

台灣自編大學教科書

機器人學導論

機構、感知、控制與具身人工智慧

全 書 合 訂 本

六篇 三十六章

編著 李世淵 (機器人叫獸)

助教 李天宇、李宇晴

2026 年 9 月

全書總目錄

導讀篇 從自動機械走向具身人工智慧 第 1~2 章 40 頁

- 第 1 章 機器人是什麼** 定義的演變、自主性光譜、Moravec 悖論
- 第 2 章 如何學機器人學** 學習地圖、即時學習策略、軟硬體環境建置

第一篇 機器人的身體與系統 第 3~7 章 107 頁

- 第 3 章 機器人的系統架構** 時間尺度金字塔、運算平台、資料流
- 第 4 章 機構與自由度** 連桿與關節、自由度分析、工作空間、URDF
- 第 5 章 驅動與傳動** 馬達選型、減速機、伺服系統、串聯彈性驅動
- 第 6 章 感測器** 編碼器、IMU、力覺、光達、相機與深度感測
- 第 7 章 嵌入式與安全** 即時系統、通訊介面、失效導向安全、安全標準

第二篇 空間數學與操作臂運動學 第 8~12 章 118 頁

- 第 8 章 座標系與空間變換** 旋轉矩陣、歐拉角與萬向節鎖、四元數、齊次變換
- 第 9 章 正向運動學** DH 參數、運動鏈、工作空間、精度與重複精度
- 第 10 章 逆向運動學** 封閉解與數值解、多重解、可達性判定
- 第 11 章 速度運動學與雅可比** 雅可比矩陣、速度—力對偶、可操作度
- 第 12 章 奇異點與冗餘** 奇異構型、冗餘自由度、零空間、分層逆運動學

第三篇 動力學、軌跡與控制 第 13~21 章 228 頁

- 第 13 章 靜力學與拉格朗日動力學** 慣性張量、標準形式、科氏力與重力項
- 第 14 章 Newton-Euler 與模擬引擎** 遞迴演算法、正向動力學、接觸模型、物理引擎
- 第 15 章 軌跡生成與運動曲線** 多項式與梯形曲線、急跳度、動力學可行性檢驗
- 第 16 章 線性回授與 PID 控制** 自然頻率與阻尼比、積分飽和、數位實作與調參
- 第 17 章 操作臂非線性控制** 計算力矩控制、重力補償、操作空間控制、自適應
- 第 18 章 最佳控制與 Riccati 方程** Bellman 原理、價值函數、LQR、倒立擺
- 第 19 章 可控性與穩定性** 可控性矩陣、Lyapunov 直接法、LaSalle 原理
- 第 20 章 力控制與柔順操作** 環境剛度、混合控制、阻抗與導納、接觸穩定性
- 第 21 章 抓取與操作** 摩擦錐、力封閉、抓取品質、欠驅動手與軟性夾爪

第四篇 感知、定位與自主移動 第 22~28 章 198 頁

- 第 22 章 機器人視覺與感知** 針孔模型、相機校正、深度恢復、手眼校正、視覺伺服
- 第 23 章 行動機器人運動學** 瞬時旋轉中心、非完整約束、里程計與 UMBmark 校正

- 第 24 章** 狀態估測與感測器融合 貝氏濾波、Kalman 與 EKF/UKF、粒子濾波、互補濾波
- 第 25 章** 同步定位與建圖 (SLAM) EKF-SLAM、FastSLAM、佔據柵格、圖最佳化、迴路閉合
- 第 26 章** 路徑規劃與導航 位形空間、A*、PRM 與 RRT、勢場與 DWA、分層導航
- 第 27 章** 無人機與飛行機器人 分配矩陣、微分平坦、串級控制、姿態估測、分級降級
- 第 28 章** 腿式機器人與運動控制 混合動態、ZMP、線性倒立擺、捕捉點、步態、全身控制

第五篇 智慧機器人與具身人工智慧 第 29 ~ 33 章 150 頁

- 第 29 章** ROS 2 與機器人軟體框架 節點與話題、服務與動作、TF2、URDF、QoS、工具生態
- 第 30 章** 模擬與 Sim-to-Real 物理引擎、保真度、領域隨機化、系統辨識、平行模擬
- 第 31 章** 機器學習與強化學習 模仿學習與 DAgger、MDP、Bellman 方程、獎勵設計、PPO
- 第 32 章** 大型模型與任務規劃 符號接地、PDDL、SayCan、VLM 與 VLA、三層防護
- 第 33 章** 具身人工智慧 Moravec 悖論的三層答案、形態計算、主動感知、倫理

第六篇 系統整合、產業應用與專題 第 34 ~ 36 章 83 頁

- 第 34 章** 系統整合與產業應用 V 模型、需求釐清、誤差預算、OEE、投資評估、安全法規
- 第 35 章** 法規、商業化與社會接受度 認證體系、產品責任、隱私、RaaS、TRL、人機互動設計
- 第 36 章** 總整專題方法論 選題與範圍、最小可行系統、量化測試、失敗分析、發表

全書共 六篇 三十六章 924 頁

如何使用這本書

本書為台灣自編大學教科書，適用於大學部的機器人學課程，亦可供高中職 AI 機器人社團、以及自學者使用。全書分為六篇三十六章，各篇可獨立閱讀，但**建議依序學習**——因為後面各篇會反覆回收前面的觀念與伏筆。

每一章的固定結構

- **叫獸開講**：以一個真實的教學情境或思想實驗開場，點出本章要解決的核心問題。
- **本章為什麼重要**：說明本章在全書中的位置與價值。
- **學習目標與先備知識**：讓你知道讀完能做到什麼、以及需要先具備什麼。
- **核心名詞中英對照**：便於查閱與銜接英文文獻。
- **正文（約 11 節）**：由淺入深的完整推導與說明。
- **完整計算例題**：每章 2~3 題，含完整的解題過程與工程判讀。
- **程式範例**：可直接執行的 Python 程式，多數能驗證該章的例題。
- **實作**：由淺入深的動手步驟，含記錄表格。
- **國中小也看得懂**：用日常的比喻說明該章的核心概念。
- **叫獸提醒與常見錯誤**：十二個實務上最常踩的坑。
- **課堂討論、章末摘要、習題、動手做與專題延伸、參考文獻**。

三種讀法

一、**完整課程（一學年）**：依序讀完六篇。建議第一學期涵蓋第 1~21 章（導讀篇至第三篇），第二學期涵蓋第 22~36 章。

二、**單學期課程**：導讀篇 + 第一篇 + 第二篇 + 第三篇的第 13、15、16、17、20、21 章。這是最精簡但完整的機器人學核心。

三、**專題導向**：先讀第 36 章（專題方法論）與第 34 章（系統整合），再依你的專題需求查閱對應的章節。本書的各章設計為可獨立查閱。

六條貫穿全書的主線

讀完全書後，比任何單一公式更值得記住的，是這六條反覆出現的思維方式（第 33.10 節有

完整的整理)：

- 模型的價值不在完美，而在減負——不完美的模型仍然極有價值。
- 分層與時間尺度分離——讓每一層只處理自己時間尺度上的問題。
- 與其消除不確定性，不如設計對它不敏感的系統——本書最重要的工程哲學。
- 機構與控制必須協同設計——在把問題丟給控制器之前，先問能不能用機構把問題變簡單。
- 模型提供保證，學習提供泛化——不是誰取代誰，而是各自處理擅長的部分。
- 理論的各部分互相支撐——當初覺得無用的章節，往往在後面突然派上用場。

台灣自編大學教科書

@prof

機器人學導論

機構、感知、控制與具身人工智慧

導 讀 篇

從自動機械走向具身人工智慧

第 1 章 機器人學概論

第 2 章 如何學習機器人學——世界名校課程巡禮

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

2026 年 8 月

導讀篇 篇章導讀

這一篇只有兩章，卻是全書三十六章的入口。它不教你任何一條公式，任務只有兩個：告訴你這門學科的全貌，以及告訴你該怎麼走完這條路。

第 1 章從一趟「機器人小旅行」出發——工廠裡的六軸手臂、客廳裡的掃地機器人、實驗室裡的四足機器狗，三者長相天差地遠，卻共享同一套靈魂。這一章會回答「機器人是什麼」，並用一隻定居後就消化掉自己大腦的海鞘，說明一個貫穿全書的核心洞見：會動的生命，才需要腦。運動需要感知、預測、決策與空間理解，而這些正是智慧的原材料。讀完第 1 章，你會擁有一張機器人世界的全景地圖：定義與判準、分類的三把尺、一百年的發展史、六大應用領域、台灣的產業位置，以及安全與倫理的邊界。

第 2 章則從兩位新生的爭論出發——「先學數學，還是先讓機器人動起來？」阿哲主張把線性代數與微積分讀完再開始，小矧主張先做出會動的東西再回頭補。叫獸沒有選邊，而是在白板上畫了一個往上盤旋的螺旋：正確的學法不是直線，是螺旋。這一章會給你六層知識金字塔、四大數學工具的分工、程式與模擬工具的選擇，以及一趟世界名校課程巡禮——史丹佛、MIT、卡內基美隆、柏林工業大學與蘇黎世聯邦理工，看看全球最頂尖的機器人教育是怎麼安排這條螺旋的。最後，你會為自己選定一條讀法：大學生依序精讀、高中生跳躍取用，或是國中小社團從附錄的手作機器人起跳。

這一篇要回答的五個問題

- 機器人到底是什麼？它一定要長得像人嗎？
- 機器人、自動化設備與人工智慧，三者的界線在哪裡？
- 一台機器人從古代的自動人偶走到今天的人形機器人，經歷了哪些關鍵轉折？
- 要學會機器人學，我需要哪些數學、哪些工具、多少時間？
- 世界最好的大學怎麼教機器人？我該用哪一種方式走完這本書？

讀完這一篇之後

你會擁有兩樣東西：一張地圖與一份計畫。地圖讓你知道，接下來每一章在整個知識版圖中的位置——第一篇的機構與硬體、第二篇的座標與運動學、第三篇的力學與控制、第四篇的感知與自主移動、第五篇的具身人工智慧、第六篇的產業與專題。計畫則讓你知道，以自己現在的程

度與可投入的時間，該從哪一章開始、哪些可以先跳過、第一台該動手做的機器人是什麼。

然後，就可以翻開第一篇，開始拆解機器人的身體了。

本篇章節配置

章次	章名	核心問題	叫獸開講
第 1 章	機器人學概論	機器人是什麼？它在世界與台灣的位置為何？	「機器人到底一定要長得像人嗎？」—— 三站小旅行與海鞘的啟示
第 2 章	如何學習機器人學——世界名校課程巡禮	需要哪些數學與工具？該用什麼路徑學？	「先學數學，還是先讓機器人動起來？」—— 阿哲與小玆的爭論與那個螺旋

導讀篇 從自動機械走向具身人工智慧

第 1 章

機器人學概論

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】跨頁照片：左半為工廠中的六軸機械手臂，右半為草地上的四足機器狗，中央疊印一隻半透明的海鞘幼體剪影——三種截然不同的「機器人」，同一套感知—思考—行動的靈魂。

叫獸開講 「機器人到底一定要長得像人嗎？」

那一天，叫獸帶著三十位大一新生做了一趟「機器人小旅行」。第一站是台中大肚山腳下的一間工具機廠。鐵捲門一拉開，六台橘色的六軸機械手臂正在為工具機上下料：手臂前端的夾爪「唰」一聲抓起十幾公斤重的工件，轉身、放入夾頭、退開，動作乾淨俐落，全程不到八秒。一位學生小聲說：「它.....長得一點都不像人，比較像一隻會轉來轉去的長頸鹿。」

第二站回到叫獸家的客廳。一台圓盤狀的掃地機器人正貼著牆邊轉圈，遇到椅腳輕輕一碰，後退、轉個角度再前進，最後還自己回到充電座。學生們笑了：「這台更不像人，根本是一塊會跑的飛盤。」

第三站是系上的機器人實驗室。一隻四足機器狗小跑步過來，被助教故意從側面踹了一腳——牠踉蹌兩步，四條腿快速交叉調整，穩穩站住，接著若無其事地繼續走。全場歡呼。這時有位學生舉手了：「老師，工廠的手臂、家裡的飛盤、實驗室的狗，三個長得完全不一樣，為什麼都叫『機器人』？機器人到底一定要長得像人嗎？」

叫獸沒有直接回答，反而講了一個海洋生物的故事。有一種叫做「被囊動物」(tunicate，俗稱海鞘) 的動物，幼體長得像小蝌蚪：有一條會擺動的尾巴、一個簡單的腦、還有感光的眼點，能在海裡游泳，尋找一塊適合定居的岩石。可是一旦牠找到位置、把自己固定下來，接下來做的第一件事，竟然是——把自己的腦和神經系統消化吸收掉，變成一團不再移動、只負責過濾海水的「濾食袋」。神經科學家常用這個例子提醒我們一件事：大腦不是為了「思考」而存在的，大腦是為了「運動」而存在的。不需要移動的生命，就不需要腦。

這正是本書想帶大家看見的核心：機器人學研究的，不是「長得像不像人」，而是「如何讓機器在真實世界裡，動得有智慧」。運動需要感知、需要預測、需要決策、需要對空間的理解——這些恰恰就是智慧的原料。所以，工廠手臂、掃地飛盤、四足機器狗，外形天差地遠，卻共享同一套靈魂：感知—思考—行動的循環。本章，我們就從「什麼是機器人」這個看似簡單、其實吵了一百年的問題出發。

本章為什麼重要

「機器人」三個字天天出現在新聞裡，但工程師口中的機器人、新聞標題裡的機器人、科幻電影裡的機器人，常常不是同一件事。如果沒有先把定義、分類與歷史脈絡整理清楚，後面三十

五章的運動學、動力學、SLAM 與具身人工智慧，就會像沒有地基的大樓。本章是全書的「總地圖」：讀完之後，你應該能在任何場合，用工程師的語言說清楚——眼前這台機器是不是機器人、屬於哪一類、用了哪些要素、在產業與社會裡扮演什麼角色，以及接下來該去哪一章找更深入的答案。

學習目標

- 能陳述機器人的工程定義，並用「可程式、自主性、與實體世界互動」三項判準分析邊界案例。
- 能區分機器人、自動化設備與人工智慧三者的異同，並解釋「具身性」的意義。
- 能畫出並說明「感知—思考—行動—回饋」循環，指出實際機器人中對應的元件。
- 能依時間軸列舉機器人發展史上至少八個里程碑事件與其意義。
- 能依用途（工業／服務／自主）與形態（操作臂／行動／無人機／腿式）對機器人進行分類。
- 能舉出製造、醫療、農業、國防、教育等領域的機器人應用實例各一。
- 能描述台灣機器人與智慧製造產業的優勢定位與代表企業。
- 能說明 Moravec 悖論，並解釋為什麼「真實世界」是人工智慧最難的考場。
- 能討論機器人安全、就業與倫理議題，並分辨科幻設定與現行工程規範的差異。

先備知識

本章不需要任何數學與程式基礎，只需要好奇心與生活觀察力。若你已具備高中物理的力與運動概念，閱讀 1.3 節會更有感覺；若完全沒有，也不影響理解。從第 8 章開始才會用到向量與矩陣，屆時附錄 D 的數學補給站隨時待命。

核心名詞中英對照

表 1-1 本章核心名詞

中文	英文	說明
機器人	Robot	可程式、具自主性、能與實體世界互動的機器
機器人學	Robotics	研究機器人設計、感知、規劃與控制的綜合

中文	英文	說明
		學科
自動化	Automation	以機器替代人力執行既定流程的技術總稱
人工智慧	Artificial Intelligence, AI	使機器展現學習、推理與決策能力的技術
具身性	Embodiment	智慧透過身體與環境互動而存在與發展的性質
致動器	Actuator	把能量轉換為運動的元件，如馬達與汽缸
感測器	Sensor	把物理量轉換為訊號的元件，機器人的感官
末端執行器	End Effector	操作臂前端與工件互動的工具，如夾爪
自由度	Degree of Freedom, DoF	描述機構獨立運動的數目
操作臂	Manipulator	由連桿與關節組成的機械手臂
自主移動機器人	Autonomous Mobile Robot, AMR	能自行定位、規劃路徑的移動平台
協作機器人	Collaborative Robot, Cobot	可與人共享工作空間的安全型機器人
感知—思考—行動循環	Sense–Think–Act Cycle	機器人運作的基本循環架構
機器人密度	Robot Density	每萬名製造業員工對應的工業機器人台數
具身人工智慧	Embodied AI	以身體與環境互動為核心的人工智慧研究方向

1.1 什麼是機器人

1.1.1 一個吵了一百年的名詞

「機器人」這個詞其實非常年輕。1920 年，捷克劇作家恰佩克 (Karel Čapek) 發表舞台劇《R.U.R. (羅素姆萬能機器人) 》，劇中工廠大量製造人造勞工，他採用捷克文 *robota*——意思是「苦役、強迫勞動」——創造出 *robot* 一詞。有趣的是，劇中的「機器人」其實是以化學方法製造的人造生物，比較接近今天說的合成生命；但這個名字太傳神了，從此全世界都用它來稱呼「替人類工作的人造機器」。中文譯為「機器人」，一開始就埋下了一個美麗的誤會：彷彿它必須是「人」的形狀。本章一開頭的三站小旅行已經告訴我們：完全不必。

1.1.2 工程上的定義

國際標準組織在 ISO 8373 中，將工業機器人定義為「可自動控制、可重複程式化、多用途

的操作機，具有三軸以上可程式化的運動軸」。這個定義抓住了三個關鍵詞：可自動控制（不需人一步一步操作）、可重複程式化（換一支程式就能換一種工作）、多用途（不是為單一產品訂做的專用機）。國際機器人聯盟（IFR）沿用這個精神，把機器人與「專用自動化設備」清楚劃開。

不過，掃地機器人沒有三軸操作機、無人機沒有夾爪，它們難道不是機器人嗎？因此本書採取一個更廣義、也更貼近現代機器人學研究範圍的工作定義：

◎ 本書的工作定義

機器人是一部可程式化的機器，能透過感測器感知環境、透過運算進行決策，並透過致動器對實體世界產生動作，且三者形成回饋循環。

1.1.3 三項判準與邊界案例

依照這個定義，我們可以用三項判準來檢驗任何一台機器：第一，可程式性——它的行為能否用軟體改寫？第二，自主性——它是否能在沒有人逐步操控的情況下，根據感測結果自行決定動作？第三，實體互動性——它是否對真實世界（而不只是螢幕裡的世界）產生物理作用？三項越齊備，就越「機器人」。

用這把尺來量一量生活中的機器，會出現許多有趣的邊界案例。全自動洗衣機可程式（選行程）、有簡單感測（水位、重量），但它的「動作」被關在滾筒裡，對環境沒有主動性，多數工程師視之為自動化家電而非機器人。自動門有感測（紅外線）、有致動（馬達），但行為只有「開／關」一種，可程式性趨近於零，是典型的自動化裝置。3D 印表機可程式性極高、對實體世界有輸出，但幾乎沒有環境感知與自主決策，一般歸類為數位製造設備。而遙控無人機是個光譜案例：純手動遙控時，它比較像「會飛的遙控車」；一旦開啟自動返航、視覺避障、路徑跟隨，自主性上升，它就滑進了機器人的領域。

圖 1-1 機器人三判準示意圖

建議配圖：三個相交的圓（可程式性、自主性、實體互動性），交集中央標註「機器人」，周圍散布洗衣機、自動門、3D 印表機、遙控無人機、工業手臂、掃地機器人等示例的位置。

這種「光譜式」的理解，比背誦一條死板的定義更有用。在後面的章節你會發現，機器人學真正關心的不是名分，而是能力：感知得多準、想得多快、動得多穩。名詞的邊界會隨技術演進不斷移動——三十年前的「先進機器人」，今天可能只是一台普通的自動化設備。

1.2 機器人、自動化設備與人工智慧的差異

1.2.1 自動化：固定劇本的演員

自動化的歷史比機器人早得多。十八世紀的紡織機、二十世紀初福特汽車的生產線、以凸輪與連桿驅動的自動車床，都是自動化的典範。傳統自動化設備的特徵是「固定劇本」：機構被設計來完成一件事，而且把這件事做到極致——速度快、精度高、成本低。代價是彈性趨近於零：換一種產品，往往意味著整台設備要重新設計。工廠裡的可程式邏輯控制器（PLC）讓劇本可以改寫，但多數專用機的「身體」仍然是為單一任務打造的。

1.2.2 機器人：可換劇本的演員

機器人的革命性在於「通用的身體 + 可換的劇本」。同一台六軸手臂，今天裝上焊槍是焊接機器人，明天換上吸盤是堆疊機器人，後天裝上相機與螺絲起子又成了組裝機器人。這種「一副身體、百種任務」的通用性，正是 ISO 定義中「多用途、可重複程式化」的精髓，也是機器人與專用自動化設備最重要的分水嶺。

1.2.3 人工智慧：沒有身體的大腦

人工智慧則是另一個維度的概念：它是「軟體的智慧」，關心的是學習、推理、辨識與決策，本身不必有身體。一個下棋程式、一個聊天模型、一套推薦系統，都是 AI，但它們的輸入與輸出都停留在數位世界。機器人恰好相反：它一定有身體，但不一定聰明——許多工業手臂三十年來只是忠實重播示教好的軌跡，沒有一絲「思考」。

因此可以這樣總結三者的關係：自動化是「會做事的機器」，AI 是「會思考的軟體」，而機器人是兩者的交會——當 AI 取得身體、當自動化設備獲得感知與決策，機器人就誕生了。近年最熱門的「具身人工智慧」（Embodied AI）一詞，說的正是這場交會：智慧不再只活在雲端，而是進入身體、透過與物理世界的碰撞來學習。這也呼應了被囊動物的啟示——會動，才需要腦；反過來說，腦只有透過動，才能真正理解世界。

表 1-2 自動化設備、機器人與人工智慧的比較

面向	專用自動化設備	機器人	人工智慧（純軟體）
身體	有，為單一任務設計	有，通用可換工具	無
可程式性	低～中（改參數為主）	高（改任務、改軌跡）	極高
環境感知	少量、固定	多元、可擴充	以資料形式輸入

面向	專用自動化設備	機器人	人工智慧（純軟體）
自主決策	幾乎沒有	低到高皆有	核心能力
典型例子	自動車床、瓶裝線	六軸手臂、AMR、無人機	語言模型、影像辨識
彈性	低	高	極高
對實體世界的作用	有	有	無（需透過機器人）

1.3 機器人的基本要素：感知、思考、行動與回饋

1.3.1 感知—思考—行動循環

拆開任何一台機器人——不論是百萬元的工業手臂還是三百元的循跡小車——都會看到同一套骨架：感知（Sense）、思考（Think）、行動（Act），三者首尾相接、循環不息。感測器把物理世界翻譯成訊號；運算單元根據訊號與目標做出決策；致動器把決策化為力量與運動；而運動改變了世界，新的世界又被感測器讀進來——這就是回饋（Feedback），機器人得以「邊做邊修正」的關鍵。

圖 1-2 感知—思考—行動—回饋循環

建議配圖：環狀流程圖。感測器（眼）→ 控制器（腦）→ 致動器（肌肉）→ 環境 → 回到感測器；環中央標註「回饋讓誤差被不斷修正」。

1.3.2 感知：機器人的感官

感知分為兩類。內部感知量測機器人自己的狀態：關節轉了幾度（編碼器）、身體傾斜多少（IMU 慣性測量單元）、馬達電流多大。外部感知量測環境：距離（超音波、紅外線、光達）、影像（相機、深度相機）、力量（力/力矩感測器）、位置（GNSS 衛星定位）。人類伸手拿杯子時，閉著眼睛也大致知道手在哪裡——那是肌肉與關節裡的本體感覺在工作；機器人的編碼器與 IMU，就是它的本體感覺。第 6 章將把這些感官一一拆給你看。

1.3.3 思考：從一行 if 到一顆大模型

「思考」的光譜極寬。最簡單的是規則式決策：「若前方距離小於二十公分，則左轉」，一行 if-else 就是一顆微型的腦。往上是控制理論：用數學模型計算該輸出多少力矩，才能又快又穩地到達目標（第 16~19 章）。再往上是規劃：在充滿障礙的空間裡搜尋一條可行路徑（第 26 章）。更上層是學習：讓機器人從資料與嘗試錯誤中自己長出策略（第 31 章），乃至用大型語言模型聽懂「幫我把桌面整理乾淨」這種人類指令（第 32~33 章）。值得記住的是：層級高不

代表一定比較好——掃地機器人早期只靠隨機碰撞加簡單規則，一樣把地掃完了。工程的智慧在於為任務選擇「剛剛好」的思考深度。

1.3.4 行動與回饋：閉迴路的力量

行動由致動器完成：直流馬達、伺服馬達、步進馬達、液壓缸、氣壓缸，甚至人工肌肉（第 5 章）。而讓行動變得可靠的，是回饋。想像家裡的冷氣：溫度感測器讀到室溫高於設定值就啟動壓縮機、低於設定值就停機——這個最古老的閉迴路概念，放大一千倍，就是機器手臂能把定位誤差壓在零點零幾公釐的原因。沒有回饋的系統稱為開迴路，像閉著眼睛投籃；有回饋的系統稱為閉迴路，像邊看邊調整的投籃。第 16 章的 PID 控制，會把這個直覺變成可以計算的數學。

◎ 與人體的對照

感測器 = 感官（眼、耳、皮膚、內耳前庭）；控制器 = 大腦與小腦；致動器 = 肌肉；機構 = 骨骼；電源 = 心血管與飲食；通訊匯流排 = 神經。下次觀察自己拿起一杯水，試著標出這六個子系統各在哪一瞬間工作——你會發現人體是一台完成度高得驚人的機器人。

1.4 機器人的歷史：自動人偶、Unimate 到人形機器人

1.4.1 古代與近代：機械夢的起點

讓機器代替人做事的夢想極其古老。古希臘的希羅（Hero of Alexandria）設計過以水力與氣壓驅動的自動裝置與神殿自動門；中國古籍記載了指南車與傳說中的木牛流馬；文藝復興時期，達文西在手稿中留下了一具以纜繩與滑輪驅動、能坐起與揮臂的機械騎士設計。十八世紀是自動人偶（automata）的黃金年代：法國工匠沃康松（Jacques de Vaucanson）打造了栩栩如生的機械鴨；瑞士鐘錶家族雅克德羅（Jaquet-Droz）製作出能蘸墨寫字的機械男孩；日本江戶時代的「茶運人偶」則能端著茶杯走向客人、奉茶後轉身返回。這些人偶沒有感知與決策，本質上是精密的發條劇本機，但它們證明了一件事：純機械也能演出「像是有生命」的動作——這正是今日凸輪、連桿與步態機構的祖先。

1.4.2 名字與想像：從 R.U.R. 到機器人三原則

1920 年恰佩克的《R.U.R.》為這個夢想命了名。1940 年代起，科幻作家艾西莫夫（Isaac Asimov）在一系列短篇中提出著名的「機器人學三原則」構想——大意是機器人不得傷害人類、須服從人類命令、並在不違反前兩者的前提下保護自己——並且創造了 robotics（機器人學）這個學科名稱。三原則是文學設定而非工程規範（真實世界靠的是安全標準與風險評估，見 1.10

節)，但它把「機器與人如何共處」這個問題，提前半個世紀放上了檯面。

1.4.3 工業機器人的誕生：Unimate 與示教重播時代

1954 年，美國發明家戴沃爾 (George Devol) 申請了「可程式物件搬運裝置」專利；他與恩格伯格 (Joseph Engelberger) ——後來被尊稱為「機器人學之父」——共同創立 Unimation 公司。1961 年，第一台工業機器人 Unimate 進駐通用汽車的工廠，負責從壓鑄機中取出高溫金屬件：又燙、又重、又危險，正是典型的 3D 工作 (Dirty、Dangerous、Demanding)。Unimate 以液壓驅動、以磁鼓記憶示教的動作序列，開啟了「示教—重播」時代：工程師牽著手臂走一遍，手臂就忠實重播一萬遍。

接下來的二十年是操作臂的黃金進化期：1969 年史丹佛大學的 Scheinman 設計出全電動、六自由度的 Stanford Arm，奠定電驅手臂架構；1973 年德國 KUKA 推出六軸電動機械手臂 Famulus；1978 年 Unimation 推出經典的 PUMA 手臂，同年日本學者牧野洋發表 SCARA 構型——四軸、水平面高速取放的專家，至今仍是電子組裝線的主力。1970 年代的日本更以驚人的速度把工業機器人普及到汽車與電子業，成為「機器人王國」。

1.4.4 會移動、會走路：從 Shakey 到 ASIMO

在工廠之外，另一條路線悄悄展開。1966 至 1972 年，美國史丹佛研究院 (SRI) 打造了第一台會「思考」的移動機器人 Shakey：它裝有相機與碰撞感測器，能感知環境、規劃路徑、推理任務步驟——今天路徑規劃課必教的 A* 演算法，正是為 Shakey 發展出來的。1986 年，本田汽車秘密啟動雙足行走研究，歷經 E 系列與 P 系列原型，於 2000 年發表震撼全球的人形機器人 ASIMO：身高一米二、能走路、上下樓梯、與人握手。1997 年，NASA 的旅居者號 (Sojourner) 成為第一台在火星上行駛的輪式機器人；2002 年，iRobot 推出掃地機器人 Roomba，機器人第一次大規模走進千家萬戶的客廳。

1.4.5 二十一世紀：動得越來越像生物，想得越來越像人

2000 年，達文西手術系統獲美國 FDA 核准，外科醫師開始透過機器手臂執行微創手術。2005 年，波士頓動力公司 (Boston Dynamics) 的四足機器人 BigDog 在崎嶇山路上馱著重物、被人踹了一腳仍能穩住的影片轟動網路；同年 DARPA 無人車挑戰賽由史丹佛的 Stanley 奪冠，點燃自動駕駛時代。2010 年代，丹麥 Universal Robots 帶起協作機器人風潮——不需圍籬、與人並肩工作；電商倉庫裡成千上萬台 AMR 開始搬運貨架。2013 年 DARPA 機器人挑戰賽推動人

形機器人在災難場景中開門、開車、爬瓦礫；其後 Atlas 的後空翻影片一次次刷新大眾對機器運動能力的想像。

2020 年代，兩條歷史線終於交會：深度學習與大型模型讓「思考」突飛猛進，硬體成本下降讓四足與人形平台走出實驗室。特斯拉發表人形機器人 Optimus、多家新創投入通用人形；研究界則把視覺—語言—動作整合為 VLA 模型，讓機器人「看得懂畫面、聽得懂人話、做得出動作」。機器人學正站在「具身人工智慧」的門口——這也是本書書名副標的由來，完整的故事將在第 33 章展開。

表 1-3 機器人發展簡明年表

年代	事件	意義
18 世紀	沃康松機械鴨、雅克德羅寫字人偶、日本茶運人偶	純機械自動人偶的黃金時代
1920	恰佩克《R.U.R.》創造 robot 一詞	「機器人」概念與名稱誕生
1940 年代	艾西莫夫提出機器人三原則構想並命名 robotics	機器人倫理想像與學科名稱出現
1961	Unimate 進駐通用汽車工廠	第一台工業機器人商業運轉
1969	Stanford Arm 問世	全電動六自由度手臂架構奠基
1966–1972	SRI 移動機器人 Shakey	感知—規劃—行動整合的先驅，A* 演算法誕生
1973 / 1978	KUKA Famulus / PUMA 與 SCARA	六軸電驅與水平關節構型普及
1997	旅居者號登陸火星	行星探測機器人時代開啟
2000	本田 ASIMO、達文西手術系統	人形行走與醫療機器人里程碑
2002	iRobot Roomba 上市	消費型服務機器人大眾化
2005	BigDog 與 DARPA 無人車挑戰賽	動態腿式運動與自駕技術突破
2010 年代	協作機器人與倉儲 AMR 興起	人機共工與物流自動化普及
2020 年代	人形機器人熱潮與 VLA 具身智慧	AI 大模型與機器人身體的歷史性交會

1.5 工業機器人、服務機器人與自主機器人

1.5.1 依用途的第一層分類

國際機器人聯盟 (IFR) 把全世界的機器人統計分成兩大陣營。第一陣營是工業機器人：安

裝在生產環境中、執行製造任務的可程式操作機，例如焊接、噴塗、搬運、鎖螺絲、上下料。第二陣營是服務機器人：為人類或設備提供服務、而非直接從事工業生產的機器人，又細分為專業服務機器人（物流 AMR、農業機器人、手術機器人、清潔消毒機器人、巡檢機器人）與消費服務機器人（掃地機器人、割草機器人、陪伴與教育機器人）。近年全球工業機器人的年安裝量已站上每年約五十萬台的量級，總運轉量突破四百萬台；而服務機器人的成長率更快，是機器人產業最有活力的新戰場。

1.5.2 自主性光譜：從遙控到全自主

「自主機器人」不是與前兩者並列的第三類，而是一把跨越所有類別的量尺。同樣是移動平台：拆彈機器人多半由人員全程遙控，自主性低；倉儲 AMR 能自行定位、規劃路徑、避開行人，自主性中高；火星探測車因為地球指令來回要十幾分鐘，必須具備高度自主的地形判讀與行駛能力。工程上常把這條光譜粗分為：人工遙控、輔助操作（人下大方向、機器管細節）、半自主（機器執行、人監督）、到全自主（機器獨立完成任務）。自主性越高，對感知、定位與決策的要求越重——這正是本書第四篇與第五篇要處理的主戰場。

圖 1-3 機器人分類與自主性光譜

建議配圖：橫軸為自主性（遙控→輔助→半自主→全自主），縱軸分工業／專業服務／消費服務三列，將拆彈機器人、示教重播手臂、協作手臂、AMR、掃地機器人、火星車等貼在對應位置。

1.5.3 機器人密度：一個國家自動化程度的溫度計

IFR 每年公布「機器人密度」：每一萬名製造業員工對應的工業機器人運轉台數。近年南韓長年居冠（每萬人超過一千台），新加坡緊追在後，德國、日本名列前茅；台灣憑藉電子與半導體產業的高度自動化，密度同樣位居全球前段班，顯著高於世界平均。這個指標的巧妙之處在於它把「機器換人」翻譯成可以跨國比較的數字——例題 1-1 我們就來親手算一次。

1.6 操作臂、行動機器人、無人機與腿式機器人

1.6.1 依形態的第二層分類

如果說 1.5 節是問「機器人做什麼」，本節就是問「機器人長什麼樣子」。形態決定了它擅長的運動方式，也決定了本書後面各章的分工：操作臂的數學在第二、三篇，輪式移動在第 23 章，無人機在第 27 章，腿式在第 28 章。

1.6.2 操作臂：固定基座上的萬能手

操作臂（機械手臂）由連桿與關節串成，基座固定，靠末端執行器工作。依構型可分為：串聯式——關節一個接一個像人的手臂，工作空間大、最常見；並聯式——多條支鏈同時撐住一個平台，如高速取放的 Delta 機器人與飛行模擬器底下的 Stewart 平台，剛性高、速度快但工作空間小；SCARA——水平面剛、垂直向柔的四軸構型，是電子組裝的速度之王；以及協作機器人——限制力量與速度、內建碰撞偵測，可與人共享空間，近十年成長最快。

1.6.3 行動機器人：把工作空間變成整個世界

輪式是效率最高的地面移動方式：差速兩輪（掃地機器人）結構最簡單；汽車式轉向適合高速；全向輪與麥克納姆輪能橫著走、原地平移，是狹窄廠房的最愛；履帶則專攻鬆軟與瓦礫地形。產業上常見兩個名詞：AGV（無人搬運車）沿磁條或色帶等固定路徑行駛，AMR（自主移動機器人）則靠地圖與感測器自由規劃路線——兩者的差別，本質上就是 1.5 節那把自主性量尺的刻度差。

1.6.4 無人機與腿式：離開平地的兩種答案

多旋翼無人機以四個以上的螺旋槳直接對抗重力，結構簡單、可懸停，成為空拍、巡檢與農業噴灑的主力；定翼機航程遠但不能懸停；垂直起降複合翼則試圖兩者兼得。腿式機器人用離散的立足點取代連續的輪跡，能上樓梯、跨障礙、走碎石坡；四足是穩定與機動的甜蜜點，雙足與人形則瞄準「為人類建造的環境」——樓梯、門把、工具全是為人手人腳設計的，人形機器人不必改造環境就能融入。此外還有水下機器人（ROV / AUV）、蛇形機器人、軟體機器人與微型機器人等特化形態，各自征服一種極端環境。

表 1-4 四大形態的比較

形態	代表構型	強項	弱項	典型應用	本書章節
操作臂	六軸串聯、SCARA、Delta、協作	精準、有力、可換工具	基座固定、工作範圍有限	焊接、組裝、取放	第 8~21 章
輪式行動	差速、麥克納姆、AGV / AMR	平地效率高、承載大	怕樓梯與崎嶇地形	倉儲物流、服務導覽	第 23~26 章
無人機	多旋翼、定翼、複合翼	三維空間、視野佳	續航短、載重小、怕風雨	空拍、巡檢、農噴	第 27 章
腿式	四足、雙足人形	地形適應力最強	控制難、能耗高、成本高	巡檢、救災、通用任務	第 28 章

1.7 應用領域：製造、醫療、農業、國防、教育與娛樂

1.7.1 製造：機器人的大本營

製造業至今仍占機器人裝置量的絕大多數。汽車業的點焊與噴塗線是六軸手臂的經典舞台；電子業的取放、鎖付與檢測讓 SCARA 與小型六軸大顯身手；金屬加工的工具機上下料、鑄件去毛邊，把工人從噪音與粉塵中解放出來。搭配機器視覺後，機器人從「盲目重播」升級為「看著做」：亂料夾取、瑕疵檢測、精密對位，成為智慧製造的核心單元（第 22、34 章）。

1.7.2 醫療與照護：在人體上工作的機器

醫療機器人把「穩定」與「精細」推到極致。手術輔助系統讓醫師的手部動作經過濾震與縮放，在患者體內以微創方式完成縫合；骨科與神經外科機器人負責高精度的定位與鑽孔；復健機器人帶著中風患者的手腳做上千次一致的訓練動作；外骨骼幫助脊髓損傷者重新站立。高齡化社會更催生了照護與陪伴機器人：搬運病患、送藥巡房、提醒服藥與陪伴對話——台灣同樣面對高齡化，這是機器人與每個家庭最近的距離。

1.7.3 農業：從產業道路飛進田裡

農業機器人要對付的是最「不標準」的工作環境：果實大小不一、光線多變、地面泥濘。採果機器人用視覺辨識成熟度、以柔性夾爪摘取；除草機器人用影像區分作物與雜草、精準點噴以減少農藥；擠乳機器人讓乳牛自己排隊「上工」；而農用無人機的噴藥、施肥與巡田，已是台灣農村實際可見的風景——第 27 章將以巡田無人機為完整案例，拆解自主導航、水位量測與病害辨識的整合設計。

1.7.4 國防、救災與極限環境

拆彈機器人代替防爆人員接近可疑物；搜救機器人鑽進倒塌建築的縫隙尋找生命跡象；無人機在災區快速建立空中影像地圖；核電廠除役與化學污染區的作業，機器人更是唯一能長時間停留的「人員」。深海與太空是另一組極限考場：水下機器人檢修離岸風機的水下基座——這對發展離岸風電的台灣尤其重要；火星探測車則要在無法即時遙控的星球上自主行駛。軍用自主武器的倫理爭議，留待 1.10 節討論。

1.7.5 教育與娛樂：把好奇心變成人才

教育機器人是門檻最低、影響最深遠的應用：從圖形化程式的循跡小車、積木式機械手臂，

到大專的自製機器人競賽，動手做的每一步都在累積機構、電子、程式與團隊合作的能力。娛樂領域則有主題樂園的機械偶、展演用的機器狗群舞、以及陪伴型桌面機器人。本書附錄 A~L 的十二個社團手作單元，正是為這條「從玩具到專業」的路徑鋪的第一段路。

1.8 台灣機器人產業與智慧製造情境

1.8.1 黃金縱谷：世界級的精密機械聚落

從台中大肚山到彰化、雲林一帶，聚集了上千家工具機與零組件廠商，被稱為精密機械的「黃金縱谷」。這個聚落的特色是垂直分工極細、供應鏈半徑極短：一台工具機或機械手臂需要的鑄件、滾珠螺桿、線性滑軌、減速機、控制器、鈑金與組裝，往往在一小時車程內就能湊齊。這種聚落效應，正是台灣切入機器人產業的先天優勢。

1.8.2 關鍵零組件與整機的代表選手

在關鍵零組件端，上銀科技（HIWIN）的滾珠螺桿與線性滑軌是全球工具機與自動化設備的重要供應者，並積極布局諧波減速機與醫療機器人；台達電子從電源與變頻器起家，發展出伺服驅動、SCARA 機器人與整廠自動化方案；和大工業等傳動齒輪廠商切入減速機構。在整機與系統端，達明機器人（Techman Robot）以「內建視覺」的協作機器人打進國際市場，出貨量居全球協作機器人前列；鴻海集團自製 Foxbot 用於自家電子組裝廠；盟立自動化等系統整合商則負責把機器人、輸送線與資訊系統整合成完整產線。

1.8.3 半導體：看不見的機器人大軍

走進台灣的先進晶圓廠，天花板上川流不息的自動搬運天車（OHT）沿著軌道運送晶圓盒，配合無塵室裡的搬運機器人與自動倉儲，構成全球最繁忙的室內物流系統之一——半導體業對潔淨度與稼動率的極端要求，使它成為自動化程度最高的產業，也是台灣機器人密度名列世界前茅的重要原因。從晶圓傳送、封裝測試到面板搬運，「看不見的機器人」早已是護國群山的隱形地基。

1.8.4 智慧製造與人才：本書的在地任務

面對少子化與缺工，政府近年持續推動智慧機械與智慧製造政策，鼓勵中小企業導入機器人、機聯網與 AI 品檢。對學生而言，這代表明確的職涯訊號：機構設計、電控整合、機器視覺、ROS 2 軟體與資料分析的複合型人才，正是產業最渴求的組合。本書每章的「台灣產業案例」欄

位，就是要把課本裡的數學，接到你未來可能上班的那條產線上。

◎ 台灣產業案例

一家中部的金屬加工廠導入兩台協作機器人做 CNC 工具機上下料：白天與老師傅並肩工作、夜間獨立顧機。導入後夜班產能增加、老師傅轉任品質與排程管理。這個小案例濃縮了台灣智慧製造的典型路徑——不是機器取代人，而是機器接手 3D（危險、骯髒、單調）工作，人升級去做機器做不到的判斷。

1.9 機器人是否具有智慧：真實世界中的 AI

1.9.1 Moravec 悖論：難的很簡單，簡單的很難

1980 年代，機器人學者莫拉維克（Hans Moravec）等人注意到一個弔詭的現象：對人類而言最「高級」的智力活動——下棋、解代數、通過考試——電腦很早就做得比多數人好；但對人類而言毫不費力的事——伸手拿杯子、在人群中走路、認出朋友的臉——機器卻難如登天。這個現象後來被稱為 Moravec 悖論。演化給了我們數億年打磨的感知與運動系統，這些能力深植神經、不假思索，反而最難被工程複製；而邏輯與棋藝只有幾千年歷史，恰好是符號運算的舒適圈。

1.9.2 真實世界：最嚴格的考場

為什麼真實世界這麼難？第一是不確定性：感測器有雜訊、馬達有誤差、地面會打滑，任何模型都只是近似——這逼出了本書第 24 章整套機率工具。第二是接觸的物理：抓取、碰撞、摩擦的力學極其複雜，模擬與現實永遠有落差。第三是即時性：球飛過來不會等你算完再落地，控制迴路必須在毫秒級完成。第四是開放性：實驗室可以控制變因，馬路與果園不行——光線、行人、風雨隨時改寫題目。一個只活在數位世界的 AI 不必面對這些；機器人必須全部面對。

1.9.3 機器人學 = 在真實世界中研究智慧

因此許多學者主張：機器人學是理解智慧最誠實的途徑。機器學習與符號推理在資料與棋盤上大放異彩，但唯有把演算法裝進身體、丟進物理世界，才會暴露「智慧到底還缺什麼」。運動需要預測、需要空間表徵、需要因果理解——被囊動物用「定居就吃掉腦」提醒過我們：這些能力正是為了行動而演化出來的。近年的具身人工智慧研究，正是把這個古老洞見變成工程綱領：讓 AI 透過身體與環境的互動來學習世界。

1.9.4 今天的機器人有多聰明？

誠實的答案是：在窄任務上超人，在常識上像幼兒。焊接手臂的重複精度勝過任何工匠，但它不知道「焊歪了會漏水」；掃地機器人能建出全屋地圖，但不明白「地上那張是重要文件不該吸走」。大型語言模型與視覺語言模型的出現，第一次讓機器人有機會「借用」人類累積在文字與影像中的常識（第 32～33 章），這是否能補上最後一塊拼圖，正是當代機器人學最激動人心的未解之題。學完本書，你將有足夠的裝備親自參與這場辯論。

1.10 機器人安全、倫理與社會影響

1.10.1 安全：現實世界的「機器人法則」

科幻的三原則寫在小說裡，工程師的三原則寫在標準與風險評估裡。傳統工業機器人的安全哲學是隔離：圍籬、光柵、安全門，人進去、機器就停。協作機器人則改用限制：限制力量、限制速度、監測碰撞，讓人機可以共享空間——國際標準（如 ISO 10218 與協作應用技術規範）明訂了功率與力量限制、速度與間距監控等協作模式。每一個機器人專案上線前的風險評估（第 36 章）並不浪漫，卻是真正保護人命的法則。此外還有功能安全設計：緊急停止按鈕、冗餘迴路、失效時導向安全狀態——第 7 章與第 34 章會落到電路與產線層面。

1.10.2 工作：取代、轉型與新生

「機器人會不會搶走工作？」是社會最關切的問題。歷史經驗顯示，自動化確實會取代特定「任務」，但同時創造新的職務：Unimate 取代了壓鑄取件工，也創造了機器人工程師、系統整合師與維護技師。真正的挑戰在轉型的陣痛——被取代的往往是重複性高的職位，新職缺卻要求新技能。合理的社會回應不是拒絕機器人（缺工與高齡化也不允許），而是教育與再訓練：讓人往「判斷、創造、照顧與整合」移動，把 3D 工作交給機器。這也是各國討論機器人稅、基本收入等政策的背景。

1.10.3 軍用、隱私與演算法偏見

自主武器是最尖銳的倫理戰場：是否允許機器在無人審核下決定攻擊目標？國際機器人武器控制委員會（ICRAC）等組織長期呼籲禁止「致命自主武器系統」，聯合國亦持續辯論相關規範——工程師無法迴避自己作品的用途問題。隱私是第二條戰線：掃地機器人畫的居家地圖、陪伴機器人聽到的對話、巡檢無人機拍下的影像，資料流向何方？最後是演算法偏見：若人臉辨識對

某些族群錯誤率較高，裝上機器人身體後，偏見就從螢幕走進了現實。負責任的機器人工程師必須把「這個功能可能被如何濫用」放進設計檢核表。

1.10.4 本書的立場

本書不迴避這些議題，也不販賣恐慌。技術本身沒有善惡，但設計有取捨、部署有後果。我們將在第 33 章討論 VLA 模型的安全限制、在第 35 章討論人機互動的信任設計、在第 36 章把風險評估變成可操作的工程流程。從第一章開始請記住：學會讓機器人動起來只是一半的功課，另一半是想清楚它該為誰而動、如何安全地動。

1.11 本書的學習地圖

1.11.1 六篇一附錄的旅程

本書的結構呼應機器人本身的構造。導讀篇（第 1~2 章）建立全局視野與學習方法。第一篇（第 3~7 章）拆解機器人的身體：機構、馬達、感測器與電子系統。第二篇（第 8~12 章）學會機器人的幾何語言：座標轉換、正逆運動學與雅可比。第三篇（第 13~21 章）進入力與控制的世界：動力學、軌跡、PID、最佳控制、力控制與抓取。第四篇（第 22~28 章）讓機器人走出基座：視覺、機率狀態估測、SLAM、路徑規劃、無人機與腿式。第五篇（第 29~33 章）替機器人裝上現代大腦：ROS 2、模擬、機器學習、LLM / VLM 與 VLA 具身智慧。第六篇（第 34~36 章）回到產業與專題實作。附錄 A~L 則是十二個社團手作單元，供國中小與入門者從玩中學。

圖 1-4 全書學習地圖

建議配圖：以一台機器人的剖面圖為中心，身體部位對應各篇——骨骼與肌肉指向第一篇、幾何座標指向第二篇、力與控制指向第三篇、眼睛與地圖指向第四篇、大腦指向第五篇、工廠背景指向第六篇；下方一排小機器人代表附錄十二單元。

1.11.2 三種讀法

大學修課讀法：依章節順序閱讀，完成每章例題、程式與實作，一學期可完成前四篇，第二學期進入五、六篇與總整專題。高中專題讀法：以故事層與圖解為主，跳過「進階推導」專欄，重點放在第 1~7 章、第 16 章 PID、第 22 章視覺與附錄 F~L 的實作。國中小社團讀法：從附錄 A 開始動手做，每完成一台機器人，翻回「螺旋銜接」指定的章節讀該章的「國中小也看得懂」專欄——小時候做過的每一台機器人，都會在長大後的課本裡重逢。

1.11.3 學習本章之後

下一章（第 2 章）我們將討論「如何學習機器人學」：需要哪些數學、哪些工具、世界名校怎麼教，以及模擬器與實體機器人如何搭配。然後，旅程正式開始。

完整計算例題

例題 1-1 機器人密度

題目：甲國製造業就業人口 250 萬人，工業機器人運轉總量 30 萬台；乙國製造業就業人口 40 萬人，運轉量 4.4 萬台。求兩國的機器人密度，並比較其自動化程度。

解：機器人密度定義為每一萬名製造業員工的機器人台數。甲國： $300000 \div 250 = 1200$ （台 / 萬人）。乙國： $44000 \div 40 = 1100$ （台 / 萬人）。兩國皆遠高於世界平均（約一百多台 / 萬人的量級），屬於高度自動化經濟體；甲國密度略高於乙國。此指標消除了經濟體規模差異，使跨國比較成為可能——但請注意它只計工業機器人，未反映服務機器人與產業結構（電子業密度天然高於食品業）的差異。

例題 1-2 自動化投資回收期

題目：某工廠評估導入一套協作機器人上下料系統，總建置成本 150 萬元。導入後每年可節省人力成本 48 萬元、因夜班稼動增加的年產值貢獻 30 萬元；系統每年維護與耗材費 18 萬元。求簡單投資回收期。

解：每年淨效益 = $48 + 30 - 18 = 60$ （萬元 / 年）。簡單回收期 = 建置成本 $\div$ 年淨效益 = $150 \div 60 = 2.5$ （年）。一般製造業對自動化投資的心理門檻約在二至三年，本案落在可接受範圍。進一步的評估還應計入良率提升、工安風險下降與人員轉任的價值——這些「軟效益」常常才是導入成敗的關鍵，第 34 章將完整展開投資效益分析。

程式範例：三十行的感知—思考—行動

以下 Python 程式在一維世界裡模擬一台避障小車：小車每一步「感知」與前方牆壁的距離（帶有雜訊），「思考」決定速度（距離越近越慢，太近則停止並警示），「行動」則更新位置。麻雀雖小，五臟俱全——這正是每一台機器人的最小骨架。

```
import random
```

```

wall = 100.0          # 牆壁位置 (公分)
pos = 0.0             # 小車位置
dt = 0.1              # 每步時間 (秒)

def sense(pos):
    """感知：量測與牆的距離，模擬感測器雜訊"""
    true_dist = wall - pos
    noise = random.gauss(0, 0.5)    # 高斯雜訊
    return true_dist + noise

def think(dist):
    """思考：依距離決定速度 (簡單規則式決策)"""
    if dist > 30:
        return 20.0                # 遠：全速前進
    elif dist > 10:
        return dist * 0.5          # 中：距離越近越慢
    else:
        return 0.0                 # 近：停止

def act(pos, v):
    """行動：依速度更新位置"""
    return pos + v * dt

for step in range(200):
    d = sense(pos)                  # 感知
    v = think(d)                   # 思考
    pos = act(pos, v)              # 行動 (回饋：新位置影響下次感知)
    if v == 0.0:
        print(f"第 {step} 步：距離 {d:.1f} cm · 已安全停止")
        break
    if step % 20 == 0:
        print(f"第 {step} 步：距離 {d:.1f} cm · 速度 {v:.1f} cm/s")

```

請試著修改 `think()` 的規則：把停止距離改成 5 公分會發生什麼事？把雜訊標準差加大到 5 會發生什麼事？你會親眼看到「感測雜訊」如何逼出更聰明的思考——這個伏筆將在第 24 章的濾波器理論中收回。

ROS 2 與模擬實作：第一次接觸

本章的實作目標很單純：在你的電腦上讓第一隻「機器龜」動起來，親身體驗感知—思考—行動的分工。步驟概要如下（完整安裝細節見第 29 章與課程網站）：

1. 在 Ubuntu 環境安裝 ROS 2（建議 LTS 版本），或使用課程提供的虛擬機映像檔。

2. 開啟終端機，執行 turtlesim 模擬節點：`ros2 run turtlesim turtlesim_node`，畫面會出現一隻小龜。
3. 另開終端機執行鍵盤遙控節點：`ros2 run turtlesim turtle_teleop_key`，用方向鍵駕駛小龜。
4. 執行 `ros2 topic list` 觀察系統中的「話題」：你的按鍵化成速度指令（思考的輸出），小龜的位置回報（感知）源源不絕地流動。
5. 思考題：在這個系統裡，誰是感測器？誰是控制器？誰是致動器？把答案寫進實驗紀錄。

這隻小龜看似玩具，卻是 ROS 2 節點、話題與訊息機制的完整縮影——第 29 章我們將用同一套機制，指揮真正的機器人。

國中小也看得懂：掃地機器人的三步驟

想像掃地機器人是一隻圓圓的小動物。它的「眼睛」是前面的碰撞板和小小的紅外線燈：碰到椅子腳，它就知道「前面有東西」。它的「腦」是一顆小電腦，想法很簡單：撞到了？後退一點、轉個彎、再前進。它的「腳」是兩個輪子，一邊轉快、一邊轉慢，就能轉彎。眼睛看→腦袋想→輪子動，一圈一圈重複，整個房間就掃完了。所有的機器人——工廠裡的大手臂、天上的無人機、會跑的機器狗——肚子裡都裝著這一模一樣的三步驟。你在附錄 F 做的循跡小車，也是這三步驟：眼睛看黑線、腦袋算偏多少、輪子調速度。做完再回來讀這一段，你就是全班最懂機器人的。

叫獸提醒與常見錯誤

- 誤解一：「機器人 = 人形」。人形只是形態之一，且因控制困難、成本高，反而是最晚成熟的形態。判斷機器人看三判準，不看長相。
- 誤解二：「AI = 機器人」。AI 是軟體智慧，機器人是具有身體的機器；有 AI 沒身體不是機器人，有身體沒 AI 也可以是機器人（示教重播手臂）。
- 誤解三：「自動 = 智慧」。自動門很自動，但毫無智慧。自動化描述「不需人操作」，智慧描述「能因應變化」——兩者是不同的軸。
- 誤解四：「機器人一定搶工作」。機器人取代的是任務不是人，且優先取代危險、骯髒、單調的 3D 任務；真正的課題是轉型與再訓練。

- 誤解五：「科幻三原則是工程規範」。真實世界靠的是安全標準、風險評估與功能安全設計，不是小說設定。
- 用語提醒：本書統一使用「操作臂」指稱固定基座的機械手臂（俗稱機器手臂、機械手臂皆通），教學時請留意口語與正式名詞的對應，避免與「機械手（末端的手爪）」混淆。

課堂討論

1. 你的手機是機器人嗎？用 1.1 節的三判準逐項分析，並和同學辯論邊界案例（提示：想想相機自動對焦與震動馬達）。
2. 如果被囊動物的故事是對的——大腦為運動而生——那麼「只會聊天、沒有身體」的語言模型，能發展出完整的智慧嗎？正反方各找兩個論點。
3. 你家附近的傳統小工廠適合導入機器人嗎？從成本、批量、彈性與人力四個角度分析，並用例題 1-2 的方法粗估回收期。
4. 查一則最近的機器人新聞，判斷報導中的「機器人」屬於哪一類（用途×形態×自主性），並檢查記者的用詞是否精確。

章末摘要

- 機器人是可程式化、能感知—決策—行動並與實體世界互動的機器；判準是可程式性、自主性與實體互動性，而非外形。
- 自動化是會做事的機器，AI 是會思考的軟體，機器人是兩者的交會；具身性主張智慧需要身體與環境的互動。
- 感知—思考—行動—回饋循環是一切機器人的共同骨架；閉迴路回饋是精準與穩定的來源。
- 機器人史的主線：自動人偶（純機械）→ Unimate（工業示教重播）→ Shakey 與 ASIMO（感知與行走）→ 協作與 AMR（人機共工）→ 人形與 VLA（具身智慧）。
- 分類三把尺：用途（工業 / 專業服務 / 消費服務）、形態（操作臂 / 輪式 / 無人機 / 腿式）、自主性（遙控到全自主）。
- 台灣優勢：精密機械聚落、關鍵零組件（傳動、減速、驅動）、協作機器人整機與全球最高等級的半導體自動化。
- Moravec 悖論提醒我們：感知與運動才是智慧的深水區；機器人學是在真實世界中研究智慧

的學科。

- 安全靠標準與風險評估而非科幻原則；就業課題的解方是轉型與教育；工程師須對作品的社會後果負責。

觀念題與計算題

觀念題

1. 寫出本書對機器人的工作定義，並說明三項判準各自防範哪一種「假機器人」。
2. 以自動販賣機為例，逐項檢驗三判準，說明它是否為機器人。
3. 解釋「一副身體、百種任務」如何區分機器人與專用自動化設備。
4. 畫出感知—思考—行動—回饋循環，並以四足機器狗被踹一腳仍站穩的情境，標註每個方塊發生的事。
5. 為什麼說 Unimate 開啟的是「示教—重播」時代？這個時代的機器人缺少循環中的哪一塊？
6. 比較 AGV 與 AMR 的差異，並指出這個差異落在哪一把分類量尺上。
7. 舉出 Moravec 悖論的兩個例子（一個機器擅長、一個機器不擅長），並解釋成因。
8. 協作機器人與傳統工業機器人的安全哲學有何不同？各用一個詞概括。
9. 台灣的機器人密度為何名列世界前茅？請由產業結構解釋。
10. 「機器人取代的是任務不是人」——你同意嗎？以本章內容支持或反駁。

計算題

1. 某經濟體製造業就業 180 萬人、機器人密度 950 台 / 萬人，求工業機器人運轉總量；若五年後就業降至 160 萬人、運轉量成長至 20 萬台，密度變為多少？
2. 一套機器人系統建置成本 240 萬元，年節省人力 64 萬元、年增產值 36 萬元、年維護費 25 萬元。求簡單回收期；若公司要求回收期不得超過三年，此案是否過關？再假設政府補助建置成本的三成，回收期變為多少？

動手做與專題延伸

本章實作：十種自動化設備觀察

在校園、住家與街上尋找十種「會自己動」的設備，逐一填寫下表，用三判準判斷它是否為機器人，並在班上分享最有爭議的一台。

表 1-5 自動化設備觀察表（範例列供參考）

設備名稱	感知（有哪些感知？）	思考（誰在決策？）	行動（誰在出力？）	程式性	自主性	是機器人嗎？
掃地機器人	碰撞板、紅外線、輪速	內建控制器與地圖演算法	兩個驅動輪與吸塵馬達	高	高	是
自動門	紅外線人體感測	固定邏輯（偵測到即開）	門機馬達	極低	低	否（自動化裝置）
（自行填寫八列）						

專題延伸

- 訪談一位在工廠或倉儲工作的親友，記錄現場有哪些自動化與機器人設備、導入前後工作內容的變化，寫成八百字田野筆記。
- 挑一部機器人題材的電影或影集，用本章知識寫一篇「工程師影評」：哪些設定符合現實？哪些違反了感知—思考—行動的基本物理？
- 進階：完成本章 Python 避障小車後，將牆壁改為「會移動的行人」（位置隨時間變化），觀察原本的規則何時失效——這將是你對第 26 章動態障礙物問題的第一次直覺。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson.
- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer.
- Toussaint, M., Introduction to Robotics 課程講義, TU Berlin / University of Stuttgart (本書系參考之德式圖解講義)。
- International Federation of Robotics (IFR), World Robotics Report (每年發布之工業與服務機器人統計)。
- ISO 8373, Robotics — Vocabulary (機器人名詞定義國際標準)。
- ISO 10218 及 ISO/TS 15066 (工業與協作機器人安全標準與技術規範)。

- Moravec, H., Mind Children: The Future of Robot and Human Intelligence, Harvard University Press.
- Brooks, R. A., "Intelligence without Representation," Artificial Intelligence, vol. 47, 1991.
- Čapek, K., R.U.R. (Rossum's Universal Robots), 1920 (robot 一詞出處之劇作) 。
- IEEE Spectrum Robotics 專欄與 IFR 官方網站 (產業動態與統計之延伸閱讀) 。

第 2 章

如何學習機器人學

——世界名校課程巡禮

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】一張向上盤旋的螺旋樓梯照片，每一階梯上放置一台由簡到繁的機器人——震動蟲、循跡小車、自平衡車、機械手臂、四足機器狗——象徵螺旋式學習：同一個問題，每繞一圈，工具更強、理解更深。

叫獸開講 「先學數學，還是先讓機器人動起來？」

開學第二週，叫獸的研究室來了兩位一年級學生。他們是高中同班同學，一起考進智慧機器人學程，卻在第一堂課後吵了起來，決定找老師評理。

阿哲說：「我覺得應該先把數學打好。線性代數、微積分、機率論，這些不學會，後面的運動學、動力學根本看不懂。我打算大一大二先把數學修完，大三再開始碰機器人。」他把一疊列印好的課表推到桌上，上面用螢光筆標了滿滿的必修數學。

小妤搖頭：「那太慢了。我國中就在社團做過循跡小車，也玩過 Arduino。我覺得應該先讓機器人動起來，遇到問題再回頭補數學。你先學三年數學，中間一台機器人都沒做出來，早就忘光了。」她從背包裡拿出一台自己焊的小車，輪子還沾著賽道的黑膠帶屑。

叫獸聽完，沒有站在任何一邊，反而問了一個問題：「你們兩個都說得有道理。但我想先問——阿哲，你知不知道，六軸手臂為什麼需要用到矩陣？小妤，你知不知道，你那台循跡小車為什麼在轉彎太急的時候會衝出賽道？」

兩個人都愣住了。阿哲確實不知道矩陣在機器人裡具體要拿來做什麼，他只知道「線性代數很重要」；小妤知道車子會衝出去，但她的解法是「把速度調慢一點」，至於為什麼調慢就好、該調多慢，她說不上來。

叫獸笑了：「這就是問題所在。阿哲的路線，是把工具磨得很利，卻不知道要砍哪棵樹；小妤的路線，是拿著斧頭一直砍，卻不知道換一把利刃可以省一半力氣。機器人學最迷人也最難的地方，就在於它是一門『雙向』的學問——理論要落到會動的機器上才有意義，動手做遇到瓶頸時，才會知道該去學哪個數學。」

他站起來，在白板上畫了一個往上盤旋的螺旋：「正確的學法不是直線，是螺旋。做一台循跡小車，遇到轉彎衝出去，回頭學 PID 控制；學會 PID，再做一台會平衡的兩輪車，發現需要角度感測，回頭學 IMU 與濾波；學會濾波，再做定位建圖，發現需要機率論.....每繞一圈，你的手更巧、你的數學更深，而且你永遠知道，這個公式是為了解決哪一個具體的痛苦而存在的。」

小妤問：「那我們該從哪裡開始繞第一圈？」叫獸把那台沾著膠帶屑的小車拿起來，說：「你已經開始了。而阿哲——你要做的，是把你手上那些數學，一個一個接到真實的機器上。這一章，我就帶你們看看，全世界最頂尖的大學是怎麼安排這條螺旋的。」

本章為什麼重要

第 1 章回答了「機器人是什麼」，本章回答「機器人學該怎麼學」。這不是可有可無的方法論閒談：機器人學橫跨機械、電機、電子、控制、資訊與人工智慧六大領域，是典型的「寬而深」學科。沒有學習策略的人，很容易掉進兩個陷阱——一是被數學公式淹沒，讀了兩年還沒讓任何東西動起來；二是一直停留在接線與複製程式碼，遇到瓶頸卻不知道該補哪塊理論。本章提供三樣東西：一張知識地圖（知道全貌在哪）、一份數學與工具清單（知道要準備什麼）、一套經過世界名校驗證的學習路徑（知道別人怎麼走成功的）。讀完之後，你手上會有一份屬於自己的四年（或三年、或一學期）學習計畫。

學習目標

- 能畫出機器人學的知識階層圖，並說明各層之間的依賴關係。
- 能說明「螺旋式學習」的意義，並為自己規劃至少三圈的學習循環。
- 能列舉機器人學所需的四大數學工具，並各舉一個在本書中的具體應用章節。
- 能說明向量、矩陣、三角函數與微積分在機器人學中分別解決什麼問題。
- 能區分 Python、C/C++ 與圖形化程式在機器人開發中的分工與適用場合。
- 能說明模擬器、數位分身與實體機器人三者的關係與各自的優缺點。
- 能比較 Stanford、MIT、CMU、柏林工業大學與 ETH Zürich 五所名校機器人課程的側重差異。
- 能說明問題導向學習（PBL）與專題導向學習的差異，並設計一個屬於自己的入門專題。
- 能依自己的身分（大學生 / 高中生 / 國中小社團）選定本書的閱讀路線與進度。
- 能完成本書所需的軟硬體環境建置清單，並評估自己的裝備缺口。

先備知識

僅需第 1 章的內容。本章不進行任何推導，所有數學都只談「它能解決什麼問題」，不談「怎麼算」。若你在閱讀 2.3、2.4 節時感到陌生，那完全正常——這正是本章的目的：讓你知道自己還缺什麼，而不是現在就學會它。

核心名詞中英對照

表 2-1 本章核心名詞

中文	英文	說明
螺旋式學習	Spiral Learning	同一主題在不同深度反覆出現、逐圈加深的學習法
問題導向學習	Problem-Based Learning, PBL	以真實問題驅動知識學習的教學法
專題導向學習	Project-Based Learning	以完成具體作品為目標的教學法
先備知識	Prerequisite	學習某主題前必須具備的基礎知識
模擬器	Simulator	以軟體重現機器人與環境物理行為的工具
數位分身	Digital Twin	與實體設備即時對應、同步更新的虛擬模型
模擬到實機落差	Sim-to-Real Gap	模擬環境與真實世界之間的行為差異
整合開發環境	Integrated Development Environment, IDE	整合編輯、編譯與除錯的程式開發軟體
版本控制	Version Control	追蹤程式碼變更歷史的管理系統，如 Git
開源軟體	Open Source Software	原始碼公開、可自由使用與修改的軟體
機器人作業系統	Robot Operating System, ROS	機器人軟體開發的中介框架與生態系
總整專題	Capstone Project	整合全部所學、於學程末期完成的綜合性專題

2.1 機器人學的知識階層

2.1.1 六層金字塔

機器人學的知識可以整理成一座六層的金字塔，由下而上分別是：物理與數學基礎、機構與硬體、幾何與運動學、力學與控制、感知與規劃、學習與智慧。每一層都以下一層為地基，但地基不必全部完工才能往上蓋——這正是與傳統理工科系「先修課全部修完才能修進階課」的線性思維最大的不同。

最底層是物理與數學基礎：牛頓力學、線性代數、微積分、機率論。第二層是機構與硬體：連桿、關節、馬達、感測器、電子電路（第 3～7 章）。第三層是幾何與運動學：座標轉換、正逆運動學、雅可比矩陣（第 8～12 章）。第四層是力學與控制：動力學、軌跡生成、PID 與最佳控制（第 13～21 章）。第五層是感知與規劃：視覺、狀態估測、SLAM、路徑規劃（第 22～28

章)。最上層是學習與智慧：機器學習、強化學習、大型模型與具身智慧(第 29~33 章)。

圖 2-1 機器人學知識階層金字塔

建議配圖：六層金字塔，由下而上為物理與數學、機構與硬體、幾何與運動學、力學與控制、感知與規劃、學習與智慧。右側標註對應章節；左側以一條向上盤旋的箭頭穿過各層，代表螺旋式學習不必逐層完工。

2.1.2 依賴關係：哪些是硬性、哪些是軟性

層與層之間的依賴有強有弱，分辨清楚可以省下大量時間。硬性依賴是「不懂前者就真的看不懂後者」：不懂矩陣乘法，就無法理解齊次變換；不懂微分方程，就無法理解 PD 控制的解；不懂機率的條件與邊際分佈，就無法理解 Bayes 濾波。軟性依賴則是「先跳過也能懂個大概」：不懂拉格朗日力學，仍然可以會用 PID 讓手臂動起來；不懂 SLAM 的數學，仍然可以操作 ROS 2 的 SLAM 套件建出地圖。

對初學者而言，最有效率的策略是：硬性依賴老老實實補，軟性依賴大膽先跳。本書每章開頭的「先備知識」欄位，就是為此設計的——它明確告訴你，這一章真正需要什麼、什麼可以之後再說。

2.1.3 三種常見的學習誤區

誤區一是「數學無底洞」：認為要先把所有數學讀完才能開始，結果永遠沒有開始。事實上機器人學用到的數學範圍雖廣，但深度有限——線性代數用到矩陣運算與奇異值分解、微積分用到偏導數與微分方程、機率用到 Bayes 定理與高斯分佈，多數在大一大二的課程範圍內。

誤區二是「接線就是機器人」：能把馬達接起來讓車子跑，就以為學會了機器人。硬體是入場券不是畢業證書；真正的門檻在於，當車子跑偏、手臂抖動、地圖漂移時，你能不能診斷出是機構、控制還是感知出了問題。

誤區三是「只讀論文不動手」：能講出十種 SLAM 演算法的差異，卻沒有讓任何一台機器人真的跑起來。機器人學是一門會被現實無情打臉的學科——摩擦力、雜訊、電磁干擾、電池電壓下降，這些課本上一行帶過的東西，會在實驗室裡花掉你整個週末。動手，是唯一能學到這些的方法。

2.2 由觀察、組裝、程式到數學模型

2.2.1 四階段學習法

把叫獸白板上的螺旋展開，每一圈都包含四個階段：觀察、組裝、程式、數學模型。這四階段的順序不是隨意的——它符合人類認知由具體到抽象的自然歷程。

觀察階段：看真實的機器人如何運作。看掃地機器人怎麼繞開椅腳、看工廠手臂怎麼取放工件、看影片裡的機器狗怎麼在雪地上恢復平衡。這階段的產出是「問題」——為什麼它要這樣動？如果換成我，我會怎麼做？

組裝階段：親手把一台會動的東西做出來。從附錄 A 的震動機器人到附錄 F 的循跡小車，重點不在做得多精緻，而在於你必須面對真實的物理：螺絲會鬆、電線會斷、電池會沒電、馬達左右轉速不一樣。

程式階段：讓它按照你的意志動作。從圖形化積木到 Python，從「前進三秒」到「偵測到黑線就修正」。這階段你會第一次體驗到「感知—思考—行動」不是抽象概念，而是三段真實的程式碼。

數學模型階段：用公式描述並預測它的行為。到這裡你才會問：為什麼 P 增益調太大會震盪？該調多少才不震盪？——而這正是第 16 章要回答的問題。因為你已經親眼看過震盪，這個公式對你來說就不再是死背，而是解藥。

表 2-2 四階段學習法的內容與產出

階段	做什麼	典型活動	產出	常見卡關
觀察	看真實機器人如何運作	工廠參訪、影片分析、拆解舊設備	一份「為什麼」清單	只看熱鬧不看門道
組裝	親手做出會動的東西	附錄 A~E 手作單元	一台會動的機器	只求能動、不求理解
程式	讓它按你的意志動作	圖形化程式、Python 控制	可重複執行的行為	抄程式碼而不改參數
數學模型	用公式描述與預測行為	推導、模擬、參數設計	可預測、可設計的系統	推導完不回頭驗證實機

2.2.2 螺旋的三圈範例

第一圈（入門）：組裝循跡小車 → 用圖形化程式讓它跑 → 發現轉彎衝出去 → 學會「偏差越大、修正越大」的比例概念。此圈的數學深度：小學高年級的比例。

第二圈（進階）：改用 Python 控制 → 加入速度限制與微分項 → 發現原地抖動 → 學會 PD 控制與阻尼的概念（第 16 章）。此圈的數學深度：高中的微分直覺。

第三圈（專業）：改用兩輪自平衡車 → 加入 IMU → 發現感測器雜訊讓角度亂跳 → 學會互補濾波與 Kalman 濾波（第 24 章）→ 進一步推導系統的動力學方程與 LQR 控制器（第 18 章）。此圈的數學深度：大學的線性代數、微分方程與機率論。

同一個主題（讓車子穩定地走），在三圈之中被反覆處理，但每一圈的工具都更強、理解都更深。這就是螺旋式學習的威力——也是本書為什麼要把附錄 A~L 的社團實作與正文章節用「螺旋銜接」欄綁在一起的原因。

◎ 叫獸的建議

如果你是大一新生，請在第一學期就做至少一台會動的機器人，不論多簡陋。原因不是技術上的，是心理上的：親手讓某個東西動起來的成就感，會支撐你度過後面兩年枯燥的數學課。而當你在解那些矩陣時，腦中會有一台具體的機器在轉動——這會讓一切變得完全不同。

2.3 機器人學所需的數學基礎

2.3.1 四大工具與它們的任務

機器人學的數學不是無邊無際的。它主要用到四樣工具，而且每一樣都有明確的任務。

線性代數負責「空間」：機器人在三維空間中的位置與姿態、座標系之間的轉換、多個關節的速度如何合成末端速度——這些全部靠向量與矩陣描述。它是機器人學最核心、使用最頻繁的數學，出現在第 8~12 章的每一頁。

微積分負責「變化」：速度是位置的導數、加速度是速度的導數、能量的變化率決定運動方程。控制系統的穩定性分析、軌跡的平滑度、最佳化的梯度下降，全都建立在微分之上。它主要出現在第 13~21 章。

機率論負責「不確定性」：感測器有雜訊、馬達有誤差、環境會變化。當機器人問「我在哪裡」時，正確的答案不是一個點，而是一個機率分佈。這是第 24、25 章的靈魂。

最佳化負責「選擇」：逆運動學有無限多組解，選哪一組？路徑有無限多條，走哪一條？控制器參數有無限多種，用哪一組？最佳化就是把「選擇」變成「求極值」的數學。它貫穿第 10 章的 IK、第 18 章的最佳控制與第 26 章的軌跡最佳化。

表 2-3 四大數學工具與本書對應章節

數學工具	負責的問題	關鍵概念	本書主要章節
線性代數	空間與姿態	向量、矩陣、行列式、秩、奇異值分解、特徵值	第 8 ~ 12、19 章
微積分	變化與運動	導數、偏導數、微分方程、梯度、泰勒展開	第 13 ~ 21 章
機率論	不確定性	條件機率、Bayes 定理、高斯分佈、期望值	第 24 ~ 25、31 章
最佳化	選擇與設計	目標函數、約束、梯度下降、Gauss-Newton	第 10、18、26 章

2.3.2 要學到多深

學生最常問的問題是：「這些數學要學到什麼程度？」誠實的答案是：以本書的範圍而言，扎實的大一大二數學就足夠。具體而言，線性代數需要熟悉矩陣乘法、反矩陣、行列式、正交矩陣與特徵值，並對奇異值分解有概念性理解；微積分需要熟悉偏導數與鏈鎖律，並能解簡單的二階常微分方程；機率論需要熟悉條件機率、Bayes 定理、期望值與變異數，以及多變量高斯分佈的基本性質；最佳化則需要理解「找極值就是找梯度為零的點」，並知道 Gauss-Newton 法的基本想法。

如果你打算往研究所發展，還會用到李群與李代數（描述旋轉的更嚴謹框架）、凸最佳化、隨機過程與資訊理論。但那是第二階段的事——本書會在相關章節的「進階推導」專欄中指出方向，不會讓初學者卡在門口。

2.3.3 補救策略：即時學習

如果你的數學基礎不夠，不必因此停下腳步。最有效的策略是「即時學習」（just-in-time learning）：不預先把整本線性代數讀完，而是在讀到第 8 章需要旋轉矩陣時，才回頭把正交矩陣的性質補起來。這樣做有兩個好處：一是動機明確，你知道這個概念要拿來做什麼；二是記憶牢固，因為它立刻被用在一個具體的問題上。附錄 D 的數學補給站正是為此設計，它只收錄本書真正會用到的部分，每一則都附上「這在第幾章會用到」的標記。

2.4 向量、矩陣、三角函數與微積分

2.4.1 向量：位置與方向的語言

向量是機器人學的第一個詞彙。末端執行器的位置是一個三維向量、關節角度的集合是一個 n 維向量、施加的力與力矩也是向量。你需要熟悉的操作有：加減（位移的疊加）、內積（計算

投影與夾角，也用於功與能量）、外積（計算力矩與法向量，在第 11 章的雅可比推導中大量使用）、範數（計算距離與誤差大小）。

一個具體的例子：機器手臂末端目前在位置 p ，目標在 p^* ，那麼誤差向量就是 $p^* - p$ ，它的範數是距離，它的方向指出該往哪走。第 10 章的逆運動學，本質上就是不斷縮短這個誤差向量的過程。

2.4.2 矩陣：轉換與映射的語言

矩陣在機器人學中主要扮演兩個角色。第一是描述座標轉換：一個 3×3 的旋轉矩陣描述姿態，一個 4×4 的齊次變換矩陣同時描述位置與姿態（第 8 章）。第二是描述線性映射：雅可比矩陣把關節速度映射到末端速度（第 11 章）、慣性矩陣把加速度映射到力矩（第 13 章）。

矩陣的一些性質在機器人學中有極其具體的物理意義，這是本書會反覆強調的重點。例如：旋轉矩陣是正交矩陣，所以它的反矩陣等於轉置——這讓座標的反向轉換變得極其便宜；雅可比矩陣的秩下降代表機器人進入奇異點，某個方向的運動能力瞬間消失（第 12 章）；慣性矩陣是正定矩陣，所以它必定可逆，這保證了正向動力學一定有解（第 14 章）。當你在線性代數課上覺得「特徵值到底有什麼用」時，請記住：第 19 章會用特徵值的正負，判斷一台機器人會不會失控。

2.4.3 三角函數：關節與姿態的橋樑

三角函數是連接「關節轉了幾度」與「末端在哪裡」的橋樑。平面二連桿手臂的正向運動學，就是兩個餘弦與兩個正弦的組合；逆向運動學則需要反三角函數，並且會遇到「一個位置對應兩組角度」的多解問題（手肘朝上或朝下，第 10 章）。程式實作上有一個實務要點：務必使用 $\text{atan2}(y, x)$ 而非 $\text{atan}(y/x)$ ，因為前者能正確判斷四個象限——這個小細節，是初學者除錯時最常掉進去的坑之一。

2.4.4 微積分：從運動到控制

微積分在機器人學的第一個用途是描述運動的層級關係：位置微分得到速度，速度微分得到加速度，加速度再微分得到急動度（jerk，第 15 章討論軌跡平滑度時會用到）。第二個用途是建立運動方程：拉格朗日方法用能量對廣義座標的偏導數，直接推導出動力學方程（第 13 章）。第三個用途是分析與設計控制器：PD 控制的閉環系統是一個二階常微分方程，它的解決定了系統是震盪、臨界阻尼還是過阻尼（第 16 章）。第四個用途是最佳化：梯度是函數上升最快的方

向，梯度下降就是沿著反方向走到谷底（第 10、26 章）。

◎ 數學不是門檻，是捷徑

許多學生把數學視為機器人學的門檻，其實它是捷徑。沒有數學，你只能靠試誤法調參數，一組一組試到天亮；有了數學，你可以直接算出「要讓系統在零點五秒內穩定且不超越， K_p 該設多少、 K_d 該設多少」（第 16 章）。前者是碰運氣，後者是工程。

2.5 力學、電子電路與控制基礎

2.5.1 力學：機器人為什麼會動、會抖、會壞

機器人是實體的機器，因此古典力學是它的基本法則。你需要的力學知識包括：牛頓定律（力等於質量乘加速度，這是第 14 章 Newton-Euler 遞迴的出發點）、力矩與轉動慣量（決定馬達選型，第 5 章）、能量與功（拉格朗日方法的基礎，第 13 章）、摩擦與背隙（決定實際精度的隱形殺手）、剛性與共振（決定手臂能跑多快而不抖動，第 27 章）。

一個實務上的例子：學生設計手臂時常常只計算「馬達扭力夠不夠舉起負載」，卻忘了計算「加速時需要多少額外扭力」與「連桿的彈性變形有多大」。前者讓手臂舉不動，後者讓手臂末端在高速運動時抖動不止——兩者都是力學問題。

2.5.2 電子電路：讓訊號與能量正確流動

電子電路在機器人中扮演兩條路線：訊號路線（感測器讀值、通訊、控制指令）與功率路線（馬達驅動、電池供電）。初學者需要掌握的最小知識集合是：歐姆定律與功率計算（估算電池撐多久、線徑要多粗）、數位與類比訊號的差別、PWM 如何控制馬達轉速、常見通訊介面（UART、I²C、SPI）的用途、以及最重要的——共地、接地與電磁干擾的基本觀念。

為什麼最後一項最重要？因為機器人是「大電流馬達」與「微弱訊號感測器」共存的系統。馬達啟動瞬間的電流突波，會讓電源電壓瞬間下降、讓感測器讀值亂跳、甚至讓微控制器重新開機——這正是第 1 章與第 7 章都會提到的「明明接好了電線，為什麼一動就重開機」的元凶。學會分離電源、加裝濾波電容、正確佈置接地，往往比多寫一百行程式更能救活一台機器人。

2.5.3 控制基礎：從開迴路到閉迴路

控制是機器人學的中樞神經。入門階段你需要建立三個直覺。第一，開迴路與閉迴路的差別：前者是「我叫你走三秒」，後者是「我看著你走到定位為止」——後者才能對抗摩擦、負載變化

與外力干擾。第二，回授的三種成分：比例（現在差多少）、微分（正在以多快的速度接近）、積分（過去累積了多少沒補上的誤差），這就是 PID。第三，穩定性的概念：增益太小反應遲鈍，增益太大則會震盪甚至發散——這個「太大会爆炸」的直覺，是第 19 章 Lyapunov 穩定性分析的樸素版本。

好消息是，這三個直覺不需要數學就能先建立，你可以在附錄 F 的循跡小車上親眼看到。壞消息是，要把它們變成可設計、可預測的工程，就必須回到微分方程——這就是第 16 章存在的理由。

2.6 Python、C/C++ 與圖形化程式

2.6.1 三種語言的分工

機器人開發不是只用一種語言。實務上，三種工具各有各的位置，硬要用一種打天下反而事倍功半。

圖形化程式（Scratch、mBlock、micro:bit MakeCode、Blockly）以拖拉積木取代打字，適合國中小社團與完全零基礎的入門者。它的價值不在於能做多複雜的專案，而在於它讓學習者把注意力集中在「邏輯」上，而非「分號忘了打」。附錄 B、F、G、H 的社團單元皆以此為主要工具。

Python 是今日機器人研究與快速原型的主流語言。它語法簡潔、函式庫豐富（NumPy 做矩陣運算、OpenCV 做影像處理、PyTorch 做深度學習），且是 ROS 2 的官方支援語言之一。本書的所有課文程式範例都以 Python 撰寫，理由很簡單：讀者可以把注意力放在演算法本身，而不是記憶體管理。

C/C++ 則是即時控制與嵌入式系統的王者。當你需要在「一毫秒內完成一次控制迴路運算」，或是在只有幾十 KB 記憶體的低控制器上跑程式時，C/C++ 幾乎是唯一選擇。工業機器人控制器、飛控韌體、ROS 2 的核心底層，全都是 C++ 寫的。Arduino 使用的也是 C++ 的簡化框架。

表 2-4 三種程式工具的比較與適用場合

工具	優點	缺點	典型場合	本書使用處
圖形化程式	零門檻、視覺化、不會語法錯誤	難以處理複雜邏輯與大型專案	國中小社團、概念示範	附錄 B、F、G、H

工具	優點	缺點	典型場合	本書使用處
Python	語法簡潔、函式庫豐富、開發快	執行速度較慢、即時性不足	演算法原型、資料分析、AI	全書課文範例
C/C++	執行快、可即時、可跑在微控制器	語法嚴格、開發較慢、易出錯	嵌入式韌體、即時控制迴路	第 7、16 章、附錄 C ~ E

2.6.2 學習順序建議

對零基礎者，建議的順序是：圖形化程式（建立邏輯直覺）→ Python（學會真正的程式設計）→ C/C++（處理即時與嵌入式）。對已有程式基礎的大學生，可直接從 Python 開始，在第 7 章接觸微控制器時再學 Arduino 風格的 C++。

有一個常見的迷思需要澄清：許多學生認為「C++ 比較高級、Python 是玩具」。事實正好相反——現代機器人系統幾乎都是混合架構：底層的即時控制迴路用 C++ 寫、上層的規劃與決策用 Python 寫，兩者透過 ROS 2 的訊息機制溝通。會用對地方，比會用哪一種更重要。

2.6.3 不只是語言：三樣必備的工程習慣

除了語言本身，有三樣工程習慣會決定你走得多遠。第一是版本控制（Git）：機器人專案的程式碼會不斷修改，沒有版本控制，你會在「昨天明明可以跑」的地獄裡浪費無數時間。第二是註解與文件：三個月後的你，就是另一個人；沒有註解的程式碼，等於沒寫。第三是模組化：把感知、決策、控制拆成獨立的函式或節點，才能單獨測試、單獨除錯——這正是 ROS 2 節點架構背後的哲學（第 29 章）。

◎ 除錯的黃金法則

當機器人不動時，請依序檢查：一、有電嗎（電池電壓、電源開關）？二、線接對了嗎（正負、訊號腳位）？三、程式跑到那一行了嗎（加 print 或 LED 指示）？四、感測器讀值合理嗎（印出來看）？五、才是懷疑演算法。初學者最常犯的錯，是第一步就跳到第五步，花三小時檢查演算法，最後發現是電池沒電。

2.7 模擬、數位分身與實體機器人的關係

2.7.1 為什麼要用模擬器

模擬器是用軟體重現機器人與環境的物理行為。它有四個不可取代的價值。第一是零成本試錯：在模擬中撞牆一百次不會壞任何東西，在真實世界撞一次可能就是幾萬元的維修費。第二是

速度：強化學習可能需要百萬次嘗試，真實機器人做不到，模擬器可以用比實時快數十倍的速度跑。第三是可重複性：真實世界的每一次實驗條件都略有不同，模擬器可以完美重現同一個場景，方便比較演算法優劣。第四是安全性：測試一個尚未驗證的控制器，在模擬中失控只是畫面亂跳，在真機上失控可能傷人。

常見的機器人模擬器包括：Gazebo（與 ROS 2 深度整合，本書第 30 章使用）、Webots（教學友善、跨平台）、Isaac Sim（NVIDIA 出品，圖形逼真、適合 AI 訓練）、MuJoCo（物理精確、廣泛用於強化學習研究）、CoppeliaSim（原名 V-REP，教學界常用）。工業界則有 ABB RobotStudio、FANUC ROBOGUIDE 等原廠離線程式軟體。

2.7.2 數位分身：模擬器的產業版本

數位分身（Digital Twin）比一般模擬器更進一步：它與實體設備即時連線、同步更新狀態。工廠裡的實體產線在運轉，電腦裡的虛擬產線同步顯示每一台機器的位置、溫度與稼動率。它的用途包括虛擬調試（在設備到廠前先驗證程式）、預知保養（從數據預測何時該換軸承）與製程最佳化（在虛擬世界試新排程，不影響生產）。第 30 章會完整展開這個主題。

2.7.3 模擬到實機的落差

必須誠實面對的是：模擬永遠不等於現實。這個差距稱為 Sim-to-Real Gap，主要來源有四個。一是接觸與摩擦的物理難以精確建模——這是模擬中最不可靠的部分，也是為什麼在模擬中完美的抓取，到真機上常常滑落。二是感測器雜訊的特性難以完整重現——真實的相機有動態範圍限制、真實的 LiDAR 在玻璃上會失效。三是致動器的非理想性——背隙、摩擦、發熱後的性能衰減。四是環境的不可預測性——模擬中的世界永遠乾淨整齊，真實的實驗室地板上有電線與灰塵。

縮小這個落差有幾種策略：領域隨機化（在模擬中隨機改變摩擦係數、質量、光照，逼迫策略學會通用）、系統辨識（用真機數據校正模擬參數）、以及混合訓練（模擬中預訓練、真機上微調）。這些方法在第 31 章會詳細討論。

表 2-5 模擬器與實體機器人的比較

面向	模擬器	實體機器人
成本	極低（軟體授權或免費）	高（硬體、維修、耗材）
速度	可快於實時數十倍	受物理限制，無法加速

面向	模擬器	實體機器人
可重複性	完美重現	每次條件略有差異
安全性	失敗無代價	失敗可能損壞或傷人
真實性	有 Sim-to-Real 落差	百分之百真實
適合階段	演算法開發、大量訓練、驗證	最終驗證、實務展示、產業應用

2.7.4 正確的使用姿態

模擬與實機不是二選一，而是分工。本書建議的流程是：先在模擬器中驗證演算法邏輯是否正確（省時間），再在實機上驗證是否能對抗真實世界的雜訊（省面子）。最糟糕的兩種做法是：只在模擬中完成專題，畢業展示時才發現真機根本跑不動；或是完全不用模擬，把整個學期耗在硬體除錯上，演算法一行都沒寫完。

2.8 名校課程巡禮：Stanford CS223A 與 Craig 教科書傳統

2.8.1 史丹佛的機器人學傳統

史丹佛大學是現代機器人學的发源地之一。1969 年 Scheinman 在此設計出 Stanford Arm，奠定了電動六自由度手臂的基本架構；而 John J. Craig 的《Introduction to Robotics: Mechanics and Control》正是源自史丹佛早期的機器人課程講義，這本教科書自 1986 年初版以來，成為全世界最廣泛採用的機器人學教材，也是本書數理骨架的主要參考。

史丹佛的機器人學導論課程（長期由 Oussama Khatib 教授主講）建立了一套經典的教學序列：空間描述與轉換 → 正向運動學（DH 參數）→ 逆向運動學 → 雅可比與速度靜力 → 動力學 → 軌跡生成 → 線性控制 → 非線性控制 → 力控制。這個序列的邏輯極其嚴謹：先描述位置，再描述速度，再描述力，最後控制力——每一步都建立在前一步之上。Khatib 教授本人也是操作空間控制（Operational Space Control）理論的提出者，本書第 17 章即介紹此方法。

2.8.2 這個傳統的優點與侷限

Craig 傳統的最大優點是數理嚴謹、體系完整：讀完之後，你能真正計算一台機器手臂該輸出多少力矩。它至今仍是所有機器人課程的骨幹，本書第二篇與第三篇即依循此架構。

但它也有明確的侷限：成書年代較早，內容集中於固定基座的操作臂，對移動機器人、機率狀態估測、SLAM、機器學習與現代 AI 幾乎沒有著墨。因此本書在保留其骨架的同時，補上了第

四篇與第五篇——這正是「經典骨架 + 現代血肉」的編寫策略。

2.9 MIT：機器人導論與現代機器操作

2.9.1 系統整合的教學傳統

麻省理工學院的機器人課程以「系統整合」見長。其機器人導論課程涵蓋平面與空間運動學、運動規劃、操作臂與移動機器人的機構設計、多剛體動力學、控制器、致動器、感測器、嵌入式軟體，並以動手專題收尾。與史丹佛的「深挖操作臂數學」相比，MIT 更強調「一台完整的機器人系統需要哪些零件，以及它們如何協同」。

這種取向對台灣的工程教育特別有參考價值：許多學生數學考得很好，卻無法把馬達、感測器、控制器與程式整合成一台會動的機器。本書第一篇（第 3~7 章）用整整五章講機器人的身體與系統，正是採納了這個教學傳統。

2.9.2 現代機器操作：把深度學習接進來

MIT 近年的現代機器操作課程（Robotic Manipulation）代表了另一條前沿路線：它把深度學習與三維幾何感知、無碰撞運動規劃、任務與動作規劃、不確定性下的規劃，以及模型式與學習式控制，整合成一個完整的操作系統。課程的核心問題不再是「如何讓手臂到達某個座標」，而是「如何讓機器人在雜亂的桌面上，看到一個沒見過的物體並把它撿起來」。

這個轉變非常關鍵：它代表機器人學的重心，正從「已知環境中的精確運動」移向「未知環境中的穩健操作」。本書第 21 章（抓取）、第 22 章（視覺）與第 31~33 章（學習與具身智慧）即呼應此方向。

2.10 CMU：感測、思考、身體與互動四大領域

2.10.1 四大支柱的架構

卡內基美隆大學擁有全球規模最大的機器人研究機構之一（Robotics Institute），其課程架構以四個基礎領域組成：感測與感知（Sensing and Perception）、動作與思考（Cognition / Reasoning）、機器人的身體（Mechanisms / Robot Bodies），以及與環境的互動（Interaction with the Environment）。

這個四分法的洞見在於：它不把機器人視為一台機械裝置，而視為一個「感知—推理—規

劃—行動」的完整智慧系統。任何一個機器人問題，都可以被定位到這四個領域之一，或它們的交集。這也是本書採用「感知—思考—行動」作為第 1 章核心框架的原因。

2.10.2 自主移動與 SLAM 的重鎮

CMU 同時是自主移動機器人與 SLAM 研究的重鎮。其 16-311 機器人導論課程涵蓋 PID 控制、Bug 演算法、非完整約束與運動規劃；16-833 機器人定位與建圖課程則專攻機率狀態估測、粒子濾波、圖最佳化 SLAM 等主題。本書第四篇（第 22~28 章）的架構，以及本書系另一本《地圖建構與同步定位》三十一章教科書，皆大量參考此教學脈絡。

◎ 給學生的觀察

比較這三所學校，你會看到一個有趣的分工：史丹佛把「操作臂的數學」做到極致，MIT 把「系統整合與現代操作」做到極致，CMU 把「感知與自主移動」做到極致。沒有一所學校能同時做全部——這也提醒我們，機器人學是一門必須靠團隊分工的學科。你不必全能，但必須知道自已的位置，以及如何與其他位置的人合作。

2.11 柏林工業大學與 ETH Zürich：圖解教學與 ROS 2 生態

2.11.1 德式教學：大量圖解與系統結構

德國的機器人教材有一個鮮明的特色：圖片極多、系統結構清晰、循序解說。本書的重要參考之一，是柏林工業大學 Marc Toussaint 教授的機器人學講義，其內容涵蓋運動學、動力學、路徑規劃（Bug、勢場、PRM、RRT）、機率、移動機器人、SLAM、控制理論（HJB、Riccati、Lyapunov）、強化學習、抓取與腿式運動——範圍遠比傳統操作臂教材寬廣，且大量使用示意圖降低理解門檻。

這種教學風格對初學者極為友善：一張好的示意圖，往往勝過三頁的文字推導。本書在每個重要概念都盡量安排「一張系統架構圖、一張座標示意圖」，即是採納此精神。同時，Toussaint 講義中對最佳控制、可控性與 Lyapunov 穩定性的完整處理，也是本書第 18、19 章獨立成章的依據。

2.11.2 ETH Zürich：ROS 2 與機器人學習

蘇黎世聯邦理工學院代表了機器人教育的另一個前沿。其 ROS 2 課程系統性地涵蓋節點、主題、訊息、服務、動作、URDF 模型、Gazebo 模擬、RViz 視覺化、TF 座標轉換，以及感測器

與致動器的閉迴路控制整合——這是今日機器人軟體工程的實務標準，本書第 29 章即依此架構編寫。

其機器人學習課程則更進一步，涵蓋模仿學習、強化學習、策略學習、生成模型、Transformer、視覺—語言—動作（VLA）模型與機器人基礎模型。ETH 同時也是四足機器人 ANYmal 的誕生地，在腿式運動與強化學習控制方面有標誌性的成果——本書第 28 章與第 31 章的內容方向即與此呼應。

表 2-6 五所名校機器人課程的側重比較

學校	代表課程 / 教材	核心側重	本書對應章節
Stanford	CS223A、Craig 教科書	操作臂數學：運動學、動力學、控制	第二、三篇（第 8~21 章）
MIT	機器人導論、現代機器操作	系統整合、感知與學習式操作	第一篇、第 21~22、31 章
CMU	16-311、16-833	感知、自主移動、SLAM	第四篇（第 22~28 章）
TU Berlin	Toussaint 機器人講義	圖解教學、規劃、最佳控制、機率	第 18~19、24~26 章
ETH Zürich	ROS 2、機器人學習	軟體工程、腿式運動、具身 AI	第 28~33 章

2.11.3 台灣的位置與本書的取舍

巡禮一圈之後，回到台灣。我們的優勢在於：精密機械與電子產業聚落完整，學生有大量接觸真實設備的機會；技職教育體系動手能力強。我們的挑戰在於：數理課程與實作課程常常各自為政，學生要嘛只會算不會做，要嘛只會做不會算。

因此本書的取舍是：以 Craig 的數理骨架保證嚴謹（不能只會做）、以德式圖解降低門檻（不能被公式擋住）、以 MIT 與 CMU 的系統與感知架構補足現代性（不能只有操作臂）、以 ETH 的 ROS 2 與具身 AI 銜接前沿（不能與國際脫節）、再以台灣的產業案例與社團實作落地（不能與在地脫節）。這五條線交織起來，就是你手上這本書。

2.12 問題導向學習與專題導向學習

2.12.1 兩種學習法的差異

問題導向學習 (Problem-Based Learning, PBL) 以一個真實問題為起點：「如何讓機器人在停電後知道自己在哪裡？」學習者為了解決問題，被迫去學習所需的知識。問題本身沒有標準答案，學習的軌跡也因人而異。

專題導向學習 (Project-Based Learning) 則以一件具體作品為終點：「做出一台能自主巡邏教室並回報異常的機器人。」它有明確的交付物、時程與規格，過程中需要分工、整合與測試。

兩者常被混用，但差別很重要：PBL 的核心是「理解」，專題的核心是「產出」。機器人學的理想教學是兩者結合——用問題驅動理解，用專題驗證能力。本書每章的「課堂討論」偏向 PBL，「動手做與專題延伸」偏向專題導向，第 36 章則是完整的總整專題方法論。

表 2-7 問題導向與專題導向學習的比較

面向	問題導向學習 (PBL)	專題導向學習
起點	一個真實問題	一個作品目標
核心	理解與探究	產出與整合
成果	解法與洞見	可展示的作品
評量	推理過程、學習歷程	作品完成度、規格達成率
時程	彈性，可短可長	明確，有里程碑
本書對應	課堂討論、叫獸開講	動手做、附錄實作、第 36 章

2.12.2 設計一個好的入門專題

一個好的機器人入門專題應該符合五個條件。第一，可完成：以現有的時間、預算與能力，確實做得出來——過度野心的專題百分之九十以死機收場。第二，可測量：有客觀的成功標準，例如「循跡完成一圈的時間低於三十秒」，而非「跑得很順」。第三，可分解：能拆成機構、電子、程式、整合四個子任務，讓團隊分工。第四，有痛點：在製作過程中必定會遇到某個技術瓶頸——這個瓶頸正是學習發生的地方。第五，可延伸：完成後能加上新功能，成為下一圈螺旋的起點。

以循跡小車為例：可完成（一週）、可測量（單圈秒數與出界次數）、可分解（車架 / 馬達驅動 / 感測器 / 程式）、有痛點（轉彎衝出去→需要 PID）、可延伸（加超音波避障→加相機辨識號誌）。這正是它成為全世界機器人入門經典的原因。

2.12.3 從失敗中學習

機器人專題最重要的教育價值，往往來自失敗。馬達燒掉、程式當機、比賽當天車子不動——這些挫折會逼出真正的工程能力：系統性除錯、風險備援、時間管理。本書第 36 章特別設有「失敗原因分析與設計改良」一節，並建議所有專題都要撰寫「失敗紀錄」：記下每一次故障的現象、猜測、驗證與結論。三年後你會發現，這份紀錄比任何一份成功報告都更有價值。

2.13 三種讀法與學習環境建置

2.13.1 大學生讀法

大學修課讀法建議依章節順序閱讀，並在每章完成三件事：讀懂例題、跑通程式、完成動手做。時程上，第一學期可完成導讀篇與第一、二篇（第 1~12 章），重點在建立機構直覺與座標運動學基礎；第二學期進入第三、四篇（第 13~28 章），重點在控制與感知；第三學期（或大三上）進入第五、六篇（第 29~36 章）與總整專題。若課程只有一學期，建議路徑為：第 1~8 章 → 第 10~11 章 → 第 15~16 章 → 第 22~26 章 → 第 29 章，捨去最佳控制與具身 AI 的深入部分。

2.13.2 高中專題讀法

高中生以故事層與圖解為主，跳過所有「進階推導」與矩陣推導細節。建議路徑為：第 1~7 章（完整讀，建立全貌與硬體知識）→ 第 16 章 PID（只讀直覺與實作，跳過微分方程解）→ 第 22 章視覺（重點在 YOLO 應用）→ 第 32 章 LLM 應用 → 附錄 F~L 的實作單元。搭配科展或專題競賽，一年內足以完成一件有深度的作品。

2.13.3 國中小社團讀法

完全從附錄 A 開始，動手優先。每完成一台機器人，翻回「螺旋銜接」欄指定的章節，只讀該章的「國中小也看得懂」專欄。建議順序為：附錄 A 震動機器人 → B 投石戰車 → F 循跡小車 → G 超音波避障 → H 藍牙遙控 → D 四伺服雙足 → J 機器狗 → K YOLO 資源回收 → L VLM 機器人。這條路線橫跨機構、控制、感知與 AI，等於用九台機器人走完一遍機器人學的全景。

圖 2-2 三種讀法的路徑圖

建議配圖：以本書 36 章與 12 附錄為橫軸方格，用三條不同顏色的路徑線標出大學生（依序全走）、高中生（跳躍式）、國中小（由附錄起跳並回連正文）的行進路線。

2.13.4 軟硬體環境建置

本書的實作分為三個層級，讀者可依資源選擇。輕量級（預算約一千至三千元）：micro:bit 或 Arduino Uno、直流馬達與驅動模組、紅外線循跡感測器、超音波模組、伺服馬達數顆、麵包板與杜邦線。中量級（約五千至一萬五千元）：ESP32 開發板、無刷馬達與電變、IMU 模組、藍牙 / WiFi 模組、六軸小型機械手臂套件、鋰電池與充電器。重量級（實驗室等級）：Raspberry Pi 5 或 NVIDIA Jetson、RGB-D 相機、二維光達、差速輪底盤、協作機械手臂（如有）。

軟體方面則建議建置以下環境：Python 3 與 NumPy、Matplotlib、OpenCV；Arduino IDE 或 PlatformIO；Ubuntu 系統與 ROS 2（可用虛擬機或 WSL2）；Gazebo 或 Webots 模擬器；Git 版本控制；以及一套順手的編輯器（VS Code 為主流選擇）。第 29 章會提供完整的安裝步驟。

表 2-8 三個層級的軟硬體建置清單

層級	主要硬體	主要軟體	適用對象	可完成的本書內容
輕量級	micro:bit / Arduino、直流馬達、紅外線與超音波感測器、伺服馬達	圖形化程式、Arduino IDE	國中小社團、高中入門	附錄 A~H、第 16 章 PID 實作
中量級	ESP32、IMU、無刷馬達、小型機械手臂套件、鋰電池	Python、Arduino C++、簡易模擬器	高中專題、大學一二年級	附錄 D~J、第 8~17 章實作
重量級	Raspberry Pi 5 / Jetson、RGB-D 相機、光達、差速底盤	Ubuntu、ROS 2、Gazebo、PyTorch	大學專業課程、專題研究	全書，含第 22~33 章

完整計算例題

例題 2-1 學習時數的規劃

題目：某大學生每週可投入機器人學習 12 小時，一學期以 18 週計。若希望在一學期內完成本書第 1~12 章（共 12 章），且每章需要「閱讀 2 小時、例題與程式 2 小時、動手做 3 小時」，請評估此規劃是否可行，並提出調整建議。

解：每章所需時數 = $2 + 2 + 3 = 7$ (小時)。12 章共需 $12 \times 7 = 84$ (小時)。一學期可投入時數 = $12 \times 18 = 216$ (小時)。表面上時數充裕 ($84 \div 216 \approx 39\%$)。但實務上必須保留：數學補救 (約 30 小時)、硬體採購與除錯 (約 40 小時，機器人專案的除錯時間通常等於或大於開發時間)、以及期中期末考週的中斷 (約 4 週、48 小時)。調整後可用時數 $\approx 216 - 30 - 40 - 48 = 98$ (小時)，仍高於 84 小時，規劃可行但緩衝不大。建議：將第 9~12 章的動手做部分改為模擬器實作 (省去硬體除錯時間)，或把第 12 章移至下學期。

例題 2-2 專題可行性評估

題目：三人團隊有 10 週時間、預算 8000 元，欲製作「能在教室內自主巡邏並辨識障礙物的機器人」。已知：底盤與馬達 2500 元、Raspberry Pi 與相機 4000 元、感測器與電池 1500 元。請評估可行性並拆解工作。

解：預算 = $2500 + 4000 + 1500 = 8000$ (元)，恰好用盡，無備品緩衝——建議先削減至 7000 元以保留 1000 元的損壞備援 (機器人專案中馬達與感測器的損耗率不可忽視)。時程拆解建議：第 1~2 週機構組裝與馬達驅動測試；第 3~4 週感測器整合與資料讀取；第 5~6 週自主移動與避障邏輯；第 7~8 週影像辨識整合；第 9 週系統整合測試；第 10 週報告與展示。人力分工：一人負責機構與電子、一人負責移動控制、一人負責影像辨識，整合階段全員投入。風險點：影像辨識 (第 7~8 週) 若延誤將壓縮整合時間，建議提前於第 5 週開始平行進行資料蒐集與模型訓練。

程式範例：檢查你的開發環境

以下 Python 程式檢查本書所需的套件是否安裝完成，並印出版本資訊。請在開始第 3 章之前執行一次，確認環境無誤。

```
import sys

print(f"Python 版本：{sys.version.split()[0]}")
print("-" * 40)

packages = {
    "numpy":      "矩陣與向量運算 (第 8 章起大量使用)",
    "matplotlib": "繪圖與資料視覺化",
    "scipy":      "科學計算、最佳化與訊號處理",
    "cv2":        "OpenCV 影像處理 (第 22 章)",
}
```



```

missing = []
for name, purpose in packages.items():
    try:
        mod = __import__(name)
        ver = getattr(mod, "__version__", "未知")
        print(f"[OK] {name:12s} v{ver:10s} {purpose}")
    except ImportError:
        print(f"[缺少] {name:12s} {'':11s} {purpose}")
        missing.append(name)

print("-" * 40)
if missing:
    pipname = {"cv2": "opencv-python"}
    cmds = " ".join(pipname.get(m, m) for m in missing)
    print(f"請執行安裝指令：pip install {cmds}")
else:
    print("環境檢查完成，可以開始第 3 章了！")

```

若出現缺少套件的訊息，依提示執行 `pip` 安裝即可。建議使用虛擬環境（`venv` 或 `conda`）管理本書專用的套件，避免與其他專案衝突。

ROS 2 與模擬實作：建立你的學習工作區

本章的實作目標是建立一個往後三十四章都會用到的工作區與筆記系統。步驟如下：

1. 在電腦上建立資料夾結構：`robotics_course/`，其下分為 `notes/`（筆記）、`code/`（各章程式）、`sim/`（模擬檔案）、`hardware/`（電路圖與零件清單）、`logs/`（實驗與失敗紀錄）。
2. 在該資料夾執行 `git init`，建立版本控制。撰寫第一份 `README.md`，寫下你的學習目標與預定進度。
3. 安裝 Python 環境並執行上一節的環境檢查程式，將輸出結果貼入 `notes/ch02_env.md`。
4. （進階）安裝 Ubuntu 與 ROS 2，或在 Windows 上啟用 WSL2 安裝。第 29 章會用到，提前安裝可避免屆時卡關。
5. 建立 `logs/failures.md`，格式為：日期 | 現象 | 猜測原因 | 驗證方法 | 結論。往後每一次除錯都記錄一則。

這五個步驟看似瑣碎，卻是專業與業餘的分水嶺。三十四章之後，這個資料夾會成為你最有價值的作品集——包括那份失敗紀錄。

國中小也看得懂：學機器人就像學騎腳踏車

學做機器人，就像學騎腳踏車。你不會先讀完一本《腳踏車力學原理》才敢上車，對不對？你會先跨上去、踩踩看、摔幾次、慢慢就會了。等你騎得很順之後，如果想騎得更快、想爬更陡的山，這時候你才會想知道：齒輪怎麼調？坐墊多高最省力？這些「原理」就變得有用了。學機器人也一樣：先做一台會動的小車（附錄 A 到 F），玩到你開始問「為什麼它轉彎會衝出去？」，那就是你該翻開課本的時候了。記住兩件事：第一，先動手，再問為什麼；第二，壞掉不要怕，每個工程師的桌上都有一堆燒掉的零件，那是他們學會的證據。

叫獸提醒與常見錯誤

- 誤區一：等數學全部學完才開始。數學是工具不是門票，用到再學（即時學習）效率最高，也記得最牢。
- 誤區二：只抄程式碼不改參數。抄來的程式能跑，不代表你懂；把 K_p 改成兩倍看看會怎樣，才是真的在學。
- 誤區三：跳過模擬直接上實機。實機除錯耗時是模擬的十倍以上，先在模擬中確認邏輯正確，再上機驗證物理。
- 誤區四：迷信模擬結果。模擬中完美的抓取，在真機上常常滑落。Sim-to-Real 落差是必然的，必須實測。
- 誤區五：不做版本控制。「昨天明明可以跑」是機器人實驗室最常聽到的哀嚎，Git 是最便宜的保險。
- 誤區六：專題訂得太大。十週的專題請訂五週能做完的目標，剩下五週會被除錯與整合吃掉——這是經驗法則，不是悲觀。
- 誤區七：失敗不記錄。同一個錯誤犯第二次，代價是雙倍的時間。失敗紀錄是機器人工程師最重要的個人資產。

課堂討論

1. 阿哲與小玆的爭論，你比較同意誰？請提出第三種路線，並說明它如何避開兩人各自的缺點。
2. 以你目前的數學程度，本書哪一章最可能讓你卡住？你打算用什麼策略處理（先跳過、即時

補、還是提前準備)？

3. 如果系上只能開一門機器人課(一學期三學分)，你認為應該教哪些主題？請從本書 36 章中選出十章並說明理由。
4. 模擬器讓學習成本大幅下降，但也可能讓學生失去接觸真實硬體的機會。你認為理想的模擬與實機比例是多少？為什麼？
5. 五所名校的課程各有側重，如果要為台灣的產業需求設計一門課，你會偏向哪一所的風格？請結合第 1 章的台灣產業內容作答。

章末摘要

- 機器人學的知識可分為六層金字塔：物理數學、機構硬體、幾何運動學、力學控制、感知規劃、學習智慧；層間依賴分硬性與軟性，硬性補足、軟性可跳。
- 最有效的學習法是螺旋式而非線性：觀察 → 組裝 → 程式 → 數學模型，每繞一圈加深一層，同一主題反覆出現於不同深度。
- 四大數學工具各司其職：線性代數管空間、微積分管變化、機率論管不確定性、最佳化管選擇；深度以扎實的大一大二程度為足。
- 程式工具三分工：圖形化入門、Python 做原型與演算法、C/C++ 做即時與嵌入式；版本控制、註解與模組化是三樣必備工程習慣。
- 模擬器提供零成本、高速、可重複與安全的試錯環境，但存在 Sim-to-Real 落差；正確姿態是先模擬驗證邏輯、再實機驗證物理。
- 五所名校側重不同：Stanford 深耕操作臂數學、MIT 強調系統整合與現代操作、CMU 專精感知與 SLAM、TU Berlin 圖解與規劃控制、ETH 領先 ROS 2 與具身 AI。
- 本書策略為五線交織：Craig 數理骨架、德式圖解、MIT 與 CMU 的系統與感知、ETH 的軟體與前沿、台灣產業與社團實作。
- 好的入門專題需符合五條件：可完成、可測量、可分解、有痛點、可延伸；失敗紀錄的價值往往高於成功報告。
- 三種讀法各有路徑：大學生依序精讀、高中生跳躍取用、國中小由附錄起跳並回連正文專欄。

觀念題與計算題

觀念題

1. 畫出機器人學的六層知識階層圖，並各舉一個屬於該層的具體主題。
2. 說明「硬性依賴」與「軟性依賴」的差別，並各舉一例。
3. 以自己的話解釋螺旋式學習，並以「讓車子穩定行走」為主題設計三圈學習循環。
4. 線性代數、微積分、機率論、最佳化四大工具，各負責機器人學中的什麼問題？各舉一個本書章節。
5. 為什麼程式中計算角度應使用 $\text{atan2}(y, x)$ 而非 $\text{atan}(y/x)$ ？
6. 旋轉矩陣是正交矩陣，這個性質在工程上帶來什麼實質好處？
7. 圖形化程式、Python 與 C/C++ 在機器人開發中的分工為何？各舉一個適用場合。
8. 列舉模擬器的四個價值，以及 Sim-to-Real 落差的四個來源。
9. 比較問題導向學習與專題導向學習的差異，並說明本書哪些欄位分別對應此二者。
10. 說明一個好的入門專題應符合的五個條件，並用循跡小車逐條驗證。
11. 比較 Stanford、MIT、CMU 三校機器人課程的側重差異，並說明本書如何整合三者。
12. 為什麼「失敗紀錄」對機器人工程師特別重要？請從機器人學的學科特性說明。

計算題

1. 某學生每週可投入 8 小時學習，一學期 18 週。若每章需 7 小時，且需保留 25 小時數學補救、35 小時硬體除錯、兩週考試中斷，請計算他一學期最多能完成幾章，並建議應選讀本書哪幾章。
2. 四人團隊有 12 週、預算 15000 元，欲製作一台具 SLAM 建圖功能的巡邏機器人。已知底盤 4000 元、Jetson 6000 元、二維光達 4000 元、電池與雜項 1500 元。請計算預算餘額、評估是否需要調整，並提出十二週的里程碑分工表（至少四個里程碑）。

動手做與專題延伸

本章實作：擬定你的個人學習計畫

完成下表，作為往後一年的學習契約。建議貼在書桌前，每月檢視一次進度。

表 2-9 個人學習計畫表 (範例列供參考)

項目	內容	自我評估 / 填寫
我的身分	大學生 / 高中生 / 社團學生 / 自學者	(填寫)
每週可投入時數	含閱讀、程式與動手做	(填寫)
目標讀完章節	例：一學期完成第 1 ~ 12 章	(填寫)
數學缺口	例：機率論尚未修過，第 24 章前需補	(填寫)
硬體層級	輕量級 / 中量級 / 重量級	(填寫)
第一圈螺旋專題	例：循跡小車，四週完成	(填寫)
第二圈螺旋專題	例：自平衡兩輪車，八週完成	(填寫)
檢核里程碑	例：第 8 週前跑通第一支 Python 控制程式	(填寫)

專題延伸

- 上網查閱本章提到的五所名校中任一所的機器人課程大綱 (多數公開於課程網站)，與本書目次逐章比對，列出三個本書涵蓋而該課程未涵蓋的主題，以及三個相反的情形。
- 訪問一位大三以上的學長姐或研究生，請教他們「如果重來一次，會怎麼安排機器人學的學習順序」，並將建議整理成五百字，與本章的建議比較異同。
- 進階：挑選一個你感興趣的機器人主題 (例如四足步態或視覺 SLAM)，用本章的四階段學習法，規劃一份為期十二週的自學計畫，明確列出每階段的產出與驗收標準。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson.
- Siciliano, B., Sciavicco, L., Villani, L. and Oriolo, G., Robotics: Modelling, Planning and Control, Springer.
- Lynch, K. M. and Park, F. C., Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press.
- Toussaint, M., Introduction to Robotics 課程講義, TU Berlin / University of Stuttgart.
- Thrun, S., Burgard, W. and Fox, D., Probabilistic Robotics, MIT Press.
- LaValle, S. M., Planning Algorithms, Cambridge University Press (線上全文公開)。
- Strang, G., Introduction to Linear Algebra, Wellesley-Cambridge Press (線性代數推薦教

材)。

- Macnab, A., "Spiral Curriculum" 相關教育學文獻；Bruner, J. S., The Process of Education, Harvard University Press (螺旋式課程的理論來源)。
- ROS 2 官方文件 (docs.ros.org) 與 Gazebo、Webots 官方教學資源。
- 各校公開課程資源：Stanford CS223A、MIT OpenCourseWare 機器人相關課程、CMU 16-311 / 16-833 課程網站、ETH Zürich ROS 2 課程。

台灣自編大學教科書

[@prof](#)

機器人學導論

機構、感知、控制與具身人工智慧

第一篇

機器人的身體與系統

第 3 章 機器人的組成與系統架構

第 4 章 機構、連桿與自由度

第 5 章 致動器、馬達與傳動系統

第 6 章 感測器與機器人的感官

第 7 章 機器人電子系統與嵌入式控制

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

2026 年 8 月

第一篇 篇章導讀

導讀篇建立了視野與方法，從這一篇開始，我們正式動手拆解機器人。這五章的任務只有一個：讓你能夠拿到一份需求規格，就畫得出系統方塊圖、算得出自由度、選得出馬達與感測器、設計得出電源與通訊架構，並列出完整的零件清單與成本。

編排的順序刻意由整體到局部。第 3 章先建立系統觀，把機器人拆解為七大子系統並理解它們之間的關係——這一步不能省略，因為先學零件再學系統的人，往往認得很多元件卻不知道怎麼組成一台會動的機器。有了系統的地圖，接下來四章才逐一深入：第 4 章分析機構與自由度，決定機器人能做什麼的物理上限；第 5 章選定致動器，賦予它力量；第 6 章配置感測器，賦予它感知；第 7 章則用電子系統把這一切連接起來，讓訊號與能量正確流動。

這一篇有一個貫穿全程的暗線：**故障**。第 3 章開場，一台醫院送藥機器人因為一條磨破的電線而報廢；第 5 章開場，一顆馬達被自己發的熱燒死；第 7 章開場，一台循跡車在比賽中反覆重開機，程式卻一行錯都沒有。這三則故事不是為了嚇唬讀者，而是要傳達一件教科書很少明說的事——機器人工程的成敗，往往不取決於你懂多少高深的演算法，而取決於你有沒有把最基本的物理現實照顧好。

這一篇要回答的六個問題

- 一台機器人由哪些子系統組成？它們如何分工與協作？
- 為什麼上位機與下位機必須分家？什麼任務算「即時」？
- 自由度是什麼？怎麼算？為什麼它不等於馬達數量？
- 馬達該選多大？為什麼幾乎每個關節都需要減速機？
- 各種感測器的物理原理決定了什麼樣的極限與失效模式？
- 為什麼超過一半的「程式問題」其實是電的問題？

讀完這一篇之後

你將擁有一位機器人系統工程師的基本功：能用工程師的語言描述一台機器人的完整構成、能做扭力與慣量的選型計算、能判讀感測器規格書中沒有明說的限制、能設計安全的電源與急停架構，並能在機器人故障時用系統性的方法快速定位問題。

但是，能組出一台會動的機器人，與能讓它精確地走到指定位置，中間還隔著一整片數學的

海洋。第二篇開始，我們將學習描述空間的語言——座標系、旋轉矩陣、齊次變換與運動學，把本篇建立的物理直覺，轉化為可以計算、可以最佳化、可以交給電腦執行的數學模型。

換句話說：第一篇教你把機器人組起來，第二篇開始教你把它算清楚。

本篇章節配置

章次	章名	核心內容	叫獸開講
第 3 章	機器人的組成與系統架構	七大子系統、上下位機、即時性、感知—規劃—控制架構	「機器人有骨骼、肌肉、神經和大腦嗎？」——解剖一台報廢的送藥機器人
第 4 章	機構、連桿與自由度	關節類型、Grübler 公式、串聯與並聯、冗餘、工作空間	「為什麼人手很靈活，機器手臂卻常卡住？」——手掌貼牆卻能抬起手肘
第 5 章	致動器、馬達與傳動系統	扭力轉速曲線、減速機、慣量匹配、選型七步驟	「同樣是馬達，為什麼玩具車馬達不能做機器手臂？」——被自己發的熱燒死
第 6 章	感測器與機器人的感官	編碼器、IMU 互補濾波、測距原理、規格指標、校正	「閉著眼睛也能摸到鼻子，機器人做得到嗎？」——沒有感測器會說它壞了
第 7 章	機器人電子系統與嵌入式控制	GPIO、ADC、PWM、通訊、電源壓降、EMI、中斷、急停	「明明接好了電線，為什麼一動就重新開機？」——電源在說謊

第一篇 機器人的身體與系統

第 3 章

機器人的組成與系統架構

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】一台被完全拆解的服務型機器人「解剖照」：鋁擠型骨架、兩顆帶減速機的馬達、主控板與驅動板、光達與相機、鋰電池、線束與紅色急停按鈕，分門別類地攤放在工作檯上，旁邊各以標籤註明其在系統中的角色——骨骼、肌肉、大腦、感官、心臟、神經、保險。

叫獸開講 「機器人有骨骼、肌肉、神經和大腦嗎？」

實驗室的解剖檯上，躺著一台壽終正寢的服務型機器人。它曾經在某家醫院的走廊送藥，服役三年，最後因為主機板燒毀而退役，被廠商捐給了叫獸的實驗室。今天下午的課，主題只有一個：把它拆開。

學生們圍成一圈，帶著手套與螺絲起子。第一層外殼卸下，露出一根鋁擠型的骨架，上面鑽滿了螺絲孔。「這是它的骨骼，」叫獸說，「決定它有多重、多硬、能承載多少東西。」接著是兩顆帶減速機的直流馬達，齒輪箱上還沾著三年份的灰塵。「這是肌肉。注意看，馬達本身轉得很快但沒有力氣，靠減速機把轉速換成扭力——就像人的肌肉配上槓桿一樣的骨頭。」

再往裡，一片主機板與四張小電路板現身。「這裡是大腦與脊髓。」叫獸指著中間那片：「這是主控電腦，跑地圖與路徑規劃，一秒鐘想十次。旁邊這四片是馬達驅動器，一秒鐘想一千次——它們負責把大腦說的『往前走』翻譯成每個輪子每毫秒該轉多快。」

有位學生舉手：「為什麼要分兩層？一台電腦全部做完不就好了？」叫獸拿起一條被燒黑的排線：「因為它們的時間尺度差一百倍。大腦可以偶爾卡頓零點一秒，你不會發現；但如果馬達控制迴路卡頓零點一秒，機器人就會撞牆。這叫『上位機與下位機』的分工，是所有機器人系統的共同架構。」

拆到最後，只剩一顆鋰電池、幾捆線與一個紅色的蘑菇頭按鈕。學生把零件在桌上排成一排：骨架、馬達、感測器、控制板、電池、線束、急停開關——七堆。叫獸環顧一圈：「現在請告訴我，這台機器人是怎麼死的？」

學生們翻找燒毀的主機板，卻在電池旁邊發現了答案：一條電源線的絕緣皮被機構的邊角磨破了，長期摩擦後短路，燒穿了電源模組，連帶燒毀主機板。「三年的研發、幾十萬的成本，」叫獸把那條線舉起來，「死於一條沒有加保護套管的電線。這就是為什麼我們要用一整章來談『系統』——因為機器人不是零件的加總，而是零件之間關係的總和。任何一個環節的疏忽，都能讓其他所有的精妙設計歸零。」

本章為什麼重要

第 1 章談了機器人是什麼，第 2 章談了怎麼學，從本章開始進入實質內容。本章的任務是建立「系統觀」——在深入每一個子系統（第 4~7 章的機構、馬達、感測器、電子）之前，先看

清楚它們如何組合成一台完整的機器人。這個順序很重要：如果先學馬達再學系統，你會知道很多零件卻不知道怎麼組；先建立系統觀，每學一個零件你都知道它在整體中的位置與責任。本章同時建立三個貫穿全書的重要概念：感知—規劃—控制的軟體分層、上位機與下位機的硬體分工、以及即時與非即時運算的時間尺度差異。這三個概念會在第 29 章的 ROS 2、第 34 章的產線整合中反覆出現。

學習目標

- 能列舉機器人系統的七大子系統，並說明各自的功能與相互關係。
- 能繪製一台機器人的系統方塊圖，標示訊號流與功率流。
- 能區分控制器、嵌入式電腦與微控制器的角色差異與選型考量。
- 能說明感測器系統的訊號鏈路：物理量、轉換、調理、取樣到數值。
- 能計算機器人的功率需求與電池續航時間。
- 能比較常見通訊介面的頻寬、距離與適用場合。
- 能說明感知—規劃—控制三層架構的資料流與各層時間尺度。
- 能解釋即時系統的意義，並判斷哪些任務必須即時、哪些不必。
- 能說明上位機與下位機的分工原則，並舉出實際的硬體組合。
- 能比較集中式與分散式機器人架構的優缺點與適用時機。

先備知識

第 1 章的感知—思考—行動循環概念。本章涉及少量電學計算（歐姆定律、功率與電池容量），若不熟悉可先閱讀附錄 D.1 的基本電學速成，或在閱讀 3.5 節時再回頭補充。本章不需要任何微積分或矩陣運算。

核心名詞中英對照

表 3-1 本章核心名詞

中文	英文	說明
子系統	Subsystem	構成完整系統的功能單元
機械本體	Mechanical Structure	機器人的骨架、連桿與外殼

中文	英文	說明
控制器	Controller	執行控制演算法、輸出指令的運算單元
微控制器	Microcontroller Unit, MCU	整合處理器與周邊介面的單晶片
單板電腦	Single Board Computer, SBC	具作業系統的小型電腦，如 Raspberry Pi
驅動器	Driver	將控制訊號放大為馬達所需功率的電路
訊號調理	Signal Conditioning	放大、濾波、隔離感測器訊號的處理
類比數位轉換器	Analog-to-Digital Converter, ADC	將連續電壓轉為數值的元件
取樣頻率	Sampling Rate	每秒讀取訊號的次數
即時系統	Real-Time System	必須在規定時限內完成回應的系統
控制週期	Control Cycle	控制迴路重複執行一次的時間間隔
上位機	Host Computer / High-Level Controller	負責規劃與決策的高階運算單元
下位機	Low-Level Controller	負責即時控制與硬體介面的運算單元
現場匯流排	Fieldbus	工業設備間的即時通訊網路，如 CAN、EtherCAT
系統方塊圖	System Block Diagram	以方塊與箭頭表示子系統與訊號流的圖

3.1 機器人的機械本體

3.1.1 骨架的三個任務

機械本體是機器人的骨骼與軀殼，承擔三個任務。第一是支撐：承受自身重量、負載重量與運動時的慣性力。第二是定位：保證各個關節與感測器的相對位置精確且穩定——如果相機的安裝支架會晃動，再好的視覺演算法也算不準物體位置。第三是保護：外殼隔絕粉塵、水氣與碰撞，同時避免人員接觸到運動件與高壓電。

這三個任務常常互相衝突。要支撐得穩就要粗壯，但粗壯就重；要保護周全就要密封，但密封就散熱困難。機器人的機構設計，本質上是在剛性、重量、成本與可維護性之間找平衡點——第 4 章與第 27 章會深入這個主題。

3.1.2 常見的結構形式與材料

實務上常見的結構形式有：鋁擠型骨架（模組化、易改裝，教學與研發首選）、鈹金外殼

(量產成本低、剛性佳)、鑄件(大型工業手臂的關節座，剛性最高但開模成本高)、碳纖維(輕量高剛性，用於無人機與競賽機器人)、以及 3D 列印件(快速原型與異形零件)。

材料的選擇有一個容易被忽略的關鍵指標：比剛度(剛度除以密度)。對移動機器人與機械手臂而言，增加的每一克重量都要消耗能量去搬運，並降低可用負載——這就是為什麼賽車與機器人都愛用鋁與碳纖維，而不是更便宜的鋼。

3.1.3 一個新手常犯的錯：忽略累積誤差

學生自製機器手臂時，常常每一節都做得「差不多」，結果末端偏差達數公分。原因是誤差會沿著運動鏈累積：第一關節的安裝偏差零點五度，經過三十公分的連桿放大後，末端就偏了將近三公釐；三個關節加起來，誤差可能達到一公分以上。這個現象在第 9 章的正向運動學中會有數學上的解釋——連桿越長、關節越多，對安裝精度的要求越高。

◎ 拆解練習的價值

如果你的實驗室或社團有報廢的印表機、掃地機器人或影印機，請務必拆開它。你會看到：工程師如何用最便宜的方式達成定位(塑膠齒輪加光遮斷器)、如何佈線避免磨損(套管與束帶)、如何做防呆(接頭形狀不同就插不進去)。這些細節在教科書裡佔不到半頁，卻是量產工程的精華。

3.2 控制器與嵌入式電腦

3.2.1 三個層級的運算單元

機器人的「大腦」通常不是一顆晶片，而是一組分工的運算單元。依運算能力與即時性由低到高排列，可分為三個層級。

微控制器(MCU)：如 Arduino 使用的 ATmega、STM32、ESP32。特點是內建類比數位轉換器、PWM 輸出、UART / I²C / SPI 介面，開機即執行程式，沒有作業系統的延遲，時序極為穩定。適合做馬達控制迴路、感測器讀取、安全監控。運算能力有限(通常數十至數百 MHz、數十至數百 KB 記憶體)，無法跑影像辨識。

單板電腦(SBC)：如 Raspberry Pi、樹莓派同級產品。具備完整的 Linux 作業系統，可跑 Python、OpenCV 與 ROS 2，能處理影像、地圖與網路。缺點是作業系統會造成不可預測的延遲(排程、記憶體管理)，不適合直接做毫秒級的控制迴路。

邊緣 AI 運算平台：如 NVIDIA Jetson 系列，在單板電腦的基礎上加入 GPU 或專用 AI 加速

器，能即時執行深度學習模型（物件偵測、語意分割、VLA 模型）。功耗與成本較高，通常只在需要視覺 AI 的機器人上。

表 3-2 三個層級運算單元的比較

層級	代表產品	運算能力	即時性	典型任務	本書章節
微控制器 MCU	Arduino、 STM32、ESP32	低（MHz 級）	極佳（無 OS 延遲）	馬達控制、感測器讀 取、急停監控	第 7、16 章
單板電腦 SBC	Raspberry Pi 5	中（GHz 級 CPU）	普通（OS 排 程延遲）	路徑規劃、SLAM、 ROS 2 節點	第 25、29 章
邊緣 AI 平 台	NVIDIA Jetson 系 列	高（含 GPU / NPU）	普通	影像辨識、深度學習 推論、VLA	第 22、31 ~ 33 章

3.2.2 選型的三個問題

面對琳瑯滿目的開發板，選型只需回答三個問題。第一，最嚴苛的即時要求是多少？如果需要一毫秒的控制迴路，就必須有 MCU。第二，最重的運算任務是什麼？如果要跑 YOLO 物件偵測，就需要 GPU 或 NPU。第三，開發時間有多少？MCU 的開發較繁瑣但穩定，SBC 上用 Python 開發極快但需處理作業系統的複雜性。

多數實用的機器人採用組合方案：SBC 或 Jetson 當上位機負責規劃與感知，MCU 當下位機負責即時控制，兩者用 UART 或 CAN 連接。這正是 3.8 節要展開的主題，也是開頭故事中那台送藥機器人的架構。

3.3 感測器系統

3.3.1 訊號鏈路：從物理世界到程式變數

感測器不是插上去就有數字。一個完整的感測器系統包含五個環節，任何一環出問題，程式讀到的都是垃圾。

第一是換能：把物理量轉成電訊號。應變片把力轉成電阻變化、光電二極體把光轉成電流、編碼器把角度轉成脈波。第二是訊號調理：微弱的訊號需要放大，高頻雜訊需要濾除，高壓或不同接地的訊號需要隔離。第三是類比數位轉換（ADC）：把連續的電壓轉成數值，解析度（幾位元）決定了能分辨多細的差異。第四是取樣：以固定頻率重複讀取，取樣頻率必須至少是訊號最高頻率的兩倍（Nyquist 定理），否則會產生混疊失真。第五是數值處理：單位換算、校正、濾

波，最後才成為程式裡那個叫做 distance 或 angle 的變數。

圖 3-1 感測器訊號鏈路

建議配圖：由左至右五個方塊——物理量（力／光／角度）→ 換能器 → 訊號調理（放大、濾波、隔離）→ ADC 與取樣 → 數值處理（校正、濾波）→ 程式變數。下方標註各環節常見的問題：靈敏度不足、雜訊、解析度不夠、混疊、單位錯誤。

3.3.2 內部感測與外部感測

如第 1 章所述，感測器分為兩大類。內部感測告訴機器人「我自己怎麼了」：關節角度（編碼器）、姿態與角速度（IMU）、馬達電流（間接反映負載與碰撞）、電池電壓。外部感測告訴機器人「外面有什麼」：距離（超音波、紅外線、光達）、影像（相機、深度相機）、力與接觸（力覺感測器、觸覺開關）、位置（GNSS、UWB 室內定位）。

一個關鍵的工程原則是：內部感測是控制的基礎，外部感測是自主的基礎。沒有編碼器，馬達連轉了幾度都不知道，連 PID 都做不了；沒有外部感測，機器人只能重播固定動作，無法應對環境變化。這也解釋了為什麼工業手臂可以只有編碼器（環境固定），而移動機器人必須有豐富的外部感測（環境未知）。

3.3.3 感測器融合的必要性

沒有任何單一感測器是完美的。編碼器精確但會累積誤差（輪子打滑時它一無所知）；IMU 反應快但會漂移；GNSS 不漂移但在室內失效且更新慢；相機資訊豐富但怕光線變化。實務上的解法是融合：用多個互補的感測器彼此校正。輪速計提供短期精確的位移、IMU 提供快速的姿態變化、光達提供絕對的環境參照，三者融合後的定位精度遠優於任何單一來源。這正是第 24 章 Kalman 濾波器的核心價值。

3.4 致動器與驅動器

3.4.1 致動器：能量轉換的終點

致動器把電能（或液壓、氣壓的能量）轉換成機械運動。機器人上最常見的是電動致動器：直流馬達（便宜、控制簡單）、無刷直流馬達（效率高、壽命長、功率密度大）、步進馬達（開迴路即可定位、低速扭力大）、伺服馬達（內建位置回授，直接下角度指令）。液壓致動器功率密度極高，用於大型工程機械與早期的人形機器人；氣壓致動器則柔順、便宜，常用於夾爪。第 5 章會完整比較。

3.4.2 驅動器：被忽略的關鍵環節

控制器輸出的是幾毫安培的訊號，馬達需要的是幾安培甚至數十安培的電流——中間必須有驅動器 (Driver) 做功率放大。這是初學者最常忽略、卻最常燒毀的環節。

驅動器的核心是功率電晶體組成的 H 橋電路，透過 PWM 訊號控制導通比例，決定馬達的等效電壓與轉速；改變橋臂的導通組合，則決定轉向。選用驅動器時必須注意三個規格：持續電流（能長時間承受的電流）、峰值電流（啟動或堵轉瞬間的電流，通常是持續電流的數倍）、以及散熱能力。學生常見的失敗是：只看馬達的額定電流選驅動器，忽略了堵轉電流可能是額定值的五到十倍，結果第一次撞牆就燒掉驅動器。

3.4.3 致動器與機構的匹配

馬達不是越大越好。選型時要同時考慮：所需扭力（含加速時的慣性扭力，不只是靜態負載）、所需轉速、減速比（減速機把高轉速低扭力換成低轉速高扭力）、以及慣量匹配（馬達轉子慣量與負載慣量相差太大時，控制會不穩定或反應遲鈍）。這幾項的計算會在第 5 章展開，本章只需建立一個觀念：致動器、減速機與機構是一個整體，不能分開選。

3.5 電源、電池與電源管理

3.5.1 為什麼電源是機器人的第一殺手

在教學現場，機器人「動不了」的原因排行榜上，電源問題長年高居第一。原因是電源同時牽涉三個容易出錯的面向：電壓不對（燒毀元件或帶不動）、電流不足（一啟動就重開機）、以及接地不當（訊號亂跳、通訊中斷）。而且電源問題的症狀往往偽裝成別的問題——程式看起來當機、感測器讀值突然亂跳、通訊莫名斷線，追查半天才發現是電池電壓在馬達啟動時瞬間掉到臨界值以下。

3.5.2 常見電池與關鍵規格

機器人常用的電池有三類。鋰聚合物電池 (LiPo) 放電電流大、能量密度高，是無人機與競賽機器人的主流，但過充過放會膨脹起火，必須使用專用充電器與平衡充。鋰離子電池 (Li-ion ，如 18650) 能量密度高、循環壽命長，適合續航需求高的移動機器人。鉛酸電池笨重但便宜且耐用，多見於大型 AGV 與工業設備。

選電池時要看四個規格：標稱電壓（決定能不能直接驅動馬達）、容量（以 mAh 或 Ah 表

示，決定續航）、放電倍率（C 數，決定能瞬間輸出多大電流）、以及重量。一個常被忽略的陷阱是放電倍率：一顆 2000 mAh、10C 的電池最大可放 20 A，但同樣容量 1C 的電池只能放 2 A——後者接上大電流馬達，電壓會瞬間崩塌。

3.5.3 電源架構：分離與穩壓

良好的機器人電源架構有兩個原則。第一是分離：功率電源（馬達、加熱器）與訊號電源（微控制器、感測器）盡量分開供電，或至少在電源入口分開走線，最後單點共地。這能避免馬達的電流突波拉低控制器電壓。第二是穩壓：電池電壓會隨放電下降（鋰電池從 4.2 V 降到 3.0 V），必須經過降壓（Buck）或升壓（Boost）模組穩壓後，才供給需要穩定電壓的元件。

此外還需要保護電路：過流保護（保險絲）、過放保護（低電壓切斷）、逆接保護（防止電池接反），以及反電動勢保護（馬達停止瞬間會產生反向電壓，需以二極體或電容吸收）。這些元件加起來成本不到兩百元，卻能保住價值數萬元的主控板。

圖 3-2 機器人電源架構示意圖

建議配圖：電池 → 主開關與保險絲 → 分為兩路：上路經降壓模組供給控制器與感測器（訊號電源，5 V / 3.3 V），下路直接供給馬達驅動器（功率電源）；兩路在電池負極單點共地。急停開關串接於功率電源路徑上。

3.5.4 續航估算

續航時間的粗略估算方式為：續航（小時）= 電池容量（Ah）× 可用比例 ÷ 平均消耗電流（A）。可用比例通常取 0.8（保留 20% 避免過放）。平均消耗電流需分項估算：主控電腦、感測器、馬達（依工作週期加權）。例題 3-1 會完整示範這個計算。

實務上有兩個經驗法則。第一，馬達的平均消耗遠低於峰值，但電源系統必須按峰值設計。第二，實測續航通常比計算值短 20% 至 30%，因為電池老化、低溫、與轉換效率損失。設計時務必留有餘裕。

3.6 通訊系統

3.6.1 機器人內部的三層通訊

一台機器人內部的通訊可分為三層。晶片級：微控制器與感測器晶片之間，使用 I²C（兩線、可掛多個裝置、速度較慢）或 SPI（四線、速度快、適合高速感測器如 IMU）。板級：控制板之間，使用 UART（簡單的點對點序列通訊）或 CAN Bus（多節點、抗雜訊強、有優先權機制，汽

車與工業標準)。系統級：上位機與外部世界之間，使用乙太網路、WiFi 或 5G。

工業機器人與高階系統則使用現場匯流排：EtherCAT (基於乙太網的即時協定，可達微秒級同步，適合多軸協調)、PROFINET、EtherNet/IP。這些協定的共同特徵是「確定性」——不只是快，而是每次傳輸的延遲都可預測，這對多軸同步至關重要。

表 3-3 常見通訊介面比較

介面	線數	速度	距離	多節點	典型用途
I ² C	2	100k ~ 1M bps	數十公分	可 (位址定址)	感測器、EEPROM
SPI	4+	數 M ~ 數十 Mbps	數十公分	可 (片選)	高速 IMU、SD 卡、顯示器
UART	2	9.6k ~ 1M bps	數公尺	否 (點對點)	模組通訊、除錯訊息
CAN Bus	2	125k ~ 1 Mbps	數十至數百公尺	可 (廣播 + 優先權)	多馬達驅動器、車輛
乙太網路	4 ~ 8	100M ~ 1 Gbps	百公尺	可 (交換機)	相機、光達、上位機
EtherCAT	4 ~ 8	100 Mbps (微秒級同步)	百公尺	可 (串接)	工業多軸即時控制
WiFi / 藍牙	無線	數 M ~ 數百 Mbps	數十公尺	可	遙控、監控、資料回傳

3.6.2 選擇通訊介面的三個考量

選介面時考慮三件事。第一是頻寬需求：一顆 IMU 每秒送幾百筆小資料，用 I²C 綽綽有餘；一台深度相機每秒送數十 MB，就必須用 USB 3 或乙太網路。第二是即時性需求：控制指令必須準時抵達，適合 CAN 或 EtherCAT；影像串流可以容忍偶爾延遲，用 WiFi 即可。第三是抗雜訊能力：機器人內部有大電流馬達，長距離的訊號線容易受干擾，CAN 的差動訊號比 UART 的單端訊號可靠得多。

◎ 無線通訊的陷阱

許多學生把 WiFi 用在控制迴路上——遙控時感覺良好，比賽時卻因為現場數百支手機的干擾而失控。原則是：安全相關與即時控制絕不依賴無線；無線只用於監控、遙測與非即時的指令下達。急停功能更必須是硬體線路，不能透過任何通訊協定。

3.7 軟體與演算法

3.7.1 機器人軟體的三層堆疊

機器人的軟體同樣分層。最底層是韌體與驅動層：直接操作硬體暫存器，讀取編碼器、輸出 PWM、處理中斷。中間層是控制與狀態估測層：PID 迴路、運動學計算、感測器融合、里程計。最上層是規劃與決策層：路徑規劃、任務排程、行為決策、人機介面。

這種分層有兩個好處。第一是可替換性：換一顆不同的馬達，只需改韌體層，上層演算法不動。第二是可測試性：每一層可以獨立驗證——先確認編碼器讀值正確，再測試 PID 是否收斂，最後才測整體路徑追蹤。跳過分層測試而直接跑全系統，是初學者最常見的除錯災難。

3.7.2 中介框架：為什麼需要 ROS 2

當機器人的軟體模組超過五個，就會面臨一個問題：模組之間如何交換資料？自己寫通訊機制不僅費時，還很難支援跨語言（C++ 與 Python 混用）與跨機器（上位機與遠端電腦）。ROS 2 正是為此而生的中介框架：它提供節點（獨立的程式模組）、話題（發布訂閱式的資料流）、服務（請求回應式的呼叫）、動作（有進度回報的長時間任務）等機制，讓模組化開發變得標準化。

此外，ROS 2 的生態系提供了大量現成套件：SLAM、路徑規劃、感測器驅動、視覺化工具，可以直接站在巨人的肩膀上。第 29 章會完整介紹，本章只需理解它在系統中的定位——它是軟體模組之間的「神經系統」。

3.7.3 狀態機：機器人行為的骨架

除了演算法，機器人軟體還需要一個行為的組織方式，最常見的是有限狀態機（Finite State Machine）。以送藥機器人為例，狀態可能包括：待命、前往取藥點、等待裝載、前往病房、等待取藥、返回充電。每個狀態有明確的進入條件、執行內容與離開條件。狀態機的價值在於行為變得可預測、可測試、可除錯——當機器人卡住時，第一個問題永遠是「它現在在哪個狀態？」

3.8 上位機與下位機

3.8.1 為什麼要分工

開頭故事中學生的疑問——「為什麼不用一台電腦全部做完」——答案是時間尺度。控制迴路需要每一毫秒執行一次且不能延遲，這是即時任務；路徑規劃可能需要幾十毫秒計算一次，偶爾慢一點無妨，這是非即時任務。把兩者放在同一個作業系統上，作業系統的排程隨時可能為了處理網路封包而打斷控制迴路，導致馬達失控。

因此標準做法是硬體上分家：上位機（SBC 或工控機，跑 Linux 與 ROS 2）負責感知、地圖、規劃與人機介面；下位機（MCU，裸機或即時作業系統）負責馬達控制迴路、感測器高頻取樣、安全監控。兩者以 UART、CAN 或乙太網路連接，上位機下達「速度指令」，下位機回傳「編碼器與狀態」。

圖 3-3 上位機與下位機的分工架構

建議配圖：上半部為上位機（Jetson / Raspberry Pi）方塊，內含 SLAM、路徑規劃、視覺辨識、ROS 2 節點；下半部為下位機（STM32 / Arduino）方塊，內含 PID 控制迴路、編碼器讀取、急停監控。中間以雙向箭頭連接，上行標註「速度指令（10~50 Hz）」，下行標註「里程計與狀態回報（100~1000 Hz）」。下位機再連接馬達驅動器與感測器。

3.8.2 介面設計的原則

上下位機之間的介面設計，有三個實務原則。第一是抽象層級要一致：上位機應該說「以每秒零點五公尺前進、每秒零點二弧度左轉」，而不是「左輪 PWM 給 180、右輪給 150」——後者把機構細節洩漏到了上層。第二是要有超時保護：如果上位機當機或連線中斷，下位機必須在數百毫秒內自動停止馬達，而不是維持最後一個指令繼續衝出去。第三是狀態要雙向：下位機不只回傳感測值，也要回傳自身狀態（是否過流、是否急停、電池電壓），讓上位機能做出正確決策。

3.8.3 典型的硬體組合

依規模不同，常見的組合有：入門級——ESP32 單機兼任上下位（適合循跡小車，因為運算需求低）；中階——Raspberry Pi 5 加 STM32 或 Arduino，以 UART 連接（適合教學型 AMR）；高階——NVIDIA Jetson 加多顆馬達驅動器，以 CAN 或 EtherCAT 連接（適合研究型移動機器人與機械手臂）；工業級——工控機加專用運動控制卡，以 EtherCAT 連接伺服驅動器（適合產線設備）。

3.9 即時控制與非即時運算

3.9.1 什麼是即時

「即時」(Real-Time) 不等於「快」，而是「準時」。一個即時系統的定義是：不僅要算出正確答案，還必須在規定的時限 (deadline) 之前算出來，否則答案就失去價值。一個平均延遲一毫秒但偶爾延遲一百毫秒的系統，不是即時系統；一個穩定延遲五毫秒的系統，反而是。

即時性又分兩級。硬即時 (Hard Real-Time)：錯過時限會造成災難，例如手臂的力矩控制迴路、無人機的姿態控制、急停響應。軟即時 (Soft Real-Time)：錯過時限只降低品質，例如影像顯示掉幀、地圖更新慢一拍。分清楚哪些任務是硬即時，是系統設計的核心決策。

3.9.2 典型的時間尺度

機器人系統中，不同層級的控制週期有數量級的差異，理解這個「時間金字塔」對系統設計極為重要。電流迴路 (馬達驅動器內部)：約 10 ~ 100 微秒。速度與位置迴路 (下位機)：約 0.1 ~ 1 毫秒。運動學與軌跡插補：約 1 ~ 10 毫秒。狀態估測與定位：約 10 ~ 100 毫秒。路徑規劃：約 100 毫秒至數秒。任務決策與 AI 推論：數百毫秒至數秒。

這個金字塔解釋了許多設計選擇。為什麼馬達驅動器要有自己的處理器？因為 10 微秒的迴路無法交給上層。為什麼視覺不能直接進控制迴路？因為相機每秒三十幀 (33 毫秒)，比控制迴路慢了兩個數量級——這正是視覺伺服控制 (第 22 章) 的核心難題。

表 3-4 機器人系統的時間尺度金字塔

層級	典型週期	執行位置	即時性要求	範例任務
電流迴路	10 ~ 100 μ s	馬達驅動器	硬即時	PWM 換相、電流控制
速度 / 位置迴路	0.1 ~ 1 ms	下位機 MCU	硬即時	PID 控制、編碼器讀取
軌跡插補	1 ~ 10 ms	下位機或上位機	硬即時	運動學計算、軌跡生成
狀態估測	10 ~ 100 ms	上位機	軟即時	感測器融合、里程計、定位
路徑規劃	0.1 ~ 數秒	上位機	非即時	A*、RRT、避障重規劃
任務決策 / AI	數百 ms ~ 數秒	上位機 / 邊緣 AI	非即時	物件辨識、任務排程、LLM

3.9.3 如何達成即時性

達成即時性有三個層次的做法。最簡單的是裸機程式 (bare-metal)：MCU 上沒有作業系統，用計時器中斷觸發控制迴路，時序最精確，適合單一控制任務。其次是即時作業系統 (RTOS，如 FreeRTOS)：提供多任務與優先權排程，高優先權任務可搶佔低優先權任務，適

合任務較多的下位機。最複雜的是即時 Linux (PREEMPT_RT 補丁)：在完整的 Linux 上取得毫秒級的可預測性，適合需要同時跑 ROS 2 與即時控制的工控機。

此外還有一個常被忽略的實務要點：即時任務中絕對不能做「執行時間不可預測」的事——不要動態配置記憶體、不要印出除錯訊息、不要等待網路回應。這三件事是初學者破壞即時性的三大元凶。

3.10 感知—規劃—控制架構

3.10.1 三層架構的資料流

把前面各節整合起來，機器人的軟體系統呈現一個經典的三層架構：感知層、規劃層、控制層。這是第 1 章「感知—思考—行動」在工程實作上的具體化。

感知層負責回答「世界現在是什麼樣子」。它接收原始感測資料（影像、點雲、編碼器脈波、IMU 讀值），經過濾波、融合與特徵擷取，輸出一個結構化的世界模型：我在地圖上的哪個座標、周圍有哪些障礙物、目標物在哪裡。這一層的產出是「狀態」與「地圖」。

規劃層負責回答「我該做什麼」。它接收感知層的世界模型與任務目標，輸出一條可執行的軌跡或一串動作序列。這一層包含全域路徑規劃（從 A 到 B 怎麼走）、區域避障（前面突然出現行人怎麼繞）、以及任務層決策（先送藥還是先充電）。

控制層負責回答「怎麼讓身體照做」。它接收規劃層的軌跡，計算每個關節或輪子該輸出多少力矩或速度，並透過回授不斷修正誤差。這一層直接與硬體對話，也是即時性要求最高的一層。

圖 3-4 感知—規劃—控制三層架構

建議配圖：三個橫向色帶由上而下——感知層（相機、光達、IMU、編碼器 → 融合 → 狀態與地圖）、規劃層（任務目標 + 地圖 → 全域路徑 → 區域避障 → 軌跡）、控制層（軌跡 → 運動學 → PID → 驅動器 → 馬達）。右側標註各層的更新頻率；左側以一條回饋箭頭由馬達經環境回到感測器，形成閉環。

3.10.2 各層的時間尺度與耦合

三層的更新頻率遞增：規劃層最慢（0.1 ~ 數秒）、感知層居中（10 ~ 100 毫秒）、控制層最快（0.1 ~ 1 毫秒）。這個梯度是自然的——世界的變化沒有那麼快，但馬達的物理響應非常快。

但三層並非完全獨立。當感知層發現前方突然出現障礙物時，必須能快速觸發規劃層重新規

劃，甚至直接觸發控制層緊急煞車——這條「反射弧」通常繞過規劃層，直接由感知連到控制，就像人類碰到燙的東西會先縮手再思考。這種設計在機器人學中稱為反應式行為（reactive behavior），與深思熟慮的規劃式行為（deliberative behavior）互補。

3.10.3 架構的演變：從三層到端到端

值得注意的是，這個經典三層架構近年正受到挑戰。深度學習與 VLA 模型的興起，帶來了「端到端」（end-to-end）的可能性：直接把相機影像輸入神經網路，輸出馬達指令，中間不再有明確的地圖與規劃層。兩種路線各有優劣：三層架構可解釋、可除錯、可驗證安全性；端到端架構能處理難以建模的複雜情境，但像個黑盒子。目前的主流看法是混合——用學習處理感知與技能，用傳統架構保證安全與可靠。第 33 章會深入這個辯論。

3.11 集中式與分散式機器人架構

3.11.1 集中式：一個大腦說了算

集中式架構把所有運算集中在一台主控電腦上：它讀取所有感測器、執行所有演算法、輸出所有指令，周邊只有簡單的訊號轉換電路。優點是設計簡單、資料同步容易、除錯集中。缺點是：布線複雜（所有感測器的線都要拉到主控）、可靠性差（主控當機則全機癱瘓）、擴充困難（增加一顆馬達可能就得換主控板）。小型教學機器人多採此架構。

3.11.2 分散式：各司其職的節點網路

分散式架構把運算分散到多個節點：每顆馬達有自己的智慧驅動器（內含電流與速度迴路）、每組感測器有自己的處理模組，彼此以 CAN 或 EtherCAT 連接，主控只負責高層協調。優點是布線簡潔（一條匯流排串接所有節點）、可靠性高（單一節點故障可隔離）、易於擴充（新增節點只需接上匯流排並設定位址）。缺點是成本較高、需要處理通訊協定與時間同步。工業機器人、汽車與大型移動平台幾乎都採此架構。

表 3-5 集中式與分散式架構的比較

面向	集中式	分散式
運算位置	單一主控	多個智慧節點
布線	星狀，線束複雜	匯流排，串接簡潔
可靠性	低（單點故障即全癱）	高（可隔離故障節點）
擴充性	差（受主控 I/O 限制）	佳（掛上匯流排即可）

面向	集中式	分散式
成本	低	較高
除錯難度	低（狀態集中）	較高（需觀察匯流排）
典型應用	教學機器人、小型專題	工業機器人、AGV、汽車

3.11.3 混合式與實務選擇

實務上多數機器人採混合式：關鍵的即時控制分散到智慧驅動器，感知與規劃集中在一台上位機。選擇的原則是——依「軸數」與「距離」決定。軸數少（三軸以下）、距離短（同一塊板子上）採集中式即可；軸數多（六軸以上）、距離長（分佈在整台車上）則必須分散式。第 34 章討論產線整合時，會看到這個原則如何延伸到整個工廠的層級。

3.12 機器人的系統方塊圖

3.12.1 方塊圖的價值

系統方塊圖是機器人工程師的共同語言。它用方塊代表子系統、用箭頭代表訊號或功率的流向，讓整台機器人的架構在一頁紙上一目了然。它的價值有三：溝通（讓機構、電子與軟體工程師對齊認知）、設計（在動手前發現介面缺口與衝突）、除錯（故障時能快速定位到某個方塊或某條連線）。

3.12.2 畫方塊圖的五個步驟

步驟一，先畫出四個核心方塊：感測器、控制器、驅動器、致動器，並以箭頭連成閉環（別忘了從致動器經「環境」回到感測器的那條回饋線）。步驟二，加入電源系統，用不同顏色或粗線標示功率流，與訊號流區分開來。步驟三，若採上下位機架構，將控制器方塊拆成上位機與下位機，標註兩者的通訊介面與方向。步驟四，在每條連線上標註介面規格（I²C、CAN、PWM）與更新頻率。步驟五，補上安全相關元件：急停開關、保險絲、過流保護，並確認急停能直接切斷功率而不經過軟體。

圖 3-5 教學型自主移動機器人系統方塊圖範例

建議配圖：完整方塊圖。左側感測器群（光達、深度相機、IMU、編碼器）；中央上位機（Raspberry Pi 5：SLAM、路徑規劃、ROS 2）以 UART 連下位機（STM32：PID、里程計、急停監控）；右側馬達驅動器與兩顆直流馬達。下方電池經開關、保險絲、急停後分為功率路（粗線至驅動器）與訊號路（經降壓模組至上下位機與感測器），單點共地。各連線標註介面與頻率。

3.12.3 由方塊圖到規格表

方塊圖畫完後，下一步是把每個方塊與連線轉成規格表：每個元件的型號、電壓、電流、通訊介面、更新頻率、成本。這份表格就是採購清單與設計文件的基礎，也是第 36 章系統工程流程的起點。學生專題最常見的失敗，就是跳過這一步直接買零件——結果買回來才發現感測器是 5 V 訊號、主控是 3.3 V，或是驅動器的電流撐不住馬達的堵轉。

完整計算例題

例題 3-1 電池續航估算

題目：一台教學型 AMR 使用 12 V、10 Ah 的鋰電池。系統耗電如下：主控電腦（Raspberry Pi 5 含光達）平均 1.5 A；下位機與感測器 0.3 A；兩顆驅動馬達在行進時各 2.0 A，但工作週期（行進時間佔比）為 60%。取電池可用比例 80%。求理論續航時間。若實測續航為 1.8 小時，求折損率並分析可能原因。

解：先計算平均消耗電流。馬達平均電流 = $2 \text{ 顆} \times 2.0 \text{ A} \times 0.6 = 2.4 \text{ (A)}$ 。總平均電流 = $1.5 + 0.3 + 2.4 = 4.2 \text{ (A)}$ 。可用電量 = $10 \text{ Ah} \times 0.8 = 8 \text{ (Ah)}$ 。理論續航 = $8 \div 4.2 \approx 1.90 \text{ (小時)}$ ，約 1 小時 54 分。

實測 1.8 小時，折損率 = $(1.90 - 1.80) \div 1.90 \approx 5.3\%$ ，屬於合理範圍。可能原因包括：降壓模組的轉換效率（通常 85~92%，會多耗約 10% 的電）、電池老化導致實際容量低於標稱、地毯或斜坡使馬達電流高於估算值、以及低溫下鋰電池內阻上升。若折損率超過 20%，則應懷疑有異常耗電（如驅動器待機電流過高）或電池已嚴重老化。

例題 3-2 控制週期與取樣頻率設計

題目：某差速輪機器人的輪子編碼器每圈輸出 1000 個脈波，輪子直徑 10 公分，最高速度 1.0 m/s。（a）求最高速度時的脈波頻率。（b）若下位機的控制週期為 1 ms，每個週期能計數幾個脈波？此解析度對應的速度誤差為何？（c）若改用 10 ms 控制週期，速度解析度如何變化？

解：（a）輪子周長 = $\pi \times 0.1 \approx 0.314 \text{ (m)}$ 。最高速度時每秒轉數 = $1.0 \div 0.314 \approx 3.18 \text{ (轉/秒)}$ 。脈波頻率 = $3.18 \times 1000 \approx 3183 \text{ (Hz)}$ 。

（b）1 ms 內的脈波數 = $3183 \times 0.001 \approx 3.18 \text{ (個)}$ 。由於計數必須是整數，實際只會讀到 3 個或 4 個，量化誤差達 ± 1 個脈波。一個脈波對應的距離 = $0.314 \div 1000 = 0.314 \text{ (mm)}$ ，

在 1 ms 內對應的速度解析度 = $0.000314 \div 0.001 = 0.314$ (m/s) ——這高達最高速度的 31% , 完全無法用於精確的速度控制。

(c) 改用 10 ms 週期，脈波數 ≈ 31.8 個，量化誤差仍為 ± 1 個脈波，但速度解析度變為 $0.000314 \div 0.01 = 0.0314$ (m/s)，僅為最高速度的 3.1%，改善了十倍。

結論與工程啟示：控制週期越短，控制反應越快，但速度解析度越差——這是一組必須權衡的矛盾。實務解法有三：一是提高編碼器線數（如改用 4000 線）；二是採用邊緣計時法（測量脈波之間的時間間隔而非計數）；三是採用串級控制，內層電流迴路用高頻、外層速度迴路用較低頻。這個「快與準」的權衡，會在第 16 章的數位控制中再次出現。

程式範例：系統健康監控

以下 Python 程式模擬一個機器人的系統監控模組：定期檢查各子系統的狀態，並在異常時分級告警。這是每一台實用機器人都應具備的基本功能——開頭故事中那台送藥機器人，若有電源異常監控，或許就不會因為一條磨破的電線而報廢。

```
import time
import random

# 各子系統的安全門檻
LIMITS = {
    "battery_v": (11.0, 12.6), # 電池電壓 (V) : 低於 11.0 需警告
    "motor_a": (0.0, 4.0), # 馬達電流 (A) : 超過 4.0 疑似堵轉
    "cpu_temp": (0.0, 75.0), # 主控溫度 (°C)
    "comm_delay": (0.0, 50.0), # 上下位機通訊延遲 (ms)
}

def read_sensors():
    """模擬讀取各子系統狀態（實機上改為真實感測器讀值）"""
    return {
        "battery_v": random.uniform(10.5, 12.6),
        "motor_a": random.uniform(0.5, 5.0),
        "cpu_temp": random.uniform(45.0, 80.0),
        "comm_delay": random.uniform(5.0, 60.0),
    }

def check_health(readings):
    """檢查狀態，回傳（等級，訊息）清單"""
    alerts = []
    for key, value in readings.items():
        lo, hi = LIMITS[key]
        if value < lo:
```



```

        alerts.append(("嚴重", f"{key} 過低：{value:.1f} ( 下限 {lo} ) "))
    elif value > hi:
        level = "嚴重" if value > hi * 1.1 else "警告"
        alerts.append((level, f"{key} 過高：{value:.1f} ( 上限 {hi} ) "))
    return alerts

def main_loop(cycles=5):
    for i in range(cycles):
        readings = read_sensors()
        alerts = check_health(readings)
        print(f"--- 第 {i+1} 次檢查 ---")
        if not alerts:
            print("所有子系統正常")
        for level, msg in alerts:
            print(f"[{level}] {msg}")
            if level == "嚴重":
                print(" → 動作：停止馬達、進入安全狀態、回報上位機")
        time.sleep(0.2)

main_loop()

```

請試著修改：加入一個「連續三次警告才升級為嚴重」的邏輯，避免單次雜訊造成誤停機。這個「去彈跳」的概念，在真實的安全監控系統中極為重要。

ROS 2 與模擬實作：畫出你的系統方塊圖

本章的實作不寫程式，而是設計。請依以下步驟，為你打算製作的機器人（或附錄中的任一）繪製完整的系統方塊圖與規格表：

1. 在紙上或繪圖軟體（建議 draw.io 或 Fritzing）畫出四個核心方塊：感測器、控制器、驅動器、致動器，並連成閉環。
2. 加入電源系統：電池、開關、保險絲、降壓模組、急停開關。以粗線標示功率流、細線標示訊號流，並標出單點共地位置。
3. 決定是否採用上下位機架構。若是，將控制器拆為兩個方塊並標註通訊介面（UART / CAN）與方向。
4. 在每條連線上標註介面規格與更新頻率，例如「I²C, 100 Hz」「PWM, 20 kHz」「UART 115200, 50 Hz」。
5. 將方塊圖轉為規格表：元件名稱、型號、工作電壓、額定電流、通訊介面、預估單價，並加總成本。

6. 檢查三件事：（一）所有元件的電壓是否相容？（二）電源的總電流是否足夠（含峰值）？（三）急停是否直接切斷功率、不經過軟體？

（進階）若已安裝 ROS 2，可執行 `rqt_graph` 觀察一個現成範例的節點圖——你會發現它就是本章方塊圖的軟體版本。第 29 章會教你如何為自己的系統建立這張圖。

國中小也看得懂：機器人的身體和我們一樣

機器人就像我們的身體，也有好幾個「系統」在一起工作。骨頭是機器人的鋁架，撐住身體不讓它垮掉。肌肉是馬達，負責出力讓輪子轉、手臂動。眼睛耳朵是感測器，看得到前面有沒有東西、知道自己轉了幾圈。大腦是控制板，負責想「該往哪走」。血管是電線，把電池的電送到每個地方。神經是訊號線，把眼睛看到的東西告訴大腦，再把大腦的命令傳給肌肉。

最重要的一件事：這些系統要「合作」才有用。如果眼睛很好但大腦沒接到線，機器人還是會撞牆；如果大腦很聰明但電池沒電，它就一動也不動。所以做機器人時，不能只顧一個部分做得很棒，要讓每個部分都健康、而且好好連在一起。這就叫做「系統」。下次你的小車不會動，記得照順序檢查：有電嗎？線接好了嗎？眼睛有看到嗎？大腦有想嗎？肌肉有動嗎？

叫獸提醒與常見錯誤

- 誤區一：先買零件再想架構。應該先畫方塊圖、列規格表，確認電壓與介面相容後再採購——順序反了會浪費大量金錢與時間。
- 誤區二：只看馬達額定電流選驅動器。堵轉電流可能是額定的五到十倍，驅動器必須按峰值選，並加上過流保護。
- 誤區三：訊號線與功率線網在一起走。馬達的電流突波會透過電磁耦合干擾訊號線，導致感測器讀值亂跳。兩者應分開走線或加隔離。
- 誤區四：把急停做成軟體功能。急停必須是硬體迴路，直接切斷功率電源。任何經過軟體或通訊協定的「急停」，在系統當機時都會失效。
- 誤區五：把即時控制放在跑作業系統的電腦上。Linux 的排程延遲不可預測，毫秒級控制迴路請交給 MCU 或即時作業系統。
- 誤區六：忽略通訊超時保護。上位機當機時，下位機若沒有超時停止機制，機器人會維持最後指令一路衝出去。

- 誤區七：不做分層測試。請依序驗證：電源正常 → 感測器讀值正確 → 單軸控制收斂 → 多軸協調 → 全系統。跳級測試會讓除錯時間暴增。
- 誤區八：布線不做保護。開頭故事的教訓——線束經過機構邊角處必須加套管、固定與圓角處理，這是最便宜的可靠度投資。

課堂討論

1. 開頭故事中的送藥機器人死於一條磨破的電線。若你是設計者，會用哪三項具體措施防止此事？請分別從機構、電子與軟體監控三方面提出。
2. 為什麼掃地機器人可以用集中式架構，而六軸工業手臂幾乎都用分散式？請從軸數、距離與可靠度三方面分析。
3. 端到端的 AI 控制（影像直接輸出馬達指令）省略了明確的規劃層。你認為在什麼場合可以接受？什麼場合絕對不行？
4. 如果你只能為機器人增加一項「監控」功能來提升可靠度，你會選什麼？請說明理由與實作方式。
5. 比較人體與機器人的系統架構：人類的「下位機」在哪裡（提示：脊髓反射）？這個對照對機器人設計有什麼啟發？

章末摘要

- 機器人由七大子系統組成：機械本體、控制器、感測器、致動器與驅動器、電源、通訊、軟體；系統的價值不在零件加總，而在關係的總和。
- 運算單元分三層：MCU（即時性佳、運算低）、SBC（可跑 OS 與 ROS 2）、邊緣 AI 平台（可跑深度學習）；實用機器人多採組合方案。
- 感測器訊號鏈路有五環：換能、訊號調理、ADC、取樣、數值處理；任一環出錯，程式讀到的都是垃圾。
- 驅動器必須按峰值電流（堵轉可達額定五至十倍）選型，並具備過流與散熱設計。
- 電源是機器人故障排行榜第一名；架構原則為功率與訊號分離、單點共地、加裝穩壓與保護電路；續航估算須保留 20 ~ 30% 餘裕。
- 通訊介面依頻寬、即時性與抗雜訊選擇；安全相關與即時控制絕不依賴無線，急停必須是硬

體迴路。

- 軟體分三層：韌體驅動、控制與狀態估測、規劃與決策；ROS 2 是模組間的神經系統，有限狀態機是行為的骨架。
- 上下位機分工的理由是時間尺度差異；介面設計三原則為抽象層級一致、超時保護、狀態雙向回報。
- 即時不等於快，而是準時；系統存在時間尺度金字塔，由電流迴路（微秒）到 AI 決策（秒），跨越六個數量級。
- 感知—規劃—控制三層架構是第 1 章循環的工程實作；集中式適合小型系統，分散式適合多軸長距離；系統方塊圖是工程師的共同語言。

觀念題與計算題

觀念題

1. 列舉機器人的七子系統，並各用一句話說明其功能。
2. 比較 MCU、SBC 與邊緣 AI 平台的差異，並各舉一個適合的任務。
3. 畫出感測器的五環訊號鏈路，並在每一環標出一個常見的失效模式。
4. 為什麼說「內部感測是控制的基礎，外部感測是自主的基礎」？請以工業手臂與移動機器人各舉一例說明。
5. 說明驅動器在系統中的角色，以及選型時必須注意的三個規格。
6. 電源架構的兩大原則是什麼？為什麼馬達啟動會導致微控制器重開機？
7. 比較 I²C、CAN Bus 與 EtherCAT 三種介面的特性與適用場合。
8. 解釋「即時不等於快」的意義，並區分硬即時與軟即時，各舉兩個機器人上的例子。
9. 說明上位機與下位機分工的理由，以及介面設計的三個原則。
10. 比較集中式與分散式架構的優缺點，並說明選擇的判準。
11. 說明感知—規劃—控制三層架構的資料流，以及「反射弧」為何要繞過規劃層。
12. 畫系統方塊圖的五個步驟為何？為什麼急停必須畫在功率路徑上而非訊號路徑上？

計算題

1. 某巡檢機器人使用 24 V、20 Ah 電池。耗電為：主控 2.0 A、光達 0.5 A、四顆馬達各 1.5 A

(工作週期 70%)、雲台 0.3 A (工作週期 20%)。電池可用比例取 80%，降壓模組效率 90%。求理論續航時間 (提示：訊號側耗電需除以轉換效率)。

2. 某機械手臂關節使用 2500 線編碼器 (每圈 2500 脈波)，減速比 50:1，馬達最高轉速 3000 rpm。(a) 求關節端的最高轉速 (度 / 秒)。(b) 求馬達端最高脈波頻率。(c) 若控制週期為 0.5 ms，計算關節角度的量化解析度 (度)。(d) 若要求角度解析度優於 0.01 度，編碼器線數至少需多少？
3. 某移動機器人採 CAN Bus 連接 6 個節點，每個節點每 10 ms 送出一則 8 位元組的訊息 (連同協定開銷共約 130 位元)。(a) 求匯流排的平均使用率 (以 500 kbps 計)。(b) 若要新增 4 個相同節點，使用率變為多少？是否仍在建議的 40% 安全上限內？

動手做與專題延伸

本章實作：機器人解剖與系統還原

找一台報廢或閒置的電子設備 (掃地機器人、噴墨印表機、光碟機、玩具遙控車皆可)，在安全前提下拆解，並完成下表。拆解前請先斷電並取出電池；若含高壓元件 (如印表機的電源模組、CRT) 請勿拆解。

表 3-6 解剖記錄表 (範例列供參考)

子系統	找到的零件	規格 / 型號	在系統中的角色	設計巧思或缺陷
機械本體	塑膠底盤與金屬支架	ABS 塑膠、厚約 2 mm	支撐與保護內部元件	以卡榫取代螺絲，組裝快但難維修
致動器	直流馬達加塑膠齒輪箱	6 V、減速比約 1:48	驅動輪子前進與轉向	齒輪為塑膠，成本低但易磨損
(自行填寫其餘五個子系統)				

完成後，請依 3.12 節的五步驟，把你拆到的零件還原成一張系統方塊圖，並回答：這台設備採用集中式還是分散式架構？它有沒有上下位機的分工？

專題延伸

- 為附錄 F 的循跡小車繪製完整系統方塊圖與規格表，並計算其電池續航時間。實測後比較理論值與實測值的差異，分析原因。

- 訪查一家自動化設備廠商或參觀工廠，記錄一台工業機器人的系統架構：主控是什麼？用什麼匯流排？急停如何設計？安全圍籬與感測如何配置？寫成一千字的參訪報告。
- 進階：為一台假想的「校園導覽機器人」做完整的系統設計——列出需求（續航四小時、能上下斜坡、能辨識人臉、能語音互動），據此選定各子系統的元件、繪製方塊圖、估算成本與續航，並標出三個最大的技術風險與因應方案。

參考文獻與延伸閱讀

- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer (第一部分：機器人系統與元件) 。
- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson.
- Corke, P., Robotics, Vision and Control, 3rd ed., Springer.
- Niku, S. B., Introduction to Robotics: Analysis, Control, Applications, Wiley (機器人子系統與致動器章節) 。
- Horowitz, P. and Hill, W., The Art of Electronics, 3rd ed., Cambridge University Press (電源、訊號調理與雜訊處理) 。
- Kopetz, H., Real-Time Systems: Design Principles for Distributed Embedded Applications, Springer (即時系統理論) 。
- ROS 2 官方文件 (docs.ros.org)：節點、話題與系統架構設計指南。
- CAN in Automation (CiA) 與 EtherCAT Technology Group 官方技術文件。
- IEC 60204-1 與 ISO 13850：機械電氣安全與緊急停止裝置相關標準。
- 各開發板官方文件：Raspberry Pi、NVIDIA Jetson、STM32、ESP32 資料手冊與應用筆記。

第 4 章

機構、連桿與自由度

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

【章首情境圖建議】三聯圖：左為一隻手掌平貼白板、手肘卻高高抬起的手臂 (冗餘自由度) ；中為一台六軸工業手臂的第三節卡在治具上 (自由度不足的困境) ；右為一台 *Delta* 機器人三條支鏈張開如飛翔的三角形。三張圖共同提問：幾個關節，才叫剛剛好？

叫獸開講 「為什麼人手看起來很靈活，機器手臂卻常常卡住？」

那天下午，叫獸在課堂上做了一個小實驗。他請一位學生上台，把右手手掌平貼在白板上，五指張開、手掌不能離開白板，然後說：「請你把手肘抬高。」

學生照做了——手掌沒動，手肘卻緩緩往上抬起，整條手臂像一扇被推開的門那樣繞著某條看不見的軸旋轉。台下一片驚呼：手的位置完全沒變，手臂的形狀卻變了。

「這個動作，」叫獸說，「是機器人學裡最重要的現象之一，叫做冗餘。你的手臂從肩膀到手掌，總共有七個自由度，但『把手掌固定在空間中的某個位置與姿態』只需要六個。多出來的那一個，就讓你可以在保持手掌不動的情況下，讓手肘上上下下。這一度自由，讓你可以在鍵盤前繞開桌角、能伸手到冰箱深處拿飲料而不撞到隔板。」

接著他打開投影片，播放一段工業現場的影片：一台六軸機械手臂正要把一支零件插進狹縫，但手臂的第三節撞上了旁邊的治具。操作員在示教器上調了半天，最後只能把整個治具挪開重來。

「這台手臂只有六個自由度，」叫獸說，「它能讓末端到達你要的位置與姿態，但『用什麼姿勢到達』是被綁死的。它沒有多餘的那一度自由，所以撞到就是撞到，沒有第二種走法。」

有學生問：「那為什麼工業手臂不做七軸就好了？」叫獸笑了：「好問題。多一軸就多一顆馬達、多一組減速機、多一段連桿——成本上升、剛性下降、控制變複雜、而且逆運動學會有無限多組解，你得再花力氣去挑一組。工程從來不是『越多越好』，而是『剛剛好』。」

他在白板上畫了三個東西：一台會爬樓梯的四足機器狗、一台在無塵室高速取放晶片的 SCARA、一台飛行模擬器底下那個六支油壓缸撐起的平台。「這三台機器，自由度從十二、到四、到六，形態完全不同——但它們共用同一套數學。今天這堂課，我們要學會兩件事：第一，怎麼數出一台機器有幾個自由度；第二，這個數字為什麼決定了它能做什麼、不能做什麼。」

下課前他留了一個問題：「回家後把手掌貼在桌上，重複剛才那個動作。然後想想看——如果你的手臂只有六個自由度，你的生活會有哪裡不方便？」

本章為什麼重要

第3章建立了系統觀，從本章開始我們逐一拆解子系統，第一站是機構——機器人的骨骼。機構決定了一台機器人「能做什麼」的物理上限：工作範圍多大、能不能繞過障礙、能不能舉起重物、精度能到多細。這些都在畫第一張設計圖時就被決定了，後面再強的控制演算法與 AI 都

無法突破。

本章的核心概念是自由度 (DoF)。它是機器人學中最基礎、也最常被誤用的名詞——很多人以為「自由度等於馬達數」，但這只在特定條件下成立。學會正確計算自由度，你才能判斷一個機構設計是否可行、是否過度約束、是否有冗餘。這個概念會在第 9 章 (正向運動學)、第 10 章 (逆向運動學的多解與無解)、第 12 章 (奇異點與冗餘) 中反覆出現，是整個第二篇的地基。

學習目標

- 能定義機構、連桿、關節與運動鏈，並辨識實際機器人中的對應部位。
- 能列舉常見關節類型及其自由度，並說明旋轉關節與移動關節的差異。
- 能區分開放鏈與閉合鏈，並說明兩者在剛性、工作空間與分析難度上的差異。
- 能說明自由度的物理意義，並解釋為何自由度不必然等於馬達數。
- 能運用 Grübler 公式計算平面與空間機構的自由度。
- 能比較串聯式與並聯式機器人的優缺點與適用場合。
- 能說明 SCARA、Delta 與 Stewart 平台的構型特徵與典型應用。
- 能定義冗餘自由度，並舉例說明其優點與代價。
- 能區分工作空間、可達空間與靈巧空間，並描述常見構型的工作空間形狀。
- 能依任務需求，為一個實際應用選定合適的機器人構型並說明理由。

先備知識

第 3 章的系統架構觀念。本章需要基本的空間想像能力與簡單的加減乘法，不需要矩陣或微積分。若能先閱讀附錄 D.1 對三維座標的說明會更順暢，但非必要——本章刻意把所有概念用圖與實物說明，數學留待第 8、9 章展開。

核心名詞中英對照

表 4-1 本章核心名詞

中文	英文	說明
機構	Mechanism	由連桿與關節組成、能傳遞運動與力的系統

中文	英文	說明
連桿	Link	機構中的剛體構件
關節	Joint	連接兩連桿並限制其相對運動的元件
運動鏈	Kinematic Chain	連桿與關節依序連接形成的鏈狀結構
自由度	Degree of Freedom, DoF	描述系統位形所需的獨立參數數目
旋轉關節	Revolute Joint	允許繞單一軸旋轉的關節，1 自由度
移動關節	Prismatic Joint	允許沿單一軸平移的關節，1 自由度
球關節	Spherical Joint	允許三軸旋轉的關節，3 自由度
開放鏈	Open Chain / Serial Chain	一端固定、一端自由的鏈狀結構
閉合鏈	Closed Chain	形成迴路的機構結構
串聯式機器人	Serial Robot	關節依序串接的機器人，如六軸手臂
並聯式機器人	Parallel Robot	多條支鏈共同支撐動平台的機器人
冗餘	Redundancy	自由度多於任務所需的情形
工作空間	Workspace	末端執行器可到達的空間範圍
靈巧工作空間	Dexterous Workspace	末端可以任意姿態到達的空間範圍
過度約束	Overconstrained	約束數超過理論所需，卻仍能運動的機構

4.1 機構與機器的基本概念

4.1.1 從機器到機構

在機械工程的傳統定義中，機器 (machine) 是能傳遞或轉換能量、完成有用功的裝置；機構 (mechanism) 則是機器中負責傳遞運動與力的部分，關注的是「怎麼動」而非「用多少能量」。一台機械手臂的馬達與減速機屬於驅動系統，而連桿與關節組成的那副骨架，就是機構。

機構學研究三件事：位置分析 (給定關節位置，末端在哪裡)、速度分析 (關節轉多快，末端跑多快)、與力分析 (末端受力多大，關節要出多少力矩)。這三件事在機器人學中分別對應第 9 章的正向運動學、第 11 章的雅可比矩陣，以及第 13 章的靜力學——所以本章其實是整個第二篇與第三篇的入口。

4.1.2 剛體假設：一個有用的謊言

機構學的一切分析都建立在「剛體假設」上：假設每根連桿都是完全不變形的剛體。這當然

不是真的——鋁桿會彎、齒輪會有背隙、軸承會有間隙。但這個假設極其有用，因為它讓運動分析變成純粹的幾何問題，可以用矩陣精確描述。

什麼時候這個謊言會失效？當手臂很長很細、當運動速度很高、當負載很重時，彈性變形就不能忽略——這就是為什麼工業手臂的連桿又粗又重（提高剛性），也是為什麼高速運動時末端會殘留振動（第 15 章的軌跡平滑度、第 27 章的結構剛性都與此有關）。初學階段先接受這個假設，但要記得它的邊界。

◎ 三個名詞的區分

機構（mechanism）：連桿與關節的組合，管運動的傳遞。機器（machine）：機構加上動力源與控制，能做有用功。機器人（robot）：機器加上感知與自主決策（第 1 章的三判準）。一台自行車是機構加機器，但不是機器人；一台六軸手臂三者皆是。

4.2 連桿、關節與運動鏈

4.2.1 連桿：機構的骨頭

連桿（link）是機構中的剛體構件。在機器人手臂上，它就是關節與關節之間的那一段結構。連桿的編號有一個慣例：固定不動的基座編為連桿 0，往末端依序編為連桿 1、2、3.....這個編號方式在第 9 章的 DH 參數法中會嚴格使用，現在先建立習慣。

連桿的設計要同時滿足三個要求：足夠的剛性（受力時變形要小）、盡可能輕（減少慣性與能耗）、以及提供精確的關節安裝面（安裝偏差會沿著鏈條累積放大，如第 3 章所述）。工程上常用的解法是中空結構——把材料集中在外圍，因為抗彎剛性主要來自截面的邊緣材料，中心的材料貢獻很小卻很重。這就是為什麼機器人連桿、腳踏車車架與飛機大樑都是中空的。

4.2.2 關節：決定運動的自由

關節（joint）連接兩根連桿，並限制它們之間的相對運動。關節的種類決定了「留下」多少自由度。兩個在空間中完全自由的剛體，彼此有六個相對自由度（三個平移、三個旋轉）；加上一個關節之後，其中一部分被約束掉，剩下的就是該關節的自由度。

最常見的兩種關節，也是機器人手臂上幾乎唯一使用的兩種：旋轉關節（revolute joint，符號 R）允許繞單一軸相對旋轉，1 個自由度，實作上就是一組軸承加馬達；移動關節（prismatic joint，符號 P）允許沿單一軸相對平移，1 個自由度，實作上是線性滑軌加螺桿或氣

壓缸。

表 4-2 常見關節類型與自由度

關節類型	英文與符號	自由度	允許的運動	機器人上的實例
旋轉關節	Revolute (R)	1	繞一軸旋轉	六軸手臂的每一軸、伺服馬達關節
移動關節	Prismatic (P)	1	沿一軸平移	SCARA 的 Z 軸、3D 印表機軸、氣壓夾爪
螺旋關節	Helical / Screw (H)	1	旋轉與平移耦合	滾珠螺桿傳動
圓柱關節	Cylindrical (C)	2	繞軸旋轉 + 沿軸平移	旋轉伸縮式取放機構
萬向關節	Universal (U)	2	兩軸旋轉	傳動軸、Delta 機器人支鏈
球關節	Spherical (S)	3	三軸旋轉	Stewart 平台支腳、人形機器人髖關節
平面關節	Planar (E)	3	平面內兩平移 + 一旋轉	平面滑動接觸

4.2.3 運動鏈與運動樹

連桿與關節依序連接就形成運動鏈 (kinematic chain)。若每根連桿最多連接兩個關節、且不形成迴路，就是簡單的鏈；若某根連桿連接三個以上的關節，運動鏈就會分岔，形成運動樹 (kinematic tree) ——人形機器人就是典型的樹狀結構，軀幹連桿同時分岔出兩條手臂、兩條腿與一個頭。

樹狀結構的分析並不比鏈狀複雜多少：每一條分支都可以獨立套用鏈狀的公式，只要記得它們共享同一個基座即可。第 9 章的正向運動學會用「從基座往末端連乘變換矩陣」的方式處理，樹狀結構只是有多條路徑要各算一次。

4.3 旋轉關節與移動關節

4.3.1 兩者的工程特性比較

雖然旋轉關節與移動關節在數學上都只是「1 個自由度」，但工程特性差異極大，選用時必須權衡。

旋轉關節的優點是：行程無限（可以連續轉動）、結構緊湊、密封容易、成本低（軸承便宜）。缺點是：末端位置與關節角度是非線性關係（三角函數，第 9 章）、力矩會隨姿態變化、且離軸越遠的位置誤差被放大越多。

移動關節的優點是：運動與位置是線性關係（好算、好控）、剛性高、在整個行程上出力一致。缺點是：行程有限（滑軌多長就只能走多遠）、佔用空間大（伸出去時機構會變長）、防塵防水困難（滑軌暴露在外）、成本較高。

表 4-3 旋轉關節與移動關節的比較

面向	旋轉關節 R	移動關節 P
行程	可無限旋轉	受滑軌長度限制
位置關係	非線性（三角函數）	線性
工作空間形狀	球狀或環狀	方正、直角座標式
空間佔用	緊湊	伸展時佔空間大
剛性	受懸臂長度影響	通常較高
防護	容易密封	滑軌暴露、需防塵罩
成本	較低	較高
典型應用	多關節手臂、人形、四足	龍門機構、3D 印表機、SCARA Z 軸

4.3.2 為什麼工業手臂幾乎都用旋轉關節

觀察市面上的六軸工業手臂，會發現六個軸全是旋轉關節。原因有三：第一，工作空間效率——用同樣的機構體積，旋轉關節能掃出大得多的工作範圍；一台佔地半平方公尺的六軸手臂，工作半徑可達一點五公尺，這是移動關節做不到的。第二，成本與可靠度——軸承與減速機的技術成熟、壽命長、可完全密封，適合工廠的粉塵油污環境。第三，仿生——人類的手臂全部由旋轉關節組成，而工廠的作業流程原本就是為人類設計的。

反過來說，需要大範圍平面運動且要求高精度時（如 PCB 檢測、雷射切割、3D 列印），龍門式的直角座標機構（全用移動關節）反而更合適：定位精度高、程式直觀（XYZ 座標直接對應）、剛性好。這說明構型選擇沒有絕對優劣，只有適不適合。

4.4 開放鏈與閉合鏈

4.4.1 開放鏈：一端固定、一端自由

開放鏈 (open chain, 又稱串聯鏈) 是最常見的機器人結構：基座固定，連桿一節接一節往外延伸，最末端是自由的末端執行器。六軸工業手臂、人的手臂、機器狗的每一條腿 (在騰空時)，都是開放鏈。

開放鏈的優點是工作空間大、結構簡單、分析容易——正向運動學只是把變換矩陣依序相乘 (第 9 章)，不需解聯立方程。缺點是剛性差 (誤差與變形逐節累積，就像一根長長的懸臂)、每個關節都要承受下游所有連桿與負載的重量 (所以靠近基座的馬達必須很大顆)、且高速運動時慣性大。

4.4.2 閉合鏈：形成迴路的結構

閉合鏈 (closed chain) 中的連桿形成了迴路：從某根連桿出發，沿著關節走一圈可以回到原點。四連桿機構、Delta 機器人的三條支鏈、Stewart 平台的六支腿，以及機器狗站立時 (雙腳同時著地、透過地面形成迴路)，都是閉合鏈。

閉合鏈的優點正是開放鏈的缺點：剛性高 (多條支鏈共同承力，像三腳架比獨腳架穩)、精度好 (誤差不累積而是被平均)、馬達可以固定在基座 (大幅降低運動部分的慣量，因此能高速)。缺點則是工作空間小、關節活動範圍受限、且運動學分析複雜——閉合鏈的正向運動學往往需要解非線性聯立方程，反而是逆運動學比較容易 (與開放鏈恰好相反，這是一個有趣的對偶現象)。

圖 4-1 開放鏈與閉合鏈的對照

建議配圖：左側為六軸串聯手臂 (開放鏈)，以虛線標出從基座到末端的單一路徑，旁註「誤差累積、工作空間大、剛性低」；右側為 Delta 機器人 (閉合鏈)，三條支鏈由基座延伸並共同連接到動平台，形成迴路，旁註「誤差平均、剛性高、馬達在基座、工作空間小」。

4.4.3 混合結構

實務上許多機器人是混合的。例如常見的六軸手臂，其第二、三軸之間常設計成平行四連桿機構 (closed loop) 來傳遞第三軸的動力，使第三軸的馬達可以往後移到基座附近，降低手臂前端的重量——這是在串聯手臂中局部嵌入閉合鏈的經典設計。人形機器人的腿部也常採此策略，把膝關節的驅動器移到大腿根部。

判斷一個機構是開放還是閉合，最簡單的方法是問：從任一根連桿出發，沿著關節能不能走回原點？能，就有閉合迴路。這個判斷會直接影響下一節自由度公式的用法。

4.5 自由度的物理意義

4.5.1 定義：描述位形所需的獨立參數數目

自由度 (Degree of Freedom, DoF) 是指完整描述一個系統的位形 (configuration , 即所有零件的位置與姿態) 所需要的獨立參數個數。這個定義聽起來抽象，但用具體例子立刻清楚。

一個點在平面上有 2 個自由度 (x, y) ; 在空間中有 3 個 (x, y, z) 。一個剛體在平面上有 3 個自由度 (x, y 加上一個旋轉角 θ) ; 在空間中有 6 個 (三個平移 x, y, z 加上三個旋轉，即滾轉、俯仰、偏航) 。這個「空間剛體六自由度」是機器人學最重要的數字之一——它解釋了為什麼工業手臂通常是六軸：六個自由度恰好足以把末端執行器擺到空間中任意位置與任意姿態。

平面剛體：DoF = 3	空間剛體：DoF = 6
--------------	--------------

這兩個數字是所有自由度計算的基礎，請務必記住。

4.5.2 自由度不等於馬達數

初學者最常見的誤解是「自由度等於馬達數量」。這在多數串聯手臂上恰巧成立，但並非通則，有三種例外。

第一種：驅動不足 (underactuated) 。馬達數少於自由度。四足機器狗在騰空瞬間，身體的六個自由度中沒有任何一個被直接驅動——它只能靠腿部的甩動與落地的接觸力間接控制姿態。人類走路也是如此：從地面到重心的這條「虛擬鏈」沒有馬達，所以走路本質上是一連串「受控的跌倒」(第 28 章) 。

第二種：冗餘驅動 (overactuated) 。馬達數多於自由度。有些並聯機器人與多指機械手會刻意讓馬達數超過自由度，藉此消除背隙、提高剛性、或分散負載——代價是各馬達之間必須協調，否則會互相「打架」而燒毀。

第三種：被動關節。有些關節完全沒有馬達，只是提供運動自由，例如 Delta 機器人支鏈上的球關節、或是機器人腳踝的被動彈簧。它們貢獻自由度，但不貢獻驅動。

◎ 三個容易混淆的自由度

機構自由度：機構整體能獨立運動的參數數目 (本節主題) 。關節自由度：單一關節允許的相對運動數 (表 4-2) 。任務自由度：完成任務所需的自由度數，例如「畫平面圖形」只需 3，「任意姿態抓取」需要 6。三者的比較決定了機器人是否足夠、是否冗餘——這正是 4.10 節的主題。

4.6 平面與空間機構的自由度：Grübler 公式

4.6.1 公式的直覺推導

計算自由度的方法很直觀：先假設所有連桿都自由漂浮，數出總自由度；再把關節加上去，每個關節扣掉它所約束的自由度；剩下的就是機構的自由度。

設一個機構有 N 根連桿（含固定基座）、 J 個關節，每個關節 i 允許 f_i 個自由度。在空間中，每根剛體有 6 個自由度，但基座是固定的，所以自由漂浮的連桿數是 $N - 1$ ，總自由度為 $6(N - 1)$ 。每個關節在空間中會約束掉 $(6 - f_i)$ 個自由度。因此：

$$\text{空間機構：DoF} = 6(N - 1) - \sum_i (6 - f_i) = 6(N - 1 - J) + \sum_i f_i$$

此即 Grübler-Kutzbach 公式的空間形式。 N 為連桿數（含基座）， J 為關節數， f_i 為第 i 個關節的自由度。

平面機構的推導完全相同，只是把 6 換成 3：

$$\text{平面機構：DoF} = 3(N - 1 - J) + \sum_i f_i$$

平面情形下，每根剛體 3 個自由度，每個關節約束 $(3 - f_i)$ 個。

4.6.2 串聯機構的特例

對於開放鏈（串聯機構），公式會退化成一個極簡的結果。開放鏈中，連桿數與關節數的關係恆為 $N = J + 1$ （每加一個關節就多一根連桿）。代入公式， $6(N - 1 - J)$ 這一項變成 $6(J + 1 - 1 - J) = 0$ ，因此：

$$\text{開放鏈：DoF} = \sum_i f_i \quad (\text{所有關節自由度的總和})$$

這就是為什麼六軸手臂（六個 1 自由度的旋轉關節）的自由度恰好是 6。

這解釋了初學者的印象為何常常正確——因為他們接觸的多半是串聯手臂。但一旦遇到閉合鏈，就必須回到完整的 Grübler 公式。

4.6.3 使用公式的三個注意事項

第一，基座要算進 N 。忘記把固定基座算進連桿數，是最常見的計算錯誤，會讓答案剛好差 6（或平面的 3）。

第二，公式算的是「理論自由度」，不保證機構真的能動到那麼多。實際機構可能因為關節行程限制、或干涉碰撞，而無法走完所有位形。

第三，也是最重要的——Grübler 公式對某些特殊機構會算錯。這類機構稱為過度約束機構

(overconstrained mechanism)，公式算出來的自由度是 0 甚至負數，但它們實際上能動。原因是公式假設所有約束彼此獨立，而這些機構因為特殊的幾何排列（例如所有軸互相平行或共點），使得部分約束重複了。經典例子是 Bennett 連桿與 Sarrus 機構。這提醒我們：公式是工具不是真理，最後仍要靠幾何直覺與實體驗證。

◎ 計算自由度的四步驟

一、數連桿數 N （記得含基座）。二、數關節數 J ，並列出每個關節的自由度 f_i 。三、判斷是平面（用 3）還是空間（用 6）機構。四、代入公式計算，並用物理直覺檢驗答案是否合理（例如：一台明顯能自由移動的機器人算出 0 自由度，一定是哪裡數錯了或遇到過度約束）。

4.7 串聯式與並聯式機器人

4.7.1 串聯式：大範圍、高彈性

串聯式機器人就是開放鏈的機器人，關節一個接一個串起來。它是工業上最主流的形態，六軸手臂、SCARA、協作機器人皆屬此類。

優點：工作空間相對於機構體積而言非常大；運動學直觀（正向運動學只需連乘）；每個關節可獨立設計與更換；容易加長臂展。缺點：剛性隨臂展下降；靠近基座的關節要承受整條手臂的重量與慣性，馬達與減速機必須很大；誤差逐節累積，末端精度受限於最差的那一節；高速時因慣性大而振動明顯。

4.7.2 並聯式：高剛性、高速度

並聯式機器人以多條支鏈同時連接基座與動平台，形成閉合鏈。經典代表是 Delta（三支鏈，取放專用）與 Stewart 平台（六支鏈，飛行模擬器與精密定位）。

優點：剛性高（力量由多條支鏈分擔）；精度好（誤差被平均而非累積）；馬達可全部固定在基座上，運動部分極輕，因此加速度可達數十個 g ，是高速取放的王者；承載自重比極佳。缺點：工作空間小（多條支鏈互相牽制）；姿態範圍受限；容易在工作空間內遇到奇異點；正向運動學需解非線性方程，計算較複雜。

表 4-4 串聯式與並聯式機器人的比較

面向	串聯式（開放鏈）	並聯式（閉合鏈）
工作空間	大	小

面向	串聯式（開放鏈）	並聯式（閉合鏈）
剛性	較低（懸臂效應）	高
精度	誤差累積	誤差平均
速度 / 加速度	中等（運動慣量大）	極高（馬達在基座）
承載自重比	較低	高
正向運動學	簡單（連乘）	複雜（解非線性方程）
逆向運動學	較複雜（多解）	相對簡單
典型代表	六軸手臂、SCARA、協作機器人	Delta、Stewart 平台
典型應用	焊接、組裝、搬運、上下料	高速取放、精密定位、模擬平台

◎ 一個記憶法

串聯像人的手臂——伸得遠、動作靈活，但舉重物時會抖。並聯像三腳架或起重機的支撐——範圍小，但穩如泰山。當任務需要「伸進去」時用串聯；需要「又快又穩」時用並聯。

4.8 SCARA、Delta 與 Stewart 平台

4.8.1 SCARA：電子業的速度之王

SCARA 全名為 Selective Compliance Assembly Robot Arm（選擇性順應裝配機器手臂），由日本學者牧野洋於 1978 年提出。它的構型是 RRPR 或 RRP：兩個平行的垂直軸旋轉關節提供水平面的運動，再加上一個垂直的移動關節與一個末端旋轉軸，總共四個自由度。

名稱中的「選擇性順應」是理解它的關鍵：這個機構在水平面方向較為順應（有一點柔性），但在垂直方向極為剛硬。這正是垂直插裝作業所需要的特性——插銷入孔時，水平方向的微小柔性可以自動修正對位誤差，而垂直方向的高剛性則保證能壓得下去。這是機構設計「順應性剛好用在對的方向」的經典案例，與第 20 章的柔順控制遙相呼應。

SCARA 的四個自由度恰好對應平面裝配任務所需：平面位置（ x, y ）、高度（ z ）與繞垂直軸的旋轉（ θ ）。它不能傾斜末端——但電子組裝幾乎所有零件都是垂直插入的，因此不需要。少兩個自由度換來的是更高的速度、更低的成本與更好的剛性，這是「剛剛好就好」的最佳範例。

4.8.2 Delta：飛翔的三角形

Delta 機器人由瑞士學者 Reymond Clavel 於 1980 年代發明，原始構想來自巧克力包裝生

產線的高速取放需求。它的結構是：三個安裝在基座上的旋轉馬達，各自驅動一根主動臂；每根主動臂再透過平行四邊形連桿（由球關節或萬向關節連接）連到中央的動平台。

這個平行四邊形設計有一個關鍵作用：它保證動平台永遠保持與基座平行，不會傾斜。因此 Delta 提供三個平移自由度（ x, y, z ），姿態被固定。若需要旋轉，通常在中央加一根伸縮傳動軸提供第四軸。

Delta 的性能極為驚人：因為三顆馬達全部固定在基座、運動部分只有輕巧的碳纖維連桿，加速度可達 10 g 以上，每分鐘可完成兩百次以上的取放。今天你在超商買到的每一盒包裝食品、每一片包裝好的藥錠，很可能都經過 Delta 機器人之手。

4.8.3 Stewart 平台：六支腳撐起的世界

Stewart 平台（又稱 Gough-Stewart 平台或 Hexapod）由六根可伸縮的支腿連接基座與動平台，每根支腿兩端各有一個球關節或萬向關節。六根腿的長度變化組合，恰好提供動平台完整的六個自由度——三平移加三旋轉。

它的特點是剛性極高、承載能力極強、且定位精度可達微米級。典型應用包括：飛行模擬器（撐起整個駕駛艙並模擬六自由度的運動）、精密機械加工的定位平台、望遠鏡的鏡面調整、以及部分手術機器人。缺點是工作空間非常小（尤其是旋轉範圍），以及正向運動學必須用數值方法求解。

圖 4-2 三種經典構型的對照

建議配圖：三格並排。左為 SCARA（俯視加側視，標出兩個垂直旋轉軸、Z 軸移動與末端旋轉，共 4 DoF）；中為 Delta（三支鏈由基座向下延伸至動平台，標註平行四邊形連桿與 3 DoF 平移）；右為 Stewart 平台（六支可伸縮腿呈交錯排列，標註 6 DoF）。下方各標註自由度、速度、剛性與典型應用。

表 4-5 三種經典構型規格對照

構型	自由度	關節配置	強項	限制	代表應用
SCARA	4	RRPR	水平高速、垂直剛硬、成本低	末端無法傾斜	電子組裝、鎖螺絲、點膠
Delta	3（可加至 4）	3 條 R + 平行四邊形支鏈	加速度極高、輕量	工作空間小、姿態固定	食品藥品高速取放、分揀
Stewart 平台	6	6 條 UPS 支腿	剛性與精度極高、承載大	工作空間與旋轉範圍極小	飛行模擬器、精密定位、手術

4.9 冗餘自由度

4.9.1 定義與人類手臂的例子

當機器人的自由度多於完成任務所需的自由度時，就稱為冗餘（redundancy）。開頭故事中人類手臂的七個自由度，對「把手掌放到指定位置與姿態」這個六自由度任務而言，就多出一個——這一個多餘的自由度，讓手肘可以在手掌固定的情況下擺動。

在數學上，冗餘意味著逆運動學有無限多組解：達成同一個末端位姿，關節角度的組合形成一個連續的集合（稱為自運動流形，self-motion manifold）。第 12 章會說明如何在這個無限多的解中，用零空間運動去選出最好的一組。

4.9.2 冗餘的四大好處

第一，避障：在末端保持軌跡不變的前提下，改變手肘或關節的姿勢以繞開障礙物——這正是開頭故事中六軸手臂做不到、而七軸手臂做得到的事。

第二，避開奇異點：奇異點是機器人失去某個方向運動能力的位形（第 12 章）。有冗餘時，可以用多出來的自由度繞開這些位形。

第三，避開關節極限：當某個關節接近行程末端時，可以重新分配各關節的角度，讓它退回安全範圍。

第四，最佳化次要目標：在完成主要任務的同時，還可以順便最小化能耗、最大化操作靈巧度、或維持一個對人類而言看起來自然的姿勢——這對協作機器人與人形機器人特別重要。

4.9.3 冗餘的代價

冗餘不是免費的。每多一個自由度，就多一顆馬達、一組減速機、一段連桿與一組線路：成本上升、重量增加、剛性下降、故障點變多。控制上也更複雜——逆運動學不再有唯一解，必須額外定義「要選哪一組」的準則（第 12 章的分層逆運動學）。

這解釋了產業現況：多數工業手臂是六軸（剛好夠用、成本最低），七軸手臂則多見於需要在狹窄空間作業或與人協作的場合。台灣廠商的協作機器人常見六軸與七軸並存的產品線，正是針對這兩種不同的需求。

4.10 工作空間與可達空間

4.10.1 三種空間的定義

工作空間 (workspace) 是末端執行器所能到達的所有位置的集合，它是機器人選型時最直接的規格。但工程上必須進一步區分三個層次。

可達工作空間 (reachable workspace)：末端至少能以某一種姿態到達的所有點。這是最寬鬆的定義，也是型錄上「工作半徑」通常指的範圍。

靈巧工作空間 (dexterous workspace)：末端能以任意姿態到達的所有點。這個範圍恆小於可達工作空間，有時甚至小得多。對需要從各種角度接近工件的任務（如焊接、噴塗），靈巧工作空間才是真正有意義的規格。

有效工作空間 (usable workspace)：扣除掉自我碰撞、關節極限、線纜干涉、以及靠近奇異點而不可用的區域後，實際能穩定作業的範圍。這通常又比靈巧工作空間再小一圈，卻是實務規劃時真正該用的數字。

圖 4-3 三種工作空間的層次關係

建議配圖：以六軸手臂為例的剖面圖，畫出三層同心的區域：最外層淺色為可達工作空間（含手臂完全伸直的邊界球面）、中層為靈巧工作空間、最內層深色為有效工作空間；並標出三個被扣除的區域：基座附近的自我碰撞區、完全伸直的邊界奇異區、以及關節極限造成的死角。

4.10.2 常見構型的工作空間形狀

直角座標式 (PPP，三個移動關節)：長方體。優點是形狀直觀、程式好寫、精度均勻；缺點是機構體積遠大於工作空間。3D 印表機與龍門式取放機屬此類。

圓柱座標式 (RPP)：中空圓柱體。一個旋轉軸加兩個移動軸，適合繞著中心排列的取放作業。

球座標式 (RRP)：中空球殼的一部分。早期的 Unimate 即為此類。

關節式 (RRR，六軸手臂)：近似球體但形狀不規則，中間有基座造成的空洞、外緣有伸直極限。這是工作空間對機構體積比最高的構型，也是主流。

SCARA：圓柱狀但較扁平，因為 Z 軸行程通常遠小於水平臂展。

Delta：一個上寬下窄的碗狀或圓錐狀區域。

4.10.3 邊界的兩個陷阱

第一個陷阱是工作空間邊界處的性能急遽下降。當手臂完全伸直時，它剛好到達可達空間的邊界，但在這個位形上，末端沿著徑向已經無法再前進——這正是第 12 章的邊界奇異點。此時即使關節仍能轉動，末端在某個方向就是動不了，且逆運動學的解會變得極不穩定。因此實務上，規劃作業位置時務必與邊界保持餘裕，通常建議保留百分之十以上。

第二個陷阱是工作空間的空洞。多數六軸手臂在基座正上方與正下方附近有無法到達的區域（因為機構會自我碰撞，或需要無限大的關節速度通過）。學生在配置工作站時，若把待取工件放在基座正前方太近的位置，常常會發現「型錄上明明說工作半徑一點五公尺，為什麼零點三公尺處反而拿不到」——答案就在這裡。

4.11 機器人結構選型

4.11.1 選型的六個問題

為一個任務挑選機器人構型，可以依序回答六個問題。第一，任務需要幾個自由度？平面搬運只需 3、垂直插裝需 4（SCARA 即可）、任意姿態作業需 6、需要避開障礙或與人協作則考慮 7。

第二，工作空間的形狀與大小為何？是長條狀的產線（考慮龍門或加裝第七軸行走軸）、還是圍繞一點的環狀配置（六軸手臂）、或是扁平的平面（SCARA）？

第三，速度與節拍要求多高？每分鐘超過六十次的取放，幾乎只有 Delta 與高速 SCARA 能勝任。

第四，負載多重？含末端夾具的總重，並記得預留百分之二十以上的餘裕，且要考慮加速時的動態負載，而非只看靜態重量。

第五，精度要求多少？請區分重複精度（回到同一點的一致性，通常是型錄上較漂亮的那個數字）與絕對精度（到達指定座標的準確度，通常差一個數量級）。多數產線任務只需重複精度，離線編程與視覺導引則需要絕對精度。

第六，環境與安全條件為何？無塵室、油污、高溫、需與人共處——這些決定了防護等級與是否必須採用協作機器人。

表 4-6 依任務選擇構型的速查表

任務需求	建議構型	理由
電子零件垂直插裝、鎖螺絲	SCARA	4 DoF 剛好夠用、水平快、垂直剛硬、成本低
食品藥品高速分揀取放	Delta	加速度極高、運動慣量小、易清潔
焊接、噴塗、任意角度作業	六軸串聯	6 DoF 可任意姿態、工作空間大
狹窄空間作業、與人協作	七軸協作機器人	冗餘可避障、力量受限較安全
大面積平面加工、3D 列印	龍門直角座標	精度均勻、程式直觀、行程可延長
飛行模擬、精密微定位	Stewart 平台	6 DoF、剛性與精度極高
跨區域搬運物料	AMR + 機械手臂	移動平台擴展工作空間 (第 23 章)

4.11.2 一個常見的選型錯誤

學生與初次導入的企業最常犯的錯誤，是只看「負載」與「工作半徑」兩個數字就下單。實際上，機器人型錄上的額定負載通常是在末端法蘭處、以標準姿態、在低速下量測的。當你裝上一支長 300 公釐的夾具、抓取一個重心偏移的工件、並要求它高速運動時，等效負載與力矩可能是額定值的兩三倍，導致手臂震動、精度下降、甚至觸發過載保護。

正確的做法是計算「負載慣量矩」(load inertia moment)：不只算重量，還要算重量乘上重心到法蘭的距離。各大廠商都提供負載能力曲線圖與線上選型工具，務必使用。這個概念會在第 5 章的馬達選型與第 14 章的動力學中再次出現。

◎ 台灣產業案例

中部某工具機廠評估工具機上下料自動化。工件重 8 公斤、夾具重 3 公斤、需在兩台機台之間搬運 (相距 2.5 公尺)、要求節拍 25 秒。初步構想是買一台大型六軸手臂放在中間，但工作半徑需 1.5 公尺以上，價格昂貴且佔地大。最後採用的方案是：中型六軸手臂 (負載 20 公斤、半徑 1.4 公尺) 加裝一支 3 公尺的第七軸行走軸 (移動關節)。這個「串聯手臂 + 線性軸」的組合，用較低的成本換到了長條狀的工作空間——正是 4.11.1 節第二個問題的實際體現。

完整計算例題

例題 4-1 平面四連桿機構的自由度

題目：一個平面四連桿機構 (如汽車雨刷或縫紉機的連桿)，由四根連桿 (含固定基座) 以四個旋轉關節首尾相連成一個封閉迴路。求其自由度。

解：這是平面機構，使用平面 Grübler 公式。連桿數 $N = 4$ (含基座)，關節數 $J = 4$ ，每個

旋轉關節 $f_i = 1$ ，故 $\sum f_i = 4$ 。

$$\text{DoF} = 3(N - 1 - J) + \sum f_i = 3(4 - 1 - 4) + 4 = 3(-1) + 4 = 1$$

平面四連桿機構的自由度為 1。

結果為 1，符合物理直覺：只要驅動其中任一根連桿（例如馬達帶動曲柄），整個機構的運動就完全確定。這正是四連桿機構在工程上如此普及的原因——一顆馬達就能產生複雜且可重複的運動軌跡。若讀者算出 4（誤用了開放鏈的公式），請回頭確認是否忽略了「封閉迴路」這個關鍵條件。

例題 4-2 六軸工業手臂與 Delta 機器人的自由度

題目：(a) 求六軸串聯工業手臂的自由度。(b) 求標準 Delta 機器人的自由度。Delta 的結構為：基座（1 根）+ 三根主動臂（3 根）+ 三組平行四邊形連桿（每組視為 2 根連桿）+ 動平台（1 根）；關節配置為：3 個旋轉關節（主動，連接基座與主動臂）+ 每條支鏈上下各 2 個球關節（共 $3 \times 4 = 12$ 個球關節）。

解 (a)：六軸手臂是開放鏈，可直接用簡化式。 $N = 7$ （基座 + 6 根連桿）， $J = 6$ ，全為旋轉關節。

$$\text{DoF} = 6(N - 1 - J) + \sum f_i = 6(7 - 1 - 6) + 6 = 0 + 6 = 6$$

六軸手臂的自由度為 6，恰好足以達成空間中任意位置與姿態。

解 (b)：Delta 為空間閉合鏈。連桿數 $N = 1$ （基座）+ 3（主動臂）+ 6（三組平行四邊形，每組 2 根）+ 1（動平台）= 11。關節數 $J = 3$ （旋轉）+ 12（球關節）= 15。自由度總和 $\sum f_i = 3 \times 1 + 12 \times 3 = 3 + 36 = 39$ 。

$$\text{DoF} = 6(11 - 1 - 15) + 39 = 6(-5) + 39 = -30 + 39 = 9$$

公式算出 9，但實際 Delta 只有 3 個自由度。

為什麼差了 6？因為每組平行四邊形連桿中的兩根連桿，可以各自繞著自身的軸線自由旋轉——這是「被動的局部自由度」（local mobility），對動平台的運動沒有任何貢獻。三組共有 6 個這樣的無效自由度。扣除後： $9 - 6 = 3$ ，正是 Delta 的三個平移自由度。

工程啟示：Grübler 公式算的是機構中所有的獨立運動參數，包含那些「空轉但不影響輸出」的局部自由度。分析並聯機構時，務必逐條檢查是否存在此類無效自由度——這也是 4.6.3 節所說「公式是工具不是真理」的具體體現。

例題 4-3 工作空間與冗餘判定

題目：某任務需要在一個 60 公分 × 40 公分的平面上，將零件垂直插入孔位，插入時零件必須保持垂直、但可繞垂直軸調整角度。(a) 此任務需要幾個自由度？(b) 若使用六軸手臂，冗餘度為何？(c) 若改用 SCARA，是否足夠？

解(a)：末端需要控制的量為：平面位置 x 、 y (2)、插入深度 z (1)、繞垂直軸的旋轉角 θ (1)。至於傾斜的兩個旋轉自由度，任務要求「保持垂直」，即這兩個量被固定為常數，不需要主動控制。故任務自由度 = 4。

解(b)：六軸手臂有 6 個自由度，任務只需 4 個，冗餘度 = $6 - 4 = 2$ 。這兩個多出來的自由度可用於避開治具干涉、或選擇對手臂更省力的姿勢。

解(c)：SCARA 的四個自由度 (x, y, z, θ) 與任務需求完全一致，冗餘度 = 0。它足夠且剛好，加上垂直方向剛性高、水平方向具選擇性順應（有利於插入時的自動對正）、速度快、成本低——因此 SCARA 是此任務的最佳選擇。這是「剛剛好就好」的典型範例。

程式範例：自由度計算器

以下 Python 程式實作 Grübler 公式，可計算平面或空間機構的自由度，並支援扣除局部自由度。程式最後以例題中的三個機構驗證。

```
JOINT_DOF = {
    "R": 1,    # 旋轉關節 Revolute
    "P": 1,    # 移動關節 Prismatic
    "H": 1,    # 螺旋關節 Helical
    "C": 2,    # 圓柱關節 Cylindrical
    "U": 2,    # 萬向關節 Universal
    "S": 3,    # 球關節 Spherical
    "E": 3,    # 平面關節 Planar
}

def grubler(n_links, joints, space=True, local_dof=0):
    """
    n_links : 連桿總數 (含固定基座)
    joints  : 關節型別清單，如 ["R", "R", "S"]
    space   : True 為空間機構(6)，False 為平面機構(3)
    local_dof : 需扣除的被動局部自由度數
    """
    m = 6 if space else 3
    j = len(joints)
```

```

sum_f = sum(JOINT_DOF[t] for t in joints)
dof = m * (n_links - 1 - j) + sum_f - local_dof
return dof, sum_f

def report(name, n, joints, space, local=0):
    dof, sum_f = grubler(n, joints, space, local)
    kind = "空間" if space else "平面"
    print(f"{name}")
    print(f"  {kind}機構 | 連桿 N={n} | 關節 J={len(joints)} | Σf={sum_f}"
          f" | 局部自由度扣除={local}")
    print(f"  → 自由度 DoF = {dof}\n")

# 例題 4-1：平面四連桿
report("平面四連桿機構", 4, ["R"]*4, space=False)

# 例題 4-2(a)：六軸串聯手臂
report("六軸工業手臂", 7, ["R"]*6, space=True)

# 例題 4-2(b)：Delta 機器人 (含 6 個被動局部自由度)
report("Delta 機器人", 11, ["R"]*3 + ["S"]*12, space=True, local=6)

# 補充：Stewart 平台 (6 條 UPS 支腿)
# 連桿：基座 1 + 每腿 2 段(上下)×6 + 動平台 1 = 14
report("Stewart 平台", 14, (["U", "P", "S"]*6), space=True, local=6)

```

執行結果應為：四連桿 1、六軸手臂 6、Delta 3、Stewart 平台 6。請試著修改：把六軸手臂改成七軸 (N=8、七個 R)，看看自由度變成多少；再想想這多出來的一個自由度，在開頭故事中對應到什麼動作。

■ 模擬實作：用 URDF 描述一個機構

機器人的機構在軟體中如何描述？ROS 2 使用 URDF (Unified Robot Description Format) ——一種以 XML 描述連桿與關節的格式。以下是一個平面兩連桿手臂的最簡 URDF 片段，展示了本章概念如何落實為程式碼：

```

<?xml version="1.0"?>
<robot name="two_link_arm">

  <!-- 連桿 0：固定基座 -->
  <link name="base_link"/>

```

```

<!-- 連桿 1 -->
<link name="link1">
  <visual>
    <geometry><box size="0.3 0.05 0.05"/></geometry>
    <origin xyz="0.15 0 0"/>
  </visual>
</link>

<!-- 關節 1：旋轉關節，連接 base_link 與 link1 -->
<joint name="joint1" type="revolute">
  <parent link="base_link"/>
  <child link="link1"/>
  <axis xyz="0 0 1"/>          <!-- 繞 Z 軸旋轉 -->
  <limit lower="-3.14" upper="3.14"
    effort="10" velocity="2.0"/>  <!-- 關節極限與能力 -->
</joint>

<!-- 連桿 2 -->
<link name="link2">
  <visual>
    <geometry><box size="0.25 0.05 0.05"/></geometry>
    <origin xyz="0.125 0 0"/>
  </visual>
</link>

<!-- 關節 2：旋轉關節，連接 link1 與 link2 -->
<joint name="joint2" type="revolute">
  <parent link="link1"/>
  <child link="link2"/>
  <origin xyz="0.3 0 0"/>      <!-- 沿連桿 1 長度偏移 -->
  <axis xyz="0 0 1"/>
  <limit lower="-2.6" upper="2.6"
    effort="5" velocity="2.0"/>
</joint>

</robot>

```

請注意這份檔案如何精確對應本章的概念：link 標籤是連桿、joint 標籤是關節、type="revolute" 指定關節類型（改成 prismatic 就變成移動關節）、parent 與 child 定義了運動鏈的連接順序、limit 則是關節行程極限——正是 4.10.3 節中限縮有效工作空間的因素之一。

實作步驟：（一）將上述內容存為 two_link.urdf。（二）若已安裝 ROS 2，執行 check_urdf two_link.urdf 驗證語法並印出運動鏈結構。（三）用 urdf_to_graphviz 產生連桿關節結構圖。（四）試著加入第三個連桿與關節，觀察自由度如何變化。第 29 章會用 RViz 把它視覺化並讓它動起來。

國中小也看得懂：數數看有幾個可以轉的地方

你有沒有玩過那種可以彎來彎去的檯燈？它有幾個「可以轉的地方」？每一個可以轉的地方，就是一個「自由度」。轉的地方越多，燈頭就能擺到越多不同的位置。

現在做個小實驗：把你的手掌平放在桌上不要移開，然後試著把手肘抬高。有沒有發現——手掌完全沒動，可是手臂的樣子變了！這是因為你的手臂「可以轉的地方」比需要的還多了一個，所以它有好幾種方法可以擺出同一個姿勢。這就叫做「多出來的自由」，它很有用——你伸手到書桌下面撿橡皮擦時，就是靠這個多出來的自由，讓手肘繞過桌角。

機器人也一樣。有的機器人手臂只有四個可以轉的地方（像工廠裡鎖螺絲的機器手），因為它只需要上下左右轉一轉就夠了；有的有六個、七個，因為它要像人一樣做各種動作。下次你看到機器人，試著數數看：它身上有幾個可以動的關節？

叫獸提醒與常見錯誤

- 誤區一：自由度等於馬達數。這只在「開放鏈且每關節皆有驅動」時成立。閉合鏈、驅動不足系統與被動關節都會打破這個等式。
- 誤區二：計算 Grübler 公式時忘記把固定基座算進連桿數 N 。這會讓答案剛好差 6（空間）或 3（平面）。
- 誤區三：對閉合鏈誤用開放鏈的簡化式（ $\text{DoF} = \Sigma f$ ）。看到迴路就必須回到完整公式。
- 誤區四：忽略被動局部自由度。並聯機構（尤其 Delta、Stewart）常有空轉但不影響輸出的自由度，必須手動扣除。
- 誤區五：把型錄上的工作半徑當成可用範圍。實際的有效工作空間要再扣除自我碰撞區、邊界奇異區與關節極限死角，建議保留 10% 以上餘裕。
- 誤區六：選型只看負載重量。必須計算負載慣量矩（重量 $\times$ 重心到法蘭的距離），並考慮加速時的動態負載。
- 誤區七：以為自由度越多越好。每增加一軸就增加成本、重量與控制複雜度，並降低剛性。工程的目標是「剛剛好」。
- 誤區八：混淆重複精度與絕對精度。型錄上漂亮的數字通常是重複精度；離線編程與視覺導引任務需要的是絕對精度，兩者常差一個數量級。

課堂討論

1. 開頭故事的問題：如果人類的手臂只有六個自由度（少掉冗餘的那一個），日常生活會有哪三件事變得困難？請具體描述動作情境。
2. 為什麼 SCARA 只用四個自由度就能主宰電子組裝市場？如果幫它加上兩個自由度變成六軸，會得到什麼、失去什麼？
3. Delta 機器人的三顆馬達全部固定在基座上，這個設計決策帶來哪些連鎖優勢？請至少列出三項，並說明它們之間的因果關係。
4. 人形機器人的腿部若採用閉合鏈設計（把膝關節驅動器移到大腿根部），會帶來什麼好處與代價？這與第 28 章的動態行走有什麼關聯？
5. 假設你要為醫院設計一台送藥並能開門的機器人。它需要幾個自由度？移動底盤與手臂的自由度該如何分配？請畫出構型草圖並說明理由。

章末摘要

- 機構是機器人的骨骼，決定「能做什麼」的物理上限；分析建立在剛體假設之上，此假設在高速、長臂、重載時會失效。
- 連桿是剛體構件，關節限制相對運動；最常用的是 1 自由度的旋轉關節（R）與移動關節（P），球關節（S）為 3 自由度。
- 開放鏈工作空間大、分析簡單但剛性低、誤差累積；閉合鏈剛性高、精度好、可高速，但工作空間小、正向運動學複雜。
- 空間剛體有 6 個自由度、平面剛體有 3 個，這是所有自由度計算的基礎。
- 自由度不等於馬達數：驅動不足、冗餘驅動與被動關節都會打破此等式。
- Grübler 公式：空間 $\text{DoF} = 6(N-1-J) + \sum f_i$ ，平面 $\text{DoF} = 3(N-1-J) + \sum f_i$ ；開放鏈退化為 $\text{DoF} = \sum f_i$ 。
- 使用公式須注意三點：基座要算進 N、需扣除被動局部自由度、過度約束機構會使公式失準。
- 串聯式範圍大彈性高，並聯式剛性強速度快；SCARA（4 DoF）主攻垂直插裝、Delta（3 DoF）主攻高速取放、Stewart（6 DoF）主攻高剛性精密定位。
- 冗餘（自由度多於任務所需）可用於避障、避奇異點、避關節極限與最佳化次要目標，代價

是成本、重量、剛性與控制複雜度。

- 工作空間分三層：可達、靈巧、有效；規劃時應使用有效工作空間並保留邊界餘裕，注意基座附近的空洞區。
- 選型六問：自由度需求、空間形狀、速度節拍、負載（含慣量矩）、精度（重複 vs 絕對）、環境與安全條件。

觀念題與計算題

觀念題

1. 定義機構、連桿、關節與運動鏈，並在一台六軸手臂上指出各自對應的部位。
2. 剛體假設是什麼？在哪三種情況下這個假設會明顯失效？
3. 比較旋轉關節與移動關節在行程、位置關係、空間佔用、防護與成本上的差異。
4. 為什麼工業手臂幾乎全用旋轉關節，而 3D 印表機全用移動關節？請從工作空間效率與精度需求分析。
5. 如何判斷一個機構是開放鏈還是閉合鏈？請描述判斷方法。
6. 說明「空間剛體有 6 個自由度」的組成，並解釋為何六軸手臂是工業主流。
7. 舉出三種「自由度不等於馬達數」的情形，各給一個機器人上的實例。
8. 寫出空間與平面的 Grübler 公式，並說明開放鏈為何可簡化為 $\text{DoF} = \sum f_i$ 。
9. 什麼是被動局部自由度？為什麼 Delta 機器人的計算必須扣除 6？
10. 比較 SCARA、Delta 與 Stewart 平台的自由度、強項與典型應用。
11. 定義冗餘自由度，列舉其四大好處與三項代價。
12. 區分可達工作空間、靈巧工作空間與有效工作空間，並說明實務規劃該用哪一個。

計算題

1. 一個平面五連桿機構（含固定基座共 5 根連桿），以 5 個旋轉關節連接成一個封閉迴路。求其自由度，並說明需要幾顆馬達才能完全控制它。
2. 某並聯機器人由 1 個基座、3 條支鏈與 1 個動平台組成。每條支鏈含 2 根連桿，關節配置由基座往上依序為：旋轉關節（R）、萬向關節（U）、球關節（S）。（a）求連桿總數 N 與關節總數 J。（b）以 Grübler 公式計算自由度。（c）若已知此機構實際只有 3 個自由度，

請推論存在幾個被動局部自由度。

3. 一台 SCARA 手臂的第一臂長 400 公釐、第二臂長 300 公釐，兩軸的關節角度範圍分別為 ± 140 度與 ± 145 度，Z 軸行程 150 公釐。(a) 求水平面上可達工作空間的最大與最小半徑。(b) 估算此工作空間的環狀面積（不考慮角度限制）。(c) 若要在距離基座 750 公釐處作業，是否可行？請說明。
4. 某六軸手臂額定負載 10 公斤（於法蘭處量測）。現欲加裝一支重 2.5 公斤、長 250 公釐的夾具，抓取一個重 6 公斤、重心位於夾具前端 80 公釐處的工件。(a) 求總重量是否超過額定負載。(b) 求負載重心到法蘭的距離（假設夾具重心在其中點）。(c) 以「等效力矩 = 總重 $\times$ 重心距離」評估，若原廠規定法蘭處最大容許力矩為 30 N·m（取 $g = 9.8 \text{ m/s}^2$ ），此配置是否安全？

動手做與專題延伸

本章實作：機構自由度普查

在校園、住家與實驗室中找出八種含有機構的裝置，完成下表。重點不在數字正確，而在於學會用結構化的方式觀察與分析機構。

表 4-7 機構自由度普查表（前兩列為範例）

裝置名稱	連桿數 N	關節型別與數量	平面 / 空間	計算自由度	實際觀察	差異原因
彎臂檯燈	4	$R \times 3$	平面	3（開放鏈）	3（可上下、前後、旋轉）	一致
汽車雨刷	4	$R \times 4$	平面	1	1（一顆馬達帶動）	閉合迴路，一致
（自行填寫六列：剪刀、折疊傘、辦公椅、自行車變速器、抽屜滑軌、玩具機器手臂等）						

專題延伸

- 用紙板、圖釘與吸管製作一個平面三連桿手臂，實測它的工作空間：在紙上標出末端能到達的所有點，畫出邊界。然後把某一個關節的活動範圍限制在 ± 90 度，重新標繪一次，觀察有效工作空間縮小了多少。

- 拆解一支折疊傘或一台舊印表機的進紙機構，畫出它的機構簡圖（用線段代表連桿、圓圈代表旋轉關節、方塊代表移動關節），並用 Grübler 公式計算自由度，與實際操作結果比對。
- 進階：為附錄 J 的十二軸四足機器狗建立機構分析——（一）單腿在騰空時是開放鏈還是閉合鏈？自由度為何？（二）當四腳同時著地時，透過地面形成了什麼結構？此時整體的自由度如何變化？（三）這個變化對控制策略有什麼影響？（提示：對照第 28 章的支撐多邊形概念）

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 1、8 章：機構與結構設計)。
- Lynch, K. M. and Park, F. C., Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press (第 2 章：位形空間與自由度，含 Grübler 公式的完整討論)。
- Tsai, L.-W., Robot Analysis: The Mechanics of Serial and Parallel Manipulators, Wiley (並聯機構分析的經典著作)。
- Merlet, J.-P., Parallel Robots, 2nd ed., Springer (Stewart 平台與並聯機器人專論)。
- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer (第 1 部分：運動學與機構)。
- Norton, R. L., Design of Machinery, McGraw-Hill (機構學經典教材，四連桿與自由度分析)。
- Clavel, R., "Delta, a Fast Robot with Parallel Geometry," Proc. 18th Int. Symp. on Industrial Robots, 1988 (Delta 機器人原始論文)。
- Makino, H. 等關於 SCARA 構型之早期文獻與日本產業機器人發展史料。
- ROS 2 官方 URDF 教學文件 (docs.ros.org)：連桿、關節與機器人模型描述。
- 各機器人廠商技術型錄與選型工具：工作空間圖、負載能力曲線與慣量矩計算工具。

第 5 章

致動器、馬達與傳動系統

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】一張工作檯上的「致動器家族合照」：由左至右依序為玩具直流馬達、RC 伺服機、步進馬達、無刷外轉子馬達、以及一顆剖開的機器人關節模組（露出內部的馬達、諧波減速機、雙編碼器與驅動板）。最右側放一顆線圈焦黑的燒毀馬達，旁註一行小字：它不是被壓垮的，是被自己發的熱燒死的。

叫獸開講 「同樣是馬達，為什麼玩具車馬達不能直接拿來做機器手臂？」

上學期末，兩位學生抱著一台自製的機械手臂衝進研究室，臉色發青。手臂的第二軸垂了下來，馬達燙得幾乎不能碰，空氣中有一股燒焦的塑膠味。

「老師，這顆馬達的規格明明夠啊！」小翰把型錄推過來，「我們算過了：手臂前端要舉 500 公克，力臂 30 公分，需要的扭力是 $0.5 \times 9.8 \times 0.3$ ，大約 1.5 牛頓米。我們買的伺服馬達標示 25 公斤公分，換算過來是 2.45 牛頓米，還有 60% 的餘裕！」

叫獸沒有馬上回答，而是問：「你們的手臂舉起重物之後，是靜止不動，還是會移動？」

「當然會動啊，它要在一秒內從水平轉到垂直。」

「那就對了。」叫獸拿起白板筆，「你算的是**靜態扭力**——手臂靜止不動、只對抗重力所需的扭力。但你的手臂還要在一秒內轉九十度，這代表它必須先加速、再減速。加速需要額外的扭力，而這個扭力等於轉動慣量乘上角加速度。你們的連桿有多長？」

「30 公分，鋁合金，大概 200 公克。」

叫獸開始算：「連桿加負載的轉動慣量，粗估大約 $0.03 \text{ kg}\cdot\text{m}^2$ 。一秒內轉 90 度，若採梯形速度曲線，角加速度大約 6 rad/s^2 。動態扭力就是 $0.03 \times 6 \approx 0.18$ 牛頓米——嗯，這部分其實不大。」他停頓了一下，「所以問題不在這裡。你們的馬達，是不是常常在快到定位的時候，還微微抖動、發出嗡嗡聲？」

兩人對看一眼，用力點頭。「對！它到定位之後會一直抖，我們以為是程式問題。」

「那才是真正的兇手。」叫獸在白板上寫下一個詞：**堵轉**。「當馬達在定位時抖動，或是手臂被卡住、被使用者用手擋住時，馬達會進入堵轉狀態——轉不動，但電流會飆到額定值的五到十倍。這時馬達幾乎不做功，所有的電能全部變成熱。你們的馬達不是被扭力壓垮的，是被自己發的熱燒死的。」

他把燒毀的馬達拆開，指著焦黑的線圈：「型錄上寫的 25 公斤公分，是**瞬間堵轉扭力**——那是它拚了老命才能撐住的極限，不是它能持續輸出的力量。可以持續使用的**額定扭力**，通常只有標示值的三分之一到二分之一。這是初學者最常掉進去的坑：把極限值當成常用值。」

小翰苦笑：「所以我們該買多大的？」叫獸把型錄翻到後面的曲線圖：「不是買多大，是先

學會看這張圖。今天這堂課，我們就從一顆馬達的內在說起——為什麼它會轉、為什麼轉速快就沒力氣、為什麼需要減速機、以及為什麼有時候馬達太大反而控制不好。」

本章為什麼重要

第 4 章決定了機器人的骨骼，本章決定它的肌肉。致動器是機器人系統中最貴、最重、最容易燒毀，也最常被選錯的元件。一台機器人的成本結構中，馬達與減速機往往佔了三到五成；而學生專題失敗的原因排行榜上，「馬達選錯」也長年名列前茅。

本章要建立三個關鍵能力。第一，看懂馬達型錄——分辨額定值與極限值、讀懂扭力轉速曲線。第二，正確計算需求——不只是靜態負載，還包括加速所需的動態扭力。第三，理解匹配——馬達、減速機與負載三者是一個整體，慣量比不當時，再強的馬達也控制不好。這些能力會直接支撐第 14 章的動力學計算與第 16 章的控制器設計。

學習目標

- 能說明致動器在機器人系統中的角色，並比較電動、液壓與氣壓三類致動器。
- 能說明直流馬達的基本原理，並解釋扭力與轉速的反比關係。
- 能比較有刷直流、無刷直流、步進與伺服馬達的特性與適用場合。
- 能讀懂馬達型錄，區分額定扭力、堵轉扭力、額定轉速與空載轉速。
- 能計算馬達的輸出功率，並由扭力與轉速需求反推所需功率。
- 能說明減速機的作用，並計算減速比對扭力、轉速與慣量的影響。
- 能比較諧波、行星與擺線三種精密減速機的特性。
- 能說明螺桿、皮帶、鏈條與鋼索四種傳動方式的優缺點。
- 能計算轉動慣量與慣量比，並判斷馬達與負載是否匹配。
- 能依任務需求完成一次完整的致動器選型計算，含靜態、動態與安全餘裕。
- 能說明馬達驅動器的功能與保護電路的必要性。

先備知識

第 3 章的驅動器概念與第 4 章的機構自由度。本章需要高中物理的力矩、轉動慣量與功率概

念，以及基本的單位換算能力。若對轉動慣量不熟悉，可先閱讀附錄 D.1 的力學速成；本章所有公式都會附上物理意義的說明與數值範例。

核心名詞中英對照

表 5-1 本章核心名詞

中文	英文	說明
致動器	Actuator	將能量轉換為機械運動的元件
扭力 (轉矩)	Torque	使物體轉動的作用，單位 N·m
額定扭力	Rated Torque	可持續輸出而不過熱的扭力
堵轉扭力	Stall Torque	轉速為零時的最大瞬間扭力
空載轉速	No-load Speed	無負載時的最高轉速
反電動勢	Back EMF	馬達旋轉時產生的反向電壓
扭力常數	Torque Constant, K_t	電流與扭力的比例係數，N·m/A
占空比	Duty Cycle	PWM 訊號中高電位所佔的比例
減速機	Gearbox / Reducer	降低轉速並放大扭力的傳動元件
減速比	Gear Ratio	輸入轉速與輸出轉速之比
背隙	Backlash	齒輪嚙合間隙造成的空轉量
轉動慣量	Moment of Inertia	物體抵抗角加速度的量度， $\text{kg}\cdot\text{m}^2$
慣量比	Inertia Ratio	折算後負載慣量與馬達轉子慣量之比
諧波減速機	Harmonic Drive	利用彈性齒輪變形的高減速比零背隙減速機
伺服系統	Servo System	具位置回授的閉迴路驅動系統
驅動器	Motor Driver	將控制訊號放大為馬達功率的電路

5.1 致動器的功能與分類

5.1.1 能量轉換的終點站

致動器 (actuator) 是機器人系統中把能量轉換成機械運動的元件。它位於整條訊號鏈路的終點：感測器把物理世界轉成訊號、控制器把訊號轉成指令、驅動器把指令轉成功率、最後由致動器把功率轉成運動。第 1 章的「感知—思考—行動」循環中，致動器就是「行動」的執行者。

依能量來源，致動器分為三大類：電動 (電能)、液壓 (液體壓力)、氣壓 (壓縮空氣)。

此外還有新興的智慧材料致動器（形狀記憶合金、介電彈性體、人工肌肉），目前多在研究階段。

5.1.2 三大類的比較

電動致動器是機器人的主流。優點是控制精準（電流與扭力成正比，容易做閉迴路）、乾淨無洩漏、能量來源方便、效率高、易於整合感測器。缺點是功率密度（單位重量能輸出的功率）不如液壓，且在低速大扭力時需要減速機。

液壓致動器的功率密度極高——這是它至今無法被完全取代的原因。挖土機、大型工程機械、以及波士頓動力早期的 BigDog 與 Atlas 都採用液壓，因為它能在很小的體積內輸出巨大的力量，且能承受劇烈衝擊。缺點是需要泵、油箱與管路（笨重且噪音大）、會漏油、維護成本高、控制精度較差。近年來人形機器人的發展趨勢已明顯從液壓轉向高功率密度的電動方案。

氣壓致動器便宜、輕巧、天生柔順（空氣可壓縮，撞到人時會自然退讓）。工廠中大量用於夾爪、推料與簡單的兩點式動作。缺點是因為空氣可壓縮，位置控制困難，通常只能做「全開 / 全關」的開關式動作。這個「天生柔順」的特性，在軟性機器人與人機協作領域反而成為優點。

表 5-2 三類致動器的比較

面向	電動	液壓	氣壓
功率密度	中	極高	低
控制精度	高（可精確位置控制）	中	低（可壓縮）
響應速度	快	快	中
剛性	高（透過減速機）	極高	低（天生柔順）
維護	少	多（漏油、濾芯）	中（需乾燥空氣）
噪音	低	高（泵浦）	高（排氣）
成本	中	高	低
典型應用	工業手臂、協作機器人、AMR	重型機械、大型腿式機器人	夾爪、推料、軟性機器人

5.2 直流馬達的原理與特性

5.2.1 為什麼會轉

直流馬達的原理是安培力：載流導體在磁場中會受力。定子（外殼上的永久磁鐵或激磁線圈）提供固定磁場，轉子上的線圈通電後受力而轉動。關鍵在於，當線圈轉過半圈後，受力方向會反

過來——因此需要換向器 (commutator) 與電刷 (brush) 在適當時機切換電流方向，讓轉子持續朝同一方向轉。這就是「有刷」馬達名稱的由來。

對機器人工程師而言，最重要的是兩個線性關係。第一，扭力與電流成正比：

$$\tau = K_t \cdot I$$

τ 為扭力 ($N \cdot m$)， K_t 為扭力常數 ($N \cdot m / A$)， I 為電流 (A)。這是電流控制即力矩控制的物理基礎。

第二，反電動勢與轉速成正比。馬達轉動時本身也是發電機，會產生反向電壓 (back EMF) 阻礙電流流入：

$$V_{emf} = K_e \cdot \omega$$

V_{emf} 為反電動勢 (V)， K_e 為反電動勢常數， ω 為角速度 (rad/s)。

5.2.2 扭力—轉速曲線：一切的關鍵

把上面兩式與電路方程結合，就能推出直流馬達最重要的特性：扭力與轉速呈線性反比。轉速越高、反電動勢越大、可流入的電流越小、因此扭力越小。這條直線的兩個端點分別是：

堵轉扭力 (stall torque)：轉速為零時的最大扭力。此時沒有反電動勢，電流達到最大 (僅受線圈電阻限制)，扭力最大——但也是發熱最嚴重的狀態。空載轉速 (no-load speed)：扭力為零時的最高轉速。

圖 5-1 直流馬達的扭力—轉速—功率—效率曲線

建議配圖：橫軸為扭力 (由 0 到堵轉扭力)，左縱軸為轉速 (由空載轉速線性下降至 0)、右縱軸為功率與效率。標出四條線：轉速 (直線下降)、電流 (直線上升)、輸出功率 (拋物線，峰值在中點)、效率 (在低扭力區達到最高後下降)。並用網底標出「連續運轉區」(約在堵轉扭力的 1/3 以內) 與「短時間可用區」。

5.2.3 最大功率點與最佳效率點不同

輸出功率是扭力與轉速的乘積： $P = \tau \cdot \omega$ 。由於兩者呈線性反比，功率曲線呈拋物線，峰值恰好落在「堵轉扭力的一半、空載轉速的一半」處。但要注意：最大功率點並不是最佳效率點。在最大功率點，馬達的效率通常只有 50% 左右，另一半的能量全變成熱。最佳效率點通常落在較低的扭力區 (約堵轉扭力的 10~30%)。

這對機器人設計的啟示是：日常運轉應該落在高效率區，只有在偶爾需要爆發力時才短暫進入高扭力區。這正是「額定扭力」與「堵轉扭力」必須分開看的原因——也是開頭故事中那顆馬達的死因。

$$P = \tau \cdot \omega \quad (\omega \text{ 單位為 rad/s ; 若用 rpm 需換算 : } \omega = 2\pi N/60)$$

馬達輸出功率的基本關係式。

5.3 無刷直流馬達

5.3.1 去掉電刷之後

有刷馬達的電刷是機械接觸元件，會磨損、會產生火花與電磁干擾、且限制了轉速與壽命。無刷直流馬達（BLDC）把這個機械換向改成電子換向：永久磁鐵移到轉子上、線圈固定在定子上，再由驅動器根據轉子位置（以霍爾感測器或無感測演算法偵測）切換三相電流的通電順序。

優點極為明顯：壽命長（無磨損件）、效率高（無電刷摩擦與壓降）、功率密度大、散熱好（線圈在外殼上容易導熱）、無火花可用於易燃環境。缺點是必須配合較複雜的三相驅動器，成本較高。

5.3.2 機器人領域的兩種重要變體

外轉子馬達（outrunner）：外殼旋轉、內部固定。轉子直徑大，因此扭力大但轉速較低。無人機的螺旋槳馬達與近年四足機器人的關節馬達多屬此類。

準直驅馬達（quasi-direct drive, QDD）：使用大扭力的外轉子馬達搭配低減速比（約 6:1 到 10:1）的行星減速機。這是近年腿式機器人的關鍵技術突破——低減速比意味著背隙小、摩擦低、且可以「反向驅動」（backdrivable），使關節能感知外力並柔順地順應衝擊。傳統高減速比方案（如諧波減速機 100:1）雖然扭力大、剛性高，但幾乎無法反向驅動，撞到東西時會硬碰硬。第 28 章討論四足機器人的落地緩衝時，會再回到這個特性。

◎ 反向驅動性為何重要

把馬達關掉，用手去轉關節：如果能輕鬆轉動，就是可反向驅動；如果紋風不動，就是不可反向驅動。可反向驅動的關節能透過馬達電流「感覺」到外力，不需額外的力覺感測器，而且撞到人時會自然退讓——這是協作機器人與腿式機器人的安全基礎。減速比越高，反向驅動性越差。

5.4 步進馬達

5.4.1 用磁極數步走

步進馬達的定子上有多組線圈，依序通電時，轉子會被磁力一格一格地「吸」過去。常見的規格是每步 1.8 度（即每轉 200 步）。它的最大特色是：不需要位置回授，只要數出送了幾個脈

波，就知道轉了幾度——這就是所謂的開迴路定位。

優點是控制簡單（脈波數即位置）、低速扭力大、保持扭力強（停止時仍能牢牢抓住位置）、成本低。這使它成為 3D 印表機、雕刻機與小型 XY 平台的主流選擇。

5.4.2 失步：開迴路的致命弱點

步進馬達的最大風險是失步（step loss）。當負載超過馬達能力、或加速過快時，轉子跟不上磁場的變化，就會「跳過」幾步。此時控制器仍然以為位置正確，但實際位置已經偏移，且這個誤差會永久累積——因為沒有回授可以修正。3D 印表機列印到一半突然「錯層」，多半就是失步造成的。

此外，步進馬達的扭力隨轉速上升而急遽下降，高速時扭力所剩無幾；且在特定轉速下會產生共振而失步。因應之道包括：加大安全餘裕（通常留 50% 以上）、限制加速度、採用微步驅動（microstepping）平滑運轉，或加裝編碼器變成閉迴路步進系統。

5.5 伺服馬達

5.5.1 伺服不是一種馬達，而是一套系統

嚴格來說，「伺服」（servo）指的不是馬達的種類，而是一套具備位置回授的閉迴路控制系統：馬達 + 編碼器 + 驅動器 + 控制迴路。馬達本身可能是有刷或無刷直流馬達，關鍵在於它有回授、能自我修正。

工業伺服系統通常採三層串級控制：最內層是電流（扭力）迴路，週期約數十微秒；中層是速度迴路，週期約數百微秒；外層是位置迴路，週期約一毫秒。這正是第 3 章時間尺度金字塔的具體實例，其控制原理將在第 16 章詳述。

5.5.2 RC 伺服機：教學與社團的主力

學生最常接觸的是 RC 伺服機（俗稱舵機），如 SG90、MG996R 等。它把馬達、減速齒輪、電位器與控制電路封裝在一個小盒子裡，只需三條線（電源、地、訊號），用 PWM 訊號的脈寬即可指定角度——標準協定是 1.0 毫秒對應 0 度、1.5 毫秒對應 90 度、2.0 毫秒對應 180 度。

使用 RC 伺服機有三個實務要點。第一，型錄標示的扭力（如「25 kg·cm」）是堵轉扭力，持續可用約為其三分之一。第二，必須使用獨立電源——伺服機啟動瞬間的電流可達 1~2 安培，

若與微控制器共用電源會造成重開機（第 3 章的經典故障）。第三，塑膠齒輪的機型在受到衝擊時容易崩齒，關節類應用建議選用金屬齒輪版本。

表 5-3 四種常見馬達的比較

馬達類型	定位方式	優點	缺點	典型應用	本書實作
有刷直流馬達	需外加編碼器	便宜、驅動簡單	電刷磨損、有火花	輪型底盤、玩具	附錄 A、F
無刷直流馬達	霍爾或編碼器	效率高、壽命長、功率密度大	驅動器複雜、成本高	無人機、四足關節	附錄 I、J
步進馬達	開迴路（脈波計數）	低速扭力大、定位簡單	會失步、高速無力	3D 印表機、XY 平台	—
RC 伺服機	內建閉迴路	整合度高、只需三條線	扭力有限、精度普通	小型手臂、雙足關節	附錄 B ~ E、J

5.6 馬達的扭力、轉速與功率計算

5.6.1 靜態扭力：對抗重力

最基本的需求是舉起負載所需的靜態扭力。對一根水平伸出的連桿：

$$\tau_{\text{static}} = m \cdot g \cdot L$$

m 為負載質量 (kg)， $g = 9.8 \text{ m/s}^2$ ， L 為重心到轉軸的距離 (m)。若連桿本身有重量，需另計其重心力矩並相加。

注意兩件事。第一，力臂 L 是「重心」到轉軸的距離，不是連桿長度——若連桿均勻，其重心在中點。第二，這是最壞情況（連桿水平時力矩最大）；當連桿轉到垂直時，重力力矩為零。設計時必須以最壞情況為準。

5.6.2 動態扭力：對抗慣性

要讓手臂加速，還需要額外的扭力，由牛頓第二定律的轉動版本給出：

$$\tau_{\text{dynamic}} = J \cdot \alpha$$

J 為轉動慣量 ($\text{kg}\cdot\text{m}^2$)， α 為角加速度 (rad/s^2)。

角加速度可由運動需求推算。若採用簡單的「加速一半、減速一半」三角形速度曲線，在時間 t 內轉過角度 θ ：

$$\alpha = 40 / t^2$$

三角形速度曲線下的角加速度。若採梯形曲線（含等速段），加速度會略大，需依實際曲線計算（第 15 章）。

5.6.3 總需求與安全餘裕

馬達所需的總扭力為兩者相加，再乘上安全係數：

$$\tau_{\text{required}} = (\tau_{\text{static}} + \tau_{\text{dynamic}}) \times SF$$

SF 為安全係數，一般取 1.5~2.0，考慮摩擦、效率損失、製造誤差與未來的負載增加。

最後，把這個扭力需求與馬達的**額定扭力**（不是堵轉扭力）比較。若使用減速機，需求扭力要除以減速比與傳動效率，才能得到馬達端的需求——這是 5.8 節的內容。例題 5-1 會完整走一遍這個流程。

5.7 減速機與齒輪比

5.7.1 為什麼幾乎每個關節都需要減速機

電動馬達有一個先天特性：它們在高轉速時效率最好，但機器人的關節需要的是低轉速、大扭力。一顆典型的無刷馬達最佳工作點在每分鐘三千轉以上，而機器手臂的關節轉速通常低於每分鐘六十轉。這中間相差五十倍以上，必須靠減速機來轉換。

減速機的作用可以用一句話概括：用轉速換扭力。減速比 N 定義為輸入轉速與輸出轉速之比，其效果為：

$$\omega_{\text{out}} = \omega_{\text{in}} / N \quad \tau_{\text{out}} = \tau_{\text{in}} \times N \times \eta$$

N 為減速比， η 為傳動效率（一般 0.7~0.95）。轉速降為 $1/N$ ，扭力放大 N 倍（再乘上效率）。

這說明了一個重要的設計槓桿：與其買一顆大十倍的馬達，不如加一組減速比 10 的減速機——後者通常更輕、更便宜、也更容易散熱。這正是所有工業機器人關節都內建減速機的原因。

5.7.2 減速機對慣量的影響：平方關係

減速機還有一個常被忽略、卻極其重要的效果：它會以平方的比例縮小負載慣量在馬達端看到的大小。

$$J_{\text{reflected}} = J_{\text{load}} / N^2$$

折算到馬達端的負載慣量與減速比的平方成反比。

這個平方關係是機器人設計的關鍵武器。假設負載慣量是 $1 \text{ kg}\cdot\text{m}^2$ ，透過減速比 100 的減速

機後，馬達「感覺到」的慣量只有 $0.0001 \text{ kg}\cdot\text{m}^2$ ——縮小了一萬倍。這使得一顆慣量極小的馬達，也能穩定地控制一個沉重的手臂。5.10 節的慣量匹配就建立在這個關係上。

但天下沒有白吃的午餐：減速比越高，反向驅動性越差（見 5.3.2 節）、背隙的影響被放大到輸出端、且系統的響應速度受限。這就是為什麼腿式機器人選擇低減速比（要柔順）、而工業手臂選擇高減速比（要剛硬與精準）的根本原因。

5.8 諧波、行星與擺線減速機

5.8.1 諧波減速機：工業手臂的心臟

諧波減速機（Harmonic Drive，又稱諧波齒輪）由三個元件組成：橢圓形的波產生器（wave generator，接馬達）、可撓的柔輪（flexspline，薄壁杯狀齒輪）、以及固定的剛輪（circular spline）。波產生器旋轉時，把柔輪撐成橢圓，使柔輪的齒在長軸兩端與剛輪嚙合。由於柔輪比剛輪少兩個齒，波產生器每轉一圈，柔輪只會相對倒退兩個齒——因此可在單級中達到 50:1 到 160:1 的極高減速比。

優點是：減速比極高且體積小、零背隙（齒是被壓著嚙合的）、精度極高、同時有多個齒接觸因此扭力大。缺點是：柔輪要反覆彈性變形，有疲勞壽命限制；剛性雖高但有非線性的扭轉柔度；價格昂貴（往往比馬達本身還貴）；且幾乎完全無法反向驅動。

這是六軸工業手臂與協作機器人手腕關節的標準配備，也是台灣廠商積極投入國產化的關鍵零組件——第 1 章提到的上銀科技即在此領域布局。

5.8.2 行星減速機：均衡的通才

行星減速機由中心的太陽輪、周圍的行星輪、外圈的環齒輪與行星架組成。動力由太陽輪輸入，行星輪同時自轉與公轉，由行星架輸出。因為多個行星輪同時分擔負載，扭力密度高且結構對稱、徑向力平衡。

單級減速比通常在 3:1 到 10:1，可多級串接達到更高比值（但效率會逐級相乘而下降）。優點是效率高（單級可達 95% 以上）、可反向驅動、成本適中、可承受高輸入轉速。缺點是有背隙（精密型可做到幾個角分，但成本上升）。這是四足機器人準直驅關節、AMR 輪組與各類通用場合的主流選擇。

5.8.3 擺線減速機：重載的硬漢

擺線減速機 (cycloidal drive) 利用偏心軸帶動擺線盤，擺線盤的曲線輪廓與固定的針齒接觸並滾動，產生減速。因為同時有多個齒接觸 (可達 60% 以上的齒同時受力)，承載能力與抗衝擊能力極強。

優點是：扭力密度最高、抗過載與衝擊能力極佳、剛性高、壽命長。缺點是結構複雜、成本高、有輕微背隙、且高速時因偏心運動會產生振動。典型應用在大型工業手臂的基座關節 (第一、二軸，承受整條手臂的重量) 與重載機械。

表 5-4 三種精密減速機的比較

特性	諧波減速機	行星減速機	擺線減速機
單級減速比	50 ~ 160	3 ~ 10	10 ~ 100
背隙	接近零	小 (精密型幾角分)	小
效率	70 ~ 85%	90 ~ 97%	80 ~ 90%
扭力密度	高	中	極高
抗衝擊	較弱 (柔輪疲勞)	中	極佳
反向驅動	幾乎不可	可	困難
成本	高	中	高
典型應用	工業手臂手腕、協作機器人	四足關節、AMR 輪組	大型手臂基座、重載

◎ 背隙的實際影響

背隙是齒輪嚙合的間隙，表現為「反轉時要空轉一小段才開始出力」。在定位任務中，背隙造成的誤差無法用位置回授完全消除 (因為編碼器多裝在馬達端，看不到輸出端的空轉)；在力控制中，背隙會讓接觸瞬間產生衝擊。這是精密機器人堅持使用零背隙減速機的原因，也是為什麼工業手臂的重複精度 (同方向) 遠優於雙向精度。

5.9 螺桿、皮帶、鏈條與鋼索傳動

5.9.1 滾珠螺桿：把旋轉變成直線

滾珠螺桿 (ball screw) 將馬達的旋轉運動轉換成直線運動：螺桿旋轉，螺帽沿軸向移動。與傳統梯形螺桿相比，滾珠螺桿在螺紋槽中放入循環滾動的鋼珠，把滑動摩擦變成滾動摩擦，效率由約 30% 提升到 90% 以上。

關鍵參數是導程 (lead) ——螺桿每轉一圈螺帽移動的距離。導程小則推力大、解析度高，但速度慢。直線運動的位置換算為：

$$x = (\theta / 2\pi) \times \text{lead} \quad F = (2\pi \times \eta \times \tau) / \text{lead}$$

x 為直線位移， θ 為螺桿轉角 (rad)， lead 為導程 (m)； F 為推力 (N)， τ 為馬達扭力 (N·m)， η 為效率。

滾珠螺桿是工具機、3D 印表機 Z 軸與直角座標機器人的核心元件，也是台灣精密機械產業的強項——上銀科技即為全球主要供應商之一。

5.9.2 同步皮帶：輕量與遠距離傳動

同步皮帶 (timing belt) 帶有齒形，能無滑動地傳遞旋轉。優點是輕、安靜、可長距離傳動、有一定彈性可吸收衝擊、且能把馬達移到遠離運動部位的地方 (降低運動慣量)。缺點是有彈性變形 (高精度定位時會造成遲滯)、需要張力調整、且會隨時間鬆弛。

典型應用是 3D 印表機的 XY 軸、SCARA 的第二軸傳動、以及協作機器人把馬達往基座移動的設計。設計時要注意皮帶張力：太鬆會跳齒，太緊會增加軸承負荷並縮短壽命。

5.9.3 鏈條與鋼索：重載與遠端驅動

鏈條傳動能承受大扭力、不易打滑，但笨重、有噪音、需要潤滑，機器人上較少用，多見於大型移動平台。

鋼索傳動 (cable/tendon drive) 則是仿生設計的代表：像人的肌腱一樣，用鋼索把驅動力從遠端傳到關節。最大優點是可以把所有馬達放在基座或手掌根部，讓末端極輕巧——這是靈巧機械手 (如 DLR 手、Shadow Hand) 的標準做法，也呼應第 21 章提到的人手生物力學。缺點是鋼索會伸長、有摩擦遲滯、且只能拉不能推 (需成對配置或加彈簧回復)。

表 5-5 四種傳動方式的比較

方式	運動轉換	效率	剛性	優點	限制	典型應用
滾珠螺桿	旋轉→直線	90%+	高	推力大、精度高	行程受限、成本高	工具機、3D 印表機 Z 軸
同步皮帶	旋轉→旋轉 / 直線	95%+	中	輕、安靜、可遠傳	有彈性、需張力調整	3D 印表機 XY、SCARA
鏈條	旋轉→旋轉	90%+	高	承載大、不打滑	重、吵、需潤滑	大型移動平台
鋼索 /	旋轉→拉力	70~	低~	末端極輕、可遠	只能拉、會伸長、	靈巧手、仿生機構

方式	運動轉換	效率	剛性	優點	限制	典型應用
鏈		90%	中	端驅動	有遲滯	

5.10 馬達慣量與負載匹配

5.10.1 為什麼太大的馬達反而不好

學生常以為「馬達越大越安全」，但過大的馬達會帶來一個問題：它自己的轉子就很重。當馬達轉子的慣量遠大於負載慣量時，馬達大部分的力氣都花在加速自己身上，系統反應遲鈍、效率低落。反過來，若負載慣量遠大於馬達轉子慣量，則系統容易振盪、控制器增益難以調高——就像用一根細牙籤去攪動一鍋濃粥。

衡量這個關係的指標是慣量比 (inertia ratio)：

$$R = J_{\text{reflected}} / J_{\text{motor}} = (J_{\text{load}} / N^2) / J_{\text{motor}}$$

J_{load} 為負載慣量， N 為減速比， J_{motor} 為馬達轉子慣量。

5.10.2 建議的慣量比範圍

業界的經驗準則是：高精度、高響應的伺服系統建議慣量比在 1 到 3 之間；一般工業應用可放寬到 5 到 10；超過 10 通常會出現整定時間拉長、易振盪、增益調不上去的問題。若慣量比過大，解法有三：提高減速比（記得是平方效果，最有效）、改用轉子慣量較大的馬達（有些廠商提供「大慣量」型號）、或降低負載慣量（減輕連桿重量、把重物往轉軸移近）。

這裡有一個重要的設計洞見：轉動慣量與質量分布的距離平方成正比。把手臂末端的一顆 500 公克馬達，往轉軸方向移近一半的距離，慣量就會降為四分之一。這正是為什麼協作機器人與四足機器人拚命把驅動器往基座移，也是 5.9.3 節鋼索傳動存在的理由。

5.10.3 常見形狀的轉動慣量

實務計算中，最常用的兩個公式是：

$$\text{細長桿繞一端} : J = (1/3) m L^2 \quad \text{質點繞轉軸} : J = m r^2$$

m 為質量 (kg)， L 為桿長， r 為質點到轉軸距離 (m)。

對一根末端帶負載的機器手臂連桿，總慣量約為兩者相加：連桿本身的 $(1/3)m_{\text{link}} L^2$ 加上末端負載的 $m_{\text{load}} L^2$ 。若需要更精確的值，可由 CAD 軟體直接查詢，或用第 14 章的平行軸定

理計算。

5.11 致動器選型流程

5.11.1 七個步驟

完整的致動器選型是一個反覆迭代的過程，但可依下列七步驟進行第一輪估算。

步驟一：確定運動需求。列出負載質量、力臂長度、運動範圍、要求的運動時間與運動曲線類型。

步驟二：計算靜態扭力。以最壞姿態（通常是水平伸出）計算重力力矩，含連桿自重。

步驟三：計算轉動慣量與動態扭力。用 5.10.3 節的公式估算 J ，由運動時間推算角加速度 α ，得 $\tau_{\text{dynamic}} = J \cdot \alpha$ 。

步驟四：加總並乘安全係數。 $\tau_{\text{required}} = (\tau_{\text{static}} + \tau_{\text{dynamic}}) \times SF$ ， SF 建議 1.5 ~ 2.0。

步驟五：選定減速比。由關節所需轉速與馬達的額定轉速決定： $N \approx \omega_{\text{motor_rated}} / \omega_{\text{joint_required}}$ 。

步驟六：反算馬達端需求並選型。 $\tau_{\text{motor}} = \tau_{\text{required}} / (N \times \eta)$ 。以此值對照馬達的額定扭力（非堵轉扭力）挑選型號。

步驟七：驗算慣量比。計算 $R = (J_{\text{load}}/N^2)/J_{\text{motor}}$ ，確認落在建議範圍內。若過大，回到步驟五提高減速比，重新迭代。

圖 5-2 致動器選型流程圖

建議配圖：流程圖。由「運動需求」出發 → 靜態扭力計算 → 動態扭力計算 → 乘安全係數 → 選減速比 → 反算馬達端扭力 → 查型錄選馬達 → 驗算慣量比；慣量比不合格時以回饋箭頭返回「選減速比」重新迭代。右側標註各步驟對應的公式編號。

5.11.2 容易忽略的三件事

第一，工作週期與發熱。馬達的額定扭力是以連續運轉為前提。若你的動作是「快速動作三秒、停十秒」，可以用較高的均方根扭力（RMS torque）作為判準，允許短時間超過額定值。反之，若是連續高負載運轉，還要考慮環境溫度與散熱條件，可能需要降額使用。

第二，摩擦與效率損失。減速機效率、軸承摩擦、皮帶預張力都會吃掉扭力。粗估時可在安

全係數中一併考慮，精算時應逐項列出。

第三，電源能力。馬達的峰值電流可能是額定的數倍，若電源或驅動器供不上這個電流，馬達就無法輸出型錄上的扭力——這是第 3 章電源問題的延伸，也是實務上「明明選對馬達卻沒力」的常見原因。

5.12 馬達驅動器與保護電路

5.12.1 H 橋與 PWM

如第 3 章所述，控制器輸出的是毫安培級的訊號，馬達需要的是安培級的電流，中間必須有驅動器做功率放大。直流馬達驅動器的核心是 H 橋電路：四顆功率電晶體排成 H 形，分別導通對角的兩顆，就能控制電流方向，進而控制轉向。

轉速控制則靠 PWM（脈寬調變）：以高頻（通常 1~20 kHz）快速開關電晶體，改變導通時間的比例（占空比）。若占空比為 60%，馬達感受到的等效電壓就約為電源電壓的 60%。之所以用開關而非線性調壓，是因為電晶體在完全導通或完全截止時發熱最少，效率遠高於線性方式。

PWM 頻率的選擇有個權衡：頻率太低（低於 2 kHz）會聽到馬達發出可聞的嘯叫聲；頻率太高則電晶體的開關損耗增加、發熱上升。實務上多數機器人採用 16~20 kHz，剛好在人耳聽覺上限之外。

5.12.2 三種必備的保護

第一是過流保護。馬達堵轉時電流會飆升到額定值的五到十倍（開頭故事的死因）。驅動器應具備電流偵測與限流功能，或至少在電源路徑上加裝保險絲。進階做法是監控電流並在超過門檻時自動降低占空比或停止輸出。

第二是反電動勢與飛輪二極體。馬達是電感性負載，當電晶體突然截止時，電感會產生極高的反向電壓尖峰，足以擊穿電晶體。解法是在馬達兩端反向並聯二極體（飛輪二極體），提供電流的回流路徑。多數整合型驅動 IC 已內建此保護，但使用分立元件時務必自行加裝。

第三是散熱。功率電晶體導通時仍有壓降，會產生熱。驅動器的持續電流規格是在有散熱片、環境通風的條件下量測的；若你的驅動器裝在密閉外殼中，實際可用電流會顯著低於標示值。實

務準則是：驅動器的持續電流規格至少要是馬達額定電流的 1.5 倍以上。

◎ 驅動器選型的三個數字

一、持續電流：必須大於馬達額定電流的 1.5 倍。二、峰值電流：必須大於馬達堵轉電流（或至少大於你設定的限流值）。三、電壓範圍：電源電壓必須落在驅動器規格內，並注意馬達煞車時的回生電壓可能超過電源電壓。三個數字都要看，只看一個就是下一台燒毀的驅動器。

5.13 機器人關節模組

5.13.1 把整個關節做成一個零件

近年最重要的產業趨勢之一，是關節模組化（joint module 或 actuator module）。傳統做法是工程師自行選購馬達、減速機、編碼器、驅動器與軸承，再自行設計外殼組裝；模組化則把這些全部整合在一個標準化的圓柱體中，對外只提供機械安裝面與一條通訊線（通常是 CAN 或 EtherCAT）。

這個轉變帶來三個好處。第一，開發時間大幅縮短——不需要自己處理散熱、軸承預壓與編碼器對位這些困難的細節。第二，可靠度提升——模組出廠前已完成校正與測試。第三，系統簡潔——串接式的匯流排取代了大量的動力線與訊號線（第 3 章的分散式架構）。

代價是成本較高、且被廠商的介面與韌體綁定。但對多數研發團隊與教學單位而言，用模組換取時間仍然划算——這也是為什麼近年的協作機器人與四足機器人幾乎清一色採用模組化關節。

5.13.2 關節模組的內部構成

一個典型的關節模組由內而外包含：無刷馬達（提供動力）、減速機（諧波或行星）、雙編碼器（馬達端測轉速與換相、輸出端測實際關節角度，可消除減速機背隙的影響）、力矩感測器（部分高階型號，用於力控制與碰撞偵測）、驅動電路板（含電流迴路與通訊介面）、以及煞車器（斷電時鎖住關節，避免手臂掉落）。

值得注意的是「雙編碼器」設計：只在馬達端裝編碼器的話，減速機的背隙與扭轉變形對控制器是不可見的；加裝輸出端編碼器後，控制器才能看到關節的真實角度。這是高階協作機器人能達到高絕對精度的關鍵，也解釋了第 4 章提到的「重複精度與絕對精度差一個數量級」現象。

5.14 台灣的關鍵零組件產業

5.14.1 傳動元件的世界級聚落

如第 1 章所述，台灣在精密機械領域有完整的產業聚落，而傳動元件正是其中最具有國際競爭力的一環。上銀科技 (HIWIN) 的滾珠螺桿與線性滑軌是全球工具機與自動化設備的重要供應來源，並延伸至諧波減速機、直驅馬達與醫療機器人；台達電子 (Delta) 從電源與變頻器技術出發，發展出完整的伺服驅動器與馬達產品線，並整合為 SCARA 機器人與整廠自動化方案。此外，和大工業、利茗機械等廠商也在減速齒輪領域深耕。

5.14.2 國產化的挑戰與機會

機器人三大關鍵零組件是減速機、伺服馬達與控制器，長期以來高階市場由日本與歐洲廠商主導（諧波減速機、RV 減速機、高階伺服系統）。台灣的機會在於：一、精密加工能力與供應鏈完整，具備切入減速機製造的基礎；二、電力電子與馬達驅動技術成熟，伺服驅動器已具國際競爭力；三、協作機器人與 AMR 等新興市場尚未被完全壟斷，是切入的窗口。

對學生的意義是：這些領域正是台灣產業最需要人才的地方。理解本章的扭力計算、慣量匹配與減速機原理，不只是為了考試，而是進入這個產業的入場券。

完整計算例題

例題 5-1 機械手臂關節的完整選型

題目：設計一支水平伸出的機械手臂關節。連桿長 400 mm、質量 0.6 kg（均勻分布）；末端負載含夾具共 1.5 kg。要求在 0.8 秒內從水平轉到垂直（90 度），採三角形速度曲線。安全係數取 1.8，減速機效率 0.9。試完成選型計算。

解（一）靜態扭力：以水平姿態為最壞情況。連桿重心在中點（0.2 m），負載在末端（0.4 m）。

$$\tau_{\text{static}} = (0.6 \times 9.8 \times 0.2) + (1.5 \times 9.8 \times 0.4) = 1.176 + 5.88 = 7.06 \text{ N}\cdot\text{m}$$

連桿自重重力矩加上負載力矩。

解（二）轉動慣量：連桿繞一端 $J = (1/3)mL^2$ ，負載視為質點 $J = mr^2$ 。

$$J = (1/3)(0.6)(0.4^2) + (1.5)(0.4^2) = 0.032 + 0.240 = 0.272 \text{ kg}\cdot\text{m}^2$$

連桿與負載的轉動慣量總和。

解（三）角加速度：轉 90 度即 $\theta = \pi/2 \approx 1.571 \text{ rad}$ ， $t = 0.8 \text{ s}$ ，三角形速度曲線 $\alpha = 4\theta/t^2$ 。

$$\alpha = 4 \times 1.571 / 0.8^2 = 6.284 / 0.64 \approx 9.82 \text{ rad/s}^2$$

所需角加速度。

解（四）動態扭力與總需求：

$$\tau_{\text{dynamic}} = J \cdot \alpha = 0.272 \times 9.82 \approx 2.67 \text{ N}\cdot\text{m}$$

克服慣性所需扭力。

$$\tau_{\text{required}} = (7.06 + 2.67) \times 1.8 = 9.73 \times 1.8 \approx 17.5 \text{ N}\cdot\text{m}$$

關節輸出端所需扭力（含安全係數）。

解（五）選減速比：關節平均角速度 = $1.571/0.8 \approx 1.96 \text{ rad/s} \approx 18.8 \text{ rpm}$ ；三角形曲線的峰值角速度是平均的兩倍，約 37.6 rpm 。若選用額定轉速 3000 rpm 的馬達：

$$N \approx 3000 / 37.6 \approx 80 \rightarrow \text{取標準值 } N = 80$$

減速比選定。

解（六）反算馬達端扭力：

$$\tau_{\text{motor}} = 17.5 / (80 \times 0.9) \approx 0.243 \text{ N}\cdot\text{m} \approx 243 \text{ mN}\cdot\text{m}$$

馬達所需的額定扭力。

因此應選擇額定扭力大於 $243 \text{ mN}\cdot\text{m}$ 、額定轉速約 3000 rpm 的無刷馬達，搭配減速比 80 的減速機。請注意：這裡比較的是馬達的**額定扭力**，不是堵轉扭力。

解（七）驗算慣量比：假設所選馬達的轉子慣量 $J_{\text{motor}} = 3.0 \times 10^{-5} \text{ kg}\cdot\text{m}^2$ 。

$$J_{\text{reflected}} = 0.272 / 80^2 = 0.272 / 6400 \approx 4.25 \times 10^{-5} \text{ kg}\cdot\text{m}^2$$

折算到馬達端的負載慣量。

$$R = 4.25 \times 10^{-5} / 3.0 \times 10^{-5} \approx 1.42$$

慣量比落在建議範圍 $1 \sim 3$ 內，匹配良好。

結論：本設計的慣量比為 1.42 ，屬於高響應伺服系統的理想範圍，選型成立。若慣量比算出來是 15 ，就應回到步驟五提高減速比（例如改用 $N = 120$ ，慣量比會降為約 0.63 倍，即約 $0.63 \times 15 \dots$ 實際為 $0.272/120^2/3e-5 \approx 0.63$ ），或改選轉子慣量較大的馬達。

例題 5-2 開頭故事的診斷

題目：回到本章開頭。學生使用標示「 $25 \text{ kg}\cdot\text{cm}$ 」的 RC 伺服機，手臂連桿 30 cm 、負載 500 g 。（a）驗算他們的靜態扭力計算是否正確。（b）分析為何馬達仍然燒毀。（c）提出兩個

改善方案。

解 (a) : $25 \text{ kg}\cdot\text{cm}$ 換算為 SI 單位 : $25 \times 0.0098 \approx 0.245 \text{ kg}\cdot\text{m}\cdot\text{g}$... 正確算法為 $25 \text{ kgf}\cdot\text{cm} = 25 \times 9.8 \text{ N} \times 0.01 \text{ m} = 2.45 \text{ N}\cdot\text{m}$ 。學生計算的靜態需求 $\tau = 0.5 \times 9.8 \times 0.3 = 1.47 \text{ N}\cdot\text{m}$ 。表面上餘裕率 $= (2.45 - 1.47)/1.47 \approx 67\%$ ，計算本身無誤。

解 (b) : 問題出在三個未被計入的因素。第一， $2.45 \text{ N}\cdot\text{m}$ 是堵轉扭力，持續可用約為其 $1/3$ ，即約 $0.82 \text{ N}\cdot\text{m}$ ——已經低於靜態需求的 $1.47 \text{ N}\cdot\text{m}$ 。第二，未計連桿自重：若連桿 200 g 、長 30 cm ，另需 $0.2 \times 9.8 \times 0.15 \approx 0.29 \text{ N}\cdot\text{m}$ 。第三，未計動態扭力與定位時的抖動堵轉發熱。實際持續需求已遠超過馬達的持續能力，長時間運轉必然過熱燒毀。

解 (c) : 方案一，改用額定 (非堵轉) 扭力大於 $(1.47+0.29)\times 1.8 \approx 3.2 \text{ N}\cdot\text{m}$ 的伺服系統，或改用無刷馬達加減速機。方案二，改變機構——縮短力臂至 15 cm ，可使力矩需求減半；或加裝配重與彈簧來平衡重力 (重力補償，第 17 章)，大幅降低持續扭力需求。方案三 (附加)：加裝電流監控，在偵測到持續大電流時自動斷電保護。

程式範例：致動器選型計算器

以下 Python 程式實作 5.11 節的七步驟選型流程，可直接輸入設計參數，自動計算扭力需求、減速比、馬達端扭力與慣量比，並判斷匹配是否合格。程式最後以例題 5-1 的數據驗證。

```
import math

G = 9.8 # 重力加速度 m/s^2

def select_actuator(link_len, link_mass, load_mass,
                    angle_deg, time_s, sf=1.8,
                    gear_eff=0.9, motor_rpm=3000,
                    motor_inertia=3.0e-5, gear_ratio=None):
    """機器人關節致動器選型計算 (七步驟)"""

    # 步驟 2：靜態扭力 (水平姿態，最壞情況)
    t_static = (link_mass * G * link_len / 2) + (load_mass * G * link_len)

    # 步驟 3：轉動慣量與動態扭力
    J = (1/3) * link_mass * link_len**2 + load_mass * link_len**2
    theta = math.radians(angle_deg)
    alpha = 4 * theta / time_s**2 # 三角形速度曲線
    t_dynamic = J * alpha
```

```

# 步驟 4：總需求含安全係數
t_required = (t_static + t_dynamic) * sf

# 步驟 5：選減速比 (依峰值轉速)
w_avg_rpm = (theta / time_s) * 60 / (2 * math.pi)
w_peak_rpm = w_avg_rpm * 2          # 三角形曲線峰值為平均兩倍
if gear_ratio is None:
    gear_ratio = round(motor_rpm / w_peak_rpm / 10) * 10

# 步驟 6：反算馬達端扭力
t_motor = t_required / (gear_ratio * gear_eff)

# 步驟 7：慣量比驗算
j_reflected = J / gear_ratio**2
ratio = j_reflected / motor_inertia

if ratio < 1:      verdict = "偏低 (馬達慣量過大，反應遲鈍)"
elif ratio <= 3:  verdict = "理想 (高響應伺服系統建議範圍)"
elif ratio <= 10: verdict = "可接受 (一般工業應用)"
else:             verdict = "過高！易振盪，建議提高減速比"

print("=" * 48)
print(f"靜態扭力  $\tau_{static}$  = {t_static:8.3f} N·m")
print(f"轉動慣量 J = {J:8.4f} kg·m2")
print(f"角加速度  $\alpha$  = {alpha:8.2f} rad/s2")
print(f"動態扭力  $\tau_{dynamic}$  = {t_dynamic:8.3f} N·m")
print(f"需求扭力(含 SF={sf}) = {t_required:8.2f} N·m (關節輸出端)")
print("-" * 48)
print(f"峰值關節轉速 = {w_peak_rpm:8.1f} rpm")
print(f"建議減速比 N = {gear_ratio:8d}")
print(f"馬達端所需額定扭力 = {t_motor*1000:8.1f} mN·m")
print("-" * 48)
print(f"折算負載慣量 = {j_reflected:.3e} kg·m2")
print(f"慣量比 R = {ratio:8.2f} → {verdict}")
print("=" * 48)
return t_motor, gear_ratio, ratio

# 驗證：例題 5-1
select_actuator(link_len=0.4, link_mass=0.6, load_mass=1.5,
                angle_deg=90, time_s=0.8)

```

請試著修改參數觀察趨勢：(一) 把連桿長度加倍，需求扭力變成幾倍？(二) 把運動時間從 0.8 秒縮短到 0.4 秒，動態扭力如何變化？(三) 把減速比從 80 改成 20，慣量比會變成多少？這三個實驗會讓你對本章的三個平方關係產生直覺。

實作：馬達特性實測

本章的實作是量測一顆真實馬達的扭力—轉速曲線，親眼驗證 5.2 節的理論。所需器材：直流馬達（含編碼器或轉速計）、可調電源供應器、電流表、滑輪與細繩、砝碼組或彈簧秤。

1. 固定馬達，在輸出軸裝上已知半徑 r 的滑輪，繞上細繩並在末端掛砝碼。負載扭力 $\tau = m \cdot g \cdot r$ 。
2. 以固定電壓驅動馬達，由空載開始逐步增加砝碼重量，記錄每一組的：砝碼質量、轉速（rpm）、電流（A）。至少取八組數據，直到馬達停轉（堵轉）。
3. 將數據換算為扭力（N·m）與角速度（rad/s），繪製扭力—轉速圖，觀察是否呈線性關係。
4. 由圖上外插求出空載轉速與堵轉扭力，與型錄標示值比較，計算誤差百分比。
5. 計算每組數據的輸出功率 $P = \tau \cdot \omega$ 與輸入功率 $P_{in} = V \cdot I$ ，繪製功率曲線與效率曲線，找出最大功率點與最佳效率點，驗證兩者確實不重合。
6. 安全注意：堵轉狀態下電流極大且發熱迅速，每次堵轉測試不得超過三秒，並立即斷電冷卻。

延伸實驗：把同一顆馬達裝上減速比 N 的減速機，重複量測，驗證輸出扭力是否約為原來的 N 倍、轉速是否降為 $1/N$ ，並由兩者的乘積變化估算減速機的傳動效率。

國中小也看得懂：馬達就像腳踏車的變速器

你有沒有騎過有變速器的腳踏車？上坡的時候要換到「輕齒輪」，踩起來很輕鬆，但騎得慢；平路換到「重齒輪」，踩起來費力，但速度快。這就是機器人身上的「減速機」在做的事情——用速度換力氣。

馬達其實跟腳踏車一樣：它轉得很快，但力氣很小。如果直接把馬達裝在機器人的手臂上，手臂會轉得超快卻舉不起任何東西。所以我們幫它裝上一組齒輪，把「轉很快、沒力氣」變成「轉很慢、力氣大」。減速機讓馬達轉一百圈，手臂才轉一圈——但力氣就變成大約一百倍！

還有一件重要的事：馬達會發熱。當你用手把機器人的手臂用力抓住不讓它動，馬達會拚命想轉卻轉不動，這時候它會非常燙，甚至燒壞。所以玩機器人的時候，千萬不要抓住正在動的手臂不放喔。如果馬達摸起來燙手，就要馬上關掉電源讓它休息。

叫獸提醒與常見錯誤

- 誤區一：把堵轉扭力當成可用扭力。型錄上漂亮的數字通常是瞬間極限值，持續可用約為其三分之一到二分之一。
- 誤區二：只算靜態扭力，忘記加速所需的動態扭力。快速運動的機構，動態扭力可能超過靜態扭力。
- 誤區三：計算力臂時用連桿長度而非重心距離。均勻連桿的重心在中點，力矩只有用全長計算的一半。
- 誤區四：忘記算連桿自身的重量。長而重的連桿，其自重力矩往往不亞於負載力矩。
- 誤區五：以為馬達越大越好。過大的轉子慣量會使系統反應遲鈍；慣量比應落在 1~10 之間。
- 誤區六：忽略減速比對慣量的平方效果。提高減速比是改善慣量比最有效的手段，效果是平方級的。
- 誤區七：驅動器只看持續電流。必須同時滿足持續電流、峰值電流與電壓範圍三項規格，並確保散熱條件與規格書一致。
- 誤區八：伺服機與微控制器共用電源。伺服機啟動瞬間的電流會拉低電壓造成重開機，必須獨立供電並共地。
- 誤區九：忽略反向驅動性。高減速比的關節無法被外力推動，撞到人時是硬碰硬——協作與腿式機器人必須慎選減速比。
- 誤區十：讓馬達長時間堵轉。這是燒毀馬達的第一元凶，務必加裝電流監控或熱保護。

課堂討論

1. 開頭故事中，若你是那兩位學生，在買馬達之前應該做哪三項計算？請依 5.11 節的流程說明。
2. 為什麼波士頓動力早期的機器人使用液壓，而近年的四足機器人多改用電動準直驅？請從功率密度、控制精度、維護與成本四方面分析。
3. 諧波減速機幾乎無法反向驅動，這對協作機器人的安全設計造成什麼挑戰？工程上有哪些補救方式（提示：力矩感測器、電流監控、第 20 章的柔順控制）？
4. 同樣要移動一個 5 公斤的物體 30 公分：方案 A 用滾珠螺桿、方案 B 用同步皮帶、方案 C

用鋼索。請分別分析三者速度、精度、成本與重量上的取捨。

5. 假設你要設計一支能與人握手的機械手臂。從致動器選型的角度，你會做哪些不同於工業手臂的決定？為什麼？

章末摘要

- 致動器分電動、液壓、氣壓三類：電動控制精準為主流、液壓功率密度最高用於重載、氣壓天生柔順用於夾爪與軟性機器人。
- 直流馬達的兩個核心關係：扭力與電流成正比 ($\tau = K_t \cdot I$)、反電動勢與轉速成正比；因此扭力與轉速呈線性反比。
- 最大功率點在堵轉扭力與空載轉速的各一半處，但此處效率僅約 50%；最佳效率點落在較低扭力區。
- 額定扭力（可持續）與堵轉扭力（瞬間極限）必須分清，後者通常是前者的二到三倍；混淆兩者是燒毀馬達的首要原因。
- 無刷馬達以電子換向取代電刷，壽命長效率高；準直驅（大扭力馬達 + 低減速比）具備反向驅動性，是腿式機器人的關鍵技術。
- 步進馬達以脈波開迴路定位，簡單但會失步；RC 伺服機整合度高，但需獨立供電且標示扭力為堵轉值。
- 扭力需求 = (靜態 + 動態) × 安全係數；靜態用重心距離計算、動態為 $J \cdot \alpha$ 、安全係數建議 1.5 ~ 2.0。
- 減速機用轉速換扭力：轉速降 $1/N$ 、扭力升 N 倍、而折算慣量降 $1/N^2$ （平方效果，是慣量匹配最有力的槓桿）。
- 諧波減速機零背隙高減速比適合工業手臂；行星減速機效率高可反驅適合腿式與 AMR；擺線減速機抗衝擊適合重載基座。
- 慣量比建議落在 1 ~ 3（高響應）或 1 ~ 10（一般工業）；過高易振盪，可提高減速比、換大慣量馬達或減輕負載改善。
- 驅動器選型須同時滿足持續電流（≥ 馬達額定 1.5 倍）、峰值電流與電壓範圍，並具備過流、飛輪二極體與散熱三項保護。
- 關節模組化整合馬達、減速機、雙編碼器、驅動器與煞車，是近年協作與四足機器人的主

流；雙編碼器是達成高絕對精度的關鍵。

觀念題與計算題

觀念題

1. 比較電動、液壓與氣壓三類致動器在功率密度、控制精度、維護與成本上的差異，各舉一個適用場合。
2. 說明直流馬達扭力與轉速呈反比的物理原因（提示：反電動勢）。
3. 在扭力—轉速曲線上標出：堵轉扭力、空載轉速、最大功率點、最佳效率點，並說明為何後兩者不重合。
4. 額定扭力與堵轉扭力有何差異？為什麼混淆兩者會導致馬達燒毀？
5. 無刷馬達相較有刷馬達有哪四項優點？代價是什麼？
6. 什麼是反向驅動性？它與減速比有何關係？為何對協作機器人與腿式機器人特別重要？
7. 步進馬達的失步是什麼？列出三種預防方法。
8. 說明減速機對轉速、扭力與慣量的三種影響，並指出哪一項是平方關係。
9. 比較諧波、行星與擺線減速機的減速比、背隙、效率與抗衝擊能力。
10. 什麼是背隙？為什麼位置回授無法完全消除背隙造成的誤差？
11. 解釋慣量比的意義與建議範圍，並列出慣量比過高時的三種改善方法。
12. 馬達驅動器必須具備哪三項保護？飛輪二極體的作用為何？

計算題

1. 某機械手臂連桿長 0.5 m、質量 1.2 kg（均勻分布），末端負載 2.0 kg，需在 1.0 秒內轉動 120 度（三角形速度曲線）。安全係數 1.8、減速機效率 0.88。（a）求靜態扭力。（b）求轉動慣量與動態扭力。（c）求關節端所需總扭力。（d）若選用額定轉速 4000 rpm 的馬達，建議減速比為何？（e）求馬達端所需額定扭力。
2. 承上題，若所選馬達的轉子慣量為 $5.0 \times 10^{-5} \text{ kg} \cdot \text{m}^2$ 。（a）計算折算慣量與慣量比。（b）判斷是否合格。（c）若不合格，欲將慣量比降到 3 以下，減速比至少需提高到多少？
3. 一顆直流馬達的規格為：空載轉速 6000 rpm、堵轉扭力 0.6 N·m、額定電壓 24 V、堵轉電流 12 A。（a）求扭力常數 K_t （提示： $\tau = K_t \cdot I$ ，堵轉時電流最大）。（b）求最大輸出功率

及其發生的轉速與扭力。(c) 在最大功率點時，輸入功率為多少？效率為何？

4. 某滾珠螺桿導程 5 mm、效率 0.9，由額定扭力 0.5 N·m 的馬達直接驅動。(a) 求可產生的最大推力。(b) 若馬達以 1500 rpm 運轉，求螺帽的移動速度 (mm/s)。(c) 若欲將推力提升一倍而保持相同馬達，可採取哪兩種方法？請各計算所需的參數變更。

動手做與專題延伸

本章實作：馬達型錄判讀練習

上網或向廠商索取三份不同類型馬達的型錄 (建議：一份 RC 伺服機、一份無刷馬達、一份含減速機的伺服馬達)，完成下表。目的是學會在真實文件中找到正確的數字。

表 5-6 馬達型錄判讀表 (第一列為範例)

項目	馬達 A (範例：RC 伺服機)	馬達 B	馬達 C
型號	MG996R		
標示扭力與種類	11 kg·cm (堵轉 · 6V)		
推估可持續扭力	約 0.36 N·m (標示值 1/3)		
額定 / 空載轉速	約 60 rpm (0.17 s/60°)		
工作電壓 / 電流	4.8 ~ 6.6 V / 堵轉約 2.5 A		
轉子慣量	型錄未提供		
是否可反向驅動	否 (塑膠蝸桿減速)		
適用場合判斷	小型手臂關節、教學用途		

專題延伸

- 為附錄 D 的四伺服雙足機器人做完整的致動器驗算：量測連桿長度與質量、估算最壞姿態的靜態扭力、與所用伺服機的持續扭力比較，判斷是否有餘裕。若不足，提出改善方案。
- 設計一個「重力補償」機構：在機械手臂的關節上加裝彈簧或配重，使手臂在任何姿態下的重力力矩接近零。計算所需的彈簧常數，並比較加裝前後馬達的持續扭力需求。(這是第 17 章重力補償控制的機構版本)
- 進階：比較兩種四足機器人關節方案的優劣——方案 A：小馬達 + 諧波減速機 (減速比 100)；方案 B：大扭力外轉子馬達 + 行星減速機 (減速比 8)。請就扭力密度、反向驅動性、抗衝擊、成本與控制難度五個面向分析，並說明為何近年學界與業界多選擇方案 B (提

示：對照第 28 章的落地衝擊）。

■ 參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 8 章：機構設計與致動器) 。
- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer (致動器與傳動系統專章) 。
- Hughes, A. and Drury, B., Electric Motors and Drives: Fundamentals, Types and Applications, 5th ed., Newnes (馬達與驅動器的經典入門書) 。
- Kenjo, T. and Sugawara, A., Stepping Motors and Their Microprocessor Controls, Oxford University Press.
- Seok, S. et al., "Design Principles for Energy-Efficient Legged Locomotion," IEEE/ASME Trans. Mechatronics, 2015 (準直驅致動器設計原理) 。
- Wensing, P. M. et al., "Proprioceptive Actuator Design in the MIT Cheetah," IEEE Trans. Robotics, 2017 (低減速比高反驅性關節設計) 。
- Harmonic Drive 公司技術手冊：諧波減速機原理與選型指南。
- 各廠商選型工具與技術文件：上銀科技 (HIWIN) 、台達電子 (Delta) 滾珠螺桿、線性滑軌與伺服系統技術資料。
- Maxon Motor 與 Faulhaber 公司的馬達技術手冊 (含扭力—轉速曲線判讀與選型方法的完整說明) 。
- IEC 60034 系列標準：旋轉電機的額定值與性能規範。

第 6 章

感測器與機器人的感官

編著：李世淵（ 機器人叫獸 ）

助教：李天宇、李宇晴

【章首情境圖建議】一台服務型機器人正面朝向一扇玻璃門的照片，畫面上疊加三層視覺化資料：光達點雲（在玻璃處出現一片空洞，彷彿門不存在）、超音波錐形波束（正確偵測到玻璃）、以及相機影像（看見門把與反光）。標題文字：三種感官，三種真相——沒有一個感測器會告訴你它壞了。

叫獸開講 「閉著眼睛也能摸到鼻子，機器人做得到嗎？」

叫獸把一位學生的眼睛蒙上，帶他站到教室中央，然後說：「請你摸到自己的鼻子。」

學生輕鬆地做到了，手指毫不猶豫地落在鼻尖上。「再來，」叫獸說，「請你走到門口。」這次學生猶豫了，伸出雙手向前試探，走了三步就停下來：「我不知道走到哪了。」

「這就是機器人感測器的兩大分類。」叫獸解開他的眼罩，「你能閉著眼睛摸到鼻子，是因為你的肌肉與關節裡有本體感覺——大腦隨時知道每一節手臂彎了幾度，不需要用看的。這叫**內部感測**。但你走不到門口，因為門在哪裡是外部世界的資訊，沒有眼睛就沒有答案。這叫**外部感測**。」

接著他請三位學生上台，分別用三種方法測量教室後牆到講台的距離：第一位用捲尺、第二位用手機的雷射測距儀、第三位用步伐數乘以自己的步長。三個答案分別是 8.42 公尺、8.4 公尺、8.7 公尺。

「哪一個是對的？」叫獸問。學生們說當然是捲尺。「錯，」叫獸說，「捲尺也有誤差，只是比較小而已。真正的重點是——**你永遠不會知道真值**。你只會有一堆帶著不同誤差的量測值。工程師的工作不是找到真值，而是知道每個量測值錯多少、以及怎麼把它們合起來得到更好的估計。」

他在白板上寫下第三位學生的方法：「步伐計數其實就是機器人的**里程計**——用輪子轉了幾圈推算走了多遠。它有一個致命弱點：如果輪子打滑了，它完全不知道。走得越遠，錯得越多，而且錯誤永遠不會自己修正。」

「那雷射測距儀呢？」有人問。「它很準，但它只在有東西可以反射時才有用。對著玻璃、對著黑色絨布、對著鏡子，它會給你完全荒謬的數字——而且它不會告訴你『我這次量錯了』，它會理直氣壯地回報一個錯誤的數字。**沒有一個感測器會告訴你它壞了**，這是初學者最大的陷阱。」

下課前，叫獸留下一個問題：「如果你手上有一個會慢慢累積誤差但很平滑的感測器（里程計），和一個不累積誤差但偶爾會抽風的感測器（雷射），你要怎麼把它們合起來？答案在第 24 章。但在那之前，你得先認識它們每一個的脾氣——這就是今天這堂課要做的事。」

本章為什麼重要

第 4 章給了機器人骨骼、第 5 章給了肌肉，本章給它感官。感測器決定了機器人「能知道什麼」，而機器人永遠不可能對它無法感知的東西做出反應——這是一條無法用演算法繞過的物理界線。再聰明的 AI，接到錯誤的感測資料，也只會做出錯誤的決定。

本章要建立三個核心觀念。第一，每種感測器都有它的物理原理，而原理決定了它的極限與失效模式——超音波為什麼怕斜面、光達為什麼怕玻璃、IMU 為什麼會漂移，都能從原理推導出來。第二，感測器的規格不只是「準不準」，還包括解析度、精度、重複性、更新率與延遲，這五個指標常常互相衝突。第三，沒有完美的感測器，所以融合是必然——這將在第 24 章展開為完整的機率理論。

學習目標

- 能區分內部感測與外部感測，並說明兩者在控制與自主性中的不同角色。
- 能說明編碼器的工作原理，並計算其解析度與四倍頻後的效果。
- 能說明 IMU 三種元件的功能，並解釋加速度計與陀螺儀為何必須互補。
- 能比較超音波、紅外線、ToF 與光達四種測距原理的優缺點與失效情境。
- 能說明相機與深度相機的成像原理，並比較三種深度取得方式。
- 能說明力覺與觸覺感測的原理及其在柔順控制中的角色。
- 能區解析度、精度、重複性、準確度與更新率五個規格指標。
- 能分析感測器的雜訊來源，並說明濾波的基本策略與代價。
- 能說明校正的必要性，並執行一次簡單的感測器校正程序。
- 能為一個指定的機器人任務，選定合適的感測器組合並說明理由。

先備知識

第 3 章的感測器訊號鏈路（換能、調理、ADC、取樣、數值處理）與第 5 章的馬達控制概念。本章涉及基本的三角函數與簡單的統計量（平均值、標準差），若不熟悉可先閱讀附錄 D。機率的完整理論留待第 24 章。

核心名詞中英對照

表 6-1 本章核心名詞

中文	英文	說明
內部感測	Proprioception	量測機器人自身狀態的感知
外部感測	Exteroception	量測外部環境的感知
編碼器	Encoder	將旋轉角度轉換為電訊號的元件
增量式編碼器	Incremental Encoder	輸出相對變化量的編碼器
絕對式編碼器	Absolute Encoder	輸出絕對角度值的編碼器
慣性測量單元	Inertial Measurement Unit, IMU	整合加速度計與陀螺儀的感測模組
漂移	Drift	誤差隨時間累積的現象
飛行時間法	Time of Flight, ToF	以光或聲波往返時間測距的原理
光達	LiDAR	以雷射掃描量測環境距離的感測器
點雲	Point Cloud	由大量三維點構成的環境表示
解析度	Resolution	感測器能分辨的最小變化量
精度	Precision	重複量測結果的一致程度
準確度	Accuracy	量測值接近真值的程度
訊噪比	Signal-to-Noise Ratio, SNR	訊號強度與雜訊強度之比
校正	Calibration	以已知標準修正感測器輸出的程序
感測器融合	Sensor Fusion	整合多感測器資訊以獲得更佳估計

6.1 感測器的角色：內部感測與外部感測

6.1.1 兩種感知，兩種任務

如開頭故事所示，感測器分為兩大類，而這個分類不只是學術上的區分，它直接對應到機器人的兩種能力。

內部感測 (proprioception) 量測機器人自己的狀態：關節轉了幾度、輪子轉了幾圈、身體傾斜多少、馬達流過多少電流、電池剩多少電。它是**控制的基礎**——沒有編碼器，連 PID 都做不了，因為你不知道現在在哪、離目標多遠。

外部感測 (exteroception) 量測外部環境：前方多遠有障礙、目標物在哪、地面是什麼材質、有沒有人靠近。它是**自主的基礎**——沒有外部感測，機器人只能重播固定動作，無法應對環

境變化。

這解釋了一個常見的現象：傳統工業手臂可以只有編碼器（因為環境是固定的、工件永遠在同一個位置），而移動機器人必須有豐富的外部感測（因為環境未知且會變）。當工業手臂要進行「亂料夾取」時，它就必須加上相機——瞬間從內部感測的世界跨入外部感測的世界。

6.1.2 主動式與被動式

外部感測還有另一種分類方式。被動式感測器只接收環境中既有的能量：相機接收環境光、麥克風接收聲音。優點是不干擾環境、功耗低、不會與其他機器人互相干擾；缺點是完全依賴環境條件（相機在黑暗中失效）。

主動式感測器則主動發射能量再接收回波：超音波發射聲波、光達發射雷射、結構光相機投射紅外線圖案。優點是不受環境光影響、可在黑暗中工作、量測距離直接；缺點是耗電、可能互相干擾（兩台超音波機器人靠近時會收到對方的回波）、且會被特定材質欺騙（黑色吸光、玻璃透光、鏡面反射）。

圖 6-1 機器人感測器全景圖

建議配圖：以一台移動操作機器人為中心的分解圖。左側標註內部感測（關節編碼器、IMU、馬達電流、電池電壓、力矩感測器），右側標註外部感測（RGB 相機、深度相機、2D / 3D 光達、超音波、紅外線、麥克風、GNSS）。各感測器以引線指向安裝位置，並標註其量測的物理量與典型更新率。

6.2 編碼器與角度感測

6.2.1 增量式編碼器

編碼器是機器人最重要的感測器，沒有之一——每一個伺服關節都需要它。最常見的光學增量式編碼器原理很簡單：一片刻有等間隔透光縫隙的圓盤隨軸旋轉，一側是光源、另一側是光電接收器，圓盤轉動時光線被反覆遮斷，產生方波脈波。數脈波就知道轉了多少角度。

關鍵設計是兩相正交輸出（A 相與 B 相），兩者相位差 90 度。這帶來兩個好處：第一，可以判斷方向（若 A 領先 B 則正轉，反之則反轉）；第二，可以做四倍頻——A、B 兩相的上升緣與下降緣共有四個邊緣，因此解析度可提升四倍。

$$\text{角度解析度} = 360^\circ / (\text{PPR} \times 4)$$

PPR 為每轉脈波數 (Pulses Per Revolution)，×4 為正交四倍頻後的效果。

例如一顆 1000 PPR 的編碼器，四倍頻後每轉可分辨 4000 個位置，解析度為 0.09 度。若後

方再接減速比 50 的減速機，關節端的解析度更可達 0.0018 度——這正是工業手臂能達到高重複精度的原因。

6.2.2 增量式的弱點與絕對式編碼器

增量式編碼器只知道「變化了多少」，不知道「現在在哪」。因此每次開機都必須執行歸原點（homing）程序：讓關節慢慢移動直到觸發原點開關，才建立起絕對位置的基準。這在工業現場是每天開機的例行程序，但對人形機器人或協作機器人來說很不方便——想像一台機器人每次開機都要先揮舞手臂找原點。

絕對式編碼器則在圓盤上刻有格雷碼（Gray code）或以磁性、電容方式編碼，任一角度都對應唯一的數值，斷電後再開機立刻知道位置。單圈絕對式只知道一圈內的位置；多圈絕對式再加上機械或電子計圈器，可記錄轉了幾圈。代價是成本較高、體積較大。

如第 5 章 5.13.2 節所述，高階關節模組採用雙編碼器：馬達端用增量式（高解析度、供換相與速度控制），輸出端用絕對式（知道關節真實角度、消除減速機背隙的影響）。

表 6-2 常見角度感測方式的比較

方式	原理	解析度	斷電記憶	成本	典型應用
光學增量式編碼器	光遮斷計數	極高（可達數萬 PPR）	無（需歸原點）	中	伺服馬達、工業手臂
光學絕對式編碼器	格雷碼盤讀取	高	有	高	協作機器人關節輸出端
磁性編碼器	霍爾或磁阻感測磁場方向	中（12~14 位元）	有（單圈）	低~中	無刷馬達換相、關節模組
電位器	電阻分壓	低（受雜訊限制）	有	極低	RC 伺服機內部、教學
旋轉變壓器	電磁耦合	高（抗惡劣環境）	有	高	汽車、航太、高溫環境

◎ 編碼器的解析度不等於精度

一顆 4000 線的編碼器可以分辨 0.09 度，但這不代表它的量測誤差只有 0.09 度。安裝偏心、圓盤刻線誤差、軸承間隙都會造成系統性誤差。解析度是「能分辨多細」，精度是「量得多準」——前者容易靠數位技術提升，後者取決於機械製造品質。第 6.8 節會完整區分這組容易混淆的概念。

6.3 IMU：加速度計、陀螺儀與地磁計

6.3.1 三種元件，三種資訊

慣性測量單元 (IMU) 通常整合三種感測元件，各自量測不同的物理量。

加速度計 (accelerometer) 量測比力 (specific force)，也就是「加速度減去重力」。靜止時它讀到的其實是重力的反作用——這個特性很有用：靜止或等速時，加速度計指向的方向就是「下」，因此可以推算傾斜角。缺點是只要機器人在加速，重力與運動加速度就混在一起，無法分離；且對振動極為敏感（馬達的震動會讓讀值劇烈跳動）。

陀螺儀 (gyroscope) 量測角速度。它的優點是不受重力與線性加速度影響、反應極快、短時間內非常準確。但它的輸出是角速度，要得到角度必須積分——而任何微小的零點偏差積分之後都會變成隨時間線性增長的角度誤差，這就是**漂移**。一顆消費級 MEMS 陀螺儀靜置不動，數分鐘後累積的角度誤差可能就達到數度。

地磁計 (magnetometer) 量測地球磁場方向，提供絕對的方位角（航向）。它不會漂移，但極易受干擾——馬達、鐵製結構、電流導線、甚至是建築物的鋼筋，都會使讀值嚴重偏移。在室內環境中，地磁計往往不可靠。

6.3.2 互補：為什麼必須融合

這三者的特性恰好互補，這是感測器融合最經典的教科書案例。

加速度計：長期正確（重力方向永遠不變），短期雜訊大（受運動與振動干擾）。陀螺儀：短期精確（反應快、平滑），長期漂移（積分累積誤差）。因此最簡單的融合方式是**互補濾波**：用高通濾波取陀螺儀的短期變化、用低通濾波取加速度計的長期趨勢，兩者加權相加：

$$\theta = \alpha \cdot (\theta_{\text{prev}} + \omega_{\text{gyro}} \cdot \Delta t) + (1 - \alpha) \cdot \theta_{\text{accel}}$$

互補濾波。 α 通常取 $0.95 \sim 0.99$ ， Δt 為取樣週期。 α 越大越信任陀螺儀（平滑但可能漂移），越小越信任加速度計（不漂移但抖動）。

這條式子只有一行，卻是自平衡車、四軸飛行器與雙足機器人姿態估測的基礎。第 24 章的 Kalman 濾波器可視為它的最佳化版本——用機率理論自動決定最佳的權重 α ，而非人工調參。

值得注意的是：加速度計與陀螺儀能提供俯仰角 (pitch) 與滾轉角 (roll)，因為重力提供了絕對參考；但**偏航角 (yaw)** 沒有任何絕對參考——重力在水平旋轉時不變。因此偏航角只能

靠陀螺儀積分（會漂移），或加上地磁計、GNSS、視覺特徵來校正。這是所有六軸 IMU 的先天限制，也是為什麼室內機器人的方向估測特別困難。

6.4 力、力矩與觸覺感測

6.4.1 應變片與力矩感測器

力覺感測的主流原理是應變規（strain gauge）：金屬箔在受力變形時電阻會改變，把四個應變片接成惠斯登電橋，就能把微小的電阻變化轉成可量測的電壓。這個訊號極其微弱（毫伏等級），必須經過高增益放大與良好的雜訊屏蔽——這是第 3 章「訊號調理」環節最典型的應用場景。

六軸力 / 力矩感測器安裝在機器手臂的末端法蘭與夾具之間，可同時量測三個方向的力與三個方向的力矩。它是精密裝配（插銷入孔）、研磨拋光、以及碰撞偵測的關鍵元件。缺點是價格昂貴（往往數萬到數十萬元）、且會降低系統剛性（因為它本身必須有一點變形才能量測）。

6.4.2 不用感測器也能感覺力

有一個省錢又優雅的替代方案：用馬達電流估算力矩。回顧第 5 章的關係式 $\tau = K_t \cdot I$ ——電流與扭力成正比，因此只要量測馬達電流，就能推估關節的輸出力矩。若已知機器人的動力學模型（第 14 章），把重力與慣性所需的力矩扣掉，剩下的就是外力造成的力矩。

這個方法的優點是零額外成本（電流本來就要量測以做控制）、且不會降低剛性。缺點是精度較差，因為摩擦力（尤其是減速機的摩擦）會混在其中，且高減速比會讓外力的訊號被稀釋——這就回到第 5 章的反向驅動性：減速比越高，越感覺不到外力。這正是協作機器人採用中低減速比並搭配關節力矩感測器的原因。

6.4.3 觸覺感測

觸覺感測器提供接觸的分布資訊，而非單點的力。常見形式包括：電阻式壓力薄膜（便宜、可做成陣列）、電容式（靈敏度高、可偵測接近）、以及近年的視覺觸覺感測器——在透明彈性體內裝一顆相機，透過觀察表面變形的影像來推算接觸力的分布與物體表面紋理。這類感測器能提供極豐富的資訊（包括滑動偵測），是第 21 章靈巧抓取研究的熱門方向。

最簡單也最實用的觸覺感測，其實是微動開關與碰撞板——掃地機器人前方的保險桿就是如此。它只回答「有沒有撞到」，卻足以支撐整套避障行為，成本不到十元。這提醒我們：感測

器的選擇取決於任務需求，不是越精密越好。

6.5 超音波與紅外線測距

6.5.1 超音波：回音定位

超音波感測器發射一段約 40 kHz 的聲波脈衝，測量回波返回的時間，再由聲速換算距離：

$$d = (v_{\text{sound}} \times t) / 2$$

d 為距離，t 為往返時間， $v_{\text{sound}} \approx 343 \text{ m/s}$ (20°C)。除以 2 是因為聲波走了來回兩趟。

優點是便宜（數十元）、不受光線影響、對透明物體（玻璃）有效——這是它相對於光學感測器的獨特優勢。缺點有四個，都源自聲波的物理特性。

第一，波束角寬（約 15~30 度）：它回報的是「這個錐形範圍內最近的東西」，無法知道確切方位，也無法分辨兩個相近的物體。第二，怕斜面：當目標表面與波束夾角過大時，聲波會被反射到別的方向而收不到回波，機器人會以為前方無障礙——這是最危險的失效模式。第三，怕吸音材質：窗簾、海綿、毛毯會吸收聲波。第四，更新率低：聲速慢，量測三公尺需要約 17.5 毫秒，且必須等回音散去才能發下一次，因此更新率通常低於 40 Hz。

此外，溫度會影響聲速（約每升高 1°C 增加 0.6 m/s），高精度應用需做溫度補償。

6.5.2 紅外線測距：三角測距與 ToF

紅外線測距有兩種截然不同的原理，常被混淆。

三角測距式（如 Sharp GP2Y 系列）：發射器與接收器相隔一段距離，物體越近，反射光落在感光陣列上的位置越偏。它的輸出是非線性的類比電壓，需要查表或曲線擬合來換算距離。特性是反應快、成本低，但量測範圍有限（通常 10~150 公分）、且受物體顏色與材質影響大——黑色物體反射弱，會被誤判為較遠。

飛行時間式（ToF，如 VL53L0X）：直接量測紅外光往返的時間（皮秒等級，靠特殊電路實現）。優點是量測結果與物體反射率關係較小、精度高（毫米級）、波束窄（可精確指向）、更新率高。缺點是強烈的環境光（尤其是陽光）會造成干擾、對透明與鏡面物體無效。近年 ToF 模組價格大幅下降，已逐漸取代三角測距式成為小型機器人的主流。

紅外線的共同弱點是：對黑色物體與玻璃無效。這正是掃地機器人常常撞到黑色沙發腳的原

因——實務上必須搭配碰撞板作為最後防線，這就是感測器互補的思想。

表 6-3 四種測距感測器的比較

感測器	原理	範圍	更新率	強項	失效情境	成本
超音波	聲波 ToF	2 cm ~ 4 m	低 (<40 Hz)	便宜、對玻璃有效	斜面、吸音材質、波束寬	極低
紅外線三角測距	光學三角	10 ~ 150 cm	高 (>100 Hz)	便宜、反應快	受顏色影響大、非線性	低
紅外線 ToF	光飛行時間	5 cm ~ 4 m	高 (>50 Hz)	精度高、波束窄	強光、透明與鏡面物體	低 ~ 中
2D 光達	雷射掃描 ToF	10 cm ~ 30 m	中 (5 ~ 20 Hz)	全周掃描、精度高、範圍大	玻璃、鏡面、強光、雨霧	中 ~ 高

6.6 光達與雷達

6.6.1 2D 光達：移動機器人的標配

光達 (LiDAR, Light Detection and Ranging) 以雷射測距搭配旋轉機構，在一個平面上掃描出數百到數千個距離點。一台典型的 2D 光達每秒旋轉 5 ~ 20 圈，每圈輸出 360 個以上的距離讀值，形成一圈環繞機器人的「距離輪廓」。

這正是掃地機器人與 AMR 能建圖與定位的關鍵。因為它一次提供整個平面的幾何資訊，可以做**掃描匹配**——把這一幀的輪廓與地圖比對，推算出機器人現在的位置。這是第 25 章 SLAM 的核心資料來源。

2D 光達的量測原理有兩種。飛行時間式 (ToF) 直接量測雷射脈衝往返時間，範圍大 (可達 30 公尺以上)、精度穩定，但成本較高。三角測距式 (如常見的低價家用型號) 則以相機觀測雷射光點的偏移，成本極低 (千元等級)，但範圍較短 (約 6 ~ 12 公尺) 且遠距精度下降明顯。

6.6.2 3D 光達與失效情境

3D 光達在垂直方向增加多線 (16、32、64、128 線) 或使用旋轉鏡面，可掃出完整的三維點雲。它是自動駕駛車與戶外機器人的主力感測器，但成本高、資料量大 (每秒數百萬點)，對運算能力要求高。近年的固態光達 (solid-state LiDAR) 以無機械旋轉件的設計降低成本並提升可靠度，正快速普及。

光達的失效情境必須牢記，因為它們在實驗室裡不常出現，卻在真實環境中致命。第一，**玻璃與鏡面**：雷射直接穿透玻璃或被鏡面反射到別處，光達會以為那裡是空的——這是服務型機器人撞上玻璃門的主因。第二，**強烈陽光**：戶外環境的紅外線背景會淹沒回波訊號。第三，**雨霧粉塵**：懸浮微粒會產生假回波，在點雲中形成不存在的障礙物。第四，**吸光材質**：黑色霧面物體反射率極低，可能量測不到。

實務上的因應方式是加裝互補感測器：用超音波偵測玻璃（聲波會被玻璃反射）、用碰撞板作為最後防線、用相機辨識材質。這再次呼應本章的核心思想——沒有任何單一感測器是完備的。

6.6.3 毫米波雷達

毫米波雷達發射無線電波（常見 24 GHz 或 77 GHz），量測回波的時間與頻率偏移。相較光達，它的最大優勢是**全天候**：雨、霧、雪、粉塵幾乎不影響它，且波長長，能穿透部分非金屬遮蔽。此外，它能直接利用都卜勒效應量測目標的**徑向速度**——這是光達做不到的，對偵測移動中的車輛與行人極為有用。

缺點是角解析度差（無法精細描繪物體輪廓）、對靜止的小物體不敏感、且金屬物體會產生強烈的多重反射（例如路邊的鐵欄杆可能被誤判為移動物體）。汽車產業的主流做法是雷達與相機融合：雷達提供可靠的距離與速度，相機提供分類與輪廓。

6.7 相機、深度相機與事件相機

6.7.1 相機：資訊最豐富的感測器

相機是所有感測器中資訊密度最高的：一張 640×480 的影像包含三十萬個像素，每個像素還有 RGB 三個通道。它能提供顏色、紋理、形狀、文字等其他感測器完全無法取得的資訊——只有相機能告訴你「那是一個紅色的可樂罐」而不只是「前方 50 公分有東西」。

代價是這些資訊**需要大量運算才能解讀**。光達給你的直接就是距離（可以立即使用），相機給你的是一堆數字，必須經過影像處理或深度學習才能變成有意義的資訊。這是第 22 章的主題。

相機的關鍵規格包括：解析度、幀率（fps）、視野角（FOV）、快門類型與動態範圍。其中兩個容易被初學者忽略但極為重要：

第一，**快門類型**。捲簾快門（rolling shutter）逐行曝光，成本低但在快速運動時會產生果

凍效應（直線變成斜線）；全域快門（global shutter）所有像素同時曝光，適合運動中的機器人與高速視覺，但價格較高。裝在移動機器人上的相機，強烈建議使用全域快門。

第二，**動態範圍**。機器人從室內走到窗邊時，畫面會瞬間過曝或過暗。人眼的動態範圍遠優於一般相機，這是視覺機器人在真實環境中最常遇到的挫折之一。

6.7.2 三種取得深度的方式

單一相機只能得到二維投影，深度資訊在成像過程中遺失了。取得深度有三種主流方法。

立體視覺（Stereo）：用兩顆已知間距（基線 B ）的相機，比對同一物體在兩張影像中的位置差（視差 d ），由三角關係算出深度：

$$Z = (f \times B) / d$$

Z 為深度， f 為焦距（像素）， B 為基線長度， d 為視差（像素）。深度誤差與距離平方成正比，故遠距離精度急遽下降。

優點是被動式（不發光、不干擾、不耗電、可在戶外用）。缺點是在**無紋理表面**（白牆、單色地板）上無法找到對應點，深度會出現大片空洞。

結構光（Structured Light）：主動投射已知的紅外線圖案（點陣或條紋），由圖案的變形推算深度。早期的 Kinect 即用此法。優點是在無紋理表面上也有效、近距離精度極高。缺點是強烈陽光會蓋過投射圖案，因此**僅適合室內**。

ToF 深度相機：對整個畫面同時量測光的飛行時間。優點是速度快、不受紋理影響、運算量低。缺點是解析度通常較低、多重反射會造成誤差（牆角處常見）、且多台設備間會互相干擾。

表 6-4 三種深度相機技術的比較

技術	原理	室內 / 室外	無紋理表面	精度特性	代表產品
立體視覺	雙目視差	皆可（被動式）	失效（無對應點）	誤差隨距離平方增加	RealSense D 系列、ZED
結構光	投射紅外圖案	僅室內	有效	近距離極準（毫米級）	Kinect v1、Astra
ToF	光飛行時間	室內為主	有效	全域一致，牆角有多重反射誤差	Kinect v2、Azure Kinect

6.7.3 事件相機：仿生的新選擇

事件相機 (event camera) 是一種仿生感測器，其原理與傳統相機截然不同。它不輸出完整的畫面幀，而是每個像素獨立監測亮度變化，只在變化超過門檻時輸出一個「事件」(包含像素座標、時間戳與變化極性)。

這帶來三個驚人的特性：極高的時間解析度 (微秒等級，遠超過每秒 30 幀的傳統相機)、極高的動態範圍 (可同時看清明亮的窗外與陰暗的室內)、以及極低的資料量與功耗 (靜止畫面幾乎不產生資料)。它非常適合高速運動的機器人 (如高速無人機避障) 與極端光照環境。缺點是需要全新的演算法 (傳統的影像處理方法完全不能用)、且無法直接取得靜態場景的外觀。目前仍以研究應用為主。

6.8 GNSS、地磁與無線電定位

6.8.1 GNSS 的原理與限制

全球衛星導航系統 (GNSS，包含美國 GPS、歐洲 Galileo、中國北斗、俄羅斯 GLONASS) 以三角定位原理運作：接收器量測來自多顆衛星的訊號傳播時間，由此推算距離，再解出自己的三維座標。理論上三顆衛星即可定位，但因接收器時鐘不夠精確，實務上至少需要四顆來同時解出時間誤差。

一般消費級 GNSS 的水平精度約 2~5 公尺，這對汽車導航足夠，但對機器人 (需要公分級精度來避開路緣) 遠遠不夠。提升精度的方法有兩種：

RTK (Real-Time Kinematic)：在已知精確位置的基準站與移動站之間比對載波相位，可將精度提升到公分級 (1~3 公分)。這是農業無人機自動導航與測量機器人的標準配備，也是第 27 章農用無人機案例的關鍵技術。缺點是需要基準站或訂閱網路 RTK 服務。

差分 GNSS (DGPS)：以基準站廣播誤差修正量，精度約 0.5~2 公尺，介於兩者之間。

GNSS 的根本限制是：**室內完全失效**、都市峽谷 (高樓之間) 因多重路徑反射而嚴重失準、樹冠下訊號衰減、且更新率低 (通常 1~10 Hz)。因此戶外機器人必定要把 GNSS 與 IMU、輪速計融合 (第 24 章)，才能在訊號短暫遺失時維持定位。

6.8.2 室內定位技術

室內無法使用 GNSS，替代方案包括：UWB (超寬頻) 標籤定位，以飛行時間測距，精度

可達 10~30 公分，但需佈建錨點；WiFi 或藍牙訊號強度指紋比對，精度約 2~5 公尺，成本低但不穩定；視覺標記（如 AprilTag、ArUco），在環境中貼上已知圖案，相機看到即可精確定位，成本極低且精度高，是實驗室與倉儲最實用的方案；以及最通用的——SLAM，用光達或相機自行建立地圖並定位，不需任何預先佈建（第 25 章）。

6.9 感測器的規格指標

6.9.1 五個必須分清的指標

讀感測器規格書時，有五個指標常被混淆，但它們的工程意義完全不同。用射箭來比喻最容易理解：

解析度 (Resolution)：能分辨的最小變化量。例如 12 位元 ADC 在 0~5 V 範圍下的解析度是 $5/4096 \approx 1.22 \text{ mV}$ 。解析度高不代表準——它只決定「刻度多細」。

精度 / 重複性 (Precision / Repeatability)：重複量測同一目標時，結果的一致程度。射箭時箭都集中在同一小區域，就是精度高（即使那個區域偏離靶心）。工業機器人型錄上漂亮的「重複精度 $\pm 0.02 \text{ mm}$ 」指的就是這個。

準確度 (Accuracy)：量測值接近真值的程度。箭落在靶心附近就是準確度高。準確度差但精度好的系統，可以透過**校正**來修正；但精度差（隨機分散）的系統無法靠校正拯救。

線性度 (Linearity)：輸出與輸入是否成正比。非線性的感測器（如紅外線三角測距）需要查表或多項式擬合來換算。

更新率與延遲 (Update Rate / Latency)：兩者不同！一顆相機可能每秒輸出 30 幀（更新率 30 Hz），但從曝光到資料抵達控制器可能耗時 100 毫秒（延遲）。在控制迴路中，延遲比更新率更致命——延遲會直接侵蝕系統的穩定裕度，這是第 16 章與第 19 章會嚴肅處理的問題。

圖 6-2 精度與準確度的靶心圖

建議配圖：四個靶心圖並排。(a) 高精度高準確度：箭群密集且在靶心；(b) 高精度低準確度：箭群密集但偏離靶心（可靠校正修正）；(c) 低精度高準確度：箭散布但平均在靶心；(d) 低精度低準確度：箭散布且偏離。下方註明：偏差可校正，隨機雜訊只能靠濾波與多次平均。

6.9.2 誤差的兩種類型

感測器誤差分為兩類，處理方式完全不同。**系統誤差 (bias)** 是固定或緩慢變化的偏移，例

如加速度計的零點偏移、編碼器的安裝偏心。它可以透過校正消除或補償——這是好消息。

隨機誤差 (noise) 則是每次量測都不同的隨機擾動，來自熱雜訊、電磁干擾、量化誤差。它無法被校正消除，只能透過濾波或多次平均來降低。統計上，取 N 次獨立量測的平均，可將隨機誤差的標準差降低為原來的 $1/\sqrt{N}$ ：

$$\sigma_{\text{mean}} = \sigma / \sqrt{N}$$

取 N 次平均可降低隨機誤差，但代價是延遲增加 N 倍。這是精度與即時性的根本權衡。

注意這個公式的殘酷之處：要把雜訊降到十分之一，需要一百次量測。對更新率 100 Hz 的感測器而言，這意味著一秒的延遲——在控制迴路中完全不可接受。這正是為什麼工程上需要更聰明的方法 (Kalman 濾波器)，它能在不犧牲即時性的前提下降低雜訊。

6.10 雜訊與濾波

6.10.1 雜訊從哪裡來

感測器讀值的雜訊有四個主要來源。**物理雜訊**：熱雜訊 (電子的隨機運動，溫度越高越嚴重)、散粒雜訊 (光子或電子到達的隨機性)，這是物理定律決定的下限，無法消除。**電子雜訊**：放大器本身的雜訊、電源的漣波、ADC 的量化誤差。**電磁干擾**：馬達的換相尖峰、PWM 的開關雜訊、無線通訊，這是機器人上最主要的雜訊來源，也是第 3 章強調「訊號線與功率線分開走」的原因。**機械振動**：馬達與減速機的振動會直接汙染 IMU 與力覺感測器的讀值。

重要的工程順序是：**先降低雜訊來源，再考慮濾波**。加裝隔離、改善接地、把訊號線遠離馬達線、在感測器旁加去耦電容，這些硬體措施的效果往往勝過任何軟體濾波，而且不會引入延遲。用軟體濾波去補救糟糕的佈線，是本末倒置。

6.10.2 三種基本濾波器

移動平均濾波 (Moving Average)：取最近 N 筆的平均。最簡單、雜訊抑制效果好，但延遲明顯 (約 $N/2$ 個取樣週期)，且對脈衝雜訊 (單一極端值) 抵抗力弱。

低通濾波 (一階 IIR)：以指數加權的方式融合新舊值：

$$y[k] = \alpha \cdot y[k-1] + (1 - \alpha) \cdot x[k]$$

α 為濾波係數 ($0 \sim 1$)。 α 越大越平滑但延遲越大。只需儲存一個變數，極省記憶體，是微控制器上最常用的濾波器。

中值濾波 (Median Filter)：取最近 N 筆的中位數。它的獨門優勢是能完全消除脈衝雜訊——當超音波偶爾回報一個荒謬的 0 或 400 公分時，中值濾波可以直接把它濾掉，而移動平均會被它拉偏。對測距感測器特別有效。

所有濾波器都遵守同一條鐵律：**平滑度與即時性不可兼得**。濾得越乾淨，延遲就越大；而延遲會侵蝕控制系統的穩定性。工程上的做法是找到剛好足夠的濾波強度，而不是追求最平滑的曲線。

表 6-5 三種基本濾波器的比較

濾波器	原理	記憶體需求	延遲	對脈衝雜訊	適用場合
移動平均	最近 N 筆平均	N 筆	約 $N/2$ 週期	抵抗力弱	一般類比訊號、電池電壓
一階低通 (IIR)	指數加權遞迴	1 筆	由 α 決定	抵抗力弱	微控制器、IMU、電流
中值濾波	最近 N 筆中位數	N 筆	約 $N/2$ 週期	完全消除	超音波、紅外線測距

6.11 校正

6.11.1 為什麼一定要校正

每一顆感測器出廠時都有個體差異：同一型號的兩顆加速度計，靜置時的讀值可能差好幾個百分點。校正就是用已知的參考標準，找出感測器輸出與真值之間的關係，並在程式中補償。

最基本的校正模型是線性的兩參數模型：

$$\text{真值} = \text{尺度係數} \times (\text{原始讀值} - \text{零點偏移})$$

尺度係數 (scale factor) 修正靈敏度誤差，零點偏移 (bias/offset) 修正零點漂移。

6.11.2 三種常見的校正程序

零點校正 (最簡單)：在已知為零的條件下量測，記錄下讀值作為偏移量。例如：把機器人靜置水平，記錄陀螺儀的三軸讀值（應為零，實際不為零），此值即為零點偏移，之後每次讀值都減掉它。這是 IMU 開機時必做的程序，而且每次開機都要重做，因為零點會隨溫度變化。

兩點校正：在兩個已知值下量測，解出尺度與偏移。例如力覺感測器：先空載 (0 kg) 記錄

一次，再掛上已知的 1 kg 砝碼記錄一次，兩點即可決定一條直線。

多點與非線性校正：對非線性感測器（如紅外線三角測距），需要在多個已知距離下量測，建立查表或擬合曲線（常用多項式或指數函數）。

更複雜的校正還包括：相機的內外參數校正（用棋盤格標定板，第 22 章）、IMU 的六面校正（將 IMU 依六個方向靜置，同時解出三軸的尺度與偏移）、以及機器人的手眼校正（求相機座標與機器人座標的關係，第 22 章）。

◎ 校正的黃金原則

一、校正必須在實際使用條件下進行（溫度、電壓、安裝方式都會影響）。二、校正參數要存起來並記錄日期——感測器會隨時間老化。三、要有驗證步驟：用第三個已知點檢查校正結果是否正確，不要只用校正用的那兩點自我驗證。四、若校正後誤差仍大，問題可能在安裝（歪斜、鬆動）而非感測器本身。

6.12 感測器配置策略

6.12.1 互補而非重複

為機器人選配感測器時，核心原則是**互補**：讓每個感測器的強項覆蓋其他感測器的弱項，而不是買一堆相同特性的感測器。

經典的互補組合有：光達（精確但怕玻璃）+ 超音波（不準但能偵測玻璃）+ 碰撞板（最後防線）；輪速里程計（平滑但會漂移）+ IMU（快速但漂移更快）+ 光達掃描匹配（不漂移但慢）；相機（能辨識類別但不知距離）+ 光達或雷達（知距離但不知類別）。

這正是自動駕駛車不惜成本裝上相機、光達與雷達三種感測器的原因——不是為了保險，而是因為它們的失效情境彼此不重疊：雨霧中光達失效但雷達有效、辨識號誌時雷達無用但相機有效。

6.12.2 由任務反推感測器

選配感測器的正確順序是由任務需求反推，而非由可用的零件出發。步驟為：一、列出機器人必須「知道」的事情（我在哪？前方有什麼？抓到東西了嗎？）；二、對每一項，列出需要的量測範圍、精度與更新率；三、找出能滿足的感測器候選；四、檢查各候選的失效情境，補上互補的感測器；五、確認運算資源與介面頻寬足夠（一台 3D 光達每秒數百萬點，微控制器無法處

理)。

表 6-6 常見任務的感測器配置建議

任務	必須知道	主要感測器	互補 / 備援	本書章節
關節位置控制	每個關節的角度	編碼器	限位開關 (歸原點)	第 16 章
移動機器人定位	自己在地圖上的位置	2D 光達 + 輪速計	IMU、視覺標記	第 25 章
室內避障	前方是否有障礙	光達或深度相機	超音波 (玻璃)、碰撞板	第 26 章
物件辨識與夾取	物體類別與 6D 位姿	RGB-D 相機	力覺感測 (接觸確認)	第 21 ~ 22 章
雙足 / 四足平衡	身體姿態與足端接觸	IMU + 關節編碼器	足底壓力感測	第 28 章
戶外導航	全域位置與航向	RTK-GNSS + IMU	輪速計、視覺里程計	第 27 章
人機協作安全	是否碰到人	關節力矩感測或電流	安全光柵、電容式接近感測	第 20、35 章

完整計算例題

例題 6-1 編碼器解析度與速度量測

題目：某差速輪機器人使用 500 PPR 增量式編碼器 (正交輸出)，直接裝在馬達軸上，馬達經減速比 30 的減速機驅動直徑 65 mm 的輪子。(a) 求輪子每轉一圈對應的編碼器計數。(b) 求機器人的位移解析度。(c) 若控制週期為 10 ms，機器人以 0.3 m/s 行進，每週期可計得幾個脈波？速度量測的量化誤差為何？

解 (a)：正交四倍頻後，馬達每轉為 $500 \times 4 = 2000$ 計數。經減速比 30，輪子轉一圈馬達需轉 30 圈：

$$\text{每輪轉計數} = 2000 \times 30 = 60000 \text{ counts/rev}$$

輪端的等效解析度。

解 (b)：輪子周長 = $\pi \times 0.065 \approx 0.2042 \text{ m}$ 。

$$\text{位移解析度} = 0.2042 / 60000 \approx 3.40 \times 10^{-6} \text{ m} \approx 3.4 \mu\text{m}$$

每一個計數對應約 3.4 微米的位移，解析度極高。

解 (c)：10 ms 內位移 = $0.3 \times 0.01 = 0.003 \text{ m} = 3 \text{ mm}$ 。計數 = $0.003 / 3.4 \times 10^{-6} \approx 882$

個脈波。量化誤差為 ± 1 個計數，對應速度誤差：

$$\Delta v = 3.4 \times 10^{-6} / 0.01 \approx 3.4 \times 10^{-4} \text{ m/s} = 0.34 \text{ mm/s}$$

相對誤差僅 $0.34/300 \approx 0.11\%$ ，速度量測品質良好。

工程對照：回顧第 3 章例題 3-2，該例的編碼器直接裝在輪軸上且線數低，量化誤差高達 31%。本例因為編碼器裝在減速機之前的馬達軸上，等效解析度被減速比放大了 30 倍——這是實務上編碼器一律安裝在馬達端的關鍵原因之一。

例題 6-2 超音波的失效分析

題目：某機器人以超音波感測器（波束角 30 度、量測範圍 2~400 cm）進行避障。（a）在距離 100 cm 處，波束覆蓋的寬度為何？（b）若前方有一面與波束軸夾角 40 度的斜牆，是否能偵測到？（c）機器人以 0.5 m/s 前進，感測器更新率 20 Hz，求兩次量測之間的移動距離，並評估避障是否安全。

解（a）：波束為圓錐，半角 15 度。在距離 $d = 100 \text{ cm}$ 處的覆蓋半徑：

$$r = d \times \tan(15^\circ) = 100 \times 0.268 \approx 26.8 \text{ cm} \rightarrow \text{覆蓋寬度約 } 53.6 \text{ cm}$$

這表示感測器無法分辨物體在此 53.6 cm 寬範圍內的確切方位。

解（b）：聲波遵守反射定律，入射角等於反射角。當牆面法線與波束軸夾角為 40 度時，反射波會偏離 80 度射出，遠超過感測器的接收範圍（約 ± 15 度）。因此無法收到回波，感測器會回報「無障礙」或超時值。一般而言，超音波在入射角超過約 20~30 度時即失效。這是超音波最危險的失效模式：它不會報錯，而是安靜地告訴你前方是空的。

解（c）：更新週期 $= 1/20 = 0.05 \text{ s}$ ，兩次量測間移動 $0.5 \times 0.05 = 0.025 \text{ m} = 2.5 \text{ cm}$ 。加上煞車距離與處理延遲，若系統總延遲為 150 ms，則反應期間已前進 7.5 cm。結論：對於低速機器人尚可接受，但必須設定足夠的安全距離（建議至少 30 cm 以上），且必須加裝碰撞板作為最後防線，以因應（b）的斜面失效情境。

例題 6-3 IMU 漂移與互補濾波

題目：某 MEMS 陀螺儀的零點偏移為 $0.05^\circ/\text{s}$ （已扣除靜態校正後的殘餘）。（a）若僅靠陀螺儀積分求角度，10 秒與 60 秒後的角度誤差各為何？（b）若加速度計的角度讀值雜訊標準差為 2 度，採互補濾波 $\alpha = 0.98$ 、取樣週期 10 ms，求濾波器的等效時間常數。（c）說明此設

計的物理意義。

解 (a)：積分後誤差隨時間線性增長：

$$\text{誤差} = 0.05^\circ/\text{s} \times t \rightarrow 10 \text{ s}: 0.5^\circ; 60 \text{ s}: 3.0^\circ$$

陀螺儀單獨使用時，誤差無界增長，長時間必然失效。

解 (b)：一階互補濾波的等效時間常數為：

$$\tau = (\alpha \times \Delta t) / (1 - \alpha) = (0.98 \times 0.01) / 0.02 = 0.49 \text{ s}$$

約 0.5 秒。

解 (c)：這代表濾波器在約 0.5 秒的時間尺度上做切換——短於 0.5 秒的快速變化交給陀螺儀（平滑、無雜訊），長於 0.5 秒的緩慢趨勢由加速度計主導（提供絕對重力參考、防止漂移）。因此陀螺儀的漂移只會累積約 0.5 秒（誤差僅 $0.05 \times 0.5 = 0.025$ 度），而加速度計的 2 度雜訊則被平均掉大部分。兩者的缺點都被有效抑制——這正是互補濾波的精髓，也是第 24 章 Kalman 濾波器的直覺基礎。

程式範例：濾波器比較與 IMU 姿態估測

以下 Python 程式模擬帶雜訊與脈衝干擾的測距訊號，比較三種濾波器的效果；接著實作互補濾波，示範如何由加速度計與陀螺儀融合出穩定的傾角。

```
import random
import math

# ----- 第一部分：三種濾波器比較 -----
class MovingAverage:
    def __init__(self, n=5):
        self.n, self.buf = n, []
    def update(self, x):
        self.buf.append(x)
        if len(self.buf) > self.n: self.buf.pop(0)
        return sum(self.buf) / len(self.buf)

class LowPass:
    def __init__(self, alpha=0.8):
        self.a, self.y = alpha, None
    def update(self, x):
        self.y = x if self.y is None else self.a*self.y + (1-self.a)*x
        return self.y

class Median:
    def __init__(self, n=5):
        self.n, self.buf = n, []
    def update(self, x):
```

```

        self.buf.append(x)
        if len(self.buf) > self.n: self.buf.pop(0)
        return sorted(self.buf)[len(self.buf)//2]

true_dist = 100.0                # 真實距離 100 cm
ma, lp, md = MovingAverage(5), LowPass(0.8), Median(5)

print("步  原始讀值  移動平均  低通  中值")
for k in range(12):
    noise = random.gauss(0, 2.0)          # 一般雜訊
    spike = 300.0 if k == 6 else 0.0      # 第 6 步注入脈衝干擾
    raw = true_dist + noise + spike
    print(f"{k:2d}  {raw:8.1f}  {ma.update(raw):8.1f}"
          f"  {lp.update(raw):7.1f}  {md.update(raw):6.1f}")
print("> 注意第 6 步的脈衝：中值濾波完全免疫，另兩者被拉偏\n")

# ----- 第二部分：互補濾波姿態估測 -----
def complementary_filter(gyro_rate, accel_angle, dt=0.01, alpha=0.98):
    """回傳一個可持續呼叫的濾波器"""
    state = {"angle": accel_angle[0]}
    out = []
    for w, a in zip(gyro_rate, accel_angle):
        # 陀螺儀積分 (短期精確) + 加速度計絕對參考 (長期不漂移)
        state["angle"] = alpha * (state["angle"] + w * dt) + (1-alpha) * a
        out.append(state["angle"])
    return out

# 模擬：真實傾角固定 10 度；陀螺儀有 0.05 deg/s 偏移；加速度計雜訊大
N, dt = 500, 0.01
true_angle = 10.0
gyro = [0.05 for _ in range(N)]          # 只有偏移，無真實旋轉
accel = [true_angle + random.gauss(0, 2.0) for _ in range(N)]
gyro_only = []
ang = true_angle
for w in gyro:
    ang += w * dt
    gyro_only.append(ang)
fused = complementary_filter(gyro, accel, dt, 0.98)

print(f"{'時間':>6} {'僅陀螺儀':>10} {'僅加速度計':>12} {'互補濾波':>10}")
for i in [0, 100, 250, 499]:
    print(f"{i*dt:5.1f}s {gyro_only[i]:10.2f} {accel[i]:12.2f} {fused[i]:10.2f}")
print(f"\n 真實角度 = {true_angle}°")
print("> 陀螺儀持續漂移、加速度計劇烈跳動、融合結果穩定且不漂移")

```

請試著修改：(一) 把 α 從 0.98 改成 0.5，觀察融合結果是否變得抖動；改成 0.999 呢？

(二) 把陀螺儀偏移加大到 0.5 °/s，看互補濾波是否仍能壓制漂移。這兩個實驗會讓你直觀理解 α 的物理意義。

實作：感測器特性實測與校正

本章的實作是實測一顆測距感測器的真實特性，並完成校正。所需器材：超音波模組（HC-SR04）或紅外線 ToF 模組、微控制器（Arduino 或 ESP32）、捲尺、以及不同材質的目標板（白紙板、黑色絨布、玻璃、金屬板、海綿）。

1. **線性度測試**：以捲尺量出 10、20、40、60、80、100、150、200 cm 的位置，在每個位置以白紙板為目標各量測 20 次，記錄平均值與標準差。繪製「實測值對真實距離」圖，觀察是否為直線。
2. **校正**：以線性回歸求出尺度係數與零點偏移，寫入程式進行補償。再取三個未用於校正的距離（如 35、75、130 cm）驗證校正效果。
3. **材質測試**：固定在 50 cm 處，分別以五種材質為目標各量測 20 次。記錄每種材質的平均值、標準差與失效次數（回報超時或荒謬值）。
4. **角度測試**：以白紙板置於 50 cm 處，將板面由垂直逐步旋轉 0° 、 10° 、 20° 、 30° 、 40° 、 50° ，記錄每個角度的成功率。找出感測器失效的臨界角度，並與例題 6-2 的理論分析比較。
5. **雜訊與濾波**：在固定距離下連續取樣 200 筆，計算標準差；分別套用移動平均、低通與中值濾波，比較三者的標準差降低程度與響應延遲（可用突然移開目標板的方式量測延遲）。
6. **綜合報告**：整理出這顆感測器的「使用說明書」——可靠工作範圍、對哪些材質失效、臨界角度、建議的濾波設定與安全裕度。

這份報告的價值在於：你將擁有對這顆感測器的第一手認識，而不是型錄上的行銷數字。往後任何專題用到它，你都知道它會在這裡背叛你。

國中小也看得懂：機器人的眼睛會騙人

閉上眼睛，用手摸摸自己的鼻子——很簡單對不對？可是閉著眼睛要走到教室門口，就很難了。這是因為你身上有兩種「感覺」：一種是知道自己身體的樣子（手彎了多少、腳站得穩不穩），另一種是知道外面有什麼（門在哪裡、前面有沒有桌子）。機器人也一樣，它需要這兩種感覺器官。

機器人用什麼當眼睛呢？有好幾種喔。**超音波**像蝙蝠一樣，發出人聽不到的聲音，等聲音撞到東西彈回來，算一算就知道多遠。但如果牆壁是斜的，聲音會彈到旁邊去，機器人就以為前面

沒東西——這就是為什麼掃地機器人有時候會撞到牆角。**雷射（光達）**用光來量，比較準，可是遇到玻璃就慘了，光直接穿過去，機器人以為那裡是空的，就一頭撞上玻璃門。**相機**看得到顏色和形狀，可以認出「那是一顆球」，但要用很多計算才看得懂。

最重要的一件事：**每個感測器都會有出錯的時候，而且它不會告訴你它錯了**。所以聰明的做法是裝好幾種不一樣的感測器，讓它們互相幫忙。就像掃地機器人，前面除了雷射，還會裝一片會被撞到的板子——萬一雷射沒看到，撞到板子還是會停下來。你在附錄 G 做的超音波避障車，也可以試試看加一個碰撞開關喔！

叫獸提醒與常見錯誤

- 誤區一：以為感測器會告訴你它出錯了。感測器失效時通常安靜地回報一個看似合理的錯誤數字，必須靠合理性檢查（範圍、變化率）主動偵測。
- 誤區二：混淆解析度與精度。解析度是刻度多細，精度是量得多準；4000 線編碼器不代表誤差只有 0.09 度。
- 誤區三：忽略延遲。更新率 30 Hz 不代表延遲只有 33 ms；在控制迴路中，延遲比更新率更致命。
- 誤區四：用軟體濾波補救糟糕的佈線。應先降低雜訊來源（隔離、接地、分離走線），濾波是最後手段且會引入延遲。
- 誤區五：對測距感測器使用移動平均。單一脈衝干擾就會拉偏平均值；測距類應優先使用中值濾波。
- 誤區六：IMU 開機不做零點校正。陀螺儀零點會隨溫度變化，每次開機都應在靜止狀態下重新量測偏移。
- 誤區七：期待六軸 IMU 提供穩定的偏航角。重力無法提供水平旋轉的參考，偏航角必然漂移，需靠地磁計、GNSS 或視覺校正。
- 誤區八：在室內信任地磁計。鋼筋、馬達與電流會嚴重扭曲磁場，室內的地磁讀值往往不可用。
- 誤區九：以為光達萬能。玻璃、鏡面、強光、雨霧粉塵都會使其失效，必須搭配互補感測器與碰撞防線。
- 誤區十：校正一次用到永遠。感測器會老化、零點會隨溫度漂移，應定期重新校正並記錄日

期。

課堂討論

1. 開頭故事中三位學生量出三個不同的距離。若要你設計一個程序來得到「最好的估計」，你會怎麼做？需要知道每個方法的什麼資訊？
2. 掃地機器人為什麼常常撞到黑色的沙發腳與玻璃門？請分別從紅外線與光達的物理原理解釋，並提出三種改善方案。
3. 自動駕駛車同時裝設相機、光達與毫米波雷達，成本高昂。若必須刪掉其中一種以降低成本，你會刪哪一種？在什麼情境下這個決定會出問題？
4. 事件相機的資料形式與傳統相機完全不同（只有變化事件、沒有完整畫面）。這對演算法設計帶來什麼挑戰？你認為它最適合用在哪些機器人任務上？
5. 有人主張「感測器越多越好」。請從成本、運算負擔、故障率、以及資料衝突（兩個感測器給出矛盾答案時怎麼辦）四個角度反駁或支持這個主張。

章末摘要

- 感測器分內部感測（自身狀態，是控制的基礎）與外部感測（環境資訊，是自主的基礎）；外部感測又分主動式與被動式。
- 編碼器是最重要的機器人感測器；正交輸出可判方向並四倍頻，解析度 = $360^\circ / (\text{PPR} \times 4)$ ；增量式需歸原點，絕對式有斷電記憶。
- 編碼器安裝在減速機之前的馬達端，可使等效解析度被減速比放大，是實務標準做法。
- IMU 三元件互補：加速度計長期正確短期雜訊大、陀螺儀短期精確長期漂移、地磁計不漂移但易受干擾。
- 互補濾波以一行公式融合加速度計與陀螺儀，是自平衡與姿態估測的基礎；六軸 IMU 的偏航角必然漂移，無絕對參考。
- 力覺可用應變片直接量測，或用馬達電流間接估算；高減速比會稀釋外力訊號，影響反向驅動與碰撞偵測能力。
- 超音波便宜且對玻璃有效，但波束寬、怕斜面（入射角超過約 $20 \sim 30$ 度即失效）、更新率低。

- 光達精確且提供全周幾何資訊，是 SLAM 的核心；但對玻璃、鏡面、強光與雨霧失效，必須有互補感測與碰撞防線。
- 深度取得有三法：立體視覺（被動、無紋理失效）、結構光（室內、無紋理有效）、ToF（快速、牆角有多重反射誤差）。
- 五個規格指標必須分清：解析度（刻度細）、精度（一致性）、準確度（接近真值）、線性度、更新率與延遲。
- 誤差分系統誤差（可校正）與隨機誤差（只能濾波或平均， σ 隨 $\sqrt{N}$ 下降，代價是延遲）。
- 應先降低雜訊來源再濾波；移動平均與低通易受脈衝干擾，中值濾波對測距感測器最有效。
- 感測器配置的核心原則是互補而非重複，並應由任務需求反推所需的感測能力。

觀念題與計算題

觀念題

1. 區分內部感測與外部感測，說明為何工業手臂可只有前者、移動機器人必須有後者。
2. 說明增量式編碼器的正交輸出如何達成方向判斷與四倍頻。
3. 比較增量式與絕對式編碼器，並說明高階關節模組為何採用雙編碼器。
4. 說明加速度計、陀螺儀與地磁計各自量測什麼、各自的優缺點為何。
5. 為什麼六軸 IMU 無法提供穩定的偏航角？有哪三種補救方式？
6. 說明用馬達電流估算外力的原理、優點與兩項限制。
7. 超音波為什麼怕斜面？請以反射定律說明，並估算失效的臨界角度。
8. 列舉光達的四種失效情境，並各提出一種互補的感測方案。
9. 比較立體視覺、結構光與 ToF 三種深度相機的原理與適用環境。
10. 說明捲簾快門與全域快門的差異，為何移動機器人建議使用後者？
11. 用射箭比喻說明解析度、精度與準確度的差異，並指出哪一種可用校正改善。
12. 為什麼「先降低雜訊來源、再濾波」是正確的工程順序？濾波的代價是什麼？
13. 說明中值濾波相對於移動平均的獨特優勢，以及它最適合的感測器類型。
14. 說明感測器配置的「互補原則」，並以自動駕駛車的三種感測器為例說明。

計算題

- 某機器手臂關節使用 2048 PPR 正交編碼器，裝於馬達端，經減速比 100 的諧波減速機。
(a) 求關節端的角度解析度 (度)。(b) 若手臂長 500 mm，求末端的位置解析度 (mm)。(c) 若改將編碼器裝在關節輸出端 (減速機之後)，解析度變為多少？由此說明馬達端安裝的優勢。
- 某 2D 光達每秒轉 10 圈，每圈輸出 360 個點，量測範圍 12 m。(a) 求角解析度 (度) 與資料率 (點 / 秒)。(b) 在 5 m 距離處，相鄰兩點的間距為多少公分？(c) 若要偵測寬 10 cm 的桌腳，在多遠的距離之外會因角解析度不足而可能漏檢 (至少需兩點打到目標)？
- 某立體視覺相機基線 $B = 6$ cm、焦距 $f = 600$ 像素、視差量測精度 ± 0.5 像素。(a) 求在 1 m、3 m、5 m 處的深度值對應的視差。(b) 求三個距離處的深度誤差 (提示：對 $Z = fb/d$ 微分， $\Delta Z \approx Z^2 \cdot \Delta d / (fB)$)。(c) 由結果說明立體視覺為何不適合遠距離量測。
- 某陀螺儀零點偏移 $0.02^\circ/\text{s}$ ，加速度計角度雜訊標準差 1.5° 。(a) 僅用陀螺儀積分，多久後角度誤差達到 1 度？(b) 採互補濾波 $\alpha = 0.99$ 、 $\Delta t = 5$ ms，求等效時間常數。(c) 在此時間常數內，陀螺儀累積的漂移為多少度？此設計是否合理？

動手做與專題延伸

本章實作：感測器規格調查表

選擇三種不同類型的感測器 (建議各選一種內部感測與兩種外部感測)，查閱其規格書並完成下表。目的是學會在真實文件中辨識關鍵指標，並看出型錄沒有明說的限制。

表 6-7 感測器規格調查表 (第一列為範例)

項目	感測器 A (範例：HC-SR04)	感測器 B	感測器 C
類型與原理	超音波測距 (聲波 ToF)		
量測範圍	2 ~ 400 cm		
解析度	約 0.3 cm		
精度 (規格書標示)	± 3 mm (理想條件)		
更新率 / 延遲	約 20 ~ 40 Hz / 約 25 ms		
工作電壓與電流	5 V / 約 15 mA		
通訊介面	觸發 / 回波脈波 (GPIO)		
已知失效情境	斜面 $> 25^\circ$ 、吸音材質、多機干擾		

項目	感測器 A (範例：HC-SR04)	感測器 B	感測器 C
建議互補感測器	碰撞板、紅外線 ToF		

專題延伸

- 為附錄 G 的超音波避障機器人加裝一顆紅外線 ToF 模組與一片碰撞板，設計一套「三層防線」的避障邏輯：光學遠距預警、超音波中距確認、碰撞板最後停止。實測三種材質（白牆、黑布、玻璃）下的避障成功率，比較加裝前後的差異。
- 使用手機的感測器記錄 App（多數手機可輸出 IMU 原始資料），把手機靜置十分鐘並記錄陀螺儀讀值。計算其零點偏移與雜訊標準差，並模擬積分後的漂移曲線，驗證本章 6.3.2 節的理論。
- 進階：為一台「圖書館還書機器人」設計完整的感測器配置。需求為：能在館內自主導航（有玻璃隔間與落地窗）、能偵測並避開行人、能辨識書架編號、能確認書本已被取走。請依 6.12.2 節的五步驟完成配置設計，列出感測器清單、預估成本、標出每個感測器的失效情境與互補方案，並繪製第 3 章格式的系統方塊圖。

參考文獻與延伸閱讀

- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer（感測與感知專章）。
- Everett, H. R., Sensors for Mobile Robots: Theory and Application, A K Peters（移動機器人感測器的經典專著）。
- Thrun, S., Burgard, W. and Fox, D., Probabilistic Robotics, MIT Press（感測器模型與不確定性的機率處理）。
- Corke, P., Robotics, Vision and Control, 3rd ed., Springer（視覺感測與影像處理）。
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer（立體視覺與深度估測）。
- Groves, P. D., Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, 2nd ed., Artech House（GNSS 與慣性導航融合）。
- Gallego, G. et al., "Event-based Vision: A Survey," IEEE TPAMI, 2022（事件相機的完整回

顧)。

- Yuan, W., Dong, S. and Adelson, E. H., "GelSight: High-Resolution Robot Tactile Sensors," Sensors, 2017 (視覺觸覺感測器)。
- 各感測器廠商技術文件：Bosch Sensortec (IMU)、SICK 與 Hokuyo (光達)、Intel RealSense 與 Stereolabs (深度相機)。
- ROS 2 感測器訊息標準 (sensor_msgs) 與驅動套件文件 (docs.ros.org)。

第 7 章

機器人電子系統與嵌入式控制

編著：李世淵（ 機器人叫獸 ）

助教：李天宇、李宇晴

【章首情境圖建議】比賽現場的雙格對照照片：左格是一台停在賽道上、LED 正在閃爍重開機的循跡車，旁邊放著萬用電表，螢幕顯示 3.8 V；右格是同一台車經過改善後的內部特寫——馬達線與訊號線分開走並各自雙絞、電源入口加了一顆 470 μ F 電容、電表穩定顯示 5.02 V。標題文字：一半的「程式問題」，其實是電在說謊。

叫獸開講 「機器人明明接好了電線，為什麼一動就重新開機？」

比賽前一天晚上十一點，實驗室的燈還亮著。小婕的循跡車在測試賽道上跑得漂亮極了——一直線加速、彎道修正、一圈只要二十八秒。她連跑五圈都完美無缺，決定收工回家。

隔天上午的正式比賽，第一圈剛過第二個彎道，車子突然停住不動，然後 LED 燈閃了一下，像是重新開機。裁判給了第二次機會，同樣的位置、同樣的症狀。第三次，車子在起跑線上就當掉了。

回到實驗室，小婕把程式從頭到尾檢查了三遍，一行問題也沒有。她甚至把程式燒回昨天測試成功的版本——還是會當。「老師，昨天明明可以跑，今天完全一樣的程式就是不行，這根本不合理！」

叫獸接過那台車，沒有看程式，而是先把萬用電表的探棒接到微控制器的電源腳位上，然後說：「你把車子舉起來，讓輪子空轉，看讀數。」電表顯示穩定的 5.02 伏特。「現在把車子放到地上，用手稍微壓住輪子，讓馬達出力。」

電表的數字瞬間跳到 3.8 伏特，然後彈回 4.6，又掉到 4.1——像心電圖一樣劇烈震盪。小婕愣住了。「這顆微控制器的最低工作電壓是 4.5 伏特，」叫獸說，「每次馬達用力，電壓就掉到門檻以下，它就重新開機。你的程式一行錯都沒有，是**電源**在說謊。」

「可是昨天為什麼可以跑？」「因為昨天你的電池是滿的。今天比賽前你只充了一半，內阻上升，同樣的電流造成的壓降就更大。而且昨天賽道是新的，摩擦力小；今天的賽道鋪了一層防滑漆，馬達要出更多力。」叫獸把電池、電線與馬達畫成一個迴路，「這三件事單獨看都不致命，加在一起就跨過了臨界點。」

他接著做了三件事：把馬達電源與控制器電源分開、在微控制器的電源腳旁加了一顆 470 微法的電容、把訊號線從馬達線旁邊移開重新綁紮。整個過程不到二十分鐘。車子再放上賽道，怎麼壓輪子都不再重開機。

「記住，」叫獸把那顆小小的電容放在小婕手心，「在機器人的世界裡，超過一半的『程式問題』其實是電的問題。而它們最狡猾的地方在於——症狀看起來永遠像是別的東西壞了。今天這堂課，我們就來學怎麼讓電乖乖聽話。」

本章為什麼重要

本章是第一篇的收尾。第 4 章給了骨骼、第 5 章給了肌肉、第 6 章給了感官，本章要處理的是把它們全部連起來的神經與血管——電子系統。這是機器人工程中最不浪漫、卻最常決定成敗的一環。

在教學現場，機器人故障原因的統計結果始終一致：電源與佈線問題佔了將近一半，遠超過演算法錯誤。更麻煩的是，這類問題的症狀極具欺騙性——程式當機、感測器讀值亂跳、通訊斷線、馬達失控，看起來全都像軟體問題。學會用電的角度診斷，能省下你未來無數個週末。

本章同時是第一篇通往第二篇的橋樑：讀完本章，你已經理解了一台機器人的完整實體構成；接下來的第 8 章開始，我們將進入數學的世界，學習如何精確描述它的運動。

學習目標

- 能比較微控制器與單板電腦的架構差異，並依任務需求選型。
- 能說明 GPIO、ADC、PWM 與計時器四種周邊的功能與典型用途。
- 能計算 ADC 的解析度與量測誤差，並說明取樣時的注意事項。
- 能說明 PWM 的原理，並計算頻率、占空比與等效電壓。
- 能比較 UART、I²C、SPI 與 CAN 四種介面的特性並正確選用。
- 能計算機器人的電流需求，並設計符合需求的電源架構。
- 能說明接地、共地與電磁干擾的成因，並提出至少五項抑制措施。
- 能說明中斷的機制與使用時機，並辨識中斷服務常式中的禁忌。
- 能設計符合安全規範的緊急停止電路。
- 能依系統性方法診斷機器人的電子故障。

先備知識

第 3 章的系統架構與上下位機分工、第 5 章的驅動器與 PWM 概念、第 6 章的感測器訊號鏈路。本章需要基本電學（歐姆定律、功率計算、串並聯），若不熟悉可先閱讀附錄 D.1。本章不需要微積分。

核心名詞中英對照

表 7-1 本章核心名詞

中文	英文	說明
微控制器	Microcontroller Unit, MCU	整合處理器與周邊介面的單晶片
通用輸入輸出	General Purpose I/O, GPIO	可程式設定為輸入或輸出的接腳
類比數位轉換器	Analog-to-Digital Converter, ADC	將電壓轉換為數值的元件
脈寬調變	Pulse Width Modulation, PWM	以脈波寬度比例控制等效功率的技術
占空比	Duty Cycle	PWM 週期中高電位所佔比例
中斷	Interrupt	由事件觸發、暫停主程式的即時處理機制
中斷服務常式	Interrupt Service Routine, ISR	中斷發生時執行的函式
上拉電阻	Pull-up Resistor	使接腳在無訊號時維持高電位的電阻
去耦電容	Decoupling Capacitor	抑制電源瞬間壓降的旁路電容
共地	Common Ground	各子系統的參考電位連接於同一點
接地迴路	Ground Loop	多條接地路徑形成迴路而引入雜訊
電磁干擾	Electromagnetic Interference, EMI	電磁能量對電路造成的干擾
光耦合器	Optocoupler	以光傳遞訊號並電氣隔離的元件
降壓轉換器	Buck Converter	將高電壓轉為較低電壓的開關式電源
緊急停止	Emergency Stop, E-Stop	以硬體迴路強制切斷動力的安全裝置
看門狗計時器	Watchdog Timer	程式當機時自動重啟系統的計時器

7.1 微控制器與單板電腦

7.1.1 兩種架構的根本差異

第 3 章已介紹過運算單元的三個層級，本節深入其硬體架構的差異。微控制器 (MCU) 與單板電腦 (SBC) 的根本不同，不在於運算速度，而在於**確定性**。

MCU 是「裸機」執行：程式從第一行跑到最後一行，沒有作業系統在背後排程。當你寫下「每 1 毫秒讀一次編碼器」，它就真的每 1 毫秒讀一次，誤差在微秒等級。它的記憶體是靜態配置的、周邊是直接映射到暫存器的、中斷延遲是可計算的。這種可預測性正是即時控制所需要的。

SBC 則跑著完整的作業系統（通常是 Linux），有虛擬記憶體、程序排程、快取機制與網路堆疊。它的運算能力可能是 MCU 的百倍，但你無法保證某一行程式一定在什麼時候執行——作

業系統隨時可能為了處理網路封包或磁碟 I/O 而打斷你的程式。這就是第 3 章所說「軟即時」的來源。

7.1.2 常見平台的比較

Arduino 系列 (AVR 或 STM32 核心)：生態系極為成熟、函式庫豐富、社群龐大，是教學與快速原型的首選。缺點是運算能力有限、且其抽象層 (如 `digitalWrite`) 效率較低，不適合高速控制。

STM32 系列：工業界最主流的 MCU 之一，型號齊全 (從幾十元的 F0 到內建 FPU 與 DSP 指令的 F4、H7)，周邊功能強大 (多組計時器、CAN、乙太網路)，是自製伺服驅動器與飛控的標準選擇。學習曲線較陡。

ESP32：內建 WiFi 與藍牙的雙核心 MCU，價格低廉且效能不錯，一顆核心可跑通訊、另一顆跑控制。是物聯網與教學型機器人的性價比之王，本書附錄多個單元採用此平台。

Raspberry Pi 5：完整的 Linux 電腦，可跑 Python、OpenCV 與 ROS 2，適合當上位機。注意它的 GPIO 沒有 ADC，且不適合做毫秒級控制迴路。

NVIDIA Jetson 系列：具備 GPU 的邊緣 AI 平台，能即時執行深度學習推論，是視覺機器人的主力，但功耗與成本較高。

表 7-2 常見開發平台的比較

平台	核心	時脈	ADC	即時性	適用角色	本書應用
Arduino Uno	AVR 8-bit	16 MHz	10-bit × 6	極佳	入門下位機	附錄 B ~ E
ESP32	Xtensa 雙核 32-bit	240 MHz	12-bit × 18	佳	下位機 / 輕量上位機	附錄 F ~ I
STM32F4	ARM Cortex-M4	168 MHz	12-bit × 16	極佳	伺服驅動、飛控	第 16 章
Raspberry Pi 5	ARM Cortex-A76 ×4	2.4 GHz	無 (需外接)	普通 (Linux)	上位機、ROS 2	第 29 章
Jetson Orin Nano	ARM ×6 + GPU	1.5 GHz	無 (需外接)	普通	視覺 AI 上位機	第 22、31 章

7.2 數位輸入輸出與 GPIO

7.2.1 數位訊號的電壓門檻

數位接腳只認識兩種狀態：高電位（邏輯 1）與低電位（邏輯 0）。但實體世界的電壓是連續的，因此晶片規格書會定義四個門檻： V_{IH} （判定為高的最低輸入電壓）、 V_{IL} （判定為低的最髙輸入電壓）、 V_{OH} （輸出高時的最低電壓）、 V_{OL} （輸出低時的最髙電壓）。介於 V_{IL} 與 V_{IH} 之間的電壓是**未定義區**，讀值可能隨機跳動。

這帶出機器人上一個極常見的問題：**電壓準位不相容**。5 V 系統（Arduino Uno、多數感測器模組）與 3.3 V 系統（ESP32、STM32、Raspberry Pi）混用時，若把 5 V 的訊號直接接到 3.3 V 的接腳上，輕則讀值錯誤、重則燒毀晶片。解法是使用位準轉換器（level shifter）、分壓電阻，或選用支援 5 V 容忍（5V tolerant）的接腳。

7.2.2 上拉、下拉與浮空

一個未連接任何東西的輸入接腳稱為**浮空**（floating），它的電位會隨環境電磁場隨機變化，讀值毫無意義。因此所有輸入接腳都必須有確定的預設電位。

上拉電阻（pull-up）把接腳經電阻接到電源，使其預設為高電位；按鈕按下時接地，變為低電位。這是最常見的接法，因為多數 MCU 內建可程式啟用的上拉電阻，不需外加零件。下拉電阻（pull-down）則相反，預設為低電位。

電阻值的選擇是一個權衡：太大（如 1 M Ω ）抗雜訊能力差、容易受干擾；太小（如 100 Ω ）則耗電且增加接腳負擔。實務上常用 4.7 k Ω 到 10 k Ω 。

7.2.3 開關彈跳與消除

機械開關在切換瞬間，金屬觸點會反覆彈跳數次，在數毫秒內產生多個假訊號。若程式直接計數，按一次可能被讀成三次。消除方式有兩種：硬體消抖（RC 濾波電路或施密特觸發器）與軟體消抖（偵測到變化後延遲 10~50 毫秒再確認，或要求連續數次讀值一致）。第 6 章提到的碰撞板、以及所有按鈕與限位開關，都必須做消抖處理。

◎ 限位開關的接線常識

安全相關的限位開關應採用**常閉接點**（NC）而非常開（NO）。原因是：若電線斷裂或接觸不良，常閉接點會呈現「觸發」狀態使機器停止（安全側失效），而常開接點會呈現「未觸發」，機器繼

續運轉直到撞毀。這個「失效導向安全」(fail-safe) 的設計原則，貫穿整個工業安全領域。

7.3 類比訊號與 ADC

7.3.1 解析度與量測誤差

類比數位轉換器把連續電壓轉成整數。一個 n 位元的 ADC 把參考電壓範圍分成 2^n 個階層，其解析度（每一階代表的電壓）為：

$$\text{LSB} = V_{\text{ref}} / 2^n$$

LSB (Least Significant Bit) 即最小可分辨的電壓變化。 V_{ref} 為參考電壓。

例如 Arduino Uno 的 10 位元 ADC 搭配 5 V 參考電壓： $\text{LSB} = 5/1024 \approx 4.88 \text{ mV}$ 。ESP32 的 12 位元 ADC 搭配 3.3 V： $\text{LSB} = 3.3/4096 \approx 0.81 \text{ mV}$ 。

此外還有**量化誤差**，其大小為 $\pm 0.5 \text{ LSB}$ 。這是數位化的本質限制，無法消除，只能靠提高位元數或縮小量測範圍來降低。

7.3.2 提升有效解析度的三個方法

第一，**縮小參考電壓**。若要量測的訊號只有 0~1 V，使用 5 V 參考等於浪費了 80% 的量測範圍。改用 1.1 V 或 2.5 V 的內部參考電壓，可大幅提升解析度。

第二，**訊號放大**。用運算放大器把微弱訊號放大到接近滿量程再進 ADC。這是力覺感測器（毫伏等級訊號）的必要處理。

第三，**過取樣與平均**。以高於所需的頻率取樣再平均，可提升有效位元數。理論上每過取樣 4 倍可增加 1 位元的有效解析度——但如第 6 章所述，這會增加延遲，且僅在訊號本身有適度雜訊時才有效（需要雜訊來「攪動」量化階層）。

7.3.3 取樣的三個陷阱

第一，**輸入阻抗**。ADC 內部有取樣電容需要充電，若訊號源的輸出阻抗太高（如高阻值的分壓電路），電容充不飽就會讀值偏低。解法是降低分壓電阻值，或加一級電壓隨耦器（voltage follower）。

第二，**混疊 (aliasing)**。如第 3 章所述，取樣頻率必須至少是訊號最高頻率的兩倍。若訊號中含有高於此的成分（例如馬達 PWM 的 20 kHz 雜訊），它會被折疊成低頻的假訊號，且無

法在數位端分辨。因此高精度量測必須在 ADC 之前加**類比抗混疊低通濾波器**——這是軟體濾波無法補救的。

第三，**共用參考電壓的污染**。多數 MCU 以電源電壓作為 ADC 參考。若電源因馬達啟動而下降 5%，所有的 ADC 讀值也會同步偏移 5%——這正是開頭故事中「感測器讀值亂跳」的另一種表現。精密量測應使用獨立的參考電壓 IC。

7.4 PWM 與馬達控制

7.4.1 原理與參數計算

PWM 以固定頻率的方波，透過改變高電位所佔的比例（占空比）來控制等效功率。對馬達這類具有電感（因而對電流變化有慣性）的負載，快速的開關會被自然平滑成穩定的等效電壓：

$$V_{\text{eff}} = V_{\text{supply}} \times D$$

D 為占空比 (0~1)。例如 12 V 電源、占空比 60%，馬達感受到約 7.2 V。

PWM 的解析度取決於計時器的位元數。以 8 位元計時器為例，占空比只有 256 階；若用於伺服馬達的角度控制（脈寬 1~2 ms 佔 20 ms 週期的 5~10%），實際可用的階層只剩約 13 階，角度解析度會粗到約 14 度——完全不堪用。因此伺服控制應使用 16 位元計時器，或使用專用的伺服驅動 IC。

7.4.2 頻率的選擇

PWM 頻率的選擇有三個考量。太低（低於約 2 kHz）會產生人耳可聽見的嘯叫，且馬達電流漣波大、發熱增加。太高則功率電晶體的開關損耗上升，驅動器發熱。此外還要考慮馬達的電氣時間常數——頻率必須高到讓電流在一個週期內不會有太大變化。

實務建議值：直流馬達 16~25 kHz（超出人耳聽覺上限）；無刷馬達 16~40 kHz；LED 調光 1 kHz 以上即可；RC 伺服機則固定為 50 Hz（這是協定規定，不可更改）。

7.4.3 從占空比到速度：一個常見誤解

初學者常以為「占空比 50% 就是最高速的一半」，這並不正確。馬達的轉速取決於等效電壓減去負載造成的壓降，而且存在**啟動死區**——占空比太低時，扭力不足以克服靜摩擦，馬達根本不轉。實測上，多數直流馬達要到 15~25% 的占空比才會開始轉動，而且左右兩顆馬達的死區還可能不同（這是循跡車直線走不直的常見原因）。

正確的做法有二：一是實測每顆馬達的死區並在程式中補償（把輸出映射到死區以上的範圍）；二是加裝編碼器做閉迴路速度控制，讓 PID 自動補償這些非線性——這正是第 16 章的主題。

7.5 通訊介面：UART、I²C、SPI 與 CAN

7.5.1 四種介面的原理

UART（非同步序列埠）：最簡單的點對點通訊，只需 TX、RX 兩條線。雙方必須事先約定相同的鮑率（如 115200 bps），因為沒有時脈線同步。優點是簡單、支援長距離（搭配 RS-485 可達千公尺）。缺點是只能一對一、且鮑率不匹配時會產生亂碼。

I²C：兩線（SDA 資料、SCL 時脈）匯流排，可掛接多達 127 個裝置，每個裝置有唯一位址。優點是接線極省，適合連接多顆感測器。缺點是速度較慢（標準 100 kHz、快速模式 400 kHz）、傳輸距離短（數十公分）、且必須外加上拉電阻（通常 4.7 k Ω ）——這是初學者最常忘記的一步，會導致完全無法通訊。

SPI：四線（MOSI、MISO、SCLK、CS）全雙工介面，速度可達數十 Mbps。適合高速裝置如 IMU、SD 卡、顯示器。缺點是每增加一個裝置就需要多一條 CS（片選）線，接腳消耗大。

CAN Bus：兩線差動匯流排，專為惡劣環境設計。它的三大特色是：差動訊號（抗雜訊能力極強）、多主架構（任何節點皆可主動發送）、以及基於訊息 ID 的優先權仲裁（多個節點同時發送時，ID 較小者自動勝出且不損失資料）。這使它成為汽車與工業機器人的標準，也是多馬達驅動器連接的首選。

表 7-3 四種通訊介面的比較

介面	線數	速度	距離	節點數	同步	典型用途
UART	2 (TX/RX)	9.6k ~ 1 Mbps	數公尺 (RS-485 可達 1 km)	2 (點對點)	非同步	上下位機通訊、除錯輸出
I ² C	2 (SDA/SCL)	100k ~ 1 Mbps	< 1 m	多 (位址定址)	同步	感測器、EEPROM、擴充板
SPI	4+ (含 CS)	1 ~ 50 Mbps	< 30 cm	多 (片選)	同步	IMU、SD 卡、顯示器、ADC
CAN	2 (CANH/CANL)	125k ~ 1 Mbps	40 m ~ 1 km (依速率)	多 (廣播 + 仲裁)	非同步差	馬達驅動器、車輛、工業

介面	線數	速度	距離	節點數	同步	典型用途
					動	

7.5.2 選擇的判準與實務要點

選擇的順序是：先問距離（超過一公尺就排除 I²C 與 SPI）、再問節點數（多節點選 I²C 或 CAN）、再問速度（高速選 SPI 或 CAN）、最後問環境雜訊（馬達旁邊選 CAN 的差動訊號）。

三個實務要點。第一，I²C 忘記接上拉電阻是最常見的除錯陷阱，且部分模組已內建上拉，多個模組並聯時上拉電阻會過小而導致通訊失敗。第二，CAN 匯流排兩端必須各接一顆 120 Ω 終端電阻，否則訊號反射會造成通訊錯誤。第三，長距離的 UART 應改用 RS-485（差動訊號），而非直接拉長 TTL 電平的線。

7.6 電源系統設計

7.6.1 為什麼馬達會讓微控制器重開機

回到開頭故事。要理解那個現象，必須認識一個常被忽略的事實：**電線與電池都有內阻**。當馬達啟動時瞬間抽取大電流 I ，這個電流流過電池內阻 R_{int} 與導線電阻 R_{wire} ，就會產生壓降：

$$V_{drop} = I \times (R_{int} + R_{wire})$$

電流越大、內阻越高，壓降越嚴重。這個壓降發生在電源本身，因此所有共用此電源的元件都會受害。

舉例：一顆老化的鋰電池內阻 0.15 Ω，加上細導線與接頭的 0.1 Ω，總計 0.25 Ω。當兩顆馬達同時啟動抽取 6 A 時，壓降達 $6 \times 0.25 = 1.5$ V。原本 6 V 的電源瞬間只剩 4.5 V——正好跨過微控制器的工作門檻。這就是開頭故事的完整解釋，也說明了為什麼「昨天可以、今天不行」：電池充電狀態與地面摩擦力的變化，剛好把系統推過了臨界點。

7.6.2 電源架構的四個原則

原則一：功率與訊號分離。馬達、加熱器等大電流負載應由獨立的線路供電，不與控制器和感測器共用。若電池只有一顆，則在電池端就分成兩條路徑（各自有自己的開關與保護），而不是共用一條線再分岔。

原則二：單點共地。所有子系統的地線必須連接（否則訊號沒有共同參考電位，通訊會失敗），但應該匯集於一個點（通常是電池負極或電源分配板）。若各處隨意互連，會形成接地迴

路 (ground loop) ——地線本身的電阻使不同位置的「地」電位不同，馬達電流流過地線時會在其上產生電壓，這個電壓會直接疊加到感測器訊號上。

原則三：就近去耦。在每顆晶片的電源腳與地之間，跨接一顆 $0.1\ \mu\text{F}$ 的陶瓷電容 (高頻旁路)，並在電源入口處加一顆 $100 \sim 1000\ \mu\text{F}$ 的電解電容 (提供瞬間電流的儲能池)。這是抑制電源雜訊最便宜、最有效的手段——開頭故事中叫獸加的那顆電容就是後者。

原則四：穩壓與保護。電池電壓會隨放電下降 (鋰電從 $4.2\ \text{V}$ 掉到 $3.0\ \text{V}$)，需經降壓 (Buck) 或升壓 (Boost) 模組穩壓後供給邏輯電路。同時應具備保險絲 (過流)、逆接保護二極體、以及低電壓切斷 (過放保護)。

圖 7-1 機器人電源分配架構圖

建議配圖：由電池出發，經主開關與保險絲後分為兩路。上路 (訊號電源，細線)：經降壓模組至 $5\ \text{V}/3.3\ \text{V}$ ，供給 MCU、感測器與通訊模組，各晶片旁標註 $0.1\ \mu\text{F}$ 去耦電容。下路 (功率電源，粗線)：經急停開關與大電容至馬達驅動器與馬達。兩路的地線最終匯於電池負極的單一星形接地點。圖中並標出量測壓降的建議測點位置。

7.6.3 電流預算

設計電源前應先做電流預算表：列出每個元件的**靜態電流** (正常工作) 與**峰值電流** (啟動或堵轉)，分別加總。電源供應能力必須滿足峰值總和，導線線徑則依持續電流選擇。

一個常見的疏漏是：只加總靜態電流。實務上馬達堵轉電流可達額定的五到十倍 (第 5 章)，若電源按靜態值設計，第一次撞牆就會全系統斷電。建議做法是：電源容量取「所有元件靜態電流總和 + 最大單一負載的峰值電流」，並保留 30% 餘裕。

7.7 接地、雜訊與電磁干擾

7.7.1 干擾的三條路徑

電磁干擾 (EMI) 從干擾源傳到受害電路，有三條路徑，各有不同的抑制方法。

傳導耦合：透過共用的電源線或地線傳遞。這是機器人上最主要的路徑——馬達的電流突波經電源線傳遍全機。抑制方法：電源分離、去耦電容、LC 濾波器、以及必要時使用獨立電池。

電容耦合：兩條靠近的導線之間存在寄生電容，高頻訊號會透過它「跳」過去。這是訊號線與馬達線網在一起時的元兇。抑制方法：拉開距離 (有效距離與間距成反比)、垂直交叉而非平行行走線、加接地隔離層。

電感耦合：變化的電流產生磁場，在鄰近的迴路中感應出電壓。迴路面積越大，感應越強。

抑制方法：**雙絞線**（把去線與回線絞在一起，使兩者的感應電壓互相抵消）、減小迴路面積、使用差動訊號（如 CAN、RS-485）。

7.7.2 七項實務抑制措施

一、訊號線與功率線分開走，若必須交叉則採垂直交叉。二、所有訊號採雙絞或使用屏蔽線，屏蔽層僅在一端接地（兩端接地會形成接地迴路）。三、在馬達端子上並聯 $0.1\ \mu\text{F}$ 陶瓷電容以吸收換相火花。四、縮短所有訊號線長度，尤其是類比訊號與高速數位訊號。五、以光耦合器或數位隔離器隔離不同電壓域（特別是編碼器與驅動器之間）。六、大電流迴路的去線與回線平行緊靠，減小迴路面積。七、在感測器端而非 MCU 端做訊號放大，使長線上傳遞的是強訊號。

◎ 一個免費的雜訊抑制技巧

把馬達的兩條電源線互相絞緊（雙絞），是完全免費卻極其有效的做法。原理是：去線與回線的電流方向相反，緊靠絞合時它們產生的磁場互相抵消，對外輻射大幅降低。同理，把感測器的訊號線與其地線絞在一起，可顯著提升抗干擾能力。這個技巧在自製機器人上的效果，往往比昂貴的屏蔽線更明顯——因為多數學生的屏蔽線根本沒有正確接地。

7.7.3 隔離：最徹底的解法

當干擾嚴重到無法用上述方法解決時，最終手段是**電氣隔離**：讓兩個電路之間沒有任何導體連接，訊號透過光（光耦合器）、磁（隔離變壓器）或電容耦合傳遞。這樣一來，干擾就無法從一側傳到另一側。

工業機器人上，編碼器訊號、急停迴路、以及與 PLC 的介面通常都是隔離的。代價是成本增加、速度受限（一般光耦合器的頻寬有限，高速訊號需用高速型號）、且隔離側需要獨立的電源。

7.8 中斷與即時程式設計

7.8.1 輪詢與中斷

MCU 要偵測事件（按鈕按下、編碼器脈波、通訊資料到達）有兩種方式。**輪詢（polling）**是在主迴圈中反覆檢查狀態，優點是簡單、時序可預測；缺點是浪費運算資源，且若主迴圈中有耗時的運算，就可能錯過短暫的事件。

中斷 (interrupt) 則由硬體在事件發生時主動打斷主程式，跳去執行中斷服務常式 (ISR)，處理完再返回原處。優點是反應極快 (微秒等級)、不會漏掉事件、且主程式可以專心做其他事。這是編碼器計數、精確計時與高速通訊的唯一正解——一顆 1000 線編碼器在高速旋轉時每秒可產生數萬個脈波，用輪詢必定漏計。

7.8.2 中斷服務常式的三大禁忌

ISR 執行期間，其他中斷通常被暫時阻擋，因此它必須**極短**。三個絕對禁忌是：

禁忌一：在 ISR 中使用延遲函式 (如 delay)。這會癱瘓整個系統，因為計時本身也可能依賴中斷。

禁忌二：在 ISR 中做序列輸出 (如 Serial.print)。序列傳輸極慢 (傳送一個字元在 115200 bps 下需約 87 微秒)，且其本身依賴中斷，容易造成死結。除錯時應在 ISR 中設定旗標，由主迴圈負責輸出。

禁忌三：在 ISR 中做複雜運算 (浮點數學、迴圈、動態記憶體配置)。ISR 應只做最必要的事：讀取數值、更新計數器、設定旗標，其餘交給主迴圈。

此外還有一個常見錯誤：ISR 與主程式共用的變數必須宣告為 **volatile**，否則編譯器可能做最佳化而快取舊值，導致主程式讀不到 ISR 的更新。對於多位元組的變數 (如 32 位元的計數器)，主程式讀取時還應暫時關閉中斷，避免讀到「讀了一半就被更新」的破碎數值。

7.8.3 計時器與看門狗

硬體計時器是 MCU 中最有用的周邊之一，能在完全不佔用 CPU 的情況下產生精確的時間基準。控制迴路應由計時器中斷觸發 (例如每 1 毫秒觸發一次)，而非在主迴圈中用延遲函式湊時間——後者的週期會隨主迴圈中其他程式的執行時間而變動，破壞控制系統的假設。

看門狗計時器 (watchdog) 是安全設計的重要一環：它是一個持續倒數的計時器，程式必須定期「餵狗」(重置計數)。若程式當機或陷入無窮迴圈而沒有餵狗，計時器歸零後會自動重啟系統。使用要點是：餵狗的動作應放在確認系統健康的位置 (例如控制迴路完成一輪之後)，而不是隨便放在主迴圈開頭——否則即使控制邏輯已經卡死，看門狗仍會被餵飽而失去意義。

7.9 安全電路與緊急停止

7.9.1 為什麼急停必須是硬體

第 3 章已強調：急停必須是硬體迴路，直接切斷動力，不經過任何軟體或通訊協定。理由很簡單——急停存在的目的，正是為了應對「軟體已經失控」的情況。若急停按鈕只是接到 MCU 的一個 GPIO，然後由程式決定要不要停止馬達，那麼當程式當機時，急停按鈕就完全失效了。

正確的做法是：急停開關（採常閉接點）串聯在功率電源的路徑上，或用來切斷驅動器的致能訊號與繼電器線圈。按下時，物理上切斷馬達的動力來源。同時可另接一路訊號給 MCU，讓軟體知道急停被觸發並做出對應處置（如記錄狀態、通知上位機），但這只是輔助，不是主要機制。

圖 7-2 緊急停止電路示意圖

建議配圖：電池 → 保險絲 → 急停開關（常閉接點，紅色蘑菇頭符號）→ 主接觸器 / 繼電器 → 馬達驅動器 → 馬達。另由急停開關並接一條細線（經上拉電阻）至 MCU 的 GPIO 作狀態偵測。圖中標註：粗線為功率路徑（急停直接切斷）、細線為訊號路徑（僅供軟體知情）。並註明常閉接點的失效導向安全特性。

7.9.2 安全設計的四個層次

第一層：本質安全。從設計上就限制危險——降低馬達功率、限制運動速度、加裝機械限位擋塊、使用圓角避免夾傷。這是最可靠的一層，因為它不依賴任何主動元件。

第二層：硬體保護。急停開關、限位開關（常閉）、保險絲、過流保護、安全繼電器。這些元件不依賴軟體運作。

第三層：軟體監控。程式檢查電流、溫度、位置是否超限，並在異常時停止。同時包含看門狗與通訊超時保護（第 3 章：上位機斷線時下位機必須自動停止）。

第四層：操作規範。測試時墊高機器人使輪子懸空、手不進入運動範圍、通電前先確認佈線、實驗室備有滅火器。這一層常被學生忽略，卻是教學現場最重要的一層。

7.9.3 學生實驗室的安全清單

在自製機器人上，最低限度應具備：一、易於按下的急停開關，位置明顯且不需繞到機器背後。二、電源總開關與保險絲。三、鋰電池使用專用充電器與平衡充，充電時不離人、置於防火袋中。四、測試新程式時，機器人必須架高使輪子離地。五、馬達或電池若發熱異常（無法用手持續接觸），立即斷電。六、機器手臂測試時，所有人退出其工作半徑之外。

7.10 配線、接頭與機構整合

7.10.1 線束設計的六個要點

配線是機器人可靠度的隱形基礎——回顧第 3 章開頭那台因電線磨破而報廢的送藥機器人，這一節就是那個教訓的正面教材。

一、**線徑依電流選擇**。導線過細會發熱、壓降增大。實務參考：0.5 mm² (約 20 AWG) 可承載約 3 A，1.0 mm² (18 AWG) 約 6 A，2.0 mm² (14 AWG) 約 12 A。訊號線可用細線，但馬達線務必足夠粗——這也是降低 7.6.1 節壓降的直接手段。

二、**保留活動餘量**。經過關節的線束必須有足夠長度，並以「U 形迴繞」或螺旋方式配置，讓關節轉動時線束是彎曲而非被拉扯。切勿拉直後剛好夠長。

三、**固定與防磨**。線束每隔約 10 公分應以束帶或線夾固定；經過金屬邊角處必須加套管 (如蛇管、編織網管) 或在孔洞裝上護線環。這是最便宜的可靠度投資。

四、**預留維修長度**。接頭處保留一小段餘線，讓未來重新壓接時不必更換整條線。

五、**標示**。每條線兩端貼上標籤 (來源—去處—功能)。三個月後的你會非常感謝現在的自己。

六、**應力消除**。線材與接頭連接處是最脆弱的點，應以熱縮套管或束帶做應力消除，避免反覆彎折造成斷芯——這種斷裂常發生在絕緣皮內部，外觀完全正常卻時通時斷，是最難診斷的故障。

7.10.2 接頭的選擇

杜邦接頭：便宜、方便，但接觸電阻大、易鬆脫，僅適合訊號與原型測試，**絕不可用於馬達電流**。XT60 / XT30 接頭：可承載大電流，是電池與馬達的標準選擇。JST-XH：常用於平衡充電與訊號，有防呆設計。螺絲端子台：可靠、可承載大電流，但體積大且需工具。航空接頭：防水防塵、耐震，用於戶外與工業場合。

一個共通原則是**防呆**：不同功能的接頭應使用不同的形式或顏色，讓錯誤的插法在物理上不可能發生。這比在說明書上寫「請注意勿插錯」有效一百倍。

7.11 電子系統除錯方法

7.11.1 系統性的五步驟診斷

面對「機器人不動了」，最沒有效率的做法是立刻開始改程式。正確的順序是由物理層往上、由簡單往複雜檢查。

步驟一：確認有電。用電表量測電池電壓、各降壓模組的輸出、以及 MCU 電源腳位的實際電壓。注意要在**負載狀態**下量測（如開頭故事所示，空載時一切正常）。

步驟二：確認接線。目視檢查所有接頭是否鬆脫、有無短路、極性是否正確。用電表的通斷模式測試可疑的線材（尤其是經過關節反覆彎折的）。

步驟三：確認程式有在跑。加入 LED 閃爍或序列輸出，確認程式執行到哪一行、有沒有卡在某個迴圈或不斷重開機（若序列埠不斷輸出開機訊息，就是重開機）。

步驟四：確認輸入正確。把所有感測器讀值印出來，檢查是否合理。許多「演算法錯誤」其實是輸入資料就已經錯了。

步驟五：才懷疑演算法。前四步都確認無誤後，才進入邏輯除錯。

這個順序的價值在於：前四步都很快（各幾分鐘），而它們能解決約八成的問題。學生最常犯的錯，是直接跳到步驟五，花三小時檢查演算法，最後發現是電池沒電。

7.11.2 三種必備工具

萬用電表：最基本也最重要。用來量電壓（診斷電源）、電阻與通斷（診斷斷線）、以及電流（診斷耗電異常）。學生應學會使用通斷模式快速檢查線路。

邏輯分析儀：可同時觀察多條數位訊號的時序，是診斷 I²C、SPI、UART 通訊問題的利器。低價的 USB 型號僅需數百元，卻能省下無數猜測的時間——它能直接告訴你「主機有沒有發出訊號」「從機有沒有回應」。

示波器：能觀察類比波形，用於診斷電源漣波、雜訊、PWM 波形與訊號完整性。開頭故事中的電壓瞬間下降，用電表只能看到跳動的數字，用示波器則能清楚看到那個下陷的波形。

表 7-4 常見故障症狀與診斷方向

症狀	最可能原因	檢查方法	常見解法
馬達一動就重開機	電源壓降（7.6.1 節）	負載狀態下量 MCU 電源電壓	分離電源、加大電容、換粗線

症狀	最可能原因	檢查方法	常見解法
感測器讀值亂跳	電磁干擾或共地不良	馬達停止時是否正常	訊號線遠離馬達、雙絞、加濾波
I ² C 完全無回應	缺上拉電阻或位址錯誤	邏輯分析儀觀察 SDA/SCL	加 4.7 kΩ 上拉、掃描位址
通訊時通時斷	接頭鬆脫或線材內部斷芯	彎折線材時觀察是否中斷	重新壓接、加應力消除
編碼器計數遺漏	使用輪詢而非中斷	檢查程式架構	改用硬體中斷或編碼器介面
直線走不直	左右馬達死區不同	分別實測兩馬達啟動占空比	死區補償或閉迴路控制
伺服機抖動	電源不足或訊號受干擾	量測伺服電源電壓	獨立供電、加大電容
程式偶爾當機	ISR 過長或 volatile 缺漏	檢查中斷服務常式	縮短 ISR、加 volatile、啟用看門狗

7.12 由電子系統邁向運動學

7.12.1 第一篇的總結

走到這裡，第一篇「機器人的身體與系統」已經完整。回顧這五章：第 3 章建立了系統觀，把機器人拆解為七大子系統並理解它們的關係；第 4 章分析了機構與自由度，決定機器人能做什麼的物理上限；第 5 章選定了致動器，賦予它力量；第 6 章配置了感測器，賦予它感知；第 7 章則把這一切用電子系統連接起來，讓訊號與能量正確流動。

此刻你應該能夠：拿到一份機器人規格需求，畫出系統方塊圖、計算自由度、選定馬達與減速機、配置感測器組合、設計電源架構與通訊介面，並列出完整的零件清單與成本。這已經是一位機器人系統工程師的基本功。

7.12.2 接下來：從「能動」到「精確地動」

但是，能組出一台會動的機器人，與能讓它精確地走到指定位置，中間還隔著一整片數學的海洋。第 8 章開始的第二篇，我們將回答一個看似簡單卻極為深刻的問題：**當每個關節轉了某個角度時，末端執行器究竟在空間中的哪個位置？**

這個問題的答案需要一套精確描述空間的語言——座標系、旋轉矩陣、齊次變換。它將把

你在本篇中建立的物理直覺，轉化為可以計算、可以最佳化、可以交給電腦執行的數學模型。第 4 章數過的自由度，將在第 9 章變成矩陣連乘的維度；第 5 章算過的扭力，將在第 13 章成為動力學方程的一項；第 6 章讀到的編碼器數值，將成為運動學計算的輸入。

換句話說：第一篇教你把機器人組起來，第二篇開始教你把它算清楚。

完整計算例題

例題 7-1 電源壓降診斷

題目：某循跡車使用 7.4 V 鋰電池（內阻 $0.12\ \Omega$ ），經 5 V 降壓模組供給 ESP32（工作電壓下限 3.0 V，模組本身需輸入至少 6.0 V 才能穩壓）。馬達與 ESP32 共用同一條電源線，導線與接頭總電阻 $0.08\ \Omega$ 。兩顆馬達正常行進時各抽 0.8 A，堵轉時各抽 3.5 A。（a）求正常行進與堵轉時的電池端電壓。（b）判斷是否會造成 ESP32 重開機。（c）提出兩種改善方案並計算改善後的結果。

解（a）：正常行進總電流 $= 0.8 \times 2 + 0.1\ (\text{ESP32}) = 1.7\ \text{A}$ 。

$$V_{\text{drop}} = 1.7 \times (0.12 + 0.08) = 1.7 \times 0.20 = 0.34\ \text{V} \rightarrow \text{電池端} = 7.4 - 0.34 = 7.06\ \text{V}$$

正常行進時的壓降。

堵轉時總電流 $= 3.5 \times 2 + 0.1 = 7.1\ \text{A}$ 。

$$V_{\text{drop}} = 7.1 \times 0.20 = 1.42\ \text{V} \rightarrow \text{電池端} = 7.4 - 1.42 = 5.98\ \text{V}$$

堵轉時的壓降。

解（b）：降壓模組需要至少 6.0 V 輸入才能維持 5 V 輸出。堵轉時電池端只剩 5.98 V，已低於門檻，模組輸出將開始下降，ESP32 電源不穩而重開機。**會發生故障**，且恰好落在臨界點附近——這解釋了為何有時可以、有時不行（電池電量與摩擦條件的微小變化即可跨越門檻）。

解（c）：方案一，降低線路電阻。改用較粗的導線與 XT60 接頭，將 $0.08\ \Omega$ 降至 $0.03\ \Omega$ 。堵轉壓降 $= 7.1 \times 0.15 = 1.07\ \text{V}$ ，電池端 $= 6.33\ \text{V} > 6.0\ \text{V}$ ，通過但餘裕不足。

方案二，電源分離。將 ESP32 改用獨立電池或在其電源支路加裝大電容與二極體隔離。此時馬達電流不再影響 ESP32 的供電路徑，即使馬達堵轉，ESP32 仍維持穩定——這是最徹底的解法。

方案三，更換電池。改用內阻 $0.05\ \Omega$ 的新電池，堵轉壓降 $= 7.1 \times 0.13 = 0.92\ \text{V}$ ，電池端 $= 6.48\ \text{V}$ ，改善明顯。建議同時採用方案一與方案二。

例題 7-2 ADC 量測設計

題目：欲以 ESP32 (12 位元 ADC、參考電壓 $3.3\ \text{V}$) 監測 $12\ \text{V}$ 鋰電池的電壓，需以分壓電阻降壓。(a) 設計分壓比並選擇電阻值。(b) 求電壓量測的解析度。(c) 若分壓電阻選用 $100\ \text{k}\Omega$ 與 $33\ \text{k}\Omega$ ，可能產生什麼問題？(d) 此電路的持續耗電為多少？

解 (a)：電池滿電約 $12.6\ \text{V}$ ，需降至 $3.3\ \text{V}$ 以下，取安全裕度設計為滿量程對應 $13.2\ \text{V}$ ，分壓比 $= 3.3/13.2 = 1/4$ 。取 $R1 = 30\ \text{k}\Omega$ (上)、 $R2 = 10\ \text{k}\Omega$ (下)：

$$V_{\text{out}} = V_{\text{in}} \times R2/(R1+R2) = 13.2 \times 10/40 = 3.3\ \text{V}$$

分壓比 1:4，滿足量程需求。

解 (b)：ADC 解析度 $\text{LSB} = 3.3/4096 \approx 0.806\ \text{mV}$ ，換算回電池端需乘上分壓比的倒數 4：

$$\text{電壓解析度} = 0.806\ \text{mV} \times 4 \approx 3.22\ \text{mV}$$

可分辨約 3.2 毫伏的電池電壓變化，對電量監測綽綽有餘。

解 (c)： $100\ \text{k}\Omega$ 與 $33\ \text{k}\Omega$ 的組合分壓比雖然接近 ($33/133 \approx 1/4$)，但輸出阻抗過高 (約 $25\ \text{k}\Omega$)。ESP32 的 ADC 建議源阻抗低於 $10\ \text{k}\Omega$ ，否則取樣電容充電不足，會導致讀值偏低且不穩定。這正是 7.3.3 節第一個陷阱。解法是降低電阻值 (如 $30\ \text{k}\Omega/10\ \text{k}\Omega$ ，輸出阻抗 $7.5\ \text{k}\Omega$) 或加一級電壓隨耦器。

解 (d)：以 $30\ \text{k}\Omega + 10\ \text{k}\Omega$ 計算，總電阻 $40\ \text{k}\Omega$ ：

$$I = 12.6 / 40000 \approx 0.315\ \text{mA}$$

持續耗電約 0.32 毫安。若電池為 $2000\ \text{mAh}$ ，此電路單獨可消耗約 6300 小時，影響甚微。但若追求極低待機功耗，可加入 MOSFET 在不量測時切斷分壓電路。

例題 7-3 PWM 解析度與死區補償

題目：某直流馬達由 8 位元 PWM (0~255) 驅動，實測左馬達在占空比 48 以下不轉、右馬達在 38 以下不轉，兩者在 255 時轉速相同。(a) 說明此差異對直線行走的影響。(b) 設計死區補償的映射公式。(c) 補償後，控制指令 0~100% 對應的實際 PWM 值為何 (以左馬達為例，計算 0%、50%、100%)。

解 (a)：若程式對兩馬達下達相同的 PWM 值 (例如 45)，左馬達完全不轉、右馬達已開

始轉動，機器人會原地打轉。即使 PWM 值高於兩者的死區（例如 100），因起始點不同，兩馬達的實際轉速仍有差異，導致機器人持續偏向一側。

解（b）：把控制指令 u （0~100%）線性映射到「死區以上」的可用範圍：

$$\text{PWM} = \text{deadzone} + (255 - \text{deadzone}) \times (u / 100) \quad (u > 0 \text{ 時})$$

$u = 0$ 時直接輸出 0（停止），避免馬達在死區內發熱卻不轉。

解（c）：左馬達 $\text{deadzone} = 48$ 。 $u = 0\%$ 時 $\text{PWM} = 0$ （停止）； $u = 50\%$ 時 $\text{PWM} = 48 + 207 \times 0.5 = 48 + 103.5 \approx 152$ ； $u = 100\%$ 時 $\text{PWM} = 48 + 207 = 255$ 。右馬達則以 38 代入， $u = 50\%$ 時 $\text{PWM} = 38 + 217 \times 0.5 \approx 147$ 。

補償後，同樣的控制指令會讓兩馬達產生接近的轉速，直線行走顯著改善。但這只是開迴路補償，仍無法對抗負載變化與電池電壓下降——真正的解法是加裝編碼器做閉迴路控制（第 16 章）。

程式範例：中斷式編碼器與安全監控

以下 Arduino/ESP32 風格的 C++ 程式示範本章三個核心概念：中斷式編碼器計數（含 volatile 與臨界區保護）、死區補償的 PWM 輸出、以及包含看門狗與通訊超時的安全監控。

```
#include <Arduino.h>

// ----- 接腳定義 -----
const int ENC_A = 2, ENC_B = 3;          // 編碼器（需為中斷腳）
const int PWM_PIN = 5, DIR_PIN = 4;      // 馬達驅動
const int ESTOP_PIN = 7;                 // 急停狀態偵測（常閉，低電位=正常）

// ----- 中斷共用變數必須加 volatile -----
volatile long encoder_count = 0;
volatile bool estop_triggered = false;

// ----- 中斷服務常式：極短，只做必要的事 -----
void IRAM_ATTR encoderISR() {
    // 正交解碼：由 B 相狀態判斷方向
    if (digitalRead(ENC_B) == HIGH) encoder_count++;
    else encoder_count--;
    // 禁忌：此處不可用 delay()、Serial.print()、浮點運算
}

void IRAM_ATTR estopISR() {
```

```

    estop_triggered = true;          // 只設旗標，處置交給主迴圈
}

// ----- 安全讀取多位元組變數 (避免讀到破碎值) -----
long readEncoderSafely() {
    noInterrupts();                  // 進入臨界區
    long value = encoder_count;
    interrupts();                    // 離開臨界區
    return value;
}

// ----- 死區補償的馬達輸出 (例題 7-3) -----
const int DEADZONE = 48;            // 實測值，每顆馬達不同
void setMotor(float percent) {      // percent: -100 ~ +100
    if (fabs(percent) < 1.0) {      // 指令過小則直接停止
        analogWrite(PWM_PIN, 0);
        return;
    }
    digitalWrite(DIR_PIN, percent > 0 ? HIGH : LOW);
    int pwm = DEADZONE + (255 - DEADZONE) * (fabs(percent) / 100.0);
    analogWrite(PWM_PIN, constrain(pwm, 0, 255));
}

// ----- 主程式 -----
unsigned long last_cmd_time = 0;     // 上位機最後通訊時間
const unsigned long COMM_TIMEOUT = 500; // 通訊超時 500 ms

void setup() {
    Serial.begin(115200);
    pinMode(ENC_A, INPUT_PULLUP);    // 內建上拉，避免浮空
    pinMode(ENC_B, INPUT_PULLUP);
    pinMode(ESTOP_PIN, INPUT_PULLUP); // 常閉接點：斷線時變高 = 觸發
    pinMode(PWM_PIN, OUTPUT);
    pinMode(DIR_PIN, OUTPUT);

    attachInterrupt(digitalPinToInterrupt(ENC_A), encoderISR, RISING);
    attachInterrupt(digitalPinToInterrupt(ESTOP_PIN), estopISR, CHANGE);
    last_cmd_time = millis();
}

void loop() {
    // 安全檢查一：急停
    if (estop_triggered || digitalRead(ESTOP_PIN) == HIGH) {
        setMotor(0);
        Serial.println("[緊急停止] 動力已切斷，請排除後重置");
        delay(200);
        return;                      // 不執行任何控制邏輯
    }

    // 安全檢查二：通訊超時 (上位機當機時自動停止，見第 3 章)

```

```

if (millis() - last_cmd_time > COMM_TIMEOUT) {
    setMotor(0);
    Serial.println("[通訊逾時] 上位機失聯，馬達已停止");
    delay(200);
    return;
}

// 正常控制流程
long count = readEncoderSafely();
float target_percent = 40.0;          // 實際應由上位機指令或 PID 決定
setMotor(target_percent);

static unsigned long t_print = 0;    // 序列輸出放主迴圈，絕不放 ISR
if (millis() - t_print > 200) {
    Serial.printf("編碼器: %ld 輸出: %.1f%%\n", count, target_percent);
    t_print = millis();
}
}

```

請注意程式中體現的四個本章原則：一、ISR 極短且只設旗標；二、共用變數宣告 `volatile` 並以臨界區保護多位元組讀取；三、輸入腳啟用上拉避免浮空；四、急停採常閉接點（斷線即觸發，失效導向安全）並在軟體端加上通訊超時保護。

實作：電源壓降實測與改善

本章實作是重現並解決開頭故事的問題，親眼看見「電的謊言」。所需器材：附錄 F 的循跡車或任何雙馬達平台、萬用電表（有示波器更佳）、不同粗細的導線、電解電容（470 ~ 1000 μF ）、以及一顆額外的電池或行動電源。

- 基準量測：**將電表接在 MCU 的電源腳與地之間。輪子懸空、馬達空轉，記錄電壓。
- 負載測試：**把車放在地面並用手輕壓車體增加負載（或讓輪子頂住牆），記錄電壓的最低值。若電表有最小值保持功能請啟用；若有示波器，記錄電壓下陷的波形與持續時間。
- 臨界點測試：**逐步增加負載直到 MCU 重開機（觀察 LED 或序列輸出）。記錄此時的電壓，與 MCU 規格書的工作電壓下限比較。
- 改善一：加電容。**在 MCU 電源入口並聯一顆 470 μF 電解電容（注意極性），重複步驟 2 ~ 3，記錄改善幅度。
- 改善二：換粗線。**將馬達電源線換成更粗的導線並縮短長度，重複量測。
- 改善三：電源分離。**以獨立電池供給 MCU（務必與馬達電源共地），重複量測。

7. **綜合比較**：製作表格比較四種情況（原始、加電容、換粗線、電源分離）下的最低電壓與是否重開機，並依 7.6.1 節的公式反推各情況下的等效線路電阻。
8. **延伸觀察**：在馬達運轉時，用電表量測感測器的讀值波動幅度，比較改善前後的差異——你會發現電源改善同時也降低了感測雜訊。

國中小也看得懂：為什麼機器人會突然昏倒

你有沒有遇過這種情形：家裡開冷氣的時候，電燈會忽然暗一下？機器人身上也會發生一樣的事情！

馬達是一個非常「吃電」的傢伙。當它開始轉動、或是被卡住轉不動的時候，會一口氣把電池的電吸走很多。這時候，電池能給其他零件的電就不夠了——機器人的小電腦（控制板）吃不飽，就會突然「昏倒」，然後重新開機。這就是為什麼有時候機器人明明程式沒問題，卻走一走就停住重來。

怎麼辦呢？有三個好方法。第一，**用粗一點的電線**——就像喝珍珠奶茶要用粗吸管一樣，電線太細，電就跑不過去。第二，**裝一顆電容**——它像一個小水塔，平常存一點電，馬達突然大口喝電的時候，它就趕快補上去。第三，**分開供電**——給小電腦一顆自己的電池，這樣馬達再怎麼吃電也搶不到它的。

還有一件最重要的事：**急停按鈕一定要是真的切斷電**，不能只是叫程式停下來。因為萬一程式當掉了，它就不會理你按按鈕了。真正的急停按鈕是直接把電切斷，就算程式壞掉，機器人也一定會停。做機器人的時候，記得先想「怎麼讓它安全地停下來」，再想「怎麼讓它動起來」喔！

叫獸提醒與常見錯誤

- 誤區一：一遇到問題就改程式。超過半數的「程式問題」其實是電源或佈線問題，應依 7.11.1 節的五步驟由物理層往上診斷。
- 誤區二：只在空載時量測電源電壓。壓降只在馬達出力時才出現，必須在負載狀態下量測。
- 誤區三：5 V 與 3.3 V 系統直接互接。必須使用位準轉換器或確認接腳為 5 V 容忍，否則可能燒毀晶片。
- 誤區四：輸入接腳浮空。未使用上拉或下拉的輸入腳讀值隨機，所有按鈕與開關都應啟用上拉。

- 誤區五：忘記 I²C 的上拉電阻。這是 I²C 完全無回應的第一名原因；反之多個模組並聯時上拉過小也會失敗。
- 誤區六：在 ISR 中使用 delay 或 Serial.print。ISR 必須極短，只做讀值、計數與設旗標。
- 誤區七：中斷共用變數未宣告 volatile。編譯器最佳化會導致主程式讀到過期的值；多位元組變數還需臨界區保護。
- 誤區八：用主迴圈的延遲函式決定控制週期。應改用計時器中斷，否則週期會隨程式負載飄移，破壞控制器設計的前提。
- 誤區九：把急停做成軟體功能。急停必須是硬體迴路直接切斷動力，並採常閉接點以達成失效導向安全。
- 誤區十：訊號線與馬達線網在一起走。電容與電感耦合會嚴重汙染訊號，應分開走線並使用雙絞。
- 誤區十一：用杜邦線傳輸馬達電流。杜邦接頭接觸電阻大且易鬆脫，大電流應使用 XT60 或端子台。
- 誤區十二：線束拉直後剛好夠長。經過關節的線束必須保留活動餘量並做防磨保護，否則反覆彎折後斷裂或磨破短路。

課堂討論

1. 開頭故事中，若小婕在比賽前一天做了哪一項簡單的測試，就能提前發現問題？請設計一個五分鐘可完成的「賽前健檢」程序。
2. 為什麼安全用的限位開關要用常閉接點而非常開接點？請以「電線斷裂」與「接頭鬆脫」兩種失效情境分別說明後果。
3. 有人主張「反正有看門狗，程式當機會自動重啟，不需要急停」。請反駁這個說法，並舉出至少兩種看門狗無法處理的危險情境。
4. 一台機器人的 I²C 感測器在馬達不轉時讀值正常、馬達一轉就通訊失敗。請列出三種可能原因與對應的驗證方法。
5. 比較兩種除錯策略：（甲）先看程式碼找邏輯錯誤；（乙）先用電表量測物理量。以本章的故障統計為依據，說明為何後者更有效率，並討論什麼情況下前者才是對的。

章末摘要

- MCU 與 SBC 的根本差異在確定性而非速度：MCU 裸機執行、時序可預測，適合即時控制；SBC 有作業系統排程延遲，適合規劃與感知。
- 數位輸入需注意電壓準位相容（5 V 與 3.3 V 不可直接互接）、避免浮空（啟用上拉或下拉）、以及機械開關的彈跳消除。
- 安全相關的開關應採常閉接點，以達成失效導向安全——斷線時系統停止而非繼續運轉。
- ADC 解析度 $\text{LSB} = V_{\text{ref}}/2^n$ ，量化誤差為 $\pm 0.5 \text{ LSB}$ ；提升有效解析度可縮小參考電壓、放大訊號或過取樣平均。
- ADC 三大陷阱：輸入阻抗過高、混疊（需類比抗混疊濾波器）、以及參考電壓受電源波動污染。
- PWM 等效電壓 = 電源電壓 $\times$ 占空比；頻率建議 16 ~ 25 kHz（超出人耳）；馬達存在啟動死區，需補償或以閉迴路控制。
- 四種通訊介面：UART 點對點、I²C 省線但短距且需上拉、SPI 高速但耗接腳、CAN 差動抗雜訊且有優先權仲裁。
- 電源壓降 $V_{\text{drop}} = I \times (R_{\text{int}} + R_{\text{wire}})$ 是「馬達一動就重開機」的根本原因；電池老化與負載增加會共同推過臨界點。
- 電源架構四原則：功率與訊號分離、單點共地、就近去耦（0.1 μF 陶瓷 + 大電解）、穩壓與保護。
- EMI 有傳導、電容、電感三條耦合路徑；雙絞線是免費且極有效的抑制手段，最徹底的解法是電氣隔離。
- 中斷是編碼器計數與精確計時的唯一正解；ISR 三禁忌為延遲函式、序列輸出與複雜運算，共用變數須加 volatile。
- 控制週期應由計時器中斷觸發而非主迴圈延遲；看門狗應在確認系統健康處餵食才有意義。
- 安全設計四層次：本質安全、硬體保護、軟體監控、操作規範；急停必須是硬體迴路且不經軟體。
- 除錯五步驟：有電 $\rightarrow$ 接線 $\rightarrow$ 程式在跑 $\rightarrow$ 輸入正確 $\rightarrow$ 才懷疑演算法；前四步能解決約八成的問題。

觀念題與計算題

觀念題

1. 說明 MCU 與 SBC 在確定性上的差異，並各舉兩個適合的機器人任務。
2. 什麼是浮空輸入？為什麼會造成讀值不穩？如何解決？
3. 為什麼安全用限位開關應採常閉接點？請以電線斷裂的情境說明。
4. 說明開關彈跳現象及其兩種消除方法。
5. 計算 12 位元 ADC 搭配 3.3 V 參考的解析度，並說明降低參考電壓為何能提升有效解析度。
6. 什麼是混疊？為什麼它必須在 ADC 之前用類比濾波器處理，而不能靠軟體補救？
7. 說明 PWM 的原理與等效電壓公式，並解釋為何頻率建議設在 16 kHz 以上。
8. 什麼是馬達的啟動死區？它如何造成循跡車直線走不直？請提出開迴路與閉迴路兩種解法。
9. 比較 UART、I²C、SPI 與 CAN 的線數、速度、距離與節點數，並各舉一個典型用途。
10. 寫出電源壓降的公式，並解釋為何「昨天可以跑、今天不行」是合理的現象。
11. 說明電源架構的四個原則，並解釋單點共地與接地迴路的差異。
12. 說明 EMI 的三條耦合路徑及其各自的抑制方法，並解釋雙絞線為何有效。
13. 列出中斷服務常式的三大禁忌，並說明 volatile 關鍵字的作用。
14. 為什麼急停必須是硬體迴路？請說明軟體急停在什麼情況下會失效。
15. 說明電子除錯的五個步驟，並解釋為何「先改程式」是最沒效率的做法。

計算題

1. 某機器人使用 11.1 V 鋰電池（內阻 $0.10\ \Omega$ ），導線與接頭電阻 $0.06\ \Omega$ 。四顆馬達正常運轉各抽 1.2 A，堵轉各抽 5.0 A，控制系統另耗 0.5 A。（a）求正常與全部堵轉時的電池端電壓。（b）若降壓模組需至少 9 V 輸入，堵轉時是否會失效？（c）若要在全部堵轉時仍維持 9 V 以上，總線路電阻（含電池內阻）最多可為多少？
2. 某系統以 10 位元 ADC（參考 5 V）量測力覺感測器，感測器輸出為 0~20 mV。（a）不放大大時，滿量程對應幾個 ADC 階層？（b）解析度對應多少毫伏？（c）若要讓滿量程佔滿整個 ADC 範圍，放大器增益應為多少？（d）放大後的解析度為何？改善了幾倍？
3. 某馬達以 20 kHz PWM 驅動，電源 12 V，占空比 65%。（a）求 PWM 週期（微秒）與高

電位持續時間。(b) 求等效電壓。(c) 若使用 8 位元 PWM 暫存器，65% 對應的設定值為何？(d) 若改用此 8 位元 PWM 控制 RC 伺服機 (50 Hz、脈寬 1~2 ms)，可用的角度階層有幾階？解析度為幾度？此結果說明什麼？

4. 某 CAN 匯流排以 500 kbps 運作，連接 8 個馬達驅動器節點，每個節點每 5 ms 回報一次狀態（每則訊息含協定開銷共 130 位元），上位機每 10 ms 對每個節點下達一次指令（同樣 130 位元）。(a) 求匯流排的總資料率與使用率。(b) 若建議使用率上限為 40%，此設計是否合格？(c) 最多可再增加幾個相同的節點？

動手做與專題延伸

本章實作：機器人電氣健檢表

為你手上的機器人（或附錄中的任一台）完成下列電氣健檢。這份表格應成為每次比賽或展示前的例行程序。

表 7-5 機器人電氣健檢表（第一列為範例）

檢查項目	檢查方法	合格標準	實測結果	處置
MCU 負載電壓	馬達堵轉時量測電源腳	高於規格下限 10% 以上	4.72 V (下限 4.5 V)	合格，餘裕偏低，建議加電容
電池電量與內阻	充飽後量開路電壓與負載電壓	壓降低於標稱值 10%		
急停功能	運轉中按下急停	馬達立即停止（硬體切斷）		
線束固定與防磨	目視檢查並活動關節	無鬆脫、無磨損、有活動餘量		
接頭牢固度	輕拉每個接頭	無鬆脫		
訊號與功率分離	目視檢查佈線	未網綁在一起、無平行長距離		
感測器雜訊	馬達運轉時觀察讀值波動	波動小於量程的 2%		
通訊穩定度	連續運轉五分鐘觀察	無斷線、無亂碼		

專題延伸

- 為附錄 F 的循跡車實測左右兩顆馬達的啟動死區，實作例題 7-3 的補償公式，並以「直線行

走 3 公尺的橫向偏移量」量化補償前後的改善效果。

- 設計並實作一套完整的急停系統：包含常閉急停按鈕、繼電器或接觸器、狀態偵測回授到 MCU、以及軟體端的處置邏輯。繪製電路圖並實測「按下到馬達完全停止」的時間。
- 進階：用邏輯分析儀或示波器診斷一個真實的通訊問題。刻意製造三種故障（拔除 I²C 上拉電阻、將訊號線與馬達線綑在一起、鮑率設定不一致），分別觀察並記錄波形特徵，整理成一份「通訊故障波形辨識手冊」——這將是你未來除錯時最有價值的參考資料。

參考文獻與延伸閱讀

- Horowitz, P. and Hill, W., The Art of Electronics, 3rd ed., Cambridge University Press (電子電路的經典聖經)。
- Ott, H. W., Electromagnetic Compatibility Engineering, Wiley (EMI 與接地設計的權威著作)。
- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer (機器人硬體與系統整合)。
- Yiu, J., The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed., Newnes (MCU 架構與中斷機制)。
- Valvano, J. W., Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers (嵌入式即時系統設計)。
- Barr, M. and Massa, A., Programming Embedded Systems, 2nd ed., O'Reilly (嵌入式 C 程式設計與除錯)。
- IEC 60204-1：機械電氣設備安全通則；ISO 13850：緊急停止裝置設計原則。
- ISO 13849-1：控制系統安全相關部件的設計（安全等級 PL 與風險評估）。
- CAN in Automation (CiA) 技術文件與 Bosch CAN Specification 2.0。
- 各晶片原廠技術文件：STM32 參考手冊、ESP32 技術參考手冊、Arduino 官方文件與應用筆記。

台灣自編大學教科書

@prof

機器人學導論

機構、感知、控制與具身人工智慧

第 二 篇

空間數學與操作臂運動學

第 8 章 空間描述、姿態與齊次轉換

第 9 章 操作臂正向運動學

第 10 章 逆向運動學

第 11 章 雅可比矩陣、速度與靜力

第 12 章 奇異點、冗餘與多任務運動

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

2026 年 9 月

第二篇 篇章導讀

第一篇讓你能把一台機器人**組起來**，這一篇要讓你把它**算清楚**。

五章的內容環環相扣，構成一個完整的邏輯鏈。第 8 章先建立描述空間的語言——座標系、旋轉矩陣、四元數與齊次變換。這一章不談機器人，只談幾何，但它是後面所有計算的字母表：沒有這套語言，連「手臂末端在哪裡」都無法精確表達。

接下來的四章，依序回答關於機械手臂的四個問題。第 9 章（正向運動學）回答**手在哪裡**：把每個關節的角度代入 DH 參數，連乘變換矩陣就得到末端位姿。這是唯一必定有解、且解唯一的問題。第 10 章（逆向運動學）反過來問**手該怎麼擺**：已知目標位置，各關節該轉幾度？這困難得多——可能無解、可能有八組解、也可能有無限多解。第 11 章（雅可比矩陣）進到速度層級，回答**手能怎麼動**：關節轉多快，末端就跑多快？而同一個矩陣的轉置，還意外地連結了力與扭矩。第 12 章（奇異點與冗餘）則回答**手何時動不了**，並說明多餘的自由度該怎麼用。

這一篇有一條貫穿全程的暗線：**退化**。第 8 章的萬向節鎖、第 9 章手臂完全伸直、第 10 章兩組逆解合而為一、第 11 章可操作度歸零——這四個看似不相干的現象，在第 12 章被證明是同一件事：雅可比矩陣失去滿秩。一個線性代數課上看似抽象的概念「矩陣的秩」，就這樣成為理解機器人運動極限的鑰匙。

這一篇要回答的六個問題

- 如何用數字精確描述「一個物體在空間中的位置與方向」？
- 為什麼描述旋轉需要四種不同的表示法，而每一種都會在某處背叛你？
- 已知所有關節角度，末端執行器究竟在哪裡？
- 已知目標位置，各關節該轉幾度？為什麼答案可能有八個、或一個都沒有？
- 關節速度與末端速度之間的「兌換率」為什麼會隨姿態改變？
- 為什麼有些姿態下，機器人再怎麼用力也往某個方向動不了？

讀完這一篇之後

你將擁有完整分析一支機械手臂運動能力的工具：能建立 DH 參數表並計算末端位姿、能求出封閉解或以數值迭代逼近逆解、能構造雅可比矩陣並由它推算速度與力、能診斷奇異點並提出處理策略、能運用冗餘自由度在完成任務的同時最佳化次要目標。

但這一篇的所有分析都是**運動學的**——只談位置、速度與幾何，完全沒有考慮質量、慣性與力的動態效應。這隱含了一個假設：關節可以瞬間達到任何我們指定的速度。這個假設在低速時尚可接受，但在高速運動、重負載或需要精確力控制時就會失效。

第三篇將處理這些問題：手臂有質量、會因重力下垂、加速時需要額外的力矩、旋轉時會產生科氏力。屆時你會發現，本篇建立的雅可比矩陣仍然無所不在——它將出現在慣性矩陣、科氏項與操作空間控制的每一個公式中。

換句話說：第二篇教你機器人**能不能到那裡**，第三篇教你**該用多大的力氣到那裡**。

本篇章節配置

章次	章名	核心內容	叫獸開講
第 8 章	空間描述、姿態與齊次轉換	座標系、旋轉矩陣、歐拉角、四元數、齊次變換、轉換樹	「杯子就在桌上，機器人為什麼還要問：它在哪一個座標系？」——阿波羅十三號的萬向節鎖
第 9 章	操作臂正向運動學	DH 參數、連桿變換、平面手臂、SCARA、六軸手臂、標定	「知道每一顆馬達轉幾度，怎麼算出機器手指尖的位置？」——調了三十次滑桿卻算不出來
第 10 章	逆向運動學	可達性、多解結構、解耦、Pieper 準則、數值迭代、DLS	「人看到杯子就伸手，機器人卻可能算出八種姿勢。」——無解、多解與無限多解
第 11 章	雅可比矩陣、速度與靜力	幾何構造、角速度、虛功原理、速度力對偶、可操作度	「為什麼機器手臂在某些姿勢突然變得很笨重？」——不斷改變的兌換率
第 12 章	奇異點、冗餘與多任務運動	秩與 SVD、三類奇異點、零空間、次要目標、分層逆運動學	「手臂伸得太直，反而最沒有力？」——四個現象，同一個原因

第二篇 空間數學與操作臂運動學

第 8 章

空間描述、姿態與齊次轉換

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為阿波羅十三號登月艙內的慣性導航平台特寫——三個同心萬向環，其中兩環已接近對齊，旁邊放著任務手冊上標示紅色禁區的姿態球圖；右半為現代六軸機械手臂，其基座、法蘭、工具與工件上各疊印一組彩色座標軸（ X 紅、 Y 綠、 Z 藍），並以虛線箭頭標出彼此的變換關係。標題文字：從萬向節鎖到四元數——描述「朝向」的五十年。

叫獸開講 「杯子就在桌上，機器人為什麼還要問：它在哪一個座標系？」

一九七〇年四月十四日，阿波羅十三號的服務艙氧氣罐爆炸。太空人在漆黑的太空中，用僅存的登月艙電力維生，準備繞過月球背面返航。返航前必須執行一次精準的引擎點火，而這需要先讓太空船轉到正確的姿態。

問題來了：導航系統的慣性平台，是靠三個同心的機械環（萬向架）維持一個不隨太空船轉動的參考方向。任務手冊上有一條紅色的警告——某些姿態組合會導致「萬向節鎖」，三個環中的兩個會排列到同一平面上，這時整個平台會瞬間失去一個轉動自由度，導航資料全部作廢，必須用肉眼觀測星星重新校準。在氧氣與電力都所剩無幾的當下，這是他們絕對不能承受的損失。

地面控制中心花了數小時，替太空人算出一條「繞開禁區」的旋轉路徑。他們必須用特定的順序、特定的角度轉動太空船，才能既到達目標姿態、又不讓那三個環對齊。

叫獸講到這裡停下來，拿出一個玩具陀螺儀教具，讓三個環自由轉動。「你們看，」他把中間那個環轉了九十度，「現在最外圈和最內圈的軸，是不是變成同一個方向了？」學生們湊近看——確實，原本互相垂直的三個轉軸，有兩個重合了。「這時候不管我怎麼轉這兩個環，都只能得到同一種轉動。三個自由度變成兩個，有一個方向永遠到不了。」

「更麻煩的是，」他繼續轉動教具，「這不是機械故障，這是**數學本身的問題**。用三個角度描述空間旋轉，無論你怎麼選這三個軸，一定會存在這樣的奇異點。這是幾何學的定理，不是工程師偷懶。」

有學生問：「那現在的機器人怎麼處理？」叫獸在白板上寫下四個字：**四元數**。「今天的無人機、遊戲引擎、機器手臂，內部幾乎都不用歐拉角來儲存姿態，而是用一個四維的數學物件。它沒有奇異點、計算快、內插平滑——代價是完全不直觀，你看著四個數字，腦中浮現不出任何畫面。」

他把玩具放回桌上：「所以這一章我們要學四種描述姿態的方法。不是因為老師要為難你們，而是因為**每一種都有它擅長的場合**：跟人溝通用歐拉角、內部計算用四元數、做座標轉換用矩陣。工程師的專業，就在於知道什麼時候該用哪一種——以及知道每一種會在哪裡背叛你。」

本章為什麼重要

從本章開始，我們進入機器人學的數學核心。第一篇讓你能組出一台會動的機器人，第二篇

要讓你能精確描述並計算它的運動——而這一切的起點，是回答一個問題：**如何用數字描述「一個物體在空間中的位置與方向」**？

這個問題比想像中深刻。位置很簡單，三個座標就夠了；但「方向」（姿態）卻是機器人學中最微妙的概念之一。它有多種表示法，各有優劣與陷阱，而且旋轉不像位移那樣可以隨意交換順序——先轉 X 再轉 Y，與先轉 Y 再轉 X，結果完全不同。

本章建立的座標轉換工具，將是後續所有章节的共同語言：第 9 章的正向運動學是一連串齊次變換的連乘；第 10 章的逆運動學是求解這些變換的反問題；第 22 章的手眼校正是求相機座標與機器人座標的關係；第 25 章的 SLAM 是不斷估計機器人座標系與地圖座標系的變換。可以說，學不好本章，後面二十八章都會卡住。

學習目標

- 能定義座標系、位置向量與姿態，並說明右手座標系的慣例。
- 能寫出並解釋二維與三維旋轉矩陣，並驗證其正交性質。
- 能說明旋轉矩陣的三種幾何詮釋（座標軸、旋轉算子、座標變換）。
- 能使用歐拉角與滾轉—俯仰—偏航描述姿態，並解釋萬向節鎖的成因。
- 能說明軸角表示法與 Rodrigues 公式的意義。
- 能定義單位四元數，說明其與軸角的關係及無奇異點的優勢。
- 能比較四種姿態表示法的優缺點並依場合正確選用。
- 能建構齊次變換矩陣，並用它同時表達平移與旋轉。
- 能正確判斷變換的相乘順序，並區分「向前框架變換」與「向後座標變換」。
- 能計算齊次變換矩陣的反矩陣並說明其物理意義。
- 能說明基座、工具、工件與相機座標系的關係，並繪製轉換樹。
- 能以 Python 實作座標轉換並驗證計算結果。

先備知識

本章需要基本的線性代數：向量加減、內積與外積、矩陣乘法、反矩陣與行列式的概念。若這些概念已經生疏，請先閱讀附錄 D.1 的數學速成，或在遇到時隨時回查。第 2 章曾說明「即時

學習」的策略——本章正是實踐它的最佳時機：你會看到每一個數學工具都對應到一個具體的機器人問題。

三角函數（正弦、餘弦、反正切）在本章大量使用。特別提醒：程式中計算角度務必使用 $\text{atan2}(y, x)$ 而非 $\text{atan}(y/x)$ ，理由已在第 2 章 2.4.3 節說明。

核心名詞中英對照

表 8-1 本章核心名詞

中文	英文	說明
座標系 / 框架	Coordinate Frame	描述位置與方向所依據的參考系統
位姿	Pose	位置與姿態的合稱
姿態	Orientation / Attitude	物體在空間中的朝向
旋轉矩陣	Rotation Matrix	描述姿態的 3×3 正交矩陣
特殊正交群	Special Orthogonal Group, $SO(3)$	所有三維旋轉矩陣所構成的集合
正交矩陣	Orthogonal Matrix	行（列）向量彼此正交且為單位長度的矩陣
右手定則	Right-hand Rule	決定座標軸與旋轉正方向的慣例
歐拉角	Euler Angles	以三次連續旋轉描述姿態的三個角度
滾轉俯仰偏航	Roll-Pitch-Yaw, RPY	繞固定軸的三個旋轉角，航空慣例
萬向節鎖	Gimbal Lock	兩軸對齊導致失去一個自由度的奇異現象
軸角表示法	Axis-Angle Representation	以旋轉軸與旋轉角描述姿態
Rodrigues 公式	Rodrigues' Rotation Formula	由軸角求旋轉矩陣的公式
四元數	Quaternion	以四個數描述旋轉的數學物件
齊次座標	Homogeneous Coordinates	以增加一維的方式統一表示平移與旋轉
齊次變換矩陣	Homogeneous Transformation Matrix	同時描述平移與旋轉的 4×4 矩陣
特殊歐氏群	Special Euclidean Group, $SE(3)$	所有剛體變換所構成的集合
轉換樹	Transform Tree, TF	描述多座標系間關係的樹狀結構

8.1 座標系、位置與姿態

8.1.1 為什麼需要座標系

「杯子在桌上」這句話對人類足夠了，對機器人卻毫無意義。機器人需要知道的是：杯子相對於什麼、在哪個方向、距離多遠。這個「相對於什麼」，就是座標系（coordinate frame，又譯參考框架）。

一個座標系由一個原點與三個互相垂直的單位向量（座標軸）構成。在機器人學中，我們幾乎總是使用**右手座標系**：伸出右手，食指指向 X 軸、中指指向 Y 軸、拇指指向 Z 軸；三者互相垂直。這個慣例的價值在於統一——若團隊中有人用左手系，所有的外積計算與旋轉方向都會反過來，錯誤會非常隱晦難查。

同樣地，**旋轉的正方向**也由右手定則決定：右手拇指指向轉軸的正方向，四指彎曲的方向就是正的旋轉方向（逆時針，由軸的正方向往原點看）。

8.1.2 位置：一個向量

點 P 在座標系 {A} 中的位置，以一個三維向量表示，習慣寫作 ${}^A\mathbf{P}$ ，左上標表示「相對於座標系 A」：

$${}^A\mathbf{P} = [p_x \ p_y \ p_z]^T$$

左上標 A 表示此向量是在座標系 {A} 中量測的。這個上標慣例在本書中會嚴格遵守，因為它是避免混淆的關鍵。

同一個物理點，在不同座標系中會有完全不同的數值。這不是矛盾，而是必然——就像同一個地點，用經緯度描述與用「從火車站往北三百公尺」描述，數字完全不同但指的是同一處。機器人學中大量的錯誤，都源自於忘記標明「這個數字是在哪個座標系裡」。

8.1.3 姿態：三個座標軸的方向

描述姿態的直覺方法是：把物體上的座標系 {B} 的三個座標軸，用座標系 {A} 的語言表達出來。設 {B} 的三個單位軸向量在 {A} 中分別為 ${}^A\mathbf{x}_B$ 、 ${}^A\mathbf{y}_B$ 、 ${}^A\mathbf{z}_B$ ，把它們並排成一個矩陣：

$${}^A\mathbf{R}_B = [{}^A\mathbf{x}_B \ {}^A\mathbf{y}_B \ {}^A\mathbf{z}_B] \quad (3 \times 3 \text{ 矩陣, 三個行向量})$$

這就是旋轉矩陣。它的每一行，是 {B} 的一個座標軸在 {A} 中的方向。

這個定義極為重要，因為它讓旋轉矩陣不再是抽象的數字表格，而是有明確的幾何意義：看矩陣的第一行，就知道 B 的 X 軸指向哪裡。8.2 節會由此推導出旋轉矩陣的所有性質。

圖 8-1 兩個座標系與位置向量

建議配圖：三維透視圖。世界座標系 $\{A\}$ (原點在左下，三軸以粗箭頭標示 X_A 、 Y_A 、 Z_A) 與物體座標系 $\{B\}$ (原點偏右上方，三軸為 X_B 、 Y_B 、 Z_B 且相對 $\{A\}$ 有旋轉)。由 $\{A\}$ 原點指向 $\{B\}$ 原點的向量標為 ${}^A P_{B_{or}}^g$ ，並在 $\{B\}$ 上放一個機械手爪示意物體。旁註：位置由向量描述、姿態由三軸方向描述。

8.2 旋轉矩陣與 SO(3)

8.2.1 二維旋轉：從最簡單的開始

先看平面上的旋轉。若座標系 $\{B\}$ 相對 $\{A\}$ 逆時針轉了角度 θ ，則 $\{B\}$ 的 X 軸在 $\{A\}$ 中的方向是 $[\cos \theta, \sin \theta]^T$ ， Y 軸則是 $[-\sin \theta, \cos \theta]^T$ 。依 8.1.3 節的定義並排成矩陣：

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

二維旋轉矩陣。注意負號的位置——它出現在右上角，這是逆時針為正的右手慣例。

驗算一個直覺：若把點 $[1, 0]^T$ (在 X 軸上) 用 $R(90^\circ)$ 旋轉，得到 $[0, 1]^T$ (跑到 Y 軸上)，符合逆時針轉 90 度的預期。

8.2.2 三維的三個基本旋轉

三維空間中，繞三個座標軸的旋轉矩陣分別為：

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c\theta & -s\theta \\ 0 & s\theta & c\theta \end{bmatrix}$$

繞 X 軸旋轉。 $c\theta$ 與 $s\theta$ 為 $\cos \theta$ 與 $\sin \theta$ 的簡寫。注意 X 那一列與那一行保持不變。

$$R_y(\theta) = \begin{bmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{bmatrix}$$

繞 Y 軸旋轉。注意負號的位置與 X 、 Z 不同——這是初學者最常抄錯的一個。

$$R_z(\theta) = \begin{bmatrix} c\theta & -s\theta & 0 \\ s\theta & c\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

繞 Z 軸旋轉。 Z 那一列與那一行保持不變。

記憶技巧：繞哪一軸旋轉，該軸對應的那一行與那一列就是單位向量（保持不變），剩下的 2×2 子矩陣就是二維旋轉矩陣。唯一的例外是 R_y 的負號位置與另外兩個相反——原因是 $Z \rightarrow X$ 的循環順序與 $X \rightarrow Y$ 、 $Y \rightarrow Z$ 不同。

8.2.3 旋轉矩陣的性質

由 8.1.3 節的定義（三個行是互相垂直的單位向量），可以直接推出旋轉矩陣的核心性質：

$$\mathbf{R}^T \mathbf{R} = \mathbf{I} \Rightarrow \mathbf{R}^{-1} = \mathbf{R}^T$$

旋轉矩陣是正交矩陣，其反矩陣等於轉置。這是機器人學中最有價值的一條性質。

為什麼這條性質這麼重要？因為求一般 3×3 矩陣的反矩陣需要計算行列式與伴隨矩陣，運算量大且有數值誤差；而轉置只是把元素位置對調，幾乎不花任何運算。這意味著座標的反向轉換是免費的——這個特性在每秒要做上千次轉換的機器人系統中，價值極高。

此外，旋轉矩陣的行列式恆為 $+1$ ：

$$\det(\mathbf{R}) = +1$$

行列式為 $+1$ 代表這是「純旋轉」（保持右手系）。若 $\det(\mathbf{R}) = -1$ ，則包含了鏡射，會把右手系變成左手系，在物理上對應不到任何剛體運動。

所有滿足這兩個條件（正交且行列式為 $+1$ ）的 3×3 矩陣，構成一個集合，稱為**特殊正交群** $\mathbf{SO}(3)$ 。它是一個群（group），意思是：兩個旋轉矩陣相乘仍是旋轉矩陣（封閉性）、有單位元素（單位矩陣，代表不轉）、每個元素有反元素（反向旋轉）。這個結構保證了旋轉的組合永遠有意義。

8.2.4 旋轉矩陣的三種詮釋

同一個旋轉矩陣，在不同場合有三種讀法，能靈活切換是理解本章的關鍵。

詮釋一：座標軸的描述。 $\mathbf{AR}_B$ 的三個行，是 $\{B\}$ 的三個座標軸在 $\{A\}$ 中的方向（8.1.3 節的定義）。

詮釋二：座標的轉換。若已知點在 $\{B\}$ 中的座標 $\mathbf{BP}$ ，可求其在 $\{A\}$ 中的座標： $\mathbf{AP} = \mathbf{AR}_B \cdot \mathbf{BP}$ 。這是實務上最常用的一種。

詮釋三：旋轉的算子。 $\mathbf{R}$ 也可以視為一個「動作」：把一個向量在同一個座標系內轉動。例如 $\mathbf{R}_z(90^\circ)$ 作用在向量上，把它繞 Z 軸轉 90 度。

詮釋二與詮釋三在數學上是同一個式子，但物理意義相反：詮釋二是「物體不動，換個角度描述它」；詮釋三是「座標系不動，把物體轉動」。這個「主動旋轉 vs 被動旋轉」的區別是初學者混淆的主要來源，第 8.7 節會再從變換的角度深入。

◎ 一個檢查錯誤的好習慣

拿到一個聲稱是旋轉矩陣的數值矩陣時，做兩個快速檢查：一、每一行的長度是否為 1 （平方和為

1) ? 二、任兩行的內積是否為 0 (互相垂直) ? 若有偏差 (例如 0.998 而非 1.000) , 通常是數值誤差累積所致 , 可用正交化 (如 Gram-Schmidt 或 SVD) 修正。長期累積不修正 , 會導致機器人的姿態逐漸「歪掉」。

8.3 歐拉角與 RPY

8.3.1 用三個角度描述姿態

旋轉矩陣有九個數字 , 卻只有三個自由度 (因為有六個正交條件約束) , 顯得非常冗餘。歐拉角則用最少的三個數字來描述姿態 : 把姿態拆解成繞三個軸的連續旋轉。

關鍵在於 , 這三次旋轉可以有不同的**軸序**與**參考基準** , 因此歐拉角有多達二十四種慣例 , 這是實務上混亂的根源。兩個主要的分類軸是 :

內旋 (intrinsic) vs 外旋 (extrinsic) : 內旋是繞著「轉動後的新座標軸」旋轉 (動軸) , 外旋是繞著「固定不動的原座標軸」旋轉 (定軸) 。

軸序 : 常見的有 Z-Y-X、Z-X-Z、X-Y-Z 等。航空領域常用 Z-Y-X , 古典力學常用 Z-X-Z 。

8.3.2 滾轉—俯仰—偏航 (RPY)

機器人與航空領域最常用的是 RPY 慣例 : 繞**固定軸**依序旋轉 , 先繞 X 軸轉滾轉角 γ (roll) 、再繞 Y 軸轉俯仰角 β (pitch) 、最後繞 Z 軸轉偏航角 α (yaw) 。對應的旋轉矩陣為 :

$$R_{RPY}(\alpha, \beta, \gamma) = R_z(\alpha) \cdot R_y(\beta) \cdot R_x(\gamma)$$

注意相乘順序 : 先做的旋轉寫在右邊。這是繞固定軸旋轉的慣例 , 稱為外旋。

這個「先做的寫右邊」規則常令初學者困惑。可以這樣理解 : 矩陣作用在向量上時 , 是由右往左依序作用的 ($R_z \cdot R_y \cdot R_x \cdot P$, 最先碰到 P 的是 R_x) 。因此最先執行的旋轉必須寫在最右邊。

一個有用的定理是 : 繞**固定軸**依序 **X-Y-Z** 旋轉 , 等價於繞**動軸**依序 **Z-Y-X** 旋轉。這解釋了為什麼有些文獻說 RPY 是「Z-Y-X 歐拉角」、有些說是「X-Y-Z 固定角」——它們指的是同一件事。

對飛行器與移動機器人而言 , RPY 的直覺意義是 : 滾轉是機身左右傾斜、俯仰是機頭上下、偏航是機頭左右轉向。這個直覺性正是它在人機介面上不可取代的原因——你可以對操作員說

「機頭抬高十度」，但不能說「四元數的第二個分量增加 0.087」。

8.3.3 萬向節鎖：歐拉角的致命缺陷

回到開頭的阿波羅十三號。萬向節鎖發生在**中間那次旋轉達到 ± 90 度**時。以 Z-Y-X 慣例為例，當俯仰角 $\beta = 90^\circ$ 時，把它代入矩陣運算會發現：第一次的 Z 軸旋轉與第三次的 X 軸旋轉，在效果上變成了繞同一個軸的旋轉。

當 $\beta = 90^\circ$ 時， α 與 γ 只以 $(\alpha - \gamma)$ 的組合出現

兩個自由度退化成一個。此時給定一個姿態， α 與 γ 有無限多組解，數值計算會失去唯一性且極不穩定。

這造成三個實際問題。第一，**失去一個自由度**：無論怎麼調整 α 與 γ ，都有一個方向的旋轉到不了。第二，**數值不穩定**：在接近奇異點時，微小的姿態變化會導致角度值劇烈跳動（例如從 179° 跳到 -179° ）。第三，**內插失敗**：若在兩個姿態之間對三個角度做線性內插，路徑會扭曲甚至發生翻轉。

必須強調：這不是某種慣例的缺陷，而是數學的必然。**任何以三個數描述三維旋轉的方法，都必然存在奇異點**。要避開它，只能增加維度——這就是四元數（四個數）與旋轉矩陣（九個數）存在的理由。

圖 8-2 萬向節鎖的幾何解釋

建議配圖：三格連續圖。左格：三個同心環（外環繞 Z、中環繞 Y、內環繞 X）互相垂直，三個轉軸指向不同方向，標註「三個獨立自由度」。中格：中環轉了 90 度，此時外環軸與內環軸變成平行（重合），以紅色虛線標出兩軸重合。右格：無論轉動外環或內環，物體都只繞同一軸轉動，標註「自由度由 3 降為 2」。下方附阿波羅任務手冊上禁區示意。

8.4 軸角表示法與 Rodrigues 公式

8.4.1 歐拉旋轉定理

有一個優美的定理（歐拉旋轉定理）指出：**任何三維旋轉，都可以表示為繞某一個固定軸旋轉某一個角度**。無論姿態多複雜，都不需要三次旋轉，一次就夠——只要選對軸。

因此可以用一個單位向量 $\mathbf{k}$ （旋轉軸）與一個純量 θ （旋轉角）來描述姿態，這稱為軸角表示法。它常被寫成一個三維向量：

$$\mathbf{r} = \theta \cdot \mathbf{k} \quad (\text{旋轉向量 / Rotation Vector})$$

向量的方向是旋轉軸，長度是旋轉角（弧度）。這是最精簡的表示法，只需三個數且具有明確的幾何意義。

8.4.2 Rodrigues 公式

由軸角求旋轉矩陣的公式稱為 Rodrigues 公式：

$$\mathbf{R} = \mathbf{I} + \sin \theta \cdot [\mathbf{k}]_{\times} + (1 - \cos \theta) \cdot [\mathbf{k}]_{\times}^2$$

其中 $[\mathbf{k}]_{\times}$ 是 $\mathbf{k}$ 的反對稱矩陣 (skew-symmetric matrix)，定義於下式。

$$[\hat{\mathbf{k}}]_{\times} = \begin{bmatrix} 0 & -k_z & k_y \\ k_z & 0 & -k_x \\ -k_y & k_x & 0 \end{bmatrix}$$

反對稱矩陣，滿足 $[\mathbf{k}]_{\times} \cdot \mathbf{v} = \mathbf{k} \times \mathbf{v}$ (外積)。它是連接向量外積與矩陣運算的橋樑，在第 11 章推導雅可比矩陣時會大量使用。

反對稱矩陣是一個值得熟悉的工具。它的核心性質是：把外積運算轉換成矩陣乘法。在機器人學中，角速度與線速度的關係、力矩與力的關係，都會用到這個轉換。

8.4.3 優點與陷阱

軸角表示法的優點是幾何意義明確、只需三個數、且在小角度時有優雅的近似 (當 θ 很小， $\mathbf{R} \approx \mathbf{I} + [\mathbf{r}]_{\times}$ ，這是第 11 章微分運動學的基礎)。

但它仍有奇異點：當 $\theta = 0$ 時，旋轉軸 $\mathbf{k}$ 完全沒有定義 (不轉的話，繞哪個軸都一樣)；當 θ 接近 180° 時， $\mathbf{k}$ 與 $-\mathbf{k}$ 對應同一個旋轉，數值計算會不穩定。此外，兩個旋轉向量無法直接相加來合成旋轉——這是初學者常犯的錯誤 (旋轉不可交換，所以不能像位移那樣疊加)。

8.5 四元數

8.5.1 定義與幾何意義

四元數是十九世紀數學家 Hamilton 發明的數系，可視為複數的三維推廣。一個四元數由四個實數組成：

$$\mathbf{q} = q_0 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k} \quad \text{或寫作} \quad \mathbf{q} = (q_0, \mathbf{q}) \quad \text{其中} \quad \mathbf{q} = [q_1, q_2, q_3]^T$$

q_0 稱為純量部 (實部)， $\mathbf{q}$ 稱為向量部 (虛部)。

用於描述旋轉的是單位四元數 (長度為 1)。它與軸角表示法有一個極為優雅的對應關係：若旋轉軸為單位向量 $\mathbf{k}$ 、旋轉角為 θ ，則

$$q_0 = \cos(\theta/2), \quad \mathbf{q} = \sin(\theta/2) \cdot \mathbf{k}$$

注意是半角。這個 $\theta/2$ 是四元數所有神奇性質的來源，也是它最反直覺的地方。

為什麼是半角？直觀的理解是：四元數作用在向量上時，需要「左乘一次、右乘一次」($q \cdot v \cdot q^{-1}$)，因此每次貢獻一半的角度。這也導致一個重要現象： q 與 $-q$ 代表**完全相同的旋轉**（因為 θ 與 $\theta+360^\circ$ 是同一個姿態，但半角差了 180° ，使四個分量全部變號）。這稱為**雙重覆蓋**（double cover），在實作中必須注意——比較兩個四元數是否相同時，要同時檢查 q 與 $-q$ 。

8.5.2 三大優勢

優勢一：無奇異點。四元數在整個旋轉空間中都表現良好，不存在萬向節鎖。這是它成為無人機飛控與遊戲引擎標準的首要原因。

優勢二：計算效率高。兩個四元數相乘（合成兩個旋轉）需要 16 次乘法與 12 次加法；兩個 3×3 旋轉矩陣相乘則需要 27 次乘法與 18 次加法。且四元數只需儲存 4 個數而非 9 個。在每秒數千次姿態更新的系統中，這個差距很可觀。

優勢三：內插平滑。在兩個姿態之間平滑過渡時，四元數可用球面線性內插（SLERP），得到的是繞固定軸的等速旋轉——這是最自然的路徑。相較之下，對歐拉角做線性內插會產生扭曲的路徑，對旋轉矩陣的元素做內插甚至會產生非旋轉矩陣。這正是第 15 章姿態軌跡規劃必用四元數的原因。

8.5.3 基本運算

旋轉的合成（相當於矩陣相乘）用四元數乘法（Hamilton 積）：

$$q \cdot p = (q_0 p_0 - \mathbf{q} \cdot \mathbf{p}, \quad q_0 \mathbf{p} + p_0 \mathbf{q} + \mathbf{q} \times \mathbf{p})$$

注意四元數乘法不可交換（ $q \cdot p \neq p \cdot q$ ），這正確反映了旋轉不可交換的事實。

反向旋轉（相當於矩陣轉置）用共軛：

$$q^* = (q_0, -\mathbf{q}) \quad (\text{對單位四元數, } q^{-1} = q^*)$$

只需把向量部變號，運算成本極低——與旋轉矩陣的轉置一樣便宜。

旋轉一個向量 v ：

$$v' = q \cdot (0, v) \cdot q^*$$

把向量寫成純量部為 0 的四元數，左乘 q 、右乘 q^* 。實作上通常會展開成更高效的形式，或先轉成旋轉矩陣再作用。

8.5.4 最大的缺點：不直觀

四元數的唯一但致命的缺點是：人類完全無法直觀理解。給你一組數值 (0.707, 0, 0.707, 0)，你的腦中浮現不出任何畫面（它其實是繞 Y 軸轉 90 度）。因此實務上的標準做法是：內部用四元數儲存與計算，介面上轉成歐拉角顯示給人看。

這正是本章開頭叫獸強調的重點——不同表示法各有其舞台。無人機的飛控軟體內部用四元數運算，但地面站的螢幕上顯示的是「俯仰 12 度、滾轉 -3 度」；機器手臂的控制器內部用旋轉矩陣，但示教器上讓操作員輸入的是 RPY 角。

表 8-2 四種姿態表示法的比較

表示法	參數數目	奇異點	合成運算	內插	直觀性	典型用途
旋轉矩陣	9 (3 自由度)	無	矩陣相乘 (27 乘法)	困難 (需正交化)	中	座標轉換、運動學計算
歐拉角 / RPY	3	有 (萬向節鎖)	需先轉成矩陣	會扭曲	高	人機介面、規格標示
軸角 / 旋轉向量	3	有 ($\theta=0$ 與 180°)	不可直接相加	尚可	中高	小角度近似、微分運動學
單位四元數	4 (3 自由度)	無	四元數乘法 (16 乘法)	優 (SLERP)	極低	飛控、遊戲引擎、姿態濾波

◎ 選用指南

與人溝通、顯示與輸入 → 歐拉角 / RPY。座標轉換與運動學推導 → 旋轉矩陣（可與平移合併為齊次變換）。內部儲存、姿態更新與內插 → 四元數。微小旋轉的線性化分析 → 軸角 / 旋轉向量。四者之間可以自由轉換，實務上經常在同一個系統中並存——重點是每次轉換都要清楚知道自己在哪一種表示法中。

8.6 齊次座標與齊次變換矩陣

8.6.1 把平移與旋轉合而為一

到目前為止，旋轉用矩陣描述、平移用向量描述。一個完整的剛體位姿變換要同時包含兩者：

$$\mathbf{AP} = \mathbf{AR}_B \cdot \mathbf{BP} + \mathbf{AP}_{B_{or}^g}$$

點在 {B} 中的座標先經旋轉，再加上 {B} 原點在 {A} 中的位置。這是最直接的寫法，但有一個缺點——它是「乘法加上加法」的混合形式。

為什麼混合形式不好？因為當我們要串接多次變換時（第 9 章的運動鏈有六、七次），這個式子會層層嵌套成一個難以整理的長串。若能把它變成純粹的矩陣乘法，就可以直接連乘，形式極為簡潔。

解法是一個巧妙的技巧：**升高一個維度**。把三維座標寫成四維，最後一個元素固定為 1：

$$\tilde{\mathbf{P}} = [p_x \ p_y \ p_z \ 1]^T$$

這稱為齊次座標 (*homogeneous coordinates*)。多出來的那個 1，是讓平移能被「乘」進去的關鍵。

8.6.2 齊次變換矩陣

有了齊次座標，就可以把旋轉與平移打包成一個 4×4 矩陣：

$${}^A\mathbf{T}_B = \begin{bmatrix} {}^A\mathbf{R}_B & {}^A\mathbf{p}_{B_o.r^g} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & 1 \end{bmatrix}$$

左上 3×3 是旋轉、右上 3×1 是平移、左下三個 0 與右下的 1 是為了維持矩陣運算的一致性。

驗證它確實做對了事：

$${}^A\tilde{\mathbf{P}} = {}^A\mathbf{T}_B \cdot {}^B\tilde{\mathbf{P}} \Rightarrow \begin{bmatrix} {}^A\mathbf{p} \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} {}^B\mathbf{p} \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} \cdot {}^B\mathbf{p} + \mathbf{p} \\ 1 \end{bmatrix}$$

展開後的上三列正是 8.6.1 節的式子，而最後一列給出 $1 = 1$ （恆成立）。齊次變換確實等價於「旋轉後再平移」。

所有這種形式的 4×4 矩陣構成的集合，稱為**特殊歐氏群 SE(3)**，代表所有可能的剛體運動。它同樣是一個群：兩個剛體變換的組合仍是剛體變換、有單位元素、每個元素有反元素。

8.6.3 為什麼要那么多餘的一列

學生常問：最後一列 $[0 \ 0 \ 0 \ 1]$ 看起來毫無作用，為什麼要加？答案有三個。

第一，**維持矩陣可乘性**。只有方陣才能連乘任意多次並保持形式一致，這正是第 9 章運動學連乘的需求。

第二，**保證反矩陣存在**。 4×4 的齊次變換矩陣行列式為 1（非零），必定可逆。

第三，**可擴充性**。在電腦圖學中，把最後一列改成非零值可以表示透視投影；把右下角的 1 改成其他值可以表示縮放。機器人學中不用這些功能，但這正是相機成像模型（第 22 章）使用同一套框架的原因。

8.7 變換的組合與相乘順序

8.7.1 連鎖法則

齊次變換最強大的地方在於它們可以直接串接。若已知 {A} 到 {B} 的變換、以及 {B} 到 {C} 的變換，則 {A} 到 {C} 的變換為：

$${}^A T_C = {}^A T_B \cdot {}^B T_C$$

上下標的「消去律」：右邊相鄰的 B 與 B 互相抵消，剩下 A 到 C 。這是檢查式子有沒有寫錯的最快方法。

這條消去律極其實用。例如第 9 章的六軸手臂正向運動學：

$${}^0 T_6 = {}^0 T_1 \cdot {}^1 T_2 \cdot {}^2 T_3 \cdot {}^3 T_4 \cdot {}^4 T_5 \cdot {}^5 T_6$$

從基座 (0) 到末端 (6) 的變換，就是把每一節連桿的變換依序相乘。相鄰的上下標兩兩抵消，剩下 0 到 6。

看到這裡，第 4 章數過的自由度、第 5 章選過的關節，就在這一行式子裡被統一起來了——每一個 ${}^{i-1}T_i$ 都包含一個關節變數。這就是機器人運動學的核心方程式。

8.7.2 順序至關重要

矩陣乘法不可交換，因此 $T_1 \cdot T_2 \neq T_2 \cdot T_1$ 。這在物理上完全合理：先向前走再左轉，跟先左轉再向前走，到達的位置完全不同。

親手做一個實驗最能建立直覺：拿一本書平放在桌上，先繞書的長邊翻轉 90 度、再繞短邊翻轉 90 度，記住最終姿態；把書放回原位，改成先繞短邊、再繞長邊，你會得到完全不同的姿態。這個「不可交換性」是三維旋轉最重要也最違反直覺的性質。

8.7.3 左乘與右乘：定軸與動軸

既然順序重要，就必須弄清楚：新的變換應該乘在左邊還是右邊？這對應到兩種不同的物理操作。

右乘 = 相對於當前的自身座標系 (動軸)。 $T_{\text{new}} = T_{\text{current}} \cdot T_{\text{relative}}$ 。例如機器手臂「沿著自己夾爪的方向前進 5 公分」，這個運動是相對於夾爪自身的座標系，因此右乘。

左乘 = 相對於固定的參考座標系 (定軸)。 $T_{\text{new}} = T_{\text{relative}} \cdot T_{\text{current}}$ 。例如「把整個物體繞世界座標系的 Z 軸旋轉 30 度」，這是相對於外部固定座標系，因此左乘。

這個規則可以用 8.7.1 節的消去律驗證： ${}^A T_C = {}^A T_B \cdot {}^B T_C$ 中，若 {B} 是當前位姿、{C} 是相對於 {B} 的新變換，則新變換寫在右邊——正是右乘的形式。

圖 8-3 左乘與右乘的差異

建議配圖：兩列對照圖。上列（右乘／動軸）：機械手臂末端沿著自身 Z 軸方向前進，箭頭隨夾爪姿態改變方向；下列（左乘／定軸）：整個手臂繞世界座標系的 Z 軸旋轉，箭頭固定指向世界的垂直方向。各列標出對應的矩陣式子與適用情境。

8.7.4 「向前框架變換」與「向後座標變換」

這是一個常令人困惑但極為重要的觀念。同一個矩陣 ${}^A T_B$ 有兩種讀法，兩者方向相反：

框架的變換（向前）： ${}^A T_B$ 描述了如何從框架 {A} 移動到框架 {B}——先平移、再旋轉，A 走到 B。

座標的變換（向後）： ${}^A T_B$ 把在 {B} 中的座標轉換成 {A} 中的座標，即 $A P = {}^A T_B \cdot B P$ 。方向是 B 到 A。

兩者方向相反並非矛盾，而是數學上的必然：框架與座標是**對偶**的關係。想像你把座標系往右移動 3 公尺，那麼原本靜止不動的物體，在新座標系中的座標就會往左變 3 公尺。框架向右，座標向左——這就是「框架變換協變、座標變換逆變」的直觀說法。

實務上的建議是：永遠使用上下標記號（ ${}^A T_B$ ），並嚴格遵守消去律。只要式子的上下標能正確消去，方向就不會錯，不需要每次都在腦中重新推理。

8.8 齊次變換的反矩陣

8.8.1 不要用一般的反矩陣公式

要求 ${}^B T_A$ （{A} 相對於 {B}），可以對 ${}^A T_B$ 求反矩陣。但千萬不要用一般的 4×4 矩陣求逆演算法——那需要大量運算且有數值誤差。利用齊次變換的結構，可以寫出封閉形式的解：

$${}^A T_B^{-1} = {}^B T_A = \begin{bmatrix} {}^A R_B^T & -{}^A R_B^T \cdot {}^A P_{B,org} \\ 0 & 1 \end{bmatrix}$$

旋轉部分取轉置（利用正交性），平移部分則是「先反向旋轉、再取負號」。

8.8.2 為什麼平移部分不是直接取負

初學者常誤以為反矩陣的平移部分就是 $-p$ 。這是錯的，正確答案是 $-R^T \cdot p$ 。可以這樣理解：

原本的變換是「先旋轉、再平移」。要反過來，必須「先反平移、再反旋轉」——順序也要顛倒。而在新（旋轉後的）座標系中表達那個反向平移，就必須先用 R^T 把它轉換過來，因此得到 $-R^T \cdot p$ 。

這個公式的運算成本極低：一次轉置（免費）加上一次 3×3 矩陣與向量的乘法（9 次乘法）。相較於一般 4×4 求逆的數十次運算，效率高出許多，且不會有數值不穩定的問題。這也是機器人系統中座標轉換能夠即時執行的原因之一。

8.9 機器人系統中的座標系

8.9.1 五個標準座標系

一台典型的工業機器人工作站中，至少有五個必須明確定義的座標系。

世界座標系 {W}：整個工作站的絕對參考，通常固定在地面或工作台的某個角落。多台機器人協同作業時，這是它們共同的語言。

基座座標系 {B}：固定在機器人底座上。它與世界座標系的關係（ ${}^W T_B$ ）在安裝完成後就固定了，由現場測量或校正得到。

手腕 / 法蘭座標系 {F}：位於機器人第六軸的法蘭面上。從 {B} 到 {F} 的變換，正是正向運動學要計算的（第 9 章），它隨關節角度而變。

工具座標系 {T}：定義在末端執行器的作用點上（例如夾爪的中心、焊槍的槍尖）。從 {F} 到 {T} 的變換由工具的幾何決定，稱為工具中心點（TCP）設定。這是實務上最常需要重新設定的參數——換一支夾具就要重設一次。

工件座標系 {O}：定義在工件或治具上。程式中的路徑通常以工件座標系描述，這樣當工件位置移動時，只需更新一個變換矩陣，所有路徑點自動跟著移動，不必逐點重教。這是離線編程（第 30 章）的基礎。

$${}^W T_T = {}^W T_B \cdot {}^B T_F(q) \cdot {}^F T_T$$

工具在世界座標系中的位姿。中間那一項是關節角度 q 的函數（正向運動學），其餘兩項在標定後固定。

8.9.2 轉換樹與 ROS 2 的 TF

當座標系數量增加（加上相機、光達、多個工件），手動管理所有變換會變得極為繁瑣。ROS 2 的 TF 系統提供了系統化的解法：把所有座標系組織成一棵樹，每個節點是一個座標系，每條邊是一個變換。

樹狀結構的關鍵性質是：任兩個座標系之間，只有唯一一條路徑。因此系統可以自動計算任

意兩個座標系之間的變換——只要沿著樹走一趟，把路上的變換連乘（往上走時取反矩陣）即可。使用者只需要問「相機看到的這個點，在機器人基座座標系中是哪裡？」系統就會自動找出路徑並完成計算。

TF 系統還處理了一個實務上極重要的問題：**時間**。感測器資料有延遲，相機在時刻 t 拍到的影像，對應的機器人姿態是 t 時刻的而非現在的。TF 會儲存變換的歷史紀錄，讓你能查詢「在時刻 t 時，這兩個座標系的關係是什麼」。這對視覺伺服與 SLAM 至關重要，第 29 章會詳細說明。

圖 8-4 機器人工作站的轉換樹

建議配圖：樹狀圖。根節點為 *world*；其下分支到 *robot_base*（固定變換，安裝時標定）與 *table*（工作台）。*robot_base* 下依序串接 *link1* ~ *link6*（各由關節角度決定，標註「隨 q 變化」），*link6* 下接 *tool0* 與 *tcp*（工具中心點，標註「換夾具需重設」）。*table* 下接 *workpiece* 與 *camera_link*，*camera_link* 下接 *camera_optical_frame*。每條邊標註變換名稱與是否為固定變換。

8.10 表示法之間的轉換

8.10.1 旋轉矩陣轉歐拉角

實務上經常需要在表示法之間轉換。由旋轉矩陣求 RPY 角（Z-Y-X 慣例）的公式為：

$$\begin{aligned}\beta &= \text{atan2}(-r_{31}, \sqrt{r_{11}^2 + r_{21}^2}) \\ \alpha &= \text{atan2}(r_{21}/\cos \beta, r_{11}/\cos \beta) \\ \gamma &= \text{atan2}(r_{32}/\cos \beta, r_{33}/\cos \beta)\end{aligned}$$

r_{ij} 表示旋轉矩陣的第 i 列第 j 行元素。務必使用 *atan2* 以正確處理象限。

注意這組公式在 $\cos \beta = 0$ （即 $\beta = \pm 90^\circ$ ）時會除以零——這正是萬向節鎖在數值上的表現。程式必須特別處理這個情形：通常的作法是偵測到 $|r_{31}|$ 接近 1 時，把 γ 設為 0，並將剩餘的旋轉全部歸給 α 。

8.10.2 旋轉矩陣轉四元數

由旋轉矩陣求四元數的基本關係為：

$$\begin{aligned}q_0 &= \frac{1}{2}\sqrt{1 + r_{11} + r_{22} + r_{33}} \\ q_1 &= (r_{32} - r_{23})/(4q_0), \quad q_2 = (r_{13} - r_{31})/(4q_0), \quad q_3 = (r_{21} - r_{12})/(4q_0)\end{aligned}$$

當 q_0 接近 0（旋轉角接近 180° ）時，此式會數值不穩定。

穩健的實作（Shepperd 方法）會先比較矩陣的跡（trace）與三個對角元素，選出四個分量中絕對值最大的那一個先計算，再由它推出其餘三個。這避免了除以極小數的問題。實務上直接使用成熟的函式庫（如 SciPy 的 Rotation 類別）即可，不需自行實作。

8.10.3 旋轉矩陣轉軸角

由旋轉矩陣求旋轉角與旋轉軸：

$$\theta = \arccos((\text{tr}(\mathbf{R}) - 1)/2), \quad \text{其中 } \text{tr}(\mathbf{R}) = r_{11} + r_{22} + r_{33}$$

旋轉角由矩陣的跡直接求得。

$$\mathbf{k} = 1/(2 \sin \theta) \cdot [r_{32} - r_{23}, \quad r_{13} - r_{31}, \quad r_{21} - r_{12}]^T$$

旋轉軸。當 $\sin \theta = 0$ ($\theta = 0$ 或 180°) 時失效，需特別處理。

這組公式有一個實用的副產品：它提供了衡量兩個姿態差異的方法。若要知道姿態 $\mathbf{R}_1$ 與 $\mathbf{R}_2$ 差多少，可計算 $\mathbf{R}_{\text{error}} = \mathbf{R}_1^T \cdot \mathbf{R}_2$ ，再取其旋轉角 θ 作為「姿態誤差的大小」。這個純量在第 10 章的逆運動學迭代與第 15 章的軌跡誤差評估中都會用到。

表 8-3 表示法轉換的注意事項

轉換方向	關鍵公式	數值陷阱	建議作法
RPY $\rightarrow$ 矩陣	$\mathbf{R}_z \cdot \mathbf{R}_y \cdot \mathbf{R}_x$ 連乘	無	直接計算
矩陣 $\rightarrow$ RPY	atan2 組合	$\beta = \pm 90^\circ$ 時除以零	偵測奇異點並特別處理
軸角 $\rightarrow$ 矩陣	Rodrigues 公式	$\theta = 0$ 時軸無定義	θ 極小時回傳單位矩陣
矩陣 $\rightarrow$ 軸角	由 $\text{tr}(\mathbf{R})$ 求 θ	$\theta = 0$ 或 180° 時軸不穩定	θ 接近 180° 時改用四元數路徑
四元數 $\rightarrow$ 矩陣	二次式展開	無 (最穩健)	直接計算
矩陣 $\rightarrow$ 四元數	由 $\text{tr}(\mathbf{R})$ 求 q_0	q_0 接近 0 時不穩定	採 Shepperd 方法選最大分量

8.11 常見錯誤與除錯技巧

8.11.1 三個最常見的錯誤來源

錯誤一：慣例不一致。這是實務上最大的陷阱。不同的軟體、廠商與文獻，可能使用不同的歐拉角軸序、不同的四元數分量順序（有些把純量部放第一位 $[w, x, y, z]$ ，有些放最後 $[x, y, z, w]$ ）、甚至不同的角度單位（度或弧度）。整合兩個系統時，若沒有明確確認慣例，會得到看似合理卻完全錯誤的結果。**建議：**任何介面文件的第一頁就寫清楚慣例。

錯誤二：上下標搞反。把 $\mathbf{A}_T \mathbf{B}$ 誤用為 $\mathbf{B}_T \mathbf{A}$ ，是初學者最常見的計算錯誤。症狀是機器人往完全相反或垂直的方向移動。**建議：**嚴格使用上下標記號並檢查消去律。

錯誤三：相乘順序顛倒。把左乘寫成右乘，或把 $\mathbf{R}_z \cdot \mathbf{R}_y \cdot \mathbf{R}_x$ 寫成 $\mathbf{R}_x \cdot \mathbf{R}_y \cdot \mathbf{R}_z$ 。症狀是小角

度時看起來正確（因為小角度旋轉近似可交換），大角度時完全錯亂——這使得它特別難以發現。
建議：用大角度（如 90 度）測試，錯誤會立刻放大顯現。

8.11.2 四個實用的除錯技巧

技巧一：用單位測試驗證恆等式。任何座標轉換程式碼寫完後，立刻驗證三件事： $T \cdot T^{-1} = I$ 、旋轉矩陣的正交性、以及「轉過去再轉回來」是否回到原點。這三個測試能抓出絕大多數的錯誤。

技巧二：用極端值測試。輸入 0 度應得到單位矩陣、輸入 90 度應得到座標軸的置換、輸入 180 度應得到明顯的翻轉。這些結果可以用心算驗證。

技巧三：視覺化。用 RViz、Matplotlib 或任何 3D 繪圖工具，把座標系畫出來。人眼對「座標軸指向奇怪的方向」極為敏感，一眼就能看出矩陣錯誤——這比盯著九個數字有效得多。

技巧四：檢查行列式。若某個「旋轉矩陣」的行列式是 -1 ，代表它包含鏡射，通常是因為某一軸的方向弄反了（例如誤用了左手座標系）。

◎ 一個真實的教訓

某團隊整合視覺系統與機器手臂時，發現夾取位置總是在 Y 方向偏移。檢查了兩週的相機校正參數都找不出問題，最後發現：相機驅動輸出的四元數是 $[x, y, z, w]$ 順序，而機器人 API 期待的是 $[w, x, y, z]$ 。四個數字順序錯了一位，造成的旋轉錯誤剛好在某些姿態下不明顯。這個故事的教訓是：先驗證慣例，再檢查數值。

8.12 由座標系邁向運動學

8.12.1 本章建立的工具

回顧本章，我們建立了一整套描述空間的語言：座標系與位置向量描述「在哪裡」；旋轉矩陣、歐拉角、軸角與四元數描述「朝向哪裡」；齊次變換矩陣把兩者合而為一；而變換的連乘與反矩陣，讓我們能在任意多個座標系之間自由換算。

這套工具的威力在於它的**通用性**。同一個 4×4 矩陣，可以描述機器手臂的一節連桿、相機與機器人的相對安裝、工件在輸送帶上的位置、甚至是無人機在世界地圖中的位姿。整本書後續的所有幾何運算，都建立在這個基礎之上。

8.12.2 接下來：讓變換動起來

本章的所有變換都是**靜態**的——描述某一瞬間的空間關係。但機器人的本質是運動，因此接下來三章要讓這些變換動起來。

第 9 章（正向運動學）：把每一個關節角度對應到一個變換矩陣，將它們連乘起來，就能算出「給定所有關節角度，末端執行器在空間中的哪裡」。第 8.7.1 節那條 ${}^0T_6 = {}^0T_1 \cdot {}^1T_2 \cdots {}^5T_6$ 的式子，將在第 9 章被完整展開。

第 10 章（逆向運動學）：反過來問「要讓末端到達指定位姿，各關節該轉幾度」。這是機器人實際作業時真正需要回答的問題，也困難得多——可能無解、可能有多組解。

第 11 章（雅可比矩陣）：由位置關係推進到**速度**關係。當關節以某個角速度轉動時，末端的線速度與角速度是多少？答案是一個矩陣，而它也將揭示奇異點的數學本質（第 12 章）。

換句話說：第 8 章教你**描述**空間，第 9 至 12 章教你**計算**運動。

完整計算例題

例題 8-1 書本旋轉問題

題目：一本書平放於桌面，其座標系 {B} 與世界座標系 {W} 完全重合。現將書向右平移 1 單位，接著繞原點（世界座標系的 Z 軸）逆時針旋轉 45 度。（a）求 WT_B 。（b）書上有一點在書的座標系中為 ${}^BP = [1, 1, 0]^T$ ，求其在世界座標系中的座標。（c）若世界座標系中有一點 ${}^WQ = [0, 1, 0]^T$ ，求其在書的座標系中的座標。

解（a）：注意題目說「繞世界座標系的 Z 軸旋轉」——這是相對於固定座標系，因此**左乘**（8.7.3 節）。先平移的變換為 T_{trans} （平移 $[1,0,0]$ ），再繞世界 Z 軸旋轉的變換為 T_{rot} ：

$${}^WT_B = T_{rot} \cdot T_{trans}$$

左乘：後執行且相對於固定座標系的變換寫在左邊。

以 $\cos 45^\circ = \sin 45^\circ = 0.7071$ 代入計算：

$${}^WT_B = \begin{bmatrix} 0.7071 & -0.7071 & 0 & 0.7071 \\ 0.7071 & 0.7071 & 0 & 0.7071 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

旋轉部分為繞 Z 軸 45 度；平移部分為原平移向量 $[1,0,0]$ 經旋轉後變成 $[0.7071, 0.7071, 0]$ 。

解（b）：直接代入 ${}^WP = {}^WT_B \cdot {}^BP$ 。以齊次座標 ${}^BP = [1,1,0,1]^T$ ：

$$\begin{aligned} {}^W P &= [0.7071(1) - 0.7071(1) + 0.7071, \quad 0.7071(1) + 0.7071(1) + 0.7071, \quad 0]^T \\ &= [0.7071, \quad 2.1213, \quad 0]^T \end{aligned}$$

書上該點在世界座標系中的位置。

解 (c)：需要反向轉換，用 8.8.1 節的公式求 ${}^B T_W = {}^W T_B^{-1}$ 。旋轉部分取轉置（即繞 Z 軸 -45 度），平移部分為 $-R^T \cdot p$ ：

$$-R^T \cdot p = -\begin{bmatrix} 0.7071 & 0.7071 \\ -0.7071 & 0.7071 \end{bmatrix} \begin{bmatrix} 0.7071 \\ 0 \end{bmatrix} = -\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

平移部分為 $[-1, 0, 0]^T$ ，符合直覺：從書的角度看，世界原點在它的左後方。

代入 ${}^B Q = {}^B T_W \cdot {}^W Q$ ，其中 ${}^W Q = [0, 1, 0, 1]^T$ ：

$$\begin{aligned} {}^B Q &= [0.7071(0) + 0.7071(1) - 1, \quad -0.7071(0) + 0.7071(1) + 0, \quad 0]^T \\ &= [-0.2929, \quad 0.7071, \quad 0]^T \end{aligned}$$

世界座標中的該點，在書的座標系中的位置。

驗算技巧：把 (b) 的答案 ${}^W P$ 代回 ${}^B T_W$ 計算，應該要得回 $[1, 1, 0]^T$ 。這種「來回轉換」的驗算能抓出絕大多數的計算錯誤。

例題 8-2 萬向節鎖的數值驗證

題目：以 Z-Y-X 的 RPY 慣例，已知 α （偏航）= 30° 、 β （俯仰）= 90° 、 γ （滾轉）= 20° 。

(a) 求旋轉矩陣。(b) 證明此時 α 與 γ 只以其差值影響結果。(c) 說明由此矩陣反求 RPY 角時會遇到什麼問題。

解 (a)：代入 $R = R_z(30^\circ) \cdot R_y(90^\circ) \cdot R_x(20^\circ)$ 。由於 $\beta = 90^\circ$ ， $\cos \beta = 0$ 、 $\sin \beta = 1$ ，計算後可得矩陣的第一行為 $[0, 0, -1]^T$ 。展開其餘元素會發現，所有元素都只包含 $\sin(\alpha - \gamma)$ 與 $\cos(\alpha - \gamma)$ 的組合：

$$\text{當 } \beta = 90^\circ : R = \begin{bmatrix} 0 & \sin(\alpha - \gamma) & \cos(\alpha - \gamma) \\ 0 & \cos(\alpha - \gamma) & -\sin(\alpha - \gamma) \\ -1 & 0 & 0 \end{bmatrix}$$

α 與 γ 僅以差值 $(\alpha - \gamma)$ 出現，兩個自由度退化為一個。

解 (b)：本例中 $\alpha - \gamma = 30^\circ - 20^\circ = 10^\circ$ 。若改為 $\alpha = 50^\circ$ 、 $\gamma = 40^\circ$ （差值同為 10° ），會得到完全相同的矩陣。這證明了無限多組 (α, γ) 對應到同一個姿態。

解 (c)：由此矩陣反求 RPY 時，8.10.1 節的公式中 $\cos \beta = 0$ ， α 與 γ 的計算都會除以零。程式必須偵測此情形：當 $|r_{31}|$ 接近 1 時，慣例上把 γ 設為 0，並將全部的旋轉歸給 α （此例中 $\alpha = 10^\circ$ 、 $\gamma = 0^\circ$ ）。雖然數值不同，但代表的姿態完全相同。

工程啟示：這正是無人機、太空船與人形機器人的姿態控制堅持使用四元數的原因。任何可能經歷大角度姿態變化的系統，都不應在內部使用歐拉角儲存姿態。

例題 8-3 工具中心點與工件座標系

題目：某機器人基座在世界座標系中的位置為 $[2, 1, 0]^T$ ，且繞 Z 軸旋轉了 90° 。經正向運動學計算，法蘭面相對於基座的變換 ${}^B T_F$ 之平移部分為 $[0.5, 0, 0.8]^T$ 、旋轉為單位矩陣。所裝夾具的 TCP 相對於法蘭面沿其 Z 軸前伸 0.15 公尺（無旋轉）。（a）求 TCP 在世界座標系中的位置。（b）若工件座標系原點在世界座標的 $[3, 2, 0.5]^T$ 且無旋轉，求 TCP 在工件座標系中的位置。

解（a）：依 8.9.1 節的連鎖式 ${}^W T_T = {}^W T_B \cdot {}^B T_F \cdot {}^F T_T$ 。

先求 ${}^B T_T = {}^B T_F \cdot {}^F T_T$ 。由於 ${}^B T_F$ 的旋轉為單位矩陣，TCP 相對基座的平移即為 $[0.5, 0, 0.8]^T + [0, 0, 0.15]^T = [0.5, 0, 0.95]^T$ 。

再左乘 ${}^W T_B$ 。基座繞 Z 軸轉 90° ，其旋轉矩陣把 $[x, y, z]$ 映射為 $[-y, x, z]$ ，因此：

$$\begin{aligned} {}^W P_T &= R_z(90^\circ) \cdot [0.5, 0, 0.95]^T + [2, 1, 0]^T \\ &= [0, 0.5, 0.95]^T + [2, 1, 0]^T = [2, 1.5, 0.95]^T \end{aligned}$$

TCP 在世界座標系中的位置。注意基座的旋轉使得「機器人自己的前方 0.5 公尺」變成了世界座標的 Y 方向。

解（b）：工件座標系無旋轉，因此 ${}^O T_W$ 的旋轉為單位矩陣、平移為 $-[3, 2, 0.5]^T$ ：

$${}^O P_T = {}^W P_T - {}^W P_{O_{or}^B} = [2, 1.5, 0.95]^T - [3, 2, 0.5]^T = [-1, -0.5, 0.45]^T$$

TCP 在工件座標系中的位置。

實務意義：程式若以工件座標系撰寫，當工件被移動到新位置時，只需更新 ${}^W T_O$ 一個矩陣，所有路徑點自動跟隨，不必逐點重教。這正是離線編程與快速換線的基礎（第 30 章）。

程式範例：座標轉換工具箱

以下 Python 程式實作本章的核心運算：基本旋轉矩陣、RPY 與矩陣互轉、齊次變換的建構、快速求逆、以及變換的連乘。程式最後以例題 8-1 與 8-2 驗證，並包含 8.11.2 節建議的單位測試。

```
import numpy as np

# ----- 基本旋轉矩陣 -----
def rot_x(t):
    c, s = np.cos(t), np.sin(t)
```

```

return np.array([[1, 0, 0], [0, c, -s], [0, s, c]])

def rot_y(t):
    c, s = np.cos(t), np.sin(t)
    return np.array([[c, 0, s], [0, 1, 0], [-s, 0, c]])

def rot_z(t):
    c, s = np.cos(t), np.sin(t)
    return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])

# ----- RPY ( Z-Y-X 固定軸 ) 與矩陣互轉 -----
def rpy_to_rot(roll, pitch, yaw):
    """先繞 X(roll)、再 Y(pitch)、最後 Z(yaw)；先做的寫右邊"""
    return rot_z(yaw) @ rot_y(pitch) @ rot_x(roll)

def rot_to_rpy(R):
    """回傳 (roll, pitch, yaw) · 含萬向節鎖處理"""
    if abs(R[2, 0]) > 0.99999:
        # 萬向節鎖：pitch = ±90°
        pitch = -np.sign(R[2, 0]) * np.pi / 2
        roll = 0.0
        yaw = np.arctan2(-R[0, 1], R[1, 1])
        print(" [警告] 偵測到萬向節鎖 · roll 已設為 0")
        return roll, pitch, yaw
    pitch = np.arctan2(-R[2, 0], np.hypot(R[0, 0], R[1, 0]))
    yaw = np.arctan2(R[1, 0], R[0, 0])
    roll = np.arctan2(R[2, 1], R[2, 2])
    return roll, pitch, yaw

# ----- 齊次變換 -----
def make_T(R=np.eye(3), p=np.zeros(3)):
    T = np.eye(4)
    T[:3, :3] = R
    T[:3, 3] = p
    return T

def inv_T(T):
    """利用結構快速求逆：R 取轉置、平移為 -R^T·p ( 8.8.1 節 ) """
    R, p = T[:3, :3], T[:3, 3]
    return make_T(R.T, -R.T @ p)

def apply_T(T, point):
    """把 3D 點以齊次座標作轉換"""
    return (T @ np.append(point, 1.0))[:3]

# ----- 驗證一：例題 8-1 -----
print("=== 例題 8-1：書本旋轉 ===")
T_trans = make_T(p=np.array([1, 0, 0]))
T_rot = make_T(R=rot_z(np.deg2rad(45)))
T_WB = T_rot @ T_trans

```

先平移

再繞世界 Z 軸旋轉

左乘 (相對固定座標系)


```

P_B = np.array([1, 1, 0])
print(f"(b) 書上點在世界座標: {apply_T(T_WB, P_B).round(4)}")

Q_W = np.array([0, 1, 0])
T_BW = inv_T(T_WB)
print(f"(c) 世界點在書座標: {apply_T(T_BW, Q_W).round(4)}")
print(f"    來回驗算 (應回到 [1,1,0]): "
      f"{apply_T(T_BW, apply_T(T_WB, P_B)).round(4)}")

# ----- 驗證二：例題 8-2 萬向節鎖 -----
print("\n=== 例題 8-2：萬向節鎖 ===")
R1 = rpy_to_rot(np.deg2rad(20), np.deg2rad(90), np.deg2rad(30))
R2 = rpy_to_rot(np.deg2rad(40), np.deg2rad(90), np.deg2rad(50))
print(f"(30°, 90°, 20°) 與 (50°, 90°, 40°) 是否相同矩陣: "
      f"{np.allclose(R1, R2)} ← 差值同為 10°")
r, p_, y = rot_to_rpy(R1)
print(f"反求 RPY: roll={np.rad2deg(r):.1f}° "
      f"pitch={np.rad2deg(p_):.1f}° yaw={np.rad2deg(y):.1f}°")

# ----- 單位測試 (8.11.2 節建議) -----
print("\n=== 單位測試 ===")
R = rpy_to_rot(0.3, -0.5, 1.2)
print(f"正交性  $R^T \cdot R = I$  : {np.allclose(R.T @ R, np.eye(3))}")
print(f"行列式  $\det(R) = 1$ : {np.isclose(np.linalg.det(R), 1.0)}")
T = make_T(R, np.array([1.5, -2.0, 0.7]))
print(f"求逆  $T \cdot T^{-1} = I$  : {np.allclose(T @ inv_T(T), np.eye(4))}")
print(f"RPY 來回轉換一致: "
      f"{np.allclose(rpy_to_rot(*rot_to_rpy(R)), R)}")

```

請試著修改：(一) 把例題 8-1 的左乘改成右乘 ($T_{\text{trans}} @ T_{\text{rot}}$)，觀察結果如何改變，並解釋其物理意義的差異。(二) 把 pitch 從 90° 改成 89.9° ，觀察 rot_to_rpy 的輸出是否劇烈變化——這就是接近奇異點時的數值不穩定。

實作：座標系視覺化與手動驗證

本章的實作分為紙上與電腦兩部分，目的是建立空間直覺——這是純看公式無法獲得的。

第一部分：紙上實驗 (不需電腦)

1. 取一本書，在封面上用便利貼標出三個軸：長邊為 X、短邊為 Y、垂直書面向上為 Z (右手系)。

2. 把書平放，此時書的座標系與桌面座標系重合。先繞 X 軸翻轉 90° 、再繞 Y 軸翻轉 90° ，記錄最終三軸的指向。
3. 把書放回原位，改為先繞 Y 軸、再繞 X 軸，記錄結果。比較兩者——親身驗證旋轉的不可交換性。
4. 手算這兩種順序的旋轉矩陣 ($R_y \cdot R_x$ 與 $R_x \cdot R_y$)，確認計算結果與實物觀察一致。

第二部分：電腦視覺化

1. 使用 Matplotlib 的 3D 繪圖，撰寫一個函式 `draw_frame(T, label)`，能將任一齊次變換矩陣所代表的座標系以三支彩色箭頭 (X 紅、Y 綠、Z 藍) 畫出。
2. 畫出世界座標系與例題 8-3 中的基座、法蘭、TCP、工件四個座標系，確認它們的相對位置符合直覺。
3. 製作動畫：讓某個座標系繞世界 Z 軸連續旋轉，同時另一個座標系繞自身 Z 軸旋轉，直觀比較左乘與右乘的差異。
4. (進階) 實作 SLERP 四元數內插與歐拉角線性內插，各自產生 20 個中間姿態並畫出，比較兩者路徑的平滑程度——你會清楚看到歐拉角內插的扭曲。
5. (若已安裝 ROS 2) 執行 `rviz2` 並發布幾個 TF 座標系，觀察 TF 樹如何自動計算任兩個座標系之間的關係。

國中小也看得懂：轉的順序不一樣，結果就不一樣

玩過「機器人指令遊戲」嗎？你對同學說「向前走三步，然後右轉」，跟說「右轉，然後向前走三步」，他會走到完全不同的地方對不對？這就是機器人數學裡最重要的一件事：**順序很重要！**

再試一個更好玩的實驗。拿一本課本平放在桌上，記住封面朝上、書名朝著你。第一次：先把書往前翻 90° (讓封面朝向你)，再往右翻 90° 。記住現在書長什麼樣子。第二次：把書放回原位，這次先往右翻 90° ，再往前翻 90° 。是不是完全不一樣？明明轉的角度都一樣，順序換了，結果就不同——這件事連大人都常常搞錯喔！

那機器人怎麼知道東西在哪裡呢？它需要一個「參考點」。就像你說「橡皮擦在鉛筆盒裡」，機器人要說「橡皮擦在我的攝影機前面 30 公分、右邊 10 公分」。如果換一個位置看，同樣一塊

橡皮擦，數字就完全不一樣了。所以機器人身上會有好多組「參考點」——底座一組、手臂一組、攝影機一組——它必須不停地把這些數字換來換去，才知道手該伸到哪裡。這種換算，就是這一章大哥哥大姐姐們在學的東西！

叫獸提醒與常見錯誤

- 誤區一：以為旋轉可以像位移一樣交換順序。 $R_1 \cdot R_2 \neq R_2 \cdot R_1$ ，先轉 X 再轉 Y 與先轉 Y 再轉 X 結果完全不同。
- 誤區二：不標明座標系。同一個點在不同座標系中數值完全不同，所有向量都應標註上標（如 ${}^A P$ ）。
- 誤區三：上下標搞反，把 ${}^A T_B$ 誤用為 ${}^B T_A$ 。用消去律檢查：相鄰的上下標應能兩兩抵消。
- 誤區四：左乘右乘混淆。相對於自身座標系的變換用右乘（動軸），相對於固定座標系的用左乘（定軸）。
- 誤區五：以為 R_y 的負號位置與 R_x 、 R_z 相同。 R_y 的負號在左下角，這是最常抄錯的一個公式。
- 誤區六：在內部儲存中使用歐拉角。任何可能經歷大角度變化的系統都應改用四元數，避免萬向節鎖。
- 誤區七：用一般矩陣求逆算齊次變換的反矩陣。應使用 8.8.1 節的封閉解，運算量低且數值穩定。
- 誤區八：反矩陣的平移部分直接取負。正確為 $-R^T \cdot p$ ，不是 $-p$ 。
- 誤區九：四元數分量順序不一致。有些系統用 $[w, x, y, z]$ 、有些用 $[x, y, z, w]$ ，整合時必須確認。
- 誤區十：對旋轉矩陣的元素做線性內插。結果不再是旋轉矩陣；姿態內插應使用四元數 SLERP。
- 誤區十一：忽略數值誤差累積。連續多次矩陣相乘後正交性會逐漸喪失，應定期正交化修正。
- 誤區十二：程式中用 atan 而非 atan2 。前者無法判斷象限，會導致角度計算在特定區域錯誤。

課堂討論

1. 阿波羅十三號的地面控制中心花數小時算出「繞開萬向節鎖」的旋轉路徑。若當時已有四元數技術並應用於導航平台，這個問題還會存在嗎？請區分「數學表示法的奇異點」與「機械結構的奇異點」。
2. 旋轉矩陣用九個數描述三個自由度，冗餘度很高；四元數用四個數；歐拉角只用三個。為什麼工程上不總是選最精簡的？請從奇異點、計算效率與直觀性三方面討論。
3. 為什麼「任何用三個數描述三維旋轉的方法都必然有奇異點」？試以「地球儀上的經緯度在南北極會發生什麼」作類比說明。
4. 工件座標系的設計讓「工件移動時不必重教路徑」。請設計一個實際情境說明這個好處，並討論若工件不只平移還有旋轉時，需要如何處理。
5. TF 系統儲存變換的「歷史紀錄」以處理感測器延遲。若沒有這個機制，視覺伺服會發生什麼問題？請以「相機在 t 時刻拍照、影像處理耗時 100 毫秒、手臂持續運動」的情境具體說明。

章末摘要

- 座標系由原點與三個互相垂直的單位軸構成；機器人學統一使用右手座標系與右手定則。
- 旋轉矩陣 A_R_B 的每一行，是 $\{B\}$ 的座標軸在 $\{A\}$ 中的方向；它有三種詮釋：座標軸描述、座標轉換、旋轉算子。
- 旋轉矩陣為正交矩陣 ($R^{-1} = R^T$) 且 $\det(R) = +1$ ，構成 $SO(3)$ 群；轉置即求逆的特性使反向轉換幾乎免費。
- 歐拉角 / RPY 用三個角度描述姿態，直觀但存在萬向節鎖——中間旋轉達 $\pm 90^\circ$ 時失去一個自由度。
- 任何以三個數描述三維旋轉的方法都必然有奇異點，這是數學的必然而非工程的缺陷。
- 軸角表示法源自歐拉旋轉定理（任何旋轉可化為繞單一軸的一次旋轉），由 Rodrigues 公式轉為矩陣。
- 單位四元數以半角形式編碼軸角，無奇異點、計算效率高、可用 SLERP 平滑內插，但完全不直觀。
- 實務策略：內部用四元數儲存計算、介面用歐拉角顯示、幾何推導用旋轉矩陣。

- 齊次座標把三維點升為四維（末位補 1），使平移能以矩陣乘法表達；齊次變換矩陣 4×4 統合旋轉與平移，構成 $SE(3)$ 。
- 變換連乘遵守消去律 $AT_C = AT_B \cdot BT_C$ ，這是正向運動學 ${}^0T_6 = {}^0T_1 \dots {}^5T_6$ 的基礎。
- 右乘代表相對自身座標系（動軸）、左乘代表相對固定座標系（定軸）；矩陣乘法不可交換，順序至關重要。
- 同一矩陣有「框架向前變換」與「座標向後變換」兩種對偶讀法；嚴格使用上下標記號可避免混淆。
- 齊次變換的反矩陣有封閉解：旋轉取轉置、平移為 $-R^T \cdot p$ ，切勿使用一般矩陣求逆。
- 機器人工作站的五個標準座標系：世界、基座、法蘭、工具（TCP）、工件；ROS 2 的 TF 以樹狀結構自動管理並處理時間戳。

觀念題與計算題

觀念題

1. 說明右手座標系與右手定則的內容，並解釋為何團隊必須統一慣例。
2. 旋轉矩陣的每一行代表什麼幾何意義？請由此說明為何它必為正交矩陣。
3. 為什麼 $R^{-1} = R^T$ 這條性質在工程上特別有價值？
4. 若某矩陣的行列式為 -1 ，代表什麼？在物理上可能對應到什麼錯誤？
5. 說明旋轉矩陣的三種詮釋，並區分「主動旋轉」與「被動旋轉」。
6. 解釋內旋與外旋的差異，並說明「繞固定軸 X-Y-Z」為何等價於「繞動軸 Z-Y-X」。
7. 萬向節鎖如何發生？為何說它是數學的必然而非工程缺陷？
8. 敘述歐拉旋轉定理，並說明軸角表示法的兩個奇異點各在何處。
9. 四元數為何使用半角？由此導致的「雙重覆蓋」現象在實作上需注意什麼？
10. 列舉四元數的三大優勢與一項致命缺點，並說明實務上的因應策略。
11. 齊次座標為何要在末位補 1？多出來的那一列有哪三個作用？
12. 說明變換連乘的消去律，並解釋為何它是檢查式子正確性的最快方法。
13. 區分左乘與右乘的物理意義，各舉一個機器人操作的實例。
14. 為何齊次變換反矩陣的平移部分是 $-R^T \cdot p$ 而非 $-p$ ？請說明其物理推理。

15. 列出機器人工作站的五個標準座標系及其用途，並說明以工件座標系撰寫程式的好處。

計算題

- 座標系 {B} 相對於 {A} 先繞 A 的 Z 軸旋轉 90° ，再沿 A 的 X 軸平移 3 單位。(a) 求 A^T_B 。(b) 點 $BP = [1, 2, 0]^T$ 在 {A} 中的座標為何？(c) 求 B^T_A 並驗證 $A^T_B \cdot B^T_A = I$ 。
- 已知 RPY 角為 $roll = 30^\circ$ 、 $pitch = 45^\circ$ 、 $yaw = 60^\circ$ (Z-Y-X 固定軸慣例)。(a) 寫出對應的旋轉矩陣 (可保留至小數點後三位)。(b) 求其行列式與驗證正交性。(c) 由此矩陣反求 RPY 角，確認與原值一致。
- 某旋轉繞單位軸 $k = [0, 0, 1]^T$ 轉 60° 。(a) 以 Rodrigues 公式求旋轉矩陣。(b) 求對應的單位四元數。(c) 驗證此四元數的長度為 1。(d) 求此四元數的共軛，並說明其代表的旋轉。
- 機器人基座在世界座標的位置為 $[1, 2, 0]^T$ 且繞 Z 軸轉 -90° 。正向運動學求得法蘭相對基座的變換：平移 $[0.6, 0.2, 0.9]^T$ 、旋轉為繞基座 Y 軸 90° 。工具相對法蘭沿其 Z 軸前伸 0.2 公尺。(a) 求 W^T_B 、 B^T_F 與 W_T 。(b) 求 TCP 在世界座標系中的位姿 (位置與旋轉矩陣)。(c) 若工件座標系位於世界座標 $[2, 3, 0.4]^T$ 且繞 Z 軸轉 45° ，求 TCP 在工件座標系中的位置。

動手做與專題延伸

本章實作：座標系關係表

為你手邊的機器人 (或附錄中的任一台) 建立完整的座標系清單與轉換關係表。這份表格是後續所有幾何計算的基礎文件。

表 8-4 座標系關係表 (前兩列為範例)

座標系	原點位置	軸向定義	父座標系	變換是否固定	取得方式
world	工作台左後角	X 向右、Y 向前、Z 向上	(根)	—	人為定義
base_link	機器人底座中心	X 為機器人正前方	world	固定	安裝後量測或標定
(自行填寫：各連桿、末端工具、相機、工件等)					

專題延伸

- 撰寫一個完整的座標轉換 Python 模組，包含四種表示法之間的所有轉換函式，並為每一個函式撰寫單元測試（驗證恆等式、極端值與來回轉換一致性）。這個模組將在第 9~11 章持續使用。
- 以 Matplotlib 製作互動式的萬向節鎖示範程式：以三個滑桿控制 RPY 角，即時繪出三個旋轉環與物體姿態。當 pitch 接近 90° 時，用紅色標示出兩軸重合的狀態，並顯示「自由度由 3 降為 2」的警告。
- 進階：實作一個簡易的 TF 系統。以字典儲存各座標系的父子關係與變換矩陣，並撰寫 `lookup_transform(from_frame, to_frame)` 函式，能自動在樹中尋找路徑（往上走時取反矩陣）並連乘出結果。以例題 8-3 的四個座標系測試，並與手算結果比對。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 2 章：空間描述與變換，本章的主要參考)。
- Lynch, K. M. and Park, F. C., Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press (第 3 章：剛體運動，含李群觀點)。
- Siciliano, B., Sciavicco, L., Villani, L. and Oriolo, G., Robotics: Modelling, Planning and Control, Springer (第 2 章：運動學基礎)。
- Murray, R. M., Li, Z. and Sastry, S. S., A Mathematical Introduction to Robotic Manipulation, CRC Press ($SO(3)$ 與 $SE(3)$ 的嚴謹數學處理)。
- Kuipers, J. B., Quaternions and Rotation Sequences, Princeton University Press (四元數的經典入門專書)。
- Shoemake, K., "Animating Rotation with Quaternion Curves," SIGGRAPH 1985 (SLERP 內插的原始論文)。
- Shepperd, S. W., "Quaternion from Rotation Matrix," Journal of Guidance and Control, 1978 (穩健的矩陣轉四元數演算法)。
- Diebel, J., "Representing Attitude: Euler Angles, Unit Quaternions, and Rotation Vectors," Stanford University, 2006 (各種姿態表示法的完整整理與轉換公式)。

- Toussaint, M., "3D Geometry Notes," TU Berlin 課程講義 (本書系參考的德式圖解講義) 。
- ROS 2 官方 TF2 教學文件 (docs.ros.org) : 轉換樹的概念、時間戳查詢與實作。

第 9 章

操作臂正向運動學

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

【章首情境圖建議】一支六軸工業機器人的「透視爆炸圖」：機構以半透明呈現，每一個關節處疊印一組彩色連桿座標系 (X 紅、 Y 綠、 Z 藍)，六組座標系之間以發光的箭頭串連成一條由基座通往夾爪的鏈；每個箭頭旁標註對應的變換矩陣符號 0T_1 、 1T_2 5T_6 。畫面下緣以一行公式收尾： ${}^0T_6 = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3 \cdot {}^3T_4 \cdot {}^4T_5 \cdot {}^5T_6$ 。標題文字：六次接力，從基座算到指尖。

叫獸開講 「知道每一顆馬達轉幾度，怎麼算出機器手指尖的位置？」

實驗室裡有一支學生自製的三軸機械手臂。叫獸請小翰示範：「請你讓夾爪停在距離基座正前方 35 公分、高度 20 公分的位置。」

小翰坐到電腦前，開始調整三個滑桿。第一軸轉了 30 度，手臂歪向一邊；再調第二軸 45 度，夾爪抬得太高；調第三軸補償，又太靠近了。他來回試了兩分鐘，滑桿調了三十幾次，夾爪終於停在大約對的位置——用尺一量，差了 2.8 公分。

「很好，」叫獸說，「現在請你告訴我，剛才那三個角度分別是多少？」小翰看了看螢幕：「28.4 度、52.1 度、負 71.6 度。」

「那麼，如果我現在把這三個數字報給你，你能不能在不看手臂的情況下，用紙筆算出夾爪會停在哪裡？」小翰愣住了。他能靠試誤把手臂移到想要的位置，卻算不出來為什麼是這三個數字。

叫獸拿起白板筆：「這就是正向運動學要解決的問題——**已知所有關節角度，求末端執行器在空間中的位姿**。聽起來好像沒什麼用，畢竟實務上我們通常是先知道目標位置、再反推角度。但如果你連正向都算不出來，逆向根本無從談起。」

他在白板上畫了一條由三段線構成的鏈：「而且，正向運動學遠不只是『算個位置』。它是機器人的**數位分身**：有了它，你可以在電腦裡模擬手臂的每一個動作而不用真的動它；你可以檢查某個姿態會不會撞到桌子；你可以計算工作空間的邊界；你可以把編碼器讀到的角度，即時換算成螢幕上顯示的座標。沒有正向運動學，機器人就只是一堆各轉各的馬達。」

「更關鍵的是，」叫獸在鏈條的每一段旁邊寫下一個矩陣符號，「上一章我們學了齊次變換，還記得那條消去律嗎？ $AT_C = AT_B \cdot BT_C$ 。今天我們要做的時候，就是把它用六次——把六個關節的變換串起來，從基座一路乘到夾爪。整個正向運動學，其實就是一連串的**矩陣連乘**。」

他轉過身：「難的不是乘法，是**怎麼替每一節連桿建立座標系**。同一支手臂，一百個工程師可能畫出一百種座標系配置，算出來的矩陣長得完全不一樣——但夾爪的位置必須是同一個。所以五十年前有兩位學者提出了一套規則，讓全世界的工程師畫出同一組座標系。這套規則，叫做 DH 參數法。」

本章為什麼重要

正向運動學是機器人學的第一個完整的計算成果。它把第 4 章的機構、第 6 章的編碼器讀值、第 8 章的座標變換全部串聯起來，回答一個核心問題：**這台機器人現在的手，究竟在哪裡？**

它的用途遠比表面看起來廣。在模擬與離線編程中，它讓你能在電腦裡預演整套動作；在碰撞檢測中，它提供每一節連桿的即時位置；在工作空間分析中，它讓你能掃描出機器人能到達的範圍；在示教與監控中，它把關節角度即時轉換成操作員看得懂的笛卡兒座標。而最重要的是——第 10 章的逆向運動學、第 11 章的雅可比矩陣、第 13 章的動力學，全部建立在正向運動學的基礎上。

本章也是全書第一次要求讀者親手完成完整的矩陣推導。這個過程可能會有點辛苦，但走過一遍之後，你對機器人的理解會產生質變——從「知道它會動」變成「知道它為什麼動到那裡」。

學習目標

- 能定義正向運動學問題，並說明其在模擬、監控與規劃中的用途。
- 能說明運動鏈的連桿與關節編號慣例。
- 能列出四個 DH 參數並說明各自的幾何意義。
- 能依標準 DH 規則為一支機械手臂建立連桿座標系並填寫參數表。
- 能寫出單一連桿的齊次變換矩陣並解釋其四個組成旋轉與平移。
- 能區分標準 DH 與修正式 DH 的差異，並說明混用的後果。
- 能推導平面 2R 與 3R 手臂的正向運動學封閉解。
- 能建立 SCARA 與六軸手臂的 DH 參數表並計算末端位姿。
- 能以 Python 實作任意串聯手臂的正向運動學。
- 能說明運動學模型的誤差來源與標定的必要性。

先備知識

第 8 章的全部內容，特別是齊次變換矩陣、變換的連乘與消去律、以及基本旋轉矩陣。第 4 章的連桿、關節與運動鏈概念。需要熟練的矩陣乘法與三角函數運算。

本章的推導量較大，建議準備紙筆邊讀邊算。若某個步驟卡住，多半是第 8 章的某個觀念還

不夠熟——回頭複習比硬撐下去有效率得多。

核心名詞中英對照

表 9-1 本章核心名詞

中文	英文	說明
正向運動學	Forward Kinematics, FK	由關節變數求末端位姿的問題
關節變數	Joint Variable	旋轉關節的角度或移動關節的位移
關節空間	Joint Space	以關節變數為座標的空間
任務空間 / 笛卡兒空間	Task Space / Cartesian Space	以末端位姿描述的空間
連桿座標系	Link Frame	固定在每一節連桿上的座標系
Denavit–Hartenberg 參數	DH Parameters	描述相鄰連桿關係的四個參數
連桿長度	Link Length, a	兩相鄰關節軸的公垂線長度
連桿扭轉角	Link Twist, α	兩相鄰關節軸之間的夾角
連桿偏移	Link Offset, d	沿關節軸方向的偏移量
關節角	Joint Angle, θ	繞關節軸的旋轉角
公垂線	Common Normal	與兩條空間直線都垂直的線段
標準 DH	Standard DH	座標系置於連桿遠端的慣例
修正式 DH	Modified DH	座標系置於連桿近端的慣例 (Craig 版本)
工具中心點	Tool Center Point, TCP	末端執行器的作用點
運動學標定	Kinematic Calibration	以量測修正運動學模型參數的程序

9.1 正向運動學問題

9.1.1 問題定義

正向運動學的問題陳述極為簡潔：給定一組關節變數，求末端執行器相對於基座的位姿。

已知： $\mathbf{q} = [q_1, q_2, \dots, q_n]^T$ 求： 0T_n (末端相對基座的齊次變換)

$\mathbf{q}$ 是關節變數向量，旋轉關節用角度、移動關節用位移。 0T_n 是一個 4×4 齊次變換矩陣，同時包含位置與姿態。

這個從關節空間到任務空間的映射，通常寫成一個函數：

$$x = f(q) \quad \text{其中 } x \in \mathbb{R}^6 \text{ (位置 3 + 姿態 3)}, q \in \mathbb{R}^n$$

第 8 章末提到的「運動圖」(kinematic map)就是這個 f 。它是一個非線性函數(因為含三角函數)，但重要的是——它總是有唯一解。

9.1.2 為什麼正向比逆向簡單

正向運動學有一個極為友善的性質：**給定關節角度，末端位姿必定存在且唯一**。這在物理上顯而易見——把每個馬達轉到指定角度，手臂只會擺出一種姿態，不可能同時出現在兩個地方。

這與第 10 章的逆向運動學形成鮮明對比。逆向問題可能無解(目標超出工作空間)、可能有多組解(手肘朝上或朝下)、甚至可能有無限多解(冗餘手臂)。正向運動學的唯一性，使它成為整個運動學體系中最穩固的基石——也是為什麼所有逆運動學的數值方法，內部都要反覆呼叫正向運動學。

9.1.3 五個實際用途

用途一：即時狀態顯示。編碼器每毫秒回報關節角度，經正向運動學換算成笛卡兒座標，讓操作員在示教器上看到「TCP 現在在 $X=520, Y=180, Z=340$ 」。

用途二：模擬與離線編程。在電腦中預演整套動作，驗證軌跡是否合理、節拍是否達標，不需佔用實體設備(第 30 章)。

用途三：碰撞檢測。正向運動學不只算末端，還能算出每一節連桿的位姿。有了它，才能判斷手臂會不會撞到自己、撞到治具或撞到人。

用途四：工作空間分析。掃描所有可能的關節組合，計算對應的末端位置，就能繪出第 4 章討論的工作空間形狀與邊界。

用途五：逆向運動學的基礎。數值型逆運動學(第 10 章)的每一次迭代，都需要用正向運動學計算「目前走到哪裡」，才能算出還差多遠。

圖 9-1 正向與逆向運動學的關係

建議配圖：中央為機器手臂圖示。左側方框「關節空間 $q = [\theta_1, \theta_2, \dots, \theta_n]$ 」，右側方框「任務空間 $x = (\text{位置}, \text{姿態})$ 」。由左至右的粗箭頭標註「正向運動學 $f(q)$ ：唯一解、直接計算」；由右至左的虛線箭頭標註「逆向運動學 $f^{-1}(x)$ ：可能無解 / 多解 / 無限多解」。下方註明：正向是連乘、逆向是解方程。

9.2 運動鏈與編號慣例

9.2.1 連桿與關節的編號

要把手臂拆解成一連串的變換，第一步是建立統一的編號。機器人學的慣例是：

連桿編號：固定不動的基座是連桿 0，往末端依序為連桿 1、2、.....、n。因此一支 n 軸手臂共有 n+1 根連桿（含基座）。

關節編號：連接連桿 i-1 與連桿 i 的關節，編號為關節 i。因此關節 1 連接基座與第一根連桿，關節 n 連接連桿 n-1 與連桿 n。

關節變數：關節 i 的變數記為 q_i 。若為旋轉關節， $q_i = \theta_i$ （角度）；若為移動關節， $q_i = d_i$ （位移）。

這個編號方式的好處是：關節 i 的運動，直接對應到連桿 i 的移動。從基座往末端走，每經過一個關節就前進一個編號。

9.2.2 連桿座標系的目標

接下來要在每一根連桿上「釘」一個座標系 $\{i\}$ ，使得連桿 i 相對於連桿 i-1 的變換 ${}^{i-1}T_i$ 可以只用少數幾個參數描述。理想上，我們希望：

- 每個變換只包含一個變數（該關節的 q_i ），其餘都是機構製造時就固定的常數。
- 參數數目盡可能少，方便建立模型與標定。
- 全世界的工程師依同一套規則，會得到同一組參數，方便交流與交叉驗證。

直覺上，描述兩個空間中的座標系關係需要六個參數（三平移三旋轉）。但 Denavit 與 Hartenberg 在 1955 年提出的巧妙作法，透過對座標系軸向的特殊選擇，把這六個參數壓縮到只需要四個。這就是 DH 參數法。

9.3 Denavit–Hartenberg 參數

9.3.1 核心洞見：沿著公垂線建立座標系

DH 法的關鍵洞見是：空間中任兩條直線（兩個關節軸），必定存在一條**公垂線**（同時垂直於兩者的線段），而且除了兩軸平行的特殊情形外，這條公垂線是唯一的。

若把座標系的軸向與這條公垂線對齊，就能消去兩個自由度，使六個參數降為四個。具體的規則是：

- **Z 軸沿著關節軸**： Z_i 軸與關節 $i+1$ 的轉軸重合（標準 DH）。
- **X 軸沿著公垂線**： X_i 軸沿著 Z_{i-1} 與 Z_i 的公垂線，方向由 Z_{i-1} 指向 Z_i 。
- **Y 軸由右手定則決定**： $Y_i = Z_i \times X_i$ 。

正因為 X 軸被強制與公垂線對齊，才使得「沿 Y 軸的平移」與「繞 Y 軸的旋轉」這兩個自由度自動歸零，四個參數就足夠了。

9.3.2 四個參數的幾何意義

依照上述規則建立座標系後，連桿 i 與 $i-1$ 的關係由四個參數完全決定：

表 9-2 DH 四參數的定義與意義

參數	符號	定義	幾何意義	何時為變數
連桿長度	a_{i-1}	沿 X_{i-1} 由 Z_{i-1} 量到 Z_i 的距離	公垂線的長度，即連桿實際長度	永遠是常數
連桿扭轉角	α_{i-1}	繞 X_{i-1} 由 Z_{i-1} 轉到 Z_i 的角度	相鄰兩關節軸的夾角	永遠是常數
連桿偏移	d_i	沿 Z_i 由 X_{i-1} 量到 X_i 的距離	沿關節軸方向的錯開量	移動關節時為變數
關節角	θ_i	繞 Z_i 由 X_{i-1} 轉到 X_i 的角度	關節的轉動角度	旋轉關節時為變數

記憶方法：**a** 與 **α** 描述「連桿本身」的幾何（兩個關節軸之間的關係，出廠就固定）；**d** 與 **θ** 描述「關節」的狀態（其中一個是變數）。這也是為什麼一個關節只貢獻一個自由度——四個參數中恰好只有一個會變。

圖 9-2 DH 四參數的幾何定義

建議配圖：三維透視圖。畫出兩條不平行、不相交的空間直線代表關節軸 Z_{i-1} 與 Z_i ，以及它們之間的公垂線（標為 X_{i-1} ）。標註四個參數： a_{i-1} （公垂線長度，沿 X_{i-1} ）、 α_{i-1} （兩軸夾角，繞 X_{i-1} ）、 d_i （沿 Z_i 的偏移，由 X_{i-1} 量到 X_i ）、 θ_i （繞 Z_i 的轉角）。用不同顏色區分「常數參數」（ a 、 α ，藍色）與「可變參數」（ d 、 θ ，紅色）。

9.3.3 建立座標系的六個步驟

實務上為一支手臂建立 DH 參數表，可依下列步驟進行：

1. **畫出機構簡圖**：以線段代表連桿、圓柱代表旋轉關節、方塊代表移動關節，標出所有關節軸的方向。

2. 標定所有 Z 軸： Z_i 沿關節 $i+1$ 的軸線（標準 DH）。方向可任選，但一旦選定就不能改。
3. 建立基座座標系 $\{0\}$ ： Z_0 沿關節 1 的軸線；原點與 X_0 方向可自由選擇，通常選在基座底面中心並讓 X_0 指向機器人正前方以簡化參數。
4. 逐一建立各連桿座標系：對 $i = 1$ 到 $n-1$ ，找出 Z_{i-1} 與 Z_i 的公垂線，原點置於公垂線與 Z_i 的交點， X_i 沿公垂線由 Z_{i-1} 指向 Z_i 。
5. 處理末端座標系 $\{n\}$ ： Z_n 通常沿末端執行器的接近方向（approach direction）， X_n 的選擇以使參數簡潔為原則。
6. 填寫參數表並驗算：對每一行讀出四個參數。驗算方法是把所有關節設為零位，用手（或模型）比對計算結果與實際幾何是否吻合。

◎ 三種特殊情形的處理

一、兩軸平行：公垂線不唯一，可自由選擇位置，通常選能讓 $d_i = 0$ 的那一條以簡化參數。二、兩軸相交：公垂線長度 $a = 0$ ， X 軸取兩軸外積的方向（ $X_i = \pm Z_{i-1} \times Z_i$ ）。三、兩軸重合：這是移動關節常見的情形， $a = 0$ 且 $\alpha = 0$ 。這三種情形在實際機器人上非常普遍——正因為工程師刻意讓軸平行或相交，才使機構容易製造且逆運動學有封閉解（第 10 章）。

9.4 連桿變換矩陣

9.4.1 由四個參數組成變換

有了四個 DH 參數，相鄰連桿之間的變換可以拆解成四個基本動作的連乘。以修正式 DH（Craig 慣例）為例，從座標系 $\{i-1\}$ 到 $\{i\}$ 的變換為：

$${}^{i-1}T_i = R_x(\alpha_{i-1}) \cdot D_x(a_{i-1}) \cdot R_z(\theta_i) \cdot D_z(d_i)$$

四個動作依序為：繞 X 轉 α 、沿 X 移 a 、繞 Z 轉 θ 、沿 Z 移 d 。前兩者跨越連桿、後兩者繞著關節。

這個順序有清楚的物理意義：先「走過連桿」（由前一個關節軸移動到這一個關節軸），再「轉動關節」。把四個矩陣乘開，得到標準的連桿變換矩陣：

$${}^{i-1}T_i = \begin{bmatrix} c\theta_i & -s\theta_i & 0 & a_{i-1} \\ s\theta_i c\alpha_{i-1} & c\theta_i c\alpha_{i-1} & -s\alpha_{i-1} & -s\alpha_{i-1}d_i \\ s\theta_i s\alpha_{i-1} & c\theta_i s\alpha_{i-1} & c\alpha_{i-1} & c\alpha_{i-1}d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

修正式 DH 的連桿變換矩陣。 c 與 s 分別代表 $\cos$ 與 $\sin$ 。這是本章最重要的一個公式，後續所有計算都由它出發。

9.4.2 讀懂這個矩陣

這個矩陣看似複雜，但結構清晰。右上角的平移欄 $[a_{i-1}, -s\alpha_{i-1}d_i, c\alpha_{i-1}d_i]^T$ 告訴我們：座標系原點的位移由 a 沿 X 、 d 沿 Z （再被 α 旋轉）所構成。左上角的 3×3 旋轉部分，則是 $R_x(\alpha) \cdot R_z(\theta)$ 的乘積。

特別注意第一列 $[c\theta, -s\theta, 0, a]$ ：它完全不含 α ，這是因為 X 軸的方向不受繞 X 軸旋轉的影響。這類結構性的觀察，是驗算矩陣有沒有抄錯的好方法。

9.4.3 串起整條運動鏈

有了每一節的變換，第 8 章的消去律就派上用場了：

$${}^0T_n = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3 \cdot \dots \cdot {}^{n-1}T_n$$

正向運動學的核心方程式。相鄰的上下標兩兩抵消，剩下由基座 0 到末端 n 。

再加上末端工具的固定變換，就得到 TCP 在基座座標系中的位姿：

$${}^0T_{TCP} = {}^0T_n \cdot {}^nT_{TCP}$$

${}^nT_{TCP}$ 由工具幾何決定，換夾具時需重新設定（第 8.9.1 節）。

至此，正向運動學的完整解答已經呈現：把每個關節角度代入各自的連桿變換矩陣，然後全部乘起來。剩下的工作，就是把它算對——這正是接下來三節要示範的。

9.4.4 由矩陣讀出位置與姿態

計算出 0T_n 之後，如何取出我們要的資訊？依第 8.6.2 節的結構：

$${}^0T_n = \begin{bmatrix} \hat{n} & \hat{o} & \hat{a} & p \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

右上 3×1 的 p 就是末端位置；左上 3×3 的三個行向量 $\hat{n}$ 、 $\hat{o}$ 、 $\hat{a}$ 分別是末端座標系的 X 、 Y 、 Z 軸方向。

在機器人學傳統上，這三個行向量有專門的名稱： $\hat{a}$ (approach) 是接近向量，通常沿夾爪的伸入方向； $\hat{o}$ (orientation) 是姿態向量，沿夾爪開合的方向； $\hat{n}$ (normal) 是法線向量，由 $\hat{o} \times \hat{a}$ 決定。工業機器人的示教器上顯示的姿態，往往就是由這三個向量換算出的 RPY 角（第 8.10.1 節）。

9.5 標準 DH 與修正式 DH

9.5.1 兩種慣例的差異

DH 參數法有兩種常見的慣例，兩者的差別在於座標系放在連桿的哪一端，以及四個基本動

作的順序。這是機器人學中一個惱人但必須弄清楚的分歧。

標準 DH (Denavit-Hartenberg 原始版本)：座標系 $\{i\}$ 放在連桿 i 的遠端 (靠近下一個關節)， Z_i 軸沿關節 $i+1$ 的軸線。變換順序為：

$${}^{i-1}T_i = R_z(\theta_i) \cdot D_z(d_i) \cdot D_x(a_i) \cdot R_x(\alpha_i)$$

標準 DH：先轉關節、再走連桿。注意參數的下標都是 i 。

修正式 DH (Craig 版本)：座標系 $\{i\}$ 放在連桿 i 的近端 (靠近前一個關節)， Z_i 軸沿關節 i 的軸線。變換順序為：

$${}^{i-1}T_i = R_x(\alpha_{i-1}) \cdot D_x(a_{i-1}) \cdot R_z(\theta_i) \cdot D_z(d_i)$$

修正式 DH：先走連桿、再轉關節。注意 a 與 α 的下標是 $i-1$ 。

9.5.2 該用哪一種

兩種慣例在數學上完全等價，都能得到正確的末端位姿，但參數值不同、座標系位置不同。實務上的選擇準則是：

修正式 DH 的優點：座標系 $\{i\}$ 的 Z 軸就是關節 i 的轉軸，這使得「第 i 個關節的變數對應到第 i 個座標系」的關係更直觀。此外，在動力學推導中 (第 14 章的 Newton-Euler 遞迴)，把座標系放在連桿近端更為自然，因為連桿的質心與慣量都在該座標系中描述。本書與 Craig 教科書一致，採用修正式 DH。

標準 DH 的優點：歷史悠久，許多經典文獻與商用軟體使用此慣例。若讀者需要與既有的參數表對接，必須確認對方使用哪一種。

◎ 最重要的一條警告

絕對不可混用兩種慣例。用標準 DH 的參數表配上修正式 DH 的變換矩陣公式，會得到一個看似合理、實則完全錯誤的結果——而且在零位附近誤差很小，不易察覺，直到手臂大幅運動時才會暴露。實務上的建議是：任何 DH 參數表都必須在標題明確註明「標準」或「修正式」，並在文件中寫出所使用的變換矩陣公式。這是機器人專案交接時最常出問題的環節之一。

9.5.3 DH 法的限制

DH 參數法雖然優雅，但有三個限制值得知道。

限制一：平行軸的奇異性。當相鄰兩關節軸接近平行時，公垂線的位置變得極為敏感——

軸向的微小偏差會導致公垂線位置大幅跳動。這使得 DH 參數在標定時對這類機構的數值穩定性較差。實務上會改用其他參數化方式（如 Hayati 模型）來處理。

限制二：只適用於串聯機構。 DH 法建立在「一根連桿接一根連桿」的鏈狀結構上。對第 4 章的並聯機器人（Delta、Stewart 平台），必須改用其他方法（通常是對每條支鏈分別建模，再加上閉環約束方程）。

限制三：不夠直觀。現代的機器人軟體（如 ROS 2 的 URDF）多半直接以「父連桿到子連桿的完整 6 參數變換」來描述，不再要求座標系必須沿公垂線放置。這樣參數雖多了兩個，但工程師可以把座標系放在任何方便的位置（例如 CAD 模型的自然原點），大幅降低建模門檻。DH 法在今日主要的價值，是在**理論推導與教學**中提供最精簡的表達。

9.6 平面 2R 與 3R 手臂

9.6.1 平面 2R 手臂的正向運動學

最簡單也最重要的例子是平面兩連桿手臂：兩個旋轉關節、軸線互相平行且垂直於運動平面，連桿長度分別為 L_1 與 L_2 。

這個機構的正向運動學可以用純幾何推導。第一節連桿末端（即第二個關節）的位置為：

$$x_1 = L_1 \cos \theta_1, \quad y_1 = L_1 \sin \theta_1$$

第一節末端位置。

第二節連桿相對於水平線的角度是 $\theta_1 + \theta_2$ （因為 θ_2 是相對於第一節的角度），因此末端位置為：

$$\begin{aligned} x &= L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ y &= L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{aligned}$$

平面 2R 手臂的正向運動學。這是全機器人學最常被引用的一組方程式，請務必熟記。

末端的姿態（連桿 2 相對於基座的角度）則為 $\varphi = \theta_1 + \theta_2$ 。因此這個機構有三個輸出量（ x 、 y 、 φ ）但只有兩個自由度——這說明它無法獨立控制位置與姿態，這一點在第 10 章討論逆運動學時會再回來。

9.6.2 以 DH 法驗證

同樣的結果也可用 DH 法得到。建立修正式 DH 參數表：

表 9-3 平面 2R 手臂的修正式 DH 參數

i	α_{i-1}	a_{i-1}	d_i	θ_i
1	0	0	0	θ_1 (變數)
2	0	L_1	0	θ_2 (變數)
3 (末端)	0	L_2	0	0

代入 9.4.1 節的連桿變換矩陣 (因所有 $\alpha = 0$ ，矩陣大幅簡化)，連乘三個矩陣後可得：

$${}^0T_3 = \begin{bmatrix} c_{12} & -s_{12} & 0 & L_1 c_1 + L_2 c_{12} \\ s_{12} & c_{12} & 0 & L_1 s_1 + L_2 s_{12} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

其中 $c_1 = \cos \theta_1$ 、 $c_{12} = \cos(\theta_1 + \theta_2)$ ， s 同理。右上角的平移欄正是幾何推導的結果，兩種方法完全一致。

這個驗證很有價值：它證明了 DH 法這套看似繁瑣的規則，得到的答案與直覺的幾何推導完全相同。對於簡單機構，幾何法更快；但對於六軸手臂這種空間機構，幾何推導幾乎不可能，DH 法的系統性就變得不可或缺。

9.6.3 平面 3R 手臂

再加一節連桿，就成為平面 3R 手臂。依同樣的邏輯推展：

$$\begin{aligned} x &= L_1 c_1 + L_2 c_{12} + L_3 c_{123} \\ y &= L_1 s_1 + L_2 s_{12} + L_3 s_{123} \\ \phi &= \theta_1 + \theta_2 + \theta_3 \end{aligned}$$

其中 $c_{123} = \cos(\theta_1 + \theta_2 + \theta_3)$ 。三個自由度對應三個輸出 (x, y, ϕ)，恰好可獨立控制平面位置與姿態。

平面 3R 手臂是一個重要的教學範例，因為它是最小的「完整」機器人——在平面世界中，剛體有三個自由度 (x, y, ϕ)，而它恰好有三個關節。這使它成為研究逆運動學多解問題 (第 10 章) 與奇異點 (第 12 章) 的標準範例。

圖 9-3 平面 2R 與 3R 手臂的幾何關係

建議配圖：左圖為平面 2R 手臂——基座在原點，第一節長 L_1 與 X 軸夾角 θ_1 ，第二節長 L_2 與第一節夾角 θ_2 ，標出末端位置 (x, y) 及其分解的四個投影分量 ($L_1 \cos \theta_1$ 、 $L_2 \cos(\theta_1 + \theta_2)$ 等)。右圖為 3R 手臂，另標出末端姿態角 $\phi = \theta_1 + \theta_2 + \theta_3$ 。兩圖下方各附正向運動學方程式。

9.7 SCARA 機器人

9.7.1 構型回顧與座標系建立

如第 4.8.1 節所述，SCARA 是 RRPR 構型：兩個垂直軸的旋轉關節提供水平面運動、一個垂

直移動關節提供高度、末端再一個旋轉關節控制方向。

由於所有旋轉軸都平行（皆為垂直方向），移動軸也與之平行，這使得所有的 α 參數皆為 0（或 180° ），DH 參數表格外簡潔。設第一臂長 a_1 、第二臂長 a_2 、基座高度 d_1 ：

表 9-4 SCARA 機器人的修正式 DH 參數

i	α_{i-1}	a_{i-1}	d_i	θ_i	關節類型
1	0	0	d_1	θ_1 (變數)	旋轉
2	0	a_1	0	θ_2 (變數)	旋轉
3	0	a_2	d_3 (變數)	0	移動
4	180°	0	d_4	θ_4 (變數)	旋轉

第 4 行的 $\alpha = 180^\circ$ 是因為 SCARA 的第四軸（末端旋轉）習慣上讓 Z 軸朝下（工具向下作業），因此相對於前一個座標系翻轉了 180° 。

9.7.2 正向運動學結果

連乘四個變換矩陣後，可得末端位姿：

$$\begin{aligned}x &= a_1 c_1 + a_2 c_{12} \\y &= a_1 s_1 + a_2 s_{12} \\z &= d_1 - d_3 \\\phi &= \theta_1 + \theta_2 - \theta_4\end{aligned}$$

SCARA 的正向運動學。注意 x 、 y 與平面 $2R$ 完全相同（因為水平運動確實就是 $2R$ ）， z 由移動關節單獨決定， ϕ 為末端繞垂直軸的方向。

這組方程式展現了 SCARA 構型的優雅之處：**運動被完全解耦**。水平位置只由 θ_1 、 θ_2 決定；高度只由 d_3 決定；末端方向只由三者的組合決定。這種解耦使得逆向運動學有簡潔的封閉解，控制也更為直觀——這正是它在電子組裝線上如此普及的數學原因。

◎ 解耦的工程價值

當運動學方程式能夠解耦時，好處是全面的：逆運動學有封閉解（計算快、無迭代、必定收斂）、控制器可以分軸獨立設計、故障診斷更容易定位、而且操作員的直覺與機器行為一致（「我要它高一點」就只調 Z 軸）。第 4 章說 SCARA 是「剛剛好就好」的最佳範例，本節提供了數學上的證據。

9.8 六軸工業機器人

9.8.1 典型構型與球形手腕

絕大多數的六軸工業機器人採用相似的構型：前三軸（腰、肩、肘）決定末端**位置**，後三軸（手腕的三個旋轉軸）決定末端**姿態**。而後三軸的軸線通常**交會於同一點**，稱為**球形手腕**（spherical wrist）。

球形手腕的設計不是為了機構方便，而是為了**數學**。當後三軸交於一點時，逆向運動學可以被拆解成兩個獨立的子問題：先由手腕中心點的位置解出前三軸、再由姿態解出後三軸。這稱為**運動學解耦**（kinematic decoupling），它使得六軸手臂的逆運動學存在封閉解。若沒有球形手腕，逆運動學就只能用數值迭代求解（第 10 章）。

這是機構設計服務於演算法的經典案例：工程師刻意犧牲一些機構自由度（讓三軸交於一點會增加設計難度），換取計算上的巨大便利。

9.8.2 PUMA 型手臂的 DH 參數

以經典的 PUMA 560 構型為例（許多現代六軸手臂沿用此架構），其修正式 DH 參數表為：

表 9-5 六軸手臂（PUMA 型）的修正式 DH 參數

i	α_{i-1}	a_{i-1}	d_i	θ_i	說明
1	0	0	0	θ_1	腰部旋轉（繞垂直軸）
2	-90°	0	0	θ_2	肩部俯仰
3	0	a_2	d_3	θ_3	肘部俯仰
4	-90°	a_3	d_4	θ_4	手腕旋轉 1
5	90°	0	0	θ_5	手腕俯仰
6	-90°	0	0	θ_6	手腕旋轉 2

觀察這張表可以學到幾件事。第一， α 只出現 0、 $\pm 90^\circ$ 三種值——這不是巧合，而是因為工業機器人的關節軸總是互相平行或垂直，這樣才容易製造且容易分析。第二， a_2 是最大的參數（通常數百公釐），它就是上臂的長度，決定了機器人的工作半徑。第三，後三軸的 a 與 d 大多為 0，這正是球形手腕「三軸交於一點」的數學表現。

9.8.3 連乘的實務作法

六個 4×4 矩陣連乘，手算約需一小時且極易出錯。實務上有三種作法：

作法一：符號運算軟體。用 SymPy、Mathematica 或 MATLAB 的 Symbolic Toolbox，可以得到含 $\theta_1 \sim \theta_6$ 的解析表達式。這對理論分析與推導逆運動學公式很有用，但表達式會非常龐大（展開後可能數百項）。

作法二：數值計算。給定具體的關節角度，直接用 NumPy 做矩陣連乘，毫秒內完成。這是實務控制中的標準作法，本章的程式範例即採此法。

作法三：現成函式庫。使用 Robotics Toolbox、Pinocchio、KDL 等函式庫，只需輸入 DH 參數或 URDF 檔案即可。這是工程專案的最佳選擇——這些函式庫經過大量驗證，且效能經過最佳化。

值得強調的是：**理解推導過程與使用現成工具並不衝突**。你需要理解原理才能除錯（當結果不對時，知道該檢查哪個環節）、才能判斷工具的輸出是否合理、也才能在遇到非標準機構時自行建模。

圖 9-4 六軸工業機器人的座標系配置

建議配圖：六軸手臂側視圖與立體圖並列。在每一個關節處畫出該連桿座標系的三軸（ X 紅、 Y 綠、 Z 藍），並標註各軸的 DH 參數（ a_2 、 a_3 、 d_3 、 d_4 等實際尺寸）。特別以虛線圓圈標出後三軸交會的「手腕中心點」，並註明「球形手腕使逆運動學可解耦」。

9.9 運動學誤差與標定

9.9.1 理論與現實的落差

正向運動學給出的是**理論位置**。把它與實際位置比對，往往會發現數公釐甚至數公分的差距。這個落差來自五個誤差源：

誤差一：製造公差。連桿的實際長度與圖面標稱值有差異，通常在 ± 0.1 mm 等級。對 $a_2 = 500$ mm 的上臂，0.1 mm 的誤差在末端就是 0.1 mm——看似不大，但六個關節的誤差會累積。

誤差二：裝配誤差。關節軸線的實際方向與理論方向有夾角偏差。這是最嚴重的一類：0.1 度的軸線偏斜，在 500 mm 外會造成約 0.87 mm 的位置誤差（ $500 \times \tan 0.1^\circ \approx 0.87$ ）。這正是第 3.1.3 節提到的「誤差沿運動鏈累積放大」。

誤差三：關節零位偏移。編碼器的零點與理論零位不完全一致。這通常是最大的單一誤差源，但也最容易修正（只需在軟體中加一個偏移量）。

誤差四：彈性變形。第 4.1.2 節的剛體假設在此失效。手臂在重力與負載下會彎曲，末端下垂可達數公釐；且變形量隨姿態與負載而變，難以用固定參數補償。

誤差五：溫度效應。鋁的線膨脹係數約 $23 \times 10^{-6} / ^\circ\text{C}$ ，一支 1 公尺長的手臂在溫度變化 10°C 時會伸長約 0.23 mm。這是精密應用中不可忽略的因素，也是為什麼高精度工作站需要恆溫環境。

9.9.2 重複精度與絕對精度

這些誤差解釋了第 4.11.1 節提到的現象：**重複精度遠優於絕對精度**。

重複精度 (repeatability) 是「回到同一組關節角度時，末端位置的一致程度」。由於上述誤差都是系統性的 (同樣的角度總是產生同樣的誤差)，它們在重複定位時會互相抵消，因此重複精度可達 $\pm 0.02 \text{ mm}$ 。

絕對精度 (absolute accuracy) 是「到達程式指定的笛卡兒座標時，實際位置與指令值的差距」。這時所有的模型誤差都會直接顯現，典型值在 ± 0.5 到 $\pm 2 \text{ mm}$ ——比重複精度差了一到兩個數量級。

這個差距決定了兩種截然不同的使用方式。**示教編程**只依賴重複精度：操作員手動把機器人移到正確位置並記錄關節角度，之後重播即可，模型誤差完全不影響。**離線編程與視覺導引**則依賴絕對精度：程式給出的是笛卡兒座標，機器人必須靠運動學模型換算成關節角度——模型有多準，定位就有多準。

9.9.3 運動學標定

運動學標定的目的，是用實際量測資料反推出**真實的 DH 參數**，取代圖面上的理論值。標準流程為：

- 1. 量測：**用雷射追蹤儀、球桿儀或相機系統，量測數十到數百個不同姿態下的實際末端位置。
- 2. 建模：**建立誤差模型，把 DH 參數的偏差視為未知數，寫出「實際位置 = $f(\text{關節角}, \text{DH 參數} + \text{參數偏差})$ 」。
- 3. 辨識：**以最小平方或非線性最佳化，求出使預測值與量測值差距最小的參數偏差。這在數學上與第 10 章的數值逆運動學屬於同一類問題。
- 4. 驗證：**用另一組未參與辨識的姿態驗證標定結果，避免過度擬合。

5. 寫入：把修正後的參數寫入控制器，取代原有的理論參數。

經過標定的機器人，絕對精度可由 $\pm 1 \text{ mm}$ 提升到 $\pm 0.2 \text{ mm}$ 以內，這對離線編程與視覺導引作業是決定性的改善。近年許多廠商已將標定納入出廠程序，並提供現場再標定的工具。

9.10 由正向邁向逆向

9.10.1 本章的成果

走完本章，你已經掌握了機器人學的第一項核心計算能力。給定任何一支串聯機械手臂與一組關節角度，你能夠：建立連桿座標系、填寫 DH 參數表、寫出各節的變換矩陣、連乘求得末端位姿、並由結果矩陣讀出位置與姿態。

更重要的是，你理解了一件事：**機器人的運動學，本質上是一連串座標系的接力**。從基座出發，每經過一個關節就轉換一次座標系，一路傳遞到夾爪。第 8 章的齊次變換不再是抽象的數學，而是描述真實機構的工具。

9.10.2 接下來的三個挑戰

第 10 章：逆向運動學。現在我們能由關節角求位置，但實務上需要的是反過來——「我要夾爪到那個位置，各關節該轉幾度？」這個問題困難得多：可能無解、可能有八組解、也可能有無限多解。我們將學習幾何解法、代數解法與數值迭代法，並理解為什麼球形手腕如此重要。

第 11 章：雅可比矩陣。本章處理的是靜態的位置關係。當關節開始以某個速度轉動時，末端會以多快的速度移動？答案是一個矩陣——雅可比矩陣。它不只連結速度，也連結力與力矩，是控制與力覺的橋樑。

第 12 章：奇異點與冗餘。當雅可比矩陣失去滿秩時，機器人會失去某個方向的運動能力。本章 9.8.1 節提到的手臂完全伸直、以及第 8 章的萬向節鎖，都將在那裡得到統一的數學解釋。

換句話說：第 9 章讓你算出手在哪裡，第 10 章讓你決定手該怎麼擺，第 11、12 章讓你知道手能怎麼動。

完整計算例題

例題 9-1 平面 2R 手臂的正向運動學

題目：某平面 2R 手臂， $L_1 = 300 \text{ mm}$ 、 $L_2 = 200 \text{ mm}$ 。(a) 當 $\theta_1 = 30^\circ$ 、 $\theta_2 = 45^\circ$ 時，

求末端位置與姿態。(b) 當 $\theta_1 = 0^\circ$ 、 $\theta_2 = 0^\circ$ 時，求末端位置，並說明此姿態的特殊意義。(c) 求此手臂末端可達的最大與最小距離。

解 (a)：直接代入 9.6.1 節的公式。先算 $\theta_1 + \theta_2 = 75^\circ$ 。

$$x = 300 \cos 30^\circ + 200 \cos 75^\circ = 300(0.8660) + 200(0.2588) = 259.81 + 51.76 = 311.57 \text{ mm}$$

末端 X 座標。

$$y = 300 \sin 30^\circ + 200 \sin 75^\circ = 300(0.5) + 200(0.9659) = 150.00 + 193.19 = 343.19 \text{ mm}$$

末端 Y 座標。

末端姿態 $\varphi = \theta_1 + \theta_2 = 75^\circ$ 。因此末端位姿為 $(311.57, 343.19, 75^\circ)$ ，距離基座 $\sqrt{(311.57^2 + 343.19^2)} \approx 463.5 \text{ mm}$ 。

解 (b)： $\theta_1 = \theta_2 = 0^\circ$ 時， $x = 300 + 200 = 500 \text{ mm}$ 、 $y = 0$ 。此時手臂**完全伸直**，末端位於工作空間的外邊界。這個姿態極為特殊：它是第 4.10.3 節提到的邊界奇異點，末端在徑向已無法再前進，且逆向運動學在此處會失去唯一性——這一點將在第 12 章詳細分析。

解 (c)：最大距離發生在完全伸直時： $r_{\max} = L_1 + L_2 = 500 \text{ mm}$ 。最小距離發生在完全折疊時 ($\theta_2 = 180^\circ$)：

$$r_{\min} = |L_1 - L_2| = |300 - 200| = 100 \text{ mm}$$

最小可達距離。工作空間是一個內徑 100 mm、外徑 500 mm 的圓環（不考慮關節極限）。

工程觀察：若兩連桿等長 ($L_1 = L_2$)，則 $r_{\min} = 0$ ，工作空間是一個完整的圓，中間沒有「洞」。這正是許多教學型手臂採用等長連桿的原因——工作空間最完整。但代價是完全折疊的姿態在實體上會發生自我碰撞，因此實際仍有內部限制。

例題 9-2 SCARA 的正向運動學與工作空間

題目：某 SCARA 機器人 $a_1 = 400 \text{ mm}$ 、 $a_2 = 300 \text{ mm}$ 、 $d_1 = 500 \text{ mm}$ ，Z 軸行程 d_3 範圍為 0~150 mm。(a) 當 $\theta_1 = 60^\circ$ 、 $\theta_2 = -30^\circ$ 、 $d_3 = 80 \text{ mm}$ 、 $\theta_4 = 20^\circ$ 時，求 TCP 位姿。(b) 求工作空間的水平環狀範圍與垂直範圍。(c) 若要在半徑 750 mm 處作業，是否可行？

解 (a)：代入 9.7.2 節的公式。 $\theta_1 + \theta_2 = 30^\circ$ 。

$$x = 400 \cos 60^\circ + 300 \cos 30^\circ = 400(0.5) + 300(0.8660) = 200.00 + 259.81 = 459.81 \text{ mm}$$

$$y = 400 \sin 60^\circ + 300 \sin 30^\circ = 400(0.8660) + 300(0.5) = 346.41 + 150.00 = 496.41 \text{ mm}$$

水平位置。

$$z = d_1 - d_3 = 500 - 80 = 420 \text{ mm}$$

$$\phi = \theta_1 + \theta_2 - \theta_4 = 60^\circ + (-30^\circ) - 20^\circ = 10^\circ$$

高度與末端方向。

TCP 位姿為 (459.81, 496.41, 420, $\phi=10^\circ$)，水平距離基座 $\sqrt{(459.81^2 + 496.41^2)} \approx 676.5$ mm。

解 (b)：水平方向的可達範圍為內徑 $|400 - 300| = 100$ mm、外徑 $400 + 300 = 700$ mm 的環狀區域。垂直方向為 z 由 $500 - 150 = 350$ mm 到 $500 - 0 = 500$ mm，行程 150 mm。整體工作空間是一個扁平的中空圓柱體。

解 (c)：750 mm 超過最大可達半徑 700 mm，不可行。且即使是 700 mm 也是完全伸直的邊界奇異點，實務上應保留至少 10% 餘裕（第 4.10.3 節），因此建議的最遠作業半徑約為 630 mm。若必須在 750 mm 處作業，解決方案有三：更換更長臂展的機型、將機器人加裝在第七軸行走軸上、或重新配置工作站佈局讓工件更靠近。

例題 9-3 三軸空間手臂的 DH 建模與計算

題目：某三軸手臂構型為：第一軸繞垂直 Z 軸旋轉（腰）、第二軸繞水平軸旋轉（肩）、第三軸繞平行於第二軸的水平軸旋轉（肘）。上臂長 $a_2 = 400$ mm、前臂長 $a_3 = 350$ mm，基座高 $d_1 = 300$ mm。（a）建立修正式 DH 參數表。（b）當 $\theta_1 = 45^\circ$ 、 $\theta_2 = 30^\circ$ 、 $\theta_3 = -60^\circ$ 時，求末端位置。

解 (a)：依 9.3.3 節步驟。 Z_0 沿第一軸（垂直）， Z_1 沿第二軸（水平），兩軸垂直相交於基座頂端，故 $\alpha_0 = \dots$ 需注意：由 Z_0 轉到 Z_1 需繞 X_0 轉 -90° ，且兩軸相交故 $a_0 = 0$ 。第二、三軸平行，故 $\alpha_1 = 0$ 、 $a_1 = a_2$ （上臂長）。

表 9-6 三軸手臂的修正式 DH 參數

i	α_{i-1}	a_{i-1}	d_i	θ_i
1	0	0	$d_1 = 300$	θ_1
2	-90°	0	0	θ_2
3	0	$a_2 = 400$	0	θ_3

i	α_{i-1}	a_{i-1}	d_i	θ_i
末端	0	$a_3 = 350$	0	0

解 (b)：這類「腰 + 平面二連桿」的構型，可用一個技巧簡化計算：先在垂直平面內當作平面 2R 手臂計算，再繞 Z 軸旋轉 θ_1 。

在垂直平面內，令 r 為水平投影距離、 z 為高度：

$$r = a_2 \cos \theta_2 + a_3 \cos(\theta_2 + \theta_3) = 400 \cos 30^\circ + 350 \cos(-30^\circ) \\ = 400(0.8660) + 350(0.8660) = 346.41 + 303.11 = 649.52 \text{ mm}$$

水平投影距離。注意 $\theta_2 + \theta_3 = 30^\circ + (-60^\circ) = -30^\circ$ ，而 $\cos(-30^\circ) = \cos 30^\circ$ 。

$$z = d_1 + a_2 \sin \theta_2 + a_3 \sin(\theta_2 + \theta_3) = 300 + 400(0.5) + 350(-0.5) \\ = 300 + 200 - 175 = 325 \text{ mm}$$

高度。

再繞 Z 軸旋轉 $\theta_1 = 45^\circ$ ，將水平投影分解到 X、Y：

$$x = r \cos \theta_1 = 649.52 \cos 45^\circ = 459.28 \text{ mm} \\ y = r \sin \theta_1 = 649.52 \sin 45^\circ = 459.28 \text{ mm} \\ z = 325 \text{ mm}$$

末端位置。

驗算技巧：末端到基座軸心的水平距離應為 $\sqrt{(459.28^2 + 459.28^2)} = 649.5 \text{ mm}$ ，與 r 相符。此外，由於 $\theta_2 + \theta_3 = -30^\circ$ 與 $\theta_2 = 30^\circ$ 對稱，前臂與上臂相對於水平線的角度大小相同、方向相反，這種對稱性使 r 的兩項相等——這正是計算結果中 346.41 與 303.11 之比恰為 400:350 的原因。

程式範例：通用正向運動學求解器

以下 Python 程式實作通用的串聯手臂正向運動學：輸入 DH 參數表與關節角度，輸出各連桿座標系與末端位姿。程式以本章三個例題驗證，並示範工作空間掃描。

```
import numpy as np

def dh_transform(alpha, a, d, theta):
    """修正式 DH (Craig 慣例) 的連桿變換矩陣 (9.4.1 節)"""
    ca, sa = np.cos(alpha), np.sin(alpha)
    ct, st = np.cos(theta), np.sin(theta)
    return np.array([
        [ ct, -st, 0, a ],
        [ st*ca, ct*ca, -sa, -sa*d ],
        [ st*sa, ct*sa, ca, ca*d ],
        [ 0, 0, 0, 1 ]
    ])
```



```

])

def forward_kinematics(dh_table, q):
    """
    dh_table: 每列為 (alpha, a, d, theta, joint_type)
               joint_type: 'R' 旋轉 (theta 為變數) 、'P' 移動 (d 為變數) 、
                           'F' 固定 (無變數)

    q: 關節變數陣列
    回傳: (末端 4x4 變換, 各連桿座標系原點位置串列)
    """
    T = np.eye(4)
    origins = [T[:3, 3].copy()]
    qi = 0
    for alpha, a, d, theta, jtype in dh_table:
        if jtype == 'R':
            theta = theta + q[qi]; qi += 1
        elif jtype == 'P':
            d = d + q[qi]; qi += 1
        T = T @ dh_transform(alpha, a, d, theta)
        origins.append(T[:3, 3].copy())
    return T, origins

def pose_from_T(T):
    """由變換矩陣取出位置與 RPY 姿態 (第 8.10.1 節)"""
    p = T[:3, 3]
    R = T[:3, :3]
    if abs(R[2, 0]) > 0.99999: # 萬向節鎖
        pitch = -np.sign(R[2, 0]) * np.pi/2
        roll, yaw = 0.0, np.arctan2(-R[0, 1], R[1, 1])
    else:
        pitch = np.arctan2(-R[2, 0], np.hypot(R[0, 0], R[1, 0]))
        yaw = np.arctan2(R[1, 0], R[0, 0])
        roll = np.arctan2(R[2, 1], R[2, 2])
    return p, np.rad2deg([roll, pitch, yaw])

d = np.deg2rad

# ----- 驗證一：例題 9-1 平面 2R -----
print("=== 例題 9-1：平面 2R 手臂 ===")
planar_2r = [
    (0, 0, 0, 0, 'R'), # 關節 1
    (0, 300, 0, 0, 'R'), # 關節 2 (a = L1)
    (0, 200, 0, 0, 'F'), # 末端 (a = L2)
]
T, _ = forward_kinematics(planar_2r, [d(30), d(45)])
p, rpy = pose_from_T(T)
print(f"末端位置: x={p[0]:.2f}, y={p[1]:.2f} mm")
print(f"末端姿態: yaw={rpy[2]:.1f}° (應為 75°)")

# ----- 驗證二：例題 9-2 SCARA -----
print("\n=== 例題 9-2：SCARA ===")
scara = [

```

```

    (0,      0,    500, 0, 'R'),    # theta1, d1=500
    (0,      400,    0, 0, 'R'),    # a1=400
    (0,      300,    0, 0, 'P'),    # a2=300, d3 為變數
    (np.pi, 0,      0, 0, 'R'),    # alpha=180°, theta4
]
T, _ = forward_kinematics(scara, [d(60), d(-30), -80, d(20)])
p, _ = pose_from_T(T)
print(f"TCP 位置: x={p[0]:.2f}, y={p[1]:.2f}, z={p[2]:.2f} mm")
print(f"水平距離: {np.hypot(p[0], p[1]):.1f} mm (應約 676.5)")

# ----- 驗證三：例題 9-3 三軸空間手臂 -----
print("\n=== 例題 9-3：三軸空間手臂 ===")
arm3 = [
    (0,      0, 300, 0, 'R'),
    (-np.pi/2, 0, 0, 0, 'R'),
    (0,      400, 0, 0, 'R'),
    (0,      350, 0, 0, 'F'),
]
T, origins = forward_kinematics(arm3, [d(45), d(30), d(-60)])
p, _ = pose_from_T(T)
print(f"末端位置: x={p[0]:.2f}, y={p[1]:.2f}, z={p[2]:.2f} mm")
print(f"各關節位置: {[np.round(o, 1).tolist() for o in origins]}")

# ----- 應用：工作空間掃描 (9.1.3 節用途四) -----
print("\n=== 平面 2R 工作空間掃描 ===")
pts = []
for t1 in np.linspace(-np.pi, np.pi, 60):
    for t2 in np.linspace(-np.pi, np.pi, 60):
        T, _ = forward_kinematics(planar_2r, [t1, t2])
        pts.append(T[:2, 3])
pts = np.array(pts)
r = np.linalg.norm(pts, axis=1)
print(f"可達半徑範圍: {r.min():.1f} ~ {r.max():.1f} mm")
print(f"理論值 (|L1-L2| ~ L1+L2) : 100.0 ~ 500.0 mm")

```

請試著修改：（一）把 SCARA 的 α_4 由 180° 改成 0° ，觀察末端姿態如何改變。（二）為例題 9-3 的手臂增加一個手腕軸，觀察 DH 表如何擴充。（三）將工作空間掃描的結果以 Matplotlib 畫成散布圖，你會看到第 4.10.2 節所描述的環狀工作空間。

實作：為實體手臂建立運動學模型

本章的實作是為一支真實的機械手臂（建議使用附錄 E 的六軸人形手臂、或任何三軸以上的教學型手臂）建立完整的運動學模型，並驗證其準確度。

1. **繪製機構簡圖**：測量並記錄每一節連桿的長度、各關節軸的方向與相對位置。以線段與圓柱符號繪出機構簡圖，標出所有關節軸線。

2. **建立座標系**：依 9.3.3 節的六步驟，逐一標定 Z 軸、找出公垂線、建立各連桿座標系。將座標系直接畫在機構簡圖上。
3. **填寫 DH 參數表**：逐行讀出四個參數，注意標明使用的是修正式 DH。
4. **程式實作**：將參數表輸入本章的程式範例，計算數組不同關節角度下的末端位置。
5. **零位驗證**：將所有關節設為零位，比對程式計算結果與實際手臂的幾何量測值。若不符，通常是座標系建立或參數符號有誤。
6. **多點驗證**：選取至少五組不同的關節角度組合，實際擺出該姿態並用尺（或方格紙、雷射測距）量測末端位置，記錄理論值與實測值。
7. **誤差分析**：計算每一組的誤差量與方向，填入下表。分析誤差是系統性偏移（每次都往同一方向偏，可用零位補償修正）還是隨機分散（可能是背隙或關節鬆動）。
8. **零位標定**：若發現系統性誤差，嘗試調整各關節的零位偏移量，使誤差最小化。這是 9.9.3 節標定流程的簡化版。

表 9-7 運動學驗證記錄表（第一列為範例）

組別	關節角度 ($\theta_1, \theta_2, \theta_3$)	理論位置 (x, y, z)	實測位置	誤差量	誤差方向
1	(0°, 0°, 0°)	(750, 0, 300)	(742, 3, 296)	9.4 mm	偏內側、偏低
2~5	(自行填寫)				

國中小也看得懂：一節一節接力算出手在哪裡

伸出你的手臂，先把肩膀往前抬 45 度，再把手肘彎 90 度。現在你的手掌在哪裡？

你可能會說「在我胸前」——但如果要用數字說清楚呢？比如說「在我肩膀前面 20 公分、往上 10 公分」。這就是機器人每一毫秒都在做的計算。它的每個關節裡都有一個「角度感應器」，隨時知道自己轉了幾度；然後電腦要靠這些角度，算出手到底伸到哪裡去了。

為什麼要算？想想看：如果機器人不知道自己的手在哪裡，它怎麼知道會不會撞到桌子？怎麼知道有沒有碰到要抓的東西？就像你閉著眼睛走路會撞到門，機器人不會算，就什麼事都做不好。

這個計算其實有個小訣竅：**一節一節接力**。先算出第一節手臂的末端在哪裡，把那裡當成新的起點，再算第二節，再把那裡當起點算第三節……一直算到手掌。就像玩接力賽跑，每一棒交

給下一棒。你在附錄 E 做的六軸人形機器人，裡面的程式就是這樣一節一節算出手腳位置的喔！

叫獸提醒與常見錯誤

- 誤區一：混用標準 DH 與修正式 DH。參數表與變換矩陣公式必須來自同一種慣例，混用會得到「看似合理但完全錯誤」的結果。
- 誤區二：忘記標明慣例。任何 DH 參數表都應在標題註明是標準還是修正式，並附上所用的變換矩陣公式。
- 誤區三：Z 軸方向前後不一致。Z 軸方向可自由選擇，但一旦選定就必須貫徹到底；中途改變會使 α 與 θ 的符號全錯。
- 誤區四：把 a 當成連桿的實體長度。 a 是兩關節軸公垂線的長度，若軸線有偏移，它未必等於連桿外觀的長度。
- 誤區五：忽略 d 參數。當兩關節軸有沿軸向的錯開時（如 PUMA 的 d_3 、 d_4 ），漏掉 d 會造成數公分的誤差。
- 誤區六：兩軸平行時隨意選公垂線卻未保持一致。平行軸的公垂線不唯一，應選定使 $d = 0$ 的那一條並貫徹。
- 誤區七：程式中角度單位混用。DH 參數表常以度為單位書寫，但三角函數需要弧度，轉換遺漏是最常見的低級錯誤。
- 誤區八：忘記末端工具的變換。 0T_n 只到法蘭面，加上夾具後必須再乘 ${}^nT_{TCP}$ ，否則會差一個工具長度。
- 誤區九：以為理論計算就等於實際位置。製造、裝配、零位、變形與溫度誤差合計可達數公釐，精密應用必須標定。
- 誤區十：用重複精度的規格去承諾絕對精度的任務。離線編程與視覺導引需要的是絕對精度，兩者可差一到兩個數量級。

課堂討論

1. 開頭故事中，小翰花兩分鐘試誤才把夾爪移到目標位置，誤差 2.8 公分。若他有正向運動學程式但沒有逆向運動學，能否加速這個過程？請設計一個只用正向運動學的搜尋策略，並討論其效率。

2. 為什麼六軸工業機器人幾乎都採用「後三軸交於一點」的球形手腕？這個設計犧牲了什麼、換來了什麼？請從機構製造與演算法兩方面討論。
3. 現代的 URDF 用六個參數描述連桿關係，比 DH 的四個參數多。既然如此，為什麼教科書仍然要教 DH 法？請討論兩者各自的適用場合。
4. 某機器人的重複精度為 $\pm 0.02 \text{ mm}$ ，絕對精度為 $\pm 1.5 \text{ mm}$ 。請各設計一個「只需重複精度」與「必須絕對精度」的實際應用，並說明理由。
5. 若一支手臂的第二軸與第三軸並非完全平行（裝配偏差 0.2° ），對正向運動學的計算結果會有多大影響？請估算在 500 mm 臂展下的末端誤差，並討論該用什麼方法補救。

章末摘要

- 正向運動學由關節變數求末端位姿，其解必定存在且唯一，是逆運動學、雅可比與動力學的共同基礎。
- 五大用途：即時狀態顯示、模擬與離線編程、碰撞檢測、工作空間分析、以及逆運動學的迭代基礎。
- 編號慣例：基座為連桿 0、末端為連桿 n ；關節 i 連接連桿 $i-1$ 與 i ； n 軸手臂共有 $n+1$ 根連桿。
- DH 法的核心洞見是沿公垂線建立座標系（ Z 沿關節軸、 X 沿公垂線），使描述兩座標系關係的六個參數壓縮為四個。
- 四個 DH 參數： a 與 α 描述連桿本身（恆為常數）、 d 與 θ 描述關節狀態（其一為變數）。
- 修正式 DH 的連桿變換為 $R_x(\alpha) \cdot D_x(a) \cdot R_z(\theta) \cdot D_z(d)$ ，先走連桿再轉關節；本書採此慣例。
- 標準 DH 與修正式 DH 數學等價但參數值與座標系位置不同，絕對不可混用，且必須明確標註。
- 正向運動學即連桿變換的連乘： ${}^0T_n = {}^0T_1 \cdot {}^1T_2 \cdots {}^{n-1}T_n$ ，再乘 ${}^nT_{TCP}$ 得工具位姿。
- 由結果矩陣可讀出：右上 3×1 為位置，左上 3×3 的三行為末端座標系的 $\hat{n}$ 、 $\hat{o}$ 、 $\hat{a}$ 三軸方向。
- 平面 2R 手臂： $x = L_1 C_1 + L_2 C_{12}$ 、 $y = L_1 S_1 + L_2 S_{12}$ 、 $\varphi = \theta_1 + \theta_2$ ；工作空間為內徑 $|L_1 - L_2|$ 、外徑 $L_1 + L_2$ 的圓環。

- SCARA 的運動學完全解耦（水平由前兩軸、高度由移動軸、方向由三者組合），這是其逆運動學有簡潔封閉解的原因。
- 六軸手臂的球形手腕（後三軸交於一點）使逆運動學得以解耦成位置與姿態兩個子問題，是封閉解存在的關鍵。
- 運動學誤差有五源：製造公差、裝配誤差、零位偏移、彈性變形、溫度效應；其中裝配誤差因沿鏈累積而最為嚴重。
- 重複精度（ $\pm 0.02 \text{ mm}$ ）遠優於絕對精度（ $\pm 0.5 \sim 2 \text{ mm}$ ）；示教編程只需前者，離線編程與視覺導引需要後者。
- 運動學標定以實測資料反推真實 DH 參數，可將絕對精度提升至 $\pm 0.2 \text{ mm}$ 以內。

觀念題與計算題

觀念題

1. 定義正向運動學問題，並說明為何其解必定存在且唯一。
2. 列舉正向運動學的五個實際用途，並各舉一個具體情境。
3. 說明連桿與關節的編號慣例，並指出 n 軸手臂共有幾根連桿。
4. DH 法如何把描述兩座標系關係所需的六個參數壓縮為四個？關鍵的幾何洞見是什麼？
5. 說明四個 DH 參數的定義與幾何意義，並指出哪兩個恆為常數。
6. 寫出修正式 DH 的四個基本動作及其順序，並說明其物理意義。
7. 比較標準 DH 與修正式 DH 的差異，並說明混用會造成什麼後果。
8. 當相鄰兩關節軸平行、相交、重合時，DH 參數各有什麼特徵？應如何處理公垂線的選擇？
9. 說明由 0T_n 矩陣中如何讀出末端的位置與姿態，並解釋 $\hat{n}$ 、 $\hat{o}$ 、 $\hat{a}$ 的意義。
10. 為何 SCARA 的正向運動學能夠完全解耦？這帶來哪四項工程好處？
11. 什麼是球形手腕？它對逆向運動學有何決定性的影響？
12. 列舉運動學誤差的五個來源，並指出何者因沿鏈累積而影響最大。
13. 區分重複精度與絕對精度，並各舉一個依賴該精度的應用。
14. 說明運動學標定的五個步驟，以及為何需要保留一組未參與辨識的資料做驗證。

計算題

1. 某平面 2R 手臂 $L_1 = 250 \text{ mm}$ 、 $L_2 = 250 \text{ mm}$ 。(a) 當 $\theta_1 = 45^\circ$ 、 $\theta_2 = 90^\circ$ 時求末端位置與姿態。(b) 求工作空間的內外半徑，並說明等長連桿的特殊性。(c) 若末端須到達 (300, 200)，請驗證此點是否落在工作空間內 (提示：計算其距原點的距離)。
2. 某平面 3R 手臂 $L_1 = 300$ 、 $L_2 = 250$ 、 $L_3 = 150 \text{ mm}$ 。當 $\theta_1 = 20^\circ$ 、 $\theta_2 = 40^\circ$ 、 $\theta_3 = -30^\circ$ 時，(a) 求末端位置與姿態角 φ 。(b) 求第二關節與第三關節的位置。(c) 若要維持末端位置不變但改變姿態角，說明是否可行及原因。
3. 某 SCARA 機器人 $a_1 = 350 \text{ mm}$ 、 $a_2 = 250 \text{ mm}$ 、 $d_1 = 450 \text{ mm}$ 、Z 軸行程 $0 \sim 200 \text{ mm}$ 。(a) 當 $\theta_1 = -40^\circ$ 、 $\theta_2 = 70^\circ$ 、 $d_3 = 120 \text{ mm}$ 、 $\theta_4 = 15^\circ$ 時求 TCP 位姿。(b) 求工作空間的水平環狀範圍與垂直範圍。(c) 若在此工作空間中要放置一個直徑 400 mm 的圓形治具，其中心應距基座多遠才能讓整個治具都在可達範圍內？
4. 某三軸手臂 $d_1 = 350 \text{ mm}$ 、 $a_2 = 450 \text{ mm}$ 、 $a_3 = 400 \text{ mm}$ ，構型同例題 9-3。(a) 建立修正式 DH 參數表。(b) 當 $\theta_1 = 60^\circ$ 、 $\theta_2 = 45^\circ$ 、 $\theta_3 = -90^\circ$ 時求末端位置。(c) 求此手臂在垂直平面內的最大水平可達距離，以及達到該距離時的關節角度組合。(d) 若第二軸的零位有 $+1^\circ$ 的偏移未被修正，估算在 (b) 的姿態下造成的末端位置誤差。

動手做與專題延伸

本章實作：DH 參數表建立練習

為下列三種機構分別建立修正式 DH 參數表，並以本章程式驗證。這是掌握 DH 法最有效的練習方式——建議按難度順序完成。

- **入門**：三軸桌上型手臂 (腰 + 肩 + 肘)，如附錄 C 的吃錢幣機器人手臂或任何三軸教學套件。
- **進階**：五軸手臂 (三軸位置 + 兩軸手腕)，如常見的教學型機械手臂套件。注意手腕兩軸的軸線關係。
- **挑戰**：六軸協作機器人。可由網路取得公開的 URDF 檔案 (如 UR5、Franka Emika)，由其中讀出連桿尺寸，反推 DH 參數表，再與文獻上公布的參數表比對。

專題延伸

- **運動學視覺化工具**：延續第 8 章的 `draw_frame` 函式，撰寫一個能繪出整支手臂的程式——輸入 DH 參數與關節角度，畫出所有連桿 (連接各座標系原點的線段) 與各連桿座標系。加

上滑桿控制各關節角度，即時更新圖形，做出一個互動式的運動學模擬器。

- **工作空間視覺化**：以蒙地卡羅法隨機取樣數萬組關節角度，計算對應的末端位置並繪成三維散布圖或點雲。加入關節極限的限制，比較有無限制時工作空間的差異——你會直觀看到第 4.10 節所說的「有效工作空間遠小於可達工作空間」。
- **進階：誤差敏感度分析**。為六軸手臂的每一個 DH 參數加上微小擾動（如 a 加 0.5 mm、 α 加 0.1° ），計算末端位置的變化量。由此找出「最敏感的參數」——這正是運動學標定時應優先辨識的參數，也是製造時應優先管控的公差項目。將結果整理成敏感度排序表。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 3 章：操作臂運動學，本章的主要參考，修正式 DH 的來源)。
- Denavit, J. and Hartenberg, R. S., "A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices," ASME Journal of Applied Mechanics, 1955 (DH 參數法的原始論文)。
- Lynch, K. M. and Park, F. C., Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press (第 4 章：正向運動學，含指數積公式的替代方法)。
- Siciliano, B., Sciavicco, L., Villani, L. and Oriolo, G., Robotics: Modelling, Planning and Control, Springer (第 2 章：運動學，採用標準 DH)。
- Spong, M. W., Hutchinson, S. and Vidyasagar, M., Robot Modeling and Control, 2nd ed., Wiley (DH 法與運動鏈建模)。
- Hayati, S. and Mirmirani, M., "Improving the Absolute Positioning Accuracy of Robot Manipulators," Journal of Robotic Systems, 1985 (平行軸問題與改良的參數化方法)。
- Roth, Z. S., Mooring, B. W. and Ravani, B., "An Overview of Robot Calibration," IEEE Journal on Robotics and Automation, 1987 (運動學標定的經典回顧)。
- Corke, P., Robotics, Vision and Control, 3rd ed., Springer (含 Robotics Toolbox 的實作範例)。
- ROS 2 URDF 官方文件與 Pinocchio、KDL 等運動學函式庫的技術文件。
- 各機器人廠商技術手冊中的 DH 參數表與運動學規格（多數廠商會在技術文件中公開）。

第 10 章

逆向運動學

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】同一個馬克杯、同一個手掌位置，卻是八張疊合的手臂剪影——每一張的肩、肘、腕呈現不同的構型（肩前／肩後、肘上／肘下、腕翻轉／不翻轉），以不同深淺的顏色區分。畫面右側另有一隻伸長卻構不到杯子的手臂，旁註「無解」；左下角一隻手掌固定而手肘上下擺動的殘影，旁註「無限多解」。標題文字：一個目標，八種答案——或者，沒有答案。

叫獸開講 「人看到杯子就伸手，機器人卻可能算出八種姿勢。」

叫獸在桌上放了一個馬克杯，請小好伸手去拿。她毫不猶豫地伸手、握住、拿起，全程不到一秒。

「請問，」叫獸說，「你剛才的肩關節轉了幾度？手肘呢？手腕呢？」小好笑了：「我怎麼可能知道。」

「但你的大腦知道，而且是在零點三秒內算出來的。」叫獸在白板上寫下一個方程式： $f(q) = x$ 。「上一章我們學的是已知 q 求 x ，這叫正向運動學，代進去乘一乘就有答案。今天我們要做的是反過來——已知你要的 x ，求 q 。這叫逆向運動學，而它困難得多。」

他請小好再拿一次杯子，但這次要求：「手肘必須抬高、貼著耳朵。」小好照做了，動作警扭但確實拿到了杯子。「注意，」叫獸對全班說，「同一個杯子、同一個手掌位置，她用了**兩種完全不同的姿勢**。這就是逆向運動學的第一個麻煩：**答案不只一個**。」

接著他把杯子往前推到桌子最遠處，超出小好坐著能伸到的範圍。小好伸長了手臂，指尖離杯子還有十公分。「這是第二個麻煩：**可能無解**。目標超出工作空間時，方程式沒有實數解——但機器人不會像人一樣停下來說『我構不到』，它可能會算出一個複數，然後控制器把它當成一個很大的角度指令送給馬達。」他停頓了一下，「這就是為什麼很多學生的第一支機械手臂，是在執行第一個逆運動學指令時撞爛的。」

最後他讓小好把手掌固定在杯子上不准動，然後說：「現在請你上下擺動手肘。」小好照做——手掌紋絲不動，手肘卻能上下擺動一大段。「第三個麻煩：**可能有無限多解**。這就是第 4 章講的冗餘。」

叫獸在白板上把三個麻煩並排寫下：**無解、多解、無限多解**。「正向運動學是代數，逆向運動學是解方程式——而且是一組非線性的三角方程式。有些機構運氣好，能解出漂亮的封閉公式；有些機構解不出來，只能用數值方法一步一步逼近。今天這堂課，我們兩種都要學。」

下課前他補了一句：「還有一件事。人類拿杯子時，從來不需要『算』角度——我們是靠反覆練習長出來的直覺。這件事在第 31 章會再出現：現代的機器人學習，某種程度上就是想讓機器也長出這種直覺，跳過解方程式這一步。但在那之前，我們得先學會老老實實地解它。」

本章為什麼重要

逆向運動學是機器人**實際作業時真正需要回答的問題**。沒有人會對機器人說「請把第三軸轉到 47.2 度」，人們說的是「把那個零件放到這個孔裡」。從笛卡兒座標到關節角度的翻譯，就是逆向運動學。

它也是機器人學中第一個「困難」的問題。正向運動學只需要把數字代進公式；逆向運動學則要解一組非線性方程，而且必須面對無解、多解與數值不穩定等現實。理解這些困難的來源，你才能診斷「為什麼機器人到不了這個點」「為什麼它突然用一個奇怪的姿勢過去」這類實務問題。

本章的內容也是後續章節的關鍵基礎：第 11 章的雅可比矩陣正是數值逆運動學的核心工具；第 12 章的奇異點解釋了為何某些位置的逆解會爆炸；第 15 章的軌跡生成需要對路徑上每一點求逆解；第 26 章的路徑規劃則要在關節空間與任務空間之間反覆換算。

學習目標

- 能定義逆向運動學問題，並說明其與正向運動學在本質上的三個差異。
- 能判斷一個目標位姿是否落在工作空間內，並說明無解時的處置方式。
- 能分析平面 2R 手臂的多解結構，並用幾何法求出封閉解。
- 能運用餘弦定理與 atan2 求解平面手臂的逆運動學。
- 能說明六軸手臂的八組解結構及其對應的機構構型。
- 能說明運動學解耦原理，並解釋球形手腕為何使封閉解成為可能。
- 能推導數值逆運動學的迭代公式，並說明雅可比反矩陣的角色。
- 能比較 Jacobian 轉置法、偽逆法與阻尼最小平方方法的特性。
- 能說明阻尼最小平方方法 (DLS) 如何處理奇異點附近的數值爆炸。
- 能在逆運動學中納入關節極限與避障等額外約束。
- 能以 Python 實作封閉解與數值解，並比較兩者的效能與適用場合。

先備知識

第 8 章的齊次變換與姿態表示、第 9 章的正向運動學與 DH 參數。本章需要三角函數的和差

角公式、餘弦定理，以及矩陣的反矩陣概念。數值方法部分會用到偏導數的概念（但完整的雅可比推導留待第 11 章）。

本章是全書目前為止數學最密集的一章。建議的讀法是：先確實掌握 10.3 節的平面 2R 幾何解（這是所有直覺的來源），再進入數值方法。若某段推導卡住，先跳過細節、抓住結論，之後再回頭補。

核心名詞中英對照

表 10-1 本章核心名詞

中文	英文	說明
逆向運動學	Inverse Kinematics, IK	由末端位姿求關節變數的問題
封閉解 / 解析解	Closed-form Solution	以有限次代數運算直接求得的解
數值解	Numerical Solution	以迭代逼近求得的解
多解	Multiple Solutions	同一位姿對應多組關節角度
手肘向上 / 向下	Elbow-up / Elbow-down	平面二連桿的兩種構型
構型	Configuration	逆解中對應特定機構姿勢的解分支
運動學解耦	Kinematic Decoupling	將位置與姿態拆為兩個子問題求解
球形手腕	Spherical Wrist	後三軸交於一點的手腕構型
Pieper 準則	Pieper's Criterion	六軸手臂存在封閉解的充分條件
雅可比矩陣	Jacobian Matrix	關節速度到末端速度的線性映射
偽逆	Pseudoinverse	非方陣的廣義反矩陣
阻尼最小平方法	Damped Least Squares, DLS	以阻尼項抑制奇異點附近數值爆炸的方法
零空間	Null Space	不影響末端運動的關節速度所張的空間
收斂	Convergence	迭代結果趨近於解的性質
殘差	Residual	目前位姿與目標位姿的差距

10.1 逆向運動學問題

10.1.1 問題定義

逆向運動學的問題陳述是正向運動學的反轉：

已知： ${}^0T_{\text{目標}}$ （末端的期望位姿） 求： $\mathbf{q} = [q_1, q_2, \dots, q_n]^T$

求出使 $f(q) = x$ 目標成立的所有關節變數組合。

這是一組非線性方程式。以六軸手臂為例， 0T_6 有 12 個有意義的元素（ 3×3 旋轉加 3×1 平移），但因旋轉矩陣的正交約束，實際只有 6 個獨立方程式，對應 6 個未知數 $\theta_1 \sim \theta_6$ 。方程式數與未知數數相等，看似「剛好可解」——但這些方程式充滿 $\sin$ 與 $\cos$ 的乘積與和，並非線性代數能直接處理。

10.1.2 三個本質差異

逆向運動學與正向運動學有三個本質差異，這正是它困難的根源。

差異一：解未必存在。當目標位姿落在工作空間之外，方程式無實數解。此外即使位置可達，某些姿態組合也可能不可達（靈巧工作空間小於可達工作空間，見第 4.10.1 節）。

差異二：解未必唯一。同一個末端位姿，往往對應多組關節角度。平面 2R 有兩組（手肘上 / 下）、六軸手臂典型有八組、冗餘手臂則有無限多組。控制器必須有一套規則來決定「選哪一組」。

差異三：未必有封閉解。正向運動學永遠可以直接算出來；逆向運動學只有在機構滿足特定條件時（如球形手腕）才有封閉公式，否則只能數值迭代。

表 10-2 正向與逆向運動學的比較

面向	正向運動學	逆向運動學
問題	已知 q 求 x	已知 x 求 q
數學本質	代入求值	解非線性方程組
解的存在性	永遠存在	可能無解（超出工作空間）
解的唯一性	唯一	常有多組解，冗餘時無限多
計算方式	矩陣連乘，直接得解	封閉解或數值迭代
計算成本	低且固定	封閉解低、數值解高且不定
實務角色	模擬、監控、碰撞檢測	路徑執行、示教、實際作業

10.1.3 兩大類解法

逆向運動學的解法分為兩大陣營，各有明確的適用場合。

封閉解（解析解）：用代數與幾何推導出直接的公式。優點是計算極快（微秒等級）、能求

出**全部**的解、且不會有收斂失敗的問題。缺點是必須針對每一種機構個別推導，且只有特定構型才存在。工業機器人控制器內部使用的都是封閉解。

數值解（迭代法）：由一個初始猜測出發，反覆修正直到誤差夠小。優點是**通用**——任何機構都適用，且容易加入額外約束（關節極限、避障、姿勢偏好）。缺點是計算較慢、只能找到一組解（取決於初始猜測）、在奇異點附近可能不穩定或不收斂。研究型機器人與冗餘手臂多採此法。

實務上兩者常結合使用：先用封閉解快速求出所有解、再用數值法做精細修正或處理額外約束。

10.2 解的存在性與工作空間

10.2.1 可達性判斷

在求解之前，應先判斷目標是否可達。最基本的檢查是距離判斷——以平面 2R 手臂為例，末端到基座的距離 r 必須滿足：

$$|L_1 - L_2| \leq r \leq L_1 + L_2$$

外側界限為完全伸直、內側界限為完全折疊。超出此範圍則無解。

對空間手臂，可達性的判斷較為複雜，但基本原則相同：先檢查位置是否在可達工作空間內，再檢查該位置能否達成所要求的姿態（靈巧工作空間）。實務上常用的作法是：先嘗試求解，若解出的角度含複數或超出關節極限，即判定為不可達。

10.2.2 無解時的處置

當目標不可達時，控制器絕不能把錯誤的計算結果直接送給馬達——這正是開頭故事中「第一支手臂撞爛」的原因。正確的處置方式有四種：

方式一：拒絕並報錯。回報「目標超出工作空間」，停止動作。這是最安全的作法，工業控制器的標準行為。

方式二：投影到最近可達點。若目標在工作空間外，取工作空間邊界上最接近的點。這在遙操作與視覺伺服中常用——操作員把目標拉到太遠時，手臂會伸到極限並停住，而非亂動。

方式三：放寬姿態要求。許多任務對姿態的要求並不嚴格（例如吸盤只需垂直向下，繞垂直

軸的角度無所謂)。放寬一個姿態自由度，往往就能讓原本不可達的点變為可達。

方式四：改變基座位置。若機器人裝在移動平台或行走軸上，可先移動基座再求解。這是行動操作機器人 (mobile manipulator) 的標準策略。

◎ 實務上最常見的「假不可達」

學生回報「手臂到不了這個點」時，有超過一半的情況並非真的超出工作空間，而是三個原因之一：一、**關節極限**——數學上有解，但某個角度超出機構允許範圍；二、**自我碰撞**——姿態可達但手臂會撞到自己；三、**姿態要求過嚴**——位置可達但要求的姿態在該處無法達成。診斷時應分別檢查這三項，而非直接認定超出工作空間。

10.3 平面 2R 手臂的封閉解

10.3.1 幾何解法：餘弦定理

平面 2R 手臂是理解逆運動學多解結構的最佳範例。給定目標位置 (x, y) ，求 θ_1 與 θ_2 。

第一步：求 θ_2 。觀察由基座、肘關節、末端所構成的三角形，其三邊分別為 L_1 、 L_2 與 $r = \sqrt{x^2 + y^2}$ 。由餘弦定理：

$$r^2 = L_1^2 + L_2^2 - 2L_1L_2 \cos(180^\circ - \theta_2) = L_1^2 + L_2^2 + 2L_1L_2 \cos \theta_2$$

注意三角形內角為 $180^\circ - \theta_2$ ，因為 θ_2 是連桿 2 相對於連桿 1 延長線的夾角。

整理後得到：

$$\cos \theta_2 = (x^2 + y^2 - L_1^2 - L_2^2) / (2 L_1 L_2)$$

求解 θ_2 的關鍵式。若右式的絕對值大於 1，代表目標不可達 (10.2.1 節)。

這裡出現第一個關鍵點：由 $\cos \theta_2$ 求 θ_2 時，會得到兩個解——正的與負的：

$$\theta_2 = \pm \arccos((x^2 + y^2 - L_1^2 - L_2^2) / (2 L_1 L_2))$$

正號對應手肘向下 (elbow-down)、負號對應手肘向上 (elbow-up)。這就是開頭故事中小妍的兩種拿杯姿勢。

第二步：求 θ_1 。已知 θ_2 後， θ_1 可由兩個角度相減求得：目標點的方位角，減去「第一節連桿與目標方向的夾角」：

$$\theta_1 = \text{atan2}(y, x) - \text{atan2}(L_2 \sin \theta_2, L_1 + L_2 \cos \theta_2)$$

第一項是目標的方位角、第二項是第一節連桿相對於目標方向的偏轉角。務必使用 atan2 以正確處理四個象

限。

由於 θ_2 有兩個值，代入後會得到兩個對應的 θ_1 ——因此總共兩組解。

10.3.2 兩組解的幾何意義

這兩組解在幾何上構成一個對稱：以「基座到末端的連線」為軸，兩種構型互為鏡像。手肘向上的解，肘關節位於連線上方；手肘向下的解，肘關節位於連線下方。

兩者的末端位置完全相同，但整條手臂佔據的空間完全不同。這在實務上極為重要：某一組解可能會撞到工件或治具，另一組則不會。控制器選擇構型的常見準則有三：

- **維持構型連續性**：整條軌跡都使用同一構型，避免中途切換（切換時手臂必須大幅擺動，既危險又耗時）。
- **最小關節移動量**：選擇離目前姿態最近的那一組解，減少運動時間與能耗。
- **避開障礙與關節極限**：檢查每一組解是否會發生碰撞或超出行程，選擇可行的那一組。

圖 10-1 平面 2R 手臂的兩組逆解

建議配圖：同一個目標點 (x, y) ，畫出兩種手臂構型——實線為手肘向上（肘關節在基座與末端連線的上方）、虛線為手肘向下（肘關節在下方），兩者以連線為軸呈鏡像對稱。標註 θ_1 、 θ_2 的定義角度，並在中間畫出由 L_1 、 L_2 、 r 構成的三角形與餘弦定理的關係。右側附兩組解的數值範例。

10.3.3 特殊情形

三種特殊情形需要在程式中特別處理。

情形一：目標在邊界 ($r = L_1 + L_2$)。此時 $\cos \theta_2 = 1$ ， $\theta_2 = 0$ ，兩組解合而為一（手臂完全伸直）。這是第 12 章要討論的**邊界奇異點**——在此處，末端已無法再沿徑向前進。

情形二：目標在內界 ($r = |L_1 - L_2|$)。此時 $\cos \theta_2 = -1$ ， $\theta_2 = 180^\circ$ ，手臂完全折疊。同樣是奇異點，且實體上通常會自我碰撞。

情形三：目標在原點且 $L_1 = L_2$ 。此時任何 θ_1 配上 $\theta_2 = 180^\circ$ 都是解——出現無限多解。這是一個退化的情形，數值上會導致 $\text{atan2}(0, 0)$ 的不定值，程式必須特別處理。

10.4 代數解法與三角恆等式技巧

10.4.1 代數解法的基本策略

幾何法直觀但難以推廣到空間機構。代數解法則系統性得多：把正向運動學方程式 ${}^0T_n(q)$

= $T_{\text{目標}}$ 的兩邊逐元素比對，得到一組方程式，再設法逐一解出各個關節變數。

關鍵技巧是設法讓某個未知數單獨出現。常用的手法是在方程式兩邊同乘某個變換的反矩陣，把已知的部分移到一邊。例如：

$${}^0T_1^{-1}(\theta_1) \cdot T_{\text{目標}} = {}^1T_2 \cdot {}^2T_3 \cdot \dots \cdot {}^{n-1}T_n$$

左邊只含 θ_1 與已知量，右邊不含 θ_1 。比對兩邊的某些元素，就能解出 θ_1 。

這個「逐步剝離」的策略是所有封閉解推導的核心。Craig 教科書中 PUMA 手臂的完整推導即採此法，過程冗長但邏輯清晰。

10.4.2 三種常見的三角方程式型式

推導過程中會反覆遇到三種型式的三角方程式，熟悉它們的解法能大幅加速推導。

型式一： $\sin \theta = a$ 、 $\cos \theta = b$ （兩者皆已知）。這是最理想的情形，直接用 atan2 得到唯一解：

$$\theta = \text{atan2}(a, b)$$

只要 a 與 b 都已知， θ 就唯一確定（不會有象限模糊）。這是為何要用 atan2 而非 $\arcsin$ 或 $\arccos$ 的根本原因。

型式二： $a \sin \theta + b \cos \theta = c$ 。這類方程式用「三角函數合成」處理。令 $\rho = \sqrt{a^2 + b^2}$ 、 $\varphi = \text{atan2}(b, a)$ ，則：

$$\rho \sin(\theta + \varphi) = c \Rightarrow \theta = \text{atan2}(c, \pm \sqrt{\rho^2 - c^2}) - \varphi$$

有兩組解（ $\pm$ 號），對應多解結構。當 $\rho < |c|$ 時無實數解，代表目標不可達。

型式三：兩式聯立，如 $a \sin \theta + b \cos \theta = c$ 與 $a \cos \theta - b \sin \theta = d$ 。此時可直接解出：

$$\sin \theta = (ac - bd)/(a^2 + b^2), \quad \cos \theta = (ad + bc)/(a^2 + b^2) \Rightarrow \theta = \text{atan2}(\dots)$$

化為型式一，得唯一解。

10.4.3 解的個數從何而來

觀察上述型式可以理解多解的來源：每一次開平方根或反餘弦，都會產生一個 $\pm$ 的分岔。平面 2R 有一次分岔（ θ_2 的正負），故有 2 組解；六軸手臂在推導過程中會遇到三次分岔，故有 $2^3 = 8$ 組解。

這三次分岔在機構上各有明確意義：肩部（前 / 後）、肘部（上 / 下）、腕部（翻轉 / 不翻

轉)。工業機器人的示教器上通常會顯示目前的構型代碼(如 "NUT" 或 "FUT")，正是在標示這三個二選一的狀態。程式設計師必須理解這些代碼，否則會遇到「同樣的座標，機器人卻用完全不同的姿勢過去」的困惑。

10.5 運動學解耦與球形手腕

10.5.1 Pieper 準則

並非所有六軸機構都有封閉解。1968 年 Pieper 提出一個充分條件 (Pieper 準則)：若六軸機器人的任意三個相鄰關節軸交會於一點、或三軸互相平行，則存在封閉解。

絕大多數工業機器人採用第一種情形：後三軸 (手腕) 交會於一點，構成第 9.8.1 節介紹的球形手腕。這不是巧合，而是刻意的設計決策——為了讓逆運動學可解。

10.5.2 解耦的核心洞見

球形手腕帶來的關鍵好處是：手腕中心點的位置，完全不受後三軸角度的影響。因為後三軸都繞著同一個點旋轉，無論它們怎麼轉，那個交會點都不會移動。

這使得問題可以拆成兩半。設末端工具的位姿為 (p, R) ，手腕中心到末端的距離為 d_6 (沿末端 Z 軸方向)，則手腕中心點的位置為：

$$p_{\text{wrist}} = p - d_6 \cdot \hat{a} \quad (\hat{a} \text{ 為末端座標系的 Z 軸方向，即 } R \text{ 的第三行})$$

由末端位姿直接算出手腕中心。這一步不需要任何關節角度資訊。

有了手腕中心的位置，問題就分解為：

- **子問題一 (位置)：**求 $\theta_1, \theta_2, \theta_3$ ，使前三軸把手腕中心送到 p_{wrist} 。這是一個三自由度的位置問題，可用類似平面 2R 的方法求解。
- **子問題二 (姿態)：**已知 $\theta_1, \theta_2, \theta_3$ 後，前三軸造成的旋轉 0R_3 為已知。剩下的旋轉必須由後三軸補足，因此求解 ${}^3R_6 = {}^0R_3^T \cdot R$ 目標，這是一個純姿態問題，等同於由旋轉矩陣求歐拉角 (第 8.10.1 節)。

兩個三自由度的問題，遠比一個六自由度的問題容易處理——這就是解耦的價值。

圖 10-2 運動學解耦示意

建議配圖：六軸手臂側視圖。以紅色圓點標出手腕中心 (後三軸交會處)，並用箭頭表示由末端位姿 p 沿 $-\hat{a}$ 方向退 d_6 得到 p_{wrist} 。左半部以藍色標示前三軸 ($\theta_1, \theta_2, \theta_3$) 負責把手腕中心送到位置；右半部以綠色標示

後三軸 (θ_4 、 θ_5 、 θ_6) 負責調整姿態。下方以兩個方框標註「子問題一：位置」與「子問題二：姿態」。

10.5.3 六軸手臂的八組解

依 10.4.3 節的分析，六軸手臂的解在推導過程中經歷三次二選一的分岔，因此共有 $2^3 = 8$ 組解：

表 10-3 六軸手臂的八組解結構

分岔	兩種選擇	機構意義	產生位置
肩部構型	前 / 後 (Front / Back)	手臂向前伸或向後反折跨過基座	求 θ_1 時的兩個相差 180° 的解
肘部構型	上 / 下 (Up / Down)	肘關節在肩腕連線的上方或下方	求 θ_3 時的 $\arccos$ 正負號
腕部構型	翻轉 / 不翻轉 (Flip / No-flip)	手腕以兩種等效姿態達成同一方向	求 θ_5 時的 $\arccos$ 正負號

這八組解在數學上完全等價（末端位姿相同），但在工程上差異巨大：某些解會超出關節極限、某些會發生自我碰撞、某些會撞到工件。實務上控制器的選解策略通常是：先過濾掉不可行的解（超限、碰撞），再從剩下的解中選擇離目前姿態最近的一組。

值得注意的是，當機器人接近奇異點時，某些解會互相靠攏甚至合併——例如手臂完全伸直時，肘上與肘下的解合而為一。這正是第 12 章要探討的現象。

10.6 數值逆運動學的原理

10.6.1 為什麼需要數值方法

封閉解雖然快速精確，卻有三個限制：必須逐一針對每種機構推導（工作量大且易錯）、只適用於滿足 Pieper 準則的構型、且難以納入額外約束（避障、關節極限、姿勢偏好）。

數值方法則完全通用。它的基本策略是：從一個初始猜測 q_0 出發，反覆問「目前差多遠？該往哪個方向調整？」直到誤差足夠小。這正是最佳化的思維，也是第 2 章提到「最佳化負責選擇」的具體實踐。

10.6.2 誤差與雅可比

設目前的關節角度為 q ，用正向運動學算出目前位姿 $f(q)$ ，與目標的差距（殘差）為：

$$\mathbf{e} = \mathbf{x}_{\text{目標}} - \mathbf{f}(\mathbf{q})$$

殘差向量。位置部分是簡單的向量相減；姿態部分則需用第 8.10.3 節的方法（由 $R_{\text{error}} = R_{\text{current}}^T \cdot R_{\text{target}}$ 取軸角）轉為三維向量。

接下來的關鍵問題是：關節該調整多少，才能減少這個誤差？答案來自一個線性近似——當關節有微小變化 $\Delta \mathbf{q}$ 時，末端位姿的變化為：

$$\Delta \mathbf{x} \approx \mathbf{J}(\mathbf{q}) \cdot \Delta \mathbf{q}$$

$\mathbf{J}$ 是雅可比矩陣 (Jacobian)，它把關節空間的微小變化映射到任務空間。這是第 11 章的主題，本章先接受這個關係並使用它。

把它反過來，就得到迭代的修正量：

$$\Delta \mathbf{q} = \mathbf{J}^{-1} \cdot \mathbf{e} \quad \mathbf{q}_{\text{new}} = \mathbf{q} + \alpha \cdot \Delta \mathbf{q}$$

α 是步長 ($0 < \alpha \leq 1$)，用於控制收斂速度與穩定性。這就是牛頓 - 拉夫森法在逆運動學上的應用。

10.6.3 迭代演算法

完整的數值逆運動學演算法如下：

1. **初始化**：給定初始猜測 $\mathbf{q}_0$ （通常用機器人目前的姿態，這是最好的猜測）。
2. **正向運動學**：計算 $\mathbf{f}(\mathbf{q})$ ，得到目前位姿。
3. **計算殘差**： $\mathbf{e} = \mathbf{x}_{\text{目標}} - \mathbf{f}(\mathbf{q})$ 。若 $\|\mathbf{e}\|$ 小於容差，則收斂成功、結束。
4. **計算雅可比**： $\mathbf{J}(\mathbf{q})$ （第 11 章會說明如何計算）。
5. **求修正量**： $\Delta \mathbf{q} = \mathbf{J}^+ \cdot \mathbf{e}$ （ $\mathbf{J}^+$ 為反矩陣或偽逆）。
6. **更新**： $\mathbf{q} \leftarrow \mathbf{q} + \alpha \cdot \Delta \mathbf{q}$ ，並將各關節限制在允許範圍內。
7. **檢查迭代次數**：若超過上限仍未收斂，回報失敗；否則回到步驟 2。

注意步驟 2 呼叫了正向運動學——這正是第 9.1.2 節所說「所有數值逆運動學內部都要反覆呼叫正向運動學」的具體體現。一次數值逆解通常需要 5 到 50 次迭代，每次都要算一遍正向運動學與雅可比矩陣，因此計算量遠高於封閉解。

◎ 初始猜測的重要性

數值法只會找到「離初始猜測最近」的那一組解。這帶來兩個實務結論：一、在連續軌跡上求解時，應以上一個點的解作為下一個點的初始猜測——這不只加速收斂，也自然維持了構型連續性（10.3.2 節）。二、若要找出所有的解，必須從多個不同的初始猜測分別出發，這稱為多重起始

(multi-start) 策略。

10.7 三種數值方法的比較

10.7.1 Jacobian 轉置法

最簡單的方法是直接用雅可比的**轉置**取代反矩陣：

$$\Delta \mathbf{q} = \boldsymbol{\alpha} \cdot \mathbf{J}^T \cdot \mathbf{e}$$

轉置法。不需求反矩陣，計算極為簡單。

這個作法看似隨便，其實有堅實的理論基礎： $\mathbf{J}^T \cdot \mathbf{e}$ 恰好是誤差平方和 $\frac{1}{2} \|\mathbf{e}\|^2$ 對 $\mathbf{q}$ 的負梯度方向。因此這實際上是**梯度下降法**——沿著讓誤差下降最快的方向走。

優點是計算成本極低、在奇異點附近完全穩定（因為不需求逆）。缺點是收斂慢（可能需要數百次迭代），且步長 α 需要仔細調整——太大會震盪、太小則收斂太慢。它適合對即時性要求不高、或作為其他方法的備援。

10.7.2 偽逆法

當關節數 n 不等於任務維度 m 時， $\mathbf{J}$ 是 $m \times n$ 的非方陣，沒有一般意義的反矩陣，此時使用 **Moore-Penrose 偽逆**：

$$\mathbf{J}^+ = \mathbf{J}^T (\mathbf{J} \mathbf{J}^T)^{-1} \quad (\text{當 } n > m, \text{ 即冗餘情形})$$

偽逆給出的是「範數最小」的解，即在所有能達成目標的關節速度中，選擇最小的那一組。

偽逆法收斂快（二次收斂），是數值逆運動學的標準方法。但它有一個致命弱點：**在奇異點附近， $(\mathbf{J} \mathbf{J}^T)$ 接近奇異，其反矩陣的元素會趨於無窮大，導致算出的 $\Delta \mathbf{q}$ 極大——機器人會突然高速甩動。這在實體機器上極其危險。**

此外，冗餘機器人的偽逆解還有一個優雅的擴充。由於零空間的存在，可以在不影響末端運動的前提下加入次要目標：

$$\Delta \mathbf{q} = \mathbf{J}^+ \cdot \mathbf{e} + (\mathbf{I} - \mathbf{J}^+ \mathbf{J}) \cdot \mathbf{z}$$

第二項是零空間投影， $\mathbf{z}$ 可以是任何次要目標（如遠離關節極限、避開障礙、維持舒適姿勢）。這正是第 4.9.2 節所說冗餘四大好處的數學實現，第 12 章會詳細展開。

10.7.3 阻尼最小平方法 (DLS)

為了解決偽逆在奇異點附近爆炸的問題，Wampler 與 Nakamura 分別於 1986 年提出阻尼最小平方方法，在求逆時加入一個阻尼項：

$$\Delta \mathbf{q} = \mathbf{J}^T (\mathbf{J} \mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \cdot \mathbf{e}$$

λ 為阻尼係數。加入 $\lambda^2 \mathbf{I}$ 後，即使 $\mathbf{J} \mathbf{J}^T$ 奇異，整個矩陣仍然可逆，數值不會爆炸。

這個方法的物理意義是：它不再追求「完全消除誤差」，而是在「減少誤差」與「限制關節運動量」之間取得平衡。數學上，它最小化的目標函數是：

$$\min \|\mathbf{J} \cdot \Delta \mathbf{q} - \mathbf{e}\|^2 + \lambda^2 \|\Delta \mathbf{q}\|^2$$

第一項要求接近目標、第二項懲罰過大的關節運動。 λ 越大越保守（運動平緩但收斂慢）， λ 越小越積極（收斂快但接近奇異點時危險）。

實務上常用**可變阻尼**策略：平常用很小的 λ （接近偽逆的效能），偵測到接近奇異點時自動增大 λ （犧牲精度換取安全）。判斷是否接近奇異點的指標，通常是雅可比矩陣的最小奇異值（第 12 章）。

表 10-4 三種數值方法的比較

方法	修正公式	收斂速度	奇異點附近	計算成本	適用場合
Jacobian 轉置	$\alpha \cdot \mathbf{J}^T \cdot \mathbf{e}$	慢（線性）	穩定	極低	即時性要求低、教學示範
偽逆	$\mathbf{J}^+ \cdot \mathbf{e}$	快（二次）	爆炸（危險）	中	遠離奇異點的一般情形
阻尼最小平方（DLS）	$\mathbf{J}^T (\mathbf{J} \mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \cdot \mathbf{e}$	中～快	穩定（犧牲精度）	中	實務首選，尤其含奇異點的軌跡

◎ 為什麼 DLS 是實務首選

在真實機器人上，軌跡經過或接近奇異點是常態而非例外（例如手臂伸直去拿遠處的物件）。偽逆法在這種情況下會產生數百度每秒的關節速度指令，輕則觸發過速保護、重則損壞機構或傷人。

DLS 犧牲的只是那一小段路徑的精度（末端會稍微偏離理想路徑），換來的是全程的安全與穩定。這是典型的工程權衡——完美的數學解，未必是最好的工程解。

10.8 最佳化觀點與額外約束

10.8.1 把逆運動學寫成最佳化問題

數值逆運動學可以更一般地表述為一個最佳化問題：

$$\mathbf{q}^* = \operatorname{argmin}_{\mathbf{q}} \|\mathbf{f}(\mathbf{q}) - \mathbf{x}_{\text{目標}}\|_C^2 + \|\mathbf{q} - \mathbf{q}_0\|_W^2$$

第一項是任務誤差（以矩陣 C 加權，表示各方向的重要性）、第二項是正則項（懲罰偏離初始姿態 $\mathbf{q}_0$ ，以矩陣 W 加權）。

這個形式的威力在於它的**可擴充性**。任何額外的需求，都可以化為目標函數中的一項或一個約束條件：

- **關節極限**：加入不等式約束 $\mathbf{q}_{\min} \leq \mathbf{q} \leq \mathbf{q}_{\max}$ ，或在目標函數中加入接近極限時急遽上升的懲罰項。
- **避障**：加入「與障礙物距離大於安全值」的約束，或加入距離越近懲罰越大的項（這與第 26 章的人工勢場法相通）。
- **姿勢偏好**：第二項的 $\mathbf{q}_0$ 可設為「自然姿勢」，使解在不影響任務的前提下傾向於舒適姿態。這對人形與協作機器人特別重要。
- **多任務優先權**：不同的任務給予不同的權重 C ，重要的任務權重高、次要的任務權重低。第 12 章的分層逆運動學是其嚴格化版本。

10.8.2 關節極限的處理

關節極限是最常見也最必要的約束。三種處理方式各有優劣：

方式一：直接截斷（clamping）。每次迭代後把超限的關節拉回邊界。實作最簡單，但可能導致收斂停滯——關節被卡在邊界上，而演算法仍不斷推它往外。

方式二：加權迴避。當某關節接近極限時，增大其在 W 矩陣中的權重，使演算法傾向於少動它、多動其他關節。這是較優雅的作法，需要冗餘自由度才有效。

方式三：零空間投影。對冗餘機器人，利用 10.7.2 節的零空間項，把「遠離關節極限」設為次要目標。這樣主任務完全不受影響，次要目標在剩餘的自由度中被最佳化。

10.8.3 姿態誤差的表示

最後一個實務要點：位置誤差是簡單的向量相減，但**姿態誤差該怎麼表示**？這是初學者實作時最常卡住的地方。

錯誤的作法是把 RPY 角相減——這在角度跨越 $\pm 180^\circ$ 時會產生巨大的假誤差（ 179° 與

−179° 實際只差 2°，相減卻得到 358°)。

正確的作法有兩種。**軸角法**：計算 $R_{\text{error}} = R_{\text{current}}^T \cdot R_{\text{target}}$ ，再由此矩陣取出軸角表示（第 8.10.3 節），得到的 $\theta \cdot k$ 即為三維的姿態誤差向量，其大小恰為需要旋轉的角度。**四元數法**：計算 $q_{\text{error}} = q_{\text{current}} * \otimes q_{\text{target}}$ ，取其向量部（乘以 2）作為誤差向量，並注意處理雙重覆蓋（若純量部為負則取 $-q_{\text{error}}$ ）。

兩種方法都能給出「大小正確、方向正確」的姿態誤差向量，且在任何姿態下都不會有跳變問題。這也再次印證了第 8 章的結論——內部計算應使用四元數或旋轉矩陣，而非歐拉角。

10.9 封閉解與數值解的選擇

10.9.1 效能與特性的比較

兩類方法的差異，可以用一張表清楚呈現：

表 10-5 封閉解與數值解的全面比較

面向	封閉解	數值解
適用機構	僅滿足 Pieper 準則者	任意機構
求解時間	微秒等級，固定	毫秒等級，不固定
解的完整性	求出全部解（如 8 組）	只得一組（依初始猜測）
收斂保證	必定得解或判定無解	可能不收斂
奇異點附近	解合併，但數值穩定	需 DLS 才穩定
額外約束	難以納入	容易納入（最佳化框架）
冗餘機器人	無法處理（解無限多）	天然支援（零空間）
開發成本	高（需逐機構推導）	低（通用程式碼）
典型使用者	工業機器人控制器	研究、模擬、冗餘與行動操作

10.9.2 實務上的混合策略

成熟的機器人系統往往結合兩者。常見的混合策略有三種：

策略一：封閉解為主、數值解為輔。對標準六軸手臂使用封閉解求出八組解，再用簡單的規則篩選；只有在加裝了第七軸或外部行走軸（使系統變成冗餘）時，才切換到數值解。

策略二：封閉解提供初始猜測。先用簡化模型的封閉解得到一個近似答案，再用數值法對完

整模型（含工具偏移、標定修正）做精細收斂。這能大幅減少迭代次數。

策略三：分層處理。位置用封閉解（快），姿態與額外約束用數值最佳化（靈活）。這在視覺伺服與力控制中很常見。

10.9.3 現代趨勢：學習式逆運動學

近年出現第三條路線：用神經網路直接學習 $x \rightarrow q$ 的映射。訓練時用正向運動學大量產生 (q, x) 資料對（這是免費且無限的資料，因為正向運動學計算極快），再訓練網路做反向預測。

優點是推論極快（一次前向傳播）、天然處理冗餘、且能學到「人類覺得自然」的姿勢偏好（若訓練資料來自人類示範）。缺點是精度通常不如傳統方法（需要後接數值修正）、且缺乏收斂保證與可解釋性。目前多用於**提供高品質的初始猜測**，而非完全取代傳統方法。第 31 章與第 33 章會回到這個主題，討論學習如何與傳統運動學互補。

10.10 軌跡上的逆運動學

10.10.1 連續求解的三個要求

實務上很少只求解單一個點——通常是對一整條軌跡上的數百個點連續求解。這帶來三個額外要求。

要求一：構型連續性。整條軌跡應使用同一組構型分支。若中途由「肘上」切換到「肘下」，手臂必須做一次大幅度的翻轉動作——耗時、危險，且末端會偏離預定路徑。作法是：每個點都選擇離前一點最近的解。

要求二：關節速度連續。相鄰兩點的關節角度差應平順變化。若某處出現跳變，代表可能跨越了奇異點或切換了構型，應在執行前偵測並處理。

要求三：全程可達。應在執行前對整條軌跡預先求解一遍（離線驗證），確認每一點都有解且不超出關節極限。工業機器人的程式在下載到控制器時，通常會做這種預檢。

10.10.2 奇異點穿越問題

若軌跡必須經過或接近奇異點，會發生第 12 章要詳述的問題：某些關節需要以極高的速度轉動。處理方式有四種：

- **繞道：**修改路徑，讓它避開奇異點附近的區域。這是最安全的作法，但末端路徑會與原始規

劃有偏差。

- **減速**：接近奇異點時自動降低末端速度，使關節速度維持在限制內。這保持了路徑精度，但節拍會拉長。
- **DLS 阻尼**：使用 10.7.3 節的阻尼法，容許末端稍微偏離路徑以換取關節速度的可控。
- **改變構型或基座位置**：若機器人裝在行走軸上，可移動基座使目標落在較「舒服」的區域。

工業控制器通常內建這些機制，並在觸發時向操作員發出警告（如「接近奇異點，速度已降低」）。理解這些訊息的意義，是機器人程式設計師的基本素養。

10.11 由位置邁向速度

10.11.1 本章的成果與遺留問題

本章完成了機器人運動學的第二塊拼圖。你現在能夠：判斷目標是否可達、用幾何與代數法求出平面手臂的封閉解、理解六軸手臂的八組解結構與解耦原理、實作數值迭代求解、並用 DLS 處理奇異點附近的數值問題。

但本章刻意留下了一個未解的環節：**雅可比矩陣 J 究竟是什麼？怎麼算出來？**我們一直把它當成黑盒子使用，只知道它把關節速度映射到末端速度。這正是下一章的主題。

10.11.2 接下來：雅可比與奇異點

第 11 章：雅可比矩陣。我們將從頭推導 J 的每一個元素，理解它的兩種形式（幾何雅可比與解析雅可比），並發現一個美妙的對偶性——同一個矩陣不只連結速度，還連結**力與力矩**（ J^T 把末端受力映射到關節扭矩）。這使雅可比成為連接運動學與力學的橋樑，也是第 13 章動力學與第 20 章力控制的基礎。

第 12 章：奇異點與冗餘。本章多次提到「奇異點」——手臂完全伸直、兩軸對齊、解合併、數值爆炸。第 12 章將用雅可比矩陣的**秩與奇異值分解**，為這些現象給出統一的數學解釋，並說明冗餘機器人如何利用零空間繞開它們。

換句話說：第 9 章算出手在哪裡、第 10 章決定手該怎麼擺、第 11 章計算手能怎麼動、第 12 章解釋手何時動不了。四章合起來，才是完整的操作臂運動學。

完整計算例題

例題 10-1 平面 2R 手臂的完整逆解

題目：某平面 2R 手臂 $L_1 = 300 \text{ mm}$ 、 $L_2 = 200 \text{ mm}$ （與例題 9-1 相同）。目標末端位置為 $(311.57, 343.19) \text{ mm}$ 。（a）判斷是否可達。（b）求出所有解。（c）驗證其中一組解，並說明兩組解的機構意義。

解（a）：先求目標到基座的距離：

$$r = \sqrt{(311.57^2 + 343.19^2)} = \sqrt{(97076 + 117779)} = \sqrt{214855} \approx 463.52 \text{ mm}$$

目標距離。

可達範圍為 $|300 - 200| = 100 \leq r \leq 300 + 200 = 500$ 。463.52 落在此範圍內，目標可達。

解（b）：先求 θ_2 。代入 10.3.1 節的公式：

$$\begin{aligned} \cos \theta_2 &= (r^2 - L_1^2 - L_2^2) / (2 L_1 L_2) = (214855 - 90000 - 40000) / (2 \times 300 \times 200) \\ &= 84855 / 120000 = 0.70713 \end{aligned}$$

由於 $|0.70713| < 1$ ，確認有實數解。

$$\theta_2 = \pm \arccos(0.70713) = \pm 45.00^\circ$$

兩組解： $\theta_2 = +45^\circ$ （肘下）與 $\theta_2 = -45^\circ$ （肘上）。

接著求對應的 θ_1 。目標方位角為 $\text{atan2}(343.19, 311.57) = 47.76^\circ$ 。

解一（ $\theta_2 = +45^\circ$ ）：

$$\begin{aligned} \theta_1 &= 47.76^\circ - \text{atan2}(200 \sin 45^\circ, 300 + 200 \cos 45^\circ) \\ &= 47.76^\circ - \text{atan2}(141.42, 441.42) = 47.76^\circ - 17.76^\circ = 30.00^\circ \end{aligned}$$

第一組解： $(\theta_1, \theta_2) = (30^\circ, 45^\circ)$ 。

解二（ $\theta_2 = -45^\circ$ ）：

$$\begin{aligned} \theta_1 &= 47.76^\circ - \text{atan2}(200 \sin(-45^\circ), 300 + 200 \cos(-45^\circ)) \\ &= 47.76^\circ - \text{atan2}(-141.42, 441.42) = 47.76^\circ - (-17.76^\circ) = 65.52^\circ \end{aligned}$$

第二組解： $(\theta_1, \theta_2) = (65.52^\circ, -45^\circ)$ 。

解（c）：驗證解一。用第 9 章的正向運動學代回：

$$\begin{aligned} x &= 300 \cos 30^\circ + 200 \cos 75^\circ = 259.81 + 51.76 = 311.57 \checkmark \\ y &= 300 \sin 30^\circ + 200 \sin 75^\circ = 150.00 + 193.19 = 343.19 \checkmark \end{aligned}$$

與目標完全吻合，且正是例題 9-1 的輸入角度——正向與逆向互為反運算，驗證成功。

機構意義：解一（ $30^\circ, 45^\circ$ ）的肘關節位於基座與末端連線的下方；解二（ $65.52^\circ, -45^\circ$ ）

的肘關節位於上方。兩者末端位置相同、末端姿態不同 (φ 分別為 75° 與 20.52°) ——這說明平面 2R 手臂無法獨立控制位置與姿態 (只有兩個自由度卻有三個輸出量，見第 9.6.1 節)。

例題 10-2 工作空間邊界與無解判定

題目：同上手臂。(a) 目標 (480, 200) 是否可達？(b) 目標 (500, 0) 的解為何？有幾組？(c) 目標 (50, 50) 是否可達？

解 (a)： $r = \sqrt{(480^2 + 200^2)} = \sqrt{(230400 + 40000)} = \sqrt{270400} = 520.00 \text{ mm}$ 。超過最大可達距離 500 mm，不可達。

若強行代入公式： $\cos \theta_2 = (270400 - 130000)/120000 = 1.1700 > 1$ ， $\arccos$ 無實數解。程式必須在此處攔截並回報錯誤，而非讓 NaN 傳遞下去——這正是開頭故事中「手臂撞爛」的根源。

解 (b)： $r = 500 \text{ mm}$ ，恰為最大可達距離。

$$\cos \theta_2 = (250000 - 130000)/120000 = 1.0000 \Rightarrow \theta_2 = 0^\circ$$

只有一個解 ($\pm 0^\circ$ 相同)，兩組解合而為一。

$\theta_1 = \text{atan2}(0, 500) - \text{atan2}(0, 500) = 0^\circ$ 。因此唯一解為 $(0^\circ, 0^\circ)$ ，手臂完全伸直。這是邊界奇異點：在此處，手臂失去了沿徑向運動的能力 (無論關節怎麼轉，末端都無法再往外)，且逆解由兩組退化為一組。第 12 章會用雅可比矩陣的秩來精確描述這個現象。

解 (c)： $r = \sqrt{(2500 + 2500)} = \sqrt{5000} \approx 70.71 \text{ mm}$ 。小於最小可達距離 100 mm，不可達——這是工作空間中央的「空洞」(第 4.10.3 節)。 $\cos \theta_2 = (5000 - 130000)/120000 = -1.0417 < -1$ ，同樣無實數解。

工程結論：完整的可達性檢查必須同時檢驗上下界。實務程式應在計算 $\arccos$ 之前先檢查其引數是否落在 $[-1, 1]$ ，並依 10.2.2 節的四種方式處置。

例題 10-3 數值逆運動學的迭代過程

題目：以平面 2R 手臂 ($L_1 = 300$ 、 $L_2 = 200$) 為例，目標 (311.57, 343.19)，初始猜測 $q_0 = (0^\circ, 0^\circ)$ 。以偽逆法、步長 $\alpha = 1.0$ 進行迭代，說明前兩次迭代的計算過程。(平面 2R 的雅可比矩陣為下式，第 11 章將推導其來源。)

$$J = \begin{bmatrix} -L_1 s_1 & -L_2 s_{12} & -L_2 s_{12} \end{bmatrix}$$

$$\begin{bmatrix} L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix}$$

平面 2R 手臂的雅可比矩陣 (僅位置部分, 2×2)。

迭代一： $q = (0^\circ, 0^\circ)$ ，正向運動學得 $f(q) = (500, 0)$ 。

$$e = (311.57, 343.19) - (500, 0) = (-188.43, 343.19)$$

殘差向量，其範數約 391.5 mm。

在 $q = (0, 0)$ 處， $s_1 = s_{12} = 0$ 、 $c_1 = c_{12} = 1$ ，故：

$$J = \begin{bmatrix} 0 & 0 \\ 500 & 200 \end{bmatrix}$$

此處 J 的第一列全為零——矩陣奇異！

這正是例題 10-2(b) 所揭示的邊界奇異點。 J 不可逆，偽逆法在此**無法計算**。這是數值方法的一個重要教訓：**初始猜測不可選在奇異點上**。改用 DLS ($\lambda = 10$) 或改變初始猜測即可解決。

改用初始猜測 $q_0 = (10^\circ, 10^\circ)$ ： $f(q) = 300\cos 10^\circ + 200\cos 20^\circ = 295.44 + 187.94 = 483.38$ ；
 $y = 300\sin 10^\circ + 200\sin 20^\circ = 52.09 + 68.40 = 120.49$ 。

$$e = (311.57 - 483.38, 343.19 - 120.49) = (-171.81, 222.70)$$

殘差範數約 281.3 mm。

計算 J 並求解 $\Delta q = J^{-1} \cdot e$ ，可得修正量 (此處省略數值細節)，更新後的 q 會朝正確方向移動。實際執行本程式可觀察到：約 5 至 8 次迭代後，殘差降至 0.01 mm 以下，收斂到解 $(30^\circ, 45^\circ)$ 或 $(65.52^\circ, -45^\circ)$ ——收斂到哪一組，取決於初始猜測落在哪一組解的「吸引域」中。

工程啟示：這個例題展示了數值法的三個實務要點——一、初始猜測極為關鍵；二、奇異點會使偽逆法失效，DLS 是必要的保險；三、數值法只給一組解，若需要全部解必須多重起始。

程式範例：封閉解與數值解的實作與比較

以下 Python 程式同時實作平面 2R 手臂的封閉解與三種數值解法，並比較其精度、迭代次數與在奇異點附近的行為。

```
import numpy as np

L1, L2 = 300.0, 200.0          # 連桿長度 (mm)

# ----- 正向運動學 -----
def fk(q):
```

```

t1, t2 = q
x = L1*np.cos(t1) + L2*np.cos(t1+t2)
y = L1*np.sin(t1) + L2*np.sin(t1+t2)
return np.array([x, y])

# ----- 雅可比矩陣 (第 11 章將推導) -----
def jacobian(q):
    t1, t2 = q
    s1, c1 = np.sin(t1), np.cos(t1)
    s12, c12 = np.sin(t1+t2), np.cos(t1+t2)
    return np.array([
        [-L1*s1 - L2*s12, -L2*s12],
        [ L1*c1 + L2*c12,  L2*c12]
    ])

# ----- 封閉解 (10.3 節) -----
def ik_closed_form(target):
    x, y = target
    r2 = x*x + y*y
    r = np.sqrt(r2)

    # 可達性檢查 (10.2.1 節)
    if r > L1 + L2 + 1e-9:
        return [], f"不可達：距離 {r:.2f} > 最大 {L1+L2:.2f} mm"
    if r < abs(L1 - L2) - 1e-9:
        return [], f"不可達：距離 {r:.2f} < 最小 {abs(L1-L2):.2f} mm"

    c2 = (r2 - L1*L1 - L2*L2) / (2*L1*L2)
    c2 = np.clip(c2, -1.0, 1.0) # 防止浮點誤差造成 arccos 失效
    solutions = []
    for sign in (+1, -1): # 肘下 / 肘上兩組解
        t2 = sign * np.arccos(c2)
        t1 = np.arctan2(y, x) - np.arctan2(L2*np.sin(t2), L1 + L2*np.cos(t2))
        solutions.append(np.array([t1, t2]))
        if abs(np.sin(t2)) < 1e-9: # 奇異點：兩解合一
            return [solutions[0]], "邊界奇異點：兩組解合而為一"
    return solutions, "OK"

# ----- 數值解：三種方法 -----
def ik_numerical(target, q0, method="dls", lam=10.0,
                  alpha=1.0, tol=1e-3, max_iter=200):
    q = np.array(q0, dtype=float)
    for k in range(max_iter):
        e = target - fk(q)
        if np.linalg.norm(e) < tol:
            return q, k, True
        J = jacobian(q)
        if method == "transpose":
            dq = J.T @ e * 1e-6 # 梯度下降需極小步長
        elif method == "pinv":
            dq = np.linalg.pinv(J) @ e
        else:
            # DLS (10.7.3 節)
            dq = J.T @ np.linalg.solve(J @ J.T + lam**2*np.eye(2), e)

```

```

        q = q + alpha * dq
    return q, max_iter, False

d, r2d = np.deg2rad, np.rad2deg

# ----- 驗證一：例題 10-1 -----
print("=== 例題 10-1：封閉解 ===")
target = np.array([311.57, 343.19])
sols, msg = ik_closed_form(target)
print(f"狀態: {msg}")
for i, s in enumerate(sols, 1):
    name = "肘下" if s[1] > 0 else "肘上"
    print(f" 解{i}({name}):  $\theta_1={r2d(s[0]):7.2f}^\circ$   $\theta_2={r2d(s[1]):7.2f}^\circ$ "
          f" 驗算  $fk={np.round(fk(s),2)}$ ")

# ----- 驗證二：例題 10-2 無解與奇異點 -----
print("\n=== 例題 10-2：可達性判定 ===")
for tgt in ([480, 200], [500, 0], [50, 50]):
    sols, msg = ik_closed_form(np.array(tgt, dtype=float))
    print(f" 目標 {tgt}: {msg} → 解的數目 = {len(sols)}")

# ----- 驗證三：三種數值方法比較 -----
print("\n=== 例題 10-3：數值方法比較 ( 初始猜測  $10^\circ, 10^\circ$  ) ===")
q0 = [d(10), d(10)]
for m in ("transpose", "pinv", "dls"):
    q, it, ok = ik_numerical(target, q0, method=m)
    err = np.linalg.norm(target - fk(q))
    print(f"  {m:10s}: 迭代 {it:3d} 次 收斂={ok} "
          f" 誤差={err:8.4f} mm 解=({r2d(q[0]):6.2f}°, {r2d(q[1]):6.2f}°)")

# ----- 驗證四：奇異點上的初始猜測 -----
print("\n=== 奇異點初始猜測 ( $0^\circ, 0^\circ$ ) 的行為 ===")
for m in ("pinv", "dls"):
    q, it, ok = ik_numerical(target, [0.0, 0.0], method=m)
    err = np.linalg.norm(target - fk(q))
    print(f"  {m:10s}: 迭代 {it:3d} 次 收斂={ok} 誤差={err:10.4f} mm")
print(" → 偽逆法在奇異點上失效，DLS 仍能脫離並收斂")

# ----- 應用：軌跡上的連續求解 ( 10.10 節 ) -----
print("\n=== 軌跡連續求解：以前一點的解作為初始猜測 ===")
q = np.array([d(30), d(45)])
for i, ang in enumerate(np.linspace(0, np.pi/2, 5)):
    tgt = np.array([400*np.cos(ang) + 50, 400*np.sin(ang) + 50])
    q, it, ok = ik_numerical(tgt, q, method="dls")
    print(f"  點{i}: 目標=({tgt[0]:6.1f},{tgt[1]:6.1f}) "
          f" 迭代={it:2d}  $\theta=({r2d(q[0]):6.2f}^\circ, {r2d(q[1]):6.2f}^\circ)$ ")
print(" → 迭代次數少且構型連續，未發生肘上/肘下切換")

```

請試著修改：(一) 把 DLS 的 λ 由 10 改為 0.1 與 100，觀察收斂速度與穩定性的變化。(二) 把軌跡連續求解改成每次都用固定初始猜測 $(0^\circ, 0^\circ)$ ，觀察是否會發生構型跳變。(三) 在 `ik_closed_form` 中移除 `np.clip`，並輸入 $(480, 200)$ ，觀察 NaN 如何在系統中傳播——這正是實務上必須做可達性檢查的原因。

實作：手臂逆運動學驗證

本章實作是為實體手臂實作逆運動學並驗證其精度。建議使用附錄 C~E 的手臂，或第 9 章實作中已建立運動學模型的任一支手臂。

1. **沿用第 9 章的模型**：取用你在第 9 章建立的 DH 參數表與正向運動學程式。
2. **實作封閉解**：若手臂為平面型或三軸型，依 10.3 節推導其封閉解；若構型複雜，直接進入下一步。
3. **實作數值解**：以本章程式為基礎，實作 DLS 數值逆運動學。雅可比矩陣可先用數值微分近似（對每個關節加一個微小擾動，觀察末端位置變化量除以擾動量），第 11 章學會解析法後再替換。
4. **單點驗證**：選定一個工作空間內的目標點，求解後把角度送給手臂，用尺量測實際末端位置，記錄誤差。
5. **多解觀察**：若手臂有多組解，分別執行不同構型的解，拍照記錄兩種姿勢，並觀察哪一組會發生碰撞或超出關節極限。
6. **邊界測試**：逐步把目標點往外移，直到程式回報不可達。記錄實際的最大可達距離，並與理論值比較（差異來自關節極限與機構干涉）。
7. **軌跡測試**：規劃一段直線或圓弧軌跡（取 20~50 個點），連續求解並執行。觀察是否發生構型跳變或速度突變，並記錄整段軌跡的最大誤差。
8. **誤差來源分析**：將誤差分解為運動學模型誤差（第 9.9 節）、逆解收斂容差、以及機構背隙與剛性三部分，估算各自的貢獻。

表 10-6 逆運動學驗證記錄表（第一列為範例）

測試項	目標位置	求得關節角	實測位置	誤差	備註
單點（工作空間中央）	(250,	(28.4°,	(247,	4.2	肘下構型，正

測試項	目標位置	求得關節角	實測位置	誤差	備註
	150)	62.1°)	153)	mm	常
(自行填寫：多解、邊界、軌跡各項)					

國中小也看得懂：伸手拿東西的三個麻煩

伸手去拿桌上的橡皮擦——很簡單對吧？但你有沒有想過，你的手臂是怎麼「決定」肩膀要轉幾度、手肘要彎幾度的？你從來沒有算過，卻每次都能拿到。

機器人可沒這麼幸運，它必須用數學算出來。而且這個計算有三個麻煩的地方：

第一個麻煩：答案不只一個。 試試看——把手掌放在橡皮擦上不要動，然後把手肘抬高、再放低。手掌一直在同一個地方，手臂卻可以有好多種姿勢！機器人也一樣，它必須決定「要用哪一種姿勢過去」。有時候某一種姿勢會撞到桌子，就要換另一種。

第二個麻煩：有時候根本拿不到。 把橡皮擦推到桌子最遠的地方，你伸長手臂還是構不到。這時候人會說「我拿不到」然後放棄，但機器人如果沒寫好程式，它可能會硬要伸過去，結果就是撞壞自己。所以工程師一定要先幫機器人檢查：「這個地方我到得了嗎？」

第三個麻煩：算起來很難。 從角度算位置很簡單（第 9 章學的），但反過來從位置算角度，就像解一道很複雜的謎題。有些機器人的形狀設計得很聰明，可以直接套公式；有些就只能用「猜一猜、修一修、再猜一猜」的方法，一直逼近正確答案。你在附錄 J 做的機器狗，每走一步都要算四隻腳的角度，用的就是這一章的方法喔！

叫獸提醒與常見錯誤

- 誤區一：不做可達性檢查就求解。 $\arccos$ 的引數超出 $[-1, 1]$ 會產生 NaN，而 NaN 一旦傳到馬達指令就是災難。務必在計算前檢查並攔截。
- 誤區二：用 $\arcsin$ 或 $\arccos$ 求角度而非 $\operatorname{atan2}$ 。前兩者會遺失象限資訊，導致解在特定區域錯誤。只要 $\sin$ 與 $\cos$ 都已知，就該用 $\operatorname{atan2}$ 。
- 誤區三：忽略多解的存在。求出一組解就直接執行，可能得到會碰撞或超出關節極限的姿勢。應求出全部解再篩選。

- 誤區四：軌跡上每點都用固定初始猜測。這會導致構型跳變（肘上突然變肘下），手臂大幅甩動。應以前一點的解作為下一點的初始猜測。
- 誤區五：用偽逆法而不加阻尼。接近奇異點時會產生極大的關節速度指令，實體機器上極其危險。實務應使用 DLS。
- 誤區六：把姿態誤差用 RPY 角相減。角度跨越 $\pm 180^\circ$ 時會產生巨大的假誤差。應改用軸角或四元數表示姿態誤差。
- 誤區七：以為數值解一定會收斂。初始猜測不佳、目標不可達或落在奇異點上，都可能不收敛。必須設定迭代上限並處理失敗情形。
- 誤區八：忘記工具偏移。逆解求的通常是法蘭位姿，若程式給的是 TCP 目標，必須先扣除工具變換（第 8.9.1 節）。
- 誤區九：數值解的容差設得太嚴。要求 $1e-12$ 的精度會導致迭代次數暴增甚至無法收斂；實務上位置容差取 $0.01 \sim 0.1 \text{ mm}$ 即足夠。
- 誤區十：以為「數學上可達」就是「實際可達」。還須通過關節極限、自我碰撞與環境碰撞三重檢查。

課堂討論

1. 開頭故事中，小好兩用兩種姿勢拿到同一個杯子。若你是控制器的設計者，會用什麼準則來決定選哪一組解？請列出至少三個準則並排出優先順序。
2. 六軸手臂有八組解，但工業控制器通常只會執行其中一組。若操作員抱怨「機器人用了一個奇怪的姿勢過去」，你會如何診斷與處理？
3. DLS 犧牲末端精度來換取關節速度的可控。請設計一個情境說明這個取舍是值得的，再設計一個情境說明它可能造成問題。
4. 學習式逆運動學（用神經網路）目前主要用於提供初始猜測，而非完全取代傳統方法。請從精度、收斂保證、可解釋性與安全性四個角度討論原因。
5. 冗餘機器人的逆解有無限多組。這聽起來像是麻煩，但第 4.9.2 節說它是優點。請說明「無限多解」如何從問題轉變為工具，並舉出兩個實際應用。

章末摘要

- 逆向運動學由末端位姿求關節變數，是機器人實際作業時真正需要回答的問題。
- 與正向運動學三個本質差異：解可能不存在、可能不唯一、可能沒有封閉解。
- 求解前必須做可達性檢查；平面 2R 的可達條件為 $|L_1 - L_2| \leq r \leq L_1 + L_2$ ，超出範圍時 $\arccos$ 的引數會超出 $[-1, 1]$ 。
- 無解時的四種處置：拒絕報錯、投影到最近可達點、放寬姿態要求、移動基座。
- 實務上「假不可達」的三大原因：關節極限、自我碰撞、姿態要求過嚴。
- 平面 2R 的封閉解由餘弦定理求 θ_2 ($\pm$ 兩解，對應肘上 / 肘下)，再由 atan2 求 θ_1 。
- 代數解法的核心策略是「逐步剝離」——兩邊同乘變換的反矩陣，使未知數單獨出現。
- 多解來自推導中的開方或反餘弦分岔；六軸手臂三次分岔產生 $2^3 = 8$ 組解，對應肩前後、肘上下、腕翻轉。
- Pieper 準則：三個相鄰軸交於一點或互相平行時，存在封閉解；球形手腕即為此設計。
- 運動學解耦把六自由度問題拆為位置（前三軸）與姿態（後三軸）兩個三自由度子問題。
- 數值法以 $\Delta q = J^+ \cdot e$ 迭代逼近，內部反覆呼叫正向運動學；初始猜測決定收斂到哪一組解。
- 三種數值方法：轉置法（穩定但慢）、偽逆法（快但奇異點爆炸）、DLS（實務首選，以阻尼換穩定）。
- 冗餘機器人可用零空間投影 $(I - J^+J) \cdot z$ 加入次要目標，不影響主任務。
- 姿態誤差不可用 RPY 相減，應以軸角或四元數表示，避免 $\pm 180^\circ$ 跳變。
- 軌跡連續求解須維持構型連續性、關節速度連續與全程可達；以前一點的解作為初始猜測是標準作法。
- 封閉解快速完整但機構受限，數值解通用靈活但較慢；實務常採混合策略。

觀念題與計算題

觀念題

1. 說明逆向運動學與正向運動學的三個本質差異，並各舉一個實例。
2. 為何逆向運動學可能無解？列出四種無解時的處置方式與各自適用的場合。
3. 說明實務上「假不可達」的三個常見原因，並各提出一種診斷方法。
4. 以平面 2R 手臂說明兩組解的幾何意義，並列出三種選解準則。

5. 為何求解角度時必須使用 atan2 而非 $\arcsin$ 或 $\arccos$?
6. 說明代數解法「逐步剝離」的策略，並寫出其關鍵的矩陣運算式。
7. 六軸手臂的八組解由哪三次分岔產生？各對應什麼機構意義？
8. 敘述 Pieper 準則，並說明球形手腕為何是刻意的設計決策。
9. 說明運動學解耦的核心洞見，以及手腕中心點如何由末端位姿求得。
10. 推導數值逆運動學的迭代公式，並說明雅可比矩陣在其中的角色。
11. 比較 Jacobian 轉置法、偽逆法與 DLS 的收斂速度、奇異點行為與適用場合。
12. 說明 DLS 中阻尼係數 λ 的物理意義，以及可變阻尼策略的作法。
13. 冗餘機器人的零空間投影項 $(I - J^+J) \cdot z$ 有什麼作用？舉兩個 z 的具體選擇。
14. 為何姿態誤差不可用 RPY 角相減？請說明正確的兩種表示方式。
15. 軌跡上連續求解有哪三個要求？為何應以前一點的解作為初始猜測？

計算題

1. 某平面 2R 手臂 $L_1 = 400 \text{ mm}$ 、 $L_2 = 250 \text{ mm}$ 。目標位置為 $(300, 400) \text{ mm}$ 。(a) 判斷可達性。(b) 求出全部解（角度以度為單位，取至小數點後兩位）。(c) 分別計算兩組解的末端姿態角 φ ，並說明為何不同。(d) 若目標改為 $(600, 300)$ ，是否可達？請說明。
2. 同上手臂。(a) 求工作空間的內外半徑。(b) 當目標恰在外邊界時，有幾組解？此時 θ_2 為何？(c) 當目標恰在內邊界時， θ_2 為何？此姿態在實體上可能有什麼問題？(d) 若把 L_2 改為 400 mm （兩連桿等長），內邊界半徑變為多少？這帶來什麼好處與代價？
3. 某六軸手臂的末端目標位姿為：位置 $p = (500, 200, 400) \text{ mm}$ ，姿態的旋轉矩陣第三行（接近向量 $\hat{a}$ ）為 $(0, 0, -1)^T$ 。已知手腕中心到末端的距離 $d_6 = 100 \text{ mm}$ 。(a) 求手腕中心點 p_{wrist} 的位置。(b) 說明接下來的求解步驟（不需實際計算）。(c) 若 d_6 量測錯誤（實際為 120 mm 卻用 100 mm 計算），估算末端位置的誤差量與方向。
4. 以平面 2R 手臂（ $L_1 = 300$ 、 $L_2 = 200$ ）進行數值逆運動學。目標 $(400, 100)$ ，初始猜測 $(20^\circ, 30^\circ)$ 。(a) 計算初始的正向運動學位置與殘差向量。(b) 計算此處的雅可比矩陣。(c) 以偽逆法求第一次迭代的 Δq （步長 $\alpha = 1.0$ ），並求更新後的關節角度。(d) 計算更新後的殘差範數，確認是否減小。

動手做與專題延伸

本章實作：逆運動學求解器

延續第 9 章的正向運動學程式，實作完整的逆運動學求解器。建議依下列順序完成，每一步都應通過測試再進入下一步。

- **階段一**：平面 2R 的封閉解，含完整的可達性檢查與兩組解的輸出。以第 9 章的正向運動學驗證（求解後代回，應得到原目標）。
- **階段二**：平面 3R 的封閉解。注意這是冗餘機構（三自由度、三個輸出但位置只需兩個），需先指定末端姿態角 φ ，才能求出唯一解。
- **階段三**：通用的數值 DLS 求解器，支援任意 DH 參數表。雅可比先用數值微分實作。
- **階段四**：加入關節極限處理與多重起始策略，使程式能找出多組解並篩選出可行者。

專題延伸

- **互動式逆運動學展示**：延續第 9 章的視覺化工具，改為滑鼠點選目標點，程式即時求解並繪出手臂姿態。同時顯示所有解（以不同顏色區分構型），並在目標不可達時以紅色標示工作空間邊界。
- **吸引域視覺化**：對平面 2R 手臂，在關節空間 (θ_1, θ_2) 中均勻取樣數千個初始猜測，記錄每個猜測最終收斂到哪一組解，並以兩種顏色繪出。你會看到一幅美麗的「吸引域」圖案，直觀理解為何初始猜測如此關鍵。
- **進階：三種數值方法的效能競賽**。設計一組測試案例（含遠離奇異點、接近奇異點、通過奇異點三類），對轉置法、偽逆法與不同 λ 的 DLS 分別測試，記錄迭代次數、最終誤差、最大關節速度與失敗率。將結果整理成圖表，並依此提出「在什麼情況下該用哪一種方法」的實務建議。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 4 章：逆向運動學，含 PUMA 的完整代數推導)。
- Pieper, D. L., "The Kinematics of Manipulators Under Computer Control," PhD Thesis, Stanford University, 1968 (Pieper 準則的原始出處)。
- Wampler, C. W., "Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods," IEEE Trans. SMC, 1986 (DLS 方

法)。

- Nakamura, Y. and Hanafusa, H., "Inverse Kinematic Solutions with Singularity Robustness," ASME J. Dynamic Systems, 1986 (奇異點強健的逆運動學)。
- Buss, S. R., "Introduction to Inverse Kinematics with Jacobian Transpose, Pseudoinverse and Damped Least Squares Methods," 2009 (三種數值方法的完整比較，極佳的入門講義)。
- Lynch, K. M. and Park, F. C., Modern Robotics, Cambridge University Press (第6章：逆運動學，含數值方法與冗餘處理)。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer (第2、3章：逆運動學與冗餘)。
- Chiaverini, S., Oriolo, G. and Walker, I. D., "Kinematically Redundant Manipulators," in Springer Handbook of Robotics (冗餘機器人的逆運動學專章)。
- IKFast (OpenRAVE 專案) 與 TRAC-IK：兩套廣泛使用的開源逆運動學求解器，前者自動產生封閉解程式碼、後者結合數值最佳化。
- MoveIt 2 官方文件 (moveit.picknik.ai)：ROS 2 生態系中的運動規劃框架，內建多種逆運動學求解器可供比較。

第 11 章

雅可比矩陣、速度與靜力

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】一支六軸手臂末端沿桌面畫出一條等速直線（白色軌跡），軌跡上取三個位置，各疊印一個速度橢球——起點處橢球扁而傾斜、中段接近正圓、終點處被壓扁成細長狀。畫面右下角另疊一組力橢球（紅色），其形狀與同位置的速度橢球恰好「胖瘦相反」。左側示教器螢幕特寫，顯示末端速度固定為 100 mm/s，但關節速度欄位的數字劇烈跳動。標題文字：同一條直線，不斷改變的兌換率。

叫獸開講 「為什麼機器手臂在某些姿勢突然變得很笨重？」

實驗室的六軸手臂正在示範一段直線運動——末端沿著桌面平穩地由左向右滑過，速度固定在每秒 10 公分。叫獸請學生們看示教器上的關節速度顯示。

「注意第二軸與第三軸，」他說。螢幕上的數字不斷跳動：一開始第二軸每秒轉 8 度、第三軸每秒 -12 度；到了中段，變成 15 度與 -5 度；快到終點時，第二軸飆到 34 度而第三軸幾乎停住。

小翰皺眉：「末端明明是等速直線，為什麼關節速度變來變去？」

「因為它們之間不是固定的比例關係。」叫獸把手臂停在中段，「同樣是第二軸轉一度，在這個姿態下末端會移動 6 公釐；」他把手臂擺成接近伸直的姿態，「在這個姿態下，同樣轉一度，末端會移動 9 公釐。兩者的兌換率，隨姿態而變。」

他在白板上畫了一個方框：「這個兌換率，就是今天的主角——雅可比矩陣。它是一個會隨姿態改變的矩陣，把關節速度翻譯成末端速度。而它之所以重要，有三個理由。」

「第一，」他寫下數字 1，「上一章的數值逆運動學，我們一直把它當黑盒子用。今天要把它拆開。」

「第二，」他把手臂緩緩擺到完全伸直，示教器上某個關節的速度數字瞬間跳成一串驚人的大數，控制器隨即發出警報並停機。「在某些姿態下，這個兌換率會壞掉——末端只要動一點點，關節就得瘋狂旋轉。這就是奇異點，下一章的主題。」

「第三個理由最有趣。」叫獸走到手臂旁，用手推末端的夾爪。示教器上，各關節的電流讀數同步跳動。「我推的是末端，動的卻是關節的扭矩。同一個雅可比矩陣，不只連結速度，也連結力——而且是用它的轉置。這件事看起來像巧合，其實是一個深刻的對偶性，一百多年前的力學家早就發現了。」

他把筆放下：「所以雅可比不只是一個計算工具。它是機器人學裡運動學與力學之間的那座橋。過了這座橋，我們才能談動力學、談力控制、談協作機器人怎麼感覺到你碰它。今天，我們就來把這座橋一根一根搭起來。」

本章為什麼重要

雅可比矩陣是機器人學中使用頻率最高的一個矩陣。第 10 章的數值逆運動學需要它、第 12 章的奇異點分析靠它定義、第 13 章的動力學推導用到它、第 17 章的操作空間控制以它為核心、第 20 章的力控制與碰撞偵測更是直接建立在它的轉置之上。

它的價值來自一個關鍵性質：**線性化**。機器人的正向運動學是高度非線性的（充滿三角函數的乘積），難以直接分析與控制。但當我們只看「微小的變化」時，這個非線性關係就退化成一個線性映射——而這個線性映射的矩陣，就是雅可比。線性代數的所有工具（秩、奇異值、零空間、條件數）從此都能派上用場，這是機器人分析中最重要的一次降維打擊。

本章也將揭示一個貫穿全書的對偶關係：速度與力的對偶。 J 把關節速度映射到末端速度， J^T 則把末端力映射到關節扭矩。理解這個對偶，你會發現許多看似不相干的主題（運動、力覺、剛性、能量）其實都是同一枚硬幣的不同側面。

學習目標

- 能定義雅可比矩陣，並說明它作為「線性化的運動兌換率」的意義。
- 能由正向運動學以偏微分法推導平面手臂的雅可比矩陣。
- 能區分幾何雅可比與解析雅可比，並說明何時該用哪一種。
- 能以幾何法（軸向與位置向量的外積）直接寫出旋轉關節與移動關節的雅可比行。
- 能說明角速度為何不是姿態角的導數，以及這對雅可比的影響。
- 能運用虛功原理推導 $\tau = J^T \cdot F$ ，並說明速度與力的對偶性。
- 能說明雅可比在不同座標系之間的轉換方式。
- 能計算並解釋可操作度、速度橢球與力橢球的意義。
- 能說明雅可比在逆運動學、力控制與碰撞偵測中的應用。
- 能以 Python 實作解析與數值兩種雅可比計算並驗證其一致性。

先備知識

第 8 章的旋轉矩陣與齊次變換、第 9 章的正向運動學與 DH 參數、第 10 章對雅可比的初步使用。數學方面需要偏導數、向量外積、矩陣乘法與轉置的概念；11.8 節會用到特徵值與奇異值的基本觀念，但會以幾何直覺說明，不需要完整的線性代數理論。

第 5 章的扭力概念與第 8.4.2 節的反對稱矩陣（用於外積轉矩陣運算）在本章會再次出現。

核心名詞中英對照

表 11-1 本章核心名詞

中文	英文	說明
雅可比矩陣	Jacobian Matrix	關節速度到末端速度的線性映射矩陣
幾何雅可比	Geometric Jacobian	以線速度與角速度表示的雅可比
解析雅可比	Analytical Jacobian	以位置與姿態參數導數表示的雅可比
線速度	Linear Velocity	末端位置的時間導數
角速度	Angular Velocity	描述剛體轉動快慢與轉軸的向量
旋量 / 扭量	Twist	線速度與角速度合成的六維速度向量
扳手	Wrench	力與力矩合成的六維力向量
虛功原理	Principle of Virtual Work	由能量守恆推導力與運動關係的原理
對偶性	Duality	速度映射與力映射互為轉置的關係
可操作度	Manipulability	衡量機器人運動靈活程度的純量指標
速度橢球	Velocity Ellipsoid	描述各方向可達速度能力的橢球
力橢球	Force Ellipsoid	描述各方向可輸出力能力的橢球
奇異值分解	Singular Value Decomposition, SVD	將矩陣分解為旋轉—縮放—旋轉的方法
條件數	Condition Number	最大與最小奇異值之比，衡量數值狀況
數值微分	Numerical Differentiation	以有限差分近似導數的方法

11.1 雅可比矩陣的定義

11.1.1 由位置關係到速度關係

第 9 章建立了正向運動學：末端位姿是關節變數的函數。

$$\mathbf{x} = \mathbf{f}(\mathbf{q})$$

$\mathbf{x}$ 為末端位姿（六維：位置三維 + 姿態三維）， $\mathbf{q}$ 為關節變數（ n 維）。 $\mathbf{f}$ 是高度非線性的函數。

現在問一個新問題：若關節以某個速度轉動，末端會以多快的速度移動？把上式對時間微分，依鏈鎖律：

$$\dot{\mathbf{x}} = \partial \mathbf{f} / \partial \mathbf{q} \cdot \dot{\mathbf{q}} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}}$$

這就是雅可比矩陣的定義。 $\mathbf{J}(\mathbf{q}) = \partial \mathbf{f} / \partial \mathbf{q}$ 是一個 $m \times n$ 的矩陣（ m 為任務維度、 n 為關節數），且它是 $\mathbf{q}$ 的函數——會隨姿態改變。

展開來看，雅可比矩陣的每一個元素都是一個偏導數：

$$\mathbf{J} = \begin{bmatrix} \partial f_1 / \partial q_1 & \partial f_1 / \partial q_2 & \cdots & \partial f_1 / \partial q_n \\ \partial f_2 / \partial q_1 & \partial f_2 / \partial q_2 & \cdots & \partial f_2 / \partial q_n \\ \vdots & \vdots & \ddots & \vdots \\ \partial f_m / \partial q_1 & \partial f_m / \partial q_2 & \cdots & \partial f_m / \partial q_n \end{bmatrix}$$

第 i 列第 j 行的元素，表示「第 j 個關節的微小變化，對第 i 個任務座標的影響」。

11.1.2 三種等價的理解方式

雅可比矩陣有三種讀法，能靈活切換是掌握本章的關鍵。

讀法一：速度的兌換率。給定關節速度 $\dot{\mathbf{q}}$ ， $\mathbf{J}$ 告訴你末端速度 $\dot{\mathbf{x}}$ 。這是開頭故事中示教器上顯示的關係——同樣的關節速度，在不同姿態下會產生不同的末端速度。

讀法二：微小位移的映射。把時間微分改寫成微小量： $\Delta \mathbf{x} \approx \mathbf{J} \cdot \Delta \mathbf{q}$ 。這是第 10 章數值逆運動學的基礎——「關節動一點點，末端會跑多遠」。

讀法三：非線性函數的局部線性化。 $\mathbf{J}$ 是 $\mathbf{f}$ 在某個 $\mathbf{q}$ 處的一階泰勒展開係數。非線性的 $\mathbf{f}$ 在局部被一個線性映射近似，這使得所有線性代數的工具都能派上用場。

這三種讀法在數學上完全相同，但在不同場合各有其直覺價值。討論運動控制時用讀法一、討論逆運動學迭代時用讀法二、討論奇異點與可操作度時用讀法三。

11.1.3 為什麼叫「雅可比」

這個名稱來自十九世紀德國數學家雅可比（Carl Gustav Jacob Jacobi）。在多變數微積分中，任何從 $\mathbb{R}^n$ 映射到 $\mathbb{R}^m$ 的函數，其一階偏導數所構成的矩陣就稱為雅可比矩陣。機器人學只是這個通用概念的一個應用——但因為它的物理意義（速度、力）如此豐富，反而成為最廣為人知的應用之一。

值得注意的是，在微積分中雅可比矩陣的行列式（Jacobian determinant）用於變數變換時的體積縮放。這個「體積」的概念在機器人學中同樣有意義——它與 11.8 節的可操作度直接相關。

11.2 偏微分法：由正向運動學直接推導

11.2.1 平面 2R 手臂的完整推導

最直接的推導方式是把正向運動學方程式逐項對每個關節變數偏微分。以平面 2R 手臂為例，第 9.6.1 節已得到：

$$\begin{aligned} x &= L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ y &= L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{aligned}$$

平面 2R 的正向運動學。

對 θ_1 偏微分（注意 θ_1 同時出現在兩項中）：

$$\begin{aligned} \partial x / \partial \theta_1 &= -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2) \\ \partial y / \partial \theta_1 &= L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \end{aligned}$$

第一行的偏導數。

對 θ_2 偏微分（ θ_2 只出現在第二項中）：

$$\begin{aligned} \partial x / \partial \theta_2 &= -L_2 \sin(\theta_1 + \theta_2) \\ \partial y / \partial \theta_2 &= L_2 \cos(\theta_1 + \theta_2) \end{aligned}$$

第二行的偏導數。

把四個偏導數排成矩陣，得到平面 2R 手臂的雅可比：

$$J = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix}$$

這正是第 10 章例題 10-3 中直接給出的矩陣，現在我們知道它從何而來。

11.2.2 解讀矩陣的每一行

這個矩陣的結構有清楚的物理意義。**第一行**（對 θ_1 的偏導）描述：只轉動第一關節時，末端如何移動。觀察其形式 $[-y_{\text{end}}, x_{\text{end}}]^T$ ——這正是「繞原點旋轉」的速度方向（垂直於半徑向量）。**第二行**（對 θ_2 的偏導）形式為 $[-(y_{\text{end}} - y_2), (x_{\text{end}} - x_2)]^T$ ，其中 (x_2, y_2) 是第二關節的位置——這是「繞第二關節旋轉」的速度方向。

這個觀察揭示了一個通則：**每一個旋轉關節對末端速度的貢獻，都是「繞該關節軸旋轉」所產生的速度**。這正是 11.4 節幾何法的核心思想，它讓我們不必做任何微分就能寫出雅可比。

11.2.3 偏微分法的優劣

偏微分法的優點是概念直接、對簡單機構容易操作、且能得到解析表達式（適合理論分析）。缺點是對六軸手臂而言，正向運動學的表達式極為龐大（第 9.8.3 節提到展開後可能數百項），

逐項微分不切實際。

因此實務上有三種替代方案：幾何法（11.4 節，用向量外積直接寫出，最常用）、符號運算軟體（用 SymPy 自動微分，適合理論推導）、以及數值微分（對每個關節加微小擾動，觀察末端變化，程式最容易實作但精度較低）。

11.3 線速度與角速度

11.3.1 六維速度：旋量

空間中的剛體有六個自由度，因此其速度也是六維的：三維的線速度 $\mathbf{v}$ 與三維的角速度 $\boldsymbol{\omega}$ 。兩者合起來稱為**旋量**（twist）：

$$\mathbf{v} = \begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} \in \mathbb{R}^6 \quad \text{完整的雅可比關係：} \mathbf{v} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}}$$

上半部三列稱為位置雅可比 $\mathbf{J}_v$ （或 $\mathbf{J}_P$ ），下半部三列稱為姿態雅可比 $\mathbf{J}_\omega$ （或 $\mathbf{J}_O$ ）。

因此對六軸手臂而言， $\mathbf{J}$ 是一個 $6 \times n$ 的矩陣。若只關心位置（如點焊、取放），可只用上半的 $3 \times n$ ；若只關心姿態（如相機朝向控制），可只用下半的 $3 \times n$ 。

11.3.2 角速度的特殊性

線速度很直觀：它就是位置的時間導數， $\mathbf{v} = \dot{\mathbf{p}}$ 。但**角速度不是任何姿態表示法的導數**——這是本章最容易被忽略、卻最重要的觀念之一。

為什麼？回顧第 8 章：姿態可以用旋轉矩陣（九個數）、歐拉角（三個數）、四元數（四個數）表示。若把旋轉矩陣對時間微分，得到的是一個 3×3 矩陣 $\dot{\mathbf{R}}$ ，不是三維向量。事實上，角速度與旋轉矩陣的關係是：

$$\dot{\mathbf{R}} = [\boldsymbol{\omega}]_\times \cdot \mathbf{R} \quad \text{其中 } [\boldsymbol{\omega}]_\times \text{ 為 } \boldsymbol{\omega} \text{ 的反對稱矩陣 (第 8.4.2 節)}$$

角速度透過反對稱矩陣與旋轉矩陣的導數相關聯。這是角速度的嚴格定義。

角速度的物理意義很明確：它的方向是**瞬時轉軸**、長度是**轉動的角速率**。它是一個真正的物理量，可以直接用陀螺儀量測（第 6.3.1 節）。

相對地，歐拉角的導數（ $\dot{\theta}, \dot{\varphi}, \dot{\psi}$ ）只是三個數字的變化率，**它們不構成一個向量**——不能直接相加、不能做外積、在萬向節鎖處會發散。這正是為什麼機器人學嚴格區分「幾何雅可比」與「解析雅可比」。

◎ 一個容易犯的錯

初學者常寫出「角速度 = RPY 角的導數」這樣的式子並直接使用。這在小角度時近似成立，但在一般姿態下是錯的——兩者之間相差一個依姿態而變的轉換矩陣（11.5.2 節）。若在控制迴路中混用，會導致末端姿態的運動方向偏離預期，而且在接近萬向節鎖時完全失控。判斷方法很簡單：若你的量是由陀螺儀量到的，那是角速度；若是由角度感測器的讀值相減得到的，那是角度導數。

11.3.3 為什麼角速度可以直接相加

角速度有一個極為有用的性質：**多個轉動的角速度可以直接向量相加**。若連桿 2 相對於連桿 1 以 ω_2 轉動、連桿 1 相對於基座以 ω_1 轉動，則連桿 2 相對於基座的角速度為：

$$\omega_{2/\text{基座}} = \omega_1 + \omega_2 \text{ (皆需先表示在同一座標系中)}$$

角速度的可加性。這與有限旋轉不可交換（第 8.7.2 節）形成有趣的對比。

為何有限旋轉不可交換，無窮小的角速度卻可以相加？直觀的解釋是：不可交換性來自二階項，而在無窮小的極限下二階項可以忽略。這正是「微分」的威力——它把複雜的非線性結構，在局部化簡為簡單的線性結構。

這個可加性是 11.4 節幾何法的理論基礎：每個關節的角速度貢獻可以直接相加，不需考慮順序。

11.4 幾何雅可比的直接構造法

11.4.1 核心思想

幾何法不需要任何微分。它的思想是：**分別考慮每一個關節的獨立貢獻，再把它們加起來**。具體而言，第 j 行的雅可比，就是「只讓第 j 個關節以單位速度運動、其餘關節鎖住」時，末端所產生的速度。

這樣做的合法性來自兩點：線速度可以相加（顯然）、角速度也可以相加（11.3.3 節）。因此總速度就是各關節貢獻的向量和，而這正是矩陣乘法 $J \cdot \dot{q}$ 所表達的意思。

11.4.2 旋轉關節的貢獻

設第 j 個關節為旋轉關節，其軸向的單位向量為 $\hat{z}_{j-1}$ （在基座座標系中表示），關節位置為 p_{j-1} ，末端位置為 p_n 。當此關節以角速度 $\dot{\theta}_j$ 轉動時：

角速度貢獻：整個下游連桿繞該軸旋轉，因此角速度就是沿軸向：

$$\omega_j = \dot{z}_{j-1} \cdot \theta_j$$

旋轉關節對末端角速度的貢獻。

線速度貢獻：末端繞著該軸做圓周運動。由剛體運動學，繞軸旋轉所產生的線速度為角速度與半徑向量的外積：

$$v_j = (\dot{z}_{j-1} \times (p_n - p_{j-1})) \cdot \theta_j$$

旋轉關節對末端線速度的貢獻。 $(p_n - p_{j-1})$ 是由關節指向末端的向量。

因此旋轉關節對應的雅可比行為：

$$J_j = \begin{bmatrix} \dot{z}_{j-1} \times (p_n - p_{j-1}) \\ \dot{z}_{j-1} \end{bmatrix} \quad \begin{matrix} \text{(上三列：線速度)} \\ \text{(下三列：角速度)} \end{matrix}$$

旋轉關節的雅可比行。這是本章最實用的一個公式。

11.4.3 移動關節的貢獻

移動關節的情形更簡單。當關節沿軸向 $\dot{z}_{j-1}$ 以速度 d_j 平移時，整個下游連桿隨之平移，但不產生任何轉動：

$$J_j = \begin{bmatrix} \dot{z}_{j-1} \\ 0 \end{bmatrix} \quad \begin{matrix} \text{(上三列：線速度)} \\ \text{(下三列：角速度為零)} \end{matrix}$$

移動關節的雅可比行。

11.4.4 構造雅可比的完整流程

有了上述兩個公式，構造雅可比只需三步：

1. **執行正向運動學：**計算 ${}^0T_1, {}^0T_2, \dots, {}^0T_n$ ，由各矩陣中取出每個關節的軸向 $\dot{z}_{j-1}$ (即 ${}^0R_{j-1}$ 的第三行) 與位置 p_{j-1} (即 ${}^0T_{j-1}$ 的平移欄)。
2. **逐行填入：**對每個關節，依其類型套用 11.4.2 或 11.4.3 節的公式，得到該行的六個元素。
3. **組合成矩陣：**把 n 個行並排，得到 $6 \times n$ 的雅可比矩陣。

這個方法的優雅之處在於：它完全重用了正向運動學的計算結果。既然執行控制時本來就要算正向運動學，雅可比幾乎是「順便」得到的，額外成本極低。這也是實務上一律採用幾何法的原因。

圖 11-1 旋轉關節對末端速度的貢獻

建議配圖：三維示意圖。畫出一支多關節手臂，特別標出第 j 個旋轉關節：其軸向單位向量 $\dot{z}_{j-1}$ (藍色箭頭)、

關節位置 p_{i-1} 與末端位置 p_n 。由關節指向末端的向量 $(p_n - p_{i-1})$ 以虛線表示（綠色）。末端處畫出兩個向量：角速度貢獻 $\hat{z}_{i-1}$ （沿軸向，藍色）與線速度貢獻 $\hat{z}_{i-1} \times (p_n - p_{i-1})$ （垂直於軸與半徑所張平面，紅色）。並以弧形箭頭表示末端繞軸的圓周運動。

11.4.5 以平面 2R 驗證

用幾何法重新計算平面 2R 手臂，驗證與 11.2.1 節的偏微分結果一致。兩個關節軸都沿 Z 方向（垂直紙面），即 $\hat{z}_0 = \hat{z}_1 = [0, 0, 1]^T$ 。

第一行： $p_0 = [0, 0, 0]^T$ （基座）、 $p_n = [x, y, 0]^T$ （末端）。

$$\hat{z}_0 \times (p_n - p_0) = [0, 0, 1] \times [x, y, 0] = [-y, x, 0]^T$$

取前兩個分量得 $[-y, x]^T = [-(L_1 s_1 + L_2 s_{12}), L_1 c_1 + L_2 c_{12}]^T$ ，與偏微分結果的第一行完全相同。

第二行： $p_1 = [L_1 c_1, L_1 s_1, 0]^T$ （第二關節位置）。

$$\hat{z}_1 \times (p_n - p_1) = [0, 0, 1] \times [L_2 c_{12}, L_2 s_{12}, 0] = [-L_2 s_{12}, L_2 c_{12}, 0]^T$$

與偏微分結果的第二行完全相同。

兩種方法得到相同答案，但幾何法完全不需要微分——只需要正向運動學算出的位置與軸向。對六軸手臂而言，這個差別是「幾行程式碼」與「數頁推導」的差別。

11.5 幾何雅可比與解析雅可比

11.5.1 兩者的差異

11.4 節構造的雅可比稱為幾何雅可比 J_g ，它的下半部給出真正的物理角速度 ω 。但在某些場合，我們想要的是姿態參數的變化率——例如控制器的目標是「RPY 角以每秒 5 度變化」，這時需要的是解析雅可比 J_a ：

$$\text{幾何: } \begin{bmatrix} \dot{v} \\ \omega \end{bmatrix} = J_g \cdot \dot{q} \quad \text{解析: } \begin{bmatrix} \dot{p} \\ \dot{\phi} \end{bmatrix} = J_a \cdot \dot{q}$$

q 為姿態參數（如 RPY 三角）， ϕ 是它們的時間導數，不等於角速度 ω 。

兩者的上半部（線速度）完全相同——位置就是位置，沒有歧義。差別只在下半部。

11.5.2 兩者的轉換關係

角速度與姿態參數導數之間存在一個轉換矩陣，其形式取決於所用的姿態表示法。以 Z-Y-X 的 RPY 角為例：

$$\omega = E(\phi) \cdot \dot{\phi} \Rightarrow J_g = \begin{bmatrix} I & 0 \\ 0 & E \end{bmatrix} \cdot J_a$$

$E(\phi)$ 是一個 3×3 的轉換矩陣，其元素為姿態角的三角函數。

對 RPY 角 (roll γ 、pitch β 、yaw α)，這個矩陣為：

$$E = \begin{bmatrix} 0 & -s\alpha & c\alpha \cdot c\beta \\ 0 & c\alpha & s\alpha \cdot c\beta \\ 1 & 0 & -s\beta \end{bmatrix}$$

由角度導數映射到角速度的轉換矩陣。

關鍵觀察是：當 $\cos \beta = 0$ (即 pitch = $\pm 90^\circ$) 時， E 的行列式為零，矩陣不可逆。這正是第 8.3.3 節的萬向節鎖，在速度層面的表現——此時無法由角速度反推出唯一的角度導數。

11.5.3 實務上該用哪一種

用幾何雅可比的場合：力控制與碰撞偵測 (因為力矩對應的是物理角速度，見 11.6 節)、動力學推導 (第 13 章)、以及任何涉及物理量的計算。工業機器人控制器內部一律使用幾何雅可比。

用解析雅可比的場合：當任務直接以姿態參數描述時 (例如「保持 RPY 的 pitch 為 0」)、以及某些以歐拉角為介面的規劃演算法。但必須注意奇異點問題。

本書的預設：除非特別註明，本書後續所有的 J 皆指幾何雅可比。這與 Craig 教科書及工業界的慣例一致。

表 11-2 幾何雅可比與解析雅可比的比較

面向	幾何雅可比 J_g	解析雅可比 J_a
下半部輸出	角速度 ω (物理量)	姿態參數導數 $\dot{\phi}$
由何取得	陀螺儀可直接量測	由角度感測值微分
奇異點	只有機構奇異點	另有表示法奇異點 (萬向節鎖)
可否直接相加	可 (角速度可加)	不可
與力的對偶	成立 ($\tau = J_g^T \cdot F$)	不直接成立
典型用途	力控制、動力學、工業控制器	以歐拉角描述的任務、部分規劃演算法

11.6 虛功原理與速度—力對偶

11.6.1 由能量守恆推導

現在來處理開頭故事中最有趣的第三點：為什麼同一個雅可比矩陣也連結力？答案來自**虛功原理**——一個比機器人學古老得多的力學原理。

考慮機器人末端受到外力與力矩（合稱**扳手** $F = [f, n]^T$ ），同時各關節輸出扭矩 τ 。若系統處於靜力平衡且沒有能量損失，則關節做的功必須等於末端做的功：

$$\tau^T \cdot \delta q = F^T \cdot \delta x$$

虛功相等。 δq 為關節的虛位移、 δx 為末端的虛位移。

代入雅可比關係 $\delta x = J \cdot \delta q$ ：

$$\tau^T \cdot \delta q = F^T \cdot J \cdot \delta q$$

兩邊對任意的 δq 都成立。

由於此式對**任意**虛位移 δq 都必須成立，兩邊的係數必然相等，因此：

$$\tau = J^T \cdot F$$

靜力關係式。這是機器人學最優美的公式之一——同一個雅可比矩陣，用轉置就把末端的力映射成關節的扭矩。

11.6.2 對偶性的意義

這個結果揭示了一個深刻的對偶關係：

$$\text{速度：}\dot{x} = J \cdot \dot{q} \quad (\text{關節} \rightarrow \text{末端})$$

$$\text{力量：}\tau = J^T \cdot F \quad (\text{末端} \rightarrow \text{關節})$$

兩者的方向恰好相反，而矩陣互為轉置。這稱為運動—力對偶（kinematics-statics duality）。

方向相反這件事很值得玩味。速度是「由內往外傳」——馬達轉動，末端才移動；力則是「由外往內傳」——末端受力，關節才需要出力。而連結兩者的，竟是同一個矩陣的兩種形式。

這個對偶帶來三個重要推論。**推論一**：不需要額外的推導，就能由運動學直接得到靜力學。

推論二：雅可比的秩同時決定了「能往哪些方向運動」與「能抵抗哪些方向的力」（第 12 章）。

推論三：在奇異點處，機器人失去某方向的運動能力，但反過來說，它在該方向上可以抵抗無限大的力而不需任何關節扭矩——這解釋了為何手臂完全伸直時特別「有力」（也特別容易被卡住）。

11.6.3 三個實際應用

應用一：重力補償。已知各連桿的重量會在末端產生等效的力，用 $\tau = J^T \cdot F$ 即可算出各關節需要輸出多少扭矩來平衡它。這是第 17 章重力補償控制的基礎。

應用二：力控制。若要末端對工件施加特定的力（如研磨、裝配），可用此式反算出各關節該輸出的扭矩。這是第 20 章的核心。

應用三：碰撞偵測。反過來用：量測各關節的實際扭矩（由馬達電流估算，第 6.4.2 節），扣除重力與慣性所需的部分，剩餘的 τ_{ext} 就是外力造成的。再由 $F = (J^T)^+ \cdot \tau_{\text{ext}}$ 反推出末端受到的外力大小與方向。這正是協作機器人「感覺到你碰它」的原理——不需要任何力覺感測器，只靠電流與雅可比。

◎ 為什麼協作機器人的減速比不能太高

第 5 章提過高減速比會使關節無法反向驅動。現在可以從對偶性理解得更深：碰撞偵測依賴的是「外力造成的關節扭矩變化」。但減速比 N 會把外力對關節的影響縮小 N 倍，同時把摩擦力放大——當減速比達到 100 時，外力訊號可能已被摩擦的不確定性淹沒。這就是協作機器人採用中低減速比、或加裝關節力矩感測器的根本原因。

11.7 雅可比的座標系轉換

11.7.1 雅可比是相對於某個座標系的

一個常被忽略的重點是：雅可比矩陣的數值，取決於它是在**哪個座標系**中表達的。11.4 節構造的雅可比，其軸向 $\dot{z}$ 與位置 p 都是在基座座標系中表示的，因此得到的是「基座座標系中的雅可比」 0J 。

有時我們需要「末端座標系中的雅可比」 nJ ——例如力控制時，往往希望以工具自身的座標描述力的方向（「沿夾爪的接近方向施加 10 牛頓」）。兩者的轉換為：

$${}^nJ = \begin{bmatrix} {}^nR_0 & 0 \\ 0 & {}^nR_0 \end{bmatrix} \cdot {}^0J$$

上下兩塊各用旋轉矩陣轉換（線速度與角速度都是向量，都需要旋轉）。注意只有旋轉、沒有平移——因為速度向量的轉換不涉及原點位置。

11.7.2 參考點的改變

另一種常見的需求是改變**參考點**。11.4 節的雅可比是針對末端點 p_n 計算的；若裝上工具後，實際的作用點移到了 TCP（距離 p_n 有一個偏移 r ），雅可比該如何修正？

由剛體運動學，同一剛體上兩點的速度關係為：

$$\mathbf{v}_{TCP} = \mathbf{v}_n + \boldsymbol{\omega} \times \mathbf{r} \quad \boldsymbol{\omega}_{TCP} = \boldsymbol{\omega}_n$$

角速度處處相同（剛體不變形），但線速度會因為轉動而不同。

因此雅可比的修正為：

$$\mathbf{J}_{TCP} = \begin{bmatrix} \mathbf{I} & -[\mathbf{r}]_{\times} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \cdot \mathbf{J}_n$$

$[\mathbf{r}]_{\times}$ 為 $\mathbf{r}$ 的反對稱矩陣（第 8.4.2 節）。這個矩陣稱為伴隨變換（adjoint transformation）。

這個修正在實務上極為重要：換一支夾具，雅可比就變了。若程式中忘記更新 TCP 偏移，會導致末端在旋轉時產生預期外的平移——症狀是「機器人轉動夾爪時，工件的尖端會畫圓而非原地旋轉」。這是裝配與點膠作業中的常見故障。

11.8 可操作度、速度橢球與力橢球

11.8.1 各方向的能力並不相同

回到開頭故事的觀察：在不同姿態下，同樣的關節速度會產生不同的末端速度。更進一步，即使在同一個姿態下，往不同方向運動的能力也不相同。

要量化這件事，考慮所有「關節速度長度不超過 1」的組合，看看它們映射到末端後形成什麼形狀：

$$\{\dot{\mathbf{x}} = \mathbf{J} \cdot \mathbf{q} : \|\mathbf{q}\| \leq 1\} \Rightarrow \text{一個橢球}$$

這稱為**速度橢球**（velocity ellipsoid）。它的形狀完整描述了機器人在該姿態下各方向的運動能力。

橢球的主軸方向與長度，由雅可比的**奇異值分解**（SVD）給出：

$$\mathbf{J} = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^T \quad \text{其中 } \boldsymbol{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_m)$$

$\mathbf{U}$ 的行向量是橢球的主軸方向、 σ_i 是對應主軸的半徑長度。 σ_1 為最大奇異值、 σ_m 為最小奇異值。

直觀理解：最大奇異值的方向是「最容易動」的方向、最小奇異值的方向是「最難動」的方向。當某個奇異值趨近於零時，該方向的運動能力消失——這就是奇異點的嚴格定義，第 12 章將深入探討。

11.8.2 可操作度指標

Yoshikawa 於 1985 年提出一個純量指標來衡量整體的運動靈活度，稱為**可操作度**

(manipulability)：

$$w = \sqrt{\det(J J^T)} = \sigma_1 \cdot \sigma_2 \cdot \dots \cdot \sigma_m$$

可操作度等於所有奇異值的乘積，也就是速度橢球的體積（相差一個常數係數）。

這個指標有三個實用價值。**價值一：奇異點偵測**。當 w 趨近於零時，代表至少有一個奇異值趨近於零，機器人接近奇異點。這可用來觸發第 10.7.3 節的 DLS 阻尼調整。**價值二：姿態最佳化**。冗餘機器人可利用零空間，在不影響末端任務的前提下最大化 w ，使機器人維持在「靈活的姿態」。**價值三：工作站佈局**。設計工作站時，應把主要作業區配置在可操作度較高的區域。

但可操作度也有其限制：它是所有方向的「平均」能力（體積），無法反映「某一個特定方向」的能力。若任務只在乎某方向（如垂直插入），應直接看該方向對應的奇異值，而非整體的 w 。

11.8.3 力橢球與對偶的幾何

由 11.6 節的對偶性，可以類似地定義**力橢球**：在關節扭矩長度不超過 1 的限制下，末端能輸出的力所形成的形狀。

由於力的映射用的是 $(J^T)^{-1}$ （或偽逆），力橢球的主軸方向與速度橢球**完全相同**，但半徑長度**互為倒數**：

速度橢球半徑： σ_i	力橢球半徑： $1/\sigma_i$
--------------------	---------------------

兩者互為倒數，形成一個優雅的對偶關係。

這個結果有非常直觀的物理意義：**容易動的方向，就是不容易出力的方向；反之亦然**。這正是槓桿原理的推廣——你用一把長槓桿可以輕鬆撬動重物（力大），但槓桿另一端要移動很長的距離（速度小）。

這個對偶對工程設計有直接的指導意義。以裝配作業為例：插入方向需要**大的力**（推入）與**精細的位置控制**（對準）——這兩個需求恰好衝突。解決方案有二：一是選擇讓插入方向對應到「力橢球長軸」的姿態（有力但精度差），再靠柔順性（第 20 章）補償對位；二是改用 SCARA 這類在垂直方向天生剛硬的機構（第 4.8.1 節的「選擇性順應」）——現在你能理解，那個設計正是在操控速度—力橢球的形狀。

圖 11-2 速度橢球與力橢球的對偶

建議配圖：兩格對照圖，各畫一支平面 $2R$ 手臂在同一姿態下的末端橢球。左格為速度橢球（藍色實線），長軸沿容易運動的方向（垂直於手臂伸展方向），標註半徑 σ_1 、 σ_2 。右格為力橢球（紅色實線），主軸方向相同但長短相反，長軸沿手臂伸展方向，標註半徑 $1/\sigma_2$ 、 $1/\sigma_1$ 。中央以雙向箭頭連接並標註「互為倒數」。下方另加一格：手臂接近伸直時，速度橢球被壓扁成一條線（ $\sigma_2 \rightarrow 0$ ）、力橢球則沿該方向無限延伸——標註「奇異點」。

11.9 雅可比的應用全景

11.9.1 運動速率控制

雅可比最直接的應用是**運動速率控制**（resolved motion rate control）：操作員或上層規劃給出末端的期望速度，控制器即時反算各關節該轉多快。

$$\mathbf{q}' = \mathbf{J}^{-1}(\mathbf{q}) \cdot \dot{\mathbf{x}}_{\text{desired}} \quad (\text{或以偽逆 / DLS 取代反矩陣})$$

這正是搖桿遙操作、視覺伺服與力控制的共同底層機制。

值得注意的是，這與第 10 章的數值逆運動學在數學上是同一件事——只是解釋不同。逆運動學是「反覆修正位置誤差直到收斂」，速率控制則是「持續把速度指令翻譯成關節速度」。前者是離散的迭代、後者是連續的控制迴路，但用的都是 $\mathbf{J}$ 的逆。這也是為什麼第 10 章的所有數值技巧（偽逆、DLS、零空間）在此完全適用。

11.9.2 六大應用領域

雅可比矩陣在機器人學中的應用範圍極廣，以下是它在本書後續章節出現的位置：

表 11-3 雅可比矩陣的應用一覽

應用	使用形式	說明	章節
數值逆運動學	$\mathbf{J}^+$ 或 DLS	迭代求解關節角度	第 10 章
奇異點分析	$\text{rank}(\mathbf{J})$ 、 $\sigma_{\min}$	判斷失去運動能力的位形	第 12 章
冗餘與零空間	$(\mathbf{I} - \mathbf{J}^+ \mathbf{J})$	在不影響任務下最佳化次要目標	第 12 章
運動速率控制	$\mathbf{J}^{-1}$	由末端速度指令求關節速度	第 11、17 章
動力學推導	$\mathbf{J}$ 、 $\dot{\mathbf{J}}$	計算慣性矩陣與科氏項	第 13、14 章
操作空間控制	$\mathbf{J}$ 、 $\mathbf{J}^T$ 、 $\Lambda = (\mathbf{J} \mathbf{M}^{-1} \mathbf{J}^T)^{-1}$	直接在任務空間設計控制律	第 17 章
重力補償	$\mathbf{J}^T \mathbf{F}_{\text{gravity}}$	計算平衡重力所需的關節扭矩	第 17 章
力控制	$\boldsymbol{\tau} = \mathbf{J}^T \mathbf{F}$	由期望的末端力求關節扭矩	第 20 章
碰撞偵測	$\mathbf{F} = (\mathbf{J}^T)^+ \cdot \boldsymbol{\tau}_{\text{ext}}$	由關節扭矩反推外力	第 20、35 章
視覺伺服	$\mathbf{J}_{\text{image}} \cdot \mathbf{J}_{\text{robot}}$	由影像誤差直接驅動關節	第 22 章

11.9.3 雅可比的時間導數

在動力學與加速度層級的控制中，還會用到雅可比的時間導數 $\dot{J}$ 。把速度關係再對時間微分一次：

$$\ddot{\mathbf{x}} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}} + \dot{\mathbf{J}}(\mathbf{q}, \dot{\mathbf{q}}) \cdot \dot{\mathbf{q}}$$

末端加速度包含兩項：關節加速度的貢獻，以及因姿態改變導致雅可比本身變化的貢獻。

第二項 $\dot{\mathbf{J}} \cdot \dot{\mathbf{q}}$ 常被稱為「速度相關項」，它在物理上對應到科氏力與離心力——第 13 章動力學方程中的 $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \cdot \dot{\mathbf{q}}$ 項正源於此。這說明了一件事：那些看似神秘的科氏力，其實只是「雅可比會隨姿態改變」這個事實的必然結果。

實務上 $\dot{J}$ 通常以數值微分近似 $(\mathbf{J}(t) - \mathbf{J}(t - \Delta t)) / \Delta t$ ，因為解析推導極為繁瑣且收益有限。

11.10 由速度邁向奇異點

11.10.1 本章的成果

本章完成了運動學的第三塊拼圖。你現在能夠：由偏微分或幾何法構造任意串聯手臂的雅可比矩陣、區分幾何與解析兩種形式、運用虛功原理連結速度與力、在不同座標系與參考點之間轉換雅可比、並用奇異值與可操作度量化機器人在各方向的運動與出力能力。

更重要的是，你獲得了一個統一的視角：**雅可比是機器人學的中央樞紐**。運動學在它的左邊、力學在它的右邊、控制在它的上方、而奇異點在它的內部。本書後續二十多章中，將有超過一半會直接或間接用到它。

11.10.2 接下來：當雅可比壞掉的時候

本章多次提到一個現象：在某些姿態下，雅可比會失去滿秩，導致某個奇異值趨近於零。此時速度橢球被壓扁、可操作度歸零、逆矩陣不存在、數值計算爆炸。

第 12 章將專門處理這個現象。我們將回答：奇異點在哪裡？如何預先偵測？為什麼機構設計上無法完全避免？以及——如果機器人有冗餘自由度，能否用零空間繞開它？屆時，第 4 章的冗餘、第 8 章的萬向節鎖、第 9 章的手臂伸直、第 10 章的解合併與數值爆炸，這些散落各章的線索，將被雅可比矩陣的秩統一串起來。

完整計算例題

例題 11-1 平面 2R 手臂的雅可比與速度計算

題目：平面 2R 手臂 $L_1 = 300 \text{ mm}$ 、 $L_2 = 200 \text{ mm}$ ，目前姿態 $\theta_1 = 30^\circ$ 、 $\theta_2 = 45^\circ$ 。(a) 求此姿態下的雅可比矩陣。(b) 若兩關節皆以每秒 10 度的角速度轉動，求末端的線速度。(c) 若要末端以 100 mm/s 沿 +X 方向運動，求所需的關節角速度。

解 (a)：代入 11.2.1 節的公式。 $\theta_1 = 30^\circ$ 、 $\theta_1 + \theta_2 = 75^\circ$ 。

$$s_1 = 0.5000, c_1 = 0.8660, s_{12} = 0.9659, c_{12} = 0.2588$$

先算出四個三角函數值。

$$J = \begin{bmatrix} -300(0.5) - 200(0.9659) & -200(0.9659) \\ 300(0.866) + 200(0.2588) & 200(0.2588) \end{bmatrix} = \begin{bmatrix} -343.19 & -193.19 \\ 311.57 & 51.76 \end{bmatrix}$$

單位為 mm/rad。

驗證技巧：J 的第一行應等於 $[-y, x]^T$ (11.2.2 節)。由例題 9-1 知末端位置為 (311.57, 343.19)，故 $[-y, x] = [-343.19, 311.57]$ ✓ 與計算結果相符。

解 (b)：關節角速度需先轉為弧度： $10^\circ/\text{s} = 0.1745 \text{ rad/s}$ 。

$$\dot{x} = J \cdot \dot{q} = \begin{bmatrix} -343.19 & -193.19 \\ 311.57 & 51.76 \end{bmatrix} \begin{bmatrix} 0.1745 \\ 0.1745 \end{bmatrix} = \begin{bmatrix} -93.60 \\ 63.42 \end{bmatrix} \text{ mm/s}$$

末端線速度。

速度大小為 $\sqrt{93.60^2 + 63.42^2} \approx 113.1 \text{ mm/s}$ ，方向為 $\text{atan2}(63.42, -93.60) \approx 145.9^\circ$ (指向上方)。注意兩個關節都以相同角速度正轉，末端卻往左上移動——這正是雅可比「兌換率隨姿態而變」的具體體現。

解 (c)：需求解 $J \cdot \dot{q} = [100, 0]^T$ 。先求 J 的行列式：

$$\det(J) = (-343.19)(51.76) - (-193.19)(311.57) = -17763 + 60190 = 42427$$

行列式不為零，J 可逆，此姿態非奇異點。

$$J^{-1} = (1/42427) \begin{bmatrix} 51.76 & 193.19 \\ -311.57 & -343.19 \end{bmatrix}$$

反矩陣 (二階矩陣的反矩陣公式：主對角互換、副對角變號，再除以行列式)。

$$\dot{q} = J^{-1} \cdot \begin{bmatrix} 100 \\ 0 \end{bmatrix} = (1/42427) \begin{bmatrix} 5176 \\ -31157 \end{bmatrix} = \begin{bmatrix} 0.1220 \\ -0.7344 \end{bmatrix} \text{ rad/s} = \begin{bmatrix} 6.99 \\ -42.08 \end{bmatrix} \text{ }^\circ/\text{s}$$

所需的關節角速度。

工程觀察：要讓末端以 100 mm/s 沿 X 方向移動，第二關節需以 42 度/秒反轉——遠快於第一關節的 7 度/秒。若目標速度提高到 500 mm/s，第二關節就需要 210 度/秒，可能已超過馬達能力。這說明了一件事：**末端速度的限制，往往不是由馬達的最高轉速直接決定，而是由當前姿態下的雅可比決定。**第 15 章的軌跡規劃必須考慮這一點。

例題 11-2 靜力分析與碰撞偵測

題目：同上手臂與姿態。(a) 若末端受到一個沿 $-X$ 方向、大小 20 N 的外力，求各關節需輸出多少扭矩才能維持靜止。(b) 反過來，若量測到關節扭矩為 $\tau = [-5.0, -2.5]^T$ N·m (已扣除重力與摩擦)，求末端受到的外力。

解 (a)：使用 $\tau = J^T \cdot F$ 。外力 $F = [-20, 0]^T$ N。注意 J 的單位是 mm/rad，需轉換為 m/rad (除以 1000)。

$$J^T = \begin{bmatrix} -0.34319 & 0.31157 \\ -0.19319 & 0.05176 \end{bmatrix} \quad (\text{單位：m/rad})$$

雅可比的轉置。

$$\tau = J^T \cdot F = \begin{bmatrix} -0.34319(-20) + 0.31157(0) \\ -0.19319(-20) + 0.05176(0) \end{bmatrix} = \begin{bmatrix} 6.864 \\ 3.864 \end{bmatrix} \text{ N}\cdot\text{m}$$

各關節所需扭矩。

因此第一關節需輸出 6.86 N·m、第二關節需 3.86 N·m。注意第一關節的負擔較重——這符合直覺，因為它承受了整條手臂的力矩。

解 (b)：這是碰撞偵測的計算方向。由 $\tau = J^T \cdot F$ 反解 $F = (J^T)^{-1} \cdot \tau$ 。

$$(J^T)^{-1} = (J^{-1})^T = (1/0.042427) \begin{bmatrix} 0.05176 & -0.31157 \\ 0.19319 & -0.34319 \end{bmatrix}$$

轉置的反矩陣等於反矩陣的轉置。

$$\begin{aligned} F &= (J^T)^{-1} \cdot \begin{bmatrix} -5.0 \\ -2.5 \end{bmatrix} = \begin{bmatrix} (0.05176)(-5.0) + (-0.31157)(-2.5) \\ (0.19319)(-5.0) + (-0.34319)(-2.5) \end{bmatrix} / 0.042427 \\ &= \begin{bmatrix} 0.5202 \\ -0.1080 \end{bmatrix} / 0.042427 = \begin{bmatrix} 12.26 \\ -2.55 \end{bmatrix} \text{ N} \end{aligned}$$

推得的末端外力。

因此末端受到一個約 12.5 N、方向偏向 +X 稍微向下的力。這正是協作機器人判斷「有人碰到我了、力來自哪個方向」的計算過程——完全不需要力覺感測器，只需要馬達電流與雅可比矩陣。

實務提醒：此計算的精度受兩個因素限制。一是關節扭矩的估算誤差（摩擦力的不確定性，第 6.4.2 節）；二是接近奇異點時 $(J)^{-1}$ 會放大誤差。因此高階協作機器人仍會加裝關節力矩感測器以提升可靠度。

例題 11-3 可操作度與速度橢球

題目：同上手臂。(a) 計算目前姿態的可操作度。(b) 計算 $\theta_2 = 5^\circ$ 時的可操作度並比較。(c) 求 θ_2 為何值時可操作度最大？

解 (a)：對方陣而言， $w = \sqrt{(\det(J J^T))} = |\det(J)|$ 。由例題 11-1， $\det(J) = 42427 \text{ mm}^2/\text{rad}^2$ 。

$$w = |\det(J)| = 42427 \text{ mm}^2/\text{rad}^2$$

目前姿態的可操作度。

解 (b)：對平面 2R 手臂，行列式有一個極為簡潔的封閉形式。展開 $\det(J)$ 可得：

$$\det(J) = L_1 L_2 \sin \theta_2$$

平面 2R 手臂的可操作度只取決於 θ_2 ，與 θ_1 完全無關。

驗證： $L_1 L_2 \sin 45^\circ = 300 \times 200 \times 0.7071 = 42426 \checkmark$ 與 (a) 一致。

當 $\theta_2 = 5^\circ$ ： $w = 300 \times 200 \times \sin 5^\circ = 60000 \times 0.0872 = 5229 \text{ mm}^2/\text{rad}^2$ 。僅為原本的 12.3%——手臂接近伸直時，運動能力大幅下降。

解 (c)：由 $w = L_1 L_2 |\sin \theta_2|$ 可知，當 $\theta_2 = \pm 90^\circ$ 時 w 達到最大值 $60000 \text{ mm}^2/\text{rad}^2$ 。此時前臂與上臂互相垂直——這是平面 2R 手臂最靈活的姿態。

工程啟示：這個結果有直接的實務意義。第一，工作站佈局時應讓主要作業區落在「手肘約彎 90 度」的位置，而非讓手臂接近伸直或折疊。第二，當 $\theta_2 \rightarrow 0$ 或 180° 時 $w \rightarrow 0$ ，即為奇異點（第 12 章）。第三，可操作度與 θ_1 無關，代表可以自由旋轉基座而不影響靈活度——這是設計轉台或行走軸時的有用資訊。

程式範例：雅可比的三種計算法與應用

以下 Python 程式實作雅可比的解析法、幾何法與數值微分法，驗證三者一致，並示範速度控制、靜力分析與可操作度計算。

```

import numpy as np

L1, L2 = 0.300, 0.200          # 連桿長度 (m)

def fk(q):
    """正向運動學：回傳末端位置 (x, y)"""
    t1, t2 = q
    return np.array([L1*np.cos(t1) + L2*np.cos(t1+t2),
                     L1*np.sin(t1) + L2*np.sin(t1+t2)])

def joint_positions(q):
    """回傳各關節與末端的位置 (供幾何法使用)"""
    t1, t2 = q
    p0 = np.array([0.0, 0.0])
    p1 = np.array([L1*np.cos(t1), L1*np.sin(t1)])
    p2 = fk(q)
    return p0, p1, p2

# ----- 方法一：解析法 (11.2 節偏微分) -----
def jacobian_analytic(q):
    t1, t2 = q
    s1, c1 = np.sin(t1), np.cos(t1)
    s12, c12 = np.sin(t1+t2), np.cos(t1+t2)
    return np.array([[-L1*s1 - L2*s12, -L2*s12],
                     [ L1*c1 + L2*c12,  L2*c12]])

# ----- 方法二：幾何法 (11.4 節：z × (p_n - p_j)) -----
def jacobian_geometric(q):
    p0, p1, p2 = joint_positions(q)
    z = np.array([0.0, 0.0, 1.0])          # 兩軸皆沿 z
    cols = []
    for pj in (p0, p1):                  # 各關節位置
        r = np.append(p2 - pj, 0.0)       # 由關節指向末端
        v = np.cross(z, r)                # 線速度貢獻
        cols.append(v[:2])                # 平面問題只取 x, y
    return np.column_stack(cols)

# ----- 方法三：數值微分 -----
def jacobian_numeric(q, eps=1e-7):
    q = np.asarray(q, dtype=float)
    J = np.zeros((2, len(q)))
    for j in range(len(q)):
        dq = np.zeros_like(q); dq[j] = eps
        J[:, j] = (fk(q + dq) - fk(q - dq)) / (2*eps)  # 中央差分
    return J

d, r2d = np.deg2rad, np.rad2deg
q = np.array([d(30), d(45)])

# ----- 驗證：三種方法一致性 -----

```

```

print("=== 三種雅可比計算法的比較 (單位 m/rad) ===")
Ja, Jg, Jn = jacobian_analytic(q), jacobian_geometric(q), jacobian_numeric(q)
print(f"解析法:\n{np.round(Ja, 5)}")
print(f"幾何法與解析法一致: {np.allclose(Ja, Jg)}")
print(f"數值法與解析法一致: {np.allclose(Ja, Jn, atol=1e-6)}")
print(f"驗證第一行 = [-y, x]: {np.allclose(Ja[:,0], [-fk(q)[1], fk(q)[0]])}")

# ----- 應用一：速度正解 (例題 11-1b) -----
print("\n=== 例題 11-1(b)：關節速度 → 末端速度 ===")
qd = np.array([d(10), d(10)])
xd = Ja @ qd
print(f"末端速度: ({xd[0]*1000:.2f}, {xd[1]*1000:.2f}) mm/s")
print(f"速度大小: {np.linalg.norm(xd)*1000:.1f} mm/s "
      f"方向: {r2d(np.arctan2(xd[1], xd[0])):.1f}°")

# ----- 應用二：速度逆解 (例題 11-1c) -----
print("\n=== 例題 11-1(c)：末端速度 → 關節速度 ===")
x_des = np.array([0.100, 0.0]) # 100 mm/s 沿 +X
qd_req = np.linalg.solve(Ja, x_des)
print(f"所需關節角速度: ({r2d(qd_req[0]):.2f}, {r2d(qd_req[1]):.2f}) °/s")

# ----- 應用三：靜力對偶 (例題 11-2) -----
print("\n=== 例題 11-2：速度-力對偶 ===")
F = np.array([-20.0, 0.0]) # 末端外力 (N)
tau = Ja.T @ F
print(f"(a) 外力 {F} N → 關節扭矩 ({tau[0]:.3f}, {tau[1]:.3f}) N·m")
tau_meas = np.array([-5.0, -2.5])
F_est = np.linalg.solve(Ja.T, tau_meas)
print(f"(b) 扭矩 {tau_meas} N·m → 推得外力 ({F_est[0]:.2f}, {F_est[1]:.2f}) N")

# ----- 應用四：可操作度 (例題 11-3) -----
print("\n=== 例題 11-3：可操作度分析 ===")
def manipulability(q):
    J = jacobian_analytic(q)
    return np.sqrt(np.linalg.det(J @ J.T))

for t2_deg in (5, 30, 45, 90, 135, 175):
    w = manipulability([d(30), d(t2_deg)])
    theory = L1*L2*abs(np.sin(d(t2_deg)))
    bar = "■" * int(w/L1/L2*40)
    print(f" θ2={t2_deg:4d}° w={w:.6f} 理論值={theory:.6f} {bar}")
print(" → θ2 = 90° 時最靈活；接近 0° 或 180° 時趨近奇異")

# ----- 應用五：速度橢球 (奇異值分解) -----
print("\n=== 速度橢球的主軸 (SVD) ===")
for t2_deg in (90, 20):

```

```
J = jacobian_analytic([d(30), d(t2_deg)])
U, S, Vt = np.linalg.svd(J)
print(f"  θ2={{t2_deg:3d}}° 奇異值 σ={{S[0]:.4f}}, {{S[1]:.4f}}} "
      f"條件數={{S[0]/S[1]:7.2f}}")
print(f"          最易運動方向: {{r2d(np.arctan2(U[1,0], U[0,0])):7.1f}}° "
      f"最難運動方向: {{r2d(np.arctan2(U[1,1], U[0,1])):7.1f}}°")
print(" → 條件數越大代表各方向能力差異越懸殊，越接近奇異點")
```

請試著修改：（一）把 θ_2 設為 0.5 度，觀察條件數如何暴增、以及速度逆解算出的關節角速度有多大。（二）比較數值微分在 $\text{eps} = 1\text{e-}3$ 與 $1\text{e-}12$ 時的誤差——你會發現太大與太小都不好（前者截斷誤差大、後者浮點捨入誤差大）。（三）繪出可操作度隨 θ_2 變化的曲線，驗證 $w = L_1 L_2 |\sin \theta_2|$ 。

實作：雅可比的實驗驗證

本章實作是在實體手臂上驗證雅可比矩陣的兩個面向：速度映射與力映射。建議使用第 9、10 章已建立模型的手臂。

第一部分：速度映射驗證

1. **選定姿態**：把手臂擺到一個遠離奇異點的姿態（如平面手臂的 $\theta_2 \approx 90^\circ$ ），記錄關節角度。
2. **計算理論雅可比**：用本章程式算出該姿態的 J 。
3. **單關節測試**：只讓第一關節以固定的低速轉動一小段（如 5 度），用方格紙或尺量測末端移動的距離與方向。與 J 的第一行預測值比較。
4. **重複第二關節**：同樣測試第二關節，與 J 的第二行比較。
5. **合成測試**：兩關節同時以不同速度轉動，驗證末端速度是否等於兩者貢獻的向量和。
6. **變換姿態重測**：把手臂擺到接近伸直的姿態（ $\theta_2 \approx 10^\circ$ ），重複上述測試。你會發現：同樣的關節轉動量，末端移動的距離與方向都改變了——親眼驗證「兌換率隨姿態而變」。

第二部分：力映射驗證（需可量測關節電流的平台）

1. **基準量測**：在無外力的情況下，記錄各關節維持姿態所需的電流（此為重力與摩擦的貢獻）。
2. **施加已知外力**：在末端掛上已知重量的砝碼（例如 500 g，即約 4.9 N 向下的力），記錄各

關節電流的變化量。

3. **換算扭矩**：用第 5 章的 $\tau = Kt \cdot I$ 把電流變化換算為扭矩變化。
4. **理論比對**：用 $\tau = J \cdot F$ 計算理論值，與實測比較。誤差主要來自摩擦力的不確定性。
5. **反向推算**：改以實測扭矩用 $F = (J \tau)^{-1} \cdot \tau$ 反推外力，與已知的砝碼重量比較，評估「無感測器碰撞偵測」的實際精度。
6. **姿態影響**：在接近奇異點的姿態重複實驗，觀察反推精度如何劣化。

表 11-4 雅可比驗證記錄表（第一列為範例）

測試項	姿態 (θ_1, θ_2)	輸入	理論預測	實測值	誤差	說明
單關節速度	(30°, 90°)	θ_1 轉 5°	末端移 26.2 mm，方向 120°	25.1 mm，118°	4.2%	誤差來自背隙
(自行填寫其餘測試)						

國中小也看得懂：機器手臂也有變速器

你有沒有騎過腳踏車的變速器？換到「輕齒輪」時，腳踩一圈車子跑得比較少，但踩起來很輕鬆；換到「重齒輪」時，踩一圈跑很遠，但要用力踩。踩踏圈數和前進距離之間的比例，會隨著齒輪而改變——這就是「兌換率」。

機器手臂也有這種兌換率，而且更神奇：它會隨著手臂的姿勢改變。試試看：把你的手臂彎成 90 度（像舉重一樣），然後手肘轉一點點，看看手掌移動多少；再把手臂伸直，同樣手肘轉一點點，這次手掌移動的距離就不一樣了。同樣的關節轉動，手移動的距離卻不同——這就是這一章在算的東西。

還有一件超酷的事。這個兌換率不只管「動得多快」，還管「用得出多少力」。而且它們是相反的：容易動的方向，出力就小；不容易動的方向，反而力氣大。想想看用剪刀剪厚紙板——你會把紙板放在剪刀最靠近軸的地方，因為那裡刀刃動得慢，但力氣最大！機器手臂也一樣，工程師必須算清楚：現在這個姿勢，往哪個方向比較有力、往哪個方向比較靈活。

叫獸提醒與常見錯誤

- 誤區一：以為雅可比是固定的矩陣。它是關節角度的函數，每一次姿態改變都必須重算。
- 誤區二：把角速度當成歐拉角的導數。兩者相差一個依姿態而變的轉換矩陣，混用會導致姿態運動方向錯誤。
- 誤區三：混用幾何與解析雅可比。力的對偶關係 $\tau = J^T \cdot F$ 只對幾何雅可比成立。
- 誤區四：忘記單位一致性。雅可比的線速度部分單位是「長度/弧度」，角度必須用弧度而非度數。
- 誤區五：忘記轉換座標系。雅可比是相對於特定座標系表達的，用錯座標系會使力或速度的方向完全偏離。
- 誤區六：換工具後不更新 TCP 偏移。雅可比的參考點改變時必須用伴隨變換修正，否則旋轉時末端會產生非預期的平移。
- 誤區七：直接用 J^{-1} 而不檢查條件數。接近奇異點時反矩陣的元素會暴增，應改用 DLS (第 10.7.3 節)。
- 誤區八：以為可操作度高就是好。它是各方向的平均能力；若任務只在乎某一方向，應直接檢視該方向對應的奇異值。
- 誤區九：數值微分的步長選擇不當。太大則截斷誤差大、太小則浮點捨入誤差大，建議使用中央差分且 ϵ 取 $1e-6 \sim 1e-8$ 。
- 誤區十：忽略末端速度受姿態限制。同樣的馬達能力，在不同姿態下能達到的末端速度差異可達數倍，軌跡規劃時必須考量。

課堂討論

1. 開頭故事中，末端等速直線運動而關節速度不斷變化。請解釋為什麼，並討論這對軌跡規劃 (第 15 章) 帶來什麼限制。
2. 速度橢球與力橢球的主軸方向相同、半徑互為倒數。請以「用螺絲起子鎖螺絲」為例，說明這個對偶對操作策略的影響。
3. 協作機器人靠 $F = (J^T)^+ \cdot \tau_{\text{ext}}$ 偵測碰撞，不需力覺感測器。請討論這個方法的三個限制，以及各自的改善方向。
4. 可操作度 $w = L_1 L_2 |\sin \theta_2|$ 與 θ_1 完全無關。這個結果對工作站佈局有什麼指導意義？若手臂裝在一個可旋轉的轉台上，這個結論還成立嗎？

5. 第 8 章的萬向節鎖與本章解析雅可比的奇異點是同一回事嗎？請比較「表示法的奇異點」與「機構的奇異點」，並說明改用四元數能否解決後者。

章末摘要

- 雅可比矩陣 $J(q) = \partial f / \partial q$ 把關節速度映射到末端速度： $\dot{x} = J \cdot \dot{q}$ ，它是正向運動學的局部線性化。
- 三種等價讀法：速度的兌換率、微小位移的映射、非線性函數的局部線性化。
- 偏微分法概念直接但對六軸手臂不切實際；幾何法完全重用正向運動學結果，是實務標準作法。
- 旋轉關節的雅可比行為 $[\dot{z} \times (p_n - p_j); \dot{z}]$ ，移動關節為 $[\dot{z}; 0]$ 。
- 角速度不是任何姿態表示法的導數；它由 $\dot{R} = [\omega] \times \cdot R$ 定義，可用陀螺儀直接量測，且多個角速度可直接相加。
- 幾何雅可比輸出物理角速度、解析雅可比輸出姿態參數導數，兩者相差一個轉換矩陣 $E(\varphi)$ ，後者在萬向節鎖處奇異。
- 由虛功原理得 $\tau = J^T \cdot F$ ：同一矩陣的轉置把末端力映射為關節扭矩，這是速度—力對偶性。
- 對偶性的三大應用：重力補償、力控制、以及無感測器的碰撞偵測（ $F = (J^T)^+ \cdot \tau_{\text{ext}}$ ）。
- 雅可比的座標系轉換只需旋轉（上下兩塊各乘 R ）；參考點改變則需伴隨變換（含 $-[r] \times$ 項）。
- 速度橢球由 SVD 給出：主軸方向為 U 的行、半徑為奇異值 σ_i ；最大奇異值方向最易運動、最小奇異值方向最難。
- 可操作度 $w = \sqrt{(\det(J J^T))}$ 等於所有奇異值的乘積，可用於奇異點偵測、姿態最佳化與工作站佈局。
- 力橢球與速度橢球主軸相同但半徑互為倒數：容易動的方向不容易出力，反之亦然。
- 平面 2R 手臂的可操作度為 $w = L_1 L_2 |\sin \theta_2|$ ，僅取決於肘關節角， $\theta_2 = \pm 90^\circ$ 時最靈活。
- $J \cdot \dot{q}$ 項對應動力學中的科氏力與離心力，說明這些力源於「雅可比隨姿態改變」這一事實。
- 末端可達速度受當前姿態的雅可比限制，而非僅由馬達最高轉速決定。

觀念題與計算題

觀念題

1. 定義雅可比矩陣，並說明它的三種等價讀法。
2. 說明偏微分法與幾何法的差異，並解釋為何實務上採用後者。
3. 寫出旋轉關節與移動關節的雅可比行公式，並說明各項的物理意義。
4. 為何說「角速度不是姿態角的導數」？請說明角速度的嚴格定義。
5. 有限旋轉不可交換，但角速度可以直接相加。請解釋這個看似矛盾的現象。
6. 比較幾何雅可比與解析雅可比的差異，並各舉一個適用場合。
7. 由虛功原理推導 $\tau = J^T \cdot F$ ，並說明速度—力對偶的三個推論。
8. 說明協作機器人如何在沒有力覺感測器的情況下偵測碰撞，並列出此方法的兩個限制。
9. 為何雅可比需要做座標系轉換？換工具後為何必須用伴隨變換修正？
10. 說明速度橢球的意義，以及奇異值分解中 U 與 Σ 各代表什麼。
11. 定義可操作度並說明其三個實用價值與一個限制。
12. 說明力橢球與速度橢球的關係，並以槓桿原理類比說明其物理意義。
13. 說明 $J \cdot q$ 項的物理意義，以及它與動力學中哪一項對應。
14. 為何末端可達速度不是由馬達最高轉速直接決定？這對軌跡規劃有何影響？

計算題

1. 平面 2R 手臂 $L_1 = 400 \text{ mm}$ 、 $L_2 = 300 \text{ mm}$ ，姿態 $\theta_1 = 20^\circ$ 、 $\theta_2 = 60^\circ$ 。(a) 求雅可比矩陣。(b) 驗證第一行是否等於 $[-y, x]^T$ 。(c) 若兩關節分別以 $15^\circ/\text{s}$ 與 $-20^\circ/\text{s}$ 轉動，求末端速度的大小與方向。(d) 求此姿態的可操作度。
2. 同上手臂與姿態。(a) 若要末端以 200 mm/s 沿 $+Y$ 方向運動，求所需的關節角速度。(b) 若馬達的最大關節速度為 $60^\circ/\text{s}$ ，此速度是否可達成？(c) 在此姿態下，末端沿 $+Y$ 方向的最大可達速度為多少？
3. 同上手臂與姿態。(a) 末端受到 $F = [0, -30]^T \text{ N}$ 的外力（模擬掛上約 3 kg 的負載），求各關節所需扭矩。(b) 若量測到的關節扭矩為 $\tau = [8.0, 3.0]^T \text{ N}\cdot\text{m}$ ，反推末端外力。(c) 若第一關節的扭矩量測有 $\pm 0.5 \text{ N}\cdot\text{m}$ 的誤差，估算對推得外力的影響。
4. 某平面 2R 手臂 $L_1 = L_2 = 250 \text{ mm}$ 。(a) 寫出可操作度隨 θ_2 變化的表達式。(b) 求 $\theta_2 = 15^\circ$ 、 45° 、 90° 、 150° 時的可操作度。(c) 若要求可操作度不低於最大值的 50% ， θ_2 的

允許範圍為何？(d) 由此結果，建議工作站的主要作業區應配置在距離基座多遠的環狀範圍內？

動手做與專題延伸

本章實作：雅可比計算模組

延續第 9、10 章的程式，實作通用的雅可比計算模組。建議依下列階段完成：

- **階段一**：以幾何法 (11.4.4 節) 實作通用的 $6 \times n$ 雅可比計算，支援任意 DH 參數表與 R/P 關節類型。
- **階段二**：實作數值微分版本作為驗證基準，確認兩者在多組隨機姿態下的一致性 (誤差應小於 $1e-6$)。
- **階段三**：加入座標系轉換與 TCP 偏移的伴隨變換功能。
- **階段四**：實作可操作度、奇異值分解與條件數的計算，並整合為一個「姿態健康度」報告函式。
- **階段五**：回頭替換第 10 章數值逆運動學中的數值微分雅可比，比較計算速度的改善幅度。

專題延伸

- **速度橢球視覺化**：延續第 9、10 章的視覺化工具，在末端位置繪出速度橢球 (以 SVD 的主軸方向與奇異值為半徑)。加上滑桿控制關節角度，即時觀察橢球如何隨姿態變形——當手臂接近伸直時，你會看到橢球被壓扁成一條線。
- **可操作度地圖**：對工作空間中的每一個點，計算其可操作度 (若有多組逆解則取較大者)，以熱力圖繪出。這張圖能直觀顯示「機器人在哪些區域最靈活」，是工作站佈局的實用工具。
- **進階：力橢球與任務適配分析**。針對一個具體任務 (例如「垂直向下插入，需 50 N 推力與 0.1 mm 的水平定位精度」)，在工作空間中找出所有能同時滿足「該方向力橢球半徑足夠大」與「水平方向速度橢球半徑足夠小 (即精度高)」的區域。將結果與可操作度地圖比較，討論為何「最靈活的位置」未必是「最適合這個任務的位置」。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 5

章：雅可比——速度與靜力，本章主要參考）。

- Yoshikawa, T., "Manipulability of Robotic Mechanisms," International Journal of Robotics Research, 1985 (可操作度概念的原始論文)。
- Lynch, K. M. and Park, F. C., Modern Robotics, Cambridge University Press (第 5 章：速度運動學與靜力，含旋量與螺旋理論觀點)。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer (第 3 章：微分運動學與靜力)。
- Murray, R. M., Li, Z. and Sastry, S. S., A Mathematical Introduction to Robotic Manipulation, CRC Press (旋量與伴隨變換的嚴謹處理)。
- Salisbury, J. K. and Craig, J. J., "Articulated Hands: Force Control and Kinematic Issues," IJRR, 1982 (雅可比在多指手力控制中的應用)。
- Chiacchio, P. et al., "Global Task Space Manipulability Ellipsoids for Multiple-Arm Systems," IEEE Trans. Robotics, 1991 (可操作度橢球的延伸)。
- De Luca, A. and Mattone, R., "Sensorless Robot Collision Detection and Hybrid Force/Motion Control," ICRA 2005 (無感測器碰撞偵測的經典論文)。
- Corke, P., Robotics, Vision and Control, 3rd ed., Springer (含雅可比與可操作度的 MATLAB / Python 實作範例)。
- Pinocchio 與 KDL 函式庫技術文件：高效的雅可比計算實作，含伴隨變換與座標系轉換功能。

第 12 章

奇異點、冗餘與多任務運動

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】畫面左半為一支六軸手臂逐漸伸直的四格連續動作，末端速度橢球（藍色）隨之被壓扁，最後退化成一條線段；橢球下方標註條件數由 $3 \rightarrow 12 \rightarrow 58 \rightarrow \infty$ 的變化。右半為一支七軸手臂，末端夾爪固定在同一位置（綠色圓圈標註「末端不動」），手肘卻沿一條圓弧畫出三重殘影，旁註「零空間自運動」。中央以一條紅色分隔線區隔，上方文字：奇異點——關上一扇門；下方文字：冗餘——開啟另一扇窗。

叫獸開講 「手臂伸得太直，反而最沒有力？」

六軸手臂正在執行一段簡單的直線軌跡：由左而右，橫越工作台，速度每秒 15 公分。前半段一切順利。到了中段，手臂逐漸伸直，控制器突然發出刺耳的警報，螢幕上跳出一行紅字——**接近奇異點，第五軸速度超限**——然後手臂僵在原地不動。

小翰盯著螢幕：「奇怪，目標點明明在工作範圍內啊，程式也沒有語法錯誤。」

叫獸把手臂手動移回起點，指著第五軸：「你看這個關節。當手腕的第四軸與第六軸轉到互相平行時，會發生什麼事？」他慢慢轉動第四軸，接著轉第六軸——兩者產生了**完全相同**的末端轉動。

小紓先反應過來：「那不就是.....第 8 章的萬向節鎖？」

「幾乎一模一樣，但這次是**機構**的，不是數學表示法的。」叫獸在白板上寫下一行字：**兩個關節做同一件事，等於少了一個關節**。「你的手臂號稱有六個自由度，但在這個姿態下，它實際上只剩五個。少掉的那一個方向，你再怎麼轉馬達都到不了。」

他接著問了一個更尖銳的問題：「但為什麼控制器不是直接說『到不了』，而是說『速度超限』？」教室安靜下來。

叫獸畫了一個橢圓，然後把它壓扁成一條線：「因為問題不在『到不了』，而在『代價』。上一章的速度橢球，在這裡被壓扁了。末端想在那個消失的方向移動 0.1 公釐，關節就得轉幾百度——控制器算出了這個數字，發現超過馬達極限，只好停機保護。」他停了一下，「如果控制器沒有這個保護，會發生什麼事？手臂會用它最快的速度甩過去。這就是為什麼工業機器人的說明書上，奇異點永遠寫在安全章節裡。」

下課前，他在白板上並排寫下四件事：**第 8 章的萬向節鎖、第 9 章的手臂完全伸直、第 10 章的兩組解合而為一、第 11 章的可操作度歸零**。「這四件事，你們可能以為是四個不相干的現象。今天我要證明給你們看——它們是同一件事。而串起它們的，是一個線性代數裡最基本的概念：**矩陣的秩**。」

本章為什麼重要

奇異點是機器人學中最重要的「壞消息」。它無法用更好的控制演算法消除、無法用更強的

馬達克服，因為它是**機構幾何本身的性質**。任何機器手臂都有奇異點，這是數學的必然。

理解奇異點有三層價值。**第一層是安全**：奇異點附近的關節速度會暴增，是工業現場最常見的異常停機與機構損傷原因之一。**第二層是設計**：知道奇異點在哪裡，才能規劃工作站佈局與軌跡，讓機器人在「舒服」的區域作業。**第三層是理論**：奇異點的分析統一了前面四章的多個現象，並自然引出冗餘、零空間與多任務控制——這些正是現代協作機器人與人形機器人的核心技術。

本章也是第二篇的收尾。讀完之後，操作臂運動學的四塊拼圖（正向、逆向、速度、奇異）全部就位，你將具備完整分析一支機械手臂運動能力的工具。

學習目標

- 能以雅可比矩陣的秩定義奇異點，並說明其物理意義。
- 能區分邊界奇異點與內部奇異點，並各舉實例。
- 能運用行列式或最小奇異值判斷是否接近奇異點。
- 能說明奇異點處的三個現象：運動能力喪失、關節速度暴增、逆解退化。
- 能運用奇異值分解找出失去的運動方向與對應的關節速度方向。
- 能說明條件數的意義並用它評估姿態的數值品質。
- 能列舉六軸手臂的三類典型奇異點及其發生條件。
- 能說明五種處理奇異點的實務策略及其取捨。
- 能定義冗餘自由度並說明零空間的幾何意義。
- 能運用零空間投影實現次要目標最佳化。
- 能說明多任務優先權控制的原理與分層逆運動學的作法。

先備知識

第 11 章的雅可比矩陣（尤其是幾何構造法、速度橢球與可操作度）、第 10 章的數值逆運動學與偽逆、第 9 章的正向運動學。線性代數方面需要矩陣的秩、線性相依、零空間與奇異值分解的概念——本章會以幾何直覺重新說明這些概念，不需要事先精通。

核心名詞中英對照

表 12-1 本章核心名詞

中文	英文	說明
奇異點 / 奇異位形	Singularity	雅可比矩陣失去滿秩的機器人位形
秩	Rank	矩陣中線性獨立的行 (或列) 的數目
滿秩	Full Rank	秩等於行數與列數中較小者
邊界奇異點	Boundary Singularity	發生於工作空間邊界的奇異點
內部奇異點	Interior Singularity	發生於工作空間內部的奇異點
腕部奇異點	Wrist Singularity	手腕兩軸共線造成的奇異點
肩部奇異點	Shoulder Singularity	腕心落在第一軸線上造成的奇異點
肘部奇異點	Elbow Singularity	手臂完全伸直或折疊造成的奇異點
奇異值	Singular Value	SVD 分解中對角矩陣的元素
條件數	Condition Number	最大與最小奇異值之比
零空間	Null Space	使 $J \cdot \dot{q} = 0$ 的關節速度所張的空間
自運動	Self-motion	末端不動但關節持續變化的運動
冗餘度	Degree of Redundancy	關節數減去任務維度
零空間投影	Null Space Projection	把次要目標投影到不影響主任務的方向
任務優先權	Task Priority	多任務衝突時的執行順序安排
分層逆運動學	Hierarchical IK	依優先權逐層求解多任務的方法

12.1 奇異點的定義

12.1.1 由秩出發

第 11 章建立了速度關係 $\dot{x} = J(q) \cdot \dot{q}$ 。現在問一個問題：這個映射能覆蓋所有的末端速度方向嗎？

答案取決於雅可比矩陣的秩。矩陣 J 的秩，等於它的行向量中線性獨立的個數，也等於映射的像空間 (image) 的維度。若 J 是 6×6 的方陣：

$\text{rank}(J) = 6 \Rightarrow$ 末端可往六個方向自由運動 (非奇異)

$\text{rank}(J) < 6 \Rightarrow$ 至少有一個方向到不了 (奇異)

奇異點的嚴格定義：雅可比矩陣失去滿秩的位形。

秩下降的直觀意義是：某兩行 (或多行) 變成線性相依。換句話說，某兩個關節產生了「重

複」的運動效果——就像開頭故事中手腕的第四軸與第六軸轉出同樣的結果。兩個關節做同一件事，等於少了一個關節，於是有一個方向永遠到不了。

12.1.2 三種等價的判準

判斷是否奇異，有三種數學上等價但用途不同的方法。

判準一：行列式為零（僅適用於方陣）。

$$\det(J) = 0 \Leftrightarrow \text{奇異}$$

計算最快，但只能給出「是或否」，無法量化「有多接近」，且非方陣（冗餘機器人）無法使用。

判準二：可操作度為零（第 11.8.2 節）。

$$w = \sqrt{\det(J J^T)} = 0 \Leftrightarrow \text{奇異}$$

適用於任意形狀的矩陣，且能量化接近程度。但它是所有方向的乘積，某一方向的能力嚴重下降時， w 會被其他方向的大值稀釋。

判準三：最小奇異值為零（最推薦）。

$$\sigma_{\min} = 0 \Leftrightarrow \text{奇異}$$

直接反映「最弱的那個方向」，是實務上最可靠的指標。 $\sigma_{\min}$ 的值本身就代表該方向的運動能力大小。

實務上的標準作法是監控 $\sigma_{\min}$ ：當它低於某個門檻（例如最大奇異值的 5%）時，就啟動第 12.5 節的防護措施。

12.1.3 奇異點的三個症狀

在奇異點處，會同時出現三個彼此相關的現象——這三個症狀分別對應到前面三章遺留的謎題。

症狀一：運動能力喪失。末端在某個特定方向上完全無法移動，無論關節如何轉動。這是第 9 章例題 9-1(b) 中「手臂完全伸直後無法再沿徑向前進」的數學解釋。

症狀二：關節速度暴增。要在接近該方向（而非恰好在該方向）移動時，所需的關節速度趨於無窮大。這是開頭故事中控制器警報的原因，也是第 10 章偽逆法數值爆炸的來源。

$$\dot{q} = J^{-1} \cdot \dot{x}, \quad \text{當 } \sigma_{\min} \rightarrow 0 \text{ 時, } J^{-1} \text{ 的元素 } \rightarrow \infty$$

反矩陣的元素與奇異值的倒數成正比，因此最小奇異值趨近於零時，關節速度指令會趨於無窮大。

症狀三：逆解退化。原本分離的多組逆解，在奇異點處合併。這是第 10 章例題 10-2(b) 中「手臂完全伸直時兩組解合而為一」的現象。

◎ 為什麼症狀二比症狀一更危險

症狀一（到不了）看起來嚴重，實際上是安全的——控制器算出無解，直接報錯停止。真正危險的是症狀二：在奇異點**附近**（而非恰好在上面），數學上仍然有解，控制器會忠實地執行它算出的指令——而那個指令可能是「第五軸以每秒 800 度轉動」。若沒有速度限制保護，手臂會以最高速度暴衝。這就是為什麼實務上要監控的不是「是否恰好在奇異點」，而是「距離奇異點多近」。

12.2 邊界奇異點與內部奇異點

12.2.1 邊界奇異點

邊界奇異點發生在工作空間的邊界上——手臂完全伸直（外邊界）或完全折疊（內邊界）。此時末端已到達可達範圍的極限，無法再沿徑向往外（或往內）移動。

以平面 2R 手臂為例，第 11 章例題 11-3 已導出其可操作度為 $w = L_1 L_2 |\sin \theta_2|$ 。因此：

$$\theta_2 = 0^\circ \Rightarrow w = 0 \quad (\text{完全伸直，外邊界奇異點})$$

$$\theta_2 = 180^\circ \Rightarrow w = 0 \quad (\text{完全折疊，內邊界奇異點})$$

平面 2R 手臂的兩個奇異位形，皆只與肘關節角度有關。

邊界奇異點的一個重要特性是：**它在實務上相對容易避開**。只要規劃時保留工作空間的餘裕（第 4.10.3 節建議至少 10%），就不會接近它。此外，操作員的直覺也能幫忙——「手臂快伸直了，該把工件移近一點」是很自然的判斷。

12.2.2 內部奇異點

內部奇異點則麻煩得多。它發生在工作空間**內部**，位置往往不直觀，操作員在示教時完全感覺不到危險，直到執行軌跡時突然停機。

內部奇異點的成因是**兩個或多個關節軸線排成特殊的幾何關係**——通常是共線或平行。此時它們產生重複的運動效果，秩下降。開頭故事中手腕第四、六軸平行的情形，正是典型的內部奇異點。

這類奇異點的三個特性使它特別危險：一、位置分散在工作空間內部，難以憑直覺避開；二、可能恰好落在常用的作業區域；三、通過它時末端位置變化不大，但關節可能需要大幅翻轉。

圖 12-1 邊界奇異點與內部奇異點

建議配圖：左圖為平面 2R 手臂的工作空間（環狀區域），以紅色粗線標出外圓（完全伸直）與內圓（完全折疊）兩個邊界奇異點，並在旁畫出兩種手臂姿態的示意。右圖為六軸手臂的側視圖，以紅色標出三處內部奇異：手腕兩軸共線（腕部）、腕心通過第一軸線（肩部）、手臂完全伸直（肘部）。下方以顏色深淺表示可操作度分布，深色區為高可操作度、白色為奇異區。

12.3 六軸手臂的三類典型奇異點

12.3.1 腕部奇異點

發生條件：第四軸與第六軸共線（通常是第五軸轉到 0° 或 180° 時）。

物理現象：第四軸與第六軸的轉動效果完全相同，等於少了一個自由度。末端無法繞垂直於這條共線軸的某個方向轉動。

為何常見：許多常用姿態（如夾爪垂直向下、手腕保持水平）恰好會讓第五軸接近 0° 。這是工業現場最頻繁遇到的奇異點，也是開頭故事的情境。

典型症狀：末端只需要轉一個很小的角度，第四軸與第六軸卻同時以極高速度反向旋轉（一個 $+400^\circ/\text{s}$ 、另一個 $-400^\circ/\text{s}$ ），淨效果卻只有幾度的末端轉動。這在示教器上看起來就像手腕「發瘋似地扭動」。

12.3.2 肩部奇異點

發生條件：手腕中心點落在第一軸（腰部旋轉軸）的軸線上。

物理現象：此時無論第一軸如何轉動，手腕中心都不會移動（因為它就在轉軸上）。末端失去沿某個切線方向運動的能力。

典型症狀：當軌跡經過機器人正上方或正下方（基座軸線附近）時，第一軸需要瞬間旋轉 180° 。這是「機器人突然整個轉半圈」這類詭異行為的常見原因。

實務對策：工作站佈局時，避免讓作業路徑通過基座的正上方區域。這也是為什麼許多工業機器人的說明書會標出「基座上方的禁止作業圓柱區」。

12.3.3 肘部奇異點

發生條件：手臂完全伸直（或完全折疊），即上臂與前臂共線。

物理現象：這是邊界奇異點在空間手臂上的版本。末端無法再沿徑向（由肩部指向腕心的方

向) 移動。

典型症狀：要求末端繼續往外移動時，第二軸與第三軸的速度暴增，控制器報「超出工作空間」或「速度超限」。

實務對策：規劃時保留工作半徑的餘裕。多數控制器會在接近伸直極限時自動降速。

表 12-2 六軸手臂三類奇異點的比較

類型	發生條件	失去的運動方向	典型症狀	實務對策
腕部奇異	第 5 軸 $\approx 0^\circ$ 或 180° (第 4、6 軸共線)	繞某軸的轉動	第 4、6 軸高速反向旋轉	調整工具姿態、加入姿態餘隙
肩部奇異	腕心落在第 1 軸線上	沿切線方向的平移	第 1 軸瞬間翻轉 180°	避開基座正上方區域
肘部奇異	手臂完全伸直或折疊	沿徑向的平移	第 2、3 軸速度暴增	保留工作半徑 10% 以上餘裕

◎ 一個容易忽略的事實

奇異點的位置與機構尺寸無關，只與構型有關。無論你買的是臂展 500 mm 還是 3000 mm 的六軸手臂，只要都是球形手腕的 PUMA 構型，它們的奇異位形 (以關節角度描述) 完全相同。這意味著：在小型手臂上學到的奇異點知識，可以直接套用到大型手臂上；而在模擬器中驗證過的軌跡，也不會因為換成不同尺寸的實機而突然遇到新的奇異點。

12.4 奇異值分解的幾何解讀

12.4.1 SVD 告訴我們什麼

第 11.8.1 節已介紹奇異值分解 $J = U \Sigma V^T$ 。現在從奇異點的角度重新解讀它的三個矩陣：

U (左奇異向量)：其行向量是**任務空間**中的一組正交方向。第 i 行對應第 i 個奇異值，代表末端運動的一個主軸方向。

Σ (奇異值)：對角線上的 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_m \geq 0$ ，代表各方向的「運動增益」。 σ_i 的意義是：關節以單位速度沿第 i 個關節方向運動時，末端在第 i 個任務方向上獲得的速度。

V (右奇異向量)：其行向量是**關節空間**中的一組正交方向。第 i 行是產生第 i 個末端運動方向所需的關節速度組合。

因此 SVD 給出一組完整的「對應關係」：沿 $\mathbf{v}_i$ 方向轉動關節，末端會沿 $\mathbf{u}_i$ 方向移動，增益為 σ_i 。

12.4.2 奇異點的精確描述

有了 SVD，奇異點的描述變得極為精確：

$$\begin{aligned} \sigma_k = 0 &\Rightarrow \text{末端無法沿 } \mathbf{u}_k \text{ 方向運動} \\ &\Rightarrow \text{沿 } \mathbf{v}_k \text{ 方向轉動關節，末端完全不動（自運動）} \end{aligned}$$

第 k 個奇異值歸零時， $\mathbf{u}_k$ 是「失去的任務方向」、 $\mathbf{v}_k$ 是「無效的關節運動方向」。

這個結果同時給出了兩個重要資訊。第一，失去的是哪個方向——這對決定如何繞道極為有用。第二，哪組關節運動變成徒勞——這正是零空間的來源（12.6 節）。

以開頭故事的腕部奇異為例：SVD 會告訴你 $\mathbf{u}_k$ 是「繞某個特定軸的轉動」（末端到不了的方向），而 $\mathbf{v}_k$ 是 $[0, 0, 0, +1, 0, -1]^T$ （第四軸與第六軸等量反向轉動——確實，這樣轉的話末端完全不動）。

12.4.3 條件數：數值品質的指標

條件數定義為最大與最小奇異值之比：

$$\kappa = \sigma_{\max} / \sigma_{\min}$$

條件數。 $\kappa = 1$ 代表各方向能力完全相同（等向性，最理想）； $\kappa \rightarrow \infty$ 代表接近奇異。

條件數的實用價值在於它是無量綱的相對指標，不受機器人尺寸影響。這使得不同機型之間可以直接比較。實務上的經驗參考值為：

表 12-3 條件數的實務判讀

條件數 κ	狀態	說明	建議措施
1 ~ 5	優良	各方向能力接近，數值穩定	理想作業區，可全速運行
5 ~ 20	可接受	有明顯的方向差異但仍可控	正常作業，注意速度規劃
20 ~ 100	警戒	接近奇異，關節速度開始放大	啟動 DLS 阻尼、降低速度
> 100	危險	極接近奇異，數值不可靠	拒絕執行或大幅降速繞道

此外還有一個相關指標：等向性（isotropy），定義為 $1/\kappa$ 。它衡量速度橢球接近正球的程度。設計工作站時，讓主要作業區落在等向性高的區域，能同時獲得良好的運動性能與數值穩定

性。

12.5 奇異點的處理策略

12.5.1 五種策略

面對奇異點，工程上有五種處理方式。它們並非互斥，實務系統通常同時採用多種。

策略一：軌跡繞道（規劃層）。在路徑規劃階段就避開奇異區域。作法是把可操作度或條件數納入路徑的代價函數，讓規劃器自動偏好「舒服」的區域。優點是根本解決、不需即時處理；缺點是路徑會偏離最短路徑，且需要事先知道奇異點分布。

策略二：降速（執行層）。偵測到 $\sigma_{\min}$ 下降時，按比例降低末端速度，使關節速度維持在限制內。優點是保持路徑精度不變；缺點是節拍時間拉長，且在真正的奇異點上仍然無解（速度必須降為零）。

$$v_{\max_allowed} = \sigma_{\min} \cdot \dot{q}_{\max}$$

可容許的末端速度與最小奇異值成正比。這是降速策略的核心公式。

策略三：阻尼最小平方（數值層）。即第 10.7.3 節的 DLS。以 λ^2 阻尼項確保矩陣可逆，容許末端稍微偏離目標路徑以換取關節速度的可控。

$$\dot{q} = J^T(JJ^T + \lambda^2 I)^{-1} \cdot \dot{x}$$

可變阻尼的常見設計： $\lambda^2 = \lambda_0^2(1 - \sigma_{\min}/\sigma_{\text{門檻}})^2$ （ $\sigma_{\min}$ 低於門檻時才啟動阻尼）。

策略四：改變構型（規劃層）。利用第 10 章的多解結構，切換到另一組不會經過奇異點的解。缺點是切換過程本身需要大幅運動，只能在軌跡的起訖點進行。

策略五：增加自由度（機構層）。加裝第七軸或把手臂裝在行走軸上，使系統成為冗餘機構，即可用零空間繞開奇異點（12.6、12.7 節）。這是最根本的解決方案，也是協作機器人多採七軸設計的原因之一。

表 12-4 五種奇異點處理策略的比較

策略	層級	路徑精度	節拍影響	實作難度	適用場合
軌跡繞道	規劃	路徑改變	略增	中	離線編程、已知環境
降速	執行	完全保持	顯著增加	低	通用，工業控制器標配
DLS 阻尼	數值	略有偏差	幾乎不變	低	即時控制、遙操作

策略	層級	路徑精度	節拍影響	實作難度	適用場合
改變構型	規劃	完全保持	切換時大幅增加	中	軌跡起訖點
增加自由度	機構	完全保持	不變	高 (需硬體)	新系統設計、協作機器人

12.5.2 實務上的複合策略

成熟的工業控制器通常這樣組合：離線編程時用**策略一**檢查並修正路徑；執行時持續監控 $\sigma_{\min}$ ，接近門檻時啟動**策略二**降速；若仍不足則啟動**策略三**的 DLS 阻尼；若條件數超過危險門檻則直接拒絕執行並報錯。這是一個由粗到細的多層防護。

對協作機器人與研究平台，則多採用**策略五**——直接用七軸機構，把問題在設計階段就解決掉大半。

◎ 奇異點無法被「消除」

常有學生問：能不能設計一支沒有奇異點的機械手臂？答案是**不能**。這是一個拓撲學上的結論：任何以旋轉關節構成的串聯手臂，其工作空間中必定存在奇異位形。這與第 8 章「任何用三個數描述三維旋轉的方法必有奇異點」是同一類的數學必然。工程上能做的，只有把奇異點「移到」不常用的區域、或增加自由度讓機器人有繞道的餘裕——而不是消滅它們。

12.6 冗餘與零空間

12.6.1 冗餘的定義

第 4.9 節已介紹冗餘的概念：當機器人的自由度多於任務所需時，稱為冗餘。現在可以用雅可比精確定義：

$$\text{冗餘度 } r = n - m \quad (n \text{ 為關節數、} m \text{ 為任務維度})$$

當 $n > m$ 時， J 是「寬矩陣」（行多於列），方程式 $J \cdot \dot{q} = \dot{x}$ 的解不唯一。

最典型的例子是七軸手臂執行六維任務（ $r = 1$ ），以及人類手臂（七自由度）執行「把手掌放到指定姿」（六維任務）——正是第 4 章開頭那個「手掌貼牆卻能抬起手肘」的實驗。

12.6.2 零空間的幾何意義

冗餘帶來的關鍵結構是**零空間**（null space）：

$$N(J) = \{ \dot{q} : J \cdot \dot{q} = 0 \}$$

零空間是所有「不產生任何末端運動」的關節速度所構成的集合。

這個定義的物理意義極為優美：沿零空間方向轉動關節，末端完全不動，但手臂的形狀改變了。這種運動稱為**自運動**（self-motion）。人類手臂那個「手掌固定、手肘上下擺動」的動作，就是沿零空間的自運動。

零空間的維度等於冗餘度 r （在非奇異的情況下）。七軸手臂執行六維任務時，零空間是一維的——因此自運動只有一個方向（手肘繞著「肩腕連線」這條軸擺動），這正是為什麼人類手臂的自運動看起來像手肘畫一個圓弧。

圖 12-2 冗餘手臂的零空間自運動

建議配圖：一支七軸手臂（或人類手臂）的三張疊影，末端夾爪位置與姿態完全相同（以綠色圓圈標出並註明「末端固定」），但手肘位置不同，沿一個圓弧分布（以藍色虛線畫出手肘的移動軌跡，並標註「自運動流形」）。旁邊以小圖標出「肩—腕連線」這條自運動的旋轉軸。下方文字：末端不動，形狀可變——這就是零空間。

12.6.3 零空間出現在奇異點

這裡有一個容易混淆但極為重要的區別。零空間有兩種來源：

來源一：冗餘造成的零空間（好的）。當 $n > m$ 時，即使 J 滿秩，零空間仍然存在且維度為 $n - m$ 。這是可用的資源，讓我們能在完成任務的同時追求次要目標。

來源二：奇異造成的零空間（壞的）。當 J 失去滿秩時（無論是否冗餘），會出現額外的零空間維度。這對應到 12.4.2 節的 v_k 方向——沿該方向轉動關節是徒勞的，因為末端不動，但我們也失去了對應的 u_k 任務方向。

兩者的差別在於：冗餘的零空間**不伴隨任務方向的損失**（機器人仍能到達所有六個方向），而奇異的零空間**伴隨著任務方向的損失**。因此判斷方法是：檢查 J 的秩是否等於 $\min(m, n)$ ——是則為冗餘零空間，否則為奇異零空間。

12.7 零空間投影與次要目標

12.7.1 通解的結構

對冗餘機器人，方程式 $J \cdot \dot{q} = \dot{x}$ 有無限多解。線性代數告訴我們，其通解的結構為「特解 + 齊次解」：

$$\dot{q} = J^+ \cdot \dot{x} + (I - J^+) \cdot z$$

第一項是偽逆給出的特解（範數最小的解）、第二項是零空間中的任意元素。 z 可以是任何 n 維向量。

矩陣 $P = (I - J^+J)$ 稱為**零空間投影算子**。它有兩個關鍵性質：

$$J \cdot P = 0 \quad (\text{投影後的結果必在零空間中})$$

$$P \cdot P = P \quad (\text{冪等性，投影兩次等於投影一次})$$

這保證了第二項無論 z 取什麼值，都不會影響末端運動——主任務永遠被完美執行。

這個結構的工程意義極為重大：它讓我們可以「免費」追求次要目標。主任務（末端軌跡）由第一項保證，而第二項提供了一個完全不干擾主任務的通道，可以用來做任何我們想做的事。

12.7.2 四種常見的次要目標

次要目標透過選擇 z 來指定。標準作法是把 z 取為某個目標函數 $H(q)$ 的梯度：

$$z = \pm k \cdot \nabla H(q)$$

取正號代表最大化 H 、負號代表最小化。 k 為增益，控制次要目標的積極程度。

目標一：遠離關節極限。定義 H 為各關節與其行程中點的距離平方和，最小化它可使機器人傾向於保持在關節行程的中央：

$$H(q) = \sum_i (q_i - q_{\text{mid},i})^2 / (q_{\text{max},i} - q_{\text{min},i})^2$$

常用的關節極限迴避函數。這是第 10.8.2 節提到的第三種處理方式。

目標二：最大化可操作度（避開奇異點）。取 $H(q) = w(q)$ 並最大化它。這使機器人在執行任務的同時，自動維持在靈活的姿態——這是策略五（12.5.1 節）的具體實現。

目標三：避開障礙物。定義 H 為手臂各點與障礙物的最小距離，最大化它。這讓手臂能在末端沿既定軌跡運動的同時，自動「閃避」環境中的障礙——正是第 4 章開頭那個六軸手臂撞到治具的問題的解方。

目標四：維持自然或舒適姿勢。定義 H 為與某個參考姿態 q_{ref} 的距離，最小化它。這對人形機器人與協作機器人特別重要——讓機器人的動作看起來自然，能顯著提升人類的接受度（第 35 章）。

表 12-5 四種次要目標的設計

次要目標	目標函數 $H(q)$	最佳化方向	典型應用
遠離關節極限	與行程中點的正規化距離平方和	最小化	長時間連續作業、避免中途停機

次要目標	目標函數 $H(q)$	最佳化方向	典型應用
最大化可操作度	$w = \sqrt{\det(J J^T)}$	最大化	避開奇異點、維持靈活姿態
避開障礙物	手臂各點與障礙的最小距離	最大化	狹窄空間作業、人機共處
維持自然姿勢	與參考姿態的加權距離	最小化	人形機器人、協作機器人

12.7.3 實作要點與陷阱

要點一：增益不能太大。次要目標的增益 k 過大時，雖然理論上不影響主任務，但數值誤差會累積，且關節可能以很高的速度做自運動。實務上應設定自運動的速度上限。

要點二：次要目標之間可能衝突。同時要求「遠離關節極限」與「最大化可操作度」時，兩個梯度方向可能相反。此時需要加權，或改用 12.8 節的分層架構。

要點三：投影算子的計算成本。 $P = (I - J^+J)$ 需要計算偽逆並做矩陣乘法，對高自由度系統成本不低。實務上常用 SVD 一次算出所有需要的量，或採用遞迴的分層算法。

要點四：零空間在奇異點會「變質」。如 12.6.3 節所述，接近奇異時零空間會混入「奇異造成的方向」，此時沿零空間運動可能無法真正達成次要目標。因此應同時監控 $\sigma_{\min}$ 。

12.8 多任務與分層逆運動學

12.8.1 多任務的需求

實際的機器人往往必須同時滿足多個任務。以一台雙臂協作機器為例，它可能同時需要：右手跟隨軌跡（6 維）、左手保持不動（6 維）、頭部相機朝向工件（2 維）、身體保持平衡（2 維）、以及所有關節遠離極限。若機器人只有 14 個自由度，這些任務的總維度已經超過——必然有些任務無法完全滿足。

此時需要一套機制來決定：哪些任務優先、哪些可以妥協。這就是任務優先權（task priority）的概念。

12.8.2 合併為單一大任務

最簡單的作法是把所有任務堆疊成一個「大任務」，各自加權：

$$\Phi(q) = \begin{bmatrix} \sqrt{\rho_1} (\phi_1(q) - y_1^*) \\ \sqrt{\rho_2} (\phi_2(q) - y_2^*) \\ \vdots \end{bmatrix}$$

每個任務 i 有自己的運動圖 φ_i 、目標值 y_i^* 與權重 ρ_i 。權重越大代表越重要。

對應的大雅可比矩陣就是各任務雅可比的垂直堆疊。求解方式與單一任務相同，只是矩陣變高了。

這個作法的優點是實作簡單、所有任務同時被考慮。缺點是**任務之間會互相妥協**——即使把某個任務的權重設得很高，它仍可能因為其他任務而產生誤差。對於「絕對不能違反」的任務（如關節極限、避免碰撞），這是不可接受的。

12.8.3 分層逆運動學

嚴格的作法是**分層**：先完全滿足最高優先權的任務，再在它的零空間中滿足次高優先權的任務，依此類推。

以兩層為例，主任務為 $\dot{x}_1$ （雅可比 J_1 ）、次任務為 $\dot{x}_2$ （雅可比 J_2 ）：

$$\dot{q}' = J_1^+ \dot{x}_1 + (J_2 P_1)^+ \cdot (\dot{x}_2 - J_2 J_1^+ \dot{x}_1) \quad , \text{ 其中 } P_1 = I - J_1^+ J_1$$

第一項完全滿足主任務；第二項在主任務的零空間中盡量滿足次任務。括號中的修正項扣除了主任務運動對次任務造成的「副作用」。

這個公式可以遞迴推廣到任意多層。它保證了**嚴格的優先權**：低優先權的任務絕不會干擾高優先權的任務，即使前者完全無法達成。

代價是計算成本較高（每一層都要做一次偽逆），且層數越多、可用的自由度越少——到某一層時零空間可能已經耗盡，該層以下的任務完全無法執行。

12.8.4 實務上的折衷

Toussaint 等學者指出一個實務上的觀察：**嚴格的分層在數值上未必最穩健**。因為每一層的投影都會使矩陣的條件數惡化，多層之後數值品質可能很差。

因此實務上常採取折衷：對真正不可違反的約束（碰撞、關節極限）使用嚴格分層或硬約束；對其餘任務則使用加權合併，並以權重的數量級差異（例如相差 100 倍）來近似優先權。這樣既保留了優先權的效果，又維持了數值穩定性。

這個主題在第 17 章（操作空間控制）與第 28 章（人形機器人全身控制）會再次出現，屆時還要加入動力學的考量。

12.9 第二篇的總結

12.9.1 四塊拼圖

走完第 8 至 12 章，操作臂運動學的完整體系已經建立。回顧這五章：

第 8 章建立了描述空間的語言——座標系、旋轉矩陣、四元數、齊次變換。這是所有幾何運算的字母表。

第 9 章 (正向運動學) 回答「手在哪裡」：把關節角度代入 DH 參數，連乘變換矩陣即得末端位姿。解必定存在且唯一。

第 10 章 (逆向運動學) 回答「手該怎麼擺」：由目標位姿反求關節角度。可能無解、多解或無限多解，需用封閉解或數值迭代。

第 11 章 (雅可比) 回答「手能怎麼動」：把關節速度映射到末端速度，並透過轉置連結力與扭矩。

第 12 章 (奇異點與冗餘) 回答「手何時動不了、以及多餘的自由度怎麼用」：由雅可比的秩統一解釋所有的退化現象，並用零空間開發冗餘的價值。

12.9.2 串起來的線索

本章開頭承諾要證明四個現象是同一件事，現在可以完整回答：

表 12-6 四個現象的統一解釋

現象	出現章節	表面症狀	統一的數學解釋
萬向節鎖	第 8.3.3 節	兩軸對齊，失去一個旋轉自由度	解析雅可比中的 $E(\varphi)$ 矩陣奇異
手臂完全伸直	第 9 章例題 9-1	無法再沿徑向前進	幾何雅可比失去滿秩 (肘部奇異)
兩組逆解合併	第 10 章例題 10-2	肘上與肘下的解重合	解空間在奇異點處退化
可操作度歸零	第 11 章例題 11-3	$w = L_1 L_2 \sin \theta_2 \rightarrow 0$	最小奇異值 $\sigma_{\min} \rightarrow 0$

四者的共同根源是：**雅可比矩陣失去滿秩**。這正是為什麼線性代數中一個看似抽象的概念——矩陣的秩——會成為機器人學的核心工具。

12.9.3 接下來：讓運動學動起來

到目前為止，我們的分析都是**運動學的** (kinematic) ——只談位置、速度與幾何，完全沒有考慮質量、慣性與力的動態效應。這隱含了一個假設：關節可以瞬間達到任何我們指定的速度。

這個假設在低速時尚可接受，但在高速運動、重負載或需要精確力控制時就會失效。手臂有質量、有慣性、會因重力下垂、加速時需要額外的力矩、旋轉時會產生科氏力——這些都是**動力學** (dynamic) 的範疇。

第三篇 (第 13 ~ 21 章) 將處理這些問題：第 13、14 章建立動力學方程 (Lagrange 法與 Newton-Euler 法)、第 15 章規劃考慮動態限制的軌跡、第 16 ~ 19 章設計控制器、第 20 章處理力控制、第 21 章討論抓取。屆時你會發現，本篇建立的雅可比矩陣仍然無所不在——它將出現在慣性矩陣、科氏項、以及操作空間控制的每一個公式中。

換句話說：第二篇教你機器人**能不能到那裡**，第三篇教你**該用多大的力氣到那裡**。

完整計算例題

例題 12-1 平面 2R 手臂的奇異點分析

題目：平面 2R 手臂 $L_1 = 300 \text{ mm}$ 、 $L_2 = 200 \text{ mm}$ 。(a) 求所有奇異位形。(b) 在 $\theta_2 = 5^\circ$ 時計算雅可比、行列式、奇異值與條件數。(c) 若末端要以 100 mm/s 沿最難運動的方向移動，所需的關節速度為多少？(d) 評估此姿態是否安全。

解 (a)：由第 11 章例題 11-3， $\det(J) = L_1 L_2 \sin \theta_2$ 。令其為零：

$$L_1 L_2 \sin \theta_2 = 0 \Rightarrow \theta_2 = 0^\circ \text{ 或 } 180^\circ$$

奇異位形只取決於 θ_2 ：完全伸直 (0°) 或完全折疊 (180°)，與 θ_1 完全無關。

解 (b)：取 $\theta_1 = 30^\circ$ 、 $\theta_2 = 5^\circ$ 。則 $\theta_1 + \theta_2 = 35^\circ$ 。

$$J = \begin{bmatrix} -300 \sin 30^\circ - 200 \sin 35^\circ & -200 \sin 35^\circ \\ 300 \cos 30^\circ + 200 \cos 35^\circ & 200 \cos 35^\circ \end{bmatrix} = \begin{bmatrix} -264.72 & -114.72 \\ 423.61 & 163.83 \end{bmatrix}$$

單位 mm/rad 。

$$\det(J) = L_1 L_2 \sin 5^\circ = 300 \times 200 \times 0.08716 = 5229.6 \text{ mm}^2/\text{rad}^2$$

行列式，遠小於最大值 60000。

計算奇異值 (此處直接給出 SVD 結果)： $\sigma_1 \approx 528.4$ 、 $\sigma_2 \approx 9.90 \text{ mm/rad}$ 。

$$\kappa = \sigma_1 / \sigma_2 = 528.4 / 9.90 \approx 53.4$$

條件數約 53，依表 12-3 落在「警戒」區間。

解 (c)：最難運動的方向對應最小奇異值 $\sigma_2 = 9.90 \text{ mm/rad}$ 。所需的關節速度大小為：

$$\|\dot{\mathbf{q}}\| = \|\dot{\mathbf{x}}\| / \sigma_{\min} = 100 / 9.90 \approx 10.1 \text{ rad/s} \approx 579^\circ/\text{s}$$

關節需以每秒近 580 度轉動——遠超過一般伺服馬達的能力（典型上限約 180~360 °/s）。

作為對比，若沿最容易的方向（ $\sigma_1 = 528.4$ ）移動同樣的 100 mm/s，只需 $100/528.4 \approx 0.19 \text{ rad/s} \approx 10.8^\circ/\text{s}$ 。同樣的末端速度，兩個方向所需的關節速度相差 53 倍——這正是條件數的物理意義。

解 (d)：不安全。條件數 53 已進入警戒區，且沿某方向的關節速度需求已超過馬達能力。建議措施：一、若軌跡允許，把 θ_2 增大到 30° 以上（此時 κ 降至約 10 以下）；二、若必須經過此姿態，啟動 DLS 阻尼並降低末端速度至約 30 mm/s 以下；三、規劃時把此區域標記為避開區。

例題 12-2 冗餘手臂的零空間自運動

題目：某平面 3R 手臂 $L_1 = L_2 = L_3 = 200 \text{ mm}$ ，執行「末端到達指定位置 (x, y)」的任務（僅位置，2 維）。目前姿態為 $\theta = (30^\circ, 45^\circ, -30^\circ)$ 。（a）求冗餘度。（b）求雅可比與零空間的維度。（c）求零空間的基底向量，並解釋其物理意義。（d）若要沿零空間運動使第二關節角度減小，該如何選擇 $\mathbf{z}$ ？

解 (a)：關節數 $n = 3$ 、任務維度 $m = 2$ （只控制位置，不控制姿態）。

$$\text{冗餘度 } r = n - m = 3 - 2 = 1$$

有一個冗餘自由度。

解 (b)：位置雅可比為 2×3 矩陣。各關節的累積角度為 $\theta_1 = 30^\circ$ 、 $\theta_{12} = 75^\circ$ 、 $\theta_{123} = 45^\circ$ 。

$$\mathbf{J} = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} - L_3 s_{123} & -L_2 s_{12} - L_3 s_{123} & -L_3 s_{123} \\ L_1 c_1 + L_2 c_{12} + L_3 c_{123} & L_2 c_{12} + L_3 c_{123} & L_3 c_{123} \end{bmatrix}$$

2×3 的位置雅可比。

代入數值（ $s_1=0.5$, $c_1=0.866$, $s_{12}=0.966$, $c_{12}=0.259$, $s_{123}=c_{123}=0.707$ ）：

$$\mathbf{J} = \begin{bmatrix} -434.5 & -334.5 & -141.4 \\ 346.9 & 193.2 & 141.4 \end{bmatrix}$$

單位 mm/rad。

由於 J 為 2×3 且滿秩 ($\text{rank} = 2$)，零空間維度為 $n - \text{rank}(J) = 3 - 2 = 1$ 維。

解 (c)：零空間的基底向量 v 滿足 $J \cdot v = 0$ 。解此齊次方程組可得 (取單位化後)：

$$v \approx [0.412, -0.833, 0.370]^T$$

零空間基底。沿此方向轉動關節，末端位置完全不變。

物理意義：三個關節以此比例同時轉動時，手臂的形狀改變 (中間的關節位置移動)，但末端維持在原處。這正是「手掌固定、手肘擺動」的平面版本。可驗證 $J \cdot v \approx [0, 0]^T$ 。

解 (d)：要使 θ_2 減小，次要目標可定義為 $H(q) = -\theta_2$ (最大化 $-\theta_2$ 即最小化 θ_2)，其梯度為 $\nabla H = [0, -1, 0]^T$ 。代入零空間投影：

$$q'_{\text{null}} = (I - J^+ J) \cdot k \cdot [0, -1, 0]^T$$

投影後的結果必定平行於 v (因為零空間是一維的)，方向取決於 $[0, -1, 0]$ 與 v 的內積符號。

由於 v 的第二個分量为 -0.833 (負值)，而目標梯度的第二個分量也為負，兩者內積為正，因此投影結果會沿 $+v$ 方向——即 $q' \approx k' \cdot [0.412, -0.833, 0.370]^T$ ，確實使 θ_2 減小，同時 θ_1 與 θ_3 自動調整以維持末端不動。

例題 12-3 六軸手臂的腕部奇異判定

題目：某六軸手臂目前關節角度為 $\theta = (20^\circ, -40^\circ, 60^\circ, 15^\circ, 2^\circ, -35^\circ)$ 。(a) 判斷是否接近腕部奇異點。(b) 若第五軸持續減小到 0.5° ，估算條件數的變化趨勢。(c) 提出兩種處理方式。

解 (a)：腕部奇異的條件是第五軸接近 0° 或 180° (第 4、6 軸共線)。目前 $\theta_5 = 2^\circ$ ，極為接近奇異點。

對球形手腕，其姿態部分的雅可比行列式正比於 $\sin \theta_5$ ：

$$\det(J_{\text{wrist}}) \propto \sin \theta_5 = \sin 2^\circ = 0.0349$$

僅為最大值 ($\theta_5 = 90^\circ$ 時) 的 3.5%。

解 (b)：條件數大致與 $1/\sin \theta_5$ 成正比。當 θ_5 由 2° 降至 0.5° ：

$$\kappa \text{ 的比值} \approx \sin 2^\circ / \sin 0.5^\circ = 0.0349 / 0.00873 \approx 4.0$$

條件數會惡化約 4 倍。若原本 $\kappa \approx 60$ (警戒)，將升至約 240 (危險)。

這說明了一個重要特性：接近奇異點時，狀況惡化的速度是非線性的。從 $\theta_5 = 10^\circ$ 降到 5°

只惡化 2 倍，但從 2° 降到 0.5° 卻惡化 4 倍，從 0.5° 降到 0.1° 則惡化 5 倍。這正是為何必須設定門檻並提早介入，而非等到真正逼近才處理。

解 (c)：處理方式一（調整工具姿態）：多數任務對工具繞其自身軸的旋轉角度並不敏感（例如吸盤、圓形夾爪、點焊槍）。若把工具的目標姿態繞末端 Z 軸旋轉一個角度，往往能讓 θ_5 遠離 0° ，完全避開此奇異點。這是成本最低、效果最好的作法。

處理方式二（DLS 加降速）：若姿態不可調整，則啟動阻尼並降低末端角速度。依 12.5.1 節，可容許的末端速度應與 $\sigma_{\min}$ 成正比—— $\theta_5 = 2^\circ$ 時，末端角速度應降至正常值的 3.5% 以下。這會嚴重影響節拍，因此僅作為備案。

程式範例：奇異點分析與零空間控制

以下 Python 程式實作奇異點的偵測與量化、SVD 幾何解讀、DLS 阻尼比較，以及冗餘手臂的零空間自運動。

```
import numpy as np

# ----- 平面手臂通用模型 -----
class PlanarArm:
    def __init__(self, links):
        self.L = np.array(links, dtype=float)
        self.n = len(links)

    def fk(self, q):
        """回傳末端位置 (x, y)"""
        ang = np.cumsum(q)
        return np.array([np.sum(self.L * np.cos(ang)),
                        np.sum(self.L * np.sin(ang))])

    def jacobian(self, q):
        """2xn 位置雅可比"""
        ang = np.cumsum(q)
        J = np.zeros((2, self.n))
        for j in range(self.n):
            J[0, j] = -np.sum(self.L[j:] * np.sin(ang[j:]))
            J[1, j] = np.sum(self.L[j:] * np.cos(ang[j:]))
        return J

    def analyze(self, q):
        """回傳姿態的「健康度報告」"""
        J = self.jacobian(q)
        U, S, Vt = np.linalg.svd(J)
        w = np.sqrt(np.linalg.det(J @ J.T)) # 可操作度
        kappa = S[0] / S[-1] if S[-1] > 1e-12 else np.inf
```

```

        return dict(J=J, U=U, S=S, Vt=Vt, w=w, kappa=kappa)

d, r2d = np.deg2rad, np.rad2deg

# ----- 例題 12-1：平面 2R 奇異點分析 -----
print("=== 例題 12-1：平面 2R 奇異點掃描 ===")
arm2 = PlanarArm([300.0, 200.0])
print(f"{'θ2':>6} {'det(J)':>10} {'σ_min':>9} {'κ':>9} 狀態")
for t2 in (0.5, 5, 15, 30, 60, 90, 120, 179):
    r = arm2.analyze([d(30), d(t2)])
    det = np.linalg.det(r['J'])
    if r['kappa'] < 5: st = "優良"
    elif r['kappa'] < 20: st = "可接受"
    elif r['kappa'] < 100: st = "警戒"
    else: st = "危險"
    print(f"{t2:6.1f} {det:10.1f} {r['S'][-1]:9.2f} {r['kappa']:9.1f} {st}")
print("→ 驗證 det(J) = L1·L2·sin(θ2) · 奇異點在 θ2 = 0° 與 180°")

# ----- SVD 幾何解讀 (12.4 節) -----
print("\n=== SVD 解讀：θ2 = 5° (接近奇異) ===")
r = arm2.analyze([d(30), d(5)])
print(f"奇異值 σ = ({r['S'][0]:.2f}, {r['S'][1]:.2f}) mm/rad")
print(f"最易運動方向 u1: 角度 {r2d(np.arctan2(r['U'][1,0], r['U'][0,0])):7.1f}°"
      f" 增益 {r['S'][0]:.1f}")
print(f"最難運動方向 u2: 角度 {r2d(np.arctan2(r['U'][1,1], r['U'][0,1])):7.1f}°"
      f" 增益 {r['S'][1]:.1f}")
for i, name in enumerate(("最易", "最難")):
    need = 100.0 / r['S'][i]
    print(f"沿{name}方向以 100 mm/s 運動 → 關節速度 {r2d(need):7.1f} °/s")

# ----- 偽逆 vs DLS 在奇異點附近 (12.5 節) -----
print("\n=== 偽逆 vs DLS：末端 100 mm/s 沿最難方向 ===")
x_dot = r['U'][:, 1] * 100.0 # 沿最難方向
J = r['J']
qd_pinv = np.linalg.pinv(J) @ x_dot
for lam in (0.0, 5.0, 20.0):
    if lam == 0.0:
        qd, tag = qd_pinv, "偽逆(λ=0)"
    else:
        qd = J.T @ np.linalg.solve(J @ J.T + lam**2*np.eye(2), x_dot)
        tag = f"DLS(λ={lam:.0f})"
    achieved = J @ qd
    err = np.linalg.norm(x_dot - achieved)
    print(f" {tag:12s} 關節速度 max={r2d(np.abs(qd).max()):8.1f} °/s "
          f"末端誤差={err:6.2f} mm/s")
print(" → DLS 大幅降低關節速度，代價是末端略微偏離目標")

```

```

# ----- 例題 12-2：冗餘手臂的零空間 -----
print("\n=== 例題 12-2：平面 3R 的零空間自運動 ===")
arm3 = PlanarArm([200.0, 200.0, 200.0])
q = np.array([d(30), d(45), d(-30)])
J3 = arm3.jacobian(q)
print(f"雅可比 2x3 · 秩 = {np.linalg.matrix_rank(J3)} · "
      f"零空間維度 = {3 - np.linalg.matrix_rank(J3)}")

# 求零空間基底 (SVD 的最後一個右奇異向量)
U, S, Vt = np.linalg.svd(J3)
v_null = Vt[-1]
print(f"零空間基底 v = {np.round(v_null, 4)}")
print(f"驗證 J·v = {np.round(J3 @ v_null, 10)} (應為零向量)")

# 沿零空間運動：末端不動 · 形狀改變
print("\n 沿零空間自運動 (每步 5°) : ")
P_null = np.eye(3) - np.linalg.pinv(J3) @ J3      # 零空間投影算子
q_sm = q.copy()
for step in range(4):
    pos = arm3.fk(q_sm)
    print(f" 步{step}: θ=({r2d(q_sm[0]):6.1f},{r2d(q_sm[1]):6.1f}, "
          f"{r2d(q_sm[2]):6.1f})° 末端=({pos[0]:7.2f},{pos[1]:7.2f})")
    q_sm = q_sm + P_null @ (d(5) * np.sign(v_null))
print(" → 三個關節角度都在變 · 但末端位置幾乎不動")

# ----- 次要目標：最大化可操作度 -----
print("\n=== 零空間次要目標：最大化可操作度 ===")
def manip(arm, q):
    J = arm.jacobian(q)
    return np.sqrt(np.linalg.det(J @ J.T))

q_opt = q.copy()
print(f"起始: w = {manip(arm3, q_opt):.1f}, 末端 = {np.round(arm3.fk(q_opt),2)}")
for it in range(30):
    J3 = arm3.jacobian(q_opt)
    Pn = np.eye(3) - np.linalg.pinv(J3) @ J3
    grad = np.zeros(3)      # 數值梯度
    for j in range(3):
        dq = np.zeros(3); dq[j] = 1e-5
        grad[j] = (manip(arm3, q_opt+dq) - manip(arm3, q_opt-dq)) / 2e-5
    q_opt = q_opt + 2e-6 * (Pn @ grad)      # 只沿零空間爬升
print(f"最佳化後: w = {manip(arm3, q_opt):.1f}, "
      f"末端 = {np.round(arm3.fk(q_opt),2)}")
print(f"關節角度: ({r2d(q_opt[0]):.1f}, {r2d(q_opt[1]):.1f}, "
      f"{r2d(q_opt[2]):.1f})°")
print("→ 可操作度提升 · 末端位置維持不變——這就是冗餘的價值")

```

請試著修改：（一）把 arm2 的掃描細分到 $\theta_2 = 0.1^\circ$ ，觀察條件數如何爆炸。（二）在零空間自運動中改用「遠離關節極限」作為次要目標（梯度指向各關節行程的中點），觀察姿態如何變化。（三）把平面 3R 改為控制位置與姿態（3 維任務），此時冗餘度為 0，驗證零空間會消失。

實作：奇異點的實驗觀察

本章實作是在實體手臂上親眼觀察奇異點的三個症狀。**安全提醒：本實驗必須在低速下進行，並確保緊急停止隨手可及（第 7.9 節）。**

- 繪製可操作度地圖：**以第 11 章的程式計算工作空間中各點的可操作度，繪成熱力圖。標出可操作度低於最大值 10% 的「危險區」。
- 邊界奇異點觀察：**讓末端沿一條徑向直線由內往外緩慢移動（速度設為額定的 10%），記錄各關節的速度變化。接近工作空間邊界時，觀察哪個關節的速度開始上升，以及控制器是否發出警告。
- 條件數即時監控：**撰寫程式即時計算並顯示條件數。手動操作手臂經過不同區域，觀察數值變化，找出條件數超過 20 的所有姿態。
- 內部奇異點搜尋：**若使用六軸手臂，把第五軸緩慢轉向 0° ，觀察第四軸與第六軸的行為。嘗試讓末端繞某個軸小幅轉動，記錄兩軸的速度指令。
- 三個症狀的驗證：**在接近奇異的姿態下，分別驗證：（a）某方向無法運動（嘗試沿該方向移動，觀察是否報錯）；（b）關節速度暴增（記錄速度指令）；（c）逆解退化（用第 10 章的程式求逆解，觀察多組解是否合併）。
- DLS 效果比較：**以第 10 章的程式，分別用偽逆與 DLS 執行同一段經過奇異區的軌跡（僅在模擬中執行），比較關節速度峰值與末端路徑誤差。
- 繞道策略驗證：**修改軌跡使其避開危險區，比較繞道前後的執行時間、最大關節速度與是否觸發警報。

表 12-7 奇異點觀察記錄表（第一列為範例）

測試項	姿態	條件數	最大關節速度	末端速度	控制器反應
邊界奇異	$\theta_2 = 8^\circ$	42	$312^\circ/\text{s}$	50 mm/s	警告：接近奇異點
（自行填寫其餘測試）					

國中小也看得懂：有一個方向你就是去不了

伸直你的手臂，指尖碰到牆壁。現在，試著讓指尖**再往前一點點**——做不到對不對？不管你怎麼用力、怎麼轉肩膀，手臂已經伸到極限了。這就是機器人學裡說的「奇異點」：有一個方向，你就是去不了。

再試一個更神奇的實驗。把手臂伸直（但不要碰牆），然後試著讓指尖往前移動**一公分**。你會發現：肩膀和手肘必須轉好大一個角度，指尖才會往前一點點。**越接近伸直，就要轉越多才能前進一點點**。機器人也一樣——這時候它的馬達會拚命高速旋轉，控制器就會發出警告：「太危險了，停下來！」

那怎麼辦呢？聰明的辦法是**多一個關節**。你的手臂其實有七個可以轉的地方（肩膀三個、手肘一個、手腕三個），比「把手放到某個位置」需要的六個還多一個。多出來的那一個，讓你可以在手不動的情況下擺動手肘——繞開那些「去不了」的姿勢。

所以工程師設計機器人時，會刻意讓某些機器人多一隻「手肘」。這樣它就能像你一樣，遇到卡住的時候換個姿勢再試一次。你在附錄 E、J 做的機器人，關節越多就越靈活，原因就在這裡喔！

叫獸提醒與常見錯誤

- 誤區一：以為奇異點可以靠好的演算法消除。它是機構幾何的性質，任何串聯手臂都必定存在，只能避開或繞道。
- 誤區二：只檢查「是否恰好在奇異點」。真正危險的是奇異點**附近**——此時數學上有解，控制器會忠實執行一個極大的速度指令。應監控 $\sigma_{\min}$ 或條件數。
- 誤區三：用行列式判斷冗餘機器人的奇異。非方陣沒有行列式，應改用最小奇異值或 $\sqrt{\det(JJ^T)}$ 。
- 誤區四：只看可操作度 w 。它是所有奇異值的乘積，某一方向嚴重退化時可能被其他方向的大值掩蓋。應同時檢視 $\sigma_{\min}$ 。
- 誤區五：混淆兩種零空間。冗餘造成的零空間是可用資源；奇異造成的零空間則伴隨任務方向的損失。判斷方法是檢查 J 的秩。

- 誤區六：次要目標的增益設得太大。雖然理論上不影響主任務，但數值誤差累積與過快的自運動都可能造成問題。
- 誤區七：以為分層越多越好。每一層投影都會惡化條件數，層數過多時數值品質會嚴重下降。
- 誤區八：忽略腕部奇異的普遍性。夾爪垂直向下這類常用姿態很容易落在第五軸接近 0° 的位置，是工業現場最頻繁的異常來源。
- 誤區九：規劃時只檢查起訖點。奇異點可能出現在路徑中間，必須對整條軌跡逐點檢查條件數。
- 誤區十：以為換一台更大的機器人就能避開。奇異位形只與構型有關、與尺寸無關，同構型的大小手臂奇異點完全相同。

課堂討論

1. 開頭故事中控制器報的是「速度超限」而非「無法到達」。請解釋兩者的差異，並討論為何後者反而比較安全。
2. 第 12.5 節列出五種處理奇異點的策略。請為以下三個情境各選擇最合適的策略並說明理由：（a）離線編程的點焊軌跡；（b）操作員用搖桿遙控的遠端作業；（c）新產線的機器人選型。
3. 協作機器人多採七軸設計。除了本章提到的繞開奇異點，冗餘還帶來哪些好處？代價又是什麼？請結合第 4.9.3 節討論。
4. 嚴格分層逆運動學保證了優先權，但數值品質可能較差。請設計一個「混合式」方案：哪些任務用嚴格分層、哪些用加權合併？判準是什麼？
5. 本章證明了萬向節鎖、手臂伸直、逆解合併、可操作度歸零是同一件事。請再找出本書前面章節中至少一個現象，說明它是否也能用雅可比的秩來解釋。

章末摘要

- 奇異點的定義是雅可比矩陣失去滿秩的位形；物理意義是某兩個以上的關節產生重複的運動效果，等於少了一個自由度。
- 三種判準： $\det(J) = 0$ （僅方陣）、可操作度 $w = 0$ 、最小奇異值 $\sigma_{\min} = 0$ （最推薦）。

- 奇異點的三個症狀：某方向運動能力喪失、關節速度暴增、多組逆解合併。其中第二個最危險，因為數學上仍有解。
- 邊界奇異點發生於工作空間邊界（手臂伸直或折疊），容易避開；內部奇異點位置分散且不直觀，更為危險。
- 六軸手臂的三類典型奇異：腕部（第 5 軸 $\approx 0^\circ$ ，最常見）、肩部（腕心落在第 1 軸線上）、肘部（手臂完全伸直）。
- 奇異位形只與構型有關、與機構尺寸無關，同構型的大小手臂奇異點完全相同。
- SVD 給出完整的幾何解讀： u_k 是失去的任務方向、 v_k 是徒勞的關節運動方向、 σ_k 是該方向的運動增益。
- 條件數 $\kappa = \sigma_{\max}/\sigma_{\min}$ 是無量綱指標； $\kappa < 5$ 優良、 $5 \sim 20$ 可接受、 $20 \sim 100$ 警戒、 >100 危險。
- 五種處理策略：軌跡繞道、降速、DLS 阻尼、改變構型、增加自由度；實務常採多層複合防護。
- 任何串聯旋轉關節手臂必定存在奇異點，這是數學必然，工程上只能避開而非消除。
- 冗餘度 $r = n - m$ ；零空間 $N(J)$ 是所有不產生末端運動的關節速度所構成的集合，沿其運動稱為自運動。
- 零空間有兩種來源：冗餘造成的（可用資源，不損失任務方向）與奇異造成的（伴隨任務方向損失）。
- 通解結構 $\dot{q} = J^+ \dot{x} + (I - J^+ J) \dot{z}$ ，第二項提供不干擾主任務的通道，可用於次要目標。
- 四種常見次要目標：遠離關節極限、最大化可操作度、避開障礙物、維持自然姿勢。
- 多任務可用加權合併（簡單但會互相妥協）或分層逆運動學（嚴格優先權但數值品質較差），實務常採折衷。
- 第二篇的四塊拼圖：正向（手在哪裡）、逆向（手該怎麼擺）、雅可比（手能怎麼動）、奇異與冗餘（手何時動不了、多餘自由度怎麼用）。

觀念題與計算題

觀念題

1. 以雅可比矩陣的秩定義奇異點，並說明「兩個關節做同一件事」的物理意義。

2. 比較三種奇異點判準的適用範圍與優缺點，並說明為何推薦使用最小奇異值。
3. 說明奇異點的三個症狀，並解釋為何「關節速度暴增」比「運動能力喪失」更危險。
4. 區分邊界奇異點與內部奇異點，並說明後者為何更難處理。
5. 說明六軸手臂三類典型奇異點的發生條件與典型症狀。
6. 為何說「奇異位形與機構尺寸無關」？這個結論對模擬驗證有什麼意義？
7. 說明 SVD 中 U 、 Σ 、 V 三個矩陣在奇異點分析中各代表什麼。
8. 定義條件數並說明其實務判讀標準；為何說它是「無量綱的相對指標」？
9. 列舉五種處理奇異點的策略，並比較各自在路徑精度與節拍上的影響。
10. 為何說奇異點無法被消除？請以拓撲學的必然性與第 8 章的類比說明。
11. 定義冗餘度與零空間，並說明自運動的物理意義。
12. 區分「冗餘造成的零空間」與「奇異造成的零空間」，並說明判斷方法。
13. 寫出冗餘機器人速度逆解的通解結構，並說明零空間投影算子的兩個性質。
14. 列舉四種常見的次要目標及其目標函數設計。
15. 比較加權合併與分層逆運動學的優缺點，並說明實務上的折衷作法。

計算題

1. 平面 2R 手臂 $L_1 = 400 \text{ mm}$ 、 $L_2 = 250 \text{ mm}$ 。(a) 求所有奇異位形。(b) 在 $\theta_1 = 45^\circ$ 、 $\theta_2 = 10^\circ$ 時求雅可比與行列式。(c) 求可操作度，並與最大可能值比較。(d) 若最小奇異值為 17.4 mm/rad ，求沿最難方向以 80 mm/s 運動所需的關節速度 (度/秒)，並判斷一般伺服馬達 (上限 $300^\circ/\text{s}$) 是否足夠。
2. 同上手臂。(a) 若要求條件數不超過 20，試由 $\det(J) = L_1 L_2 \sin \theta_2$ 的關係，估算 θ_2 的允許範圍 (提示：可先用程式掃描)。(b) 由此結果，建議末端的安全作業半徑範圍。(c) 若把 L_2 改為 400 mm (兩連桿等長)，內邊界奇異會發生什麼變化？
3. 某平面 3R 手臂 $L_1 = L_2 = L_3 = 150 \text{ mm}$ ，僅控制末端位置 (2 維任務)。目前姿態 $\theta = (20^\circ, 50^\circ, -40^\circ)$ 。(a) 求冗餘度與零空間維度。(b) 求 2×3 的雅可比矩陣。(c) 驗證雅可比是否滿秩。(d) 若次要目標為「使第三關節角度趨近於 0° 」，寫出對應的目標函數與梯度。
4. 某六軸手臂在某姿態下的六個奇異值為 $\sigma = (520, 380, 295, 88, 41, 6.2)$ (單位已正規化)。

(a) 求可操作度與條件數。(b) 依表 12-3 判斷其狀態。(c) 若末端各方向的速度需求皆為 100 單位，求所需關節速度的最大值。(d) 若採用 DLS 且 $\lambda = 15$ ，估算最小奇異值方向的有效增益變化 (提示：DLS 的等效增益為 $\sigma/(\sigma^2 + \lambda^2)$)。

動手做與專題延伸

本章實作：奇異點分析工具箱

延續第 9~11 章的程式，完成一套完整的運動學分析工具。這將是第二篇的總成果，也是後續章節的基礎。

- **階段一**：整合前三章的模組（正向運動學、逆運動學、雅可比），並加入本章的奇異點分析（ $\sigma_{\min}$ 、條件數、可操作度）。
- **階段二**：實作「姿態健康度報告」函式：輸入關節角度，輸出完整診斷——可操作度、條件數、最小奇異值、失去的運動方向、與最近奇異點的估計距離、以及建議措施。
- **階段三**：實作軌跡的預先檢查功能：輸入一串目標點，逐點求逆解並檢查條件數，回報所有超過警戒門檻的路段。
- **階段四**：對冗餘機器人實作零空間投影與次要目標最佳化，支援至少兩種次要目標。

專題延伸

- **可操作度地圖**：對平面手臂繪製工作空間的可操作度熱力圖，並以等高線標出條件數 20 與 100 的邊界。加入關節極限的限制，觀察可用區域如何縮小。這張圖可直接作為工作站佈局的參考。
- **奇異點穿越的動態比較**：規劃一條刻意穿越奇異點的直線軌跡，分別以偽逆、DLS（三種不同 λ ）與降速策略執行（模擬中），繪出關節速度隨時間的曲線與末端路徑誤差。整理成圖表，量化「路徑精度」與「關節速度」之間的取舍關係。
- **進階：七軸手臂的自運動視覺化**。為七軸手臂（可用公開的 URDF，如 Franka Emika 或 KUKA iiwa）實作零空間自運動：固定末端位姿，沿零空間連續運動並繪製手肘位置的軌跡。你會看到手肘畫出一個完整的圓弧——這就是「自運動流形」。進一步，在此流形上分別找出可操作度最大、最遠離關節極限、以及最不易碰撞的三個姿態，並討論它們是否重合。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 5 章：奇異點與雅可比) 。
- Nakamura, Y., Advanced Robotics: Redundancy and Optimization, Addison-Wesley (冗餘機器人與零空間的經典專著) 。
- Yoshikawa, T., "Manipulability of Robotic Mechanisms," IJRR, 1985 (可操作度與奇異點的量化指標) 。
- Chiaverini, S., "Singularity-Robust Task-Priority Redundancy Resolution," IEEE Trans. Robotics and Automation, 1997 (分層逆運動學與奇異點強健性) 。
- Siciliano, B. and Slotine, J.-J., "A General Framework for Managing Multiple Tasks in Highly Redundant Robotic Systems," ICAR 1991 (多任務優先權框架的奠基論文) 。
- Maciejewski, A. A. and Klein, C. A., "Obstacle Avoidance for Kinematically Redundant Manipulators," IJRR, 1985 (零空間避障的經典應用) 。
- Toussaint, M., Introduction to Robotics 課程講義, TU Berlin (多任務、分層逆運動學與正則化的實務討論) 。
- Lynch, K. M. and Park, F. C., Modern Robotics, Cambridge University Press (第 5 章：奇異點的旋量理論觀點) 。
- Escande, A., Mansard, N. and Wieber, P.-B., "Hierarchical Quadratic Programming," IJRR, 2014 (現代的分層最佳化求解法) 。
- 各機器人廠商的技術手冊中「奇異點」與「特異姿態」章節，含實機的奇異區域圖與規避建議。

機器人學導論

機構、感知、控制與具身人工智慧

第 三 篇

動力學、軌跡與控制

第 13 章 操作臂靜力學與拉格朗日動力學

第 14 章 Newton-Euler 動力學與模擬引擎

第 15 章 軌跡生成與運動曲線

第 16 章 線性回授與 PID 控制

第 17 章 操作臂非線性控制

第 18 章 最佳控制與 Riccati 方程

第 19 章 可控性與穩定性

第 20 章 力控制、阻抗控制與柔順操作

第 21 章 抓取與操作

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

2026 年 9 月

第三篇 篇章導讀

第二篇教你機器人**能不能到那裡**，這一篇要教你**該用多大的力氣到那裡**。

第二篇的所有分析都是運動學的——只談位置、速度與幾何，隱含一個假設：關節可以瞬間達到任何我們指定的速度。這在低速時尚可接受，但真實的手臂有質量、有慣性、受重力作用。它在加速時會抵抗你、在轉彎時會產生你沒預料到的力、在停止時因為慣性而繼續往前衝。第三篇的九章，就是要處理這些「力」的問題。

第 13、14 章 (動力學) 建立「力與運動」的數學模型。前者用拉格朗日法從能量出發，讓我們**看懂**方程的結構——慣性、科氏力、離心力與重力四類項各自從何而來；後者改用 Newton-Euler 遞迴，把複雜度由 $O(n^3)$ 降到 $O(n)$ ，讓我們**算得夠快**，能在毫秒級的控制迴路中即時執行。

第 15 章 (軌跡生成) 決定「該走哪條路」。同一條路徑，不同的時間安排會帶來截然不同的結果——慢慢加速反而更快，因為粗暴的運動會激發振動，等它平息的時間比省下來的還多。這一章也首次把動力學模型用於檢驗軌跡的可行性。

第 16、17 章 (控制) 讓機器人真的走上那條路。第 16 章從最基本的 PID 出發，建立回授的基礎與極限；第 17 章則把動力學模型放進控制器，用計算力矩控制突破 PID 的限制。核心哲學是一句話：**用模型處理已知的部分，用回授處理未知的部分**。

第 18、19 章 (最佳性與穩定性) 回答兩個更深的問題。增益該怎麼選才是最好的？——第 18 章用代價函數與 Riccati 方程給出可計算的答案。我們憑什麼相信控制器一定不會失控？——第 19 章用 Lyapunov 方法給出嚴格的證明。

第 20、21 章 (接觸與抓取) 讓機器人走出自由空間。一旦與環境接觸，前面的邏輯就完全反轉：高增益的精確位置控制反而成為最危險的設定。這兩章的核心洞見是——**與其消除不確定性，不如設計一個對不確定性不敏感的系統**。

這一篇要回答的七個問題

- 手臂有重量，加速時需要多大的力矩？關節之間又如何互相干擾？
- 六軸手臂的動力學方程展開有上萬項，控制器如何在一毫秒內算完？
- 同樣的路徑，為什麼「慢慢加速」反而比「全速衝過去」更快？

- 為什麼低速調校完美的 PID，一提速就失準？調高增益為何治標不治本？
- 控制器的增益該取多少才是「最好」的？這個問題有沒有標準答案？
- 我們憑什麼相信一個控制器一定不會失控？
- 機器人碰到東西時，為什麼精確的位置控制反而會壓碎雞蛋？

三條貫穿全篇的線索

線索一：模型的價值不在完美，而在減負。 第 13 章的動力學模型有 10~20% 的誤差，第 17 章仍能靠它把回授的負擔降低五倍。追求完美的模型往往不划算，因為剩下的部分交給回授即可。

線索二：理論的各部分互相支撐。 第 13 章那個當時看似無用的「 $\dot{M} - 2C$ 為斜對稱」性質，兩百頁後成為第 19 章穩定性證明的樞紐；第 11 章的雅可比對偶，則先後成為第 17 章操作空間控制的核心、第 20 章力映射的基礎、第 21 章抓取矩陣的原型。

線索三：機構與控制必須協同設計。 第 20 章的插銷入孔靠一個成本幾乎為零的倒角，就把容許誤差放寬 41 倍；第 21 章的欠驅動手用機構取代了大量的感測與計算。反覆出現的教訓是：在把問題丟給控制器之前，先問能不能用機構把問題變簡單。

讀完這一篇之後

你將能夠：建立機器人的完整動力學模型並在微秒級計算它、規劃兼顧速度與平順度且通過力矩檢驗的軌跡、設計並實作從 PID 到計算力矩控制的各級控制器、用最佳控制理論計算增益並用 Lyapunov 方法證明其穩定性、以及讓機器人安全地與環境接觸、溫柔地抓取物體。

但第三篇的所有技術，都建立在一個前提上：**機器人知道自己在哪裡、知道目標在哪裡**。固定基座的手臂靠編碼器就知道自己的姿態，工件的位置則由治具保證。一旦機器人要移動、要面對未知的環境，這個前提就不成立了。

第四篇「感知、定位與自主移動」將處理這些問題。屆時你會發現，第三篇的工具並未被拋棄——控制理論仍是無人機姿態穩定與四足步態的基礎。只是問題的重心，從「精確地執行」轉向了「正確地理解」。

本篇章節配置

章次	章名	核心內容	叫獸開講
第 13 章	操作臂靜力學與拉格朗日動力學	慣性張量、拉格朗日方程、標準形式、慣性矩陣、科氏力與重力項	「知道手臂要往哪動，為什麼還不知道馬達要出多少力？」——手臂是一桶會晃的水
第 14 章	Newton-Euler 動力學與模擬引擎	前向與後向遞迴、 $O(n)$ 複雜度、正向動力學、數值積分、接觸模型	「每毫秒都要算一次力矩，怎麼算才夠快？」——一萬四千項的正確答案
第 15 章	軌跡生成與運動曲線	多項式、梯形與 S 形曲線、急跳度、混成、時間縮放、動力學檢驗	「到達同一位置，為什麼有些動作像機器？」——慢慢加速，反而更快
第 16 章	線性回授與 PID 控制	開閉迴路、自然頻率與阻尼比、積分飽和、數位實作、調參方法	「機器手臂到了目標位置，為什麼還在左右發抖？」——手上立竹竿的三個祕訣
第 17 章	操作臂非線性控制	計算力矩控制、回授線性化、重力補償、操作空間控制、滑模與自適應	「搬起重物後，原本調好的控制器為什麼失準？」——讓末端像一顆點質量
第 18 章	最佳控制與 Riccati 方程	Bellman 原理、價值函數、HJB 方程、LQR、權重選擇、倒立擺	「倒立搖桿如何自己找出最省力的平衡之道？」——把「什麼叫好」寫成公式
第 19 章	可控性與穩定性	可控性矩陣、三種穩定性、Lyapunov 直接法、LaSalle 原理、伏筆回收	「控制器算出完美指令，系統卻仍翻車？」——彈珠一定會滾到碗底
第 20 章	力控制、阻抗控制與柔順操作	環境剛度、自然與人為約束、混合控制、阻抗與導納、接觸穩定性、RCC	「拿雞蛋不能只知道位置，還要知道用了多少力。」——控制兩者之間的關係
第 21 章	抓取與操作	摩擦錐、抓取矩陣、力封閉、形封閉與關入籠、品質指標、欠驅動手	「戴著厚手套、閉著眼睛，為什麼還是抓得起來？」——柔順勝過精準

第三篇 動力學、軌跡與控制

第 13 章

操作臂靜力學與拉格朗日動力學

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為一位服務生端著滿滿一盤飲料快步轉彎的照片，杯中液面明顯傾斜，並以箭頭標示慣性、離心力與科氏力三個方向；右半為六軸手臂高速轉彎的多重曝光影像，末端軌跡以紅色虛線標出偏離理想路徑的部分。中央以四色方塊並列標註動力學方程的四項： $M(q)\ddot{q}$ （藍）、 $C(q,\dot{q})\dot{q}$ （橙）、 $G(q)$ （紅）、 $F(q)$ （灰）。標題文字：手臂不是沒有重量的幾何模型——它是一桶會晃的水。

叫獸開講 「知道手臂要往哪裡動，為什麼還不知道馬達要出多少力？」

實驗室的六軸手臂已經調校了兩週，末端的重複精度達到 ± 0.05 公釐，在低速下畫圓、走直線都完美無缺。今天要驗收的是最後一項：把速度調到額定值的八成，執行同一段軌跡。

小妤按下啟動。手臂加速——然後在轉彎處明顯地「甩」了一下，末端偏離軌跡將近三公釐，接著在停止時抖了兩三下才穩住。示教器上跳出黃色警告：追隨誤差過大。

「可是低速時明明完美啊，」小妤翻著控制器參數，「是不是 PID 的增益要再調高一點？」

叫獸沒有回答，反而問了一個看起來無關的問題：「你有沒有提過裝滿水的水桶走路？」

小妤點頭。「走慢的時候，水面很平靜；一旦走快、或是突然轉彎，水就會晃出來——而且你必須用力抵抗那個晃動，否則會被水的慣性拉得踉蹌。」叫獸指著手臂，「你的機器手臂就是那桶水。前面十二章，我們把它當成一個沒有重量的幾何模型——關節說轉幾度就轉幾度，說停就停。但真實的手臂有質量、有慣性。它在加速時會抵抗你、在轉彎時會產生你沒預料到的力、在停止時因為慣性而繼續往前衝。」

他在白板上寫下三個詞：慣性、科氏力、重力。「你的 PID 控制器只知道『現在差了多少度』，它不知道『這個姿態下第二軸需要多大的力矩才能加速』，更不知道『第三軸轉動時會給第二軸一個額外的干擾力』。所以它只能事後補救——誤差出現了才修正。速度越快，補救得越不及時。」

小妤問：「那該怎麼辦？把增益調到很高？」

「那會讓它變得很硬，撞到東西時很危險，而且會激發機構的共振。」叫獸搖頭，「正確的作法是讓控制器事先知道：我要在這個姿態下、以這個加速度運動，需要多大的力矩？把這個力矩直接送給馬達，PID 就只需要修正那一點點模型誤差。這叫做前饋，而要算出那個力矩，就需要一個東西——動力學模型。」

他把筆放下：「第二篇我們問的是『手臂能不能到那裡』。從今天開始，第三篇要問的是：『該用多大的力氣到那裡』。」

本章為什麼重要

第二篇的運動學分析有一個隱含假設：關節可以瞬間達到任何指定的速度。這在低速時尚可

接受，但在高速運動、重負載或需要精確力控制時就會失效——因為機器人有質量、有慣性、受重力作用。

動力學回答的是「力與運動的關係」：要產生某個加速度，各關節需要輸出多少力矩？反之，給定力矩，機器人會怎麼運動？這個模型是三件事的基礎：**高精度高速控制**（第 17 章的計算力矩控制與前饋）、**力控制與碰撞偵測**（第 20 章，必須先扣除慣性與重力的貢獻才能辨識外力）、以及**模擬**（第 30 章的物理引擎、第 31 章的強化學習訓練環境）。

本章採用拉格朗日方法——它從能量出發，不需畫受力圖、不需處理內力，只要寫出動能與位能，微分幾次就得到運動方程。這個方法在推導與理解上極為優雅，適合建立物理直覺；第 14 章則會介紹計算效率更高的 Newton-Euler 遞迴法，兩者互補。

學習目標

- 能定義質心、轉動慣量與慣性張量，並說明其物理意義。
- 能運用平行軸定理計算物體對任意軸的轉動慣量。
- 能寫出機器人連桿的動能與位能表達式。
- 能敘述拉格朗日方程並說明其相對於牛頓法的優勢。
- 能推導單擺與平面 2R 手臂的完整動力學方程。
- 能辨識並解釋動力學方程中的慣性項、科氏力項、離心力項與重力項。
- 能說明慣性矩陣的物理意義及其正定、對稱、姿態相依三個性質。
- 能解釋耦合效應：為何一個關節的運動會影響其他關節。
- 能說明摩擦力模型及其對控制的影響。
- 能寫出機器人動力學的標準形式並說明各項的角色。
- 能以 Python 實作動力學方程並驗證能量守恆。

先備知識

第 9 章的正向運動學（用於計算各連桿質心的位置與速度）、第 11 章的雅可比矩陣（連桿質心的速度由雅可比計算）。物理方面需要高中的功、能量、動量與轉動慣量概念；數學方面需要偏導數與鏈鎖律。

本章的推導量是全書最大的一章之一。建議讀法：先確實掌握 13.5 節的單擺範例（每一步都親手算過），再進入 13.6 節的平面 2R 手臂。若中途卡住，多半是能量表達式沒寫對——回頭檢查速度的計算，而非懷疑微分。

核心名詞中英對照

表 13-1 本章核心名詞

中文	英文	說明
質心	Center of Mass, CoM	物體質量分布的平均位置
轉動慣量	Moment of Inertia	物體抵抗角加速度的量度
慣性張量	Inertia Tensor	描述剛體三維轉動慣性的 3×3 矩陣
平行軸定理	Parallel Axis Theorem	由質心軸的慣量求任意平行軸慣量的定理
廣義座標	Generalized Coordinates	能完整描述系統位形的獨立變數
拉格朗日量	Lagrangian	動能減位能， $L = T - U$
拉格朗日方程	Euler-Lagrange Equation	由能量推導運動方程的方程式
慣性矩陣	Inertia Matrix / Mass Matrix	$M(q)$ ，連結關節加速度與力矩
科氏力	Coriolis Force	兩軸同時轉動時產生的耦合力
離心力	Centrifugal Force	單軸轉動時因圓周運動產生的力
重力向量	Gravity Vector	$G(q)$ ，平衡重力所需的關節力矩
耦合	Coupling	一個關節的運動影響其他關節的現象
黏滯摩擦	Viscous Friction	與速度成正比的摩擦力
庫倫摩擦	Coulomb Friction	與速度方向相反、大小固定的摩擦力
逆動力學	Inverse Dynamics	由運動求所需力矩
正向動力學	Forward Dynamics	由力矩求所產生的運動

13.1 質量、質心與轉動慣量

13.1.1 三個基本的慣性參數

要描述一根連桿的動力學特性，需要三組參數。

質量 m ：一個純量，描述物體抵抗線加速度的能力。由牛頓第二定律 $F = ma$ ，質量越大，同樣的力產生的加速度越小。

質心位置 p_c ：一個三維向量，描述質量分布的平均位置。它的重要性在於：**重力可視為全部作用在質心上的單一力**，而剛體的平移運動也可以用質心的運動來描述。對均勻的細長連桿，質心在中點；對實際的機器人連桿（一端有馬達、另一端有減速機），質心往往明顯偏離幾何中心。

慣性張量 I ：一個 3×3 的對稱矩陣，描述物體抵抗**角**加速度的能力。它是轉動版本的「質量」，但比質量複雜得多——因為繞不同軸旋轉的難易程度不同。

13.1.2 為什麼轉動慣量是矩陣

線運動的慣性只需一個數（質量），因為無論往哪個方向推，抵抗都相同。但轉動不然：一根長桿繞其長軸旋轉很容易，繞垂直於長軸的方向旋轉則困難得多。因此需要一個矩陣來描述各方向的差異：

$$I = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix}$$

慣性張量。對角元素稱為**轉動慣量**（繞各軸的慣性），非對角元素稱為**慣性積**（描述軸之間的耦合）。矩陣恆為對稱，故只有六個獨立元素。

角動量與角速度的關係為 $L = I \cdot \omega$ 。注意這裡有一個關鍵差異：**角動量的方向未必與角速度相同**。只有當旋轉恰好繞著慣性主軸時，兩者才平行。這解釋了一個常見現象——旋轉一個形狀不規則的物體時，它會自己「翻滾」，因為角動量守恆而角速度方向不斷改變。

實務上，若能選擇適當的座標軸（慣性主軸），慣性積會全部為零，矩陣化為對角形式。CAD 軟體通常能直接輸出連桿的質量、質心與慣性張量——這是建立動力學模型最實用的資料來源。

13.1.3 常見形狀的轉動慣量

實務估算中最常用的三個公式為：

$$\text{細長桿繞質心（垂直於桿）：} I = (1/12) m L^2$$

$$\text{細長桿繞一端（垂直於桿）：} I = (1/3) m L^2$$

$$\text{質點繞距離 } r \text{ 的軸：} I = m r^2$$

第 5.10.3 節在馬達選型時已用過前兩式，此處給出完整脈絡。

13.1.4 平行軸定理

已知物體繞其質心的轉動慣量 I_c ，要求繞另一條平行軸（距離 d ）的轉動慣量：

$$I = I_c + m d^2$$

平行軸定理。這是動力學計算中使用頻率最高的定理之一。

驗證一致性：細長桿繞質心為 $(1/12)mL^2$ ，質心到端點距離 $d = L/2$ ，故繞端點為 $(1/12)mL^2 + m(L/2)^2 = (1/12 + 1/4)mL^2 = (1/3)mL^2$ ✓ 與上述公式相符。

平行軸定理在機器人動力學中的角色極為關鍵：連桿的慣量參數通常以質心座標系給出（CAD 輸出），但動力學方程需要的是繞關節軸的慣量——兩者之間就靠這條定理連結。而 $m d^2$ 這一項也解釋了第 5.10.2 節的設計原則：把重物往轉軸移近，慣量以距離的平方下降。

13.2 動能與位能

13.2.1 剛體的動能

一個運動中的剛體，其動能包含兩部分：質心的平移動能，加上繞質心的轉動動能。

$$T = \frac{1}{2} m \mathbf{v}_c^T \mathbf{v}_c + \frac{1}{2} \boldsymbol{\omega}^T I \boldsymbol{\omega}$$

剛體動能。第一項為平移（ $\mathbf{v}_c$ 為質心速度）、第二項為轉動（ $\boldsymbol{\omega}$ 為角速度、 I 為繞質心的慣性張量）。

這個「平移 + 轉動」的分解稱為 König 定理，它的價值在於把複雜的剛體運動拆成兩個獨立的部分——這正是我們能用有限個參數描述動力學的原因。

對整支機器手臂，總動能是各連桿動能的總和：

$$T = \sum_i \left[\frac{1}{2} m_i \mathbf{v}_{c_i}^T \mathbf{v}_{c_i} + \frac{1}{2} \boldsymbol{\omega}_i^T I_i \boldsymbol{\omega}_i \right]$$

n 根連桿的動能總和。

13.2.2 用雅可比表達動能

上式中的 $\mathbf{v}_{c_i}$ 與 $\boldsymbol{\omega}_i$ 都是關節速度 $\dot{\mathbf{q}}$ 的函數。運用第 11 章的雅可比概念，可以把每一根連桿質心的速度寫成：

$$\mathbf{v}_{c_i} = J_{v_i}(\mathbf{q}) \cdot \dot{\mathbf{q}} \quad \boldsymbol{\omega}_i = J_{\omega_i}(\mathbf{q}) \cdot \dot{\mathbf{q}}$$

J_{v_i} 與 J_{ω_i} 是「以連桿 i 的質心為末端」的雅可比矩陣，可用第 11.4 節的幾何法直接構造。

代入動能表達式並整理：

$$T = \frac{1}{2} \mathbf{q}^T \left[\sum_i (m_i \mathbf{J}_i \mathbf{v}_i^T \mathbf{J}_i \mathbf{v}_i + \mathbf{J}_i \boldsymbol{\omega}_i^T \mathbf{I}_i \mathbf{J}_i \boldsymbol{\omega}_i) \right] \mathbf{q}' = \frac{1}{2} \mathbf{q}^T \mathbf{M}(\mathbf{q}) \mathbf{q}'$$

定義中括號內的矩陣為慣性矩陣 $\mathbf{M}(\mathbf{q})$ 。

這是本章第一個重要成果。它顯示：機器人的動能永遠可以寫成 $\frac{1}{2} \mathbf{q}^T \mathbf{M}(\mathbf{q}) \mathbf{q}'$ 的二次型，其中 $\mathbf{M}(\mathbf{q})$ 是一個只與姿態有關（與速度無關）的 $n \times n$ 矩陣。這個矩陣將是整個動力學方程的核心，13.7 節會詳細討論它的性質。

13.2.3 位能

機器人的位能主要來自重力（若有彈簧則另計）：

$$U = \sum_i m_i g h_{ci}(\mathbf{q}) \quad (h_{ci} \text{ 為第 } i \text{ 根連桿質心的高度})$$

位能只與姿態 $\mathbf{q}$ 有關，與速度無關。這個「與速度無關」的性質，稍後會簡化拉格朗日方程的推導。

更一般的寫法是用向量形式： $U = -\sum_i m_i \mathbf{g}^T \mathbf{p}_{ci}(\mathbf{q})$ ，其中 $\mathbf{g}$ 是重力加速度向量（通常為 $[0, 0, -9.8]^T$ ）。這個寫法不需假設重力沿特定方向，適用於傾斜安裝或移動平台上的機器人。

13.3 拉格朗日方程

13.3.1 方程式與意義

拉格朗日力學的核心是一個純量函數——拉格朗日量：

$$L(\mathbf{q}, \mathbf{q}') = T(\mathbf{q}, \mathbf{q}') - U(\mathbf{q})$$

動能減位能。注意是「減」而非「加」，這一點常令初學者困惑，但它是變分原理推導的必然結果。

系統的運動方程則由拉格朗日方程（又稱 Euler-Lagrange 方程）給出：

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \mathbf{q}_i'} \right) - \frac{\partial L}{\partial \mathbf{q}_i} = \boldsymbol{\tau}_i \quad (i = 1, 2, \dots, n)$$

對每一個廣義座標各寫一式，共 n 個方程。 $\boldsymbol{\tau}_i$ 為作用在第 i 個廣義座標上的廣義力（旋轉關節為力矩、移動關節為力）。

13.3.2 為什麼這個方法更好用

與牛頓法（逐一分析每個剛體的受力）相比，拉格朗日法有四個決定性的優勢。

優勢一：不需畫受力圖。牛頓法必須逐一找出每根連桿上的所有作用力，包括關節處的內力（連桿之間互相推擠的力）。對六軸手臂而言，這是數十個力向量的分析。

優勢二：內力自動消去。關節處的約束力（constraint force）不做功，因此完全不會出現

在能量表達式中。這是最大的簡化——牛頓法必須算出這些力再消去，拉格朗日法直接跳過。

優勢三：座標系自由。可以使用任何一組能描述系統位形的獨立變數（廣義座標），不必侷限於直角座標。對機器人而言，關節角度是最自然的廣義座標。

優勢四：純量運算。動能與位能都是純量，比向量運算不易出錯。

代價是：需要先寫出能量表達式（要算出每根連桿質心的速度，這需要雅可比），且微分過程對多自由度系統會相當繁瑣。這正是第 14 章引入 Newton-Euler 遞迴法的原因——它在計算效率上遠勝拉格朗日法。

◎ 兩種方法的分工

拉格朗日法適合**推導與理解**：它能給出結構清晰的解析表達式，讓你看見慣性矩陣、科氏項與重力項各自的來源。Newton-Euler 法適合**計算與實作**：它以遞迴方式運算，複雜度為 $O(n)$ ，是控制器內部即時計算的標準作法。兩者得到的方程完全相同，只是路徑不同。本書第 13 章用前者建立理解，第 14 章用後者處理實作。

13.3.3 推導的四個步驟

運用拉格朗日法推導機器人動力學，可依下列步驟進行：

1. **選定廣義座標**：對串聯機器手臂，直接取關節變數 $q = [q_1, \dots, q_n]^T$ 。
2. **寫出動能 T**：計算每根連桿質心的速度（線速度與角速度），代入 13.2.1 節的公式並加總。
3. **寫出位能 U**：計算每根連桿質心的高度，乘上質量與重力加速度後加總。
4. **代入拉格朗日方程並微分**：對每個 q_i 計算 $\partial L / \partial q_i$ 、對時間微分、再減去 $\partial L / \partial \dot{q}_i$ ，即得該關節的運動方程。

13.4 最簡單的例子：單擺

13.4.1 建立模型

在處理機器手臂之前，先用最簡單的例子熟悉整個流程。考慮一根長度 L 、質量 m 的均勻細桿，一端以旋轉關節固定，可在垂直平面內自由擺動。關節處施加力矩 τ 。

步驟一：廣義座標。取 θ 為桿與垂直向下方向的夾角，這是唯一的自由度。

步驟二：動能。桿繞固定端旋轉，其繞該端的轉動慣量為 $I = (1/3)mL^2$ (13.1.3 節)：

$$T = \frac{1}{2} I \dot{\theta}^2 = \frac{1}{2} \cdot (1/3) m L^2 \cdot \dot{\theta}^2 = (1/6) m L^2 \dot{\theta}^2$$

純轉動的動能。此處直接用繞固定端的慣量，省去分解為平移加轉動的步驟。

步驟三：位能。質心在距關節 $L/2$ 處，其高度 (以關節為基準) 為 $-(L/2)\cos \theta$ ：

$$U = -m g (L/2) \cos \theta$$

$\theta = 0$ (垂直向下) 時位能最低，符合物理直覺。

13.4.2 推導運動方程

步驟四：代入拉格朗日方程。先寫出拉格朗日量：

$$L = T - U = (1/6) m L^2 \dot{\theta}^2 + m g (L/2) \cos \theta$$

拉格朗日量。

計算對廣義速度的偏導數：

$$\partial L / \partial \dot{\theta} = (1/3) m L^2 \dot{\theta}$$

動能對速度的偏導數，物理上就是角動量。

對時間微分：

$$d/dt (\partial L / \partial \dot{\theta}) = (1/3) m L^2 \ddot{\theta}$$

角動量的變化率。

計算對廣義座標的偏導數：

$$\partial L / \partial \theta = -m g (L/2) \sin \theta$$

位能的負梯度，物理上就是重力產生的力矩。

代入拉格朗日方程 $d/dt(\partial L / \partial \dot{\theta}) - \partial L / \partial \theta = \tau$ ：

$$(1/3) m L^2 \ddot{\theta} + m g (L/2) \sin \theta = \tau$$

單擺的完整運動方程。

13.4.3 解讀結果

這條方程可以逐項解讀，而這三項恰好對應到機器人動力學的三大類項目：

第一項 $(1/3)mL^2\ddot{\theta}$ ：慣性項。它是「轉動慣量 $\times$ 角加速度」，代表要產生角加速度所需的

力矩。注意係數 $(1/3)mL^2$ 是常數——這是單擺的特殊性，一般機器手臂的慣性係數會隨姿態變化。

第二項 $mg(L/2)\sin \theta$ ：重力項。它隨 θ 變化：垂直向下 ($\theta=0$) 時為零、水平 ($\theta=90^\circ$) 時最大。這正是第 5.6.1 節「以水平姿態為最壞情況」計算馬達扭力的理論依據。

右式 τ ：外加的關節力矩。

由此可以立即回答一個實務問題：**要讓單擺靜止在水平位置，需要多大的力矩？** 令 $\dot{\theta} = 0$ 、 $\theta = 90^\circ$ ，得 $\tau = mg(L/2)$ ——這就是重力補償（第 17 章）所需的力矩。

此外，若令 $\tau = 0$ 且假設小角度 ($\sin \theta \approx \theta$)，方程化為 $\ddot{\theta} + (3g/2L)\theta = 0$ ，這是簡諧運動方程，其自然頻率為 $\omega_n = \sqrt{3g/2L}$ 。這個結果在第 16 章討論控制器頻寬時會再出現——**控制器的頻寬必須高於系統的自然頻率**，否則無法有效抑制振動。

13.5 平面 2R 手臂的完整推導

13.5.1 模型與符號

現在處理真正的機器手臂。考慮平面 2R 手臂：連桿長度 L_1 、 L_2 ，質量 m_1 、 m_2 ，質心分別位於距各自關節 l_1 、 l_2 處（對均勻桿則 $l_i = L_i/2$ ），繞各自質心的轉動慣量為 I_1 、 I_2 。

這個例子的價值在於：它是最簡單但仍具備完整結構的機器人動力學——包含慣性、科氏力、離心力與重力四類項，且各項之間有耦合。理解它，就理解了六軸手臂動力學方程的結構。

13.5.2 第一步：各連桿質心的位置與速度

連桿 1 的質心位置與速度（由第 9 章的正向運動學）：

$$\begin{aligned} x_{c1} &= l_1 c_1, & y_{c1} &= l_1 s_1 \\ \dot{x}_{c1} &= -l_1 s_1 \dot{\theta}_1, & \dot{y}_{c1} &= l_1 c_1 \dot{\theta}_1 \end{aligned}$$

質心 1 的速度平方為 $v_{c1}^2 = l_1^2 \dot{\theta}_1^2$ （因 $s^2 + c^2 = 1$ ）。

連桿 2 的質心位置：

$$x_{c2} = L_1 c_1 + l_2 c_{12}, \quad y_{c2} = L_1 s_1 + l_2 s_{12}$$

其中 $c_{12} = \cos(\theta_1 + \theta_2)$ 。注意第一項用的是連桿 1 的全長 L_1 ，第二項用的是連桿 2 的質心距離 l_2 。這是初學者最常出錯的地方。

對時間微分並計算速度平方（過程中利用 $\cos(A-B)$ 的和差角公式化簡）：

$$v_c^2 = L_1^2 \dot{\theta}_1^2 + l_2^2 (\dot{\theta}_1 + \dot{\theta}_2)^2 + 2L_1 l_2 \dot{\theta}_1 (\dot{\theta}_1 + \dot{\theta}_2) c_2$$

質心 2 的速度平方。最後一項含 $\cos \theta_2$ ，這正是耦合的來源——它使動能無法拆解成兩個獨立的部分。

13.5.3 第二步：動能與位能

兩根連桿的動能總和（各含平移與轉動兩部分）：

$$T = \frac{1}{2} m_1 l_1^2 \dot{\theta}_1^2 + \frac{1}{2} I_1 \dot{\theta}_1^2 + \frac{1}{2} m_2 [L_1^2 \dot{\theta}_1^2 + l_2^2 (\dot{\theta}_1 + \dot{\theta}_2)^2 + 2L_1 l_2 \dot{\theta}_1 (\dot{\theta}_1 + \dot{\theta}_2) c_2] + \frac{1}{2} I_2 (\dot{\theta}_1 + \dot{\theta}_2)^2$$

總動能。注意連桿 2 的角速度是 $\theta_1 + \theta_2$ （絕對角速度），而非僅 θ_2 。

位能（設重力沿 $-Y$ 方向）：

$$U = m_1 g l_1 s_1 + m_2 g (L_1 s_1 + l_2 s_{12})$$

總位能。

13.5.4 第三步：代入拉格朗日方程

推導過程冗長，此處直接給出整理後的結果（讀者可對照本章程式範例的符號運算驗證）。

為簡化書寫，定義三個常數組合：

$$\begin{aligned} \alpha &= m_1 l_1^2 + I_1 + m_2 (L_1^2 + l_2^2) + I_2 \\ \beta &= m_2 L_1 l_2 \\ \delta &= m_2 l_2^2 + I_2 \end{aligned}$$

這三個組合是機構的固有參數，與姿態無關。

第一關節的方程：

$$\tau_1 = (\alpha + 2\beta c_2) \dot{\theta}_1 + (\delta + \beta c_2) \dot{\theta}_2 - \beta s_2 \dot{\theta}_2^2 - 2\beta s_2 \dot{\theta}_1 \dot{\theta}_2 + g_1(q)$$

含慣性、離心、科氏與重力四類項。

第二關節的方程：

$$\tau_2 = (\delta + \beta c_2) \dot{\theta}_1 + \delta \dot{\theta}_2 + \beta s_2 \dot{\theta}_1^2 + g_2(q)$$

第二關節的方程較簡單，但仍含來自第一關節的耦合項。

其中重力項為：

$$\begin{aligned} g_1(q) &= (m_1 l_1 + m_2 L_1) g c_1 + m_2 l_2 g c_{12} \\ g_2(q) &= m_2 l_2 g c_{12} \end{aligned}$$

重力產生的關節力矩，只與姿態有關、與速度無關。

13.5.5 整理成標準形式

把上述兩式寫成矩陣形式，就得到機器人動力學的標準形式：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$$

機器人動力學方程的標準形式。這是本章、乃至整個第三篇最重要的一條式子。

其中三個矩陣（向量）分別為：

$$M(q) = \begin{bmatrix} \alpha + 2\beta c_2 & \delta + \beta c_2 \\ \delta + \beta c_2 & \delta \end{bmatrix}$$

慣性矩陣，對稱且姿態相依。

$$C(q, \dot{q})\dot{q} = \begin{bmatrix} -\beta s_2 \dot{\theta}_2^2 - 2\beta s_2 \dot{\theta}_1 \dot{\theta}_2 \\ \beta s_2 \dot{\theta}_1^2 \end{bmatrix}$$

科氏力與離心力項。含速度的平方或乘積。

$$G(q) = \begin{bmatrix} (m_1 l_1 + m_2 L_1) g c_1 + m_2 l_2 g c_{12} \\ m_2 l_2 g c_{12} \end{bmatrix}$$

重力項。

圖 13-1 動力學方程各項的物理意義

建議配圖：中央為平面 2R 手臂圖示，四周以四個色塊分別標註四類項目及其物理來源。 $M(q)\ddot{q}$ （藍）：加速時抵抗的慣性，圖示為手臂加速的箭頭。離心力（綠）：單軸旋轉時連桿被「甩」向外的傾向，圖示為旋轉中的手臂與向外的虛線箭頭。科氏力（橙）：兩軸同時轉動時產生的耦合力，圖示為兩個關節同時旋轉並在末端產生側向力。 $G(q)$ （紅）：重力使手臂下垂的力矩，圖示為向下的重力箭頭與其力臂。下方標註標準形式方程式。

13.6 慣性矩陣 $M(q)$

13.6.1 四個關鍵性質

慣性矩陣是動力學方程中最重要的物件，它有四個必須理解的性質。

性質一：對稱。 $M = M^T$ 恆成立。這來自動能是二次型的事實（ $\frac{1}{2}\dot{q}^T M \dot{q}$ ），任何非對稱的部分在二次型中都會自動抵消。對稱性使 $n \times n$ 矩陣只有 $n(n+1)/2$ 個獨立元素，並可用於驗算——若你推導出的 M 不對稱，一定是算錯了。

性質二：正定。對任意非零的 $\dot{q}$ ，恆有 $\dot{q}^T M \dot{q} > 0$ 。這在物理上是必然的——動能不可能為零或負值（除非完全靜止）。正定性帶來一個極重要的推論： **M 必定可逆**。這保證了正向動力學（由力矩求加速度）永遠有唯一解：

$$\ddot{q} = M^{-1}(\tau - C\dot{q} - G)$$

正向動力學。因 M 正定必可逆，此式永遠成立——這是物理模擬（第 30 章）能夠進行的數學保證。

性質三：姿態相依。 M 是 q 的函數。以平面 2R 為例， $M_{11} = \alpha + 2\beta \cos \theta_2$ 隨 θ_2 變化：手

臂伸直 ($\theta_2 = 0$) 時 $M_{11} = \alpha + 2\beta$ 最大、手臂折疊 ($\theta_2 = 180^\circ$) 時最小。這正是開頭故事中「同樣的關節速度，在不同姿態下需要不同力矩」的數學根源，也是為何固定增益的 PID 在高速時表現不佳。

性質四：有界。 M 的特徵值有上下界，且與位置無關（對只有旋轉關節的機器人）。這個性質在第 19 章的穩定性證明中扮演關鍵角色。

13.6.2 對角元素與非對角元素

慣性矩陣的元素有清楚的物理意義。

對角元素 M_{ii} ：關節 i 單獨加速時所感受到的等效慣量。它包含了該關節下游所有連桿的貢獻——這解釋了為何靠近基座的關節（第一、二軸）需要遠大於手腕關節的馬達（第 5 章的選型考量）。

非對角元素 M_{ij} ($i \neq j$)：耦合項。它代表「關節 j 的加速度，會對關節 i 產生多少力矩」。以平面 2R 為例， $M_{12} = \delta + \beta \cos \theta_2$ 不為零，意味著：即使第一關節完全不動 ($\dot{\theta}_1 = 0$)，只要第二關節加速，第一關節仍需輸出力矩來維持不動。

這個耦合是機器人控制困難的核心來源。若把每個關節當成獨立系統各自用 PID 控制（第 16 章的作法），耦合項就成為每個關節眼中的「未知干擾」。速度越快、加速度越大，這個干擾就越顯著——這正是開頭故事中低速完美、高速失準的原因。

◎ 耦合的直觀體驗

把手臂平伸，然後只彎曲手肘（前臂快速上抬）。你會感覺到**肩膀也需要用力**——即使肩關節完全沒有轉動。這就是非對角慣性項 M_{12} 的作用。再試一次，但這次把前臂盡量靠近身體（縮短力臂），你會發現肩膀需要的力氣小得多——因為 $\beta = m_2 L_1 l_2$ 隨質心距離下降。這正是第 5 章「把重物往轉軸移近」設計原則的動力學解釋。

13.7 科氏力與離心力

13.7.1 兩種速度相關的力

$C(q, \dot{q})\dot{q}$ 這一項包含兩種物理上不同的效應，可由它們對速度的依賴形式區分。

離心力項：係數含 $\dot{q}^2$ （單一速度的平方）。物理意義是：當連桿繞某軸旋轉時，它有「被甩出去」的傾向，這會在其他關節上產生力矩。以平面 2R 為例， τ_2 中的 $\beta s_2 \dot{\theta}_1^2$ 就是離心力項——

—第一關節旋轉時，連桿 2 被向外甩，在第二關節上產生力矩。

科氏力項：係數含 $\dot{q}_i \dot{q}_j$ (兩個不同速度的乘積)。它只在**兩個關節同時運動**時出現。以平面 2R 為例， τ_1 中的 $-2\beta s_2 \dot{\theta}_1 \dot{\theta}_2$ 就是科氏力項。

科氏力的物理直覺較難掌握，但可以用一個經典的例子理解：**在旋轉的旋轉木馬上往中心走**。你會感覺到一個側向的力把你往旁邊推——這就是科氏力。它源自「你的旋轉半徑正在改變，因此角動量守恆要求切線速度改變，而改變速度就需要力」。機器人的第二關節彎曲時，連桿 2 的質心到第一軸的距離正在改變，因此產生同樣的效應。

13.7.2 為何常被忽略、又何時不能忽略

這兩類力都與速度的平方成正比，因此：

科氏力與離心力 $\propto \dot{q}^2$

速度加倍，這些力變成四倍。

這解釋了實務上的經驗法則：**低速時可以忽略、高速時絕對不能忽略**。以工業機器為例，在每秒 10 度的示教速度下，這些力可能只佔總力矩的百分之幾；但在每秒 200 度的生產速度下，可能達到百分之三十以上。

這正是開頭故事的完整解釋：低速調校時，控制器面對的幾乎只有慣性項與重力項，PID 足以應付；一旦提高到八成速度，科氏力與離心力暴增為原來的數十倍，成為 PID 無法及時補償的大干擾，於是出現軌跡偏差與振動。

13.7.3 C 矩陣的不唯一性與斜對稱性

有一個技術細節值得注意：**C 矩陣的寫法不唯一**。因為 $C(q, \dot{q})\dot{q}$ 是一個向量，把它「分解」成矩陣乘向量的方式有很多種。不同教科書可能給出不同的 C，但 $C \cdot \dot{q}$ 的結果必定相同。

在眾多寫法中，有一種特別有價值——採用 Christoffel 符號定義的 C，會使下列性質成立：

$\dot{M}(q) - 2C(q, \dot{q})$ 是斜對稱矩陣 (skew-symmetric)

即 $x^T(\dot{M} - 2C)x = 0$ 對任意 x 成立。

這個性質看似抽象，卻是第 19 章證明控制器穩定性的關鍵工具。它的物理意義是**能量守恆**：機器人內部的科氏力與離心力不會產生或消耗能量，只是在各關節之間重新分配。這與現實相符

——這些力源自慣性，而非外部能量來源。

13.8 重力項與摩擦

13.8.1 重力項 $G(q)$

重力項是四類項中最單純的：它只與姿態有關，與速度、加速度完全無關。

$$G(q) = \partial U(q) / \partial q$$

重力項就是位能對關節角度的梯度。

儘管單純，它在實務上卻極為重要，原因有三。

原因一：它通常是最大的一項。在中低速運動中，重力項往往佔總力矩的一半以上。一支水平伸展的手臂，光是「維持不動」就需要相當大的力矩。

原因二：它是恆定的偏差。與科氏力不同，重力項在靜止時仍然存在。若控制器不補償它，PID 的積分項必須持續累積來對抗重力，導致響應變慢且容易超越。這是**重力補償**（第 17 章）成為標準作法的原因。

原因三：它是碰撞偵測的最大干擾源。要由關節力矩推算外力（第 11.6.3 節），必須先精確扣除重力項。若重力模型有 5% 的誤差，在需要 50 N·m 重力補償的姿態下，就會產生 2.5 N·m 的殘差——這可能大於真實碰撞的訊號。

13.8.2 摩擦模型

摩擦力不出現在拉格朗日方程中（它是非保守力，不能由位能導出），必須另外加入。實務上常用的模型有三層：

庫倫摩擦 (Coulomb)：大小固定、方向與速度相反。

$$\tau_c = F_c \cdot \text{sign}(\dot{q})$$

在速度過零時會有不連續跳變，這是控制上的難題（造成低速爬行現象，stick-slip）。

黏滯摩擦 (Viscous)：與速度成正比。

$$\tau_v = F_v \cdot \dot{q}$$

來自潤滑油的阻力，是最容易補償的一項（線性）。

靜摩擦 (Stiction)：靜止時需要克服的最大摩擦力，通常大於運動中的庫倫摩擦。它造成「明明給了力矩卻不動，稍微加大就突然衝出去」的現象。

加入摩擦後，完整的動力學方程為：

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) + F(q) = \tau$$

含摩擦的完整形式。 $F(q)$ 通常取 $F_v\dot{q} + F_c\text{sign}(\dot{q})$ 。

13.8.3 摩擦為何難以處理

摩擦是動力學模型中最不可靠的一環，原因有四：它隨溫度變化（機器暖機後摩擦下降）、隨潤滑狀態變化（保養前後不同）、隨磨損變化（使用數年後改變）、且在低速時高度非線性（stick-slip 現象）。

這對兩件事有直接影響。第一，**高減速比的關節摩擦特別大**——減速機內部有大量齒面接觸，其摩擦力矩可能達到額定輸出的百分之十以上。第二，**碰撞偵測的靈敏度受限於摩擦的不確定性**——這正是第 11.6.3 節知識框中「協作機器人減速比不能太高」的另一個理由。

實務上的處理方式有三：一是透過實驗辨識摩擦參數（讓關節以不同速度等速運動，記錄所需力矩，擬合出 F_v 與 F_c ）；二是在低速時加入 dither（高頻微小振動）打破靜摩擦；三是採用低摩擦的機構設計（如直驅或低減速比方案）。

13.9 逆動力學與正向動力學

13.9.1 兩個方向的問題

有了標準形式 $M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau$ ，可以往兩個方向使用它。

逆動力學 (Inverse Dynamics)：已知運動，求所需力矩。

$$\tau = M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q)$$

直接代入即可，不需解方程。計算量為 $O(n)$ 或 $O(n^2)$ ，可即時執行。

這是控制上最常用的方向。給定規劃好的軌跡（ $q, \dot{q}, \ddot{q}$ 都已知），逆動力學算出「理論上需要多大的力矩」，把它當作前饋送給馬達——這正是開頭故事中叫獸提到的解決方案，也是第 17 章計算力矩控制的核心。

正向動力學 (Forward Dynamics)：已知力矩，求所產生的運動。

$$\ddot{\mathbf{q}} = \mathbf{M}^{-1}(\mathbf{q}) [\boldsymbol{\tau} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} - \mathbf{G}(\mathbf{q})]$$

需要求 $\mathbf{M}$ 的反矩陣 (或解線性方程組)，計算量較大。

這是**模擬**所需的方向。物理引擎在每個時間步驟中，由當前的力矩算出加速度，再積分得到速度與位置。第 30 章的 Gazebo、Isaac Sim 與 MuJoCo，內部做的就是這件事。

表 13-2 逆動力學與正向動力學的比較

面向	逆動力學	正向動力學
問題	已知運動 → 求力矩	已知力矩 → 求運動
公式	$\boldsymbol{\tau} = \mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{G}$	$\ddot{\mathbf{q}} = \mathbf{M}^{-1}(\boldsymbol{\tau} - \mathbf{C}\dot{\mathbf{q}} - \mathbf{G})$
是否需求逆	否 (直接代入)	是 ($\mathbf{M}$ 正定故必可逆)
計算複雜度	$O(n)$ (Newton-Euler 遞迴)	$O(n^3)$ 或 $O(n)$ (特殊演算法)
主要用途	控制前饋、力矩預測	物理模擬、預測行為
本書章節	第 14、17 章	第 14、30 章

13.9.2 為何正向動力學一定有解

回到 13.6.1 節的性質二： $\mathbf{M}$ 是正定矩陣，因此必定可逆。這保證了正向動力學永遠有唯一解——給定任何一組力矩，機器人的運動都是確定的。

這個看似理所當然的結論，其實有深刻的物理意義：它排除了「同樣的力產生不同運動」或「某些力矩組合無法產生任何運動」的可能。物理世界的因果關係，在數學上由慣性矩陣的正定性所保證。

13.10 參數辨識與模型誤差

13.10.1 參數從哪裡來

動力學模型需要每根連桿的三組參數：質量、質心位置、慣性張量。對 n 軸機器人，這是 $10n$ 個參數 ($1 + 3 + 6$)。它們的來源有三：

來源一：CAD 模型。設計階段直接由 3D 模型計算。優點是免費且完整；缺點是與實體有落差——線材、螺絲、油脂、加工公差都未計入，誤差通常在 5 ~ 15%。

來源二：實驗辨識。讓機器人執行特定的激勵軌跡，記錄關節力矩與運動資料，再用最小平方方法反推參數。這是最準確的方法，但需要專門的實驗設計。

來源三：廠商提供。部分廠商會提供動力學參數（尤其是協作機器人，因為力控制需要）。這是最省事的方式。

13.10.2 線性參數化：辨識的關鍵

動力學方程有一個極為有用的性質：**它對慣性參數是線性的。**

$$\tau = Y(q, \dot{q}, \ddot{q}) \cdot \pi$$

Y 稱為迴歸矩陣 (regressor)，只與運動有關； π 是由慣性參數組成的向量。

這個性質使參數辨識變成一個**線性最小平方問題**——只要收集足夠多的 (運動, 力矩) 資料，就能用線性代數直接解出 π ，不需非線性最佳化。這是動力學辨識能夠實用化的關鍵。

但有一個複雜之處：**並非所有參數都可辨識**。某些參數的組合永遠以固定的方式一起出現（如 13.5.4 節的 α 、 β 、 δ ），無法個別分離。可辨識的組合稱為**基本參數** (base parameters)，其數目少於 $10n$ 。實務上只需辨識這些基本參數即可，因為動力學行為只取決於它們。

13.10.3 模型誤差的影響

動力學模型永遠不完美，主要誤差源有五：慣性參數誤差（5 ~ 15%）、摩擦的不確定性（最大的一項）、關節彈性與背隙（剛體假設失效）、連桿變形（高速或重載時）、以及未建模的元件（線材、氣管的拉扯力）。

這帶來一個重要的工程觀點：**動力學模型的價值不在於完美，而在於把大部分的力矩需求「預測」出來**。即使模型只有 80% 的準確度，前饋控制也能把 PID 需要處理的誤差降低五倍——這已足以帶來顯著的性能提升。第 17 章的計算力矩控制正是建立在這個思路上：**用模型處理已知的部分，用回授處理未知的部分**。

13.11 由能量邁向遞迴

13.11.1 本章的成果

本章建立了機器人動力學的完整框架。你現在能夠：計算連桿的慣性參數與能量、運用拉格朗日方程推導運動方程、辨識並解釋慣性項、科氏力項、離心力項與重力項、理解慣性矩陣的四個性質及其工程意義、並說明逆動力學與正向動力學的用途。

最重要的是那條標準形式的方程：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) + F(q) = \tau$$

這條式子將在第三篇的每一章出現：第 15 章用它檢驗軌跡的可行性、第 16~17 章用它設計控制器、第 19 章用它證明穩定性、第 20 章用它分離外力。

13.11.2 接下來：從優雅到高效

拉格朗日法在理解上極為優雅，但它有一個實務上的致命問題：**計算效率**。要得到六軸手臂的解析方程，符號運算會產生數千項的表達式；即使得到了，每次求值也需要大量運算。控制器需要在每一毫秒完成一次逆動力學計算，這條路走不通。

第 14 章 (Newton-Euler 遞迴) 將提供解方。它放棄追求解析表達式，改用遞迴的數值計算：先由基座往末端傳遞速度與加速度（前向遞迴），再由末端往基座傳遞力與力矩（後向遞迴）。整個過程的複雜度只有 $O(n)$ ，六軸手臂的一次完整計算只需數微秒。兩種方法得到的答案完全相同，但一個適理解、一個適合實作。

第 15 章 (軌跡生成) 則會回頭使用本章的成果：規劃軌跡時不能只考慮幾何，還必須確認所需的力矩不超過馬達能力——而這正是逆動力學的工作。

換句話說：第 13 章讓你**看懂**動力學的結構，第 14 章讓你**算得夠快**，第 15 章之後則開始用**它來做事**。

完整計算例題

例題 13-1 單擺的動力學計算

題目：一根均勻細桿，長 $L = 0.6 \text{ m}$ 、質量 $m = 2.0 \text{ kg}$ ，一端以旋轉關節固定，可在垂直平面內擺動。(a) 求繞固定端的轉動慣量。(b) 當桿呈水平且靜止時，維持不動所需的力矩。(c) 當桿在水平位置且需產生 5 rad/s^2 的角加速度時，所需的總力矩。(d) 求小角度擺動時的自然頻率。

解 (a)：均勻細桿繞一端的轉動慣量 (13.1.3 節)：

$$I = (1/3) m L^2 = (1/3)(2.0)(0.6^2) = (1/3)(2.0)(0.36) = 0.24 \text{ kg}\cdot\text{m}^2$$

繞固定端的轉動慣量。

驗算 (平行軸定理)：繞質心 $I_c = (1/12)(2.0)(0.36) = 0.06$ ；質心距端點 $d = 0.3 \text{ m}$ ，故 $I = 0.06 + (2.0)(0.09) = 0.06 + 0.18 = 0.24 \checkmark$ 兩法一致。

解 (b)：水平時 $\theta = 90^\circ$ ，重力力矩最大。由 13.4.2 節的方程，令 $\ddot{\theta} = 0$ ：

$$\tau = m g (L/2) \sin 90^\circ = (2.0)(9.8)(0.3)(1.0) = 5.88 \text{ N}\cdot\text{m}$$

維持水平所需的力矩，即重力補償力矩。

解 (c)：總力矩為慣性項加重力項：

$$\tau = I \ddot{\theta} + m g (L/2) \sin \theta = (0.24)(5.0) + 5.88 = 1.20 + 5.88 = 7.08 \text{ N}\cdot\text{m}$$

總需求力矩。

工程觀察：注意慣性項 (1.20 N·m) 只佔總力矩的 17%，重力項 (5.88 N·m) 佔了 83%。這印證了 13.8.1 節的說法——在中低加速度下，重力往往是最大的一項。若加速度提高到 50 rad/s^2 ，慣性項變為 12 N·m，兩者的比例就反轉了。

解 (d)：小角度時 $\sin \theta \approx \theta$ ，且此處 θ 由垂直向下算起。方程化為：

$$I \ddot{\theta} + m g (L/2) \theta = 0 \Rightarrow \ddot{\theta} + [m g L / (2I)] \theta = 0$$

簡諧運動方程。

$$\omega_n = \sqrt{(m g L / (2I))} = \sqrt{(2.0)(9.8)(0.6) / (2 \times 0.24)} = \sqrt{(11.76/0.48)} = \sqrt{24.5} \approx 4.95 \text{ rad/s}$$

自然頻率約 4.95 rad/s ，即 0.79 Hz (週期約 1.27 秒)。

控制上的意義：若要有效控制這個系統，控制器的頻寬應至少為自然頻率的 5~10 倍 (約 4~8 Hz)，對應的取樣頻率則應再高 10 倍以上 (40~80 Hz 以上)。這是第 16 章選擇控制週期的依據之一。

例題 13-2 平面 2R 手臂的動力學計算

題目：平面 2R 手臂參數為： $L_1 = 0.4 \text{ m}$ 、 $L_2 = 0.3 \text{ m}$ 、 $m_1 = 3.0 \text{ kg}$ 、 $m_2 = 2.0 \text{ kg}$ ，兩連桿皆均勻 (故 $l_1 = 0.2$ 、 $l_2 = 0.15 \text{ m}$)， $I_1 = (1/12)m_1 L_1^2 = 0.04$ 、 $I_2 = (1/12)m_2 L_2^2 = 0.015 \text{ kg}\cdot\text{m}^2$ 。目前姿態 $\theta_1 = 30^\circ$ 、 $\theta_2 = 60^\circ$ ，關節速度 $\dot{\theta} = (1.0, -0.5) \text{ rad/s}$ ，加速度 $\ddot{\theta} = (2.0, 1.0) \text{ rad/s}^2$ 。求各關節所需力矩。

解：先計算三個常數組合 (13.5.4 節)：

$$\begin{aligned} \alpha &= m_1 l_1^2 + I_1 + m_2 (L_1^2 + l_2^2) + I_2 \\ &= 3.0(0.04) + 0.04 + 2.0(0.16 + 0.0225) + 0.015 \\ &= 0.12 + 0.04 + 0.365 + 0.015 = 0.540 \text{ kg}\cdot\text{m}^2 \end{aligned}$$

常數 α 。

$$\beta = m_2 L_1 l_2 = 2.0 \times 0.4 \times 0.15 = 0.120 \text{ kg}\cdot\text{m}^2$$

耦合常數 β 。

$$\delta = m_2 l_2^2 + I_2 = 2.0(0.0225) + 0.015 = 0.045 + 0.015 = 0.060 \text{ kg}\cdot\text{m}^2$$

常數 δ 。

代入 $\theta_2 = 60^\circ$: $c_2 = 0.5$ 、 $s_2 = 0.866$ 。慣性矩陣為：

$$M = \begin{bmatrix} \alpha + 2\beta c_2 & \delta + \beta c_2 \\ \delta + \beta c_2 & \delta \end{bmatrix} = \begin{bmatrix} 0.540 + 0.120 & 0.060 + 0.060 \\ 0.120 & 0.060 \end{bmatrix} = \begin{bmatrix} 0.660 & 0.120 \\ 0.120 & 0.060 \end{bmatrix}$$

慣性矩陣。注意其對稱性✓

慣性項： $M \cdot \ddot{q} =$

$$\begin{bmatrix} 0.660(2.0) + 0.120(1.0) \\ 0.120(2.0) + 0.060(1.0) \end{bmatrix} = \begin{bmatrix} 1.440 \\ 0.300 \end{bmatrix} \text{ N}\cdot\text{m}$$

慣性項。注意第一關節的 1.44 中，有 0.12 來自第二關節的加速度——這就是耦合。

科氏與離心項： $\beta s_2 = 0.120 \times 0.866 = 0.1039$ 。

$$\begin{aligned} \tau_1 \text{ 項} &= -\beta s_2 \dot{\theta}_2^2 - 2\beta s_2 \dot{\theta}_1 \dot{\theta}_2 = -0.1039(0.25) - 2(0.1039)(1.0)(-0.5) \\ &= -0.026 + 0.1039 = 0.0779 \text{ N}\cdot\text{m} \end{aligned}$$

第一關節的速度相關項（離心 + 科氏）。

$$\tau_2 \text{ 項} = \beta s_2 \dot{\theta}_1^2 = 0.1039(1.0)^2 = 0.1039 \text{ N}\cdot\text{m}$$

第二關節的離心力項。

重力項： $\theta_1 = 30^\circ$ ($c_1 = 0.866$)、 $\theta_1 + \theta_2 = 90^\circ$ ($c_{12} = 0$)。

$$\begin{aligned} G_1 &= (m_1 l_1 + m_2 L_1)g c_1 + m_2 l_2 g c_{12} \\ &= (3.0 \times 0.2 + 2.0 \times 0.4)(9.8)(0.866) + 2.0(0.15)(9.8)(0) \\ &= (0.6 + 0.8)(9.8)(0.866) = 11.88 \text{ N}\cdot\text{m} \end{aligned}$$

第一關節的重力項。

$$G_2 = m_2 l_2 g c_{12} = 2.0(0.15)(9.8)(0) = 0 \text{ N}\cdot\text{m}$$

第二關節的重力項。因連桿 2 恰好垂直，重力力臂為零。

總力矩：

$$\begin{aligned} \tau_1 &= 1.440 + 0.078 + 11.88 = 13.40 \text{ N}\cdot\text{m} \\ \tau_2 &= 0.300 + 0.104 + 0 = 0.404 \text{ N}\cdot\text{m} \end{aligned}$$

各關節的總需求力矩。

工程分析：第一關節的力矩中，重力佔 88.7%、慣性佔 10.7%、速度相關項僅佔 0.6%。這

是典型的中低速情形。若把速度提高五倍 ($\dot{\theta} = (5, -2.5)$)，速度相關項會變成 25 倍 (約 1.95 N·m)，佔比升至約 13%——這就是開頭故事中高速時失準的原因。

例題 13-3 耦合效應的量化

題目：承上例的機構參數。(a) 若第一關節完全靜止 ($\theta_1 = \dot{\theta}_1 = 0$)，而第二關節以 $\ddot{\theta}_2 = 10 \text{ rad/s}^2$ 加速，求第一關節需輸出多少力矩才能維持不動 (忽略重力)。(b) θ_2 為何值時這個耦合力矩最大？最小？(c) 由此提出一個降低耦合的機構設計建議。

解 (a)：由標準形式， $\tau_1 = M_{11}\ddot{\theta}_1 + M_{12}\ddot{\theta}_2 + (\text{速度項}) + G_1$ 。代入 $\ddot{\theta}_1 = 0$ 、忽略速度項與重力：

$$\tau_1 = M_{12} \cdot \ddot{\theta}_2 = (\delta + \beta c_2) \times 10 = (0.060 + 0.120 \times 0.5) \times 10 = 0.120 \times 10 = 1.20 \text{ N}\cdot\text{m}$$

第一關節必須輸出 1.20 N·m 才能「維持不動」。

這個結果值得深思：第一關節明明沒有運動，卻必須出力。若控制器只用 PID (只看位置誤差)，它必須等到第一關節真的被推動、產生誤差之後才會反應——這就是耦合造成的追隨誤差。

解 (b)： $M_{12} = \delta + \beta \cos \theta_2$ 隨 θ_2 變化：

$$\theta_2 = 0^\circ : M_{12} = 0.060 + 0.120 = 0.180 \text{ (最大, 耦合力矩 } 1.80 \text{ N}\cdot\text{m)}$$

$$\theta_2 = 180^\circ : M_{12} = 0.060 - 0.120 = -0.060 \text{ (絕對值最小)}$$

$$\theta_2 = 120^\circ : M_{12} = 0.060 - 0.060 = 0 \text{ (耦合完全消失)}$$

耦合項隨姿態變化，甚至可能為零或變號。

有趣的觀察：在 $\theta_2 = 120^\circ$ 時耦合為零——此時兩個關節在動力學上完全解耦。這種姿態稱為「動力學解耦位形」，某些高精度應用會刻意在此姿態附近作業。

解 (c)：耦合強度由 $\beta = m_2 L_1 l_2$ 決定。要降低耦合，可從三方面著手：減輕連桿 2 的質量 m_2 (用輕量材料)、縮短連桿 1 的長度 L_1 (但會犧牲工作空間)、或把連桿 2 的質心往其關節移近 (減小 l_2)——最後一項最實用，作法是把第二關節的馬達與減速機盡量往關節處配置，或採用第 5.9.2 節的皮帶傳動把馬達移到基座。這正是為何高性能機器手臂會不惜成本把驅動器往基座移。

程式範例：動力學方程的實作與驗證

以下 Python 程式實作平面 2R 手臂的完整動力學，包含逆動力學、正向動力學、能量計算與模擬，並用能量守恆驗證模型的正確性。

```
import numpy as np

class Planar2R:
    """平面 2R 手臂的動力學模型"""
    def __init__(self, L1=0.4, L2=0.3, m1=3.0, m2=2.0, g=9.8):
        self.L1, self.L2, self.m1, self.m2, self.g = L1, L2, m1, m2, g
        self.l1, self.l2 = L1/2, L2/2          # 均勻桿，質心在中點
        self.I1 = m1*L1**2/12                  # 繞質心的轉動慣量
        self.I2 = m2*L2**2/12
        # 三個常數組合 (13.5.4 節)
        self.alpha = m1*self.l1**2 + self.I1 + m2*(L1**2 + self.l2**2) + self.I2
        self.beta = m2*L1*self.l2
        self.delta = m2*self.l2**2 + self.I2

    def M(self, q):
        """慣性矩陣 (13.6 節)"""
        c2 = np.cos(q[1])
        return np.array([[self.alpha + 2*self.beta*c2, self.delta + self.beta*c2],
                        [self.delta + self.beta*c2, self.delta]])

    def C_qd(self, q, qd):
        """科氏力與離心力項 C(q,qd)·qd (13.7 節)"""
        s2 = np.sin(q[1])
        return np.array([-self.beta*s2*qd[1]**2 - 2*self.beta*s2*qd[0]*qd[1],
                        self.beta*s2*qd[0]**2])

    def G(self, q):
        """重力項 (13.8 節)"""
        c1, c12 = np.cos(q[0]), np.cos(q[0]+q[1])
        return np.array([
            (self.m1*self.l1 + self.m2*self.L1)*self.g*c1 + self.m2*self.l2*self.g*c12,
            self.m2*self.l2*self.g*c12])

    def inverse_dynamics(self, q, qd, qdd):
        """逆動力學：已知運動求力矩 (13.9.1 節)"""
        return self.M(q) @ qdd + self.C_qd(q, qd) + self.G(q)

    def forward_dynamics(self, q, qd, tau):
        """正向動力學：已知力矩求加速度"""
        return np.linalg.solve(self.M(q), tau - self.C_qd(q, qd) - self.G(q))

    def energy(self, q, qd):
        """回傳 (動能, 位能)"""
        T = 0.5 * qd @ self.M(q) @ qd          # 動能為二次型
        h1 = self.l1*np.sin(q[0])
        h2 = self.L1*np.sin(q[0]) + self.l2*np.sin(q[0]+q[1])
        U = self.m1*self.g*h1 + self.m2*self.g*h2
```

```

    return T, U

d, r2d = np.deg2rad, np.rad2deg
arm = Planar2R()

# ----- 驗證一：例題 13-2 -----
print("=== 例題 13-2：逆動力學計算 ===")
q = np.array([d(30), d(60)])
qd = np.array([1.0, -0.5])
qdd = np.array([2.0, 1.0])
print(f"常數:  $\alpha$ ={arm.alpha:.4f}  $\beta$ ={arm.beta:.4f}  $\delta$ ={arm.delta:.4f} kg·m2")
print(f"慣性矩陣 M = \n{np.round(arm.M(q), 4)}")
print(f"對稱性驗證: {np.allclose(arm.M(q), arm.M(q).T)}")
print(f"正定性驗證: {np.all(np.linalg.eigvals(arm.M(q)) > 0)}")
print(f"慣性項 M· $\ddot{q}$  = {np.round(arm.M(q) @ qdd, 4)} N·m")
print(f"科氏離心 C· $\dot{q}$  = {np.round(arm.C_qd(q, qd), 4)} N·m")
print(f"重力項 G = {np.round(arm.G(q), 4)} N·m")
tau = arm.inverse_dynamics(q, qd, qdd)
print(f"總力矩  $\tau$  = {np.round(tau, 3)} N·m")

# ----- 驗證二：逆動力學與正向動力學互為反運算 -----
print("\n=== 驗證：逆 / 正向動力學的一致性 ===")
qdd_back = arm.forward_dynamics(q, qd, tau)
print(f"由  $\tau$  反算加速度: {np.round(qdd_back, 6)}")
print(f"與原始  $\ddot{q}$  一致: {np.allclose(qdd, qdd_back)}")

# ----- 驗證三：例題 13-3 耦合效應 -----
print("\n=== 例題 13-3：耦合效應掃描 ===")
print(f"{' $\theta_2$ ':>6} {'M12':>9} {'耦合力矩':>10} ( $\ddot{\theta}_2 = 10 \text{ rad/s}^2$ )")
for t2 in (0, 30, 60, 90, 120, 150, 180):
    M12 = arm.M([0, d(t2)])[0, 1]
    print(f"{'t2':>6d} {'M12':>9.4f} {'M12*10':>10.3f} N·m"
          + (" ← 動力學解耦" if abs(M12) < 0.01 else ""))

# ----- 驗證四：能量守恆（模型正確性的最強驗證） -----
print("\n=== 能量守恆驗證（自由擺動、無外力、無摩擦） ===")
q_s = np.array([d(60), d(30)])
qd_s = np.array([0.0, 0.0])
dt = 1e-5
T0, U0 = arm.energy(q_s, qd_s)
E0 = T0 + U0
print(f"起始: T={T0:.6f} U={U0:.6f} E={E0:.6f} J")
for step in range(200000):
    # 模擬 2 秒
    qdd_s = arm.forward_dynamics(q_s, qd_s, np.zeros(2))
    qd_s = qd_s + qdd_s*dt
    # 半隱式歐拉積分
    q_s = q_s + qd_s*dt
    if (step+1) % 50000 == 0:
        T, U = arm.energy(q_s, qd_s)

```

```

    print(f"t={({step+1})*dt:.1f}s: T={T:8.5f} U={U:9.5f} "
          f"E={T+U:.6f} J 誤差={abs(T+U-E0)/abs(E0)*100:.4f}%")
print("> 總能量幾乎守恆，證明動力學模型與積分器皆正確")

# ----- 應用：重力補償力矩地圖 -----
print("\n=== 重力補償力矩（各姿態下維持靜止所需）===")
print(f"{'θ1':>5}{'θ2':>6} {'τ1':>9} {'τ2':>8}")
for t1 in (0, 45, 90):
    for t2 in (0, 90):
        G = arm.G([d(t1), d(t2)])
        print(f"{t1:5d}{t2:6d} {G[0]:9.3f} {G[1]:8.3f} N·m")
print("> θ1=0°（水平伸展）時重力負擔最大，與第 5 章選型的最壞情況一致")

```

請試著修改：（一）在能量守恆測試中加入摩擦項，觀察能量如何單調遞減（這驗證了摩擦是耗散力）。（二）把積分步長 dt 加大到 $1e-3$ ，觀察能量誤差如何增加——這說明了物理模擬為何需要小步長。（三）把 m_2 減半，觀察耦合項 β 與所需力矩的變化，驗證例題 13-3(c) 的設計建議。

實作：動力學參數的實驗辨識

本章實作是為實體手臂辨識動力學參數，並驗證模型的預測能力。建議使用第 9～12 章已建立運動學模型的手臂，且需能量測關節電流（第 5、6 章）。

- 幾何與質量量測：**拆下各連桿（或查閱 CAD），量測其質量、長度，並用「平衡點法」找出質心位置（把連桿放在細桿上，找到能平衡的位置即為質心）。
- 慣量估算：**對形狀規則的連桿，用 13.1.3 節的公式計算；對不規則者，可用「三線擺法」實測（懸掛連桿使其扭轉振盪，由週期反推慣量），或直接採用 CAD 值。
- 重力項驗證：**讓手臂在多個靜止姿態下維持不動，記錄各關節的電流，換算為力矩。與模型計算的 $G(q)$ 比較，繪出「理論 vs 實測」散布圖並計算相關係數。
- 摩擦參數辨識：**讓單一關節以多種等速轉動（其餘關節固定），記錄各速度下的穩態力矩。扣除重力項後，剩餘的力矩對速度作圖——截距為庫倫摩擦 F_c 、斜率為黏滯摩擦 F_v 。
- 慣性項驗證：**讓關節執行已知加速度的運動（如梯形速度曲線的加速段），記錄力矩。扣除重力與摩擦後，與 $M(q)\ddot{q}$ 比較。
- 耦合效應觀察：**固定第一關節（位置控制），讓第二關節快速加速，觀察第一關節的力矩是否上升。這是例題 13-3 的實驗版本。

7. **模型準確度評估**：執行一段包含加減速的完整軌跡，全程記錄實測力矩與模型預測力矩，計算誤差百分比。分析誤差主要來自哪一項（通常是摩擦）。

8. **前饋效果測試（進階）**：若控制器允許加入前饋力矩，比較「有無前饋」時的追隨誤差，量化動力學模型帶來的改善。

表 13-3 動力學參數辨識記錄表（第一列為範例）

項目	測試條件	理論值	實測值	誤差	說明
重力項 G_1	$\theta=(0^\circ, 0^\circ)$ 靜止	11.88 N·m	12.94 N·m	8.9%	含摩擦與質心估計誤差
（自行填寫其餘項目）					

國中小也看得懂：端著水桶走路的道理

提過裝滿水的水桶走路嗎？慢慢走的時候水面很平靜；但只要你**突然加速或急轉彎**，水就會晃出來——而且你必須用力抓住桶子，不然會被水拉得踉蹌一下。機器手臂也是一樣的道理喔！

前面幾章我們都假裝機器手臂「沒有重量」——說轉多少度就轉多少度。但真實的手臂是有重量的，這會帶來三件麻煩事：

第一件：要加速就要用力。 越重的手臂，要讓它動起來就要越用力，要讓它停下來也要越用力。這就是為什麼機器人的手臂要盡量做輕。

第二件：重力一直在拉它。 把手臂平舉在空中不動，其實馬達一直在用力！不信你自己把手臂平舉三十秒試試看，肩膀會很酸——那個「酸」就是你的肌肉在對抗重力。機器人的馬達也一樣。

第三件：關節會互相影響。 這個最神奇。把手臂平舉，然後只彎手肘（前臂快速抬起來），你會發現肩膀也要用力！明明肩膀沒動啊，為什麼要用力？因為前臂動起來的時候，會「拉」肩膀一下。機器人工程師必須算出這個互相影響有多大，機器人動起來才會又快又準。

這一章在做的事情，就是用數學算出這三件事。算清楚了，機器人就可以動得又快又穩——就像練過的服務生，端著滿滿一盤飲料快步走也不會灑出來。

叫獸提醒與常見錯誤

- 誤區一：把連桿長度 L 與質心距離 l 搞混。計算連桿 2 的質心位置時，第一項用 L_1 （全

長)、第二項用 I_2 (質心距離)，寫錯會導致整個模型偏差。

- 誤區二：忘記連桿 2 的絕對角速度是 $\theta_1 + \theta_2$ 而非 θ_2 。這是動能計算最常見的錯誤，會使耦合項完全消失。
- 誤區三：慣性張量的參考點弄錯。CAD 輸出的通常是繞質心的慣量，用於動力學方程時需以平行軸定理轉換。
- 誤區四：推導出的慣性矩陣不對稱。 M 必為對稱矩陣，若算出不對稱，一定是推導有誤，可據此檢查。
- 誤區五：低速調校完就直接高速運行。科氏力與離心力與速度平方成正比，速度加倍則這些力變成四倍。
- 誤區六：忽略摩擦。摩擦不出現在拉格朗日方程中（非保守力），必須另外加入，且它往往是模型誤差的最大來源。
- 誤區七：以為模型必須完美才有用。80% 準確的模型就能把 PID 需處理的誤差降低五倍，這已帶來顯著改善。
- 誤區八：把 C 矩陣的特定寫法當成唯一解。 C 的分解方式不唯一，但 $C \cdot q$ 的結果必定相同。
- 誤區九：碰撞偵測時未扣除完整的動力學項。必須扣除 $M \ddot{q}$ 、 $C \dot{q}$ 、 G 與摩擦，剩餘的才是外力，否則加速時會誤判為碰撞。
- 誤區十：模擬時積分步長取得太大。動力學模擬需要小步長（通常 1 ms 以下）才能維持能量守恆，否則系統會「無中生有」地發散。

課堂討論

1. 開頭故事中，小好想把 PID 增益調高來解決高速失準。請說明這個作法為何治標不治本，並列出它可能帶來的三個副作用。
2. 拉格朗日法「內力自動消去」是最大的優勢。請以平面 2R 手臂為例，說明若改用牛頓法需要處理哪些內力，以及為何它們在能量法中不會出現。
3. 例題 13-3 發現在 $\theta_2 = 120^\circ$ 時耦合完全消失。若你要設計一個高精度的裝配工作站，是否應該把作業點都安排在這個姿態附近？請討論其優缺點（提示：對照第 11、12 章的可操作度與奇異點）。
4. 動力學模型的誤差中，摩擦是最大也最難處理的一項。請討論三種降低摩擦影響的方法，並

比較它們在成本、性能與適用場合上的差異。

5. 第 30 章的物理模擬引擎內部執行的是正向動力學。請討論：為什麼模擬結果與真實機器人總會有落差 (Sim-to-Real Gap , 第 2.7.3 節) ？本章的哪些內容能解釋這個落差？

章末摘要

- 動力學回答「力與運動的關係」，是高速控制、力控制與物理模擬的共同基礎。
- 剛體的慣性由三組參數描述：質量（抵抗線加速度）、質心位置（重力作用點）、慣性張量（抵抗角加速度的 3×3 對稱矩陣）。
- 平行軸定理 $I = I_c + md^2$ 連結質心慣量與任意平行軸慣量，是動力學計算中使用最頻繁的定理。
- 剛體動能可分解為平移與轉動兩部分（König 定理）；用雅可比表達後，機器人總動能恆為二次型 $T = \frac{1}{2} \dot{q}^T M(q) \dot{q}$ 。
- 拉格朗日量 $L = T - U$ ；拉格朗日方程 $d/dt(\partial L/\partial \dot{q}) - \partial L/\partial q = \tau$ 由能量直接導出運動方程。
- 拉格朗日法的四大優勢：不需受力圖、內力自動消去、座標系自由、純量運算；代價是計算效率低於 Newton-Euler 法。
- 機器人動力學的標準形式： $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) + F(q) = \tau$ ，四類項分別對應慣性、速度相關力、重力與摩擦。
- 慣性矩陣 $M(q)$ 有四個性質：對稱、正定（故必可逆）、姿態相依、有界。正定性保證正向動力學永遠有唯一解。
- M 的對角元素是各關節的等效慣量；非對角元素是耦合項，代表一個關節的加速度會對其他關節產生力矩。
- 速度相關項含兩種力：離心力（係數含 $\dot{q}_i^2$ ）與科氏力（係數含 $\dot{q}_i \dot{q}_j$ ，兩軸同時運動時才出現）。
- 科氏力與離心力與速度平方成正比，低速可忽略、高速絕不可忽略——這是低速調校完美而高速失準的根本原因。
- 採 Christoffel 符號定義的 C 使 $\dot{M} - 2C$ 為斜對稱，此性質是第 19 章穩定性證明的關鍵，物理意義為內部力不產生能量。
- 重力項 $G(q) = \partial U/\partial q$ 只與姿態有關，在中低速時往往是最大的一項，也是碰撞偵測的最大

干擾源。

- 摩擦含庫倫、黏滯與靜摩擦三部分，是模型中最不可靠的一環，會隨溫度、潤滑與磨損變化。
- 逆動力學 ($\tau = M\ddot{q} + C\dot{q} + G$) 用於控制前饋；正向動力學 ($\ddot{q} = M^{-1}(\tau - C\dot{q} - G)$) 用於物理模擬。
- 動力學方程對慣性參數是線性的 ($\tau = Y \cdot \pi$)，使參數辨識成為線性最小平方問題；可辨識者稱為基本參數。

觀念題與計算題

觀念題

1. 說明質量、質心與慣性張量三者各描述什麼物理性質，以及為何轉動慣量必須用矩陣表示。
2. 敘述平行軸定理，並說明它在機器人動力學中連結了哪兩種資料。
3. 寫出剛體動能的兩部分分解，並說明為何機器人總動能恆為 $\frac{1}{2}\dot{q}^T M(q)\dot{q}$ 的形式。
4. 寫出拉格朗日方程，並列舉它相對於牛頓法的四個優勢與一個代價。
5. 寫出機器人動力學的標準形式，並說明四類項各自的物理意義。
6. 列舉慣性矩陣的四個性質，並說明正定性帶來的關鍵推論。
7. 區分慣性矩陣的對角元素與非對角元素的物理意義，並說明後者為何造成控制困難。
8. 區分離心力與科氏力，並各舉一個生活中的類比。
9. 為何說「低速時可忽略科氏力與離心力、高速時絕不可忽略」？請以定量關係說明。
10. 說明 $\dot{M} - 2C$ 為斜對稱的物理意義，以及它在何處會被用到。
11. 為何重力項在實務上特別重要？請列舉三個理由。
12. 列舉三種摩擦模型，並說明摩擦為何是動力學模型中最不可靠的一環。
13. 比較逆動力學與正向動力學的公式、計算成本與主要用途。
14. 說明「動力學方程對慣性參數是線性的」這個性質，以及它對參數辨識的意義。

計算題

1. 一根均勻細桿長 0.8 m、質量 3.0 kg，一端固定作單擺。(a) 求繞固定端的轉動慣量 (並用平行軸定理驗算)。(b) 求桿在水平位置維持靜止所需的力矩。(c) 若要在水平位置產

生 8 rad/s^2 的角加速度，求總力矩。(d) 求小角度擺動的自然頻率與週期。

2. 平面 2R 手臂： $L_1 = 0.5 \text{ m}$ 、 $L_2 = 0.4 \text{ m}$ 、 $m_1 = 4.0 \text{ kg}$ 、 $m_2 = 2.5 \text{ kg}$ (皆均勻)， $I_1 = (1/12)m_1 L_1^2$ 、 $I_2 = (1/12)m_2 L_2^2$ 。(a) 求常數 α 、 β 、 δ 。(b) 當 $\theta_2 = 45^\circ$ 時求慣性矩陣並驗證其對稱與正定。(c) 當 $\theta = (0^\circ, 45^\circ)$ 靜止時，求重力補償所需的力矩。
3. 承上題。姿態 $\theta = (20^\circ, 70^\circ)$ 、速度 $\dot{\theta} = (2.0, -1.5) \text{ rad/s}$ 、加速度 $\ddot{\theta} = (3.0, 2.0) \text{ rad/s}^2$ 。(a) 分別計算慣性項、科氏離心項與重力項。(b) 求總力矩。(c) 計算三類項各佔總力矩的百分比。(d) 若速度全部加倍而其餘不變，重新計算並比較速度相關項的變化倍數。
4. 承上題的機構參數。(a) 寫出耦合項 M_{12} 隨 θ_2 變化的表達式。(b) 求 M_{12} 的最大值、最小值及其發生的 θ_2 。(c) 求使 $M_{12} = 0$ (動力學解耦) 的 θ_2 值。(d) 若把 m_2 由 2.5 kg 減為 1.5 kg ，重新計算最大耦合力矩 (設 $\dot{\theta}_2 = 15 \text{ rad/s}^2$)，並計算改善的百分比。

動手做與專題延伸

本章實作：動力學模型模組

延續第 9~12 章的程式庫，加入動力學模組。這將是第三篇所有章节的共同基礎。

- **階段一：**實作平面 2R 手臂的完整動力學 (M、C、G、逆動力學、正向動力學)，並用能量守恆驗證。
- **階段二：**加入摩擦模型 (黏滯 + 庫倫)，驗證加入摩擦後能量單調遞減。
- **階段三：**實作三連桿或空間手臂的動力學。可用符號運算 (SymPy) 自動推導，並與數值微分的結果比對驗證。
- **階段四：**實作簡易的物理模擬器：給定初始狀態與力矩輸入，用正向動力學與數值積分模擬運動，並繪出動畫。這將是第 16 章測試控制器的環境。

專題延伸

- **能量守恆的積分器比較：**以自由擺動的平面 2R 手臂為測試案例，比較四種數值積分法 (顯式歐拉、半隱式歐拉、四階 Runge-Kutta、Verlet) 在不同步長下的能量漂移。繪出「步長 vs 能量誤差」的對數圖，並討論物理模擬器為何多採用半隱式或辛積分法。
- **力矩需求地圖：**對整個工作空間掃描，計算「以固定加速度執行運動」所需的關節力矩，繪成熱力圖。標出超過馬達額定扭力的區域——這張圖是第 15 章軌跡規劃的重要參考，也能回頭驗證第 5 章的馬達選型是否足夠。

- **進階：前饋控制的效益量化。**在你的模擬器中實作兩種控制器：（甲）純 PID；（乙）PID + 動力學前饋（計算力矩控制的簡化版）。讓兩者執行同一段高速軌跡，比較追隨誤差、最大力矩與穩定時間。接著故意讓前饋模型的參數有 20% 誤差，觀察性能下降多少——這個實驗會讓你直觀理解第 17 章「用模型處理已知、用回授處理未知」的設計哲學。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 6 章：操作臂動力學，本章主要參考)。
- Spong, M. W., Hutchinson, S. and Vidyasagar, M., Robot Modeling and Control, 2nd ed., Wiley (第 7 章：動力學，拉格朗日法的完整處理)。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer (第 7 章：動力學，含參數辨識與線性參數化)。
- Lynch, K. M. and Park, F. C., Modern Robotics, Cambridge University Press (第 8 章：動力學，含旋量觀點的表述)。
- Featherstone, R., Rigid Body Dynamics Algorithms, Springer (動力學演算法的權威專著，含高效實作)。
- Khalil, W. and Dombre, E., Modeling, Identification and Control of Robots, Butterworth-Heinemann (參數辨識的系統性方法)。
- Goldstein, H., Poole, C. and Safko, J., Classical Mechanics, 3rd ed., Addison-Wesley (拉格朗日力學的經典教材)。
- Swevers, J., Verdonck, W. and De Schutter, J., "Dynamic Model Identification for Industrial Robots," IEEE Control Systems Magazine, 2007 (工業機器人動力學辨識的實務回顧)。
- Pinocchio 與 RBDL 函式庫技術文件：高效的剛體動力學計算實作，支援逆 / 正向動力學與參數辨識。
- MuJoCo 與 Bullet 物理引擎文件：正向動力學在模擬中的實作與數值積分策略。

第 14 章

Newton-Euler 動力學與模擬引擎

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】上半部為一列火車的側視示意圖，車頭往前加速，動力沿車廂一節節向後傳遞（藍色箭頭）；同時最後一節的重量透過連結器一節節往前累積（紅色箭頭），越靠近車頭的連結器承受越大的力。下半部為一支六軸手臂，以完全相同的兩色箭頭標註前向遞迴（ ω 、 $\dot{\omega}$ 、 a ）與後向遞迴（ f 、 n ），每個關節旁標出取軸向投影得到的 τ_i 。右下角並列兩個對比框：「拉格朗日展開：14,000 項」與「Newton-Euler 遞迴：20 行程式」。

叫獸開講 「六軸機器人每毫秒都要算一次力矩，怎麼算才夠快？」

上完第 13 章之後，小翰花了整個週末，用 SymPy 推導六軸手臂的拉格朗日動力學方程。星期一早上他抱著筆電衝進研究室，臉上混合著興奮與絕望。

「老師，我算出來了……但是。」他把螢幕轉過來。第一關節的力矩方程式佔滿了整個畫面，往下捲了將近兩分鐘還沒到底。「這一條式子有一萬四千多項。」

叫獸看了一眼，笑了：「恭喜你，你推導出了正確的答案——一個完全沒有人會使用的正確答案。」

「為什麼？它是對的啊。」

「因為控制器每一毫秒就要算一次。」叫獸在白板上寫下一個數字：**1000 Hz**。「這是伺服控制迴路的典型頻率。你的公式如果要花十毫秒才算得完，控制器就只能等——而在那十毫秒裡，手臂已經移動了好幾公釐。**再正確的模型，算不快就沒有價值。**」

小翰洩氣地問：「那第 13 章學的都白費了？」

「當然不是。」叫獸搖頭，「第 13 章讓你**看懂**動力學的結構——你現在知道什麼是慣性矩陣、什麼是科氏力、為什麼關節之間會耦合。這些理解是無法取代的。但如果要**計算**，我們需要另一條路。」

他在白板上畫了一支手臂，然後畫了兩個方向的箭頭：一個由基座指向末端，一個由末端指回基座。

「想像你在一列火車上。第一節車廂加速，第二節就會感受到；第二節再傳給第三節……速度與加速度**由前往後傳遞**。」他指著向外的箭頭。「反過來，最後一節車廂的重量，會透過連結器一節一節往前壓；每一節車廂承受的力，等於它自己的重量加上後面所有車廂的重量。**力由後往前累積**。」他指著向內的箭頭。

「機器手臂完全一樣。先從基座往末端傳遞速度與加速度，算出每根連桿的運動狀態；再從末端往基座回推，算出每個關節需要承受多少力與力矩。整個過程只需要 **$O(n)$** 次運算——六軸手臂大約幾微秒就算完了。」

小翰盯著白板：「所以我週末推導的一萬四千項……」

「用這個方法，二十行程式碼就能得到同樣的答案，而且快一萬倍。」叫獸放下筆，「這就是今天要講的 Newton-Euler 遞迴法。第 13 章教你看懂，今天教你算得夠快。」

本章為什麼重要

第 13 章的拉格朗日法在理解上優雅，但它有一個致命的實務問題：計算量隨自由度呈指數成長。六軸手臂的解析方程可達上萬項，無法在毫秒級的控制迴路中即時求值。

Newton-Euler 遞迴法解決了這個問題。它放棄追求解析表達式，改用兩趟遞迴的數值計算：**前向遞迴**由基座往末端傳遞速度與加速度、**後向遞迴**由末端往基座累積力與力矩。整個過程的複雜度只有 $O(n)$ ——這意味著即使是二十自由度的人形機器人，也能在微秒級完成一次完整的逆動力學計算。

這個效率上的突破，使三件事成為可能：**即時前饋控制**（第 17 章的計算力矩控制需要每毫秒算一次逆動力學）、**高速物理模擬**（第 30 章的模擬器與第 31 章的強化學習訓練，需要每秒模擬數千步）、以及**線上碰撞偵測**（第 20 章需即時扣除慣性與重力的貢獻）。可以說，沒有高效的動力學演算法，現代機器人的許多能力都無從談起。

學習目標

- 能寫出剛體的牛頓方程與尤拉方程，並說明兩者的分工。
- 能說明角速度與角加速度在連桿之間的傳遞關係。
- 能推導質心加速度的三個組成項（切向、離心、平移）。
- 能執行前向遞迴，逐一計算各連桿的速度與加速度。
- 能執行後向遞迴，逐一計算各關節的力與力矩。
- 能說明為何遞迴法的複雜度為 $O(n)$ 而非 $O(n^3)$ 。
- 能運用遞迴法計算慣性矩陣、科氏項與重力項（複合剛體法）。
- 能比較拉格朗日法與 Newton-Euler 法的適用場合。
- 能說明正向動力學的三種演算法及其複雜度。
- 能說明物理模擬引擎的組成與數值積分的選擇。
- 能以 Python 實作 Newton-Euler 遞迴並與拉格朗日法交叉驗證。

先備知識

第 13 章的全部內容（尤其是慣性張量、標準形式方程與四類項的意義）、第 9 章的正向運動學與 DH 參數（遞迴需要各連桿之間的變換矩陣）、第 8 章的旋轉矩陣與座標系轉換、第 11 章的角速度概念與反對稱矩陣。

本章的推導以向量與矩陣運算為主，需要熟悉外積、座標系轉換與矩陣乘法。相較第 13 章，本章的數學較「機械」但概念較直觀——只要理解「傳遞」的物理圖像，公式就會自然成立。

核心名詞中英對照

表 14-1 本章核心名詞

中文	英文	說明
牛頓方程	Newton's Equation	描述質心平移的方程 $F = ma$
尤拉方程	Euler's Equation	描述剛體轉動的方程 $N = I\dot{\omega} + \omega \times I\omega$
前向遞迴	Forward Recursion / Outward Iteration	由基座往末端傳遞運動量
後向遞迴	Backward Recursion / Inward Iteration	由末端往基座累積力與力矩
連桿間力	Inter-link Force	相鄰連桿之間互相作用的力
切向加速度	Tangential Acceleration	角加速度造成的線加速度分量
向心加速度	Centripetal Acceleration	角速度造成的指向轉軸的加速度
複合剛體法	Composite Rigid Body Algorithm, CRBA	計算慣性矩陣的高效演算法
關節體演算法	Articulated Body Algorithm, ABA	$O(n)$ 複雜度的正向動力學演算法
重力技巧	Gravity Trick	以基座加速度模擬重力的技巧
數值積分	Numerical Integration	由加速度推算速度與位置的方法
辛積分法	Symplectic Integrator	能長期保持能量守恆的積分方法
剛度	Stiffness	微分方程中變化率差異極大的性質
接觸模型	Contact Model	描述物體碰撞與摩擦的數學模型
物理引擎	Physics Engine	執行剛體動力學模擬的軟體

14.1 牛頓方程與尤拉方程

14.1.1 剛體運動的兩條基本定律

Newton-Euler 法的名稱來自它所依據的兩條方程。任何剛體的運動，都可以完整地由這兩

條式子描述。

牛頓方程描述質心的平移運動：

$$\mathbf{F} = m \cdot \mathbf{a}_c$$

$\mathbf{F}$ 為作用在剛體上的合力、 m 為質量、 $\mathbf{a}_c$ 為質心的線加速度。這是大家熟悉的 $F = ma$ 。

尤拉方程描述繞質心的轉動運動：

$$\mathbf{N} = \mathbf{I} \cdot \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I} \cdot \boldsymbol{\omega})$$

$\mathbf{N}$ 為對質心的合力矩、 $\mathbf{I}$ 為繞質心的慣性張量、 $\boldsymbol{\omega}$ 與 $\dot{\boldsymbol{\omega}}$ 為角速度與角加速度。

14.1.2 尤拉方程的第二項從何而來

牛頓方程的形式很直觀，但尤拉方程多出了一項 $\boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega})$ ，這是初學者最困惑的地方。它的來源可以這樣理解：

角動量為 $\mathbf{L} = \mathbf{I} \cdot \boldsymbol{\omega}$ 。由角動量定理，力矩等於角動量的變化率： $\mathbf{N} = d\mathbf{L}/dt$ 。若在固定座標系中觀察，慣性張量 $\mathbf{I}$ 會隨剛體轉動而改變（因為物體的方位在變），這使微分變得複雜。

因此我們改在**隨剛體轉動的座標系**中計算，此時 $\mathbf{I}$ 是常數。但代價是：在旋轉座標系中對向量微分時，必須加上一個修正項：

$$d\mathbf{L}/dt|_{\text{固定}} = d\mathbf{L}/dt|_{\text{旋轉}} + \boldsymbol{\omega} \times \mathbf{L} = \mathbf{I} \cdot \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I} \cdot \boldsymbol{\omega})$$

第二項稱為**陀螺項 (gyroscopic term)**，是座標系旋轉所帶來的修正。

這一項的物理效應在生活中隨處可見：轉動中的陀螺不會倒下、騎自行車時車輪的角動量幫助維持平衡、以及旋轉一個形狀不對稱的物體時它會自己翻滾（Dzhanibekov 效應）。在機器人中，它是科氏力與離心力的來源之一。

重要的簡化：當剛體繞著其慣性主軸旋轉，且角速度沿該軸時， $\boldsymbol{\omega}$ 與 $\mathbf{I}\boldsymbol{\omega}$ 平行，外積為零，陀螺項消失。這也是為何平面問題（所有轉軸平行）的動力學比空間問題簡單得多。

14.1.3 與拉格朗日法的關係

值得強調的是：Newton-Euler 法與拉格朗日法得到的答案完全相同。它們只是推導路徑不同——前者從力的平衡出發（向量法），後者從能量出發（純量法）。

兩者的差異在於「過程中間產生了什麼」。拉格朗日法直接得到關節力矩，中間的連桿間內

力被自動消去；Newton-Euler 法則會顯式地算出每一根連桿之間的作用力。這看似多餘，實際上是一個重要的副產品——它讓我們能知道每個關節軸承受多大的力，這對機構設計與強度分析極為有用。

14.2 運動量的傳遞：前向遞迴

14.2.1 基本圖像

前向遞迴的物理圖像是開頭故事的火車：連桿 i 的運動狀態，等於連桿 $i-1$ 的運動狀態，加上關節 i 自己的貢獻。因此只要從基座（運動狀態已知，通常為靜止）出發，就能一節一節往外推算。

需要傳遞的量有四個：角速度 ω 、角加速度 $\dot{\omega}$ 、連桿原點的線加速度 a 、以及質心的線加速度 a_c 。

14.2.2 角速度的傳遞

由第 11.3.3 節，角速度可以直接相加。因此連桿 i 的角速度等於連桿 $i-1$ 的角速度（轉換到連桿 i 的座標系後），加上關節 i 自己造成的轉動：

$${}^i\omega_i = {}^iR_{i-1} \cdot {}^{i-1}\omega_{i-1} + \dot{\theta}_i \cdot \hat{z}_i$$

旋轉關節的角速度傳遞。 ${}^iR_{i-1}$ 是座標系轉換矩陣、 $\hat{z}_i$ 是關節軸的單位向量（在連桿 i 座標系中通常就是 $[0,0,1]^T$ ）。

若關節 i 是移動關節，它不產生轉動，因此第二項消失：

$${}^i\omega_i = {}^iR_{i-1} \cdot {}^{i-1}\omega_{i-1} \quad (\text{移動關節})$$

移動關節不改變角速度。

14.2.3 角加速度的傳遞

把角速度的式子對時間微分，會多出一個外積項：

$${}^i\dot{\omega}_i = {}^iR_{i-1} \cdot {}^{i-1}\dot{\omega}_{i-1} + ({}^iR_{i-1} \cdot {}^{i-1}\omega_{i-1}) \times (\dot{\theta}_i \hat{z}_i) + \dot{\theta}_i \hat{z}_i$$

角加速度的傳遞。中間的外積項來自「在旋轉座標系中微分」，它是科氏效應的直接來源。

這個外積項的物理意義是：即使關節 i 以固定的角速度轉動（ $\dot{\theta}_i = 0$ ），只要連桿 $i-1$ 本身也在轉動，連桿 i 仍會產生角加速度——因為它的轉軸方向正在改變。這正是陀螺效應在遞迴中

的體現。

14.2.4 線加速度的傳遞

線加速度的傳遞來自剛體運動學的基本公式。連桿 i 的原點相對於連桿 $i-1$ 的原點，有一個固定的位置向量 P （在連桿 $i-1$ 座標系中固定）。因此：

$${}^i a_i = {}^i R_{\{i-1\}} [{}^{i-1} \omega_{\{i-1\}} \times {}^{i-1} P_i + {}^{i-1} \omega_{\{i-1\}} \times ({}^{i-1} \omega_{\{i-1\}} \times {}^{i-1} P_i) + {}^{i-1} a_{\{i-1\}}]$$

旋轉關節的線加速度傳遞，共三項。

這三項各有明確的物理意義，值得逐一辨識：

- $\omega \times P$ （切向加速度）：角加速度造成的加速度，方向垂直於半徑，大小與角加速度成正比。這是「轉得越來越快」所產生的加速度。
- $\omega \times (\omega \times P)$ （向心加速度）：即使角速度固定不變，做圓周運動的點仍有指向轉軸的加速度，大小為 $\omega^2 r$ 。這是離心力的來源。
- $a_{\{i-1\}}$ （平移加速度）：上游連桿本身的加速度，直接傳遞下來。

質心的加速度用同樣的公式計算，只是把位置向量 P 換成「連桿原點到質心的向量」 P_c ：

$${}^i a_{ci} = {}^i \omega_i \times {}^i P_{ci} + {}^i \omega_i \times ({}^i \omega_i \times {}^i P_{ci}) + {}^i a_i$$

質心加速度。注意此處全部在連桿 i 的座標系中計算，不需再轉換。

14.2.5 重力的巧妙處理

重力需要特別處理，但有一個極為優雅的技巧：把基座的線加速度設為重力加速度的反方向。

$${}^0 a_0 = -\vec{g} = [0, 0, +9.8]^T \quad (\text{若重力沿 } -Z \text{ 方向})$$

重力技巧。設定基座「向上加速」，等效於整個系統受到向下的重力。

為什麼可行？這是愛因斯坦等效原理的一個工程應用：均勻重力場中的靜止觀察者，與無重力空間中等加速度運動的觀察者，物理上無法區分。因此讓基座「向上加速 9.8 m/s^2 」，所有連桿都會自動獲得正確的重力慣性力，不需在每一根連桿上另外加入重力項。

這個技巧的實用價值極高：它讓重力完全融入遞迴流程，不需任何特殊處理。程式中只需修改一個初始值，就能開關重力——這對模擬與測試極為方便。

◎ 驗證重力技巧

把手臂靜止不動（所有 $\dot{q} = \ddot{q} = 0$ ），只設定基座加速度為 $[0, 0, 9.8]^T$ ，執行完整的遞迴。你得到的

關節力矩，恰好就是第 13 章的重力項 $G(q)$ 。這是驗證 Newton-Euler 程式是否正確的第一個測試——它不需要任何複雜的軌跡，卻能檢查出座標轉換與遞迴邏輯的大部分錯誤。

14.3 力的累積：後向遞迴

14.3.1 基本圖像

完成前向遞迴後，我們知道了每一根連桿的加速度。現在用牛頓與尤拉方程，算出每根連桿需要多少力與力矩才能產生那樣的運動：

$$\begin{aligned} {}^iF_i &= m_i \cdot {}^i a_{ci} \quad (\text{作用在連桿 } i \text{ 質心上的合力}) \\ {}^iN_i &= {}^iI_i \cdot {}^i\dot{\omega}_i + {}^i\omega_i \times ({}^iI_i \cdot {}^i\omega_i) \quad (\text{對質心的合力矩}) \end{aligned}$$

由牛頓與尤拉方程直接計算。這是每根連桿「所需」的淨力與淨力矩。

但這個「合力」是從哪裡來的？答案是：來自上游連桿的推力，減去傳給下游連桿的力。因此可以由末端往基座回推——這就是後向遞迴。

14.3.2 力與力矩的平衡

考慮連桿 i 的受力平衡。它受到兩個相鄰連桿的作用：連桿 $i-1$ 透過關節 i 施加的力 f_i ，以及連桿 $i+1$ 透過關節 $i+1$ 施加的反作用力 $-f_{i+1}$ 。兩者之和必須等於淨力：

$${}^i f_i = {}^i R_{i+1} \cdot {}^{i+1} f_{i+1} + {}^i F_i$$

力的後向遞迴。連桿 i 所受的力，等於下游傳來的力，加上自己所需的淨力。

力矩的平衡較複雜，因為力作用在不同位置會產生額外的力矩：

$${}^i n_i = {}^i N_i + {}^i R_{i+1} \cdot {}^{i+1} n_{i+1} + {}^i P_{ci} \times {}^i F_i + {}^i P_{i+1} \times ({}^i R_{i+1} \cdot {}^{i+1} f_{i+1})$$

力矩的後向遞迴，共四項：自身所需的淨力矩、下游傳來的力矩、自身慣性力產生的力矩、以及下游傳來的力所產生的力矩。

最後兩項是外積形式，代表「力乘上力臂」。這正是後向遞迴中最容易出錯的部分——必須確認每個力的作用點與力臂向量是否正確。

14.3.3 取出關節力矩

遞迴完成後，得到的是每個關節處的完整力與力矩向量（六維）。但關節只能在其轉軸方向出力，其餘方向由軸承承擔。因此關節的驅動力矩就是力矩向量在轉軸上的投影：

$$\tau_i = {}^i n_i^T \cdot \hat{z}_i \quad (\text{旋轉關節})$$

$$\tau_i = {}^i\mathbf{f}_i^T \cdot \hat{\mathbf{z}}_i \quad (\text{移動關節})$$

取關節軸方向的分量。這一步是「六維力」轉為「一維關節力矩」的關鍵。

而其餘的分量（垂直於轉軸的力與力矩）雖然不是驅動力矩，卻是極有價值的資訊——它們正是軸承與結構必須承受的負荷。機構設計時，這些數值用於選定軸承規格與驗算連桿強度。這是 Newton-Euler 法相對於拉格朗日法的一個重要優勢（14.1.3 節）。

圖 14-1 Newton-Euler 兩趟遞迴的流程

建議配圖：一支三連桿手臂的示意圖，上方以藍色箭頭由基座往末端標示前向遞迴，每個關節旁標註傳遞的量（ ω 、 $\dot{\omega}$ 、 a 、 a_c ）；下方以紅色箭頭由末端往基座標示後向遞迴，每個關節旁標註累積的量（ f 、 n ），並在每個關節處以綠色標出取軸向投影得到的 τ_i 。左側標註「已知： $q, \dot{q}, \ddot{q}$ 」、右側標註「輸出： τ 」。基座處特別註明「重力技巧： $a_0 = -g$ 」。

14.4 完整演算法與複雜度分析

14.4.1 演算法總覽

把前後兩趟遞迴組合起來，就是完整的 Newton-Euler 逆動力學演算法：

1. 初始化：設 $\omega_0 = 0$ 、 $\dot{\omega}_0 = 0$ 、 $a_0 = -g$ （重力技巧）。
2. 前向遞迴（ $i = 1$ 到 n ）：依序計算 ω_i 、 $\dot{\omega}_i$ 、 a_i 、 a_{ci} ，再由牛頓尤拉方程得 F_i 、 N_i 。
3. 邊界條件：設末端所受的外力 f_{n+1} 與力矩 n_{n+1} （無外力時為零；若末端接觸環境則填入實測或期望的力）。
4. 後向遞迴（ $i = n$ 到 1 ）：依序計算 f_i 、 n_i 。
5. 取投影： $\tau_i = n_i^T \hat{\mathbf{z}}_i$ （旋轉關節），得到最終的關節力矩向量。

整個流程只需要基本的向量運算（加法、外積、矩陣乘向量），沒有任何矩陣求逆或符號運算。這正是它高效的原因。

14.4.2 為何是 $O(n)$

複雜度分析很直接：兩趟遞迴各執行 n 次，每次的運算量是固定的（幾個 3×3 矩陣乘法與外積，與 n 無關）。因此總運算量與 n 成正比，即 $O(n)$ 。

這與拉格朗日法形成鮮明對比。若直接展開拉格朗日方程的解析式，慣性矩陣有 n^2 個元素、科氏項有 n^3 個 Christoffel 符號，複雜度為 $O(n^3)$ 甚至更差（若考慮符號展開後的項數，會呈指數成長）。

表 14-2 兩種動力學方法的複雜度比較

方法	逆動力學複雜度	六軸手臂的典型運算時間	是否得到解析式	主要用途
拉格朗日 (符號展開)	$O(n^3)$ 或更差	數毫秒 (且需事先推導數小時)	是 (但可達上萬項)	理論分析、教學、小自由度系統
Newton-Euler 遞迴	$O(n)$	數微秒	否 (純數值)	即時控制、物理模擬、大自由度系統

以實際數字說明這個差距的意義：一個 20 自由度的人形機器人，用 $O(n)$ 演算法約需 20 個單位的運算；用 $O(n^3)$ 則需要 8000 個單位——相差 400 倍。這正是為何所有實用的機器人軟體 (Pinocchio、RBDL、KDL、MuJoCo) 內部都採用遞迴演算法。

14.4.3 可以順便得到什麼

Newton-Euler 遞迴的一個重要副產品，是它能用同一套程式碼算出動力學方程的各個組成部分，只要適當設定輸入：

表 14-3 用 Newton-Euler 計算各項的技巧

要計算的量	設定 $\ddot{q}$	設定 $\dot{q}$	設定重力	得到的結果
完整力矩 τ	實際值	實際值	開啟	$\tau = M\ddot{q} + C\dot{q} + G$
重力項 $G(q)$	0	0	開啟	$G(q)$
科氏離心項 $C(q, \dot{q})\dot{q}$	實際值	0	關閉	$C(q, \dot{q})\dot{q}$
慣性矩陣第 i 行	0	e_i (單位向量)	關閉	M 的第 i 行

最後一列的技巧特別實用：把加速度設為第 i 個單位向量 (只有第 i 個元素為 1，其餘為 0)，並關閉重力與速度，則得到的力矩就恰好是慣性矩陣的第 i 行。執行 n 次即可得到完整的 $M(q)$ ——這稱為**單位向量法**，複雜度為 $O(n^2)$ 。若追求更高效率，則使用專門的**複合剛體演算法** (CRBA)，複雜度同樣是 $O(n^2)$ 但常數更小。

14.5 正向動力學的三種演算法

14.5.1 問題的不對稱性

Newton-Euler 遞迴解決的是**逆動力學**：已知運動求力矩。但模擬需要的是相反方向——已

知力矩求加速度。這兩個問題的難度並不對稱。

逆動力學可以直接遞迴計算，因為運動狀態完全已知，力可以一節一節累積。正向動力學則不然：要算出連桿 1 的加速度，必須知道下游連桿施加的反作用力；但那個反作用力又取決於下游連桿的加速度，而後者又受連桿 1 影響——形成了循環依賴。這正是為何正向動力學需要更精巧的演算法。

14.5.2 方法一：組合法 ($O(n^3)$)

最直接的作法是利用第 13 章的標準形式。先用 Newton-Euler 分別算出各項，再解線性方程組：

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1} [\boldsymbol{\tau} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{G}(\mathbf{q})]$$

組合法。需要三個步驟：計算偏差項、計算慣性矩陣、解線性方程組。

具體流程為：一、用一次 Newton-Euler 遞迴（設 $\dot{\mathbf{q}} = 0$ 、開啟重力與實際速度）得到偏差項 $\mathbf{b} = \mathbf{C}\dot{\mathbf{q}} + \mathbf{G}$ ；二、用單位向量法或 CRBA 建構慣性矩陣 $\mathbf{M}$ ($O(n^2)$)；三、解 $\mathbf{M} \cdot \ddot{\mathbf{q}} = \boldsymbol{\tau} - \mathbf{b}$ ($O(n^3)$ ，通常用 Cholesky 分解，因 $\mathbf{M}$ 正定)。

總複雜度由第三步主導，為 $O(n^3)$ 。對六軸手臂完全可接受 ($n^3 = 216$)，但對高自由度系統（如 30 自由度的人形機器人， $n^3 = 27000$ ）效率就不理想。

14.5.3 方法二：關節體演算法 ($O(n)$)

Featherstone 於 1983 年提出的關節體演算法 (Articulated Body Algorithm, ABA) 將複雜度降到 $O(n)$ 。它的核心概念稱為關節體慣量 (articulated body inertia)：

把「連桿 i 以及它下游的所有連桿」視為一個整體，問一個問題：若對連桿 i 施加一個力，它會產生多大的加速度？由於下游連桿是透過關節連接（而非剛性固定），它們對連桿 i 呈現的「等效慣量」與剛性連接時不同——這個等效慣量就是關節體慣量。

ABA 用三趟遞迴完成計算：第一趟（前向）計算速度；第二趟（後向）計算關節體慣量與偏差力；第三趟（前向）由基座往末端計算加速度。每一趟都是 $O(n)$ ，因此總複雜度為 $O(n)$ 。

實務上，ABA 在自由度超過約 9 時開始優於組合法。現代的模擬器 (MuJoCo、Bullet、Pinocchio) 多半兩種都實作，依系統規模自動選擇。

14.5.4 方法三：最大座標法

前兩種方法都採用**最小座標**（以關節變數描述系統），約束自動滿足。另一條路線是**最大座標法**：把每根連桿都視為自由的六自由度剛體（共 $6n$ 個變數），再用約束方程強制它們透過關節連接。

$$\mathbf{M} \ddot{\mathbf{q}} = \boldsymbol{\tau} + \mathbf{J}^c \boldsymbol{\lambda}, \quad \mathbf{J}^c \ddot{\mathbf{q}} = -\dot{\mathbf{J}}^c \dot{\mathbf{q}}$$

$\boldsymbol{\lambda}$ 為約束力（拉格朗日乘子）、 $\mathbf{J}^c$ 為約束雅可比。需同時解出加速度與約束力。

這個方法變數多、需要解更大的方程組，且約束誤差會隨時間累積（連桿可能「散開」，需額外的穩定化處理）。但它有一個重要優勢：**能自然處理閉環機構與接觸約束**。因此電玩物理引擎與需要大量接觸的模擬（如抓取、行走）常採此法，而機器手臂的控制器則多用最小座標法。

表 14-4 三種正向動力學方法的比較

方法	複雜度	座標	閉環支援	數值穩定性	典型應用
組合法 ($\mathbf{M}^{-1}$)	$O(n^3)$	最小	困難	佳	低自由度手臂、控制器
關節體演算法 ABA	$O(n)$	最小	困難	佳	人形、四足、高自由度模擬
最大座標法	$O(m^3)$ (m 為約束數)	最大	自然支援	需穩定化	接觸豐富的模擬、電玩引擎

14.6 數值積分

14.6.1 由加速度到運動

正向動力學給出的是加速度。要得到實際的運動軌跡，必須對時間積分兩次：

$$\ddot{\mathbf{q}} \rightarrow \dot{\mathbf{q}} \rightarrow \mathbf{q}$$

積分兩次。實務上以離散的時間步驟 Δt 逐步推進，這稱為數值積分。

積分方法的選擇對模擬品質有決定性的影響——這也是第 13 章程式範例中「能量守恆」測試如此重要的原因。

14.6.2 四種常見的積分器

顯式歐拉法 (Explicit Euler)：最簡單，但最不穩定。

$$\dot{q}_{k+1} = \dot{q}_k + \ddot{q}_k \Delta t, \quad q_{k+1} = q_k + \dot{q}_k \Delta t$$

全部使用舊值更新。誤差為 $O(\Delta t)$ ，且能量會單調增加——模擬的鐘擺會越擺越高，最終發散。實務上幾乎不用。

半隱式歐拉法 (Semi-implicit / Symplectic Euler)：只改一個字，效果天差地遠。

$$\dot{q}_{k+1} = \dot{q}_k + \ddot{q}_k \Delta t, \quad q_{k+1} = q_k + \dot{q}_{k+1} \Delta t$$

位置更新時使用新的速度。這是一個辛積分器 (symplectic)，能長期保持能量在真值附近振盪而不發散。

這個看似微小的差異，是物理引擎的標準選擇。它的精度只有一階，但長期穩定性遠優於同階的顯式方法——對模擬而言，「不發散」比「每一步都很準」更重要。

四階 Runge-Kutta (RK4)：精度高（誤差 $O(\Delta t^4)$ ），每步需計算四次導數。適合需要高精度的短時間模擬（如驗證控制器），但不是辛積分器，長時間模擬仍會有能量漂移。

Verlet 系列：辛積分器，精度二階，在分子動力學與遊戲物理中廣泛使用。它的變體 (velocity Verlet) 能同時得到位置與速度，是精度與穩定性的良好折衷。

表 14-5 四種數值積分器的比較

方法	精度	每步計算量	能量行為	是否辛積分	適用場合
顯式歐拉	$O(\Delta t)$	1 次	單調增加（發散）	否	不建議使用
半隱式歐拉	$O(\Delta t)$	1 次	有界振盪	是	物理引擎標準選擇
Velocity Verlet	$O(\Delta t^2)$	1 ~ 2 次	有界振盪	是	高精度模擬、分子動力學
RK4	$O(\Delta t^4)$	4 次	緩慢漂移	否	短時間高精度、控制器驗證

14.6.3 步長的選擇

步長 Δt 的選擇是精度與速度的權衡，但還受一個更嚴格的限制：**穩定性**。若系統中含有高頻動態（剛性接觸、高增益控制器、彈性關節），步長必須小於該動態的特徵時間，否則模擬會發散。

實務上的經驗值為：機器手臂的自由運動可用 1 ~ 5 ms；含接觸的模擬需要 0.1 ~ 1 ms；含高剛度彈簧或剛性碰撞則可能需要 0.01 ms 以下。這正是為何含接觸的模擬遠比自由運動昂貴——第 30 章討論模擬器選擇時會再回到這一點。

◎ 為何能量守恆是最好的驗證

動力學程式的錯誤（座標轉換寫反、外積順序顛倒、力臂用錯）往往不會造成明顯的異常，模擬看起來「還算合理」，卻是錯的。但能量守恆是一個極為嚴格的檢驗：只要有任何一處推導錯誤，能量幾乎必定會漂移或發散。因此標準的驗證流程是：關閉重力以外的所有外力與摩擦，讓系統自由運動數秒，檢查總能量是否維持在初值的 0.1% 以內。若通過此測試，模型與積分器幾乎可以確定是正確的。

14.7 接觸與碰撞

14.7.1 模擬中最困難的部分

到目前為止，我們處理的都是「自由運動」——機器人在空中揮舞，不接觸任何東西。但機器人的價值恰恰在於**接觸**：抓取物體、插入零件、行走時腳踩地面。而接觸正是物理模擬中最困難、最不可靠的部分。

困難的來源有三。**第一，接觸是不連續的**：物體要嘛接觸、要嘛不接觸，狀態瞬間切換，這對數值積分極不友善。**第二，接觸力極大且作用時間極短**：碰撞瞬間的力可達數千牛頓，持續時間卻只有毫秒，需要極小的步長才能捕捉。**第三，摩擦是非線性且不唯一的**：庫倫摩擦模型在某些配置下有多組解（甚至無解），這是著名的 Painlevé 悖論。

14.7.2 兩類接觸模型

懲罰法（Penalty Method）：允許物體輕微互相穿透，並依穿透深度產生一個彈簧—阻尼力把它們推開。

$$F_{\text{contact}} = k \cdot d + c \cdot \dot{d} \quad (d \text{ 為穿透深度})$$

懲罰法。實作簡單、力是連續的，適合梯度式的最佳化與學習。缺點是需要極大的剛度 k 才能避免明顯穿透，而大 k 導致方程剛性化，必須用極小的步長。

約束法（Constraint-based）：以不等式約束強制「不得穿透」，接觸力由求解一個線性互補問題（LCP）或凸最佳化問題得到。

優點是物理上更嚴謹、不會穿透、可用較大步長。缺點是求解成本高，且在多接觸點的情況下可能無唯一解。多數現代引擎（MuJoCo、Bullet 的部分模式）採用經過正則化的凸最佳化形式，在嚴謹性與可解性之間取得平衡。

14.7.3 為何模擬的抓取常常不準

這解釋了第 2.7.3 節提到的 Sim-to-Real Gap 中最頑固的一環：**在模擬中完美的抓取，到真機上常常滑落**。原因可歸納為四點：

- **摩擦係數不確定**：真實材料的摩擦係數隨表面狀態、濕度、壓力而變，模擬中通常設為一個固定值。
- **接觸面積被簡化**：真實的夾爪與物體之間是面接觸（且會變形），模擬中常簡化為數個點接觸。
- **柔軟性未建模**：真實的夾爪墊、物體表面都會變形，這種順應性對抓取穩定性極為關鍵，但難以精確模擬。
- **接觸模型本身的近似**：懲罰法的穿透、約束法的正則化，都與真實物理有落差。

因應之道是第 31 章會詳述的**領域隨機化**：在訓練時隨機改變摩擦係數、質量、接觸剛度等參數，強迫學到的策略對這些不確定性具備強健性。這是目前處理接觸不確定性最有效的實務方法。

14.8 物理模擬引擎

14.8.1 引擎的四大組成

一個完整的物理模擬引擎包含四個模組，動力學只是其中之一：

一、碰撞偵測 (Collision Detection)：判斷哪些物體正在接觸，並計算接觸點、法向量與穿透深度。這通常是計算量最大的部分，分為粗略階段 (broad phase，用包圍盒快速排除不可能碰撞的配對) 與精確階段 (narrow phase，計算實際的接觸幾何)。

二、動力學求解 (Dynamics Solver)：即本章的正向動力學，計算在給定力與約束下的加速度。

三、約束求解 (Constraint Solver)：處理關節約束、接觸約束與摩擦，通常以迭代方式求解（如投影高斯—賽德爾法）。

四、數值積分 (Integrator)：14.6 節的內容，由加速度推進到下一個狀態。

14.8.2 常見引擎的比較

機器人領域常用的物理引擎各有取捨：

表 14-6 常見物理引擎的比較

引擎	特色	接觸模型	速度	典型用途
MuJoCo	為機器人與強化學習設計，接觸模型平滑可微	凸最佳化（軟接觸）	極快	強化學習訓練、控制研究
Bullet	開源、成熟、廣泛整合	LCP / 順序脈衝	快	遊戲、通用機器人模擬
ODE	歷史悠久、Gazebo 預設	LCP	中	ROS 生態系、教學
PhysX	NVIDIA 出品、GPU 加速	混合	極快（GPU）	Isaac Sim、大規模平行模擬
DART	重視動力學精確性	約束法	中	學術研究、精確動力學

選擇的原則是：看重接觸的真實性選 DART 或 Bullet；看重速度與可微性（用於學習）選 MuJoCo；需要 GPU 大規模平行訓練選 Isaac Sim（PhysX）；需要與 ROS 2 整合則優先考慮 Gazebo 支援的引擎。第 30 章會從模擬應用的角度再次比較。

14.8.3 可微分模擬：新的方向

近年出現一個重要趨勢：**可微分物理模擬**。傳統模擬器只能回答「給定這些參數，會發生什麼」；可微分模擬器還能回答「若要得到期望的結果，參數該如何調整」——因為它能計算模擬結果對輸入參數的梯度。

這帶來三個應用：**系統辨識**（由觀測到的真實運動，用梯度下降反推物理參數，比第 13.10 節的傳統辨識更通用）、**策略最佳化**（直接對控制參數求梯度，比強化學習的隨機探索更有效率）、以及**機構最佳化**（把連桿尺寸也當作可微參數，自動設計出最佳機構）。

挑戰在於接觸的不連續性——碰撞的瞬間梯度不存在或無意義。因此可微分模擬器（如 MuJoCo 的軟接觸、Brax、Dojo）多半採用平滑化的接觸模型，以可微性換取部分物理真實性。這個取舍的合理性仍在研究中，是第 31、33 章會觸及的前沿主題。

14.9 兩種方法的分工與實務選擇

14.9.1 各自的舞台

走完第 13、14 兩章，可以清楚地劃分兩種方法的分工。

拉格朗日法的價值在於「看見結構」。它給出的解析表達式讓我們能辨識慣性矩陣、科氏項與重力項，理解耦合從何而來、哪些參數影響哪些效應。第 13 章例題 13-3 那個「 $\theta_2 = 120^\circ$ 時耦合完全消失」的發現，只有透過解析式才看得出來。此外，控制理論的穩定性證明（第 19 章）也需要解析形式的性質（如 $\dot{M} - 2C$ 的斜對稱性）。

Newton-Euler 法的價值在於「算得夠快」。所有需要即時計算的場合——控制器前饋、物理模擬、線上碰撞偵測——都採用它。它也是所有機器人動力學函式庫的內部實作。

因此實務上的標準流程是：**用拉格朗日法理解與分析，用 Newton-Euler 法實作與計算**。兩者互相驗證——若你的遞迴程式與解析式在同一組輸入下得到不同結果，必定有一方寫錯了。這正是本章程式範例的核心測試。

14.9.2 現代實務：直接用函式庫

必須誠實地說：在真實的工程專案中，很少有人從頭實作動力學演算法。成熟的開源函式庫已經處理了所有的細節與最佳化：

- **Pinocchio**：C++ 實作，效能極高，支援解析導數（可微分），是目前研究界的主流選擇。
- **RBDL**（Rigid Body Dynamics Library）：輕量、易於整合，廣泛用於人形機器人研究。
- **KDL**（Kinematics and Dynamics Library）：ROS 生態系的傳統選擇，功能完整但效能較普通。
- **Drake**：包含動力學、最佳化與控制的完整框架，由 MIT 與 TRI 開發。

那麼為何還要學這一章？三個理由。**理由一：除錯**。當函式庫的輸出不符預期時，你必須知道它在算什麼、可能哪裡出錯（最常見的是慣性參數的座標系定義不一致）。**理由二：非標準機構**。閉環、柔性關節、浮動基座等情形，往往需要自行擴充。**理由三：理解性能瓶頸**。知道哪些操作是 $O(n)$ 、哪些是 $O(n^3)$ ，才能在系統設計時做出正確的取捨。

14.10 由動力學邁向軌跡與控制

14.10.1 本章的成果

本章完成了動力學的計算工具。你現在能夠：運用牛頓與尤拉方程描述剛體運動、執行前向與後向遞迴計算逆動力學、理解 $O(n)$ 複雜度的來源、選擇合適的正向動力學演算法與數值積分器、並理解物理模擬引擎的組成與接觸模型的限制。

結合第 13 章，你已經掌握了機器人動力學的完整圖像：知道方程長什麼樣、每一項代表什麼、以及如何在微秒內算出它。

14.10.2 接下來：把動力學用起來

第 13、14 章建立的是工具，接下來六章要把它用起來。

第 15 章 (軌跡生成)：規劃運動時不能只考慮幾何路徑，還必須確認所需的力矩不超過馬達能力、加速度不引起振動。這需要逆動力學來驗證軌跡的可行性。

第 16 章 (線性回授與 PID)：先從最簡單的控制器出發，理解回授的基本原理與極限——並看見為何純 PID 在高速時會失效 (正如第 13 章開頭的故事)。

第 17 章 (非線性控制)：把動力學模型加入控制器，用計算力矩控制與操作空間控制克服 PID 的限制。這是本章遞迴演算法最直接的應用場合。

第 18、19 章 (最佳控制與穩定性)：用更嚴謹的數學工具設計控制器並證明其穩定性。第 13 章提到的 $\dot{M} - 2C$ 斜對稱性，將在此發揮關鍵作用。

第 20 章 (力控制)：當機器人接觸環境時，必須同時控制運動與力。動力學模型讓我們能區分「慣性造成的力」與「外界施加的力」。

換句話說：第 13 章讓你**看懂**、第 14 章讓你**算快**、第 15 章之後開始**用它做事**。

完整計算例題

例題 14-1 單連桿的 Newton-Euler 遞迴

題目：一根均勻連桿，長 $L = 0.5 \text{ m}$ 、質量 $m = 2.0 \text{ kg}$ ，繞固定端在垂直平面內轉動 (即第 13 章的單擺)。目前 $\theta = 60^\circ$ (由垂直向下量起)、 $\dot{\theta} = 2.0 \text{ rad/s}$ 、 $\ddot{\theta} = 3.0 \text{ rad/s}^2$ 。以 Newton-Euler 遞迴計算所需的關節力矩，並與第 13 章的拉格朗日結果比對。

解：設連桿座標系 $\{1\}$ 的原點在關節處、 X 軸沿連桿方向、 Z 軸垂直於運動平面。質心在 $P_c = [L/2, 0, 0]^T = [0.25, 0, 0]^T$ 。繞質心的慣性張量中，只有 Z 方向的分量參與運動： $I_{zz} = (1/12)mL^2 = (1/12)(2.0)(0.25) = 0.04167 \text{ kg}\cdot\text{m}^2$ 。

前向遞迴。基座靜止， $\omega_0 = 0$ 、 $\dot{\omega}_0 = 0$ 。依重力技巧 (14.2.5 節)，設基座加速度為重力的反向。在連桿座標系中，需將重力向量轉換過來——當連桿與垂直方向夾 60° 時：

$${}^1\mathbf{a}_1 = {}^1\mathbf{R}_0 \cdot (-\mathbf{g}^*) = [g \sin \theta, g \cos \theta, 0]^T = [9.8(0.866), 9.8(0.5), 0]^T \\ = [8.487, 4.900, 0]^T \text{ m/s}^2$$

關節處的線加速度 (僅含重力效應，因基座固定) 。

角速度與角加速度直接由關節給出：

$${}^1\boldsymbol{\omega}_1 = [0, 0, 2.0]^T \text{ rad/s}, \quad {}^1\dot{\boldsymbol{\omega}}_1 = [0, 0, 3.0]^T \text{ rad/s}^2$$

沿 Z 軸的轉動。

質心加速度 (14.2.4 節的三項)：

$${}^1\mathbf{a}_{c1} = \dot{\boldsymbol{\omega}} \times \mathbf{P}_c + \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{P}_c) + \mathbf{a}_1$$

三項相加。

$$\dot{\boldsymbol{\omega}} \times \mathbf{P}_c = [0, 0, 3.0] \times [0.25, 0, 0] = [0, 0.75, 0]^T \quad (\text{切向})$$

$$\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{P}_c) = -\omega^2 \mathbf{P}_c = [-(4.0)(0.25), 0, 0]^T = [-1.0, 0, 0]^T \quad (\text{向心})$$

前兩項。

$${}^1\mathbf{a}_{c1} = [-1.0 + 8.487, 0.75 + 4.900, 0]^T = [7.487, 5.650, 0]^T \text{ m/s}^2$$

質心總加速度。

牛頓與尤拉方程：

$${}^1\mathbf{F}_1 = m \cdot \mathbf{a}_{c1} = 2.0 \times [7.487, 5.650, 0]^T = [14.974, 11.300, 0]^T \text{ N}$$

淨力。

$${}^1\mathbf{N}_1 = I \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (I\boldsymbol{\omega}) = [0, 0, 0.04167 \times 3.0]^T + 0 = [0, 0, 0.125]^T \text{ N}\cdot\text{m}$$

淨力矩。因 $\boldsymbol{\omega}$ 與 $I\boldsymbol{\omega}$ 平行 (皆沿 Z)，陀螺項為零。

後向遞迴。末端無外力 ($f_2 = 0$ 、 $n_2 = 0$)，故：

$${}^1\mathbf{f}_1 = {}^1\mathbf{F}_1 = [14.974, 11.300, 0]^T \text{ N}$$

關節受力。

$${}^1\mathbf{n}_1 = {}^1\mathbf{N}_1 + \mathbf{P}_c \times {}^1\mathbf{F}_1 = [0, 0, 0.125]^T + [0.25, 0, 0] \times [14.974, 11.300, 0]^T$$

力矩累積。

$$\mathbf{P}_c \times \mathbf{F} = [0.25, 0, 0] \times [14.974, 11.300, 0] = [0, 0, 0.25 \times 11.300]^T = [0, 0, 2.825]^T$$

外積計算。

$${}^1\mathbf{n}_1 = [0, 0, 0.125 + 2.825]^T = [0, 0, 2.950]^T \text{ N}\cdot\text{m}$$

關節總力矩向量。

取投影：關節軸為 Z 軸，故 $\tau = n_1^T \hat{z} = 2.950 \text{ N}\cdot\text{m}$ 。

與拉格朗日法比對（第 13.4.2 節）：

$$\tau = (1/3)mL^2\ddot{\theta} + mg(L/2)\sin\theta = (1/3)(2.0)(0.25)(3.0) + (2.0)(9.8)(0.25)(0.866) \\ = 0.500 + 2.450 = 2.950 \text{ N}\cdot\text{m} \checkmark$$

兩種方法完全一致，驗證成功。

觀察：Newton-Euler 法的步驟較多，但每一步都是簡單的向量運算；拉格朗日法一步到位，但需要事先推導出那條公式。對單連桿而言後者顯然較快，但當自由度增加到六時，情況會完全逆轉。

例題 14-2 複雜度的實際意義

題目：某控制器的迴路頻率為 1 kHz（每毫秒執行一次），處理器每秒可執行 10^8 次浮點運算。假設 Newton-Euler 遞迴每個自由度需 150 次浮點運算，而拉格朗日展開式的運算量約為 $30n^3$ 次。（a）計算六軸手臂用兩種方法的運算時間。（b）計算 20 自由度人形機器人的情形。（c）判斷各情形是否能滿足即時要求。

解（a）：六軸手臂（ $n = 6$ ）。

$$\text{Newton-Euler : } 150 \times 6 = 900 \text{ 次運算} \rightarrow 900 / 10^8 = 9 \mu\text{s}$$

佔 1 ms 週期的 0.9%，遊刃有餘。

$$\text{拉格朗日 : } 30 \times 6^3 = 30 \times 216 = 6480 \text{ 次運算} \rightarrow 6480 / 10^8 = 64.8 \mu\text{s}$$

佔週期的 6.5%，仍可接受但已高出 7 倍。

解（b）：20 自由度人形機器人（ $n = 20$ ）。

$$\text{Newton-Euler : } 150 \times 20 = 3000 \text{ 次} \rightarrow 30 \mu\text{s}$$

佔週期的 3%，仍然輕鬆。

$$\text{拉格朗日 : } 30 \times 20^3 = 30 \times 8000 = 240000 \text{ 次} \rightarrow 2400 \mu\text{s} = 2.4 \text{ ms}$$

超過 1 ms 的控制週期，無法即時執行。

解（c）：六軸手臂兩種方法都能滿足即時要求；但 20 自由度系統只有 Newton-Euler 可行，拉格朗日法會使控制迴路超時。

工程意義：這個對比說明了為何複雜度分析不是學術遊戲。 $O(n)$ 與 $O(n^3)$ 的差別，在 $n = 6$ 時只是「快 7 倍」，在 $n = 20$ 時卻是「能做」與「不能做」的差別。而現代機器人（人形、四足、雙臂）的自由度普遍在 20 以上，且模擬時還需要以比實時快數十倍的速度執行——此時演算法效率就是決定性的。

例題 14-3 積分器的能量行為比較

題目：以單擺為例（ $L = 0.5 \text{ m}$ 、 $m = 2.0 \text{ kg}$ ，無摩擦、無驅動），由 $\theta = 90^\circ$ 、 $\dot{\theta} = 0$ 靜止釋放。步長 $\Delta t = 0.01 \text{ s}$ 。（a）寫出顯式歐拉與半隱式歐拉的更新式。（b）計算第一步之後的狀態與總能量。（c）說明兩者長期行為的差異。

解（a）：單擺方程為 $\ddot{\theta} = -(mgL/2I)\sin\theta$ ，其中 $I = (1/3)mL^2 = 0.1667 \text{ kg}\cdot\text{m}^2$ 。故：

$$\ddot{\theta} = -[(2.0)(9.8)(0.25)/0.1667] \sin\theta = -29.4 \sin\theta$$

角加速度。

$$\text{顯式歐拉：}\dot{\theta}_{k+1} = \dot{\theta}_k + \ddot{\theta}_k \Delta t, \quad \theta_{k+1} = \theta_k + \dot{\theta}_k \Delta t$$

$$\text{半隱式歐拉：}\dot{\theta}_{k+1} = \dot{\theta}_k + \ddot{\theta}_k \Delta t, \quad \theta_{k+1} = \theta_k + \dot{\theta}_{k+1} \Delta t$$

唯一差別在位置更新使用舊速度或新速度。

解（b）：初始 $\theta_0 = \pi/2$ 、 $\dot{\theta}_0 = 0$ ，故 $\ddot{\theta}_0 = -29.4 \sin(90^\circ) = -29.4 \text{ rad/s}^2$ 。

初始能量： $T = 0$ 、 $U = -mg(L/2)\cos(90^\circ) = 0$ ，故 $E_0 = 0 \text{ J}$ （以水平位置為基準）。

顯式歐拉： $\dot{\theta}_1 = 0 + (-29.4)(0.01) = -0.294 \text{ rad/s}$ ； $\theta_1 = \pi/2 + (0)(0.01) = 1.5708 \text{ rad}$ （位置沒有改變，因為用的是舊速度 0）。

$$E_1 = \frac{1}{2}(0.1667)(0.294^2) + (-2.0 \times 9.8 \times 0.25 \times \cos 90^\circ) = 0.00721 + 0 = 0.00721 \text{ J}$$

能量憑空增加了！這正是顯式歐拉會發散的原因。

半隱式歐拉： $\dot{\theta}_1$ 同樣為 -0.294 rad/s ；但 $\theta_1 = \pi/2 + (-0.294)(0.01) = 1.5679 \text{ rad}$ （位置有變化）。

$$E_1 = 0.00721 + (-4.9 \cos(1.5679)) = 0.00721 - 0.00436 = 0.00285 \text{ J}$$

誤差小得多，且不會單調累積。

解（c）：長期行為的差異是決定性的。顯式歐拉的能量每一步都增加，模擬中的單擺會越擺越高，數十秒後完全發散、失去物理意義。半隱式歐拉的能量在真值附近有界振盪，即使模擬

數小時也不會發散——擺動的幅度可能略有偏差，但物理行為始終合理。

這正是所有物理引擎採用辛積分器的原因：對長時間模擬而言，「不發散」比「每一步都精確」重要得多。

程式範例：Newton-Euler 遞迴的實作與交叉驗證

以下 Python 程式實作平面手臂的 Newton-Euler 遞迴逆動力學，並與第 13 章的拉格朗日解析式交叉驗證。同時示範用遞迴法萃取慣性矩陣、重力項，以及三種積分器的能量行為比較。

```
import numpy as np

class PlanarNE:
    """平面 n 連桿手臂的 Newton-Euler 遞迴動力學"""
    def __init__(self, L, m, g=9.8):
        self.L = np.array(L, float)      # 各連桿長度
        self.m = np.array(m, float)      # 各連桿質量
        self.n = len(L)
        self.lc = self.L / 2              # 均勻桿，質心在中點
        self.I = self.m * self.L**2 / 12  # 繞質心的轉動慣量 (Z 軸)
        self.g = g

    def inverse_dynamics(self, q, qd, qdd, gravity=True):
        """Newton-Euler 遞迴逆動力學 (14.4.1 節的五步驟)"""
        n = self.n
        # ----- 步驟 1：初始化 (重力技巧) -----
        w = 0.0                          # 角速度 (平面問題為純量，沿 Z)
        wd = 0.0                          # 角加速度
        # 基座線加速度：向上加速等效於重力 (14.2.5 節)
        a = np.array([0.0, self.g if gravity else 0.0])
        F, N, ac = [], [], []

        # ----- 步驟 2：前向遞迴 -----
        th = 0.0                          # 累積絕對角度
        for i in range(n):
            th_prev = th
            th += q[i]
            # 由前一連桿座標系轉到本連桿座標系 (旋轉 q[i])
            c, s = np.cos(q[i]), np.sin(q[i])
            R = np.array([[c, s], [-s, c]])  # ^i R_{i-1}
            a = R @ a                        # 先把上游加速度轉換過來
            w = w + qd[i]                    # 角速度累加 (14.2.2 節)
            wd = wd + qdd[i]                 # 角加速度累加
            # 質心加速度：切向 + 向心 + 平移 (14.2.4 節)
            r = np.array([self.lc[i], 0.0])
            a_c = np.array([-w*w*r[0], wd*r[0]]) + a
```



```

ac.append(a_c)
F.append(self.m[i] * a_c)          # 牛頓方程
N.append(self.I[i] * wd)          # 尤拉方程 (平面・陀螺項為 0)
# 準備傳給下一連桿：連桿末端 (長度 L) 的加速度
rL = np.array([self.L[i], 0.0])
a = np.array([-w*w*rL[0], wd*rL[0]]) + a

# ----- 步驟 3-4：後向遞迴 -----
f = np.zeros(2)                   # 末端無外力
nz = 0.0
tau = np.zeros(n)
for i in range(n-1, -1, -1):
    if i < n-1:                   # 把下游的力轉回本座標系
        c, s = np.cos(q[i+1]), np.sin(q[i+1])
        R_next = np.array([[c, -s], [s, c]]) # ^i R_{i+1}
        f = R_next @ f
        r = np.array([self.lc[i], 0.0])
        rL = np.array([self.L[i], 0.0])
        # 力矩 = 自身淨力矩 + 下游力矩 + 兩個力臂項 (14.3.2 節)
        nz = N[i] + nz + np.cross(r, F[i]) + np.cross(rL, f)
        f = f + F[i]
        tau[i] = nz               # 步驟 5：取 Z 軸投影
return tau

# ----- 用遞迴法萃取各項 (14.4.3 節的技巧) -----
def gravity_vector(self, q):
    z = np.zeros(self.n)
    return self.inverse_dynamics(q, z, z, gravity=True)

def coriolis_vector(self, q, qd):
    return self.inverse_dynamics(q, qd, np.zeros(self.n), gravity=False)

def mass_matrix(self, q):
    """單位向量法：逐行萃取慣性矩陣"""
    M = np.zeros((self.n, self.n))
    z = np.zeros(self.n)
    for i in range(self.n):
        e = np.zeros(self.n); e[i] = 1.0
        M[:, i] = self.inverse_dynamics(q, z, e, gravity=False)
    return M

def forward_dynamics(self, q, qd, tau):
    b = self.inverse_dynamics(q, qd, np.zeros(self.n), gravity=True)
    return np.linalg.solve(self.mass_matrix(q), tau - b)

def energy(self, q, qd):
    T = 0.5 * qd @ self.mass_matrix(q) @ qd
    U, th, x, y = 0.0, 0.0, 0.0, 0.0
    for i in range(self.n):
        th += q[i]
        U += self.m[i]*self.g*(y + self.lc[i]*np.sin(th))
        x += self.L[i]*np.cos(th); y += self.L[i]*np.sin(th)
    return T, U

```

```

d = np.deg2rad

# ----- 驗證一：例題 14-1 單連桿・與拉格朗日比對 -----
print("=== 例題 14-1：單連桿交叉驗證 ===")
arm1 = PlanarNE([0.5], [2.0])
q, qd, qdd = np.array([d(60)]), np.array([2.0]), np.array([3.0])
tau_ne = arm1.inverse_dynamics(q, qd, qdd)
# 拉格朗日解析式 (第 13.4.2 節) :  $\tau = (1/3)mL^2\ddot{\theta} + mg(L/2)\sin\theta$ 
m, L = 2.0, 0.5
tau_lag = (1/3)*m*L**2*qdd[0] + m*9.8*(L/2)*np.sin(q[0])
print(f"Newton-Euler : {tau_ne[0]:.4f} N·m")
print(f"拉格朗日解析 : {tau_lag:.4f} N·m")
print(f"兩者一致 : {np.isclose(tau_ne[0], tau_lag)}")

# ----- 驗證二：平面 2R・與第 13 章解析式比對 -----
print("\n=== 平面 2R：與第 13 章解析式交叉驗證 ===")
L1, L2, m1, m2 = 0.4, 0.3, 3.0, 2.0
arm2 = PlanarNE([L1, L2], [m1, m2])
l1, l2 = L1/2, L2/2
I1, I2 = m1*L1**2/12, m2*L2**2/12
alpha = m1*l1**2 + I1 + m2*(L1**2 + l2**2) + I2
beta = m2*L1*l2
delta = m2*l2**2 + I2

q = np.array([d(30), d(60)])
qd = np.array([1.0, -0.5])
qdd = np.array([2.0, 1.0])

# 解析式 (第 13.5.5 節)
c2, s2 = np.cos(q[1]), np.sin(q[1])
M_lag = np.array([[alpha + 2*beta*c2, delta + beta*c2],
                  [delta + beta*c2, delta]])
C_lag = np.array([-beta*s2*qd[1]**2 - 2*beta*s2*qd[0]*qd[1],
                  beta*s2*qd[0]**2])
c1, c12 = np.cos(q[0]), np.cos(q[0]+q[1])
G_lag = np.array([(m1*l1 + m2*L1)*9.8*c1 + m2*l2*9.8*c12,
                  m2*l2*9.8*c12])
tau_lag2 = M_lag @ qdd + C_lag + G_lag

print(f"慣性矩陣 NE:\n{np.round(arm2.mass_matrix(q), 5)}")
print(f"慣性矩陣 拉格朗日:\n{np.round(M_lag, 5)}")
print(f"一致: {np.allclose(arm2.mass_matrix(q), M_lag)}")
print(f"重力項 NE: {np.round(arm2.gravity_vector(q), 5)}")
print(f"重力項 拉格朗日: {np.round(G_lag, 5)} 一致: "
      f"{np.allclose(arm2.gravity_vector(q), G_lag)}")
print(f"總力矩 NE: {np.round(arm2.inverse_dynamics(q, qd, qdd), 5)}")
print(f"總力矩 拉格朗日: {np.round(tau_lag2, 5)}")
print(f"一致: {np.allclose(arm2.inverse_dynamics(q, qd, qdd), tau_lag2)}")

```

```
# ----- 驗證三：例題 14-3 積分器能量行為 -----
print("\n=== 例題 14-3：三種積分器的能量行為 ===")
def simulate(method, steps=3000, dt=0.005):
    q, qd = np.array([d(90)]), np.array([0.0])
    T0, U0 = arm1.energy(q, qd); E0 = T0 + U0
    errs = []
    for k in range(steps):
        if method == "rk4":
            def deriv(s):
                qq, vv = s[:1], s[1:]
                return np.concatenate([vv, arm1.forward_dynamics(qq, vv, np.zeros(1))])
            s = np.concatenate([q, qd])
            k1 = deriv(s); k2 = deriv(s + dt/2*k1)
            k3 = deriv(s + dt/2*k2); k4 = deriv(s + dt*k3)
            s = s + dt/6*(k1 + 2*k2 + 2*k3 + k4)
            q, qd = s[:1], s[1:]
        else:
            qdd_ = arm1.forward_dynamics(q, qd, np.zeros(1))
            if method == "explicit":
                q_new = q + qd*dt # 用舊速度
                qd = qd + qdd_*dt
                q = q_new
            else: # semi-implicit
                qd = qd + qdd_*dt
                q = q + qd*dt # 用新速度
            if (k+1) % 600 == 0:
                T, U = arm1.energy(q, qd)
                errs.append(abs(T + U - E0))
    return errs

for m_ in ("explicit", "semi-implicit", "rk4"):
    e = simulate(m_)
    print(f" {m_:14s} 能量誤差演變: " +
          " ".join(f"{x:.2e}" for x in e))
print(" → 顯式歐拉單調發散；半隱式有界；RK4 最精確但非辛積分")
```

請試著修改：（一）把積分步長由 0.005 改為 0.02，觀察顯式歐拉多快發散、半隱式是否仍然穩定。（二）把 PlanarNE 擴充為三連桿，驗證遞迴程式是否仍與解析式一致（這是檢驗遞迴邏輯正確性的最佳測試）。（三）用 time 模組測量 mass_matrix 與 inverse_dynamics 的執行時間，驗證前者約為後者的 n 倍。

實作：動力學演算法的驗證與效能測試

本章實作以程式驗證為主，目標是建立一套可信賴的動力學計算工具，並量化其效能。

1. **交叉驗證**：以平面 2R 手臂為測試案例，用 Newton-Euler 遞迴與第 13 章的拉格朗日解析式計算同一組輸入，比對 M 、 $C \cdot q$ 、 G 與總力矩。在至少 100 組隨機的 $(q, \dot{q}, \ddot{q})$ 下驗證兩者誤差小於 $1e-10$ 。

2. **擴充至三連桿**：把遞迴程式擴充到三連桿，並用 SymPy 自動推導對應的拉格朗日方程作為驗證基準。這一步能檢驗遞迴邏輯是否真的通用（很多錯誤只在三連桿以上才會顯現）。
3. **重力項單獨驗證**：設 $\dot{q} = \ddot{q} = 0$ ，執行遞迴，確認得到的力矩恰為重力補償力矩。用不同姿態測試，並與手算的 $\sum m_i g \cdot (\text{力臂})$ 比對。
4. **能量守恆測試**：關閉重力以外的所有外力，自由模擬 10 秒，記錄總能量的漂移百分比。分別以三種積分器與三種步長（0.001、0.005、0.02 s）測試，製作 3×3 的比較表。
5. **效能量測**：用 time 模組量測逆動力學、慣性矩陣與正向動力學的執行時間。改變連桿數 $n = 2, 4, 6, 10, 20$ ，繪出「 n vs 執行時間」的對數圖，驗證逆動力學為 $O(n)$ 、慣性矩陣為 $O(n^2)$ 。
6. **與函式庫比對（進階）**：安裝 Pinocchio 或 RBDL，用相同的機構參數建立模型，比對你的實作與函式庫的輸出。若有差異，最可能的原因是慣性張量的座標系定義不同。
7. **即時性評估**：計算你的實作在目標硬體（如 Raspberry Pi 或 STM32）上能支援的最高控制頻率，並判斷是否滿足 1 kHz 的需求。

表 14-7 動力學驗證記錄表（前兩列為範例）

測試項	測試條件	預期結果	實測結果	判定
交叉驗證	100 組隨機輸入，2R 手臂	誤差 $< 1e-10$	最大誤差 $3.2e-14$	通過
能量守恆	半隱式歐拉， $dt=0.005$ ，10 秒	漂移 $< 0.1\%$	0.03%	通過
（自行填寫其餘測試）				

國中小也看得懂：來回傳兩次的傳話遊戲

你有沒有玩過「傳話遊戲」？第一個人說一句話，一個一個傳下去，傳到最後一個人。機器手臂算力氣的方法，其實就像玩兩次傳話遊戲——一次往前傳，一次往後傳。

第一次：從肩膀傳到手指（傳「動作」）。想像你坐在火車上。第一節車廂動了，第二節就會跟著動；第二節再帶動第三節……機器手臂也一樣：肩膀轉動的時候，手肘會跟著被帶動，手腕又被手肘帶動。所以電腦先從最靠近身體的關節開始，一個一個算出「這一節現在動得多快」。

第二次：從手指傳回肩膀（傳「力氣」）。這次方向相反。想想看你手上拿著一本很重的

書：手指要出一點力氣抓住書、手腕要出更多力氣（因為要撐住書和手掌）、手肘要出更多、肩膀最辛苦（要撐住書和整條手臂）。越靠近身體的關節，要出的力氣越大——這就是為什麼機器手臂靠近底座的馬達總是最大顆的。

這個「來回傳兩次」的方法有多厲害呢？如果用另一種方法（直接寫出一條大公式），六軸機器人的公式會長到有一萬多項，印出來要好幾十頁！但用傳話的方法，只要二十行程式就寫完了，而且快一萬倍。這就是為什麼聰明的方法這麼重要——答案一樣，但一個做得到、一個做不到。

叫獸提醒與常見錯誤

- 誤區一：忘記尤拉方程的陀螺項 $\omega \times (I\omega)$ 。它在平面問題中為零，但在空間運動中不可忽略，遺漏會導致高速旋轉時的誤差。
- 誤區二：座標系轉換的方向弄反。前向遞迴用 ${}^iR_{i-1}$ （把上游量轉到本座標系），後向遞迴用 ${}^iR_{i+1}$ （把下游量轉回本座標系），兩者互為轉置。
- 誤區三：後向遞迴的力臂用錯。自身慣性力的力臂是質心向量 P_c ，下游傳來的力的力臂是連桿全長向量 P_{i+1} ，混用會造成明顯誤差。
- 誤區四：忘記重力技巧的符號。基座加速度應設為 $-g\vec{}$ （即向上為正），設反了會使重力方向顛倒。
- 誤區五：慣性張量的座標系不一致。慣性張量必須表示在該連桿自身的座標系中，若使用 CAD 輸出值，需確認其參考座標系與 DH 座標系是否一致。
- 誤區六：用顯式歐拉法做長時間模擬。它的能量單調增加，必定發散；應改用半隱式歐拉或其他辛積分器。
- 誤區七：模擬步長取得太大。含接觸或高剛度的系統需要極小步長（0.1 ms 以下），否則會穿透或發散。
- 誤區八：以為模擬中的抓取成功就代表實機可行。接觸模型的簡化（摩擦、變形、接觸面積）是 Sim-to-Real Gap 最頑固的來源。
- 誤區九：只驗證單連桿就相信程式正確。許多遞迴邏輯的錯誤只在三連桿以上才顯現，務必以多連桿與能量守恆雙重驗證。
- 誤區十：忽略複雜度分析。 $O(n)$ 與 $O(n^3)$ 在 $n=6$ 時只差 7 倍，在 $n=20$ 時卻是「能做」與

「不能做」的差別。

課堂討論

1. 開頭故事中小翰推導出一萬四千項的正確公式，叫獸卻說它「沒有價值」。請討論：在什麼情況下，解析式仍然是必要的？（提示：對照第 19 章的穩定性證明與第 13 章的耦合分析）
2. Newton-Euler 法會顯式算出連桿之間的內力，拉格朗日法則自動消去它們。請討論這個「多餘的資訊」在哪些工程場合反而是優點。
3. 重力技巧把重力轉化為基座的加速度，這用到了等效原理。請討論：這個技巧在什麼情況下會失效？（提示：考慮非均勻重力場，或裝在移動平台、太空站上的機器人）
4. 物理引擎採用辛積分器而非高階的 RK4，犧牲了單步精度換取長期穩定。請討論這個取捨在哪些應用中是正確的、哪些應用中反而應該選 RK4。
5. 可微分物理模擬能計算「結果對參數的梯度」，但必須把接觸平滑化。請討論這個近似可能帶來什麼問題，以及它對學習出來的策略在真機上的表現有何影響。

章末摘要

- Newton-Euler 法以牛頓方程 ($F = ma_c$) 與尤拉方程 ($N = I\dot{\omega} + \omega \times I\omega$) 為基礎，後者的陀螺項來自在旋轉座標系中微分角動量。
- 逆動力學以兩趟遞迴完成：前向遞迴由基座往末端傳遞 ω 、 $\dot{\omega}$ 、 a 、 a_c ；後向遞迴由末端往基座累積 f 、 n 。
- 質心加速度含三項：切向 ($\omega \times P$)、向心 ($\omega \times (\omega \times P)$) 與平移（上游傳來的 a ）。
- 重力技巧：把基座線加速度設為 $-g$ ，即可讓重力自動融入遞迴，不需在每根連桿另加重力項。
- 後向遞迴的力矩累積含四項：自身淨力矩、下游力矩、自身慣性力的力矩、下游力的力矩。
- 關節驅動力矩為力矩向量在轉軸上的投影；其餘分量是軸承與結構承受的負荷，是機構設計的重要資訊。
- 遞迴法的複雜度為 $O(n)$ ，遠優於拉格朗日展開的 $O(n^3)$ ； $n=6$ 時差 7 倍， $n=20$ 時則是能否即時的分界。
- 用同一套遞迴程式可萃取各項：設 $\ddot{q}=\dot{q}=0$ 得 $G(q)$ ；設 $\ddot{q}=0$ 且無重力得 $C\dot{q}$ ；設 $\ddot{q}=e_i$ 得 M

的第 i 行 (單位向量法) 。

- 正向動力學有三法：組合法 $O(n^3)$ 、關節體演算法 ABA $O(n)$ 、最大座標法 (自然支援閉環與接觸) 。
- 數值積分中，顯式歐拉能量單調增加必發散；半隱式歐拉是辛積分器，能量有界振盪，是物理引擎的標準選擇。
- 對長時間模擬而言，「不發散」比「單步精確」更重要——這是辛積分器優於 RK4 的理由。
- 接觸是模擬中最困難的部分：不連續、力大時間短、摩擦非線性且可能無唯一解。
- 接觸模型分懲罰法 (簡單、可微、需小步長) 與約束法 (嚴謹、不穿透、成本高) 兩類。
- 模擬抓取常與實機不符，源於摩擦不確定、接觸面積簡化、柔軟性未建模與接觸模型近似；因應之道是領域隨機化。
- 物理引擎含四模組：碰撞偵測、動力學求解、約束求解與數值積分；不同引擎在速度、精確性與可微性上各有取捨。
- 實務分工：用拉格朗日法理解與分析、用 Newton-Euler 法實作與計算，兩者互相驗證。

觀念題與計算題

觀念題

1. 寫出牛頓方程與尤拉方程，並說明尤拉方程中陀螺項 $\omega \times (I\omega)$ 的來源與物理意義。
2. 說明前向遞迴傳遞哪四個量，以及角速度與角加速度的傳遞公式為何不同 (後者多出一項外積) 。
3. 列出質心加速度的三個組成項，並說明各自的物理意義。
4. 說明重力技巧的原理，以及它為何等效於在每根連桿加入重力。
5. 寫出後向遞迴中力矩累積的四項，並說明兩個力臂項的差別。
6. 為何關節驅動力矩只是力矩向量的一個分量？其餘分量有何工程價值？
7. 分析 Newton-Euler 遞迴的複雜度為何是 $O(n)$ ，並與拉格朗日展開比較。
8. 說明如何用同一套遞迴程式分別萃取 $M(q)$ 、 $C(q, \dot{q})\dot{q}$ 與 $G(q)$ 。
9. 比較三種正向動力學演算法的複雜度、座標選擇與閉環支援能力。

10. 為何顯式歐拉法的能量會單調增加？半隱式歐拉只改一處為何就能解決？
11. 說明何謂辛積分器，以及為何物理引擎偏好它而非高階的 RK4。
12. 列舉接觸模擬困難的三個來源，並比較懲罰法與約束法的優缺點。
13. 說明模擬中的抓取為何常與實機不符，並提出因應之道。
14. 列舉物理引擎的四個組成模組，並說明哪一個通常是計算瓶頸。
15. 說明拉格朗日法與 Newton-Euler 法在實務上的分工，並各舉兩個適用場合。

計算題

1. 一根均勻連桿長 0.8 m、質量 3.0 kg，繞固定端在垂直平面轉動。目前 $\theta = 45^\circ$ （由垂直向下量起）、 $\dot{\theta} = 1.5 \text{ rad/s}$ 、 $\ddot{\theta} = 4.0 \text{ rad/s}^2$ 。（a）計算繞質心的轉動慣量。（b）以 Newton-Euler 遞迴計算質心加速度的三個分量。（c）求淨力與淨力矩。（d）求關節力矩，並與拉格朗日公式 $\tau = (1/3)mL^2\ddot{\theta} + mg(L/2)\sin\theta$ 比對。
2. 某控制器頻率為 2 kHz，處理器每秒可執行 2×10^8 次浮點運算。設 Newton-Euler 每自由度需 180 次運算、拉格朗日展開需 $40n^3$ 次。（a）計算 7 軸協作機器人兩種方法的運算時間與週期佔比。（b）計算 30 自由度人形機器人的情形。（c）若模擬需以實時的 50 倍速度執行，30 自由度系統用 Newton-Euler 是否可行？
3. 以單擺（ $L = 0.6 \text{ m}$ 、 $m = 2.5 \text{ kg}$ 、 $I = (1/3)mL^2$ ）由 $\theta = 60^\circ$ 、 $\dot{\theta} = 0$ 靜止釋放，步長 $\Delta t = 0.02 \text{ s}$ 。（a）計算初始角加速度。（b）分別以顯式歐拉與半隱式歐拉計算第一步後的 θ 與 $\dot{\theta}$ 。（c）計算兩者第一步後的總能量，並與初始能量比較。（d）預測長時間模擬後兩者的行為差異。
4. 某六軸手臂的動力學計算需求如下：逆動力學 900 次運算、慣性矩陣（單位向量法）為逆動力學的 n 倍、正向動力學需額外的 Cholesky 分解（約 $n^3/3$ 次運算）。處理器每秒 10^8 次運算。（a）分別計算三者的運算時間。（b）若模擬迴路需同時執行正向動力學與碰撞偵測（假設後者需 $200 \mu\text{s}$ ），求可達的最高模擬頻率。（c）若要達到 1 kHz 的模擬頻率，碰撞偵測的時間預算最多為多少？

動手做與專題延伸

本章實作：動力學計算模組升級

延續第 13 章的動力學模組，加入 Newton-Euler 遞迴實作。這將是第 15 ~ 20 章所有控制

與規劃程式的計算核心。

- **階段一：**實作平面 n 連桿的 Newton-Euler 遞迴逆動力學，並與第 13 章的解析式交叉驗證（2R 與 3R 皆須通過）。
- **階段二：**實作 14.4.3 節的萃取技巧，用同一套遞迴取得 M 、 $C \cdot \dot{q}$ 與 G ，並驗證 $\tau = M\ddot{q} + C\dot{q} + G$ 恆成立。
- **階段三：**實作正向動力學與三種數值積分器，完成能量守恆的 3×3 比較表（三種積分器 $\times$ 三種步長）。
- **階段四：**擴充至空間手臂（含 3×3 慣性張量與完整的座標轉換），並以第 9 章的 DH 參數表為輸入。

專題延伸

- **複雜度的實驗驗證：**為 $n = 2$ 到 30 的平面手臂，實測逆動力學、慣性矩陣與正向動力學的執行時間，繪出對數—對數圖並用線性回歸求出斜率。驗證三者的斜率分別接近 1、2、3（對應 $O(n)$ 、 $O(n^2)$ 、 $O(n^3)$ ）。這個實驗能讓抽象的複雜度分析變成具體的曲線。
- **積分器的相圖比較：**以單擺為對象，用三種積分器各模擬 100 秒，在相平面（ θ 對 $\dot{\theta}$ ）上繪出軌跡。理想的守恆系統應該是封閉曲線；你會看到顯式歐拉的軌跡向外螺旋發散、半隱式歐拉維持接近封閉的曲線、RK4 則緩慢向內收縮。這張圖是理解辛積分器價值的最佳視覺教材。
- **進階：接觸模擬的初探。**實作一個最簡單的接觸情境——單擺末端碰到地面。分別用懲罰法（彈簧阻尼）與速度反轉法（碰撞時速度乘上恢復係數）實作，比較兩者在不同步長下的行為。觀察懲罰法的剛度 k 增大時，穩定所需的步長如何急遽下降——這將讓你親身體會為何含接觸的模擬如此昂貴。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 6 章：Newton-Euler 遞迴的完整推導，本章主要參考)。
- Featherstone, R., Rigid Body Dynamics Algorithms, Springer, 2008 (動力學演算法的權威專著，含 ABA 的完整推導)。
- Luh, J. Y. S., Walker, M. W. and Paul, R. P. C., "On-Line Computational Scheme for

Mechanical Manipulators," ASME J. Dynamic Systems, 1980 (遞迴 Newton-Euler 演算法的原始論文) 。

- Walker, M. W. and Orin, D. E., "Efficient Dynamic Computer Simulation of Robotic Mechanisms," ASME, 1982 (複合剛體演算法 CRBA) 。
- Hairer, E., Lubich, C. and Wanner, G., Geometric Numerical Integration, Springer (辛積分器的數學理論) 。
- Todorov, E., Erez, T. and Tassa, Y., "MuJoCo: A Physics Engine for Model-Based Control," IROS 2012 (MuJoCo 的設計理念與軟接觸模型) 。
- Stewart, D. E., "Rigid-Body Dynamics with Friction and Impact," SIAM Review, 2000 (接觸與摩擦的數學困難 · 含 Painlevé 悖論) 。
- Carpentier, J. et al., "The Pinocchio C++ Library," SII 2019 (高效動力學函式庫的設計與實作) 。
- Freeman, C. D. et al., "Brax: A Differentiable Physics Engine for Large Scale Rigid Body Simulation," 2021 (可微分模擬的代表性工作) 。
- 各物理引擎官方文件：MuJoCo、Bullet、ODE、PhysX、DART 的技術手冊與接觸模型說明。

第 15 章

軌跡生成與運動曲線

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】上半部並列兩組四階曲線圖（位置／速度／加速度／急跳度），左組為梯形速度曲線（加速度呈階梯狀跳變，急跳度出現無限大的紅色脈衝），右組為 S 形七段曲線（加速度連續，急跳度為有限的階梯）。下半部為同一支手臂執行同一段路徑的兩次多重曝光：左側末端在終點處出現抖動殘影並標註「殘餘振動 0.65 秒」，右側平順停止並標註「立即穩定」。標題文字：同樣的路徑，不同的時間安排——慢慢加速，反而更快。

叫獸開講 「到達同一個位置，為什麼有些動作像機器，有些動作像人？」

叫獸在白板上畫了兩條一模一樣的路線：由 A 點到 B 點的一段直線。「這兩條路徑完全相同，」他說，「但我要讓機器人用兩種方式走過去。」

第一次執行：手臂突然啟動、以固定速度衝過去、到達 B 點瞬間停止。末端在停止時劇烈抖動，花了將近一秒才穩下來，示教器上跳出振動警告。

第二次執行：手臂緩緩加速、中段維持高速、接近終點時逐漸減速、平順地停在 B 點。全程沒有任何抖動，而且——總時間比第一次還短了 0.3 秒。

「同樣的路徑、同樣的起訖點，」叫獸說，「但一個又抖又慢，一個又穩又快。差別在哪裡？」

小好舉手：「在於**怎麼走**——什麼時候加速、什麼時候減速。」

「正確。這就是**路徑與軌跡**的差別。」叫獸在白板上寫下兩個詞。「路徑只說『經過哪些點』，是純幾何的問題，第二篇處理過了。軌跡則多了一個東西——**時間**。它說的是『在第幾秒鐘應該在哪裡』。」

他接著問了一個更尖銳的問題：「那為什麼第一次會抖？」

學生們沉默了一下。叫獸自己回答：「因為第一次的速度是瞬間跳到最大值的——這代表加速度是**無限大**。而力矩等於慣量乘上加速度，無限大的加速度需要無限大的力矩。馬達當然給不出來，於是它盡力而為，然後手臂就像被一鎚子敲了一下——**你激發了整個機構的振動**。」

他在白板上寫下四個詞，由上往下：**位置、速度、加速度、急跳度**。「第 13 章我們學了力矩正比於加速度。而加速度的變化率，叫做急跳度（jerk）。急跳度太大，就代表力矩瞬間改變——這會激發機構的彈性振動、傷害減速機、也讓乘客暈車。所以電梯、高鐵、機器手臂的運動曲線，都是為了控制急跳度而設計的。」

小翰問：「那為什麼第二次反而更快？」

「因為第一次的加速度超過馬達能力，控制器只好自動降速保護。」叫獸敲了敲白板，「這正是本章的核心：**軌跡規劃不只是幾何問題，它必須知道機器人的動力學極限**。你不能規劃一條馬達跑不動的軌跡。今天我們就來學，怎麼設計一條又快、又穩、又不超過極限的軌跡。」

本章為什麼重要

軌跡生成是連接「規劃」與「控制」的橋樑。第二篇告訴我們機器人能到哪裡、第 13 ~ 14 章告訴我們動起來需要多少力矩，而本章要決定的是：**在每一個時刻，機器人應該在哪裡、以多快的速度、多大的加速度運動。**

這個決定的品質，直接影響四件事。**節拍時間**：軌跡設計得好，同樣的路徑可以縮短數十個百分點的時間——這對每天執行數萬次的產線意義重大。**精度**：平滑的軌跡讓控制器容易追隨，粗糙的軌跡則會產生追隨誤差。**機構壽命**：頻繁的力矩突變會加速減速機與軸承的磨損。**振動**：如開頭故事所示，不當的軌跡會激發機構共振，使末端在停止後仍抖動不止。

本章也是第 13、14 章的第一個實際應用：規劃軌跡時，我們會用逆動力學檢驗「這條軌跡所需的力矩是否超過馬達能力」，並據此調整時間分配。這正是「模型」開始為「決策」服務的起點。

學習目標

- 能區分路徑與軌跡，並說明時間參數化的意義。
- 能區分關節空間與任務空間的軌跡規劃，並說明各自的優缺點。
- 能推導三次與五次多項式軌跡的係數，並說明邊界條件的作用。
- 能設計梯形速度曲線，並計算其時間分配與可行性條件。
- 能說明 S 形曲線（含急跳度限制）的優勢與代價。
- 能說明急跳度對振動、機構壽命與舒適度的影響。
- 能規劃通過多個路徑點的連續軌跡，並處理速度連續性。
- 能規劃笛卡兒空間的直線與圓弧軌跡。
- 能運用四元數 SLERP 規劃姿態軌跡。
- 能運用逆動力學檢驗軌跡的動力學可行性並進行時間縮放。
- 能以 Python 實作各類軌跡並比較其速度、加速度與急跳度曲線。

先備知識

第 9 ~ 10 章的正逆運動學（任務空間軌跡需要逐點求逆解）、第 11 章的雅可比（速度的轉

換與極限分析)、第 13~14 章的動力學(力矩可行性檢驗)、第 8 章的四元數(姿態內插)。

數學方面需要多項式、微分與積分的基本運算。本章的推導不難，但需要細心處理邊界條件——大部分的錯誤都來自於此。

核心名詞中英對照

表 15-1 本章核心名詞

中文	英文	說明
路徑	Path	空間中的幾何曲線，不含時間資訊
軌跡	Trajectory	含時間參數化的路徑
時間參數化	Time Parameterization	為路徑指定各點的通過時刻
路徑點	Via Point / Waypoint	軌跡必須通過或接近的中間點
急跳度	Jerk	加速度的時間導數
梯形速度曲線	Trapezoidal Velocity Profile	等加速—等速—等減速的速度曲線
S 形曲線	S-curve	加速度連續變化的平滑速度曲線
混成	Blending	在路徑點附近以曲線平順銜接
球面線性內插	SLERP	四元數之間的平滑姿態內插
時間最佳軌跡	Time-optimal Trajectory	在限制下用時最短的軌跡
時間縮放	Time Scaling	調整整體執行時間以滿足限制
殘餘振動	Residual Vibration	運動停止後仍持續的振動
輸入整形	Input Shaping	以延遲疊加抵消振動的技術
路徑點混成半徑	Blending Radius	在路徑點處偏離的容許距離
笛卡兒空間	Cartesian Space	以末端位姿描述的空間

15.1 路徑與軌跡

15.1.1 多了一個維度：時間

如開頭故事所述，路徑與軌跡的差別在於時間。

路徑 (path) 是空間中的一條幾何曲線，通常以一個從 0 到 1 的參數 s 描述： $p(s)$ 。它只回答「經過哪些點、順序為何」，不涉及速度。第 26 章的路徑規劃演算法 (A^* 、RRT) 產生的就是路徑。

軌跡 (trajectory) 則把時間加進來： $p(t)$ 。它回答「在第 t 秒鐘應該在哪裡」。有了時間，速度與加速度就自然定義了：

$$\text{速度 } v(t) = dp/dt, \quad \text{加速度 } a(t) = d^2p/dt^2, \quad \text{急跳度 } j(t) = d^3p/dt^3$$

軌跡的各階導數。控制器需要位置與速度作為追隨目標，動力學則需要加速度來計算力矩。

由路徑產生軌跡的過程稱為**時間參數化**：給定 $p(s)$ ，選擇一個函數 $s(t)$ ，使得 $p(t) = p(s(t))$ 。這個選擇有無限多種可能，而本章的核心正是「如何選得好」。

15.1.2 好軌跡的五個標準

什麼叫「好的軌跡」？工程上有五個標準，它們常常互相衝突。

- **平滑**：位置、速度、加速度應連續。加速度不連續代表力矩突變，會激發振動。
- **快速**：在滿足限制的前提下，總時間越短越好——這直接決定產線的節拍。
- **可行**：不超過關節的位置、速度、加速度與力矩極限。這是硬性條件。
- **精確**：末端應準確通過指定的路徑點，或在容許誤差內。
- **可預測**：軌跡應能事先完整計算，以便驗證與安全檢查。

「快速」與「平滑」的衝突最為明顯：要縮短時間就得加大加速度，而加大加速度就會增加力矩突變。整章的技術，本質上都是在這個張力之間尋找最佳點。

15.1.3 三個層次的規劃

完整的運動規劃通常分三層執行：

第一層：路徑規劃（第 26 章）。決定要經過哪些點，避開障礙物。輸出是一串路徑點或一條幾何曲線。

第二層：軌跡生成（本章）。為路徑加上時間，產生連續的位置、速度與加速度指令。

第三層：軌跡追隨（第 16~17 章）。控制器實際驅動馬達，使機器人盡可能貼合規劃的軌跡。

三層各司其職但互相影響：路徑若有尖角，軌跡就必須在該處減速；軌跡若超出動力學極限，控制器就無法追隨。本章的內容處於中間層，必須同時考慮上下兩層的需求。

15.2 關節空間與任務空間

15.2.1 兩種規劃方式

軌跡可以在兩個不同的空間中規劃，這是一個必須及早決定的關鍵選擇。

關節空間規劃：直接為每個關節角度設計時間函數 $q_i(t)$ 。作法是先用逆運動學求出起點與終點的關節角度，再在關節空間中內插。

任務空間規劃：先為末端位姿設計軌跡 $x(t)$ ，再逐點用逆運動學求出對應的關節角度。

15.2.2 各自的優缺點

關節空間規劃的優點：計算量小（不需逐點求逆解）、必定平滑（多項式在關節空間中直接定義）、不會遇到奇異點問題（因為完全不做逆運動學）、且不可能超出關節極限（可直接在關節空間設限）。

關節空間規劃的缺點：末端路徑不可預測。即使起訖點都正確，中間過程末端會走出一條奇怪的曲線——可能撞到障礙物、可能離開工作區域。這是它最大的問題。

任務空間規劃的優點：末端路徑完全可控（可以精確畫直線、圓弧），這對焊接、塗膠、切割等連續路徑作業是必要條件。

任務空間規劃的缺點：需逐點求逆運動學（計算量大）、可能在中途遇到奇異點（第 12 章）、可能因路徑超出工作空間而無解、且關節速度可能在某些姿態下暴增。

表 15-2 關節空間與任務空間規劃的比較

面向	關節空間	任務空間
規劃對象	各關節角度 $q(t)$	末端位姿 $x(t)$
計算量	小（直接內插）	大（逐點逆運動學）
末端路徑	不可預測（曲線）	完全可控（直線、圓弧）
奇異點	不受影響	可能遇到，需處理
關節極限	容易保證	需逐點檢查
典型應用	點對點取放、避開障礙的大範圍移動	焊接、塗膠、切割、裝配
工業指令	MOVJ（關節運動）	MOVL / MOVC（直線 / 圓弧運動）

15.2.3 實務上的選擇

工業機器人的程式中，兩種指令並存且各有明確用途：**MOVJ (關節運動)** 用於「從 A 大範圍移動到 B，中間怎麼走無所謂」的場合，因為它最快也最安全；**MOVL (直線運動)** 用於「必須沿直線」的場合，如焊縫、膠道。

一個典型的取放循環會混用兩者：由待機點 **MOVJ** 到取料點上方（快速移動）、**MOVL** 垂直下降取料（精確直線）、**MOVJ** 移動到放料點上方（快速移動）、**MOVL** 垂直下降放料。這樣既保證了必要的精度，又不浪費時間在不必要的直線運動上。

◎ 關節空間規劃的意外路徑

一個常見的教學現場事故：學生用關節空間指令讓手臂從桌面左側移動到右側，起訖點都在桌面上方安全高度。執行時手臂卻**穿過桌面**——因為中間過程末端沿著一條下凹的曲線移動。這說明了關節空間規劃的風險：起訖點安全不代表過程安全。因應之道是加入中間路徑點，或改用任務空間規劃並做碰撞檢查。

15.3 多項式軌跡

15.3.1 三次多項式

最基本的軌跡形式是多項式。考慮單一關節由 q_0 移動到 q_f ，總時間為 T 。若要求起訖點的位置與速度都指定，共有四個邊界條件，因此需要四個係數——即三次多項式：

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

三次多項式軌跡。

邊界條件為： $q(0) = q_0$ 、 $\dot{q}(0) = 0$ 、 $q(T) = q_f$ 、 $\dot{q}(T) = 0$ （假設起訖皆靜止）。代入求解可得：

$$a_0 = q_0, \quad a_1 = 0, \quad a_2 = 3(q_f - q_0)/T^2, \quad a_3 = -2(q_f - q_0)/T^3$$

三次多項式的係數。

其速度與加速度為：

$$\dot{q}(t) = 6(q_f - q_0)(t/T^2)(1 - t/T) \quad \ddot{q}(t) = 6(q_f - q_0)(1 - 2t/T)/T^2$$

速度呈拋物線（中點最大）、加速度呈直線（由正變負）。

關鍵觀察：最大速度發生在中點，其值為：

$$v_{\max} = 1.5 \times (q_f - q_0)/T$$

最大速度是平均速度的 1.5 倍。這個係數在檢驗速度極限時很有用。

最大加速度發生在起點與終點，其值為 $6(q_f - q_0)/T^2$ 。注意加速度在 $t=0$ 時由零瞬間跳到最大值——這代表急跳度無限大，會激發振動。這是三次多項式的主要缺點。

15.3.2 五次多項式

要解決加速度不連續的問題，可以再增加兩個邊界條件： $\ddot{q}(0) = 0$ 與 $\ddot{q}(T) = 0$ 。共六個條件，需要六個係數，即五次多項式：

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

五次多項式軌跡。

在起訖皆靜止且加速度為零的條件下，係數為：

$$a_0 = q_0, a_1 = a_2 = 0, a_3 = 10\Delta q/T^3, a_4 = -15\Delta q/T^4, a_5 = 6\Delta q/T^5 \quad (\Delta q = q_f - q_0)$$

五次多項式的係數。

其最大速度與最大加速度為：

$$v_{\max} = 1.875 \Delta q/T \quad (\text{在中點}) \quad a_{\max} = 5.774 \Delta q/T^2$$

五次多項式的極值。相較三次多項式，速度峰值高了 25%、加速度峰值也較高。

取捨：五次多項式的加速度連續（急跳度有限），因此運動更平順、振動更小；代價是同樣的時間內需要更高的峰值速度與加速度。若馬達能力足夠，五次多項式是較好的選擇；若已接近極限，則需延長時間或改用其他曲線。

圖 15-1 三次與五次多項式軌跡的比較

建議配圖：四列子圖並排比較兩種多項式。第一列為位置（兩者皆為 S 形，五次略陡）；第二列為速度（三次為拋物線、五次為鐘形，峰值較高）；第三列為加速度（三次為直線且在 $t=0$ 與 $t=T$ 處有跳變，五次為連續的正弦狀曲線，兩端為零）；第四列為急跳度（三次為常數但在端點有無限大脈衝，五次為連續曲線）。在三次多項式的加速度跳變處以紅色圈標註「振動來源」。

15.4 梯形速度曲線

15.4.1 為何工業界偏好梯形

多項式軌跡在整段時間中速度都在變化，只有一瞬間達到最大速度。這在時間效率上並不理想——馬達大部分時間都沒有全力運轉。

梯形速度曲線則不然：它以固定加速度加速到最大速度、維持一段等速、再以固定減速度停止。速度曲線呈梯形，因而得名。它的優勢是在滿足相同的速度與加速度極限下，**用時最短**——這是時間最佳控制的直接結果（第 18 章會說明其理論根據）。

15.4.2 時間分配計算

設總位移為 Δq 、最大速度為 $v_{\max}$ 、最大加速度為 $a_{\max}$ 。先計算加速段所需的時間與位移：

$$t_a = v_{\max} / a_{\max}, \quad \Delta q_a = \frac{1}{2} a_{\max} t_a^2 = v_{\max}^2 / (2 a_{\max})$$

加速段的時間與位移。減速段相同。

若總位移足夠長 ($\Delta q \geq 2\Delta q_a$)，則存在等速段：

$$t_c = (\Delta q - 2\Delta q_a) / v_{\max} \quad \text{總時間 } T = 2t_a + t_c = \Delta q / v_{\max} + v_{\max} / a_{\max}$$

有等速段的情形。總時間的公式很優雅：位移除以最大速度，再加上一個固定的「加減速代價」。

15.4.3 三角形退化情形

若位移太短 ($\Delta q < 2\Delta q_a$)，機器人還沒加速到 $v_{\max}$ 就必須開始減速。此時等速段消失，速度曲線退化為三角形：

$$v_{\text{peak}} = \sqrt{(a_{\max} \cdot \Delta q)} \quad T = 2\sqrt{(\Delta q / a_{\max})}$$

三角形情形。實際達到的峰值速度低於 $v_{\max}$ 。

判斷條件為：

$$\text{若 } \Delta q \geq v_{\max}^2 / a_{\max} \rightarrow \text{梯形} ; \text{否則} \rightarrow \text{三角形}$$

判斷準則。程式中必須先做此判斷，否則會算出負的等速時間。

實務意義：短距離移動時，機器人根本達不到額定的最高速度。這解釋了一個常見現象——把機器人的速度參數從 80% 調到 100%，短距離的取放動作卻幾乎沒有變快。因為瓶頸在加速度而非速度，此時該調的是加速度參數。

15.4.4 梯形曲線的缺點

梯形曲線的加速度是階梯狀的：在 $t=0$ 瞬間由 0 跳到 $a_{\max}$ 、在 t_a 瞬間由 $a_{\max}$ 跳到 0、在減速起點再跳到 $-a_{\max}$ 。每一次跳變都是無限大的急跳度。

這正是開頭故事中第一次執行會抖動的原因（該案例更極端，連加速段都沒有）。因此高性能的控制器多半採用下一節的 S 形曲線，在梯形的基礎上把每個轉角「磨圓」。

15.5 S 形曲線與急跳度限制

15.5.1 把梯形的角磨圓

S 形曲線（S-curve）的作法是在梯形速度曲線的每個轉折處，加入一段加速度線性變化的過渡。這使加速度成為連續函數，急跳度成為有限的階梯狀函數。

完整的 S 形曲線有七個階段：加加速（jerk 為正）、等加速、減加速（jerk 為負）、等速、加減速、等減速、減減速。因此又稱為**七段式速度曲線**。

急跳度： $j(t) \in \{+j_{\max}, 0, -j_{\max}\}$

加速度： $a(t)$ 為連續的梯形

速度： $v(t)$ 為連續且平滑的 S 形

三階曲線的層級關係。每往下積分一次，曲線就平滑一階。

15.5.2 時間分配

S 形曲線的時間計算比梯形複雜，因為要處理更多的退化情形（可能沒有等速段、可能沒有等加速段、甚至兩者都沒有）。基本的三個時間參數為：

$t_j = a_{\max} / j_{\max}$ （急跳段時間）

$t_a = v_{\max} / a_{\max} + t_j$ （完整加速段時間，含兩段急跳）

$T = \Delta q / v_{\max} + v_{\max} / a_{\max} + a_{\max} / j_{\max}$ （總時間，完整七段時）

S 形曲線的時間分配。注意總時間比梯形多出 $a_{\max}/j_{\max}$ 這一項——這就是平滑的代價。

代價的量化：若 $a_{\max} = 5 \text{ m/s}^2$ 、 $j_{\max} = 50 \text{ m/s}^3$ ，則多出的時間為 0.1 秒。對一個原本 1 秒的動作而言，這是 10% 的節拍損失。但換來的是振動大幅降低——實務上往往因為不必等待振動衰減，**總體反而更快**（如開頭故事所示）。

15.5.3 急跳度為何重要

急跳度雖然是三階導數，聽起來很抽象，但它有四個非常具體的工程意義。

意義一：力矩的變化率。由 $\tau = M \ddot{q}$ ，力矩正比於加速度；因此急跳度正比於力矩的變化率。力矩瞬間改變會對減速機的齒面、軸承與聯軸器產生衝擊負荷，加速磨損。

意義二：激發結構振動。任何機構都有其自然頻率（第 13 章例題 13-1 算過單擺的自然頻率）。加速度的階梯狀跳變在頻域上含有寬廣的頻率成分，必定包含機構的共振頻率，因而激發振動。這正是開頭故事中「像被鎚子敲了一下」的物理解釋。

意義三：乘坐舒適度。人體對加速度的變化特別敏感。電梯與高鐵的加減速曲線都經過精心設計以限制急跳度——國際標準建議電梯的急跳度不超過約 1.5 m/s^3 。

意義四：追隨精度。控制器需要時間反應。急跳度大代表指令變化快，控制器來不及跟上，追隨誤差因而增加。

15.5.4 殘餘振動與輸入整形

即使限制了急跳度，機構的振動仍無法完全消除。有一個巧妙的技術可以進一步抑制：**輸入整形**（input shaping）。

它的原理是：既然一個脈衝會激發振動，那麼再送一個**時間錯開半個振動週期**的脈衝，兩者激發的振動恰好反相而互相抵消。具體作法是把原始的軌跡指令與一組脈衝序列做卷積：

最簡單的 ZV 整形器：在 $t=0$ 送出 A_1 、在 $t = T_d/2$ 送出 A_2 （ T_d 為阻尼振動週期）

ZV (Zero Vibration) 整形器。代價是總時間增加半個振動週期。

輸入整形在半導體設備、晶圓搬運與大型龍門機構中應用廣泛，因為這些系統的結構柔性大、振動問題嚴重。它的限制是需要準確知道系統的自然頻率與阻尼比——頻率估計誤差超過約 10% 時效果就會明顯下降。

表 15-3 四種軌跡形式的比較

形式	加速度	急跳度	時間效率	振動	適用場合
三次多項式	線性、端點跳變	端點無限大	中	明顯	教學、低速動作
五次多項式	連續、端點為零	有限但不受控	較差	小	需平滑起停的動作
梯形速度	階梯狀跳變	跳變處無限大	最佳	明顯	一般工業運動、對時間敏感
S 形（七段）	連續梯形	有限且可設限	略差於梯形	最小	高速高精度、柔性機構

15.6 多路徑點與混成

15.6.1 通過多個點的問題

實際的作業往往需要通過一連串路徑點。最單純的作法是把每一段都當作獨立的點對點運動，在每個路徑點停止再啟動。這樣做正確但極沒效率——每次停止再加速都浪費時間，而且會產生大量的加減速衝擊。

因此需要**連續通過**：機器人在路徑點處不停止，而是以一定速度直接通過。這帶來一個新問題：在路徑點處的速度該設多少？

15.6.2 路徑點速度的決定

三種常見的策略：

策略一：使用者指定。由程式明確指定每個路徑點的速度。最靈活但最麻煩。

策略二：啟發式規則。若路徑點前後的運動方向相同，取兩段平均速度；若方向相反（轉折），則設為零。這是工業控制器常見的預設行為。

若 $\text{sign}(\Delta q_{\text{前}}) = \text{sign}(\Delta q_{\text{後}})$: $v_k = (v_{\text{前}} + v_{\text{後}})/2$; 否則 $v_k = 0$

常見的啟發式規則。避免在轉折處產生不合理的速度。

策略三：整體最佳化。把所有路徑點的速度視為未知數，以「總時間最短」或「加速度平方積分最小」為目標求解。這能得到最佳結果，但計算量大，通常用於離線編程。

15.6.3 混成：不精確通過反而更好

有一個重要的實務技巧：**允許機器人不精確通過路徑點**。

在轉折處，若要求精確通過，機器人必須減速到零再轉向（因為速度方向要瞬間改變，需要無限大的加速度）。但若允許它在路徑點附近「切彎」——沿一段圓弧或多項式曲線平順轉向——就能維持較高的速度通過。

這個容許的偏離距離稱為**混成半徑**（blending radius），在工業機器人程式中通常以參數指定（如 ABB 的 zone data、FANUC 的 CNT 值）。

混成半徑越大 → 速度越快、路徑偏離越多

混成半徑為零 → 精確通過但必須停止

混成半徑是速度與精度的直接取舍參數。

實務原則：在自由空間的移動段（避開障礙即可），使用大的混成半徑以提高速度；在作業段（如焊縫、裝配路徑），使用零混成以保證精度。一個典型的取放程式，可能在移動段用 100

mm 的混成、在取放點用精確定位。

圖 15-2 路徑點混成的效果

建議配圖：俯視圖，三個路徑點 $P1$ 、 $P2$ 、 $P3$ 呈直角轉折。左圖為零混成：路徑精確通過 $P2$ ，但速度曲線顯示在 $P2$ 處降為零再重新加速，總時間較長。右圖為有混成半徑 r ：路徑在 $P2$ 附近以圓弧切過（以虛線圖標示混成區域），末端與 $P2$ 的最大偏離為 r ，但速度曲線維持在較高水準，總時間顯著縮短。下方以數字標註兩者的時間差異。

15.7 笛卡兒空間軌跡

15.7.1 直線運動

任務空間中最基本的軌跡是直線。給定起訖位置 p_0 與 p_f ，路徑為：

$$p(s) = p_0 + s(p_f - p_0), \quad s \in [0, 1]$$

直線路徑的參數化。

接著用前面任一種時間曲線為 s 賦予時間： $s(t)$ 可以是三次多項式、五次多項式、梯形或 S 形。這種「幾何路徑 + 時間曲線」的分離設計極為實用——換一種時間曲線不需重新設計路徑。

直線運動的執行需要逐點逆運動學。實務上通常以固定的插補週期（如每 $1 \sim 4$ ms）計算一個點，求逆解後送給關節控制器。這意味著一秒鐘要執行數百次逆運動學——這正是第 10 章強調封閉解重要性的原因。

15.7.2 圓弧運動

圓弧運動需要三個點來定義：起點、中間點與終點（三點決定一個圓）。控制器由這三點計算出圓心與半徑，再沿圓弧內插。

圓弧運動的一個重要特性是向心加速度：

$$a_c = v^2 / R$$

沿圓弧運動時，即使速度大小不變，仍有指向圓心的加速度。半徑越小、速度越快，向心加速度越大。

這帶來一個實務限制：小半徑的圓弧必須減速。若不減速，向心加速度會超過機構或馬達的能力。多數控制器會自動依此公式限制圓弧段的速度——這也解釋了為何機器人在走小圓弧時明顯變慢。

15.7.3 姿態軌跡與 SLERP

末端不只有位置，還有姿態。姿態的內插不能用線性方式——如第 8 章所述，對旋轉矩陣的元素做線性內插，結果不再是旋轉矩陣；對歐拉角做線性內插，路徑會扭曲且可能經過萬向節鎖。

正確的作法是四元數球面線性內插 (SLERP)：

$$q(s) = [\sin((1-s)\Omega) \cdot q_0 + \sin(s\Omega) \cdot q_f] / \sin \Omega, \quad \text{其中 } \cos \Omega = q_0 \cdot q_f$$

SLERP 公式。它產生的是繞固定軸的等速旋轉——兩個姿態之間最自然的路徑。

實作上有兩個要點。**要點一：處理雙重覆蓋。**由於 q 與 $-q$ 代表同一姿態（第 8.5.1 節），若 $q_0 \cdot q_f < 0$ ，應先將 q_f 取負號，否則會走「遠路」（繞超過 180 度）。**要點二：小角度時改用線性內插。**當 Ω 很小時 $\sin \Omega \rightarrow 0$ ，公式會數值不穩定，此時用線性內插再正規化即可。

15.7.4 位置與姿態的時間協調

一個容易被忽略的細節：位置與姿態應使用**同一個時間參數** $s(t)$ ，才能保證兩者同步完成。若各自使用不同的時間曲線，可能出現「位置已到達但姿態還在轉」的情形，導致末端在終點附近做出奇怪的動作。

但有時這是刻意的設計。例如在噴塗作業中，可能希望姿態提前調整到位、位置再緩慢通過——此時就需要為兩者設計不同的時間曲線。這種情形應在程式中明確標註，避免誤解。

15.8 動力學可行性與時間縮放

15.8.1 軌跡必須通過動力學檢驗

到目前為止，我們規劃軌跡時只考慮了運動學極限（速度、加速度）。但真正的限制是**力矩**——而力矩與加速度的關係並非簡單的比例，它取決於姿態（第 13 章的 $M(q)$ 隨姿態變化）與速度（科氏力與速度平方成正比）。

因此完整的軌跡規劃必須包含一個檢驗步驟：對規劃好的軌跡，逐點用**逆動力學**計算所需力矩，確認不超過馬達能力。

$$\text{對每個時刻 } t: \quad \tau(t) = M(q(t))\ddot{q}(t) + C(q(t), \dot{q}(t))\dot{q}(t) + G(q(t))$$

$$\text{檢驗: } |\tau_i(t)| \leq \tau_{\max,i} \quad \text{對所有 } i \text{ 與 } t \text{ 成立}$$

動力學可行性檢驗。這正是第 14 章的 $O(n)$ 遞迴演算法派上用場的地方。

15.8.2 時間縮放：優雅的修正方法

若檢驗失敗（某處力矩超限），最直接的修正方法是**時間縮放**：把整條軌跡的執行時間拉長

k 倍。

時間縮放有一個優美的數學性質。設原軌跡為 $q(t)$ ，縮放後為 $\tilde{q}(t) = q(t/k)$ 。則：

$$\dot{\tilde{q}} = \dot{q}/k, \quad \ddot{\tilde{q}} = \ddot{q}/k^2$$

速度按 $1/k$ 縮放、加速度按 $1/k^2$ 縮放。

因此動力學方程中各項的縮放行為為：慣性項 $M \ddot{q}$ 按 $1/k^2$ 縮放、科氏項 $C \dot{q}$ 也按 $1/k^2$ (因為含速度平方)、但重力項 G 完全不變 (它與速度無關)。

$$\tilde{\tau} = (1/k^2)(M \ddot{\tilde{q}} + C \dot{\tilde{q}}) + G$$

時間縮放後的力矩。這是本節最重要的關係式。

這個結果有兩個重要的推論。**推論一**：把時間拉長一倍 ($k = 2$)，動態部分的力矩降為四分之一——時間縮放對降低力矩極為有效。**推論二**：無論怎麼放慢，**重力項都無法降低**。若重力所需的力矩就已超過馬達能力，任何時間縮放都救不了，只能減輕負載或更換馬達。

15.8.3 時間最佳軌跡

既然時間縮放能保證可行，一個自然的問題是：在滿足所有限制的前提下，最短的時間是多少？

這稱為**時間最佳路徑參數化** (Time-Optimal Path Parameterization, TOPP)。它的核心想法是：沿著給定的路徑，在每一點計算「在此處最大可容許的速度與加速度」(受力矩極限、向心加速度、關節速度等限制)，再用類似「相平面分析」的方法找出最快的通過方式。

時間最佳解有一個典型特徵：在每一個時刻，至少有一個關節處於其極限 (力矩飽和或速度飽和)。這稱為 **bang-bang** 特性，與第 18 章的最佳控制理論一致。若你的軌跡在整段時間中都沒有任何一個關節達到極限，那它一定不是時間最佳的——還有加速的空間。

◎ 為何實務上很少用時間最佳軌跡

時間最佳軌跡在理論上很吸引人，但實務應用有限，原因有三。一、**不夠平滑**：bang-bang 的力矩切換會產生大的急跳度，激發振動。二、**沒有餘裕**：所有關節都在極限運轉，任何模型誤差或外部干擾都會導致追隨失敗。三、**模型依賴**：需要精確的動力學參數，而現實中它們有 10~20% 的不確定性。因此實務上多採「次佳但穩健」的作法：用 80~90% 的極限值規劃 S 形曲線，保留餘裕給控制器與不確定性。

15.9 多軸協調與同步

15.9.1 最慢的關節決定一切

多軸運動時有一個基本問題：各關節的移動量不同、能力也不同，若各自以最快速度執行，它們會在不同時刻到達終點——這會使末端走出不可預測的路徑。

正確的作法是**同步**：讓所有關節同時開始、同時結束。作法是先計算每個關節「單獨執行時所需的最短時間」，取其中最大者作為共同的執行時間，再讓其餘關節放慢配合。

$$T = \max_i (T_i), \text{ 其中 } T_i \text{ 為關節 } i \text{ 的最短可行時間}$$

同步時間。決定總時間的那個關節稱為主導軸 (dominant axis)。

實務意義：這解釋了為何「調快某一軸的速度參數」往往沒有效果——若它不是主導軸，調快也沒用。要縮短節拍，必須先找出主導軸，再針對它最佳化（減少該軸的移動量、或提高該軸的加減速能力）。

15.9.2 三種同步等級

工業控制器通常提供三種同步等級，精細程度不同：

無同步：各軸獨立執行，最快但路徑不可預測。已很少使用。

終點同步：所有軸同時結束，但中間過程的速度比例不固定。實作簡單，是關節運動 (MOVJ) 的標準行為。

全程同步：所有軸的位置在整個過程中維持固定比例（即所有軸的正規化時間曲線完全相同）。這使關節空間中的路徑成為一條直線，末端路徑雖仍是曲線但完全可預測。多數現代控制器採此作法。

15.9.3 外部軸與多機協同

當系統包含外部軸（如第 4 章提到的第七軸行走軸、旋轉工作台）或多台機器人時，同步問題更為複雜。此時需要一個共同的時間基準，並確保所有設備的插補週期對齊。

這正是第 3 章提到的 EtherCAT 等現場匯流排的價值所在——它們提供微秒級的**分散式時鐘同步**，使多個驅動器能在同一時刻執行指令。若同步誤差達到毫秒級，高速協同作業（如兩台機器人共同搬運一個大工件）就會產生內應力甚至損壞工件。

15.10 實務上的軌跡規劃流程

15.10.1 完整的七個步驟

整合本章內容，一個完整的軌跡規劃流程如下：

1. **取得路徑**：由使用者示教、離線編程或路徑規劃演算法（第 26 章）取得一串路徑點。
2. **選擇空間**：依任務需求決定各段使用關節空間或任務空間（15.2.3 節）。
3. **設定混成**：為每個路徑點指定混成半徑——作業段設零、移動段設大值（15.6.3 節）。
4. **時間參數化**：選擇時間曲線（梯形或 S 形），依速度與加速度極限計算各段時間。
5. **多軸同步**：計算各軸所需時間，取最大值統一，找出主導軸（15.9.1 節）。
6. **動力學檢驗**：逐點計算逆動力學，確認力矩不超限；若超限則時間縮放並回到步驟 5（15.8 節）。
7. **輸出指令**：以固定的插補週期（1~4 ms）產生位置、速度、加速度序列，送給控制器。

步驟 6 與 5 之間的迭代是關鍵。實務上通常從一個保守的時間估計開始，逐步縮短直到某處觸及極限，這稱為**時間最佳化迴圈**。

15.10.2 線上與離線規劃

離線規劃：在執行前完整計算整條軌跡。優點是可以做完整的檢驗（碰撞、力矩、奇異點）、可以做全域最佳化。缺點是無法應對環境變化。適用於固定的產線作業。

線上規劃：邊執行邊計算，通常只規劃未來數十毫秒。優點是能即時反應（如視覺伺服追蹤移動的物件、或人突然接近時減速）。缺點是無法做全域最佳化，且必須保證計算能在控制週期內完成。

現代的協作機器人多採**混合式**：離線規劃出名目軌跡，執行時再依感測資訊做線上調整（如偵測到人接近就即時做時間縮放降速）。這種「離線骨架 + 線上微調」的架構，兼顧了最佳性與反應性。

15.11 由軌跡邁向控制

15.11.1 本章的成果

本章完成了「規劃」到「控制」之間的橋樑。你現在能夠：區分路徑與軌跡、在關節空間與

任務空間之間做出正確選擇、設計多項式與梯形 / S 形速度曲線、規劃通過多個路徑點的連續軌跡與笛卡兒直線圓弧、用 SLERP 內插姿態、以逆動力學檢驗可行性並用時間縮放修正、以及處理多軸同步。

更重要的是，你理解了幾個貫穿全章的取舍：**快速與平滑的張力**（急跳度限制的代價）、**精度與速度的取舍**（混成半徑）、以及**最佳與穩健的權衡**（為何實務不用時間最佳軌跡）。這些取舍的判斷力，是工程師與程式產生器的差別所在。

15.11.2 接下來：讓機器人真的走上這條軌跡

但軌跡只是**指令**。規劃出完美的軌跡，不代表機器人就會照著走——馬達有摩擦、機構有彈性、負載會變化、外界有干擾。把指令變成實際運動，是**控制**的工作。

第 16 章（線性回授與 PID）：從最基本的控制器出發。我們將分析一維點質量的 PD 與 PID 控制，理解比例、微分、積分三項各自的作用，以及為何增益不能無限調高。這一章會解釋第 13 章開頭故事中「調高增益」為何治標不治本。

第 17 章（非線性控制）：把第 13、14 章的動力學模型加入控制器。計算力矩控制（computed torque control）用逆動力學做前饋，把非線性的機器人「線性化」，使 PID 得以有效運作。這正是第 13 章叫獸提到的解方。

第 18~19 章：用最佳控制與 Lyapunov 理論，設計更好的控制器並嚴格證明其穩定性。

換句話說：第 15 章決定**該走哪條路**，第 16 章之後決定**如何真的走上去**。

完整計算例題

例題 15-1 多項式軌跡的設計與檢驗

題目：某關節需由 0° 移動到 90° ，時間 $T = 2.0$ 秒，起訖皆靜止。關節的速度極限為 $60^\circ/\text{s}$ 、加速度極限為 $120^\circ/\text{s}^2$ 。（a）設計三次多項式軌跡並求其最大速度與加速度。（b）改用五次多項式，重新計算。（c）判斷兩者是否滿足極限。（d）若五次多項式不可行，求所需的最短時間。

解（a）： $\Delta q = 90^\circ$ ， $T = 2.0 \text{ s}$ 。由 15.3.1 節：

$$v_{\max} = 1.5 \Delta q / T = 1.5 \times 90 / 2.0 = 67.5^\circ/\text{s}$$

三次多項式的最大速度。

$$a_{\max} = 6 \Delta q/T^2 = 6 \times 90/4.0 = 135 \text{ }^\circ/\text{s}^2$$

三次多項式的最大加速度 (發生在起訖點)。

解 (b) : 由 15.3.2 節 :

$$v_{\max} = 1.875 \Delta q/T = 1.875 \times 90/2.0 = 84.4 \text{ }^\circ/\text{s}$$

五次多項式的最大速度。

$$a_{\max} = 5.774 \Delta q/T^2 = 5.774 \times 90/4.0 = 129.9 \text{ }^\circ/\text{s}^2$$

五次多項式的最大加速度 (發生在約 $t = 0.21T$ 與 $0.79T$ 處)。

解 (c) : 三次多項式 : 速度 $67.5 > 60$ (超限 12.5%)、加速度 $135 > 120$ (超限 12.5%)。

不可行。

五次多項式 : 速度 $84.4 > 60$ (超限 40.7%)、加速度 $129.9 > 120$ (超限 8.3%)。更不可行。

這個結果值得注意 : 五次多項式雖然更平滑 , 但峰值速度高出 25% , 反而更容易超出速度極限。平滑不是免費的。

解 (d) : 由時間縮放的關係 , 速度按 $1/k$ 、加速度按 $1/k^2$ 縮放。分別求兩個限制所需的縮放倍數 :

$$\text{速度限制 : } k_v = 84.4/60 = 1.407$$

$$\text{加速度限制 : } k_a = \sqrt{129.9/120} = \sqrt{1.0825} = 1.040$$

兩個限制各自要求的縮放倍數。

取較大者 $k = 1.407$ (速度為主導限制) :

$$T_{\min} = 1.407 \times 2.0 = 2.81 \text{ 秒}$$

五次多項式的最短可行時間。此時速度恰好達到 $60 \text{ }^\circ/\text{s}$, 加速度為 $129.9/1.407^2 = 65.6 \text{ }^\circ/\text{s}^2$ (remaining 餘裕充足)。

工程觀察 : 此案例中速度是瓶頸 , 加速度餘裕很大——這代表若改用梯形速度曲線 (能在較短時間內達到並維持最高速度) , 可以更快完成。下一個例題將驗證這一點。

例題 15-2 梯形速度曲線的設計

題目 : 承上題的關節 ($\Delta q = 90^\circ$ 、 $v_{\max} = 60 \text{ }^\circ/\text{s}$ 、 $a_{\max} = 120 \text{ }^\circ/\text{s}^2$)。(a) 設計梯形速

度曲線並求總時間。(b) 與例題 15-1 的五次多項式比較。(c) 若移動量改為 20° ，重新計算並說明差異。(d) 改用 S 形曲線 ($j_{\max} = 600^\circ/\text{s}^3$)，求總時間。

解 (a)：先檢查是否有等速段 (15.4.3 節)：

$$v_{\max}^2/a_{\max} = 60^2/120 = 3600/120 = 30^\circ < \Delta q = 90^\circ \rightarrow \text{有等速段 (梯形)}$$

判斷條件成立。

$$t_a = v_{\max}/a_{\max} = 60/120 = 0.5 \text{ s} \quad (\text{加速段時間})$$

$$T = \Delta q/v_{\max} + v_{\max}/a_{\max} = 90/60 + 60/120 = 1.5 + 0.5 = 2.0 \text{ 秒}$$

梯形曲線的總時間。

解 (b)：梯形曲線只需 2.0 秒，而五次多項式需 2.81 秒——梯形快了 29%。原因是梯形曲線讓馬達在大部分時間中都以最高速度運轉，而多項式只在瞬間達到峰值速度。

這印證了 15.4.1 節的說法：在相同的速度與加速度限制下，梯形速度曲線是時間最佳的。

解 (c)： $\Delta q = 20^\circ$ ，重新檢查：

$$v_{\max}^2/a_{\max} = 30^\circ > \Delta q = 20^\circ \rightarrow \text{無等速段 (三角形)}$$

退化為三角形曲線。

$$v_{\text{peak}} = \sqrt{a_{\max} \cdot \Delta q} = \sqrt{120 \times 20} = \sqrt{2400} = 49.0^\circ/\text{s}$$

$$T = 2\sqrt{\Delta q/a_{\max}} = 2\sqrt{20/120} = 2 \times 0.408 = 0.816 \text{ 秒}$$

三角形情形的峰值速度與總時間。

關鍵觀察：實際達到的峰值速度只有 $49^\circ/\text{s}$ ，遠低於額定的 $60^\circ/\text{s}$ 。因此提高速度參數對這個動作完全沒有效果——瓶頸在加速度。若要縮短時間，必須提高 $a_{\max}$ 。這正是 15.4.3 節提到的實務現象。

解 (d)：S 形曲線的總時間 (15.5.2 節，以 $\Delta q = 90^\circ$ 計算)：

$$T = \Delta q/v_{\max} + v_{\max}/a_{\max} + a_{\max}/j_{\max} = 1.5 + 0.5 + 120/600 = 1.5 + 0.5 + 0.2 = 2.2 \text{ 秒}$$

S 形曲線的總時間。

比梯形多了 0.2 秒 (10%)，這就是平滑的代價。但換來的是加速度連續、急跳度有限、振動大幅降低。若原本因振動需要在終點等待 0.3 秒才能進行下一動作，改用 S 形反而總體更快——這正是開頭故事的情境。

例題 15-3 動力學可行性與時間縮放

題目：某平面 2R 手臂 (參數同第 13 章例題 13-2 : $L_1=0.4$ 、 $L_2=0.3$ m , $m_1=3.0$ 、 $m_2=2.0$ kg) 執行一段軌跡。在某個時刻，姿態 $\theta = (30^\circ, 60^\circ)$ 、速度 $\dot{\theta} = (2.0, -1.0)$ rad/s、加速度 $\ddot{\theta} = (4.0, 2.0)$ rad/s²。已知兩關節的力矩極限分別為 $\tau_{\max} = (15, 3)$ N·m。(a) 計算此時刻所需的力矩。(b) 判斷是否可行。(c) 若不可行，求所需的時間縮放倍數。(d) 驗證縮放後的力矩。

解 (a)：沿用第 13 章的常數： $\alpha = 0.540$ 、 $\beta = 0.120$ 、 $\delta = 0.060$ kg·m²。 $\theta_2 = 60^\circ$ 時 $c_2 = 0.5$ 、 $s_2 = 0.866$ 。

$$M = \begin{bmatrix} 0.660 & 0.120 \\ 0.120 & 0.060 \end{bmatrix} \quad M \cdot \ddot{q} = \begin{bmatrix} 0.660(4.0) + 0.120(2.0) \\ 0.120(4.0) + 0.060(2.0) \end{bmatrix} = \begin{bmatrix} 2.880 \\ 0.600 \end{bmatrix}$$

慣性項。

$$\begin{aligned} \beta s_2 &= 0.1039 \\ C \text{ 項}_1 &= -0.1039(1.0)^2 - 2(0.1039)(2.0)(-1.0) = -0.104 + 0.416 = 0.312 \\ C \text{ 項}_2 &= 0.1039(2.0)^2 = 0.416 \end{aligned}$$

科氏與離心項。

重力項 (由第 13 章例題 13-2，同樣的姿態)： $G = (11.88, 0)^T$ N·m。

$$\begin{aligned} \tau_1 &= 2.880 + 0.312 + 11.88 = 15.07 \text{ N}\cdot\text{m} \\ \tau_2 &= 0.600 + 0.416 + 0 = 1.02 \text{ N}\cdot\text{m} \end{aligned}$$

所需力矩。

解 (b)： $\tau_1 = 15.07 > 15$ (超限 0.5%)、 $\tau_2 = 1.02 < 3$ (可行)。因此第一關節超限，軌跡不可行。

解 (c)：由 15.8.2 節，時間縮放後： $\tilde{\tau} = (1/k^2)(M\ddot{q} + Cq) + G$ 。對第一關節：

$$(1/k^2)(2.880 + 0.312) + 11.88 \leq 15 \Rightarrow 3.192/k^2 \leq 3.12 \Rightarrow k^2 \geq 1.023 \Rightarrow k \geq 1.011$$

所需的時間縮放倍數。

取 $k = 1.05$ (留一些餘裕)。整條軌跡的時間拉長 5%。

解 (d)：驗證縮放後的力矩：

$$\tilde{\tau}_1 = 3.192/1.05^2 + 11.88 = 3.192/1.1025 + 11.88 = 2.895 + 11.88 = 14.78 \text{ N}\cdot\text{m} \checkmark$$

低於 15 N·m 的極限，可行。

重要觀察：注意重力項 $11.88 \text{ N}\cdot\text{m}$ 佔了極限 $15 \text{ N}\cdot\text{m}$ 的 79%，動態部分只剩 $3.12 \text{ N}\cdot\text{m}$ 的空間。這說明了兩件事：一、此姿態下馬達的餘裕極小，是整條軌跡的瓶頸；二、**若重力項本身就超過 $15 \text{ N}\cdot\text{m}$ ，任何時間縮放都無濟於事——這正是 15.8.2 節推論二的實例。**此時只能減輕負載、改變姿態，或更換更大的馬達。

程式範例：軌跡生成器與可行性檢驗

以下 Python 程式實作四種軌跡形式（三次、五次、梯形、S 形），比較其速度、加速度與急跳度特性，並示範動力學可行性檢驗與時間縮放。

```
import numpy as np

# ===== 各類軌跡生成器 =====
def cubic(q0, qf, T, t):
    """三次多項式 (15.3.1 節)：起訖靜止"""
    dq = qf - q0
    s = t / T
    q = q0 + dq*(3*s**2 - 2*s**3)
    qd = dq*(6*s - 6*s**2)/T
    qdd = dq*(6 - 12*s)/T**2
    return q, qd, qdd

def quintic(q0, qf, T, t):
    """五次多項式 (15.3.2 節)：起訖靜止且加速度為零"""
    dq = qf - q0
    s = t / T
    q = q0 + dq*(10*s**3 - 15*s**4 + 6*s**5)
    qd = dq*(30*s**2 - 60*s**3 + 30*s**4)/T
    qdd = dq*(60*s - 180*s**2 + 120*s**3)/T**2
    return q, qd, qdd

def trapezoidal(q0, qf, vmax, amax, t):
    """梯形速度曲線 (15.4 節)，自動處理三角形退化"""
    dq = qf - q0
    sgn = np.sign(dq) if dq != 0 else 1.0
    D = abs(dq)
    if D >= vmax**2/amax:
        # 有等速段
        ta = vmax/amax
        T = D/vmax + vmax/amax
        vp = vmax
    else:
        # 三角形退化
        vp = np.sqrt(amax*D)
        ta = vp/amax
        T = 2*ta
    if t <= 0:
        q, qd, qdd = 0.0, 0.0, 0.0
    elif t < ta:
        q, qd, qdd = 0.5*amax*t**2, amax*t, amax
    elif t < T-ta:
        q, qd, qdd = 0.5*vp*ta + vp*(t-ta), vp, 0.0
    elif t <= T:
        td = T - t
        q, qd, qdd = D - 0.5*amax*td**2, amax*td, -amax
```

```

else:
    q, qd, qdd = D, 0.0, 0.0
    return q0 + sgn*q, sgn*qd, sgn*qdd, T

def scurve_time(D, vmax, amax, jmax):
    """S 形曲線的總時間 (15.5.2 節·完整七段情形)"""
    return D/vmax + vmax/amax + amax/jmax

# ===== 驗證：例題 15-1、15-2 =====
print("=== 例題 15-1：多項式軌跡 ===")
dq, T = 90.0, 2.0
print(f"三次多項式：v_max={1.5*dq/T:6.2f} °/s   a_max={6*dq/T**2:7.2f} °/s²")
print(f"五次多項式：v_max={1.875*dq/T:6.2f} °/s   a_max={5.774*dq/T**2:7.2f} °/s²")
print(f"極限：      v_max= 60.00 °/s   a_max= 120.00 °/s²")
kv, ka = 1.875*dq/T/60, np.sqrt(5.774*dq/T**2/120)
print(f"五次多項式所需縮放：k_v={kv:.3f}   k_a={ka:.3f} → 取 k={max(kv,ka):.3f}")
print(f"最短可行時間：{max(kv,ka)*T:.2f} 秒")

print("\n=== 例題 15-2：梯形與 S 形曲線 ===")
for D in (90.0, 20.0):
    _, _, _, Ttrap = trapezoidal(0, D, 60.0, 120.0, 0)
    kind = "梯形" if D >= 60**2/120 else "三角形"
    vp = 60.0 if D >= 60**2/120 else np.sqrt(120*D)
    print(f"位移 {D:5.1f}°: {kind:4s}   峰值速度={vp:5.1f} °/s   總時間={Ttrap:.3f} s")
Ts = scurve_time(90.0, 60.0, 120.0, 600.0)
print(f"S 形曲線 (90°, j_max=600): 總時間={Ts:.3f} s   "
      f" ( 比梯形多 {(Ts-2.0)/2.0*100:.0f}%) ")

# ===== 四種軌跡的特性比較 =====
print("\n=== 四種軌跡的峰值比較 (Δq=90°, T=2.0s) ===")
ts = np.linspace(0, T, 2001)
for name, fn in (("三次多項式", cubic), ("五次多項式", quintic)):
    q, qd, qdd = np.array([fn(0, dq, T, t) for t in ts]).T
    jerk = np.gradient(qdd, ts)
    print(f"{name}: |v|max={np.abs(qd).max():6.2f}   "
          f"|a|max={np.abs(qdd).max():7.2f}   "
          f"|j|max={np.abs(jerk).max():8.1f} °/s³")

# ===== 動力學可行性檢驗 (例題 15-3) =====
print("\n=== 例題 15-3：動力學可行性與時間縮放 ===")
class Arm2R:
    def __init__(s, L1=0.4, L2=0.3, m1=3.0, m2=2.0, g=9.8):
        s.L1, s.L2, s.m1, s.m2, s.g = L1, L2, m1, m2, g
        s.l1, s.l2 = L1/2, L2/2
        s.I1, s.I2 = m1*L1**2/12, m2*L2**2/12
        s.a = m1*s.l1**2 + s.I1 + m2*(L1**2 + s.l2**2) + s.I2
        s.b = m2*L1*s.l2
        s.d = m2*s.l2**2 + s.I2
    def tau(s, q, qd, qdd):
        c2, s2 = np.cos(q[1]), np.sin(q[1])
        M = np.array([[s.a + 2*s.b*c2, s.d + s.b*c2],
                      [s.d + s.b*c2, s.d]])

```

```

C = np.array([-s.b*s2*qd[1]**2 - 2*s.b*s2*qd[0]*qd[1],
              s.b*s2*qd[0]**2])
c1, c12 = np.cos(q[0]), np.cos(q[0]+q[1])
G = np.array([(s.m1*s.l1 + s.m2*s.l1)*s.g*c1 + s.m2*s.l2*s.g*c12,
              s.m2*s.l2*s.g*c12])
return M @ qdd + C + G, M @ qdd + C, G

arm = Arm2R()
q = np.deg2rad([30, 60])
qd = np.array([2.0, -1.0])
qdd = np.array([4.0, 2.0])
tau_max = np.array([15.0, 3.0])

tau, dyn, grav = arm.tau(q, qd, qdd)
print(f"動態部分 (Mq̈+Cq̇): {np.round(dyn, 3)} N·m")
print(f"重力部分 G: {np.round(grav, 3)} N·m")
print(f"總力矩 τ: {np.round(tau, 3)} N·m")
print(f"力矩極限: {tau_max} N·m")
print(f"是否可行: {np.all(np.abs(tau) <= tau_max)}")

# 時間縮放 (15.8.2 節：動態項按 1/k² 縮放，重力不變)
k_need = 1.0
for i in range(2):
    margin = tau_max[i] - abs(grav[i])
    if margin <= 0:
        print(f"關節{i+1}: 重力已超限，時間縮放無效！")
    elif abs(dyn[i]) > margin:
        k_need = max(k_need, np.sqrt(abs(dyn[i])/margin))
print(f"所需時間縮放倍數: k = {k_need:.4f}")

k = 1.05
tau_scaled = dyn/k**2 + grav
print(f"以 k={k} 縮放後的力矩: {np.round(tau_scaled, 3)} N·m")
print(f"縮放後是否可行: {np.all(np.abs(tau_scaled) <= tau_max)}")
print(f"重力佔極限比例: ")
    f"{abs(grav[0])/tau_max[0]*100:.1f}% ← 動態餘裕僅剩 "
    f"{(1-abs(grav[0])/tau_max[0])*100:.1f}%"

```

請試著修改：（一）把 S 形曲線的 $j_{\max}$ 由 600 降到 200，觀察總時間如何增加。（二）在動力學檢驗中把姿態改為 $\theta = (0^\circ, 90^\circ)$ （手臂水平伸展），觀察重力項如何暴增、動態餘裕如何被壓縮。（三）繪出四種軌跡的位置、速度、加速度與急跳度四階曲線，直觀比較它們的平滑程度。

實作：軌跡品質的實驗評估

本章實作是在實體手臂上比較不同軌跡形式的實際表現，量化「平滑」的價值。

1. **基準設定**：選定一段點對點運動（建議移動量較大，如某關節轉 90° ），並確認手臂在該路徑上無碰撞風險。
2. **四種軌跡實作**：分別以三次多項式、五次多項式、梯形速度、S 形曲線規劃同一段運動，時間參數設為相同（或各自的最短可行時間）。
3. **振動量測**：在末端加裝加速度計（可用手機的感測器 App，第 6 章的實作已用過），記錄每種軌跡執行時的末端加速度。特別記錄**運動停止後**的殘餘振動——這是最能區分軌跡品質的指標。
4. **振動量化**：計算殘餘振動的峰值振幅與衰減至 5% 所需的時間（穩定時間）。製作四種軌跡的比較表。
5. **總時間比較**：記錄「開始運動」到「振動衰減至可接受範圍」的總時間。你可能會發現：S 形曲線雖然運動時間較長，但總時間反而最短——這是本章最重要的實驗結論。
6. **追隨誤差量測**：若控制器能輸出追隨誤差，記錄每種軌跡的最大誤差。平滑的軌跡應有較小的誤差。
7. **極限探索**：逐步縮短執行時間，找出每種軌跡開始出現異常（超限警告、明顯振動、追隨誤差暴增）的臨界時間。
8. **混成效果測試**：規劃一段含三個路徑點的直角轉折路徑，分別以零混成與不同混成半徑執行，記錄總時間與實際路徑偏離量，繪出「混成半徑 vs 時間」與「混成半徑 vs 偏離量」兩條曲線。

表 15-4 軌跡品質比較表（第一列為範例）

軌跡形式	運動時間	殘餘振動峰值	穩定時間	總時間	最大追隨誤差
梯形速度	2.00 s	0.42 g	0.65 s	2.65 s	1.8 mm
（自行填寫其餘三種）					

國中小也看得懂：為什麼捷運不會突然衝出去

坐過捷運嗎？有沒有注意過，車子**啟動的時候不會突然衝出去**，而是慢慢加速；快到站的時候也是慢慢減速，最後才停下來。為什麼不乾脆一下子加速到最快呢？因為那樣的話，站著的乘客會全部跌倒！

機器手臂也一樣。如果讓它「說走就走、說停就停」，會發生兩件壞事：第一，馬達根本做

不到（就像你沒辦法瞬間從站著變成全速跑步）；第二，整支手臂會**晃來晃去**，像用力甩一根竹竿之後，竿子還會抖好久。

所以工程師要幫機器人設計一條「**什麼時候該多快**」的計畫表。這張表叫做「軌跡」。好的軌跡長得像一座梯形的小山：慢慢加速上去、中間維持最快速度、再慢慢減速下來。

這裡有一件很有趣的事：**慢慢加速反而更快！**聽起來很奇怪對不對？但想想看——如果你猛力甩手臂，它會抖個不停，你必須等它不抖了才能做下一件事。如果你溫和地移動，它一停就穩住了，馬上可以做下一件事。加起來，反而是溫和的比較快。

下次你玩附錄的機器手臂時，試試看兩種方式：一次讓它「咻」地衝過去，一次讓它慢慢加速慢慢停。用眼睛看看哪一種抖得比較厲害，再用手機的碼表量量看，哪一種真正「穩下來」的時間比較短。你會親眼看到這一章的道理喔！

叫獸提醒與常見錯誤

- 誤區一：以為關節空間規劃的起訖點安全就代表全程安全。中間過程末端會走出曲線，可能穿過桌面或撞到障礙物。
- 誤區二：在需要直線的作業（焊接、塗膠）使用關節運動指令。末端路徑會偏離直線，造成焊縫或膠道歪斜。
- 誤區三：以為五次多項式一定比三次好。它雖然加速度連續，但峰值速度高出 25%，更容易超出速度極限。
- 誤區四：短距離移動時調高速度參數。此時瓶頸在加速度，速度根本達不到額定值，應調整的是加速度參數。
- 誤區五：忽略急跳度。加速度的跳變會激發機構共振、加速減速機磨損，且使追隨誤差增加。
- 誤區六：在轉折處要求精確通過。這會迫使機器人減速至零，大幅增加節拍時間；應在非作業段使用混成半徑。
- 誤區七：對姿態用歐拉角做線性內插。路徑會扭曲且可能經過萬向節鎖；應使用四元數 SLERP。
- 誤區八：SLERP 未處理雙重覆蓋。若 $q_0 \cdot q_f < 0$ 而未把 q_f 取負，機器人會繞遠路（超過

180 度)。

- 誤區九：只做運動學檢驗而不檢驗力矩。速度與加速度在限制內，不代表力矩也在限制內——後者還取決於姿態與重力。
- 誤區十：以為時間縮放萬能。重力項不隨時間縮放而降低；若重力本身就超限，只能減輕負載或換馬達。
- 誤區十一：追求時間最佳軌跡。它沒有餘裕、對模型誤差敏感、且急跳度大；實務應保留 10 ~ 20% 的餘裕。
- 誤區十二：忽略多軸同步。各軸獨立執行會使末端路徑不可預測；且調快非主導軸的速度對節拍毫無幫助。

課堂討論

1. 開頭故事中，平滑的軌跡「反而更快」。請從三個角度解釋這個現象：控制器的降速保護、殘餘振動的等待時間、以及追隨誤差。
2. 混成半徑是速度與精度的直接取舍。請為以下三個情境各建議一個混成策略：(a) 點焊（只需在特定点停留）；(b) 連續弧焊（整條路徑都要精確）；(c) 從料架取料再放到輸送帶（自由空間移動）。
3. 時間縮放無法降低重力項。請討論這對「機器人在什麼姿態下最容易超出力矩極限」有何啟示，並提出三種設計層面的因應方式。
4. 實務上很少使用時間最佳軌跡。請討論：在什麼情況下追求時間最佳是值得的？需要哪些配套條件（提示：模型精度、控制器性能、機構剛性）？
5. 輸入整形以「延遲半個振動週期再送第二個脈衝」來抵消振動。請討論這個方法的三個前提條件，以及當這些條件不滿足時會發生什麼。

章末摘要

- 路徑是幾何曲線（不含時間），軌跡是時間參數化的路徑；由路徑產生軌跡的過程稱為時間參數化。
- 好軌跡的五個標準：平滑、快速、可行、精確、可預測；其中「快速」與「平滑」存在根本的張力。

- 關節空間規劃計算快、無奇異點問題，但末端路徑不可預測；任務空間規劃路徑可控，但需逐點逆解且可能遇奇異點。
- 三次多項式的 $v_{\max} = 1.5\Delta q/T$ 、 $a_{\max} = 6\Delta q/T^2$ ，加速度在端點跳變（急跳度無限大）。
- 五次多項式加速度連續，但 $v_{\max} = 1.875\Delta q/T$ 高出 25%，更容易觸及速度極限——平滑不是免費的。
- 梯形速度曲線在相同的速度與加速度限制下用時最短： $T = \Delta q/v_{\max} + v_{\max}/a_{\max}$ 。
- 當 $\Delta q < v_{\max}^2/a_{\max}$ 時退化為三角形，實際峰值速度低於額定值——此時提高速度參數無效，瓶頸在加速度。
- S 形曲線以七段結構限制急跳度，總時間多出 $a_{\max}/j_{\max}$ ，換取加速度連續與振動降低。
- 急跳度的四個意義：力矩變化率、激發結構振動、乘坐舒適度、追隨精度。
- 多路徑點的速度可由使用者指定、啟發式規則或整體最佳化決定；混成半徑是速度與精度的直接取舍參數。
- 笛卡兒直線採「幾何路徑 + 時間曲線」分離設計；圓弧須依 $a_c = v^2/R$ 限制速度；姿態須用四元數 SLERP 並處理雙重覆蓋。
- 軌跡必須通過動力學檢驗：逐點計算 $\tau = M\ddot{q} + C\dot{q} + G$ 並確認不超限。
- 時間縮放 k 倍後，速度按 $1/k$ 、加速度按 $1/k^2$ 、動態力矩按 $1/k^2$ 縮放，但重力項完全不變。
- 時間最佳軌跡具 bang-bang 特性（恆有關節處於極限），但因不夠平滑、無餘裕、依賴模型，實務少用。
- 多軸同步取各軸所需時間的最大值，決定總時間的是主導軸；調快非主導軸對節拍無幫助。
- 實務流程七步驟：取得路徑、選擇空間、設定混成、時間參數化、多軸同步、動力學檢驗、輸出指令。

觀念題與計算題

觀念題

1. 區分路徑與軌跡，並說明時間參數化的意義。
2. 列舉好軌跡的五個標準，並說明哪兩者之間存在根本的張力。

3. 比較關節空間與任務空間規劃的優缺點，並各舉兩個適用場合。
4. 為何關節空間規劃「起訖點安全不代表全程安全」？請提出兩種因應方式。
5. 推導三次多項式的係數，並說明為何其加速度在端點會跳變。
6. 說明五次多項式相對於三次多項式的優勢與代價。
7. 寫出梯形速度曲線的總時間公式，並說明三角形退化的判斷條件與實務意義。
8. 說明 S 形曲線的七個階段，以及它相對於梯形曲線多出的時間代價。
9. 列舉急跳度的四個工程意義，並說明它與機構振動的關係。
10. 說明混成半徑的作用，並解釋為何「允許不精確通過」反而是好的設計。
11. 為何姿態內插必須使用 SLERP 而非歐拉角線性內插？實作時需注意哪兩個要點？
12. 說明圓弧運動為何必須依半徑限制速度，並寫出相關公式。
13. 寫出時間縮放後各項力矩的變化關係，並說明為何重力項不隨縮放降低。
14. 說明時間最佳軌跡的 bang-bang 特性，以及實務上少用它的三個原因。
15. 說明多軸同步的三種等級，以及「主導軸」的概念對節拍最佳化的意義。

計算題

1. 某關節需由 -30° 移動到 120° ，速度極限 $90^\circ/\text{s}$ 、加速度極限 $200^\circ/\text{s}^2$ 。(a) 以梯形速度曲線設計，判斷是否有等速段並求總時間。(b) 求加速段的時間與位移。(c) 若改用五次多項式且時間相同，求其峰值速度與加速度，並判斷是否可行。(d) 若五次多項式不可行，求最短可行時間。
2. 承上題的關節。(a) 若移動量改為 30° ，重新判斷曲線形狀並求總時間與峰值速度。(b) 計算「把速度極限提高到 $120^\circ/\text{s}$ 」對此動作總時間的影響。(c) 計算「把加速度極限提高到 $300^\circ/\text{s}^2$ 」的影響。(d) 由此說明應優先調整哪個參數。
3. 某 S 形曲線的參數為 $v_{\max} = 100 \text{ mm/s}$ 、 $a_{\max} = 500 \text{ mm/s}^2$ 、 $j_{\max} = 5000 \text{ mm/s}^3$ ，位移 300 mm 。(a) 求急跳段時間 t_j 與完整加速段時間 t_a 。(b) 求總時間。(c) 與梯形曲線比較，計算多出的時間百分比。(d) 若 $j_{\max}$ 降為 1000 mm/s^3 ，重新計算總時間。
4. 某平面 2R 手臂在某時刻的動態力矩為 $(M\ddot{q} + C\dot{q}) = (4.5, 1.2)^T \text{ N}\cdot\text{m}$ 、重力項 $G = (10.5, 0.8)^T \text{ N}\cdot\text{m}$ ，力矩極限為 $(14, 2.5)^T \text{ N}\cdot\text{m}$ 。(a) 計算總力矩並判斷可行性。(b) 計算各關節

的動態餘裕。(c) 求所需的時間縮放倍數。(d) 若把負載加倍使重力項變為 $(18, 1.4)^T$ ，重新判斷可行性並說明結論。

動手做與專題延伸

本章實作：軌跡生成模組

延續前面章節的程式庫，加入軌跡生成模組。這將是第 16 ~ 17 章控制器測試的訊號來源。

- **階段一**：實作四種基本軌跡（三次、五次、梯形、S 形），統一介面為 $\text{traj}(t) \rightarrow (q, \dot{q}, \ddot{q})$ ，並繪出四階曲線比較。
- **階段二**：實作多路徑點軌跡，支援使用者指定或啟發式的路徑點速度，並加入混成功能。
- **階段三**：實作笛卡兒空間的直線與圓弧軌跡，含 SLERP 姿態內插與逐點逆運動學。
- **階段四**：整合第 14 章的逆動力學，實作可行性檢驗與自動時間縮放的迴圈。
- **階段五**：實作多軸同步，找出主導軸並統一時間。

專題延伸

- **四階曲線視覺化**：為四種軌跡形式繪製位置、速度、加速度、急跳度的四階疊圖。特別標註三次多項式在端點的急跳度脈衝與梯形曲線在轉折處的跳變。這張圖是理解「平滑」概念最直觀的教材。
- **節拍最佳化實驗**：設計一個包含五個路徑點的取放循環，系統性地變化三個參數（速度比例、加速度比例、混成半徑），記錄總節拍時間與路徑偏離量。繪出三維的參數—性能圖，找出最佳的參數組合。這模擬了產線調機的真實工作。
- **進階：時間最佳路徑參數化 (TOPP) 的簡化實作**。對一條給定的路徑，在每個路徑點計算「受力矩極限所允許的最大加速度」，再用前向—後向掃描法（先由起點正向加速掃描、再由終點反向減速掃描、取兩者的較小值）求出時間最佳的速度剖面。與固定的梯形曲線比較節拍時間，並觀察最佳解的 bang-bang 特性。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 7 章：軌跡生成，本章主要參考)。
- Biagiotti, L. and Melchiorri, C., Trajectory Planning for Automatic Machines and

Robots, Springer (軌跡規劃的完整專著，含各類運動曲線的詳細分析) 。

- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer (第 4 章：軌跡規劃) 。
- Bobrow, J. E., Dubowsky, S. and Gibson, J. S., "Time-Optimal Control of Robotic Manipulators Along Specified Paths," IJRR, 1985 (時間最佳路徑參數化的奠基論文) 。
- Pham, Q.-C., "A General, Fast, and Robust Implementation of the Time-Optimal Path Parameterization Algorithm," IEEE Trans. Robotics, 2014 (現代 TOPP 演算法的高效實作) 。
- Singer, N. C. and Seering, W. P., "Preshaping Command Inputs to Reduce System Vibration," ASME J. Dynamic Systems, 1990 (輸入整形技術的原始論文) 。
- Shoemake, K., "Animating Rotation with Quaternion Curves," SIGGRAPH 1985 (SLERP 姿態內插) 。
- Kunz, T. and Stilman, M., "Time-Optimal Trajectory Generation for Path Following with Bounded Acceleration and Velocity," RSS 2012 (含加速度限制的軌跡生成) 。
- Ruckig 開源函式庫：即時的 S 形軌跡生成器，支援線上重新規劃，廣泛用於現代機器人系統。
- MoveIt 2 與 Ruckig、TOPP-RA 的整合文件：ROS 2 生態系中軌跡生成與時間參數化的實務作法。

第 16 章

線性回授與 PID 控制

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為一位學生在手掌上平衡竹竿的三格連續照片：第一格蒙眼（竿子傾倒，標註「開迴路」）、第二格張眼但動作誇張（竿子劇烈擺盪，標註「只有 P 」）、第三格動作細微而穩定（標註「 PID 」）。右半為對應的階躍響應曲線圖，三條曲線分別為等幅振盪（ $\zeta=0$ ）、有超越的衰減（ $\zeta=0.3$ ）與平順到位（ $\zeta=0.707$ ），並以虛線標出 $\pm 2\%$ 容許帶。中央以三個大字母 P 、 I 、 D 串連兩側，下方註記： P 看現在、 I 看過去、 D 看未來。

叫獸開講 「機器手臂到了目標位置，為什麼還在左右發抖？」

叫獸請小翰上台，蒙上眼睛，然後在他手上放了一根竹竿，竿頂立著一顆保麗龍球。「請你把球維持在竿子頂端不要掉下來。」

小翰試了三秒，球就掉了。「我看不到啊！」

「對，這就是**開迴路**。」叫獸解開眼罩，「你只能憑記憶動作，不知道現在的狀況。現在張開眼睛再試一次。」這次小翰撐了將近半分鐘——他能看到球往哪邊倒，就往哪邊移動竿子。

「這叫**閉迴路**：你量測誤差、據以修正。這就是所有控制的核心。」叫獸在白板上畫了一個圓圈：目標 → 比較 → 控制 → 系統 → 量測 → 回到比較。

接著他問了一個更深的問題：「小翰，你剛才是**怎麼**修正的？球往右倒，你就把竿子往右移——移多少？」

「憑感覺……倒得越多，我移得越多。」

「這就是**比例控制**。」叫獸寫下 P。「但如果只有這樣，會發生什麼事？」他請小翰再試一次，但這次要求「只能看球目前偏多少，不准預測」。小翰的動作變得非常誇張——球在左右之間劇烈擺盪，撐不到五秒。

「因為你反應太慢了。等球已經倒了才修正，它已經有速度了，你把它扳回來時它又衝過頭。」叫獸寫下第二個字母 D。「真正厲害的人會看球**倒的速度**，提前反應——球還沒倒多少，但倒得很快，就要趕快修。這叫**微分控制**，它的作用像剎車。」

最後他在竿子上綁了一小塊配重，讓竿子微微向左傾斜。小翰重新開始，這次球雖然沒掉，但總是停在偏左一點的位置，怎麼都回不到正中央。

「因為你的比例控制需要**誤差存在**才會出力。誤差變小，你出的力就變小，小到剛好抵消配重時就平衡了——於是留下一個永遠消不掉的偏差。」叫獸寫下第三個字母 I。「解法是：把過去累積的誤差加起來，只要偏差持續存在，修正量就持續增加，直到把它推回中央為止。」

白板上留下三個字母：P、I、D。「這三個字母，」叫獸說，「統治了全世界九成以上的工業控制迴路——從你家的冷氣、汽車的定速巡航，到六軸機器手臂的每一個關節。今天，我們就把它們徹底搞懂。」

本章為什麼重要

第 15 章規劃出了完美的軌跡，但那只是**指令**。真實的機器人有摩擦、有彈性、負載會變、外界有干擾——把指令變成實際運動，是控制器的工作。

PID 是控制領域最重要的實用工具。據統計，工業界九成以上的控制迴路採用 PID 或其變體，而所有的機器人關節伺服控制，內層都是 PID。理解它不只是為了會調參數，更是為了理解**回授的本質與極限**：為什麼增益不能無限調高、為什麼快速與穩定互相衝突、以及為什麼在機器人上單靠 PID 不夠（第 13 章開頭的故事）。

本章的分析對象刻意選擇最簡單的系統——一維點質量。這不是簡化的教學玩具，而是有深刻理由的：第 17 章的計算力矩控制，正是要把複雜的多軸機器人「線性化」成 n 個獨立的一維點質量。因此把一維系統徹底搞懂，就掌握了現代機器人控制的核心。

學習目標

- 能區分開迴路與閉迴路控制，並說明回授的三個價值。
- 能繪製控制系統的方塊圖並辨識各元件的角色。
- 能推導一維點質量在 P 控制下的閉環方程並說明其振盪特性。
- 能推導 PD 控制的閉環二階方程，並由係數求出自然頻率與阻尼比。
- 能說明過阻尼、臨界阻尼與欠阻尼三種響應的特徵與適用場合。
- 能說明積分項如何消除穩態誤差，以及它帶來的代價。
- 能解釋積分飽和現象並提出至少兩種防止方法。
- 能運用時域規格（上升時間、超越量、整定時間）進行增益設計。
- 能說明數位實作的三個議題：取樣週期、微分雜訊、離散化。
- 能說明前饋控制的原理及其與回授的分工。
- 能以 Python 實作 PID 控制器並在動力學模擬中測試。

先備知識

第 13 章的動力學（尤其是一維點質量與二階系統的概念）、第 15 章的軌跡生成（提供控制器的追隨目標）、第 7 章的嵌入式實作（PWM、中斷、控制週期）。

數學方面需要二階常微分方程的解法（特徵方程與根的性質）。若對此不熟悉，可先閱讀附錄 D 的微分方程速成——本章會以物理直覺重新說明每一個數學結果，不會讓推導成為理解的障礙。

核心名詞中英對照

表 16-1 本章核心名詞

中文	英文	說明
開迴路	Open-loop	不使用量測回授的控制方式
閉迴路	Closed-loop	以量測誤差修正輸出的控制方式
回授	Feedback	將輸出量測值送回輸入端比較
前饋	Feedforward	依模型預先計算所需輸入
誤差	Error	目標值與實際值之差
增益	Gain	控制器對誤差的放大倍數
比例控制	Proportional Control	輸出與當前誤差成正比
微分控制	Derivative Control	輸出與誤差變化率成正比
積分控制	Integral Control	輸出與誤差累積量成正比
自然頻率	Natural Frequency, ω_n	系統無阻尼時的振盪頻率
阻尼比	Damping Ratio, ζ	描述振盪衰減快慢的無量綱參數
穩態誤差	Steady-state Error	時間趨於無窮時殘留的誤差
超越量	Overshoot	響應超過目標值的最大百分比
整定時間	Settling Time	響應進入並維持在容許帶內所需時間
積分飽和	Integral Windup	積分項過度累積導致的響應惡化
取樣週期	Sampling Period	數位控制器執行一次的時間間隔

16.1 開迴路與閉迴路

16.1.1 兩種控制哲學

開迴路控制依據事先的計算或經驗直接下達指令，不檢查結果。例如：「馬達通電 3 秒，應該會轉 90 度。」它的優點是簡單、便宜、不會不穩定（因為沒有回授就沒有振盪的可能）。

缺點是任何未預期的因素都會造成誤差且無法修正：電池電壓下降、摩擦力變化、負載改變、

機構磨損——全部都會累積成無法察覺的偏差。第 5 章的步進馬達失步（5.4.2 節）正是開迴路的典型風險。

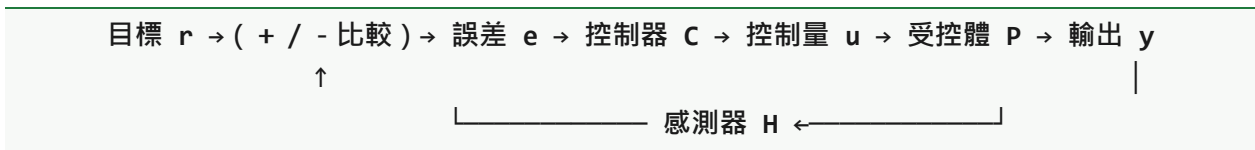
閉迴路控制則持續量測實際輸出，計算與目標的誤差，據以調整控制量。這帶來三個決定性的價值：

- **抗干擾**：外界的擾動造成誤差後會被偵測並修正。
- **降低對模型的依賴**：即使系統參數估計不準（如負載不明），回授仍能把誤差壓下來。
- **改變系統動態**：可以讓一個原本反應緩慢的系統變快、讓一個容易振盪的系統變穩。

代價則是：需要感測器（成本）、可能不穩定（增益過高會發散）、且引入雜訊（感測器的雜訊會被放大送進控制量）。

16.1.2 控制系統的方塊圖

標準的閉迴路控制系統包含五個元件：



閉迴路方塊圖。誤差 $e = r - y$ （若感測器增益為 1）。

對機器人的單一關節而言： r 是第 15 章規劃的目標角度、 C 是 PID 控制器、 u 是送給驅動器的力矩指令、 P 是關節的動力學（第 13 章）、 y 是實際角度、 H 是編碼器（第 6 章）。可以看到，本書前面各章的內容在此匯集成一個完整的迴路。

圖 16-1 機器人關節的控制迴路

建議配圖：完整的方塊圖。左側為軌跡生成器（第 15 章）輸出目標位置、速度與加速度；中央為 PID 控制器（三個並聯的方塊 K_p 、 K_i 、 K_d ）；控制量經驅動器（第 5、7 章）送至馬達；受控體標註為關節動力學 $M(q)\ddot{q} + C\dot{q} + G$ （第 13 章）；輸出經編碼器（第 6 章）回授。圖中並以虛線標出第 17 章將加入的前饋路徑（由軌跡直接經逆動力學至控制量）。各方塊下方標註對應的章節編號。

16.2 一維點質量：所有控制的起點

16.2.1 為何從最簡單的系統開始

考慮一個質量為 m 的物體，在無摩擦的水平面上，受控制力 u 作用。其運動方程為：

$$m \ddot{x} = u$$

一維點質量。這是最簡單的動態系統，卻包含了控制問題的所有本質。

為何不直接研究六軸手臂？三個理由。**理由一：本質相同**。多軸系統的每個關節，在忽略耦合後就是一個一維系統。**理由二：可解析分析**。一維系統的閉環方程有封閉解，可以精確算出增益與性能的關係；多軸系統只能數值模擬。**理由三：第 17 章的目標**。計算力矩控制的核心思想，正是用動力學模型把多軸系統「化為」 n 個獨立的一維點質量——因此一維分析不是簡化，而是最終要達成的理想狀態。

16.2.2 控制目標

控制的目標是讓 $x(t)$ 追隨目標 $x_d(t)$ 。定義誤差：

$$e = x_d - x$$

誤差。控制的任務就是讓 $e \rightarrow 0$ ，且過程要快、要穩、不要振盪。

最基本的情形是**定點調節** (regulation)： x_d 為固定值，把物體從某處移到目標點並停住。進階的情形是**軌跡追隨** (tracking)： $x_d(t)$ 隨時間變化，物體必須持續跟上——這正是第 15 章規劃的軌跡所需要的。本章先分析定點調節，16.9 節再處理軌跡追隨。

16.3 比例控制與振盪

16.3.1 最直覺的控制律

最自然的想法是：誤差越大，出力越大。

$$u = K_p \cdot e = K_p (x_d - x)$$

比例控制 (P control)。 K_p 稱為比例增益。

代入運動方程並整理 (設 x_d 為常數，故 $\ddot{e} = -\ddot{x}$)：

$$m \ddot{x} = K_p (x_d - x) \Rightarrow m \ddot{e} + K_p e = 0 \Rightarrow \ddot{e} + (K_p/m) e = 0$$

閉環方程。這是一個標準的簡諧振盪方程。

16.3.2 結果：永不停止的振盪

上式的解為：

$$e(t) = e_0 \cos(\omega t), \quad \text{其中 } \omega = \sqrt{K_p/m}$$

純比例控制的響應。誤差以固定振幅永遠振盪，不會衰減。

這個結果與物理直覺完全吻合：**比例控制器等效於一根彈簧**。誤差就是彈簧的伸長量， K_p 就是彈簧常數。一個質量掛在無阻尼的彈簧上，會永遠上下振盪——這正是我們得到的結果。

◎ 比例控制 = 彈簧

這個類比極為有用，值得牢記。 K_p 越大，等效彈簧越硬，振盪頻率越高 ($\omega = \sqrt{K_p/m}$)，系統反應越快——但也越容易被激發振動。 K_p 越小，系統越「軟」，反應慢但也溫和。第 20 章的阻抗控制，正是刻意利用這個類比，把 K_p 設計成期望的機構剛度。

16.3.3 為何純 P 控制不可用

純比例控制有兩個致命問題。**問題一：永遠振盪**。理想情況下振幅不衰減；實際系統因有摩擦會緩慢衰減，但過程漫長且難以預測。這正是開頭故事中小翰「只看偏多少、不准預測」時球劇烈擺盪的原因。

問題二：穩態誤差。若系統受到持續的干擾力 d (如重力、摩擦)，平衡時必須有 $K_p \cdot e = d$ ，因此：

$$e_{ss} = d / K_p$$

穩態誤差與干擾成正比、與增益成反比。要消除它只能把 K_p 調到無限大——但那會使系統極度振盪且放大雜訊。

這正是開頭故事中「綁上配重後球總是偏左」的現象。解決這兩個問題，分別需要 D 項與 I 項。

16.4 微分控制與阻尼

16.4.1 加入「剎車」

要抑制振盪，需要一個與**速度**反向的力——就像空氣阻力或油壓阻尼器。這就是微分控制：

$$u = K_p e + K_d \dot{e}$$

PD 控制。 K_d 稱為微分增益。 $\dot{e}$ 是誤差的變化率。

注意：在定點調節中 x_d 為常數，故 $\dot{e} = -\dot{x}$ 。因此微分項實際上就是 $-K_d \dot{x}$ ——一個與運動速度反向的力，正是阻尼的定義。

16.4.2 閉環方程與二階系統

代入運動方程：

$$m \ddot{e} + K_d \dot{e} + K_p e = 0$$

PD 控制的閉環方程。這是機械工程中最著名的方程之一——質量—阻尼器—彈簧系統。

這個結果揭示了一個優美的對應關係：

$$m \leftrightarrow \text{質量} \quad K_d \leftrightarrow \text{阻尼係數} \quad K_p \leftrightarrow \text{彈簧常數}$$

PD 控制器把控制問題轉化為一個虛擬的機械系統。控制器設計，就是在「設計」這個虛擬系統的彈簧與阻尼器。

將方程改寫為標準二階形式：

$$\ddot{e} + 2\zeta\omega_n \dot{e} + \omega_n^2 e = 0$$

二階系統的標準形式。 ω_n 為自然頻率、 ζ 為阻尼比。

比較係數可得兩個關鍵公式：

$$\omega_n = \sqrt{K_p/m} \quad \zeta = K_d / (2\sqrt{K_p m})$$

由增益求系統參數。這兩式是 PD 控制器設計的核心。

16.4.3 三種響應

二階系統的行為完全由阻尼比 ζ 決定，可分為三種情形：

欠阻尼 ($\zeta < 1$)：系統會超越目標再回擺，逐漸衰減。振盪頻率為 $\omega_d = \omega_n\sqrt{1-\zeta^2}$ 。反應快但有超越量。

臨界阻尼 ($\zeta = 1$)：不振盪的最快響應。這是理論上的最佳點——在所有不超越的情形中，它的整定最快。

過阻尼 ($\zeta > 1$)：完全不振盪，但反應遲緩。適用於絕對不容許超越的場合。

表 16-2 三種阻尼情形的比較

情形	阻尼比 ζ	超越量	特徵	適用場合
欠阻尼	$0 < \zeta < 1$	有 (ζ 越小越大)	反應最快、有振盪	允許小幅超越、追求速度
臨界阻尼	$\zeta = 1$	無	不振盪的最快響應	理論最佳、常作為設計目標
過阻尼	$\zeta > 1$	無	反應遲緩	絕對禁止超越 (如插入作業)
無阻尼	$\zeta = 0$	100%	永不衰減的振盪	純 P 控制，不可用

圖 16-2 不同阻尼比的階躍響應

建議配圖：單一座標圖，橫軸為正規化時間 $\omega_n t$ 、縱軸為輸出（目標值為 1）。繪出五條曲線並標註阻尼比： $\zeta=0$ （等幅振盪）、 $\zeta=0.3$ （明顯超越與振盪）、 $\zeta=0.707$ （小幅超越約 4.3%，常用設計點）、 $\zeta=1$ （臨界阻尼，無超越且最快）、 $\zeta=2$ （過阻尼，緩慢）。以虛線標出 $\pm 2\%$ 的容許帶，並標註 $\zeta=0.707$ 與 $\zeta=1$ 的整定時間比較。

16.4.4 實務上的常用設計點

雖然 $\zeta = 1$ 是「不超越的最快」，實務上最常用的設計點卻是 $\zeta = 0.707$ （即 $1/\sqrt{2}$ ）。原因是它允許約 4.3% 的小幅超越，換來的是明顯更短的上升時間與整定時間——在多數應用中這個交換是划算的。此外， $\zeta = 0.707$ 在頻域上對應到最平坦的頻率響應（Butterworth 特性），對雜訊的處理也較為理想。

在機器人上則需依任務調整：取放動作可用 $\zeta \approx 0.7$ 追求速度；精密插入應用 $\zeta \geq 1$ 避免超越造成撞擊；力控制（第 20 章）則往往需要更高的阻尼以確保接觸穩定。

16.5 積分控制與穩態誤差

16.5.1 消除頑固的偏差

16.3.3 節指出：在持續干擾下，比例控制必然留下穩態誤差 $e_{ss} = d/K_p$ 。要消除它，需要一個「只要誤差存在就持續增強」的機制——這就是積分項：

$$u = K_p e + K_i \int e \, dt + K_d \dot{e}$$

完整的 PID 控制律。 K_i 為積分增益。

積分項的關鍵性質是：只要誤差不為零，積分值就持續累積，控制量就持續增加。因此系統唯一可能的平衡狀態，就是誤差恰好為零——此時積分停止增長，而累積下來的值恰好提供了抵消干擾所需的力。

$$\text{平衡時：} K_i \int e \, dt = d, \quad \text{而 } e = 0$$

積分項「記住」了需要多大的力來抵消干擾，使誤差得以歸零。

這正是開頭故事中「把過去累積的誤差加起來」的作法。它的物理意義是：控制器學會了這個干擾的大小，並持續提供補償。

16.5.2 積分的三個代價

積分項雖能消除穩態誤差，卻帶來三個明顯的代價。

代價一：降低穩定性。加入積分後，閉環系統由二階變為三階。三階系統若增益配置不當會不穩定——這是 PID 調參中最容易出問題的地方。粗略的規則是： K_i 越大越容易振盪。

代價二：增加超越量。積分項在誤差消除後仍保有累積值，這使系統傾向於「衝過頭」。因此加入積分幾乎必然使超越量增加。

代價三：反應遲滯。積分是對過去的累積，本質上是「向後看」的，因此反應較慢。這與微分項（向前預測）恰成對比。

三項作用可以這樣總結：**P 看現在、I 看過去、D 看未來**。三者互補，但也互相牽制。

16.5.3 積分飽和與防護

積分項有一個惡名昭彰的實務問題：**積分飽和**（integral windup）。

問題發生在控制量達到物理極限時（馬達力矩飽和、PWM 達到 100%）。此時即使控制器要求更大的輸出，實際系統也無法提供，誤差持續存在，積分項卻仍在不斷累積——可能累積到極大的值。

當系統終於接近目標時，這個巨大的積分值仍在推動系統，造成嚴重的超越；而要「消掉」這個積分值，必須讓誤差反向累積同樣長的時間。結果是巨大的超越加上漫長的振盪。

三種標準的防護方法：

- **條件積分（Conditional Integration）**：偵測到控制量飽和時，暫停積分累積。最簡單有效，是嵌入式系統的首選。
- **積分限幅（Clamping）**：直接限制積分項的最大值，通常設為控制量上限的某個比例。實作簡單但需調整限幅值。
- **反飽和回授（Back-calculation）**：把「要求輸出」與「實際輸出」的差值回饋去減少積分項，飽和越嚴重、積分被削減越多。效果最平順，是工業控制器的標準作法。

◎ 一個常見的現場故障

某輸送設備在正常運轉時控制良好，但每次「開機時工件卡住」之後就會失控暴衝。原因就是積分飽和：卡住期間馬達力矩飽和、誤差持續、積分累積到極大值；工件一鬆脫，那個巨大的積分值立刻推動馬達全速運轉。這類故障的特徵是「平常都好，異常之後更糟」——只要看到這個模式，第一個該檢查的就是積分飽和防護。

16.6 增益設計與時域規格

16.6.1 由規格反推增益

實務上的設計流程不是「隨便試增益」，而是**先訂規格、再算增益**。常用的三個時域規格為：

上升時間 t_r ：輸出由 10% 上升到 90% 所需的時間。它衡量反應速度。

超越量 M_p ：最大超過目標值的百分比。它由阻尼比唯一決定：

$$M_p = \exp(-\pi\zeta / \sqrt{1-\zeta^2}) \times 100\%$$

超越量與阻尼比的關係。 $\zeta=0.707$ 對應 4.3%、 $\zeta=0.5$ 對應 16.3%、 $\zeta=0.3$ 對應 37.2%。

整定時間 t_s ：輸出進入並維持在目標值 $\pm 2\%$ 帶內所需的時間。近似公式為：

$$t_s \approx 4 / (\zeta \omega_n)$$

整定時間近似式（2% 準則）。注意它只與 $\zeta \omega_n$ 的乘積有關——這個乘積就是指數衰減的速率。

16.6.2 設計流程

標準的三步驟設計流程：

1. 由超越量規格決定 ζ 。若要求超越量不超過 5%，則 $\zeta \geq 0.69$ ，取 $\zeta = 0.707$ 。
2. 由整定時間規格決定 ω_n 。由 $t_s \approx 4/(\zeta \omega_n)$ 反推： $\omega_n \approx 4/(\zeta t_s)$ 。
3. 由 ζ 與 ω_n 計算增益： $K_p = m \omega_n^2$ 、 $K_d = 2 \zeta \omega_n m$ 。

這個流程把「調參數」從試誤變成了設計。例題 16-2 會完整走一遍。

16.6.3 增益的上限：三個限制

由 $K_p = m \omega_n^2$ 可知，要反應更快（大 ω_n ）就需要更大的增益。那麼能否無限提高？答案是不能，有三個限制。

限制一：致動器飽和。大增益會要求大力矩，但馬達有極限（第 5 章）。一旦飽和，實際的閉環行為就與設計不符，且可能觸發積分飽和。

限制二：雜訊放大。微分項會放大感測器雜訊——若編碼器讀值有微小抖動，微分後會變成劇烈的雜訊，再乘上 K_d 送給馬達，造成馬達嗡嗡作響甚至發熱。這是 K_d 的主要限制。

限制三：未建模動態。最根本也最危險的限制。真實的機構並非剛體——連桿有彈性、減速機有扭轉柔度、皮帶會伸長。這些「未建模」的柔性有其自身的共振頻率。若控制器的頻寬

ω_n 接近或超過這個共振頻率，就會激發共振，系統立刻失控。

經驗法則：控制器頻寬 $\omega_n < (1/3 \sim 1/5) \times$ 最低結構共振頻率

這是機電系統設計的黃金法則。想要更高的頻寬，必須先提高機構的剛性。

這正是第 13 章開頭故事「調高增益治標不治本」的完整答案：增益調高會依序遇到雜訊放大（馬達嗡嗡叫）、致動器飽和（力矩不足）、最後是激發共振（劇烈振動甚至損壞）。真正的解方不是提高增益，而是加入前饋（16.9 節與第 17 章）。

16.7 數位實作

16.7.1 從連續到離散

前面的分析都在連續時間中進行，但實際的控制器是數位的——每隔一個固定的取樣週期 T 執行一次。這帶來三個必須處理的議題。

議題一：微分的近似。導數必須用差分近似：

$$\dot{e}_k \approx (e_k - e_{k-1}) / T$$

後向差分。最簡單但對雜訊極為敏感——雜訊被除以一個小數 T ，等於被放大了 $1/T$ 倍。

議題二：積分的近似。積分用累加近似：

$$\int e \, dt \approx \sum e_k \cdot T$$

矩形積分。也可用梯形法提高精度，但實務上矩形法通常已足夠。

議題三：取樣延遲。從量測到輸出至少延遲一個週期，這相當於在迴路中加入了純延遲，會降低穩定裕度。

16.7.2 取樣週期的選擇

取樣週期太長會導致三個問題：延遲增大（穩定性下降）、離散化誤差、以及可能的混疊（第 6.9 節）。太短則增加運算負擔。

經驗法則是：

$$\text{取樣頻率 } f_s \geq (10 \sim 20) \times \text{閉環頻寬 } f_{bw} \quad (f_{bw} = \omega_n / 2\pi)$$

取樣頻率的選擇準則。若閉環頻寬為 10 Hz，取樣頻率應至少 100 ~ 200 Hz。

這解釋了第 3 章的時間尺度金字塔：機器人的關節位置迴路通常在 1 kHz、速度迴路數 kHz、電流迴路可達數十 kHz——每一層的取樣頻率都遠高於其閉環頻寬。

16.7.3 微分項的雜訊處理

微分放大雜訊是 PID 數位實作最大的實務難題。三種標準對策：

對策一：微分項濾波。在微分後加一個一階低通濾波器（第 6.10.2 節）。這是最常見的作法，稱為「不完全微分」。

$$D_k = \alpha D_{k-1} + (1-\alpha) K_d (e_k - e_{k-1})/T$$

帶濾波的分項。 α 越大濾波越強，但延遲也越大。

對策二：用速度量測取代微分。若系統有速度感測器（或編碼器能可靠地計算速度），直接使用速度值，避免對位置微分。這是伺服驅動器的標準作法。

對策三：對量測值微分而非誤差。在追隨變化的目標時，目標值的突變會使誤差微分產生巨大脈衝（稱為 derivative kick）。改為只對輸出 y 微分（並取負號），可避免這個問題：

$$u = K_p e + K_i \int e dt - K_d \dot{y}$$

微分作用於輸出而非誤差。這是工業 PID 控制器的標準實作。

16.8 PID 調參方法

16.8.1 手動調參的標準流程

即使有 16.6 節的設計公式，實務上仍需微調——因為真實系統有摩擦、彈性與參數不確定性。標準的手動調參流程為：

1. **全部歸零：** $K_p = K_i = K_d = 0$ 。
2. **先調 K_p ：**逐步增大直到系統出現持續的等幅振盪，記下此時的增益（稱為臨界增益 K_u ）與振盪週期 T_u ，然後把 K_p 降到約 K_u 的一半。
3. **再調 K_d ：**逐步增大以抑制振盪，直到超越量符合規格。若馬達開始發出嗡嗡聲或高頻抖動，代表 K_d 過大（雜訊放大）。
4. **最後調 K_i ：**只在確實存在穩態誤差時才加入，由小值開始逐步增大，直到誤差在可接受時間內消除。若出現緩慢的振盪或超越量明顯增加，代表 K_i 過大。

5. 加入防飽和：確認積分飽和防護已啟用（16.5.3 節）。
6. 全工作範圍驗證：在不同姿態、不同負載下測試——機器人的等效慣量隨姿態變化（第 13 章的 $M(q)$ ），單一組增益未必在所有姿態下都適用。

16.8.2 Ziegler-Nichols 法

最著名的經驗調參法是 Ziegler-Nichols 法。以臨界增益法為例：先求出臨界增益 K_u 與振盪週期 T_u （步驟同上），再依下表設定：

表 16-3 Ziegler-Nichols 臨界增益法的參數建議

控制器類型	K_p	K_i	K_d
P	$0.5 K_u$	—	—
PI	$0.45 K_u$	$1.2 K_p / T_u$	—
PID	$0.6 K_u$	$2 K_p / T_u$	$K_p T_u / 8$

必須注意的限制：Ziegler-Nichols 法的設計目標是「對干擾的快速抑制」，其結果通常有較大的超越量（約 25%），對追求精度的機器人應用往往過於激進。它適合作為**起點**，之後再依 16.8.1 節的流程微調。此外，求臨界增益的過程需要讓系統實際振盪，這在實體機器人上有風險，應在低速、無負載、有防護的情況下進行。

16.8.3 機器人的特殊考量

將 PID 應用於機器人關節時，有三個特殊考量。

考量一：增益隨姿態調整。等效慣量 $M(q)$ 隨姿態變化可達數倍（第 13.6.1 節）。若用固定增益，在低慣量姿態下會過於激進、在高慣量姿態下又太遲鈍。解法是**增益排程**（gain scheduling）：依姿態查表或計算調整增益，使 ω_n 與 ζ 維持恆定。

考量二：重力補償優先。重力是持續存在的干擾（第 13.8.1 節），若讓積分項去對抗它，會導致響應遲緩與超越。正確作法是用**前饋直接補償重力**，讓積分項只需處理摩擦等較小的殘餘干擾。這是第 17 章的第一個主題。

考量三：關節間的耦合。獨立的關節 PID 把耦合項（第 13.6.2 節的 M_{12} ）視為未知干擾。低速時影響小，高速時則成為主要誤差來源——這正是第 13 章開頭故事的核心，也是第 17 章計算力矩控制要解決的問題。

16.9 軌跡追隨與前饋

16.9.1 追隨誤差：回授的先天限制

前面的分析都針對定點調節。但機器人的實際任務是追隨第 15 章規劃的軌跡——目標值持續變化。此時會出現一個純回授控制無法避免的問題：**追隨誤差**。

原因很根本：**回授控制必須先有誤差才會出力**。當目標以固定速度移動時，控制器需要持續輸出力量來維持運動，而這個力量來自 $K_p \cdot e$ ，因此誤差 e 不可能為零。速度越快、所需的力越大、誤差也越大。

$$\text{等速追隨時的穩態誤差：} e_{ss} \approx (m \ddot{x}_d + \text{摩擦}) / K_p$$

追隨誤差與所需的力成正比、與增益成反比。這是純回授的先天限制。

這解釋了一個常見現象：機器人在定點時精度很高（ $\pm 0.02 \text{ mm}$ ），但在運動中的動態精度差得多（ $\pm 1 \text{ mm}$ 以上）。工業機器人型錄上標示的「重複精度」是靜態的，動態的「路徑精度」通常要差一到兩個數量級。

16.9.2 前饋：不等誤差發生就先出力

解決之道是**前饋**（feedforward）：既然我們**事先知道**目標軌跡（第 15 章已完整規劃），就能預先計算出所需的力，直接送給馬達，不必等誤差出現。

$$u = u_{ff} + u_{fb} = m \ddot{x}_d + [K_p e + K_d \dot{e}]$$

前饋加回授的控制律。第一項由模型預先計算、第二項處理模型誤差與干擾。

這個架構的思想極為重要，可以總結為一句話：**用模型處理已知的部分，用回授處理未知的部分**。

效果的量化：若模型完全正確，前饋項會提供全部所需的力，誤差理論上為零，回授項不需出力。若模型有 20% 的誤差，回授項只需處理那 20%——相較於純回授要處理 100%，誤差降為五分之一。

16.9.3 三個層次的前饋

前饋可以由粗到細分為三個層次，複雜度與效果遞增：

表 16-4 三個層次的前饋

層次	前饋項	所需資訊	誤差改善	本書章節
速度前饋	$K_v \cdot \dot{x}_d$	目標速度	小 (僅補償黏滯摩擦)	本章
加速度前饋	$m \cdot \ddot{x}_d$	目標加速度、質量估計	中 (補償慣性)	本章
完整動力學前饋	$M(q)\ddot{q}_d + C\dot{q}_d + G$	完整動力學模型	大 (補償慣性、耦合、重力)	第 17 章

第三層次正是第 17 章的**計算力矩控制**。它把第 13、14 章建立的動力學模型直接用於前饋，是本書「模型服務於控制」思想的完整實現。

◎ 前饋與回授的分工

一個生活化的比喻：開車轉彎時，有經驗的駕駛在**進入彎道前**就開始轉方向盤（前饋——依據對彎道的預知）；同時也持續看著車道線做微調（回授——處理路面、風、車況的不確定性）。新手駕駛只會做後者，於是總是壓線後才修正，開得又慢又晃。機器人控制完全相同：純 PID 是新手駕駛、加入前饋才是老手。

16.10 PID 的極限

16.10.1 PID 做得好的事

公允地說，PID 在多數場合表現極佳，這也是它統治工業界的原因。它的優勢是：**不需要模型**（只需調三個參數）、**實作簡單**（幾行程式碼）、**魯棒**（對參數變化有一定容忍度）、**直覺**（三項的作用清楚可解釋）。

對於變化緩慢、耦合弱、干擾小的系統，PID 幾乎總是足夠。這涵蓋了工業界九成以上的控制迴路。

16.10.2 PID 做不好的事

但在機器人上，PID 會遇到四個結構性的限制：

限制一：非線性。機器人的動力學是非線性的（第 13 章），而 PID 是線性控制器。在某個工作點調好的增益，換到另一個姿態就可能不適用。

限制二：耦合。獨立關節 PID 把耦合項視為干擾。低速時可忽略，高速時（科氏力與速度平方成正比）成為主要誤差源。

限制三：追隨誤差。如 16.9.1 節所述，純回授必然有追隨誤差，且與速度成正比。

限制四：增益上限。如 16.6.3 節所述，增益受雜訊、飽和與結構共振三重限制，無法靠調高增益解決上述問題。

這四點合起來，正是第 13 章開頭故事的完整診斷：**低速時 PID 完美（非線性與耦合效應小），高速時失準（四個限制同時發作），而調高增益碰到天花板。**真正的解方是引入模型——這就是第 17 章的主題。

16.11 由線性邁向非線性

16.11.1 本章的成果

本章建立了控制的基礎。你現在能夠：區分開迴路與閉迴路、理解 P、I、D 三項各自的物理意義（彈簧、記憶、阻尼）、由時域規格反推增益、辨識並防止積分飽和、處理數位實作的取樣與雜訊問題、以及理解前饋與回授的分工。

最重要的是三個核心洞見：**PD 控制等效於一個虛擬的彈簧—阻尼系統**（16.4.2 節）——這個觀點將在第 20 章的阻抗控制中發揮關鍵作用；**增益有三重上限**（16.6.3 節），因此「調高增益」不是萬能解；以及**用模型處理已知、用回授處理未知**（16.9.2 節）——這是現代控制的核心哲學。

16.11.2 接下來：把模型放進控制器

第 17 章（非線性控制）將直接處理本章留下的四個限制。核心方法是**計算力矩控制**：用第 13、14 章的動力學模型做前饋，把非線性的機器人「線性化」成 n 個獨立的一維點質量——而那正是本章分析得最透徹的系統。屆時你會發現，本章的所有結論（ ζ 、 ω_n 、增益設計）都能直接套用，只是作用在「已被線性化」的系統上。

同時第 17 章也會介紹**重力補償**（把 $G(q)$ 直接前饋，這是最簡單也最有效的一步）與**操作空間控制**（直接在笛卡兒空間設計控制律）。

第 18~19 章則提供更嚴謹的工具：最佳控制（如何系統性地選擇「最好」的增益）與 Lyapunov 穩定性（如何**證明**控制器不會失控，而非只是實驗上看起來穩定）。

換句話說：第 16 章教你**不用模型也能控制**，第 17 章教你**用模型控制得更好**，第 18~19 章

教你證明它真的可靠。

完整計算例題

例題 16-1 PD 控制的參數分析

題目：某機器人關節的等效慣量 $m = 0.5 \text{ kg}\cdot\text{m}^2$ (此處以轉動慣量表示，控制量為力矩)。
 設定 $K_p = 200 \text{ N}\cdot\text{m}/\text{rad}$ 、 $K_d = 20 \text{ N}\cdot\text{m}\cdot\text{s}/\text{rad}$ 。(a) 求自然頻率與阻尼比。(b) 判斷響應類型並估算超越量。(c) 求整定時間。(d) 若要達到臨界阻尼， K_d 應設為多少？

解 (a)：由 16.4.2 節的公式：

$$\omega_n = \sqrt{(K_p/m)} = \sqrt{(200/0.5)} = \sqrt{400} = 20 \text{ rad/s}$$

自然頻率，約 3.18 Hz。

$$\zeta = K_d / (2\sqrt{(K_p m)}) = 20 / (2\sqrt{(200 \times 0.5)}) = 20 / (2 \times 10) = 1.0$$

阻尼比恰為 1。

解 (b)： $\zeta = 1.0$ 恰為臨界阻尼。這是不振盪的最快響應，超越量為 0%。

解 (c)：由 16.6.1 節的近似式：

$$t_s \approx 4/(\zeta \omega_n) = 4/(1.0 \times 20) = 0.2 \text{ 秒}$$

整定時間約 0.2 秒。

解 (d)：題目已是臨界阻尼，故 $K_d = 20$ 即為答案。若要驗證：臨界阻尼要求 $\zeta = 1$ ，即 $K_d = 2\sqrt{(K_p m)} = 2\sqrt{100} = 20 \checkmark$

延伸觀察：若把 K_p 加倍到 400，則 $\omega_n = \sqrt{800} \approx 28.3 \text{ rad/s}$ (反應更快)，但若 K_d 維持 20， ζ 會降為 $20/(2\sqrt{200}) = 0.707$ ——變成欠阻尼，出現約 4.3% 的超越。這說明了一個重要的實務要點：**調整 K_p 時必須同步調整 K_d ，才能維持期望的阻尼比。**具體關係為 $K_d \propto \sqrt{K_p}$ 。

例題 16-2 由規格設計增益

題目：某關節等效慣量 $m = 0.8 \text{ kg}\cdot\text{m}^2$ 。設計規格要求：超越量不超過 5%、整定時間不超過 0.3 秒。(a) 由規格決定 ζ 與 ω_n 。(b) 計算所需的 K_p 與 K_d 。(c) 驗證設計。(d) 若該關節的最低結構共振頻率為 25 Hz，判斷此設計是否安全。

解 (a)：由超越量決定 ζ 。由 16.6.1 節的公式，要求 $M_p \leq 5\%$ ：

$$\exp(-\pi\zeta/\sqrt{1-\zeta^2}) \leq 0.05 \Rightarrow \pi\zeta/\sqrt{1-\zeta^2} \geq 3.0 \Rightarrow \zeta \geq 0.690$$

取常用值 $\zeta = 0.707$ (對應超越量 4.3% · 滿足規格)。

由整定時間決定 ω_n ：

$$t_s \approx 4/(\zeta\omega_n) \leq 0.3 \Rightarrow \omega_n \geq 4/(0.707 \times 0.3) = 18.86 \text{ rad/s}$$

取 $\omega_n = 20 \text{ rad/s}$ (留一些餘裕)。

解 (b)：由 16.6.2 節的公式：

$$K_p = m \omega_n^2 = 0.8 \times 400 = 320 \text{ N}\cdot\text{m/rad}$$

比例增益。

$$K_d = 2 \zeta \omega_n m = 2 \times 0.707 \times 20 \times 0.8 = 22.6 \text{ N}\cdot\text{m}\cdot\text{s/rad}$$

微分增益。

解 (c)：驗證 $\omega_n = \sqrt{(320/0.8)} = \sqrt{400} = 20 \checkmark$ 、 $\zeta = 22.6/(2\sqrt{(320 \times 0.8)}) = 22.6/(2 \times 16) = 0.707 \checkmark$ 。預期超越量 $4.3\% < 5\% \checkmark$ 、整定時間 $4/(0.707 \times 20) = 0.283 \text{ s} < 0.3 \text{ s} \checkmark$ 。設計符合所有規格。

解 (d)：控制器頻寬 $\omega_n = 20 \text{ rad/s} = 3.18 \text{ Hz}$ 。結構共振頻率為 25 Hz 。比值為 $25/3.18 \approx 7.9$ 倍。

依 16.6.3 節的黃金法則 (頻寬應小於共振頻率的 $1/3$ 到 $1/5$ ，即比值至少 $3 \sim 5$)，本設計的比值 7.9 相當安全，還有提高頻寬的空間。若要進一步縮短整定時間，可把 ω_n 提高到約 30 rad/s (比值仍有 5.2)，此時 $t_s \approx 0.19$ 秒。

工程判斷：這個餘裕分析是控制器設計中最容易被忽略卻最重要的一步。許多「調不好」的系統，實際上是因為要求的頻寬已經逼近結構共振——此時無論怎麼調參數都會振盪，唯一的解方是提高機構剛性或降低性能要求。

例題 16-3 追隨誤差與前饋的效益

題目：承例題 16-2 的關節 ($m = 0.8$ 、 $K_p = 320$ 、 $K_d = 22.6$)。現要追隨一段等加速度軌跡，加速度為 $\ddot{x}_d = 5 \text{ rad/s}^2$ 。(a) 求純 PD 控制下的追隨誤差。(b) 若加入加速度前饋且質量估計準確，求誤差。(c) 若質量估計有 20% 誤差 (估為 0.64)，求誤差。(d) 比較三者並說明結論。

解 (a)：純 PD 控制時，穩態下系統以固定加速度運動，所需的力矩為 $m \cdot \ddot{x}_d$ ，這個力矩必須由 $K_p \cdot e$ 提供（穩態時 $\dot{e} \approx 0$ ）：

$$K_p \cdot e_{ss} = m \ddot{x}_d \Rightarrow e_{ss} = m \ddot{x}_d / K_p = (0.8 \times 5) / 320 = 4.0 / 320 = 0.0125 \text{ rad}$$

追隨誤差約 $0.0125 \text{ rad} \approx 0.716^\circ$ 。

若關節後接一支 0.5 m 的連桿，末端誤差約為 $0.0125 \times 0.5 = 6.25 \text{ mm}$ ——這對精密作業是不可接受的。

解 (b)：加入前饋 $u_{ff} = m \ddot{x}_d = 0.8 \times 5 = 4.0 \text{ N}\cdot\text{m}$ 。此力矩恰好提供全部所需，回授項不需出力：

$$e_{ss} = 0 \text{ rad}$$

誤差理論上為零。這是完美模型下的理想結果。

解 (c)：質量估計為 $\hat{m} = 0.64$ （低估 20%），前饋提供 $0.64 \times 5 = 3.2 \text{ N}\cdot\text{m}$ ，尚缺 $4.0 - 3.2 = 0.8 \text{ N}\cdot\text{m}$ ，由回授補足：

$$e_{ss} = 0.8 / 320 = 0.0025 \text{ rad} \approx 0.143^\circ$$

誤差降為純回授的 $1/5$ ，末端誤差約 1.25 mm 。

解 (d)：三者比較如下表：

表 16-5 三種控制方式的追隨誤差比較

控制方式	前饋力矩	回授需補足	追隨誤差	末端誤差 (0.5 m 臂)
純 PD	0 N·m	4.0 N·m	0.0125 rad	6.25 mm
前饋 (模型準確)	4.0 N·m	0 N·m	0 rad	0 mm
前饋 (模型誤差 20%)	3.2 N·m	0.8 N·m	0.0025 rad	1.25 mm

結論：這個例題量化了 16.9.2 節的核心論點——誤差的改善倍數，恰等於模型的準確度倍數。模型準確度 80%（誤差 20%），誤差就降為 $1/5$ 。這也說明了為何「不完美的模型仍然極有價值」：即使模型只有 80% 準確，效果也遠勝於把增益調高五倍（後者會遭遇 16.6.3 節的三重限制）。這正是第 17 章的核心思想。

程式範例：PID 控制器與階躍響應分析

以下 Python 程式實作完整的 PID 控制器（含防飽和與微分濾波），在一維點質量與平面 2R 手臂上測試，並比較不同增益、有無前饋的效果。

```
import numpy as np

# ===== PID 控制器 =====
class PID:
    """含防積分飽和與微分濾波的 PID 控制器"""
    def __init__(self, kp, ki=0.0, kd=0.0, dt=0.001,
                 u_max=np.inf, d_filter=0.0, derivative_on_output=True):
        self.kp, self.ki, self.kd, self.dt = kp, ki, kd, dt
        self.u_max = u_max
        self.alpha = d_filter          # 微分濾波係數 (0=不濾波)
        self.d_on_out = derivative_on_output  # 對輸出微分，避免 derivative kick
        self.integral = 0.0
        self.prev_e = None
        self.prev_y = None
        self.d_term = 0.0

    def reset(self):
        self.integral = 0.0
        self.prev_e = self.prev_y = None
        self.d_term = 0.0

    def update(self, r, y, ydot=None, u_ff=0.0):
        e = r - y
        # ----- 比例項 -----
        p_term = self.kp * e
        # ----- 微分項 -----
        if ydot is not None:          # 有速度量測則直接使用 (最佳)
            raw_d = -self.kd * ydot if self.d_on_out else 0.0
        elif self.d_on_out and self.prev_y is not None:
            raw_d = -self.kd * (y - self.prev_y) / self.dt
        elif self.prev_e is not None:
            raw_d = self.kd * (e - self.prev_e) / self.dt
        else:
            raw_d = 0.0
        self.d_term = self.alpha*self.d_term + (1-self.alpha)*raw_d
        # ----- 積分項 (含條件積分防飽和) -----
        i_term = self.ki * self.integral
        u_unsat = u_ff + p_term + i_term + self.d_term
        u = np.clip(u_unsat, -self.u_max, self.u_max)
        if abs(u_unsat - u) < 1e-9:    # 未飽和才累積積分
            self.integral += e * self.dt
        self.prev_e, self.prev_y = e, y
        return u

# ===== 一維點質量模擬 =====
def simulate_1d(kp, kd, ki=0.0, m=0.8, T=1.0, dt=0.001,
               target=1.0, disturbance=0.0, u_max=np.inf):
```

```

pid = PID(kp, ki, kd, dt, u_max=u_max)
x, v = 0.0, 0.0
ts, xs, us = [], [], []
for k in range(int(T/dt)):
    u = pid.update(target, x, ydot=v)
    a = (u + disturbance) / m          # 含外部干擾
    v += a*dt; x += v*dt              # 半隱式歐拉 (第 14 章)
    ts.append(k*dt); xs.append(x); us.append(u)
return np.array(ts), np.array(xs), np.array(us)

def analyze(ts, xs, target=1.0):
    """計算時域規格：超越量、整定時間、穩態誤差"""
    Mp = (xs.max() - target)/target*100 if xs.max() > target else 0.0
    band = 0.02*target
    idx = np.where(np.abs(xs - target) > band)[0]
    ts_settle = ts[idx[-1]] if len(idx) else 0.0
    return Mp, ts_settle, target - xs[-1]

# ----- 驗證：例題 16-1 與 16-2 -----
print("=== 例題 16-1：不同阻尼比的響應 ===")
m = 0.5
print(f"{'K_p':>6}{ 'K_d':>7}{ 'wn':>7}{ 'ζ':>7}{ '超越量':>9}{ '整定時間':>10}")
for kp, kd in ((200, 20), (200, 10), (200, 30), (400, 20)):
    wn = np.sqrt(kp/m); zeta = kd/(2*np.sqrt(kp*m))
    ts, xs, _ = simulate_1d(kp, kd, m=m, T=1.0)
    Mp, tset, _ = analyze(ts, xs)
    print(f"{kp:6.0f}{kd:7.1f}{wn:7.1f}{zeta:7.3f}{Mp:8.1f}%{tset:9.3f}s")

print("\n=== 例題 16-2：由規格設計增益 ===")
m2, zeta_d, wn_d = 0.8, 0.707, 20.0
kp = m2*wn_d**2; kd = 2*zeta_d*wn_d*m2
print(f"設計：ζ={zeta_d}, wn={wn_d} → K_p={kp:.1f}, K_d={kd:.2f}")
ts, xs, _ = simulate_1d(kp, kd, m=m2, T=0.8)
Mp, tset, _ = analyze(ts, xs)
print(f"實測：超越量={Mp:.2f}% (規格 <5%) 整定時間={tset:.3f}s (規格 <0.3s)")
print(f"結構共振 25 Hz vs 控制頻寬 {wn_d/(2*np.pi):.2f} Hz "
      f"→ 比值 {25/(wn_d/(2*np.pi)):.1f} 倍 (安全) ")

# ----- 穩態誤差與積分項 -----
print("\n=== 積分項消除穩態誤差 (持續干擾 -3 N) ===")
for ki, tag in ((0.0, "PD "), (300.0, "PID ")):
    ts, xs, _ = simulate_1d(kp, kd, ki, m=m2, T=2.0, disturbance=-3.0)
    _, _, ess = analyze(ts, xs)
    print(f" {tag}: 穩態誤差 = {ess:8.5f} "
          f" (理論 PD 值 = {3.0/kp:.5f}) ")

# ----- 積分飽和示範 -----
print("\n=== 積分飽和防護的效果 (力矩上限 5 N·m) ===")
for prot, tag in ((True, "有防護"), (False, "無防護")):
    pid = PID(kp, 300.0, kd, 0.001, u_max=5.0)
    x, v, xs2 = 0.0, 0.0, []

```



```

for k in range(3000):
    e = 1.0 - x
    p_t, d_t = pid.kp*e, -pid.kd*v
    i_t = pid.ki*pid.integral
    u_un = p_t + i_t + d_t
    u = np.clip(u_un, -5.0, 5.0)
    if prot and abs(u_un - u) > 1e-9:
        pass # 飽和時暫停積分
    else:
        pid.integral += e*0.001
        a = u/m2; v += a*0.001; x += v*0.001
        xs2.append(x)
xs2 = np.array(xs2)
print(f" {tag}: 最大超越 = {(xs2.max()-1.0)*100:6.2f}% "
      f"最終值 = {xs2[-1]:.4f}")

# ----- 例題 16-3：前饋的效益 -----
print("\n=== 例題 16-3：前饋控制的追隨誤差 ===")
acc = 5.0
for m_hat, tag in ((0.0, "純 PD"),
                   (0.8, "前饋(模型準確)"),
                   (0.64, "前饋(誤差 20%)")):
    pid = PID(kp, 0.0, kd, 0.001)
    x, v = 0.0, 0.0
    for k in range(1500):
        t = k*0.001
        xd, vd = 0.5*acc*t**2, acc*t # 等加速度目標
        u = pid.update(xd, x, ydot=v, u_ff=m_hat*acc)
        a = u/m2; v += a*0.001; x += v*0.001
        err = 0.5*acc*1.5**2 - x
    print(f" {tag}: 追隨誤差 = {err:9.6f} rad "
          f"= {np.rad2deg(err):7.4f}° 末端(0.5m) = {err*500:7.3f} mm")

```

請試著修改：（一）把 K_d 設為 0 觀察純 P 控制的等幅振盪。（二）在微分項加入模擬的感測雜訊（在 y 上加高斯雜訊），比較有無微分濾波的控制量抖動程度。（三）把質量 m 改為隨時間變化（模擬機器人姿態改變導致的慣量變化），觀察固定增益的性能如何劣化——這將說明第 17 章增益排程的必要性。

實作：實體關節的 PID 調參

本章實作是在實體平台上完成一次完整的 PID 調參，並量化各項的作用。建議使用附錄 F 的循跡車馬達、或任何具備編碼器回授的關節。

- 系統辨識：**先量測系統的等效慣量。作法是施加已知的階躍力矩，記錄角加速度，由 $\tau = J\alpha$ 反推 J 。重複三次取平均。

2. **理論設計**：依 16.6.2 節的流程，設定規格（建議：超越量 $< 10\%$ 、整定時間 $< 0.5\text{ s}$ ），計算理論增益 K_p 與 K_d 。
3. **純 P 測試**：只設 K_p （用理論值），觀察階躍響應。記錄振盪頻率並與理論值 $\omega_n = \sqrt{(K_p/J)}$ 比較。這一步驗證你的慣量估計是否正確。
4. **加入 K_d** ：由 0 逐步增加到理論值，記錄每一步的超越量與整定時間。找出實際的臨界阻尼點，並與理論值比較（差異來自摩擦提供的額外阻尼）。
5. **雜訊上限探索**：繼續增大 K_d 直到馬達出現可聽見的嗡嗡聲或高頻抖動。記錄此臨界值——這就是雜訊放大的實際上限（16.6.3 節限制二）。
6. **穩態誤差量測**：在系統上施加一個固定的外力（如掛砝碼或用手輕壓），量測穩態誤差，驗證 $e_{ss} = d/K_p$ 的關係。
7. **加入 K_i** ：由小值逐步增加直到穩態誤差在 1 秒內消除。觀察超越量的增加幅度。
8. **積分飽和實驗**：在系統啟動前先用手擋住不讓它動 3 秒，再放開。分別測試有無防飽和保護的行為差異——你會親眼看到 16.5.3 節描述的暴衝現象。
9. **前饋效益驗證**：規劃一段等加速度軌跡（第 15 章），分別以純 PID 與加入加速度前饋執行，比較追隨誤差。驗證例題 16-3 的結論。
10. **共振頻率探索**（選做，須謹慎）：在低增益下用手快速撥動連桿末端，用加速度計或觀察振動記錄其自然振動頻率。這就是結構共振頻率，據以檢驗你的控制頻寬是否安全。

表 16-6 PID 調參記錄表（前兩列為範例）

步驟	K_p	K_d	K_i	超越量	整定時間	穩態誤差	觀察
純 P（理論值）	320	0	0	持續振盪	—	0.012 rad	頻率 3.2 Hz，接近理論 3.18 Hz
加入 K_d （理論值）	320	22.6	0	5.8%	0.31 s	0.012 rad	略高於理論，摩擦影響
（自行填寫其餘步驟）							

國中小也看得懂：在手上立竹竿的三個祕訣

玩過在手掌上立一根竹竿的遊戲嗎？如果閉上眼睛，竿子一秒就倒了；張開眼睛，你就能一

直讓它立著。為什麼？因為你**看得到它往哪邊倒，就往哪邊移動手**。這個「看→修正」的循環，就是機器人控制最重要的道理。

但光是「往倒的方向移動」還不夠。有三件事要一起做，工程師叫它們 **P、I、D**：

P 是「現在差多少」。竿子歪得越多，你就要移動得越多。這很直覺對不對？可是只做這件事的話，竿子會左右晃個不停——因為你總是等它歪了才反應，扳回來的時候又衝過頭了。

D 是「正在多快地變糟」。高手會看竿子倒的速度：雖然現在只歪一點點，但倒得很快，就要趕快修！這就像騎腳踏車時的煞車——不是等快撞到才煞，而是看到前面有狀況就先減速。有了 D，晃動就會停下來。

I 是「已經歪了多久」。如果竿子上黏了一塊黏土讓它老是往左倒，你會發現不管怎樣它總是偏左一點點。這時候要記住：「我已經偏左很久了，要再多用一點力！」把過去累積的偏差加起來一起修正，就能真正回到正中央。

記住這句話就好：**P 看現在、D 看未來、I 看過去**。你家的冷氣、電梯、汽車的定速巡航，還有工廠裡所有的機器手臂，通通都在用這三個字母控制。你在附錄 F 做的循跡車，如果只用 P 會蛇行，加上 D 就會變得又直又穩——不信你試試看！

叫獸提醒與常見錯誤

- 誤區一：以為調高 K_p 就能提高精度。增益受雜訊放大、致動器飽和與結構共振三重限制，超過就會振盪或失控。
- 誤區二：調 K_p 卻不同步調 K_d 。阻尼比 $\zeta = K_d / (2\sqrt{K_p m})$ ， K_p 加倍時 K_d 需乘以 $\sqrt{2}$ 才能維持相同阻尼。
- 誤區三：忘記積分飽和防護。這是「平常都好、卡住之後暴衝」這類故障的第一嫌疑。
- 誤區四：對誤差微分而非對輸出微分。目標值突變時會產生 derivative kick，使控制量出現巨大脈衝。
- 誤區五：微分項未濾波。感測器雜訊被除以取樣週期（放大 $1/T$ 倍）再乘上 K_d ，會使馬達發熱與異音。
- 誤區六：取樣頻率不足。應至少為閉環頻寬的 10~20 倍，否則延遲會嚴重侵蝕穩定裕度。
- 誤區七：用同一組增益走遍所有姿態。機器人的等效慣量隨姿態變化可達數倍，應考慮增益

排程。

- 誤區八：用積分項去對抗重力。重力是可預測的，應以前饋直接補償；讓積分處理它會導致響應遲緩與超越。
- 誤區九：以為定點精度就是動態精度。純回授必然有追隨誤差，且與速度成正比，兩者可差一到兩個數量級。
- 誤區十：忽略結構共振。控制頻寬應低於最低共振頻率的 $1/3 \sim 1/5$ ；若已逼近，任何調參都無法解決，只能提高機構剛性。
- 誤區十一：直接套用 Ziegler-Nichols 的結果。它針對干擾抑制設計，超越量可達 25%，對機器人往往過於激進，應作為起點而非終點。
- 誤區十二：在實體機器人上求臨界增益。讓系統實際振盪有損壞風險，應在低速、無負載、有防護的條件下進行，或改用模型計算。

課堂討論

1. 開頭故事中，小翰蒙眼、只看位置、加上配重三種情境，分別對應 PID 的哪個面向？請為每種情境設計一個機器人上的對應實例。
2. PD 控制等效於一個虛擬的彈簧—阻尼系統。請討論這個觀點如何幫助我們設計「與人接觸時安全」的機器人（提示：對照第 20 章的阻抗控制與第 5 章的反向驅動性）。
3. 第 13 章的故事中，小好想調高增益解決高速失準。請用本章 16.6.3 節與 16.10.2 節的內容，完整說明為何這個方向行不通，並指出正確的解方。
4. 例題 16-3 顯示「模型準確度 80% 就能使誤差降為 $1/5$ 」。請討論：若模型準確度只有 50%，效益如何？在什麼情況下建立模型的成本會超過其效益？
5. 增益排程依姿態調整增益，使 ω_n 與 ζ 維持恆定。請討論這個作法與第 17 章「用動力學模型線性化」的關係——兩者是互補還是替代？

章末摘要

- 開迴路不用回授，簡單但無法修正誤差；閉迴路以量測誤差修正，帶來抗干擾、降低模型依賴與改變系統動態三個價值。
- 比例控制等效於彈簧： $u = K_p e$ 使閉環成為簡諧振盪，永不衰減，且在持續干擾下留下穩

態誤差 $e_{ss} = d/K_p$ 。

- 微分控制等效於阻尼器；PD 控制的閉環方程 $m\ddot{e} + K_d \dot{e} + K_p e = 0$ 是標準的質量—阻尼—彈簧系統。
- 由增益求系統參數： $\omega_n = \sqrt{(K_p/m)}$ 、 $\zeta = K_d/(2\sqrt{(K_p m)})$ ；調 K_p 時須同步調 K_d 以維持阻尼比。
- 阻尼比決定響應類型： $\zeta < 1$ 欠阻尼（有超越）、 $\zeta = 1$ 臨界阻尼（不振盪的最快響應）、 $\zeta > 1$ 過阻尼；實務常用 $\zeta = 0.707$ （超越 4.3%）。
- 積分項消除穩態誤差，因其「記住」了抵消干擾所需的力；代價是降低穩定性、增加超越量、反應遲滯。
- P 看現在、I 看過去、D 看未來——三者互補但互相牽制。
- 積分飽和發生在控制量飽和時，防護方法有條件積分、限幅與反飽和回授；症狀是「異常之後更糟」。
- 設計流程：由超越量規格定 ζ 、由整定時間規格定 ω_n 、再計算 $K_p = m\omega_n^2$ 、 $K_d = 2\zeta\omega_n m$ 。
- 增益有三重上限：雜訊放大（限制 K_d ）、致動器飽和、以及最根本的結構共振（頻寬應低於共振頻率的 $1/3 \sim 1/5$ ）。
- 數位實作三議題：微分放大雜訊（需濾波或用速度量測）、積分近似、取樣延遲；取樣頻率應為閉環頻寬的 $10 \sim 20$ 倍。
- 微分應作用於輸出而非誤差，以避免目標突變時的 derivative kick。
- 純回授在軌跡追隨時必然有誤差 $e_{ss} \approx m \ddot{x}_d / K_p$ ，因為回授必須先有誤差才會出力。
- 前饋依模型預先計算所需的力： $u = u_{ff} + u_{fb}$ ，核心思想是「用模型處理已知的部分、用回授處理未知的部分」。
- 誤差的改善倍數約等於模型的準確度倍數——模型 80% 準確，誤差降為 $1/5$ 。
- PID 在機器人上的四個結構性限制：非線性、耦合、追隨誤差、增益上限；解方是引入動力學模型（第 17 章）。

觀念題與計算題

觀念題

1. 比較開迴路與閉迴路控制，並列舉回授的三個價值與三個代價。
2. 說明比例控制為何等效於彈簧，以及為何純 P 控制會產生等幅振盪。
3. 推導純 P 控制在持續干擾下的穩態誤差公式，並說明為何不能靠調高 K_p 消除。
4. 寫出 PD 控制的閉環方程，並說明它與機械系統的對應關係。
5. 寫出由增益求 ω_n 與 ζ 的公式，並說明調整 K_p 時 K_d 應如何配合。
6. 比較欠阻尼、臨界阻尼與過阻尼三種響應，並說明為何實務常用 $\zeta = 0.707$ 而非 $\zeta = 1$ 。
7. 說明積分項消除穩態誤差的機制，以及它帶來的三個代價。
8. 解釋積分飽和的成因與典型症狀，並列舉三種防護方法。
9. 說明由時域規格設計增益的三個步驟。
10. 列舉限制增益的三個因素，並說明何者最為根本。
11. 說明數位實作的三個議題，以及微分項雜訊的三種對策。
12. 為何微分應作用於輸出而非誤差？這避免了什麼問題？
13. 說明為何純回授在軌跡追隨時必然有誤差，並寫出其近似公式。
14. 說明前饋的原理與「用模型處理已知、用回授處理未知」的思想。
15. 列舉 PID 在機器人應用上的四個結構性限制。

計算題

1. 某關節等效慣量 $m = 1.2 \text{ kg}\cdot\text{m}^2$ ，設定 $K_p = 300 \text{ N}\cdot\text{m}/\text{rad}$ 、 $K_d = 36 \text{ N}\cdot\text{m}\cdot\text{s}/\text{rad}$ 。(a) 求 ω_n 與 ζ 。(b) 判斷響應類型並估算超越量。(c) 求整定時間。(d) 若要達到臨界阻尼， K_d 應改為多少？(e) 若改為 $\zeta = 0.707$ ， K_d 應為多少？
2. 某關節 $m = 0.6 \text{ kg}\cdot\text{m}^2$ 。規格要求超越量不超過 10%、整定時間不超過 0.25 秒。(a) 由超越量決定 ζ (提示： $M_p = 10\%$ 對應 $\zeta \approx 0.591$)。(b) 由整定時間決定 ω_n 。(c) 計算 K_p 與 K_d 。(d) 若該關節的結構共振頻率為 18 Hz，判斷此設計是否安全。(e) 若不安全，應如何修改規格？
3. 承上題設計的控制器。系統受到 $2.5 \text{ N}\cdot\text{m}$ 的持續干擾 (如重力)。(a) 求純 PD 的穩態誤差。(b) 若連桿長 0.4 m，求末端誤差。(c) 若加入積分項且要求誤差在 0.5 秒內消除，估算所需的 K_i (提示：可由 $K_i \approx \omega_n^2 m / T_i$ 估算， T_i 取 0.5 秒)。(d) 改用重力前饋補償，說明其相對於積分項的三個優點。

4. 某關節 $m = 0.9 \text{ kg}\cdot\text{m}^2$ 、 $K_p = 400$ 、 $K_d = 30$ ，追隨一段軌跡，其最大加速度為 8 rad/s^2 、最大速度為 3 rad/s 。(a) 求純 PD 在最大加速度處的追隨誤差。(b) 若質量估計為 0.75 (誤差 16.7%)，加入加速度前饋後的誤差為何？(c) 計算誤差改善的倍數。(d) 若連桿長 0.6 m ，比較兩者的末端誤差，並判斷是否滿足 $\pm 1 \text{ mm}$ 的精度要求。

動手做與專題延伸

本章實作：PID 控制器模組

延續前面章節的程式庫，加入控制模組。這將與第 13~15 章的動力學與軌跡模組整合成完整的模擬平台。

- **階段一**：實作完整的 PID 類別，含條件積分防飽和、微分濾波、對輸出微分三項防護。
- **階段二**：實作階躍響應分析函式，自動計算超越量、上升時間、整定時間與穩態誤差。
- **階段三**：整合第 13 章的動力學模擬，在平面 2R 手臂上測試獨立關節 PID，觀察耦合造成的誤差。
- **階段四**：加入第 15 章的軌跡生成，測試軌跡追隨，並比較有無加速度前饋的效果。
- **階段五**：實作增益排程——依姿態計算等效慣量 $M(q)$ 的對角元素，動態調整增益使 ω_n 與 ζ 維持恆定。比較固定增益與排程增益的性能差異。

專題延伸

- **參數空間地圖**：對一維點質量系統，在 (K_p, K_d) 平面上掃描數百組增益組合，計算每組的超越量與整定時間，繪成兩張熱力圖。在圖上標出 $\zeta = 0.707$ 與 $\zeta = 1$ 的等高線，以及「超越量 $< 5\%$ 且整定時間 $< 0.3 \text{ s}$ 」的可行區域。這張圖能讓抽象的設計公式變成直觀的地圖。
- **雜訊與增益的取舍實驗**：在模擬的感測器讀值上加入不同強度的高斯雜訊，對每種雜訊強度找出「控制量抖動不超過額定 10% 」所允許的最大 K_d 。繪出「雜訊強度 vs 最大可用 K_d 」曲線，量化 16.6.3 節的限制二。
- **進階：耦合效應的量化研究**。在平面 2R 手臂上，讓第一關節維持定點 (PID 控制)，第二關節執行不同速度的正弦運動。記錄第一關節的位置誤差隨第二關節速度的變化，繪出曲線。你應該會看到誤差隨速度平方成長——這正是第 13 章科氏力的直接證據，也是第 17 章計算力矩控制必要性的實驗依據。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 9 章：操作臂的線性控制，本章主要參考) 。
- Åström, K. J. and Murray, R. M., Feedback Systems: An Introduction for Scientists and Engineers, 2nd ed., Princeton University Press (回授控制的現代入門經典，線上免費) 。
- Åström, K. J. and Hägglund, T., PID Controllers: Theory, Design, and Tuning, 2nd ed., ISA (PID 的權威專著，含各類調參方法與防飽和技術) 。
- Ogata, K., Modern Control Engineering, 5th ed., Pearson (控制理論的標準教科書，時域規格與二階系統分析) 。
- Ziegler, J. G. and Nichols, N. B., "Optimum Settings for Automatic Controllers," ASME Transactions, 1942 (經驗調參法的原始論文) 。
- Spong, M. W., Hutchinson, S. and Vidyasagar, M., Robot Modeling and Control, 2nd ed., Wiley (第 6 章：獨立關節控制) 。
- Franklin, G. F., Powell, J. D. and Emami-Naeini, A., Feedback Control of Dynamic Systems, 8th ed., Pearson (含數位控制與離散化的完整處理) 。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer (第 8 章：運動控制，含前饋與增益排程) 。
- 各伺服驅動器廠商技術手冊：台達、三菱、安川等廠商的伺服調機指南，含實務的調參流程與共振抑制濾波器設定。
- MATLAB Control System Toolbox 與 Python 的 python-control 套件文件：控制系統設計與分析的實作工具。

第 17 章

操作臂非線性控制

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩張並列的追隨誤差曲線圖：上圖標註「固定增益 PID 」，誤差隨姿態改變而劇烈起伏，加上負載後整體抬升；下圖標註「計算力矩控制」，誤差維持在一條平坦的低線，換姿態與加負載後幾乎不變。右半為一位工程師用單手輕推六軸手臂末端、手臂順著推力平順移動的照片，旁註「零力控制： $\tau = G(q)$ 」。中央以方塊圖串起兩層架構——內層「 $M(q)u + Cq + G$ 」抵消非線性、外層「 $\ddot{q}_d + K\dot{e} + Kpe$ 」做線性控制。標題文字：讓末端表現得像一顆一維點質量。

叫獸開講 「機器手臂搬起重物後，原本調好的控制器為什麼失準？」

小好把第 13 章那台手臂又推了出來。她花了三天，用 Ziegler-Nichols 方法把六個關節的 PID 都調到最佳——在額定速度的三成下，追隨誤差只有 0.08 公釐，漂亮極了。

「現在，」叫獸從架上拿下一個五公斤的配重塊，裝到末端法蘭上，「重跑同一段軌跡。」

手臂啟動，第一個彎道就明顯下墜，示教器上的追隨誤差跳到 2.4 公釐。小好皺眉：「那我重調一次增益。」

「可以。」叫獸點頭，「但我還有第二個問題。」他把配重塊拆掉，改把手臂的作業區從機器人的正前方移到側前方四十五度。同樣的軌跡、同樣的增益、沒有負載——誤差還是變成了 1.6 公釐。

小好愣住了：「連姿態換了都不行？」

「因為你調的那組增益，只對『某一個特定姿態、某一種特定負載』是最佳的。」叫獸在白板上寫下第 13 章的方程式： $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$ 。「你看這個 M ——它是 q 的函數。手臂伸直時和折疊時，同一個關節的等效慣量可能差三倍。你的 PID 增益是固定的，但系統本身在變。」

「這就像，」他想了想，「你在學開車，教練幫你調好了方向盤的靈敏度。但這台車有時候是空車、有時候載滿貨、有時候在柏油路、有時候在冰上——同一組設定不可能都好用。」

小好問：「那怎麼辦？每個姿態各調一組增益？」

「那叫增益排程，確實有人這樣做，但六個關節乘上無限多種姿態，你調不完。」叫獸搖頭，「更好的辦法是——既然我們已經有動力學模型了，為什麼不直接用它？」

他在白板上畫了一個方塊圖：「讓控制器先用逆動力學算出『這個姿態、這個加速度，理論上需要多少力矩』，把它直接送出去。這叫前饋。剩下的模型誤差，才交給 PID 修正。」

「這樣做有一個很美的結果。」叫獸在方塊圖旁邊寫下一行字：讓末端表現得像一顆一維點質量。「第 16 章我們分析的那個最簡單的模型 $m\ddot{x} = f$ ，其實可以被『製造』出來——只要控制器把非線性的部分全部抵消掉，剩下的系統在數學上就真的變成了一顆點質量。而點質量的控制，我們上一章已經完全解決了。」

本章為什麼重要

第 16 章的 PID 建立在一個假設上：系統是線性、非時變的。但第 13 章告訴我們，機器人完全不是這樣——慣性矩陣隨姿態變化、科氏力與速度平方成正比、重力隨姿態改變、關節之間互相耦合。PID 之所以還能用，是因為它把這些通通當成「干擾」來被動修正；而干擾越大（高速、重載），修正就越跟不上。

本章的核心思想是**主動出擊**：既然我們已經在第 13、14 章建立了動力學模型，就應該用它預先算出所需的力矩，而不是等誤差出現才補救。這個轉變帶來三個層次的成果：**計算力矩控制**把非線性系統精確線性化、**操作空間控制**讓我們直接在任務空間設計行為、而**重力補償**則是其中最簡單卻最實用的一項——許多協作機器人的「手動導引」功能就只靠它。

本章也是全書「模型」與「控制」交會的地方。第 9~12 章的運動學、第 13~14 章的動力學，都將在這裡被真正地使用起來。

學習目標

- 能說明機器人系統的四個非線性來源，並解釋固定增益 PID 為何不足。
- 能推導計算力矩控制律，並說明其如何達成回授線性化。
- 能說明計算力矩控制的兩個層次：內層線性化與外層線性控制。
- 能實作重力補償，並說明它在手動導引與零力示教中的角色。
- 能說明逆動力學控制與計算力矩控制的關係。
- 能說明模型誤差對回授線性化的影響，並估算殘餘誤差。
- 能比較關節空間控制與操作空間控制的適用場合。
- 能推導操作空間的等效慣量 Λ 並說明其物理意義。
- 能說明滑模控制、自適應控制與魯棒控制的基本思想與適用時機。
- 能說明模型預測控制的基本架構與其相對於 PID 的優勢。
- 能以 Python 實作計算力矩控制並與 PID 比較追隨性能。

先備知識

第 13 章的動力學標準形式與各項的物理意義、第 14 章的 Newton-Euler 遞迴（實作前饋需

要它的計算效率)、第 16 章的 PD / PID 控制與二階系統分析、第 11 章的雅可比矩陣(操作空間控制的核心工具)、第 15 章的軌跡生成(提供 $\mathbf{q}_d$ 、 $\dot{\mathbf{q}}_d$ 、 $\ddot{\mathbf{q}}_d$)。

本章是全書控制部分的關鍵轉折。建議讀法：務必先徹底理解 17.2 節的計算力矩控制——它是後續所有進階控制的基礎。若 17.7 節的操作空間控制推導感到吃力，可先跳過細節、掌握結論，之後回頭補。

核心名詞中英對照

表 17-1 本章核心名詞

中文	英文	說明
計算力矩控制	Computed Torque Control, CTC	以逆動力學抵消非線性的控制法
回授線性化	Feedback Linearization	以非線性回授使閉環系統成為線性
逆動力學控制	Inverse Dynamics Control	計算力矩控制的別稱
前饋	Feedforward	依模型預先計算的控制輸入
重力補償	Gravity Compensation	抵消重力所需的關節力矩
零力控制	Zero-force Control	使機器人可被輕易手動推動的模式
手動導引	Hand Guiding / Lead-through	以手直接帶動機器人示教的方式
操作空間	Operational Space	以末端位姿描述的控制空間
操作空間慣量	Operational Space Inertia, Λ	末端在任務空間中感受到的等效慣量
動態解耦	Dynamic Decoupling	使各方向的運動互不影響
滑模控制	Sliding Mode Control	以切換律強制系統沿滑動面運動
顫振	Chattering	滑模控制中高頻切換造成的振動
自適應控制	Adaptive Control	線上估測未知參數的控制法
魯棒控制	Robust Control	對模型不確定性維持性能的控制法
模型預測控制	Model Predictive Control, MPC	以滾動時域最佳化求解控制輸入
滾動時域	Receding Horizon	每步重新求解未來一段時間的最佳解

17.1 非線性從何而來

17.1.1 四個非線性來源

回顧第 13 章的動力學方程，可以清楚辨識出四個非線性來源：

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) + F(q) = \tau$$

機器人動力學標準形式。四項各自貢獻不同性質的非線性。

來源一：慣性矩陣隨姿態變化。 $M(q)$ 是 q 的函數。以第 13 章的平面 2R 為例， $M_{11} = \alpha + 2\beta\cos\theta_2$ ——當 θ_2 由 0° 變到 180° ， M_{11} 由 0.78 變到 0.30，相差 2.6 倍。這代表同樣的關節加速度，在不同姿態下需要差 2.6 倍的力矩。

來源二：關節之間的耦合。 M 的非對角元素不為零，一個關節的加速度會對其他關節產生力矩（第 13 章例題 13-3 量化過：第一關節明明沒動卻要出 1.2 N·m）。對固定增益的獨立關節 PID 而言，這是無法預測的干擾。

來源三：速度相關項與速度平方成正比。 $C(q,\dot{q})\dot{q}$ 中的科氏力與離心力隨速度平方增長。速度加倍，這些力變成四倍——這正是低速調校完美、高速失準的根源。

來源四：重力隨姿態變化。 $G(q)$ 在水平伸展時最大、垂直時為零。PID 必須靠積分項慢慢累積來對抗它，導致響應遲鈍。

17.1.2 為何固定增益 PID 不足

第 16 章的分析建立在 $m\ddot{x} = f$ 這個線性模型上，由此推出的自然頻率 $\omega_n = \sqrt{(K_p/m)}$ 與阻尼比 $\zeta = K_d/(2\sqrt{(K_p \cdot m)})$ ，都假設 m 是常數。

但在機器人上，「 m 」實際上是 $M(q)$ ——它會變。這意味著：

$$\omega_n(q) = \sqrt{(K_p/M(q))}, \quad \zeta(q) = K_d/(2\sqrt{(K_p \cdot M(q))})$$

有效的自然頻率與阻尼比隨姿態變化。同一組增益，在不同姿態下產生完全不同的動態行為。

具體而言：在**慣量小的姿態**（手臂折疊），有效 ω_n 大、 ζ 小——系統反應快但容易振盪；在**慣量大的姿態**（手臂伸展），有效 ω_n 小、 ζ 大——系統反應遲鈍。

這正是開頭故事的完整解釋。小好正在前方調好的增益，換到側前方四十五度時， $M(q)$ 改變了，於是阻尼比也變了——原本的臨界阻尼可能變成欠阻尼（振盪）或過阻尼（遲鈍）。加上五公斤負載後， M 更是整體放大，情況更嚴重。

◎ 增益排程：一個治標的解法

面對 $M(q)$ 的變化，一個直觀的作法是**增益排程**（gain scheduling）：預先為不同的姿態區域各調一組增益，執行時依當前姿態切換。這在單一自由度或姿態變化範圍小的系統上確實可行（如汽車

的變增益轉向助力)。但對六軸手臂而言，姿態空間是六維的，要涵蓋所有組合需要天文數字的參數，且切換時可能產生不連續。因此工業機器人幾乎都不採此路，而是直接使用本章的模型式方法。

17.2 計算力矩控制

17.2.1 核心理念：先抵消，再控制

計算力矩控制 (Computed Torque Control, CTC) 的想法極為優雅：既然我們知道非線性項是什麼，就用控制輸入把它們精確抵消掉。

具體作法是把控制力矩設計成兩部分：

$$\tau = M(q) \cdot u + C(q, \dot{q}) \dot{q} + G(q)$$

計算力矩控制律。後兩項抵消非線性， u 是待設計的「虛擬控制輸入」。

把它代入系統方程 $M \ddot{q} + C \dot{q} + G = \tau$ ：

$$M \ddot{q} + C \dot{q} + G = M u + C \dot{q} + G \quad \Rightarrow \quad M \ddot{q} = M u \quad \Rightarrow \quad \ddot{q} = u$$

驚人的結果：閉環系統變成了 $\ddot{q} = u$ 。

這個結果的意義極為重大。原本複雜的非線性耦合系統，經過這個控制律之後，變成了 n 個獨立的雙積分器——每個關節都是一個「加速度就是輸入」的簡單系統，彼此完全不耦合，且沒有任何非線性。

$$\ddot{q}_i = u_i \quad (i = 1, 2, \dots, n)$$

n 個獨立的雙積分器。這正是第 16 章的一維點質量模型 ($m = 1$) ！

17.2.2 外層：線性控制器

既然系統已經變成雙積分器，第 16 章的所有分析就完全適用。設計 u 為 PD 形式：

$$u = \ddot{q}_d + K_d(\dot{q}_d - \dot{q}) + K_p(q_d - q)$$

外層控制律。第一項是前饋加速度，後兩項是 PD 回授。

代入 $\ddot{q} = u$ 並整理，令誤差 $e = q_d - q$ ：

$$\ddot{e} + K_d \dot{e} + K_p e = 0$$

誤差動態方程。這是一個線性、非時變、且各關節完全解耦的二階系統。

這條式子的美妙之處在於：它不含 M 、 C 、 G 任何一項。無論機器人處於什麼姿態、以多快的速度運動、帶多重的負載，誤差的收斂行為都完全相同。這徹底解決了開頭故事的問題。

增益的選擇直接套用第 16 章的公式：

$$K_p = \omega_n^2, \quad K_d = 2\zeta\omega_n$$

由期望的自然頻率與阻尼比反推增益。通常取 $\zeta = 1$ (臨界阻尼) 以避免超越。

圖 17-1 計算力矩控制的兩層架構

建議配圖：方塊圖。左側輸入為軌跡規劃器輸出的 q_d 、 $\dot{q}_d$ 、 $\ddot{q}_d$ 。外層 (藍色區塊) 為線性控制器： $\ddot{q}_d$ 直接前饋，加上 K_p 、 K_d 對誤差的回授，輸出虛擬控制 u 。內層 (綠色區塊) 為線性化補償：以 u 乘上 $M(q)$ ，再加上 $C(q, \dot{q})\dot{q}$ 與 $G(q)$ ，輸出實際力矩 τ 送給機器人。機器人的 q 、 $\dot{q}$ 回授至兩層。圖下方標註：內層把非線性抵消成 $\ddot{q} = u$ ，外層再用第 16 章的 PD 設計。

17.2.3 為何叫「計算力矩」

這個名稱來自它的實作方式：控制器在每一個週期都要計算出「維持這個運動所需的力矩」——也就是第 14 章的逆動力學。這正是為何第 14 章的 $O(n)$ 遞迴演算法如此重要：CTC 需要在每一毫秒完成一次完整的逆動力學計算。

值得注意的是，實作上通常不會分別計算 M 、 C 、 G 再組合，而是直接呼叫一次 Newton-Euler 遞迴，傳入 $(q, \dot{q}, u)$ 作為輸入：

$$\tau = \text{InverseDynamics}(q, \dot{q}, u) \quad \text{其中 } u = \ddot{q}_d + K_d \dot{e} + K_p e$$

實作技巧。把 u 當作「期望加速度」傳入逆動力學，一次遞迴即可得到完整的 τ ，複雜度 $O(n)$ 。

17.3 回授線性化的觀點

17.3.1 更一般的框架

計算力矩控制是一個更廣泛的概念——回授線性化 (feedback linearization) ——在機器人上的特例。

回授線性化的一般想法是：對於一個非線性系統，若能找到一個非線性的回授控制律與座標變換，使閉環系統在新座標下成為線性系統，就能套用所有線性控制理論的工具。

機器人動力學特別適合這個方法，原因有二：一、 $M(q)$ 恆為正定因此必可逆 (第 13.6.1 節的性質二)，這保證了 $\tau = M \cdot u + \dots$ 這個變換永遠可行；二、控制輸入的維度等於自由度 (每個關節都有一個馬達)，這使系統是「完全驅動」的 (fully actuated)。

17.3.2 完全驅動與驅動不足

這裡有一個重要的分野。**完全驅動系統**（如固定基座的機械手臂）可以精確回授線性化；**驅動不足系統**（underactuated，如第 5.5 節提到的四足機器人騰空階段、兩輪自平衡車、四旋翼無人機）則不行——因為某些自由度沒有直接的控制輸入。

這解釋了為何本章的方法主要用於機械手臂，而人形機器人與無人機需要更複雜的控制策略（第 27、28 章）。對它們而言，「浮動基座」的六個自由度沒有馬達直接驅動，只能透過與地面或空氣的接觸間接控制。

17.3.3 與線性化的差異

必須釐清一個容易混淆的概念：回授線性化**不是**在某個工作點附近做泰勒展開的「近似線性化」。

近似線性化（第 19 章會用到）是在平衡點附近取一階近似，只在該點附近有效，離開就失準。**回授線性化**則是精確的：只要模型正確，在**整個狀態空間**中閉環系統都嚴格等於 $\ddot{q} = u$ ，沒有任何近似。

代價是：回授線性化需要**完整且準確的模型**。近似線性化只需要在一個點上的資訊。這個取捨，正是下一節要討論的。

17.4 模型誤差的影響

17.4.1 現實中的不完美

計算力矩控制的推導假設模型完全準確。但第 13.10.3 節已指出，實際的動力學模型有 10~20% 的誤差，主要來自慣性參數不準、摩擦難以建模、關節彈性與未建模的線材拉力。

設控制器使用的是估計模型 $\hat{M}$ 、 $\hat{C}$ 、 $\hat{G}$ ，實際系統為 M 、 C 、 G 。控制律為：

$$\tau = \hat{M}u + \hat{C}\dot{q} + \hat{G}$$

使用估計模型的控制律。

代入真實系統 $M\ddot{q} + C\dot{q} + G = \tau$ 並整理：

$$\ddot{q} = M^{-1}\hat{M}u + M^{-1}[(\hat{C} - C)\dot{q} + (\hat{G} - G)]$$

閉環方程。若模型完美（ $\hat{M} = M$ 等），退化為 $\ddot{q} = u$ ；否則會有兩個偏差。

偏差一： $M^{-1}M \neq I$ 。這使有效增益偏離設計值。若 M 高估了 20%，有效的 K_p 與 K_d 都會被放大 1.2 倍——阻尼比不變（因為兩者同比例變化），但自然頻率上升 10%。這通常影響不大。

偏差二：括號中的殘差項。這是一個「等效干擾」，會造成穩態誤差與追隨誤差。它的大小正比於模型誤差與運動的劇烈程度。

17.4.2 為何仍然有效

關鍵的觀察是：即使模型有 20% 的誤差，CTC 仍然消除了 80% 的非線性。留給 PD 回授處理的殘差，只有原本的五分之一。

這帶來兩個好處。第一，**追隨誤差大幅降低**——干擾小了五倍，同樣的增益下誤差也小約五倍。第二，**增益可以調得更低**——因為不需要靠高增益去對抗大干擾，系統可以更「軟」，這對安全與抗雜訊都有利。

◎ 模型式控制的正確心態

初學者常有一個誤解：認為模型式控制需要「精確」的模型才能用，因此在參數不確定時乾脆放棄。這是錯的。正確的心態是：**模型處理已知的部分，回授處理未知的部分**。一個 80% 準確的模型，就能讓回授的負擔降低五倍——這已經是巨大的改善。追求 99% 的模型精度往往不划算，因為剩下的 1% 由回授處理即可。第 13 章的結論在此再次適用：模型的價值不在完美，而在把大部分的需求「預測」出來。

17.4.3 實務上的折衷版本

完整的 CTC 需要即時計算 M 、 C 、 G 三項。在計算資源有限或模型不完整時，有兩個常用的簡化版本。

版本一：只做重力補償（17.5 節）。只計算 $G(q)$ ，忽略 M 與 C 。這適用於中低速運動，因為此時重力往往佔總力矩的 70~90%（第 13 章例題 13-2 中佔 88.7%）。計算量極小，效果顯著。

版本二：固定慣量近似。用一個常數矩陣 M （例如取工作區域中央姿態的 M 值）取代 $M(q)$ ，並忽略 C 。這保留了「解耦」的好處，但省去了每週期重算 M 的成本。

這兩個版本在實務上極為常見——許多工業機器人控制器的預設模式，其實就是重力補償加上獨立關節 PID。完整的 CTC 通常只在高速或高精度需求時才啟用。

17.5 重力補償

17.5.1 最簡單也最實用的模型式控制

重力補償是計算力矩控制的極簡版本：只補償 $G(q)$ ，其餘交給 PD 回授。

$$\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) + G(q)$$

PD 加重力補償。這是機器人控制中投資報酬率最高的一項改良。

它的價值在於三件事。**第一，消除穩態誤差**。純 PD 控制在有重力時必定有穩態誤差——因為必須維持一個非零的誤差才能產生足以平衡重力的力矩：

$$\text{純 PD 的穩態誤差：} e_{ss} = G(q)/K_p$$

誤差與重力成正比、與增益成反比。補償重力後， $e_{ss} = 0$ 。

第二，避免依賴積分項。第 16 章提到積分項雖能消除穩態誤差，但會降低相位裕度、拖慢響應、並帶來積分飽和的風險。重力補償直接從源頭消除誤差，不需積分項。

第三，計算量極小。 $G(q)$ 只需一次正向運動學加上簡單的加總（第 14.4.3 節的技巧：設 $\ddot{q} = 0$ 執行一次遞迴即可），遠比完整的 CTC 便宜。

17.5.2 理論保證

第 16 章的知識框提過 Takegaki-Arimoto 定理，現在可以完整陳述：

◎ PD 加重力補償的穩定性定理

對於任何機械手臂，控制律 $\tau = K_p(q_d - q) - K_d \dot{q} + G(q)$ （其中 K_p 、 K_d 為任意正定矩陣）能保證定點調節的**全域漸近穩定**——無論初始位置在哪裡，機器人必定收斂到目標且誤差趨於零。這個結論不需要知道 $M(q)$ 與 $C(q, \dot{q})$ 的任何資訊，只需要 $G(q)$ 。這正是重力補償如此重要的理論依據，第 19 章會用 Lyapunov 方法給出完整證明。

必須注意這個定理的適用範圍：它保證的是**定點調節**（ q_d 為常數），不保證時變軌跡的追隨精度。要追隨快速的軌跡，仍然需要完整的 CTC 或至少加上加速度前饋。

17.5.3 零力控制與手動導引

重力補償有一個極為重要的應用：**零力控制**（zero-force control）。若把控制律中的 PD 項全部去掉，只保留重力補償：

$$\tau = G(q)$$

純重力補償。機器人恰好抵消自身重量，但不主動維持位置。

此時機器人會**浮在空中**——它不會因重力下墜，但也不抗拒外力。用手輕輕一推，它就順著推動的方向移動；放手後停在原處。這正是協作機器人「**手動導引示教**」(hand guiding / lead-through teaching) 的原理：操作員直接抓著手臂把它帶到想要的位置，按下記錄鍵存下該點。

這個功能徹底改變了機器人的示教方式。傳統的示教器需要操作員理解座標系與運動方向，用按鈕一軸一軸地移動；手動導引則任何人都能上手——這是協作機器人易用性的關鍵，也是它們能進入中小企業的重要原因。

實作上有三個要點。**要點一：重力模型必須準確**。若補償不足，手臂會緩慢下墜；若過度補償，手臂會自己往上飄。兩者都會讓操作員感到不安。**要點二：必須補償摩擦**。減速機的摩擦會讓手臂「推不動」或「推了就停不下來」，因此高階系統會額外加入摩擦補償。**要點三：安全限制**。零力模式下機器人幾乎不受控，必須限制速度並設定虛擬牆（軟體邊界）避免碰撞。

17.5.4 重力補償的機構替代方案

值得一提的是，重力也可以用**機構**而非控制來補償。第 5 章的專題延伸提過這個作法：在關節上加裝彈簧或配重，使其產生的力矩恰好抵消重力力矩。

這種被動重力補償的優點是：不消耗能量、斷電時手臂也不會掉落、且能大幅降低馬達的持續扭力需求（第 5 章例題 5-2 的改善方案二）。缺點是增加機構重量與慣量、只對特定的負載範圍有效、且設計複雜。

實務上兩者常並用：大型手臂用彈簧平衡器承擔大部分的重量，控制器再用軟體補償剩餘的部分。這是「機構與控制協同設計」的典型範例。

17.6 關節空間的軌跡追隨

17.6.1 完整的控制架構

把第 15 章的軌跡生成與本章的 CTC 結合，就得到完整的關節空間軌跡追隨架構：

1. **軌跡生成器**（第 15 章）：依任務需求產生 $q_d(t)$ 、 $\dot{q}_d(t)$ 、 $\ddot{q}_d(t)$ 三者。注意必須提供加

速度——這是 CTC 前饋所必需的。

2. 誤差計算： $e = q_d - q$ ， $\dot{e} = \dot{q}_d - \dot{q}$ 。
3. 外層控制律： $u = \ddot{q}_d + K_d \dot{e} + K_p e$ 。
4. 內層線性化： $\tau = \text{InverseDynamics}(q, \dot{q}, u)$ ，一次 Newton-Euler 遞迴即可（第 14 章）。
5. 力矩限制：檢查並限制 τ 不超過馬達能力，若觸及飽和則須有對應處理（見 17.6.3 節）。
6. 送出指令：把 τ 轉換為電流指令送給驅動器（第 5 章的 $\tau = K_t I$ ）。

整個迴路通常以 1 kHz 執行。以第 14 章例題 14-2 的估算，六軸手臂的一次逆動力學約需 9 μs ，佔週期的不到 1%——完全可行。

17.6.2 三種控制律的比較

為了看清楚各種方法的差異，把三種控制律並列：

表 17-2 三種關節空間控制律的比較

控制律	公式	所需模型	計算量	追隨性能	適用場合
獨立關節 PID	$\tau = K_p e + K_d \dot{e} + K_i \int e$	無	極低	低速尚可、高速差	教學、低速、簡單系統
PD + 重力補償	$\tau = K_p e + K_d \dot{e} + G(q)$	只需 $G(q)$	低	定點調節優異、追隨中等	定位、手動導引、多數工業應用
計算力矩控制	$\tau = M(q)u + C \dot{q} + G$	完整動力學	中 ($O(n)$)	優異	高速、高精度、重載變化

實務上的選擇原則：若運動緩慢（額定速度 30% 以下）且負載固定，PD 加重力補償通常就足夠；若需要高速、或負載會大幅變化、或要求高追隨精度，則採用完整 CTC。

17.6.3 力矩飽和的處理

一個實務上必須處理的問題是**力矩飽和**：控制器算出的 τ 可能超過馬達能力。此時若直接截斷，會產生兩個問題。

問題一：線性化被破壞。CTC 的推導假設 τ 能被精確執行；一旦飽和， $\ddot{q} \neq u$ ，閉環不再是雙積分器，性能保證失效。

問題二：積分飽和（若有積分項）。誤差持續存在但控制器已無力修正，積分項會不斷累積（第 16 章的 windup 問題）。

因應之道有三：一、**事前預防**——在第 15 章的軌跡規劃階段就做動力學可行性檢驗（15.8 節），確保規劃的軌跡不會導致飽和。這是最好的方法。二、**動態降速**——偵測到接近飽和時，即時放慢軌跡的執行速度（時間縮放）。三、**抗飽和機制**——採用第 16 章的反算法或條件積分，避免積分項失控。

17.7 操作空間控制

17.7.1 為何要在任務空間控制

到目前為止，控制都在關節空間進行：規劃 $q_d(t)$ 、控制關節誤差。但許多任務的需求其實是在**任務空間**中描述的：「末端沿這條直線移動」、「末端對工件施加 10 N 的力」、「末端保持在這個位置不動，但可以被推」。

關節空間控制要達成這些需求，必須先把任務轉成關節軌跡（逐點逆運動學），再控制關節。這有三個缺點：一、**逆運動學的計算負擔**；二、**關節誤差與末端誤差的關係隨姿態變化**（同樣的關節誤差，在不同姿態下造成的末端偏差不同）；三、**難以直接指定末端的動態行為**（如剛度、阻尼）。

操作空間控制（operational space control，由 Khatib 於 1987 年提出）直接在任務空間中設計控制律，繞過這些問題。

17.7.2 推導：把動力學投影到任務空間

推導的起點是第 11 章的雅可比關係。末端速度與加速度為：

$$\dot{x} = J \dot{q} \quad \ddot{x} = J \ddot{q} + \dot{J} \dot{q}$$

任務空間與關節空間的速度、加速度關係。第二項來自雅可比隨姿態變化（第 11.9.3 節）。

我們希望設計控制律，使末端的行為像一顆在任務空間中的點質量：

$$\ddot{x} = u_x \quad (u_x \text{ 為任務空間的虛擬控制輸入})$$

目標：讓末端表現得像一顆一維點質量——正是開頭故事中叫獸說的那句話。

由 $\ddot{x} = J \ddot{q} + \dot{J} \dot{q} = u_x$ 反解關節加速度：

$$\ddot{q} = J^{-1}(u_x - \dot{J} \dot{q}) \quad (\text{或用偽逆 } J^+ \text{，若冗餘})$$

所需的關節加速度。

代入 CTC 的控制律 $\tau = M \ddot{q} + C \dot{q} + G$:

$$\tau = M J^{-1}(\ddot{u}_x - \dot{J} \dot{q}) + C \dot{q} + G$$

操作空間控制律的初步形式。

Khatib 進一步證明，這可以改寫成一個更優雅、且直接以「末端力」為輸入的形式：

$$\tau = J^T [\Lambda(q) \ddot{u}_x + \mu(q, \dot{q}) + p(q)]$$

操作空間控制律。 J^T 把任務空間的力映射為關節力矩（第 11.6 節的對偶性）。

其中三項的意義分別為：

$$\Lambda(q) = (J M^{-1} J^T)^{-1} \quad (\text{操作空間慣量矩陣})$$

$$\mu(q, \dot{q}) = \Lambda J M^{-1} C \dot{q} - \Lambda \dot{J} \dot{q} \quad (\text{任務空間的科氏離心項})$$

$$p(q) = \Lambda J M^{-1} G \quad (\text{任務空間的重力項})$$

三個投影後的量。它們是關節空間的 M 、 C 、 G 在任務空間中的等效物。

17.7.3 操作空間慣量 Λ 的物理意義

$\Lambda = (J M^{-1} J^T)^{-1}$ 這個矩陣值得特別說明，它有一個非常直觀的物理意義：末端在各方向上感受到的等效慣量。

想像你用手推機器人的末端。往某些方向推很輕鬆（等效慣量小），往另一些方向推很沉重（等效慣量大）。 Λ 就精確描述了這個「各方向推起來多重」的分布——它是第 11.8 節速度橢球與力橢球概念在動力學層面的延伸。

Λ 的三個重要性質：一、對稱正定（繼承自 M 的正定性）；二、隨姿態變化；三、在奇異點處退化——當 J 失去滿秩時， $J M^{-1} J^T$ 變成奇異， Λ 的某些元素趨於無窮大。這在物理上完全合理：在奇異點的失去方向上，末端「推不動」，等效慣量無限大。

17.7.4 動態解耦與冗餘

操作空間控制的一個關鍵成果是**動態解耦**：經過上述控制律，末端在任務空間的六個自由度變成互相獨立的雙積分器。這意味著可以為每個方向獨立設計控制器——例如在 Z 方向用高剛度（精確定位）、在 X - Y 方向用低剛度（順應性），這正是第 20 章力控制與阻抗控制的基礎。

對冗餘機器人，操作空間控制還能自然地結合第 12 章的零空間投影：

$$\tau = J^T [\Lambda \ddot{u}_x + \mu + p] + (I - J^T J) \tau_{\text{null}}$$

第二項是動態一致的零空間投影，其中 $J = M^{-1}J$ Λ 為動態一致的廣義逆。 τ_{null} 可用於次要目標（避關節極限、維持姿勢等）。

注意這裡用的是「動態一致」的零空間投影，而非第 12 章的運動學版本。兩者的差別在於：運動學版本保證零空間運動不影響末端位置，動態版本則保證不影響末端受力——後者在力控制中更為重要。

圖 17-2 關節空間控制與操作空間控制的架構對比

建議配圖：上下兩列方塊圖。上列（關節空間控制）：任務目標 $x_d \rightarrow$ 逆運動學 $\rightarrow q_d \rightarrow$ 關節 PD/CTC $\rightarrow \tau \rightarrow$ 機器人；標註「需逐點逆解、誤差關係隨姿態變化」。下列（操作空間控制）：任務目標 $x_d \rightarrow$ 直接在任務空間計算誤差與 $u_x \rightarrow$ 乘上 Λ 得任務空間力 $\rightarrow$ 經 J^T 轉為關節力矩 $\tau \rightarrow$ 機器人；末端位姿 x 由正向運動學回授。標註「不需逆運動學、末端動態解耦、可直接指定剛度」。右側另附冗餘機器人的零空間分支。

17.8 進階非線性控制概觀

17.8.1 滑模控制

計算力矩控制依賴準確的模型。當模型誤差較大時，一個強健的替代方案是滑模控制（sliding mode control）。

它的核心想法是：先定義一個「滑動面」——一個誤差空間中的超平面，只要系統狀態落在這個面上，誤差就會自動收斂：

$$s = \dot{e} + \lambda e = 0 \quad (\text{滑動面})$$

當 $s = 0$ 時， $\dot{e} = -\lambda e$ ，誤差以指數衰減至零。 λ 決定收斂速度。

控制律的設計則是：無論系統在滑動面的哪一側，都用足夠強的力把它推回面上：

$$\tau = \tau_{eq} + K \cdot \text{sign}(s)$$

τ_{eq} 為等效控制（依名目模型計算）， $K \cdot \text{sign}(s)$ 為切換項。只要 K 大於模型誤差的上界，就能保證收斂。

優點：對模型不確定性與外部干擾極為強健，且不需要精確的參數，只需知道誤差的上界。

缺點： sign 函數造成控制輸入高頻切換，產生顫振（chattering）——這會激發機構振動、加速磨損、並產生噪音。

實務上的緩解方法是用飽和函數（saturation）取代 sign ，在滑動面附近設一個「邊界層」使控制連續。代價是犧牲了理論上的有限時間收斂，改為收斂到一個小鄰域內。

17.8.2 自適應控制

自適應控制 (adaptive control) 採取另一條路線：既然參數不準，就線上估測它們。

它利用第 13.10.2 節的關鍵性質——動力學方程對慣性參數是線性的：

$$\tau = Y(q, \dot{q}, \ddot{q}) \cdot \pi$$

Y 為迴歸矩陣 (只與運動有關)、 π 為慣性參數向量。這個線性結構是自適應控制可行的數學基礎。

控制器同時做兩件事：用當前的參數估計值 $\hat{\pi}$ 計算控制力矩、並依追隨誤差調整 $\hat{\pi}$ 。經典的 Slotine-Li 自適應控制器能證明：追隨誤差保證收斂到零，即使初始的參數估計完全錯誤。

優點：能自動適應未知或變化的負載——例如機器人抓起一個未知重量的物件後，控制器會自動學會補償它。**缺點：**參數收斂需要「持續激勵」(軌跡必須夠豐富才能辨識出所有參數)；且收斂過程中性能較差；實作也較複雜。

17.8.3 魯棒控制

魯棒控制 (robust control) 的哲學介於兩者之間：不追求估測真實參數，也不用高頻切換，而是設計一個對「所有可能的參數組合」都能維持性能的控制器。

典型作法是在 CTC 的基礎上加入一個補償項，其大小依模型誤差的上界設計。它比滑模控制平滑 (無顫振)、比自適應控制簡單 (無需線上估測)，但性能通常較保守——因為必須為最壞情況做準備。

表 17-3 四種非線性控制方法的比較

方法	核心思想	模型需求	對不確定性	主要缺點	適用場合
計算力矩控制	用模型精確抵消非線性	完整且準確	敏感	依賴模型精度	模型可靠、高速高精度
滑模控制	強力推回滑動面	只需誤差上界	極強健	顫振	模型很不確定、干擾大
自適應控制	線上估測未知參數	需知結構 (Y 矩陣)	強 (會收斂)	需持續激勵、收斂慢	負載未知或會變化
魯棒控制	為最壞情況設計	需知誤差上界	強健	性能保守	不確定性有界且已知

17.8.4 模型預測控制

模型預測控制 (Model Predictive Control, MPC) 代表另一個層次的思路。前述方法都是

「當下反應」——依當前誤差計算控制輸入。MPC 則是「向前看」：

1. 在每個控制週期，用模型預測未來一段時間（預測時域）內系統的行為。
2. 求解一個最佳化問題：找出未來的控制序列，使某個代價函數最小，同時滿足所有約束（力矩極限、關節極限、避障）。
3. 只執行這個序列的**第一步**。
4. 下一個週期，以新的量測值重新求解——這稱為滾動時域（receding horizon）。

MPC 最大的優勢是能明確處理約束。前述所有方法都是「先算出控制輸入，再截斷到極限」；MPC 則在求解時就把極限納入考慮，因此不會發生飽和造成的性能崩壞。此外它天然支援多目標權衡與未來資訊（如已知的軌跡）。

代價是計算量。每個週期都要解一個最佳化問題，這在過去只能用於慢速系統（化工製程的控制週期以分鐘計）。但隨著求解器與硬體的進步，近年 MPC 已能在毫秒級執行——四足機器人的即時步態控制（第 28 章）就大量使用 MPC，這是近十年腿式機器人能力躍進的關鍵技術之一。

MPC 與第 18 章的最佳控制關係密切：它本質上是「有限時域的最佳控制問題，反覆求解」。第 18 章將建立最佳控制的理論基礎，屆時可以更完整地理解 MPC 的數學結構。

17.9 多任務的動態版本

17.9.1 由運動學到動力學的優先權

第 12.8 節介紹了分層逆運動學：以嚴格的優先權處理多個任務，低優先權的任務在高優先權任務的零空間中執行。那是**運動學**層次的處理——只考慮位置與速度。

操作空間控制讓我們能把同樣的想法推廣到**動力學**層次。差別在於：運動學版本保證「低優先權任務不影響高優先權任務的運動」；動力學版本則保證「不影響高優先權任務所受的力」。

以兩層任務為例（主任務雅可比 J_1 、次任務 J_2 ）：

$$\tau = J_1^T F_1 + N_1^T J_2^T F_2, \quad \text{其中 } N_1 = I - J_1^T J_1^T$$

動態一致的分層控制。 N_1 是動態一致的零空間投影算子， $J_1 = M^{-1}J_1^T \Lambda_1$ 為動態一致廣義逆。

這個架構是人形機器人**全身控制**（whole-body control）的核心。一台人形機器人可能同

時需要：維持平衡（最高優先權）、右手跟隨軌跡、左手保持不動、注視目標、以及所有關節遠離極限——分層架構讓這些任務能有序地共存。第 28 章會回到這個主題。

17.9.2 為何動態一致性重要

使用運動學版本的零空間投影，在動力學層次上會產生一個微妙的問題：次任務的力矩雖然不會改變主任務的**加速度**，卻可能改變主任務所需的**力**。

在純運動控制中這無妨；但在**力控制**（第 20 章）中就是嚴重問題——例如機器人一手抵著工件維持 10 N 的接觸力，另一手在做次要動作，若次要動作干擾了接觸力，工件可能被推移或損壞。動態一致的投影正是為了避免這種干擾。

17.10 實務上的控制架構

17.10.1 工業機器人的典型架構

真實的工業機器人控制器並非只用單一種方法，而是一個多層次的複合架構：

1. **軌跡插補層**（10~100 Hz）：依第 15 章的方法產生 q_d 、 $\dot{q}_d$ 、 $\ddot{q}_d$ ，並做動力學可行性檢查。
2. **動力學前饋層**（1 kHz）：以逆動力學計算名目力矩 $\tau_{ff} = M \ddot{q}_d + C \dot{q}_d + G$ ，這是「理論上需要的力矩」。
3. **位置 / 速度回授層**（1~4 kHz）：PID 修正實際的追隨誤差，輸出 τ_{fb} 。
4. **電流迴路層**（10~20 kHz）：驅動器內部的 PI 控制，確保實際電流跟上力矩指令（第 5 章）。
5. **安全監控層**：檢查力矩、速度、位置是否超限，異常時觸發保護。

總力矩為 $\tau = \tau_{ff} + \tau_{fb}$ 。這個架構的精神就是本章反覆強調的：**前饋處理可預測的部分**，**回授處理不可預測的部分**。

17.10.2 三個實務要點

要點一：前饋用期望值，回授用實際值。 τ_{ff} 應以 q_d 、 $\dot{q}_d$ 計算（而非實際的 q 、 $\dot{q}$ ）。理由是：若用實際值，感測雜訊會直接進入前饋通道被放大；用期望值則前饋是純粹的「開迴路預測」，平滑無雜訊。

要點二：微分項用速度回授而非誤差微分。回顧第 16 章，若對誤差微分，當 $\dot{q}_d$ 快速變化時會產生大的尖峰。實務上多採 $\tau_{fb} = K_p \cdot e - K_d \cdot \dot{q}$ （直接對實際速度做阻尼），這稱為「速度回授」形式。

要點三：模型與實機的參數必須同步。若更換了夾具或負載，控制器的動力學參數必須同步更新，否則前饋會系統性偏差。多數協作機器人提供「負載辨識」功能，能自動估測末端負載的質量與質心。

◎ 為何協作機器人特別需要模型式控制

協作機器人有兩個需求，使它們比傳統工業機器人更依賴動力學模型。**需求一：碰撞偵測**（第 11.6.3 節）。要由關節力矩推算外力，必須先精確扣除 $M \ddot{q}$ 、 $C \dot{q}$ 、 G 三項——模型誤差直接轉化為誤判或漏判。**需求二：手動導引**（17.5.3 節）。零力控制完全建立在重力與摩擦的精確補償上。因此協作機器人廠商通常會在出廠前做完整的動力學參數辨識，並提供線上的負載自動辨識功能——這在傳統工業機器人上並非標準配備。

17.11 由控制律邁向最佳性與穩定性

17.11.1 本章的成果

本章完成了模型式控制的核心。你現在能夠：辨識機器人的四個非線性來源、推導計算力矩控制並理解其回授線性化的本質、實作重力補償與零力控制、分析模型誤差的影響、在關節空間與操作空間之間選擇合適的控制架構、並理解滑模、自適應、魯棒與 MPC 四種進階方法的思想。

最重要的是，你理解了那個貫穿全章的設計哲學：**用模型處理已知的部分，用回授處理未知的部分**。這不只是控制技術，更是工程思維——面對複雜系統時，先用理解把問題縮小，剩下的才交給反饋修正。

17.11.2 兩個尚未回答的問題

但本章留下了兩個重要的問題。

問題一：增益該怎麼選才是「最好」的？第 16 章告訴我們 $K_p = \omega_n^2$ 、 $K_d = 2\zeta\omega_n$ ，但 ω_n 該取多少？調高則反應快但控制量大、調低則省力但遲鈍。這中間有沒有一個「最佳」的權衡點？這需要先定義什麼叫「好」——**第 18 章（最佳控制與 Riccati 方程）**將以代價函數的形式回答這個問題，並導出 LQR 這個能自動算出最佳增益的方法。

問題二：我們怎麼知道控制器一定不會失控？ 本章的推導都建立在「模型正確」的假設上，並以直覺論證了模型誤差的影響。但這不是嚴格的證明。**第 19 章（可控性與穩定性）**將引入 Lyapunov 方法，嚴格證明本章各種控制律的穩定性——包括 17.5.2 節那個「PD 加重力補償保證全域漸近穩定」的定理。

第 20 章（力控制）則會把本章的操作空間控制推進一步：不只控制末端的運動，還要控制末端與環境之間的力。屆時 Λ 矩陣與動態解耦的價值將完全展現。

換句話說：第 17 章告訴你**控制器該長什麼樣**，第 18 章告訴你**參數該怎麼選最好**，第 19 章告訴你**為什麼它一定有效**。

完整計算例題

例題 17-1 計算力矩控制的完整計算

題目：平面 2R 手臂（參數同第 13 章例題 13-2： $L_1=0.4$ 、 $L_2=0.3$ m， $m_1=3.0$ 、 $m_2=2.0$ kg， $\alpha=0.540$ 、 $\beta=0.120$ 、 $\delta=0.060$ kg·m²）。目前狀態 $q = (30^\circ, 60^\circ)$ 、 $\dot{q} = (1.0, -0.5)$ rad/s。軌跡要求 $q_d = (32^\circ, 58^\circ)$ 、 $\dot{q}_d = (1.2, -0.4)$ rad/s、 $\ddot{q}_d = (2.0, 1.0)$ rad/s²。控制增益取 $\omega_n = 20$ rad/s、 $\zeta = 1$ （臨界阻尼）。（a）求 K_p 、 K_d 。（b）計算虛擬控制 u 。（c）計算實際力矩 τ 。（d）驗證閉環誤差動態。

解（a）：由第 16 章的公式，臨界阻尼時：

$$K_p = \omega_n^2 = 20^2 = 400 \quad (1/s^2) \quad K_d = 2\zeta\omega_n = 2(1)(20) = 40 \quad (1/s)$$

增益。注意 CTC 的增益是無量綱化的（作用在加速度層級），與直接 PID 的增益單位不同。

解（b）：先計算誤差（換算為弧度）：

$$\begin{aligned} e &= q_d - q = (32-30, 58-60)^\circ = (2, -2)^\circ = (0.0349, -0.0349) \text{ rad} \\ \dot{e} &= \dot{q}_d - \dot{q} = (1.2-1.0, -0.4-(-0.5)) = (0.2, 0.1) \text{ rad/s} \end{aligned}$$

位置與速度誤差。

$$\begin{aligned} u &= \ddot{q}_d + K_d \cdot \dot{e} + K_p \cdot e \\ &= (2.0, 1.0) + 40(0.2, 0.1) + 400(0.0349, -0.0349) \\ &= (2.0, 1.0) + (8.0, 4.0) + (13.96, -13.96) \\ &= (23.96, -8.96) \text{ rad/s}^2 \end{aligned}$$

虛擬控制輸入，即「期望的關節加速度」。

解（c）：把 u 當作期望加速度代入逆動力學。先計算各項（ $\theta_2=60^\circ$ ， $c_2=0.5$ 、 $s_2=0.866$ ）：

$$M = \begin{bmatrix} \alpha + 2\beta c_2 & \delta + \beta c_2 \\ \delta + \beta c_2 & \delta \end{bmatrix} = \begin{bmatrix} 0.660 & 0.120 \\ 0.120 & 0.060 \end{bmatrix}$$

慣性矩陣。

$$M \cdot u = \begin{bmatrix} 0.660(23.96) + 0.120(-8.96) \\ 0.120(23.96) + 0.060(-8.96) \end{bmatrix} = \begin{bmatrix} 15.813 - 1.075 \\ 2.875 - 0.538 \end{bmatrix} = \begin{bmatrix} 14.738 \\ 2.337 \end{bmatrix}$$

慣性項。注意這裡用的是實際的 $\dot{q}$ 計算 C 項，但用 u 取代 $\ddot{q}$ 。

科氏離心項（用實際速度 $\dot{q} = (1.0, -0.5)$ ， $\beta_{s_2} = 0.1039$ ）：

$$C \dot{q} = \begin{bmatrix} -0.1039(0.25) - 2(0.1039)(1.0)(-0.5) \\ 0.1039(1.0)^2 \end{bmatrix} = \begin{bmatrix} -0.026 + 0.104 \\ 0.104 \end{bmatrix} = \begin{bmatrix} 0.078 \\ 0.104 \end{bmatrix}$$

科氏與離心項。

重力項（由第 13 章例題 13-2，同姿態）： $G = (11.88, 0)^T \text{ N}\cdot\text{m}$ 。

$$\tau = M u + C \dot{q} + G = \begin{bmatrix} 14.738 + 0.078 + 11.88 \\ 2.337 + 0.104 + 0 \end{bmatrix} = \begin{bmatrix} 26.70 \\ 2.44 \end{bmatrix} \text{ N}\cdot\text{m}$$

最終送出的力矩。

解（d）：若模型完全準確，代入系統方程可得 $\ddot{q} = u$ 。因此誤差動態為：

$$\ddot{e} + 40\dot{e} + 400e = 0$$

閉環誤差方程。這是臨界阻尼的二階系統，其解為 $e(t) = (A + Bt)e^{-20t}$ 。

誤差的包絡線時間常數為 $1/20 = 0.05$ 秒，因此約 4 個時間常數（0.2 秒）後誤差衰減至 2% 以下。重要的是：這個收斂行為與姿態、速度、負載完全無關——這正是 CTC 的價值。

例題 17-2 重力補償與穩態誤差

題目：承上手臂。（a）若採用純 PD 控制（無重力補償）， $K_p = 320 \text{ N}\cdot\text{m}/\text{rad}$ ，求在 $q = (0^\circ, 90^\circ)$ 姿態下維持靜止的穩態誤差。（b）若加入重力補償，穩態誤差為何？（c）若要用純 PD 把誤差壓到 0.5° 以下，需要多大的 K_p ？（d）討論此增益的可行性。

解（a）：先計算該姿態的重力項。 $q = (0^\circ, 90^\circ)$ 時， $c_1 = 1$ 、 $c_{12} = \cos(90^\circ) = 0$ 。

$$G_1 = (m_1 l_1 + m_2 l_1)g \cdot c_1 + m_2 l_2 g \cdot c_{12} = (3.0 \times 0.2 + 2.0 \times 0.4)(9.8)(1) + 0 = (0.6 + 0.8)(9.8) = 13.72 \text{ N}\cdot\text{m}$$

第一關節的重力力矩（此姿態為水平伸展，重力最大）。

純 PD 在穩態時（ $\dot{q} = 0$ 、 $\ddot{q} = 0$ ）： $K_p \cdot e = G$ ，故：

$$e_{ss} = G_1/K_p = 13.72/320 = 0.0429 \text{ rad} = 2.46^\circ$$

穩態誤差。手臂會比目標位置低 2.46 度。

解 (b)：加入重力補償後，控制律為 $\tau = K_p \cdot e + K_d \cdot \dot{e} + G(q)$ 。穩態時 $K_p \cdot e + G = G$ ，故 $e_{ss} = 0$ 。誤差完全消除。

解 (c)：要求 $e_{ss} \leq 0.5^\circ = 0.00873 \text{ rad}$ ：

$$K_p \geq G_1/e_{ss} = 13.72/0.00873 = 1572 \text{ N}\cdot\text{m/rad}$$

所需的比例增益，是原本的 4.9 倍。

解 (d)：這個增益不可行，理由有三。

理由一：激發共振。由 $\omega_n = \sqrt{(K_p/M)}$ ，取 $M_{11} \approx 0.66$ (該姿態下)： $\omega_n = \sqrt{(1572/0.66)} = 48.8 \text{ rad/s} \approx 7.8 \text{ Hz}$ 。若機構的第一共振頻率在 10~20 Hz (大型手臂常見)，控制器頻寬已太接近，會激發振動。

理由二：放大雜訊。編碼器的量化雜訊若為 0.001 rad，經 $K_p = 1572$ 放大後產生 1.57 N·m 的力矩抖動——這會造成明顯的機械噪音與磨損。

理由三：安全風險。高增益使機器人變得極「硬」，碰撞時的衝擊力大幅增加，這對協作應用是不可接受的。

結論：這正是重力補償的價值——用一行模型計算，達成了增益調高五倍也達不到的效果，而且沒有任何副作用。這是本章「用模型處理已知部分」哲學最直接的例證。

例題 17-3 模型誤差的影響評估

題目：承例題 17-1 的情境。假設控制器使用的模型有誤差：末端多了一個未建模的 1.0 kg 負載 (安裝在連桿 2 末端，距第二關節 0.3 m)。(a) 計算真實的動力學參數變化。(b) 估算重力項的誤差。(c) 估算此誤差造成的穩態位置偏差 (設 $K_p = 400$ 、外層為 CTC)。(d) 提出兩種因應方式。

解 (a)：未建模負載 $m_L = 1.0 \text{ kg}$ 位於 $r = 0.3 \text{ m}$ 處。它對各項的影響為：

$$\begin{aligned}\Delta\delta &= m_L r^2 = 1.0(0.09) = 0.090 \text{ kg}\cdot\text{m}^2 \\ \Delta\beta &= m_L L_1 r = 1.0(0.4)(0.3) = 0.120 \text{ kg}\cdot\text{m}^2 \\ \Delta\alpha &= m_L (L_1^2 + r^2) = 1.0(0.16 + 0.09) = 0.250 \text{ kg}\cdot\text{m}^2\end{aligned}$$

慣性參數的增量。注意 β 增加了 100% (由 0.120 變為 0.240) ——耦合大幅增強。

解 (b)：重力項的增量 (在 $q = (30^\circ, 60^\circ)$ ， $c_{12} = \cos(90^\circ) = 0$)：

$$\Delta G_1 = m_L (L_1 c_1 + r \cdot c_{12}) g = 1.0(0.4 \times 0.866 + 0.3 \times 0)(9.8) = 1.0(0.3464)(9.8) =$$

$$\Delta G_2 = m_L \cdot r \cdot c_{12} \cdot g = 0 \text{ N}\cdot\text{m}$$

重力項的誤差。第一關節少補償了 $3.4 \text{ N}\cdot\text{m}$ 。

解 (c)：這個未補償的力矩，會被外層 PD 視為干擾。在 CTC 架構下，等效的干擾加速度為：

$$\Delta \ddot{q} = M^{-1} \cdot \Delta G \quad M^{-1} = (1/\det) \begin{bmatrix} 0.060 & -0.120 \\ -0.120 & 0.660 \end{bmatrix}, \det = 0.660(0.060) - 0.120^2 = 0.0252$$

慣性矩陣的反矩陣。

$$\Delta \ddot{q}_1 = (0.060 \times 3.395)/0.0252 = 0.2037/0.0252 = 8.08 \text{ rad/s}^2$$

等效干擾加速度。

穩態時 $K_p \cdot e = \Delta \ddot{q}$ (在加速度層級)，故：

$$e_{ss} = \Delta \ddot{q}_1 / K_p = 8.08/400 = 0.0202 \text{ rad} = 1.16^\circ$$

穩態誤差約 1.16° 。

解 (d)：方式一：負載辨識。讓機器人執行一段激勵軌跡，由實測力矩反推負載參數 (第 13.10 節)，再更新模型。多數協作機器人提供此功能，通常只需十幾秒。

方式二：加入積分項。在外層 PD 加上積分項變成 PID，可消除這個常值誤差。代價是第 16 章提到的響應變慢與積分飽和風險。實務上兩者常並用：先用負載辨識消除大部分誤差，剩餘的小殘差交給積分項。

程式範例：計算力矩控制與 PID 的對比模擬

以下 Python 程式在平面 2R 手臂上實作三種控制器 (獨立關節 PID、PD 加重力補償、計算力矩控制)，執行同一段軌跡並比較追隨誤差。同時示範模型誤差與負載變化的影響。

```
import numpy as np

class Arm2R:
    """平面 2R 手臂動力學 (沿用第 13、14 章)"""
    def __init__(s, L1=0.4, L2=0.3, m1=3.0, m2=2.0, mL=0.0, g=9.8):
        s.L1, s.L2, s.g = L1, L2, g
        s.m1, s.m2, s.mL = m1, m2, mL          # mL 為末端額外負載
        s.l1, s.l2 = L1/2, L2/2
        I1, I2 = m1*L1**2/12, m2*L2**2/12
        # 把末端負載併入連桿 2 的等效參數
        m2e = m2 + mL
```

```

l2e = (m2*s.l2 + mL*L2)/m2e if m2e > 0 else s.l2
I2e = I2 + m2*(l2e-s.l2)**2 + mL*(L2-l2e)**2
s.m2e, s.l2e = m2e, l2e
s.alpha = m1*s.l1**2 + I1 + m2e*(L1**2 + l2e**2) + I2e
s.beta = m2e*L1*l2e
s.delta = m2e*l2e**2 + I2e

def M(s, q):
    c2 = np.cos(q[1])
    return np.array([[s.alpha + 2*s.beta*c2, s.delta + s.beta*c2],
                    [s.delta + s.beta*c2, s.delta]])

def Cqd(s, q, qd):
    sn = np.sin(q[1])
    return np.array([-s.beta*sn*qd[1]**2 - 2*s.beta*sn*qd[0]*qd[1],
                    s.beta*sn*qd[0]**2])

def G(s, q):
    c1, c12 = np.cos(q[0]), np.cos(q[0]+q[1])
    return np.array([(s.m1*s.l1 + s.m2e*s.l1)*s.g*c1 + s.m2e*s.l2e*s.g*c12,
                    s.m2e*s.l2e*s.g*c12])

def forward_dynamics(s, q, qd, tau):
    return np.linalg.solve(s.M(q), tau - s.Cqd(q, qd) - s.G(q))

# ----- 三種控制器 -----
def ctrl_pid(model, q, qd, ref, gains, state):
    """獨立關節 PID (無模型)"""
    qd_ref, qdd_ref, q_ref = ref
    Kp, Kd, Ki = gains
    e, ed = q_ref - q, qd_ref - qd
    state['I'] += e * state['dt']
    return Kp*e + Kd*ed + Ki*state['I']

def ctrl_pd_gravity(model, q, qd, ref, gains, state):
    """PD + 重力補償 (17.5 節)"""
    qd_ref, qdd_ref, q_ref = ref
    Kp, Kd, _ = gains
    return Kp*(q_ref - q) + Kd*(qd_ref - qd) + model.G(q)

def ctrl_ctc(model, q, qd, ref, gains, state):
    """計算力矩控制 (17.2 節)"""
    qd_ref, qdd_ref, q_ref = ref
    wn, zeta = state['wn'], state['zeta']
    Kp_a, Kd_a = wn**2, 2*zeta*wn # 加速度層級的增益
    u = qdd_ref + Kd_a*(qd_ref - qd) + Kp_a*(q_ref - q)
    return model.M(q) @ u + model.Cqd(q, qd) + model.G(q) # 逆動力學

# ----- 軌跡：五次多項式 (第 15 章) -----
def quintic(q0, qf, T, t):
    t = np.clip(t, 0, T); s = t/T
    dq = qf - q0
    return (q0 + dq*(10*s**3 - 15*s**4 + 6*s**5),

```

```

dq*(30*s**2 - 60*s**3 + 30*s**4)/T,
dq*(60*s - 180*s**2 + 120*s**3)/T**2)

# ----- 模擬 -----
def simulate(ctrl, gains, plant, model, T=1.0, dt=1e-4, wn=20.0, zeta=1.0):
    q0 = np.deg2rad([10.0, 20.0]); qf = np.deg2rad([70.0, -30.0])
    q, qd = q0.copy(), np.zeros(2)
    state = {'I': np.zeros(2), 'dt': dt, 'wn': wn, 'zeta': zeta}
    max_err, tau_max = 0.0, 0.0
    n = int(T/dt)
    for k in range(n):
        t = k*dt
        q_r, qd_r, qdd_r = quintic(q0, qf, T*0.8, t)
        tau = ctrl(model, q, qd, (qd_r, qdd_r, q_r), gains, state)
        tau = np.clip(tau, -60, 60) # 力矩飽和
        qdd = plant.forward_dynamics(q, qd, tau) # 真實系統
        qd = qd + qdd*dt; q = q + qd*dt # 半隱式歐拉
        if t > 0.05:
            max_err = max(max_err, np.abs(np.rad2deg(q_r - q)).max())
            tau_max = max(tau_max, np.abs(tau).max())
    final_err = np.abs(np.rad2deg(qf - q)).max()
    return max_err, final_err, tau_max

# ===== 情境一：模型完全準確 =====
print("=== 情境一：模型準確、無額外負載 ===")
plant = Arm2R(); model = Arm2R()
gains_pid = (np.array([300.,300.]), np.array([30.,30.]), np.array([100.,100.]))
gains_pd = (np.array([300.,300.]), np.array([30.,30.]), None)
print(f"{'控制器':<18}{'最大追隨誤差':>14}{'終點誤差':>12}{'最大力矩':>12}")
for name, fn, g in (("獨立關節 PID", ctrl_pid, gains_pid),
                    ("PD + 重力補償", ctrl_pd_gravity, gains_pd),
                    ("計算力矩控制 CTC", ctrl_ctc, gains_pd)):
    me, fe, tm = simulate(fn, g, plant, model)
    print(f"{name:<18}{me:12.4f}°{fe:11.4f}°{tm:11.2f} N·m")

# ===== 情境二：末端加上 1 kg 未建模負載 =====
print("\n=== 情境二：實機多 1.0 kg 負載，控制器模型未更新 ===")
plant2 = Arm2R(mL=1.0) # 真實系統有負載
model2 = Arm2R(mL=0.0) # 控制器模型沒有
for name, fn, g in (("獨立關節 PID", ctrl_pid, gains_pid),
                    ("PD + 重力補償", ctrl_pd_gravity, gains_pd),
                    ("計算力矩控制 CTC", ctrl_ctc, gains_pd)):
    me, fe, tm = simulate(fn, g, plant2, model2)
    print(f"{name:<18}{me:12.4f}°{fe:11.4f}°{tm:11.2f} N·m")

# ===== 情境三：更新模型後的 CTC =====
print("\n=== 情境三：CTC 且模型已更新（負載辨識後） ===")
me, fe, tm = simulate(ctrl_ctc, gains_pd, plant2, Arm2R(mL=1.0))
print(f"{'CTC (模型已更新)':<18}{me:12.4f}°{fe:11.4f}°{tm:11.2f} N·m")

```

```
# ===== 情境四：不同速度下的性能 =====
print("\n=== 情境四：速度對三種控制器的影響（最大追隨誤差） ===")
print(f"{'總時間':>8}{'PID':>12}{'PD+G':>12}{'CTC':>12}")
for T in (2.0, 1.0, 0.5, 0.3):
    r = []
    for fn, g in ((ctrl_pid, gains_pid), (ctrl_pd_gravity, gains_pd),
                  (ctrl_ctc, gains_pd)):
        me, _, _ = simulate(fn, g, plant, model, T=T)
        r.append(me)
    print(f"{T:7.1f}s{r[0]:11.3f}°{r[1]:11.3f}°{r[2]:11.3f}°")
print("→ 速度越快，PID 誤差惡化越明顯；CTC 幾乎不受影響")

# ===== 例題 17-2：穩態誤差驗證 =====
print("\n=== 例題 17-2：純 PD 的重力造成穩態誤差 ===")
arm = Arm2R()
q_test = np.deg2rad([0.0, 90.0])
G = arm.G(q_test)
for Kp in (320.0, 800.0, 1572.0):
    print(f"Kp={Kp:7.1f}: G1={G[0]:6.2f} N·m "
          f"穩態誤差={np.rad2deg(G[0]/Kp):6.3f}°")
print(f"加入重力補償後：穩態誤差 = 0.000° ← 一行模型勝過五倍增益")
```

請試著修改：（一）把情境四的總時間縮短到 0.2 秒，觀察 PID 是否會完全失控。（二）在 CTC 的模型中把慣性參數全部乘上 0.8（模擬 20% 模型誤差），觀察誤差增加多少——你會發現它仍遠優於 PID。（三）把重力補償改成只補償 90%，觀察殘餘的穩態誤差是否與理論值 $0.1G/Kp$ 相符。

實作：實體手臂的模型式控制

本章實作是在實體手臂上實現重力補償與計算力矩控制，並量化其相對於 PID 的改善。**安全提醒**：模型式控制若參數錯誤可能導致手臂暴衝，務必先在低增益、低速下測試，並確保緊急停止隨手可及。

- 前置作業**：沿用第 13 章實作中辨識出的動力學參數（質量、質心、慣量、摩擦）。若尚未完成，請先回頭完成該實作。
- 重力補償驗證**：把手臂設為只輸出 $G(q)$ 的模式（無 PD 項）。逐一測試多個姿態，觀察手臂是否能「浮」在原處。若緩慢下墜代表補償不足、若自行上飄代表過度補償，據此微調參數。
- 零力控制體驗**：在重力補償正確後，用手推動手臂各關節，感受推動的難易。記錄「推動所需的最小力」——這反映了未補償的摩擦大小。

4. **摩擦補償**：加入 $\tau_f = F_v \cdot \dot{q} + F_c \cdot \text{sign}(\dot{q})$ 的摩擦補償項，重複上一步，量化推動所需力量的下降幅度。注意 sign 函數在零速附近會抖動，應改用平滑的近似。
5. **穩態誤差比較**：設定一個水平伸展的目標姿態，分別以「純 PD」與「PD + 重力補償」執行，量測穩態位置誤差。驗證例題 17-2 的理論值 $e_{ss} = G/K_p$ 。
6. **軌跡追隨比較**：規劃一段中速軌跡（第 15 章），分別以三種控制器執行，記錄全程的追隨誤差。繪出三條誤差曲線於同一張圖。
7. **速度掃描**：把同一段軌跡的執行時間逐步縮短（如 3、2、1、0.6 秒），記錄各控制器的最大追隨誤差。繪出「速度 vs 誤差」曲線——你應該會看到 PID 的曲線急遽上升，而 CTC 的曲線相對平坦。
8. **負載測試**：在末端加裝已知重量的配重，重複步驟 6。分別測試「模型未更新」與「模型已更新」兩種情況，量化負載辨識的價值。
9. **安全性比較（選作）**：在低速下讓手臂輕觸一個固定障礙物，比較高增益 PID 與低增益 CTC 的接觸力大小。這能直觀展示「模型式控制可用較低增益達成同等精度」對安全性的意義。

表 17-4 控制器性能比較記錄表（第一列為範例）

控制器	執行時間	最大追隨誤差	穩態誤差	最大力矩	備註
獨立關節 PID	1.0 s	2.84°	0.42°	38.2 N·m	轉彎處明顯落後
（自行填寫其餘測試）					

國中小也看得懂：端一杯水走過房間的兩種方法

想像你要端一杯滿滿的水走過房間。有兩種方法：

第一種方法：邊走邊看。你盯著水面，看到水往左邊晃，就趕快往右邊調整；看到往右晃，就往左調。這樣可以走到，但水會一直晃來晃去，而且你必須非常專心。這就是「PID 控制」——看到偏了才修正。

第二種方法：先想好再走。如果你已經知道「等一下要轉彎，轉彎的時候水會往外甩」，那你可以在轉彎之前就先把杯子往內傾斜一點點，剛好抵消那個甩出去的力。這樣水根本不會晃，你也不用一直緊張地盯著。這就是這一章的「計算力矩控制」——用知識預先準備好。

那第一種方法就沒用了嗎？當然有用！因為總會有你沒想到的事——地板突然有一塊不平、有人突然撞到你。這時候還是要靠「邊走邊看」來救。

所以最好的方法是**兩個一起用**：能預測的事情先算好（這叫前饋），沒預測到的事情再用眼睛修正（這叫回授）。厲害的機器人工程師就是這樣做的。

最後告訴你一個很酷的小知識：這一章有一個叫做「重力補償」的技巧，就是讓機器人的馬達剛好出力抵消自己的重量。做到之後，機器手臂會像在無重力的太空一樣**浮在空中**——你用一根手指就能把它推來推去！很多新型的機器人就是用這個方法，讓工人可以直接用手教它動作，不需要寫程式。

叫獸提醒與常見錯誤

- 誤區一：以為調高 PID 增益就能解決高速誤差。系統的有效阻尼比隨姿態變化，固定增益不可能在所有姿態下都最佳。
- 誤區二：前饋項使用實際的 q 、 $\dot{q}$ 計算。應使用期望值 q_d 、 $\dot{q}_d$ ，否則感測雜訊會直接進入前饋通道被放大。
- 誤區三：忘記軌跡必須提供加速度。CTC 的前饋需要 $\ddot{q}_d$ ，若軌跡生成器只輸出位置與速度，前饋效果會大打折扣。
- 誤區四：以為模型必須完美才能用 CTC。80% 準確的模型就能把干擾降低五倍——模型的價值不在完美而在大幅減負。
- 誤區五：更換夾具或負載後不更新模型參數。這會造成系統性的前饋偏差，應使用負載辨識功能。
- 誤區六：零力控制未補償摩擦。只補償重力的話，手臂仍會因減速機摩擦而「推不動」，體驗很差。
- 誤區七：在高增益下追求精度而忽略安全。模型式控制的價值之一，就是能用較低增益達成同等精度，這對協作應用至關重要。
- 誤區八：忽略力矩飽和對線性化的破壞。一旦飽和， $\ddot{q} \neq u$ ，CTC 的性能保證即失效，應在規劃階段就避免。
- 誤區九：混淆回授線性化與近似線性化。前者在整個狀態空間精確成立（但需完整模型），後者只在工作點附近有效。

- 誤區十：對驅動不足系統套用回授線性化。人形機器人的浮動基座、無人機的姿態，都無法精確線性化，需要其他方法。
- 誤區十一：在操作空間控制中使用運動學版本的零空間投影。力控制場合必須用動態一致版本，否則次任務會干擾主任務的接觸力。
- 誤區十二：以為滑模控制的顫振可以忽略。高頻切換會激發機構振動、加速磨損並產生噪音，實務上必須用邊界層緩解。

課堂討論

1. 開頭故事中，小好調好的增益在換姿態與加負載後都失效。請用 $\omega_n(q)$ 與 $\zeta(q)$ 的公式，定量解釋這兩種情況下系統動態各自如何改變。
2. 例題 17-2 顯示「一行重力補償勝過五倍增益」。請討論：還有哪些機器人控制問題適合用「模型」而非「增益」來解決？請舉兩例並說明理由。
3. 手動導引示教徹底改變了機器人的易用性。請討論它對「誰能使用機器人」的影響，並分析這個功能需要哪三項技術同時到位（提示：對照第 5、6、11 章）。
4. 四種進階方法（滑模、自適應、魯棒、MPC）各有適用場合。請為以下三個情境各選一種並說明理由：（a）機器人抓取重量未知的物件；（b）在有大幅外力干擾的環境作業；（c）四足機器人的即時步態控制。
5. 本章的哲學是「用模型處理已知、用回授處理未知」。請討論這個原則在機器人以外的領域有何對應（提示：可考慮天氣預報、庫存管理、或第 31 章的機器學習）。

章末摘要

- 機器人的四個非線性來源：慣性隨姿態變化、關節耦合、速度平方項、重力隨姿態變化。
- 固定增益 PID 的有效自然頻率與阻尼比隨姿態變化（ $\omega_n = \sqrt{K_p/M(q)}$ 、 $\zeta = K_d/(2\sqrt{K_p \cdot M(q)})$ ），因此不可能在所有姿態下都最佳。
- 計算力矩控制 $\tau = M(q)u + C\dot{q} + G$ 使閉環系統成為 $\ddot{q} = u$ ，即 n 個獨立的雙積分器。
- 外層設計 $u = \ddot{q}_d + K_d \dot{e} + K_p e$ ，得到誤差動態 $\ddot{e} + K_d \dot{e} + K_p e = 0$ ——不含 M 、 C 、 G ，與姿態負載完全無關。
- 實作上直接呼叫一次逆動力學 $\tau = \text{InverseDynamics}(q, \dot{q}, u)$ ，複雜度 $O(n)$ 。

- 回授線性化是精確的（整個狀態空間成立），不同於只在工作點附近有效的近似線性化；但需要完整模型且僅適用於完全驅動系統。
- 模型誤差造成兩個偏差：有效增益偏移與等效干擾；但 80% 準確的模型仍能把干擾降低五倍。
- 重力補償是投報率最高的改良：消除穩態誤差（純 PD 的 $e_{ss} = G/K_p$ ）、避免依賴積分項、計算量極小。
- Takegaki-Arimoto 定理：PD 加重力補償保證定點調節的全域漸近穩定，不需 M 與 C 的任何資訊。
- 純重力補償（ $\tau = G(q)$ ）即為零力控制，是協作機器人手動導引示教的原理，實作需同時補償摩擦。
- 三種控制律的選擇：低速固定負載用 PD 加重力補償；高速、負載變化或高精度需求用完整 CTC。
- 操作空間控制 $\tau = J^T[\Lambda \dot{u}_x + \mu + p]$ 直接在任務空間設計，其中 $\Lambda = (J M^{-1} J^T)^{-1}$ 為末端的等效慣量。
- 操作空間控制達成動態解耦，使各方向可獨立設計剛度——這是第 20 章力控制的基礎。
- 四種進階方法：滑模（強健但顫振）、自適應（估測參數但需持續激勵）、魯棒（保守）、MPC（能處理約束但計算量大）。
- 實務架構為多層複合：軌跡插補、動力學前饋、位置速度回授、電流迴路、安全監控；前饋用期望值、回授用實際值。
- 核心哲學：用模型處理已知的部分，用回授處理未知的部分。

觀念題與計算題

觀念題

1. 列舉機器人動力學的四個非線性來源，並各說明其對控制的影響。
2. 為何固定增益的 PID 無法在所有姿態下都達到最佳性能？請以 ω_n 與 ζ 的公式說明。
3. 推導計算力矩控制律，並證明其閉環系統為 $\ddot{q} = u$ 。
4. 為何說 CTC 的誤差動態「與姿態、速度、負載無關」？這帶來什麼實務好處？
5. 說明回授線性化與近似線性化的差異，以及各自的適用條件。

6. 什麼是完全驅動與驅動不足系統？為何後者無法精確回授線性化？
7. 分析模型誤差對 CTC 的兩種影響，並說明為何「80% 準確的模型仍然有用」。
8. 推導純 PD 控制在有重力時的穩態誤差公式，並說明重力補償如何消除它。
9. 敘述 Takegaki-Arimoto 定理的內容與適用範圍。
10. 說明零力控制的原理與三個實作要點，以及它對機器人易用性的意義。
11. 比較三種關節空間控制律的模型需求、計算量與適用場合。
12. 說明操作空間控制的優勢，並解釋 $\Lambda = (J M^{-1} J^T)^{-1}$ 的物理意義。
13. 為何操作空間控制在奇異點處會失效？請由 Λ 的定義說明。
14. 比較滑模、自適應、魯棒三種方法的核心思想與主要缺點。
15. 說明 MPC 的四個步驟，以及它相對於前述方法的最大優勢。

計算題

1. 某平面 2R 手臂 $\alpha = 0.72$ 、 $\beta = 0.18$ 、 $\delta = 0.09 \text{ kg}\cdot\text{m}^2$ 。目前 $q = (45^\circ, 90^\circ)$ 、 $\dot{q} = (1.5, -0.8) \text{ rad/s}$ ，軌跡要求 $q_d = (48^\circ, 87^\circ)$ 、 $\dot{q}_d = (1.6, -0.7) \text{ rad/s}$ 、 $\ddot{q}_d = (3.0, 1.5) \text{ rad/s}^2$ 。取 $\omega_n = 25 \text{ rad/s}$ 、 $\zeta = 1$ 。(a) 求 K_p 、 K_d 。(b) 求虛擬控制 u 。(c) 求慣性矩陣 M 。(d) 求 $M \cdot u$ 。
2. 承上題手臂 ($m_1 = 3.5$ 、 $m_2 = 2.2 \text{ kg}$ 、 $L_1 = 0.45$ 、 $L_2 = 0.35 \text{ m}$ 、質心在中點)。(a) 計算 $q = (0^\circ, 0^\circ)$ (水平完全伸展) 姿態下的重力項 G 。(b) 若採純 PD 且 $K_p = 400 \text{ N}\cdot\text{m/rad}$ ，求穩態誤差 (度)。(c) 若要求誤差小於 0.3° ，需要多大的 K_p ？(d) 計算該 K_p 下的自然頻率，並判斷是否可能激發 15 Hz 的機構共振。
3. 某手臂在末端加裝了一個未建模的夾具，質量 0.8 kg 、質心距法蘭 0.12 m 。已知該姿態下 $M = [[0.85, 0.15], [0.15, 0.08]]$ 、 $L_1 = 0.4 \text{ m}$ 、 $c_1 = 0.7$ 、 $c_{12} = 0.3$ 。(a) 估算重力項的誤差 ΔG 。(b) 求 M 的反矩陣。(c) 計算等效干擾加速度。(d) 若 CTC 的 $K_p = 625$ ($\omega_n = 25$)，求穩態位置誤差。
4. 某六軸手臂的控制器需在 1 kHz 執行。已知：逆動力學需 900 次浮點運算、雅可比需 400 次、 Λ 矩陣的計算 (含 M^{-1} 與 3×3 求逆) 需約 1200 次。處理器每秒可執行 10^8 次運算。(a) 計算關節空間 CTC 的運算時間與週期佔比。(b) 計算操作空間控制的運算時間與佔比。(c) 若還需加上碰撞偵測 ($500 \mu\text{s}$)，判斷兩者是否仍可行。(d) 提出一個降低操作空間控制計算量的策略。

動手做與專題延伸

本章實作：控制器模組

延續第 13 ~ 16 章的程式庫，建立完整的控制器模組。這將是第 18 ~ 20 章的實驗平台。

- **階段一**：實作三種關節空間控制器（獨立 PID、PD 加重力補償、CTC），統一介面為 $\text{ctrl}(q, \dot{q}, \ddot{q}, \text{ref}) \rightarrow \tau$ 。
- **階段二**：建立模擬環境，整合第 15 章的軌跡生成與第 14 章的正向動力學，能執行完整的閉環模擬並記錄誤差曲線。
- **階段三**：加入模型誤差的模擬能力（真實系統與控制器模型使用不同參數），量化各控制器的強健性。
- **階段四**：實作操作空間控制，包含 Λ 矩陣計算與 J^T 映射，並與關節空間控制比較末端的追隨精度。
- **階段五**：實作力矩飽和與抗飽和機制，觀察飽和對 CTC 性能的破壞程度。

專題延伸

- **性能地圖**：對整個工作空間掃描，計算 PID 與 CTC 在各姿態下的追隨誤差，繪成兩張熱力圖並列比較。你會看到 PID 的誤差分布極不均勻（在慣量變化大的區域惡化），而 CTC 的分布相對均勻——這是本章理論最直觀的視覺證明。
- **模型誤差的敏感度分析**：系統性地為每一個動力學參數（各連桿的質量、質心、慣量）加入 $\pm 20\%$ 的誤差，分別量測 CTC 的追隨誤差變化。找出「最敏感的參數」——這正是實務上參數辨識時應優先確保精度的項目，可與第 9 章的運動學敏感度分析對照。
- **進階：自適應控制的實作與觀察**。實作 Slotine-Li 自適應控制器，在末端加上未知負載的情境下執行。繪出「參數估計值隨時間收斂」的曲線與「追隨誤差隨時間下降」的曲線。特別觀察：若軌跡過於單調（例如只在單一方向來回），參數可能不會收斂到真值——這就是「持續激勵條件」的實際體現。設計一段「豐富」的軌跡再試一次，比較兩者的差異。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 10 章：非線性控制，本章主要參考)。

- Khatib, O., "A Unified Approach for Motion and Force Control of Robot Manipulators: The Operational Space Formulation," IEEE J. Robotics and Automation, 1987 (操作空間控制的奠基論文) 。
- Takegaki, M. and Arimoto, S., "A New Feedback Method for Dynamic Control of Manipulators," ASME J. Dynamic Systems, 1981 (PD 加重力補償的穩定性證明) 。
- Slotine, J.-J. E. and Li, W., Applied Nonlinear Control, Prentice Hall (滑模與自適應控制的經典教材) 。
- Spong, M. W., Hutchinson, S. and Vidyasagar, M., Robot Modeling and Control, 2nd ed., Wiley (第 8 ~ 10 章：非線性與多變數控制) 。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer (第 8 章：運動控制，含各類控制律的完整比較) 。
- Slotine, J.-J. E. and Li, W., "On the Adaptive Control of Robot Manipulators," IJRR, 1987 (Slotine-Li 自適應控制器) 。
- Nakanishi, J. et al., "Operational Space Control: A Theoretical and Empirical Comparison," IJRR, 2008 (多種操作空間控制方法的實證比較) 。
- Rawlings, J. B., Mayne, D. Q. and Diehl, M., Model Predictive Control: Theory, Computation, and Design, Nob Hill (MPC 的權威教材) 。
- Di Carlo, J. et al., "Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control," IROS 2018 (MPC 在四足機器人上的代表性應用) 。

第 18 章

最佳控制與 Riccati 方程

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】畫面中央為一台小車上立著一根桿子的倒立擺，桿子微微右傾、小車正往右加速追趕，並以虛線標出「支點必須移到重心下方」。左側疊一張取舍曲線圖（橫軸為最大控制力、縱軸為穩定時間），曲線上標出三個點： Q 小（省力但慢）、 Q 適中、 Q 大（快但費力），並註明「這條線上的每一點都是最佳的——選哪一個取決於你在乎什麼」。右側為一個由後往前計算的網格圖，紅色箭頭由終點往起點逐層推算價值函數。標題文字：不再試誤——把「什麼叫好」寫成公式，最佳解就會自己出現。

叫獸開講 「輕便車上的倒立搖桿，如何自己找出最省力的平衡之道？」

小翰把第 17 章的計算力矩控制寫完了，模擬跑得很漂亮。但他有一個問題卡了三天。

「老師，公式說 $K_p = \omega n^2$ 、 $K_d = 2\zeta\omega n$ 。可是 ωn 該取多少？我試了 10、20、50、100——每一個看起來都『可以』，但我不知道哪一個是對的。」

叫獸沒有直接回答，而是在白板上畫了兩條曲線。「 ωn 取 100 的時候，誤差收斂得很快對不對？那你有沒有看馬達的力矩？」

小翰想了想：「……很大。有時候會撞到飽和。」

「那 ωn 取 10 呢？」「力矩很小，但誤差要好幾秒才收斂。」

「所以，」叫獸把兩條曲線並排，「這根本不是一個『對或錯』的問題，而是一個**取捨**的問題。你想要收斂快，就要付出大力矩的代價；你想要省力，就要忍受慢。而『哪一個最好』——**取決於你認為快有多重要、省力有多重要。**」

他在白板上寫下一個式子： $J = \int (\text{誤差的代價} + \text{用力的代價}) dt$ 。「工程上有一個很優雅的作法：把你的兩個願望寫成一個**代價函數**。誤差大要罰、用力大也要罰。然後問：有沒有一組控制律，能讓這個總代價最小？」

小好舉手：「這聽起來像最佳化。可是控制是一個隨時間變化的過程，不是一個固定的數值——要怎麼對『一整條軌跡』做最佳化？」

「好問題。這正是最佳控制與一般最佳化最大的差別。」叫獸拿出一顆玻璃彈珠與一段彎曲的軌道，「想像彈珠要從 A 滾到 B。你不是在選一個數字，你是在選**一整條路徑**——而路徑有無限多條。」

他停頓了一下，「1950 年代，一位叫 Bellman 的數學家想出了一個絕妙的辦法。他說：不要一次想整條路。**先問一個問題——如果我已經走到中途某個點，剩下的路該怎麼走最好？**如果你能回答『從任何一點出發的最佳走法』，那整條路的答案就自動出來了。」

「這個想法叫做**動態規劃**，它的核心是一句話：**最佳路徑的任何一段，本身也必定是最佳的。**聽起來像廢話，但它把一個無限維的問題，變成了一個可以遞迴求解的問題。」

叫獸在白板角落畫了一台小車，上面立著一根桿子。「今天結束時，你們會親手算出一個東

西——一個能自動平衡倒立擺的控制器。而且不是靠試誤調參數，是算出來的最佳解。」

本章為什麼重要

第 16、17 章教會了我們設計控制器的結構，卻留下一個懸而未決的問題：參數該怎麼選？增益調高則反應快但耗能大、容易飽和、對雜訊敏感；調低則省力但遲鈍。這個取捨沒有唯一答案——除非我們先明確定義「什麼叫好」。

最佳控制提供了這個框架。它要求設計者把目標寫成一個代價函數，然後在數學上求出使代價最小的控制律。這帶來三個層次的價值：

價值一：從試誤到計算。LQR（線性二次調節器）能由系統模型與兩個權重矩陣，直接算出最佳的回授增益——不需要 Ziegler-Nichols 那樣的試誤。對多輸入多輸出系統（如六軸手臂、四足機器人），這個優勢尤其明顯，因為手動調參幾乎不可行。

價值二：處理欠驅動系統。第 17 章的計算力矩控制只適用於完全驅動系統。但倒立擺、兩輪自平衡車、四旋翼無人機、人形機器人的浮動基座——這些都是欠驅動的，回授線性化行不通。LQR 則能直接處理。

價值三：連結多個領域。本章的 Bellman 方程與價值函數，正是第 31 章強化學習的數學基礎；而 MPC（第 17.8.4 節）本質上就是「有限時域的最佳控制反覆求解」。理解本章，等於同時打開了三扇門。

學習目標

- 能定義最佳控制問題的三個要素：系統動態、代價函數與邊界條件。
- 能說明離散與連續時間最佳控制問題的差異。
- 能敘述 Bellman 最佳性原理，並說明其為何能簡化問題。
- 能定義價值函數，並說明其物理意義。
- 能推導離散時間的 Bellman 方程並用動態規劃求解簡單問題。
- 能寫出 Hamilton-Jacobi-Bellman 方程並說明其意義與求解困難。
- 能定義線性二次調節器問題，並說明 Q 、 R 權重矩陣的作用。
- 能推導 Riccati 微分方程，並說明有限時域與無限時域的差異。

- 能求解代數 Riccati 方程並計算最佳回授增益 K 。
- 能將 LQR 應用於一維點質量與倒立擺，並解釋所得增益的物理意義。
- 能說明最佳控制與逆運動學、軌跡最佳化及強化學習的關係。

先備知識

第 13~14 章的動力學模型（提供系統方程）、第 16 章的二階系統分析與增益的物理意義、第 17 章的控制架構。數學方面需要：矩陣運算、偏導數、以及狀態空間表示法的基本概念（本章 18.2 節會完整介紹）。

本章是全書數學密度最高的章節之一。建議讀法：**18.1~18.3 節建立直覺，務必徹底理解 Bellman 原理**；18.4~18.6 節的推導可先掌握結論與意義，細節之後回頭補；18.7 節之後回到具體應用，會重新變得直觀。若時間有限，至少要讀懂 18.6 節的 LQR 與 18.8 節的實例。

核心名詞中英對照

表 18-1 本章核心名詞

中文	英文	說明
最佳控制	Optimal Control	求使代價函數最小的控制律
代價函數	Cost Function	衡量控制表現好壞的量化指標
狀態	State	完整描述系統當前情況的變數集合
狀態空間表示	State-space Representation	以一階微分方程組描述系統的形式
價值函數	Value Function	由某狀態出發的最小可能代價
最佳性原理	Principle of Optimality	最佳路徑的任一段本身也是最佳的
動態規劃	Dynamic Programming	由後往前遞迴求解最佳決策的方法
Bellman 方程	Bellman Equation	價值函數所滿足的遞迴關係式
HJB 方程	Hamilton-Jacobi-Bellman Equation	連續時間版本的 Bellman 方程
線性二次調節器	Linear Quadratic Regulator, LQR	線性系統加二次代價的最佳控制
Riccati 方程	Riccati Equation	LQR 中決定最佳增益的矩陣方程
代數 Riccati 方程	Algebraic Riccati Equation, ARE	無限時域下的穩態 Riccati 方程
狀態回授	State Feedback	以 $u = -Kx$ 形式的控制律
權重矩陣	Weight Matrix	代價函數中調整各項相對重要性的矩陣
倒立擺	Inverted Pendulum	重心在支點上方的不穩定系統

中文	英文	說明
欠驅動	Underactuated	控制輸入數少於自由度的系統

18.1 最佳控制問題的定義

18.1.1 三個要素

一個最佳控制問題由三個要素構成。

要素一：系統動態。描述狀態如何隨控制輸入演變。連續時間形式為：

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$$

$\mathbf{x}$ 為狀態向量、 $\mathbf{u}$ 為控制輸入。對機器人而言， $\mathbf{x}$ 通常包含位置與速度（如 $\mathbf{x} = [q, \dot{q}]^T$ ）， $\mathbf{u}$ 則是關節力矩。

要素二：代價函數。量化「什麼叫好」。標準形式為：

$$J = \varphi(\mathbf{x}(T)) + \int_0^T L(\mathbf{x}, \mathbf{u}) dt$$

φ 稱為終端代價（到達終點時的懲罰）、 L 稱為運行代價（過程中每一刻的懲罰）。

要素三：邊界條件與約束。初始狀態 $\mathbf{x}(0) = \mathbf{x}_0$ 、可能的終端條件、以及控制與狀態的限制（如力矩極限）。

問題就是：找出控制訊號 $\mathbf{u}(t)$ （ t 由 0 到 T ），使 J 最小。

18.1.2 與一般最佳化的差別

這與第 10 章的逆運動學最佳化有一個本質差異。逆運動學是在有限維空間中找一個點（一組關節角度）；最佳控制則是在函數空間中找一條曲線（整條控制訊號 $\mathbf{u}(t)$ ）。這是無限維的問題。

正如叫獸的彈珠比喻：你不是在選一個數字，而是在選一整條路徑，而路徑有無限多條。這使得直接套用梯度下降之類的方法變得困難——你要對「一個函數」求導數。

Bellman 的貢獻，正是提供了一個繞過這個困難的方法。

18.1.3 為何機器人需要最佳控制

在機器人上，最佳控制回答的問題形形色色：

- **最省能：**以最少的能量完成搬運（電池供電的移動機器人特別在意）。

- **最快速**：在力矩極限下用最短時間完成動作（第 15.8.3 節的時間最佳軌跡）。
- **最平順**：最小化急跳度以減少振動（第 15.5 節）。
- **最穩定**：在維持平衡的前提下最小化控制量（倒立擺、雙足行走）。
- **多目標權衡**：同時考慮精度、能耗與時間，以權重調整優先順序。

這些看似不同的需求，在最佳控制的框架下都只是「換一個代價函數」而已——這正是這個框架的威力所在。

18.2 狀態空間表示法

18.2.1 把高階系統化為一階

最佳控制理論建立在狀態空間表示上。它的核心技巧是：把任何高階微分方程，化為一組一階微分方程。

以第 16 章的一維點質量為例，其方程為二階： $m \ddot{x} = f$ 。定義狀態向量：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \text{位置} \\ \text{速度} \end{bmatrix}$$

狀態向量。選擇位置與速度作為狀態，是機械系統的標準作法。

則系統可寫成兩個一階方程：

$$\begin{aligned} \dot{x}_1 &= x_2 \quad (\text{位置的導數是速度}) \\ \dot{x}_2 &= f/m \quad (\text{速度的導數是加速度}) \end{aligned}$$

化為一階系統。

寫成矩陣形式：

$$\dot{\mathbf{x}} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 0 \\ 1/m \end{bmatrix} u \quad \text{即} \quad \dot{\mathbf{x}} = \mathbf{A} \mathbf{x} + \mathbf{B} u$$

線性狀態空間形式。 $\mathbf{A}$ 稱為系統矩陣、 $\mathbf{B}$ 稱為輸入矩陣。這是控制理論中最標準的表示法。

18.2.2 為何要這樣做

這個轉換有三個好處。**好處一：統一形式**。無論原系統是幾階，都化為 $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, u)$ 的一階形式，理論可以統一處理。**好處二：完整描述**。狀態的定義是「知道現在的狀態與未來的輸入，就能完全決定未來」——這正是 Bellman 原理成立的前提。**好處三：矩陣工具**。線性系統可用線性代數的全套工具分析（特徵值、可控性、可觀測性——第 19 章的主題）。

18.2.3 機器人的狀態空間

對 n 自由度的機器手臂，狀態向量為 $2n$ 維：

$$\mathbf{x} = \begin{bmatrix} \mathbf{q} \end{bmatrix} \in \mathbb{R}^{2n}, \quad \mathbf{u} = \boldsymbol{\tau} \in \mathbb{R}^n$$

狀態含所有關節的位置與速度。六軸手臂的狀態空間是 12 維的。

其動態方程來自第 13 章：

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{q}} \\ \ddot{\mathbf{q}} \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{q}} \\ \mathbf{M}^{-1}(\boldsymbol{\tau} - \mathbf{C} \dot{\mathbf{q}} - \mathbf{G}) \end{bmatrix}$$

機器人的狀態空間形式。注意這是非線性的（因 $\mathbf{M}$ 、 $\mathbf{C}$ 、 $\mathbf{G}$ 都是 $\mathbf{q}$ 的函數）。

這帶出一個重要的實務策略：若已經套用第 17 章的計算力矩控制，系統就變成了 $\mathbf{q}'' = \mathbf{u}$ ，此時狀態方程化為線性的雙積分器——LQR 就能直接應用。這是模型式控制與最佳控制結合的標準作法。

18.3 Bellman 最佳性原理

18.3.1 核心洞見

Bellman 於 1950 年代提出的最佳性原理，可以用一句話表達：

◎ Bellman 最佳性原理

最佳路徑的任何一段，本身也必定是最佳的。換句話說：無論之前如何到達當前狀態，剩下的最佳決策只取決於當前狀態，而與過去的歷史無關。

這個原理有一個生活化的驗證。假設從台北到高雄的最快路線經過台中。那麼「台中到高雄」這一段，必定也是台中到高雄的最快路線——否則你可以把它替換成更快的走法，使整條路線更快，這與原路線是最快的假設矛盾。

這個看似顯然的觀察，威力極大：它把「一次決定整條路徑」的無限維問題，轉化為「在每個狀態決定下一步」的遞迴問題。

18.3.2 價值函數

為了把原理數學化，定義價值函數（value function）：

$$V(\mathbf{x}, t) = \min_{\mathbf{u}} \int_t^T L(\mathbf{x}, \mathbf{u}) d\tau + \varphi(\mathbf{x}(T))$$

由時刻 t 、狀態 $\mathbf{x}$ 出發，若之後都採取最佳決策，所能達到的最小總代價。

價值函數的意義非常直觀：它是「站在這個狀態，未來最少還要付出多少代價」。狀態越好（離目標近、速度合適），價值函數越小。

這個概念在第 31 章的強化學習中會再次出現，且是核心中的核心。強化學習的許多演算法（Q-learning、DQN、Actor-Critic）本質上都是在估計價值函數——只是它們用資料學習，而本章用模型計算。

18.3.3 為何價值函數如此有用

如果我們知道了價值函數，最佳控制就變得極為簡單：在每一個時刻，選擇讓「當下代價 + 下一步的價值」最小的控制輸入。這是一個局部的、有限維的最佳化——遠比原本的無限維問題容易。

$$u^*(x, t) = \operatorname{argmin}_u [L(x, u) + (\text{下一步的價值})]$$

最佳控制律。只需在當前狀態做一次局部最佳化。

這種「由價值函數導出控制律」的結構稱為**回授形式的解**——它給出的不是一條預定的軌跡，而是一個依狀態決定動作的規則。這比開迴路的軌跡強健得多：即使系統受到干擾偏離了原路徑，控制律仍然知道該怎麼做。

18.4 動態規劃與 Bellman 方程

18.4.1 離散時間的遞迴關係

在離散時間下，Bellman 原理可以寫成一個精確的遞迴關係。設系統為 $x_{k+1} = f(x_k, u_k)$ ，代價為 $\sum L(x_k, u_k)$ ，則：

$$V_k(x) = \min_u [L(x, u) + V_{k+1}(f(x, u))]$$

Bellman 方程（離散時間）。當前的價值 = 當下的代價 + 下一步的價值（取最小）。

這條式子是本章的核心。它的結構極其優雅：把一個跨越 N 步的最佳化問題，化為 N 次「只看一步」的最佳化。

18.4.2 由後往前求解

Bellman 方程的求解方向是由後往前（backward），這是初學者最不習慣的一點。

原因是：終點的價值是已知的（ $V_N(x) = \varphi(x)$ ，即終端代價）。有了 V_N ，就能用

Bellman 方程算出 V_{N-1} ；有了 V_{N-1} ，就能算出 V_{N-2} ——如此逐步往前推。

1. 初始化： $V_N(x) = \varphi(x)$ （終端代價，通常已知）。
2. 遞迴：對 $k = N-1, N-2, \dots, 0$ ，計算 $V_k(x) = \min_u [L(x,u) + V_{k+1}(f(x,u))]$ ，同時記錄使其最小的 $u^*_k(x)$ 。
3. 輸出：得到每個時刻、每個狀態下的最佳控制 $u^*_k(x)$ ——這是一個完整的控制策略。

這個「由後往前」的邏輯有一個直觀的解釋：要知道現在該怎麼走，必須先知道未來的每一步值多少。就像下棋時要往後推算——先想清楚殘局怎麼贏，才能決定現在該走哪一步。

圖 18-1 動態規劃的由後往前求解

建議配圖：一個網格狀的狀態—時間圖。橫軸為時間步 $k = 0$ 到 N ，縱軸為離散化的狀態。最右側 ($k = N$) 的每個狀態標註其終端代價值。以紅色箭頭由右往左，逐層計算各狀態的價值函數，每個節點旁標註 $V_k(x)$ 的數值與最佳動作 u^* 。左下角另附一個小圖，顯示由 x_0 出發沿最佳策略走出的路徑（綠色粗線）。標題文字：先知未來值多少，才知道現在該怎麼走。

18.4.3 維數詛咒

動態規劃在理論上完美，但有一個致命的實務問題：必須把狀態空間離散化並逐格計算。

若每個狀態維度分成 100 格，則：一維系統需要 100 格、二維需要 10^4 格、六維需要 10^{12} 格——而六軸手臂的狀態空間是 12 維的，需要 10^{24} 格。這遠超過任何電腦的能力。

這就是 Bellman 自己命名的維數詛咒 (curse of dimensionality)。它是第 26 章路徑規劃放棄格點法、改用取樣法 (PRM、RRT) 的原因，也是第 31 章強化學習用神經網路近似價值函數的原因。

那麼動態規劃還有什麼用？答案是：在特殊結構下，Bellman 方程有封閉解。而最重要的一個特例，就是「線性系統 + 二次代價」——這正是 18.6 節的 LQR。

18.5 Hamilton-Jacobi-Bellman 方程

18.5.1 由離散走向連續

18.4 節的 Bellman 方程是離散時間的。把時間步長 Δt 趨近於零，就得到連續時間的版本。

推導的思路是：在一個微小的時間 Δt 內，代價約為 $L(x,u) \cdot \Delta t$ ，而狀態變化約為 $\dot{x} \cdot \Delta t$ 。代入離散的 Bellman 方程並對價值函數做泰勒展開：

$$V(\mathbf{x}, t) = \min_{\mathbf{u}} [L(\mathbf{x}, \mathbf{u})\Delta t + V(\mathbf{x} + \mathbf{f}(\mathbf{x}, \mathbf{u})\Delta t, t + \Delta t)]$$

離散形式。展開右邊的 V 並保留一階項。

$$V(\mathbf{x}, t) \approx \min_{\mathbf{u}} [L\Delta t + V(\mathbf{x}, t) + (\partial V / \partial \mathbf{x})^T \mathbf{f}(\mathbf{x}, \mathbf{u})\Delta t + (\partial V / \partial t)\Delta t]$$

泰勒展開。兩邊的 $V(\mathbf{x}, t)$ 消去，再除以 Δt 並令 $\Delta t \rightarrow 0$ ：

$$-\partial V / \partial t = \min_{\mathbf{u}} [L(\mathbf{x}, \mathbf{u}) + (\partial V / \partial \mathbf{x})^T \mathbf{f}(\mathbf{x}, \mathbf{u})]$$

Hamilton-Jacobi-Bellman (HJB) 方程。這是連續時間最佳控制的基本方程。

18.5.2 方程的解讀

HJB 方程雖然形式緊湊，但每一項都有明確的意義。

左邊的 $-\partial V / \partial t$ 表示「價值隨時間的變化率」——隨著時間推進、剩餘時間減少，價值通常會下降。

右邊括號中的第一項 $L(\mathbf{x}, \mathbf{u})$ 是當下的代價；第二項 $(\partial V / \partial \mathbf{x})^T \mathbf{f}(\mathbf{x}, \mathbf{u})$ 是「因為狀態移動而導致的價值變化」。控制輸入 $\mathbf{u}$ 的選擇，就是在這兩者之間權衡：用力大則當下代價高，但能把狀態推向價值更低的方向。

這正是所有控制取舍的數學本質。第 17 章的增益選擇問題，在這裡得到了精確的表述。

18.5.3 為何 HJB 難解

HJB 是一個偏微分方程，且是非線性的（因為含 $\min$ 運算）。對一般的 $\mathbf{f}$ 與 L ，它沒有解析解，數值求解又面臨 18.4.3 節的維數詛咒。

因此實務上有三條路：

- **特例求解**：找出有封閉解的特殊情形——這就是 LQR（18.6 節）。
- **近似求解**：用函數近似器（如神經網路）逼近價值函數——這是第 31 章深度強化學習的路線。
- **局部求解**：不求全域的價值函數，只在當前狀態附近做有限時域的最佳化——這就是 MPC（第 17.8.4 節）。

這三條路涵蓋了現代機器人控制的主要方法。理解它們共同的起點是 HJB，就能看清這些看似不同的技術之間的血緣關係。

18.6 線性二次調節器 (LQR)

18.6.1 問題設定

LQR 是 HJB 方程有封閉解的最重要特例。它要求兩個條件：**系統是線性的、代價是二次的**。

$$\text{系統：}\dot{\mathbf{x}} = \mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}$$

$$\text{代價：} J = \int_0^\infty (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}) dt$$

LQR 問題。Q 為狀態權重矩陣 (半正定)、R 為控制權重矩陣 (正定)。

代價函數的兩項意義極為直觀：

$\mathbf{x}^T \mathbf{Q} \mathbf{x}$ ：狀態偏離的懲罰。Q 越大，代表越不能容忍誤差，控制器會更積極。若 Q 是對角矩陣，其對角元素分別代表各個狀態變數的重要程度。

$\mathbf{u}^T \mathbf{R} \mathbf{u}$ ：控制用力的懲罰。R 越大，代表越不希望用大力氣，控制器會更保守。R 必須正定 (不能為零)，否則最佳解會是無限大的控制輸入。

因此 Q/R 的比值決定了控制器的積極程度——這正是叫獸開場時說的「取決於你認為快有多重要、省力有多重要」的數學化。

18.6.2 猜測解的形式

求解 HJB 的關鍵技巧是「猜」出價值函數的形式。對 LQR 問題，一個合理的猜測是二次型：

$$V(\mathbf{x}) = \mathbf{x}^T \mathbf{P} \mathbf{x}$$

猜測價值函數為二次型。P 為對稱正定矩陣 (待求)。

這個猜測的合理性來自：系統是線性的、代價是二次的，因此積分後的總代價「應該」也是二次的。(嚴格的證明需要驗證這個解確實滿足 HJB 方程——下面的推導就是驗證過程。)

代入 HJB 方程 (無限時域下 $\partial V / \partial t = 0$)：

$$0 = \min_{\mathbf{u}} [\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + (2\mathbf{P}\mathbf{x})^T (\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u})]$$

代入 $\partial V / \partial \mathbf{x} = 2\mathbf{P}\mathbf{x}$ 。

18.6.3 求最佳控制律

先對 u 求極小。對括號內的式子關於 u 求偏導並令其為零：

$$\partial / \partial \mathbf{u} [\mathbf{u}^T \mathbf{R} \mathbf{u} + 2\mathbf{x}^T \mathbf{P} \mathbf{B} \mathbf{u}] = 2\mathbf{R} \mathbf{u} + 2\mathbf{B}^T \mathbf{P} \mathbf{x} = 0$$

一階必要條件。

$$\mathbf{u}^* = -\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}\mathbf{x} \quad \text{即} \quad \mathbf{u}^* = -\mathbf{K}\mathbf{x}, \text{ 其中 } \mathbf{K} = \mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}$$

最佳控制律。它是一個線性的狀態回授——這是 LQR 最重要的結果。

這個結果值得強調：**最佳控制律是狀態的線性函數**。它不是一條預先算好的軌跡，而是一個「看當前狀態決定用多少力」的規則。這正是 18.3.3 節提到的回授形式解，具有天然的抗干擾能力。

18.6.4 Riccati 方程

把 $\mathbf{u}^*$ 代回 HJB 方程，整理後得到 $\mathbf{P}$ 必須滿足的條件：

$$\mathbf{A}^T\mathbf{P} + \mathbf{P}\mathbf{A} - \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P} + \mathbf{Q} = 0$$

*代數 Riccati 方程 (ARE)。*這是一個矩陣二次方程，解出 $\mathbf{P}$ 後即可得增益 $\mathbf{K} = \mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}$ 。

這條方程是本章的核心成果。它的意義是：**給定系統模型 (A、B) 與設計者的偏好 (Q、R)，最佳增益就唯一確定了**——不需要試誤，直接解方程即可。

ARE 的求解在數值上是成熟的問題，所有控制軟體都內建 (MATLAB 的 `lqr`、Python 的 `scipy.linalg.solve_continuous_are`)。實務上工程師只需要選擇 $\mathbf{Q}$ 與 $\mathbf{R}$ ，剩下交給求解器。

◎ 為何叫 Riccati

這個名稱來自十八世紀義大利數學家 Jacopo Riccati，他研究的是形如 $\dot{y} = a(t) + b(t)y + c(t)y^2$ 的一階非線性微分方程。控制理論中的矩陣版本 (含二次項 $\mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}$) 與之結構相同，因此沿用其名。有趣的是，Riccati 本人研究它時純粹出於數學興趣，完全無法預見兩百年後它會成為太空船導航、飛機自動駕駛與機器人控制的核心——這是純數學意外造福工程的又一個例子。

18.6.5 有限時域與微分 Riccati 方程

若代價函數的積分只到有限時間 T (而非無限)，則價值函數會隨時間變化， $\mathbf{P}$ 也成為時間的函數。此時 Riccati 方程變成微分方程：

$$-\dot{\mathbf{P}}(t) = \mathbf{A}^T\mathbf{P} + \mathbf{P}\mathbf{A} - \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P} + \mathbf{Q}, \quad \text{終端條件 } \mathbf{P}(T) = \mathbf{Q}_f$$

*微分 Riccati 方程。*需由終點往回積分求解 (呼應 18.4.2 節的「由後往前」)。

此時增益也隨時間變化： $\mathbf{K}(t) = \mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}(t)$ 。一個有趣的現象是：**當 T 夠大時， $\mathbf{P}(t)$ 在大部分時間內會趨於一個常數值**——正是 ARE 的解。只有在接近終點的一小段時間， $\mathbf{P}$ 才會變化。

這解釋了為何實務上多用無限時域版本：對於持續運行的系統（如平衡控制、軌跡追隨），常數增益已經是最佳的，不需要每個時刻重算。

表 18-2 有限時域與無限時域 LQR 的比較

面向	有限時域	無限時域
代價積分	0 到 T	0 到 ∞
Riccati 方程	微分方程（需反向積分）	代數方程（一次解出）
增益 K	隨時間變化 K(t)	常數 K
終端代價	可指定 Q_f	無（隱含於穩定性要求）
計算量	較大（需儲存整條 K(t)）	小（一組常數）
適用場合	有明確終點的任務（如著陸）	持續運行（平衡、追隨）

18.7 權重矩陣的選擇

18.7.1 從物理意義出發

LQR 把「調增益」的問題轉化為「選 Q 與 R」的問題。這是否只是換湯不換藥？並非如此——因為 Q 與 R 有明確的物理意義，比抽象的增益容易選擇得多。

實務上最常用的作法是採用對角矩陣，並用 Bryson 法則設定初值：

$$Q_{ii} = 1 / (x_{i,max})^2, \quad R_{jj} = 1 / (u_{j,max})^2$$

Bryson 法則。 $x_{i,max}$ 是「第 i 個狀態可容許的最大偏差」、 $u_{j,max}$ 是「第 j 個控制輸入可接受的最大值」。

這個法則的邏輯是：把每一項都正規化，使各項在達到其容許上限時對代價的貢獻相當。例如容許位置誤差 0.01 m、容許力矩 20 N·m，則 $Q_{11} = 1/0.0001 = 10000$ 、 $R_{11} = 1/400 = 0.0025$ 。

Bryson 法則給出的是起點，之後再依實測結果微調。但相較於盲目試誤，它通常能在一兩次調整內得到可用的結果。

18.7.2 調整的直覺

有了初值後，調整的方向非常直觀：

表 18-3 LQR 權重調整的效果

觀察到的現象	原因	調整方向
收斂太慢	狀態懲罰不足	提高 Q (或降低 R)
力矩太大、常飽和	控制懲罰不足	提高 R (或降低 Q)
某個狀態誤差偏大	該狀態權重不足	提高該狀態對應的 Q_{ii}
振盪	速度狀態的懲罰不足	提高速度對應的 Q_{ii} (等效於加大阻尼)
對雜訊過度反應	增益過高	整體提高 R

一個實用的技巧是：只調整 Q 與 R 的比值，而非絕對值。因為 LQR 的解只取決於 Q/R 的相對大小——把兩者同時乘以 10，得到的增益完全相同。因此實務上常固定 $R = I$ ，只調整 Q 。

18.7.3 LQR 與 PD 的關係

一個有啟發性的觀察：對一維點質量系統，LQR 的解恰好就是一個 PD 控制器。

回顧 18.2.1 節，一維點質量的狀態為 $x = [\text{位置}, \text{速度}]^T$ 。LQR 的控制律 $u = -Kx$ 展開後為：

$$u = -k_1 \cdot (\text{位置}) - k_2 \cdot (\text{速度})$$

這正是 PD 控制的形式： k_1 對應 K_p 、 k_2 對應 K_d 。

因此 LQR 並沒有發明新的控制器結構，而是告訴我們該用多大的 K_p 與 K_d 。第 17 章留下的問題，在此得到了回答。

更重要的是，LQR 的優勢在多變數系統中才完全展現。對六軸手臂（12 維狀態、6 個輸入）， K 是一個 6×12 的矩陣，含 72 個增益——手動調參幾乎不可能，但 LQR 能一次算出全部，且保證是該代價函數下的最佳解。

18.8 LQR 的性質與限制

18.8.1 三個優良性質

性質一：保證穩定。只要系統可控（第 19 章會嚴格定義）且 Q 、 R 選得合理，LQR 閉環系統必定穩定。這是一個強而有力的保證——不需要另外驗證。

性質二：良好的裕度。LQR 閉環系統有一個著名的結果：增益裕度至少為 $[0.5, \infty)$ 、相位裕度至少 60° 。這代表即使實際的系統增益是模型的一半或無限倍，系統仍然穩定——這是相當寬裕的容錯能力。

性質三：解的唯一性。ARE 的正定解是唯一的，因此最佳增益也是唯一的。不會有「不同的初值收斂到不同解」的問題。

18.8.2 四個實務限制

限制一：需要線性模型。機器人本質上是非線性的。因應之道有二：在工作點附近做**近似線性化**（第 19 章），或先用第 17 章的計算力矩控制把系統線性化為 $\ddot{\mathbf{q}} = \mathbf{u}$ ，再套用 LQR。後者是機器人上的標準作法。

限制二：需要完整的狀態量測。LQR 的控制律 $\mathbf{u} = -\mathbf{K}\mathbf{x}$ 需要知道所有狀態。若某些狀態無法直接量測（如速度雜訊太大），必須加入**狀態觀測器**（如 Kalman 濾波器，第 24 章）。LQR 加上 Kalman 濾波器的組合稱為 **LQG**（Linear Quadratic Gaussian），是經典控制理論的巔峰成果之一。

限制三：無法處理約束。LQR 的推導假設 $\mathbf{u}$ 可以取任意值。實際的力矩極限、關節限位都無法納入。若最佳解超出極限而被截斷，性能保證就失效——這正是 MPC 存在的理由（第 17.8.4 節）。

限制四：二次代價未必反映真實目標。有些目標本質上不是二次的，例如「時間最短」（第 15.8.3 節）或「絕不能超過某個誤差」。強行用二次代價近似，可能得到不理想的結果。

表 18-4 LQR 與其他控制方法的比較

方法	增益來源	處理非線性	處理約束	計算量	適用場合
手動調 PID	試誤	否	否	極低	單輸入單輸出、低要求
計算力矩控制	指定 ω_n 、 ζ	是（抵消）	否	中	完全驅動的機械手臂
LQR	解 ARE	需先線性化	否	低（離線算好）	多變數、欠驅動、平衡控制
MPC	線上最佳化	是	是	高	有硬約束、需前瞻
強化學習	資料學習	是	可（軟性）	訓練極高、推論低	難以建模的複雜任務

◎ 為何 LQR 在機器人上如此重要

LQR 特別適合三類機器人問題。一、**欠驅動系統的平衡**：倒立擺、兩輪自平衡車、四旋翼的姿態控

制——這些無法用第 17 章的回授線性化處理，LQR 是最標準的解。二、**多變數耦合系統**：當狀態與輸入都是多維時，手動調參不可行，LQR 能一次算出所有增益。三、**作為更複雜方法的基礎**：MPC 的終端代價常用 LQR 的價值函數、iLQR (迭代 LQR) 是軌跡最佳化的主流方法之一、而強化學習的許多演算法在線性情形下會退化為 LQR。理解 LQR，等於掌握了一把通往現代控制的鑰匙。

18.9 倒立擺：欠驅動系統的經典範例

18.9.1 為何倒立擺是控制理論的試金石

倒立擺 (cart-pole) 是控制領域最經典的測試問題：一台可左右移動的小車，上面用鉸鏈立著一根桿子。控制輸入只有小車的水平推力，目標是讓桿子維持直立。

它之所以成為標竿，有三個原因。**原因一：本質不穩定**。放手就會倒，必須持續控制——這排除了所有開迴路方法。**原因二：欠驅動**。系統有兩個自由度 (小車位置、桿子角度)，卻只有一個控制輸入。第 17 章的回授線性化在此完全失效。**原因三：非最小相位**。要讓桿子往右倒回來，小車必須**先往右移** (把支點移到重心下方)——控制的初始反應方向與最終目標相反，這在直覺上極為違反常識。

這三個特性，恰好也是雙足機器人、兩輪自平衡車與火箭垂直著陸的共同特性。因此掌握倒立擺，就掌握了這一大類問題的核心。

18.9.2 線性化模型

完整的倒立擺動力學是非線性的，但在直立點 ($\theta \approx 0$) 附近可以線性化。設小車質量 M 、桿質量 m 、桿長 $2l$ (質心在中點 l 處)，狀態向量取：

$$\mathbf{x} = [s, \dot{s}, \theta, \dot{\theta}]^T$$

狀態：小車位置、小車速度、桿角度 (由垂直向上量起)、桿角速度。

在 $\theta \approx 0$ 、 $\dot{\theta} \approx 0$ 的假設下 ($\sin \theta \approx \theta$ 、 $\cos \theta \approx 1$ 、 $\theta^2 \approx 0$)，線性化後的系統矩陣為：

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -mg/M & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & (M+m)g/(Ml) & 0 \end{bmatrix}$$

系統矩陣。第四列的正號是關鍵——它代表「角度越大，角加速度越大」，即不穩定。

$$\mathbf{B} = [0, 1/M, 0, -1/(Ml)]^T$$

輸入矩陣。注意推力對桿角的影響是負的——往右推小車，桿子會相對往左倒，這正是非最小相位的來源。

18.9.3 為何 LQR 特別適合

面對這個系統，LQR 有三個優勢。

優勢一：自然處理欠驅動。LQR 不要求輸入數等於自由度，它只是求解「用這一個輸入，如何最好地影響四個狀態」。

優勢二：自動協調多目標。我們同時希望桿子直立且小車不要跑太遠。這兩個目標會衝突（要扶正桿子就得移動小車），LQR 用 Q 矩陣的相對權重自動權衡。

優勢三：一次算出四個增益。控制律 $u = -Kx$ 中， K 是 1×4 的矩陣，含四個增益。手動調這四個數字極其困難（它們互相耦合），LQR 直接算出。

例題 18-3 與本章程式範例將完整實作這個系統。

18.10 與其他主題的關聯

18.10.1 與逆運動學的關係

第 10.8 節把逆運動學寫成一個最佳化問題：最小化任務誤差加上正則項。這與 LQR 的代價函數結構完全相同——都是「目標誤差的二次型 + 控制量的二次型」。

差別在於：逆運動學是靜態的（一次求解一個位形），LQR 是動態的（考慮整條時間軌跡）。但兩者的數學工具（二次型最佳化、偽逆與正則化）是相通的。第 10.7.3 節的阻尼最小平方方法，其實就是一個「單步的 LQR」。

18.10.2 與軌跡最佳化的關係

第 15 章的軌跡生成用的是預設的曲線形式（多項式、梯形、S 形）。更一般的作法是軌跡最佳化：直接把整條軌跡當作最佳化變數，最小化某個代價函數（時間、能耗、平順度）並滿足動力學與約束。

這正是一個最佳控制問題。求解方法有兩大類：間接法（由 HJB 或 Pontryagin 最大原理導出必要條件再求解）與直接法（把時間離散化，化為一個大型非線性規劃問題）。實務上直接法更常用，因為容易加入約束。

其中一個重要的方法是 iLQR（迭代 LQR）：對非線性系統，在當前軌跡附近線性化、用

LQR 求解、更新軌跡、再重複。它結合了 LQR 的高效與對非線性的處理能力，是目前四足機器人與操作規劃的主流工具之一。

18.10.3 與強化學習的關係

這是本章最重要的連結。第 31 章的強化學習，其數學基礎正是本章的 Bellman 方程。

兩者的對應關係如下：

表 18-5 最佳控制與強化學習的術語對照

最佳控制	強化學習	說明
狀態 x	狀態 s	相同概念
控制輸入 u	動作 a	相同概念
代價 L (要最小化)	獎勵 r (要最大化)	符號相反，本質相同
價值函數 $V(x)$	狀態價值 $V(s)$	相同概念
系統動態 $f(x,u)$ (已知)	環境轉移 (未知，需探索)	最關鍵的差異
Bellman 方程	Bellman 方程	完全相同
動態規劃	動態規劃 / 時序差分學習	後者用取樣估計代替完整計算

核心差異只有一個：模型是否已知。最佳控制假設我們知道 $f(x,u)$ ，因此可以直接計算；強化學習則假設環境未知，必須透過與環境互動來學習。

這個差異帶來完全不同的方法論，但**目標與數學結構完全相同**。理解本章，第 31 章會輕鬆許多——屆時你會發現許多「新概念」其實是舊識。

18.11 由最佳性邁向穩定性

18.11.1 本章的成果

本章回答了第 17 章留下的問題：增益不是憑感覺調的，而是可以由代價函數計算出來的。你現在能夠：把控制問題形式化為最佳控制問題、理解 Bellman 最佳性原理與價值函數、寫出並解讀 HJB 方程、推導並求解 LQR 與 Riccati 方程、用 Bryson 法則選擇權重、以及把 LQR 應用於欠驅動系統如倒立擺。

更重要的是，你看見了一個統一的框架。**動態規劃、LQR、MPC、iLQR、強化學習——這些看似不同的技術，全部源自同一條 Bellman 方程。**差別只在於：模型是否已知、時域是有限還

是無限、以及用什麼方法近似求解。

18.11.2 接下來：從「最好」到「保證」

但本章有一個微妙的缺口。LQR 保證了「在這個代價函數下最好」，也宣稱閉環系統會穩定——但我們還沒有嚴格證明過任何一個控制器的穩定性。

第 16 章分析二階系統時，我們看了微分方程的解；第 17 章推導計算力矩控制時，我們得到了線性的誤差方程。但這些都是針對特定的、簡化的情形。對於一般的非線性系統，如何證明「這個控制器一定不會失控」？

第 19 章（可控性與穩定性）將提供工具。我們會學到兩個核心概念：**可控性**（系統是否有可能被控制到任意狀態——這是 LQR 能運作的前提）與 **Lyapunov 穩定性**（一個不需解出微分方程就能判定穩定性的強大方法）。屆時我們將回頭嚴格證明第 17.5.2 節那個「PD 加重力補償保證全域漸近穩定」的定理，並看見第 13 章那個「 $\dot{M} - 2C$ 為斜對稱」的性質如何在證明中扮演關鍵角色。

換句話說：第 18 章告訴你什麼是最好的，第 19 章告訴你為什麼它一定安全。

完整計算例題

例題 18-1 一維點質量的 LQR 設計

題目：質量 $m = 2 \text{ kg}$ 的物體在無摩擦平面上，控制輸入為力 f 。要求設計 LQR 控制器，使物體回到原點。（a）寫出狀態空間模型。（b）取 $Q = \text{diag}(1, 0)$ 、 $R = 1$ ，求解 ARE 並得到最佳增益。（c）計算閉環系統的自然頻率與阻尼比。（d）若改為 $Q = \text{diag}(100, 0)$ ，重新計算並比較。

解（a）：取狀態 $x = [\text{位置}, \text{速度}]^T$ 。由 18.2.1 節：

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1/m \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$$

狀態空間模型。

解（b）：設 $P = [[p_1, p_2], [p_2, p_3]]$ （對稱）。代入 ARE： $A^T P + PA - PBR^{-1}B^T P + Q = 0$ 。

先計算各項。 $A^T P + PA$ ：

$$A^T P = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} p_1 & p_2 \\ p_2 & p_3 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ p_1 & p_2 \end{bmatrix}, \quad P A = \begin{bmatrix} 0 & p_1 \\ 0 & p_2 \end{bmatrix}$$

兩項相加得 $[[0, p_1], [p_1, 2p_2]]$ 。

再計算 $PBR^{-1}B^T P \cdot B^T P = [0, 0.5] \cdot P = [0.5p_2, 0.5p_3]$ ：

$$PBR^{-1}B^T P = \begin{bmatrix} 0.5p_2 \\ 0.5p_3 \end{bmatrix} \begin{bmatrix} 0.5p_2 & 0.5p_3 \end{bmatrix} = \begin{bmatrix} 0.25p_2^2 & 0.25p_2p_3 \\ 0.25p_2p_3 & 0.25p_3^2 \end{bmatrix}$$

二次項。

代入 ARE 並逐項比對 ($Q = \text{diag}(1, 0)$)：

$$(1,1) \text{ 項: } 0 - 0.25p_2^2 + 1 = 0 \Rightarrow p_2^2 = 4 \Rightarrow p_2 = 2$$

$$(1,2) \text{ 項: } p_1 - 0.25p_2p_3 = 0 \Rightarrow p_1 = 0.5p_3$$

$$(2,2) \text{ 項: } 2p_2 - 0.25p_3^2 + 0 = 0 \Rightarrow p_3^2 = 8p_2 = 16 \Rightarrow p_3 = 4$$

解得 $p_2 = 2$ 、 $p_3 = 4$ 、 $p_1 = 2$ 。(取正定解，故各值取正。)

$$K = R^{-1}B^T P = [0.5p_2, 0.5p_3] = [1.0, 2.0]$$

最佳增益。控制律為 $f = -1.0 \cdot (\text{位置}) - 2.0 \cdot (\text{速度})$ 。

解 (c)：閉環系統為 $\dot{x} = (A - BK)x$ 。代入：

$$A - BK = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -0.5 & -1.0 \end{bmatrix}$$

閉環系統矩陣。

其特徵方程為 $\lambda^2 + 1.0\lambda + 0.5 = 0$ 。對照第 16 章的標準形式 $\lambda^2 + 2\zeta\omega_n\lambda + \omega_n^2$ ：

$$\omega_n = \sqrt{0.5} = 0.707 \text{ rad/s}, \quad \zeta = 1.0/(2 \times 0.707) = 0.707$$

自然頻率與阻尼比。

有趣的觀察：LQR 自動選出了 $\zeta = 0.707$ ——正是第 16.4.4 節提到的實務最佳值！這不是巧合，而是二次代價函數的自然結果。這個例子生動說明了 LQR 的價值：它把工程師的經驗法則，變成了數學的必然結論。

解 (d)： $Q = \text{diag}(100, 0)$ 代表更重視位置精度。重複計算：

$$p_2^2 = 4 \times 100 = 400 \Rightarrow p_2 = 20 \\ p_3^2 = 8p_2 = 80 \Rightarrow p_3 = 8.944$$

新的 P 元素。

$$K = [0.5 \times 10, 0.5 \times 8.944] = [5.0, 4.472]$$

新的增益。

$$\omega_n = \sqrt{(0.5 \times 5.0)} = \sqrt{2.5} = 1.58 \text{ rad/s}, \quad \zeta = (0.5 \times 4.472)/(2 \times 1.58) = 0.707$$

新的動態參數。

關鍵觀察：Q 增大 100 倍後， ω_n 增為 2.24 倍（ $\sqrt{100}$ 的四次方根，即 $100^{0.25}$ ），但阻尼比 ζ 仍然是 0.707。這揭示了 LQR 的一個優美性質：改變 Q/R 的比值會調整系統的速度，但阻尼比保持在最佳值——控制器自動維持了「不振盪」的品質。

例題 18-2 Bryson 法則的應用

題目：某機器人關節（等效慣量 $J = 0.5 \text{ kg}\cdot\text{m}^2$ ）採用 LQR 控制。設計要求：位置誤差不超過 0.02 rad、速度誤差不超過 0.5 rad/s、力矩不超過 10 N·m。（a）用 Bryson 法則設定 Q 與 R。（b）說明若實測發現力矩經常飽和，該如何調整。（c）若改為要求位置誤差不超過 0.005 rad，Q 如何改變？增益大約變為幾倍？

解（a）：由 18.7.1 節的 Bryson 法則：

$$Q_{11} = 1/(0.02)^2 = 1/0.0004 = 2500 \quad (\text{位置})$$

$$Q_{22} = 1/(0.5)^2 = 1/0.25 = 4 \quad (\text{速度})$$

$$R = 1/(10)^2 = 1/100 = 0.01 \quad (\text{力矩})$$

初始權重設定。

因此 $Q = \text{diag}(2500, 4)$ 、 $R = 0.01$ 。注意 Q_{11} 遠大於 Q_{22} ——這反映了「位置精度要求嚴格得多」這個設計意圖。

解（b）：力矩飽和代表控制器太積極，應提高 R（或等效地降低 Q）。實務作法是把 R 逐步加倍（ $0.01 \rightarrow 0.02 \rightarrow 0.04$ ），每次重新解 ARE 並測試，直到力矩落在極限內。

由於 LQR 的解只取決於 Q/R 的比值，也可以固定 $R = 1$ 而縮小 Q——兩種作法等價，選擇習慣的即可。

解（c）：位置容許誤差由 0.02 縮為 0.005（四分之一），則：

$$Q_{11} = 1/(0.005)^2 = 40000 \quad (\text{原本的 16 倍})$$

新的位置權重。

由例題 18-1 的規律，增益大約與 $Q^{0.5}$ 的四次方根相關。更精確地說，對此類系統：

$$K_p \propto \sqrt{(Q_{11}/R)} \Rightarrow \text{增益約變為 } \sqrt{16} = 4 \text{ 倍}$$

增益的變化。位置增益約增為 4 倍。

工程判讀：要求精度提高 4 倍，需要增益提高約 4 倍，而力矩也會相應增大。若原本的力矩餘裕不足，這個要求可能無法達成——此時應考慮改善機構（降低摩擦、提高剛性）而非一味提高增益。

例題 18-3 倒立擺的 LQR 設計

題目：倒立擺參數為：小車質量 $M = 1.0 \text{ kg}$ 、桿質量 $m = 0.2 \text{ kg}$ 、桿長 $2l = 0.6 \text{ m}$ （故 $l = 0.3 \text{ m}$ ）、 $g = 9.8 \text{ m/s}^2$ 。（a）寫出線性化的 A、B 矩陣。（b）驗證開迴路系統不穩定。（c）說明 Q 與 R 的選擇考量。（d）解釋所得增益的物理意義。

解（a）：代入 18.9.2 節的公式：

$$\begin{aligned} -mg/M &= -(0.2)(9.8)/1.0 = -1.96 \\ (M+m)g/(Ml) &= (1.2)(9.8)/((1.0)(0.3)) = 11.76/0.3 = 39.2 \end{aligned}$$

兩個關鍵元素。

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -1.96 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 39.2 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 0 \\ 1.0 \\ 0 \\ -3.333 \end{bmatrix}$$

線性化模型。B 中的 $-1/(Ml) = -1/0.3 = -3.333$ 。

解（b）：求 A 的特徵值。由結構可知有兩個零特徵值（對應小車位置與速度的積分），另兩個滿足：

$$\lambda^2 = 39.2 \Rightarrow \lambda = \pm 6.26$$

其中 $\lambda = +6.26$ 為正實部特徵值 $\Rightarrow$ 系統不穩定。

這個正特徵值的物理意義是：任何微小的角度偏差，都會以 $e^{(6.26t)}$ 指數放大。時間常數為 $1/6.26 = 0.16$ 秒——大約每 0.16 秒偏差就放大 2.7 倍。這解釋了為何倒立擺必須以高頻率控制（至少 50 Hz 以上）。

解（c）：四個狀態的權重考量如下：

- **桿角度 θ** ：最重要——倒了就完全失敗。應給最大權重。
- **桿角速度 $\dot{\theta}$** ：次重要——它預示了未來的角度變化，適度懲罰有助於阻尼。
- **小車位置 s** ：中等——不希望小車跑太遠（軌道有限），但可以容忍一定範圍。
- **小車速度 $\dot{s}$** ：最不重要——只要最終停下即可，過程中的速度不需嚴格限制。

典型的選擇是 $Q = \text{diag}(1, 1, 100, 10)$ 、 $R = 1$ （角度權重遠大於位置）。

解 (d)：解 ARE 後（見本章程範例）會得到形如 $K = [k_1, k_2, k_3, k_4]$ 的增益，控制律為：

$$u = -k_1 \cdot s - k_2 \cdot \dot{s} - k_3 \cdot \theta - k_4 \cdot \dot{\theta}$$

控制律的四項。

物理解讀： k_3 與 k_4 是「扶正桿子」的主力——桿子往右倒（ $\theta > 0$ ），控制器輸出負的力（往左推小車），使支點移到重心下方，桿子就會被「接住」。 k_1 與 k_2 則負責「把小車拉回原點」，但它們必須很小心——因為要把小車往左移，反而必須先讓桿子往左傾，這正是非最小相位的體現。

實際計算會發現 k_3 的符號與直覺相反（往右倒卻要往右推？）——這是因為 B 矩陣中角度那一項是負的。這個符號的糾結，正是倒立擺讓初學者困惑的地方，也是為何手動調參極其困難、而 LQR 能直接算出正確答案的價值所在。

程式範例：LQR 設計與倒立擺模擬

以下 Python 程式實作 ARE 的求解、LQR 增益計算，並完整模擬倒立擺的平衡控制。同時驗證例題 18-1 的手算結果與 Q 權重的影響。

```
import numpy as np
from scipy.linalg import solve_continuous_are

def lqr(A, B, Q, R):
    """求解連續時間 LQR：回傳增益 K、Riccati 解 P、閉環特徵值"""
    P = solve_continuous_are(A, B, Q, R)
    K = np.linalg.solve(R, B.T @ P)          # K = R^-1 B^T P
    eig = np.linalg.eigvals(A - B @ K)
    return K, P, eig

# ===== 驗證一：例題 18-1 一維點質量 =====
print("=== 例題 18-1：一維點質量 LQR (m = 2 kg) ===")
m = 2.0
A = np.array([[0., 1.], [0., 0.]])
B = np.array([[0.], [1/m]])

for q11 in (1.0, 100.0):
    Q = np.diag([q11, 0.0]); R = np.array([[1.0]])
    K, P, eig = lqr(A, B, Q, R)
    wn = np.sqrt(K[0,0]/m)                # 閉環 wn
    zeta = (K[0,1]/m)/(2*wn)              # 閉環 ζ
    print(f"Q11={q11:6.0f}: K = [{K[0,0]:.4f}, {K[0,1]:.4f}] "
```

```

    f"wn = {wn:.4f} rad/s    ζ = {zeta:.4f}")
    print(f"          P = {np.round(P.flatten(), 4)}    ")
    f"閉環極點 = {np.round(eig, 4)}")
print("→ 注意：Q 改變 100 倍 · ζ 始終維持 0.707 (最佳阻尼)")

# ===== 驗證二：Q/R 比值的效果 =====
print("\n=== Q/R 比值對性能的影響 ===")
print(f"'Q11/R':>8}'{Kp':>10}'{Kd':>10}'{wn':>10}'{ζ':>8}'{ '最大力矩':>12}")
for ratio in (0.1, 1, 10, 100, 1000):
    Q = np.diag([ratio, 0.0]); R = np.array([[1.0]])
    K, _, _ = lqr(A, B, Q, R)
    wn = np.sqrt(K[0,0]/m); zeta = (K[0,1]/m)/(2*wn)
    f_max = K[0,0]*1.0 # 初始偏差 1 m 時的力
    print(f"{ratio:8.1f}{K[0,0]:10.3f}{K[0,1]:10.3f}"
          f"{wn:10.3f}{zeta:8.3f}{f_max:11.2f} N")
print("→ 提高 Q 使反應變快 · 但所需力矩也同比例增加")

# ===== 例題 18-3：倒立擺 =====
print("\n=== 例題 18-3：倒立擺 LQR 設計 ===")
M_cart, m_pole, l, g = 1.0, 0.2, 0.3, 9.8

A_ip = np.array([
    [0., 1.,          0., 0.],
    [0., 0., -m_pole*g/M_cart, 0.],
    [0., 0.,          0., 1.],
    [0., 0., (M_cart+m_pole)*g/(M_cart*l), 0.]])
B_ip = np.array([[0.], [1/M_cart], [0.], [-1/(M_cart*l)]])

print(f"開迴路特徵值: {np.round(np.linalg.eigvals(A_ip), 4)}")
unstable = [e for e in np.linalg.eigvals(A_ip) if e.real > 1e-9]
print(f"→ 有 {len(unstable)} 個正實部特徵值 · 系統不穩定")
print(f"→ 發散時間常數 = {1/max(e.real for e in unstable):.3f} 秒")

# Q 的選擇：角度最重要 (18.9.3 節)
Q_ip = np.diag([1.0, 1.0, 100.0, 10.0])
R_ip = np.array([[1.0]])
K_ip, P_ip, eig_ip = lqr(A_ip, B_ip, Q_ip, R_ip)
print(f"\nQ = diag(1, 1, 100, 10), R = 1")
print(f"最佳增益 K = [{K_ip[0,0]:8.3f} {K_ip[0,1]:8.3f} "
      f"{K_ip[0,2]:8.3f} {K_ip[0,3]:8.3f}]"
      f"          ( 位置    速度    角度    角速度 ) ")
print(f"閉環極點 = {np.round(eig_ip, 3)}")
print(f"→ 全部落在左半平面 · 系統已穩定")

# ===== 非線性模擬 (驗證線性化控制器的有效性) =====
def cartpole_dynamics(x, u, M=1.0, mp=0.2, l=0.3, g=9.8):
    """完整的非線性倒立擺動力學"""
    s, sd, th, thd = x
    sin_t, cos_t = np.sin(th), np.cos(th)

```

```

den = M + mp*sin_t**2
sdd = (u + mp*sin_t*(1*thd**2 - g*cos_t)) / den
thdd = (-u*cos_t - mp*1*thd**2*sin_t*cos_t + (M+mp)*g*sin_t) / (1*den)
return np.array([sd, sdd, thd, thdd])

def simulate_cartpole(K, x0, T=5.0, dt=0.001, u_max=20.0):
    x = np.array(x0, float)
    max_angle, max_u, max_s = 0.0, 0.0, 0.0
    for k in range(int(T/dt)):
        u = float(-K @ x)
        u = np.clip(u, -u_max, u_max)
        xd = cartpole_dynamics(x, u)
        x = x + xd*dt
        max_angle = max(max_angle, abs(np.rad2deg(x[2])))
        max_u = max(max_u, abs(u)); max_s = max(max_s, abs(x[0]))
        if abs(x[2]) > np.pi/2:
            return None, max_angle, max_u, max_s      # 倒了
    return x, max_angle, max_u, max_s

print("\n=== 非線性模擬：由不同初始傾角出發 ===")
print(f"{'初始角度':>10}{'結果':>8}{'最大角度':>12}{'最大力':>10}{'小車位移':>12}")
for deg in (5, 10, 20, 30, 45):
    xf, ma, mu, ms = simulate_cartpole(K_ip, [0., 0., np.deg2rad(deg), 0.])
    status = "穩定" if xf is not None else "失敗"
    print(f"{deg:9d}°{status:>8}{ma:11.2f}°{mu:9.2f} N{ms:11.3f} m")
print("> 線性化設計的控制器，在相當大的角度範圍內仍然有效")

# ===== Q 權重對倒立擺行為的影響 =====
print("\n=== Q 權重的影響 ( 初始傾角 15° ) ===")
print(f"{'Q(角度)':>10}{'Q(位置)':>10}{'最大角度':>12}{'最大力':>10}{'小車位移':>12}")
for q_th, q_s in ((10., 1.), (100., 1.), (1000., 1.), (100., 50.)):
    Q_t = np.diag([q_s, 1.0, q_th, 10.0])
    K_t, _, _ = lqr(A_ip, B_ip, Q_t, R_ip)
    xf, ma, mu, ms = simulate_cartpole(K_t, [0., 0., np.deg2rad(15), 0.])
    print(f"{q_th:10.0f}{q_s:10.0f}{ma:11.2f}°{mu:9.2f} N{ms:11.3f} m")
print("> 角度權重高則桿子更穩但小車跑更遠；位置權重高則小車收斂快但角度波動大")

```

請試著修改：（一）把倒立擺的 R 由 1 提高到 100，觀察增益如何下降、最大力如何減小、但恢復是否變慢。（二）把初始角度加大到 60 度，觀察線性化控制器何時失效——這說明了線性化的有效範圍。（三）在 `cartpole_dynamics` 中加入摩擦項，觀察 LQR 是否仍然有效（它應該相當強健）。

實作：LQR 控制器的設計與驗證

本章實作以模擬與設計為主，並可延伸到實體的兩輪自平衡車或倒立擺平台。

1. **建立狀態空間模型**：為你的系統（可選一維點質量、單關節手臂、或倒立擺）寫出 A 、 B 矩

陣。若是非線性系統，先在工作點附近線性化。

2. **開迴路分析**：計算 A 的特徵值，判斷系統是否穩定。若有正實部特徵值，計算其發散時間常數，據此決定所需的控制頻率（建議至少為時間常數倒數的 10 倍）。
3. **Bryson 法則設定初值**：依 18.7.1 節，由各狀態的容許誤差與控制的可用範圍，設定 Q 與 R 的初值。
4. **求解 ARE**：用 scipy 或 MATLAB 求得 P 與 K ，並檢查閉環特徵值是否全部落在左半平面。
5. **線性模擬**：以線性模型模擬閉環響應，記錄收斂時間、超越量與最大控制量。
6. **非線性模擬**：改用完整的非線性模型重複模擬，比較與線性模擬的差異。逐步加大初始偏差，找出線性化控制器失效的臨界點。
7. **權重掃描**：系統性地改變 Q 與 R （建議固定 $R = 1$ ，只掃描 Q ），記錄「收斂時間 vs 最大控制量」的關係，繪出取舍曲線（Pareto front）。這張圖能直觀展示最佳控制的本质——沒有免費的午餐。
8. **強健性測試**：故意讓控制器使用的模型參數與真實系統相差 $\pm 20\%$ （如質量估計錯誤），觀察 LQR 是否仍能穩定。驗證 18.8.1 節提到的增益裕度性質。
9. **（進階）實體驗證**：若有兩輪自平衡車或倒立擺平台，將設計好的 K 燒錄進去測試。注意實體系統有感測雜訊、量化誤差與致動器延遲，通常需要把增益降低 20~30% 才能穩定運行。

表 18-6 LQR 設計記錄表（第一列為範例）

Q 設定	R 設定	增益 K	閉環極點	收斂時間	最大控制量
diag(1, 0)	1	[1.00, 2.00]	$-0.5 \pm 0.5j$	6.0 s	1.0 N
（自行填寫其餘設計）					

國中小也看得懂：在手掌上立掃把的祕密

你有沒有玩過「在手掌上立掃把」的遊戲？（第 16 章我們玩過立竹竿，這次是更難的版本。）

要立住掃把，你的手必須一直移動。但你有沒有想過一件很奇怪的事——當掃把往右倒的時候，你的手要往哪邊移？

答案是：**往右**！這聽起來很怪對不對？掃把往右倒，你卻要往右追。但仔細想想就懂了：你要把「支撐點」移到掃把重心的下方，它才會被「接住」。如果你往左移，掃把只會倒得更快。

這件事有趣的地方在於：**如果你想讓掃把往左移動一點點，你必須先讓它往左倒一下下，然後再追過去。** 也就是說——**想往左，要先往左倒。**這種「動作方向和目的地相反」的系統，是工程師最頭痛的一類。

那要怎麼算出「該移動多快」呢？這一章教的方法很聰明。工程師會先問自己兩個問題：

問題一：我最不能忍受什麼？ 是掃把倒下去？還是手移動太遠？

問題二：我願意付出多少力氣？ 用力大一點就穩，但會很累。

把這兩個答案寫成一個「分數」——歪太多要扣分、太用力也要扣分。然後用數學算出「怎麼做才能扣最少分」。算出來的答案，就是最好的控制方法！

最酷的是：這個方法不用試誤。以前的工程師要花好幾天慢慢調參數，現在只要說清楚「我在乎什麼」，電腦就能直接算出最佳答案。

叫獸提醒與常見錯誤

- 誤區一：以為 LQR 能取代模型。它需要準確的 A 、 B 矩陣；模型錯了，算出的「最佳」增益也是錯的。
- 誤區二： R 設為零。 R 必須正定，否則最佳解是無限大的控制輸入，ARE 無解。
- 誤區三：只調 Q 與 R 的絕對值。LQR 的解只取決於 Q/R 的比值，兩者同乘一個常數得到完全相同的增益。
- 誤區四：忘記 LQR 需要完整的狀態量測。若某些狀態無法量測，必須加入觀測器（如 Kalman 濾波器）構成 LQG。
- 誤區五：忽略約束。LQR 不處理力矩極限；若最佳解超限而被截斷，穩定性保證即失效，應改用 MPC 或降低增益。
- 誤區六：在遠離工作點處使用線性化的 LQR。線性化只在小範圍內有效，大偏差時可能失控——本章程式範例的角度掃描即在驗證此點。
- 誤區七：用直覺調倒立擺的四個增益。它們高度耦合且部分符號違反直覺（非最小相位），手動調參幾乎不可能成功。

- 誤區八：把有限時域的 $K(t)$ 用在持續運行的系統上。持續運行應使用無限時域的常數增益，計算簡單且已是最佳。
- 誤區九：以為動態規劃能直接用在高維系統。維數詛咒使其在超過三、四維時就不可行，必須改用 LQR、MPC 或函數近似。
- 誤區十：把實體系統的增益直接用模擬值。實機有感測雜訊、量化誤差與致動器延遲，通常需要降低 20~30% 才能穩定。
- 誤區十一：忽略 LQR 與強化學習的血緣關係。兩者共用 Bellman 方程，差別只在模型是否已知——理解前者會讓後者容易得多。

課堂討論

1. 例題 18-1 發現 LQR 自動選出 $\zeta = 0.707$ ，恰好是第 16 章的實務最佳值。請討論：這是巧合嗎？工程師的經驗法則與數學最佳解之間，通常是什麼關係？
2. 維數詛咒使動態規劃無法直接用於高維系統。請比較本書中三種因應策略——LQR（特例封閉解）、MPC（局部有限時域）、強化學習（函數近似）——各自「犧牲了什麼、換到了什麼」。
3. 倒立擺是非最小相位系統：想往左，要先往左倒。請再舉出兩個生活或工程中的非最小相位例子，並討論它們為何難以直覺控制。
4. LQR 保證增益裕度至少 $[0.5, \infty)$ 、相位裕度至少 60° 。請討論這個保證在實務上的價值，以及它為何在加入觀測器（LQG）後可能失效。
5. 本章的代價函數 $J = \int (x^T Q x + u^T R u) dt$ 把所有目標都化為二次型。請討論：哪些機器人任務的目標**不適合**用二次型表達？此時該怎麼辦？

章末摘要

- 最佳控制問題由三要素構成：系統動態 $\dot{x} = f(x, u)$ 、代價函數 J 、以及邊界條件與約束。
- 它與一般最佳化的本質差異：在函數空間中找一整條曲線（無限維），而非在有限維空間中找一個點。
- 狀態空間表示把高階系統化為一階形式 $\dot{x} = Ax + Bu$ ，是控制理論的統一語言。
- Bellman 最佳性原理：最佳路徑的任何一段本身也是最佳的；剩餘決策只取決於當前狀態。

與歷史無關。

- 價值函數 $V(x)$ 是「由此狀態出發，未來最少還要付出的代價」；知道它就能導出回授形式的最佳控制律。
- 離散時間的 Bellman 方程 $V_k(x) = \min_u [L + V_{k+1}(f)]$ 由後往前遞迴求解，稱為動態規劃。
- 動態規劃面臨維數詛咒：狀態每增一維，計算量指數成長，超過三四維即不可行。
- 連續時間版本為 HJB 方程 $-\partial V/\partial t = \min_u [L + (\partial V/\partial x)^T f]$ ，一般無解析解。
- LQR 是 HJB 有封閉解的最重要特例：線性系統加二次代價，價值函數為 $V = x^T P x$ 。
- 最佳控制律為線性狀態回授 $u = -Kx$ ，其中 $K = R^{-1}B^T P$ ， P 由代數 Riccati 方程 $A^T P + PA - PBR^{-1}B^T P + Q = 0$ 解出。
- Q 懲罰狀態偏差、 R 懲罰控制用力；解只取決於 Q/R 的比值；Bryson 法則以各量的容許上限倒數平方設定初值。
- LQR 的三個優良性質：保證穩定、增益裕度至少 $[0.5, \infty)$ 與相位裕度至少 60° 、解唯一。
- 四個限制：需線性模型、需完整狀態量測、無法處理約束、二次代價未必反映真實目標。
- 對一維點質量，LQR 的解就是一個 PD 控制器；其價值在於為多變數系統一次算出所有增益。
- 倒立擺是欠驅動、不穩定且非最小相位的系統，回授線性化失效，LQR 是標準解法。
- 最佳控制與強化學習共用 Bellman 方程，核心差異僅在於系統模型是否已知。

觀念題與計算題

觀念題

1. 說明最佳控制問題的三個要素，並解釋它與一般最佳化問題的本質差異。
2. 為何要把高階系統化為狀態空間的一階形式？這帶來哪三個好處？
3. 敘述 Bellman 最佳性原理，並以一個生活化的例子說明其正確性。
4. 定義價值函數，並說明為何「知道價值函數就等於解決了問題」。
5. 寫出離散時間的 Bellman 方程，並說明為何必須由後往前求解。
6. 什麼是維數詛咒？本書中有哪三種因應策略？

7. 寫出 HJB 方程並解讀其中每一項的物理意義。
8. 說明 LQR 問題的兩個前提條件，以及 Q 、 R 兩個權重矩陣的作用。
9. 為何 R 必須正定而 Q 可以只是半正定？
10. 寫出代數 Riccati 方程與最佳增益的公式，並說明其工程意義。
11. 比較有限時域與無限時域 LQR 的差異，並說明實務上多用後者的原因。
12. 說明 Bryson 法則的邏輯，並解釋為何它是一個好的起點。
13. 列舉 LQR 的三個優良性質與四個實務限制。
14. 為何說「對一維點質量，LQR 的解就是 PD 控制器」？LQR 的真正價值在哪裡？
15. 說明倒立擺的三個特性（不穩定、欠驅動、非最小相位），以及為何 LQR 特別適合它。
16. 列出最佳控制與強化學習的術語對照，並指出兩者最關鍵的差異。

計算題

1. 質量 $m = 4 \text{ kg}$ 的物體在無摩擦平面上，控制輸入為力。（a）寫出狀態空間模型 A 、 B 。
（b）取 $Q = \text{diag}(4, 0)$ 、 $R = 1$ ，手算求解 ARE 得到 P 與 K 。（c）計算閉環的自然頻率與阻尼比。（d）驗證阻尼比是否為 0.707。
2. 某關節的等效慣量 $J = 0.8 \text{ kg}\cdot\text{m}^2$ 。設計要求：位置誤差不超過 0.01 rad 、速度誤差不超過 0.4 rad/s 、力矩不超過 $15 \text{ N}\cdot\text{m}$ 。（a）用 Bryson 法則設定 Q 與 R 。（b）若實測發現收斂太慢，說明應如何調整並解釋理由。（c）若把位置容許誤差改為 0.002 rad ， Q_{11} 變為幾倍？增益約變為幾倍？
3. 某倒立擺 $M = 1.5 \text{ kg}$ 、 $m = 0.3 \text{ kg}$ 、 $l = 0.4 \text{ m}$ 、 $g = 9.8$ 。（a）計算 A 矩陣中的 $-mg/M$ 與 $(M+m)g/(Ml)$ 兩個元素。（b）計算 B 矩陣的兩個非零元素。（c）由 $\lambda^2 = (M+m)g/(Ml)$ 求不穩定極點與發散時間常數。（d）依此建議最低的控制頻率（取時間常數倒數的 10 倍）。
4. 某二階系統 $\dot{x} = Ax + Bu$ ，其中 $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$ 、 $B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ 。（a）計算開迴路特徵值並判斷穩定性。（b）取 $Q = I$ 、 $R = 1$ ，寫出 ARE 的三個純量方程式（設 $P = \begin{bmatrix} p_1 & p_2 \\ p_2 & p_3 \end{bmatrix}$ ）。（c）說明求解此方程組的步驟（不需完整求出數值）。（d）若求得 $K = [k_1, k_2]$ ，寫出閉環系統矩陣 $A - BK$ 。

動手做與專題延伸

本章實作：LQR 設計工具

延續前面章節的程式庫，加入最佳控制模組。

- **階段一**：實作 ARE 求解與 LQR 增益計算（可用 `scipy`，但建議也自行實作一個簡單的迭代解法以理解原理）。
- **階段二**：實作 Bryson 法則的自動權重設定，輸入各狀態與控制的容許範圍，自動產生 Q 與 R 。
- **階段三**：整合第 14 章的動力學模擬，建立「設計 → 模擬 → 評估」的完整迴圈，能自動掃描權重並輸出取舍曲線。
- **階段四**：實作有限時域的微分 Riccati 方程（反向積分），觀察 $K(t)$ 如何在大部分時間趨於常數。
- **階段五**：把 LQR 與第 17 章的計算力矩控制結合——先用 CTC 線性化為 $\ddot{q} = u$ ，再用 LQR 設計外層增益。比較這個組合與手動指定 ω_n 、 ζ 的差異。

專題延伸

- **取舍曲線（Pareto Front）視覺化**：對一維點質量系統，掃描 Q/R 由 0.01 到 10000，記錄每組增益對應的「穩定時間」與「最大控制量」，以這兩者為軸繪出散布圖。你會得到一條凸曲線——這就是「快與省力」的取舍前緣，任何在曲線上的點都是最佳的，選哪一個取決於你的偏好。這張圖是理解最佳控制本質最好的教材。
- **倒立擺的吸引域分析**：對設計好的 LQR 控制器，在「初始角度 × 初始角速度」的平面上取樣數千個初始狀態，模擬並記錄哪些能成功穩定。將成功與失敗以不同顏色繪出，得到一張「吸引域」圖。改變 Q 與 R 再繪一次，觀察吸引域如何變化——你會發現更積極的控制器（高 Q ）有更大的吸引域，但代價是更大的控制量。
- **進階：iLQR 的實作與應用**。實作迭代 LQR：對非線性系統，在當前軌跡上逐點線性化、用有限時域 LQR 求解、更新軌跡、重複直到收斂。用它求解一個「倒立擺擺起」（swing-up）問題——由下垂位置擺盪到直立位置。這個任務無法用單純的 LQR 解決（因為需要遠離線性化點的大範圍運動），但 iLQR 能找到漂亮的解。這將讓你親身體會第 17.8.4 節的 MPC 與本章 LQR 之間的橋樑。

參考文獻與延伸閱讀

- Bellman, R., Dynamic Programming, Princeton University Press, 1957 (動態規劃與最佳性原理的原始著作) 。
- Kalman, R. E., "Contributions to the Theory of Optimal Control," Bol. Soc. Mat. Mexicana, 1960 (LQR 與 Riccati 方程在控制上的奠基論文) 。
- Bertsekas, D. P., Dynamic Programming and Optimal Control, Athena Scientific (動態規劃與最佳控制的權威教材) 。
- Anderson, B. D. O. and Moore, J. B., Optimal Control: Linear Quadratic Methods, Prentice Hall (LQR 與 LQG 的經典專著) 。
- Bryson, A. E. and Ho, Y.-C., Applied Optimal Control, Taylor & Francis (Bryson 法則的出處，工程導向的最佳控制教材) 。
- Kirk, D. E., Optimal Control Theory: An Introduction, Dover (適合初學者的入門教材，含大量範例) 。
- Todorov, E. and Li, W., "A Generalized Iterative LQG Method for Locally-Optimal Feedback Control," ACC 2005 (iLQR/iLQG 的代表性論文) 。
- Tedrake, R., Underactuated Robotics, MIT 課程講義 (線上公開) (欠驅動系統控制的優秀教材，含大量倒立擺與行走機器人範例) 。
- Toussaint, M., Introduction to Robotics 課程講義, TU Berlin (本書系參考的德式講義，含最佳控制與 Riccati 方程的簡潔處理) 。
- Sutton, R. S. and Barto, A. G., Reinforcement Learning: An Introduction, 2nd ed., MIT Press (強化學習的權威教材，第 4 章即為動態規劃，可與本章對照閱讀) 。

第 19 章

可控性與穩定性

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

【章首情境圖建議】畫面中央為一個三維碗狀曲面，橫軸為位置誤差、縱軸為速度，高度代表 *Lyapunov* 函數 V ；一條螺旋軌跡由碗緣持續往下滑向碗底，旁註「 $V \leq 0$ 」。左側疊一張複平面圖，標出左半平面 (連續系統穩定區) 與單位圓 (離散系統穩定區)，並以箭頭表示極點配置把不穩定極點由右半平面搬到左半平面。右側為一支機械手臂由數個差異極大的初始姿態同時出發、最終全部收斂到同一目標姿態的多重曝光，旁註「全域漸近穩定」。標題文字：不必解出方程，也能證明它一定會到。

叫獸開講 「控制器算出了最完美的指令，系統卻仍然翻車——問題出在哪裡？」

小妤把第 18 章的 LQR 用在一台自製的兩輪自平衡車上。她仔細寫出 A 、 B 矩陣、選好 Q 與 R 、解出 ARE 、燒錄增益——然後車子在啟動的瞬間就往前撲倒，一路加速衝到桌邊摔了下去。

「我照著公式算的啊，」她把筆電推過來，「 Q 和 R 都是正定的， ARE 也解出來了，程式沒有報錯。」

叫獸看了一眼她的 A 矩陣，指著其中一項：「你這裡的符號寫反了。」

「那.....重算一次就好了？」

「當然可以。但我想問你一個更重要的問題。」叫獸把筆電闔上，「在你按下啟動按鈕之前，你怎麼知道這個控制器一定不會失控？」

小妤愣住了。「模擬跑過了啊。」

「模擬跑的是你寫錯的那個模型。它在你的模型裡穩得很，但那個模型不是真實的車子。」叫獸在白板上寫下三個字：**保證**。「工程師和玩家的差別，就在這裡。玩家說『我試過了，沒問題』；工程師說『我可以證明它不會出問題』。」

他停頓了一下：「更根本的問題是——有些系統根本就控制不了。你有沒有想過，如果那台車的兩個輪子被焊死在同一根軸上，不管你的控制器多聰明，它都轉不了彎？這不是控制器的問題，是系統本身的問題。在動手設計之前，你應該先問：這個系統，究竟能不能被控制到我想要的任何狀態？」

小翰舉手：「那要怎麼問？」

「有一個矩陣可以回答。」叫獸寫下 $C = [B, AB, A^2B, \dots]$ 。「把它算出來，看它的秩夠不夠。夠，代表你想去哪就去得了；不夠，代表有些狀態你永遠到不了——再厲害的演算法也沒用。」

接著他在白板另一邊畫了一個碗，裡面放了一顆珠子。「至於穩定性，有一個更漂亮的方法，連微分方程都不用解。想像你的系統是一顆珠子在碗裡滾動。只要你能找到一個『高度函數』，證明珠子每一刻的高度都在下降，那你根本不需要知道它走的路徑長什麼樣——它終究會滾到碗底。」

「這個想法是一位俄國數學家 Lyapunov 在一百三十年前提出的。今天，我們要用它來證明

一件事：第 17 章那個 PD 加重力補償的控制器，無論初始位置在哪裡、無論機器人多重、無論姿態多複雜，都保證會收斂。而且證明過程中，會用到第 13 章那個看似無用的性質—— $\dot{M} - 2C$ 是斜對稱矩陣。」

本章為什麼重要

前面三章（16、17、18）教會了我們設計控制器，但都留下了同一個缺口：我們憑什麼相信它一定有效？第 16 章分析了簡化的二階系統、第 17 章假設模型完美、第 18 章宣稱 LQR 保證穩定卻沒有證明。本章要補上這個缺口。

這不是純理論的潔癖。在機器人上，「失控」的代價是實實在在的：手臂暴衝可能傷人、自平衡車摔壞、四足機器人在測試場翻覆。而最危險的失控，往往發生在模擬中沒測到的邊界情況——這正是為何需要「證明」而非「測試」。

本章提供兩把工具。可控性回答「這個系統有沒有可能被控制」——這是設計之前的體檢，能在動手前就篩掉不可能的方案。Lyapunov 穩定性則提供一個不需解微分方程就能判定穩定性的方法，它對非線性系統同樣有效，是機器人控制中最重要的理論工具。

本章也是三個伏筆的回收處：第 13 章的 $\dot{M} - 2C$ 斜對稱性、第 17 章的 Takegaki-Arimoto 定理、以及第 18 章 LQR 的穩定性保證，都將在此得到嚴格的證明。

學習目標

- 能定義可控性，並說明其在控制系統設計中的角色。
- 能建構可控性矩陣並以其秩判斷系統是否完全可控。
- 能舉例說明不可控系統，並解釋其物理成因。
- 能區分 Lyapunov 穩定、漸近穩定與指數穩定三個定義。
- 能以特徵值判斷線性系統的穩定性。
- 能運用極點配置設計狀態回授增益。
- 能說明 Lyapunov 直接法的核心思想與三個條件。
- 能為簡單系統構造 Lyapunov 函數並證明其穩定性。
- 能運用能量作為 Lyapunov 候選函數，證明機器人控制器的穩定性。

- 能說明 $\dot{\mathbf{M}} - 2\mathbf{C}$ 斜對稱性在穩定性證明中的關鍵作用。
- 能說明可觀測性的意義及其與可控性的對偶關係。

先備知識

第 13 章的動力學方程與慣性矩陣性質（尤其是正定性與 $\dot{\mathbf{M}} - 2\mathbf{C}$ 的斜對稱性）、第 16~18 章的控制器設計、第 18 章的狀態空間表示法。

數學方面需要：矩陣的秩與特徵值、正定矩陣的概念、以及基本的微分運算。本章的證明看似抽象，但每一步都有清晰的物理對應——建議邊讀邊回想「碗中的珠子」這個圖像。

核心名詞中英對照

表 19-1 本章核心名詞

中文	英文	說明
可控性	Controllability	能否以控制輸入將系統帶到任意狀態
可控性矩陣	Controllability Matrix	判斷可控性的矩陣 $[\mathbf{B}, \mathbf{AB}, \mathbf{A}^2\mathbf{B}, \dots]$
完全可控	Completely Controllable	可控性矩陣滿秩的系統
可觀測性	Observability	能否由輸出推斷出完整的內部狀態
平衡點	Equilibrium Point	系統可以永遠停留的狀態
Lyapunov 穩定	Lyapunov Stable	初始擾動小則後續偏差也小
漸近穩定	Asymptotically Stable	不僅有界且最終收斂到平衡點
指數穩定	Exponentially Stable	以指數速率收斂
全域穩定	Globally Stable	對任意初始狀態皆成立
Lyapunov 函數	Lyapunov Function	類似能量、沿軌跡遞減的純量函數
直接法	Direct Method	不需解方程即可判定穩定性的方法
半負定	Negative Semi-definite	恆小於等於零的二次型
LaSalle 不變性原理	LaSalle's Invariance Principle	處理 V 半負定情形的定理
極點配置	Pole Placement	指定閉環特徵值以設計增益的方法
穩定裕度	Stability Margin	系統距離不穩定的餘裕
斜對稱矩陣	Skew-symmetric Matrix	滿足 $\mathbf{S}^T = -\mathbf{S}$ 的矩陣

19.1 可控性：能不能控制

19.1.1 問題的意義

在設計控制器之前，應該先問一個更基本的問題：**這個系統有沒有可能被控制？**

可控性的正式定義是：對於任意的初始狀態 x_0 與任意的目標狀態 x_f ，是否存在一個控制訊號 $u(t)$ ，能在有限時間內把系統由 x_0 帶到 x_f ？若答案為是，稱系統**完全可控**。

這個問題之所以重要，是因為**不可控是系統的先天缺陷，任何控制器都無法彌補**。正如開頭故事中兩輪被焊死的車子——不管演算法多聰明，它就是轉不了彎。及早發現這一點，能避免把時間浪費在注定失敗的設計上。

19.1.2 可控性矩陣

對於線性系統 $\dot{x} = Ax + Bu$ ，Kalman 給出了一個優雅的判準。建構**可控性矩陣**：

$$C = [B, AB, A^2B, \dots, A^{n-1}B]$$

可控性矩陣。 n 為狀態的維度。矩陣的大小為 $n \times (n \cdot m)$ ，其中 m 為輸入的數目。

$$\text{系統完全可控} \iff \text{rank}(C) = n$$

Kalman 可控性判準。秩必須等於狀態的維度。

為什麼是這個形式？直觀的解釋是： B 代表「輸入能直接推動的方向」； AB 代表「經過一次系統動態演化後，這個推力會傳到哪些方向」； A^2B 則是傳兩次之後……把所有這些方向加起來，若能張滿整個狀態空間，就代表沒有任何方向是控制不到的。

為何只算到 $A^{n-1}B$ 就停？這來自 Cayley-Hamilton 定理： A^n 可以表示為 $I, A, \dots, A^{n-1}$ 的線性組合，因此再往後算不會產生新的方向。

19.1.3 不可控的例子

例子一：兩個完全對稱的子系統共用一個輸入。設有兩台完全相同的馬達，接在同一個訊號上。無論怎麼下指令，它們永遠做一模一樣的動作——「兩者的差」這個狀態永遠是零，無法被控制。這正是開頭故事中焊死的輪子。

例子二：控制輸入影響不到某個狀態。若某個關節的馬達壞了（或根本沒裝），該關節的角度就成為不可控狀態。

例子三：機構的對稱性。某些並聯機構在特定姿態下會出現對稱的自由度，使控制輸入無法區分兩種運動模式。

◎ 可控性與奇異點的關係

第 12 章的奇異點與本章的不可控性有相似之處，但**不完全相同**。奇異點是**瞬時**的現象：在該位形下，雅可比失去滿秩，末端某個方向的速度無法產生；但只要離開那個位形，能力就恢復了。不可控則是**結構性**的缺陷：無論在哪個狀態，都有一部分狀態空間永遠到不了。用一個比喻：奇異點像車子開到了死巷（倒退出來就好），不可控則像車子根本沒有方向盤。實務上，機器手臂在奇異點附近會出現「暫時性的方向失能」，這在數學上更接近「可控性變差」而非「完全不可控」。

19.1.4 可控性與 LQR 的關係

第 18.8.1 節提到「只要系統可控，LQR 必定穩定」。現在可以理解這個條件的意義：若系統不可控，代表有些狀態根本無法被影響——若這些狀態本身又是不穩定的，那麼無論怎麼設計控制器都無法拯救。

嚴格來說，LQR 的穩定性只需要一個較弱的條件：**可穩定 (stabilizable)**——即所有**不穩定的模態**都是可控的。已經穩定的模態即使不可控也無妨，因為它們會自己收斂。

19.2 可觀測性與對偶

19.2.1 能不能「看見」

可控性問的是「能不能推動系統」，可觀測性則問相反的問題：**能不能由量測到的輸出，推斷出系統完整的內部狀態？**

這在機器人上是實際的問題。第 18.8.2 節提過，LQR 的控制律 $u = -Kx$ 需要知道**所有狀態**。但實務上：關節速度往往不直接量測（而是由位置差分得到，雜訊大）、末端負載的狀態未知、柔性關節的變形量無法量測。這時就需要**觀測器**由可量測的訊號估計出完整狀態。

觀測器能否存在，取決於系統是否可觀測。設輸出方程為 $y = Cx$ ，則**可觀測性矩陣**為：

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

可觀測性矩陣。系統完全可觀測 $\Leftrightarrow \text{rank}(\mathcal{O}) = n$ 。

19.2.2 優雅的對偶關係

比較兩個矩陣的形式，會發現一個漂亮的對稱性：

$$(A, B) \text{ 可控} \iff (A^T, B^T) \text{ 可觀測}$$

可控性與可觀測性的對偶原理。

這代表兩個問題在數學上是同一個問題的兩面。因此**所有關於可控性的結果，都可以直接翻譯成關於可觀測性的結果**——包括設計方法。事實上，Kalman 濾波器（第 24 章）的增益計算與 LQR 的增益計算，用的是形式完全相同的 Riccati 方程，只是把 A 換成 A^T 。

這個對偶在工程上的意義是：**估計與控制是同一件事的兩面**。控制是「決定要輸出什麼」，估計是「推斷輸入了什麼」——兩者共用同一套數學。第 24 章討論狀態估測時會再次見到這個結構。

19.3 穩定性的三個定義

19.3.1 平衡點

談穩定性之前，必須先定義**平衡點**：系統可以永遠停留不動的狀態。數學上：

$$x_e \text{ 是平衡點} \iff f(x_e, 0) = 0$$

在該狀態下，若不施加控制，狀態的導數為零，因此系統保持不動。

以倒立擺為例，有兩個平衡點：桿子垂直向下（穩定）與桿子垂直向上（不穩定）。兩者都滿足「不動就一直不動」的條件，但性質截然不同——這正是需要區分不同穩定性等級的原因。

19.3.2 三個層次

穩定性有三個層次，由弱到強：

層次一：Lyapunov 穩定 (Stable)。「初始擾動夠小，之後的偏差就會夠小。」形式化的說法是：對任意的 $\varepsilon > 0$ ，存在 $\delta > 0$ ，使得 $\|x(0) - x_e\| < \delta$ 時，恆有 $\|x(t) - x_e\| < \varepsilon$ 。

注意這個定義**不要求收斂**——它只要求「不會越跑越遠」。一個無阻尼的單擺（永遠等幅擺動）就是 Lyapunov 穩定但不收斂的例子，正如第 16.3.2 節的純比例控制。

層次二：漸近穩定 (Asymptotically Stable)。在 Lyapunov 穩定的基礎上，還要求最終收斂：

$$\lim_{t \rightarrow \infty} x(t) = x_e$$

漸近穩定。這是控制設計通常追求的目標。

層次三：指數穩定 (Exponentially Stable) 。不僅收斂，還要求收斂速度至少是指數的：

$$\|x(t) - x_e\| \leq \alpha \|x(0) - x_e\| e^{-\lambda t}$$

指數穩定。 λ 稱為收斂率，它給出了「多快收斂」的定量保證。

三者的關係是：指數穩定 $\Rightarrow$ 漸近穩定 $\Rightarrow$ Lyapunov 穩定。反之則不然。

表 19-2 三種穩定性的比較

等級	要求	會收斂嗎	有速度保證嗎	典型例子
Lyapunov 穩定	偏差有界	不一定	無	無阻尼單擺、純 P 控制
漸近穩定	最終收斂到平衡點	會	無	有摩擦的單擺、PD 控制
指數穩定	以指數速率收斂	會	有	線性穩定系統、CTC 閉環

19.3.3 局部與全域

還有一個重要的區分：穩定性是局部還是全域的。

局部穩定：只有當初始狀態夠靠近平衡點時才成立。倒立擺的 LQR 控制器就是局部的——傾角太大時（第 18 章程式範例中約 45 度以上），線性化失效，控制器會失敗。這個「夠靠近」的範圍稱為**吸引域**（region of attraction）。

全域穩定：對任意初始狀態都成立。第 17.5.2 節的 PD 加重力補償就是全域漸近穩定——無論手臂一開始擺在哪裡，都保證會收斂。這是一個非常強的保證，也是本章要證明的核心結果。

19.4 線性系統的穩定性：特徵值判準

19.4.1 由解的形式看穩定性

對線性系統 $\dot{x} = Ax$ ，其解可以用矩陣指數表示： $x(t) = e^{At} x(0)$ 。若 A 可對角化，則解可分解為各特徵模態的組合：

$$x(t) = \sum_i c_i v_i e^{\lambda_i t}$$

各項為特徵向量 v_i 乘上指數項。 λ_i 為特徵值。

由此立刻可以看出穩定性的判準：每一項是否衰減，取決於 $e^{\lambda_i t}$ 是否趨近於零。若 $\lambda = a$

+ bj 為複數，則 $|e^{(\lambda t)}| = e^{(at)}$ ——只取決於實部。

線性系統漸近穩定 $\Leftrightarrow$ 所有特徵值的實部皆為負

特徵值判準。即所有極點落在複平面的左半平面。

虛部的作用則是決定振盪：實部決定衰減快慢、虛部決定振盪頻率。這與第 16 章二階系統的分析完全一致—— $\zeta\omega_n$ 是實部（衰減）、 ω_d 是虛部（振盪）。

19.4.2 閉環系統的特徵值

加入狀態回授 $u = -Kx$ 後，閉環系統為：

$$\dot{x} = Ax + B(-Kx) = (A - BK)x$$

閉環系統。穩定性取決於 $A - BK$ 的特徵值。

這給出了控制設計的一個直接視角：設計 K ，就是在移動閉環系統的特徵值。開迴路不穩定的系統（如倒立擺，第 18.9 節算出有 $\lambda = +6.26$ ），可以透過適當的 K 把特徵值搬到左半平面。

19.4.3 極點配置

既然穩定性取決於特徵值，一個自然的設計方法是：直接指定期望的閉環特徵值，反推 K 。這稱為極點配置（pole placement）。

有一個重要的定理：若系統完全可控，則可以透過狀態回授把閉環特徵值配置到任意指定的位置。這正是可控性最重要的實用意義——它保證了設計的自由度。

極點配置與 LQR 的比較很有啟發性：

表 19-3 極點配置與 LQR 的比較

面向	極點配置	LQR
設計者指定	期望的閉環極點位置	權重矩陣 Q 、 R
直覺性	直接（極點決定響應）	間接（需理解權重意義）
多輸入系統	解不唯一，需額外準則	解唯一
控制量大小	無保證，可能過大	由 R 直接控制
穩定裕度	無保證	保證 $[0.5, \infty)$ 增益裕度
適用場合	低階、單輸入、需精確指定響應	多變數、需權衡控制量

實務上兩者常互補：先用 LQR 得到一組合理的增益，再檢查其閉環極點是否符合預期；或

用極點配置得到初值，再用 LQR 微調。

19.4.4 線性化與局部穩定性

對非線性系統 $\dot{x} = f(x)$ ，可以在平衡點附近做一階泰勒展開：

$$\dot{x} \approx A x, \quad \text{其中 } A = \partial f / \partial x |_{x = x_e} \quad (\text{雅可比矩陣})$$

近似線性化。注意這與第 17.3.3 節的回授線性化完全不同——這只在平衡點附近有效。

Lyapunov 第一法（間接法）指出：若線性化系統的所有特徵值實部皆為負，則原非線性系統在該平衡點**局部**漸近穩定。

這個方法的優點是簡單（只需算特徵值），缺點有二：**只能保證局部穩定**（不知道吸引域多大），且**當有特徵值實部為零時無法判定**（線性項不足以決定行為，需看高階項）。

這些限制，正是下一節 Lyapunov 直接法要克服的。

19.5 Lyapunov 直接法

19.5.1 碗中珠子的想法

Lyapunov 於 1892 年提出的方法，核心想法是開頭故事中那個「碗中珠子」的圖像：

◎ Lyapunov 直接法的核心思想

若能找到一個類似「高度」或「能量」的純量函數 $V(x)$ ，滿足：一、它在平衡點為零、在其他地方為正（碗底最低）；二、沿著系統的軌跡，它持續下降（珠子一直往下滾）——那麼系統必定會收斂到平衡點。而且完全不需要解出微分方程。

這個方法的威力在於「不需要解方程」。對非線性系統，解析解幾乎不存在；但只要能找到一個下降的函數，就能斷定收斂。這使 Lyapunov 方法成為非線性控制中最重要工具。

19.5.2 三個條件

形式化地說，若存在一個純量函數 $V(x)$ （稱為 **Lyapunov 函數**）滿足下列條件，則平衡點 $x_e = 0$ 是穩定的：

條件一（正定）： $V(0) = 0$ ，且 $V(x) > 0$ 對所有 $x \neq 0$

條件二（遞減）： $\dot{V}(x) \leq 0$ 對所有 x

條件三（徑向無界）：當 $\|x\| \rightarrow \infty$ 時 $V(x) \rightarrow \infty$

Lyapunov 穩定性定理。條件三只在要證明全域穩定時才需要。

三個條件的物理意義是：**條件一**確保 V 像一個「碗」——平衡點在碗底；**條件二**確保狀態沿著碗壁往下滑，不會往上爬；**條件三**確保碗是「無限深」的，不會有狀態從碗的邊緣溜走。

若進一步有 $\dot{V} < 0$ （嚴格負定，而非只是小於等於零），則平衡點是**漸近穩定**——不只有界，還會真正收斂到零。

19.5.3 關鍵技術：計算 $\dot{V}$

實際運用時，最關鍵的一步是計算 V 沿系統軌跡的時間導數。由鏈鎖律：

$$\dot{V} = (\partial V / \partial \mathbf{x})^T \dot{\mathbf{x}} = (\partial V / \partial \mathbf{x})^T \mathbf{f}(\mathbf{x})$$

注意這裡代入了系統方程 $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ 。這是整個方法的精髓——把系統的動態「灌」進 V 的導數中。

這一步是初學者最容易忽略的： $\dot{V}$ 不是隨便的導數，而是「沿著系統實際運動的軌跡」的導數。因此它必然包含系統的動態資訊，也因此才能反映系統的行為。

19.5.4 方法的優點與難處

優點一：適用於非線性系統。特徵值判準只適用於線性系統，Lyapunov 方法則無此限制。

優點二：可證明全域穩定。線性化只能給出局部結果，Lyapunov 方法（配合條件三）能證明全域性質。

優點三：不需解方程。這在實務上是決定性的——多數機器人系統的閉環方程根本無法解析求解。

難處：如何找到 V ？這是 Lyapunov 方法唯一的、也是最大的困難。定理只說「若能找到」，卻沒說「怎麼找」。對一般的非線性系統，構造 Lyapunov 函數是一門藝術。

但機器人有一個巨大的優勢：它是機械系統，天然就有一個現成的候選——**能量**。這正是下一節的主題。

19.6 能量作為 Lyapunov 函數

19.6.1 為何能量是天然的候選

物理系統的能量有三個性質，恰好對應 Lyapunov 函數的要求：**一、能量恆為非負**（動能為正、位能可調整基準使其為正）；**二、能量在平衡點最小**（穩定平衡就是能量的局部極小點）；**三、有耗散時能量會下降**（摩擦、阻尼會消耗能量）。

因此對機械系統，「證明穩定」往往等價於「證明能量持續下降」——這是一個極其自然的物理論證，而 Lyapunov 方法把它變成了嚴格的數學工具。

19.6.2 最簡單的例子：質量—彈簧—阻尼

先用最簡單的例子熟悉流程。考慮 $m\ddot{x} + c\dot{x} + kx = 0$ (第 16 章的 PD 控制閉環系統)。取總能量為 Lyapunov 候選：

$$V = \frac{1}{2} m \dot{x}^2 + \frac{1}{2} k x^2$$

動能加彈性能。顯然 $V \geq 0$ ，且僅在 $x = \dot{x} = 0$ 時為零——滿足條件一。

計算 $\dot{V}$ 並代入系統方程 ($\ddot{x} = -(c\dot{x} + kx)/m$)：

$$\dot{V} = m \dot{x} \ddot{x} + k x \dot{x} = \dot{x}(m \ddot{x} + k x) = \dot{x}(-c \dot{x}) = -c \dot{x}^2$$

關鍵的一步：彈性項 $kx\dot{x}$ 與 $m\dot{x}\ddot{x}$ 中的對應項互相抵消，只剩下阻尼項。

$$\dot{V} = -c \dot{x}^2 \leq 0$$

只要阻尼 $c > 0$ ，能量就持續下降——系統穩定。

這個計算揭示了一個深刻的事實：在能量的觀點下，彈簧不消耗能量（只是在動能與位能之間搬運），只有阻尼才真正耗散能量。這正是第 16.3.2 節「純比例控制會永遠振盪」的物理解釋——沒有阻尼項， $\dot{V} = 0$ ，能量守恆，振盪不衰減。

19.6.3 $\dot{V}$ 半負定的問題與 LaSalle 原理

上例中 $\dot{V} = -c\dot{x}^2$ 只是半負定——當 $\dot{x} = 0$ 但 $x \neq 0$ 時（例如彈簧被拉開但瞬間靜止）， $\dot{V} = 0$ 。嚴格說來，這只能證明 Lyapunov 穩定，不足以證明漸近穩定。

解決這個問題的工具是 LaSalle 不變性原理：

◎ LaSalle 不變性原理

若 $\dot{V} \leq 0$ ，考慮集合 $E = \{x : V(x) = 0\}$ 。系統的軌跡最終會收斂到 E 中最大的不變集（即進去後就不會出來的部分）。若這個不變集只包含平衡點，則系統漸近穩定。

套用到上例： $E = \{(x, \dot{x}) : \dot{x} = 0\}$ 。但若系統停在 $\dot{x} = 0$ 而 $x \neq 0$ 處，由方程 $m\ddot{x} = -kx \neq 0$ ，代表 $\ddot{x} \neq 0$ ，狀態立刻會離開 $\dot{x} = 0$ ——因此無法「停留」在 E 中。 E 中唯一的不變集只有原點，故系統漸近穩定。

LaSalle 原理在機器人控制的穩定性證明中幾乎必定會用到，因為機械系統的 V 通常只含速

度項（半負定）。

19.7 伏筆回收：PD 加重力補償的穩定性證明

19.7.1 要證明什麼

現在具備了所有工具，可以嚴格證明第 17.5.2 節提到的 Takegaki-Arimoto 定理。要證明的敘述是：

◎ 待證定理

對於任意的機械手臂，控制律 $\tau = K_p(q_d - q) - K_d \dot{q} + G(q)$ （其中 K_p 、 K_d 為任意正定矩陣、 q_d 為常數目標）能保證閉環系統全域漸近穩定。也就是說：無論初始姿態在哪裡、無論機器人的慣性參數是多少，關節必定收斂到 q_d 且速度趨於零。

值得再次強調這個定理的驚人之處：控制器完全不需要知道 $M(q)$ 與 $C(q, \dot{q})$ ，只需要重力項 $G(q)$ 。這是為何重力補償在實務上如此重要。

19.7.2 建立閉環方程

由第 13 章的動力學方程與上述控制律：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = K_p e - K_d \dot{q} + G(q) \quad (\text{其中 } e = q_d - q)$$

代入控制律。注意 $G(q)$ 在兩邊出現，直接抵消。

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + K_d \dot{q} - K_p e = 0$$

閉環系統方程。重力項已完全消失——這正是重力補償的作用。

注意 q_d 為常數，故 $\dot{e} = -\dot{q}$ 、 $\ddot{e} = -\ddot{q}$ 。這個關係在後續計算中會用到。

19.7.3 構造 Lyapunov 函數

依 19.6.1 節的思路，取「動能 + 虛擬彈性位能」作為候選：

$$V = \frac{1}{2} \dot{q}^T M(q) \dot{q} + \frac{1}{2} e^T K_p e$$

Lyapunov 候選函數。第一項是機器人的實際動能（第 13.2.2 節）、第二項是控制器產生的「虛擬彈簧」的位能。

驗證條件一（正定）：由第 13.6.1 節， $M(q)$ 恆為正定，故第一項 ≥ 0 且僅在 $\dot{q} = 0$ 時為零； K_p 為正定矩陣，故第二項 ≥ 0 且僅在 $e = 0$ 時為零。因此 $V \geq 0$ ，且僅在 $(e, \dot{q}) = (0, 0)$ 時為零——條件一成立。

驗證條件三（徑向無界）：當 $\|e\|$ 或 $\|q\|$ 趨於無窮時， V 也趨於無窮（因兩個矩陣皆正定）——條件三成立，故可證明全域穩定。

19.7.4 計算 V ：斜對稱性登場

這是整個證明最關鍵的一步。對 V 求時間導數：

$$V = \dot{q}^T M \dot{q} + \frac{1}{2} \dot{q}^T \dot{M} \dot{q} + e^T K_p e$$

注意第二項——因為 M 是 q 的函數，會隨時間變化，微分時必須包含 $\dot{M}$ 這一項。這一項的存在正是機器人動力學的特色。

由閉環方程解出 $M \ddot{q} = -C \dot{q} - K_d \dot{q} + K_p e$ ，代入：

$$V = \dot{q}^T (-C \dot{q} - K_d \dot{q} + K_p e) + \frac{1}{2} \dot{q}^T \dot{M} \dot{q} + e^T K_p e$$

代入系統動態。

利用 $\dot{e} = -\dot{q}$ ，第三項變為 $-e^T K_p \dot{q}$ 。又因 K_p 對稱， $\dot{q}^T K_p e = e^T K_p \dot{q}$ ，故第一項中的 $\dot{q}^T K_p e$ 與第三項恰好抵消：

$$V = -\dot{q}^T C \dot{q} - \dot{q}^T K_d \dot{q} + \frac{1}{2} \dot{q}^T \dot{M} \dot{q}$$

剩下三項。

整理前兩項中含 C 與 $\dot{M}$ 的部分：

$$V = \frac{1}{2} \dot{q}^T (\dot{M} - 2C) \dot{q} - \dot{q}^T K_d \dot{q}$$

關鍵時刻：括號中出現了第 13.7.3 節那個「看似無用」的組合。

回顧第 13 章的性質：採 Christoffel 符號定義的 C 使 $\dot{M} - 2C$ 為斜對稱矩陣。而斜對稱矩陣有一個重要性質：

$$\text{對任意向量 } x \text{ 與斜對稱矩陣 } S : \quad x^T S x = 0$$

斜對稱矩陣的二次型恆為零。證明： $x^T S x$ 是純量，故等於其轉置 $x^T S^T x = -x^T S x$ ，因此 $2x^T S x = 0$ 。

因此第一項完全消失：

$$V = -\dot{q}^T K_d \dot{q} \leq 0$$

得證。由於 K_d 正定， V 恆為非正——能量持續下降。

19.7.5 由 LaSalle 完成證明

V 只是半負定（當 $\dot{q} = 0$ 時為零），因此需要 LaSalle 原理。考慮集合 $E = \{(e, \dot{q}) : \dot{q} = 0\}$ 。

假設系統能停留在 E 中，則恆有 $\dot{q} = 0$ 且 $\ddot{q} = 0$ 。代入閉環方程：

$$M \cdot 0 + C \cdot 0 + K_d \cdot 0 - K_p e = 0 \Rightarrow K_p e = 0 \Rightarrow e = 0 \quad (\text{因 } K_p \text{ 正定必可逆})$$

E 中唯一的不變集是原點。

因此由 LaSalle 原理，系統全域漸近收斂到 $e = 0$ 、 $\dot{q} = 0$ 。定理得證。

◎ 這個證明告訴我們什麼

這個證明有三個值得深思的地方。一、 $\dot{M} - 2C$ 的斜對稱性不是數學巧合，而是能量守恆的體現——第 13.7.3 節說過，這個性質的物理意義是「內部的科氏力與離心力不產生也不消耗能量，只是重新分配」。正因如此，它在能量的計算中才會完全消失。二、證明完全不需要知道 M 與 C 的具體形式——只用到了 M 正定與 $\dot{M} - 2C$ 斜對稱兩個結構性質。這正是定理如此強大的原因。三、 K_d 是唯一的耗散來源——與 19.6.2 節的結論一致：彈簧（ K_p ）不耗能，只有阻尼（ K_d ）才讓系統真正收斂。

圖 19-1 PD 加重力補償的能量下降圖像

建議配圖：左半為一個三維的碗狀曲面，橫軸為位置誤差 e 、縱軸為速度 $\dot{q}$ 、高度為 Lyapunov 函數 V 。碗底標註平衡點 $(0,0)$ 。一條螺旋狀的軌跡由碗緣沿著曲面往下滑向碗底，並以箭頭標註「 $V = -\dot{q}^T K_d \dot{q} \leq 0$ 」。右半為對應的機器手臂多重曝光：由一個任意的初始姿態逐步收斂到目標姿態，並在旁列出證明的四個關鍵步驟（正定性、代入動態、斜對稱項消失、LaSalle）。

19.8 其他控制器的穩定性

19.8.1 計算力矩控制

第 17 章的計算力矩控制，其穩定性證明反而最簡單——因為閉環系統已經是線性的。

回顧第 17.2.2 節，CTC 的誤差動態為：

$$\ddot{e} + K_d \dot{e} + K_p e = 0$$

線性、非時變、且各關節解耦。

這是 n 個獨立的二階線性系統。由 19.4.1 節的特徵值判準，只要 K_p 、 K_d 為正定矩陣（通常取對角矩陣），所有特徵值的實部皆為負，系統指數穩定。

這也顯示了 CTC 的一個優勢：它提供的是指數穩定（有明確的收斂速率保證），而 PD 加重力補償只能保證漸近穩定（會收斂但速率不定）。代價則是需要完整的動力學模型。

19.8.2 LQR 的穩定性

第 18.8.1 節宣稱「LQR 保證穩定」，現在可以看出證明的思路：LQR 的價值函數 $V(x) = x^T P x$ 本身就是一個 Lyapunov 函數。

條件一：由 Riccati 方程的正定解 P ， $V = x^T P x$ 恆為正定。

條件二：計算閉環系統 $\dot{x} = (A - BK)x$ 下的 V ，經過整理可得：

$$\dot{V} = -x^T(Q + K^T R K)x \leq 0$$

由於 Q 半正定、 R 正定，括號中的矩陣至少半正定，故 $\dot{V} \leq 0$ 。

這個結果有一個優美的解讀： V 恰好等於「當下的代價」的負值。也就是說，價值函數（未來的總代價）以「當下正在付出的代價」的速率遞減——這完全符合直覺，因為每過一刻，「剩餘的代價」就少掉了「剛付出的代價」。

因此最佳性與穩定性在 LQR 中是同一件事的兩面。這是第 18 章與本章之間最深刻的連結。

19.8.3 三種控制器的穩定性比較

表 19-4 三種控制器的穩定性性質

控制器	所需模型	穩定性等級	範圍	證明工具
PD（無重力補償）	無	有穩態誤差，非漸近穩定	全域有界	Lyapunov + LaSalle
PD + 重力補償	只需 $G(q)$	漸近穩定	全域	Lyapunov + LaSalle + 斜對稱性
計算力矩控制	完整動力學	指數穩定	全域（模型準確時）	特徵值判準（閉環為線性）
LQR（線性系統）	A 、 B 矩陣	指數穩定	全域（線性系統）	價值函數即 Lyapunov 函數
LQR（線性化後）	工作點的 A 、 B	指數穩定	局部（吸引域內）	Lyapunov 第一法

19.8.4 模型誤差下的穩定性

實務上還有一個重要問題：若模型有誤差，穩定性還成立嗎？

對 PD 加重力補償：若 $G(q)$ 的估計有誤差 ΔG ，則閉環方程中會殘留一項 ΔG 。此時系統不

再收斂到 $e = 0$ ，而是收斂到 $K_p e = \Delta G$ ，即 $e_{ss} = K_p^{-1} \Delta G$ 。系統仍然穩定（誤差有界），只是有穩態誤差——這與第 17 章例題 17-2 的分析一致。

對 **計算力矩控制**：模型誤差使閉環方程變為 $\ddot{e} + K_d \dot{e} + K_p e = \delta$ （ δ 為模型誤差造成的等效干擾）。只要 δ 有界，系統就是**輸入—狀態穩定**（ISS, Input-to-State Stable）——誤差有界且正比於 δ 的大小。這正是第 17.4.2 節「80% 準確的模型仍然有用」的嚴格說法。

這類「有界干擾下仍保持有界誤差」的性質，是實務上真正需要的保證。純粹的漸近穩定在有雜訊與模型誤差的現實中並不存在，ISS 才是實用的穩定性概念。

19.9 離散時間系統的穩定性

19.9.1 數位控制器的現實

第 7 章與第 16.7 節提過，實際的控制器都是數位的：以固定週期取樣、計算、輸出。因此嚴格的分析必須在**離散時間**下進行。

離散系統的形式為：

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k$$

離散狀態方程。 $\mathbf{A}_d$ 與 $\mathbf{B}_d$ 由連續系統的 $\mathbf{A}$ 、 $\mathbf{B}$ 與取樣週期 T 換算而得。

19.9.2 穩定性判準的差異

離散系統的穩定性判準與連續系統形式相似，但判定的區域不同：

連續系統穩定 $\Leftrightarrow$ 所有特徵值的實部 < 0 （左半平面）

離散系統穩定 $\Leftrightarrow$ 所有特徵值的絕對值 < 1 （單位圓內）

兩者的穩定區域。這是初學者最容易混淆的一點。

為何是單位圓？因為離散系統的解為 $\mathbf{x}_k = \mathbf{A}_d^k \mathbf{x}_0$ ，各模態的行為由 λ^k 決定。若 $|\lambda| < 1$ ，則 $\lambda^k \rightarrow 0$ ；若 $|\lambda| > 1$ ，則發散。

兩者的對應關係為 $\lambda_d = e^{(\lambda_c T)}$ 。可以驗證： λ_c 的實部為負 $\Leftrightarrow |\lambda_d| < 1$ 。連續的左半平面，經過指數映射後恰好變成離散的單位圓內部。

19.9.3 取樣週期對穩定性的影響

這裡有一個重要的實務結論：**取樣週期太長會使原本穩定的系統變得不穩定。**

直觀的解釋是：數位控制器在兩次取樣之間是「盲目」的——它保持上一次的輸出不變，看不見這段期間系統的變化。取樣越慢，這段盲目期越長，等效於引入了延遲。而延遲會侵蝕相位裕度，過多的延遲必然導致不穩定。

實務上的經驗法則是：

取樣頻率 $\geq 10 \times$ 閉環頻寬 （最低要求）

取樣頻率 $\geq 20 \sim 30 \times$ 閉環頻寬 （建議值）

取樣頻率的選擇。閉環頻寬約等於設計的 ω_n 。

以第 18 章的倒立擺為例：其不穩定極點為 6.26 rad/s (約 1 Hz)，閉環設計的頻寬可能在 3~5 Hz，因此取樣頻率至少應為 50 Hz，建議 100 Hz 以上。這正是第 18 章例題 18-3 中「發散時間常數 0.16 秒 $\Rightarrow$ 控制頻率至少 50 Hz」的依據。

反過來說，這也解釋了為何機器手臂的伺服迴路要跑到 1~4 kHz (第 3 章的時間尺度金字塔)——因為要控制的機構共振頻率可達數十至數百 Hz。

19.10 自適應控制的穩定性概觀

19.10.1 為何自適應控制的證明特別困難

第 17.8.2 節介紹的自適應控制，其穩定性證明是一個經典的挑戰。困難在於：系統中有兩組動態在同時演變——狀態的收斂與參數估計的收斂——而且它們互相影響。

Slotine-Li 自適應控制器的證明思路，正是 Lyapunov 方法的漂亮應用。它的 Lyapunov 候選函數同時包含兩部分：

$$V = \frac{1}{2} s^T M(q) s + \frac{1}{2} \tilde{\pi}^T \Gamma^{-1} \tilde{\pi}$$

第一項是「追隨誤差」的能量 (s 為第 17.8.1 節的滑動面變數)、第二項是「參數估計誤差」 $\tilde{\pi} = \pi - \pi$ 的能量。 Γ 為自適應增益矩陣。

計算 V 後，經過與 19.7.4 節類似的操作 (包括斜對稱性的抵消)，可以證明 $\dot{V} \leq 0$ 。因此系統穩定，且追隨誤差收斂到零。

19.10.2 一個微妙但重要的區別

這個證明有一個常被誤解的結論：追隨誤差保證收斂到零，但參數估計不保證收斂到真值。

原因是 V 只包含追隨誤差的項，不包含參數誤差的項。因此參數誤差只保證有界，不保證趨

於零。要讓參數也收斂，需要額外的**持續激勵條件** (persistent excitation) ——軌跡必須「足夠豐富」，才能把所有參數都激發出來讓演算法辨識。

實務上的意義是：若機器人只做單調的重複動作，自適應控制器仍能把軌跡追隨得很好，但它學到的參數可能是錯的。一旦換一個新軌跡，性能可能突然變差。這是第 17 章專題延伸中要求驗證的現象。

19.11 由穩定性邁向力控制

19.11.1 本章的成果

本章補上了前三章的理論缺口。你現在能夠：判斷系統是否可控與可觀測、區分三個層次的穩定性、以特徵值判斷線性系統的穩定性、運用極點配置設計增益、以 Lyapunov 直接法證明非線性系統的穩定性、以及理解離散化對穩定性的影響。

最重要的是，你完成了三個伏筆的回收：第 13 章的 $\dot{M} - 2C$ 斜對稱性在 19.7.4 節成為證明的關鍵；第 17 章的 Takegaki-Arimoto 定理在 19.7 節得到完整證明；第 18 章 LQR 的穩定性保證在 19.8.2 節被揭示為「價值函數即 Lyapunov 函數」。

這三個回收展現了一件事：理論的各個部分不是孤立的知識點，而是互相支撐的結構。第 13 章那個當時看似無用的數學性質，兩百頁後成為證明的樞紐——這正是系統性學習的價值。

19.11.2 接下來：當機器人碰到東西

到目前為止，第三篇的所有控制都是**運動控制**：規劃一條軌跡，讓機器人精確地跟隨它。這隱含一個假設——機器人在自由空間中運動，不與環境接觸。

但機器人的價值恰恰在於接觸：抓取物件、插入零件、研磨表面、與人握手。一旦接觸發生，純粹的位置控制就會出問題：若目標位置在工件內部（哪怕只差 0.1 公釐），控制器會不斷加大力矩試圖到達那個「不可能的位置」，結果不是壓壞工件就是損壞機器人。

第 20 章（力控制與阻抗控制）將處理這個問題。我們會學到：如何直接控制接觸力、如何用「混合位置 / 力控制」在不同方向分別控制運動與力、以及最優雅的**阻抗控制**——不直接控制位置或力，而是控制兩者之間的**關係**，讓機器人表現得像一個可調整的彈簧。屆時第 17 章的操作空間控制與本章的穩定性工具都將派上用場。

換句話說：第 19 章保證了機器人**不會失控**，第 20 章讓它學會**溫柔地接觸**。

完整計算例題

例題 19-1 可控性判定

題目：(a) 判斷第 18 章的一維點質量系統是否完全可控。(b) 某系統 $A = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$ 、 $B = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ ，判斷其可控性並解釋物理意義。(c) 某系統 $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$ 、 $B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ ，判斷其可控性。

解 (a)：一維點質量 ($m = 2$) 的模型為 $A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ 、 $B = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$ 。

$$AB = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0 \end{bmatrix}$$

計算 AB 。

$$C = [B, AB] = \begin{bmatrix} 0 & 0.5 \\ 0.5 & 0 \end{bmatrix}$$

可控性矩陣。

其行列式為 $0 \times 0 - 0.5 \times 0.5 = -0.25 \neq 0$ ，故 $\text{rank}(C) = 2 = n$ 。系統完全可控。

物理意義：施力可以改變速度，速度又改變位置——因此透過適當的施力序列，可以把物體帶到任意的位置與速度組合。

解 (b)：計算 $AB = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ 。

$$C = [B, AB] = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$$

第二列全為零。

行列式為零， $\text{rank}(C) = 1 < 2$ 。系統不可控。

物理意義：這個系統的兩個狀態是完全解耦的 (A 為對角矩陣)，而 B 的第二個元素為零——代表控制輸入完全影響不到第二個狀態。 x_2 會依自己的動態 $\dot{x}_2 = 2x_2$ 發散，而我們無能為力。這正是開頭故事中「有些系統根本控制不了」的數學形式。

解 (c)：計算 $AB = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -3 \end{bmatrix}$ 。

$$C = [B, AB] = \begin{bmatrix} 0 & 1 \\ 1 & -3 \end{bmatrix}$$

可控性矩陣。

行列式為 $0 \times (-3) - 1 \times 1 = -1 \neq 0$ ， $\text{rank} = 2$ 。系統完全可控，因此可以用極點配置或 LQR 把閉環極點放到任意位置。

例題 19-2 Lyapunov 穩定性證明

題目：考慮非線性系統 $\dot{x} = -x^3$ 。(a) 求其平衡點。(b) 用線性化判斷穩定性，並說明遇到的困難。(c) 用 Lyapunov 直接法證明其漸近穩定。(d) 判斷是否為指數穩定。

解 (a)：令 $\dot{x} = 0$ ，得 $-x^3 = 0$ ，故唯一平衡點為 $\mathbf{x}_e = 0$ 。

解 (b)：線性化需計算 $\partial f / \partial x = -3x^2$ ，在 $x = 0$ 處：

$$A = \partial f / \partial x \big|_{x=0} = -3(0)^2 = 0$$

線性化的系統矩陣為零。

特徵值 $\lambda = 0$ ，實部為零。由 19.4.4 節，Lyapunov 第一法在此無法判定——線性項不足以決定行為，必須看高階項。這正是線性化方法的限制。

解 (c)：取最簡單的 Lyapunov 候選：

$$V(x) = \frac{1}{2} x^2$$

顯然 $V(0) = 0$ 、 $V(x) > 0$ 對 $x \neq 0$ (條件一成立)，且 $x \rightarrow \pm\infty$ 時 $V \rightarrow \infty$ (條件三成立)。

計算 $\dot{V}$ 並代入系統方程：

$$\dot{V} = x \dot{x} = x(-x^3) = -x^4$$

代入 $\dot{x} = -x^3$ 。

對所有 $x \neq 0$ ， $-x^4 < 0$ (嚴格負定)。因此 $\dot{V} < 0$ ，由 Lyapunov 定理，系統全域漸近穩定。

這個例子完美展示了直接法的價值：線性化束手無策的問題，Lyapunov 方法只用兩行就解決了。

解 (d)：雖然漸近穩定，但不是指數穩定。可以解出精確解驗證：

$$x(t) = x_0 / \sqrt{1 + 2x_0^2 t}$$

解析解。當 t 很大時， $x(t) \approx 1/\sqrt{2t}$ ——這是代數衰減 (如 $t^{-1/2}$)，比任何指數衰減都慢。

工程意義：漸近穩定只保證「最終會到」，不保證「多快到」。在控制設計中，若要求明確的收斂速率，必須額外驗證指數穩定性——這正是第 17 章的 CTC (指數穩定) 優於 PD 加重力

補償（僅漸近穩定）的地方。

例題 19-3 單關節 PD 加重力補償的穩定性驗證

題目：考慮第 13 章的單擺（ $I = (1/3)mL^2$ ，質量 $m = 2 \text{ kg}$ 、長 $L = 0.6 \text{ m}$ ），採用控制律 $\tau = K_p(\theta_d - \theta) - K_d \dot{\theta} + mg(L/2)\sin \theta$ ，取 $K_p = 20 \text{ N}\cdot\text{m}/\text{rad}$ 、 $K_d = 4 \text{ N}\cdot\text{m}\cdot\text{s}/\text{rad}$ 。（a）寫出閉環方程。（b）構造 Lyapunov 函數並驗證條件一。（c）計算 V 。（d）用 LaSalle 原理完成證明。（e）驗證此為單自由度情形下 19.7 節定理的特例。

解（a）：單擺的動力學為 $I \ddot{\theta} + mg(L/2)\sin \theta = \tau$ 。代入控制律：

$$I \ddot{\theta} + mg(L/2)\sin \theta = K_p(\theta_d - \theta) - K_d \dot{\theta} + mg(L/2)\sin \theta$$

重力項在兩邊抵消。

$$I \ddot{\theta} + K_d \dot{\theta} - K_p e = 0 \quad (\text{其中 } e = \theta_d - \theta)$$

閉環方程。這是一個線性的二階系統——因為單自由度時 M 為常數、 C 為零。

代入數值： $I = (1/3)(2)(0.36) = 0.24 \text{ kg}\cdot\text{m}^2$ 。

$$0.24 \ddot{\theta} + 4 \dot{\theta} - 20 e = 0$$

數值化的閉環方程。

解（b）：依 19.7.3 節的形式取：

$$V = \frac{1}{2} I \dot{\theta}^2 + \frac{1}{2} K_p e^2 = \frac{1}{2}(0.24)\dot{\theta}^2 + \frac{1}{2}(20)e^2 = 0.12 \dot{\theta}^2 + 10 e^2$$

Lyapunov 候選函數。

驗證條件一：兩項的係數皆為正（ $I > 0$ 且 $K_p > 0$ ），故 $V \geq 0$ ，且僅在 $\dot{\theta} = 0$ 且 $e = 0$ 時為零。條件一成立。此外 V 隨 $|e|$ 或 $|\dot{\theta}|$ 增大而無界，條件三亦成立。

解（c）：

$$\dot{V} = I \dot{\theta} \ddot{\theta} + K_p e \dot{e}$$

求導。注意單自由度時 $i = 0$ ，故無 $\frac{1}{2} \dot{q}^T \dot{M} \dot{q}$ 項——這是單擺比多連桿手臂簡單之處。

由閉環方程 $I \ddot{\theta} = K_p e - K_d \dot{\theta}$ ，且 $\dot{e} = -\dot{\theta}$ ：

$$\dot{V} = \dot{\theta}(K_p e - K_d \dot{\theta}) + K_p e(-\dot{\theta}) = K_p e \dot{\theta} - K_d \dot{\theta}^2 - K_p e \dot{\theta} = -K_d \dot{\theta}^2$$

彈性項抵消，只剩阻尼項。

$$V = -4\theta^2 \leq 0$$

半負定。能量持續下降。

解 (d) : $V = 0$ 僅當 $\theta = 0$ 。考慮 $E = \{(e, \theta) : \theta = 0\}$ 。若系統停留在 E 中，則 $\dot{\theta} \equiv 0$ 且 $\ddot{\theta} \equiv 0$ 。代入閉環方程：

$$0.24(0) + 4(0) - 20e = 0 \Rightarrow e = 0$$

E 中唯一的不變集是原點。

由 LaSalle 原理，系統**全域漸近穩定**——無論初始角度為何（即使桿子一開始朝下、目標朝上），最終都會收斂到目標。

解 (e) : 這確實是 19.7 節定理的單自由度特例。對照可知：

- $M(q) \rightarrow I$ ：單自由度時慣量為常數，故 $\dot{M} = 0$ 。
- $C(q, \dot{q}) \rightarrow 0$ ：只有一個關節，沒有科氏力與離心力的耦合。
- $\dot{M} - 2C = 0 - 0 = 0$ ：斜對稱項自動為零（零矩陣既對稱又斜對稱），因此在 19.7.4 節中「消失」的那一項，在單自由度情形下根本不會出現。

這說明了為何多自由度的證明需要斜對稱性：單自由度時運氣好（該項為零），多自由度時該項不為零，必須靠斜對稱性才能消掉。第 13 章那個「看似無用」的性質，正是為此而存在。

程式範例：可控性檢查與 Lyapunov 函數的數值驗證

以下 Python 程式實作可控性與可觀測性的檢查、極點配置、以及 Lyapunov 函數沿軌跡的數值驗證——直接「看見」能量下降的過程。

```
import numpy as np
from scipy.linalg import solve_continuous_are

# ===== 可控性與可觀測性 =====
def controllability_matrix(A, B):
    n = A.shape[0]
    cols = [B]
    for i in range(1, n):
        cols.append(A @ cols[-1])
    return np.hstack(cols)

def observability_matrix(A, C):
    n = A.shape[0]
    rows = [C]
    for i in range(1, n):
```

```

        rows.append(rows[-1] @ A)
    return np.vstack(rows)

def check_system(name, A, B, C=None):
    n = A.shape[0]
    Cm = controllability_matrix(A, B)
    r = np.linalg.matrix_rank(Cm)
    print(f"{name}")
    print(f"    可控性矩陣秩 = {r} / {n} → ")
    print(f"        f'{'完全可控' if r == n else '不可控 ( 有 %d 個狀態控制不到 )' % (n-r)}")
    if C is not None:
        Om = observability_matrix(A, C)
        ro = np.linalg.matrix_rank(Om)
        print(f"    可觀測性矩陣秩 = {ro} / {n} → ")
        print(f"        f'{'完全可觀測' if ro == n else '不可觀測'}")
    eig = np.linalg.eigvals(A)
    unstable = [e for e in eig if e.real > 1e-9]
    print(f"    開迴路特徵值 = {np.round(eig, 4)} → ")
    print(f"        f'{'不穩定 (%d 個正實部)' % len(unstable) if unstable else '穩定'}")

print("=== 例題 19-1：可控性判定 ===")
# (a) 一維點質量
check_system("(a) 一維點質量 m=2",
              np.array([[0., 1.], [0., 0.]]),
              np.array([[0.], [0.5]]))
# (b) 解耦系統，輸入影響不到第二狀態
check_system("(b) 解耦系統 ( B 第二列為零 )",
              np.array([[1., 0.], [0., 2.]]),
              np.array([[1.], [0.]])
# (c) 一般二階系統
check_system("(c) 二階系統",
              np.array([[0., 1.], [-2., -3.]]),
              np.array([[0.], [1.]])

# ===== 極點配置 =====
print("\n=== 極點配置：把不穩定系統搬到左半平面 ===")
A = np.array([[0., 1.], [2., 0.]])      # 特徵值 ±1.414 · 不穩定
B = np.array([[0.], [1.]])
print(f"開迴路特徵值: {np.round(np.linalg.eigvals(A), 4)} ← 不穩定")

def place_poles_2nd(A, B, desired):
    """二階單輸入系統的極點配置 ( 可控標準型法 ) """
    # 期望特徵多項式 s^2 + a1 s + a0
    a1 = -(desired[0] + desired[1])
    a0 = desired[0] * desired[1]
    # 開迴路特徵多項式係數
    tr, det = np.trace(A), np.linalg.det(A)
    # 對此結構 A=[[0,1],[a,b]], B=[[0],[1]] · K 可直接推得
    return np.array([[a0 + A[1,0], a1 + A[1,1]]])

```

```

for poles in ([-2., -3.], [-5., -6.]):
    K = place_poles_2nd(A, B, poles)
    eig_cl = np.linalg.eigvals(A - B @ K)
    print(f" 期望極點 {poles}: K = {np.round(K.flatten(), 3)}  "
          f"實際閉環極點 = {np.round(np.sort(eig_cl.real), 4)}")

# ===== Lyapunov 函數的數值驗證 =====
print("\n=== 例題 19-3：單擺 PD+重力補償的 Lyapunov 驗證 ===")
m, L, g = 2.0, 0.6, 9.8
I = m*L**2/3
Kp, Kd = 20.0, 4.0
theta_d = np.deg2rad(90.0)          # 目標：水平

def closed_loop(state):
    """閉環動態：I·θ̈ + Kd·θ̇ - Kp·e = 0"""
    th, thd = state
    e = theta_d - th
    thdd = (Kp*e - Kd*thd) / I
    return np.array([thd, thdd])

def lyapunov(state):
    th, thd = state
    e = theta_d - th
    return 0.5*I*thd**2 + 0.5*Kp*e**2

def lyapunov_dot(state):
    """理論值 Ḃ = -Kd·θ̇²"""
    return -Kd * state[1]**2

print(f"I = {I:.4f} kg·m², Kp = {Kp}, Kd = {Kd}")
print(f"\n{'時間':>7}{ 'θ (度)':>10}{ 'θ̇':>10}{ 'V':>12}"
      f"{'Ḃ(數值)':>12}{ 'Ḃ(理論)':>12}")

for th0_deg in (-90.0, 200.0):      # 兩個差異極大的初始角
    print(f"\n 初始角度 = {th0_deg}° (目標 90°)")
    s = np.array([np.deg2rad(th0_deg), 0.0])
    dt = 1e-4
    V_prev = lyapunov(s)
    for k in range(int(3.0/dt)):
        sd = closed_loop(s)
        s = s + sd*dt
        if (k+1) % 5000 == 0:
            V = lyapunov(s)
            Vdot_num = (V - V_prev)/(5000*dt)
            print(f"{{(k+1)*dt:6.2f}}s{{np.rad2deg(s[0]):10.3f}"
                  f"{{s[1]:10.4f}}{V:12.5f}}{Vdot_num:12.5f}"
                  f"{{lyapunov_dot(s):12.5f}}")
            V_prev = V
    print(f"  → 最終誤差 = {np.rad2deg(theta_d - s[0]):.6f}°  "
          f" ( 收斂到目標 · 驗證全域漸近穩定 ) ")

```



```
# ===== 驗證  $\dot{V}$  恆為非正 =====
print("\n=== 驗證  $\dot{V} \leq 0$  在整個狀態空間成立 ===")
worst = 0.0
for th in np.linspace(-2*np.pi, 2*np.pi, 200):
    for thd in np.linspace(-10, 10, 200):
        worst = max(worst, lyapunov_dot(np.array([th, thd])))
print(f" $\dot{V}$  在取樣範圍內的最大值 = {worst:.2e} "
      f"(應  $\leq 0$ ，僅在  $\theta' = 0$  時等於 0)")

# ===== LQR 的價值函數即 Lyapunov 函數 =====
print("\n=== 19.8.2 節驗證：LQR 的  $V = x^T P x$  為 Lyapunov 函數 ===")
A2 = np.array([[0., 1.], [0., 0.]])
B2 = np.array([[0.], [0.5]])
Q, R = np.diag([1., 0.]), np.array([[1.]])
Pm = solve_continuous_are(A2, B2, Q, R)
K2 = np.linalg.solve(R, B2.T @ Pm)
Acl = A2 - B2 @ K2
print(f"P = \n{np.round(Pm, 4)}")
print(f"P 是否正定: {np.all(np.linalg.eigvals(Pm) > 0)}")
# 理論： $\dot{V} = -x^T(Q + K^T R K)x$ 
S_theory = Q + K2.T @ R @ K2
S_actual = -(Acl.T @ Pm + Pm @ Acl) # 由  $\dot{V} = x^T(Acl^T P + P Acl)x$ 
print(f"Q + K^T R K = \n{np.round(S_theory, 4)}")
print(f"-(Acl^T P + P Acl) = \n{np.round(S_actual, 4)}")
print(f"兩者相等: {np.allclose(S_theory, S_actual)} "
      f"→ 驗證  $\dot{V} = -x^T(Q + K^T R K)x$ ")
```

請試著修改：（一）把 K_d 設為 0（去掉阻尼），觀察 V 是否停止下降——這會重現第 16.3.2 節的永久振盪。（二）把單擺的初始角度設為 720 度（轉了兩圈），驗證全域穩定性確實成立。（三）把可控性檢查用在第 18 章的倒立擺 A 、 B 矩陣上，驗證它是完全可控的——這正是 LQR 能穩定它的前提。

實作：穩定性的實驗驗證

本章實作以模擬為主，目標是「親眼看見」抽象的穩定性概念。

- 1. 可控性檢查工具：**為你的系統（單關節、平面 2R、或倒立擺）建立狀態空間模型，實作可控性與可觀測性矩陣的計算，並判斷其秩。試著故意「弄壞」系統（如把 B 的某一列設為零），觀察秩如何下降。
- 2. 穩定性等級的視覺化：**分別模擬三個系統——無阻尼單擺（Lyapunov 穩定）、有阻尼單擺（漸近穩定）、線性 PD 閉環（指數穩定）。在同一張圖上繪出三者的狀態隨時間變化，直觀比較三種穩定性的差異。

3. **Lyapunov 函數的數值驗證**：對第 17 章的 PD 加重力補償控制器，計算並繪出 $V(t)$ 與 $\dot{V}(t)$ 的曲線。驗證 V 單調下降、 $\dot{V}$ 恆為非正、且 $\dot{V}$ 只在速度為零的瞬間觸及零。
4. **相平面軌跡與能量等高線**：在相平面 (e 對 $\dot{e}$) 上繪出 Lyapunov 函數的等高線 (同心橢圓)，再疊上系統的實際軌跡。你會看到軌跡持續穿越等高線往內收斂——這是本章最直觀的視覺化。
5. **吸引域的探索**：對倒立擺的 LQR 控制器，在「初始角度 $\times$ 初始角速度」平面上大量取樣，標記哪些初始條件能成功穩定。繪出吸引域的邊界，並比較不同 Q 、 R 設定下吸引域的大小。
6. **取樣週期的影響**：把連續時間的控制器離散化，逐步加大取樣週期 (1 ms、5 ms、20 ms、50 ms.....)，記錄系統從穩定變成不穩定的臨界週期。與 19.9.3 節的經驗法則比較。
7. **模型誤差下的穩定性**：故意讓控制器的重力模型有 20%、50%、100% 的誤差，觀察穩態誤差如何增大、以及系統是否仍然穩定 (應該仍然穩定，只是有穩態誤差)。這驗證了 19.8.4 節的 ISS 性質。
8. **斜對稱性的數值驗證**：對平面 2R 手臂，隨機取樣多組 $(q, \dot{q})$ ，數值計算 $\dot{M}$ 與 C ，驗證 $x^T(\dot{M} - 2C)x \approx 0$ 對任意向量 x 成立。這是第 13 章性質的直接驗證。

表 19-5 穩定性驗證記錄表 (第一列為範例)

測試項	系統 / 設定	理論預期	模擬結果	判定
Lyapunov 遞減	PD+重力補償, $K_d=4$	$\dot{V} = -4\dot{\theta}^2 \leq 0$	最大 $\dot{V} = -1.2e-9$	通過
(自行填寫其餘測試)				

國中小也看得懂：彈珠一定會滾到碗底

想像你把一顆彈珠放進一個碗裡。不管你從碗的哪一邊放進去，彈珠最後都會滾到碗底停下來，對不對？

為什麼你這麼確定？你並沒有算出彈珠會走什麼路線、轉幾圈、花多久時間——你只知道一件事：彈珠會一直往低的地方滾。只要它一直在下降，它就一定會到最低的地方。

這一章教的就是這個道理，只是用在機器人上。工程師想知道「我的機器人會不會失控」，但如果要算出它每一秒鐘的位置，會複雜到算不出來。所以他們改用彈珠的方法：

第一步：找一個「高度」——通常是機器人的**能量**（動得越快、離目標越遠，能量就越高）。

第二步：證明這個能量**一直在減少**。

第三步：既然能量一直減少，機器人最後一定會停在「能量最低」的地方——也就是我們要它去的位置！

這個方法是一百三十年前一位俄國數學家想出來的，聰明的地方在於：**不用知道路線，只要知道方向是往下的就夠了。**

還有一件重要的事：這一章還教我們問一個更基本的問題——「這台機器**有沒有可能**被控制？」想想看，如果一台遙控車的兩個輪子被黏在一起，不管你的遙控器多厲害，它都轉不了彎。這不是遙控器的問題，是車子本身的問題。所以聰明的工程師會先檢查：**我想去的每個地方，這台機器都有辦法去嗎？** 確定可以，才開始寫程式。

叫獸提醒與常見錯誤

- 誤區一：跳過可控性檢查就開始設計控制器。不可控是系統的先天缺陷，任何演算法都無法彌補，應在設計前先確認。
- 誤區二：混淆連續與離散系統的穩定區域。連續系統看左半平面（實部為負），離散系統看單位圓內（絕對值小於 1）。
- 誤區三：以為 Lyapunov 穩定就代表會收斂。它只保證「不會跑遠」，收斂需要漸近穩定，速度保證則需要指數穩定。
- 誤區四：計算 V 時忘記代入系統方程。 $\dot{V} = (\partial V / \partial x)^T \cdot f(x)$ 中的 $f(x)$ 必須是實際的系統動態，否則計算毫無意義。
- 誤區五： V 只是半負定就宣稱漸近穩定。需要 LaSalle 不變性原理才能完成證明。
- 誤區六：以為線性化的結果就是全部。它只給出局部結論，且當特徵值實部為零時完全無法判定（如例題 19-2）。
- 誤區七：忽略取樣週期對穩定性的影響。取樣太慢等效於引入延遲，會使原本穩定的系統變得不穩定。
- 誤區八：以為自適應控制的參數一定會收斂到真值。只有在持續激勵條件滿足時才會，否則

只保證追隨誤差收斂。

- 誤區九：把奇異點與不可控混為一談。前者是瞬時的位形問題（離開就恢復），後者是結構性的先天缺陷。
- 誤區十：只追求漸近穩定而忽略收斂速率。實務上往往需要指數穩定的速率保證，這是 CTC 優於 PD 加重力補償之處。
- 誤區十一：以為有模型誤差就不穩定。多數控制器具備 ISS 性質——有界的模型誤差只造成有界的追隨誤差，系統仍然穩定。

課堂討論

1. 開頭故事中叫獸說「工程師和玩家的差別，在於能不能證明」。請討論：在什麼情況下，模擬測試不足以取代理論證明？請舉出兩個機器人上的具體情境。
2. 19.7 節的證明完全不需要知道 $M(q)$ 與 $C(q, \dot{q})$ 的具體形式，只用到「 M 正定」與「 $\dot{M}-2C$ 斜對稱」兩個結構性質。請討論這種「只靠結構性質的證明」有什麼工程價值？
3. 第 13 章的 $\dot{M} - 2C$ 斜對稱性在當時看似無用，兩百頁後成為證明的關鍵。請回顧本書，找出另一個類似的「伏筆」——某個當時看似次要、後來卻成為關鍵的性質。
4. LQR 的價值函數恰好就是 Lyapunov 函數，這使得「最佳」與「穩定」成為同一件事的兩面。請討論這個巧合的深層原因（提示：兩者都與「能量 / 代價的遞減」有關）。
5. 例題 19-2 的系統漸近穩定但非指數穩定，收斂速度為 $t^{-1/2}$ 。請討論：在機器人控制中，這種「會到但很慢」的收斂在什麼場合可以接受？什麼場合不行？

章末摘要

- 可控性回答「系統能否被帶到任意狀態」；可控性矩陣 $C = [B, AB, \dots, A^{n-1}B]$ 滿秩即為完全可控。
- 不可控是結構性的先天缺陷，任何控制器都無法彌補；這與第 12 章的奇異點（瞬時位形問題）本質不同。
- 可觀測性回答「能否由輸出推斷完整狀態」，與可控性形成對偶： (A, B) 可控 $\Leftrightarrow (A^T, B^T)$ 可觀測。
- 穩定性有三個層次：Lyapunov 穩定（有界）、漸近穩定（收斂）、指數穩定（有收斂速率）。

保證)。

- 線性系統的判準：連續系統看特徵值實部是否為負，離散系統看特徵值絕對值是否小於 1。
- 若系統完全可控，則可用極點配置把閉環特徵值放到任意位置；LQR 則以權重間接決定極點且保證裕度。
- Lyapunov 直接法：若能找到正定的 V 且 $\dot{V} \leq 0$ ，則系統穩定；不需解微分方程，適用於非線性系統。
- 計算 $\dot{V}$ 時必須代入系統方程，這是把系統動態「灌入」證明的關鍵步驟。
- V 只是半負定時，需用 LaSalle 不變性原理：軌跡收斂到 $\{V = 0\}$ 中最大的不變集。
- 機械系統的能量是天然的 Lyapunov 候選函數：正定、在平衡點最小、且有耗散時遞減。
- PD 加重力補償的穩定性證明：取 $V = \frac{1}{2} \dot{q}^T M \dot{q} + \frac{1}{2} e^T K_p e$ ，計算 $\dot{V}$ 時 $\dot{M} - 2C$ 的斜對稱性使該項消失，得 $\dot{V} = -\dot{q}^T K_d \dot{q} \leq 0$ ，再由 LaSalle 完成證明。
- 此證明只用到「 M 正定」與「 $\dot{M} - 2C$ 斜對稱」兩個結構性質，完全不需要參數的具體數值——這正是定理強大之處。
- 計算力矩控制的閉環為線性，可直接用特徵值判準證明指數穩定；LQR 的價值函數 $V = x^T P x$ 本身即為 Lyapunov 函數，且 $\dot{V} = -x^T (Q + K^T R K) x$ 。
- 取樣週期過長等效於引入延遲，會侵蝕穩定裕度；建議取樣頻率為閉環頻寬的 20 ~ 30 倍。
- 自適應控制保證追隨誤差收斂，但參數估計的收斂需要額外的持續激勵條件。
- 模型誤差下多數控制器具 ISS 性質：有界的誤差只造成有界的偏差，系統仍然穩定。

觀念題與計算題

觀念題

1. 定義可控性，並說明為何應在設計控制器之前先檢查它。
2. 寫出可控性矩陣的形式，並解釋 B 、 AB 、 A^2B 各代表什麼「方向」。
3. 為何可控性矩陣只需算到 $A^{n-1}B$ ？
4. 舉出兩個不可控系統的例子，並說明其物理成因。
5. 比較不可控性與第 12 章的奇異點，說明兩者的本質差異。
6. 說明可觀測性的意義，以及它與可控性的對偶關係。

7. 區分 Lyapunov 穩定、漸近穩定與指數穩定，並各舉一例。
8. 說明連續與離散系統穩定性判準的差異，並解釋兩者為何以 $\lambda_d = e^{(\lambda_c T)}$ 對應。
9. 敘述 Lyapunov 直接法的三個條件，並說明各自的物理意義。
10. 為何計算 V 時必須代入系統方程？若不代入會發生什麼？
11. 敘述 LaSalle 不變性原理，並說明它為何在機械系統的證明中幾乎必定會用到。
12. 為何能量是機械系統天然的 Lyapunov 候選函數？請由三個性質說明。
13. 說明 $\dot{M} - 2C$ 的斜對稱性在 19.7 節證明中扮演的角色，以及其物理意義。
14. 為何說「LQR 的價值函數就是 Lyapunov 函數」？ V 的表達式有何直觀意義？
15. 說明取樣週期對穩定性的影響機制，並給出實務上的選擇準則。
16. 什麼是輸入—狀態穩定 (ISS)？為何說它比純粹的漸近穩定更符合實務需求？

計算題

1. 某系統 $A = \begin{bmatrix} 0 & 1 \\ -4 & -5 \end{bmatrix}$ 、 $B = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$ 、 $C = [1, 0]$ 。(a) 計算可控性矩陣並判斷可控性。(b) 計算可觀測性矩陣並判斷可觀測性。(c) 計算開迴路特徵值並判斷穩定性。(d) 若要把閉環極點配置到 -4 與 -6 ，求所需的 K (提示：比較特徵多項式係數)。
2. 考慮非線性系統 $\dot{x} = -x + x^3$ 。(a) 求所有平衡點。(b) 在 $x = 0$ 處線性化並判斷局部穩定性。(c) 取 $V = \frac{1}{2}x^2$ 計算 $\dot{V}$ ，並找出 $\dot{V} < 0$ 成立的 x 範圍。(d) 由此估計原點的吸引域，並說明為何此系統不是全域穩定。
3. 某單關節手臂 $I = 0.5 \text{ kg}\cdot\text{m}^2$ 、重力力矩為 $mg(L/2)\sin\theta$ 且 $mg(L/2) = 8 \text{ N}\cdot\text{m}$ 。採用 $\tau = K_p(\theta_d - \theta) - K_d\dot{\theta} + 8\sin\theta$ ，取 $K_p = 30$ 、 $K_d = 6$ 。(a) 寫出閉環方程。(b) 構造 Lyapunov 函數 V 並寫出其表達式。(c) 計算 $\dot{V}$ 並驗證其為半負定。(d) 計算閉環系統的特徵值，判斷是欠阻尼、臨界阻尼還是過阻尼。
4. 某控制系統的閉環頻寬設計為 $\omega_n = 30 \text{ rad/s}$ 。(a) 換算為赫茲。(b) 依 19.9.3 節的建議，求最低與建議的取樣頻率。(c) 若實際使用 100 Hz 取樣，判斷是否足夠。(d) 若系統的機構共振頻率為 80 Hz ，討論取樣頻率是否還需提高，並說明理由。

動手做與專題延伸

本章實作：穩定性分析工具箱

延續前面章節的程式庫，加入穩定性分析模組。這將是驗證所有控制器設計的標準工具。

- **階段一**：實作可控性與可觀測性矩陣的計算與秩判定，並整合為「系統體檢報告」函式。
- **階段二**：實作極點配置（可用 `scipy.signal.place_poles`），並與 LQR 的結果比較。
- **階段三**：實作 Lyapunov 函數的數值驗證工具——給定 V 的表達式與系統動態，在狀態空間中大量取樣，檢查 $V > 0$ 與 $V \leq 0$ 是否處處成立。
- **階段四**：實作吸引域的估計：對非線性系統大量取樣初始條件，判定哪些能收斂，繪出吸引域的邊界。
- **階段五**：實作離散化與臨界取樣週期的自動搜尋，驗證 19.9.3 節的經驗法則。

專題延伸

- **相平面與能量等高線的視覺化**：對第 17 章的 PD 加重力補償控制器（單關節版本），在相平面上繪製 Lyapunov 函數的等高線（同心橢圓），並疊上多條由不同初始條件出發的軌跡。你會看到所有軌跡都持續往內穿越等高線、螺旋收斂到原點。這張圖是本章所有理論最直觀的呈現，建議做成動畫。
- **吸引域的比較研究**：對倒立擺，分別用（甲）LQR、（乙）極點配置（極點放得較遠）、（丙）極點配置（極點放得較近）三種方法設計控制器，各自繪出吸引域。比較「控制越積極，吸引域是否越大」，以及代價（最大控制量）如何變化。這個實驗能量化「性能與強健性」的取捨。
- **進階：斜對稱性的完整驗證與其重要性**。對平面 2R 手臂，用符號運算（SymPy）推導 $M(q)$ 與採 Christoffel 符號定義的 $C(q, \dot{q})$ ，計算 $\dot{M} - 2C$ 並驗證其斜對稱性。接著做一個對照實驗：故意使用一個「錯誤的」 C 矩陣分解（仍滿足 $C \cdot \dot{q}$ 正確，但不具斜對稱性），重新執行 19.7 節的證明步驟，觀察 V 中會殘留什麼項、以及這是否會破壞穩定性結論。這個實驗能讓你深刻理解為何 C 的「特定分解方式」如此重要。

參考文獻與延伸閱讀

- Khalil, H. K., Nonlinear Systems, 3rd ed., Prentice Hall（非線性系統與 Lyapunov 理論的權威教材）。
- Slotine, J.-J. E. and Li, W., Applied Nonlinear Control, Prentice Hall（工程導向的非線性控制，含機器人應用）。

- Takegaki, M. and Arimoto, S., "A New Feedback Method for Dynamic Control of Manipulators," ASME J. Dynamic Systems, 1981 (PD 加重力補償的原始證明) 。
- Kalman, R. E., "On the General Theory of Control Systems," IFAC Congress, 1960 (可控性與可觀測性概念的提出) 。
- Lyapunov, A. M., The General Problem of the Stability of Motion, 1892 (中譯與英譯本；穩定性理論的原始著作) 。
- LaSalle, J. P., "Some Extensions of Liapunov's Second Method," IRE Trans. Circuit Theory, 1960 (不變性原理) 。
- Spong, M. W., Hutchinson, S. and Vidyasagar, M., Robot Modeling and Control, 2nd ed., Wiley (第 6、8 章：穩定性與非線性控制) 。
- Ortega, R. and Spong, M. W., "Adaptive Motion Control of Rigid Robots: A Tutorial," Automatica, 1989 (機器人自適應控制的穩定性分析) 。
- Sontag, E. D., "Smooth Stabilization Implies Coprime Factorization," IEEE Trans. Automatic Control, 1989 (輸入—狀態穩定 ISS 概念的提出) 。
- Tedrake, R., Underactuated Robotics, MIT 課程講義 (線上公開) (含 Lyapunov 方法在欠驅動機器人上的大量應用與吸引域分析) 。

第 20 章

力控制、阻抗控制與柔順操作

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩格對照照片：上格是夾爪以位置控制夾生雞蛋、蛋殼碎裂蛋液流出，旁註「位置誤差 $0.5\text{ mm} \rightarrow$ 接觸力 1500 N 」；下格是同一夾爪以阻抗控制輕握完好的雞蛋，旁註「 $K_d = 500\text{ N/m} \rightarrow$ 接觸力 0.25 N 」。右半為機器人末端與虛擬平衡點之間以三根粗細不同的彈簧與阻尼器相連的示意圖，下方橫軸為剛度的連續頻譜，左端標「純力控制」、右端標「純位置控制」，中間標出協作、裝配、研磨等應用的位置。標題文字：不控制位置，也不控制力——控制兩者之間的關係。

叫獸開講 「拿雞蛋不能只知道位置，還必須知道用了多少力。」

叫獸在桌上放了一顆生雞蛋，然後請小翰用機器手臂把它夾起來。

小翰很有信心。他量好了雞蛋的寬度是 44 公釐，於是把夾爪的目標位置設為 43.5 公釐——留 0.5 公釐的「夾持餘量」。按下執行，夾爪緩緩合攏——「啪」的一聲，蛋黃流了一桌。

「再給你一次機會。」叫獸換上第二顆蛋。小翰改設 44.2 公釐，這次夾爪停在蛋殼外側，完全沒碰到；他把手臂抬起來，蛋留在桌上。

「所以，」叫獸說，「44.2 太鬆、43.5 太緊。你要不要試試 43.9？」

小翰試了。這次蛋沒破，但也夾不穩——抬到一半就滑了下來。他開始試 43.8、43.85、43.87.....

「停。」叫獸打斷他，「你發現問題了嗎？你在用位置去控制一件本質上是力的事情。」

他在白板上寫了一個計算：「雞蛋殼的剛度大約是每公釐 3000 牛頓。如果你的位置指令比實際接觸點深了 0.5 公釐，接觸力會是多少？」

「3000 乘以 0.5.....1500 牛頓？」小翰倒抽一口氣，「那不是可以壓碎石頭了嗎。」

「這就是為什麼蛋會破。」叫獸點頭，「而且更糟的是——你的位置控制器不知道自己壓爆了蛋。它只看到『還差 0.5 公釐沒到位』，於是繼續加大力矩。第 17 章我們花了整章讓它精確追隨位置，現在這個優點反而變成致命傷。」

小紓舉手：「那改成控制力就好了？」

「一半對。」叫獸把雞蛋放回桌上，「但想想：在夾爪還沒碰到蛋之前，你要控制什麼力？此時無論怎麼下指令，力都是零——控制器會發瘋似地往前衝，直到撞上為止。純力控制在自由空間中是不穩定的。」

他停頓了一下，「所以真正的答案不是『控制位置』也不是『控制力』，而是控制兩者之間的關係。」

叫獸伸出手，輕輕握住那顆蛋，然後說：「你看我的手。我沒有『把手指移到 43.9 公釐』，也沒有『輸出 2 牛頓的力』。我做的是——讓我的手像一個軟硬適中的彈簧。碰到硬的東西就退讓，碰到軟的東西就順從。這個『像什麼樣的彈簧』，就是今天要學的**阻抗**。」

本章為什麼重要

第三篇前七章的所有控制方法，都建立在一個隱含假設上：**機器人在自由空間中運動，不與環境接觸**。這個假設下，位置控制越精確越好，增益越高越理想。

但機器人的價值恰恰在於接觸。抓取、裝配、研磨、拋光、擦拭、與人握手——這些任務中，機器人必須與環境交換力。而一旦接觸發生，前面的邏輯就完全反轉：**高增益的精確位置控制，反而成為最危險的設定**。原因如開頭故事所示——環境的剛度極高，微小的位置誤差會產生巨大的接觸力。

本章要建立三個層次的能力。**第一，直接力控制**：讓機器人輸出指定的接觸力。**第二，混合位置 / 力控制**：在不同的方向分別控制運動與力（例如在垂直方向壓住、在水平方向移動）。**第三，阻抗控制**：不直接控制位置或力，而是控制兩者的**動態關係**——讓機器人表現得像一個程式的彈簧與阻尼器。這是最優雅也最實用的方法，是協作機器人、力覺裝配與人機互動的共同基礎。

本章也是第 11 章對偶性、第 17 章操作空間控制、第 19 章穩定性理論的共同匯流處——這三個工具將在此完全發揮作用。

學習目標

- 能說明位置控制在接觸任務中失效的原因，並量化其接觸力。
- 能說明純力控制在自由空間中不穩定的原因。
- 能區分力控制、位置控制與阻抗控制三者的本質差異。
- 能說明力 / 力矩感測的三種方式及其優缺點。
- 能設計直接力控制器並說明其積分項的必要性。
- 能運用自然約束與人為約束分析任務，建構選擇矩陣。
- 能設計混合位置 / 力控制器並說明其在操作空間中的實作。
- 能推導阻抗控制律，並說明質量、阻尼、剛度三個參數的物理意義。
- 能區分阻抗控制與導納控制，並依系統條件選擇適當者。
- 能說明接觸穩定性問題與剛度上限的成因。

- 能說明主動柔順與被動柔順的分工，並舉例說明 RCC 等機構。
- 能設計精密插銷入孔的柔順策略並分析其成功條件。

先備知識

第 11 章的雅可比與速度—力對偶 ($\tau = J^T F$ 是本章的基礎)、第 17 章的操作空間控制與 Λ 矩陣、第 13 章的動力學模型、第 19 章的穩定性概念、第 6.4 節的力覺感測器。

本章的數學不比前幾章困難，但**概念的轉換較大**——必須從「控制位置」的思維切換到「控制關係」。建議讀法：務必先徹底理解 20.1 節為何位置控制會失效，這是後續所有方法的動機來源。

核心名詞中英對照

表 20-1 本章核心名詞

中文	英文	說明
力控制	Force Control	控制機器人與環境間接觸力的方法
柔順性	Compliance	受力時產生位移的性質，剛度的倒數
剛度	Stiffness	單位位移所產生的力
阻抗	Impedance	描述力與運動之間動態關係的算子
導納	Admittance	阻抗的反向關係：由力決定運動
環境剛度	Environment Stiffness	接觸物體的剛度特性
自然約束	Natural Constraint	由環境幾何決定、無法違背的限制
人為約束	Artificial Constraint	由任務指定、可自由選擇的目標
選擇矩陣	Selection Matrix	指定各方向由位置或力控制的對角矩陣
混合控制	Hybrid Control	不同方向分別做位置與力控制的架構
主動柔順	Active Compliance	以控制軟體實現的柔順性
被動柔順	Passive Compliance	以機構彈性實現的柔順性
遠端順應中心	Remote Center of Compliance, RCC	使旋轉中心落在工件尖端的順應機構
接觸不穩定	Contact Instability	接觸時產生高頻振盪的現象
彈跳	Bouncing	接觸與脫離反覆交替的現象
插銷入孔	Peg-in-Hole	精密裝配的經典測試任務
倒角	Chamfer	孔口的斜面，用於引導插入

20.1 為何位置控制在接觸中失效

20.1.1 剛度的量級問題

開頭故事揭示的問題，本質是一個量級問題。考慮機器人末端與環境接觸時的力：

$$F = K_{\text{env}} \cdot \Delta x$$

接觸力等於環境剛度乘上「侵入深度」。Δx 是位置指令超過實際接觸面的量。

關鍵在於 K_{env} 的量級。下表列出常見環境的剛度：

表 20-2 常見環境的剛度量級

環境	典型剛度	產生 10 N 力所需的侵入深度	產生 100 N 所需深度
海綿、軟墊	1 ~ 10 N/mm	1 ~ 10 mm	10 ~ 100 mm
人體軟組織	5 ~ 50 N/mm	0.2 ~ 2 mm	2 ~ 20 mm
雞蛋殼	~ 3000 N/mm	0.003 mm	0.03 mm
塑膠工件	$10^3 \sim 10^4$ N/mm	0.001 ~ 0.01 mm	0.01 ~ 0.1 mm
金屬工件	$10^5 \sim 10^6$ N/mm	$10^{-5} \sim 10^{-4}$ mm	$10^{-4} \sim 10^{-3}$ mm

這張表說明了一切。對金屬工件而言，只要位置誤差達到千分之一公釐，接觸力就可能達到數百牛頓。而第 9.9.2 節告訴我們，機器人的絕對精度只有 ± 0.5 到 ± 2 公釐——比所需的精度差了三個數量級。

換句話說：用位置控制去接觸剛性環境，在物理上是不可能成功的。這不是控制器調得不夠好，而是問題的本質。

20.1.2 控制器的「盲目」

更危險的是控制器的反應。位置控制器只看位置誤差，當末端被工件擋住而無法到達目標時，它會判定「還差得遠」，於是持續加大力矩——這正是第 16 章積分飽和的物理版本。

結果有三種：輕則工件變形、中則工件或工具損壞、重則機器人本體受損或觸發過載保護緊急停機。開頭故事中的雞蛋只是最溫和的示範。

20.1.3 那麼純力控制呢？

既然位置控制不行，是否應該改用純力控制？答案是「不完全」。

在**接觸狀態**下，力控制確實有效：直接指定期望的接觸力，控制器據此調整力矩。但在**自由空間**中，力控制會出問題：

此時無論機器人怎麼移動，測到的力都是零。若期望力為 5 N，誤差恆為 5 N，控制器會持續加速前進，直到撞上東西為止——而撞上的瞬間，累積的動能會產生巨大的衝擊力。

自由空間中： $F_{\text{measured}} = 0$ ， $F_{\text{error}} = F_d - 0 = F_d \neq 0 \Rightarrow$ 控制器持續加速

純力控制在自由空間中不穩定。這是它無法單獨使用的根本原因。

因此實務上必須有一套機制，處理「接觸」與「未接觸」兩種狀態的切換——這正是 20.4 節混合控制與 20.5 節阻抗控制要解決的問題。

◎ 一個核心的觀念轉換

位置控制與力控制看似對立，其實是**同一個問題的兩個極端**。位置控制是「無限剛硬」——無論受多大的力都不退讓；力控制是「無限柔軟」——無論位置偏多少都只維持指定的力。而真實世界的互動需要的是**中間值**：像人手一樣，既能施力又能退讓。這個「中間值」就是**阻抗**，也是本章的核心概念。理解這一點，後面所有的方法都會變得自然。

20.2 力與力矩的感測

20.2.1 三種感測方式

要控制力，首先要能量測力。第 6.4 節已介紹過原理，此處從控制的角度比較三種方式。

方式一：六軸力 / 力矩感測器。安裝在末端法蘭與工具之間，直接量測三個方向的力與三個方向的力矩。優點是精度高（解析度可達 0.01 N）、頻寬高（可達 1 kHz 以上）、且直接量測到作用點附近的力。缺點是價格昂貴（數萬至數十萬元）、增加末端重量與長度、且本身有柔性會降低系統剛度。

方式二：關節力矩感測器。在每個關節內部安裝扭矩感測器。優點是能偵測作用在手臂**任何位置**的碰撞（不只末端）、不影響末端剛性、且可用於全身的碰撞偵測。缺點是成本更高（每個關節都要一個）、且要由關節力矩反推末端力需要經過雅可比（在奇異點附近精度會劣化）。這是高階協作機器人的標準配備。

方式三：馬達電流估測。如第 11.6.3 節所述，用 $F = (J^T)^+ \cdot \tau_{\text{ext}}$ 由電流反推外力。優點是零

額外成本。缺點是精度受摩擦不確定性限制（第 13.8.3 節），且高減速比會稀釋外力訊號。

表 20-3 三種力感測方式的比較

方式	精度	成本	偵測範圍	對剛性影響	典型應用
六軸力 / 力矩感測器	高 (0.01 N)	高	僅末端	略降低	精密裝配、力控研磨
關節力矩感測器	中高 (0.1 N·m)	極高	全身	無	協作機器人、人機互動
馬達電流估測	低 (1 ~ 5 N)	零	全身	無	碰撞偵測、手動導引

20.2.2 力訊號的處理

力訊號的處理有三個實務要點，處理不當會直接導致控制失敗。

要點一：重力與慣性補償。感測器量到的是「工具重量 + 慣性力 + 真實外力」的總和。必須先扣除前兩者：

$$F_{\text{external}} = F_{\text{measured}} - F_{\text{gravity}}(q) - F_{\text{inertia}}(q, \dot{q}, \ddot{q})$$

工具的重力隨姿態變化、慣性力隨加速度變化，兩者都必須即時補償。這正是第 13、14 章動力學模型的直接應用。

要點二：濾波與延遲的取舍。力訊號雜訊大（尤其是電流估測法），需要濾波；但濾波會引入延遲，而延遲會嚴重損害接觸穩定性（20.7 節）。實務上濾波器的截止頻率通常設在 50 ~ 200 Hz 之間。

要點三：座標系的一致性。感測器輸出的是在感測器自身座標系中的力。要用於控制，必須轉換到任務座標系——這需要第 8 章的座標轉換與第 11.7 節的伴隨變換（因為力矩會隨參考點改變）。

20.3 直接力控制

20.3.1 最基本的形式

直接力控制的目標是讓接觸力追隨指定值 F_d 。最直觀的作法是模仿位置控制的形式：

$$\tau = J^T [F_d + K_f (F_d - F_{\text{measured}})]$$

直接力控制。第一項是前饋（直接輸出期望的力），第二項是回授修正。

注意 J^T 的出現——這正是第 11.6 節速度—力對偶的直接應用：把任務空間中期望的力，映

射為關節空間的力矩。

20.3.2 為何需要積分項

純比例的力控制有一個穩態誤差問題。考慮環境為彈簧（剛度 K_{env} ），機器人的位置為 x 、環境表面在 x_e ，則接觸力為 $F = K_{env}(x - x_e)$ 。

在穩態下，控制器輸出的力必須恰好等於接觸力，此時若只有比例項，會有殘餘誤差。加入積分項可以消除：

$$\tau = J^T [F_d + K_f e_F + K_i \int e_F dt], \quad e_F = F_d - F$$

含積分的力控制器。積分項對力控制而言幾乎是必需的，這與位置控制中積分項只是「可選」不同。

原因在於：位置控制中，重力等常值干擾可以用模型補償（第 17 章的重力補償）；但力控制中，環境剛度 K_{env} 通常未知且會變化（不同工件、不同接觸點），無法用模型精確補償，只能靠積分項自動適應。

20.3.3 直接力控制的限制

直接力控制有三個限制，使它很少單獨使用。

限制一：無法處理自由空間（20.1.3 節）。必須有額外的邏輯偵測接觸狀態並切換控制模式，而切換的瞬間容易產生衝擊。

限制二：無法控制切線方向的運動。研磨作業需要「垂直壓住、水平移動」，純力控制只能管前者。

限制三：對環境剛度敏感。同一組增益在軟環境（如海綿）與硬環境（如金屬）上的表現差異極大，前者可能反應遲鈍、後者可能振盪。

這三個限制，分別引出了接下來的三種方法：混合控制（解決限制二）、阻抗控制（解決限制一與三）、以及被動柔順（輔助解決限制三）。

20.4 混合位置 / 力控制

20.4.1 自然約束與人為約束

Mason 於 1981 年提出了一個關鍵的分析框架，用於決定「哪個方向該控位置、哪個方向該控力」。

自然約束 (natural constraint)：由環境的幾何形狀決定，機器人**無法違背**的限制。例如末端貼在平面上時，垂直於平面的**位置**被約束（不能穿透）；沿著平面滑動時，切線方向的**力**被約束（無摩擦時為零）。

人為約束 (artificial constraint)：由任務指定、設計者可以自由選擇的目標。在自然約束「不管」的方向上，我們才能下指令。

兩者有一個優美的互補關係：

在任一方向上：自然約束了位置 $\Rightarrow$ 可人為指定力

自然約束了力 $\Rightarrow$ 可人為指定位置

互補性原理。同一個方向不可能同時控制位置與力——這在物理上是矛盾的。

以「用板擦擦黑板」為例：垂直於黑板的方向，位置被自然約束（板擦不能陷進去），因此我們可以指定**壓力**；平行於黑板的方向，力被自然約束（沒有阻擋），因此我們可以指定**移動軌跡**。這正是人類擦黑板時直覺在做的事。

表 20-4 常見接觸任務的約束分析

任務	自然約束 (位置)	自然約束 (力)	人為指定 (力)	人為指定 (位置)
平面研磨 / 擦拭	垂直平面方向	平面內兩方向	垂直壓力	平面內軌跡
軸孔插入	徑向兩方向	軸向	徑向力為零	軸向深度
旋開螺絲	軸向與徑向	繞軸旋轉	軸向壓力	旋轉角度
沿邊緣描摹	垂直邊緣方向	沿邊緣方向	接觸壓力	沿邊緣位置
拋光曲面	曲面法向	曲面切向	法向壓力	曲面上的路徑

20.4.2 選擇矩陣

把上述分析形式化，引入一個對角的**選擇矩陣** S ：

$$S = \text{diag}(s_1, s_2, \dots, s_6), \quad s_i \in \{0, 1\}$$

$s_i = 1$ 代表第 i 個方向做位置控制、 $s_i = 0$ 代表做力控制。

控制律則把兩者結合：

$$F_{\text{command}} = S \cdot F_{\text{position_control}} + (I - S) \cdot F_{\text{force_control}}$$

混合控制律。S 選出位置控制的方向、(I-S) 選出力控制的方向，兩者互補。

以平面研磨為例（設 Z 為法向）： $S = \text{diag}(1, 1, 0, 1, 1, 1)$ —— X 、 Y 方向與三個旋轉方向做位置控制，只有 Z 方向做力控制。

20.4.3 在操作空間中實作

完整的混合控制律建立在第 17.7 節的操作空間控制之上：

$$\tau = J^T [\Lambda(q) \cdot S \cdot u_x + (I - S) \cdot F_d + \mu + p]$$

混合位置／力控制。位置控制的方向經過 Λ 做動態解耦、力控制的方向直接指定期望力。

這裡可以清楚看見第 17 章操作空間控制的價值：它使各方向的動態完全解耦，因此才能在不同方向施加不同性質的控制而互不干擾。若在關節空間做混合控制，各方向會透過 $M(q)$ 互相耦合，效果會大打折扣。

20.4.4 混合控制的困難

混合控制在理論上優雅，但實務上有三個困難。

困難一：需要準確的環境幾何。選擇矩陣的方向必須與實際的接觸面法向一致。若工件擺放有 5 度的偏差，控制方向就錯了 5 度，可能導致「本該控力的方向在控位置」——後果如 20.1 節所述。

困難二：接觸狀態的切換。從自由空間接近工件時是純位置控制，接觸後才切換為混合控制。切換的瞬間若處理不當會產生衝擊。

困難三：無法處理不確定的接觸。若工件位置有誤差、或接觸點在移動（如沿著曲面），選擇矩陣必須即時更新——這需要準確的即時感測。

這些困難促使了另一條路線的發展：不試圖區分方向，而是統一控制「力與位置的關係」——這就是阻抗控制。

20.5 阻抗控制

20.5.1 控制「關係」而非控制「量」

阻抗控制的核心思想，正是開頭故事中叫獸握住雞蛋時說的那句話：不是控制位置，也不是控制力，而是控制兩者之間的關係。

具體而言，我們希望機器人的末端表現得像一個質量—彈簧—阻尼系統：

$$M_d \ddot{\tilde{x}} + B_d \dot{\tilde{x}} + K_d \tilde{x} = F_{ext} \quad , \text{ 其中 } \tilde{x} = x - x_d$$

目標阻抗關係。 x_d 是「虛擬平衡位置」、 F_{ext} 是外界施加的力。三個參數分別為期望的慣量、阻尼與剛度。

這條式子的意義極為直觀：當外力推動末端時，末端會像一個彈簧一樣退讓。退讓多少由 K_d 決定、退讓的過程有多平順由 B_d 決定、對衝擊的反應有多鈍由 M_d 決定。

關鍵的洞見是：這一條式子同時涵蓋了位置控制與力控制兩個極端：

$K_d \rightarrow \infty$: 完全不退讓 $\Rightarrow$ 純位置控制

$K_d \rightarrow 0$: 完全順從外力 $\Rightarrow$ 純力控制 (重力補償模式)

阻抗控制是一個連續的頻譜，兩端分別是位置控制與力控制。

20.5.2 控制律的推導

如何讓機器人真的表現出上述行為？從第 17.7 節的操作空間控制出發。機器人在任務空間的實際動力學為：

$$\Lambda(q) \ddot{x} + \mu(q, \dot{q}) + p(q) = F_{control} + F_{ext}$$

任務空間的動力學。 $F_{control}$ 是控制器施加的等效末端力、 F_{ext} 是外界的力。

我們希望它表現為目標阻抗 (設 $M_d = \Lambda$ ，即不改變真實慣量，這是最簡單也最常用的選擇)。比較兩式，可得所需的控制力：

$$F_{control} = \mu + p - B_d \dot{\tilde{x}} - K_d \tilde{x}$$

阻抗控制律。前兩項抵消機器人自身的非線性與重力，後兩項產生虛擬的彈簧與阻尼。

再由對偶性映射為關節力矩：

$$\tau = J^T [\mu + p - B_d(\dot{x} - \dot{x}_d) - K_d(x - x_d)]$$

阻抗控制的完整實作。注意這與第 17.7 節的操作空間控制形式幾乎相同——只是外層的控制律改為彈簧阻尼形式。

20.5.3 三個參數的物理意義與選擇

剛度 K_d ：最重要的參數。它決定「被推開多少」：

$$\text{穩態時：} \tilde{x} = F_{ext} / K_d$$

剛度越大越硬。單位為 N/m 或 N/rad 。

選擇的原則是與環境剛度匹配或更低。若 K_d 遠大於環境剛度，行為接近位置控制 (危險)；

若遠小於環境剛度，機器人幾乎不施力（無法完成任務）。實務上常取 K_d 在 $100 \sim 5000 \text{ N/m}$ 之間——請注意這比 20.1 節表中的金屬工件剛度（ 10^8 N/m ）小了四到五個數量級，這正是安全的來源。

阻尼 B_d ：決定平順度。它抑制振盪，防止接觸時的彈跳。常用的選擇是讓系統維持臨界阻尼：

$$B_d = 2\zeta\sqrt{K_d \cdot M_d}, \quad \text{通常取 } \zeta = 0.7 \sim 1.0$$

由第 16 章的二階系統理論直接套用。

慣量 M_d ：通常取為實際慣量。若刻意設定 $M_d \neq \Lambda$ ，控制器必須「主動抵消」真實慣量並「製造」虛擬慣量——這需要準確量測外力 F_{ext} ，且對感測雜訊極為敏感。因此除非有特殊需求（如讓沉重的手臂「感覺起來很輕」以利手動導引），實務上多取 $M_d = \Lambda$ 。

表 20-5 阻抗參數的選擇指引

應用場景	剛度 K_d	阻尼比 ζ	說明
自由空間精確定位	高（ $5000+ \text{ N/m}$ ）	1.0	接近位置控制
接觸剛性工件（裝配）	中低（ $100 \sim 500 \text{ N/m}$ ）	$0.7 \sim 1.0$	允許順應以吸收位置誤差
研磨、拋光	低（法向）/ 高（切向）	1.0	各方向使用不同剛度
人機協作、手動導引	極低（ $0 \sim 50 \text{ N/m}$ ）	0.7	接近零力控制
接觸軟物（人體、食品）	極低（ $10 \sim 100 \text{ N/m}$ ）	1.0	避免壓傷

20.5.4 阻抗控制的優勢

優勢一：統一處理接觸與非接觸。不需要偵測接觸狀態、不需要切換控制模式。在自由空間中， $F_{\text{ext}} = 0$ ，控制律退化為 PD 位置控制；接觸後 $F_{\text{ext}} \neq 0$ ，末端自然退讓。這徹底解決了 20.3.3 節與 20.4.4 節的切換問題。

優勢二：本質安全。剛度是設計出來的上限——無論位置誤差多大，接觸力最多是 K_d 乘上誤差。以 $K_d = 500 \text{ N/m}$ 、位置誤差 5 mm 為例，最大接觸力只有 2.5 N 。這使機器人在碰到人時不會造成傷害，是協作機器人安全性的根本保證。

優勢三：不需要力感測器（基本版本）。20.5.2 節的控制律中並沒有出現 F_{ext} ——控制器只需要位置與速度回授。這是因為「像彈簧一樣退讓」這個行為，本身就是由 PD 形式的控制律

自然產生的。（若要精確控制力或改變虛擬慣量，才需要力感測。）

優勢四：各方向可獨立設定。 K_d 是一個矩陣，可以在不同方向設定不同的剛度——這自然涵蓋了混合控制的功能，且不需要硬性的 0/1 選擇矩陣。例如插銷入孔時，軸向設高剛度（推進）、徑向設低剛度（順應對位）。

圖 20-1 阻抗控制的彈簧模型

建議配圖：中央為機器人末端執行器，其與「虛擬平衡位置 x_d 」之間以三個彈簧與阻尼器連接（分別代表 X 、 Y 、 Z 三個方向，且彈簧粗細不同以表示剛度不同）。左側標註「自由空間： $F_{ext} = 0$ ，末端停在 x_d 」，右側標註「接觸： $F_{ext} \neq 0$ ，末端退讓 $\tilde{x} = F/K_d$ 」。下方以一條橫軸表示剛度的連續頻譜，左端標「 $K_d \rightarrow 0$ ：純力控制 / 零力模式」、右端標「 $K_d \rightarrow \infty$ ：純位置控制」，中間標出各種應用的典型位置。

20.6 阻抗控制與導納控制

20.6.1 兩種實作方式

實現「力與運動的目標關係」有兩種相反的實作路徑，容易混淆但差異重要。

阻抗控制 (Impedance Control)：「量測運動，輸出力」。控制器讀取位置與速度，依目標阻抗關係算出應施加的力，再透過力矩控制輸出。

$$\text{量測 } x, \dot{x} \Rightarrow \text{計算 } F = -K_d \tilde{x} - B_d \dot{\tilde{x}} \Rightarrow \text{輸出力矩 } \tau = J^T F$$

阻抗控制的資料流。內層是力矩控制。

導納控制 (Admittance Control)：「量測力，輸出運動」。控制器讀取外力，依目標關係算出末端「應該移動到哪裡」，再交給一個高剛度的位置控制器執行。

$$\text{量測 } F_{ext} \Rightarrow \text{積分目標阻抗方程求出 } x_{ref} \Rightarrow \text{交給位置控制器追隨}$$

導納控制的資料流。內層是位置控制。

20.6.2 該選哪一種

兩者的選擇取決於**機器人本身的特性與環境**，這是一個重要的工程判斷。

阻抗控制適合：本身可反向驅動的機器人（低減速比、低摩擦——第 5 章的準直驅設計）、以及**剛性環境**。因為它的內層是力矩控制，天生就「軟」，接觸硬物時能自然退讓。

導納控制適合：高減速比、高摩擦、難以做力矩控制的機器人（多數傳統工業機器人）、以及**柔軟環境**。它需要力感測器，但只要有了感測器，即使是很「硬」的機器人也能表現出柔順行

為。

一個有用的判準是：

環境硬 + 機器人軟（可反驅）→ 阻抗控制

環境軟 + 機器人硬（高減速比）→ 導納控制

互補原則。若兩者都硬（硬機器人接觸硬環境），無論用哪種方法都很難穩定——這是接觸控制中最困難的情形。

表 20-6 阻抗控制與導納控制的比較

面向	阻抗控制	導納控制
資料流	量測運動 → 輸出力	量測力 → 輸出運動
內層控制	力矩 / 電流控制	位置控制
是否需力感測器	基本版不需要	必需
適合的機器人	可反向驅動（低減速比）	高減速比、高摩擦
適合的環境	剛性環境	柔軟環境
自由空間的表現	良好（退化為 PD）	可能因雜訊而漂移
典型代表	協作機器人、腿式機器人	傳統工業機器人加力覺感測

20.6.3 摩擦的影響

阻抗控制有一個實務上的隱憂：**關節摩擦會破壞柔順性**。若減速機的摩擦力矩相當於末端 10 N 的力，那麼小於 10 N 的外力根本推不動機器人——無論你把 K_d 設得多低，機器人「感覺起來」都是硬的。

這正是第 5 章「協作機器人減速比不能太高」與第 11.6.3 節知識框的另一個理由。因應方式有三：使用低減速比的準直驅設計、加入摩擦補償（第 17.5.3 節）、或改用導納控制（因為力感測器直接量到外力，摩擦由內層的位置控制器克服）。

20.7 接觸穩定性

20.7.1 為何接觸時容易振盪

接觸控制有一個特有的穩定性問題：**機器人在自由空間中穩定，一接觸就開始高頻振盪**。這種現象稱為接觸不穩定（contact instability），嚴重時會產生刺耳的噪音、損傷工件、甚至損壞機構。

成因可以用第 19 章的觀點理解。接觸後，環境的剛度加入了閉環系統，使等效的閉環剛度大幅提高：

$$\text{接觸前的等效剛度} = K_d$$

$$\text{接觸後的等效剛度} = K_d + K_{env} \approx K_{env} \quad (\text{因 } K_{env} \gg K_d)$$

接觸使系統的剛度暴增數個數量級，自然頻率隨之上升。

而系統的自然頻率上升後，原本足夠的取樣頻率可能變得不足（第 19.9.3 節），且原本設計的阻尼比也會下降：

$$\omega_n = \sqrt{K/M} \text{ 上升} \Rightarrow \zeta = B/(2\sqrt{KM}) \text{ 下降}$$

接觸後阻尼比下降，系統從穩定變成欠阻尼甚至不穩定。

20.7.2 三個關鍵因素

因素一：取樣延遲。數位控制器的取樣週期造成的延遲，會侵蝕相位裕度。接觸時系統頻寬提高，同樣的延遲造成的相位滯後更嚴重。這是接觸不穩定最主要的原因。

因素二：力感測器的濾波延遲。為了降低雜訊而加的低通濾波器，其延遲直接進入控制迴路。這造成一個兩難：濾波太少則雜訊大、太多則不穩定。

因素三：關節與結構的柔性。減速機的扭轉柔度、連桿的彎曲柔度，在高剛度接觸時會被激發成振動模態。這是為何研磨、拋光等作業常有明顯的顫振（chatter）。

20.7.3 可實現的剛度上限

上述分析導出一個重要的結論：**控制器能實現的最大剛度是有上限的**，且這個上限與取樣頻率、延遲、感測雜訊有關。

$$K_{max} \propto 1 / (\text{取樣週期} \times \text{總延遲})$$

大致的關係。取樣頻率越高、延遲越小，能穩定實現的剛度上限越高。

實務上的經驗值是：以 1 kHz 控制、良好的力感測器，可穩定實現的剛度上限約在數千 N/m 的量級——遠低於金屬工件的 10^8 N/m。這再次印證了 20.1 節的結論：**用純軟體控制去「模擬」高剛度接觸是不可行的。**

因應之道有三：**一、提高控制頻率與降低延遲**（成本高，且有物理極限）；**二、降低目標剛度**（犧牲精度換穩定，最常用）；**三、加入被動柔順**（在機構上加入彈性，使等效環境剛度降低

——這正是下一節的主題）。

◎ 為何研磨作業特別困難

研磨結合了本章所有的困難：環境剛度極高（金屬）、需要維持穩定的法向力（力控制）、同時需要沿曲面移動（位置控制）、且工具旋轉會產生高頻擾動。因此工業上的力控研磨幾乎都採用「主動 + 被動」的混合方案：機器人做粗略的力控制與軌跡追隨，工具端則加裝一個氣壓或彈簧式的**主動順應裝置**（active compliance device），由它負責高頻的力調節。這個分工的原理是：機器人負責大範圍、低頻的運動，順應裝置負責小範圍、高頻的力——各自做自己擅長的事。

20.8 被動柔順與 RCC

20.8.1 用機構取代控制

主動柔順（用控制器實現）有頻寬限制（20.7.3 節）；被動柔順（用機構彈性實現）則沒有這個問題——**彈簧的反應是瞬間的，不需要感測、計算與致動**。

因此在需要高頻順應的場合，被動柔順往往是更好的選擇。常見的被動柔順元件包括：彈簧、橡膠墊、氣壓缸（空氣可壓縮，天然柔順——第 5.1.2 節）、以及專門設計的柔性機構。

被動柔順的缺點是**不可調整**：彈簧一旦裝上，剛度就固定了。若任務需要在不同階段使用不同的剛度，就必須用主動柔順，或設計可切換的機構。

20.8.2 遠端順應中心（RCC）

被動柔順最著名的設計是**遠端順應中心裝置**（Remote Center of Compliance, RCC），由 Draper 實驗室於 1970 年代為軸孔裝配開發。

它的巧妙之處在於：透過特殊的彈性元件配置，使整個裝置的「順應中心」落在**裝置外部**——恰好在插銷的**尖端**位置。

為何這個設計如此關鍵？考慮插銷入孔時的兩種誤差：

平移誤差：插銷中心與孔中心有橫向偏移。此時倒角會產生一個橫向力，把插銷推正——需要**橫向的柔順**。

角度誤差：插銷有傾斜。此時需要旋轉來修正——但關鍵是**繞哪一點旋轉**？若繞裝置本體旋轉，尖端會產生額外的橫向位移，反而使對位變差；若能繞**尖端**旋轉，就能純粹修正角度而不影響已對好的位置。

RCC 正是讓旋轉中心落在尖端，使兩種修正互不干擾。這使得插銷能「自己找到」孔的位置，成功率大幅提升。

圖 20-2 RCC 的作用原理

建議配圖：左右對照。左圖為「無 RCC」——插銷傾斜時繞著法蘭處旋轉，尖端因而橫向偏移，箭頭標示「角度修正反而破壞了位置對準」。右圖為「有 RCC」——以虛線標出順應中心恰好在插銷尖端，插銷繞尖端旋轉，尖端位置不變，箭頭標示「角度與位置的修正互相獨立」。下方另附兩個小圖，顯示插銷接觸倒角時橫向力如何自動引導對位。

20.8.3 主動與被動的分工

實務上最佳的方案通常是兩者結合，各自負責擅長的部分：

表 20-7 主動柔順與被動柔順的分工

面向	主動柔順（控制）	被動柔順（機構）
剛度可調	可即時改變	固定
頻寬	受取樣與延遲限制（數十 Hz）	極高（機構的自然頻率）
可實現的剛度範圍	有上限（數千 N/m）	可極低或極高
成本	需感測器與運算	機構成本，通常較低
對衝擊的反應	有延遲，可能來不及	瞬間，天然吸收
適合處理	大範圍、低頻、需調整的順應	小範圍、高頻、固定的順應

典型的組合是：機器人用阻抗控制處理公分級的位置誤差與低頻的力調節，末端的 RCC 或彈性夾具處理零點幾公釐的高頻對位。這個「粗調靠控制、微調靠機構」的分工，是精密裝配的標準作法。

20.9 精密裝配：插銷入孔

20.9.1 為何這個任務如此經典

插銷入孔（peg-in-hole）是機器人裝配研究的標準測試任務。它看似簡單，卻濃縮了接觸控制的所有困難：位置誤差不可避免、環境剛度極高、接觸狀態不斷改變、且失敗的代價明確（卡住或損壞）。

問題的核心是一個量級落差：孔與軸的配合間隙通常只有零點幾公釐，而機器人的絕對定位精度是數公釐（第 9.9.2 節）。純靠位置精度是不可能成功的。

20.9.2 插入過程的三個階段

階段一：接近 (Approach)。末端在自由空間中移動到孔口上方。此時無接觸，阻抗控制退化為位置控制，可用較高的剛度以求快速精確。

階段二：搜尋與對位 (Search & Align)。插銷接觸到工件表面或倒角。此時需要低的橫向剛度，讓插銷能被倒角的斜面「推」到正確位置。若剛度太高，插銷會頂在表面上而不會滑入。

階段三：插入 (Insertion)。插銷進入孔中，開始沿軸向下滑。此時需要軸向適度的推力 (力控制) 與橫向的高順應 (避免卡住)。

三個階段的參數需求不同，這正是阻抗控制優於混合控制之處——只需連續調整 K_d 矩陣，不需切換控制模式。

表 20-8 插銷入孔各階段的阻抗參數

階段	軸向剛度	橫向剛度	旋轉剛度	軸向力指令	說明
接近	高	高	高	無	自由空間快速精確定位
搜尋對位	低	極低	極低	小 (1~5 N)	讓倒角引導，避免頂住
插入	中 (或力控)	極低	極低	中 (5~20 N)	軸向推進，橫向完全順從
完成確認	高	中	中	無	回到位置控制以確認深度

20.9.3 三種提高成功率的策略

策略一：倒角 (Chamfer)。在孔口做出斜面，把「精確對準」的要求轉化為「落在倒角範圍內」。倒角寬度若為 1 公釐，容許的位置誤差就從零點幾公釐放寬到 1 公釐——這是機構設計為控制減負的典型範例，成本極低而效果顯著。

策略二：螺旋搜尋 (Spiral Search)。若首次接觸未能進入孔中，讓末端在保持軸向壓力的同時，沿一條螺旋路徑緩慢移動。當經過孔口時，軸向的順應會使插銷自動落入。這是一個「用時間換空間」的策略，能大幅提升對位置不確定性的容忍度。

策略三：主動晃動 (Wiggle / Dither)。在插入過程中疊加一個小幅度的旋轉或橫向擺動。這能打破靜摩擦 (第 13.8.2 節的 stick-slip)、幫助插銷找到最順暢的方向，並避免「三點卡死」 (jamming) 的情形。

20.9.4 卡死與楔住

插入失敗有兩種模式，成因不同。

楔住 (Wedging)：插銷與孔壁在兩個接觸點上互相楔緊，此時無論施加多大的軸向力都無法推入——力越大楔得越緊。成因通常是插入初期角度誤差過大。

卡死 (Jamming)：施加的力與力矩的組合方向不對，使插銷「頂住」而無法前進。與楔住不同，卡死可以透過**改變施力方向**來解除。

兩者的區別在實務上很重要：偵測到插入失敗時，若是卡死，可以嘗試調整力矩方向或加入晃動；若是楔住，則必須退出重來。判斷的依據通常是「軸向力持續增大但深度不變」——這是楔住的特徵。

20.10 人機協作的安全

20.10.1 阻抗控制作為安全基礎

第 1 章與第 7 章提過協作機器人的安全哲學是「限制」而非「隔離」。本章的阻抗控制，正是這個哲學的技術實現。

關鍵在於 20.5.4 節的優勢二：**剛度是設計出來的上限**。以 $K_d = 300 \text{ N/m}$ 為例，即使機器人的位置指令與人體位置差了 10 公分，最大接觸力也只有 30 N——這遠低於造成傷害的門檻。相對地，一個高增益的位置控制器在同樣的情況下可能輸出數千牛頓。

20.10.2 碰撞的三個階段與對應措施

國際標準（如 ISO/TS 15066）把人機碰撞分為三個階段，各需不同的技術。

階段一：碰撞前 (Pre-collision)。以視覺或接近感測偵測人員接近，主動降速或停止。這屬於第 22、26 章的範疇。

階段二：碰撞中 (Impact)。碰撞發生的最初數十毫秒，力量由機器人的**慣性主導**——此時控制器根本來不及反應。降低這個衝擊的唯一方法是**降低有效慣量**（輕量化設計、低減速比）與**加裝被動緩衝**（軟性外殼）。這是機構設計的責任，控制無能為力。

階段三：碰撞後 (Post-impact)。控制器偵測到異常力（第 11.6.3 節的無感測器碰撞偵測，或關節力矩感測），立即切換為零力模式或反向退讓。這裡的關鍵指標是**偵測延遲**——從碰撞到反應的時間，通常要求在 10 毫秒以內。

三個階段的分工說明了一個重要觀念：**安全不是單靠控制達成的**。控制只能處理階段一與階段三；階段二的衝擊完全由機構的慣量與柔性決定。這也是第 5 章強調準直驅設計、第 4 章討論輕量化的深層原因。

20.10.3 力與壓力的區別

最後一個實務要點：安全標準規範的往往不是**力**，而是**壓力**（力除以接觸面積）。同樣是 50 N 的力，透過一個鈍圓的外殼分散開來可能完全無害，但若集中在一個尖角上就可能造成穿刺傷。

因此協作機器人的設計除了控制之外，還必須注意：外殼的圓角化、避免夾點（pinch point）、以及在可能接觸的部位加裝軟性覆蓋。這再次呼應第 7.9 節的「本質安全」——最可靠的安全來自設計本身，而非主動的控制。

20.11 由力控制邁向抓取

20.11.1 本章的成果

本章讓機器人學會了「接觸」。你現在能夠：分析位置控制在接觸任務中失效的原因並量化接觸力、比較三種力感測方式、設計直接力控制與混合位置 / 力控制、運用自然與人為約束分析任務、推導並實作阻抗控制、依系統條件選擇阻抗或導納控制、理解接觸穩定性的限制與剛度上限、以及運用主動與被動柔順的分工解決精密裝配問題。

最重要的是那個貫穿全章的觀念轉換：**從控制「量」到控制「關係」**。位置控制與力控制是同一個頻譜的兩個極端，而阻抗控制涵蓋了整個頻譜——這使它成為現代機器人與環境互動的通用語言。

20.11.2 接下來：從接觸到抓取

本章處理的是「單點接觸」——機器人用一個末端點與環境互動。但**抓取**是更複雜的問題：多個接觸點同時作用、必須考慮摩擦錐、必須判斷物體會不會滑落或旋轉。

第 21 章（抓取與操作）將處理這些問題。我們會學到：**力封閉**（force closure）——判斷一組接觸點能否抵抗任意方向的外力；**形封閉與關入籠**（caging）——不靠摩擦而靠幾何限制物體；**摩擦錐**與正線性跨度的數學；以及**抓取品質指標**——如何量化「這個抓法有多穩」。

屆時本章的柔順概念會再次出現，並帶出一個重要的洞見——第 21 章的「手套思想實驗」將說明：**在抓取中，柔順往往比精準更重要**。這與本章的結論一脈相承。

換句話說：第 20 章讓機器人學會**溫柔地碰**，第 21 章讓它學會**穩穩地抓**。

完整計算例題

例題 20-1 接觸力的量級分析

題目：某機器人以位置控制接近一個金屬工件（環境剛度 $K_{env} = 10^5 \text{ N/mm}$ ）。（a）若位置指令比實際接觸面深 0.01 mm，求接觸力。（b）若機器人的定位誤差為 $\pm 0.5 \text{ mm}$ ，求最壞情況的接觸力。（c）改用阻抗控制， $K_d = 500 \text{ N/m}$ ，重新計算（b）的接觸力。（d）比較並說明。

解（a）：直接代入 $F = K_{env} \cdot \Delta x$ 。注意單位換算：

$$F = 10^5 \text{ N/mm} \times 0.01 \text{ mm} = 1000 \text{ N}$$

僅 0.01 公釐的侵入，就產生 1000 牛頓的力（約等於 100 公斤重）。

解（b）：位置誤差 0.5 mm 時：

$$F = 10^5 \text{ N/mm} \times 0.5 \text{ mm} = 5 \times 10^4 \text{ N} = 50000 \text{ N}$$

五萬牛頓——這遠超過任何機器人的能力，實際上會先觸發過載保護、或造成工件與機構的永久損壞。

解（c）：阻抗控制下，機器人表現為一個剛度 K_d 的彈簧，與環境的彈簧**串聯**。串聯彈簧的等效剛度為：

$$1/K_{eq} = 1/K_d + 1/K_{env} \Rightarrow K_{eq} = (K_d \cdot K_{env}) / (K_d + K_{env})$$

串聯剛度公式。

代入數值（注意單位統一為 N/m： $K_d = 500 \text{ N/m}$ 、 $K_{env} = 10^8 \text{ N/m}$ ）：

$$K_{eq} = (500 \times 10^8) / (500 + 10^8) \approx 500 \text{ N/m}$$

等效剛度幾乎完全由較軟的那一個決定。這是串聯彈簧的重要性質。

$$F = 500 \text{ N/m} \times 0.5 \text{ mm} = 500 \times 0.0005 = 0.25 \text{ N}$$

接觸力僅 0.25 牛頓。

解（d）：位置控制產生 50000 N，阻抗控制產生 0.25 N——相差二十萬倍。

核心結論：當一個軟彈簧與一個硬彈簧串聯時，等效剛度接近軟的那一個。因此**只要機器人夠「軟」，環境有多硬都無所謂**。這正是阻抗控制安全性的數學根源，也解釋了為何協作機器人能安全地與人接觸。

例題 20-2 阻抗參數的設計

題目：某協作機器人需要執行「把零件輕放到桌面上」的任務。要求：(i) 接觸力不超過 20 N；(ii) 能容忍 ± 8 mm 的位置不確定性；(iii) 接觸過程不振盪。末端的等效慣量 $\Lambda = 5$ kg。(a) 求剛度 K_d 的上限。(b) 依臨界阻尼求阻尼 B_d 。(c) 計算接觸時的自然頻率，並判斷所需的最低控制頻率。(d) 若改為接觸人手（要求力不超過 5 N），重新設計 K_d 。

解 (a)：最壞情況下位置誤差為 8 mm，此時接觸力不得超過 20 N：

$$K_d \leq F_{\max} / \Delta x_{\max} = 20 \text{ N} / 0.008 \text{ m} = 2500 \text{ N/m}$$

剛度上限。取 $K_d = 2000 \text{ N/m}$ 以保留餘裕。

解 (b)：臨界阻尼 ($\zeta = 1$) 時：

$$B_d = 2\zeta\sqrt{(K_d \cdot M_d)} = 2 \times 1 \times \sqrt{(2000 \times 5)} = 2\sqrt{10000} = 2 \times 100 = 200 \text{ N}\cdot\text{s/m}$$

阻尼係數。

解 (c)：接觸後系統的剛度變為 K_d 與 K_{env} 的串聯。若桌面為木質 ($K_{\text{env}} \approx 10^6 \text{ N/m}$)，則等效剛度仍約為 2000 N/m（由較軟者主導）：

$$\omega_n = \sqrt{(K_{\text{eq}}/M_d)} = \sqrt{(2000/5)} = \sqrt{400} = 20 \text{ rad/s} \approx 3.18 \text{ Hz}$$

接觸時的自然頻率。

依第 19.9.3 節的準則，取樣頻率應為閉環頻寬的 20~30 倍：

$$f_{\text{sample}} \geq 20 \times 3.18 \approx 64 \text{ Hz} \cdot \text{建議 } 100 \text{ Hz 以上}$$

最低控制頻率。實務上力控制通常以 500~1000 Hz 執行，餘裕充足。

解 (d)：接觸人手時要求力不超過 5 N。但人手本身是柔軟的 ($K_{\text{hand}} \approx 20 \text{ N/mm} = 2 \times 10^4 \text{ N/m}$)，且人會退讓，因此位置誤差的定義不同。保守起見仍以 8 mm 計算：

$$K_d \leq 5 \text{ N} / 0.008 \text{ m} = 625 \text{ N/m}$$

接觸人體時的剛度上限，約為接觸桌面時的四分之一。

實務意義：這說明了為何協作機器人在偵測到人接近時，除了降速之外，還應**降低剛度**。許多協作機器人提供「協作模式」，其本質就是切換到較低的 K_d 。

例題 20-3 插銷入孔的容差分析

題目：某軸孔配合的規格為：孔徑 20.05 mm、軸徑 20.00 mm (間隙 0.05 mm)，孔口有 1.0 mm 寬、45° 的倒角。機器人的定位精度為 ± 0.8 mm。(a) 判斷純位置控制是否可行。(b) 加入倒角後，容許的位置誤差為多少？(c) 若定位精度仍不足，計算螺旋搜尋所需的搜尋半徑。(d) 設計此任務的阻抗參數。

解 (a)：純位置控制要求軸心與孔心的偏差小於間隙的一半：

$$\text{容許偏差} = (20.05 - 20.00)/2 = 0.025 \text{ mm}$$

而機器人的精度為 ± 0.8 mm——相差 32 倍，完全不可行。

解 (b)：有倒角時，只要軸的邊緣落在倒角範圍內，斜面就能引導它滑入。容許的偏差為：

$$\text{容許偏差} = \text{倒角寬度} + \text{間隙}/2 = 1.0 + 0.025 = 1.025 \text{ mm}$$

倒角把容許誤差放寬了 41 倍。

由於機器人精度 ± 0.8 mm < 1.025 mm，加入倒角後任務變為可行。這生動說明了 20.9.3 節「機構設計為控制減負」的價值——一個成本幾乎為零的倒角，解決了控制上極其困難的問題。

解 (c)：假設機器人精度較差 (± 1.5 mm)，超出倒角的容許範圍。此時需要螺旋搜尋。搜尋半徑應涵蓋最大可能的誤差：

$$\text{搜尋半徑} \geq \text{定位誤差} - \text{容許偏差} = 1.5 - 1.025 = 0.475 \text{ mm}$$

取 0.6 mm 以保留餘裕。螺旋的螺距應小於容許偏差的兩倍 (約 2 mm)，以確保不會「跳過」孔口。

解 (d)：依 20.9.2 節的階段分析設計參數：

- **軸向 (Z)：**插入階段用力控制，指定 10 N 的推力。或用中等剛度 $K_z = 1000$ N/m 配合位置指令。
- **橫向 (X、Y)：**極低剛度 $K_{xy} = 50 \sim 200$ N/m，讓倒角能輕易推動插銷對位。
- **旋轉 (繞 X、Y)：**極低剛度，允許插銷自由調整傾角以順應孔的方向。
- **繞 Z 旋轉：**視任務而定；若插銷有方向性 (如有鍵槽) 則需高剛度，否則可設低值。

驗證橫向剛度是否適當：倒角的斜面 (45°) 在 10 N 軸向推力下，產生的橫向分力約為 10 N。若 $K_{xy} = 200 \text{ N/m}$ ，則橫向可移動 $10/200 = 0.05 \text{ m} = 50 \text{ mm}$ ——遠大於所需的 1 mm，因此順應性充足。若 K_{xy} 設為 10000 N/m，橫向只能移動 1 mm，可能不足以完成對位。

程式範例：阻抗控制的模擬與比較

以下 Python 程式模擬機器人末端接觸環境的過程，比較位置控制與阻抗控制的接觸力差異，並示範阻抗參數對行為的影響與接觸穩定性問題。

```
import numpy as np

# ===== 一維接觸模擬環境 =====
class ContactSim:
    """末端沿一維運動，在 x_wall 處遇到剛度 K_env 的牆"""
    def __init__(s, m=5.0, x_wall=0.0, K_env=1e8, B_env=100.0):
        s.m, s.x_wall, s.K_env, s.B_env = m, x_wall, K_env, B_env

    def contact_force(s, x, v):
        """環境施加的力（僅在侵入時作用）"""
        pen = x - s.x_wall          # 侵入深度（正值代表已侵入）
        if pen <= 0:
            return 0.0
        f = s.K_env*pen + s.B_env*max(v, 0.0)
        return -f                  # 方向與運動相反

    def step(s, x, v, u, dt):
        f_ext = s.contact_force(x, v)
        a = (u + f_ext) / s.m
        v = v + a*dt
        x = x + v*dt
        return x, v, f_ext

# ===== 三種控制器 =====
def position_control(x, v, x_d, Kp=50000.0, Kd=500.0):
    """高增益位置控制（第 17 章的思維）"""
    return Kp*(x_d - x) - Kd*v

def impedance_control(x, v, x_d, Kd_imp=500.0, Bd=200.0):
    """阻抗控制（20.5 節）：像一個彈簧阻尼器"""
    return Kd_imp*(x_d - x) - Bd*v

def force_control(x, v, F_d, F_meas, Kf=0.5, Ki=20.0, state=None):
    """直接力控制（20.3 節）·含積分項"""
    e = F_d - abs(F_meas)
    state['I'] += e * state['dt']
    return -(F_d + Kf*e + Ki*state['I'])    # 負號：往牆的方向推
```

```

# ===== 模擬主程式 =====
def run(ctrl, label, x_d=0.005, T=1.5, dt=1e-5, **kw):
    """x_d = 0.005 代表指令位置比牆面深 5 mm (模擬定位誤差)"""
    sim = ContactSim()
    x, v = -0.05, 0.0 # 由牆前 50 mm 出發
    state = {'I': 0.0, 'dt': dt}
    f_max, f_final, contact_t = 0.0, 0.0, None
    for k in range(int(T/dt)):
        f_ext = sim.contact_force(x, v)
        if ctrl is force_control:
            u = ctrl(x, v, kw.get('F_d', 10.0), f_ext, state=state)
        else:
            u = ctrl(x, v, x_d, **kw)
        u = np.clip(u, -3000, 3000) # 致動器力矩上限
        x, v, f_ext = sim.step(x, v, u, dt)
        if f_ext != 0.0:
            if contact_t is None: contact_t = k*dt
            f_max = max(f_max, abs(f_ext))
            f_final = abs(f_ext)
    return label, f_max, f_final, x - sim.x_wall

print("=== 位置控制 vs 阻抗控制：接觸金屬牆 (K_env = 1e8 N/m) ===")
print("指令位置比牆面深 5 mm (模擬定位誤差)")
print(f"{'控制器':<28}{'最大接觸力':>14}{'穩態接觸力':>14}{'侵入深度':>14}")
for ctrl, label, kw in (
    (position_control, "位置控制 (Kp=50000)", {}),
    (impedance_control, "阻抗控制 (Kd=2000)", {'Kd_imp': 2000.0, 'Bd': 200.0}),
    (impedance_control, "阻抗控制 (Kd=500)", {'Kd_imp': 500.0, 'Bd': 100.0}),
    (impedance_control, "阻抗控制 (Kd=100)", {'Kd_imp': 100.0, 'Bd': 45.0})):
    lab, fmax, ffin, pen = run(ctrl, label, **kw)
    print(f"{'lab':<28}{'fmax':12.1f} N{'ffin':12.2f} N{'pen*1000':12.4f} mm")
print("> 剛度越低，接觸力越小 (例題 20-1 的驗證)")

# ===== 例題 20-1：串聯剛度 =====
print("\n=== 例題 20-1：串聯彈簧的等效剛度 ===")
K_env = 1e8
print(f"{'K_d (N/m)':>12}{'K_eq (N/m)':>14}{'誤差 0.5mm 的力':>16}")
for Kd_imp in (1e8, 1e5, 1e4, 2000, 500):
    K_eq = (Kd_imp*K_env)/(Kd_imp + K_env)
    print(f"{'Kd_imp':12.0f}{'K_eq':14.1f}{'K_eq*0.0005':14.2f} N")
print("> 等效剛度幾乎完全由較軟者決定")

# ===== 例題 20-2：阻抗參數設計 =====
print("\n=== 例題 20-2：阻抗參數設計 ===")
Lam, F_max, dx_max = 5.0, 20.0, 0.008
Kd_lim = F_max/dx_max
print(f"剛度上限 K_d ≤ {F_max}/{dx_max} = {Kd_lim:.0f} N/m → 取 2000 N/m")
Kd_use = 2000.0
Bd = 2*1.0*np.sqrt(Kd_use*Lam)
wn = np.sqrt(Kd_use/Lam)

```

```

print(f"臨界阻尼 B_d = 2√(K_d·M_d) = {Bd:.1f} N·s/m")
print(f"自然頻率 ω_n = √(K_d/M_d) = {wn:.1f} rad/s = {wn/(2*np.pi):.2f} Hz")
print(f"建議最低取樣頻率 = 20 × {wn/(2*np.pi):.2f} = {20*wn/(2*np.pi):.0f} Hz")
Kd_human = 5.0/0.008
print(f"接觸人手 (力 ≤ 5 N) : K_d ≤ {Kd_human:.0f} N/m "
      f" ( 僅為接觸桌面的 {Kd_human/Kd_lim*100:.0f}% ) ")

# ===== 阻尼比對接觸行為的影響 =====
print("\n=== 阻尼比對接觸的影響 ( K_d = 2000 N/m, M = 5 kg ) ===")
print(f"'ζ':>6}{ 'B_d':>10}{ '最大接觸力':>14}{ '穩態力':>12} 行為")
Kd_imp = 2000.0
for zeta in (0.2, 0.5, 0.7, 1.0, 2.0):
    Bd_t = 2*zeta*np.sqrt(Kd_imp*5.0)
    _, fmax, ffin, _ = run(impedance_control, "", Kd_imp=Kd_imp, Bd=Bd_t)
    behav = ("嚴重超越 / 彈跳" if zeta < 0.4 else
             "略有超越" if zeta < 0.9 else
             "平順" if zeta <= 1.2 else "反應遲鈍")
    print(f"{zeta:6.1f}{Bd_t:10.1f}{fmax:12.1f} N{ffin:10.2f} N {behav}")
print("→ 阻尼不足會在接觸瞬間產生衝擊與彈跳")

# ===== 例題 20-3：插銷入孔容差 =====
print("\n=== 例題 20-3：插銷入孔容差分析 ===")
hole, peg, chamfer, robot_acc = 20.05, 20.00, 1.0, 0.8
clearance = (hole - peg)/2
print(f"間隙半徑 = {clearance:.3f} mm")
print(f"純位置控制容許偏差 = {clearance:.3f} mm "
      f" ( 機器人精度 ±{robot_acc} mm → {'可行' if robot_acc <= clearance else '不可行'} ) ")
tol_chamfer = chamfer + clearance
print(f"有倒角後容許偏差 = {tol_chamfer:.3f} mm "
      f" ( → {'可行' if robot_acc <= tol_chamfer else '不可行'} . "
      f"容差放寬 {tol_chamfer/clearance:.0f} 倍 ) ")
for acc in (1.5, 2.0):
    r = max(0.0, acc - tol_chamfer)
    print(f"若精度為 ±{acc} mm: 需螺旋搜尋，半徑 ≥ {r:.3f} mm → 建議 {r*1.25:.2f} mm")

# 橫向順應性驗證
print("\n 橫向剛度的順應性驗證 ( 45° 倒角、軸向推力 10 N ) : ")
F_lateral = 10.0
for Kxy in (10000., 1000., 200., 50.):
    disp = F_lateral/Kxy*1000
    print(f" K_xy = {Kxy:7.0f} N/m → 橫向可位移 {disp:7.2f} mm "
          f"{'✓ 充足' if disp > 1.0 else 'X 不足'}")

```

請試著修改：（一）把環境剛度改為 $1e4$ N/m（橡膠），觀察位置控制是否變得可行——這說明「環境軟」時位置控制的困難會大幅降低。（二）在阻抗控制中加入 5 ms 的力回授延遲，

觀察是否出現振盪（驗證 20.7 節的接觸不穩定）。（三）把致動器的力上限降到 100 N，觀察位置控制的行為如何改變。

實作：柔順控制的實驗

本章實作是在實體手臂上實現柔順行為並量化其效果。**安全提醒：力控制的參數錯誤可能導致機器人失控或損壞工件，務必從極低的剛度與極低的速度開始測試。**

- 前置作業：**確認第 17 章的重力補償已正確實作（阻抗控制建立在其上）。若有力 / 力矩感測器，完成其歸零與重力補償校正。
- 剛度的直接體驗：**實作阻抗控制，設定不同的 K_d （如 5000、1000、200 N/m），用手推動末端並感受其軟硬差異。用彈簧秤量測「推開 10 mm 所需的力」，驗證是否符合 $F = K_d \times \Delta x$ 。
- 阻尼的影響：**固定 K_d ，改變 B_d 使 ζ 分別為 0.3、0.7、1.0、2.0。快速推動末端後放手，觀察其回彈行為——低阻尼會來回振盪、臨界阻尼平順回位、過阻尼緩慢回位。
- 接觸力量測：**在末端裝一支筆，讓機器人以不同的 K_d 接觸一個電子秤，設定的目標位置比秤面「深」5 mm。記錄各剛度下的實際接觸力，與理論值 $K_d \times 0.005$ 比較。
- 位置控制的對照：**在安全的前提下（用海綿或彈簧秤，勿用硬物），以位置控制重複上一步。觀察接觸力如何暴增，並記錄控制器是否觸發過載保護。
- 方向性剛度：**把 K_d 設為非對角矩陣（如 Z 方向 2000 N/m、X-Y 方向 100 N/m），用手推動末端，感受不同方向的軟硬差異。這是混合控制的阻抗版本。
- 接觸穩定性探索：**逐步提高 K_d ，找出開始出現振盪或嗡嗡聲的臨界值。記錄此臨界剛度，並嘗試提高控制頻率或降低力訊號的濾波，觀察臨界值是否上升（驗證 20.7.3 節）。
- 插銷入孔實測：**準備一組有倒角與無倒角的軸孔試件（可 3D 列印）。分別以位置控制與阻抗控制嘗試插入各二十次，記錄成功率。這個對照實驗能最直觀地展示柔順控制的價值。
- （進階）螺旋搜尋：**對定位誤差較大的情形，實作螺旋搜尋策略，記錄搜尋所需的平均時間與成功率的提升。

表 20-9 柔順控制實驗記錄表（第一列為範例）

實驗項目	設定	理論預期	實測結果	說明
剛度驗證	$K_d = 1000 \text{ N/m}$ ，推 10 mm	10.0 N	11.3 N	誤差來自摩擦

實驗項目	設定	理論預期	實測結果	說明
(自行填寫其餘實驗)				

國中小也看得懂：讓機器人的手變成一根彈簧

試試看這個實驗：用**兩根手指**輕輕拿起一顆生雞蛋。你有沒有想過，你是怎麼知道該用多大的力氣的？

其實你**沒有**特別去想。你不會說「我要把手指移到 43.9 公釐的位置」，也不會說「我要出 2 牛頓的力」。你只是——輕輕地握住它。

但機器人不一樣。如果我們叫機器人「把手指合到 43.5 公釐」，會發生什麼事？蛋殼只要被壓進去 0.5 公釐，就會產生大約 1500 牛頓的力——那可以壓碎石頭！蛋當然一秒就爆了。

那如果改成「用 2 牛頓的力去夾」呢？也不行。因為在手指還沒碰到蛋之前，力是零，機器人會覺得「還不夠用力」，於是拚命往前衝，撞上去的瞬間反而更慘。

所以聰明的做法是**第三種**：讓機器人的手變得像一根彈簧。

想像你的手指上裝了兩根軟軟的彈簧。當彈簧碰到蛋，蛋會把彈簧壓回去一點點，這時彈簧產生的力剛剛好——不會太大（因為彈簧很軟），也不會是零（因為確實被壓縮了）。而且你**完全不需要知道蛋有多大**，彈簧會自己調整！

這就是這一章最重要的概念：**不要控制位置，也不要控制力，而是控制「軟硬」**。工程師叫它「阻抗控制」。

這個方法還有一個很棒的好處：**安全**。如果機器人的手是「很軟的彈簧」，就算它不小心撞到你，也只會輕輕碰一下——因為軟彈簧根本產生不了大力氣。這就是為什麼現在的新型機器人可以直接和人一起工作，不需要用鐵籠子把它關起來。

叫獸提醒與常見錯誤

- 誤區一：用位置控制去接觸剛性環境。環境剛度可達 10^8 N/m，微米級的位置誤差就會產生數百牛頓的力，這在物理上不可能成功。
- 誤區二：以為純力控制可以取代位置控制。在自由空間中測到的力恆為零，控制器會持續加

速直到撞上，這是本質上的不穩定。

- 誤區三：忘記扣除工具重力與慣性力。力感測器量到的是總和，必須用動力學模型即時補償，否則會把自重誤判為外力。
- 誤區四：混合控制的方向與實際接觸面不一致。若工件擺放有角度偏差，本該控力的方向變成控位置，後果與誤區一相同。
- 誤區五：同一個方向同時控制位置與力。這在物理上是矛盾的——自然約束與人為約束必定互補。
- 誤區六：把阻抗剛度設得太高。剛度上限受取樣頻率與延遲限制，設得太高會導致接觸振盪而非更精確。
- 誤區七：忽略摩擦對柔順性的破壞。高減速比的關節摩擦會使機器人「感覺起來很硬」，無論 K_d 設得多低。
- 誤區八：在硬機器人上用阻抗控制、在軟環境中用導納控制。應遵循互補原則：硬環境配可反驅的機器人用阻抗、軟環境配高減速比機器人用導納。
- 誤區九：力訊號濾波過度。濾波降低雜訊但增加延遲，而延遲是接觸不穩定的主因，必須權衡。
- 誤區十：以為柔順控制能取代機構設計。倒角、RCC 等被動措施成本極低卻效果顯著，且沒有頻寬限制。
- 誤區十一：以為安全只靠控制。碰撞的最初數十毫秒由慣量主導，控制器來不及反應，必須靠輕量化與被動緩衝。
- 誤區十二：只看力而忽略壓力。安全標準規範的是壓力（力除以面積），同樣的力集中在尖角上可能造成傷害。

課堂討論

1. 例題 20-1 顯示阻抗控制的接觸力比位置控制小二十萬倍。請用「串聯彈簧」的觀點解釋這個現象，並討論它對協作機器人安全設計的意義。
2. 阻抗控制的基本版本不需要力感測器，卻能產生柔順行為。請解釋這是為什麼，並討論在什麼情況下仍然必須加裝力感測器。
3. 20.6.2 節提出「硬環境配軟機器人用阻抗、軟環境配硬機器人用導納」的互補原則。請討

論：若兩者都硬（傳統工業機器人接觸金屬工件），有哪些可行的解決方案？

4. 一個 1 公釐的倒角，把插銷入孔的容許誤差放寬了 41 倍（例題 20-3）。請再舉出兩個「機構設計為控制減負」的例子，並討論這種思維對系統設計的啟示。
5. 20.10.2 節指出碰撞的第二階段（衝擊瞬間）完全由機構決定，控制無能為力。請討論這對機器人設計的優先順序有何影響——在追求更好的控制演算法之前，應該先做什麼？

章末摘要

- 位置控制在接觸剛性環境時必然失效：環境剛度可達 10^8 N/m，微米級誤差即產生數百牛頓的力。
- 純力控制在自由空間中不穩定：測得的力恆為零，控制器會持續加速直到撞擊。
- 力感測有三種方式：六軸力 / 力矩感測器（精度高但貴）、關節力矩感測器（可全身偵測）、馬達電流估測（免費但精度低）。
- 力訊號必須扣除工具重力與慣性力，並注意濾波延遲與座標系轉換。
- 直接力控制需要積分項，因為環境剛度未知且會變化，無法用模型補償。
- 自然約束由環境幾何決定、人為約束由任務指定，兩者互補：同一方向不可能同時控制位置與力。
- 混合位置 / 力控制以選擇矩陣 S 指定各方向的控制模式，並建立在操作空間的動態解耦之上。
- 阻抗控制的目標是讓末端表現為 $M_d \ddot{x} + B_d \dot{x} + K_d x = F_{ext}$ ，即一個可程式的質量—彈簧—阻尼系統。
- 阻抗控制涵蓋整個頻譜： $K_d \rightarrow \infty$ 為純位置控制、 $K_d \rightarrow 0$ 為純力控制。
- 阻抗控制的四大優勢：統一處理接觸與非接觸、本質安全（剛度即力的上限）、基本版不需力感測器、各方向可獨立設定剛度。
- 串聯彈簧的等效剛度由較軟者主導——只要機器人夠軟，環境有多硬都無所謂，這是安全性的數學根源。
- 阻抗控制「量測運動輸出力」適合可反驅機器人與硬環境；導納控制「量測力輸出運動」適合高減速比機器人與軟環境。
- 接觸使閉環剛度暴增、自然頻率上升、阻尼比下降，因而容易振盪；可實現的剛度有上限，

與取樣頻率及延遲有關。

- 主動柔順可調但受頻寬限制，被動柔順（如 RCC）不可調但反應瞬間；實務上採「粗調靠控制、微調靠機構」的分工。
- 插銷入孔的三個階段需要不同的阻抗參數；倒角、螺旋搜尋與主動晃動是提高成功率的三種策略。
- 人機碰撞的三階段中，衝擊瞬間由機構慣量主導，控制只能處理碰撞前的偵測與碰撞後的退讓。

觀念題與計算題

觀念題

1. 說明位置控制在接觸剛性環境時失效的原因，並以量級分析支持你的論點。
2. 為何純力控制在自由空間中不穩定？請描述控制器的行為。
3. 比較三種力感測方式的精度、成本、偵測範圍與適用場合。
4. 力訊號處理的三個要點為何？為何重力補償對力控制特別重要？
5. 為何直接力控制幾乎必須加入積分項？這與位置控制中的積分項有何不同？
6. 區分自然約束與人為約束，並說明兩者的互補性原理。
7. 以「用板擦擦黑板」為例，分析其自然約束與人為約束，並寫出選擇矩陣。
8. 寫出阻抗控制的目標關係式，並說明三個參數的物理意義。
9. 為何說阻抗控制涵蓋了位置控制與力控制兩個極端？
10. 列舉阻抗控制的四大優勢，並說明「本質安全」的數學根據。
11. 說明串聯彈簧的等效剛度性質，以及它對接觸安全的意義。
12. 區分阻抗控制與導納控制，並說明選擇兩者的互補原則。
13. 為何摩擦會破壞阻抗控制的柔順性？有哪三種因應方式？
14. 說明接觸不穩定的三個關鍵因素，以及可實現剛度有上限的原因。
15. 比較主動柔順與被動柔順，並說明 RCC 的設計原理。
16. 說明插銷入孔的三個階段及其阻抗參數需求，並列出三種提高成功率的策略。

計算題

1. 某機器人接觸一個塑膠工件 ($K_{env} = 5 \times 10^3 \text{ N/mm}$)。(a) 若位置指令深入 0.2 mm ，求接觸力。(b) 若改用阻抗控制且 $K_d = 800 \text{ N/m}$ ，求同樣位置誤差下的接觸力。(c) 計算兩者的比值。(d) 求串聯後的等效剛度。
2. 某協作機器人末端等效慣量 $\Lambda = 8 \text{ kg}$ ，要求接觸力不超過 30 N 且能容忍 $\pm 12 \text{ mm}$ 的位置不確定性。(a) 求剛度上限並選定 K_d 。(b) 依 $\zeta = 0.8$ 求阻尼 B_d 。(c) 計算自然頻率 (rad/s 與 Hz)。(d) 依第 19 章的準則建議最低控制頻率。
3. 某軸孔配合：孔徑 12.03 mm 、軸徑 12.00 mm ，孔口倒角寬 0.8 mm 。機器人定位精度 $\pm 0.6 \text{ mm}$ 。(a) 求間隙半徑與純位置控制的容許偏差。(b) 判斷純位置控制是否可行。(c) 加入倒角後的容許偏差為何？是否可行？(d) 若精度劣化為 $\pm 1.2 \text{ mm}$ ，求所需的螺旋搜尋半徑。
4. 某力控研磨作業：法向要求維持 25 N 的壓力，工件表面有 $\pm 1.5 \text{ mm}$ 的起伏。(a) 若用位置控制且工件剛度為 $2 \times 10^4 \text{ N/mm}$ ，求表面起伏造成的力變化範圍。(b) 改用阻抗控制，求使力變化不超過 $\pm 10 \text{ N}$ 所需的 K_d 上限。(c) 在此剛度下，維持 25 N 平均壓力所需的平均侵入深度為何？(d) 討論此深度是否合理。

動手做與專題延伸

本章實作：柔順控制模組

延續第 17 ~ 19 章的控制器程式庫，加入力控制與阻抗控制模組。

- **階段一**：建立一維的接觸模擬環境 (含環境剛度與阻尼)，實作位置控制與阻抗控制並比較接觸力。
- **階段二**：實作直接力控制 (含積分項) 與混合位置 / 力控制 (含選擇矩陣)，在模擬中驗證各方向的行為。
- **階段三**：實作完整的六自由度阻抗控制，整合第 17 章的操作空間控制與第 11 章的雅可比。
- **階段四**：實作導納控制版本，並在「硬機器人 + 軟環境」與「軟機器人 + 硬環境」兩種配置下比較兩者的表現。
- **階段五**：加入力感測的雜訊與延遲模型，找出各配置下的臨界剛度，驗證 20.7.3 節的結論。

專題延伸

- **剛度—接觸力關係圖**：系統性地掃描 K_d (由 10 到 10^6 N/m)，對固定的位置誤差記錄接觸力，繪成對數—對數圖。你會看到曲線在 K_d 遠小於 K_{env} 時呈線性、在接近 K_{env} 時飽和——這張圖直觀展示了串聯彈簧的性質與「軟者主導」的原理。
- **接觸穩定性的邊界圖**：以「控制頻率」為橫軸、「目標剛度」為縱軸，對每個組合模擬接觸過程並判定是否穩定，繪出穩定與不穩定的分界線。加入不同的力感測延遲 (0 、 2 、 5 、 10 ms) 各繪一條線，量化延遲對可實現剛度的影響。
- **進階：插銷入孔的策略比較**。建立一個二維的插銷入孔模擬 (含倒角幾何與摩擦)，實作四種策略——(甲) 純位置控制、(乙) 阻抗控制、(丙) 阻抗控制加螺旋搜尋、(丁) 阻抗控制加主動晃動。對每種策略在不同的初始位置誤差下各模擬 100 次，統計成功率與平均插入時間，繪出「位置誤差 vs 成功率」曲線。這個實驗能完整呈現本章各項技術的實際價值。

參考文獻與延伸閱讀

- Hogan, N., "Impedance Control: An Approach to Manipulation, Parts I-III," ASME J. Dynamic Systems, 1985 (阻抗控制的奠基論文)。
- Mason, M. T., "Compliance and Force Control for Computer Controlled Manipulators," IEEE Trans. SMC, 1981 (自然約束與人為約束的框架)。
- Raibert, M. H. and Craig, J. J., "Hybrid Position/Force Control of Manipulators," ASME J. Dynamic Systems, 1981 (混合位置 / 力控制的原始論文)。
- Whitney, D. E., "Quasi-Static Assembly of Compliantly Supported Rigid Parts," ASME J. Dynamic Systems, 1982 (RCC 與插銷入孔的理論分析)。
- Khatib, O., "A Unified Approach for Motion and Force Control of Robot Manipulators," IEEE J. Robotics and Automation, 1987 (操作空間中的統一運動與力控制)。
- Siciliano, B. and Villani, L., Robot Force Control, Kluwer Academic (力控制的完整專著)。
- Colgate, J. E. and Hogan, N., "Robust Control of Dynamically Interacting Systems," Int. J. Control, 1988 (接觸穩定性與被動性理論)。
- Albu-Schäffer, A. et al., "Soft Robotics: From Torque Feedback-Controlled Lightweight Robots to Intrinsically Compliant Systems," IEEE Robotics & Automation Magazine, 2008 (DLR 輕量手臂與柔順控制的實務)。

- ISO/TS 15066:2016, Robots and robotic devices — Collaborative robots (人機協作的力與壓力限制規範) 。
- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson (第 9 章：力控制) 。

第 21 章

抓取與操作

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為一隻戴著厚工作手套的手正握住一瓶礦泉水的照片，手套明顯變形貼合瓶身，旁註「摸不清楚，卻抓得穩」。右半為三格對照：上格是剛性兩指夾爪與圓柱體的單點接觸（標註「點接觸，位置差 2 mm 即失敗」）；中格是柔性指墊變形後的面接觸（標註「軟指接觸，可抵抗扭轉」）；下格是欠驅動手指自動包絡不規則物體的示意。畫面下緣以摩擦錐圖示串起——兩個相對的圓錐與其間的連線，標註「連線落在雙錐內 $\Rightarrow$ 力封閉」。標題文字：抓取的成功，往往不是來自精準，而是來自順應。

叫獸開講 「戴著厚手套、閉著眼睛，為什麼還是抓得起來？」

叫獸拿出一副厚厚的工作手套，遞給小妤：「戴上，然後把眼睛閉起來。」

小妤照做了。叫獸在她面前的桌上放了一瓶礦泉水：「桌上有個東西，把它拿起來。」

小妤的手伸出去，摸索了兩三秒，碰到了瓶身，手掌順著曲面滑動了一下，然後五指收攏——穩穩地把水瓶提了起來。全程不到五秒。

「你剛才知道那是什麼嗎？」叫獸問。「不知道，摸起來像個圓柱體。」「知道它有多重嗎？」「不知道。」「知道它的中心在哪裡嗎？」「也不知道。」

叫獸轉向全班：「她戴著厚手套、閉著眼睛、不知道物體是什麼、多重、重心在哪——但她五秒鐘就拿起來了。現在請你們想想：**如果要一台機器人做同樣的事，需要什麼？**」

學生們開始列舉：高解析度的相機、精確的物體辨識、三維重建、質量與摩擦係數的估計、精密的力覺感測、毫米級的手指定位.....

「你們說的都對，」叫獸點頭，「而且這正是過去三十年機器人抓取研究的主流方向——把每一件事都測得更準、算得更精。但小妤剛才什麼都不知道，她是怎麼成功的？」

教室安靜了幾秒。小翰試著回答：「因為.....手套很軟？」

「非常接近了。」叫獸拿起那副手套，「厚手套讓她摸不清楚，這是壞事。但厚手套同時也讓她的手指**能夠變形、能夠貼合、能夠在滑動時自動調整**——這是好事。而在抓取這件事上，後者比前者重要得多。」

他在白板上寫下一行字：**抓取的成功，往往不是來自精準，而是來自順應。**

「這是第 20 章那個雞蛋的延續。人手抓東西時，手指的軟墊會變形、會攤開成一片接觸面、會在物體要滑動時自動增加摩擦。這些都不是『算出來』的，是**機構本身的性質**。」

小妤把手套脫下來：「那我們今天要學什麼？」

「兩件事。」叫獸豎起兩根手指，「第一，**什麼叫做抓穩了**——這需要嚴格的數學，因為工程師不能只憑感覺說『看起來很穩』。第二，**在什麼情況下，該靠算的、什麼時候該靠機構的柔軟**。學會這兩件事，你就理解了現代機器人抓取的全貌。」

本章為什麼重要

抓取是機器人與環境互動最基本、也最普遍的形式。從工廠的取放、倉儲的分揀、到服務型機器人的居家操作，幾乎所有任務都由「抓起某個東西」開始。

但抓取同時也是機器人學中最頑固的難題之一。第 1 章提到的 Moravec 悖論在此表現得最為極端：人類幼兒兩歲就能靈巧地抓取任意物體，而這件事至今仍是機器人研究的前沿。原因在於抓取結合了本書幾乎所有的困難——不確定的物體幾何、未知的摩擦與質量、多點同時接觸、以及接觸力學的本質複雜性（第 14.7 節）。

本章要建立三個層次的理解。**第一，理論基礎**：用力封閉、形封閉與摩擦錐等概念，嚴格定義「什麼叫抓穩了」。**第二，設計思維**：理解柔順性在抓取中的關鍵角色，以及軟性夾爪為何能用簡單的機構解決複雜的問題。**第三，現代方法**：了解學習式抓取如何繞過建模的困難，直接由資料學習「哪裡可以抓」。

本章也是第三篇的收尾。第 13~20 章建立的動力學、控制與力控制工具，都將在此匯聚——抓取正是它們的綜合應用。

學習目標

- 能說明抓取問題的三個組成：接觸建模、抓取分析與抓取規劃。
- 能描述三種接觸模型及其可傳遞的力與力矩。
- 能定義摩擦錐並說明其在抓取分析中的角色。
- 能定義力封閉，並判斷一組接觸點是否構成力封閉。
- 能運用正線性跨度的概念判斷抓取的完整性。
- 能區分力封閉、形封閉與關入籠三個概念。
- 能說明常用的抓取品質指標及其物理意義。
- 能敘述抓取合成的經典定理與所需的最少接觸點數。
- 能說明柔順性在抓取中的作用，並解釋為何順應往往勝過精準。
- 能比較剛性夾爪、軟性夾爪與欠驅動手的設計哲學。
- 能說明學習式抓取的基本流程與其相對於解析法的優劣。

先備知識

第 11 章的雅可比與速度—力對偶、第 20 章的力控制與柔順概念（本章可視為其在多接觸點情形的推廣）、第 13 章的動力學基礎、第 6.4 節的力覺與觸覺感測。

數學方面需要：向量與矩陣運算、線性獨立與跨度（span）的概念、以及基本的凸集合直覺。本章的數學不深，但**幾何直覺很重要**——建議多用手邊的物品實際比劃，體會各個概念的物理意義。

核心名詞中英對照

表 21-1 本章核心名詞

中文	英文	說明
抓取	Grasp	以多個接觸點限制物體運動的行為
接觸模型	Contact Model	描述接觸點能傳遞何種力的模型
點接觸	Point Contact	僅能傳遞法向力的理想接觸
軟指接觸	Soft Finger Contact	可傳遞法向力、摩擦力與繞法向的力矩
摩擦錐	Friction Cone	不發生滑動的接觸力方向所構成的錐體
摩擦係數	Coefficient of Friction	最大靜摩擦力與法向力之比
抓取矩陣	Grasp Matrix	把接觸力映射為物體所受合力的矩陣
力封閉	Force Closure	能以接觸力抵抗任意方向外力的性質
形封閉	Form Closure	僅靠幾何即可限制物體的性質
關入籠	Caging	物體被包圍而無法逃脫但仍可移動
正線性跨度	Positive Linear Span	以非負係數線性組合所能達到的集合
抓取品質	Grasp Quality	量化抓取穩固程度的指標
欠驅動手	Underactuated Hand	驅動器少於自由度的機械手
自適應抓取	Adaptive Grasping	手指依物體形狀自動貼合的抓取
軟性夾爪	Soft Gripper	以柔性材料製成的夾持裝置
抓取姿勢偵測	Grasp Pose Detection	由感測資料預測可行抓取位姿

21.1 抓取問題的組成

21.1.1 三個層次

「抓取」看似單一動作，實際上包含三個層次的問題，各有不同的研究方法。

層次一：接觸建模。單一接觸點能傳遞什麼樣的力與力矩？這取決於接觸的幾何（點、線、面）與摩擦特性。這是最底層的物理基礎，21.2 節處理。

層次二：抓取分析。給定一組接觸點，判斷這個抓法是否「穩」——能否抵抗任意方向的外力？有多穩？這是本章數學的核心，21.3 至 21.6 節處理。

層次三：抓取規劃。面對一個物體，該把手指放在哪裡？這是搜尋與決策問題，也是現代學習式方法的主戰場，21.9 節處理。

三個層次的關係是：分析回答「這個抓法好不好」，規劃則是「找出好的抓法」。前者是後者的評分函數。

21.1.2 為何抓取如此困難

抓取的困難來自五個因素的疊加：

- **物體資訊不完整：**形狀可能只看得到一半（遮蔽）、質量與重心未知、材質與摩擦係數難以由視覺判斷。
- **接觸力學本質複雜：**如第 14.7 節所述，摩擦是非線性的、接觸是不連續的、且某些配置下的接觸力沒有唯一解。
- **多點同時作用：**各接觸點的力互相影響，且必須滿足不滑動的條件——這是一個帶約束的多變數問題。
- **位置誤差不可避免：**機器人的定位精度（第 9.9.2 節）遠大於精確抓取所需的容差。
- **物體會動：**抓取過程中物體可能被推動、旋轉或滑落，使原本規劃的接觸點失效。

這五個因素解釋了為何「精確建模再精確執行」的路線在抓取上特別吃力——不確定性太多，且每一項都難以消除。這也預示了本章的核心洞見：**與其消除不確定性，不如設計一個對不確定性不敏感的系統。**

21.2 接觸模型與摩擦錐

21.2.1 三種接觸模型

抓取分析的第一步，是決定用什麼模型描述接觸點。常用的有三種，複雜度遞增。

模型一：無摩擦點接觸 (Frictionless Point Contact)。只能傳遞沿接觸法向的**推力**（不能拉）。每個接觸點只貢獻 1 個力的自由度。適用於極光滑的表面，是最保守的假設。

模型二：有摩擦點接觸 (Point Contact with Friction, PCwF)。除法向力外，還能傳遞切向的摩擦力。每個接觸點貢獻 3 個自由度（三維空間中的力向量）。這是最常用的模型。

模型三：軟指接觸 (Soft Finger Contact)。除上述三個力之外，還能傳遞繞接觸法向的**力矩**——因為柔軟的指尖會攤開成一小片接觸面，能抵抗扭轉。每個接觸點貢獻 4 個自由度。

表 21-2 三種接觸模型的比較

模型	可傳遞	自由度	適用情形	保守程度
無摩擦點接觸	法向推力	1	極光滑表面、冰面	最保守
有摩擦點接觸	法向力 + 兩個切向摩擦力	3	一般剛性指尖	標準假設
軟指接觸	上述 + 繞法向的力矩	4	橡膠指墊、人手	較樂觀

模型三的重要性常被低估。能抵抗扭轉，代表物體不容易在手中打轉——這正是人類用軟指墊抓握筆或螺絲起子時能穩定施力的原因。這也是本章一再強調的主題：**柔軟不只是「安全」**，它實際上增加了抓取的能力。

21.2.2 摩擦錐

有摩擦的接觸點所能施加的力並非任意——它必須滿足庫倫摩擦定律，否則就會滑動。設接觸法向為 $\hat{n}$ 、摩擦係數為 μ ，則不滑動的條件是：

$$\|f_t\| \leq \mu \cdot f_n \quad (\text{切向力不超過法向力的 } \mu \text{ 倍})$$

庫倫摩擦條件。 f_n 為法向分量（必須為正，即推而非拉）、 f_t 為切向分量。

滿足此條件的所有力向量，在空間中構成一個以法線為軸的**圓錐**，稱為**摩擦錐**（friction cone）。錐的半頂角為：

$$\alpha = \arctan(\mu)$$

摩擦錐的半頂角。 $\mu = 0.5$ 時 $\alpha \approx 26.6^\circ$ ； $\mu = 1.0$ 時 $\alpha = 45^\circ$ 。

摩擦錐是抓取分析的核心幾何物件。它的意義是：**接觸力必須落在錐內**，否則手指會滑動。因此所有的抓取分析，本質上都是在問「錐內的力能組合出什麼效果」。

圖 21-1 摩擦錐與接觸力的限制

建議配圖：左圖為單一接觸點的三維示意——物體表面、接觸法線 $\hat{n}$ 、以及以法線為軸的圓錐（半頂角 $\alpha = \arctan \mu$ ）。錐內畫一個綠色的合法力向量、錐外畫一個紅色的非法力向量並標註「會滑動」。右圖為二維簡化版，顯示兩個手指從兩側夾住物體時的兩個摩擦錐，並以陰影標出「兩錐的正線性組合所能產生的合力範圍」。下方標註 μ 值增大時錐角變寬、可用力的範圍變大。

21.2.3 摩擦係數的不確定性

摩擦係數是抓取規劃中最不可靠的參數之一。同樣是「橡膠對塑膠」，實際的 μ 可能因表面粗糙度、清潔度、濕度、接觸壓力而在 0.3 到 1.2 之間變動——變化範圍可達四倍。

這帶來一個重要的工程原則：抓取規劃時應使用保守的 μ 估計值。若實際 μ 大於假設值，抓取只會更穩；反之則可能失敗。實務上常取 $\mu = 0.3 \sim 0.5$ 作為設計值，即使指墊材料的標稱值更高。

這也再次呼應第 14.7.3 節：模擬中的抓取常與實機不符，摩擦係數的不確定性是主因之一，而因應之道正是第 31 章的領域隨機化。

21.3 抓取矩陣與力封閉

21.3.1 抓取矩陣

要分析多個接觸點的共同效果，需要把各接觸力對物體產生的合力與合力矩表達出來。設第 i 個接觸點的位置為 p_i （相對於物體質心）、施加的力為 f_i ，則它對物體產生：

力： f_i 力矩： $p_i \times f_i$

單一接觸點的貢獻。力矩由力臂與力的外積決定（第 11 章的反對稱矩陣在此再次出現）。

把所有接觸點的貢獻加總，寫成矩陣形式：

$$w = G \cdot f, \quad \text{其中 } G = \begin{bmatrix} I & I & \dots & I \\ [p_1] \times & [p_2] \times & \dots & [p_n] \times \end{bmatrix}$$

抓取矩陣 G 。 $w = [力; 力矩]$ 是物體所受的合成扳手（wrench）、 f 是所有接觸力串接而成的向量。

抓取矩陣的角色，類似第 11 章的雅可比——它是一個線性映射，把「各接觸點的力」轉換為「物體所受的合力與合力矩」。兩者的對照很有啟發性：雅可比連結關節與末端，抓取矩陣連結手指與物體。

21.3.2 力封閉的定義

現在可以嚴格定義「抓穩了」的意思。

◎ 力封閉 (Force Closure) 的定義

一組接觸點構成**力封閉**，若且唯若：對於作用在物體上的**任意外力**與外力矩，都存在一組滿足摩擦錐約束的接觸力，能夠將其平衡。

關鍵在於「**任意**」兩個字。這代表無論外力從哪個方向來、多大（在手指出力能力範圍內），抓取都能抵抗。這正是工程上「抓穩了」的嚴格意義。

數學上，力封閉的條件可以表述為：

摩擦錐內的接觸力，經抓取矩陣 G 映射後，其「**正線性跨度**」必須涵蓋整個扳手空間（三維時為 $\mathbb{R}^6$ ）

力封閉的數學條件。

21.3.3 為何是「正」線性跨度

這裡有一個容易被忽略但極重要的細節：為何是**正線性跨度**，而非一般的線性跨度？

原因是：**手指只能推，不能拉**。接觸力的法向分量必須為正（ $f_n \geq 0$ ），否則手指就離開物體了。因此在組合各接觸力時，係數只能是非負的。

一般線性跨度： $\{ \sum c_i v_i : c_i \in \mathbb{R} \}$ （係數可正可負）

正線性跨度： $\{ \sum c_i v_i : c_i \geq 0 \}$ （係數必須非負）

兩者的差別。正線性跨度要涵蓋整個空間，需要更多的向量。

這個限制帶來一個直觀的推論：在平面上，要用正線性組合涵蓋二維的所有方向，至少需要三個方向不共面的向量（想像三個互成 120 度的向量）。若只有兩個相反的向量，只能涵蓋一條線，無法涵蓋整個平面。這正是下一節「最少接觸點數」定理的幾何直覺。

21.4 最少接觸點數與抓取合成

21.4.1 經典定理

抓取理論中有幾個關於「最少需要幾個接觸點」的經典結果。

◎ 力封閉的最少接觸點數

平面（2D）情形：以有摩擦的點接觸，至少需要 2 個接觸點（若兩摩擦錐能互相涵蓋）；以無摩擦接觸則至少需要 4 個。**空間（3D）情形**：以有摩擦的點接觸，至少需要 3 個接觸點；以無摩擦

接觸則至少需要 7 個。

這些數字揭示了摩擦的價值：**有摩擦時所需的接觸點數大幅減少**。這正是為何人手抓東西時，用兩三根手指就足夠——因為皮膚提供了可觀的摩擦。

21.4.2 二指抓取的條件

平面上兩指抓取的力封閉條件有一個優美的幾何判準：

兩個接觸點的連線，必須同時落在兩個摩擦錐之內

二指力封閉條件。這稱為「連線在雙錐內」判準。

直觀的理解是：兩指相夾時，力必定沿著兩點的連線方向傳遞。若這條連線落在兩個摩擦錐內，代表兩指都能沿此方向施力而不滑動，物體就被夾住了。若連線偏離錐外，手指會滑開。

這個判準有一個實用的推論：**摩擦係數越大，錐角越寬，可行的抓取位置就越多**。當 $\mu = 0$ 時，連線必須恰好垂直於兩個接觸面（機率為零），因此無摩擦的二指抓取幾乎不可能——這與 21.4.1 節「無摩擦需要 4 點」的結論一致。

21.4.3 抓取合成的實務作法

由理論走向實務，「找出一組好的接觸點」稱為**抓取合成**（grasp synthesis）。傳統的解析法流程為：

1. **取得物體的幾何模型**：由 CAD、三維掃描或點雲重建（第 22 章）。
2. **採樣候選接觸點**：在物體表面均勻或依啟發式規則取樣，並計算各點的法向。
3. **組合並篩選**：列舉所有可能的接觸點組合，用力封閉條件篩掉不可行者。
4. **評分排序**：用 21.6 節的品質指標為可行的抓法評分。
5. **運動學可行性檢查**：確認機械手能以該姿態到達（逆運動學有解、不碰撞）。
6. **選定並執行**：取分數最高且可行的抓法。

這個流程在理論上完整，但實務上有三個問題：**一、組合爆炸**—— n 個候選點取 3 個的組合數為 $O(n^3)$ ，計算量大；**二、依賴精確的幾何模型**——而真實場景中物體常被遮蔽；**三、對摩擦係數敏感**——而該值不確定。

這三個問題正是 21.9 節學習式方法興起的原因。但在此之前，21.5 至 21.8 節要先探討另一

條同樣重要的路線：用機構的柔順性繞過這些困難。

21.5 形封閉與關入籠

21.5.1 形封閉：完全不靠摩擦

力封閉允許使用摩擦力。但摩擦係數不確定（21.2.3 節），若能設計一個完全不依賴摩擦的抓取，可靠度會高得多。這就是**形封閉**（form closure）。

◎ 形封閉的定義

一組接觸點構成**形封閉**，若且唯若：即使所有接觸都是**無摩擦**的，物體仍然無法產生任何運動（包含平移與旋轉）。也就是說，物體被幾何形狀完全鎖死。

形封閉比力封閉**嚴格**得多——它是力封閉的特例（形封閉必定力封閉，反之不然）。代價是需要更多的接觸點：平面需要 4 點、空間需要 7 點（21.4.1 節的無摩擦情形）。

實務上，形封閉主要出現在**治具與夾具設計**中，而非機械手抓取。工廠的定位治具就是典型的形封閉——用數個定位銷與夾持點把工件完全鎖死，即使工件表面很滑也不會移動。這種可靠度正是量產所需要的。

表 21-3 力封閉與形封閉的比較

面向	力封閉	形封閉
是否依賴摩擦	是	否
平面最少接觸點	2（有摩擦）	4
空間最少接觸點	3（有摩擦）	7
對摩擦係數的敏感度	高	無
可靠度	中（依賴 μ 的估計）	高
典型應用	機械手抓取	定位治具、夾具

21.5.2 關入籠：不抓緊也能控制

在力封閉與「完全沒抓住」之間，還有一個重要的中間狀態：**關入籠**（caging）。

◎ 關入籠的定義

物體被手指**包圍**，使其無法逃離某個有限的區域，但在該區域內仍可自由移動與旋轉。物體並未被夾緊，卻也跑不掉。

一個生活化的例子：用手掌與五指圍成一個「碗」的形狀去撈一顆彈珠。彈珠在手中可以滾動，但無論怎麼滾都出不去——這就是關入籠。

關入籠的價值在於三點：

- **對位置誤差極不敏感**：不需要精確對準接觸點，只要「圍住」即可。這大幅降低了對感測與定位精度的要求。
- **不會壓壞物體**：因為沒有夾緊，接觸力可以很小甚至為零——這對易碎物（水果、蛋、玻璃）特別有價值。
- **可作為抓取的中間步驟**：先關入籠確保物體不會跑掉，再逐步收攏成為力封閉。這是一個穩健的兩階段策略。

這個「先圍住、再收緊」的策略，正是人類抓取不熟悉物體時的自然做法——也是開頭故事中小好戴著手套摸索時所做的事。她的手掌先形成一個包圍的姿態，接觸後才收攏。

21.5.3 三個概念的層次關係

把三個概念放在一起，可以看出清楚的層次：

形封閉 $\subset$ 力封閉 $\subset$ 關入籠 $\subset$ 有接觸

由嚴格到寬鬆。越往右對精度的要求越低、可靠度的保證越弱。

工程上的選擇原則是：**用最寬鬆但足夠的那一個**。若任務只是「把物體搬到旁邊」，關入籠可能就夠了；若要「精確地插入孔中」，則需要力封閉；若要「在振動的環境中固定住」，才需要形封閉。過度追求嚴格的封閉條件，會不必要地提高對感測與控制的要求。

21.6 抓取品質指標

21.6.1 為何需要量化

力封閉是一個「是或否」的判斷，但實務上我們需要比較：兩個都滿足力封閉的抓法，哪一個更好？這就需要**抓取品質指標**（grasp quality metric）。

品質指標的作用有三：在抓取合成時作為排序的依據（21.4.3 節的步驟四）、在執行前評估風險、以及作為學習式方法的訓練標籤（21.9 節）。

21.6.2 四個常用指標

指標一：最小奇異值 (ϵ 指標)。這是最廣泛使用的指標。它衡量「在最不利的方向上，抓取能抵抗多大的外力」：

$$Q_{\epsilon} = \min_{\|w\|=1} \max_{\|f\|} \{ \|f\| : Gf = w, f \text{ 在摩擦錐內} \} \text{的倒數}$$

直觀地說：在扳手空間中，找出「能抵抗的外力集合」的內接球半徑。半徑越大，抓取越穩。

這個指標與第 11.8 節的可操作度有相似的結構——都是在問「最弱的方向有多弱」。若 $Q_{\epsilon} = 0$ ，代表某個方向完全無法抵抗，即不構成力封閉。

指標二：抓取體積 (Volume 指標)。取「能抵抗的外力集合」的體積，而非內接球半徑。它反映整體的能力而非最弱方向，與可操作度 $w = \sqrt{\det(JJ^T)}$ 的思路一致。

指標三：接觸點分布。直觀的啟發式指標：接觸點越分散、越接近物體重心的對稱位置，抓取越穩。計算簡單，常用於快速篩選。

指標四：對誤差的強健性。刻意為接觸點加入位置擾動（如 $\pm 2 \text{ mm}$ ），檢查抓取是否仍然力封閉。通過擾動測試的比例即為強健性分數。

表 21-4 四種抓取品質指標的比較

指標	衡量什麼	計算成本	優點	缺點
最小奇異值 ϵ	最弱方向的抵抗力	中（需解最佳化）	有明確物理意義、最通用	對接觸點位置敏感
抓取體積	整體抵抗能力	中	反映全方向能力	可能掩蓋單一弱方向
接觸點分布	幾何對稱性	極低	快速、直觀	只是啟發式、不嚴謹
誤差強健性	對位置誤差的容忍	高（需多次採樣）	最貼近實務需求	計算量大

21.6.3 指標與實務的落差

必須誠實指出：理論上的高品質抓取，在實機上未必成功。原因有三。

原因一：模型與現實的落差。品質指標建立在「物體幾何已知、摩擦係數已知、接觸點精確」的假設上，而這三者在真實場景中都不成立。

原因二：只考慮靜力學。多數指標只分析靜態的力平衡，未考慮抓取過程——手指接近時

可能推動物體、閉合時可能使物體旋轉。

原因三：忽略柔順性。指標假設剛性接觸，但真實的指墊會變形，這改變了接觸的性質（從點接觸變成面接觸），使實際的抓取往往比理論預測更穩。

因此近年的趨勢是：**用學習的方式直接由實驗資料預測成功率**，而非依賴解析的品質指標。這正是 21.9 節的主題。但理論指標仍有其價值——它們提供了物理直覺，也是產生訓練資料的重要工具。

21.7 手套思想實驗：柔順勝過精準

21.7.1 回到開頭的實驗

現在有了理論工具，可以重新分析開頭那個手套實驗。小好戴著厚手套、閉著眼睛，卻能在五秒內拿起未知的水瓶。從本章的角度看，發生了什麼？

第一，手套增加了接觸面積。剛性指尖與圓柱體接觸是「點接觸」，柔軟的手套則會變形貼合，形成一小片「面接觸」。依 21.2.1 節的模型，這使接觸從「有摩擦點接觸」升級為「軟指接觸」——多了抵抗扭轉的能力。

第二，手套吸收了位置誤差。她的手指位置必定不精確（閉著眼睛），但柔軟的材料可以壓縮數公釐而不產生過大的力——這正是第 20 章阻抗控制的機構版本。剛性手指若位置差 3 公釐，不是碰不到就是把瓶子推倒。

第三，手套的變形自動增加了穩定性。當瓶子開始下滑時，接觸面會因剪切而變形，這增加了接觸面積與正向壓力，摩擦力隨之上升——這是一個**自動的負回授**，不需要任何感測或計算。

第四，多指的自適應包絡。五指收攏時，先碰到的手指會停止、後碰到的繼續前進，直到全部貼合。這使接觸點自動分布在物體周圍，形成良好的力封閉——而這完全不需要事先知道物體的形狀。

◎ 本章的核心洞見

在抓取中，柔順性不只是「安全考量」，它實際上取代了大量的感測與計算。一個柔軟的、能自適應的手，用極少的資訊就能完成剛性手需要精確建模才能達成的任務。這與第 20 章的結論一脈相承：與其消除不確定性，不如設計一個對不確定性不敏感的系統。工程上，這意味著**機構設計與控制策略必須協同考慮**——把問題丟給控制器解決之前，先問：能不能用機構把問題變簡單？

21.7.2 人手的生物力學啟示

人手是這個原則的極致體現，有四個值得工程借鏡的設計。

設計一：分層的柔軟度。指尖的皮膚最軟、皮下組織次之、骨頭最硬。這個梯度使指尖能大範圍貼合，同時保有施力的能力。

設計二：筋腱驅動。肌肉在前臂，透過肌腱驅動手指（第 5.9.3 節）。這使手指本身極輕，慣量小、反應快、且撞擊時不會造成傷害。

設計三：欠驅動的耦合。人手有 27 個自由度，但真正能獨立控制的遠少於此——多數手指的關節是耦合運動的。這大幅簡化了控制，卻不太影響抓取能力。

設計四：豐富的觸覺。指尖的機械受器密度極高，能偵測壓力、振動與微小滑動。滑動偵測尤其關鍵——它讓人能在物體開始滑落的瞬間自動加大握力，這個反射的延遲只有約 70 毫秒。

這四項設計中，前三項是**機構**的、只有第四項是**感測**的。這個比例本身就說明了：人手的靈巧，主要來自機構而非感知。

21.8 軟性夾爪與欠驅動手

21.8.1 三種設計哲學

機械手的設計可以分為三種路線，各自代表不同的哲學。

路線一：剛性多指手。模仿人手的形狀與自由度，每個關節獨立驅動，配備力覺與觸覺感測。代表如 Shadow Hand、DLR Hand。優點是理論上能完成任何人手能做的操作；缺點是極其昂貴（數百萬元）、控制複雜、且可靠度低（自由度高、故障點多）。

路線二：欠驅動手。自由度高但驅動器少，透過機構的耦合（連桿、彈簧或筋腱）讓手指自動適應物體形狀。代表如 Robotiq 的自適應夾爪、Barrett Hand。優點是成本適中、控制極簡（往往只需一個「閉合」指令）、且抓取穩健；缺點是無法做精細的手內操作（in-hand manipulation）。

路線三：軟性夾爪。以矽膠、氣動或柔性材料製成，沒有明確的關節。代表如氣動的軟指（Soft Robotics 公司）、真空夾爪、以及顆粒堵塞式夾爪（jamming gripper）。優點是極其安全、能抓取形狀不規則或易碎的物體、成本低；缺點是出力小、精度低、且難以建模與控制。

表 21-5 三種機械手設計的比較

面向	剛性多指手	欠驅動手	軟性夾爪
自由度	多 (15~24)	多 (但驅動少)	連續變形
驅動器數	等於自由度	1~4	1~3
控制複雜度	極高	低	極低
適應未知形狀	需精確建模	機構自動適應	天然適應
可做手內操作	可	有限	幾乎不可
出力	中	大	小
成本	極高	中	低
典型應用	研究、遙操作	工業取放、物流分揀	食品、生鮮、易碎品

21.8.2 欠驅動的機構智慧

欠驅動手的核心巧思，在於用機構實現「自動分配」。以最常見的連桿式欠驅動手指為例：

手指有兩個關節但只有一個驅動器，兩者之間以連桿與彈簧連接。當手指閉合時：若近端指節未碰到物體，兩個關節一起彎曲（像剛體一樣）；若近端指節碰到物體而停住，驅動力會自動轉移到遠端指節，使其繼續彎曲直到也接觸物體。

結果是：手指自動包絡物體的輪廓，無論物體是什麼形狀。這個行為完全由機構產生，不需要任何感測器、不需要知道物體形狀、也不需要任何計算。

這是「機構智慧」（mechanical intelligence）的典範——把原本需要感測與計算才能達成的功能，用機構的巧妙設計直接實現。它的可靠度遠高於任何軟體方案，因為沒有感測器會壞、沒有程式會當。

圖 21-2 欠驅動手指的自適應包絡

建議配圖：四格連續動作圖，顯示一根雙關節欠驅動手指抓取一個不規則物體的過程。第一格：手指張開接近物體。第二格：近端指節接觸物體並停止。第三格：驅動力自動轉移，遠端指節繼續彎曲。第四格：兩指節都貼合物體表面，形成包絡抓取。每格旁標註驅動器的施力路徑（以紅色箭頭表示力如何在連桿間轉移），並在下方對照一根剛性手指抓同樣物體時只能形成單點接觸的情形。

21.8.3 真空與吸附

在工業實務中，還有一類極為重要但常被學術界忽略的方案：真空吸盤。

它的優勢在特定場合極為突出：只需要一個接觸面（不需要環繞物體，因此可以從貨箱中取

出緊密排列的物品)、吸附力大且穩定、成本極低、動作極快。物流業的包裹分揀、電子業的面板搬運、食品業的包裝，大量使用真空方案。

限制則是：物體表面必須平滑且不透氣（多孔材料如布料、粗糙的紙箱難以吸附）、且無法抵抗大的剪切力與扭矩。

工程上的啟示：面對一個抓取任務，應先問「能不能用吸盤？」而非直接考慮多指手。這是「用最簡單的方案解決問題」的工程原則——與 21.5.3 節「用最寬鬆但足夠的封閉條件」是同一種思維。

◎ 顆粒堵塞夾爪：一個優雅的巧思

有一種軟性夾爪的設計極具啟發性：一個裝滿咖啡渣（或其他細顆粒）的彈性袋。使用時把袋子壓在物體上，顆粒會流動並包覆物體的輪廓；接著抽真空，顆粒之間互相擠壓而「堵塞」，整個袋子瞬間變得堅硬，牢牢鎖住物體的形狀。放開時只需通氣，袋子又變回柔軟。這個設計的巧妙之處在於：它在同一個裝置中同時實現了「柔軟以適應形狀」與「堅硬以牢固抓持」兩個矛盾的需求，而切換只需要一個真空閥。它能抓取幾乎任何形狀的物體，且完全不需要知道物體是什麼——這是機構智慧的又一個典範。

21.9 學習式抓取

21.9.1 為何轉向學習

21.4.3 節列出了解析式抓取合成的三個困難：組合爆炸、依賴精確幾何模型、對摩擦係數敏感。而真實場景中還有更根本的問題——**物體常常是沒見過的**。

倉儲分揀是最典型的例子：一個電商倉庫可能有數十萬種商品，不可能為每一種都建立 CAD 模型。傳統方法在此完全失效。

學習式方法的思路是繞過建模：**不去理解物體是什麼，而是直接學習「哪裡看起來可以抓」**。這與人類的直覺相符——我們看到一個陌生物體時，並不會先在腦中重建它的三維模型再計算力封閉，而是「一眼就知道該抓哪裡」。

21.9.2 基本流程

典型的學習式抓取系統包含四個步驟：

1. **感測：**以 RGB-D 相機取得場景的彩色影像與深度圖（第 6.7 節、第 22 章）。

2. **抓取姿勢偵測**：以神經網路預測一組候選的抓取位姿（夾爪的位置、方向與開度），並為每個候選給出成功率的預測分數。
3. **篩選與排序**：過濾掉運動學不可達或會碰撞的候選，取分數最高者。
4. **執行與回饋**：執行抓取，並記錄成功或失敗——這個結果可以回饋作為新的訓練資料。

其中步驟二是核心。常見的網路輸出形式有兩種：**離散式**（把影像切成網格，為每個位置與角度預測分數）與**生成式**（直接回歸出抓取位姿的參數）。

21.9.3 訓練資料從哪裡來

學習需要大量的標註資料，而「這個抓法會不會成功」的標註成本極高。實務上有三種來源：

來源一：模擬產生。在物理模擬器（第 14.8 節）中生成大量物體與抓取嘗試，自動標註成功與否。優點是資料量可以極大（數百萬筆）、完全自動化；缺點是 Sim-to-Real Gap——模擬中的接觸與摩擦不夠真實（第 14.7.3 節）。因應之道是**領域隨機化**：在模擬中隨機改變摩擦係數、質量、物體尺寸與光照，強迫網路學到對這些變異不敏感的特徵。

來源二：解析式標註。用 21.6 節的品質指標（如 ϵ 指標）自動為模擬中的抓取評分，作為訓練標籤。這結合了理論與學習的優勢——用理論產生資料，用學習達成泛化。

來源三：真實機器人自我監督。讓機器人不斷嘗試抓取，用「抓起來了沒有」作為標籤。優點是資料完全真實；缺點是速度慢（每次嘗試需數秒）、且需要自動化的重置機制。曾有研究讓多台機器人並行運轉數月，收集數十萬次嘗試。

21.9.4 優勢與限制

優勢一：能處理未知物體。這是相對於解析法最大的突破。網路學到的是「什麼樣的局部幾何適合夾持」這種通用特徵，因此能泛化到訓練時沒見過的物體。

優勢二：不需完整的三維模型。可以直接由單一視角的深度圖預測，不需重建完整幾何——這在物體互相遮蔽的雜亂場景中尤其重要。

優勢三：推論極快。一次前向傳播即可（數十毫秒），遠快於解析法的組合搜尋。

限制一：缺乏保證。網路給出的是「預測的成功率」，沒有力封閉那樣的數學保證。在安全關鍵的場合，這是嚴重的缺點。

限制二：難以處理特殊需求。若任務要求「必須從特定方向抓」或「抓取後要能施加特定的力」，純粹的成功率預測無法表達這些約束。

限制三：對訓練分布外的物體仍會失敗。若遇到訓練資料中完全沒有的形態（如極細長、極柔軟、透明物體），性能會明顯下降。透明與反光物體尤其困難，因為深度相機在這些表面上根本量不到正確的距離（第 6.7 節）。

表 21-6 解析式與學習式抓取的比較

面向	解析式（力封閉分析）	學習式（抓取姿勢偵測）
需要物體模型	需要精確幾何	不需要
處理未知物體	困難	擅長
計算時間	秒級（組合搜尋）	毫秒級（一次推論）
理論保證	有（力封閉條件）	無（僅機率預測）
對摩擦係數	敏感	隱含在資料中
處理雜亂場景	困難	擅長
可解釋性	高	低
典型應用	已知工件的產線取放	倉儲分揀、服務型機器人

21.9.5 混合式方法

實務上最有效的往往是兩者結合：**用學習產生候選、用解析驗證品質**。

具體流程是：神經網路快速給出數十個候選抓取位姿（利用其泛化能力與速度），再對排名前幾名的候選，用 21.6 節的品質指標做嚴格驗證（利用理論的可靠性），最後執行通過驗證且運動學可行者。

這個組合兼顧了泛化能力與安全保證，是目前工業應用的主流架構。它也體現了本書一再出現的原則——**模型與學習不是互相取代，而是各自處理擅長的部分**（呼應第 17 章的「模型處理已知、回授處理未知」）。

21.10 由抓取到操作

21.10.1 抓取只是開始

本章大部分內容討論的是**抓取（grasping）**——把物體拿起來。但真正的任務往往需要操

作 (manipulation) ——拿起來之後，還要對它做些什麼：插入、旋轉、擺放、組裝、使用工具。

兩者的難度差距很大。抓取是一個「靜態」的問題 (找到穩定的接觸配置)，操作則是「動態」的 (在維持抓握的同時控制物體的運動與受力)。

21.10.2 三種操作方式

方式一：手臂操作 (Arm Manipulation)。抓穩後，物體與手固連，由手臂的運動帶動物體。這是最簡單也最常用的方式——本書第 9 ~ 20 章的所有技術都直接適用，因為「手加物體」可視為一個新的剛體。

方式二：手內操作 (In-hand Manipulation)。在不放開的前提下，用手指改變物體在手中的姿態——例如把一支筆在手中轉個方向。這極其困難，需要多指手、精確的觸覺、以及對滾動與滑動接觸的建模。人類做這件事毫不費力，機器人至今仍相當笨拙。

方式三：非抓取式操作 (Non-prehensile Manipulation)。不抓起物體，而是用推、撥、翻、滾等方式改變其位置——例如把桌上的書推到桌邊再翻起來。這在物體太大、太重或無法抓取時特別有用，也常作為抓取前的「預處理」 (把物體推到方便抓的位置)。

21.10.3 操作中的力控制

操作階段的力控制，正是第 20 章技術的直接應用。以「把抓著的插銷插入孔中」為例：

- **軸向**：以力控制施加推力 (第 20.3 節)。
- **橫向**：以低剛度的阻抗控制允許順應對位 (第 20.5 節)。
- **手指握力**：必須足夠大以防滑落，但不能大到壓壞物體——這本身就是一個力控制問題。
- **滑動偵測**：若觸覺感測到物體開始滑動，立即增加握力 (模仿人類的 70 毫秒反射)。

這說明了第 20 章與本章的關係：**第 20 章處理「手與環境」的接觸，本章處理「手與物體」的接觸，而操作任務中兩者同時發生**。這也是抓取與操作特別困難的原因——它是一個多重接觸、多重約束的耦合問題。

21.11 第三篇的總結

21.11.1 九章的軌跡

第 21 章至此結束，第三篇「動力學、軌跡與控制」的九章也全部完成。回顧這條路徑：

第 13、14 章（動力學）建立了「力與運動」的數學模型。前者用拉格朗日法讓我們看懂方程的結構，後者用 Newton-Euler 遞迴讓我們算得夠快。

第 15 章（軌跡生成）決定「該走哪條路」，並首次把動力學模型用於檢驗軌跡的可行性。

第 16、17 章（控制）讓機器人真的走上那條路。前者建立回授的基礎與極限，後者用模型前饋突破 PID 的限制——核心哲學是「用模型處理已知、用回授處理未知」。

第 18、19 章（最佳性與穩定性）回答了兩個更深的問題：參數該怎麼選才是最好的（LQR 與 Riccati 方程），以及我們憑什麼相信它一定有效（Lyapunov 方法）。

第 20、21 章（接觸與抓取）讓機器人走出自由空間，學會與環境互動。核心洞見是：與其消除不確定性，不如設計一個對不確定性不敏感的系統——柔順性不只是安全考量，它實際上取代了大量的感測與計算。

21.11.2 三條貫穿的線索

回顧九章，有三條線索貫穿始終。

線索一：模型的價值不在完美，而在減負。第 13 章的動力學模型有 10~20% 的誤差，第 17 章仍能靠它把回授的負擔降低五倍；第 18 章的 LQR 需要線性模型，卻能為多變數系統一次算出所有增益。反覆出現的訊息是：不完美的模型仍然極有價值。

線索二：理論的各部分互相支撐。第 13 章那個「看似無用」的 $\dot{\mathbf{M}} - 2\mathbf{C}$ 斜對稱性，在第 19 章成為證明的樞紐；第 11 章的雅可比對偶，在第 17 章成為操作空間控制的核心、在第 20 章成為力映射的基礎、在本章成為抓取矩陣的原型。

線索三：機構與控制必須協同設計。第 15 章的軌跡受限於馬達能力、第 17 章的協作機器人需要低減速比、第 20 章的插銷入孔靠一個倒角就把容差放寬 41 倍、本章的欠驅動手用機構取代了感測與計算。反覆出現的教訓是：在把問題丟給控制器之前，先問能不能用機構把問題變簡單。

21.11.3 接下來：讓機器人走出實驗室

第三篇的所有技術，都建立在一個前提上：機器人知道自己在哪裡、知道目標在哪裡。固定

基座的手臂靠編碼器就能知道自己的姿態，工件的位置則由治具保證。

但一旦機器人要移動、要面對未知的環境，這個前提就不成立了。第四篇「感知、定位與自主移動」（第 22～28 章）將處理這些問題：如何用視覺理解環境（第 22 章）、行動機器人如何運動（第 23 章）、如何在充滿不確定性的世界中估計自己的狀態（第 24 章）、如何一邊移動一邊建立地圖（第 25 章）、如何規劃避開障礙的路徑（第 26 章）、以及無人機（第 27 章）與腿式機器人（第 28 章）這兩種特殊的移動形態。

屆時你會發現，第三篇的工具並未被拋棄——控制理論仍然是無人機姿態穩定與四足步態的基礎，而動力學模型則是它們規劃運動的依據。只是問題的重心，從「精確地執行」轉向了「正確地理解」。

完整計算例題

例題 21-1 摩擦錐與二指抓取判定

題目：一個長方體工件被兩指從相對的兩面夾持。指墊與工件之間的摩擦係數 $\mu = 0.4$ 。（a）求摩擦錐的半頂角。（b）若兩接觸點的連線與接觸面法線夾角為 15° ，判斷是否構成力封閉。（c）若夾角為 30° ，重新判斷。（d）若要容許 30° 的夾角，摩擦係數至少需要多少？

解（a）：由 21.2.2 節的公式：

$$\alpha = \arctan(\mu) = \arctan(0.4) = 21.8^\circ$$

摩擦錐的半頂角。

解（b）：依 21.4.2 節的判準，兩接觸點的連線必須落在兩個摩擦錐之內。連線與法線的夾角為 15° ，而錐的半頂角為 21.8° ：

$$15^\circ < 21.8^\circ \Rightarrow \text{連線落在錐內} \Rightarrow \text{構成力封閉} \checkmark$$

抓取穩固。

解（c）：夾角 30° 時：

$$30^\circ > 21.8^\circ \Rightarrow \text{連線落在錐外} \Rightarrow \text{不構成力封閉} \times, \text{手指會滑動}$$

抓取失敗。

解（d）：要容許 30° 的夾角，需要 $\alpha \geq 30^\circ$ ：

$$\mu \geq \tan(30^\circ) = 0.577$$

所需的最小摩擦係數。

工程判讀：一般橡膠指墊對塑膠的摩擦係數約在 0.5 ~ 0.8 之間，因此 0.577 是可以達成的。但考慮到 21.2.3 節提到的摩擦不確定性（表面可能有油污或灰塵使 μ 下降），設計時應留有餘裕——實務上會建議改變夾持位置，使夾角降到 15° 以下，或改用有齒紋的指墊以提高 μ 。

例題 21-2 平面三指抓取的力封閉分析

題目：一個圓形物體（半徑 $r = 50 \text{ mm}$ ）被三根手指抓持，接觸點分別位於圓周上 0° 、 120° 、 240° 的位置。摩擦係數 $\mu = 0.5$ 。（a）求各接觸點的法線方向與摩擦錐。（b）判斷此抓取能否抵抗任意方向的**平移**外力。（c）判斷能否抵抗**旋轉**外力（力矩）。（d）若把三個接觸點改為 0° 、 30° 、 60° ，重新分析。

解（a）：圓形物體的法線沿半徑方向指向圓心。三個接觸點的法線方向分別為 180° 、 300° 、 60° （即指向圓心）。摩擦錐半頂角為 $\arctan(0.5) = 26.6^\circ$ ，因此各接觸點可施加的力方向範圍為：

接觸點 1 (0°)：力方向 $180^\circ \pm 26.6^\circ$ ，即 $153.4^\circ \sim 206.6^\circ$

接觸點 2 (120°)：力方向 $300^\circ \pm 26.6^\circ$ ，即 $273.4^\circ \sim 326.6^\circ$

接觸點 3 (240°)：力方向 $60^\circ \pm 26.6^\circ$ ，即 $33.4^\circ \sim 86.6^\circ$

三個摩擦錐涵蓋的方向範圍。

解（b）：三個錐的中心方向分別為 180° 、 300° 、 60° ，兩兩相差 120° 。由 21.3.3 節的正線性跨度概念：三個互成 120° 的向量，其正線性組合可以涵蓋整個平面。

直觀的驗證：任取一個方向（如 0° ），可以由 300° 與 60° 兩個方向的正組合產生（兩者的角平分線正是 0° ）。因此能抵抗任意方向的**平移**外力。

解（c）：對於力矩，需要考慮切向力。由於每個接觸點的力可以偏離法線 $\pm 26.6^\circ$ ，其切向分量為：

$$f_t = f_n \cdot \tan(26.6^\circ) = 0.5 f_n$$

最大切向分量。

此切向力對圓心產生的力矩為：

$$M = f_t \times r = 0.5 f_n \times 0.05 = 0.025 f_n \text{ (N}\cdot\text{m, 若 } f_n \text{ 以 N 為單位)}$$

單一接觸點的最大力矩貢獻。三指同向施加切向力時可疊加，故總力矩為 $0.075 f_n$ 。

由於三指可以同時施加順時針或逆時針的切向力，**能抵抗兩個方向的力矩**。因此此抓取構成完整的力封閉。

解 (d)：若三點集中在 0° 、 30° 、 60° ，則三個法線方向為 180° 、 210° 、 240° ——**全部落在同一個半平面內**。

由正線性跨度的性質，這三個方向的非負組合只能產生「指向左方的力」，**無法產生指向右方 (0° 方向) 的力**。因此物體會被往右推出去，**不構成力封閉**。

核心結論：接觸點的分布比數量更重要。三個聚集的接觸點不如兩個分散的接觸點。這正是 21.6.2 節「接觸點分布」作為品質指標的依據，也解釋了為何人手抓握時會自然地把拇指與其他手指分置於物體兩側。

例題 21-3 柔順性的效益量化

題目：某夾爪抓取一個直徑 40 mm 的圓柱體。夾爪的定位誤差為 ± 2 mm。(a) 若指墊為剛性（等效剛度 10^5 N/m），求位置誤差造成的夾持力變化範圍（設標稱夾持力為 20 N）。(b) 若改用柔性指墊（剛度 2000 N/m），重新計算。(c) 比較兩者，並說明柔性指墊如何提升抓取的成功率。(d) 若物體是易碎品，容許的最大夾持力為 30 N，判斷兩種指墊是否可行。

解 (a)：剛性指墊時，位置誤差直接轉換為變形量，進而產生力的變化：

$$\Delta F = K \times \Delta x = 10^5 \text{ N/m} \times 0.002 \text{ m} = 200 \text{ N}$$

位置誤差 ± 2 mm 造成的力變化。

因此實際夾持力範圍為 20 ± 200 N。負值代表夾不到（誤差往外 2 mm 時，指墊根本沒接觸）；正值 220 N 則遠超標稱值。

這個結果說明剛性指墊在此誤差下根本不可用——不是抓不到，就是夾壞。

解 (b)：柔性指墊時：

$$\Delta F = 2000 \text{ N/m} \times 0.002 \text{ m} = 4 \text{ N}$$

力的變化僅 4 N。

實際夾持力範圍為 $20 \pm 4 = 16 \sim 24$ N。這是一個完全可接受的範圍。

解 (c)：兩者的力變化相差 50 倍。柔性指墊透過三個機制提升成功率：

- **吸收位置誤差**： $\pm 2 \text{ mm}$ 的誤差只造成 $\pm 4 \text{ N}$ 的力變化，遠在可接受範圍內。
- **增加接觸面積**：柔性材料變形後形成面接觸，使接觸模型由「點接觸」升級為「軟指接觸」（21.2.1 節），增加抵抗扭轉的能力。
- **提高等效摩擦**：面接觸的實際摩擦力大於理論的點接觸值，因為材料的微觀嵌合效應。

解 (d)：容許最大力 30 N 時：

剛性指墊：實際力最高可達 220 N，遠超 30 N 的上限。**不可行**——物體必定損壞。

柔性指墊：實際力最高 $24 \text{ N} < 30 \text{ N}$ 。**可行**，且還有 6 N 的餘裕。

工程結論：這個例題量化了本章的核心洞見。要讓剛性夾爪可行，必須把定位精度提升到 $\pm 0.05 \text{ mm}$ 以下（使 $\Delta F < 5 \text{ N}$ ）——這需要極昂貴的機構與感測。而換一個成本數百元的柔性指墊，就直接解決了問題。**這正是「用機構把問題變簡單」的最佳例證**，也與第 20 章例題 20-3 中「一個倒角把容差放寬 41 倍」是同一種思維。

程式範例：摩擦錐與力封閉的計算

以下 Python 程式實作平面抓取的力封閉判定：計算摩擦錐、建構抓取矩陣、以正線性跨度判斷是否力封閉，並計算抓取品質指標。

```
import numpy as np
from itertools import product

# ===== 平面抓取分析 =====
class PlanarGrasp:
    """平面 (2D) 抓取分析：接觸點在物體邊界上"""
    def __init__(s, mu=0.5):
        s.mu = mu
        s.alpha = np.arctan(mu)          # 摩擦錐半頂角
        s.contacts = []                  # [(位置, 法線方向角), ...]

    def add_contact(s, pos, normal_angle_deg):
        """pos: (x, y) 相對物體質心; normal_angle: 法線指向物體內部的角度"""
        s.contacts.append((np.array(pos, float), np.deg2rad(normal_angle_deg)))

    def cone_edges(s):
        """回傳所有摩擦錐的邊界方向 (每個接觸點兩條邊)"""
        edges = []
        for pos, n_ang in s.contacts:
```

```

    for sign in (-1, 1):
        ang = n_ang + sign*s.alpha
        edges.append((pos, np.array([np.cos(ang), np.sin(ang)])))
    return edges

def grasp_matrix(s):
    """建構抓取矩陣 G：把各錐邊的單位力映射為扳手 [Fx, Fy, Mz]"""
    cols = []
    for pos, d in s.cone_edges():
        fx, fy = d
        mz = pos[0]*fy - pos[1]*fx      # 二維外積 = 力矩
        cols.append([fx, fy, mz])
    return np.array(cols).T            # 3 x (2n)

def is_force_closure(s, n_samples=360):
    """
    力封閉判定：檢查錐邊的正線性組合能否涵蓋整個扳手空間。
    方法：對大量方向取樣，檢查每個方向是否都能被正組合達成。
    """
    G = s.grasp_matrix()
    if G.shape[1] < 3:
        return False, 0.0
    # 必要條件：原點必須在錐邊所張凸包的內部
    # 以線性規劃的對偶形式檢查：不存在一個方向 w 使所有 G 的行都在其負半空間
    min_margin = np.inf
    for i in range(n_samples):
        th = 2*np.pi*i/n_samples
        for mz_dir in (-1., 0., 1.):
            w = np.array([np.cos(th), np.sin(th), mz_dir*0.05])
            w = w/np.linalg.norm(w)
            # 檢查是否有任一錐邊在此方向上有正投影
            proj = G.T @ w
            min_margin = min(min_margin, proj.max())
    return min_margin > 1e-9, max(min_margin, 0.0)

def report(s, name):
    fc, q = s.is_force_closure()
    print(f"{name}")
    print(f"  摩擦係數  $\mu$  = {s.mu:.2f} · 摩擦錐半頂角 = "
          f"{np.rad2deg(s.alpha):.1f}°")
    print(f"  接觸點數 = {len(s.contacts)}")
    print(f"  力封閉判定 = {'✓ 成立' if fc else 'X 不成立'}")
    print(f"  品質指標  $\approx$  {q:.4f}")
    return fc, q

# ===== 例題 21-1：二指抓取 =====
print("=== 例題 21-1：二指抓取的夾角分析 ===")
mu = 0.4
print(f" $\mu$  = {mu} → 摩擦錐半頂角 = {np.rad2deg(np.arctan(mu)):.1f}°\n")
print(f"{'連線與法線夾角':>16}{'判定':>10}")
for ang in (0, 10, 15, 21, 25, 30):
    ok = ang < np.rad2deg(np.arctan(mu))

```

```

    print(f"{ang:14d}°{'✓ 力封閉' if ok else 'X 會滑動':>12}")
mu_need = np.tan(np.deg2rad(30))
print(f"\n 要容許 30° 夾角 · 需  $\mu \geq \tan(30^\circ) = \{mu\_need:.3f\}$ ")

# ===== 例題 21-2：三指抓取圓形物體 =====
print("\n=== 例題 21-2：三指抓取圓形物體 ( r = 50 mm ) ===")
r = 0.05

# (b)(c) 三指均勻分布 0°, 120°, 240°
g1 = PlanarGrasp(mu=0.5)
for a in (0, 120, 240):
    th = np.deg2rad(a)
    pos = (r*np.cos(th), r*np.sin(th))
    g1.add_contact(pos, a + 180)      # 法線指向圓心
g1.report("三指均勻分布 (0°, 120°, 240°)")

# (d) 三指聚集 0°, 30°, 60°
g2 = PlanarGrasp(mu=0.5)
for a in (0, 30, 60):
    th = np.deg2rad(a)
    pos = (r*np.cos(th), r*np.sin(th))
    g2.add_contact(pos, a + 180)
g2.report("三指聚集 (0°, 30°, 60°)")
print("> 接觸點的分布比數量更重要")

# ===== 接觸點分布對品質的影響 =====
print("\n=== 接觸點分布角度對抓取品質的影響 ( 三指、 $\mu=0.5$  ) ===")
print(f"'分布跨度':>10}{'接觸點角度':>22}{'力封閉':>10}{'品質':>10}")
for span in (60, 120, 180, 240, 300, 360):
    g = PlanarGrasp(mu=0.5)
    angles = [i*span/3 for i in range(3)]
    for a in angles:
        th = np.deg2rad(a)
        g.add_contact((r*np.cos(th), r*np.sin(th)), a + 180)
    fc, q = g.is_force_closure()
    ang_str = ", ".join(f"{a:.0f}°" for a in angles)
    print(f"{span:9d}°{'ang_str':>22}{'✓' if fc else 'X':>9}{q:10.4f}")
print("> 接觸點越分散 ( 跨度接近 360° ) · 品質越高")

# ===== 摩擦係數的影響 =====
print("\n=== 摩擦係數對二指抓取的影響 ( 夾角固定 20° ) ===")
print(f"{'μ':>8}{'錐角':>10}{'判定':>12}")
for mu_t in (0.1, 0.2, 0.3, 0.36, 0.4, 0.6, 1.0):
    a = np.rad2deg(np.arctan(mu_t))
    print(f"{mu_t:8.2f}{a:9.1f}°{'✓ 力封閉' if a > 20 else 'X 會滑動':>14}")

# ===== 例題 21-3：柔順性的效益 =====
print("\n=== 例題 21-3：指墊剛度對夾持力的影響 ===")
F_nominal, pos_err, F_limit = 20.0, 0.002, 30.0

```

```

print(f"標稱夾持力 = {F_nominal} N · 定位誤差 = ±{pos_err*1000:.0f} mm · "
      f"容許上限 = {F_limit} N\n")
print(f"{'指墊類型':<16}{'剛度 (N/m)':>14}{'力變化':>12}{'力範圍':>18}{'可行?':>8}")
for name, K in (("剛性金屬", 1e5), ("硬塑膠", 2e4),
               ("橡膠墊", 2000.), ("軟矽膠", 500.)):
    dF = K*pos_err
    lo, hi = F_nominal - dF, F_nominal + dF
    ok = (hi <= F_limit) and (lo > 0)
    print(f"{name:<16}{K:14.0f}{dF:10.1f} N"
          f"{lo:9.1f} ~{hi:6.1f} N{'✓' if ok else 'X':>7}")
print("→ 柔性指墊把力的變化縮小 50 倍，使原本不可行的任務變為可行")

# 反推：剛性指墊需要多好的定位精度？
K_rigid = 1e5
err_needed = (F_limit - F_nominal)/K_rigid
print(f"\n 若堅持用剛性指墊 (K = {K_rigid:.0e} N/m) : ")
print(f" 定位精度需達 ±{err_needed*1000:.3f} mm 才能滿足力的上限")
print(f" 這比目前的 ±{pos_err*1000:.0f} mm 嚴格 "
      f"{pos_err/err_needed:.0f} 倍 — 機構成本將大幅上升")

```

請試著修改：（一）把二指抓取的摩擦係數設為 0.1（如濕滑表面），觀察可容許的夾角縮小到多少。（二）為三指抓取加入第四指，觀察品質指標的提升幅度是否顯著（你會發現邊際效益遞減）。（三）把接觸點的位置加入 $\pm 5^\circ$ 的隨機擾動，重複計算一百次，統計力封閉成立的比例——這就是 21.6.2 節的「誤差強健性」指標。

實作：抓取實驗與柔順性驗證

本章實作以實體抓取實驗為主，重點在於**親手驗證柔順性的價值**。所需器材：任何具備夾爪的機械手臂（或第 5 章的伺服夾爪自製）、不同硬度的指墊材料（金屬、硬塑膠、橡膠、矽膠）、以及一組測試物體。

- 測試物體準備：**準備至少八種物體，涵蓋不同的形狀（圓柱、方塊、球、不規則）、材質（金屬、塑膠、布、海綿）與重量。另準備一顆生雞蛋作為易碎品的極端測試（可先用水煮蛋練習）。
- 摩擦係數量測：**用斜面法量測各種指墊材料與各種物體之間的摩擦係數——把物體放在指墊材料上，緩緩傾斜直到滑動，記錄臨界角度 θ ，則 $\mu = \tan \theta$ 。製作一張材料配對的摩擦係數表。
- 摩擦錐驗證：**依量得的 μ 計算摩擦錐半頂角，並用例題 21-1 的判準預測「哪些夾持角度會

成功」。實際測試並比對預測的準確度。

4. **指墊硬度比較**：用同一支夾爪，分別裝上四種硬度的指墊，各抓取所有測試物體十次。記錄成功率、並用彈簧秤量測夾持力。製作「指墊硬度 vs 成功率」的圖表。
5. **位置誤差容忍度測試**：刻意讓夾爪的目標位置偏離物體中心（0、1、2、3、5 mm），對各種指墊測試成功率。這是例題 21-3 的實驗版本——你應該會看到柔性指墊的容忍範圍明顯較大。
6. **手套實驗的重現**：讓同學戴上厚手套並蒙眼，嘗試抓取各種物體，記錄成功率與所需時間。再改為戴薄手套或不戴手套（但仍蒙眼）重複。討論結果是否符合本章的預期。
7. **接觸點分布實驗**：若使用三指以上的夾爪，改變手指的分布角度（聚集 vs 分散），測試抓取穩定性。用手施加不同方向的外力，記錄物體開始滑動時的力量大小——這是力封閉品質的實測。
8. **易碎品挑戰**：以生雞蛋為目標，測試各種指墊與夾持力設定。記錄「能成功抓起且不破」的參數範圍寬度——這個範圍越寬，代表系統越穩健。
9. **（進階）欠驅動手指製作**：用紙板、橡皮筋與棉線製作一根雙關節的欠驅動手指（21.8.2 節），驗證其自適應包絡的行為。與剛性手指抓取不規則物體的效果比較。

表 21-7 抓取實驗記錄表（第一列為範例）

測試項目	指墊材料	物體	設定	成功率	觀察
位置誤差容忍	橡膠 ($\mu=0.62$)	圓柱 $\phi 40$	偏移 2 mm	10/10	指墊變形吸收誤差
（自行填寫其餘測試）					

國中小也看得懂：抓東西「軟」比「準」更重要

來玩一個遊戲：戴上一副厚厚的手套，然後閉上眼睛，請旁邊的人在桌上放一樣東西，你去把它拿起來。

你會發現——居然拿得起來！明明你看不見、也摸不清楚，卻還是成功了。為什麼？

秘密就在那副**厚手套**。你可能以為手套讓事情變難了（確實，你摸不清楚形狀），但它同時也幫了你三個大忙：

第一，手套很軟，可以貼合。 不管東西是圓的、方的還是奇形怪狀，軟軟的手套都會壓扁

去配合它，變成一大片接觸面。接觸面越大，就抓得越牢。

第二，手套可以吸收錯誤。 你的手指位置一定不準（因為閉著眼睛嘛），可能差了好幾公釐。但手套會壓縮，所以差一點點沒關係。如果你的手指是硬邦邦的鐵做的，差 3 公釐不是碰不到、就是把東西撞飛了。

第三，東西快滑掉的時候，手套會自動抓更緊。 這是最神奇的——當物體開始往下滑，手套的材料會被拉扯變形，反而讓接觸變得更緊密、摩擦力變大，於是就止住了。這完全不需要你去「想」，是材料自己做到的！

所以這一章最重要的一句話是：抓東西這件事，「軟」比「準」更重要。

科學家花了三十年想讓機器人抓東西，一開始都想用超級精密的相機和超級準確的手指。後來才發現：做一隻軟軟的手，比做一隻超準的手有用多了。現在很多機器人的手都是矽膠做的，軟軟的、可以彎，抓草莓、抓雞蛋都沒問題——而且它根本不知道自己抓的是什麼！

叫獸提醒與常見錯誤

- 誤區一：以為接觸點越多抓得越穩。分布比數量重要——三個聚集的接觸點可能不如兩個分散的（例題 21-2）。
- 誤區二：忘記手指只能推不能拉。因此是「正」線性跨度而非一般線性跨度，這使所需的接觸點數增加。
- 誤區三：使用樂觀的摩擦係數。實際 μ 可能因油污、灰塵、濕度而降低數倍，設計時應取保守值並留餘裕。
- 誤區四：混淆力封閉與形封閉。前者允許用摩擦、後者完全不依賴摩擦；形封閉更嚴格但需要更多接觸點。
- 誤區五：忽略關入籠這個中間選項。許多任務不需要完全夾緊，「圍住不讓它跑掉」就足夠且穩健得多。
- 誤區六：過度追求嚴格的封閉條件。應用最寬鬆但足夠的那一個，過度嚴格會不必要地提高感測與控制的要求。
- 誤區七：以為理論品質指標高就一定成功。指標建立在幾何已知、摩擦已知、接觸點精確的假設上，且只考慮靜力學。

- 誤區八：把柔順性只當成安全考量。它實際上增加了抓取能力（面接觸能抵抗扭轉）並取代了大量的感測與計算。
- 誤區九：直接考慮多指手而不先評估簡單方案。真空吸盤、欠驅動手在許多工業場合更快、更便宜、更可靠。
- 誤區十：學習式抓取完全取代解析方法。前者缺乏保證，實務上最佳解是「學習產生候選、解析驗證品質」的混合式。
- 誤區十一：忽略深度相機在透明與反光物體上的失效。這類物體是學習式抓取的主要失敗來源（第 6.7 節）。
- 誤區十二：把抓取與操作混為一談。抓穩是靜態問題，操作是動態問題；手內操作至今仍是研究前沿。

課堂討論

1. 開頭的手套實驗中，厚手套同時降低了感知能力並提升了柔順性，而結果是成功的。請討論：這對「該把資源投在感測還是機構」這個工程決策有何啟示？
2. 例題 21-3 顯示柔性指墊把力變化縮小 50 倍，使原本需要 $\pm 0.05 \text{ mm}$ 精度的任務變為只需 $\pm 2 \text{ mm}$ 。請再舉出兩個「用機構降低對感測或控制要求」的例子（可回顧第 20 章）。
3. 顆粒堵塞夾爪在同一裝置中實現了「柔軟以適應」與「堅硬以固定」兩個矛盾的需求。請討論這種「可切換的物理性質」還能應用在哪些機器人問題上？
4. 學習式抓取沒有力封閉那樣的數學保證。請討論：在哪些應用中這個缺點是可接受的？在哪些應用中不可接受？如何設計混合式系統來兼顧？
5. 人手的靈巧主要來自機構（分層柔軟度、筋腱驅動、欠驅動耦合）而非感知。請討論：這是否意味著機器人手的研究方向應該調整？多指靈巧手的研究價值何在？

章末摘要

- 抓取問題分三個層次：接觸建模（單點能傳遞什麼力）、抓取分析（這個抓法穩不穩）、抓取規劃（該抓哪裡）。
- 三種接觸模型：無摩擦點接觸（1 自由度）、有摩擦點接觸（3 自由度）、軟指接觸（4 自由度，可抵抗扭轉）。

- 摩擦錐的半頂角 $\alpha = \arctan(\mu)$ ；接觸力必須落在錐內，否則會滑動。
- 摩擦係數不確定性大（可差四倍），設計時應取保守值。
- 抓取矩陣 G 把各接觸力映射為物體所受的合力與合力矩，角色類似雅可比。
- 力封閉：能以摩擦錐內的接觸力抵抗任意方向的外力；數學條件是接觸力的正線性跨度涵蓋整個扳手空間。
- 因手指只能推不能拉，故為「正」線性跨度，這使所需的接觸點數增加。
- 力封閉的最少接觸點數：平面有摩擦 2 點、空間有摩擦 3 點；無摩擦則分別需 4 點與 7 點。
- 平面二指抓取的判準：兩接觸點的連線必須落在兩個摩擦錐之內。
- 形封閉完全不依賴摩擦（更嚴格、需更多接觸點），關入籠則只包圍不夾緊（最寬鬆、對誤差不敏感）。
- 抓取品質指標有四類：最小奇異值 ε 、抓取體積、接觸點分布、誤差強健性；但理論指標與實機成功率仍有落差。
- 接觸點的分布比數量更重要——三個聚集的接觸點可能不如兩個分散的。
- 柔順性在抓取中的四個作用：增加接觸面積、吸收位置誤差、滑動時自動增加摩擦、多指自適應包絡。
- 本章核心洞見：抓取的成功往往不是來自精準，而是來自順應；柔順性取代了大量的感測與計算。
- 三種機械手設計：剛性多指手（能力強但昂貴複雜）、欠驅動手（機構自動適應、控制極簡）、軟性夾爪（安全、適應性強但出力小）。
- 學習式抓取直接由感測資料預測可行的抓取位姿，擅長處理未知物體與雜亂場景，但缺乏理論保證；實務最佳解是混合式。

觀念題與計算題

觀念題

1. 說明抓取問題的三個層次，並指出各自的核心問題。
2. 列舉抓取困難的五個因素，並說明它們為何難以用「精確建模」的路線解決。
3. 比較三種接觸模型可傳遞的力與自由度，並說明軟指接觸為何特別重要。

4. 定義摩擦錐並寫出其半頂角公式；說明摩擦係數的不確定性對設計的影響。
5. 說明抓取矩陣的作用，並將其與雅可比矩陣類比。
6. 定義力封閉，並解釋為何條件中「任意」兩字是關鍵。
7. 為何力封閉的條件是「正」線性跨度而非一般線性跨度？這帶來什麼推論？
8. 寫出力封閉的最少接觸點數（平面與空間、有摩擦與無摩擦），並說明摩擦的價值。
9. 敘述平面二指抓取的力封閉判準，並解釋其幾何直覺。
10. 比較力封閉、形封閉與關入籠三個概念，並說明選擇的原則。
11. 列舉四種抓取品質指標，並說明理論指標與實機成功率落差的三個原因。
12. 說明柔順性在抓取中的四個作用，並解釋「順應勝過精準」的意義。
13. 比較剛性多指手、欠驅動手與軟性夾爪的設計哲學與適用場合。
14. 說明欠驅動手指如何達成自適應包絡，以及這體現了什麼設計原則。
15. 說明學習式抓取的四個步驟與三種訓練資料來源。
16. 比較解析式與學習式抓取的優缺點，並說明混合式方法的架構。

計算題

1. 某夾爪的指墊與工件的摩擦係數 $\mu = 0.6$ 。(a) 求摩擦錐半頂角。(b) 若兩接觸點的連線與法線夾角為 25° ，判斷是否力封閉。(c) 求此 μ 值下可容許的最大夾角。(d) 若表面被油污染使 μ 降為 0.25，重新判斷 (b) 的情形。
2. 一個正方形物體（邊長 60 mm）被四指抓持，接觸點位於四個邊的中點。 $\mu = 0.5$ 。(a) 求各接觸點的法線方向。(b) 判斷此抓取能否抵抗任意方向的平移力。(c) 若移除其中一指（剩三指），重新判斷。(d) 若四指改為集中在相鄰的兩個邊上，分析結果。
3. 某夾爪標稱夾持力 15 N，定位誤差 ± 3 mm。(a) 若指墊剛度為 5×10^4 N/m，求夾持力的變化範圍。(b) 若物體容許的最大夾持力為 25 N，判斷是否可行。(c) 求使夾持力變化不超過 ± 8 N 所需的指墊剛度上限。(d) 若堅持使用 (a) 的剛性指墊，求所需的定位精度。
4. 某三指夾爪抓取半徑 40 mm 的圓柱體，三指均勻分布（相隔 120° ）， $\mu = 0.45$ ，每指的法向力為 12 N。(a) 求摩擦錐半頂角與各指的最大切向力。(b) 求單指能產生的最大力矩（對圓心）。(c) 求三指同向施力時的總力矩。(d) 若物體重 1.5 kg，判斷此抓取能否在

垂直提起時抵抗重力（提示：比較三指的最大切向力總和與重量）。

動手做與專題延伸

本章實作：抓取分析工具

延續第 20 章的力控制模組，加入抓取分析功能。

- **階段一**：實作平面抓取的摩擦錐計算與二指力封閉判定，用例題 21-1 驗證。
- **階段二**：實作抓取矩陣的建構與多指的力封閉判定（可用線性規劃或凸包判斷），用例題 21-2 驗證。
- **階段三**：實作抓取品質指標（至少包含 ϵ 指標與接觸點分布指標），並比較不同抓取配置的評分。
- **階段四**：實作簡易的抓取合成——給定一個平面多邊形物體，自動搜尋所有可行的二指抓取，並依品質排序。
- **階段五**：加入誤差強健性分析——對接觸點加入隨機擾動，統計力封閉成立的比例。

專題延伸

- **抓取品質地圖**：對一個二維物體（如不規則多邊形），窮舉所有可能的二指抓取位置對，計算每一組的品質指標，繪成熱力圖。你會看到「好抓的位置」形成明顯的區塊——這正是學習式方法試圖用網路學會的東西。比較這張圖與你的直覺是否一致。
- **柔順性的量化實驗**：製作或取得三種不同硬度的指墊，設計一組實驗量化「柔順性換來多少誤差容忍」。具體作法是：對每種指墊，逐步增加夾爪的定位誤差，記錄抓取成功率降到 50% 時的誤差值。繪出「指墊剛度 vs 容許誤差」曲線，並與例題 21-3 的理論預測比較。
- **進階：欠驅動手指的設計與分析**。設計一根雙關節的欠驅動手指（可用 3D 列印或紙板製作），以彈簧與連桿實現力的自動分配。（一）分析其在不同物體形狀下的接觸點分布；（二）量測抓取不同直徑圓柱體時的接觸力分配；（三）與剛性手指比較，量化「自適應」帶來的成功率提升。若有能力，可進一步用第 13 章的靜力學分析推導其力分配的理論值，並與實測比較。

參考文獻與延伸閱讀

- Murray, R. M., Li, Z. and Sastry, S. S., A Mathematical Introduction to Robotic

Manipulation, CRC Press (第 5 章：多指抓取的完整數學理論) 。

- Bicchi, A. and Kumar, V., "Robotic Grasping and Contact: A Review," ICRA 2000 (抓取理論的經典回顧論文) 。
- Nguyen, V.-D., "Constructing Force-Closure Grasps," IJRR, 1988 (力封閉抓取合成的奠基工作) 。
- Rodriguez, A., Mason, M. T. and Ferry, S., "From Caging to Grasping," IJRR, 2012 (關入籠與抓取的關係) 。
- Ferrari, C. and Canny, J., "Planning Optimal Grasps," ICRA 1992 (ϵ 品質指標的提出) 。
- Birglen, L., Laliberté, T. and Gosselin, C., Underactuated Robotic Hands, Springer (欠驅動手的設計與分析) 。
- Brown, E. et al., "Universal Robotic Gripper Based on the Jamming of Granular Material," PNAS, 2010 (顆粒堵塞夾爪) 。
- Mahler, J. et al., "Dex-Net 2.0: Deep Learning to Plan Robust Grasps with Synthetic Point Clouds and Analytic Grasp Metrics," RSS 2017 (結合解析指標與深度學習的代表性工作) 。
- Levine, S. et al., "Learning Hand-Eye Coordination for Robotic Grasping with Deep Learning and Large-Scale Data Collection," IJRR, 2018 (大規模真實資料的自我監督學習) 。
- Siciliano, B. and Khatib, O. (Eds.), Springer Handbook of Robotics, 2nd ed., Springer (抓取與靈巧操作專章) 。

台灣自編大學教科書

[@prof](#)

機器人學導論

機構、感知、控制與具身人工智慧

第 四 篇

感知、定位與自主移動

第 22 章 機器人視覺與感知

第 23 章 行動機器人運動學

第 24 章 狀態估測與感測器融合

第 25 章 同步定位與建圖 (SLAM)

第 26 章 路徑規劃與導航

第 27 章 無人機與飛行機器人

第 28 章 腿式機器人與運動控制

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

2026 年 9 月

第四篇 篇章導讀

第三篇教你機器人該用多大的力氣動起來，這一篇要教你它如何知道自己在哪裡、又該往哪裡去。

前三篇的所有技術，都建立在一個前提上：機器人知道自己在哪裡、也知道目標在哪裡。固定基座的手臂靠編碼器就知道姿態，工件的位置則由治具保證。但一旦機器人要移動、要面對未知的環境，這個前提就崩解了——而第四篇的七章，就是要重建它。

第 22 章（視覺與感知） 建立由影像走向三維世界的完整鏈條。相機把三維壓成二維，丟掉的深度永遠救不回來；本章要處理的，正是如何用校正、雙視角、深度感測與先驗知識，把那個維度補回來。

第 23 章（行動機器人運動學） 處理輪型機器人的運動。它的核心概念是**非完整約束**——輪子不能側滑，這使得「當下能走的方向」被限制了，但「長期能到的地方」卻沒有。理解這個看似矛盾的性質，就理解了為何路邊停車要來回好幾次。

第 24 章（狀態估測） 是本篇的數學核心。里程計短期準但會漂移、外部感測不漂移但更新慢——該相信哪一個？答案是**依可信度加權**，而這正是 Kalman 濾波器的全部祕密。本章也會揭曉第 6 章那條互補濾波公式的真正來源。

第 25 章（SLAM） 解開機器人學最著名的雞生蛋難題：要建圖必須先定位、要定位又必須先有地圖。解法出乎意料地優雅——不要分開解，一起解。而全章最戲劇性的時刻，是迴路閉合時整張地圖被瞬間扭正。

第 26 章（路徑規劃） 決定該往哪裡走。它的核心抽象是**位形空間**——把複雜形狀的碰撞問題，轉化為「一個點是否落在禁區」。而在畫不出來的六維空間裡，取樣式方法給出了「不建構、只查詢」的突破。

第 27、28 章（飛行與行走） 則是全書技術的綜合考驗。無人機六個自由度卻只有四顆馬達、且失控即墜落；腿式機器人自由度更高，且每走一步接觸狀態就要切換一次。兩者都必須靠**分層架構與簡化模型**才能駕馭。

這一篇要回答的七個問題

- 相機看到零件在第 320 個像素，機器手臂該把夾爪移到哪裡？

- 兩輪小車為什麼不能往正右方平移？停車為什麼要來回好幾次？
- 兩個定位系統給出不同答案，該相信哪一個？還是各取一半？
- 要建圖得先定位、要定位得先有地圖——這個循環怎麼打破？
- 六維空間長什麼樣？機器人如何在一個看不見的空間裡找路？
- 六個自由度只有四顆馬達，無人機憑什麼飛得這麼穩？
- 為什麼說「走路的本質，是一連串被打斷的跌倒」？

三條貫穿全篇的線索

線索一：量化不確定性，而非忽略它。 第三篇的世界是確定的——給定關節角度，末端位置唯一確定。本篇的世界則充滿雜訊：每一個感測讀值、每一次辨識、每一步落腳都可能出錯。因應之道不是假裝它不存在，而是用協方差、誤差橢圓與捕捉點裕度**把它算出來**。

線索二：依可信度融合，而非二選一。 這是第 24 章的核心，也是整篇的主軸。里程計與視覺、光達與 IMU、模型與學習——正確的作法從來不是切換或覆蓋，而是依各自的不確定性加權。這個原則在 SLAM 的迴路閉合、無人機的姿態估測、腿式機器人的狀態融合中反覆出現。

線索三：用簡化模型處理複雜系統。 第 26 章把機器人簡化為位形空間中的一個點、第 27 章用小角度近似把姿態與位置解耦、第 28 章更把三十自由度的人形機器人化為一根倒立擺。這些抽象使原本不可解的問題變得可解——而底層仍用完整模型執行，兩者透過分層架構銜接。

讀完這一篇之後

你將能夠：校正相機並由影像推算三維位置、分析各種輪型的運動約束、實作 Kalman 與粒子濾波器融合多重感測、建立 SLAM 系統並處理迴路閉合、以 A* 與 RRT 規劃路徑並設計分層導航、以及理解無人機與腿式機器人這兩類欠驅動系統的控制架構。

到這裡，機器人已經具備了完整的**物理能力**：它能看見、能移動、能定位、能導航、能飛行也能行走。但它仍然缺少一件關鍵的東西——**理解與決策**。目前為止的所有能力，都建立在「人類明確地告訴它該做什麼」之上。

第五篇「智慧機器人與具身人工智慧」將處理這個轉變。屆時你會發現，前四篇建立的所有能力，都將成為第五篇的執行基礎——一個大型模型可以理解「幫我把桌上的杯子拿到廚房」，但要真正做到，需要視覺辨識、抓取、導航與控制。**智慧與身體，缺一不可。**

本篇章節配置

章次	章名	核心內容	叫獸開講
第 22 章	機器人視覺與感知	針孔模型、相機校正、深度恢復、特徵匹配、手眼校正、視覺伺服	「相機看到零件在第 320 個像素，手臂該移到哪裡？」——相機會說謊
第 23 章	行動機器人運動學	輪型與約束、瞬時旋轉中心、差速與阿克曼、非完整性、里程計	「兩輪小車為什麼不能往正右方平移？」——瞬時受限，全域自由
第 24 章	狀態估測與感測器融合	貝氏濾波、協方差、Kalman 與 EKF/UKF、粒子濾波、互補濾波	「兩個定位系統都不完全對，該相信哪一個？」——依可信度加權
第 25 章	同步定位與建圖 (SLAM)	雞生蛋問題、EKF-SLAM、FastSLAM、佔據柵格、圖最佳化、迴路閉合	「要建圖得先定位，要定位得先有地圖——怎麼辦？」——一起解
第 26 章	路徑規劃與導航	位形空間、A* 與啟發函數、PRM 與 RRT、勢場與 DWA、分層導航	「六維空間長什麼樣？」——既然畫不出來，就用取樣去探索
第 27 章	無人機與飛行機器人	分配矩陣、欠驅動、微分平坦、串級控制、姿態估測、分級降級	「六個自由度只有四顆馬達，憑什麼飛得穩？」——時間尺度分離
第 28 章	腿式機器人與運動控制	混合動態、ZMP、線性倒立擺、捕捉點、步態、全身控制、強化學習	「走路的本質，是一連串被打斷的跌倒。」——30 自由度化為一根倒立擺

第四篇 感知、定位與自主移動

第 22 章

機器人視覺與感知

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為一張筆直鐵軌延伸至地平線的照片，兩條平行軌道在遠方交會成一點，旁註「相機會說謊：三維壓成二維，深度永遠救不回來」。中央為針孔相機模型的幾何示意——物體、投影中心、成像平面與相似三角形，並以一條由像素射出的虛線延伸成空間中的射線，標註「影像上的一點 $\Leftrightarrow$ 空間中的一條射線」。右半為一支機械手臂末端裝著相機正對工件，畫面上疊印四個座標系（像素、相機、末端、基座）與串接它們的變換箭頭。標題文字：從像素走向三維世界。

叫獸開講 「相機看到零件在第 320 個像素，手臂該移到哪裡？」

叫獸在投影幕上打出一張照片：一條筆直的鐵軌延伸到遠方，兩條軌道在地平線處交會成一個點。

「請問，」他指著螢幕，「這兩條鐵軌，在真實世界裡是不是平行的？」全班點頭。「那照片上呢？」「相交了。」

「所以第一件事：**相機會說謊**。」叫獸在白板上寫下這五個字，「它把三維的世界壓成二維的影像，過程中丟掉了一整個維度——**深度**。而丟掉的資訊，永遠救不回來。」

他換了一張投影片：一張照片上有一顆看起來很大的橘子。「這顆橘子有多大？」小翰說：「大概拳頭大吧。」叫獸按下一鍵，鏡頭拉遠——那其實是一顆離鏡頭很近的小彈珠。「你剛才判斷錯了。為什麼？因為你潛意識假設了它的距離。**單張照片無法區分「大而遠」與「小而近」**，這叫做**尺度歧義**。」

小妤舉手：「那用兩台相機不就好了？像人的兩隻眼睛。」

「很好，這確實是一條路。但我要先問一個更基礎的問題。」叫獸拿出一台工業相機與一支雷射筆，「假設現在我用相機看到一個零件在畫面的第 320 個像素、第 240 個像素的位置。請問——**機器手臂該把夾爪移到哪裡？**」

教室安靜了。小翰試著說：「畫面中央？」「畫面中央是相機的中央，不是機器人的中央。」叫獸搖頭，「像素座標是相機的語言，機器人的關節聽不懂。中間至少缺了三件事：**相機的內部參數**（畫面上一個像素對應多少角度）、**相機與機器人的相對位置**（相機裝在哪裡、朝哪個方向）、以及**深度**（那個零件離相機多遠）。少了任何一項，這個轉換就做不出來。」

他在白板上畫了一條鏈：**像素 → 相機座標 → 機器人座標 → 關節角度**。「這條鏈的後半段，我們在第 8 章與第 10 章已經學會了。今天要補上的是前半段——**如何從影像走向三維世界**。」

最後他補了一句：「還有一件事你們必須有心理準備。第二篇與第三篇的世界很乾淨：給定關節角度，末端位置是唯一確定的。但從今天開始，**每一個數字都帶著不確定性**。光線會變、陰影會動、玻璃會反光、物體會被遮住。感知不是精確的計算，而是在**充滿雜訊的證據中做出最合理的推測**。這個心態的轉換，比任何一條公式都重要。」

本章為什麼重要

第三篇的所有技術都建立在一個前提上：**機器人知道自己在哪裡、知道目標在哪裡**。固定基座的手臂靠編碼器就知道姿態，工件位置則由治具保證。但一旦要面對未知的環境，這個前提就崩解了。

視覺是彌補這個缺口最重要的工具。它提供的資訊密度遠超其他感測器——一張影像包含數十萬個像素，能同時告訴你物體的位置、形狀、顏色、紋理、文字甚至狀態。這是光達或超音波做不到的。

但代價是：**這些資訊需要大量的處理才能使用**（第 6.7.1 節）。光達直接給你距離，相機給你的是一堆數字。本章的任務，就是建立由「像素」走向「三維世界」的完整鏈條：相機模型告訴我們投影如何發生、校正讓我們知道相機的真實參數、深度感測補回丟失的維度、手眼校正把相機座標接回機器人座標，而視覺伺服則讓機器人能直接依影像回授來運動。

本章也是第四篇的起點。第 23～28 章的移動機器人、定位、SLAM 與自主導航，全部依賴感知作為輸入。理解視覺的能力**與極限**，是後續所有章节的基礎。

學習目標

- 能說明針孔相機模型，並寫出投影方程。
- 能區分相機的內部參數與外部參數，並說明各自的物理意義。
- 能說明鏡頭畸變的成因與校正方法。
- 能敘述相機校正的流程並說明棋盤格標定板的作用。
- 能說明由影像回推三維位置所缺少的資訊，並列舉三種補足方式。
- 能比較常見的影像特徵並說明特徵匹配的流程。
- 能說明手眼校正的兩種構型與其求解問題的形式。
- 能區分眼在手上與眼在手外的優缺點與適用場合。
- 能說明視覺伺服的兩種架構及其差異。
- 能推導影像雅可比並說明其與第 11 章雅可比的串接關係。
- 能說明深度學習在物體偵測與位姿估計中的角色與限制。

- 能分析視覺系統常見的失效情境並提出因應方式。

先備知識

第 8 章的座標系與齊次變換（本章大量使用，尤其是轉換樹的概念）、第 6.7 節的相機與深度感測器、第 11 章的雅可比（視覺伺服需要串接影像雅可比與機器人雅可比）、第 10 章的逆運動學。

數學方面需要：齊次座標、矩陣運算、以及基本的最小平方法概念。本章的推導不深，但座標系的追蹤極為重要——建議隨時畫圖標記每個量屬於哪個座標系，這是避免混淆的關鍵。

核心名詞中英對照

表 22-1 本章核心名詞

中文	英文	說明
針孔相機模型	Pinhole Camera Model	以單一投影中心描述成像的理想模型
焦距	Focal Length	投影中心到成像平面的距離
主點	Principal Point	光軸與成像平面的交點
內部參數	Intrinsic Parameters	描述相機自身成像特性的參數
外部參數	Extrinsic Parameters	描述相機在世界中位姿的參數
鏡頭畸變	Lens Distortion	鏡頭造成的影像幾何失真
相機校正	Camera Calibration	求出內外部參數的程序
尺度歧義	Scale Ambiguity	單張影像無法區分大而遠與小而近
對極幾何	Epipolar Geometry	描述雙視角之間幾何關係的理論
視差	Disparity	同一點在左右影像中的位置差
特徵點	Feature Point	影像中易於重複偵測與匹配的位置
描述子	Descriptor	描述特徵點鄰域外觀的向量
手眼校正	Hand-eye Calibration	求相機與機器人之間變換的程序
眼在手上	Eye-in-hand	相機安裝於機器人末端的構型
眼在手外	Eye-to-hand	相機固定於環境中的構型
視覺伺服	Visual Servoing	以視覺回授直接驅動機器人運動
影像雅可比	Image Jacobian	相機運動與影像特徵變化的關係矩陣
位姿估計	Pose Estimation	由影像推算物體的位置與姿態

22.1 針孔相機模型

22.1.1 最簡單也最有用的模型

相機成像的物理過程極為複雜（涉及透鏡、光圈、感光元件），但工程上有一個極簡卻極有效的近似：**針孔相機模型**。

想像一個不透光的盒子，前方有一個極小的孔。外界的光線只能穿過這個孔投射到後方的成像平面上。由於孔極小，物體上每一點只有一條光線能通過——因此成像是清晰的，且遵守簡單的相似三角形關係。

設相機座標系的原點在針孔（稱為**投影中心**）、Z 軸沿光軸指向前方。空間中一點 $P = (X, Y, Z)$ 投影到成像平面上的位置為：

$$x = f \cdot X/Z, \quad y = f \cdot Y/Z$$

投影方程。f 為焦距（投影中心到成像平面的距離）。這是本章最基本的一條式子。

22.1.2 除以 Z：一切問題的根源

上式中的 **除以 Z** 是整個機器人視覺的核心困難所在。它帶來三個直接的後果：

後果一：深度資訊被摧毀。同一個 (x, y) 可以對應無限多個 (X, Y, Z) ——只要 X/Z 與 Y/Z 的比值相同即可。這在幾何上意味著：**影像上的一個點，對應到空間中的一整條射線**。

後果二：尺度歧義。這正是開頭故事中「橘子還是彈珠」的問題。物體變大兩倍、距離也變遠兩倍，影像完全相同。

後果三：非線性。除法使投影成為非線性運算，這使得許多線性代數的工具無法直接套用——這也是齊次座標（第 8.6 節）在視覺中如此重要的原因，它能把除法「藏」到最後一步。

$$\text{影像上的一點} \iff \text{空間中的一條射線}$$

這個對應關係值得牢記。它解釋了為何要恢復三維，必須引入額外的資訊（第 22.4 節）。

22.1.3 由米制到像素

上述的 (x, y) 單位是長度（如公釐），但影像處理的單位是**像素**。兩者的轉換需要三個參數：

$$u = f_x \cdot X/Z + c_x$$

$$v = f_y \cdot Y/Z + c_y$$

像素座標。 f_x 、 f_y 為以像素為單位的焦距， c_x 、 c_y 為主點（光軸與成像平面的交點，通常接近影像中心）。

為何 f_x 與 f_y 可能不同？因為感光元件的像素未必是正方形。實務上多數相機兩者接近相等，但校正時仍分開估計。

寫成矩陣形式（使用齊次座標）：

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

中間那個 3×3 矩陣稱為內部參數矩陣 K 。 s 為尺度因子（實際上就是 Z ），這正是齊次座標把除法延後的技巧。

22.2 內部參數、外部參數與畸變

22.2.1 內外部參數的分工

完整的成像過程包含兩個步驟，各由一組參數描述。

外部參數（Extrinsic）：描述相機在世界中的位姿，即世界座標系到相機座標系的變換。它就是第 8 章的齊次變換矩陣：

$${}^cP = {}^cR_W \cdot {}^Wp + {}^c t_W \quad \text{即} \quad {}^cT_W = [R \mid t]$$

外部參數。共 6 個自由度（3 平移 + 3 旋轉）。它描述的是「相機在哪裡」，與相機本身無關。

內部參數（Intrinsic）：描述相機自身的成像特性，即由相機座標系到像素座標的投影。就是 22.1.3 節的矩陣 K ，共 4 個參數（ f_x 、 f_y 、 c_x 、 c_y ）。它描述的是「相機長什麼樣」，與相機在哪裡無關。

兩者串接就是完整的投影：

$$s \cdot \tilde{p} = K \cdot [R \mid t] \cdot \tilde{P}$$

完整的相機投影方程。 $\tilde{P}$ 為世界座標的齊次表示， $\tilde{p}$ 為像素座標的齊次表示。 $K \cdot [R \mid t]$ 合稱為投影矩陣。

◎ 為何要區分內外部參數

這個區分有極重要的工程意義。內部參數在相機出廠後就固定了（除非變焦或更換鏡頭），因此只需校正一次，之後可以永久使用。外部參數則隨相機的安裝位置而變，每次移動相機都必須重新求取。實務上這意味著：內部參數校正可以在實驗室從容進行；而外部參數（尤其是相機與機器人的關係）則必須在現場做手眼校正（22.6 節），且機器人一旦被碰撞移位就要重做。

22.2.2 鏡頭畸變

針孔模型假設光線直線傳播，但真實的鏡頭會使光線彎折，造成**畸變**。最主要的兩類是：

徑向畸變 (Radial Distortion)：由鏡片的球面形狀造成，使影像呈桶形 (barrel，中央凸出) 或枕形 (pincushion，邊緣拉伸)。廣角鏡頭的桶形畸變尤其明顯。

$$x_d = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6), \quad r^2 = x^2 + y^2$$

徑向畸變模型。k₁、k₂、k₃ 為畸變係數。注意畸變量與 r 有關——離影像中心越遠，畸變越嚴重。

切向畸變 (Tangential Distortion)：由鏡頭與感光元件未完全平行造成，通常較小。以 p₁、p₂ 兩個係數描述。

因此完整的相機模型共有 **4 個內部參數 + 5 個畸變係數 = 9 個參數** (外加每張影像的 6 個外部參數)。

22.2.3 畸變對機器人的影響

畸變的實務影響常被低估。一個典型的廣角鏡頭在影像邊緣的畸變可達數十個像素——若不校正，直接把像素座標換算成空間位置，誤差可能達到數公分。

因此標準的處理流程是：**先校正畸變、再做任何幾何計算**。多數視覺函式庫提供 undistort 函式，能把畸變影像轉換成符合針孔模型的理想影像。

一個實務要點：**畸變校正會裁掉影像邊緣** (因為校正後的有效區域是不規則的)。若視野本來就緊繃，校正後可能不夠用——這在選鏡頭時就應納入考量。

22.3 相機校正

22.3.1 校正在做什麼

相機校正的目標，是求出 22.2 節所述的內部參數與畸變係數。它的基本原理是：

◎ 校正的基本邏輯

拍攝一個幾何尺寸已知的物體 (標定板)，我們就同時知道了「空間中的點在哪裡」與「它成像在哪個像素」。有了足夠多的這種對應關係，就能反推出相機的參數——這是一個**最小平方問題**：找出一組參數，使得「用該參數預測的像素位置」與「實際觀測到的像素位置」差距最小。

最常用的標定板是**棋盤格**，原因有三：角點的位置可以用亞像素精度精確偵測 (黑白交界處

的梯度極大)、格子的尺寸精確已知、且平面的假設簡化了數學。

22.3.2 標準流程

Zhang 於 2000 年提出的方法是目前的標準，流程為：

1. **準備標定板**：列印或訂製一張棋盤格，量測其格子的精確尺寸。板子必須**夠平**——貼在玻璃或壓克力板上是常見作法。
2. **拍攝多張影像**：從不同的角度與距離拍攝標定板，建議 15 ~ 30 張。**關鍵是姿態要多樣**——若所有照片都是正對著板子拍的，某些參數會無法分離。
3. **偵測角點**：以程式自動找出每張影像中所有的棋盤格角點，並以亞像素精度精修。
4. **初始估計**：由平面單應性 (homography) 的性質，線性求解出參數的初始值。
5. **非線性最佳化**：以重投影誤差為目標函數，用 Levenberg-Marquardt 等方法精修所有參數 (含畸變係數)。
6. **驗證**：檢查重投影誤差——把已知的三維點用求得的參數投影回影像，與實際偵測的角點比較。

22.3.3 如何判斷校正好不好

重投影誤差 (reprojection error) 是最重要的品質指標：

$$\text{重投影誤差} = \text{平均} || \text{預測的像素位置} - \text{實際偵測的像素位置} ||$$

以像素為單位。這個數字直接反映模型與現實的吻合程度。

經驗判準是：

表 22-2 重投影誤差的判讀

重投影誤差	品質	說明
< 0.3 像素	優良	可用於精密應用 (如視覺量測)
0.3 ~ 0.5 像素	良好	一般機器人應用足夠
0.5 ~ 1.0 像素	可接受	精度要求不高時可用
> 1.0 像素	不佳	應重新校正，檢查板子是否平整、角點偵測是否正確

三個常見的校正失敗原因：一、**標定板不夠平** (列印在紙上拿在手中拍是最常見的錯誤)；二、**拍攝姿態不夠多樣** (都從同一角度拍，參數無法分離)；三、**影像模糊或曝光不當** (角點偵

測不準)。

22.4 由影像回到三維

22.4.1 缺少的那一個維度

回到 22.1.2 節的核心問題：影像上的一點對應空間中的一條射線。要確定該點在射線上的哪個位置，必須引入額外的資訊。共有三類方法。

22.4.2 方法一：第二個視角

若同一個點能被兩個位置不同的相機看到，就有兩條射線，它們的交點即為該點的三維位置。這稱為三角測量 (triangulation)。

第 6.7.2 節已介紹過立體視覺的深度公式：

$$Z = (f \cdot B) / d$$

B 為基線 (兩相機的間距)、 d 為視差 (同一點在兩影像中的像素位置差)。

這個公式有一個關鍵的性質：深度誤差與距離的平方成正比。

$$\Delta Z \approx (Z^2 / (f \cdot B)) \cdot \Delta d$$

微分上式所得。這意味著遠處的深度估計會急遽劣化——這是立體視覺的根本限制。

雙視角之間的幾何關係由對極幾何 (epipolar geometry) 描述。它的核心結論是：左影像中的一個點，在右影像中的對應點必定落在一條特定的直線上 (稱為對極線)。這把二維的搜尋降為一維，是立體匹配演算法的基礎。

22.4.3 方法二：主動測距

與其被動地推算深度，不如主動地量測它。這就是第 6.7.2 節的結構光與 ToF 深度相機——它們直接輸出每個像素的距離。

優點是不需要紋理 (立體視覺在白牆上會失效)、計算量小。缺點如第 6.7.2 節所述：結構光只能室內使用、ToF 在牆角有多重反射誤差、且兩者對透明與反光物體都會失效。

22.4.4 方法三：先驗知識

第三條路是利用已知的資訊。若我們已經知道物體的尺寸或形狀，就能由影像中的大小反推距離。

這在工業上極為常見。例如：已知工件的 CAD 模型，由影像中的輪廓即可解出其六自由度位姿（這稱為 **PnP 問題**，Perspective-n-Point）。或者更簡單的作法——使用**視覺標記**（如 AprilTag、ArUco），它們的尺寸與圖案都是已知的，因此單張影像就能算出完整的位姿。

◎ 視覺標記：工程上的務實選擇

在研究論文中，用視覺標記常被認為是「作弊」——因為它繞過了物體辨識的困難。但在工程實務上，這往往是最正確的決定。一張成本幾元的 AprilTag，能提供公釐級的位姿估計、毫秒級的計算、以及極高的可靠度。若你的應用允許在物體或治具上貼標記（例如自家產線的載具、AGV 的充電站、機器人的校正基準），**就應該優先採用**。把辨識的困難交給演算法，只在無法貼標記時才是必要的。這與第 20 章「用一個倒角解決插銷入孔」是同一種工程思維。

表 22-3 三種恢復深度方法的比較

方法	所需條件	優點	限制	典型應用
立體視覺	兩個視角、有紋理	被動、可戶外	無紋理失效、遠距精度差	戶外導航、自駕車
結構光 / ToF	主動投射	不需紋理、計算量小	室內限制、怕透明反光	室內操作、桌面抓取
先驗知識 (PnP)	已知模型或標記	單張影像即可、精度高	需事先建模或貼標記	工業定位、標記導航

22.5 影像特徵與匹配

22.5.1 為何需要特徵

一張影像有數十萬個像素，但其中絕大多數是「不特別」的——一片白牆上的任一點看起來都一樣。若要在兩張影像之間找對應關係（立體視覺、運動估計、物體辨識都需要），逐像素比對既慢又不可靠。

特徵點（feature point）的思想是：只挑出那些「容易被重複找到、且容易區分」的位置。好的特徵點應具備三個性質：

- **可重複性**：從不同角度、不同光線下拍攝，同一個實體位置都能被偵測到。
- **獨特性**：其鄰域的外觀足夠特別，不會與其他位置混淆。
- **局部性**：只依賴小範圍的鄰域，因此部分遮蔽時仍然有效。

哪些位置符合這些條件？答案是**角點**——在兩個方向上灰階都有劇烈變化的位置。相對地，平坦區域（各方向都無變化）與邊緣（只有一個方向有變化）都不適合：平坦區無法定位、邊緣則有「沿邊滑動」的歧義。

22.5.2 偵測與描述兩個步驟

特徵的處理分為兩個獨立的步驟。

步驟一：偵測 (Detection)。找出影像中的特徵點位置。經典的方法是 Harris 角點偵測——計算每個像素鄰域的梯度結構矩陣，若其兩個特徵值都大，代表兩個方向都有變化，即為角點。

步驟二：描述 (Description)。為每個特徵點計算一個**描述子 (descriptor)**——一個能刻畫其鄰域外觀的向量。之後比對兩張影像的特徵時，就是比對這些向量的相似度。

好的描述子需要具備**不變性**：即使物體旋轉、縮放、或光線改變，同一個特徵點的描述子應該保持相似。這是描述子設計的核心挑戰。

表 22-4 常見特徵演算法的比較

演算法	特點	旋轉不變	尺度不變	速度	適用場合
Harris	經典角點偵測，無描述子	否	否	極快	簡單追蹤、教學
SIFT	尺度空間極值，描述子強健	是	是	慢	高精度匹配、離線處理
SURF	SIFT 的加速版	是	是	中	折衷方案
ORB	FAST 偵測 + 二元描述子	是	部分	極快	即時 SLAM、嵌入式
深度學習特徵	由網路學習偵測與描述	是	是	需 GPU	困難場景、大視角變化

實務上的選擇：即時系統（如第 25 章的視覺 SLAM）幾乎都用 **ORB**——它的二元描述子可以用位元運算比對，速度比 SIFT 快數十倍，精度損失有限。離線的高精度應用則仍偏好 SIFT。

22.5.3 匹配與外點剔除

有了描述子，匹配的作法是為每個特徵找出另一張影像中描述子最相似的那一個。但這必然會產生**錯誤匹配 (outlier)**——相似的紋理、重複的圖案、或被遮蔽的區域都會造成誤配。

因此匹配之後必須做**外點剔除**。標準的作法是 **RANSAC**（隨機取樣一致性）：

1. 隨機挑選最少數目的匹配點（例如求基礎矩陣需要 8 對）。

2. 由這些點計算出一個幾何模型（如兩視角之間的變換）。
3. 檢查所有匹配點是否符合這個模型，統計「內點」的數目。
4. 重複多次，保留內點最多的那個模型，並剔除其餘的外點。

RANSAC 的核心思想值得一記：與其試圖用所有資料（包含錯誤的）擬合出一個模型，不如假設「有一小群正確的資料」並找出它們。這使它對高比例的外點（甚至超過 50%）仍然有效，是感知系統中最重要的穩健技術之一。

22.6 手眼校正

22.6.1 問題的定義

回到開頭故事的核心問題：相機看到零件在像素 (320, 240)，機器人該把夾爪移到哪裡？

即使已經完成相機校正（知道內部參數）並取得了深度（知道零件在**相機座標系**中的三維位置），仍然缺少最後一塊拼圖：**相機座標系與機器人座標系之間的變換**。求出這個變換的程序，稱為**手眼校正**（hand-eye calibration）。

22.6.2 兩種構型

依相機的安裝位置，有兩種構型，其求解的未知量不同。

眼在手上（Eye-in-hand）：相機固定在機器人末端，隨手臂一起移動。要求的是「相機相對於末端法蘭」的變換 T_C ——這是一個固定值（只要相機不鬆脫）。

眼在手外（Eye-to-hand）：相機固定在環境中（如架在工作台上方），不隨機器人移動。要求的是「相機相對於機器人基座」的變換 B_T ——同樣是固定值。

表 22-5 兩種手眼構型的比較

面向	眼在手上	眼在手外
相機位置	末端法蘭	固定於環境
待求變換	T_C （法蘭到相機）	B_T （基座到相機）
視野	隨手臂移動，可主動改變	固定
解析度	可靠近物體取得高解析度	受限於固定距離
遮蔽問題	手臂本身可能擋住視線	手臂常擋住工件
末端負載	增加（相機重量與線材）	無影響

面向	眼在手上	眼在手外
典型應用	精密裝配、視覺伺服、探索未知場景	產線取放、大範圍監看

22.6.3 $AX = XB$ 問題

手眼校正在數學上歸結為一個經典的問題形式。作法是：讓機器人移動到多個不同的姿態，每個姿態都拍攝同一個固定的標定板。

在兩個姿態之間，可以得到兩個「相對運動」：機器人末端的運動 A （由正向運動學算出，第 9 章）與相機的運動 B （由標定板的兩次觀測算出）。這兩者透過待求的變換 X 相連：

$$A \cdot X = X \cdot B$$

手眼校正常式。A 為末端的相對運動、B 為相機的相對運動、X 為待求的手眼變換。

這條式子的直觀意義是：「先移動再換算」與「先換算再移動」必須得到相同的結果——這是變換的一致性要求。

求解需要至少兩組（實務上建議 10 組以上）不同的相對運動，且這些運動必須包含足夠的旋轉多樣性——若所有運動都繞同一個軸旋轉， X 的某些分量無法被確定。這與相機校正需要多樣的拍攝姿態是同樣的道理。

22.6.4 實務要點

要點一：運動要有足夠的旋轉。純平移的運動組合無法求解旋轉部分。建議規劃的姿態包含繞不同軸的顯著旋轉（至少 30 度以上）。

要點二：驗證方法。求出 X 後，應以獨立的資料驗證：讓機器人移動到一個未參與校正的姿態，用 X 預測標定板在影像中的位置，與實際觀測比較。誤差應在數個像素以內。

要點三：什麼時候要重做。相機被碰撞、鏡頭被更換或調焦、機器人被重新安裝、末端加裝了新的工具——這些情況都必須重新校正。實務上建議定期驗證（如每月一次），以及在精度異常時優先懷疑手眼參數。

圖 22-1 兩種手眼構型與座標系關係

建議配圖：左圖為眼在手上——機械手臂末端裝有相機，畫出基座座標系 $\{B\}$ 、法蘭座標系 $\{F\}$ 、相機座標系 $\{C\}$ 與工件座標系 $\{O\}$ ，並以箭頭標出四個變換： ${}^B T_F$ （正向運動學，已知）、 ${}^F T_C$ （待求的手眼變換）、 ${}^O T_O$ （相機量測，已知）。右圖為眼在手外——相機架在工作台上方，同樣標出四個座標系，待求的是 ${}^B T_C$ 。兩圖下方

各附一個閉環方程，說明變換鏈必須自治。

22.7 視覺伺服

22.7.1 兩種架構

前面的討論都是「看一次、算一次、動一次」的開迴路作法：拍照、計算目標位置、規劃軌跡、執行。這個流程有一個弱點——執行過程中若有誤差（機器人定位誤差、物體移動），沒有機會修正。

視覺伺服（visual servoing）則把視覺放進控制迴路中：持續拍攝、持續比對目前影像與目標影像的差異、持續調整運動。這是一個閉迴路的作法，能自動補償各種誤差。

依「誤差在哪個空間中定義」，有兩種架構。

位置式視覺伺服（PBVS, Position-Based）：先由影像估計出物體的三維位姿，再在笛卡兒空間中計算誤差並控制。

$$\text{誤差 } e = x_{\text{current}} - x_{\text{desired}} \quad (\text{三維位姿的差})$$

PBVS。誤差有明確的物理意義（公釐、度），軌跡在空間中是直線，直觀易懂。

影像式視覺伺服（IBVS, Image-Based）：直接在影像空間中定義誤差——比較特徵點目前的像素位置與期望的像素位置。

$$\text{誤差 } e = s_{\text{current}} - s_{\text{desired}} \quad (\text{像素座標的差})$$

IBVS。不需要估計三維位姿，因此對相機校正誤差與深度估計誤差較不敏感。

表 22-6 PBVS 與 IBVS 的比較

面向	位置式 PBVS	影像式 IBVS
誤差定義空間	笛卡兒空間（三維）	影像空間（像素）
需要位姿估計	是	否
對校正誤差	敏感	較不敏感
空間軌跡	直線，可預測	可能彎曲甚至繞遠路
影像軌跡	特徵可能離開視野	特徵保持在畫面內
大旋轉時	表現良好	可能退離（camera retreat）
適用場合	已知模型、大範圍運動	未知模型、精密對位

兩者的取捨很有意思：**PBVS** 保證空間軌跡漂亮但可能讓特徵跑出畫面；**IBVS** 保證特徵留在畫面但空間軌跡可能很怪。實務上也有混合式（2.5D 視覺伺服）試圖兼顧兩者。

22.7.2 影像雅可比

IBVS 的核心是回答一個問題：**相機移動時，影像上的特徵點會怎麼移動？** 這個關係由**影像雅可比**（image Jacobian，又稱互動矩陣）描述：

$$\dot{s} = L(s, Z) \cdot v_{\text{camera}}$$

s 為特徵的像素座標、 v_{camera} 為相機的六維速度（三平移三旋轉）、 L 為影像雅可比。

對單一個特徵點 (u, v) ，其影像雅可比為（此處以歸一化座標表示）：

$$L = \begin{bmatrix} -f/Z & 0 & u/Z & uv/f & -(f+u^2/f) & v \\ 0 & -f/Z & v/Z & f+v^2/f & -uv/f & -u \end{bmatrix}$$

單點的影像雅可比（ 2×6 ）。注意它**含有深度 Z** ——這是 IBVS 仍需要深度估計的原因，但好消息是它對 Z 的誤差相當不敏感。

由於一個點只提供 2 個方程式，而相機有 6 個自由度，因此**至少需要 3 個特徵點**（提供 6 個方程式）才能完整控制。實務上常用 4 個以上以增加穩健性。

22.7.3 控制律與雅可比的串接

有了影像雅可比，控制律的設計就與第 11、17 章完全一致——這是本書工具重複使用的又一個例子。

$$v_{\text{camera}} = -\lambda \cdot L^+ \cdot e$$

IBVS 的控制律。 L^+ 為影像雅可比的偽逆（第 10.7.2 節）、 λ 為增益。這使誤差以指數衰減。

最後，把相機速度轉換為關節速度，需要**串接兩個雅可比**：

$$\dot{q} = J_{\text{robot}}^+ \cdot {}^B V_C \cdot L^+ \cdot (-\lambda e)$$

完整的鏈條：影像誤差 $\rightarrow$ 相機速度 $\rightarrow$ 末端速度 $\rightarrow$ 關節速度。 ${}^B V_C$ 是相機速度到末端速度的伴隨變換（第 11.7 節）。

這條式子清楚展現了本書各章工具的串接：**影像雅可比（本章）+ 伴隨變換（第 11 章）+ 機器人雅可比的偽逆（第 10、11 章）**。理解了每一塊，整條鏈就自然成立。

◎ 視覺伺服的取樣率問題

視覺伺服有一個先天的困難：**相機的更新率遠低於控制迴路**。典型的相機是 30 Hz，而伺服迴路需

要 1 kHz——相差 33 倍。這意味著在兩幀影像之間，控制器有 33 個週期「沒有新資訊」。實務上的作法有三：一、以編碼器的高頻資訊做內插（在兩幀之間用運動學預測）；二、使用高幀率相機（可達 200 Hz 以上，但成本高）；三、降低視覺迴路的增益，使其反應速度與更新率匹配。這正是第 6.9.1 節「延遲比更新率更致命」在視覺上的體現。

22.8 深度學習在機器人視覺中的角色

22.8.1 三類任務

近十年，深度學習徹底改變了機器人視覺的能力邊界。它主要處理三類任務。

任務一：物體偵測與分割。在影像中找出物體的位置（邊界框）與類別，或逐像素標出物體的輪廓（語意分割、實例分割）。這是傳統方法極難處理的問題——因為「什麼是一個杯子」無法用幾何規則描述。

任務二：六自由度位姿估計。不只知道物體在哪裡，還要知道它的完整姿態。這是抓取（第 21 章）與裝配所必需的。傳統方法需要 CAD 模型與特徵匹配；學習式方法則能直接由影像回歸出位姿，且能處理無紋理或對稱的物體。

任務三：深度與結構估計。由單張影像預測深度圖（單目深度估計），或由多張影像重建三維結構。單目深度估計在物理上是不可能的（22.1.2 節的尺度歧義），但網路能從**先驗知識**（物體的典型大小、透視線索、紋理梯度）做出合理的推測——這正是人類單眼也能大致判斷距離的原理。

22.8.2 與傳統方法的分工

深度學習並未取代傳統的幾何方法，兩者的分工相當清楚：

表 22-7 傳統幾何方法與深度學習的分工

面向	傳統幾何方法	深度學習
擅長	精確的幾何計算、多視角關係	語意理解、辨識、處理未知物體
需要	校正參數、幾何模型	大量標註資料
精度	高（可達亞像素）	中（受訓練資料限制）
泛化	只要幾何成立就有效	受限於訓練分布
可解釋	高	低
典型應用	校正、三角測量、SLAM 後端	偵測、分割、位姿估計、抓取點預測

實務上的標準架構是混合式：用深度學習做「理解」（這是什麼、大概在哪裡），用幾何方法做「精算」（精確的位置與姿態）。例如視覺抓取系統：先用網路偵測出物體並給出粗略的抓取點，再用點雲配準（幾何方法）精修位姿。

這與第 21.9.5 節的結論完全一致，也再次呼應第 17 章的原則——**不同的工具處理各自擅長的部分**。

22.8.3 資料與泛化的現實

學習式方法的最大挑戰是**資料**。標註一張分割影像可能需要數分鐘，而訓練通常需要數萬張。因應之道有三：

方式一：合成資料。在模擬器中渲染大量場景並自動產生標註。優點是免費且無限；缺點是與真實影像有落差（視覺的 **Sim-to-Real Gap**——渲染的光影、材質、雜訊都不夠真實）。因應之道同樣是**領域隨機化**：隨機改變光照、材質、背景與相機參數。

方式二：預訓練與微調。使用在大規模通用資料集上預訓練的模型，再用少量的任務專屬資料微調。這是目前最實用的作法。

方式三：自我監督。讓機器人自己收集資料——例如抓取後由「有沒有拿起來」自動產生標籤（第 21.9.3 節）。

必須誠實面對的限制：無論用什麼方法，網路在**訓練分布之外**的表現都會下降。而機器人的部署環境往往與訓練環境不同（不同的光線、不同的背景、沒見過的物體）。因此安全關鍵的系統絕不能只依賴網路的輸出，必須有幾何驗證或其他感測器的交叉檢查。

22.9 視覺系統的失效與因應

22.9.1 七種常見的失效情境

視覺系統在實驗室裡常常表現優異，一到現場就問題百出。以下七種情境涵蓋了絕大多數的失敗原因。

情境一：光線變化。這是最常見也最頑固的問題。日光隨時間與天氣改變、室內燈光可能閃爍（與相機快門產生頻閃）、機器人自身可能在工件上投下陰影。**因應**：使用主動光源（環形燈、

同軸光)並遮蔽環境光；固定曝光參數而非自動曝光；選用高動態範圍的相機。

情境二：反光與鏡面。金屬工件的高光會使影像局部飽和，深度相機在鏡面上完全失效。**因應：**使用偏光片濾除鏡面反射；改用漫射光源；或改用非光學的感測方式（如力覺探測）。

情境三：透明物體。玻璃、透明塑膠對深度相機幾乎是「不存在」的（第 6.7 節）。這是目前仍未完全解決的難題。**因應：**使用背景對比（在物體後方放特殊背景）；使用偏光或熱像；或改用觸覺確認。

情境四：遮蔽。物體被其他物體或機器人自身擋住。**因應：**多視角配置；主動視覺（移動相機以取得更好的視角）；或利用先驗模型推測被遮蔽的部分。

情境五：無紋理表面。白牆、單色平面上，立體視覺找不到對應點。**因應：**改用主動測距；或投射隨機圖案（這正是結構光的原理）。

情境六：運動模糊。相機或物體移動時，曝光期間的位移造成影像模糊。**因應：**縮短曝光時間（需要更強的光源）；使用全域快門（第 6.7.1 節）；或降低運動速度。

情境七：校正漂移。相機被碰撞、溫度變化使機構變形、鏡頭鬆動。**因應：**定期驗證重投影誤差；設置固定的校正基準（如工作台上的 AprilTag）供系統自我檢查。

22.9.2 系統設計的三個原則

面對這些失效，有三個貫穿性的設計原則。

原則一：控制環境比改進演算法有效。與其寫更聰明的程式去對抗變化的光線，不如加一個遮光罩與固定光源。工業視覺的成功，八成來自打光的設計而非演算法。

原則二：不要只依賴單一感測。視覺失效時應有備援——力覺可以確認接觸、光達可以偵測障礙、碰撞板是最後防線。這是第 6.12 節「互補而非重複」原則的延續。

原則三：設計失效的偵測機制。比「不失效」更重要的是「知道自己失效了」。例如：檢查偵測到的特徵點數目是否足夠、重投影誤差是否異常、深度圖的有效像素比例是否過低。一旦偵測到異常，系統應停止並求援，而非用錯誤的資料繼續動作。

◎ 工業視覺的第一課：打光

初學者常把 90% 的時間花在演算法上、10% 花在打光；資深工程師的比例恰好相反。原因是：**好的打光能讓困難的問題變簡單，而壞的打光會讓簡單的問題變不可能。**常見的打光技巧包括——背

光（拍輪廓，對量測外形極有效）、**同軸光**（拍平整反光表面，如晶圓）、**環形光**（通用照明）、**低角度掠射光**（凸顯表面刮痕與凹凸）、**藍光或紅外**（濾除環境光干擾）。面對一個視覺問題時，第一個該問的是「能不能換個打光方式讓目標特徵變得明顯？」而不是「該用什麼演算法？」

22.10 感知的心態轉換

22.10.1 由確定到機率

開頭故事中叫獸提到的「心態轉換」，值得在此展開。

第二篇與第三篇的世界是**確定性**的：給定關節角度，末端位置由運動學唯一確定；給定力矩，運動由動力學唯一確定。誤差來自模型的不完美，但問題本身有唯一的答案。

感知的世界則不然。同一個場景，換個光線就得到不同的影像；同一張影像，可能對應多種三維解釋（22.1.2 節的射線歧義）；同一個特徵，可能被錯誤匹配。**感知不是計算一個答案，而是在不完整、有雜訊的證據中做出最合理的推測。**

這個轉換帶來三個方法論上的改變：

- **輸出應包含不確定性。**不只回報「物體在 (x, y, z) 」，還應回報「這個估計的信賴度」。這使後續的決策能適當地加權。
- **應主動處理外點。**感知資料必定含有錯誤（誤匹配、雜訊、失效的像素），演算法必須設計成對此穩健——RANSAC（22.5.3 節）正是這種思維的代表。
- **應整合多個來源。**單一感測的估計不可靠，融合多個互補的來源才能得到穩健的結果——這正是第 24 章的主題。

22.10.2 為第 24 章鋪路

這個心態轉換，直接引出了第 24 章的核心工具：**機率狀態估測**。

屆時我們會學到：如何用機率分布表示「我大概在這裡，但不確定有多大」、如何在移動時正確地累積不確定性、以及如何在取得新的觀測時最佳地更新估計（Kalman 濾波器與粒子濾波器）。

而本章的視覺，正是那些「觀測」的主要來源。第 25 章的 SLAM 則會把兩者結合——一邊用視覺觀測環境、一邊估計自己的位置、同時建立地圖。

22.11 由看見到理解

22.11.1 本章的成果

本章建立了由影像走向三維世界的完整鏈條。你現在能夠：用針孔模型描述成像過程、區分內外部參數並執行相機校正、理解深度為何丟失以及三種恢復的方式、運用特徵匹配與 RANSAC 建立影像間的對應、執行手眼校正把相機接回機器人座標、設計視覺伺服控制律、以及判斷深度學習與傳統幾何方法各自的適用場合。

更重要的是那個心態的轉換：**感知不是精確的計算，而是在雜訊中的合理推測**。這將貫穿整個第四篇。

22.11.2 接下來：讓機器人動起來

本章讓機器人「看見」了環境，但它仍然固定在原地。第四篇接下來的六章，要讓它真正走出去。

第 23 章 (行動機器人運動學)：輪式機器人如何運動？差速輪、麥克納姆輪、阿克曼轉向各有什麼特性？為何有些機器人「無法直接橫 move」（非完整約束）？

第 24 章 (狀態估測)：在充滿雜訊的世界中，如何最佳地估計自己的位置？Kalman 濾波器如何融合里程計與感測器？

第 25 章 (SLAM)：如何一邊移動、一邊建立地圖、同時定位自己？這個「雞生蛋、蛋生雞」的問題如何求解？

第 26 章 (路徑規劃)：如何找出一條避開障礙的路徑？A*、RRT 等演算法如何運作？

第 27、28 章：無人機與腿式機器人這兩種特殊的移動形態。

換句話說：第 22 章讓機器人**看見**世界，第 23 ~ 28 章讓它**在世界中行動**。

完整計算例題

例題 22-1 相機投影與視野計算

題目：某工業相機的感光元件為 1/2.5 吋（有效尺寸 $5.70 \times 4.28 \text{ mm}$ ）、解析度 1920×1440 像素，搭配焦距 8 mm 的鏡頭。（a）求以像素為單位的焦距 f_x 與 f_y 。（b）若相機距離

工作平面 500 mm，求視野的實際尺寸。(c) 求每個像素對應工作平面上的實際尺寸(解析度)。(d) 若要量測 0.5 mm 的特徵，判斷此配置是否足夠。

解(a)：像素尺寸為感光元件尺寸除以像素數：

$$\text{像素寬} = 5.70 / 1920 = 0.002969 \text{ mm/pixel}$$

$$\text{像素高} = 4.28 / 1440 = 0.002972 \text{ mm/pixel}$$

兩者幾乎相同，代表像素接近正方形。

$$f_x = 8 / 0.002969 = 2694 \text{ pixels}$$

$$f_y = 8 / 0.002972 = 2692 \text{ pixels}$$

以像素為單位的焦距。這正是內部參數矩陣 K 中的值。

解(b)：由投影方程的相似三角形關係，視野(FOV)的實際尺寸為：

$$\text{視野寬} = (\text{感光元件寬} / \text{焦距}) \times \text{距離} = (5.70 / 8) \times 500 = 356.3 \text{ mm}$$

$$\text{視野高} = (4.28 / 8) \times 500 = 267.5 \text{ mm}$$

工作距離 500 mm 處的視野約 $356 \times 268 \text{ mm}$ 。

解(c)：每像素對應的實際尺寸：

$$\text{解析度} = 356.3 / 1920 = 0.1856 \text{ mm/pixel}$$

約 0.19 公釐。

解(d)：要量測 0.5 mm 的特徵，該特徵在影像中只佔 $0.5 / 0.1856 \approx 2.7$ 個像素。

判定：不足。一般的經驗法則是「待測特徵至少要橫跨 3~5 個像素才能可靠偵測，若要精確量測則需 10 個像素以上」。因此此配置勉強能「看到」該特徵，但無法可靠量測。

改善方案有三：一、縮短工作距離(如降到 250 mm，解析度提升一倍，但視野減半)；二、使用更長的焦距鏡頭(如 16 mm，同樣使解析度加倍)；三、使用更高解析度的相機。三者的取捨在於視野、成本與景深——這正是視覺系統選型的核心權衡。

例題 22-2 立體視覺的深度精度

題目：某立體相機的基線 $B = 120 \text{ mm}$ 、焦距 $f = 1200$ 像素、視差量測精度 $\Delta d = \pm 0.25$ 像素。(a) 求距離 1 m、3 m、5 m 處的視差值。(b) 求三個距離處的深度誤差。(c) 若要求深度誤差不超過 $\pm 10 \text{ mm}$ ，最遠可測到多遠？(d) 若把基線加大到 240 mm，重新計算(c)。

解(a)：由 $Z = fB/d$ 反解 $d = fB/Z$ ：

$$Z = 1000 \text{ mm} : d = 1200 \times 120 / 1000 = 144.0 \text{ pixels}$$

$$Z = 3000 \text{ mm} : d = 1200 \times 120 / 3000 = 48.0 \text{ pixels}$$

$$Z = 5000 \text{ mm} : d = 1200 \times 120 / 5000 = 28.8 \text{ pixels}$$

距離越遠，視差越小——這是精度劣化的根源。

解 (b)：由 22.4.2 節的誤差公式 $\Delta Z = Z^2 \cdot \Delta d / (f \cdot B)$ ：

$$Z = 1000 : \Delta Z = 1000^2 \times 0.25 / (1200 \times 120) = 250000 / 144000 = 1.74 \text{ mm}$$

$$Z = 3000 : \Delta Z = 3000^2 \times 0.25 / 144000 = 15.6 \text{ mm}$$

$$Z = 5000 : \Delta Z = 5000^2 \times 0.25 / 144000 = 43.4 \text{ mm}$$

深度誤差與距離的平方成正比。距離增為 5 倍，誤差增為 25 倍。

解 (c)：令 $\Delta Z \leq 10 \text{ mm}$ ：

$$Z^2 \times 0.25 / 144000 \leq 10 \Rightarrow Z^2 \leq 5.76 \times 10^6 \Rightarrow Z \leq 2400 \text{ mm}$$

最遠約 2.4 公尺。

解 (d)：基線加倍為 240 mm 時，分母加倍：

$$Z^2 \times 0.25 / 288000 \leq 10 \Rightarrow Z^2 \leq 11.52 \times 10^6 \Rightarrow Z \leq 3394 \text{ mm}$$

最遠約 3.4 公尺，改善了 41% (即 $\sqrt{2}$ 倍)。

工程判讀：基線加倍只讓有效距離增加 $\sqrt{2}$ 倍，而非 2 倍——因為誤差與 Z^2 成正比。這說明了**加大基線的效益是遞減的**。此外加大基線還有兩個代價：**共同視野縮小**（近處的物體可能只有一台相機看得到）、以及**體積增大**。這正是自駕車的立體相機基線通常在 20~50 公分（而非更大）的原因。

例題 22-3 像素座標到機器人座標的完整轉換

題目：某眼在手外的系統中，相機固定在工作台上方。已知：內部參數 $f_x = f_y = 800 \text{ pixels}$ 、主點 $(c_x, c_y) = (640, 480)$ ；相機到機器人基座的變換 ${}^B T_C$ 為——旋轉為繞 X 軸 180° （相機朝下）、平移為 $(300, 200, 900) \text{ mm}$ 。現在相機偵測到一個零件在像素 $(740, 380)$ ，且由深度相機測得該點的深度 $Z_C = 600 \text{ mm}$ 。（a）求該點在相機座標系中的三維位置。（b）求其在機器人基座座標系中的位置。（c）若深度量測有 $\pm 10 \text{ mm}$ 的誤差，估算對基座座標的影響。

解 (a)：由 22.1.3 節的投影方程反解。已知像素座標與深度：

$$\begin{aligned} X_C &= (u - c_x) \cdot Z_C / f_x = (740 - 640) \times 600 / 800 = 100 \times 0.75 = 75.0 \text{ mm} \\ Y_C &= (v - c_y) \cdot Z_C / f_y = (380 - 480) \times 600 / 800 = -100 \times 0.75 = -75.0 \text{ mm} \end{aligned}$$

$$Z_C = 600.0 \text{ mm}$$

相機座標系中的位置。注意 Y 為負值，因為該像素在主點的上方（影像座標 v 向下為正）。

解（b）：套用齊次變換 ${}^B P = {}^B R_C \cdot {}^C P + {}^B t_C$ 。繞 X 軸旋轉 180° 的旋轉矩陣為：

$${}^B R_C = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

繞 X 軸 180° ： Y 與 Z 皆變號。

$${}^B R_C \cdot {}^C P = (75.0, -(-75.0), -600.0) = (75.0, 75.0, -600.0)$$

旋轉後的向量。

$${}^B P = (75.0 + 300, 75.0 + 200, -600.0 + 900) = (375.0, 275.0, 300.0) \text{ mm}$$

零件在機器人基座座標系中的位置。這就是可以直接送給逆運動學的目標。

合理性檢查： $Z = 300 \text{ mm}$ 為正值，代表零件在基座上方 300 mm 處——合理（相機在 900 mm 高，零件在其下方 600 mm 處）。

解（c）：深度誤差 $\Delta Z_C = \pm 10 \text{ mm}$ 會同時影響三個分量。由（a）的關係式：

$$\begin{aligned} \Delta X_C &= (u - c_x)/f_x \times \Delta Z_C = (100/800) \times 10 = \pm 1.25 \text{ mm} \\ \Delta Y_C &= (v - c_y)/f_y \times \Delta Z_C = (-100/800) \times 10 = \mp 1.25 \text{ mm} \\ \Delta Z_C &= \pm 10 \text{ mm} \end{aligned}$$

橫向誤差遠小於深度誤差。

經旋轉後，在基座座標中的誤差為 $\Delta X_B = \pm 1.25$ 、 $\Delta Y_B = \pm 1.25$ 、 $\Delta Z_B = \mp 10 \text{ mm}$ 。

重要觀察：深度誤差主要影響垂直方向（因為相機朝下）。這對抓取有直接的意義——垂直方向的 10 mm 誤差可能導致夾爪撞到工件或抓空。因應之道有二：一、加入力控制或阻抗控制（第 20 章），讓垂直方向能順應；二、以「先移動到上方安全高度、再緩慢下降直到接觸」的策略執行。這正是實務上視覺抓取幾乎總是搭配力覺或柔順控制的原因。

程式範例：相機模型、校正與座標轉換

以下 Python 程式實作本章的核心計算：相機投影與反投影、視野與解析度計算、立體視覺深度精度分析、以及像素到機器人座標的完整轉換鏈。

```
import numpy as np

# ===== 相機模型 =====
class Camera:
```

```

"""針孔相機模型 (含畸變) """
def __init__(s, fx, fy, cx, cy, dist=None):
    s.K = np.array([[fx, 0, cx], [0, fy, cy], [0, 0, 1]], float)
    s.fx, s.fy, s.cx, s.cy = fx, fy, cx, cy
    s.dist = np.zeros(5) if dist is None else np.array(dist, float)

def project(s, P_cam):
    """三維點 (相機座標) → 像素座標 (22.1.3 節) """
    X, Y, Z = P_cam
    if Z <= 0:
        return None # 在相機後方
    u = s.fx * X/Z + s.cx
    v = s.fy * Y/Z + s.cy
    return np.array([u, v])

def backproject(s, pixel, Z):
    """像素座標 + 深度 → 三維點 (相機座標) """
    u, v = pixel
    X = (u - s.cx) * Z / s.fx
    Y = (v - s.cy) * Z / s.fy
    return np.array([X, Y, Z])

def fov(s, width_px, height_px, distance):
    """求指定距離處的視野尺寸與解析度"""
    w = width_px * distance / s.fx
    h = height_px * distance / s.fy
    return w, h, w/width_px

def apply_distortion(s, x, y):
    """套用徑向與切向畸變 (22.2.2 節) """
    k1, k2, p1, p2, k3 = s.dist
    r2 = x*x + y*y
    radial = 1 + k1*r2 + k2*r2**2 + k3*r2**3
    x_d = x*radial + 2*p1*x*y + p2*(r2 + 2*x*x)
    y_d = y*radial + p1*(r2 + 2*y*y) + 2*p2*x*y
    return x_d, y_d

# ===== 例題 22-1：視野與解析度 =====
print("=== 例題 22-1：相機配置分析 ===")
sensor_w, sensor_h = 5.70, 4.28 # mm
res_w, res_h = 1920, 1440
focal_mm = 8.0

px_w, px_h = sensor_w/res_w, sensor_h/res_h
fx, fy = focal_mm/px_w, focal_mm/px_h
cam = Camera(fx, fy, res_w/2, res_h/2)
print(f"像素尺寸: {px_w*1000:.3f} × {px_h*1000:.3f} μm")
print(f"焦距(像素): fx = {fx:.0f}, fy = {fy:.0f}")

for dist in (250, 500, 1000):
    w, h, res = cam.fov(res_w, res_h, dist)
    n_px = 0.5 / res # 0.5 mm 特徵佔幾個像素

```

```

verdict = ("X 不足" if n_px < 3 else
           "△ 勉強" if n_px < 10 else "√ 充足")
print(f"距離 {dist:5d} mm: 視野 {w:6.1f} × {h:6.1f} mm "
      f"解析度 {res:.4f} mm/px 0.5mm 特徵佔 {n_px:4.1f} px {verdict}")

# ===== 例題 22-2：立體視覺深度精度 =====
print("\n=== 例題 22-2：立體視覺的深度精度 ===")
def stereo_error(Z, f, B, dd=0.25):
    d = f*B/Z
    dZ = Z**2 * dd / (f*B)
    return d, dZ

f_st, B_st = 1200.0, 120.0
print(f"基線 B = {B_st:.0f} mm, 焦距 f = {f_st:.0f} px, 視差精度 ±0.25 px")
print(f"'距離(mm)':>10{'視差(px)':>12{'深度誤差(mm)':>16{'相對誤差':>12}}")
for Z in (500, 1000, 2000, 3000, 5000):
    d, dZ = stereo_error(Z, f_st, B_st)
    print(f"{Z:10d}{d:12.1f}{dZ:14.2f}{dZ/Z*100:11.2f}%")

# 最遠可測距離
for B in (120.0, 240.0):
    Z_max = np.sqrt(10.0 * f_st * B / 0.25)
    print(f"基線 {B:5.0f} mm → 誤差 ≤ 10 mm 的最遠距離 = {Z_max:.0f} mm")
print("→ 基線加倍，有效距離只增加 √2 倍（因誤差與 Z2 成正比）")

# ===== 例題 22-3：完整座標轉換鏈 =====
print("\n=== 例題 22-3：像素 → 相機 → 機器人基座 ===")
cam2 = Camera(800., 800., 640., 480.)

def make_T(R, t):
    T = np.eye(4); T[:3,:3] = R; T[:3,3] = t
    return T

# 相機朝下：繞 X 軸 180°
R_BC = np.array([[1,0,0],[0,-1,0],[0,0,-1]], float)
t_BC = np.array([300., 200., 900.])
T_BC = make_T(R_BC, t_BC)

pixel, depth = np.array([740., 380.]), 600.
P_cam = cam2.backproject(pixel, depth)
P_base = (T_BC @ np.append(P_cam, 1.0))[0:3]
print(f"像素座標:      ({pixel[0]:.0f}, {pixel[1]:.0f}), 深度 {depth:.0f} mm")
print(f"相機座標系:      ({P_cam[0]:7.2f}, {P_cam[1]:7.2f}, {P_cam[2]:7.2f}) mm")
print(f"機器人基座系:    ({P_base[0]:7.2f}, {P_base[1]:7.2f}, {P_base[2]:7.2f}) mm")

# 驗證：把結果投影回像素應得到原值
P_cam_check = (np.linalg.inv(T_BC) @ np.append(P_base, 1.0))[0:3]
print(f"反向驗證(投影回像素): {np.round(cam2.project(P_cam_check), 2)}")

```



```

# 深度誤差的影響
print("\n 深度誤差 ±10 mm 的影響:")
for dz in (-10., 0., 10.):
    Pc = cam2.backproject(pixel, depth + dz)
    Pb = (T_BC @ np.append(Pc, 1.0))[0:3]
    print(f" ΔZ={dz:+6.1f} mm → 基座座標 ({Pb[0]:7.2f}, {Pb[1]:7.2f}, {Pb[2]:7.2f})")
print("→ 深度誤差主要轉化為垂直方向的誤差 (相機朝下時)")

# ===== 畸變的影響 =====
print("\n=== 鏡頭畸變對邊緣的影響 (k1 = -0.25 桶形畸變) ===")
cam_d = Camera(800., 800., 640., 480., dist=[-0.25, 0.08, 0, 0, 0])
print(f"'歸一化半徑':>12}'{'畸變後':>12}'{'像素偏移':>12}")
for r in (0.0, 0.2, 0.4, 0.6, 0.8):
    xd, yd = cam_d.apply_distortion(r, 0.0)
    shift = abs(xd - r) * 800
    print(f"{r:12.2f}{xd:12.4f}{shift:10.1f} px")
print("→ 離中心越遠畸變越嚴重，邊緣可達數十像素，必須校正")

# ===== 影像雅可比 (22.7.2 節) =====
print("\n=== 影像雅可比：相機運動 → 特徵點速度 ===")
def image_jacobian(u, v, Z, f=1.0):
    """單點的 2x6 影像雅可比 (歸一化座標)"""
    return np.array([
        [-f/Z, 0, u/Z, u*v/f, -(f + u*u/f), v],
        [0, -f/Z, v/Z, f + v*v/f, -u*v/f, -u]])

u_n, v_n, Z_n = 0.2, 0.15, 1.0
L = image_jacobian(u_n, v_n, Z_n)
print(f"特徵點(歸一化) = ({u_n}, {v_n}), 深度 = {Z_n} m")
print(f"影像雅可比 L (2x6) =\n{np.round(L, 3)}")

motions = [("相機前進 +Z", [0,0,0.1,0,0,0]),
           ("相機右移 +X", [0.1,0,0,0,0,0]),
           ("相機繞 Z 旋轉", [0,0,0,0,0,0.1])]
for name, v_cam in motions:
    s_dot = L @ np.array(v_cam, float)
    print(f" {name:14s} → 特徵速度 ({s_dot[0]:+7.4f}, {s_dot[1]:+7.4f})")
print("→ 前進使特徵向外擴散、右移使特徵向左、旋轉使特徵繞中心轉")

```

請試著修改：(一) 把例題 22-1 的焦距改為 16 mm，觀察視野與解析度如何改變，驗證「焦距加倍則解析度加倍、視野減半」。(二) 在立體視覺分析中把視差精度改為 ± 0.1 像素 (次像素匹配)，觀察有效距離如何延伸。(三) 在座標轉換中故意把手眼變換的旋轉設錯 5 度，觀察對最終位置的影響——這說明了手眼校正精度的重要性。

實作：相機校正與視覺定位

本章實作是建立一套完整的視覺定位系統，從校正到座標轉換。所需器材：一台 USB 相機或工業相機、棋盤格標定板（可自行列印並貼在硬板上）、以及第 9 章已建立運動學模型的機械手臂。

- 製作標定板：**列印一張 9×6 內角點的棋盤格（格子邊長建議 20~30 mm），貼在玻璃或壓克力板上確保平整。用游標卡尺精確量測格子的實際邊長。
- 相機校正：**從至少 20 個不同的角度與距離拍攝標定板，用 OpenCV 的 `calibrateCamera` 求出內部參數與畸變係數。**記錄重投影誤差**，依表 22-2 判斷品質。若超過 0.5 像素，檢查板子平整度與拍攝多樣性後重做。
- 畸變校正驗證：**拍攝一張含有直線的場景（如門框、桌邊），比較校正前後的影像。用尺在螢幕上量測，驗證校正後的直線確實是直的。
- 視野與解析度量測：**把標定板放在已知距離處，由影像中格子的像素數反推解析度，與例題 22-1 的理論計算比較。
- 手眼校正：**把標定板固定在工作台上，讓機器人移動到 15 個以上姿態（確保包含繞不同軸的顯著旋轉），每個姿態記錄關節角度與拍攝影像。用 OpenCV 的 `calibrateHandEye` 求出手眼變換。
- 手眼驗證：**移動到一個未參與校正的姿態，用求得的變換預測標定板某個角點在影像中的位置，與實際偵測比較。誤差應在 5 像素以內。
- 端到端定位測試：**在工作台上放一個目標物（可貼 AprilTag），由影像計算其在機器人基座座標中的位置，命令機器人移動到該位置上方。用尺量測實際偏差。
- 誤差來源分析：**分別引入三種誤差——相機校正誤差、手眼校正誤差、深度量測誤差——各自估計其對最終定位精度的貢獻，找出主要瓶頸。
- （進階）視覺伺服：**實作簡單的 IBVS——以四個特徵點為目標，用影像雅可比計算相機速度，驅動機器人使特徵收斂到期望位置。觀察特徵在影像中的運動軌跡。

表 22-8 視覺系統驗證記錄表（第一列為範例）

項目	設定	理論值	實測值	誤差	備註
相機校正	25 張標定影像	—	重投影誤差 0.28 px	—	品質優良
（自行填寫其餘項目）					

國中小也看得懂：照片會騙人

看看窗外的馬路，或找一張鐵軌的照片。兩條鐵軌明明是平行的，為什麼照片上看起來會相交在遠方？

因為相機（還有你的眼睛）做了一件很厲害、但也很可惜的事：把立體的世界壓成了一張平的圖片。壓平的時候，有一個資訊被丟掉了——那就是「有多遠」。

來做個實驗：閉上一隻眼睛，然後伸出兩根食指，試著讓它們在空中對準碰在一起。是不是有點困難？現在張開兩隻眼睛再試一次——立刻就對準了！

為什麼？因為兩隻眼睛看到的畫面稍微不一樣。左眼看到的東西比較偏右，右眼看到的比較偏左。而且——越近的東西，兩眼看到的差別越大。你的大腦就是用這個「差別」來判斷距離的。

機器人也用同樣的方法！有些機器人身上裝了兩台相機，就像兩隻眼睛，靠比較兩張照片的差別來算距離。這叫做「立體視覺」。

不過這裡有一個有趣的問題：越遠的東西，兩張照片的差別越小。所以機器人看近的東西很準，看遠的東西就很不準。想想看你自己——你能準確說出手上這本書離你多遠嗎？可以。那遠方那棟大樓呢？就只能猜了。機器人跟你一樣！

最後再告訴你一件事：相機看到的東西是用「第幾個點」（像素）來描述的，但機器人的手臂只聽得懂「往前幾公分、往右幾公分」。所以工程師必須做一件叫做「校正」的工作，教會機器人把「照片上的位置」翻譯成「真實世界的位置」。這就像幫兩個說不同語言的人當翻譯一樣！

叫獸提醒與常見錯誤

- 誤區一：以為單張影像就能得到三維位置。影像上的一點對應空間中的一整條射線，必須額外提供深度資訊。
- 誤區二：忽略鏡頭畸變。廣角鏡頭在影像邊緣的畸變可達數十像素，未校正就做幾何計算會有數公分的誤差。
- 誤區三：校正時標定板不夠平。列印在紙上拿在手中拍是最常見的錯誤，必須貼在硬板上。
- 誤區四：校正時拍攝姿態不夠多樣。都從同一角度拍攝會使某些參數無法分離，應涵蓋不同

角度與距離。

- 誤區五：不檢查重投影誤差就相信校正結果。這是唯一能量化校正品質的指標，應低於 0.5 像素。
- 誤區六：手眼校正時運動缺乏旋轉。純平移的運動組合無法求解旋轉部分，必須包含繞不同軸的顯著旋轉。
- 誤區七：相機被碰撞後不重做手眼校正。這是視覺系統精度突然劣化最常見的原因。
- 誤區八：忽略立體視覺的深度誤差隨距離平方增長。同樣的配置在 1 m 處誤差 2 mm、5 m 處可能就是 43 mm。
- 誤區九：混淆內部參數與外部參數。前者出廠後固定（校正一次即可），後者隨安裝位置而變（移動就要重做）。
- 誤區十：只依賴視覺而無備援。視覺對透明、反光、無紋理物體都會失效，必須有力覺或其他感測作為交叉檢查。
- 誤區十一：把時間花在演算法而非打光上。好的打光能讓困難的問題變簡單，這是工業視覺的第一課。
- 誤區十二：忽略視覺伺服的取樣率限制。相機典型 30 Hz 而伺服迴路需 1 kHz，兩者的落差必須妥善處理。

課堂討論

1. 開頭故事中「橘子還是彈珠」的例子說明了尺度歧義。請討論：人類單眼看世界時如何克服這個問題？機器人能否借用同樣的策略？
2. 22.4.4 節提到「使用視覺標記在研究上像作弊，在工程上卻往往是最正確的決定」。請討論：在哪些情況下應該堅持不用標記？判斷的準則是什麼？
3. PBVS 保證空間軌跡漂亮但特徵可能離開視野，IBVS 保證特徵留在畫面但空間軌跡可能很怪。請為三個情境各選一種並說明理由：（a）精密插件對位；（b）大範圍移動到目標上方；（c）追蹤移動中的物體。
4. 22.9.2 節主張「控制環境比改進演算法有效」。請討論這個原則的適用邊界——在什麼情況下必須改進演算法而不能只靠控制環境？
5. 本章的心態轉換是「由確定到機率」。請回顧第二、三篇，找出一個當時被視為確定、但其

實也含有不確定性的量，並討論若要處理其不確定性需要做什麼改變。

章末摘要

- 針孔相機模型的投影方程為 $x = fX/Z$ 、 $y = fY/Z$ ；其中的「除以 Z 」摧毀了深度資訊，使影像上一點對應空間中一條射線。
- 內部參數 (f_x, f_y, c_x, c_y) 描述相機自身特性、出廠後固定；外部參數 (R, t) 描述相機在世界中的位姿、隨安裝而變。
- 鏡頭畸變分徑向與切向兩類，離影像中心越遠越嚴重；必須先校正畸變再做任何幾何計算。
- 相機校正以已知幾何的標定板求出參數；品質以重投影誤差判斷，應低於 0.5 像素。
- 恢復深度有三種方式：第二個視角（立體視覺）、主動測距（結構光 / ToF）、先驗知識（PnP 或視覺標記）。
- 立體視覺的深度誤差與距離的平方成正比；加大基線的效益遞減（只增加 $\sqrt{}$ 倍）。
- 特徵處理分偵測與描述兩步；ORB 適合即時系統、SIFT 適合高精度離線處理。
- 匹配必定產生外點，須以 RANSAC 剔除——其思想是「假設有一小群正確資料並找出它們」。
- 手眼校正求相機與機器人之間的變換，歸結為 $AX = XB$ 問題，需要包含足夠旋轉多樣性的運動。
- 眼在手上視野可主動改變、可靠近取得高解析度；眼在手外不增加末端負載、視野固定。
- PBVS 在笛卡兒空間定義誤差（軌跡直觀但特徵可能離開視野）；IBVS 在影像空間定義誤差（特徵留在畫面但空間軌跡可能彎曲）。
- 影像雅可比描述相機運動與特徵點速度的關係；控制律需串接影像雅可比、伴隨變換與機器人雅可比的偽逆。
- 深度學習擅長語意理解與處理未知物體，傳統幾何方法擅長精確計算；實務採混合式架構。
- 視覺的七種常見失效：光線變化、反光、透明、遮蔽、無紋理、運動模糊、校正漂移。
- 系統設計三原則：控制環境比改進演算法有效、不依賴單一感測、設計失效偵測機制。
- 感知的心態轉換：不是精確的計算，而是在不完整且有雜訊的證據中做出最合理的推測。

觀念題與計算題

觀念題

1. 寫出針孔相機的投影方程，並說明「除以 Z 」帶來的三個後果。
2. 區分內部參數與外部參數，並說明兩者在工程實務上的不同意義。
3. 說明徑向畸變與切向畸變的成因，以及為何畸變在影像邊緣特別嚴重。
4. 敘述相機校正的六個步驟，並說明棋盤格為何是理想的標定板。
5. 說明重投影誤差的定義與判讀標準，並列舉三個校正失敗的常見原因。
6. 列舉恢復深度的三種方式，並比較其所需條件與限制。
7. 推導立體視覺的深度誤差公式，並說明加大基線為何效益遞減。
8. 說明好的特徵點應具備的三個性質，以及為何角點適合而邊緣不適合。
9. 說明 RANSAC 的四個步驟與其核心思想。
10. 寫出手眼校正方程 $AX = XB$ ，並解釋其直觀意義。
11. 比較眼在手上與眼在手外的優缺點，並各舉兩個適用場合。
12. 比較 PBVS 與 IBVS 的誤差定義空間、對校正誤差的敏感度與軌跡特性。
13. 說明影像雅可比的作用，以及為何 IBVS 至少需要三個特徵點。
14. 說明視覺伺服的完整控制鏈，並指出其中串接了哪三個矩陣。
15. 比較深度學習與傳統幾何方法的擅長領域，並說明混合式架構的作法。
16. 列舉視覺系統的七種常見失效情境，並各提出一種因應方式。

計算題

1. 某相機感光元件 $6.4 \times 4.8 \text{ mm}$ 、解析度 1280×960 ，鏡頭焦距 12 mm 。(a) 求像素尺寸與以像素為單位的焦距。(b) 求工作距離 400 mm 處的視野尺寸與解析度。(c) 若要可靠量測 0.3 mm 的特徵 (需佔 5 個像素)，求所需的最大工作距離。(d) 若改用 25 mm 鏡頭，重新計算 (c)。
2. 某立體相機基線 150 mm 、焦距 1000 像素、視差精度 ± 0.2 像素。(a) 求 2 m 與 4 m 處的視差與深度誤差。(b) 求深度相對誤差不超過 1% 的最遠距離。(c) 若把視差精度提升到 ± 0.05 像素 (次像素匹配)，重新計算 (b)。(d) 比較「提升視差精度」與「加大基線一倍」哪一種改善較多。
3. 某眼在手上的系統，相機內部參數 $f_x = f_y = 950$ 、主點 $(640, 512)$ 。手眼變換 T_C 為：無

旋轉、平移 (50, 0, 80) mm。目前正向運動學得法蘭位姿 ${}^B T_F$ 為：繞 Z 軸旋轉 90° 、平移 (400, 100, 500) mm。相機偵測到目標在像素 (800, 400)、深度 350 mm。(a) 求目標在相機座標系中的位置。(b) 求其在法蘭座標系中的位置。(c) 求其在基座座標系中的位置。

4. 某 IBVS 系統以四個特徵點控制，影像雅可比為 8×6 矩陣。特徵點的目前位置與期望位置的誤差為 e (8 維向量)，增益 $\lambda = 0.5$ 。(a) 說明為何需要用偽逆而非反矩陣。(b) 若某時刻 L 的最小奇異值為 0.02，討論可能發生的問題。(c) 提出兩種因應方式 (提示：回顧第 10.7.3 節與第 12.5 節)。(d) 若只用兩個特徵點，判斷系統是否可控並說明理由。

動手做與專題延伸

本章實作：視覺工具模組

建立本章的視覺計算模組，這將是第 23 ~ 28 章的感知基礎。

- **階段一**：實作相機模型類別 (投影、反投影、畸變、視野計算)，並用例題 22-1 與 22-3 驗證。
- **階段二**：以 OpenCV 實作相機校正流程，並自動產生校正報告 (含重投影誤差與參數表)。
- **階段三**：實作立體視覺的深度計算與精度分析工具，能繪出「距離 vs 深度誤差」曲線。
- **階段四**：實作完整的座標轉換鏈 (像素 $\rightarrow$ 相機 $\rightarrow$ 手眼 $\rightarrow$ 基座)，並整合第 9 章的正向運動學。
- **階段五**：實作影像雅可比與 IBVS 控制律，在模擬中驗證特徵收斂。

專題延伸

- **視覺系統選型計算器**：撰寫一個工具，輸入「待測特徵尺寸、所需視野、可用工作距離」，自動計算並建議合適的相機解析度與鏡頭焦距組合，並標示各方案的取舍 (視野 vs 解析度 vs 景深)。這是視覺工程師實務上最常做的計算，把它自動化能省下大量時間。
- **校正品質的敏感度研究**：系統性地探討校正品質的影響因素——分別改變標定影像的數目 (5、10、20、40 張)、姿態多樣性 (全部正對 vs 涵蓋各角度)、以及標定板的平整度 (紙張 vs 硬板)。記錄各情況下的重投影誤差與參數估計值，量化每個因素的貢獻。這個實驗能建立對校正的直覺，避免日後盲目重做。
- **進階：PBVS 與 IBVS 的軌跡比較**。在模擬中實作兩種視覺伺服，讓相機由同一個初始位姿收斂到同一個目標位姿。分別繪出：(一) 相機在三維空間中的運動軌跡；(二) 特徵點在

影像平面上的運動軌跡。重複測試多種初始姿態，特別包含「需要繞光軸旋轉 180 度」的情形——你會看到 IBVS 出現著名的「相機退離」現象（相機先後退再前進），而 PBVS 則會讓特徵跑出視野。這個實驗能讓你深刻理解兩者的取舍。

參考文獻與延伸閱讀

- Hartley, R. and Zisserman, A., Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press (多視角幾何的權威著作)。
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer (電腦視覺的完整教材，線上免費提供)。
- Corke, P., Robotics, Vision and Control, 3rd ed., Springer (機器人視覺的實作導向教材，含大量程式範例)。
- Zhang, Z., "A Flexible New Technique for Camera Calibration," IEEE TPAMI, 2000 (目前標準相機校正方法的原始論文)。
- Tsai, R. Y. and Lenz, R. K., "A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/Eye Calibration," IEEE Trans. Robotics and Automation, 1989 (手眼校正的經典方法)。
- Chaumette, F. and Hutchinson, S., "Visual Servo Control, Part I & II," IEEE Robotics & Automation Magazine, 2006-2007 (視覺伺服的權威教學論文)。
- Fischler, M. A. and Bolles, R. C., "Random Sample Consensus," Communications of the ACM, 1981 (RANSAC 的原始論文)。
- Rublee, E. et al., "ORB: An Efficient Alternative to SIFT or SURF," ICCV 2011 (ORB 特徵)。
- OpenCV 官方文件：相機校正、手眼校正、特徵偵測與匹配的完整 API 與教學。
- Olson, E., "AprilTag: A Robust and Flexible Visual Fiducial System," ICRA 2011 (視覺標記系統)。

第 23 章

行動機器人運動學

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為路邊停車的俯視連續圖：一台車經由「前進打右、後退打左」的四個步驟，逐步挪進與車身平行的車位；每一步以箭頭標出當下的運動方向，並以紅色虛線標註「無法直接橫移」。右半為三種構型的瞬時旋轉中心對照——差速驅動（ICR 可落在兩輪共軸的任意位置，含原地旋轉）、阿克曼轉向（ICR 為前後輪軸線的交點，故有最小轉彎半徑）、麥克納姆全向平台（無側滑約束，可任意方向平移）。畫面下緣文字：瞬時受限，全域自由——當下不能做的事，繞個路就能做到。

叫獸開講 「兩輪小車為什麼不能往正右方平移？」

叫獸把一台自製的差速輪小車放在地上，然後對全班說：「請你們告訴我怎麼讓它往**正右方**移動——注意，是平移，車頭方向不能改變。」

小翰馬上說：「右輪往後轉、左輪往前轉。」叫獸照做，車子原地打轉了半圈——車頭轉了，但位置沒動。

小妤說：「那兩輪同速往前？」車子直直往前開——位置動了，但方向是前方不是右方。

學生們接連提出各種組合，車子不是原地轉、就是往前後走、要不就畫一個弧。二十分鐘過去，沒有人能讓它「正右方平移」。

「因為那是**做不到的**。」叫獸終於揭曉，「不是你們笨，是這台車在物理上就沒有這個能力。輪子只能滾，不能橫著滑——這叫做**非完整約束**。」

他在白板上寫下一個對比：「第 4 章我們數過自由度。這台車在平面上的位形有三個變數： x 、 y 、還有車頭角度 θ 。但它只有兩個馬達。」

小翰想了想：「那不就是欠驅動嗎？就像第 18 章的倒立擺。」

「很接近，但**性質完全不同**，而且這個差別非常微妙。」叫獸拿起小車，「倒立擺是『有些狀態你到不了』——桿子倒下去就爬不起來。但這台車不一樣：**停車場裡任何一個位置與角度，它都到得了**。你只是不能『直接』過去，必須來回打方向盤，像路邊停車那樣。」

他在白板上畫了兩個詞：**瞬時受限、全域自由**。「這就是非完整系統最迷人的地方——當下能走的方向被限制了，但長期能到的地方沒有被限制。這兩件事同時成立，聽起來矛盾，卻是真的。」

小妤問：「那為什麼不乾脆設計成能橫移的？」

「可以啊，那叫麥克納姆輪，掃地機器人展場上那種能橫著走的平台就是。」叫獸點頭，「但它的代價是：輪子結構複雜、容易卡髒東西、效率低、承載差、而且**里程計極不準**。工程上永遠是取捨——多數場合，兩個馬達加兩個輪子的簡單方案就夠了，剩下的用**規劃**來補。」

「所以今天要學兩件事。第一，**這些車子能怎麼動**——各種輪型的運動學模型。第二，**當下不能做的事，怎麼靠規劃繞過去**。學完之後，你會理解為什麼路邊停車需要來回三次，以及為

什麼機器人也一樣。」

本章為什麼重要

第二篇與第三篇處理的都是**固定基座**的機械手臂：基座不動，只有關節在轉。但機器人的另一大類——**移動機器人**——面對的是完全不同的問題。

差別的核心在於**約束的性質**。機械手臂的關節可以獨立控制，想讓末端往哪個方向動都可以（除非遇到第 12 章的奇異點）。但輪型機器人不同：**輪子只能沿其滾動方向前進，不能側滑**。這個看似微小的限制，帶來了機器人學中最深刻也最違反直覺的一類問題。

本章要建立三個層次的理解。**第一，運動學模型**：各種輪型構型（差速、阿克曼、全向）能產生什麼運動，以及如何由輪速推算車體運動。**第二，非完整約束的本質**：為何「當下受限」不等於「到不了」，以及這對控制與規劃的深遠影響。**第三，里程計與其極限**：如何由輪子的轉動推算位置，以及為何這個推算必定會漂移——這正是第 24、25 章要處理的問題。

本章也是第四篇的樞紐。第 24 章的狀態估測需要本章的運動模型作為預測方程、第 25 章的 SLAM 建立在里程計之上、第 26 章的路徑規劃必須考慮本章的運動約束。

學習目標

- 能說明輪子的四種基本類型及其約束特性。
- 能推導差速驅動機器人的正向與逆向運動學。
- 能說明瞬時旋轉中心的概念並用它分析各種輪型構型。
- 能定義非完整約束，並區分其與完整約束的差異。
- 能說明「瞬時受限但全域可達」的意義及其對規劃的影響。
- 能區分非完整約束與欠驅動，並說明兩者的關係。
- 能推導阿克曼轉向的運動學模型並說明其與差速驅動的差異。
- 能說明全向輪與麥克納姆輪的原理、優勢與代價。
- 能實作里程計的積分計算，並分析其誤差來源。
- 能說明里程計必定漂移的原因，並提出改善策略。
- 能為特定應用選擇合適的移動平台構型並說明理由。

先備知識

第 4 章的自由度與機構分析、第 8 章的座標系與旋轉矩陣（二維版本）、第 6.2 節的編碼器、第 11 章的雅可比概念（本章的運動學矩陣扮演類似角色）。

數學方面需要：二維旋轉、簡單的微分方程積分、以及矩陣的秩。本章的數學比第三篇簡單得多，但幾何直覺與物理意義更為關鍵——建議多用手邊的玩具車或椅子實際比劃。

核心名詞中英對照

表 23-1 本章核心名詞

中文	英文	說明
移動機器人	Mobile Robot	能在環境中移動的機器人
差速驅動	Differential Drive	以兩獨立驅動輪控制的構型
阿克曼轉向	Ackermann Steering	汽車式的前輪轉向構型
全向輪	Omni Wheel	可側向滑動的特殊輪子
麥克納姆輪	Mecanum Wheel	輪面有斜向滾子的全向輪
瞬時旋轉中心	Instantaneous Center of Rotation, ICR	剛體瞬時繞其轉動的點
完整約束	Holonomic Constraint	只限制位形、可積分的約束
非完整約束	Nonholonomic Constraint	限制速度方向、不可積分的約束
位形空間	Configuration Space	描述機器人所有可能位形的空間
可達性	Reachability / Controllability	能否到達位形空間中的任意點
李括號	Lie Bracket	衡量兩個運動方向組合能否產生新方向
里程計	Odometry	由輪子轉動推算位置的方法
航位推算	Dead Reckoning	由運動歷史推算當前位置
輪距	Wheel Base / Track Width	兩驅動輪之間的距離
打滑	Slip	輪子與地面之間的相對滑動
系統誤差	Systematic Error	可重複、可校正的誤差
隨機誤差	Random Error	不可預測、只能統計描述的誤差

23.1 輪子的類型與約束

23.1.1 四種基本輪型

移動機器人的能力，根本上取決於使用什麼輪子。常見的有四種。

類型一：標準固定輪 (Fixed Wheel)。最常見的輪子，只能繞輪軸滾動，轉向軸固定在車體上。它施加一個關鍵的約束：**輪子不能沿其軸向側滑**。差速驅動車的兩個驅動輪就是這一類。

類型二：可轉向輪 (Steerable Wheel)。在固定輪的基礎上，增加一個垂直的轉向軸，可以改變滾動方向。汽車的前輪、以及許多 AGV 的驅動輪屬於此類。

類型三：腳輪 (Caster Wheel)。轉向軸與輪心有一段偏移（像購物車的輪子）。它會自動對齊運動方向，因此**不施加任何約束**——純粹是支撐用途。

類型四：全向輪 (Omni / Mecanum Wheel)。輪面上安裝了許多可自由轉動的小滾子，使輪子在滾動的同時也能側向滑動。它**解除了不能側滑的約束**，代價見 23.6 節。

表 23-2 四種輪型的比較

輪型	自由度	是否施加約束	承載	里程計精度	典型應用
標準固定輪	1 (滾動)	是 (不可側滑)	高	高	差速車、AGV 驅動輪
可轉向輪	2 (滾動 + 轉向)	是 (不可側滑)	高	中高	汽車前輪、舵輪式 AGV
腳輪	2 (被動)	否	中	不適用 (被動)	支撐用、購物車
全向 / 麥克納姆輪	2 (滾動 + 側滑)	否	低	低	全向平台、比賽機器人

23.1.2 「不能側滑」這個約束

標準輪的約束可以精確地寫出來。設輪子的位置為 (x, y) 、輪面方向（滾動方向）與車體 X 軸夾角為 α ，則不可側滑的條件是：**輪子在其軸向的速度分量必須為零**。

$$-\dot{x} \sin(\theta + \alpha) + \dot{y} \cos(\theta + \alpha) = 0$$

不可側滑約束。 θ 為車體的朝向角。這條式子說：速度向量必須垂直於輪軸，也就是必須沿著輪面方向。

這條方程式看似簡單，卻是本章所有討論的根源。它限制的是**速度**（含 $\dot{x}$ 、 $\dot{y}$ ），而非位置——這正是它與第 4 章那些機構約束的根本差異，也是 23.3 節「非完整」一詞的由來。

23.1.3 瞬時旋轉中心

有一個極為有用的幾何工具能統一分析各種輪型構型：**瞬時旋轉中心**（Instantaneous Center of Rotation, ICR）。

◎ 瞬時旋轉中心

任何平面剛體的瞬時運動，都可以描述為「繞某一點的純旋轉」。這個點就是瞬時旋轉中心。若剛體在做純平移，則 ICR 位於無窮遠處。

ICR 的價值在於一個關鍵性質：**每一個標準輪的軸線，都必須通過 ICR**。原因很直觀——輪子只能沿輪面方向滾動，代表它的速度垂直於輪軸；而繞 ICR 旋轉的點，其速度也垂直於「該點到 ICR 的連線」。兩者要相符，輪軸就必須指向 ICR。

這給出了一個強大的分析工具：

- **差速驅動**：兩輪共軸，ICR 必在該共同軸線上（可以是軸線上任一點，因此可以原地旋轉）。
- **阿克曼轉向**：後輪共軸、前輪可轉向，ICR 為「後輪軸線」與「前輪軸線」的交點——這也是為何汽車無法原地旋轉（ICR 永遠在車體側方一定距離外）。
- **三輪或多輪皆固定**：若三個輪軸不交於一點，則系統被過度約束，只能直行或完全不能動。

圖 23-1 三種構型的瞬時旋轉中心

建議配圖：三格俯視圖。左格為差速驅動車——兩驅動輪共軸，虛線延伸的軸線上標出 ICR 可位於的任意位置（包含兩輪中點，即原地旋轉）。中格為阿克曼轉向車——後輪軸線與兩前輪軸線交會於一點（ICR），並標註內外前輪的轉角不同（阿克曼幾何）。右格為全向輪平台——三個或四個全向輪，標註「無側滑約束，ICR 可在平面上任意位置」。三格下方各標註對應的可行運動：原地旋轉、最小轉彎半徑、任意方向平移。

23.2 差速驅動運動學

23.2.1 最常見的構型

差速驅動（differential drive）是移動機器人最常見的構型：兩個同軸的獨立驅動輪，加上一至兩個腳輪支撐。掃地機器人、多數 AGV、教學小車都採用它。

它之所以普及，有四個原因：**機構極簡**（兩顆馬達、兩個輪子）、**能原地旋轉**（機動性佳，適合狹窄空間）、**運動學簡單**（易於控制與里程計）、**成本低**。

23.2.2 正向運動學

設兩輪的線速度分別為 v_L (左輪) 與 v_R (右輪)，輪距 (兩輪中心距) 為 L 。車體中心的線速度與角速度為：

$$v = (v_R + v_L)/2 \quad \omega = (v_R - v_L)/L$$

差速驅動的正向運動學。線速度是兩輪的平均、角速度正比於兩輪的差。這是本章最重要的一組公式。

三種典型情形可以立刻讀出：

- $v_R = v_L$: $\omega = 0$ ，車子直行。
- $v_R = -v_L$: $v = 0$ ，車子原地旋轉 (ICR 在兩輪中點)。
- $v_L = 0$: ICR 在左輪處，車子繞左輪畫圓。

接著把車體速度轉換到世界座標系。設車體位姿為 (x, y, θ) ：

$$\begin{aligned}\dot{x} &= v \cos \theta \\ \dot{y} &= v \sin \theta \\ \dot{\theta} &= \omega\end{aligned}$$

車體的運動方程。這三條式子構成一個非線性系統，是第 24~26 章的共同基礎。

23.2.3 逆向運動學

反過來，若要達成指定的 (v, ω) ，各輪應轉多快？由 23.2.2 節的兩式聯立求解：

$$v_R = v + \omega L/2 \quad v_L = v - \omega L/2$$

差速驅動的逆向運動學。注意這個逆解永遠存在且唯一——與第 10 章手臂逆運動學的多解與無解形成鮮明對比。

最後換算為馬達的角速度 (設輪半徑為 r)：

$$\omega_R = v_R / r \quad \omega_L = v_L / r$$

輪子的角速度。這才是真正送給馬達控制器的指令。

23.2.4 轉彎半徑與速度限制

由 v 與 ω 可求出瞬時的轉彎半徑：

$$R = v / \omega$$

轉彎半徑。 $\omega = 0$ 時 $R \rightarrow \infty$ (直行) ; $v = 0$ 時 $R = 0$ (原地旋轉)。

實務上有一個重要的限制：**輪速不能超過馬達的最高速度**。這意味著 v 與 ω 不能任意組合——高速行進時，可用的轉向能力會下降。

$$|v| + |\omega|L/2 \leq v_{\max}$$

速度限制條件。這在 (v, ω) 平面上構成一個菱形的可行區域。

這個限制在軌跡追隨時必須考慮。若控制器算出的 (v, ω) 超出可行區域，直接截斷會改變運動的方向；較好的作法是等比例縮小兩者，保持轉彎半徑不變而只降低速度。

23.3 非完整約束

23.3.1 兩種約束的根本差異

現在可以正式回答開頭故事的問題。約束分為兩大類，差別在於能否積分。

完整約束 (Holonomic)：只限制位形本身，可以寫成 $f(q) = 0$ 的形式。例如第 4 章的閉環機構——連桿的長度限制了各關節角度的組合。它的效果是直接減少位形空間的維度。

非完整約束 (Nonholonomic)：限制的是速度，形式為 $f(q, \dot{q}) = 0$ ，且無法積分成只含 q 的形式。輪子的不可側滑就是典型例子。

完整約束： $f(q) = 0 \Rightarrow$ 減少可達位形的維度

非完整約束： $f(q, \dot{q}) = 0$ (不可積分) $\Rightarrow$ 限制速度方向，但不減少可達位形

兩者的關鍵差異。這是本章最重要的概念。

23.3.2 為何「不可積分」如此關鍵

若不可側滑的約束能夠積分，就會得到一條形如 $g(x, y, \theta) = 0$ 的方程——那代表車子被限制在一個曲面上，永遠到不了曲面外的位形。

但實際上它積分不出來。這意味著：約束只在每個瞬間限制速度的方向，卻沒有在位形空間中劃出任何禁區。車子當下不能側移，但透過一連串的前進、後退與轉向，它可以到達平面上的任何位置與任何朝向。

這正是路邊停車的數學解釋：你不能把車橫著推進車位，但可以透過「前進打右、後退打左」的組合逐步挪進去。每一步都遵守約束，組合起來卻達成了約束「禁止」的效果。

◎ 李括號：為何組合能產生新方向

數學上，這個現象由李括號 (Lie bracket) 描述。設車子有兩個可控的運動方向：前進 g_1 與轉向 g_2 。李括號 $[g_1, g_2]$ 定義為「先做 g_1 再做 g_2 ，減去先做 g_2 再做 g_1 」的淨效果。由於兩者不可交換（正如第 8 章的旋轉），這個差值不為零——而且它恰好指向側移的方向！Chow-Rashevskii

定理指出：若這些李括號能張滿整個位形空間，系統就是完全可控的。這給出了「瞬時受限但全域自由」的嚴格數學基礎，也解釋了為何「來回打方向盤」能產生側移。

23.3.3 自由度的三個數字

非完整系統有三個容易混淆的「自由度」數字，必須分清：

表 23-3 差速驅動機器人的三個自由度

名稱	數值	意義
位形空間維度	3	描述位形需要 (x, y, θ) 三個變數
可控自由度（瞬時）	2	當下只能控制 v 與 ω 兩個量
可達自由度（全域）	3	長期而言可到達任意 (x, y, θ)

第一與第三個數字相等，這是關鍵。它代表系統雖然瞬時受限，卻沒有失去任何可達性。這也是非完整系統與欠驅動系統的分界（見下節）。

23.3.4 與欠驅動的區別

開頭故事中小翰的疑問值得深究：非完整與欠驅動有何不同？

兩者的共同點是「控制輸入數少於位形空間維度」。但**後果完全不同**：

非完整系統（如差速車）：可控自由度 $<$ 位形維度，但**可達自由度 = 位形維度**。所有位形都到得了，只是路徑受限。

欠驅動系統（如倒立擺、無人機）：某些狀態根本無法穩定或到達。倒立擺的桿子若倒下超過某個角度就再也扶不起來（第 18 章的吸引域）。

因此兩者的**難處不同**：非完整系統的挑戰在**規劃**（如何找出一條合乎約束的路徑），欠驅動系統的挑戰在**控制**（如何穩定住不穩定的狀態）。這也預示了本章與第 26 章的關係——非完整約束會直接影響路徑規劃的方法。

23.4 阿克曼轉向

23.4.1 汽車式構型

阿克曼轉向（Ackermann steering）是汽車採用的構型：後輪固定驅動、前輪可轉向。自駕車、大型 AGV 與戶外移動平台多採此設計。

它的核心幾何要求是：**轉彎時，內外側前輪的轉角必須不同**。原因由 ICR 分析立刻可知——內外前輪到 ICR 的距離不同，因此其軸線指向 ICR 的角度也不同。若兩前輪同角度轉動（如簡單的平行連桿），輪胎必定有一個在打滑。

$$\tan \delta_{\text{inner}} = L/(R - W/2) \quad \tan \delta_{\text{outer}} = L/(R + W/2)$$

阿克曼幾何。 L 為軸距（前後輪距）、 W 為輪距（左右輪距）、 R 為轉彎半徑。內輪轉角恆大於外輪。

23.4.2 單車模型

為了簡化分析，常把左右輪合併為中間的一個輪子，稱為**單車模型**（bicycle model）。設前輪轉角為 δ 、後輪中心的速度為 v ：

$$\begin{aligned}\dot{x} &= v \cos \theta \\ \dot{y} &= v \sin \theta \\ \dot{\theta} &= (v/L) \tan \delta\end{aligned}$$

單車模型的運動方程。這是自駕車與路徑追隨演算法最常用的模型。

與差速驅動比較，可以看出兩個關鍵差異：

差異一：角速度與線速度耦合。 $\dot{\theta} = (v/L)\tan \delta$ 中含有 v ——車子不動就無法轉向。這與差速驅動能原地旋轉（ $v = 0$ 而 $\omega \neq 0$ ）形成鮮明對比。

差異二：最小轉彎半徑。前輪轉角有機械極限 δ_{max} ，因此：

$$R_{\text{min}} = L / \tan \delta_{\text{max}}$$

最小轉彎半徑。這是阿克曼構型的根本限制，直接影響路徑規劃（第 26 章的 Dubins 路徑即由此而來）。

23.4.3 為何汽車採用這個構型

既然阿克曼的機動性明顯不如差速驅動（不能原地轉、有最小轉彎半徑），為何汽車都採用它？三個理由：

- **高速穩定性：**差速驅動在高速下容易因兩輪速度的微小差異而偏擺；阿克曼的前輪轉向提供了穩定的方向控制。
- **承載與效率：**後輪可用單一動力源加差速器驅動，機構簡單且能承受大扭力。
- **輪胎磨耗：**正確的阿克曼幾何使所有輪子都是純滾動，不會側滑，大幅延長輪胎壽命。差速驅動在轉彎時腳輪必定有側滑。

這是一個典型的工程取捨：**用機動性換取高速穩定性、承載能力與效率**。因此低速、室內、

需要靈活的場合用差速驅動；高速、戶外、大負載的場合用阿克曼。

表 23-4 差速驅動與阿克曼轉向的比較

面向	差速驅動	阿克曼轉向
原地旋轉	可	不可
最小轉彎半徑	0	$L/\tan \delta_{\max}$
高速穩定性	較差	佳
機構複雜度	極簡	中 (需轉向連桿與差速器)
輪胎磨耗	較大 (腳輪側滑)	小 (純滾動)
運動學	解耦 (v 與 ω 獨立)	耦合 (θ 正比於 v)
典型應用	掃地機器人、室內 AGV	汽車、自駕車、戶外平台

23.5 全向移動平台

23.5.1 解除約束的代價

既然非完整約束帶來這麼多麻煩，能不能乾脆消除它？答案是可以——用**全向輪**。

全向輪的原理是在輪面上安裝許多可自由轉動的小滾子。輪子在被驅動時沿滾動方向前進，同時可以沿滾子的方向自由滑動。這使「不可側滑」的約束消失了。

麥克納姆輪 (Mecanum wheel) 是最常見的變體：滾子與輪軸成 45 度角。四個麥克納姆輪 (兩兩鏡像配置) 的組合，能產生平面上的完整三自由度運動——前後、左右、原地旋轉，且三者可任意組合。

23.5.2 麥克納姆輪的運動學

設四個輪子依序為左前、右前、左後、右後，輪半徑 r ，車體半長 L 、半寬 W 。由車體速度 (v_x, v_y, ω) 求各輪角速度：

$$\begin{aligned}\omega_{LF} &= (v_x - v_y - (L+W)\omega)/r \\ \omega_{RF} &= (v_x + v_y + (L+W)\omega)/r \\ \omega_{LR} &= (v_x + v_y - (L+W)\omega)/r \\ \omega_{RR} &= (v_x - v_y + (L+W)\omega)/r\end{aligned}$$

麥克納姆輪的逆運動學。注意四個式子中 v_y 與 ω 的符號組合各不相同——這正是產生全向運動的關鍵。

這組公式有一個值得注意的結構：**四個輪子、三個自由度**。這代表系統是**冗餘驅動的** (第 5.4.2 節提過的 *overactuated*)。好處是可以分散負載；壞處是若四輪的速度不一致 (如某輪打

滑)，會產生內部的「打架」——輪子互相對抗，浪費能量並加速磨損。

23.5.3 全向平台的三大代價

代價一：里程計精度極差。滾子與地面的接觸面積小、且滾動方向不斷改變，打滑遠比標準輪嚴重。實務上麥克納姆平台的里程計誤差可達標準差速車的數倍——這使它幾乎必須依賴外部定位（第 25 章的 SLAM 或視覺標記）。

代價二：承載與效率低。滾子是點接觸或線接觸，承載能力遠不如整個輪面著地的標準輪。同時，斜向的滾子使部分驅動力被「浪費」在無用的方向上，效率典型只有標準輪的 70~80%。

代價三：對地面敏感。滾子的縫隙容易卡入砂石與毛髮；不平整的地面會使某些輪子懸空，瞬間失去該輪的貢獻，造成運動偏差。因此麥克納姆平台幾乎只用於平整的室內地面。

◎ 為何工廠的 AGV 很少用麥克納姆輪

展場上的麥克納姆平台總是令人驚豔——能橫著走、能斜著走、能原地打轉。但走進真實的工廠，你會發現絕大多數的 AGV 仍是差速驅動或阿克曼構型。原因正是上述三個代價：工廠地面有油污與碎屑、需要長時間高負載運行、且對定位精度要求嚴格。全向能力是一個「好用但昂貴」的功能，只有在空間極度受限（如狹窄走道中的橫向靠泊）時才值得付出代價。這再次體現了工程上的取舍思維——多數場合，用規劃補足運動限制，比用機構解除限制更划算。

23.5.4 三輪全向構型

另一種常見的全向設計是三個全向輪呈 120 度均勻配置。它比麥克納姆簡單（三個輪子、三個自由度，剛好不冗餘）、運動學更對稱，常見於機器人足球賽（RoboCup）等競賽場合。

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix} = \begin{bmatrix} 1 & -\sin(\theta_1) & \cos(\theta_1) & R \\ - & -\sin(\theta_2) & \cos(\theta_2) & R \\ r & -\sin(\theta_3) & \cos(\theta_3) & R \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}$$

三輪全向平台的逆運動學。 θ_i 為各輪的安裝角度（通常為 0° 、 120° 、 240° ）、 R 為輪子到中心的距離。

這個 3×3 矩陣是可逆的（只要三輪不共線），因此正向運動學也有唯一解——這與差速驅動一樣，比機械手臂的逆運動學單純得多。

23.6 履帶與其他構型

23.6.1 履帶式（差速轉向）

履帶車（如挖土機、軍用機器人、救災機器人）在運動學上可視為「加寬的差速驅動」——

—左右履帶的速度差產生轉向。

但有一個關鍵差異：**履帶轉向必定伴隨大量的滑動**。因為履帶與地面是長條形接觸，轉彎時履帶的前後段必須相對地面滑移。這稱為**滑移轉向**（skid steering）。

後果是：

- **里程計極不準**：滑移量難以預測，且隨地面材質變化極大。實務上履帶車的航向誤差可達每公尺數度。
- **能耗高**：大量能量消耗在克服滑動摩擦上。
- **地面破壞**：轉向時會刮傷地面，這在室內或草坪上是問題。

但它的優勢也極為突出：**接地面積大、越野能力強、能爬樓梯與跨越障礙**。因此在戶外崎嶇地形、災難現場與軍事應用中不可取代。這類應用通常依賴 GNSS 或光達定位，而非里程計。

23.6.2 構型選擇的決策

整合前面各節，可以給出一個選型的決策準則：

表 23-5 移動平台構型的選型指引

需求	建議構型	理由
室內、狹窄空間、低速	差速驅動	能原地旋轉、機構簡單、里程計佳
高速、戶外、大負載	阿克曼轉向	高速穩定、承載佳、輪胎磨耗小
需要橫向靠泊、極窄走道	麥克納姆 / 全向	唯一能側移的方案，但需接受其代價
崎嶇地形、爬樓梯、越野	履帶	接地面積大、越野能力強
競賽、高機動性、平整場地	三輪全向	運動學對稱、機構較麥克納姆簡單
超大負載、需精確路徑	多舵輪（獨立轉向）	各輪可獨立轉向，兼具全向與承載

23.7 里程計

23.7.1 由輪子推算位置

里程計（odometry）是最基本的定位方法：由輪子轉了多少，推算車子走了多遠、轉了多少角度。它屬於**航位推算**（dead reckoning）的一種——不依賴任何外部參照，純粹累積內部感測的資訊。

對差速驅動車，每個控制週期 Δt 內的計算為：

1. 讀取編碼器：得到左右輪的增量脈波數 ΔN_L 、 ΔN_R 。
2. 換算輪子位移： $\Delta s_L = 2\pi r \cdot \Delta N_L / (\text{PPR} \times 4 \times N_{\text{gear}})$ ，右輪同理（第 6.2.1 節的四倍頻與減速比）。
3. 計算車體位移與轉角： $\Delta s = (\Delta s_R + \Delta s_L) / 2$ ， $\Delta \theta = (\Delta s_R - \Delta s_L) / L$ 。
4. 更新位姿：積分到世界座標系。

23.7.2 積分方法的選擇

第四步的積分有兩種常見作法，精度差異值得注意。

方法一：直線近似（最簡單）。假設每個週期內車子沿直線移動：

$$\begin{aligned} x &\leftarrow x + \Delta s \cdot \cos \theta \\ y &\leftarrow y + \Delta s \cdot \sin \theta \\ \theta &\leftarrow \theta + \Delta \theta \end{aligned}$$

直線近似。實作最簡單，但當 $\Delta \theta$ 較大時誤差明顯。

方法二：中點近似（推薦）。用該週期的中間角度計算位移方向：

$$\begin{aligned} x &\leftarrow x + \Delta s \cdot \cos(\theta + \Delta \theta / 2) \\ y &\leftarrow y + \Delta s \cdot \sin(\theta + \Delta \theta / 2) \\ \theta &\leftarrow \theta + \Delta \theta \end{aligned}$$

中點近似。只多一個除以 2，精度卻顯著提升——這是實務上的標準作法。

方法三：精確圓弧。若 $\Delta \theta \neq 0$ ，該週期的軌跡實際上是一段圓弧（半徑 $R = \Delta s / \Delta \theta$ ），可以精確積分：

$$\begin{aligned} x &\leftarrow x + R[\sin(\theta + \Delta \theta) - \sin \theta] \\ y &\leftarrow y - R[\cos(\theta + \Delta \theta) - \cos \theta] \\ \theta &\leftarrow \theta + \Delta \theta \end{aligned}$$

圓弧積分。理論上最精確，但當 $\Delta \theta \rightarrow 0$ 時 $R \rightarrow \infty$ ，需特別處理數值問題。

實務建議：控制週期夠短（如 10 ~ 50 ms）時，中點近似的精度已足夠，且無數值問題。只有在低頻更新或高速旋轉時，才值得使用圓弧積分。

23.7.3 里程計的價值與定位

里程計有三個無可取代的優點：更新率極高（可達 kHz 級）、短期精度極佳、永不失效（不受環境影響，黑暗、無特徵、無 GNSS 都能運作）。

但它有一個致命缺陷：**誤差會累積且永不修正**。這正是第 6 章開頭故事中「步伐計數」的問題——走得越遠，錯得越多。

因此里程計在系統中的定位是：**提供高頻的短期預測，再由低頻但不漂移的外部感測（光達、視覺、GNSS）定期校正**。這正是第 24 章感測器融合與第 25 章 SLAM 的核心架構。

23.8 里程計的誤差分析

23.8.1 為何必定漂移

里程計的本質是**積分**——把每個微小的位移累加起來。而積分有一個數學上的必然性質：

◎ 積分的宿命

任何量測都有誤差。當我們把一連串帶誤差的量測**累加**起來時，誤差也會被累加。而且關鍵在於：**沒有任何機制能讓誤差減小**——里程計不知道自己錯了，因為它沒有任何外部參照可以比對。這使得漂移不是「品質不好」的問題，而是**方法本身的宿命**。

更糟的是角度誤差的放大效應：一個微小的航向誤差，會隨著行進距離**線性放大**成位置誤差。

$$\text{位置誤差} \approx \text{行進距離} \times \text{航向誤差 (弧度)}$$

以 1 度 (0.0175 rad) 的航向誤差為例，走 10 公尺後橫向偏差就達 17.5 公分。

這解釋了一個實務現象：**里程計的位置誤差主要來自角度而非距離**。因此改善里程計最有效的方向，是提升航向的精度（例如融合陀螺儀，第 24 章）。

23.8.2 誤差的兩大類

Borenstein 的經典分類把里程計誤差分為兩類，處理方式完全不同。

系統誤差 (Systematic)：可重複、可預測的誤差，主要來自三個源頭：

- **輪徑不等**：兩輪的實際直徑有微小差異（製造公差、磨耗、胎壓）。這使機器人在指令直行時實際上會畫一個大弧。
- **輪距量測不準**：公式中的 L 若與實際不符，所有的角度計算都會有比例誤差。
- **輪徑量測不準**：使所有的距離計算都有比例誤差。

隨機誤差 (Non-systematic)：不可預測，主要來自：

- **打滑**：地面濕滑、加速過猛、碰撞。這是最嚴重也最難處理的一項。

- 地面不平：輪子跳動、瞬間懸空。
- 編碼器量化誤差：有限的解析度（第 6.2.1 節）。
- 離散取樣誤差：兩次取樣之間的運動被近似為直線或圓弧。

23.8.3 UMBmark 校正法

系統誤差是可以校正的，Borenstein 與 Feng 於 1996 年提出的 UMBmark 是標準方法。

其原理極為巧妙：讓機器人沿著一個正方形路徑行走，分別以順時針與逆時針各走數圈，然後量測回到起點時的偏差。

關鍵洞見是：兩種誤差在順逆時針下的表現不同。輪徑不等造成的偏差在順逆時針時方向相反（會互相抵消或加成），而輪距誤差造成的偏差則在兩者中同向。因此由兩組實驗的結果，可以分離出這兩個誤差源，並計算出對應的修正係數。

1. 準備：在地面標出一個邊長 2~4 公尺的正方形，起點做精確標記。
2. 順時針實驗：讓機器人沿正方形走五圈，記錄最終位置與起點的偏差 ($\Delta x, \Delta y$)。重複五次取平均。
3. 逆時針實驗：同樣走五圈五次，記錄偏差。
4. 計算修正係數：由兩組偏差的和與差，分別解出輪徑比修正係數 E_d 與輪距修正係數 E_b 。
5. 驗證：套用修正係數後重做實驗，偏差應顯著減小（通常可改善 3~10 倍）。

經過 UMBmark 校正後，典型的差速驅動機器人可以把系統誤差由數個百分點降到 0.5% 以下——這是一個投報率極高的程序，只需要一個下午與一條捲尺。

23.8.4 誤差的量化與模型

為了在第 24 章的感測器融合中使用里程計，必須先量化其誤差。常用的模型是把誤差視為與運動量成正比的隨機變數：

$$\sigma_{\Delta s^2} = k_s \cdot |\Delta s| \quad \sigma_{\Delta \theta^2} = k_{\theta} \cdot |\Delta \theta| + k_{s\theta} \cdot |\Delta s|$$

里程計的誤差模型。變異數正比於運動量——走得越多、轉得越多，不確定性越大。 k 值由實驗量測。

最後一項 $k_{s\theta} \cdot |\Delta s|$ 特別值得注意：它代表直行時也會累積航向誤差（因為輪徑不等）。這一項往往是長距離漂移的主因。

表 23-6 里程計誤差來源與因應

誤差來源	類型	典型量級	因應方式
輪徑不等	系統	0.5 ~ 3%	UMBmark 校正
輪距不準	系統	0.5 ~ 2%	UMBmark 校正
編碼器量化	隨機	< 0.1%	提高解析度、裝在馬達端
打滑	隨機	0 ~ 數十 %	限制加速度、偵測異常、融合其他感測
地面不平	隨機	1 ~ 5%	改善懸吊、融合 IMU
取樣離散化	隨機	< 0.1%	提高更新率、用中點或圓弧積分

23.9 軌跡追隨控制

23.9.1 非完整系統的控制困難

有了運動學模型，下一步是控制：如何讓機器人走上指定的路徑？這裡有一個著名的理論障礙。

◎ Brockett 條件

1983 年 Brockett 證明了一個令人意外的結果：非完整系統無法用任何連續的、非時變的狀態回授控制律，穩定到一個固定的位形點。換句話說——不存在一個形如 $u = f(x)$ 的平滑控制器，能讓差速車由任意初始位姿收斂到指定的 (x, y, θ) 。

這個結論初看很違反直覺：明明第 23.3 節說過所有位形都到得了，為何不能用回授控制穩定過去？

直觀的理解是：問題出在「同時」。要讓 x 、 y 、 θ 三個量同時收斂到零，但控制輸入只有兩個。當車子已經對準目標方向、位置也對了，卻只有 θ 有誤差時——此時任何前進或後退都會破壞已經對好的位置。控制器陷入兩難。

因應之道有三：時變控制律（讓控制律隨時間變化，如加入振盪項）、不連續控制律（分階段切換，如「先轉向、再前進、最後調整角度」）、或放棄點穩定，改為軌跡追隨。實務上第三種最常用。

23.9.2 軌跡追隨與路徑追隨

實務上有兩種不同的任務，區別很重要：

軌跡追隨 (Trajectory Tracking)：追隨一條帶時間的參考軌跡 ($x_d(t)$, $y_d(t)$, $\theta_d(t)$)。目標是「在第 t 秒到達第 t 秒該到的位置」。這與第 15 章的軌跡定義一致。

路徑追隨 (Path Following)：只要求沿著幾何路徑行走，不規定何時到達某點。這在實務上更常用——因為速度可以依情況調整（遇到障礙就減速），而不會產生「落後於時間表」的累積誤差。

關鍵差異：軌跡追隨若因故落後，控制器會試圖加速趕上，可能導致危險；路徑追隨則只在乎「有沒有偏離路徑」，落後多少無所謂。因此戶外導航與自駕車幾乎都採用路徑追隨。

23.9.3 Pure Pursuit：最實用的路徑追隨法

純追蹤法 (Pure Pursuit) 是移動機器人最廣泛使用的路徑追隨演算法，因為它極為簡單且穩健。

它的思想模仿人類開車：**盯著前方一段距離的目標點，朝它轉向**。具體作法是：

1. 在參考路徑上，找出距離車子當前位置為 L_d (**前視距離**，look-ahead distance) 的那個點，稱為目標點。
2. 計算通過車子當前位置、且以當前朝向為切線、並通過目標點的**圓弧**。
3. 沿此圓弧行進，即得所需的轉彎半徑與角速度。

由幾何關係可導出極簡的公式：

$$\kappa = 2 \cdot \sin(\alpha) / L_d \quad \Rightarrow \quad \omega = v \cdot \kappa = 2 \cdot v \cdot \sin(\alpha) / L_d$$

Pure Pursuit 控制律。 κ 為曲率、 α 為「車頭方向」與「車子到目標點連線」的夾角。整個演算法只需要這一行。

前視距離 L_d 是唯一的調參，而它的影響極為直觀：

- L_d 太小：反應激烈、容易震盪（像新手駕駛只看車頭前方）。
- L_d 太大：轉彎時會「切內角」、路徑追隨不準（像看太遠而忽略眼前的彎道）。
- **實務作法**：讓 L_d 隨速度變化 ($L_d = k \cdot v + L_{\min}$)，高速時看遠、低速時看近——這正是人類駕駛的行為。

Pure Pursuit 的優點是簡單、穩健、無需調整複雜的增益；缺點是在高曲率路徑上會有穩態的內切誤差。改良版本（如 Stanley 控制器、模型預測控制）能改善此問題，但複雜度也顯著提

高。

圖 23-2 Pure Pursuit 的幾何原理

建議配圖：俯視圖。畫出一條彎曲的參考路徑（灰色），車子（含車頭方向箭頭）位於路徑旁略有偏離。以車子為圓心、 L_d 為半徑畫一個圓，與參考路徑交於一點——即目標點（紅色）。由車子當前位置出發、以車頭方向為切線、通過目標點畫一段圓弧（藍色虛線），標註其曲率 κ 與轉彎半徑 $R = 1/\kappa$ 。並標出來角 α 。右側另附兩個小圖，比較 L_d 太小（軌跡震盪）與太大（轉彎切內角）的效果。

23.10 從運動學到動力學

23.10.1 何時運動學就夠了

本章的所有討論都停留在**運動學**層次——只談速度與幾何，沒有考慮質量、慣性與力。這個簡化在多數情況下是合理的，原因有三：

- **移動機器人通常低速運行**：室內 AGV 多在每秒 0.5 ~ 1.5 公尺，慣性效應相對於摩擦與驅動 force 並不顯著。
- **輪速可由內迴路精確控制**：馬達驅動器內部的速度迴路（第 5、16 章）已經處理了動力學，上層只需下達速度指令。
- **地面提供了大量阻尼**：滾動摩擦使系統自然穩定，不像機械手臂那樣容易振盪。

因此對多數室內移動機器人而言，**運動學模型加上速度控制的內迴路，已經足夠**。這與第三篇機械手臂必須處理完整動力學的情況形成鮮明對比。

23.10.2 何時必須考慮動力學

但在四種情況下，運動學模型會失效：

情況一：高速運動。速度超過每秒 2 ~ 3 公尺時，慣性使機器人無法瞬間改變速度。此時下達的速度指令與實際運動之間會有明顯的落差。

情況二：急轉彎。向心加速度 $a = v^2/R$ 可能超過摩擦力所能提供的極限，導致側滑——此時不可側滑的約束被打破，運動學模型完全失效。這正是汽車過彎打滑的物理。

不打滑的條件： $v^2/R \leq \mu g$

以 $\mu = 0.7$ 、 $R = 2\text{ m}$ 為例，最高過彎速度僅約 3.7 m/s。這是路徑規劃時的重要限制。

情況三：重載或斜坡。重力分量與大慣量使加減速能力受限，可能無法執行規劃的速度剖面。

情況四：需要精確的力控制。例如移動機器人推動一個重物，或在崎嶇地形上維持牽引力。

這些情況下，需要引入完整的動力學模型——而第 13、14 章建立的工具（慣性、摩擦、力矩限制）就再次派上用場。第 27 章的無人機與第 28 章的腿式機器人，更是完全無法用運動學近似處理。

23.11 由運動邁向定位

23.11.1 本章的成果

本章建立了移動機器人的運動基礎。你現在能夠：分析四種輪型的約束特性、用瞬時旋轉中心分析任何構型、推導差速驅動與阿克曼轉向的運動學、理解非完整約束「瞬時受限但全域自由」的本質、實作里程計並分析其誤差、執行 UMBmark 校正、以及用 Pure Pursuit 實現路徑追隨。

最重要的是那個核心洞見：**非完整約束限制的是「怎麼走」，而非「能到哪裡」**。這使移動機器人的挑戰從「控制」轉向「規劃」——這正是第 26 章的主題。

23.11.2 接下來：里程計不夠用

但本章也留下一個明確的缺口。23.8 節證明了里程計**必定漂移**——這不是工程品質的問題，而是「積分」這個方法的宿命。走十公尺可能偏差十幾公分，走一百公尺就可能偏差一公尺以上。對於需要長時間運行的機器人，這完全不可接受。

解方有兩個層次，正是接下來兩章的主題。

第 24 章（狀態估測與感測器融合）：里程計短期準但會漂移，光達或視覺不漂移但更新慢且有雜訊——**能不能把兩者的優點結合**？這一章將建立機率的框架，用 Kalman 濾波器與粒子濾波器回答這個問題。屆時你會發現，第 6.3.2 節那個「互補濾波」的一行公式，其實是 Kalman 濾波器的簡化版本。

第 25 章（SLAM）：更進一步的問題是——如果連地圖都沒有呢？機器人如何**一邊探索未知環境、一邊建立地圖、同時還要知道自己在哪裡**？這個看似雞生蛋的問題，是機器人學最著名的難題之一。

換句話說：第 23 章讓機器人**知道自己動了多少**，第 24、25 章讓它**知道自己真正在哪裡**。

完整計算例題

例題 23-1 差速驅動的正逆運動學

題目：某差速驅動機器人的輪半徑 $r = 0.05 \text{ m}$ 、輪距 $L = 0.30 \text{ m}$ 。(a) 若左輪以 3.0 rad/s 、右輪以 5.0 rad/s 轉動，求車體的線速度、角速度與轉彎半徑。(b) 若要讓機器人以 0.4 m/s 前進、同時以 0.5 rad/s 左轉，求兩輪的角速度。(c) 若馬達的最高角速度為 12 rad/s ，求機器人的最高直行速度與最高原地旋轉角速度。(d) 若機器人以 0.5 m/s 行進，求此時可用的最大角速度。

解 (a)：先由輪的角速度求線速度：

$$\begin{aligned} v_L &= r \cdot \omega_L = 0.05 \times 3.0 = 0.15 \text{ m/s} \\ v_R &= r \cdot \omega_R = 0.05 \times 5.0 = 0.25 \text{ m/s} \end{aligned}$$

兩輪的線速度。

$$\begin{aligned} v &= (v_R + v_L)/2 = (0.25 + 0.15)/2 = 0.20 \text{ m/s} \\ \omega &= (v_R - v_L)/L = (0.25 - 0.15)/0.30 = 0.333 \text{ rad/s} \end{aligned}$$

車體的線速度與角速度。 ω 為正代表左轉（逆時針）。

$$R = v/\omega = 0.20/0.333 = 0.60 \text{ m}$$

轉彎半徑 0.6 公尺，圓心在車體左側。

解 (b)：由逆運動學公式：

$$\begin{aligned} v_R &= v + \omega L/2 = 0.40 + 0.5 \times 0.15 = 0.475 \text{ m/s} \\ v_L &= v - \omega L/2 = 0.40 - 0.5 \times 0.15 = 0.325 \text{ m/s} \end{aligned}$$

兩輪的線速度。

$$\omega_R = 0.475/0.05 = 9.5 \text{ rad/s} \quad \omega_L = 0.325/0.05 = 6.5 \text{ rad/s}$$

兩輪的角速度。

解 (c)：最高直行速度發生在兩輪同時全速：

$$v_{\max} = r \cdot \omega_{\text{motor}_{\max}} = 0.05 \times 12 = 0.60 \text{ m/s}$$

最高直行速度。

最高原地旋轉角速度發生在兩輪反向全速 ($v_R = 0.6$ 、 $v_L = -0.6$)：

$$\omega_{\max} = (0.6 - (-0.6))/0.30 = 1.2/0.30 = 4.0 \text{ rad/s} \approx 229^\circ/\text{s}$$

最高原地旋轉角速度。

解 (d) : 由 23.2.4 節的速度限制條件 :

$$|v| + |\omega|L/2 \leq v_{\max} \Rightarrow 0.5 + |\omega| \times 0.15 \leq 0.6 \Rightarrow |\omega| \leq 0.667 \text{ rad/s}$$

以 0.5 m/s 行進時，最大角速度僅 0.667 rad/s (對應最小轉彎半徑 0.75 m) 。

工程觀察：速度越快，可用的轉向能力越差。這是所有差速驅動機器人的共同限制，路徑規劃時必須考慮——高速通過的路段必須是平緩的曲線。

例題 23-2 里程計的積分與誤差累積

題目：承上機器人，編碼器為 500 PPR (正交四倍頻)、裝在馬達端、減速比 30。控制週期 20 ms。(a) 求每個編碼器計數對應的輪面位移。(b) 某週期內左輪計數 +120、右輪計數 +150，求該週期的 Δs 與 $\Delta\theta$ 。(c) 若初始位姿為 (0, 0, 0)，用中點近似更新位姿。(d) 若兩輪的實際直徑相差 1%，估算直行 10 公尺後的橫向偏差。

解 (a) : 輪端每轉的計數為 :

$$\text{每輪轉計數} = 500 \times 4 \times 30 = 60000 \text{ counts/rev}$$

四倍頻後再乘減速比 (第 6.2.1 節的技巧) 。

$$\begin{aligned} \text{每計數位移} &= 2\pi r / 60000 = 2\pi(0.05) / 60000 = 0.3142 / 60000 = 5.236 \times 10^{-6} \text{ m} \approx \\ &5.24 \mu\text{m} \end{aligned}$$

解析度極高。

解 (b) :

$$\begin{aligned} \Delta s_L &= 120 \times 5.236 \times 10^{-6} = 6.283 \times 10^{-4} \text{ m} = 0.6283 \text{ mm} \\ \Delta s_R &= 150 \times 5.236 \times 10^{-6} = 7.854 \times 10^{-4} \text{ m} = 0.7854 \text{ mm} \end{aligned}$$

兩輪的位移。

$$\begin{aligned} \Delta s &= (0.7854 + 0.6283) / 2 = 0.7069 \text{ mm} = 7.069 \times 10^{-4} \text{ m} \\ \Delta\theta &= (0.7854 - 0.6283) / 300 = 0.1571 / 300 = 5.236 \times 10^{-4} \text{ rad} = 0.030^\circ \end{aligned}$$

車體的位移與轉角。

驗算： $\Delta s / \Delta t = 7.069 \times 10^{-4} / 0.02 = 0.0353 \text{ m/s}$ ，這是一個合理的低速值。

解 (c) : 用中點近似 (23.7.2 節) :

$$\begin{aligned} x &\leftarrow 0 + 7.069 \times 10^{-4} \times \cos(\theta + 5.236 \times 10^{-4} / 2) = 7.069 \times 10^{-4} \times \cos(2.618 \times 10^{-4}) \approx 7.069 \times 10^{-4} \text{ m} \\ y &\leftarrow 0 + 7.069 \times 10^{-4} \times \sin(2.618 \times 10^{-4}) \approx 1.851 \times 10^{-7} \text{ m} \\ \theta &\leftarrow 0 + 5.236 \times 10^{-4} = 5.236 \times 10^{-4} \text{ rad} \end{aligned}$$

更新後的位姿。因轉角極小， x 幾乎等於 Δs 。

解 (d)：兩輪直徑相差 1%，代表在相同的編碼器計數下，兩輪的實際位移相差 1%。設平均輪徑對應的位移為 s ，則：

$$\Delta s_R - \Delta s_L = 0.01 \times s \text{ (誤差)} \Rightarrow \text{累積轉角 } \Delta\theta_{\text{error}} = 0.01s/L$$

輪徑不等造成的虛假轉角。

直行 10 公尺 ($s = 10 \text{ m}$)：

$$\theta_{\text{error}} = 0.01 \times 10/0.30 = 0.333 \text{ rad} = 19.1^\circ$$

航向誤差高達 19 度！

這個誤差會使實際路徑彎曲。粗略估算橫向偏差（假設誤差均勻累積，取平均角度的一半）：

$$\text{橫向偏差} \approx 10 \times \sin(19.1^\circ/2) \approx 10 \times 0.166 = 1.66 \text{ m}$$

走 10 公尺，橫向偏差達 1.66 公尺。

這個結果令人震驚：僅僅 1% 的輪徑差異，就造成如此巨大的誤差。這正是 23.8.3 節 UMBmark 校正如此重要的原因——一個下午的校正工作，能把誤差降低數倍到十倍。

例題 23-3 阿克曼轉向與 Pure Pursuit

題目：某自駕小車採阿克曼構型，軸距 $L = 0.35 \text{ m}$ 、輪距 $W = 0.28 \text{ m}$ 、前輪最大轉角 $\delta_{\text{max}} = 30^\circ$ 。(a) 求最小轉彎半徑。(b) 當轉彎半徑為 2.0 m 時，求內外前輪的轉角。(c) 以 1.2 m/s 行駛、前輪轉 20° 時，求車體的角速度。(d) 若採用 Pure Pursuit 且前視距離 $L_d = 1.0 \text{ m}$ ，目標點與車頭方向夾角 $\alpha = 25^\circ$ ，求所需的轉彎半徑與前輪轉角。

解 (a)：由 23.4.2 節的公式：

$$R_{\text{min}} = L/\tan \delta_{\text{max}} = 0.35/\tan(30^\circ) = 0.35/0.5774 = 0.606 \text{ m}$$

最小轉彎半徑約 0.61 公尺。注意這遠大於差速驅動的 0——這是阿克曼構型的根本限制。

解 (b)：由阿克曼幾何 (23.4.1 節)：

$$\begin{aligned} \tan \delta_{\text{inner}} &= L/(R - W/2) = 0.35/(2.0 - 0.14) = 0.35/1.86 = 0.1882 \Rightarrow \delta_{\text{inner}} = 10.66^\circ \\ \tan \delta_{\text{outer}} &= L/(R + W/2) = 0.35/(2.0 + 0.14) = 0.35/2.14 = 0.1636 \Rightarrow \delta_{\text{outer}} = 9.29^\circ \end{aligned}$$

內外輪轉角相差約 1.4 度。

工程意義：若採用簡單的平行連桿（兩輪同角度），必有一輪打滑。轉彎半徑越小，內外輪

的角度差越大，打滑越嚴重——這正是阿克曼連桿機構存在的理由。

解 (c)：由單車模型：

$$\dot{\theta} = (v/L) \cdot \tan \delta = (1.2/0.35) \times \tan(20^\circ) = 3.429 \times 0.3640 = 1.248 \text{ rad/s}$$

車體角速度約 1.25 rad/s ($71.5^\circ/\text{s}$)。

注意：若車子停止 ($v = 0$)，則 $\dot{\theta} = 0$ ——無論前輪轉多少角度都無法改變朝向。這與差速驅動能原地旋轉形成關鍵差異。

解 (d)：由 Pure Pursuit 公式：

$$\kappa = 2 \cdot \sin(\alpha) / L_d = 2 \times \sin(25^\circ) / 1.0 = 2 \times 0.4226 = 0.8452 \text{ /m}$$

$$R = 1/\kappa = 1.183 \text{ m}$$

所需的轉彎半徑。

由於 $R = 1.183 \text{ m} > R_{\min} = 0.606 \text{ m}$ ，此指令可執行。所需的前輪轉角為：

$$\delta = \arctan(L/R) = \arctan(0.35/1.183) = \arctan(0.2959) = 16.5^\circ$$

前輪應轉 16.5 度，在 30 度的極限內。

若 α 更大會如何？例如 $\alpha = 60^\circ$ 時， $\kappa = 2\sin(60^\circ)/1.0 = 1.732$ ， $R = 0.577 \text{ m} < R_{\min}$ ——超出車輛能力。此時控制器必須：一、增大前視距離 L_d （看遠一點，使轉彎更平緩）；二、減速並倒車重新對準；或三、重新規劃路徑。這正是非完整約束對規劃的直接影響。

程式範例：運動學、里程計與路徑追隨

以下 Python 程式實作差速驅動與阿克曼的運動學、三種里程計積分方法的比較、輪徑誤差的影響模擬，以及 Pure Pursuit 路徑追隨。

```
import numpy as np

# ===== 差速驅動運動學 =====
class DiffDrive:
    def __init__(s, r=0.05, L=0.30, w_max=12.0):
        s.r, s.L, s.w_max = r, L, w_max

    def forward(s, wL, wR):
        """由輪角速度求車體速度 (23.2.2 節)"""
        vL, vR = s.r*wL, s.r*wR
        return (vR + vL)/2, (vR - vL)/s.L

    def inverse(s, v, w):
```

```

    """由車體速度求輪角速度 ( 23.2.3 節 ) """
    vR, vL = v + w*s.L/2, v - w*s.L/2
    return vL/s.r, vR/s.r

def limits(s):
    v_max = s.r*s.w_max
    w_max = 2*v_max/s.L
    return v_max, w_max

def max_omega_at(s, v):
    """給定線速度時的最大角速度 ( 23.2.4 節 ) """
    v_max, _ = s.limits()
    return max(0.0, 2*(v_max - abs(v))/s.L)

# ===== 例題 23-1 =====
print("=== 例題 23-1：差速驅動運動學 ===")
dd = DiffDrive()
v, w = dd.forward(3.0, 5.0)
print(f"(a)  $\omega_L=3.0$ ,  $\omega_R=5.0 \rightarrow v=\{v:.4f\}$  m/s,  $\omega=\{w:.4f\}$  rad/s, "
      f" $R=\{v/w:.3f\}$  m)")
wL, wR = dd.inverse(0.40, 0.5)
print(f"(b)  $v=0.40$ ,  $\omega=0.5 \rightarrow \omega_L=\{wL:.2f\}$ ,  $\omega_R=\{wR:.2f\}$  rad/s")
vm, wm = dd.limits()
print(f"(c) 最高直行  $v_{\max}=\{vm:.3f\}$  m/s, "
      f"最高原地旋轉  $\omega_{\max}=\{wm:.2f\}$  rad/s ( $\{np.rad2deg(wm):.0f\}^\circ/s$ )")
print(f"(d)  $v=0.5$  m/s 時最大  $\omega=\{dd.max\_omega\_at(0.5):.4f\}$  rad/s "
      f"(最小轉彎半徑  $\{0.5/dd.max\_omega\_at(0.5):.3f\}$  m)")

# ===== 三種里程計積分法 =====
print("\n=== 里程計積分方法的精度比較 ===")
def odom_step(x, y, th, ds, dth, method="midpoint"):
    if method == "straight":
        x += ds*np.cos(th); y += ds*np.sin(th)
    elif method == "midpoint":
        x += ds*np.cos(th + dth/2); y += ds*np.sin(th + dth/2)
    else:
        # exact arc
        if abs(dth) < 1e-9:
            x += ds*np.cos(th); y += ds*np.sin(th)
        else:
            R = ds/dth
            x += R*(np.sin(th + dth) - np.sin(th))
            y -= R*(np.cos(th + dth) - np.cos(th))
    return x, y, th + dth

# 測試：走一個半徑 1 m 的完整圓，比較回到起點的誤差
R_true, n_steps = 1.0, 200
ds = 2*np.pi*R_true/n_steps
dth = 2*np.pi/n_steps
print(f"測試：{n_steps} 步走完半徑 {R_true} m 的圓 ( 理論回到原點 ) ")
for m in ("straight", "midpoint", "arc"):
    x = y = th = 0.0
    for _ in range(n_steps):
        x, y, th = odom_step(x, y, th, ds, dth, m)

```

```

err = np.hypot(x, y)
print(f"  {m:10s}: 終點=({x:8.5f},{y:8.5f}) 誤差={err*1000:7.3f} mm")
print("  → 中點近似已足夠精確，圓弧積分最準但差異不大")

# ===== 例題 23-2：輪徑誤差的影響 =====
print("\n=== 例題 23-2：輪徑不等造成的漂移 ===")
def simulate_drift(dia_error_pct, distance=10.0, L=0.30, dt_ds=0.001):
    """模擬輪徑相差 dia_error_pct % 時直行 distance 公尺的漂移"""
    x = y = th = 0.0
    n = int(distance/dt_ds)
    for _ in range(n):
        # 指令直行，但右輪實際多走 error 比例
        dsL = dt_ds*(1 - dia_error_pct/200)
        dsR = dt_ds*(1 + dia_error_pct/200)
        ds, dth = (dsR + dsL)/2, (dsR - dsL)/L
        x, y, th = odom_step(x, y, th, ds, dth, "midpoint")
    return x, y, th

print(f"'輪徑差':>8}'{終點 X':>10}'{終點 Y':>10}'{航向誤差':>12}'{橫向偏差':>12}")
for e in (0.1, 0.5, 1.0, 2.0):
    x, y, th = simulate_drift(e)
    print(f"{e:7.1f}% {x:10.3f} {y:10.3f} {np.rad2deg(th):10.2f}°"
          f" {y:11.3f} m")
print("  → 僅 1% 的輪徑差，走 10 m 就偏離超過 1.5 m")

# ===== 阿克曼轉向 (例題 23-3) =====
print("\n=== 例題 23-3：阿克曼轉向 ===")
class Ackermann:
    def __init__(s, L=0.35, W=0.28, d_max=np.deg2rad(30)):
        s.L, s.W, s.d_max = L, W, d_max
    def R_min(s):
        return s.L/np.tan(s.d_max)
    def steer_angles(s, R):
        di = np.arctan(s.L/(R - s.W/2))
        do = np.arctan(s.L/(R + s.W/2))
        return np.rad2deg(di), np.rad2deg(do)
    def yaw_rate(s, v, delta):
        return (v/s.L)*np.tan(delta)

ack = Ackermann()
print(f"(a) 最小轉彎半徑 R_min = {ack.R_min():.3f} m")
di, do = ack.steer_angles(2.0)
print(f"(b) R=2.0 m → 內輪 {di:.2f}°, 外輪 {do:.2f}° (差 {di-do:.2f}°)")
yr = ack.yaw_rate(1.2, np.deg2rad(20))
print(f"(c) v=1.2 m/s, δ=20° → θ̇={yr:.3f} rad/s ({np.rad2deg(yr):.1f}°/s)")

# ===== Pure Pursuit =====
print("\n=== Pure Pursuit 路徑追隨 ===")
def pure_pursuit(alpha_deg, Ld):
    """回傳曲率與轉彎半徑 (23.9.3 節)"""
    kappa = 2*np.sin(np.deg2rad(alpha_deg))/Ld
    R = np.inf if abs(kappa) < 1e-9 else 1/kappa
    return kappa, R

```

```

print(f"{'α':>6}{'L_d':>7}{'曲率 κ':>10}{'半徑 R':>10}{'前輪 δ':>10}{'可行?':>8}")
for alpha, Ld in ((25, 1.0), (25, 2.0), (60, 1.0), (60, 2.0), (10, 1.0)):
    k, R = pure_pursuit(alpha, Ld)
    delta = np.rad2deg(np.arctan(ack.L/R)) if np.isfinite(R) else 0.0
    ok = R >= ack.R_min()
    print(f"{alpha:5d}°{Ld:7.1f}{k:10.4f}{R:10.3f}"
          f"{'delta:9.2f'}{'✓' if ok else 'X 超出能力':>10}")
print(" → α 過大時需增大 L_d (看遠一點) 或減速重新對準")

# ===== Pure Pursuit 完整追隨模擬 =====
print("\n=== Pure Pursuit 追隨圓形路徑 (比較不同 L_d) ===")
def follow_path(Ld, v=0.5, T=20.0, dt=0.01, R_path=2.0):
    """追隨一個半徑 R_path 的圓形路徑，回傳平均橫向誤差"""
    x, y, th = R_path, -0.3, 0.0          # 起點略偏離路徑
    errs = []
    for _ in range(int(T/dt)):
        # 找路徑上距離車子 Ld 的目標點 (圓形路徑可解析求解)
        ang = np.arctan2(y, x)
        # 沿圓弧前進 Ld 對應的角度
        d_ang = Ld/R_path
        tx, ty = R_path*np.cos(ang + d_ang), R_path*np.sin(ang + d_ang)
        # 轉到車體座標
        dx, dy = tx - x, ty - y
        alpha = np.arctan2(dy, dx) - th
        kappa = 2*np.sin(alpha)/Ld
        w = v*kappa
        # 更新
        ds, dth = v*dt, w*dt
        x, y, th = odom_step(x, y, th, ds, dth, "midpoint")
        errs.append(abs(np.hypot(x, y) - R_path))
    return np.mean(errs[len(errs)//2:]), np.max(errs[len(errs)//2:])

print(f"{'L_d':>7}{'平均誤差':>14}{'最大誤差':>14} 行為")
for Ld in (0.2, 0.5, 1.0, 2.0, 4.0):
    mean_e, max_e = follow_path(Ld)
    behav = ("震盪" if Ld < 0.3 else "良好" if Ld <= 1.5
             else "切內角明顯")
    print(f"{Ld:7.1f}{mean_e*1000:12.1f} mm{max_e*1000:12.1f} mm {behav}")
print(" → L_d 太小則震盪、太大則內切；本例最佳約 0.5~1.0 m")

```

請試著修改：(一) 把輪徑誤差設為 0.1% (良好校正後的水準)，觀察 10 公尺的偏差降到多少。(二) 在 Pure Pursuit 中加入速度自適應的前視距離 $L_d = 0.5v + 0.3$ ，比較固定 L_d 的效果。(三) 把路徑改為一個有直線與急彎的組合，觀察哪一段的追隨誤差最大。

實作：里程計校正與路徑追隨

本章實作以實體移動機器人為主（可用附錄 F 的循跡車或任何差速驅動平台，需具備編碼器）。

1. **參數量測**：用游標卡尺量測輪徑（多量幾個方向取平均）、用鋼尺量測輪距（兩輪中心距）。記錄編碼器規格與減速比。
2. **里程計實作**：撰寫里程計程式，以中點近似積分。以固定週期（建議 20 ~ 50 ms）更新並輸出位姿。
3. **直線測試**：在地面拉一條直線，讓機器人指令直行 5 公尺。量測終點的實際位置與里程計輸出的差異，特別記錄**橫向偏差**與**航向偏差**。
4. **旋轉測試**：讓機器人原地旋轉 360 度（依里程計判定），量測實際的角度偏差。重複順逆時針各五次。
5. **UMBmark 校正**：依 23.8.3 節的流程，在地面標出 2×2 公尺的正方形，順逆時針各走五圈五次，記錄終點偏差。計算修正係數並套用。
6. **校正效果驗證**：重複步驟 3 與 4，量化校正前後的改善倍數。
7. **打滑實驗**：在光滑地面（如磁磚加水）與粗糙地面（如地毯）各重複直線測試，比較誤差的差異。這能直觀展示隨機誤差的影響。
8. **Pure Pursuit 實作**：規劃一條含直線與彎道的路徑（可用地面膠帶標示），實作 Pure Pursuit 控制器。
9. **前視距離調參**：測試 $L_d = 0.2$ 、 0.5 、 1.0 、 2.0 倍車長，記錄追隨誤差與行為（是否震盪、是否切內角）。找出最佳值。
10. **（進階）速度自適應**：實作 $L_d = k \cdot v + L_{\min}$ ，測試在不同速度下的表現，驗證是否優於固定 L_d 。

表 23-7 里程計校正記錄表（前兩列為範例）

測試項目	校正前	校正後	改善倍數	備註
直行 5 m 橫向偏差	18.3 cm	2.1 cm	8.7×	主要為輪徑不等
原地旋轉 360° 角度誤差	12.5°	1.8°	6.9×	主要為輪距誤差
（自行填寫其餘測試）				

國中小也看得懂：為什麼停車要來回好幾次

你有沒有看過爸媽路邊停車？車位明明就在旁邊，為什麼不能直接**橫著滑進去**，而要前進、後退、前進、後退好幾次？

答案很簡單：**輪子只能往前後滾，不能往旁邊滑**。這聽起來理所當然，但它帶來一個很有趣的結果——

當下不能做的事，繞個路就能做到。車子沒辦法直接橫移，可是只要「前進轉一點、後退轉一點」重複幾次，它就慢慢挪進車位了。你想去的地方**都去得了**，只是不能走直線過去。

機器人也一樣。掃地機器人只有兩個輪子，它不能橫著走，但它可以先原地轉個方向再往前——最後還是能走遍整個房間。

那有沒有能橫著走的輪子呢？有！叫做「麥克納姆輪」，輪子表面裝了很多斜斜的小滾輪，可以往任何方向移動，非常酷。但它有三個缺點：**容易卡髒東西、載不了重物、而且走起來不太準**。所以大部分的機器人還是用普通的輪子，寧可多轉幾次彎。

最後告訴你一件很重要的事：機器人怎麼知道自己走了多遠？它會**數輪子轉了幾圈**。但這個方法有個大問題——如果輪子打滑了，或者兩個輪子大小差了一點點，它就會算錯，而且**越走錯越多**。

有多嚴重呢？如果兩個輪子的大小只差了百分之一（大概比一根頭髮還細的差距），機器人走 10 公尺之後，會歪掉超過 1 公尺半！所以工程師必須很仔細地「校正」機器人，就像你考試前要對答案一樣。

叫獸提醒與常見錯誤

- 誤區一：以為非完整約束代表「到不了」。它只限制瞬時的運動方向，所有位形仍然可達——這與欠驅動系統有本質差異。
- 誤區二：混淆非完整與欠驅動。前者的挑戰在規劃、後者的挑戰在控制；差速車是非完整但完全可達，倒立擺則是真的有到不了的狀態。
- 誤區三：忽略速度與角速度的耦合限制。高速時可用的轉向能力會下降，超出時應等比例縮小而非直接截斷。

- 誤區四：以為阿克曼車能原地旋轉。 θ 正比於 v ，車子不動就無法改變朝向——這是與差速驅動最關鍵的差異。
- 誤區五：里程計用直線近似而非中點近似。多一個除以 2 的成本極低，精度卻顯著提升。
- 誤區六：忽略輪徑不等的影響。僅 1% 的差異，走 10 公尺就能造成 19 度的航向誤差與 1.6 公尺的偏差。
- 誤區七：跳過 UMBmark 校正。這是投報率最高的工程程序之一——一個下午能把系統誤差降低數倍到十倍。
- 誤區八：以為里程計的位置誤差主要來自距離。實際上主要來自角度——微小的航向誤差會隨距離線性放大。
- 誤區九：期待里程計能單獨長期使用。漂移是積分方法的宿命，必須與不漂移的外部感測融合（第 24 章）。
- 誤區十：為了全向能力而選麥克納姆輪，卻忽略其代價。里程計差、承載低、怕髒污，多數場合用規劃補足運動限制更划算。
- 誤區十一：Pure Pursuit 的前視距離設為固定值。應隨速度調整（高速看遠、低速看近），這才符合實際的駕駛行為。
- 誤區十二：忽略動力學的限制。急轉彎時 v^2/R 可能超過摩擦極限而側滑，此時運動學模型完全失效。

課堂討論

1. 開頭故事中，全班花二十分鐘也無法讓差速車「正右方平移」。請用瞬時旋轉中心的概念解釋為何這是不可能的，並說明為何「來回打方向盤」卻能達成同樣的效果。
2. 非完整約束「瞬時受限但全域自由」這個性質，對路徑規劃（第 26 章）會帶來什麼影響？請討論：規劃器需要額外考慮什麼？
3. 例題 23-2 顯示 1% 的輪徑差就造成 1.6 公尺的偏差。請討論還有哪些「微小的物理誤差被放大成巨大後果」的例子（可回顧第 9 章的運動學標定與第 12 章的奇異點）。
4. 麥克納姆輪解除了非完整約束，卻付出里程計、承載與可靠度的代價。請討論：在什麼情況下這個交易是划算的？並比較這與第 21 章「用機構解決問題 vs 用演算法解決問題」的思維。

5. Pure Pursuit 只有一個調參（前視距離），卻是工業上最常用的路徑追隨演算法。請討論「參數少」在工程實務上的價值，並與第 16 章 PID 調參的困難做對比。

章末摘要

- 輪子分四種：標準固定輪、可轉向輪（皆施加不可側滑約束）、腳輪（不施加約束）、全向輪（解除約束）。
- 瞬時旋轉中心（ICR）是分析各種構型的統一工具：每個標準輪的軸線都必須通過 ICR。
- 差速驅動的正向運動學： $v = (v_R + v_L)/2$ 、 $\omega = (v_R - v_L)/L$ ；逆解永遠存在且唯一。
- 差速驅動的速度限制： $|v| + |\omega|L/2 \leq v_{\max}$ ，因此高速時可用的轉向能力會下降。
- 完整約束限制位形（可積分、減少維度）；非完整約束限制速度方向（不可積分、不減少可達位形）。
- 非完整系統「瞬時受限但全域自由」：可控自由度 $2 < \text{位形維度 } 3$ ，但可達自由度仍為 3 。
- 李括號解釋了為何運動的組合能產生新方向；Chow-Rashevskii 定理給出完全可控的條件。
- 非完整與欠驅動不同：前者所有位形都可達（挑戰在規劃），後者有些狀態根本到不了（挑戰在控制）。
- 阿克曼轉向的單車模型： $\theta = (v/L)\tan \delta$ ；角速度與線速度耦合，故不能原地旋轉，且有最小轉彎半徑 $L/\tan \delta_{\max}$ 。
- 全向輪解除了非完整約束，代價是里程計差、承載低、效率低、對地面敏感。
- 履帶車的滑移轉向必然伴隨大量滑動，里程計極不準但越野能力強。
- 里程計積分應使用中點近似（比直線近似精確、比圓弧積分簡單且無數值問題）。
- 里程計必定漂移，這是積分方法的宿命——沒有外部參照就無法修正誤差。
- 位置誤差主要來自航向誤差：位置誤差 $\approx \text{行進距離} \times \text{航向誤差}$ 。
- 誤差分系統誤差（輪徑不等、輪距不準，可用 UMBmark 校正）與隨機誤差（打滑、地面不平，只能統計描述）。
- Brockett 條件：非完整系統無法用連續非時變的狀態回授穩定到定點；實務上改用路徑追隨。
- Pure Pursuit 只需一個公式 $\kappa = 2\sin(\alpha)/L_d$ 與一個調參（前視距離），是最實用的路徑追隨演算法。

觀念題與計算題

觀念題

1. 列舉四種輪型及其約束特性，並說明腳輪為何不施加約束。
2. 說明瞬時旋轉中心的定義，以及「標準輪的軸線必須通過 ICR」這個性質為何成立。
3. 推導差速驅動的正向與逆向運動學，並說明三種典型情形（直行、原地旋轉、繞單輪）。
4. 為何差速驅動的速度與角速度不能任意組合？超出限制時應如何處理？
5. 區分完整約束與非完整約束，並說明「可積分性」為何是關鍵。
6. 解釋「瞬時受限但全域自由」的意義，並以路邊停車為例說明。
7. 說明非完整約束與欠驅動的差異，並指出兩者的挑戰各在何處。
8. 寫出阿克曼轉向的單車模型，並說明它為何不能原地旋轉。
9. 為何阿克曼轉向需要內外前輪不同角度？若使用簡單的平行連桿會有什麼後果？
10. 說明全向輪的原理與三大代價，並討論其適用場合。
11. 比較里程計的三種積分方法，並說明實務上為何推薦中點近似。
12. 解釋里程計為何必定漂移，並說明為何位置誤差主要來自航向誤差。
13. 區分里程計的系統誤差與隨機誤差，並各列舉三個來源。
14. 說明 UMBmark 校正法的原理——為何順逆時針兩組實驗能分離兩種誤差源？
15. 敘述 Brockett 條件的內容，並說明實務上的三種因應方式。
16. 說明 Pure Pursuit 的原理與前視距離的影響，並解釋為何應隨速度調整。

計算題

1. 某差速驅動機器人輪半徑 $r = 0.075 \text{ m}$ 、輪距 $L = 0.42 \text{ m}$ 、馬達最高角速度 15 rad/s 。
 - (a) 若左輪 4.0 rad/s 、右輪 7.0 rad/s ，求 v 、 ω 與轉彎半徑。
 - (b) 若要以 0.6 m/s 前進並以 0.8 rad/s 右轉，求兩輪角速度並判斷是否超出馬達能力。
 - (c) 求最高直行速度與最高原地旋轉角速度。
 - (d) 求以 0.8 m/s 行進時的最小轉彎半徑。
2. 某機器人編碼器 1000 PPR（正交四倍頻）、裝於馬達端、減速比 20、輪半徑 0.06 m 、輪距 0.35 m 。
 - (a) 求每個計數對應的輪面位移。
 - (b) 某週期左輪 +200 計數、右輪 +260 計數，求 Δs 與 $\Delta \theta$ 。
 - (c) 若初始位姿為 $(1.0, 2.0, 30^\circ)$ ，用中點近似更新。
 - (d) 若兩輪直徑相

差 0.8%，估算直行 20 公尺後的航向誤差。

3. 某阿克曼車軸距 $L = 2.7 \text{ m}$ 、輪距 $W = 1.6 \text{ m}$ 、前輪最大轉角 35° 。(a) 求最小轉彎半徑。(b) 轉彎半徑 8 m 時，求內外前輪轉角及其差值。(c) 以 15 m/s 行駛、前輪轉 5° 時，求車體角速度與轉彎半徑。(d) 若路面摩擦係數 $\mu = 0.8$ ，判斷 (c) 的情形是否會側滑。
4. 某機器人採用 Pure Pursuit，前視距離 $L_d = 1.5 \text{ m}$ 。(a) 當目標點夾角 $\alpha = 30^\circ$ 時，求曲率與轉彎半徑。(b) 若車輛為差速驅動且以 1.0 m/s 行進，求所需的角速度。(c) 若改為阿克曼構型 ($L = 0.5 \text{ m}$)，求所需的前輪轉角。(d) 若 α 增大到 75° ，重新計算 (a) 並討論可能的問題與因應方式。

動手做與專題延伸

本章實作：移動機器人運動學模組

建立移動機器人的程式庫，這將是第 24 ~ 26 章的共同基礎。

- 階段一：實作差速驅動的正逆運動學與速度限制檢查，用例題 23-1 驗證。
- 階段二：實作三種里程計積分方法，並用「走完整圓應回到原點」的測試比較精度。
- 階段三：實作誤差模擬——可設定輪徑誤差、輪距誤差與隨機打滑，觀察軌跡如何偏離。
- 階段四：實作阿克曼運動學與 Pure Pursuit 控制器，並在模擬中追隨各種路徑。
- 階段五：實作 UMBmark 校正的計算部分（由順逆時針的偏差資料反推修正係數）。

專題延伸

- **非完整約束的視覺化**：撰寫一個互動程式，讓使用者用滑鼠指定目標位姿，程式顯示差速車「無法直接過去」但「能透過一連串動作到達」。加入一個「可達性熱力圖」——顯示在固定時間內能到達的位形範圍，你會看到一個特殊的形狀（並非圓形），這直觀展示了非完整約束的效果。
- **里程計誤差的蒙地卡羅模擬**：對同一條路徑執行 1000 次模擬，每次加入隨機的輪徑誤差與打滑，記錄終點位置的分布。繪出散布圖與誤差橢圓——你會看到誤差主要沿著垂直於行進方向擴散（因為航向誤差的影響最大）。這張圖正是第 24 章 Kalman 濾波器中「協方差矩陣」的直觀來源。
- **進階：三種路徑追隨演算法的比較**。實作 Pure Pursuit、Stanley 控制器與一個簡單的 MPC，在同一組測試路徑（含直線、大彎、急彎、S 型）上比較。記錄橫向誤差、控制平順

度與計算時間。討論在什麼情況下該用哪一種——你會發現簡單的 Pure Pursuit 在多數情況下已經夠好，而這正是它成為工業標準的原因。

參考文獻與延伸閱讀

- Siegwart, R., Nourbakhsh, I. R. and Scaramuzza, D., Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press (移動機器人的標準教材，第 3 章為運動學) 。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer (第 11 章：移動機器人，含非完整約束的嚴謹處理) 。
- LaValle, S. M., Planning Algorithms, Cambridge University Press (線上公開) (第 13 章：微分約束下的規劃) 。
- Brockett, R. W., "Asymptotic Stability and Feedback Stabilization," in Differential Geometric Control Theory, 1983 (Brockett 條件的原始論文) 。
- Borenstein, J. and Feng, L., "Measurement and Correction of Systematic Odometry Errors in Mobile Robots," IEEE Trans. Robotics and Automation, 1996 (UMBmark 校正法) 。
- Coulter, R. C., "Implementation of the Pure Pursuit Path Tracking Algorithm," CMU Technical Report, 1992 (Pure Pursuit 的經典技術報告) 。
- Snider, J. M., "Automatic Steering Methods for Autonomous Automobile Path Tracking," CMU Technical Report, 2009 (多種路徑追隨演算法的完整比較) 。
- Campion, G., Bastin, G. and D'Andrea-Novell, B., "Structural Properties and Classification of Kinematic and Dynamic Models of Wheeled Mobile Robots," IEEE Trans. Robotics, 1996 (輪型機器人的系統性分類) 。
- Thrun, S., Burgard, W. and Fox, D., Probabilistic Robotics, MIT Press (第 5 章：運動模型與其機率描述，銜接第 24 章) 。
- ROS 2 導航堆疊 (Nav2) 官方文件：里程計、路徑追隨控制器與移動平台整合的實務參考。

第 24 章

狀態估測與感測器融合

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩條位置估計曲線的對照：上圖是「直接覆蓋」的作法——曲線平順前進後每隔一段就猛地跳一下，轉彎處呈鋸齒狀，旁註「扔掉了資訊」；下圖是 Kalman 融合——曲線平滑且貼近真值，旁註「依可信度加權」。右半為誤差橢圓隨時間的演變序列：機器人沿路徑前進，橢圓逐步脹大並沿行進方向拉長（無量測時），經過地標時瞬間收縮成小圓。畫面下緣以兩個箭頭標註貝氏濾波的節奏——「預測：呼氣，不確定性增加」與「更新：吸氣，不確定性減少」。

叫獸開講 「兩個定位系統都不完全對，該相信哪一個？」

小妤的自製 AGV 有兩套定位系統。里程計每 20 毫秒更新一次，短時間內準得驚人；天花板上的視覺標記系統每秒只更新五次，但永遠不會累積誤差。

「兩個都有了，」她說，「所以我寫了一個切換邏輯：平常用里程計，每次看到標記就把位置覆蓋過去。」

叫獸請她示範。AGV 沿著走廊前進——螢幕上的位置平順地移動，然後每隔零點二秒，就猛地跳一下。跳動幅度有時三公分、有時七公分。到了轉彎處，車子甚至因為位置突變而急打方向盤，畫出一條鋸齒狀的軌跡。

「你看到問題了嗎？」叫獸問。小妤點頭：「跳來跳去。」

「更深一層的問題是——你在扔掉資訊。」叫獸在白板上畫了兩條線，「里程計說我在 A 點，視覺說我在 B 點，兩者差七公分。你選了 B，等於宣告『里程計完全錯了』。但里程計真的完全錯了嗎？」

「.....應該沒有那麼錯。」

「那視覺就完全對嗎？相機有雜訊、標記可能貼歪、光線會影響偵測。兩個都不完全對，也都不完全錯。」叫獸在 A 與 B 之間畫了一個點，「所以正確答案通常在中間某處。問題是——中間的哪裡？」

小翰試著說：「取平均？」

「接近了，但還不夠。」叫獸拿起兩支粗細不同的筆，「如果里程計剛校正過、只走了兩公尺，它的誤差可能只有一公分；而視覺標記在光線很暗的角落，誤差可能有十公分。這時候你還要各取一半嗎？」

教室安靜了幾秒。小妤說：「應該.....相信比較準的那個多一點。」

「這就是全部的祕密。」叫獸在白板上寫下一行字：依可信度加權平均。「而『可信度』要怎麼量化？答案是變異數——不確定性越小，權重越大。這個看似簡單的想法，在 1960 年被一位工程師寫成了嚴謹的數學，讓阿波羅太空船能夠導航到月球。它叫做 Kalman 濾波器。」

他停頓了一下，補上最後一句：「還有一件事。第 6 章我們用過一行公式做互補濾波——加

速度計配陀螺儀。當時我說『這是一個經驗法則』。今天你們會發現：那條式子其實是 Kalman 濾波器的簡化版，只是當年沒有告訴你們它從哪裡來。」

本章為什麼重要

第 23 章證明了一件殘酷的事：**里程計必定漂移**。這不是工程品質的問題，而是積分方法的宿命。但機器人不能因此就無法定位——它必須結合多種感測器，各取所長。

這個「結合」看似簡單，實際上是一個深刻的問題。不同感測器的更新率不同、精度不同、失效模式不同，而且**每一個讀值都帶著雜訊**。單純的切換或平均都會浪費資訊甚至造成危險（如開頭故事的跳動）。

本章要建立的，是一套**機率的思考框架**。核心的觀念轉換是：不再問「機器人在哪裡」，而是問「**機器人在各個位置的可能性各有多大**」。這個轉換使我們能夠：量化不確定性、依可信度融合多個來源、並在資訊不足時誠實地表達「我不確定」。

這套框架的具體工具是 Kalman 濾波器與粒子濾波器。它們不只用於定位——第 25 章的 SLAM、第 27 章的無人機姿態估計、第 28 章的腿式機器人狀態估測，全都建立在本章之上。可以說，**本章是第四篇的數學核心**。

學習目標

- 能說明機率式機器人學的核心觀念轉換及其必要性。
- 能定義信念分布並說明其與點估計的差異。
- 能敘述貝氏濾波的兩個步驟並說明各自的作用。
- 能說明高斯分布的性質及其為何適合表示不確定性。
- 能解釋協方差矩陣的幾何意義並繪製誤差橢圓。
- 能推導一維的資訊融合公式並說明權重的來源。
- 能敘述 Kalman 濾波器的五條方程式並說明各項的物理意義。
- 能說明 Kalman 增益的意義及其如何自動平衡預測與量測。
- 能說明擴展 Kalman 濾波器的線性化作法與其限制。
- 能說明粒子濾波器的原理並比較其與 Kalman 濾波器的適用場合。

- 能說明互補濾波與 Kalman 濾波器的關係。
- 能為特定應用選擇合適的估測方法並設定雜訊參數。

先備知識

第 23 章的運動模型（作為預測方程）、第 6 章的感測器特性與雜訊、第 22 章的視覺定位、第 18.2 節的狀態空間表示法。

數學方面需要：基本機率（條件機率、貝氏定理）、常態分布、矩陣運算。**不需要**測度論或嚴格的統計理論——本章會以幾何直覺與物理意義為主軸，公式的推導只在必要時進行。若對機率生疏，可先閱讀附錄 D.3。

核心名詞中英對照

表 24-1 本章核心名詞

中文	英文	說明
狀態估測	State Estimation	由感測資料推斷系統內部狀態
信念	Belief	對狀態的機率分布表示
貝氏濾波	Bayes Filter	遞迴更新信念的通用框架
預測步驟	Prediction Step	依運動模型推進信念
更新步驟	Update / Correction Step	依量測修正信念
先驗	Prior	取得量測前的信念
後驗	Posterior	取得量測後的信念
似然	Likelihood	在給定狀態下觀測到該量測的機率
高斯分布	Gaussian Distribution	以均值與變異數描述的常態分布
協方差矩陣	Covariance Matrix	描述多維不確定性與相關性的矩陣
誤差橢圓	Error Ellipse	協方差矩陣的幾何表示
Kalman 濾波器	Kalman Filter, KF	線性高斯系統的最佳估測器
Kalman 增益	Kalman Gain	決定信任預測或量測的權重矩陣
擴展 Kalman 濾波器	Extended Kalman Filter, EKF	以線性化處理非線性系統
無跡 Kalman 濾波器	Unscented Kalman Filter, UKF	以取樣點傳遞分布的方法
粒子濾波器	Particle Filter, PF	以大量樣本表示任意分布
重取樣	Resampling	依權重重重新抽取粒子的程序

中文	英文	說明
粒子退化	Particle Depletion	粒子多樣性喪失的現象
互補濾波	Complementary Filter	以高低通濾波融合兩種感測的簡化方法

24.1 機率的觀念轉換

24.1.1 從「在哪裡」到「可能在哪裡」

第二篇與第三篇的世界很乾淨：給定關節角度，末端位置是唯一確定的。但一旦進入感知的領域，這個確定性就消失了。

機率式機器人學 (probabilistic robotics) 的核心主張是：不要試圖回答「機器人在哪裡」這個問題，因為它沒有確定的答案；應該回答「機器人在各個位置的可能性各有多大」。

這個轉換帶來三個具體的好處：

- **能量化不確定性**：系統不只知道估計值，還知道這個估計有多可靠。這對決策極為重要——不確定時應該減速、應該主動去尋找地標。
- **能正確融合多個來源**：有了不確定性的量化，就能依可信度加權（開頭故事的核心）。
- **能誠實面對失敗**：當所有感測都不可靠時，機率分布會變得很寬——系統知道自己「迷路了」，而不是給出一個看似精確卻錯誤的數字。

24.1.2 信念分布

機器人對自身狀態的認知，用一個機率分布表示，稱為**信念** (belief)：

$$\text{bel}(x_t) = p(x_t \mid z_{1:t} u_{1:t})$$

在給定所有歷史量測 z 與所有控制指令 u 的條件下，狀態 x 的機率分布。

信念分布的形狀本身就承載了豐富的資訊：

表 24-2 信念分布的形狀與其意義

分布形狀	意義	典型情境
窄而高的單峰	位置確定	剛看到明確的地標
寬而矮的單峰	大致知道但不精確	長時間只靠里程計
多個峰	有數個可能位置	走廊上的對稱環境

分布形狀	意義	典型情境
接近均勻分布	完全不知道在哪裡	剛開機、被綁架問題
沿某方向拉長	某方向確定、某方向不確定	沿長廊行進（縱向不確定）

最後一列特別值得注意——它說明了為何需要**協方差矩陣**而非單一的誤差數字：不確定性在不同方向上可能截然不同。

24.1.3 三個經典問題

定位問題依難度分為三個層次，這個分類在文獻中極為常見：

位置追蹤 (Position Tracking)：初始位置已知，只需持續追蹤。不確定性始終保持在局部，是最簡單的情形。Kalman 濾波器適用。

全域定位 (Global Localization)：初始位置未知，機器人必須從頭找出自己在地圖中的哪裡。初期的信念可能遍布整個地圖，之後逐漸收斂。這需要能表示多峰分布的方法（粒子濾波器）。

綁架機器人問題 (Kidnapped Robot Problem)：機器人在運行中被「瞬間搬到」另一個地方（實務上對應打滑嚴重、被人搬動、或定位失敗後的復原）。這是最困難的——系統必須先察覺自己錯了，再重新全域定位。

三者的難度遞增，而所需的方法也不同。這是後續選擇 Kalman 或粒子濾波的重要依據。

24.2 貝氏濾波

24.2.1 兩個步驟的循環

所有的遞迴估測方法，都建立在同一個框架上：**貝氏濾波**。它由兩個交替進行的步驟構成。

步驟一：預測 (Prediction)。依運動模型，把上一時刻的信念往前推進：

$$\text{bel}^-(x_t) = \int p(x_t | x_{t-1}, u_t) \cdot \text{bel}(x_{t-1}) dx_{t-1}$$

預測步驟。積分是對所有可能的前一狀態求和。 $p(x_t | x_{t-1}, u_t)$ 稱為**運動模型**，正是第 23 章建立的內容加上雜訊。

這一步的關鍵性質是：**不確定性必然增加**。因為運動本身有誤差（打滑、輪徑不準），每走一步就多累積一分不確定。這正是第 23.8 節「里程計必定漂移」在機率語言中的表述。

步驟二：更新 (Update / Correction) 。取得量測後，依貝氏定理修正信念：

$$\text{bel}(x_t) = \eta \cdot p(z_t | x_t) \cdot \text{bel}^-(x_t)$$

更新步驟。 $p(z_t | x_t)$ 稱為量測模型 (似然)， η 為正規化常數使總機率為 1。

這一步的關鍵性質是：不確定性必然減少 (或至少不增加)。因為量測提供了新的資訊。

◎ 一呼一吸的節奏

貝氏濾波的兩個步驟，可以想像成呼吸：預測是「呼氣」——信念分布往外擴散，不確定性增加；更新是「吸氣」——信念被量測收攏，不確定性減少。整個估測過程就是這樣一呼一吸地循環。若長時間沒有量測 (例如走在沒有地標的長廊)，分布會持續擴散；一旦看到地標，又會瞬間收窄。理解這個節奏，就理解了所有濾波器的本質。

24.2.2 馬可夫假設

上述公式隱含一個重要的假設：當前狀態包含了所有必要的資訊，過去的歷史可以丟棄。這稱為馬可夫假設 (Markov assumption)。

$$p(x_t | x_{0:t-1}, z_{1:t-1}, u_{1:t}) = p(x_t | x_{t-1}, u_t)$$

馬可夫性。只需要前一個狀態與當前的控制指令。

這個假設極為關鍵——它使遞迴計算成為可能。若必須記住所有歷史，計算量會隨時間無限成長；有了馬可夫性，我們只需維護一個當前的信念分布即可。

這與第 18.3 節 Bellman 最佳性原理的結構完全相同——那裡說「剩餘的最佳決策只取決於當前狀態」，這裡說「當前的信念只取決於前一個信念」。兩者都是把「歷史」壓縮進「狀態」的思想。

實務上馬可夫假設並非總是成立 (例如未建模的系統性偏差會使誤差相關)，但它是一個極為有用的近似。

24.3 高斯分布與協方差

24.3.1 為何選擇高斯

貝氏濾波是一個通用框架，但要實作它，必須決定用什麼方式表示信念分布。最常用的選擇是高斯分布 (常態分布)，原因有四：

- **只需兩個參數**：均值 μ (最可能的位置) 與變異數 σ^2 (不確定性的尺寸)。這使儲存與計算都極為高效。
- **中央極限定理**：許多獨立誤差源的疊加，趨近於高斯分布——這在物理上經常成立。
- **線性運算下封閉**：高斯分布經過線性變換後仍是高斯分布。這是 Kalman 濾波器能夠遞迴的數學基礎。
- **兩個高斯相乘仍是高斯**：這使貝氏更新步驟有封閉解。

第三與第四點特別重要——它們保證了**信念永遠可以用兩個參數表示**，不會隨時間變得越來越複雜。

24.3.2 協方差矩陣的幾何意義

多維情形下，不確定性由**協方差矩陣** Σ 描述。以二維 (x, y) 為例：

$$\Sigma = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix}$$

協方差矩陣。對角元素是各方向的變異數，非對角元素是**相關性**。

它的幾何意義極為直觀：**協方差矩陣對應一個橢圓**（三維時為橢球）。

- **橢圓的主軸方向**： Σ 的特徵向量。這是不確定性的主要方向。
- **主軸的長度**：正比於特徵值的平方根。
- **非對角元素的意義**：若 $\sigma_{xy} \neq 0$ ，代表 x 與 y 的誤差是**相關**的——橢圓會傾斜。

這個橢圓稱為**誤差橢圓** (error ellipse)。習慣上會畫「 1σ 橢圓」(涵蓋約 39% 的機率，二維情形) 或「 3σ 橢圓」(涵蓋約 99%)。

第 23 章專題延伸提過的蒙地卡羅模擬，其散布圖的形狀正是這個橢圓——**沿垂直於行進方向拉長**，因為航向誤差的影響最大。這正是相關性的具體體現。

圖 24-1 協方差矩陣與誤差橢圓

建議配圖：四格對照圖。第一格： $\sigma_x = \sigma_y$ 、 $\sigma_{xy} = 0$ ——正圓，代表各方向不確定性相同。第二格： $\sigma_x > \sigma_y$ 、 $\sigma_{xy} = 0$ ——沿 X 軸拉長的橢圓，兩軸與座標軸對齊。第三格： $\sigma_{xy} > 0$ ——傾斜的橢圓，標註「 x 與 y 的誤差正相關」。第四格：機器人沿長廊行進的實際情形——橢圓沿行進方向拉長（縱向位置不確定），並標註特徵向量方向與特徵值的平方根。每格下方標註對應的協方差矩陣數值。

24.3.3 不確定性的傳播

當狀態經過一個變換時，其不確定性如何變化？對線性變換 $y = Ax$ ：

$$\mu_y = A \mu_x \quad \Sigma_y = A \Sigma_x A^T$$

不確定性的線性傳播。這是 Kalman 濾波器預測步驟的核心公式。

注意 $A \Sigma A^T$ 這個形式——它與第 11 章的 $J M^{-1} J^T$ 、第 17 章的 Λ 矩陣有相同的結構。這不是巧合：兩者都在做「把一個空間中的二次型，映射到另一個空間」。

對非線性變換 $y = g(x)$ ，則需要線性化（用雅可比矩陣 $G = \partial g / \partial x$ ）：

$$\Sigma_y \approx G \Sigma_x G^T$$

非線性情形的近似傳播。這是 24.6 節擴展 Kalman 濾波器的基礎，也是其誤差的來源。

24.4 一維的資訊融合

24.4.1 最簡單的情形

在進入完整的 Kalman 濾波器之前，先看最簡單的問題：兩個獨立的量測，如何融合？

設有兩個對同一個量的估計：估計一為 μ_1 ，變異數 σ_1^2 ；估計二為 μ_2 ，變異數 σ_2^2 。最佳的融合結果是：

$$\mu = (\sigma_2^2 \mu_1 + \sigma_1^2 \mu_2) / (\sigma_1^2 + \sigma_2^2)$$

融合後的均值。注意權重的分配——變異數小的（較可信的）得到較大的權重。

$$1/\sigma^2 = 1/\sigma_1^2 + 1/\sigma_2^2 \quad \text{即} \quad \sigma^2 = (\sigma_1^2 \sigma_2^2) / (\sigma_1^2 + \sigma_2^2)$$

融合後的變異數。倒數相加的形式極為優雅。

24.4.2 三個關鍵洞見

這兩條式子雖然簡單，卻包含了整個 Kalman 濾波器的精髓。

洞見一：權重與變異數成反比。這正是開頭故事的答案——不是各取一半，而是依可信度加權。若 $\sigma_1 = 1 \text{ cm}$ 、 $\sigma_2 = 10 \text{ cm}$ ，則估計一的權重是估計二的 100 倍（因為權重正比於 $1/\sigma^2$ ）。

洞見二：融合後必定更確定。由 $1/\sigma^2 = 1/\sigma_1^2 + 1/\sigma_2^2$ 可知， σ^2 必定小於 σ_1^2 與 σ_2^2 中的任何一個。即使加入一個很差的量測，結果也不會變糟——這是資訊融合最重要的性質。

洞見三：資訊是可加的。 $1/\sigma^2$ 這個量稱為**資訊量**（information），也稱**精確度**（precision）。上式說：融合就是把資訊量相加。這個觀點導出了 Kalman 濾波器的另一種等價形式——**資訊濾波器**（information filter）。

◎ 為何「差的量測也不會讓結果變糟」

這個性質常令初學者驚訝。直觀的理解是：一個變異數很大的量測，其權重接近於零——它幾乎不影響均值，但仍然提供了一點點資訊，使總變異數略微下降。這意味著「多加一個感測器」在數學上永遠是好事。當然，前提是誤差模型正確——若某個感測器有未建模的偏差 (bias) 而非純粹的隨機雜訊，它就會系統性地拉偏結果。這是實務上感測器融合最常見的失敗原因，24.9 節會詳述。

24.4.3 改寫成預測—修正的形式

把 24.4.1 節的公式重新整理，可以得到一個極有啟發性的形式。設 μ_1 為「預測值」、 μ_2 為「量測值」：

$$\mu = \mu_1 + K \cdot (\mu_2 - \mu_1), \quad \text{其中 } K = \sigma_1^2 / (\sigma_1^2 + \sigma_2^2)$$

預測—修正形式。K 就是 Kalman 增益的一維版本。

這個形式的解讀極為直觀：

- $(\mu_2 - \mu_1)$ 是「創新」(innovation)——量測與預測的差距，代表「新的資訊」。
- K 決定要相信多少。K = 0 代表完全不理會量測 (預測極準)；K = 1 代表完全採信量測 (預測極差)。
- K 自動調節：預測的不確定性 σ_1^2 越大，K 越大 (越相信量測)；量測雜訊 σ_2^2 越大，K 越小 (越相信預測)。

變異數的更新也可以寫成對應的形式：

$$\sigma^2 = (1 - K) \cdot \sigma_1^2$$

更新後的變異數。K 越大，不確定性下降越多。

這四條式子 (預測值、增益、修正、變異數更新) 就是 Kalman 濾波器的全部。接下來一節要做的，只是把它們推廣到多維並加上運動模型。

24.5 Kalman 濾波器

24.5.1 系統的假設

Kalman 濾波器適用於滿足三個條件的系統：線性的運動模型、線性的量測模型、高斯的雜訊。形式為：

$$\text{運動模型: } \mathbf{x}_t = \mathbf{A} \mathbf{x}_{t-1} + \mathbf{B} \mathbf{u}_t + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$$

$$\text{量測模型: } \mathbf{z}_t = \mathbf{H} \mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathbf{N}(\mathbf{0}, \mathbf{R})$$

A 為狀態轉移矩陣、 B 為控制輸入矩陣、 H 為量測矩陣。 Q 為過程雜訊協方差、 R 為量測雜訊協方差。

兩個雜訊矩陣的意義極為重要，且是實務上最需要調整的參數：

- Q (過程雜訊)：描述「運動模型有多不準」。輪子打滑、地面不平、模型簡化——這些都反映在 Q 中。 Q 越大，代表越不信任預測。
- R (量測雜訊)：描述「感測器有多不準」。這通常可由感測器規格或實驗量測得到。 R 越大，代表越不信任量測。

24.5.2 五條方程式

Kalman 濾波器由五條方程式構成，分為兩組。

預測步驟 (兩條)：

$$\hat{\mathbf{x}}_t^- = \mathbf{A} \hat{\mathbf{x}}_{t-1} + \mathbf{B} \mathbf{u}_t$$

①狀態預測。依運動模型推進估計值。

$$\mathbf{P}_t^- = \mathbf{A} \mathbf{P}_{t-1} \mathbf{A}^T + \mathbf{Q}$$

②協方差預測。注意 $\mathbf{A} \mathbf{P} \mathbf{A}^T$ 正是 24.3.3 節的不確定性傳播，再加上過程雜訊 Q ——不確定性必然增加。

更新步驟 (三條)：

$$\mathbf{K}_t = \mathbf{P}_t^- \mathbf{H}^T (\mathbf{H} \mathbf{P}_t^- \mathbf{H}^T + \mathbf{R})^{-1}$$

③Kalman 增益。這是整個演算法的核心，24.5.3 節詳述。

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_t^- + \mathbf{K}_t (\mathbf{z}_t - \mathbf{H} \hat{\mathbf{x}}_t^-)$$

④狀態更新。括號中的 $(\mathbf{z}_t - \mathbf{H} \hat{\mathbf{x}}_t^-)$ 就是創新——量測與預測的差距。

$$\mathbf{P}_t = (\mathbf{I} - \mathbf{K}_t \mathbf{H}) \mathbf{P}_t^-$$

⑤協方差更新。 K 越大，不確定性下降越多——不確定性必然減少。

對照 24.4.3 節的一維版本，可以看出完全相同的結構：預測 → 算增益 → 依創新修正 → 更新不確定性。多維版本只是把純量換成矩陣。

24.5.3 Kalman 增益的意義

增益公式 $\mathbf{K} = \mathbf{P}^- \mathbf{H}^T (\mathbf{H} \mathbf{P}^- \mathbf{H}^T + \mathbf{R})^{-1}$ 看似複雜，但其行為極為直觀。考慮一維的簡化情形 ($H = 1$)：

$$K = P^- / (P^- + R)$$

一維增益。分母是「預測不確定性 + 量測不確定性」，分子是「預測不確定性」。

三種極端情形立刻可讀出：

表 24-3 Kalman 增益的行為

情形	K 的值	行為	實務意義
$P^- \gg R$ (預測很不準)	$K \rightarrow 1$	幾乎完全採信量測	剛開機、長時間無量測後
$P^- \ll R$ (量測很不準)	$K \rightarrow 0$	幾乎忽略量測	感測器雜訊大、環境不佳
$P^- \approx R$ (兩者相當)	$K \approx 0.5$	各取一半	正常運作狀態
$R \rightarrow 0$ (量測完美)	$K \rightarrow 1$	完全採信量測	理想情形，實務不存在

關鍵洞見：K 是自動調節的。工程師不需要手動決定「該相信誰」——只要正確設定 Q 與 R，濾波器每一步都會依當下的不確定性自動計算出最佳的權重。這正是開頭故事中小玆所缺少的東西。

另一個重要性質：P 的演化與量測值 z 完全無關——由方程②③⑤可見，它們只含 A、H、Q、R 與 P 自身。這意味著 P 與 K 的序列可以離線預先計算，這在計算資源受限的嵌入式系統上極為有用（稱為穩態 Kalman 濾波器）。

24.5.4 為何說它是「最佳」的

Kalman 濾波器有一個強而有力的理論保證：

◎ Kalman 濾波器的最佳性

在「線性系統 + 高斯雜訊」的假設下，Kalman 濾波器是最小均方誤差（MMSE）意義下的最佳估測器——沒有任何其他方法能得到更小的估計誤差。此外，即使雜訊不是高斯的，它仍是所有線性估測器中最佳的。

這個保證與第 18 章 LQR 的地位相同——兩者都是「在特定假設下的最佳解」，且數學結構驚人地相似：LQR 解 Riccati 方程求控制增益 K，Kalman 濾波器解 Riccati 方程求估測增益 K。這正是第 19.2.2 節提到的對偶性——估計與控制是同一件事的兩面。

兩者結合稱為 LQG（Linear Quadratic Gaussian）：用 Kalman 濾波器估計狀態、再用 LQR 計算控制。這是經典控制理論的巔峰成果。

24.6 非線性的處理：EKF 與 UKF

24.6.1 機器人為何是非線性的

Kalman 濾波器要求線性系統，但機器人幾乎都不是。以第 23 章的差速驅動車為例：

$$\begin{aligned}x_t &= x_{t-1} + \Delta s \cdot \cos(\theta_{t-1} + \Delta\theta/2) \\y_t &= y_{t-1} + \Delta s \cdot \sin(\theta_{t-1} + \Delta\theta/2) \\ \theta_t &= \theta_{t-1} + \Delta\theta\end{aligned}$$

運動模型含三角函數，是非線性的。

量測模型同樣非線性。例如用光達量測到某地標的距離與角度：

$$\begin{aligned}r &= \sqrt{(x_L - x)^2 + (y_L - y)^2} \\ \phi &= \text{atan2}(y_L - y, x_L - x) - \theta\end{aligned}$$

距離與方位的量測模型，同樣非線性。

因此必須擴展 Kalman 濾波器以處理非線性。有兩條主要路線。

24.6.2 擴展 Kalman 濾波器 (EKF)

EKF 的作法最直接：在當前估計值附近做一階泰勒展開，用雅可比矩陣取代 A 與 H 。

$$\begin{aligned}G &= \partial g / \partial x \big|_{(\hat{x}_{t-1})} \quad (\text{運動模型的雅可比}) \\ H &= \partial h / \partial x \big|_{(\hat{x}_t^-)} \quad (\text{量測模型的雅可比})\end{aligned}$$

EKF 的線性化。其餘四條方程式的形式與 KF 完全相同，只是把 A 換成 G 。

注意狀態的預測與量測的預測仍使用非線性函數本身（而非線性化版本）——只有協方差的傳播使用雅可比。這個細節很重要：

$$\begin{aligned}\hat{x}_t^- &= g(\hat{x}_{t-1}, u_t) \quad (\text{用非線性函數}) \\ P_t^- &= G P_{t-1} G^T + Q \quad (\text{用雅可比})\end{aligned}$$

EKF 的預測步驟。

EKF 的三個限制：

- **線性化誤差**：泰勒展開只在小範圍內準確。若不確定性很大（ P 很大），線性化的誤差就會顯著。
- **不再是最佳的**：EKF 只是一個近似，失去了 KF 的最佳性保證。
- **可能發散**：若線性化誤差累積，估計可能完全偏離真值且 P 卻顯示「很確定」——這是最危險的失效模式。

儘管有這些限制，EKF 仍是實務上最常用的方法，因為它計算效率高、實作成熟、且在多數

機器人應用中的非線性程度尚可接受。

24.6.3 無跡 Kalman 濾波器 (UKF)

UKF 採取完全不同的思路：與其線性化函數，不如取樣分布。

它的核心是無跡變換 (unscented transform)：

1. 由當前的均值與協方差，精心挑選出 $2n+1$ 個代表點（稱為 **sigma points**），使它們的樣本均值與協方差恰好等於原分布。
2. 把每一個 sigma point 直接代入非線性函數（不做任何近似）。
3. 由變換後的點，重新計算均值與協方差。

UKF 的優勢：不需要計算雅可比（對複雜模型是很大的便利）、精度達二階（EKF 只有一階）、且對強非線性系統更穩健。

UKF 的代價：計算量約為 EKF 的 2~3 倍。但由於 sigma points 的數目只有 $2n+1$ （線性成長），對中等維度的系統仍完全可接受。

表 24-4 EKF 與 UKF 的比較

面向	EKF	UKF
處理非線性的方式	線性化（泰勒展開）	取樣傳遞（sigma points）
需要雅可比	需要（手推或數值微分）	不需要
近似精度	一階	二階
計算量	低	中（約 2~3 倍）
強非線性下的穩健性	較差，可能發散	較佳
實作難度	中（需推導雅可比）	低（只需模型函數）
典型應用	多數機器人定位、SLAM	姿態估計、強非線性系統

實務建議：若模型的雅可比容易推導且非線性不強，用 EKF；若模型複雜（雅可比難推）或非線性顯著（如姿態估計含大角度旋轉），UKF 往往是更好的選擇。

24.7 粒子濾波器

24.7.1 放棄高斯假設

EKF 與 UKF 都假設信念是高斯分布（單峰）。但 24.1.3 節提到的**全域定位**問題中，初始的信念可能遍布整個地圖、或有多個候選位置——這根本不是高斯分布。

粒子濾波器（particle filter）徹底放棄了參數化的表示，改用**大量的樣本**來表示任意形狀的分布。

$$\text{bel}(\mathbf{x}_t) \approx \{(\mathbf{x}_t^{(i)}, w_t^{(i)})\}, i = 1, \dots, M$$

以 M 個帶權重的粒子表示信念。每個粒子是一個「可能的位置假設」，權重代表其可信度。

這個表示法的威力在於：它能表示任何形狀的分布——多峰、不對稱、甚至不連續。代價則是需要大量的粒子（典型 1000 ~ 100000 個），計算量遠高於 Kalman 家族。

24.7.2 三個步驟

粒子濾波器的每一次迭代包含三個步驟。

步驟一：預測（採樣）。對每一個粒子，依運動模型加上隨機雜訊，往前推進：

$$\mathbf{x}_t^{(i)} \sim p(\mathbf{x}_t | \mathbf{x}_{t-1}^{(i)}, \mathbf{u}_t)$$

每個粒子獨立地「走一步」，且各自加上不同的隨機擾動。這使粒子群逐漸散開——對應不確定性的增加。

步驟二：加權。依量測模型，計算每個粒子的可信度：

$$w_t^{(i)} = p(z_t | \mathbf{x}_t^{(i)})$$

與量測越吻合的粒子，權重越大。例如某粒子預測「前方 2 公尺有牆」，而光達確實量到 2 公尺，該粒子就得到高權重。

步驟三：重取樣（Resampling）。依權重重新抽取 M 個粒子（權重大的可能被抽中多次、權重小的可能消失）。

重取樣是粒子濾波器的關鍵機制——它使**計算資源自動集中到高可能性的區域**。若沒有重取樣，大量粒子會停留在不可能的位置，浪費計算量。

24.7.3 粒子退化與其對策

粒子濾波器有一個惡名昭彰的失效模式：**粒子退化**（particle depletion）。

◎ 粒子退化

重取樣會複製高權重的粒子、淘汰低權重的粒子。經過多次迭代後，所有粒子可能都源自同一個祖先——**多樣性喪失，分布退化成一個點**。此時濾波器會過度自信，且一旦真實位置不在該點附近，

就再也無法恢復（因為沒有粒子在正確的地方）。

三個標準的對策：

- **只在必要時重取樣**：計算「有效粒子數」 $N_{\text{eff}} = 1/\Sigma(w^{(i)})^2$ ，只有當它低於門檻（如 $M/2$ ）時才重取樣。這是最重要的一項改善。
- **加入隨機粒子**：每次迭代注入少量（如 1~5%）隨機分布的粒子。這使系統能從綁架問題中恢復——若真實位置在某處，總會有粒子恰好落在附近。
- **低變異數重取樣**：使用系統性採樣（systematic resampling）而非簡單的多項式採樣，能顯著降低取樣造成的變異。

24.7.4 適用場合的比較

表 24-5 三種濾波器的比較

面向	KF / EKF	UKF	粒子濾波器
分布表示	高斯（均值 + 協方差）	高斯（sigma points）	任意（大量樣本）
能否多峰	否	否	是
處理非線性	線性化（一階）	取樣（二階）	完全不近似
計算量	低	中	高
維度可擴充性	佳（ $O(n^3)$ ）	佳	差（維度詛咒）
全域定位	不適用	不適用	適用
綁架問題復原	不能	不能	可（需注入隨機粒子）
典型應用	位置追蹤、感測器融合	姿態估計	全域定位（MCL）、地圖匹配

粒子濾波器最大的限制是維度詛咒：所需的粒子數隨狀態維度指數成長。因此它適用於低維問題（如二維定位的 3 維狀態），但不適用於高維（如第 25 章 SLAM 中包含數百個地標的狀態向量）。這個限制決定了兩者的分工。

在移動機器人定位中，粒子濾波器的具體應用稱為**蒙地卡羅定位**（Monte Carlo Localization, MCL），是 ROS 2 導航堆疊中 AMCL 套件的核心演算法。

24.8 伏筆回收：互補濾波

24.8.1 第 6 章那條式子從哪裡來

第 6.3.2 節在討論 IMU 時，給出了一條融合加速度計與陀螺儀的公式：

$$\theta = \alpha \cdot (\theta_{\text{prev}} + \omega \cdot \Delta t) + (1 - \alpha) \cdot \theta_{\text{accel}}$$

互補濾波。當時只說它是一個「經驗法則」， α 通常取 0.98。

現在可以揭曉它的來源。把它與 24.4.3 節的 Kalman 更新式對照：

$$\text{Kalman: } \hat{x} = \hat{x}^- + K(z - \hat{x}^-) = (1 - K) \cdot \hat{x}^- + K \cdot z$$

$$\text{互補濾波: } \theta = \alpha \cdot \theta_{\text{gyro}} + (1 - \alpha) \cdot \theta_{\text{accel}}$$

兩者結構完全相同！只要令 $\alpha = 1 - K$ ，兩條式子就一模一樣。

因此：互補濾波就是「Kalman 增益固定不變」的 Kalman 濾波器。

24.8.2 為何叫「互補」

這個名稱來自頻域的觀點。兩個感測器的誤差特性恰好互補：

- **陀螺儀**：短期極準，但積分後會**漂移**（低頻誤差大）。因此對它用**高通濾波**——只取其快速變化的部分。
- **加速度計**：長期平均準確（重力方向不變），但受運動加速度干擾而**雜訊大**（高頻誤差大）。因此對它用**低通濾波**——只取其緩慢變化的部分。

兩個濾波器的頻率響應相加恰好為 1（互補），因此不會遺漏或重複計算任何頻段的資訊。這正是「互補」一詞的由來。

$$H_{\text{highpass}}(s) + H_{\text{lowpass}}(s) = 1$$

互補的條件。這是頻域的觀點，與時域的加權平均是同一件事。

24.8.3 什麼時候該用哪一個

既然互補濾波是簡化版，為何不總是使用完整的 Kalman 濾波器？答案是**成本與收益的權衡**。

表 24-6 互補濾波與 Kalman 濾波器的比較

面向	互補濾波	Kalman 濾波器
增益	固定（手動調 α ）	自動計算（隨不確定性變化）
計算量	極低（幾次乘加）	中（矩陣運算與求逆）
需要的模型	無	運動模型、量測模型、Q、R
能否處理變動的雜訊	不能	能
能否輸出不確定性	不能	能（協方差矩陣）

面向	互補濾波	Kalman 濾波器
實作難度	極低（幾行程式）	中
典型應用	小型無人機、平衡車、教學	自駕車、專業無人機、SLAM

選擇的原則：若感測器的雜訊特性**穩定**、且不需要不確定性的輸出，互補濾波往往就夠了——它簡單、可靠、且不會因為 Q 、 R 設錯而發散。這正是為何許多商用飛控（尤其是入門級）採用它。

但在三種情況下必須使用 Kalman：一、雜訊特性會變化（如 GPS 訊號時好時壞）；二、需要不確定性的輸出以供決策（如「不確定時就減速」）；三、感測器數量多且更新率不同（Kalman 能自然處理，互補濾波則難以擴充）。

◎ 教學上的一個建議

本書刻意在第 6 章先給出互補濾波、到第 24 章才揭曉其來源。這個安排是有意的：**先讓學生用一行公式做出能運作的東西，建立信心與直覺；再回頭理解其背後的理論。**這與第 2 章「即時學習」的策略一致。實務上也建議這個順序——先用互補濾波把系統跑起來，確認硬體與整體架構沒問題，再視需求升級為 Kalman。過早引入複雜的方法，往往會讓除錯變得極為困難，因為你分不清問題是出在硬體、模型還是濾波器參數。

24.9 實務上的陷阱

24.9.1 Q 與 R 該怎麼設

Kalman 濾波器的理論很優美，但實務上最大的困難是： **Q 與 R 該設多少？**這兩個矩陣沒有標準答案，卻直接決定濾波器的行為。

R （量測雜訊）相對容易：讓感測器對著一個**靜止不動**的目標量測數千次，計算讀值的變異數即可。多數感測器的規格書也會提供標準差。

Q （過程雜訊）則困難得多，因為它描述的是「模型有多不準」——這無法直接量測。三種實務作法：

- **由物理推算：**如第 23.8.4 節的里程計誤差模型，把變異數設為正比於運動量。這是最有依據的作法。
- **由實驗調整：**先設一個初值，觀察濾波器的行為並調整（見下節的判斷準則）。

- **自適應估計**：由創新序列的統計特性線上估計 Q 與 R 。理論優雅但實作複雜，且可能不穩定。

24.9.2 如何判斷調得對不對

有一個極為實用的診斷工具：**檢查創新序列** (innovation，即 $z - H\hat{x}$)。

理論上，若 Q 與 R 設定正確，創新序列應該具備兩個性質：

性質一：均值為零 (無系統性偏差)

性質二：其變異數應等於 $S = H P^- H^T + R$ (濾波器自己預測的值)

創新的一致性檢驗。這是判斷濾波器健康與否的黃金標準。

由此可以診斷三種問題：

表 24-7 濾波器的診斷

症狀	可能原因	處理方式
創新的實際變異數 > 預測值	Q 或 R 設得太小 (過度自信)	增大 Q (更常見) 或 R
創新的實際變異數 < 預測值	Q 或 R 設得太大 (過度保守)	減小 Q 或 R
創新的均值不為零	有未建模的系統性偏差	檢查感測器校正、模型是否遺漏項
估計值跟不上真實變化	Q 太小、過度信任模型	增大 Q
估計值雜訊大、抖動	R 太小、過度信任量測	增大 R
協方差 P 持續增大	長時間無有效量測	檢查感測器是否失效

一個重要的實務原則：當不確定時，寧可把 Q 設大一點。 Q 太小會使濾波器過度自信、拒絕接受量測、最終發散且無法恢復； Q 太大則只是使估計稍微雜訊多一些，但系統仍然穩健。

24.9.3 三個常見的失效

失效一：未建模的偏差 (bias)。Kalman 濾波器假設雜訊的均值為零。若感測器有系統性偏差 (如陀螺儀的零漂、磁力計受鐵件干擾)，濾波器會被系統性地拉偏，且協方差仍顯示「很確定」——這是最危險的情形。

因應之道：把偏差本身加入狀態向量一起估計。例如狀態由 [位置, 速度] 擴充為 [位置, 速度, 感測器偏差]。這是實務上極為常用的技巧，稱為**狀態擴增** (state augmentation)。

失效二：離群值 (outlier)。單一個錯誤的量測 (如視覺誤匹配、光達打到玻璃) 可能嚴

重破壞估計。

因應之道：馬氏距離門檻 (Mahalanobis gating)。計算創新的正規化距離，若超過門檻就拒絕該量測：

$$d^2 = \mathbf{v}^T \mathbf{S}^{-1} \mathbf{v}, \quad \text{其中 } \mathbf{v} \text{ 為創新、} \mathbf{S} = \mathbf{H} \mathbf{P}^- \mathbf{H}^T + \mathbf{R}$$

馬氏距離的平方。若 d^2 超過卡方分布的門檻 (如 95% 對應的值)，就拒絕此量測。這是實務上必備的保護機制。

失效三：時間戳不同步。不同感測器的資料到達時間不同，若未正確處理時間戳，會把「五十毫秒前的量測」當成「現在的量測」使用。在高速運動時，這會造成顯著的誤差。

因應之道：所有感測資料都應帶精確的時間戳，並在融合時做時間對齊 (把狀態預測到量測的時刻，或反過來)。這正是第 8.9.2 節提到 ROS 2 的 TF 系統儲存變換歷史紀錄的原因。

24.10 多感測器融合的架構

24.10.1 常見的感測器組合

實務的機器人系統通常融合多種感測器，各自貢獻不同的資訊：

表 24-8 常見感測器的互補特性

感測器	更新率	短期精度	長期漂移	失效情境
輪式里程計	高 (100+ Hz)	高	會漂移	打滑、抬起
IMU (陀螺儀)	極高 (1000 Hz)	極高	會漂移	長時間積分
IMU (加速度計)	極高	低 (雜訊大)	不漂移	運動加速度干擾
光達 + 地圖匹配	中 (10~40 Hz)	中高	不漂移	無特徵環境、動態物體
視覺標記	中 (5~30 Hz)	高	不漂移	遮蔽、光線不足
GNSS	低 (1~10 Hz)	低 (公尺級)	不漂移	室內、隧道、多路徑
視覺里程計	中 (10~30 Hz)	高	會漂移	無紋理、快速運動、光線變化

融合的核心邏輯：用高頻但會漂移的感測器 (里程計、IMU) 提供平滑的短期估計，用低頻但不漂移的感測器 (光達、視覺標記、GNSS) 定期校正。這正是開頭故事的正確解法。

24.10.2 鬆耦合與緊耦合

多感測器融合有兩種架構，取捨明確。

鬆耦合 (Loosely Coupled)：每個感測器先各自處理成一個位姿估計，再融合這些位姿。

- **優點**：模組化、易於實作與除錯、單一感測器失效時容易偵測與隔離。
- **缺點**：資訊有損失。例如 GNSS 只收到三顆衛星時無法解出位姿，在鬆耦合下這筆資料就完全浪費了。

緊耦合 (Tightly Coupled)：把所有感測器的**原始量測**直接放入同一個濾波器。

- **優點**：資訊利用最完整、精度最高。上例中三顆衛星的資料仍能提供部分約束。
- **缺點**：實作複雜、狀態維度高、單一感測器的異常會直接污染整個估計。

實務建議：從鬆耦合開始（容易除錯），確認系統運作後，若精度不足再考慮緊耦合。多數工業應用鬆耦合已足夠；高精度需求（如自駕車的視覺慣性里程計）才使用緊耦合。

24.10.3 ROS 2 中的實務

ROS 2 生態系提供了成熟的融合工具：

- **robot_localization**：最常用的套件，提供 EKF 與 UKF 節點，能融合任意數量的里程計、IMU 與 GNSS 來源。設定檔中可指定「哪個感測器提供哪些狀態量」——這是鬆耦合的典型實作。
- **AMCL**：粒子濾波器的實作，用於已知地圖中的全域定位（第 25 章會再提及）。
- **TF2**：管理各座標系之間的變換與其時間歷史，是所有融合的基礎設施（第 8.9.2 節）。

一個標準的移動機器人定位架構是：**robot_localization** 融合里程計與 IMU 產生平滑的 **odom→base_link** 變換（高頻、會漂移）；**AMCL** 以光達與地圖匹配產生 **map→odom** 變換（低頻、不漂移）。兩者串接即得到不漂移且平滑的定位。這個**兩層架構**是 ROS 導航的標準設計，值得深入理解。

24.11 由估測邁向建圖

24.11.1 本章的成果

本章建立了機率式估測的完整框架。你現在能夠：以信念分布表示不確定性、理解貝氏濾波「一呼一吸」的節奏、運用協方差矩陣與誤差橢圓描述多維不確定性、實作 Kalman 濾波器並理解增益的自動調節、以 EKF 或 UKF 處理非線性、以粒子濾波器處理多峰分布與全域定位、並診斷濾波器的常見失效。

更重要的是那個核心洞見：**依可信度加權，而非二選一**。這個看似簡單的原則，是所有感測器融合的基礎，也回答了開頭故事中小好的困境。

我們也回收了第 6.3.2 節的伏筆——**互補濾波就是增益固定的 Kalman 濾波器**，兩者在時域與頻域上是同一件事的兩種表述。

24.11.2 接下來：如果連地圖都沒有呢

但本章的所有定位方法都有一個共同的前提：**必須有地圖**。粒子濾波器要「用光達與地圖匹配」、視覺標記要「事先知道標記的位置」、地圖匹配更是顧名思義。

問題是——**地圖從哪裡來**？人工測繪一棟建築物既昂貴又緩慢，而且環境會改變。理想的做法是讓機器人自己建立地圖。

但這裡出現了一個看似無解的循環：**要建立地圖，必須知道自己在哪裡（否則不知道把觀測到的東西畫在地圖的哪個位置）；要知道自己在哪裡，又必須有地圖（否則沒有參照）**。這就像雞生蛋、蛋生雞。

第 25 章 (SLAM) 將回答這個問題。同步定位與建圖 (Simultaneous Localization and Mapping) 是機器人學最著名的難題之一，而它的解法正是建立在本章之上——把「機器人位姿」與「地圖中所有地標的位置」**放進同一個狀態向量**，一起估計。屆時你會看到 EKF 與粒子濾波器如何被擴展來處理這個高維問題，以及為何現代的方法轉向了「圖最佳化」的觀點。

換句話說：第 24 章讓機器人在已知地圖中知道自己在哪裡，第 25 章讓它在未知環境中同時搞清楚地圖與自己的位置。

完整計算例題

例題 24-1 一維資訊融合

題目：機器人用兩種方式估計自己的位置。里程計給出 $x_1 = 10.20 \text{ m}$ ，標準差 $\sigma_1 = 0.30 \text{ m}$ ；視覺標記給出 $x_2 = 10.50 \text{ m}$ ，標準差 $\sigma_2 = 0.10 \text{ m}$ 。(a) 求融合後的估計值與標準差。(b) 計算兩者的權重比。(c) 若視覺標記在光線不佳處，標準差劣化為 0.50 m ，重新計算。(d) 比較 (a) 與 (c)，說明濾波器的行為。

解 (a)：由 24.4.1 節的公式：

$$\mu = (\sigma_2^2 \mu_1 + \sigma_1^2 \mu_2) / (\sigma_1^2 + \sigma_2^2) = (0.01 \times 10.20 + 0.09 \times 10.50) / (0.09 + 0.01)$$

代入數值。注意權重是用對方的變異數。

$$\mu = (0.1020 + 0.9450) / 0.10 = 1.0470 / 0.10 = 10.47 \text{ m}$$

融合後的估計值。

$$\sigma^2 = (\sigma_1^2 \sigma_2^2) / (\sigma_1^2 + \sigma_2^2) = (0.09 \times 0.01) / 0.10 = 0.009 \Rightarrow \sigma = 0.0949 \text{ m}$$

融合後的標準差約 9.5 公分。

關鍵觀察：融合後的標準差 0.0949 m 小於兩者中較小的 0.10 m——這印證了 24.4.2 節的洞見二：融合必定使估計更確定。

解 (b)：權重正比於 $1/\sigma^2$ ：

$$w_1 : w_2 = 1/0.09 : 1/0.01 = 11.11 : 100 = 1 : 9$$

視覺標記的權重是里程計的 9 倍。因此融合結果 10.47 明顯偏向視覺的 10.50。

解 (c)： $\sigma_2 = 0.50 \text{ m}$ 時：

$$\mu = (0.25 \times 10.20 + 0.09 \times 10.50) / (0.09 + 0.25) = (2.550 + 0.945) / 0.34 = 3.495 / 0.34 = 10.28 \text{ m}$$

融合後的估計值。

$$\sigma^2 = (0.09 \times 0.25) / 0.34 = 0.0662 \Rightarrow \sigma = 0.257 \text{ m}$$

融合後的標準差。

$$w_1 : w_2 = 1/0.09 : 1/0.25 = 11.11 : 4.0 = 2.78 : 1$$

現在里程計的權重反而較大。

解 (d)：兩種情形的對照極具啟發性：

表 24-9 例題 24-1 的結果對照

情形	視覺 σ_2	融合結果	偏向	融合後 σ
(a) 光線良好	0.10 m	10.47 m	偏向視覺 (權重 9:1)	0.095 m
(c) 光線不佳	0.50 m	10.28 m	偏向里程計 (權重 1:2.78)	0.257 m

這正是 Kalman 濾波器的價值：不需要人工判斷「現在該相信誰」，只要正確描述各感測器的不確定性，權重就會自動調整。若採用開頭故事中「直接覆蓋」的作法，兩種情形都會得到

10.50——在光線不佳時將是嚴重的錯誤。

例題 24-2 Kalman 濾波器的完整迭代

題目：一維的位置追蹤問題。狀態為位置 x ，運動模型為 $x_t = x_{t-1} + u_t$ (u 為里程計增量)，過程雜訊 $Q = 0.04 \text{ m}^2$ ；量測模型為 $z_t = x_t$ ，量測雜訊 $R = 0.09 \text{ m}^2$ 。初始估計 $\hat{x}_0 = 0$ 、 $P_0 = 0.25 \text{ m}^2$ 。(a) 若第一步的里程計 $u_1 = 1.00 \text{ m}$ ，求預測值與預測協方差。(b) 若量測值 $z_1 = 1.30 \text{ m}$ ，求 Kalman 增益。(c) 求更新後的估計值與協方差。(d) 若接連三步都沒有量測，求協方差的演變。

解 (a)：由方程①② (此處 $A = 1$ 、 $B = 1$)：

$$\hat{x}_1^- = A \hat{x}_0 + B u_1 = 1 \times 0 + 1 \times 1.00 = 1.00 \text{ m}$$

狀態預測。

$$P_1^- = A P_0 A^T + Q = 1 \times 0.25 \times 1 + 0.04 = 0.29 \text{ m}^2$$

協方差預測。不確定性由 0.25 增加到 0.29——這正是預測步驟的必然結果。

解 (b)：由方程③ ($H = 1$)：

$$K_1 = P_1^- H^T (H P_1^- H^T + R)^{-1} = 0.29 / (0.29 + 0.09) = 0.29 / 0.38 = 0.7632$$

Kalman 增益約 0.76。

判讀： $K = 0.76 > 0.5$ ，代表濾波器較信任量測。這合理，因為此時預測的不確定性 (0.29) 大於量測的不確定性 (0.09)。

解 (c)：由方程④⑤：

$$\hat{x}_1 = \hat{x}_1^- + K_1(z_1 - H \hat{x}_1^-) = 1.00 + 0.7632 \times (1.30 - 1.00) = 1.00 + 0.2290 = 1.229 \text{ m}$$

更新後的估計值。

注意：結果 1.229 落在預測值 1.00 與量測值 1.30 之間，且偏向量測——與增益 0.76 的判讀一致。

$$P_1 = (1 - K_1)P_1^- = (1 - 0.7632) \times 0.29 = 0.2368 \times 0.29 = 0.0687 \text{ m}^2$$

更新後的協方差。由 0.29 大幅降到 0.069——量測提供了顯著的資訊。

驗證：融合後的標準差為 $\sqrt{0.0687} = 0.262 \text{ m}$ ，小於預測的 $\sqrt{0.29} = 0.539 \text{ m}$ 與量測的

$\sqrt{0.09} = 0.30 \text{ m}$ ——再次印證融合必定更確定。

解 (d)：無量測時只執行預測步驟，協方差逐步累加 Q ：

$$\begin{aligned} P_2 &= 0.0687 + 0.04 = 0.1087 \\ P_3 &= 0.1087 + 0.04 = 0.1487 \\ P_4 &= 0.1487 + 0.04 = 0.1887 \text{ m}^2 \end{aligned}$$

協方差線性成長 (因 $A = 1$)。

對應的標準差由 0.262 m 增至 0.434 m 。這正是里程計漂移在機率語言中的表述——沒有外部量測時，不確定性持續增加且無法停止。若長時間如此， P 會無限增大，代表系統「完全不知道自己在哪裡」。

例題 24-3 互補濾波與 Kalman 的等價

題目：某平衡車以互補濾波融合陀螺儀與加速度計，取 $\alpha = 0.98$ ，取樣週期 5 ms 。(a) 求對應的等效 Kalman 增益 K 。(b) 若已知加速度計的角度量測雜訊 $R = 4.0 \text{ deg}^2$ ，反推此 α 對應的預測協方差 P^- 。(c) 計算互補濾波的等效時間常數。(d) 討論若改用 $\alpha = 0.90$ 會如何。

解 (a)：由 24.8.1 節的對應關係 $\alpha = 1 - K$ ：

$$K = 1 - \alpha = 1 - 0.98 = 0.02$$

等效 Kalman 增益。極小的 K 代表幾乎完全信任陀螺儀的預測。

解 (b)：由 $K = P^-/(P^- + R)$ 反解：

$$0.02 = P^-/(P^- + 4.0) \Rightarrow 0.02(P^- + 4.0) = P^- \Rightarrow 0.08 = 0.98 P^- \Rightarrow P^- = 0.0816 \text{ deg}^2$$

隱含的預測協方差，對應標準差 0.286 度 。

判讀：這代表工程師在設 $\alpha = 0.98$ 時，實質上假設了「陀螺儀在一個取樣週期內的預測誤差約 0.29 度 、而加速度計的量測誤差約 2 度 」。這個假設是否合理，取決於實際的感測器——這正是互補濾波「隱藏了假設」的地方。

解 (c)：互補濾波的等效時間常數為：

$$\tau = \alpha \Delta t / (1 - \alpha) = 0.98 \times 0.005 / 0.02 = 0.245 \text{ s}$$

時間常數約 0.25 秒 。

物理意義：陀螺儀的漂移會在約 0.25 秒 內被加速度計修正。因此慢於 0.25 秒 的漂移會被消

除，快於 0.25 秒的加速度計雜訊會被濾掉——這正是「互補」的頻域意義。

解 (d) : $\alpha = 0.90$ 時：

$$K = 0.10, \quad \tau = 0.90 \times 0.005 / 0.10 = 0.045 \text{ s}$$

增益變大五倍、時間常數縮短為 45 毫秒。

後果：更快速地修正陀螺儀漂移（好事），但也更容易受加速度計雜訊影響（壞事）。在平衡車上，這可能導致車體在振動時角度估計抖動，進而使控制器產生高頻的馬達指令。

實務結論： α 的選擇是「抗漂移」與「抗雜訊」的取捨——與第 16 章 PID 增益的取捨、第 18 章 LQR 的 Q/R 權重是同一類問題。若機器振動大，應提高 α （更信任陀螺儀）；若陀螺儀品質差、漂移快，則應降低 α 。

程式範例：Kalman 濾波器與粒子濾波器

以下 Python 程式實作一維與二維的 Kalman 濾波器、擴展 Kalman 濾波器，以及粒子濾波器，並比較它們在不同情境下的表現。

```
import numpy as np

# ===== 一維 Kalman 濾波器 =====
class KalmanFilter1D:
    """最簡單的 Kalman 濾波器 (24.5 節的五條方程式)"""
    def __init__(s, x0=0.0, P0=1.0, Q=0.04, R=0.09):
        s.x, s.P, s.Q, s.R = x0, P0, Q, R

    def predict(s, u=0.0):
        s.x = s.x + u                # ① 狀態預測
        s.P = s.P + s.Q              # ② 協方差預測 (必然增加)
        return s.x, s.P

    def update(s, z):
        K = s.P / (s.P + s.R)        # ③ Kalman 增益
        innov = z - s.x              # ④ 創新
        s.x = s.x + K*innov          # ⑤ 狀態更新
        s.P = (1 - K)*s.P            # ⑥ 協方差更新 (必然減少)
        return s.x, s.P, K, innov

# ===== 例題 24-1：一維資訊融合 =====
print("=== 例題 24-1：一維資訊融合 ===")
def fuse(m1, s1, m2, s2):
    v1, v2 = s1**2, s2**2
```

```

mu = (v2*m1 + v1*m2)/(v1 + v2)
var = v1*v2/(v1 + v2)
return mu, np.sqrt(var), (1/v1)/(1/v2)

print(f"{'情境':<12}{'里程計':>10}{'視覺':>10}{'融合結果':>12}"
      f"{'融合  $\sigma$ ':>10}{'權重比':>12}")
for label, s2 in (("光線良好", 0.10), ("光線普通", 0.25), ("光線不佳", 0.50)):
    mu, sig, ratio = fuse(10.20, 0.30, 10.50, s2)
    print(f"{label:<12}{'10.20':>10}{'10.50':>10}{mu:12.4f}"
          f"{'sig:10.4f'}{'1 : {1/ratio:.2f}':>12}")
print("→ 視覺越不準，結果越偏向里程計—權重自動調整")

# ===== 例題 24-2：完整迭代 =====
print("\n=== 例題 24-2：Kalman 濾波器的完整迭代 ===")
kf = KalmanFilter1D(x0=0.0, P0=0.25, Q=0.04, R=0.09)
x_pred, P_pred = kf.predict(u=1.00)
print(f"(a) 預測:  $\hat{x}^- = \{x\_pred:.4f\}$  m,  $P^- = \{P\_pred:.4f\}$  m2 "
      f"(由 0.25 增至  $\{P\_pred:.2f\})$ ")
x_upd, P_upd, K, innov = kf.update(z=1.30)
print(f"(b) Kalman 增益  $K = \{K:.4f\}$  "
      f"({'偏向量測' if K > 0.5 else '偏向預測'})")
print(f"(c) 更新:  $\hat{x} = \{x\_upd:.4f\}$  m,  $P = \{P\_upd:.4f\}$  m2 "
      f"(創新 =  $\{innov:.2f\})$ ")
print(f"     $\sigma$  由  $\{np.sqrt(P\_pred):.3f\}$  降至  $\{np.sqrt(P\_upd):.3f\}$  m")
print("(d) 接連三步無量測:")
for i in range(3):
    _, Pk = kf.predict(u=0.5)
    print(f"    第  $\{i+2\}$  步:  $P = \{Pk:.4f\}$  m2 ( $\sigma = \{np.sqrt(Pk):.3f\}$  m)")
print("    → 無量測時不確定性持續增加，這就是漂移")

# ===== 完整模擬：追蹤一個移動目標 =====
print("\n=== 模擬：追蹤等速運動的目標（比較三種策略）===")
np.random.seed(42)
T, dt = 200, 0.05
true_v = 1.0
true_x = np.cumsum(np.full(T, true_v*dt))
odom_bias = 1.03 # 里程計有 3% 的比例誤差
odom = np.cumsum(np.full(T, true_v*dt*odom_bias)
                 + np.random.randn(T)*0.005)
meas = true_x + np.random.randn(T)*0.30 # 視覺量測， $\sigma=0.3$  m
meas_avail = np.arange(T) % 10 == 0 # 每 10 步才有一次量測

# 策略一：只用里程計
err_odom = np.abs(odom - true_x)
# 策略二：直接覆蓋（開頭故事的錯誤作法）
est_ovr, jumps = [], []
x = 0.0
for k in range(T):
    x += true_v*dt*odom_bias + np.random.randn()*0.005

```

```

    if meas_avail[k]:
        jumps.append(abs(meas[k] - x))
        x = meas[k] # 直接覆蓋
    est_ovr.append(x)
err_ovr = np.abs(np.array(est_ovr) - true_x)
# 策略三：Kalman 融合
kf2 = KalmanFilter1D(x0=0.0, P0=0.01, Q=0.0004, R=0.09)
est_kf = []
for k in range(T):
    kf2.predict(u=true_v*dt*odom_bias + np.random.randn()*0.005)
    if meas_avail[k]:
        kf2.update(meas[k])
    est_kf.append(kf2.x)
err_kf = np.abs(np.array(est_kf) - true_x)

print(f"{'策略':<20}{'平均誤差':>12}{'最大誤差':>12}{'跳動':>12}")
print(f"{'只用里程計':<20}{err_odom.mean():10.4f} m{err_odom.max():10.4f} m"
      f"{'無':>12}")
print(f"{'直接覆蓋':<20}{err_ovr.mean():10.4f} m{err_ovr.max():10.4f} m"
      f"{'np.mean(jumps):10.4f} m")
print(f"{'Kalman 融合':<20}{err_kf.mean():10.4f} m{err_kf.max():10.4f} m"
      f"{'平滑':>12}")
print("→ Kalman 兼顧了準確度與平滑性")

# ===== 二維 EKF：差速車定位 =====
print("\n=== 擴展 Kalman 濾波器：差速車定位 ===")
class EKF_DiffDrive:
    """狀態 [x, y,  $\theta$ ] · 量測為到已知地標的距離與方位"""
    def __init__(s, x0, P0, Q, R):
        s.x, s.P, s.Q, s.R = np.array(x0, float), np.array(P0), Q, R

    def predict(s, ds, dth):
        th = s.x[2]
        # 非線性運動模型 (用函數本身)
        s.x = s.x + np.array([ds*np.cos(th + dth/2),
                              ds*np.sin(th + dth/2), dth])
        s.x[2] = np.arctan2(np.sin(s.x[2]), np.cos(s.x[2])) # 角度正規化
        # 雅可比 (用於協方差傳播)
        G = np.array([[1, 0, -ds*np.sin(th + dth/2)],
                      [0, 1, ds*np.cos(th + dth/2)],
                      [0, 0, 1]])
        s.P = G @ s.P @ G.T + s.Q

    def update(s, z, landmark):
        lx, ly = landmark
        dx, dy = lx - s.x[0], ly - s.x[1]
        q = dx*dx + dy*dy
        r = np.sqrt(q)
        z_hat = np.array([r, np.arctan2(dy, dx) - s.x[2]])
        H = np.array([[-dx/r, -dy/r, 0],
                      [dy/q, -dx/q, -1]])
        innov = z - z_hat
        innov[1] = np.arctan2(np.sin(innov[1]), np.cos(innov[1]))

```



```

S = H @ s.P @ H.T + s.R
# 馬氏距離門檻 ( 24.9.3 節 )
d2 = innov @ np.linalg.solve(S, innov)
if d2 > 9.21:                                     # 卡方 99%, 2 自由度
    return False, d2
K = s.P @ H.T @ np.linalg.inv(S)
s.x = s.x + K @ innov
s.P = (np.eye(3) - K @ H) @ s.P
return True, d2

ekf = EKF_DiffDrive([0., 0., 0.], np.diag([0.01, 0.01, 0.01]),
                    np.diag([0.001, 0.001, 0.0005]),
                    np.diag([0.04, 0.01]))
landmark = np.array([5.0, 3.0])
print(f"{'步':>4}{ 'x':>9}{ 'y':>9}{ 'θ(度)':>9}{ 'σx':>9}{ 'σy':>9}{ '量測':>10}")
for k in range(10):
    ekf.predict(0.5, np.deg2rad(5))
    tag = "無"
    if k % 3 == 2:
        dx, dy = landmark - ekf.x[:2]
        z_true = np.array([np.hypot(dx, dy) + np.random.randn()*0.2,
                            np.arctan2(dy, dx) - ekf.x[2]
                            + np.random.randn()*0.1])
        ok, d2 = ekf.update(z_true, landmark)
        tag = f"接受" if ok else f"拒絕"
    print(f"{k:4d}{ekf.x[0]:9.3f}{ekf.x[1]:9.3f}"
          f"{np.rad2deg(ekf.x[2]):9.2f}{np.sqrt(ekf.P[0,0]):9.4f}"
          f"{np.sqrt(ekf.P[1,1]):9.4f}{tag:>10}")
print("> 有量測時 σ 下降 · 無量測時 σ 持續上升")

# ===== 粒子濾波器：全域定位 =====
print("\n=== 粒子濾波器：一維全域定位 ( 走廊有三扇門 ) ===")
doors = [2.0, 5.0, 8.0]                                # 門的位置 ( 地圖 )
M = 2000
particles = np.random.uniform(0, 10, M)                # 初始：完全不知道在哪
weights = np.ones(M)/M

def sense_door(x):
    """量測：是否在門旁邊 ( ±0.4 m 內 ) """
    return any(abs(x - d) < 0.4 for d in doors)

def measurement_prob(x, saw_door):
    near = any(abs(x - d) < 0.4 for d in doors)
    return 0.9 if near == saw_door else 0.1

true_x = 2.1
moves = [0.0, 3.0, 3.0]                                # 前進序列
print(f"{'步':>4}{ '真實位置':>10}{ '看到門':>9}{ '粒子均值':>11}"
      f"{'粒子標準差':>12}{ '有效粒子':>11}")
for step, mv in enumerate(moves):
    # 預測
    true_x += mv

```

```

particles = particles + mv + np.random.randn(M)*0.15
particles = np.clip(particles, 0, 10)
# 加權
saw = sense_door(true_x)
weights = np.array([measurement_prob(p, saw) for p in particles])
weights /= weights.sum()
n_eff = 1.0/np.sum(weights**2)
print(f"{step:4d}{true_x:10.2f}{'是' if saw else '否':>9}")
      f"{np.average(particles, weights=weights):11.3f}"
      f"{np.sqrt(np.average((particles-np.average(particles,weights=weights))**2,
weights=weights)):12.3f}"
      f"{n_eff:11.0f}")
# 重取樣 ( 僅在有效粒子數過低時 · 24.7.3 節 )
if n_eff < M/2:
    idx = np.random.choice(M, M, p=weights)
    particles = particles[idx]
    weights = np.ones(M)/M
print("> 一開始標準差極大 ( 不知道在哪 ) · 逐步收斂到真實位置")
print("> 這是 Kalman 濾波器做不到的——初期是多峰分布")

```

請試著修改：(一) 在 Kalman 模擬中把 Q 設為原本的 100 倍，觀察估計是否變得雜訊很大。(二) 在 EKF 中故意讓某一次的量測有 5 公尺的誤差 (模擬離群值)，驗證馬氏距離門檻是否成功拒絕它。(三) 在粒子濾波器中把粒子數降到 50，觀察是否會發生粒子退化而收斂到錯誤的位置。

實作：感測器融合的實驗

本章實作以第 23 章的移動機器人為平台，加裝 IMU 後實現感測器融合。

- 感測器雜訊量測：**讓機器人靜止不動，連續記錄 IMU 與里程計的讀值數千筆。計算各軸的均值 (應為零或已知的偏差) 與變異數——這就是 R 矩陣的來源。
- 偏差辨識：**特別注意陀螺儀的零漂。靜止十分鐘，觀察積分後的角度如何線性增長，由斜率估算零漂量。這是必須補償的系統性誤差。
- 互補濾波實作：**以第 6.3.2 節的公式融合陀螺儀與加速度計 (或與里程計的航向)。測試不同的 α (0.90、0.95、0.98、0.995)，記錄各自的漂移速率與雜訊水準。
- 互補濾波的驗證：**把機器人放在轉盤上，緩慢旋轉一整圈後回到原位。比較純陀螺儀積分、純加速度計、與互補濾波三者的角度誤差。
- 一維 Kalman 實作：**以里程計為預測、以視覺標記 (或超音波測距到牆面) 為量測，實作一維位置的 Kalman 濾波器。

6. **Q、R 調參實驗**：系統性地改變 Q 與 R（各三個量級），記錄估計的平滑度與追隨速度。製作 3×3 的效果對照表，找出最佳組合。
7. **創新序列診斷**：記錄整個運行過程的創新序列，計算其均值與變異數，並與濾波器預測的 S 比較（24.9.2 節）。判斷 Q、R 是否設定合理。
8. **離群值測試**：故意在某幾個時刻注入錯誤的量測（如把視覺結果加上 2 公尺），驗證馬氏距離門檻能否成功拒絕。
9. **三種策略的對照**：讓機器人沿固定路徑行走 20 公尺，分別以（甲）只用里程計、（乙）直接覆蓋、（丙）Kalman 融合三種方式估計位置。記錄終點誤差與軌跡的平滑度。這是開頭故事的實驗版本。
10. **（進階）EKF 二維定位**：擴充為二維的 EKF，狀態為 (x, y, θ) ，以到已知地標的距離與方位為量測。繪出估計軌跡與誤差橢圓隨時間的演變。

表 24-10 感測器融合實驗記錄表（前兩列為範例）

測試項目	設定	終點誤差	軌跡平滑度	備註
只用里程計	—	1.82 m	平滑	系統性漂移
直接覆蓋	—	0.21 m	明顯跳動	轉彎處控制器震盪
（自行填寫其餘測試）				

國中小也看得懂：誰比較可信就相信誰多一點

假設你和朋友在猜一個西瓜有多重。你說「大概 5 公斤」，朋友說「應該有 7 公斤」。那到底是幾公斤呢？

你可能會說：取中間，6 公斤！這是一個好方法——但還不是最好的。

再想想：如果你昨天才用秤量過類似的西瓜，而你朋友只是隨口猜的，那答案應該比較靠近你的 5 公斤才對。反過來，如果朋友是水果店老闆，那就該相信他多一點。

這就是這一章最重要的道理：不是各取一半，而是「誰比較可信，就相信誰多一點」。

機器人也一樣。它有好幾種方法知道自己在哪裡：

方法一：數輪子轉了幾圈。 這個方法反應很快，但走久了會越來越不準（因為輪子會打滑）。

方法二：看天花板上的標記。 這個方法很準，但要等看到標記才知道，而且光線暗的時候會看不清楚。

聰明的機器人不會「二選一」，而是**兩個都用**：平常用輪子的資料快速更新，看到標記時就拿來修正。而且它還會判斷「現在這個標記看得清不清楚」——看得清楚就多相信一點，看不清楚就少相信一點。

這個方法有一個名字，叫做「**卡爾曼濾波器**」。它是 1960 年一位工程師發明的，後來被用在**阿波羅太空船**上——就是載人類第一次登陸月球的那艘太空船！沒有它，太空人可能就找不到回家的路了。

叫獸提醒與常見錯誤

- 誤區一：用「切換」或「覆蓋」取代融合。這會扔掉資訊並造成估計跳動，正確作法是依可信度加權。
- 誤區二：以為加入一個差的感測器會讓結果變糟。只要誤差模型正確（無偏差），多一個量測必定使估計更確定。
- 誤區三： Q 設得太小。這使濾波器過度自信、拒絕接受量測、最終發散且無法恢復；不確定時應寧可設大。
- 誤區四：忽略未建模的偏差。Kalman 假設雜訊均值為零，若感測器有系統性偏差，應把它加入狀態向量一起估計。
- 誤區五：沒有離群值保護。單一個錯誤量測就可能破壞估計，必須加入馬氏距離門檻。
- 誤區六：忽略時間戳。把舊的量測當成當前的使用，在高速運動時會造成顯著誤差。
- 誤區七：對強非線性系統直接用 EKF。線性化誤差可能導致發散，應考慮 UKF 或縮短更新週期。
- 誤區八：用 Kalman 濾波器做全域定位。它只能表示單峰分布，無法處理「可能在好幾個地方」的情形，應改用粒子濾波器。
- 誤區九：粒子濾波器每一步都重取樣。這會加速粒子退化，應只在有效粒子數低於門檻時才重取樣。
- 誤區十：粒子濾波器用於高維狀態。維度詛咒使所需粒子數指數成長，高維問題應改用 Kalman 家族。

- 誤區十一：忘記角度的正規化。角度差必須用 $\text{atan2}(\sin, \cos)$ 包裹到 $\pm\pi$ ，否則在跨越 $\pm 180^\circ$ 時會產生巨大的假創新。
- 誤區十二：過早使用複雜的方法。應先用互補濾波把系統跑起來確認硬體無誤，再視需求升級為 Kalman。

課堂討論

1. 開頭故事中小好「直接覆蓋」的方式融合兩種定位。除了跳動之外，請討論這個作法還有哪些隱患？（提示：考慮視覺偶爾誤判的情形）
2. 24.4.2 節的洞見二說「融合必定使估計更確定」，但 24.9.3 節又說「未建模的偏差會系統性拉偏結果」。這兩者矛盾嗎？請說明前者成立的前提條件。
3. Kalman 濾波器與第 18 章的 LQR 在數學上是对偶的（都解 Riccati 方程）。請討論這個對偶在工程直覺上意味著什麼——「估計」與「控制」為何是同一件事的兩面？
4. 粒子濾波器能處理多峰分布，但受維度詛咒限制。請討論：第 25 章的 SLAM 問題（狀態含數百個地標）該用哪一類方法？為什麼？
5. 本書刻意在第 6 章先給互補濾波、第 24 章才揭曉其原理。請討論這種「先用後懂」的教學安排的優缺點，以及它在工程實務上的對應（提示：對照第 2 章的即時學習策略）。

章末摘要

- 機率式機器人學的核心轉換：不問「在哪裡」，而問「在各個位置的可能性各有多大」。
- 信念分布的形狀承載資訊：窄峰代表確定、寬峰代表不確定、多峰代表有多個候選、拉長代表某方向不確定。
- 定位問題分三個層次：位置追蹤（初始已知）、全域定位（初始未知）、綁架問題（運行中突然錯位）。
- 貝氏濾波由兩步驟循環：預測（信念擴散、不確定性增加）與更新（信念收攏、不確定性減少）。
- 馬可夫假設使遞迴計算成為可能——當前狀態包含所有必要資訊，歷史可以丟棄。
- 高斯分布因「只需兩參數、線性運算下封閉、相乘仍為高斯」而成為標準選擇。
- 協方差矩陣對應誤差橢圓：特徵向量為主軸方向、特徵值平方根為半徑、非對角元素代表相

關性。

- 一維融合公式：權重與變異數成反比；融合必定使不確定性減小；資訊量 $1/\sigma^2$ 是可加的。
- Kalman 濾波器五條方程式：狀態預測、協方差預測、增益計算、狀態更新、協方差更新。
- Kalman 增益 $K = P^-(P^-+R)$ 自動調節：預測越不準則越信任量測，反之亦然。
- Kalman 濾波器在線性高斯假設下是最小均方誤差意義的最佳估測器，且與 LQR 互為對偶。
- EKF 以雅可比線性化處理非線性（一階、需推導雅可比）；UKF 以 sigma points 取樣傳遞（二階、不需雅可比）。
- 粒子濾波器以大量樣本表示任意分布，能處理多峰與全域定位，但受維度詛咒限制。
- 粒子退化的三個對策：只在有效粒子數過低時重取樣、注入隨機粒子、使用低變異數重取樣。
- 互補濾波是「增益固定」的 Kalman 濾波器： $\alpha = 1 - K$ ；其名稱來自兩感測器的頻率響應恰好互補。
- 實務三大陷阱：未建模的偏差（應狀態擴增）、離群值（應馬氏距離門檻）、時間戳不同步（應時間對齊）。
- 創新序列的一致性檢驗是診斷濾波器健康的黃金標準；不確定時應寧可把 Q 設大。
- 融合的核心邏輯：高頻但漂移的感測器提供短期平滑，低頻但不漂移的感測器定期校正。

觀念題與計算題

觀念題

1. 說明機率式機器人學的核心觀念轉換，並列舉這個轉換帶來的三個好處。
2. 定義信念分布，並說明五種分布形狀各代表什麼情境。
3. 區分位置追蹤、全域定位與綁架機器人問題，並指出各需要什麼方法。
4. 敘述貝氏濾波的兩個步驟，並說明各自對不確定性的影響。
5. 說明馬可夫假設的內容，以及它為何是遞迴計算的前提。
6. 列舉高斯分布被選用的四個理由，並說明哪兩個對遞迴特別關鍵。
7. 說明協方差矩陣的幾何意義，並解釋非對角元素代表什麼。
8. 推導一維資訊融合的公式，並說明「融合必定更確定」的原因與前提。

9. 寫出 Kalman 濾波器的五條方程式，並說明各項的物理意義。
10. 說明 Kalman 增益如何自動調節，並列出三種極端情形的行為。
11. 比較 EKF 與 UKF 的原理、精度與計算量，並說明各自的適用場合。
12. 說明粒子濾波器的三個步驟，以及重取樣為何是關鍵機制。
13. 什麼是粒子退化？請說明三種對策及其原理。
14. 推導互補濾波與 Kalman 濾波器的等價關係，並說明「互補」一詞的頻域意義。
15. 列舉感測器融合的三个實務陷阱，並說明各自的因應之道。
16. 說明創新序列的一致性檢驗，並列出三種可由此診斷的問題。

計算題

1. 兩個獨立的位置估計：GNSS 給出 25.4 m ($\sigma = 2.0$ m)、視覺里程計給出 24.8 m ($\sigma = 0.5$ m)。(a) 求融合後的估計值與標準差。(b) 求兩者的權重比。(c) 若 GNSS 進入城市峽谷使 σ 劣化為 8.0 m，重新計算。(d) 比較兩種情形並說明。
2. 一維 Kalman 濾波器： $Q = 0.09$ m²、 $R = 0.16$ m²、初始 $\hat{x} = 5.0$ m、 $P = 0.36$ m²。(a) 里程計增量 $u = 2.0$ m，求預測值與 P^- 。(b) 量測 $z = 7.5$ m，求 Kalman 增益。(c) 求更新後的估計值與協方差。(d) 若接連五步無量測，求 P 的演變並計算對應的標準差。
3. 某互補濾波器取 $\alpha = 0.995$ 、取樣週期 2 ms。(a) 求等效的 Kalman 增益。(b) 求等效時間常數。(c) 若加速度計的角度量測變異數 $R = 9.0$ deg²，反推隱含的預測協方差 P^- 。(d) 若機器振動加劇使加速度計雜訊倍增，討論應如何調整 α 。
4. 某 EKF 的創新為 $v = [0.35, -0.12]^T$ ，創新協方差 $S = [[0.04, 0.01], [0.01, 0.09]]$ 。(a) 計算 S 的反矩陣。(b) 計算馬氏距離的平方 d^2 。(c) 若門檻取卡方分布 95% (2 自由度為 5.99)，判斷此量測應接受或拒絕。(d) 若改用 99% 門檻 (9.21)，結論是否改變？

動手做與專題延伸

本章實作：狀態估測模組

延續第 23 章的移動機器人程式庫，加入狀態估測模組。這是第 25、26 章的必要基礎。

- 階段一：實作一維 Kalman 濾波器，並用例題 24-1、24-2 驗證。
- 階段二：實作二維 EKF 的差速車定位，狀態為 (x, y, θ) 、量測為到已知地標的距離與方位。

加入角度正規化與馬氏距離門檻。

- **階段三**：實作誤差橢圓的繪製，即時顯示協方差矩陣的幾何意義。
- **階段四**：實作一維與二維的粒子濾波器，含有效粒子數判斷與隨機粒子注入。
- **階段五**：實作創新序列的一致性檢驗工具，能自動診斷 Q 、 R 是否設定合理。

專題延伸

- **誤差橢圓的演化動畫**：模擬機器人沿一條路徑行進，途中經過數個地標。即時繪製協方差橢圓——你會看到橢圓在無量測時持續脹大、且沿行進方向拉長（因航向誤差累積），而在看到地標時瞬間收縮。這是本章所有理論最直觀的視覺化，強烈建議做成動畫。
- **三種濾波器的競賽**：設計一組測試情境（位置追蹤、全域定位、綁架問題、強非線性），對 EKF、UKF 與粒子濾波器各自測試。記錄估計誤差、計算時間與失敗率，製作完整的比較表。你會發現沒有一種方法在所有情境下都最好——這正是工程選型的依據。
- **進階：Q 與 R 的自動調參**。實作一個以創新序列統計為基礎的自適應方法：持續監測創新的實際變異數與濾波器預測的 S ，若兩者持續不符就自動調整 Q 。在一個模擬環境中測試（可故意讓感測器雜訊在中途改變），觀察自適應機制能否追上變化。討論這個方法的穩定性風險——若調整過於激進，濾波器可能震盪甚至發散。

參考文獻與延伸閱讀

- Thrun, S., Burgard, W. and Fox, D., Probabilistic Robotics, MIT Press（機率式機器人學的權威教材，第 2~4、7~8 章直接對應本章）。
- Kalman, R. E., "A New Approach to Linear Filtering and Prediction Problems," ASME J. Basic Engineering, 1960（Kalman 濾波器的原始論文）。
- Julier, S. J. and Uhlmann, J. K., "Unscented Filtering and Nonlinear Estimation," Proceedings of the IEEE, 2004（UKF 的完整說明）。
- Bar-Shalom, Y., Li, X.-R. and Kirubarajan, T., Estimation with Applications to Tracking and Navigation, Wiley（估測理論的工程導向專著，含大量實務細節）。
- Simon, D., Optimal State Estimation: Kalman, H_∞ , and Nonlinear Approaches, Wiley（涵蓋各類估測方法的完整教材）。
- Doucet, A., de Freitas, N. and Gordon, N. (Eds.), Sequential Monte Carlo Methods in

Practice, Springer (粒子濾波器的理論與應用) 。

- Dellaert, F. et al., "Monte Carlo Localization for Mobile Robots," ICRA 1999 (蒙地卡羅定位的奠基論文) 。
- Mahony, R., Hamel, T. and Pflimlin, J.-M., "Nonlinear Complementary Filters on the Special Orthogonal Group," IEEE Trans. Automatic Control, 2008 (互補濾波的嚴謹推廣) 。
- Bailey, T. and Durrant-Whyte, H., "Simultaneous Localization and Mapping: Part II," IEEE Robotics & Automation Magazine, 2006 (銜接第 25 章) 。
- ROS 2 robot_localization 與 AMCL 官方文件：EKF/UKF 融合與蒙地卡羅定位的實務參考。

第 25 章

同步定位與建圖 (SLAM)

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

【章首情境圖建議】左半為迴路閉合前後的地圖對照：上圖是只靠里程計建出的地圖——走廊呈弧形彎曲，繞行一圈後最後一段牆與第一段錯開約兩公尺，並以紅色虛線圈出錯位處；下圖是偵測到迴路閉合並最佳化後的同一張地圖——牆面筆直對齊、房間方正。右半為位姿圖的彈簧類比示意：一串珠子 (節點) 以彈簧 (里程計邊) 相連成環，環的兩端另接一根粗彈簧 (迴路閉合邊)，箭頭表示系統被拉回平衡的過程。畫面下緣文字：定位需要地圖，建圖需要定位——那就一起解。

叫獸開講 「要建圖得先定位，要定位得先有地圖——怎麼辦？」

叫獸把一台裝了光達的機器人放在實驗室門口，然後對全班說：「請你們讓它畫出這層樓的地圖。」

小翰很快動手：「簡單，讓它走一圈，每一步用里程計知道自己在哪，把光達看到的牆畫上去就好。」他寫完程式，機器人繞了一圈回到門口——螢幕上的地圖卻是一團扭曲的線條，走廊變成弧形，最後一段牆與第一段錯開了將近兩公尺。

「為什麼？」叫獸問。小好反應過來：「因為里程計會漂移。走到後面，它以為自己在 A 點，其實已經在 B 點了，所以把牆畫錯了位置。」

「那就先定位準一點再建圖。」小翰說。叫獸點頭：「好，那你怎麼定位？」

「用第 24 章的方法，跟地圖比對……」小翰說到一半停住了。

「地圖從哪來？」叫獸接話，「你要建圖，得先知道自己在哪；你要知道自己在哪，又得先有地圖。」他在白板上畫了一個圈：**定位 → 需要地圖 → 需要定位**。「這就是機器人學最有名的一个循環，叫做 **SLAM**——同步定位與建圖。」

教室安靜了幾秒。小好問：「那……有解嗎？」

「有，而且答案出乎意料地優雅。」叫獸在白板上寫下一行字：**不要分開解，一起解**。「與其先定位再建圖，不如把『我在哪裡』和『地標在哪裡』放進同一個未知數清單，一次全部估計出來。」

他停頓一下，補充了關鍵的一點：「而且這裡有一個很美的機制。當你觀測到一個地標時，你同時得到了兩件事的資訊——**地標在哪裡，以及你自己在哪裡**。它們透過這次觀測綁在一起了。累積夠多的觀測，這些互相牽制的關係就會把整個系統『拉』到一個一致的答案上。」

最後他拿起一條繩子，圍成一個環：「而 SLAM 最戲劇性的時刻，叫做**迴路閉合**。當機器人繞了一大圈，突然認出『這個地方我來過』——這一瞬間，累積了幾百公尺的誤差會被一口氣攤平，整張地圖會**當場扭正**。看過那個畫面的人，都不會忘記。」

本章為什麼重要

SLAM 是移動機器人能夠自主運作的關鍵技術。沒有它，機器人只能在事先人工測繪好的環

境中運作——而人工測繪既昂貴又緩慢，且環境一改變就得重來。

它的應用已經滲透到日常生活：掃地機器人建立家中的地圖、AR 眼鏡定位使用者在房間中的位置、自駕車建立高精地圖、無人機在 GPS 訊號不良處導航、倉儲 AGV 在不斷變動的貨架間穿梭。

但 SLAM 的價值不只在應用，更在於它是一個完美的整合範例。它需要：第 22 章的視覺與感知、第 23 章的運動模型、第 24 章的機率估測、第 8 章的座標變換、以及第 18 章的最佳化思維。要把 SLAM 做好，前面二十四章的知識必須全部到位——這也是為什麼它被安排在這個位置。

本章要建立三個層次的理解。**第一，問題的本質**：為何 SLAM 是雞生蛋問題，以及「一起解」為何可行。**第二，三代主流方法**：EKF-SLAM、粒子濾波 SLAM、以及現代的圖最佳化 SLAM，理解它們各自的洞見與限制。**第三，實務的關鍵**：迴路閉合、資料關聯、與長期運行的挑戰。

學習目標

- 能陳述 SLAM 問題並說明其「雞生蛋」的本質。
- 能說明為何聯合估計比先後求解更有效。
- 能比較三種地圖表示法及其適用場合。
- 能說明 EKF-SLAM 的狀態向量結構與其 $O(n^2)$ 的複雜度來源。
- 能解釋地標之間的相關性為何是 SLAM 收斂的關鍵。
- 能說明 Rao-Blackwell 分解的思想與 FastSLAM 的架構。
- 能說明佔據柵格地圖的更新原理與對數機率的優勢。
- 能說明圖最佳化 SLAM 的節點與邊，並解釋其為何成為現代主流。
- 能說明迴路閉合的作用、偵測方法與誤判的風險。
- 能說明資料關聯問題及其處理方法。
- 能比較視覺 SLAM 與光達 SLAM 的特性與適用場合。
- 能為特定應用選擇合適的 SLAM 方案並說明理由。

先備知識

第 24 章的貝氏濾波、Kalman 濾波器與粒子濾波器（本章直接建立在其上）、第 23 章的運

動模型與里程計、第 22 章的視覺與特徵匹配、第 8 章的座標變換、第 18 章的最佳化與 Bellman 原理。

本章是全書整合度最高的章節之一。建議讀法：若對第 24 章的內容有任何不確定，務必先回頭複習——SLAM 的所有方法都是那些工具的延伸。25.6 節的圖最佳化涉及較多數學，可先掌握其幾何直覺（「把彈簧系統拉到平衡」），細節之後再補。

核心名詞中英對照

表 25-1 本章核心名詞

中文	英文	說明
同步定位與建圖	SLAM	同時估計機器人位姿與環境地圖
完整 SLAM	Full SLAM	估計整條軌跡與地圖
線上 SLAM	Online SLAM	只估計當前位姿與地圖
地標	Landmark	環境中可重複辨識的特徵
特徵地圖	Feature Map	以離散地標點表示的地圖
佔據柵格地圖	Occupancy Grid Map	以格點的佔據機率表示的地圖
拓撲地圖	Topological Map	以節點與連結表示的地圖
資料關聯	Data Association	判斷觀測對應到哪一個已知地標
迴路閉合	Loop Closure	辨認出曾經到過的地方
感知混淆	Perceptual Aliasing	不同地點看起來相同的現象
圖最佳化	Graph Optimization / Pose Graph	以節點與約束邊表示並最佳化的方法
因子圖	Factor Graph	以變數與因子表示機率關係的圖
稀疏性	Sparsity	矩陣中大量元素為零的性質
前端	Front-end	處理感測資料、建立約束的模組
後端	Back-end	求解最佳化問題的模組
對數勝算比	Log-odds	以對數形式表示機率的方法
視覺 SLAM	Visual SLAM, vSLAM	以相機為主要感測的 SLAM
視覺慣性里程計	Visual-Inertial Odometry, VIO	融合視覺與 IMU 的里程計

25.1 SLAM 問題的本質

25.1.1 雞生蛋的循環

如開頭故事所述，SLAM 的核心困難是一個循環依賴：

要建圖 → 必須知道自己在哪（否則不知道把觀測畫在地圖的何處）

要定位 → 必須有地圖（否則沒有參照物）

SLAM 的雞生蛋問題。兩者互為前提。

這個循環之所以無法用「先做一個、再做另一個」的方式打破，是因為兩者的誤差會互相污染。位姿估計錯了，地圖就畫歪；地圖歪了，下次定位又更錯——誤差會正回饋地放大，這正是小翰第一次嘗試失敗的原因。

25.1.2 解法：聯合估計

SLAM 的關鍵洞見是：不要試圖打破循環，而是承認兩者互相依賴，把它們一起估計。

$$p(x_{1:t} \, m \mid z_{1:t} \, u_{1:t})$$

完整 SLAM 的機率表述。同時估計整條軌跡 $x_{1:t}$ 與地圖 m ，條件是所有的觀測 z 與控制 u 。

這個表述有一個變體，只估計當前位姿而非整條軌跡：

$$p(x_t \, m \mid z_{1:t} \, u_{1:t})$$

線上 SLAM。計算量較小，適合即時運行，但無法回頭修正過去的軌跡。

兩者的差別在第 25.6 節會顯得極為重要——迴路閉合需要修正過去的軌跡，因此本質上是完整 SLAM 的問題。這正是現代方法由濾波轉向圖最佳化的原因。

25.1.3 為何聯合估計可行

聯合估計聽起來只是把問題變大（未知數更多了），為何反而可解？關鍵在於觀測同時約束了兩者。

◎ 一次觀測，兩份資訊

當機器人觀測到「地標 A 在我的右前方 3 公尺處」時，它同時得到了兩件事的資訊：若我知道自己在哪，就知道 A 在哪；若我知道 A 在哪，就知道自己在哪。這一次觀測把「我的位姿」與「A 的位置」綁在一起了——兩者的誤差從此相關。累積夠多這樣的約束，整個系統就會被「拉」到一個唯一一致的解上。這正是 SLAM 能夠收斂的根本原因。

用一個生活化的類比：這就像玩拼圖。單看一片拼圖無法確定它的位置，但當你把許多片互相咬合起來，每一片的位置都被鄰居約束，最終整幅圖只有一種拼法。SLAM 中的「咬合」就是

觀測，而「唯一的拼法」就是最佳解。

25.1.4 SLAM 的三個子問題

實務上，SLAM 系統面對三個層次的問題：

- **狀態估計**：給定觀測與控制，求最可能的軌跡與地圖。這是 25.3 ~ 25.6 節的主題。
- **資料關聯**：這次看到的地標，是之前哪一個？還是全新的？這是 25.8 節的主題，也是實務上最容易出錯的環節。
- **迴路閉合**：機器人是否回到了曾經到過的地方？這是 25.7 節的主題，也是消除長期漂移的唯一手段。

三者的難度並不對稱。狀態估計有成熟的數學工具，反而是資料關聯與迴路閉合才是真正的難點——因為它們涉及「辨識」而非「計算」，而辨識必然有錯誤的可能。

25.2 地圖的表示

25.2.1 三種主流表示法

在討論演算法之前，必須先決定「地圖長什麼樣」。這個選擇會深刻影響後續的所有設計。

表示法一：特徵地圖 (Feature Map)。地圖是一組離散的地標點，每個地標有其座標（可能還有描述子）。例如「牆角在 (3.2, 5.1)」、「柱子在 (7.8, 2.4)」。

- **優點**：資料量小、與 EKF 天然契合（每個地標就是狀態向量中的兩三個元素）、易於做資料關聯。
- **缺點**：需要能穩定偵測的特徵；無法表示「哪裡是空的」（這對路徑規劃很重要）；不規則的環境難以用點表示。

表示法二：佔據柵格地圖 (Occupancy Grid Map)。把空間切成規則的格子，每格記錄「被佔據的機率」。這是掃地機器人與 ROS 導航最常用的表示法。

- **優點**：能表示任意形狀、明確區分「佔據 / 空曠 / 未知」三種狀態、直接可用於路徑規劃（第 26 章）。
- **缺點**：記憶體隨面積與解析度平方成長（三維時是立方）、不含語意資訊。

表示法三：拓撲地圖 (Topological Map)。只記錄「地點」與「地點之間的連結」，不含

精確的幾何。例如「房間 A 連到走廊 B、走廊 B 連到房間 C」。

- **優點**：極為精簡、適合大範圍導航與任務規劃、接近人類的空間認知方式。
- **缺點**：無法做精確定位與避障，通常需與其他表示法混用。

表 25-2 三種地圖表示法的比較

面向	特徵地圖	佔據柵格地圖	拓撲地圖
表示內容	離散地標點	每格的佔據機率	地點與連結關係
記憶體需求	小 (正比於地標數)	大 (正比於面積 $\times$ 解析度 ²)	極小
能否表示空曠區	否	是	否
可直接用於路徑規劃	否	是	部分 (高層規劃)
適用的估測方法	EKF-SLAM	粒子濾波 SLAM	圖 SLAM 的高層
典型應用	早期 SLAM、稀疏視覺 SLAM	掃地機器人、室內 AGV	大範圍導航、語意地圖

25.2.2 現代的混合表示

實務系統往往同時維護多種表示。一個典型的架構是：用特徵 (或關鍵影格) 做定位與最佳化、用佔據柵格做路徑規劃、用拓撲層做高階任務決策。

近年還興起了兩種新的表示：語意地圖 (除幾何外還標註「這是門、這是桌子」，為第 32 章的任務規劃提供基礎) 與神經隱式表示 (如 NeRF，用一個神經網路隱含地表示整個場景的幾何與外觀)。後者是目前的研究前沿，但尚未在工業上普及。

25.3 EKF-SLAM

25.3.1 把地圖放進狀態向量

EKF-SLAM 是最早的完整解法，其思想直接體現了 25.1.2 節的「一起估計」：把所有地標的座標，都放進 Kalman 濾波器的狀態向量中。

$$\mathbf{x} = [x_r, y_r, \theta_r, m_{1x}, m_{1y}, m_{2x}, m_{2y}, \dots, m_{Nx}, m_{Ny}]^T$$

EKF-SLAM 的狀態向量。前三個是機器人位姿，其餘是 N 個地標的座標。維度為 $3 + 2N$ 。

對應的協方差矩陣則是 $(3+2N) \times (3+2N)$ 的方陣。它的結構極具啟發性：

$$\Sigma = \begin{bmatrix} \Sigma_{rr} & \Sigma_{rm} \\ \Sigma_{mr}^T & \Sigma_{mm} \end{bmatrix}$$

分塊結構。 Σ_{rr} 是機器人位姿的不確定性、 Σ_{mm} 是地標的不確定性、 Σ_{rm} 是兩者之間的相關性。

25.3.2 相關性：SLAM 收斂的關鍵

那個非對角區塊 Σ_{rm} 是整個 EKF-SLAM 最重要的部分。它記錄了「機器人位姿的誤差」與「地標位置的誤差」之間的**相關性**。

為何相關性如此重要？考慮一個具體情境：

1. 機器人在某處觀測到地標 A，此時它自己的位置有 10 公分的誤差，因此 A 的估計位置也有約 10 公分的誤差——**而且是同方向的誤差**（因為源自同一個位姿誤差）。
2. 機器人移動並累積更多誤差，現在自己的位置誤差變成 30 公分。
3. 此時它再次觀測到 A。由於 A 的位置與機器人當時的位姿相關，這次觀測**同時修正了兩者**——機器人的位置誤差降回接近 10 公分。

若沒有記錄**相關性**（把 Σ_{rm} 當成零），系統會誤以為 A 的位置是獨立已知的，於是過度自信地修正，導致估計不一致甚至發散。這是 SLAM 早期研究的一個重要教訓。

◎ SLAM 的收斂定理

Dissanayake 等人於 2001 年證明了 EKF-SLAM 的三個重要性質：一、**地圖的不確定性單調遞減**——隨著觀測累積，地標的位置估計只會越來越準。二、**地標之間的相關性會趨於完全相關**——所有地標最終形成一個剛性的「骨架」，它們的相對位置極為精確，只是整體相對於世界座標系仍有不確定性。三、**地圖不確定性的下界，由第一次觀測時機器人的位姿不確定性決定**。這三個性質給出了 SLAM 為何可行的嚴格保證。

25.3.3 $O(n^2)$ 的複雜度困境

EKF-SLAM 有一個致命的實務問題：**協方差矩陣的更新是 $O(n^2)$ 的複雜度**（ n 為地標數）。

原因是：每一次觀測，即使只看到一個地標，**整個協方差矩陣的所有元素都必須更新**——因為所有地標之間都有相關性。這使得：

- **100 個地標**：矩陣有 $203 \times 203 \approx 41000$ 個元素，尚可即時處理。
- **1000 個地標**： $2003 \times 2003 \approx 400$ 萬個元素，每次更新需要數百毫秒——已經無法即時。
- **10000 個地標**：4 億個元素，完全不可行。

這個限制使 EKF-SLAM 只適用於**小規模、地標稀疏**的環境。它在教學上仍極有價值（結構

清晰、直接體現 SLAM 的本質)，但實務上已被後續方法取代。

此外它還繼承了 EKF 的所有缺點（第 24.6.2 節）：線性化誤差、只能表示單峰分布、且資料關聯錯誤無法修正——一旦把兩個不同的地標誤認為同一個，錯誤會永久留在協方差矩陣中。

25.4 粒子濾波 SLAM 與 FastSLAM

25.4.1 Rao-Blackwell 分解

FastSLAM 的突破來自一個巧妙的觀察，稱為 **Rao-Blackwell 分解**：

◎ 關鍵洞見

如果已經知道機器人的完整軌跡，那麼各個地標的位置就互相獨立了。因為地標之間的相關性，完全源自於「共用了不確定的機器人位姿」——一旦軌跡確定，這個共同原因就消失了。

這個洞見允許把聯合分布分解為兩部分：

$$p(x_{1:t}, m \mid z, u) = p(x_{1:t} \mid z, u) \cdot \prod_i p(m_i \mid x_{1:t}, z)$$

Rao-Blackwell 分解。第一項用粒子濾波器處理軌跡、第二項的每個地標用一個獨立的小型 EKF 處理。

25.4.2 FastSLAM 的架構

由上述分解，FastSLAM 的結構為：

- 每個粒子代表一條完整的軌跡假設（而非只是當前位姿）。
- 每個粒子攜帶自己的一整張地圖——由 N 個獨立的 2×2 小型 EKF 構成（每個地標一個）。
- 權重由觀測與該粒子地圖的吻合程度決定。

這個架構的三個關鍵優勢：

優勢一：複雜度降為 $O(M \log N)$ （ M 為粒子數、 N 為地標數）。因為每個地標只需維護 2×2 的小矩陣，不再有巨大的協方差矩陣。

優勢二：能處理資料關聯的不確定性。不同的粒子可以做出不同的資料關聯決定——若某個粒子關聯錯了，它的地圖會與後續觀測不符，權重下降而被淘汰。錯誤會被自動篩除，這是 EKF-SLAM 做不到的。

優勢三：能表示多峰分布，繼承了粒子濾波器處理全域定位的能力。

25.4.3 FastSLAM 的限制

FastSLAM 的主要問題是**粒子退化**（第 24.7.3 節）——而在 SLAM 中，這個問題特別嚴重。

原因是：重取樣會淘汰粒子，而**被淘汰的粒子連同它的整條軌跡歷史一起消失**。經過長時間運行後，所有粒子可能都源自同一個祖先，代表「過去的軌跡只剩一種假設」——此時系統已無法修正過去的錯誤。這使 FastSLAM 在**大迴路閉合**時表現不佳。

儘管如此，FastSLAM 的一個變體在實務上極為成功：**GMapping**，它是 ROS 中最廣泛使用的二維光達 SLAM 套件之一，採用佔據柵格地圖而非特徵地圖。它的成功關鍵在於一個改良——用當前的光達掃描來改善提議分布（proposal distribution），使粒子更集中在正確的位置附近，大幅減少所需的粒子數。

表 25-3 EKF-SLAM 與 FastSLAM 的比較

面向	EKF-SLAM	FastSLAM
核心思想	地圖進狀態向量、聯合估計	Rao-Blackwell 分解、軌跡與地圖分離
軌跡表示	只有當前位姿（線上 SLAM）	整條軌跡（每個粒子一條）
地圖表示	單一地圖	每個粒子一張地圖
複雜度	$O(N^2)$	$O(M \log N)$
能否多峰	否	是
資料關聯錯誤	無法修正	可被篩除
主要限制	地標數受限（< 幾百）	粒子退化、大迴路表現差
代表實作	教學與早期研究	GMapping（ROS）

25.5 佔據柵格地圖

25.5.1 每一格的機率

佔據柵格地圖把空間切成規則的格子（二維常用 5 公分或 10 公分），每格記錄一個機率值：這格被障礙物佔據的機率。

與特徵地圖相比，它有一個關鍵優勢：**能明確區分三種狀態**——已知佔據（機率接近 1）、已知空曠（接近 0）、未探索（0.5）。第三種狀態對自主探索極為重要，因為機器人需要知道「哪裡還沒去過」。

地圖更新的基本假設是：**各格互相獨立**。這在物理上並不完全正確（相鄰的格子通常屬於同一面牆），但它使計算大幅簡化，且實務效果良好。

$$p(m \mid z_{1:t} x_{1:t}) = \prod_i p(m_i \mid z_{1:t} x_{1:t})$$

獨立性假設。每一格可以單獨更新，不需考慮鄰居。

25.5.2 對數勝算比

直接以機率值更新會遇到兩個問題：反覆相乘導致數值下溢、以及每次更新都需要正規化。標準的解法是改用**對數勝算比**（log-odds）：

$$l(m) = \log[p(m)/(1 - p(m))]$$

對數勝算比。機率 0.5 對應 $l = 0$ ；機率趨近 1 時 $l \rightarrow +\infty$ ；趨近 0 時 $l \rightarrow -\infty$ 。

這個轉換的威力在於：**貝氏更新從乘法變成加法**。

$$l_t = l_{t-1} + l_{\text{sensor}} - l_0$$

更新規則。 l_{sensor} 是本次觀測提供的證據、 l_0 是先驗（通常為 0）。只需要加減法。

三個實務好處：一、**計算極快**（只有加減）；二、**不會數值下溢**（不會出現 0.0001×0.0001 這種問題）；三、**天然實現了「記憶」**——同一格被反覆觀測到是障礙物， l 就持續累加，越來越確定。

25.5.3 感測器逆模型

那麼 l_{sensor} 該取多少？這由**感測器逆模型**（inverse sensor model）決定。以光達的一條射線為例：

- **射線經過的格子**（在量測距離之前）：這些格子應該是空的，給予負的 l_{sensor} （如 -0.4 ）。
- **射線終點的格子**：這裡有障礙物，給予正的 l_{sensor} （如 $+0.85$ ）。
- **射線終點之後的格子**：被遮擋，無資訊， $l_{\text{sensor}} = 0$ （不更新）。

這個「沿射線標記空曠、終點標記佔據」的程序稱為**射線追蹤**（ray casting），實作上常用 Bresenham 演算法（原本用於電腦繪圖畫直線）高效完成。

兩個實務細節：一、 l 應該設定上下限（如 ± 5 ），否則長時間觀測後某格會變得「無限確定」，導致環境改變時無法更新——這對動態環境（有人走動、傢俱移動）極為重要。二、佔據

的證據 (+0.85) 通常大於空曠的證據 (-0.4)，因為「打到東西」比「沒打到」更可靠。

圖 25-1 佔據柵格地圖的射線更新

建議配圖：俯視圖。機器人位於左下，一條光達射線射向右上方並在某處打到牆面。射線經過的格子以淺藍色標示 (l 減少，趨向空曠)、終點的格子以深紅色標示 (l 增加，趨向佔據)、終點之後的格子以灰色標示 (被遮擋，不更新)。右側附一張對數勝算比的數線圖，標出 $l = 0$ (未知， $p = 0.5$)、 $l = +5$ (確定佔據)、 $l = -5$ (確定空曠)，並以箭頭表示每次觀測如何推移 l 值。下方另附三格小圖，顯示同一區域被觀測 1 次、5 次、20 次後地圖的清晰度變化。

25.6 圖最佳化 SLAM

25.6.1 觀念的轉向

EKF-SLAM 與 FastSLAM 都是濾波的思路：維護一個當前的信念，隨時間遞迴更新，過去的資料用完就丟。這個作法有一個根本的限制——無法回頭修正過去的軌跡。

但迴路閉合恰恰需要這個能力。當機器人繞了一大圈後認出「我回到起點了」，它需要把整條軌跡都調整一遍，才能消除累積的誤差。

因此現代 SLAM 轉向了完全不同的思路：把所有歷史資料都保留下來，當成一個大型的最佳化問題來解。這稱為圖最佳化 SLAM (Graph-based SLAM)，也稱位姿圖 (pose graph) 最佳化。

25.6.2 節點與邊

圖最佳化把 SLAM 表示成一個圖：

- 節點 (Node)：待估計的變數。通常是機器人在各個時刻的位姿 (有時也包含地標的位置)。
- 邊 (Edge)：兩個節點之間的約束，來自某次觀測。每條邊記錄「這兩個節點之間的相對位姿應該是多少」以及「這個約束有多可信」(資訊矩陣)。

邊有兩種來源：

里程計邊：相鄰時刻之間的約束，來自輪速或 IMU。這些邊構成一條鏈，且必定存在。

迴路閉合邊：不相鄰的兩個時刻之間的約束，來自「認出這個地方我來過」。這些邊把鏈接成環，是消除漂移的關鍵。

25.6.3 彈簧系統的直觀理解

圖最佳化有一個極為直觀的物理類比：

◎ 彈簧系統的類比

把每個節點想成一顆珠子、每條邊想成一根彈簧。彈簧的自然長度就是觀測所預期的相對位姿；彈簧的勁度就是該觀測的可信度（資訊矩陣）。若把珠子按里程計的估計擺放，所有彈簧都會被拉伸或壓縮——系統處於高能量狀態。**最佳化的過程，就是放手讓這個彈簧系統自己找到平衡。**當總彈性位能最小時，就是最一致的軌跡與地圖。

這個類比也解釋了迴路閉合的戲劇性效果：加入一條迴路閉合邊，等於在環的兩端接上一根強力彈簧。系統為了平衡這根新彈簧，會把整個環「拉正」——累積的誤差被均勻地攤到整條軌跡上。這正是開頭故事中「地圖當場扭正」的物理解釋。

25.6.4 數學形式

形式化地說，圖最佳化求解下列問題：

$$\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x}} \sum_{(i,j) \in E} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j)^T \Omega_{ij} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j)$$

圖最佳化的目標函數。 $\mathbf{e}_{ij}$ 為第 (i,j) 條邊的誤差（預期的相對位姿與實際估計的差）、 Ω_{ij} 為資訊矩陣（協方差的反矩陣）。

這是一個非線性最小平方問題，標準解法是 Gauss-Newton 或 Levenberg-Marquardt 迭代——與第 22.3.2 節的相機校正、第 10.7 節的數值逆運動學是同一類方法。

與第 18 章的關聯：這也是一個最佳化問題，但性質不同——第 18 章求的是「未來的最佳控制序列」，這裡求的是「過去的最一致解釋」。前者往前看、後者往回看，但數學工具相通。

25.6.5 稀疏性：可解的關鍵

圖最佳化的問題規模可能極大——一趟數公里的行程可能有數萬個節點。為何仍然可解？

關鍵在於**稀疏性**：每個節點只與少數幾個節點有邊相連（相鄰時刻加上少數迴路閉合）。這使得問題的**資訊矩陣（Hessian）是稀疏的**——絕大多數元素為零。

利用稀疏性，可以用專門的線性代數方法（如稀疏 Cholesky 分解）高效求解。一個包含數萬個節點的位姿圖，在現代電腦上通常能在數秒內最佳化完成。

這正是圖最佳化能取代濾波方法的根本原因：EKF-SLAM 的協方差矩陣是**稠密的**（所有地標互相相關，第 25.3.2 節），因此是 $O(n^2)$ ；而圖最佳化的資訊矩陣是**稀疏的**，因此能處理大得多

的問題。

◎ 稀疏與稠密的深層原因

為何濾波是稠密的、圖最佳化是稀疏的？關鍵在於是否邊緣化 (marginalize) 過去的變數。濾波法為了維持固定的狀態維度，會把過去的位姿「積掉」——而這個積分操作會在剩餘的變數之間產生新的相關性，使矩陣變稠密。圖最佳化則保留所有變數不做邊緣化，因此原始的稀疏結構得以保存。用更多的記憶體，換取更低的計算複雜度與回頭修正的能力——這是一個極為划算的交易，也是現代 SLAM 的主流選擇。

25.6.6 前端與後端

現代 SLAM 系統普遍採用**前端—後端**的架構分工：

前端 (Front-end)：處理感測資料，建立圖的節點與邊。包含特徵擷取、掃描匹配、資料關聯、迴路閉合偵測。這部分與感測器類型高度相關（光達與相機的前端完全不同）。

後端 (Back-end)：接收前端建立的圖，求解最佳化問題。這部分與感測器無關，是純粹的數學問題。

這個分工的價值在於**模組化**：後端可以是通用的（如 g2o、GTSAM、Ceres 等函式庫），前端則針對不同感測器分別開發。這使得研究者能專注在自己的領域，而不需重複實作最佳化器。

25.7 迴路閉合

25.7.1 為何如此關鍵

迴路閉合 (loop closure) 是 SLAM 中最戲劇性、也最關鍵的機制。它的作用是：**辨認出「這個地方我曾經來過」**，並據此建立一條連接遙遠時刻的約束邊。

沒有迴路閉合，SLAM 就退化為「里程計 + 局部建圖」——誤差仍會無止境地累積。**迴路閉合是消除長期漂移的唯一手段。**

它的效果極為顯著。一個典型的例子：機器人繞行一棟建築物一圈（約 200 公尺），回到起點時里程計的累積誤差可能達 5 ~ 10 公尺。一旦成功偵測到迴路閉合並最佳化，**整條軌跡的誤差會降到數十公分**——改善了一到兩個數量級。

25.7.2 偵測方法

如何判斷「我來過這裡」？主要有三類方法。

方法一：幾何驗證。若當前的估計位姿與某個歷史位姿接近，就嘗試把兩處的感測資料做匹配（如光達的掃描匹配）。若匹配成功且誤差小，就認定為迴路閉合。

- **優點：**可靠、能直接得到精確的相對位姿。
- **缺點：**依賴當前的位姿估計——若漂移已經很大，機器人根本不知道自己回到了附近，就不會去嘗試匹配。這是一個雞生蛋的小循環。

方法二：外觀比對（Appearance-based）。完全不依賴位姿，純粹比較「現在看到的景象」與「歷史上看過的景象」是否相似。

最著名的技術是**詞袋模型**（Bag of Words, BoW）：把影像的特徵描述子量化成「視覺詞彙」，用詞彙的分布來描述整張影像。比對時只需比較詞彙分布的相似度，速度極快（毫秒級即可搜尋數萬張歷史影像）。

- **優點：**不依賴位姿估計、能從嚴重漂移中恢復、計算高效。
- **缺點：**可能被**感知混淆**（見下節）欺騙、對光線與視角變化敏感。

方法三：學習式方法。用深度神經網路直接學習「場景描述子」，使同一地點在不同光線、季節、視角下也能匹配。這是近年的研究重點（如 NetVLAD），對長期運行的機器人特別有價值。

25.7.3 感知混淆與誤判的災難

迴路閉合最大的風險是**誤判**。而誤判的主因是**感知混淆**（perceptual aliasing）——不同的地點看起來一模一樣。

典型的例子：辦公大樓中每一層的走廊完全相同、旅館的房門排列一致、倉庫中一模一樣的貨架、對稱的建築物。

◎ 一次錯誤的迴路閉合，可能毀掉整張地圖

這是 SLAM 中最嚴重的失效模式。若系統誤認「三樓的走廊」是「二樓的走廊」，就會加入一條錯誤的強約束邊。最佳化器會忠實地執行它的任務——把兩個實際上相隔一層樓的位置「拉」到一起。結果是整張地圖被折疊、扭曲，且**這個錯誤無法自動修復**（因為那條錯誤的邊仍然存在，且它的「彈簧」很強）。相較之下，漏掉一次迴路閉合只是損失一次改善的機會，代價小得多。因此實務上的原則是：**寧可漏判，不可誤判**。

三個標準的防護機制：

- **幾何驗證**：外觀相似只是候選，必須再用幾何匹配確認（如檢查特徵點的空間分布是否一致）。這能過濾掉大部分的感知混淆。
- **時間一致性**：要求連續數個時刻都偵測到同一個迴路，才予以確認。孤立的單次匹配不採信。
- **強健核函數 (Robust Kernel)**：在最佳化的目標函數中，對誤差很大的邊自動降低其權重（如 Huber 或 Cauchy 核）。這使得少數錯誤的邊不會主導整個結果——這是後端的最後一道防線。

25.8 資料關聯

25.8.1 被低估的難題

資料關聯 (data association) 問的是：這次觀測到的特徵，對應到地圖中的哪一個地標？還是一個全新的地標？

這個問題在教科書中常被輕描淡寫（多數推導直接假設「已知關聯」），但在實務上它才是 **SLAM 最容易失敗的環節**。原因是：狀態估計有嚴謹的數學保證，而資料關聯本質上是一個**辨識**問題——必然有出錯的可能，且錯誤會污染後續的所有估計。

25.8.2 三種處理方法

方法一：最近鄰 (Nearest Neighbor)。把觀測關聯到預測位置最接近的地標。判斷時使用**馬氏距離**（第 24.9.3 節）而非歐氏距離，因為它考慮了不確定性。

關聯到地標 j ，若 $d^2_{ij} = \mathbf{v}^T \mathbf{S}^{-1} \mathbf{v}$ 最小且低於門檻

最近鄰關聯。簡單快速，但在地標密集時容易出錯。

方法二：聯合相容性 (Joint Compatibility)。不逐一關聯，而是考慮**所有觀測的整體一致性**。例如同時看到三個地標時，檢查這三個關聯組合起來是否幾何上合理。

這顯著提升了可靠度——即使個別的關聯有歧義，整體的幾何約束往往能唯一確定正確的組合。代價是計算量增加。

方法三：多重假設 (Multi-hypothesis)。不做決定，而是同時保留多種可能的關聯，讓

後續的觀測來判別。FastSLAM 的每個粒子可以做不同的關聯決定（第 25.4.2 節），正是這個思想的實現。

25.8.3 實務原則

三個經驗法則：

- **寧可新增，不可誤配。**若不確定觀測是否對應到已知地標，寧可把它當成新地標。多一個冗餘的地標只是浪費一點記憶體；誤配則會產生錯誤的約束，破壞整張地圖。
- **用描述子而非只用位置。**視覺 SLAM 中，特徵的外觀描述子提供了位置之外的判別資訊，能大幅降低誤配率。
- **設定嚴格的門檻。**馬氏距離的門檻應偏保守（如卡方 99% 而非 95%），寧可多產生一些新地標。

25.9 視覺 SLAM 與光達 SLAM

25.9.1 兩條技術路線

依主要感測器的不同，SLAM 分為兩大陣營，各有其生態系與適用場合。

光達 SLAM (LiDAR SLAM)：以雷射掃描為主要輸入。其核心操作是**掃描匹配**（scan matching）——把當前的掃描與已有的地圖（或前一次掃描）對齊，求出相對位姿。常用的演算法是 ICP（Iterative Closest Point）與其變體 NDT。

視覺 SLAM (Visual SLAM)：以相機為主要輸入。其核心操作是**特徵匹配與三角測量**——在連續影像間追蹤特徵點，由視差恢復三維結構與相機運動（第 22.4.2 節）。

表 25-4 光達 SLAM 與視覺 SLAM 的比較

面向	光達 SLAM	視覺 SLAM
感測器成本	高（數萬至數十萬元）	低（數百至數千元）
直接取得深度	是	否（需三角測量或深度相機）
對光線的依賴	無（主動測距）	高（黑暗中失效）
對紋理的依賴	無	高（白牆上失效）
資訊豐富度	低（只有幾何）	高（顏色、紋理、語意）
迴路閉合能力	較弱（幾何易混淆）	強（外觀描述子）

面向	光達 SLAM	視覺 SLAM
尺度	絕對尺度	單目有尺度歧義；雙目 / RGB-D 無此問題
典型應用	室內 AGV、自駕車、測繪	AR/VR、無人機、消費機器人

25.9.2 視覺慣性里程計 (VIO)

純視覺 SLAM 有三個弱點：快速運動時影像模糊、單目有尺度歧義、紋理不足時失效。而 IMU 恰好能補足這三點——它更新率極高（1000 Hz）、能提供絕對尺度（透過重力加速度）、且完全不依賴環境外觀。

視覺慣性里程計（Visual-Inertial Odometry, VIO）融合兩者，是目前 AR 眼鏡、消費級無人機與掃地機器人的主流方案。其互補性極為漂亮：

視覺：低頻、不漂移、需要紋理與光線

IMU：高頻、會漂移、完全不依賴環境

兩者的誤差特性恰好互補——這正是第 24.8 節互補濾波思想的高階版本。

VIO 的實作有兩種路線：濾波式（如 MSCKF，用 EKF 融合，計算量小、適合嵌入式）與最佳化式（如 VINS-Mono、OKVIS，用滑動視窗的圖最佳化，精度較高但計算量大）。近年後者逐漸成為主流，因為行動裝置的運算能力已足夠。

25.9.3 多感測器融合的趨勢

現代的高階系統普遍融合多種感測器，取各家之長：

- **光達 + IMU + 輪速**：自駕車與工業 AGV 的標準組合。光達提供精確幾何、IMU 補足高頻運動、輪速提供尺度與冗餘。
- **相機 + IMU**：消費級裝置的主流（成本考量），如 AR 眼鏡、掃地機器人。
- **光達 + 相機 + IMU + GNSS**：戶外自駕車的完整配置。GNSS 提供全域參照以消除長期漂移。

融合的原則與第 24.10 節相同：用高頻但漂移的感測器提供平滑的短期估計、用不漂移的感測器定期校正。SLAM 只是把這個原則應用到「地圖」這個特殊的參照上。

25.10 實務挑戰與現代發展

25.10.1 動態環境

經典的 SLAM 假設環境是靜態的。但真實世界充滿移動的人、車、被搬動的傢俱。

動態物體會造成兩個問題：一、**被誤認為地標**，一旦它移動，地圖就與現實不符；二、**干擾掃描匹配**，使位姿估計偏移。

三種因應策略：

- **偵測並剔除**：用語意分割（第 22.8 節）辨識出人、車等動態類別，直接排除其上的特徵。這是目前最有效的作法。
- **統計剔除**：多次觀測後，若某區域的佔據狀態反覆變化，就標記為不可靠。佔據柵格的 I 值上下限（第 25.5.3 節）正是這個機制的簡單版本。
- **明確建模**：把動態物體也納入估計（追蹤它們的運動）。這是 SLAM 與多目標追蹤的結合，複雜度高但資訊最完整。

25.10.2 長期運行與地圖維護

若機器人需要在同一環境中運行數月甚至數年，會遇到「**長期 SLAM**」的挑戰：

挑戰一：外觀變化。同一地點在白天與夜晚、夏天與冬天、燈開與燈關時看起來截然不同。這使外觀式的迴路閉合失效。

挑戰二：結構變化。傢俱被搬動、貨架被重新排列、施工改變了通道。地圖必須能更新，但又不能因為暫時的變化就把長期正確的部分抹掉。

挑戰三：地圖膨脹。持續累積資料會使地圖無限增長。需要機制來**遺忘**——刪除過時的資訊、合併重複的節點。

這些挑戰目前仍是研究前沿。實務上的折衷作法包括：定期重新建圖、維護多張不同時段的地圖、或採用「經驗地圖」（Experience Maps）的架構——為同一地點保留多個不同外觀的版本。

25.10.3 學習與 SLAM 的結合

深度學習正在改變 SLAM 的多個環節，但方式與想像的不同——它**主要強化了前端**，而非取代後端。

表 25-5 深度學習在 SLAM 中的角色

環節	傳統方法	學習方法的貢獻	是否已成熟
特徵擷取	SIFT、ORB 等手工設計	SuperPoint 等學習式特徵，對光線變化更穩健	接近成熟
特徵匹配	描述子最近鄰	SuperGlue 等，考慮全域一致性	接近成熟
迴路閉合	詞袋模型	NetVLAD 等場景描述子，跨季節仍可匹配	接近成熟
動態物體剔除	統計方法	語意分割直接辨識人車	已成熟
深度估計	立體匹配	單目深度預測網路	部分成熟
位姿估計	幾何最佳化	端到端網路直接回歸位姿	尚不成熟
地圖表示	點雲、柵格	神經隱式表示 (NeRF)	研究前沿

一個重要的觀察：學習方法在「辨識」類的任務上（特徵、匹配、迴路閉合、語意）表現優異，但在「幾何最佳化」上仍不如傳統方法。原因是幾何有嚴謹的數學結構，而神經網路難以保證滿足這些約束。

因此目前的共識是：學習做前端、幾何做後端。這與第 21.9.5 節「學習產生候選、解析驗證品質」的混合式架構、第 17 章「模型處理已知、回授處理未知」的哲學，是同一種思維的不同體現。

25.11 由建圖邁向規劃

25.11.1 本章的成果

本章解開了第 24 章結尾拋出的雞生蛋難題。你現在能夠：陳述 SLAM 問題並理解聯合估計為何可行、比較三種地圖表示法、說明 EKF-SLAM 的結構與其 $O(n^2)$ 困境、理解 Rao-Blackwell 分解與 FastSLAM、實作佔據柵格地圖的對數勝算比更新、理解圖最佳化的彈簧類比與稀疏性優勢、以及處理迴路閉合與資料關聯這兩個實務難點。

最重要的是三個核心洞見：一、一次觀測同時約束位姿與地標——這是 SLAM 可解的根本原因。二、迴路閉合是消除長期漂移的唯一手段，但寧可漏判不可誤判。三、不做邊緣化就能保持稀疏——這是圖最佳化取代濾波法的關鍵。

25.11.2 接下來：知道在哪裡，然後呢

經過第 22 ~ 25 章，機器人終於具備了完整的空間認知能力：它能看見環境（第 22 章）、

能移動（第 23 章）、能估計自己的狀態（第 24 章）、並能在未知環境中建立地圖並定位（第 25 章）。

但知道自己在哪裡，並不等於知道該怎麼走。給定一張地圖與一個目標，機器人必須規劃出一條路徑——這條路徑必須避開障礙、符合運動約束（第 23 章的非完整性）、且盡可能地短或快。

第 26 章（路徑規劃與導航）將處理這個問題。我們會學到：位形空間這個把「機器人的形狀」轉化為「一個點」的關鍵抽象、 A^* 等圖搜尋演算法、RRT 等取樣式方法（處理高維與非完整約束）、人工勢場與動態視窗法等即時避障方法、以及完整導航系統的分層架構——全域規劃負責大方向、局部規劃負責即時避障。

屆時你會發現，第 25 章的佔據柵格地圖正是路徑規劃最直接的輸入，而第 23 章的運動約束則決定了哪些路徑真正可行。

換句話說：第 25 章讓機器人知道世界長什麼樣、自己在哪裡，第 26 章讓它決定該往哪裡走。

完整計算例題

例題 25-1 EKF-SLAM 的狀態維度與複雜度

題目：某 EKF-SLAM 系統在二維環境中運行，機器人位姿為 (x, y, θ) ，每個地標以 (x, y) 表示。（a）當地圖中有 50 個地標時，求狀態向量的維度與協方差矩陣的元素數。（b）當地標增至 500 個時，重新計算。（c）若協方差更新的運算量正比於元素數，計算（b）相對於（a）的運算量倍數。（d）若（a）的更新耗時 2 毫秒，估算（b）的耗時並判斷能否以 20 Hz 即時運行。

解（a）：狀態維度為 $3 + 2N$ ：

$$\dim = 3 + 2 \times 50 = 103$$

狀態向量有 103 個元素。

$$\text{協方差元素數} = 103^2 = 10609$$

約一萬個元素。

解（b）：

$$\text{dim} = 3 + 2 \times 500 = 1003 \quad \text{協方差元素數} = 1003^2 = 1006009$$

約一百萬個元素。

解 (c) :

$$\text{倍數} = 1006009/10609 \approx 94.8 \text{ 倍}$$

地標數增加 10 倍，運算量增加約 95 倍——這正是 $O(N^2)$ 的體現。

解 (d) :

$$\text{耗時} \approx 2 \text{ ms} \times 94.8 = 189.6 \text{ ms}$$

近 190 毫秒。

20 Hz 即時運行要求每次更新在 50 毫秒內完成。 $189.6 \text{ ms} > 50 \text{ ms}$ ，無法即時運行。

工程結論：EKF-SLAM 在此配置下，地標數的上限約為 250 個左右（此時耗時約 50 ms）。這印證了 25.3.3 節的說法——它只適用於小規模環境。若環境需要更多地標，必須改用 FastSLAM ($O(M \log N)$) 或圖最佳化（利用稀疏性）。

例題 25-2 佔據柵格的對數勝算比更新

題目：某佔據柵格地圖使用對數勝算比更新。感測器逆模型設定為：偵測到障礙物時 $l_{\text{occ}} = +0.85$ 、偵測為空曠時 $l_{\text{free}} = -0.40$ ，先驗 $l_0 = 0$ （機率 0.5）。(a) 某格連續三次被偵測為障礙物，求其 l 值與對應的機率。(b) 接著兩次被偵測為空曠，重新計算。(c) 若 l 的上下限設為 ± 5.0 ，求需要連續幾次偵測為障礙物才會達到上限。(d) 討論設定上下限的必要性。

解 (a)：三次累加：

$$l = 0 + 3 \times 0.85 = 2.55$$

對數勝算比。

換算回機率（由 $l = \log[p/(1-p)]$ 反解）：

$$p = 1/(1 + e^{(-l)}) = 1/(1 + e^{(-2.55)}) = 1/(1 + 0.0781) = 0.9276$$

佔據機率約 92.8%——已相當確定此處有障礙物。

解 (b)：再累加兩次空曠的證據：

$$l = 2.55 + 2 \times (-0.40) = 2.55 - 0.80 = 1.75$$

對數勝算比下降。

$$p = 1/(1 + e^{(-1.75)}) = 1/(1 + 0.1738) = 0.8519$$

機率降至約 85.2%。

觀察：兩次「空曠」的證據，只把機率由 92.8% 降到 85.2%——這是因為 I_{occ} (0.85) 大於 $|I_{free}|$ (0.40)。這個不對稱的設定反映了「打到東西比沒打到更可靠」的物理事實 (第 25.5.3 節)。

解 (c)：由 $I_0 = 0$ 開始，每次 +0.85：

$$n \times 0.85 \geq 5.0 \Rightarrow n \geq 5.88 \Rightarrow n = 6 \text{ 次}$$

需要連續 6 次偵測為障礙物才會達到上限。

此時的機率為 $p = 1/(1 + e^{(-5.0)}) = 0.9933$ ，約 99.3%。

解 (d)：上下限是必要的，理由有三：

- **允許環境改變：**若沒有上限，某格被觀測一千次為障礙物後， I 會達到 850——此時即使障礙物被移走，需要超過兩千次的空曠觀測才能翻轉。這使地圖無法適應動態環境。
- **避免數值溢位：**長時間運行後 I 會無限增長。
- **限制單一錯誤的影響：**若感測器暫時故障產生大量錯誤讀值，上下限能限制其破壞程度。

以本例的上限 5.0 而言，一個「確定的障礙物」只需約 13 次空曠觀測 ($5.0/0.40 = 12.5$) 就能被翻轉為空曠——這是一個合理的反應速度。

例題 25-3 迴路閉合的效果估算

題目：某機器人沿一條封閉路徑行走 300 公尺後回到起點。里程計的航向誤差率為每公尺 0.05 度 (已校正後的水準)。(a) 估算回到起點時的累積航向誤差。(b) 估算終點的位置誤差 (假設誤差均勻累積)。(c) 若成功偵測到迴路閉合並執行圖最佳化，估算誤差如何分配。(d) 若迴路閉合誤判 (把起點誤認為路徑中點)，討論後果。

解 (a)：累積航向誤差：

$$\theta_{\text{error}} = 0.05^\circ/\text{m} \times 300 \text{ m} = 15^\circ$$

回到起點時，機器人的朝向估計偏差達 15 度。

解 (b)：位置誤差的估算較複雜，但可用一個簡化模型：若航向誤差線性累積，則位置誤

差約為「路徑長度 × 平均航向誤差的正弦」：

$$\text{平均航向誤差} = 15^\circ / 2 = 7.5^\circ$$

$$\text{位置誤差} \approx 300 \times \sin(7.5^\circ) \approx 300 \times 0.1305 = 39.2 \text{ m}$$

理論上限的粗估。實務上因路徑封閉（誤差部分抵消），實際誤差通常較小，典型為路徑長度的 2~5%。

取 3% 估算：

$$\text{實際位置誤差} \approx 300 \times 0.03 = 9 \text{ m}$$

回到起點時，估計位置與真實起點相差約 9 公尺。

解（c）：迴路閉合提供了一個強約束：「終點 = 起點」。圖最佳化會把這 9 公尺的誤差均勻攤到整條軌跡上。

$$\text{最佳化後的平均誤差} \approx 9 \text{ m} / \sqrt{(\text{節點數})} \text{ 或更小}$$

誤差不再集中在終點，而是分散到全程。

實務上的典型結果是：最大誤差由 9 公尺降至 0.2~0.5 公尺，改善約 20~45 倍。這正是 25.7.1 節所說「改善一到兩個數量級」的具體數字。

解（d）：若誤判——把「起點」誤認為「路徑中點」（假設在 150 公尺處），後果是：

- 最佳化器會加入一條錯誤的強約束，強迫「300 公尺處」與「150 公尺處」重合。
- 整條軌跡會被折疊——後半段被拉到前半段上方。
- 地圖上會出現重疊的牆面與虛假的通道，使路徑規劃完全失效。
- 最嚴重的是：這個錯誤無法自動修復。那條錯誤的邊仍在圖中，且其資訊矩陣可能很大（因為外觀匹配的信心很高）。

因應之道（第 25.7.3 節）：一、幾何驗證——檢查兩處的光達掃描是否真的能對齊；二、時間一致性——要求連續數次偵測；三、強健核函數——在最佳化中自動降低異常邊的權重。三者應同時採用，因為單一機制都可能被特殊情況突破。

程式範例：佔據柵格建圖與位姿圖最佳化

以下 Python 程式實作佔據柵格地圖的對數勝算比更新與射線追蹤，以及一個簡化的位姿圖最佳化，展示迴路閉合如何「扭正」整條軌跡。

```
import numpy as np
```

```
# ===== 佔據柵格地圖 =====
class OccupancyGrid:
    """對數勝算比表示的佔據柵格地圖 ( 25.5 節 ) """
    def __init__(s, w=200, h=200, res=0.05, l_occ=0.85, l_free=-0.40,
                 l_min=-5.0, l_max=5.0):
        s.w, s.h, s.res = w, h, res
        s.l = np.zeros((h, w)) # 對數勝算比 · 初值 0 ( 未知 )
        s.l_occ, s.l_free = l_occ, l_free
        s.l_min, s.l_max = l_min, l_max

    def world_to_grid(s, x, y):
        return int(x/s.res + s.w/2), int(y/s.res + s.h/2)

    def prob(s):
        """轉換為機率地圖"""
        return 1.0/(1.0 + np.exp(-s.l))

    def bresenham(s, x0, y0, x1, y1):
        """射線追蹤：回傳由 (x0,y0) 到 (x1,y1) 經過的格子"""
        cells = []
        dx, dy = abs(x1-x0), abs(y1-y0)
        sx = 1 if x0 < x1 else -1
        sy = 1 if y0 < y1 else -1
        err = dx - dy
        x, y = x0, y0
        while True:
            cells.append((x, y))
            if x == x1 and y == y1:
                break
            e2 = 2*err
            if e2 > -dy: err -= dy; x += sx
            if e2 < dx: err += dx; y += sy
        return cells

    def update_ray(s, rx, ry, angle, dist, max_range=5.0):
        """以一條光達射線更新地圖 ( 25.5.3 節 ) """
        gx0, gy0 = s.world_to_grid(rx, ry)
        hit = dist < max_range
        d = min(dist, max_range)
        ex, ey = rx + d*np.cos(angle), ry + d*np.sin(angle)
        gx1, gy1 = s.world_to_grid(ex, ey)
        cells = s.bresenham(gx0, gy0, gx1, gy1)
        for i, (cx, cy) in enumerate(cells):
            if not (0 <= cx < s.w and 0 <= cy < s.h):
                continue
            is_end = (i == len(cells)-1)
            delta = s.l_occ if (is_end and hit) else s.l_free
            s.l[cy, cx] = np.clip(s.l[cy, cx] + delta, s.l_min, s.l_max)

    def stats(s):
        p = s.prob()
        return (np.sum(p > 0.7), np.sum(p < 0.3),
                np.sum((p >= 0.3) & (p <= 0.7)))
```

```

# ===== 例題 25-2 驗證 =====

print("=== 例題 25-2：對數勝算比更新 ===")
def l2p(l): return 1.0/(1.0 + np.exp(-l))
l_occ, l_free = 0.85, -0.40
l = 0.0
print(f"{'事件':<20}{'l 值':>10}{'機率':>12}")
print(f"{'初始 (未知)':<20}{l:10.3f}{l2p(l):11.2%}")
for i in range(3):
    l += l_occ
    print(f"{'第 %d 次偵測為障礙' % (i+1):<20}{l:10.3f}{l2p(l):11.2%}")
for i in range(2):
    l += l_free
    print(f"{'第 %d 次偵測為空曠' % (i+1):<20}{l:10.3f}{l2p(l):11.2%}")
n_needed = int(np.ceil(5.0/l_occ))
print(f"\n 達到上限 l=5.0 需連續 {n_needed} 次偵測為障礙 "
      f"(機率 {l2p(5.0):.2%})")
print(f"由上限翻轉為空曠需 {int(np.ceil(10.0/abs(l_free)))} 次空曠觀測")

# ===== 建圖模擬 =====

print("\n=== 佔據柵格建圖模擬 (矩形房間) ===")
grid = OccupancyGrid(w=200, h=200, res=0.05)
# 定義一個 4x3 公尺的房間 (牆面)
def raycast_room(rx, ry, angle, w=2.0, h=1.5):
    """計算射線到矩形房間牆面的距離"""
    dx, dy = np.cos(angle), np.sin(angle)
    ts = []
    for wall, d, p in ((w, dx, rx), (-w, dx, rx), (h, dy, ry), (-h, dy, ry)):
        if abs(d) > 1e-9:
            t = (wall - p)/d
            if t > 0:
                px, py = rx + t*dx, ry + t*dy
                if -w-0.01 <= px <= w+0.01 and -h-0.01 <= py <= h+0.01:
                    ts.append(t)
    return min(ts) if ts else 5.0

np.random.seed(0)
positions = [(0.0, 0.0), (1.0, 0.5), (-1.0, -0.5), (1.2, -0.8), (-1.2, 0.9)]
for step, (rx, ry) in enumerate(positions):
    for angle in np.linspace(-np.pi, np.pi, 180, endpoint=False):
        d = raycast_room(rx, ry, angle) + np.random.randn()*0.01
        grid.update_ray(rx, ry, angle, d)
    occ, free, unk = grid.stats()
    print(f" 掃描 {step+1} 次後：佔據 {occ:5d} 格, "
          f"空曠 {free:5d} 格, 未知 {unk:5d} 格")
print("→ 隨著觀測累積，未知區域逐步減少")

# ===== 位姿圖最佳化 (迴路閉合) =====

print("\n=== 位姿圖最佳化：迴路閉合的效果 ===")
class PoseGraph2D:
    """簡化的二維位姿圖 (只估計 x, y，省略角度以簡化)"""
    def __init__(s, n):
        s.n = n

```



```

s.nodes = np.zeros((n, 2))
s.edges = [] # (i, j, dx, dy, weight)

def add_edge(s, i, j, dx, dy, w=1.0):
    s.edges.append((i, j, dx, dy, w))

def optimize(s, iters=100):
    """Gauss-Newton 迭代 (25.6.4 節)"""
    for _ in range(iters):
        H = np.zeros((2*s.n, 2*s.n))
        b = np.zeros(2*s.n)
        # 固定第一個節點 (否則系統有平移自由度)
        H[0:2, 0:2] += np.eye(2)*1e6
        for (i, j, dx, dy, w) in s.edges:
            e = (s.nodes[j] - s.nodes[i]) - np.array([dx, dy])
            A = -np.eye(2)*w
            B = np.eye(2)*w
            H[2*i:2*i+2, 2*i:2*i+2] += A.T @ A
            H[2*i:2*i+2, 2*j:2*j+2] += A.T @ B
            H[2*j:2*j+2, 2*i:2*i+2] += B.T @ A
            H[2*j:2*j+2, 2*j:2*j+2] += B.T @ B
            b[2*i:2*i+2] += A.T @ (w*e)
            b[2*j:2*j+2] += B.T @ (w*e)
        dx_all = np.linalg.solve(H, -b)
        s.nodes += dx_all.reshape(-1, 2)
        if np.max(np.abs(dx_all)) < 1e-9:
            break
    return s.nodes

# 建立一個正方形迴路：20 個節點，每邊 5 步
N = 20
true_poses = []
for i in range(N):
    side, k = i//5, i % 5
    if side == 0: true_poses.append([k*1.0, 0.0])
    elif side == 1: true_poses.append([5.0, k*1.0])
    elif side == 2: true_poses.append([5.0-k*1.0, 5.0])
    else: true_poses.append([0.0, 5.0-k*1.0])
true_poses = np.array(true_poses, float)

# 里程計有系統性偏差 (每步偏 3%)
pg = PoseGraph2D(N)
odom = np.zeros((N, 2))
bias = 1.03
for i in range(N-1):
    d_true = true_poses[i+1] - true_poses[i]
    d_odom = d_true*bias + np.random.randn(2)*0.02
    odom[i+1] = odom[i] + d_odom
    pg.add_edge(i, i+1, d_odom[0], d_odom[1], w=1.0)
pg.nodes = odom.copy()

err_before = np.linalg.norm(odom[-1] - (true_poses[-1] - true_poses[0]))
print(f"最佳化前 (僅里程計):")
print(f"終點估計 = ({odom[-1,0]:6.3f}, {odom[-1,1]:6.3f})")
print(f"真實終點 = ({true_poses[-1,0]:6.3f}, {true_poses[-1,1]:6.3f})")

```

```

print(f" 終點誤差 = {err_before:.3f} m")

# 加入迴路閉合邊：節點 19 應該回到節點 0 附近
d_loop = true_poses[0] - true_poses[N-1]
pg.add_edge(N-1, 0, d_loop[0], d_loop[1], w=5.0) # 高權重
pg.optimize()

errs = np.linalg.norm(pg.nodes - (true_poses - true_poses[0]), axis=1)
errs_before = np.linalg.norm(odom - (true_poses - true_poses[0]), axis=1)
print(f"\n 最佳化後 ( 加入迴路閉合 ) :")
print(f" 終點估計 = ({pg.nodes[-1,0]:6.3f}, {pg.nodes[-1,1]:6.3f})")
print(f" 終點誤差 = {errs[-1]:.3f} m")
print(f" 全程平均誤差: {errs_before.mean():.3f} m → {errs.mean():.3f} m")
print(f" 全程最大誤差: {errs_before.max():.3f} m → {errs.max():.3f} m")
print(f" 改善倍數: {errs_before.max()/max(errs.max(),1e-9):.1f}x")
print(f"→ 誤差被均勻攤到整條軌跡上，這就是「地圖當場扭正」")

# ===== 誤判迴路閉合的災難 =====
print(f"\n=== 誤判迴路閉合的後果 ( 例題 25-3d ) ===")
pg_bad = PoseGraph2D(N)
pg_bad.nodes = odom.copy()
for i in range(N-1):
    d = odom[i+1] - odom[i]
    pg_bad.add_edge(i, i+1, d[0], d[1], w=1.0)
# 錯誤：把節點 19 誤認為節點 10 ( 路徑中點 )
pg_bad.add_edge(N-1, 10, 0.0, 0.0, w=5.0)
pg_bad.optimize()
errs_bad = np.linalg.norm(pg_bad.nodes - (true_poses - true_poses[0]), axis=1)
print(f" 正確迴路閉合：平均誤差 {errs.mean():.3f} m, "
      f"最大 {errs.max():.3f} m")
print(f" 錯誤迴路閉合：平均誤差 {errs_bad.mean():.3f} m, "
      f"最大 {errs_bad.max():.3f} m")
print(f"→ 誤判使誤差反而增加 {errs_bad.mean()/errs.mean():.1f} 倍——"
      f"軌跡被錯誤地折疊")

```

請試著修改：（一）把迴路閉合邊的權重由 5.0 改為 0.5，觀察最佳化的效果如何減弱。（二）在建圖模擬中加入一個「動態障礙物」（某次掃描時出現、之後消失），觀察 I 值的上下限如何幫助地圖恢復。（三）把里程計的偏差由 3% 增至 10%，觀察迴路閉合的改善倍數如何變化。

實作：SLAM 系統的建立與評估

本章實作以 ROS 2 環境為主（可用模擬器或實體機器人）。若無 ROS 環境，前四項仍可用

純 Python 完成。

1. **佔據柵格建圖**：實作對數勝算比的地圖更新與 Bresenham 射線追蹤。用模擬的光達資料（或第 6 章的實體光達）建立一個房間的地圖。
2. **參數敏感度測試**：改變 l_{occ} 、 l_{free} 與上下限，觀察地圖的清晰度與對動態變化的反應速度。找出適合你的感測器的參數組合。
3. **掃描匹配實作**：實作簡化的 ICP 演算法——給定兩次光達掃描，求出它們之間的相對位姿。這是光達 SLAM 前端的核心。
4. **位姿圖最佳化**：實作二維的位姿圖最佳化（可參考本章程式），驗證迴路閉合的效果。
5. **ROS 2 SLAM 實測**：使用 `slam_toolbox`（ROS 2 中最常用的二維 SLAM 套件），讓機器人在實際環境中建圖。錄製 `rosvbag` 以便重複實驗。
6. **迴路閉合的觀察**：規劃一條封閉路徑，讓機器人繞行一圈。錄影記錄地圖在迴路閉合瞬間的變化——這是本章最值得親眼看見的現象。
7. **里程計品質的影響**：故意讓里程計的參數失準（如把輪距設錯 5%），重複建圖，比較地圖的品質。這連結了第 23 章的校正工作。
8. **動態環境測試**：在建圖過程中讓人走過機器人前方，觀察地圖上是否留下「鬼影」。測試不同的 l 值上下限對此的影響。
9. **感知混淆測試**：在一個有重複結構的環境（如相似的走廊、對稱的房間）中建圖，觀察是否發生錯誤的迴路閉合。若有，分析其成因。
10. **（進階）地圖品質評估**：以人工測量的真實尺寸為基準，量測建出的地圖中數個關鍵距離（如走廊長度、房間寬度），計算誤差百分比。這是評估 SLAM 品質的標準作法。

表 25-6 SLAM 實驗記錄表（前兩列為範例）

測試項目	設定	地圖品質	定位誤差	備註
基準建圖	<code>slam_toolbox</code> 預設	牆面清晰	終點誤差 12 cm	無迴路閉合
加入迴路閉合	繞行一圈	牆面對齊	終點誤差 3 cm	閉合瞬間地圖明顯扭正
（自行填寫其餘測試）				

■ 國中小也看得懂：蒙著眼睛畫地圖

想像你被蒙住眼睛，帶到一個從來沒去過的房間，然後有人跟你說：「請你畫出這個房間的地圖。」

你會怎麼做？大概是伸手摸索——摸到一面牆，往前走幾步，再摸到另一面牆……慢慢在腦中拼湊出房間的樣子。

但這裡有一個大問題：**你怎麼知道自己走了多遠？** 你可能以為自己走了 5 步，其實只走了 4 步半。走得越多，錯得越多。等你摸完一整圈回到門口，你畫出來的地圖可能是**歪的**——最後一面牆跟第一面牆對不起來！

這就是機器人遇到的難題，它有一個名字叫 **SLAM**（唸作「斯朗姆」）。它同時要做兩件互相依賴的事：

要畫地圖 → 必須知道自己在哪裡（不然不知道要把牆畫在哪）

要知道自己在哪裡 → 必須有地圖（不然沒有東西可以對照）

聽起來像「先有雞還是先有蛋」對不對？科學家想了很久才找到答案：**不要分開做，一起做！**

而最神奇的一刻叫做「**繞回來了**」。當機器人繞了一大圈，突然認出「咦，這個角落我來過！」——這時候它就知道：「原來我走歪了這麼多。」然後**整張地圖會在一瞬間被扭正**，歪掉的牆全部對齊！

這個畫面真的很酷，如果你有機會操作掃地機器人的建圖功能，可以留意看看——當它繞完一圈回到起點時，螢幕上的地圖會突然「跳」一下變整齊。那就是這一章講的東西！

■ 叫獸提醒與常見錯誤

- 誤區一：先定位再建圖，或先建圖再定位。兩者互為前提且誤差會互相污染，必須聯合估計。
- 誤區二：忽略地標之間的相關性。若把 Σ_{rm} 當成零，系統會過度自信地修正，導致估計不一致甚至發散。
- 誤區三：在大規模環境使用 EKF-SLAM。其 $O(N^2)$ 複雜度使地標數超過數百個就無法即時運行。
- 誤區四：佔據柵格不設 I 值的上下限。這會使地圖無法適應環境變化——障礙物被移走後仍

長期顯示為佔據。

- 誤區五：以為濾波法能修正過去的軌跡。線上 SLAM 只維護當前位姿，無法回頭調整——迴路閉合需要完整 SLAM。
- 誤區六：迴路閉合只做外觀比對而不做幾何驗證。感知混淆會導致誤判，而一次誤判可能毀掉整張地圖。
- 誤區七：把「漏判」與「誤判」等同看待。漏判只是損失一次改善機會，誤判則會產生無法自動修復的錯誤約束。
- 誤區八：資料關聯用歐氏距離而非馬氏距離。後者考慮了不確定性，在地標密集時判別力顯著較佳。
- 誤區九：不確定時強行關聯到已知地標。應遵循「寧可新增，不可誤配」的原則。
- 誤區十：忽略動態物體。移動的人或車會被誤認為地標，並在地圖上留下「鬼影」，干擾後續的定位與規劃。
- 誤區十一：以為深度學習能端到端取代 SLAM。學習在辨識類任務（特徵、迴路閉合、語意）上優異，但幾何最佳化仍應交給傳統方法。
- 誤區十二：用純視覺 SLAM 處理快速運動或低紋理環境。應融合 IMU（VIO）以補足視覺的三個弱點。

課堂討論

1. 25.1.3 節說「一次觀測同時約束了位姿與地標」。請用拼圖的類比進一步說明：為何累積夠多的約束後，整個系統只有一種一致的解？
2. 圖最佳化保留所有歷史資料而不做邊緣化，因此保持稀疏。請討論這個「用記憶體換計算量」的交易，在什麼硬體條件下會不划算？
3. 25.7.3 節主張「寧可漏判，不可誤判」。請討論這個原則的普遍性——本書中還有哪些場合採用了類似的不對稱風險考量？（提示：第 7 章的失效導向安全、第 20 章的力控制）
4. 深度學習改變了 SLAM 的前端卻未取代後端。請討論這個分工背後的原因，並與第 21.9.5 節的混合式抓取、第 17 章的模型加回授做比較。
5. 長期 SLAM 面臨「地圖必須更新，但不能因暫時的變化就抹掉長期正確的部分」的兩難。請設計一個機制來平衡這兩者，並討論其可能的失效情境。

章末摘要

- SLAM 的本質是雞生蛋問題：建圖需要定位、定位需要地圖；解法是把兩者放進同一個估計問題中聯合求解。
- 聯合估計可行的根本原因：一次觀測同時約束了機器人位姿與地標位置，累積足夠的約束後解唯一。
- 完整 SLAM 估計整條軌跡（能修正過去），線上 SLAM 只估計當前位姿（計算量小但無法回頭修正）。
- 三種地圖表示：特徵地圖（精簡、配 EKF）、佔據柵格（能表示空曠、可直接用於規劃）、拓撲地圖（極精簡、用於高階導航）。
- EKF-SLAM 把所有地標放進狀態向量，維度為 $3+2N$ ；非對角區塊 Σ_{rm} 記錄的相關性是收斂的關鍵。
- EKF-SLAM 的協方差是稠密的，更新複雜度為 $O(N^2)$ ，因此只適用於數百個地標以內的環境。
- SLAM 收斂定理：地圖不確定性單調遞減、地標間趨於完全相關、下界由首次觀測時的位姿不確定性決定。
- Rao-Blackwell 分解：已知完整軌跡時各地標互相獨立；FastSLAM 據此用粒子處理軌跡、用小型 EKF 處理各地標。
- FastSLAM 的複雜度為 $O(M \log N)$ ，且能處理資料關聯的不確定性（錯誤的粒子會被淘汰）。
- 佔據柵格使用對數勝算比 $l = \log[p/(1-p)]$ ，使貝氏更新由乘法變成加法；必須設定上下限以適應環境變化。
- 圖最佳化把 SLAM 表示為節點（位姿）與邊（約束）的圖，最佳化即「讓彈簧系統找到平衡」。
- 圖最佳化因不做邊緣化而保持稀疏，這是它能處理大規模問題並取代濾波法的關鍵。
- 前端負責建立圖（與感測器相關），後端負責求解最佳化（與感測器無關），這個分工使系統模組化。
- 迴路閉合是消除長期漂移的唯一手段，能將誤差改善一到兩個數量級。
- 迴路閉合偵測有三類：幾何驗證（依賴位姿估計）、外觀比對（詞袋模型，不依賴位姿）、

學習式描述子。

- 一次錯誤的迴路閉合可能毀掉整張地圖且無法自動修復；防護機制為幾何驗證、時間一致性與強健核函數。
- 資料關聯的實務原則：寧可新增不可誤配、用描述子輔助判別、門檻設得保守。
- 光達 SLAM 精確但昂貴且資訊單一；視覺 SLAM 便宜且資訊豐富但依賴光線與紋理；VIO 融合視覺與 IMU 補足彼此弱點。

觀念題與計算題

觀念題

1. 陳述 SLAM 問題並說明其雞生蛋的本質，以及為何無法用「先做一個再做另一個」解決。
2. 說明「一次觀測同時約束位姿與地標」的意義，並解釋這為何是 SLAM 可解的根本原因。
3. 區分完整 SLAM 與線上 SLAM，並說明為何迴路閉合本質上需要前者。
4. 比較三種地圖表示法的優缺點與適用場合。
5. 寫出 EKF-SLAM 的狀態向量結構，並說明協方差矩陣中非對角區塊的意義。
6. 說明 EKF-SLAM 為何是 $O(N^2)$ 的複雜度，以及這帶來什麼實務限制。
7. 敘述 SLAM 收斂定理的三個性質。
8. 說明 Rao-Blackwell 分解的核心洞見，以及 FastSLAM 如何據此設計。
9. 說明對數勝算比的定義與三個優點，並解釋為何必須設定上下限。
10. 說明佔據柵格的感測器逆模型，以及為何佔據的證據通常大於空曠的證據。
11. 用彈簧系統的類比說明圖最佳化，並解釋迴路閉合為何能「扭正」整條軌跡。
12. 解釋為何濾波法的矩陣是稠密的而圖最佳化是稀疏的，並說明這個差異的後果。
13. 說明前端與後端的分工，以及這個架構的價值。
14. 比較三類迴路閉合偵測方法的原理與優缺點。
15. 說明感知混淆的意義，以及為何「寧可漏判，不可誤判」。
16. 列舉三種資料關聯的方法，並說明實務上的三個原則。
17. 比較光達 SLAM 與視覺 SLAM，並說明 VIO 為何能補足純視覺的弱點。

計算題

1. 某 EKF-SLAM 在三維環境中運行，機器人位姿為 6 維、每個地標為 3 維。(a) 當有 80 個地標時，求狀態維度與協方差元素數。(b) 地標增至 400 個時重新計算。(c) 求運算量的倍數。(d) 若 (a) 耗時 5 ms，判斷 (b) 能否以 10 Hz 運行。
2. 某佔據柵格使用 $l_{occ} = +0.7$ 、 $l_{free} = -0.35$ 、上下限 ± 4.0 。(a) 某格連續四次偵測為障礙物，求 l 值與機率。(b) 接著三次偵測為空曠，重新計算。(c) 求達到上限所需的連續偵測次數。(d) 求由上限翻轉為「未知」($l = 0$) 所需的空曠觀測次數。
3. 某機器人沿封閉路徑行走 500 公尺，里程計航向誤差率為每公尺 0.08 度。(a) 求累積航向誤差。(b) 若實際位置誤差為路徑長度的 4%，求終點誤差。(c) 若迴路閉合後誤差降至 0.3 公尺，求改善倍數。(d) 若路徑長度加倍為 1000 公尺，估算迴路閉合前後的誤差。
4. 某位姿圖有 500 個節點，每個節點為二維位姿 (3 個變數)。里程計邊 499 條，迴路閉合邊 20 條。(a) 求變數總數與資訊矩陣的維度。(b) 若矩陣為稠密，求元素總數。(c) 實際上每個節點平均只與 2.08 個節點相連，估算非零元素的比例。(d) 討論稀疏性帶來的計算量差異。

動手做與專題延伸

本章實作：SLAM 模組

延續第 23、24 章的程式庫，建立 SLAM 的核心模組。

- **階段一**：實作佔據柵格地圖 (對數勝算比、Bresenham 射線追蹤、上下限)，並用模擬光達建圖。
- **階段二**：實作簡化的 EKF-SLAM (特徵地圖、二維環境)，驗證地標不確定性隨觀測遞減、且地標間逐漸相關。
- **階段三**：實作二維位姿圖最佳化 (Gauss-Newton)，並驗證迴路閉合的效果。
- **階段四**：實作簡化的掃描匹配 (ICP)，作為位姿圖邊的來源。
- **階段五**：整合為完整的 SLAM 系統——前端 (掃描匹配 + 迴路偵測) 加後端 (位姿圖最佳化)。

專題延伸

- **迴路閉合的視覺化動畫**：模擬機器人繞行一個大迴路，記錄每一步的軌跡估計與地圖。製作

動畫顯示「里程計逐漸偏離 → 偵測到迴路閉合 → 整條軌跡瞬間扭正」的完整過程。這是本章最具戲劇性的現象，也是理解圖最佳化最直觀的方式。

- **稀疏性的量化實驗：**建立不同規模的位姿圖（100、500、2000、10000 個節點），分別以稠密與稀疏的線性求解器計算，記錄執行時間。繪出「節點數 vs 時間」的對數圖，驗證稠密解法為 $O(n^3)$ 而稀疏解法接近 $O(n)$ 。這個實驗能讓抽象的複雜度差異變得具體。
- **進階：感知混淆的實驗與防護。**建立一個有重複結構的模擬環境（如四條一模一樣的走廊）。實作外觀式的迴路閉合偵測，故意不加防護機制，觀察錯誤的迴路閉合如何折疊地圖。接著逐一加入三種防護（幾何驗證、時間一致性、強健核函數），量化每一種對誤判率的降低效果。討論三者的成本與效益，並提出一個綜合的防護策略。

參考文獻與延伸閱讀

- Thrun, S., Burgard, W. and Fox, D., Probabilistic Robotics, MIT Press (第 9 ~ 13 章涵蓋佔據柵格、EKF-SLAM、FastSLAM 與圖 SLAM)。
- Durrant-Whyte, H. and Bailey, T., "Simultaneous Localization and Mapping: Part I & II," IEEE Robotics & Automation Magazine, 2006 (SLAM 的經典回顧論文)。
- Cadena, C. et al., "Past, Present, and Future of Simultaneous Localization and Mapping," IEEE Trans. Robotics, 2016 (現代 SLAM 的全面回顧，強烈推薦)。
- Dissanayake, M. W. M. G. et al., "A Solution to the Simultaneous Localization and Map Building Problem," IEEE Trans. Robotics and Automation, 2001 (EKF-SLAM 的收斂性證明)。
- Montemerlo, M. et al., "FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem," AAAI 2002 (FastSLAM 的原始論文)。
- Grisetti, G. et al., "A Tutorial on Graph-Based SLAM," IEEE Intelligent Transportation Systems Magazine, 2010 (圖最佳化 SLAM 的優秀教學文章)。
- Mur-Artal, R. and Tardós, J. D., "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras," IEEE Trans. Robotics, 2017 (視覺 SLAM 的代表性系統)。
- Qin, T., Li, P. and Shen, S., "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE Trans. Robotics, 2018 (VIO 的代表性工作)。

- Shan, T. and Englot, B., "LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping," IROS 2018 (光達 SLAM 的高效實作) 。
- ROS 2 slam_toolbox、Cartographer 與 RTAB-Map 官方文件：三套廣泛使用的開源 SLAM 系統，各有不同的設計取舍。

第 26 章

路徑規劃與導航

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為位形空間的轉換示意：上圖是工作空間——一台長方形機器人推車要繞過一根柱子，同一條路徑在車頭朝左時能過、朝右時卡住；下圖是位形空間——柱子被膨脹了機器人的半徑，機器人縮成一個點，路徑變成點的移動，旁註「在位形空間中，機器人是一個點」。右半為分層導航的對照：一張完整地圖上以藍色粗線標出 A^* 規劃的全域路徑，路徑周圍以綠色扇形疊出 DWA 的局部候選軌跡群，其中一條被選中；畫面標註「全域看得遠、局部反應快」。下緣文字：先想清楚大方向，再邊走邊閃。

叫獸開講 「六維空間長什麼樣？機器人如何在看不見的空間裡找路？」

叫獸在白板上畫了一張簡單的平面圖：一個房間、中間有一根柱子、機器人在左下角、目標在右上角。

「請小翰上來畫出機器人該走的路線。」小翰很快畫了一條繞過柱子的曲線。「很好。現在，」叫獸把機器人的圖示擦掉，換成一個長方形——一台推著大型推車的機器人，長 1.2 公尺、寬 0.8 公尺。「同樣的房間，重畫一次。」

小翰猶豫了。這次不只要考慮「路徑會不會穿過柱子」，還要考慮「機器人的車身會不會撞到柱子」——而這取決於機器人當時的**朝向**。他畫了幾筆，發現同一條路徑，車頭朝左時能過、朝右時會卡住。

「這就是路徑規劃第一個、也是最重要的觀念。」叫獸說，「你剛才不自覺地在做一件很累的事——**同時追蹤機器人的形狀與位置**。有沒有辦法讓它變簡單？」

小好舉手：「把柱子畫大一點？大到機器人可以當成一個點？」

「非常好。」叫獸在柱子外圍畫了一圈虛線，「這叫做**膨脹**。你把障礙物長胖了機器人的半徑，然後就可以把機器人當成一個**點**來規劃。這個轉換叫做**位形空間**，是整個路徑規劃的地基。」

他接著在白板上另一側寫下三個字：**點、線、面**。「但這裡有一個關鍵問題：機器人不是圓的。你的推車是長方形，轉個方向佔的空間就不一樣——所以真正的位形空間不是二維的平面，而是**三維的**：x、y、還有角度 θ 。」

小翰皺眉：「三維還好吧？」

「三維還好。但如果是第 9 章那支六軸手臂呢？」叫獸敲了敲白板，「它的位形空間是**六維**的。而規劃演算法必須在這個六維空間裡，找出一條不碰到任何障礙的路。你能想像六維空間長什麼樣嗎？」

教室安靜了。叫獸笑了：「沒有人能。這就是為什麼路徑規劃在 1990 年代之前幾乎是無解的問題——直到有人想出一個聰明的辦法：**既然畫不出來，那就不要畫。用隨機取樣去探索它。**」

最後他在白板上寫下一句話：「今天要學的，是機器人如何在一個**看不見也畫不出來**的空間裡找路。而這個問題有兩層——**先想清楚大方向，再邊走邊閃**。兩者缺一不可。」

本章為什麼重要

經過第 22 ~ 25 章，機器人已經能看見環境、能移動、能估計自己的狀態、並能建立地圖。但知道自己在哪裡，不等於知道該怎麼走。

路徑規劃回答的正是這個問題：給定當前位置、目標位置與環境地圖，找出一條可行的路徑。這條路徑必須**避開障礙**、**符合運動約束**（第 23 章的非完整性）、且**盡可能地短或快**。

這個問題的困難程度常被低估。對人類而言「繞過那根柱子」是本能，但對機器人來說，它涉及三個層次的挑戰：一、**如何表示「哪些位置是可行的」**——這需要位形空間的抽象；二、**如何在高維空間中搜尋**——六軸手臂的位形空間是六維的，人類無法想像；三、**如何處理動態與不確定性**——環境會變、感知有誤差、規劃出的路徑未必走得準。

本章也是本書兩條主線的交會處。它同時服務於**移動機器人**（在地圖上找路）與**機械手臂**（在關節空間避開障礙），而兩者用的是同一套理論。此外，本章的搜尋與取樣思想，也將在第 32 章的任務規劃中再次出現。

學習目標

- 能定義位形空間，並說明其為何是路徑規劃的核心抽象。
- 能運用障礙物膨脹把機器人簡化為點。
- 能區分路徑規劃與軌跡生成的分工。
- 能實作 Dijkstra 與 A^* 演算法並說明兩者的差異。
- 能定義啟發函數的可採納性，並說明其對最佳性的保證。
- 能說明格點式規劃在高維空間的維數詛咒。
- 能說明 PRM 與 RRT 的原理與適用場合。
- 能說明 RRT* 如何達成漸近最佳性。
- 能說明非完整約束對規劃的影響與 Dubins 路徑的意義。
- 能說明人工勢場法的原理與局部極小值問題。
- 能說明動態視窗法的原理與其如何納入動力學限制。
- 能說明分層導航架構中全域與局部規劃的分工。

先備知識

第 25 章的佔據柵格地圖（規劃的直接輸入）、第 23 章的運動約束與非完整性、第 15 章的軌跡生成（規劃的下游）、第 4 章的自由度概念、第 24 章的機率思維。

演算法方面需要：圖的表示、優先佇列、以及基本的搜尋概念。若對資料結構生疏，可先閱讀附錄 D.4。本章的數學不深，但**空間想像力很重要**——建議多畫圖，尤其是位形空間的部分。

核心名詞中英對照

表 26-1 本章核心名詞

中文	英文	說明
位形空間	Configuration Space, C-space	以機器人所有位形為點所構成的空間
自由空間	Free Space, C_free	位形空間中不發生碰撞的部分
障礙物膨脹	Obstacle Inflation	把障礙物擴大以將機器人簡化為點
完備性	Completeness	有解時必定能找到的性質
機率完備性	Probabilistic Completeness	取樣數趨於無窮時找到解的機率趨於 1
最佳性	Optimality	所找到的解是最短或代價最小的
啟發函數	Heuristic Function	估計某點到目標的剩餘代價
可採納	Admissible	啟發函數不高估真實代價的性質
一致性	Consistency	啟發函數滿足三角不等式的性質
機率路標圖	Probabilistic Roadmap, PRM	先建圖再查詢的取樣式規劃法
快速探索隨機樹	Rapidly-exploring Random Tree, RRT	以隨機取樣長出樹狀結構的規劃法
Dubins 路徑	Dubins Path	有最小轉彎半徑限制下的最短路徑
人工勢場	Artificial Potential Field	以吸引與排斥力導引運動的方法
局部極小值	Local Minimum	勢場中非目標的低點，會使機器人卡住
動態視窗法	Dynamic Window Approach, DWA	在速度空間中搜尋的局部規劃法
全域規劃	Global Planner	依完整地圖規劃大方向路徑
局部規劃	Local Planner	依即時感測避開動態障礙
代價地圖	Costmap	含膨脹層與代價值的規劃用地圖

26.1 位形空間

26.1.1 把機器人變成一個點

路徑規劃最重要的抽象是**位形空間** (configuration space , 簡稱 C-space) 。它的定義是：

◎ 位形空間的定義

位形 (configuration) 是完整描述機器人所有部位位置所需的最少參數集合。**位形空間**則是所有可能位形所構成的空間——空間中的**每一個點**，代表機器人的**一個完整姿態**。

這個轉換的威力在於：**在位形空間中，整個機器人變成了一個點**。原本「這個形狀複雜的物體會不會撞到障礙物」的困難問題，變成了「這個點會不會落在禁區內」的簡單問題。

幾個具體的例子：

表 26-2 各類機器人的位形空間

機器人	位形參數	維度	空間的形狀
平面上的圓形機器人	(x, y)	2	平面
平面上的長方形機器人	(x, y, θ)	3	平面 $\times$ 圓 (環狀)
差速驅動車	(x, y, θ)	3	同上，但有非完整約束
平面 2R 手臂	(θ_1, θ_2)	2	環面 (甜甜圈)
六軸工業手臂	($\theta_1, \dots, \theta_6$)	6	六維環面
行動操作機器人	(x, y, $\theta, \theta_1 \dots \theta_6$)	9	九維

第四列特別值得注意：平面 2R 手臂的位形空間是一個**環面** (torus , 甜甜圈的形狀) ——因為兩個關節角度都是週期性的 (0° 與 360° 是同一個位形) 。這說明位形空間未必是平坦的歐氏空間，其拓撲結構可能相當特殊。

26.1.2 自由空間與障礙

位形空間可以分成兩部分：

$$C = C_{\text{free}} \cup C_{\text{obs}}$$

C_{free} (自由空間) 是不發生碰撞的位形； C_{obs} 是會碰撞的位形。路徑規劃的目標，就是在 C_{free} 中找出一條由起點到終點的連續曲線。

關鍵的困難在於： **C_{obs} 的形狀通常無法解析地寫出來**。即使工作空間中的障礙物是一個簡單的圓，在位形空間中對應的 C_{obs} 可能是一個極為複雜的形狀——尤其當機器人有多個關節時。

這正是為何 26.4 節的取樣式方法如此重要：它們不需要知道 C_{obs} 的形狀，只需要能回答「這個點是否在 C_{obs} 中」（即碰撞檢測）。這個「不建構、只查詢」的策略，是路徑規劃在高維空間中的突破關鍵。

26.1.3 障礙物膨脹

對於平移的機器人，有一個實用的簡化：把障礙物膨脹（inflate）機器人的半徑，然後把機器人當成一個點。

$$C_{obs} = \text{障礙物} \oplus \text{機器人形狀 (Minkowski 和)}$$

數學上這稱為 Minkowski 和。對圓形機器人，就是把障礙物往外擴張一個半徑。

這正是開頭故事中小好的想法。它在實務上極為常用——ROS 導航的代價地圖（costmap）就內建了膨脹層（inflation layer）。

但要注意兩個限制：一、只對圓形機器人完全正確。非圓形機器人的膨脹量會隨朝向改變，若用最大半徑膨脹會過於保守（狹窄通道可能被誤判為不可通行）。二、膨脹會抹除細節。若膨脹半徑大於通道寬度的一半，該通道在膨脹後會完全消失。

實務上的折衷是：用外接圓半徑膨脹做粗略規劃，再用精確的機器人輪廓做碰撞檢查。ROS 的 costmap 正是採用這種「內切圓 + 外接圓」的雙半徑設計。

圖 26-1 位形空間與障礙物膨脹

建議配圖：三格對照。左格為工作空間——一個圓形機器人（半徑 r ）與一個方形障礙物，機器人繞行障礙物的路徑以虛線表示。中格為位形空間——障礙物被膨脹了 r （原障礙以實線、膨脹後以虛線圈出），機器人縮成一個點，同一條路徑變成點的移動。右格為平面 $2R$ 手臂的位形空間——橫軸 θ_1 、縱軸 θ_2 ，工作空間中的一個圓形障礙物在此對應成一片不規則的黑色區域（ C_{obs} ），並標註「四邊相連成環面」。下方文字：在位形空間中，機器人是一個點。

26.2 規劃問題的分類與判準

26.2.1 路徑、軌跡與導航

必須先釐清三個容易混淆的層次（部分已在第 15 章討論）：

- 路徑規劃（Path Planning）：找出一條幾何曲線，只關心「經過哪些位形」，不含時間。本章的主題。
- 軌跡生成（Trajectory Generation）：為路徑加上時間參數化，決定速度與加速度。第 15

章的主題。

- **導航 (Navigation)**：完整的系統，包含定位、規劃、執行與即時避障的循環。26.9 節的主題。

三者的關係是**由上而下**：導航系統呼叫路徑規劃取得幾何路徑，再交給軌跡生成加上時間，最後由控制器執行。而執行過程中若發現新的障礙，又會觸發重新規劃——形成一個閉環。

26.2.2 演算法的三個判準

評估規劃演算法有三個標準的判準，理解它們的差異極為重要。

判準一：完備性 (Completeness)。若解存在，演算法**保證**能找到；若不存在，能**判定**無解。

- **完備**：格點搜尋（在該解析度下）、Dijkstra、A*。
- **機率完備**：取樣式方法（PRM、RRT）——取樣數趨於無窮時，找到解的機率趨於 1，但**永遠無法判定無解**。
- **不完備**：人工勢場法——可能卡在局部極小值而找不到存在的解。

判準二：最佳性 (Optimality)。找到的解是否為代價最小的？

- **最佳**：Dijkstra、A*（在可採納啟發函數下）。
- **漸近最佳**：RRT*——取樣數增加時，解逐漸收斂到最佳。
- **非最佳**：RRT、人工勢場法——找到的路徑可能繞遠路。

判準三：計算效率。實務上往往是決定性的因素——一個最佳但需要一小時的演算法，對即時導航毫無用處。

◎ 三者的取捨

這三個判準通常無法兼得，而**工程上的選擇取決於應用**。倉儲 AGV 在固定地圖上運行，可以離線計算最佳路徑（選 A*）；機械手臂在雜亂的桌面上抓取，位形空間六維且障礙隨時改變，只能用取樣法快速找出**可行解**（選 RRT）；而移動機器人的即時避障需要毫秒級反應，連可行性都只能局部保證（選 DWA）。**沒有一種演算法適用於所有場合**，理解它們的取捨才是本章的核心。

26.3 圖搜尋：Dijkstra 與 A*

26.3.1 把空間變成圖

最直觀的規劃方法是：把位形空間離散化成一張圖，然後用圖搜尋演算法找最短路徑。

離散化的方式有兩種：

方式一：格點 (Grid)。把空間切成規則的格子，每格是一個節點，相鄰的格子之間有邊。

這與第 25 章的佔據柵格地圖天然契合——佔據機率低的格子就是可通行的節點。

方式二：路標圖 (Roadmap)。在自由空間中選取一些代表性的位形作為節點，再連接彼此可見的節點。26.4 節的 PRM 屬於此類。

26.3.2 Dijkstra 演算法

Dijkstra 是最經典的最短路徑演算法。它的策略是：永遠先擴展目前已知代價最小的節點。

1. 把起點的代價設為 0，其餘節點設為無窮大。
2. 從尚未處理的節點中，取出代價最小的節點。
3. 更新其所有鄰居的代價（若經由此節點更短，則更新）。
4. 標記此節點為已處理，回到步驟 2，直到目標被處理或無節點可取。

Dijkstra 保證找到最短路徑，但它有一個明顯的低效：它往所有方向均勻地擴展，像水波一樣向外蔓延。即使目標就在正東方，它也會花大量時間探索西方的節點。

26.3.3 A*：加入方向感

A* 的改進極為簡單卻極為有效：在選擇節點時，不只看「已經走了多遠」，還要估計「還要走多遠」。

$$f(n) = g(n) + h(n)$$

A^* 的評價函數。 $g(n)$ 是由起點到 n 的實際代價、 $h(n)$ 是由 n 到目標的估計代價（啟發函數）。

演算法的流程與 Dijkstra 相同，只是排序的依據由 g 換成 f 。這個小改動使搜尋有了方向感——朝向目標的節點會優先被擴展。

常用的啟發函數（在格點地圖上）：

表 26-3 常見的啟發函數

名稱	公式	適用的移動方式	特性
曼哈頓距離	$ \Delta x + \Delta y $	只能上下左右（4 連通）	可採納

名稱	公式	適用的移動方式	特性
對角線距離	$\max(\Delta x , \Delta y)$	可斜走且代價相同 (8 連通)	可採納
歐氏距離	$\sqrt{(\Delta x)^2 + (\Delta y)^2}$	任意方向	可採納但可能低估
零 ($h = 0$)	0	任意	退化為 Dijkstra

26.3.4 可採納性：最佳性的保證

A^* 保證找到最短路徑的條件是：啟發函數必須是可採納的 (admissible) 。

$$h(n) \leq h^*(n) \quad \text{對所有 } n \text{ 成立}$$

可採納性。 $h^*(n)$ 是由 n 到目標的真實最短代價。即：啟發函數絕不能高估。

為何不能高估？直觀的理解是：若 h 高估了某條路徑的剩餘代價， A^* 會認為那條路「不划算」而跳過它——但它其實可能是最短的。低估則只會使搜尋多探索一些節點，不影響最終結果的正確性。

還有一個更強的條件稱為一致性 (consistency)：

$$h(n) \leq c(n, n') + h(n') \quad \text{對任意相鄰的 } n, n' \text{ 成立}$$

一致性 (也稱單調性)。滿足此條件時，每個節點只需處理一次，演算法更高效。一致性蘊含可採納性。

◎ 啟發函數的取舍

h 越接近真實值，搜尋越高效——若 $h = h^*$ (完美的啟發)， A^* 會直接沿最短路徑走，不浪費任何探索。但完美的 h 通常無法取得 (若能取得，問題就已經解決了)。反之， $h = 0$ 時 A^* 退化為 Dijkstra——保證最佳但效率低。因此 h 的設計是一個取舍：在「可採納」的前提下，盡可能接近真實值。實務上還有一種常見的作法：故意讓 h 略微高估 (乘上一個大於 1 的權重)，這稱為加權 A^* 。它犧牲最佳性換取速度，找到的路徑保證不超過最佳解的該權重倍——這在即時系統中往往是划算的交易。

26.3.5 格點法的維數詛咒

格點法在二維與三維中效果良好，但在高維空間中會遭遇與第 18.4.3 節相同的問題：維數詛咒。

若每一維切成 100 格：

2 維： 10^4 格

3 維： 10^6 格

6 維： 10^{12} 格

9 維： 10^{18} 格

格點數隨維度指數成長。六軸手臂的位形空間根本無法建表。

這使得格點法只適用於低維問題——典型是移動機器人的二維或三維位形空間。對機械手臂與高自由度系統，必須改用下一節的取樣式方法。

26.4 取樣式規劃：PRM 與 RRT

26.4.1 核心思想：不建構，只查詢

取樣式方法的突破性洞見是：

◎ 關鍵洞見

我們不需要知道 C_{obs} 的完整形狀，只需要能回答「這個特定的位形是否碰撞」。而後者只是一次碰撞檢測——對任何維度的機器人都能在毫秒內完成。既然如此，就不要試圖建構整個空間的表示，而是隨機取樣一些點，用它們拼出一條路。

這個轉換使高維規劃成為可能。它放棄了完備性（改為機率完備）與最佳性，換取的是在任意維度中都能運作的能力。

26.4.2 機率路標圖（PRM）

PRM (Probabilistic Roadmap) 採用「先建圖、再查詢」的策略，分為兩個階段。

階段一：建構路標圖（離線）

1. 在位形空間中隨機取樣 N 個點，丟棄碰撞的點。
2. 對每個保留的點，嘗試連接其鄰近的 k 個點。
3. 若兩點之間的直線路徑無碰撞（用局部規劃器檢查），就在圖中加入這條邊。
4. 得到一張涵蓋自由空間的路標圖。

階段二：查詢（線上）

1. 把起點與終點加入路標圖，各自連接到鄰近的節點。
2. 用 A^* 或 Dijkstra 在圖上搜尋最短路徑。

PRM 的最大優勢是「多次查詢」（multi-query）：路標圖建好之後，可以用它回答任意多組起訖點的查詢，每次只需一次圖搜尋。這使它非常適合環境固定但任務多變的場合——如工廠中的機械手臂，環境（治具、機台）固定不變，但每天要執行數千次不同的取放。

缺點是：建圖階段耗時、且若環境改變（有人走過、工件移位）就必須重建。

26.4.3 快速探索隨機樹 (RRT)

RRT (Rapidly-exploring Random Tree) 採用完全不同的策略：從起點長出一棵樹，逐步探索空間，直到觸及目標。

1. 以起點為樹根。
2. 在位形空間中隨機取樣一個點 x_{rand} 。
3. 找出樹上距離 x_{rand} 最近的節點 x_{near} 。
4. 由 x_{near} 朝 x_{rand} 前進一小步（步長 ϵ ），得到新節點 x_{new} 。
5. 若 x_{near} 到 x_{new} 的路徑無碰撞，就把 x_{new} 加入樹中。
6. 重複，直到樹觸及目標附近。

RRT 的名稱來自其驚人的探索特性：這棵樹會快速地朝未探索的區域生長。原因是——空曠的區域面積大，隨機取樣落在那裡的機率高，因此樹會優先往那些方向延伸。這是一個極為優雅的自組織性質。

一個關鍵的實作技巧：目標偏置 (goal bias)。以某個機率（典型 5 ~ 10%）不做隨機取樣，而是直接取目標點作為 x_{rand} 。這使樹在探索的同時也朝目標推進，能大幅加速收斂。

26.4.4 RRT*：漸近最佳性

原始的 RRT 有一個明顯的缺點：找到的路徑通常繞遠路，因為樹的結構是隨機生長的，不會回頭優化。

RRT* (RRT-star) 加入了兩個關鍵改進：

改進一：選擇最佳父節點。加入新節點時，不是直接連到最近的節點，而是在其鄰域內尋找使總代價最小的父節點。

改進二：重新佈線 (rewiring)。加入新節點後，檢查其鄰域內的既有節點——若經由新節點能使它們的代價更低，就改變它們的父節點。

這兩個改進使 RRT* 具備漸近最佳性 (asymptotic optimality)：

當取樣數 $\rightarrow \infty$ 時，解 $\rightarrow$ 最佳解

漸近最佳。這是 Karaman 與 Frazzoli 於 2011 年證明的重要結果。

代價是計算量增加（每次都要檢查鄰域），且**收斂較慢**——RRT 可能在一秒內找到一條可行路徑，而 RRT* 可能需要數十秒才能收斂到接近最佳。

表 26-4 PRM 與 RRT 家族的比較

面向	PRM	RRT	RRT*
策略	先建圖、後查詢	由起點長樹	長樹 + 重新佈線
查詢類型	多次查詢	單次查詢	單次查詢
完備性	機率完備	機率完備	機率完備
最佳性	取決於路標密度	非最佳	漸近最佳
適合動態環境	否（需重建）	是	是
處理非完整約束	困難	容易	容易
計算成本	建圖高、查詢低	低	中高
典型應用	固定環境的手臂	動態環境、快速求解	需要品質的離線規劃

26.4.5 為何 RRT 特別適合非完整系統

RRT 有一個 PRM 所沒有的重要優勢：它**天然能處理非完整約束**（第 23.3 節）。

原因在於生長的方式。PRM 需要「連接任意兩點」——但對差速車而言，兩個位形之間未必存在直線路徑（不能側移）。RRT 則不同：它只需要「從現有節點**沿著可行的運動**前進一小步」。

具體作法是：把步驟 4 的「朝 x_{rand} 前進」改為「**在可行的控制輸入中，選擇使結果最接近 x_{rand} 的那一個**」。這樣長出的樹，每一條邊都自動滿足運動約束。

這個版本稱為 **kinodynamic RRT**，是自駕車與無人機路徑規劃的主流方法之一。

26.5 非完整約束下的規劃

26.5.1 為何不能直接用最短路徑

第 23 章證明了差速車與汽車都受**非完整約束**——它們不能側移。這對路徑規劃有直接的影響：

在位形空間中，兩點之間的直線段未必是可行的路徑。A* 在格點上找出的路徑，可能包含

「原地橫移」這種物理上做不到的動作；PRM 連接兩個節點時假設的直線路徑，對汽車而言也不存在。

因此非完整規劃需要一個關鍵的替換：把「直線」換成「該機器人真正能走的最短路徑」。

26.5.2 Dubins 路徑

對於**只能前進**且有最小轉彎半徑 R 的車輛（如飛機、某些 AGV），Dubins 於 1957 年證明了一個優美的結果：

◎ Dubins 定理

在只能前進且轉彎半徑不小於 R 的限制下，連接任意兩個位姿的**最短路徑**，必定是下列六種組合之一：LSL、LSR、RSL、RSR、RLR、LRL。其中 L 表示左轉最小半徑圓弧、R 表示右轉、S 表示直線。

這個結果的實用價值極高：只需計算六種候選路徑的長度，取最短者即可——不需要任何搜尋或最佳化。計算成本是常數時間。

典型的 Dubins 路徑長這樣：先以最小半徑轉一個彎對準目標方向、直線行駛、再轉一個彎對準目標朝向。這正是飛機航線與 AGV 路徑常見的形狀。

26.5.3 Reeds-Shepp 路徑

若車輛**可以倒車**（如汽車），問題變得更複雜但也更靈活。Reeds 與 Shepp 於 1990 年將 Dubins 的結果推廣，證明最短路徑必定屬於 **48 種**基本組合之一（含前進與後退的切換）。

Reeds-Shepp 路徑的實用意義極為重大：它讓路邊停車這類需要來回的動作有了數學上的**最佳解**。自駕車的自動停車功能，其底層就是 Reeds-Shepp 路徑的計算。

兩者的比較：

表 26-5 Dubins 與 Reeds-Shepp 路徑

面向	Dubins	Reeds-Shepp
是否可倒車	否	是
基本組合數	6 種	48 種
路徑長度	較長	較短（可倒車繞近路）
適用車輛	飛機、固定翼無人機、單向 AGV	汽車、自駕車、可倒車 AGV

面向	Dubins	Reeds-Shepp
典型應用	航線規劃、巡邏路徑	自動停車、狹窄空間掉頭

26.5.4 混合 A*

實務上最常用的非完整規劃方法是 **Hybrid A*** (混合 A*)，它結合了格點搜尋與連續運動。

與標準 A* 的差別在於：

- **節點是連續的位姿** (x, y, θ)，而非格點中心。格點只用來記錄哪些區域已探索過 (每格最多保留一個節點)。
- **擴展時使用可行的運動基元** (motion primitive) ——例如「以最大轉角左轉前進 0.5 公尺」，而非簡單的上下左右。
- **啟發函數同時考慮兩種資訊**：忽略障礙的 Reeds-Shepp 距離 (考慮運動約束)、以及忽略運動約束的柵格距離 (考慮障礙)。取兩者的最大值，兼顧兩方面。

Hybrid A* 是自駕車路徑規劃的主流方法之一，DARPA 城市挑戰賽的優勝隊伍即採用此法。它的優勢是：產生的路徑天然可行 (不需後處理平滑)、且能處理倒車與狹窄空間。

26.6 人工勢場法

26.6.1 把規劃變成物理

人工勢場法 (Artificial Potential Field) 採取了一個完全不同的思路：**不做搜尋，而是讓機器人「被吸引」到目標、「被排斥」離開障礙。**

具體作法是定義一個勢場函數，機器人沿其負梯度方向運動：

$$U(q) = U_{\text{att}}(q) + U_{\text{rep}}(q) \quad \text{運動方向：} -\nabla U(q)$$

總勢場為吸引項與排斥項之和。機器人像一顆球，沿著勢場的斜坡滾向最低點。

吸引勢通常取與目標距離的平方 (像彈簧)：

$$U_{\text{att}}(q) = \frac{1}{2} k_{\text{att}} \cdot \|q - q_{\text{goal}}\|^2 \Rightarrow F_{\text{att}} = -k_{\text{att}}(q - q_{\text{goal}})$$

吸引力正比於距離，方向指向目標。

排斥勢只在障礙物附近作用，且距離越近力越大：

$$U_{\text{rep}}(q) = \frac{1}{2} k_{\text{rep}} (1/d - 1/d_0)^2 \quad \text{若 } d < d_0, \text{ 否則為 } 0$$

d 為到最近障礙的距離、 d_0 為影響範圍。 $d \rightarrow 0$ 時排斥力 $\rightarrow \infty$ ，確保不會碰撞。

26.6.2 優點：極為簡單且即時

勢場法的三個優點使它在實務上仍有一席之地：

- **計算極快**：只需計算當前位置的梯度，不需要搜尋。可以在微秒級完成，適合高頻的即時控制迴路。
- **天然處理動態障礙**：障礙物移動時，勢場自動改變，機器人立即反應——不需重新規劃。
- **與控制器天然銜接**：輸出直接是一個速度或力的向量，可直接送給控制器（甚至與第 20 章的阻抗控制形式相同）。

26.6.3 致命弱點：局部極小值

但勢場法有一個著名的、也是致命的問題：**局部極小值**（local minimum）。

◎ 局部極小值問題

當吸引力與排斥力恰好互相抵消時，機器人會**卡住不動**——即使目標明明可以到達。最典型的情境是 **U 形障礙物**（如一個凹陷的死角）：機器人被吸引進入凹陷，但兩側的牆壁把它推回中央，形成一個穩定的平衡點。機器人就這樣停在那裡，永遠到不了目標。

這使勢場法**不具備完備性**（第 26.2.2 節）——它可能找不到明明存在的解。

三種常見的緩解方法：

- **隨機擾動**：偵測到卡住時，隨機移動一小段距離跳出極小值。簡單但不保證有效。
- **調和勢場（Harmonic Potential）**：使用滿足 Laplace 方程的勢場，數學上可證明**沒有局部極小值**。代價是需要在整個空間上數值求解偏微分方程，計算量大且需要完整地圖。
- **只作為局部規劃器**：這是實務上最常用的作法——用**全域規劃器**（如 A^* ）決定大方向，勢場法只負責局部的即時避障。由於全域路徑已經避開了大的凹陷，局部極小值的風險大幅降低。

第三種作法直接引出了 26.8 節的分層架構。

26.7 動態視窗法

26.7.1 在速度空間中搜尋

動態視窗法 (Dynamic Window Approach, DWA) 是移動機器人最廣泛使用的局部規劃器，也是 ROS 導航的預設選項之一。

它的核心洞見是：**不要在位置空間中規劃路徑，而是直接在速度空間中選擇指令。**

對差速驅動車，控制指令是 (v, ω) 。DWA 在這個二維的速度空間中搜尋最佳的組合。「動態視窗」指的是**當前可達的速度範圍**，由三個限制共同決定：

- **速度限制**： $|v| \leq v_max$ 、 $|\omega| \leq \omega_max$ (馬達能力，第 23.2.4 節)。
- **加速度限制**：在下一個控制週期 Δt 內，速度只能改變 $a \cdot \Delta t$ ——這是**動態視窗**一詞的由來。
- **安全限制**：必須能在撞到障礙物前煞停。這給出 $v \leq \sqrt{2 \cdot d \cdot a_brake}$ ，其中 d 為到障礙的距離。

$$V_d = \{ (v, \omega) : \text{同時滿足上述三個限制} \}$$

動態視窗。它是速度空間中的一個小矩形區域，隨當前速度與障礙分布而變。

26.7.2 評分函數

在動態視窗內取樣多組 (v, ω) ，對每一組**模擬未來一小段時間的軌跡**（用第 23 章的運動模型），再依三個指標評分：

$$G(v, \omega) = \alpha \cdot \text{heading}(v, \omega) + \beta \cdot \text{clearance}(v, \omega) + \gamma \cdot \text{velocity}(v, \omega)$$

DWA 的評分函數。三項分別代表朝向目標、遠離障礙、行進快速。

- **heading (朝向)**：模擬軌跡終點的朝向與目標方向的一致程度。鼓勵朝目標前進。
- **clearance (淨空)**：模擬軌跡上到最近障礙物的距離。鼓勵遠離障礙。
- **velocity (速度)**：前進速度的大小。鼓勵快速移動，避免機器人為了安全而停滯不前。

取得分最高的 (v, ω) 作為本週期的控制指令。整個過程在每個控制週期（典型 10 ~ 20 Hz）重複一次。

26.7.3 DWA 的優勢與參數

DWA 相對於勢場法的關鍵優勢是：它**明確考慮了動力學限制**。勢場法算出的方向可能是機器人根本無法立即達成的（例如要求瞬間掉頭）；DWA 則只在「下一步真正做得到」的範圍內選擇，因此產生的指令必定可執行。

三個權重的調整直接影響行為：

表 26-6 DWA 權重的影響

症狀	原因	調整方向
機器人繞遠路、不直接朝目標	α (朝向) 太小	提高 α
太貼近障礙物、有擦撞風險	β (淨空) 太小	提高 β
行進緩慢、在原地猶豫	γ (速度) 太小	提高 γ
在狹窄通道前停住不敢進入	β 太大	降低 β 或縮小膨脹半徑
轉彎時撞到內側	模擬時間太短	延長軌跡模擬時間

DWA 的限制：它是**局部**的方法——只看未來一小段時間，因此同樣會遇到局部極小值（如走進死巷）。這再次說明了為何需要全域規劃器配合。

圖 26-2 動態視窗法的速度空間搜尋

建議配圖：左半為速度空間圖——橫軸為線速度 v 、縱軸為角速度 ω 。畫出三層限制：最外層的矩形為馬達速度上限、中間的小矩形為加速度限制下的可達範圍（動態視窗，以藍色標出）、其中被安全限制排除的區域以紅色斜線標示。視窗內取樣的候選點以小圓點表示，得分最高者以綠色星號標出。右半為對應的工作空間圖——機器人前方有兩個障礙物，數條由不同 (v, ω) 產生的模擬軌跡以灰色曲線畫出，被選中的軌跡以綠色粗線標示，並標註三項評分（朝向、淨空、速度）的示意。

26.8 分層導航架構

26.8.1 為何需要分層

前面各節的方法各有所長，也各有致命的限制：

- A^* 保證最佳，但需要完整地圖且無法即時反應動態障礙（重新規劃需要數十至數百毫秒）。
- DWA 與勢場法反應極快，但只看局部，會卡在死巷。

實務系統的解法是分層：把兩者結合，各司其職。

◎ 分層導航的核心思想

全域規劃器負責「大方向」：依完整的地圖，規劃一條由當前位置到目標的路徑。更新頻率低

（0.5~2 Hz 或只在必要時），但視野涵蓋整張地圖。**局部規劃器負責「邊走邊閃」：**依即時的感測資料，決定下一個控制週期該用什麼速度。更新頻率高（10~20 Hz），但只看周圍數公尺。**兩者的分工類似人類走路——**你腦中有一個「往車站方向」的大計畫，同時眼睛不斷地避開路上的行人與坑洞。

26.8.2 完整的導航堆疊

以 ROS 2 的導航系統 (Nav2) 為例，一個完整的導航堆疊包含五個部分：

1. **定位 (Localization)**：AMCL 或 SLAM，提供機器人在地圖中的位姿 (第 24、25 章)。
2. **代價地圖 (Costmap)**：把地圖與即時感測資料融合，產生含膨脹層的規劃用地圖。分為全域 (整張地圖) 與局部 (機器人周圍數公尺) 兩張。
3. **全域規劃器 (Global Planner)**：在全域代價地圖上用 A* 或 Dijkstra 找出路徑。
4. **局部規劃器 (Local Planner / Controller)**：DWA、TEB 或 MPPI，依局部代價地圖產生速度指令。
5. **行為樹 (Behavior Tree)**：協調上述模組，並處理異常情況 (見下節)。

26.8.3 代價地圖的分層設計

代價地圖本身也是分層的，這個設計極為實用：

表 26-7 代價地圖的圖層

圖層	來源	作用
靜態層 (Static)	SLAM 建立的地圖	已知的牆壁與固定障礙
障礙層 (Obstacle)	即時的光達 / 深度相機	動態障礙、地圖上沒有的東西
膨脹層 (Inflation)	上述兩層的處理結果	把障礙擴大機器人半徑 (第 26.1.3 節)
其他自訂層	如禁行區、社交距離	人為設定的規則

各層的結果依序疊加，得到最終的代價值。**這個分層設計的價值在於模組化**——可以獨立地加入或移除某一層，而不影響其他部分。例如在醫院的機器人可以加一層「病房門口減速區」，在工廠可以加一層「堆高機通道禁行區」。

26.8.4 復原行為

真實的導航系統必定會遇到規劃失敗的情況：被人包圍、走進死巷、定位漂移導致以為自己在牆裡。因此需要**復原行為** (recovery behaviors)。

典型的復原序列 (由溫和到激烈)：

1. **清除代價地圖**：清掉障礙層的舊資料，可能是暫時經過的人已經離開了。
2. **原地旋轉**：轉一圈重新掃描環境，更新周圍的障礙資訊。

3. **後退**：往後退一小段，脫離可能的卡住位置。
4. **重新定位**：若懷疑定位錯誤，觸發全域重定位（第 24.1.3 節的綁架問題）。
5. **放棄並求援**：通知操作員或上層系統。

復原行為的設計往往比規劃演算法本身更影響系統的可用性。一個規劃器可能 99% 的情況都能成功，但若剩下的 1% 會讓機器人永久卡住，整個系統就無法無人化運行。這是實務與論文的一個重要差距——論文報告成功率，實務關心失敗時怎麼辦。

26.9 機械手臂的路徑規劃

26.9.1 與移動機器人的異同

本章至此的討論多以移動機器人為例，但路徑規劃同樣是機械手臂的核心問題——而且更困難。

相同之處：位形空間的抽象、碰撞檢測的需求、取樣式方法的適用性，兩者完全相通。

不同之處有四點：

表 26-8 移動機器人與機械手臂規劃的比較

面向	移動機器人	機械手臂
位形空間維度	2~3	6~7 (或更高)
主要方法	格點搜尋 (A*、DWA)	取樣式 (RRT、PRM)
碰撞檢測對象	機器人本體 vs 環境	各連桿 vs 環境、以及自我碰撞
目標的指定	位置 (二維點)	位姿 (需逆運動學，可能多解)
非完整約束	有 (不能側移)	無 (各關節獨立)
典型規劃時間	毫秒~數十毫秒	數十毫秒~數秒

第三列特別值得注意：機械手臂必須檢查**自我碰撞**——手臂的某一節可能撞到另一節，或撞到自己的基座。這使碰撞檢測的成本大幅增加 (n 個連桿需要 $O(n^2)$ 次配對檢查)。

26.9.2 目標的多重性

另一個機械手臂特有的問題是：**目標通常以末端位姿指定，而非關節角度**。

這帶來一個機會與一個困難。**機會**是：由第 10 章可知，同一個末端位姿可能對應多組關節

角度（六軸手臂典型有八組）。規劃器可以**把所有的逆解都當成候選目標**，只要能到達其中任何一個就算成功——這大幅提高了找到解的機率。

困難是：某些逆解可能無法到達（被障礙擋住、或超出關節極限）。因此實務的作法是：先求出所有逆解、過濾掉不可行者、再以剩餘的解作為多目標的規劃問題。

26.9.3 路徑平滑

取樣式方法（尤其是 RRT）產生的路徑通常**曲折而不自然**——因為它是由隨機取樣點連接而成的。直接執行會使機器人走出鋸齒狀的動作，既不美觀也浪費時間。

因此規劃後必須做**路徑平滑**（path smoothing）。最常用的方法是**捷徑法**（shortcutting）：

1. 隨機選取路徑上的兩個點。
2. 檢查它們之間的直線是否無碰撞。
3. 若無碰撞，就用這條直線取代原本兩點之間的曲折路段。
4. 重複數百次，路徑會逐漸被拉直。

捷徑法簡單有效，通常能把 RRT 路徑的長度縮短 20~40%。之後再交給第 15 章的軌跡生成加上時間參數化，即可執行。

完整的流程是：逆運動學（求目標關節角）→ 路徑規劃（RRT）→ 路徑平滑（捷徑法）→ 軌跡生成（第 15 章）→ 動力學檢驗（第 15.8 節）→ 控制執行（第 17 章）。這條鏈完整地串起了本書第二、三、四篇的內容。

26.10 現代發展

26.10.1 最佳化式規劃

除了搜尋與取樣，還有第三條路線：**把規劃直接表述為一個最佳化問題**。

代表性的方法是 **CHOMP** 與 **TrajOpt**。它們的作法是：先給一條初始軌跡（可能是直線，即使它會穿過障礙），然後用梯度下降逐步把它「推」出障礙區，同時最小化路徑長度與平滑度。

$$\min_{\xi} f_{\text{smooth}}(\xi) + f_{\text{obstacle}}(\xi)$$

軌跡最佳化的目標函數。第一項鼓勵平滑、第二項懲罰接近障礙。

優點：產生的軌跡天然平滑（不需後處理）、能同時考慮多個目標（平滑、避障、力矩限制）、且收斂快。

缺點：只能找到**局部**最佳解——若初始軌跡的「這一側」有障礙，它不會考慮繞到「另一側」。因此對複雜的環境，可能需要多個初始猜測。

這與第 18 章的最佳控制、第 25 章的圖最佳化屬於同一類數學工具——**都是用梯度把一個初始猜測推向更好的解**。

26.10.2 學習式規劃

深度學習在規劃中的角色，與第 25 章的 SLAM 類似——**主要是加速而非取代**。三個具體的應用方向：

- **學習取樣分布：**與其均勻隨機取樣，不如學習「哪些區域比較可能包含解」。例如在狹窄通道附近多取樣。這能使 RRT 的收斂速度提升數倍。
- **學習啟發函數：**用神經網路預測「由此點到目標的真實剩餘代價」，取代手工設計的歐氏距離。這能使 A* 的搜尋更有方向感。
- **端到端的策略：**直接由感測資料學習「該往哪走」，跳過顯式的規劃。這是第 31 章強化學習的路線，在複雜或難以建模的環境中有其價值，但缺乏保證。

目前的共識與 SLAM 相同：學習擅長「哪裡值得探索」這類需要經驗與直覺的判斷，而傳統方法擅長「這條路是否真的可行」這類需要嚴格保證的驗證。混合式架構仍是主流。

26.10.3 多機器人與社會導航

兩個近年的重要方向：

多機器人規劃：當數十台 AGV 在同一個倉庫運行時，必須避免互相碰撞與死鎖。這需要**協調**——可以是集中式（一個中央規劃器分配路徑）或分散式（各自規劃並協商）。核心的挑戰是**死鎖偵測與解除**：兩台機器人在狹窄通道相遇時，必須有一方讓路，而這需要一致的規則。

社會導航（Social Navigation）：當機器人在人群中移動時，「不碰撞」是不夠的——它還必須符合**人類的社會慣例**。例如：不要突然從人的背後竄出、與人保持適當的社交距離、在走廊上靠右行、在通過門口時禮讓。這些規則難以用傳統的代價函數表達，是目前研究的熱點，也直接關係到機器人能否被社會接受（第 35 章）。

26.11 由導航邁向飛行與行走

26.11.1 本章的成果

本章完成了自主移動的最後一塊拼圖。你現在能夠：運用位形空間把機器人簡化為一個點、實作 A* 並理解啟發函數的可採納性、運用 PRM 與 RRT 在高維空間中規劃、處理非完整約束下的路徑 (Dubins 與 Reeds-Shepp)、實作勢場法與 DWA 進行即時避障、並理解分層導航架構中全域與局部規劃的分工。

最重要的是三個核心洞見：一、**位形空間的抽象**——把「複雜形狀的碰撞」轉化為「一個點是否在禁區」。二、**不建構、只查詢**——取樣式方法放棄完整表示，換取高維空間中的可行性。三、**分層的必然性**——全域看得遠但反應慢、局部反應快但看得近，兩者缺一不可。

26.11.2 第四篇的整合

第 22 ~ 26 章共同構成了一個完整的自主移動系統：

看見 (第 22 章) → 移動 (第 23 章) → 估測 (第 24 章) → 建圖與定位 (第 25 章) →
規劃與導航 (第 26 章)

第四篇的邏輯鏈。每一章都是下一章的必要輸入。

這五章也共同體現了一個與第三篇截然不同的思維方式。第三篇的世界是**確定的**——給定關節角度，末端位置唯一確定；本篇的世界則充滿**不確定性**——每一個感測讀值都帶著雜訊、每一個估計都帶著誤差、每一次辨識都可能出錯。

因應這個轉變，本篇反覆出現三個原則：一、**量化不確定性而非忽略它** (第 24 章的協方差)；二、**依可信度融合而非二選一** (第 24 章的 Kalman 增益)；三、**寧可保守也不要誤判** (第 25 章的迴路閉合、第 26 章的膨脹半徑)。

26.11.3 接下來：離開地面

但第 22 ~ 26 章的討論都隱含一個假設：**機器人在地面上、以輪子移動**。這使問題大幅簡化——只需考慮二維或三維的位形空間、有地面提供穩定的支撐、且運動學相對單純。

第 27 章 (無人機) 與第 28 章 (腿式機器人) 將處理兩種截然不同的移動方式，它們共有一個關鍵特性：**欠驅動** (第 23.3.4 節的概念在此重現)。

無人機有六個自由度但只有四個控制輸入（四旋翼）——它無法一邊保持水平一邊橫移，必須先傾斜才能移動。而且它一刻都不能停止控制——馬達一停就會墜落。

腿式機器人在騰空階段完全無法控制質心的軌跡（只能靠落地時的觸地點修正），且必須持續維持平衡。它的位形空間維度高達數十維，且接觸狀態不斷切換。

這兩者將把第三篇的控制理論（尤其是第 18 章的 LQR 與第 19 章的穩定性）與第四篇的感知定位技術完整地結合起來——它們是全書技術的綜合考驗。

完整計算例題

例題 26-1 A* 的搜尋與啟發函數

題目：在一個 4 連通的格點地圖上，機器人由 (0,0) 走向 (5,3)，每步代價為 1。（a）計算曼哈頓距離啟發函數在起點的值。（b）若某節點在 (2,1)，計算其 f 值（設 $g = 3$ ）。（c）判斷歐氏距離在此 4 連通情形下是否可採納。（d）若把啟發函數乘上 2（加權 A*），討論其後果。

解（a）：曼哈頓距離為兩軸差值的絕對值之和：

$$h(0,0) = |5 - 0| + |3 - 0| = 5 + 3 = 8$$

起點的啟發值。由於 4 連通每步只能走一格且代價為 1，這正好等於真實的最短距離（無障礙時）。

解（b）：

$$\begin{aligned} h(2,1) &= |5 - 2| + |3 - 1| = 3 + 2 = 5 \\ f(2,1) &= g + h = 3 + 5 = 8 \end{aligned}$$

評價函數值。

判讀： $g = 3$ 代表已走三步，但曼哈頓距離顯示由 (0,0) 到 (2,1) 最少需要 3 步——因此這條路徑沒有繞路。 $f = 8$ 等於起點的 h 值，代表這條路徑仍在最短路徑上。

解（c）：歐氏距離為：

$$h_{\text{euclid}}(0,0) = \sqrt{5^2 + 3^2} = \sqrt{34} \approx 5.83$$

歐氏距離。

由於 $5.83 < 8$ （真實最短距離），歐氏距離是可採納的（沒有高估）。但它低估太多——這會使 A* 探索大量不必要的節點，效率接近 Dijkstra。

結論：在 4 連通格點上應使用曼哈頓距離，它既可採納又緊貼真實值，效率最高。若誤用歐

氏距離，結果仍正確但速度會明顯下降。

解 (d)：加權 A^* ($h' = 2h$) 時：

$$h'(0,0) = 2 \times 8 = 16 > 8 = h^*$$

高估了真實代價，不再可採納。

後果是：**不再保證找到最短路徑**，但可以證明找到的路徑長度**不超過最佳解的 2 倍**（一般地，權重為 w 時保證不超過 w 倍）。

取捨：搜尋速度通常能提升數倍到數十倍（因為 A^* 會更「急切」地衝向目標，少探索側面的節點）。在即時系統中，「1.2 倍長的路徑但快 10 倍算出來」往往比「最短路徑但來不及」更有價值。實務上常用 $w = 1.1 \sim 1.5$ 。

例題 26-2 位形空間與膨脹

題目：某圓形移動機器人半徑 $r = 0.35 \text{ m}$ ，在一個走廊中運行。走廊寬 1.2 m ，中間有一根直徑 0.4 m 的圓柱。（a）求膨脹後的圓柱等效半徑。（b）判斷機器人能否從圓柱的一側通過。（c）若機器人改為長 0.9 m 、寬 0.5 m 的長方形，求其外接圓與內切圓半徑。（d）討論用外接圓膨脹的後果。

解 (a)：圓柱原半徑 0.2 m ，膨脹機器人半徑 0.35 m ：

$$R_{\text{inflated}} = 0.2 + 0.35 = 0.55 \text{ m}$$

膨脹後的等效半徑。在位形空間中，機器人的中心不能進入這個範圍。

解 (b)：走廊寬 1.2 m ，圓柱在中央。圓柱兩側各剩：

$$\text{剩餘空間} = (1.2 - 0.4)/2 = 0.4 \text{ m (每側)}$$

工作空間中每側的實際通道寬度。

在位形空間中，機器人中心可行走的範圍為：

$$\text{可行寬度} = 0.4 - 0.35 = 0.05 \text{ m}$$

扣除機器人半徑後，中心只剩 5 公分的可行帶。

理論上可以通過 ($0.05 \text{ m} > 0$)，但**實務上極為危險**——考慮到定位誤差（第 24 章，典型數公分）與控制誤差，5 公分的餘裕遠遠不夠。

工程判斷：應視為不可通行，並在代價地圖中把此處標為高代價區，讓規劃器優先選擇其他路徑。實務上建議至少保留機器人半徑 20 ~ 30% 的餘裕（本例約 10 公分）。

解 (c)：長方形 $0.9 \times 0.5 \text{ m}$ ：

$$\text{外接圓半徑} = \sqrt{((0.9/2)^2 + (0.5/2)^2)} = \sqrt{(0.2025 + 0.0625)} = \sqrt{0.265} = 0.515 \text{ m}$$

$$\text{內切圓半徑} = \min(0.9, 0.5)/2 = 0.25 \text{ m}$$

兩個半徑。外接圓是內切圓的兩倍以上。

解 (d)：若用外接圓 (0.515 m) 膨脹，圓柱的等效半徑變為 $0.2 + 0.515 = 0.715 \text{ m}$ 。此時：

$$\text{可行寬度} = 0.4 - 0.515 = -0.115 \text{ m} < 0$$

通道被判定為完全不可通行。

但實際上，若機器人沿走廊方向擺放（長邊平行於走廊），它的等效寬度只有 0.5 m，需要的通道為 $0.5/2 = 0.25 \text{ m}$ ——0.4 m 的通道是綽綽有餘的。

這正是 26.1.3 節提到的限制：用外接圓膨脹過於保守，會誤判可通行的通道。實務的解法是 ROS costmap 的雙半徑設計——用內切圓做快速的粗略判斷（絕對安全的區域）、用外接圓標記絕對危險的區域、兩者之間的區域則用精確的多邊形碰撞檢測逐一判斷。這是「精度」與「速度」的分層取捨。

例題 26-3 動態視窗的計算

題目：某差速驅動機器人 $v_{\max} = 0.8 \text{ m/s}$ 、 $\omega_{\max} = 1.5 \text{ rad/s}$ 、線加速度上限 $a_v = 0.5 \text{ m/s}^2$ 、角加速度上限 $a_\omega = 2.0 \text{ rad/s}^2$ 。控制週期 $\Delta t = 0.1 \text{ s}$ 。當前速度 $(v, \omega) = (0.4, 0.3)$ 。(a) 求加速度限制下的動態視窗。(b) 若正前方 1.5 m 處有障礙物，煞車減速度為 0.5 m/s^2 ，求安全速度上限。(c) 綜合求出最終的動態視窗。(d) 討論若控制週期加長為 0.5 s 的影響。

解 (a)：加速度限制在一個週期內能改變的速度為：

$$\Delta v_{\max} = a_v \times \Delta t = 0.5 \times 0.1 = 0.05 \text{ m/s}$$

$$\Delta \omega_{\max} = a_\omega \times \Delta t = 2.0 \times 0.1 = 0.2 \text{ rad/s}$$

一個週期內的速度變化量。

$$v \in [0.4 - 0.05, 0.4 + 0.05] = [0.35, 0.45] \text{ m/s}$$

$$\omega \in [0.3 - 0.2, 0.3 + 0.2] = [0.10, 0.50] \text{ rad/s}$$

動態視窗。注意這個範圍極小——這正是「動態」的意義。

觀察：雖然機器人的最高速度是 0.8 m/s，但在下一個週期它最多只能達到 0.45 m/s。DWA 只在這個小範圍內搜尋，因此產生的指令必定可執行。

解 (b)：安全條件是「能在撞到障礙前煞停」。由運動學：

$$v \leq \sqrt{2 \cdot d \cdot a_{\text{brake}}} = \sqrt{2 \times 1.5 \times 0.5} = \sqrt{1.5} = 1.22 \text{ m/s}$$

安全速度上限。

由於 1.22 m/s > 0.8 m/s (馬達上限)，在此距離下安全限制不起作用——機器人可以用任何速度前進。

若障礙在 0.3 m 處： $v \leq \sqrt{2 \times 0.3 \times 0.5} = \sqrt{0.3} = 0.548 \text{ m/s}$ ——此時安全限制開始生效。

若障礙在 0.1 m 處： $v \leq \sqrt{0.1} = 0.316 \text{ m/s}$ ——限制更嚴格。

解 (c)：綜合三個限制取交集：

$$v \in [0.35, 0.45] \cap [0, 0.8] \cap [0, 1.22] = [0.35, 0.45] \text{ m/s}$$

$$\omega \in [0.10, 0.50] \cap [-1.5, 1.5] = [0.10, 0.50] \text{ rad/s}$$

最終的動態視窗。本例中由加速度限制主導。

DWA 會在這個範圍內取樣 (例如 $5 \times 5 = 25$ 組)，對每組模擬軌跡並評分，取最佳者。

解 (d)：若 $\Delta t = 0.5 \text{ s}$ ：

$$\Delta v_{\text{max}} = 0.5 \times 0.5 = 0.25 \text{ m/s} \Rightarrow v \in [0.15, 0.65]$$

$$\Delta \omega_{\text{max}} = 2.0 \times 0.5 = 1.0 \text{ rad/s} \Rightarrow \omega \in [-0.7, 1.3] \text{ (受 } \omega_{\text{max}} \text{ 限制)}$$

動態視窗大幅擴大。

這看似是好事 (選擇變多)，實際上是壞事，理由有三：

- **反應變慢：**兩次決策之間隔了 0.5 秒，機器人在這段時間內是「盲目」的。若有人突然走出來，可能來不及反應。
- **軌跡模擬的誤差變大：**DWA 假設速度在一個週期內固定，週期越長這個假設越不準。
- **與第 19.9.3 節的結論一致：**取樣週期過長會侵蝕穩定裕度。

實務建議：DWA 的控制週期應在 50 ~ 200 ms 之間 (5 ~ 20 Hz)。過短則計算來不及，過長則反應遲鈍。

程式範例：A*、RRT 與動態視窗法

以下 Python 程式實作三種代表性的規劃演算法：格點上的 A*、取樣式的 RRT，以及局部規劃的 DWA，並比較它們的行為。

```
import numpy as np
import heapq, random

# ===== 地圖與碰撞檢測 =====
class GridMap:
    def __init__(s, w=50, h=30):
        s.w, s.h = w, h
        s.grid = np.zeros((h, w), dtype=int) # 0 空曠, 1 障礙

    def add_rect(s, x0, y0, x1, y1):
        s.grid[y0:y1, x0:x1] = 1

    def free(s, x, y):
        xi, yi = int(round(x)), int(round(y))
        if not (0 <= xi < s.w and 0 <= yi < s.h):
            return False
        return s.grid[yi, xi] == 0

    def inflate(s, r):
        """障礙物膨脹 (26.1.3 節)"""
        out = s.grid.copy()
        ri = int(np.ceil(r))
        ys, xs = np.where(s.grid == 1)
        for y, x in zip(ys, xs):
            y0, y1 = max(0, y-ri), min(s.h, y+ri+1)
            x0, x1 = max(0, x-ri), min(s.w, x+ri+1)
            out[y0:y1, x0:x1] = 1
        m = GridMap(s.w, s.h); m.grid = out
        return m

# ===== A* 演算法 =====
def astar(gmap, start, goal, heuristic="manhattan", weight=1.0):
    """26.3 節的 A*。回傳 (路徑, 展開節點數)"""
    def h(n):
        dx, dy = abs(n[0]-goal[0]), abs(n[1]-goal[1])
        if heuristic == "manhattan": return dx + dy
        if heuristic == "euclidean": return np.hypot(dx, dy)
        return 0.0 # Dijkstra
    open_set = [(h(start)*weight, 0.0, start, None)]
    came, gscore, expanded = {}, {start: 0.0}, 0
    while open_set:
        f, g, cur, parent = heapq.heappop(open_set)
        if cur in came: continue
        came[cur] = parent
        expanded += 1
        if cur == goal:
            path, n = [], cur
            while n is not None:
                path.append(n); n = came[n]
            return path[::-1], expanded
        for dx, dy in ((1,0), (-1,0), (0,1), (0,-1)):
            nb = (cur[0]+dx, cur[1]+dy)
```

```

        if not gmap.free(*nb) or nb in came: continue
        ng = g + 1.0
        if ng < gscore.get(nb, 1e18):
            gscore[nb] = ng
            heapq.heappush(open_set, (ng + weight*h(nb), ng, nb, cur))
    return None, expanded

# ===== RRT 演算法 =====
def rrt(gmap, start, goal, max_iter=5000, step=1.5, goal_bias=0.1,
        goal_tol=2.0, seed=0):
    """26.4.3 節的 RRT。回傳 (路徑, 節點數)"""
    rng = random.Random(seed)
    nodes = [np.array(start, float)]
    parents = [-1]
    for it in range(max_iter):
        # 目標偏置
        if rng.random() < goal_bias:
            x_rand = np.array(goal, float)
        else:
            x_rand = np.array([rng.uniform(0, gmap.w),
                               rng.uniform(0, gmap.h)])

        # 找最近節點
        d = [np.linalg.norm(n - x_rand) for n in nodes]
        i_near = int(np.argmin(d))
        x_near = nodes[i_near]

        # 前進一步
        vec = x_rand - x_near
        norm = np.linalg.norm(vec)
        if norm < 1e-9: continue
        x_new = x_near + vec/norm*min(step, norm)

        # 碰撞檢查 (沿線取樣)
        ok = True
        for t in np.linspace(0, 1, 8):
            p = x_near + t*(x_new - x_near)
            if not gmap.free(p[0], p[1]): ok = False; break
        if not ok: continue
        nodes.append(x_new); parents.append(i_near)
        if np.linalg.norm(x_new - np.array(goal)) < goal_tol:
            path, i = [], len(nodes)-1
            while i != -1:
                path.append(tuple(nodes[i])); i = parents[i]
            return path[::-1], len(nodes)
    return None, len(nodes)

def path_length(path):
    if not path: return float('inf')
    return sum(np.linalg.norm(np.array(path[i+1]) - np.array(path[i]))
               for i in range(len(path)-1))

def shortcut(gmap, path, iters=300, seed=0):
    """26.9.3 節的捷徑法平滑"""
    rng = random.Random(seed)
    p = [np.array(q, float) for q in path]
    for _ in range(iters):
        if len(p) < 3: break
        i = rng.randint(0, len(p)-3)
        j = rng.randint(i+2, len(p)-1)

```

```

    ok = True
    for t in np.linspace(0, 1, 20):
        q = p[i] + t*(p[j] - p[i])
        if not gmap.free(q[0], q[1]): ok = False; break
    if ok: p = p[:i+1] + p[j:]
    return [tuple(q) for q in p]

# ===== 建立測試地圖 =====
gm = GridMap(50, 30)
gm.add_rect(15, 0, 18, 20)          # 上方伸下的牆
gm.add_rect(30, 10, 33, 30)        # 下方伸上的牆
gm.add_rect(40, 5, 44, 12)         # 一個方塊
start, goal = (2, 15), (47, 15)

print("=== A* : 不同啟發函數的比較 ===")
print(f"{'啟發函數':<14}{'權重':>6}{'路徑長度':>10}{'展開節點':>10}{'最佳?':>8}")
for hname, w in (("dijkstra", 1.0), ("euclidean", 1.0),
                 ("manhattan", 1.0), ("manhattan", 1.5),
                 ("manhattan", 3.0)):
    path, exp = astar(gm, start, goal, hname, w)
    opt = "是" if w == 1.0 else f"≤{w:.1f}×"
    print(f"{'hname':<14}{'w':>6.1f}{'len(path)-1':>10d}{'exp':>10d}{'opt':>8}")
print("→ 曼哈頓最有效率；加權 A* 用最佳性換速度")

# ===== RRT 與平滑 =====
print("\n=== RRT : 取樣式規劃 ===")
print(f"{'種子':>6}{'節點數':>9}{'原始長度':>11}{'平滑後':>10}{'縮短':>9}")
for seed in range(5):
    p, n = rrt(gm, start, goal, seed=seed)
    if p is None:
        print(f"{'seed':>6d}{'n':>9d}{'失敗':>11}"); continue
    L0 = path_length(p)
    ps = shortcut(gm, p, seed=seed)
    L1 = path_length(ps)
    print(f"{'seed':>6d}{'n':>9d}{'L0':>11.2f}{'L1':>10.2f}{'(1-L1/L0)*100':>8.1f}%")
print("→ RRT 每次結果不同 (隨機性) ; 捷徑法可縮短 20~40%")

# ===== 目標偏置的影響 =====
print("\n=== 目標偏置對 RRT 的影響 (10 次平均) ===")
print(f"{'偏置率':>8}{'成功率':>9}{'平均節點數':>12}{'平均長度':>11}")
for gb in (0.0, 0.05, 0.1, 0.3, 0.6):
    ns, Ls, ok = [], [], 0
    for sd in range(10):
        p, n = rrt(gm, start, goal, goal_bias=gb, seed=sd)
        if p: ok += 1; ns.append(n); Ls.append(path_length(p))
    if ns:
        print(f"{'gb':>8.2f}{'ok':>10d}{'np.mean(ns)':>12.0f}{'np.mean(Ls)':>11.2f}")
    else:
        print(f"{'gb':>8.2f}{'ok':>10d}{'-'>12}{'-'>11}")
print("→ 適度的目標偏置 (5~10%) 能加速收斂；過高則易卡在障礙前")

```

```
# ===== 動態視窗法 (例題 26-3) =====
print("\n=== 例題 26-3：動態視窗的計算 ===")
class DWA:
    def __init__(s, v_max=0.8, w_max=1.5, av=0.5, aw=2.0, dt=0.1):
        s.v_max, s.w_max, s.av, s.aw, s.dt = v_max, w_max, av, aw, dt

    def window(s, v, w, obs_dist=None, a_brake=0.5):
        vs = [max(-s.v_max, v - s.av*s.dt), min(s.v_max, v + s.av*s.dt)]
        ws = [max(-s.w_max, w - s.aw*s.dt), min(s.w_max, w + s.aw*s.dt)]
        if obs_dist is not None:
            v_safe = np.sqrt(2*obs_dist*a_brake)
            vs[1] = min(vs[1], v_safe)
        return vs, ws

dwa = DWA()
vs, ws = dwa.window(0.4, 0.3)
print(f"(a) 加速度限制:  $v \in [{vs[0]:.3f}, {vs[1]:.3f}]$  m/s, "
      f" $w \in [{ws[0]:.3f}, {ws[1]:.3f}]$  rad/s")
print(f"\n(b) 不同障礙距離的安全速度上限:")
for d in (1.5, 0.5, 0.3, 0.1):
    v_safe = np.sqrt(2*d*0.5)
    binding = "生效" if v_safe < 0.8 else "不生效"
    print(f"    障礙 {d:.1f} m  $\rightarrow v \leq {v_safe:.3f}$  m/s ({binding})")
vs2, ws2 = dwa.window(0.4, 0.3, obs_dist=0.3)
print(f"\n(c) 障礙 0.3 m 時的最終視窗: "
      f" $v \in [{vs2[0]:.3f}, {vs2[1]:.3f}]$ ,  $w \in [{ws2[0]:.3f}, {ws2[1]:.3f}]$ ")
print(f"\n(d) 控制週期的影響:")
for dt in (0.05, 0.1, 0.3, 0.5):
    d2 = DWA(dt=dt)
    v2, w2 = d2.window(0.4, 0.3)
    print(f"     $\Delta t={dt:.2f}$ s  $\rightarrow v$  範圍寬度  ${v2[1]-v2[0]:.3f}$  m/s, "
          f" $w$  範圍寬度  ${w2[1]-w2[0]:.3f}$  rad/s")
print("     $\rightarrow$  週期越長視窗越大，但反應越遲鈍 (見 19.9.3 節)")

# ===== 膨脹的影響 (例題 26-2) =====
print("\n=== 例題 26-2：膨脹半徑對可通行性的影響 ===")
gm2 = GridMap(40, 20)
gm2.add_rect(18, 8, 22, 12) # 走廊中央的柱子
print(f"{ '膨脹半徑':>10}{ 'A* 結果':>12}{ '路徑長度':>11}")
for r in (0, 1, 2, 3, 4):
    gi = gm2.inflate(r) if r > 0 else gm2
    p, _ = astar(gi, (2, 10), (37, 10))
    if p:
        print(f"{r:10d}{ '成功':>12}{len(p)-1:11d}")
    else:
        print(f"{r:10d}{ '無解':>12}{ '-':>11}")
print("\n $\rightarrow$  膨脹過大會使原本可通行的路徑消失 (例題 26-2d)")
```

請試著修改：(一) 把 A* 的權重設為 5.0，觀察路徑長度增加多少、展開節點減少多少。

(二) 把 RRT 的步長由 1.5 改為 0.5 與 4.0，觀察節點數與成功率的變化。(三) 在地圖中加入一個 U 形障礙，比較 A* (能繞出) 與勢場法 (會卡住) 的差異。

實作：導航系統的建立與調校

本章實作以 ROS 2 Nav2 為主 (可用模擬器或實體機器人)。若無 ROS 環境，前四項可用純 Python 完成。

- 1. A* 實作與比較：**在自建的格點地圖上實作 Dijkstra、A* (曼哈頓)、加權 A*。記錄三者的路徑長度與展開節點數，驗證表 26-3 的預期。
- 2. 啟發函數的實驗：**對同一張地圖，測試四種啟發函數 (零、歐氏、曼哈頓、對角線)，繪出「展開節點數 vs 啟發函數」的長條圖。討論為何曼哈頓在 4 連通下最有效率。
- 3. RRT 實作：**實作基本的 RRT 與目標偏置，在含狹窄通道的地圖上測試。執行 20 次，統計成功率與路徑長度的變異——親身體會取樣式方法的隨機性。
- 4. 路徑平滑：**實作捷徑法，量化平滑前後的路徑長度差異。觀察平滑後的路徑是否更接近人類的直覺。
- 5. 膨脹半徑的調校：**在 Nav2 的代價地圖中，測試不同的 inflation_radius (0.2、0.35、0.5、0.8 m)。記錄機器人能否通過狹窄通道、以及是否會過於貼近牆面。找出適合你的機器人的值。
- 6. DWA 權重調校：**系統性地改變 DWA 的三個權重 (path_distance_bias、goal_distance_bias、occdist_scale)，觀察機器人的行為變化。製作表 26-6 的實測版本。
- 7. 局部極小值測試：**在環境中放置一個 U 形障礙 (如三面圍起的死角)，讓機器人的目標在其後方。觀察：(甲) 只用局部規劃器時是否卡住；(乙) 加入全域規劃器後是否能繞出。這是 26.6.3 節的實驗版本。
- 8. 動態障礙測試：**在機器人行進途中放入障礙物 (如紙箱)，觀察局部規劃器的反應時間與避讓行為。測試不同的控制週期對反應速度的影響。
- 9. 復原行為測試：**故意讓機器人陷入困境 (用障礙物包圍)，觀察 Nav2 的復原行為序列是否被觸發、以及是否成功脫困。
- 10. (進階) 多目標導航：**規劃一條經過多個中途點的巡邏路徑，觀察系統如何處理連續的導航目標，並記錄總時間與路徑效率。

表 26-9 導航系統調校記錄表（前兩列為範例）

參數	設定值	行為表現	問題	調整方向
inflation_radius	0.5 m	與牆保持適當距離	無法通過 0.9 m 走道	降至 0.35 m
occdist_scale	0.02	貼近障礙物行走	多次擦撞警告	提高至 0.1
（自行填寫其餘測試）				

國中小也看得懂：走迷宮的兩個層次

你有沒有玩過走迷宮的遊戲？從入口走到出口，中間有很多牆擋住。你會怎麼找路？

最笨的方法是**每一條路都試試看**，走到死路就退回來換一條。這樣一定找得到，但很花時間。電腦有一個方法就是這樣，叫做「**Dijkstra**」（唸作「戴克斯特拉」）——它像水一樣往四面八方慢慢流開，直到流到出口。

聰明一點的方法是：**朝著出口的方向優先找**。如果出口在右邊，那就先試右邊的路，不要浪費時間往左找。這個方法叫做「**A 星**」（寫作 A^* ），是現在最常用的找路方法——連你玩的電動遊戲裡，那些會追著你跑的角色，用的也是它！

但機器人找路還有一個麻煩：**它自己有身體**。你在迷宮裡是一個小點，但機器人是一台大車，路太窄它就過不去。

工程師想到一個很聰明的辦法：**把牆壁「加厚」**。假設機器人半徑是 30 公分，那就把每一面牆都往外長胖 30 公分。這樣一來，就可以把機器人當成一個「點」來找路了——只要那個點不碰到加厚後的牆，真正的機器人就一定不會撞到！

最後還有一件事。機器人找路要分成**兩個層次**，就像你去朋友家：

大方向：先看地圖決定「搭捷運到某站再走過去」。這個計畫不會常常改。

邊走邊閃：路上有人擋住、有水坑、有腳踏車，你會隨時繞開。這個要一直做。

機器人也一樣——腦中有一個大計畫，眼睛不停地看著前方閃避。兩個都要有，缺一個都會出問題！

叫獸提醒與常見錯誤

- 誤區一：忘記機器人有體積。必須用位形空間或障礙物膨脹處理，否則規劃出的路徑會使機器人擦撞。
- 誤區二：對非圓形機器人用外接圓膨脹。這過於保守，會誤判可通行的狹窄通道；應採內切圓加精確碰撞檢測的雙層設計。
- 誤區三：膨脹半徑設得過大。可能使原本可通行的通道完全消失（例題 26-2d）。
- 誤區四：使用不可採納的啟發函數卻期待最佳解。h 高估時 A* 不再保證最短路徑。
- 誤區五：在高維空間使用格點法。六維以上會遭遇維數詛咒，必須改用取樣式方法。
- 誤區六：以為 RRT 找到的是最短路徑。原始 RRT 非最佳，需要 RRT* 或後處理的路徑平滑。
- 誤區七：期待取樣式方法能判定無解。它們只有機率完備性——找不到解時，無法區分是「無解」還是「取樣不夠」。
- 誤區八：只用勢場法或 DWA 而無全域規劃器。局部方法會卡在 U 形障礙的局部極小值中。
- 誤區九：忽略非完整約束。對汽車式機器人，位形空間中的直線段未必可行，應使用 Dubins、Reeds-Shepp 或 Hybrid A*。
- 誤區十：DWA 的控制週期設得過長。視窗雖然變大但反應遲鈍，且軌跡模擬的誤差增加。
- 誤區十一：忽略復原行為的設計。規劃器 99% 成功率仍不足以無人化運行——關鍵在於剩下 1% 失敗時怎麼辦。
- 誤區十二：機械手臂規劃時忘記檢查自我碰撞。連桿之間可能互撞，這需要 $O(n^2)$ 的配對檢查。

課堂討論

1. 26.4.1 節的「不建構、只查詢」洞見使高維規劃成為可能。請討論這個思維在本書其他章節的對應（提示：第 18 章的維數詛咒、第 24 章的粒子濾波器）。
2. 加權 A* 犧牲最佳性換取速度，保證路徑不超過最佳解的 w 倍。請討論在什麼應用中這個交易是划算的？什麼應用中不可接受？
3. 分層導航架構中，全域規劃器與局部規劃器可能給出衝突的建議（如全域說往左、局部說往右）。請設計一個仲裁機制，並討論其可能的失效情境。
4. 26.8.4 節主張「復原行為的設計往往比規劃演算法本身更影響系統可用性」。請討論這個觀察對研究與實務優先順序的啟示。

5. 社會導航要求機器人符合人類的社會慣例（如靠右行、保持社交距離）。請討論這些規則該如何形式化為代價函數，以及會遇到什麼困難。

章末摘要

- 位形空間是路徑規劃的核心抽象：空間中每一點代表機器人的一個完整姿態，機器人因而簡化為一個點。
- 障礙物膨脹（Minkowski 和）使平移機器人可視為點；對非圓形機器人應採內切圓與外接圓的雙層設計。
- 演算法的三個判準：完備性（有解必找到）、最佳性（找到最短）、計算效率；三者通常無法兼得。
- 格點搜尋完備且最佳，取樣式方法僅機率完備，勢場法不完備。
- A^* 的評價函數 $f = g + h$ ；啟發函數可採納（ $h \leq h^*$ ）時保證找到最短路徑。
- h 越接近真實值搜尋越高效； $h = 0$ 時退化為 Dijkstra；加權 A^* 犧牲最佳性換速度，保證不超過 w 倍。
- 格點法在高維遭遇維數詛咒，六維以上不可行。
- 取樣式方法的核心洞見：不建構 C_{obs} ，只需回答「這個位形是否碰撞」。
- PRM 先建圖後查詢，適合固定環境的多次查詢；RRT 由起點長樹，適合動態環境的單次查詢。
- RRT 會快速朝未探索區域生長；目標偏置（5~10%）能顯著加速收斂。
- RRT* 透過選擇最佳父節點與重新佈線達成漸近最佳性，代價是收斂較慢。
- RRT 天然能處理非完整約束，因為它只需「沿可行運動前進一步」而非「連接任意兩點」。
- Dubins 路徑（僅前進）有 6 種基本組合；Reeds-Shepp（可倒車）有 48 種，是自動停車的基礎。
- Hybrid A^* 結合格點搜尋與連續運動基元，是自駕車規劃的主流方法。
- 人工勢場法計算極快且能處理動態障礙，但會卡在局部極小值，因此只適合作為局部規劃器。
- DWA 在速度空間中搜尋，明確考慮加速度與煞停限制，因此產生的指令必定可執行。
- 分層導航：全域規劃器看得遠但更新慢，局部規劃器反應快但視野小，兩者缺一不可。

- 代價地圖採分層設計（靜態、障礙、膨脹、自訂），復原行為的設計對系統可用性至關重要。

觀念題與計算題

觀念題

1. 定義位形空間，並說明它為何是路徑規劃的核心抽象。
2. 說明障礙物膨脹的原理與其兩個限制。
3. 為何平面 2R 手臂的位形空間是一個環面？請說明其拓撲意義。
4. 區分完備性、機率完備性與不完備，並各舉一個演算法為例。
5. 說明 A* 的評價函數，並解釋可採納性為何是最佳性的保證。
6. 為何啟發函數不能高估？低估會有什麼後果？
7. 說明加權 A* 的原理與其提供的保證。
8. 說明格點法在高維空間的維數詛咒，並計算六維時的格點數。
9. 敘述取樣式方法的核心洞見，並說明它為何使高維規劃成為可能。
10. 比較 PRM 與 RRT 的策略、適用場合與優缺點。
11. 說明 RRT 為何會「快速朝未探索區域生長」，以及目標偏置的作用。
12. 說明 RRT* 的兩個改進，以及漸近最佳性的意義。
13. 為何 RRT 比 PRM 更適合處理非完整約束？
14. 說明 Dubins 路徑的六種組合，以及 Reeds-Shepp 的推廣。
15. 說明人工勢場法的原理與局部極小值問題，並列舉三種緩解方法。
16. 說明動態視窗的三個限制，以及 DWA 相對於勢場法的關鍵優勢。
17. 說明分層導航架構的分工，以及代價地圖的分層設計。
18. 為何說「復原行為的設計往往比規劃演算法更影響系統可用性」？

計算題

1. 在 8 連通格點上（斜走代價 $\sqrt{2}$ ），機器人由 (0,0) 走向 (8,5)。（a）計算對角線距離啟發函數的值。（b）計算曼哈頓距離的值。（c）判斷兩者是否可採納。（d）說明應使用哪一個並解釋理由。

2. 某長方形移動機器人長 1.0 m、寬 0.6 m。(a) 求外接圓與內切圓半徑。(b) 若走廊寬 1.5 m，判斷用外接圓膨脹後是否可通行。(c) 若機器人沿走廊方向擺放，實際需要的最小走廊寬度為何？(d) 討論兩者的差異與實務作法。
3. 某差速機器人 $v_{\max} = 1.0 \text{ m/s}$ 、 $\omega_{\max} = 2.0 \text{ rad/s}$ 、 $a_v = 0.8 \text{ m/s}^2$ 、 $a_\omega = 3.0 \text{ rad/s}^2$ 、 $\Delta t = 0.1 \text{ s}$ 。當前 $(v, \omega) = (0.6, -0.5)$ 。(a) 求加速度限制下的動態視窗。(b) 若前方 0.4 m 有障礙、煞車減速度 0.8 m/s^2 ，求安全速度上限。(c) 求最終的動態視窗。(d) 若在此視窗中均勻取樣 5×5 組，列出 v 與 ω 的取樣值。
4. 某六軸手臂的位形空間為六維，每個關節角度範圍 $\pm 180^\circ$ 。(a) 若每維切成 36 格 (每格 10°)，求總格點數。(b) 若每個格點的碰撞檢測需 0.1 ms ，求建表所需的總時間。(c) 若改用 RRT 且每次迭代需 0.5 ms 、平均需 3000 次迭代，求規劃時間。(d) 比較兩者並說明為何取樣式方法是唯一可行的選擇。

動手做與專題延伸

本章實作：路徑規劃模組

延續第 23 ~ 25 章的程式庫，完成導航系統的最後一塊。

- 階段一：實作格點地圖與障礙物膨脹，並實作 Dijkstra 與 A^* (支援多種啟發函數)。
- 階段二：實作 RRT 與目標偏置，並加入捷徑法路徑平滑。
- 階段三：實作 RRT^* ，比較其與 RRT 在路徑品質與收斂時間上的差異。
- 階段四：實作人工勢場法與 DWA，並在含 U 形障礙的地圖上比較兩者的局部極小值行為。
- 階段五：整合為分層導航系統——全域用 A^* 、局部用 DWA，並實作路徑追隨與重新規劃的觸發邏輯。

專題延伸

- 演算法的視覺化競賽：對同一組測試地圖 (含開闊區、狹窄通道、U 形陷阱、迷宮)，以動畫呈現五種演算法 (Dijkstra、 A^* 、加權 A^* 、RRT、 RRT^*) 的搜尋過程。記錄各自的展開節點數、路徑長度與執行時間，製作完整的比較表。這個專題能讓抽象的演算法差異變得直觀。
- 位形空間的視覺化：對平面 2R 手臂，在工作空間中放置數個圓形障礙物。掃描所有的 (θ_1, θ_2) 組合，判斷是否碰撞，繪出位形空間的黑白圖。你會看到工作空間中的簡單圓形障礙，在

位形空間中變成極為複雜的形狀——這是理解「為何不建構 C_{obs} 」最有力的證據。接著在這張圖上執行 A^* ，觀察路徑在兩個空間中的對應關係。

- **進階：非完整約束的規劃比較。**實作三種方法規劃汽車式機器人的路徑：（甲）忽略約束的 A^* （產生不可行路徑）、（乙）Dubins 路徑（只能前進）、（丙）Hybrid A^* （可倒車）。在一個包含狹窄停車位的場景中測試，比較三者的可行性與路徑長度。特別觀察 Hybrid A^* 如何自動產生「前進、倒車、再前進」的停車動作——這正是例題中路邊停車的自動化版本。

參考文獻與延伸閱讀

- LaValle, S. M., Planning Algorithms, Cambridge University Press（線上公開）（路徑規劃的權威教材，涵蓋本章所有主題）。
- Choset, H. et al., Principles of Robot Motion: Theory, Algorithms, and Implementations, MIT Press（規劃演算法的完整教材，含位形空間的詳細處理）。
- Hart, P. E., Nilsson, N. J. and Raphael, B., "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Trans. SSC, 1968（ A^* 演算法的原始論文）。
- Kavraki, L. E. et al., "Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces," IEEE Trans. Robotics and Automation, 1996（PRM 的奠基論文）。
- LaValle, S. M. and Kuffner, J. J., "Randomized Kinodynamic Planning," IJRR, 2001（RRT 的原始論文）。
- Karaman, S. and Frazzoli, E., "Sampling-Based Algorithms for Optimal Motion Planning," IJRR, 2011（RRT* 與漸近最佳性的證明）。
- Khatib, O., "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," IJRR, 1986（人工勢場法的原始論文）。
- Fox, D., Burgard, W. and Thrun, S., "The Dynamic Window Approach to Collision Avoidance," IEEE Robotics & Automation Magazine, 1997（DWA 的原始論文）。
- Dolgov, D. et al., "Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments," IJRR, 2010（Hybrid A^* 在自駕車上的應用）。

- ROS 2 Nav2 官方文件與 OMPL (Open Motion Planning Library) ：導航堆疊與規劃演算法函式庫的實務參考。

第 27 章

無人機與飛行機器人

編著：李世淵（ 機器人叫獸 ）

助教：李天宇、李宇晴

【章首情境圖建議】左半為四旋翼往前飛行的側視圖——機身明顯前傾，推力向量垂直於機身並分解為「向上抵抗重力」與「向前推進」兩個分量，旁註「想往前，必須先低頭」。機身上以正反槳的箭頭標出兩對相反的旋轉方向。右半為串級控制的四層巢狀方框圖，由外而內標註頻率：軌跡規劃 10 Hz、位置控制 50 Hz、姿態控制 500 Hz、角速度控制 1000 Hz，各層以不同深淺的綠色區隔，並在位置與姿態之間標出轉換公式 $\theta_d = a_x/g$ 。下緣文字：欠驅動且本質不穩定——一刻都不能停止控制。

叫獸開講 「六個自由度只有四顆馬達，它憑什麼飛得這麼穩？」

叫獸把一台四旋翼無人機放在桌上，接上電池，螺旋槳開始緩緩轉動。「小翰，我要它往右平移一公尺，但機身不能傾斜。做得到嗎？」

小翰想了幾秒：「右邊兩個螺旋槳.....不對，那是轉向。」他試著在腦中組合四顆馬達的轉速，發現無論怎麼配，只要想產生向右的力，機身就必定會傾斜。

「這是第 23 章那台差速車的空中版本。」叫獸說，「它有六個自由度——三個位置、三個姿態——但只有**四顆馬達**。想往右移動，你**必須先向右傾斜**，讓推力有一個水平分量。位置與姿態是綁在一起的，不能獨立控制。」

小好舉手：「所以跟差速車一樣，是非完整約束？」

「不一樣，而且差別很致命。」叫獸把無人機拿起來，「差速車不能側移，但你**放手它就停在那裡**。無人機呢？」他做出鬆手的姿勢，「馬達一停，它就從三十公尺高摔下來。」

他在白板上寫下兩個詞：**欠驅動、本質不穩定**。「差速車是欠驅動但穩定；無人機是欠驅動而且不穩定。它一刻都不能停止控制——這是第 18 章那個倒立擺的三維版本，而且是在空中。」

接著他問了一個更尖銳的問題：「那為什麼四旋翼還能飛得這麼穩？我看過的競速無人機可以做出翻滾動作再穩住。」

小翰試著回答：「因為控制器很快？」

「這是一半的答案。真正的關鍵在於**分層**。」叫獸在白板上畫了兩個方框，「姿態控制跑在**500 到 1000 Hz**——它只管『機身該傾斜幾度』；位置控制只跑**50 Hz**——它只管『該往哪個方向去』，然後把答案翻譯成『請傾斜這麼多度』交給下層。」

「為什麼要分開？」叫獸自問自答，「因為**姿態的動態比位置快十倍以上**。姿態是由力矩直接驅動的、慣量小、反應極快；位置則要透過『傾斜 → 產生水平分力 → 加速 → 積分成速度 → 再積分成位置』，中間隔了好幾層。**把快的和慢的分開處理，各自用合適的頻率**——這就是無人機能穩定飛行的祕密。」

最後他把無人機放回桌上：「今天你們會發現，第 18 章的 LQR、第 19 章的 Lyapunov、第 24 章的互補濾波，全部會在這一章派上用場。**這是全書控制理論的第一次綜合考驗。**」

本章為什麼重要

無人機是過去二十年機器人領域最成功的商業化案例之一。從航拍、農噴、巡檢、測繪、物流到災難搜救，它已經是一個成熟的產業。而在技術上，它更是一個**極佳的教學載體**——因為它把前面各章的理論逼到了必須全部用上的程度。

它的三個特性使它與前面所有的機器人都不同：

- **欠驅動且本質不穩定**：六個自由度只有四個輸入，且失去控制就會墜落。第 23 章的差速車雖然也欠驅動，但停下來仍是安全的。
- **沒有支撐點**：機械手臂固定在基座上、輪型機器人有地面支撐，而無人機**完全靠自己的推力懸浮**。任何一個環節失效都是災難。
- **動態極快**：姿態的時間常數只有數十毫秒，這要求 500 Hz 以上的控制頻率——比機械手臂的伺服迴路還快。

因此本章不只是「介紹一種機器人」，而是把第三篇的控制理論與第四篇的感知定位技術真正整合起來的第一個完整案例。第 18 章的 LQR、第 19 章的穩定性、第 24 章的狀態估測、第 26 章的路徑規劃，都會在這裡以具體的形式重現。

學習目標

- 能說明四旋翼的推力與力矩產生原理，並解釋為何需要正反槳。
- 能寫出四旋翼由馬達轉速到推力與力矩的分配矩陣。
- 能推導四旋翼的簡化動力學模型並辨識其非線性項。
- 能說明四旋翼的欠驅動特性及其對控制的影響。
- 能說明微分平坦性的意義及其在軌跡規劃上的價值。
- 能說明串級控制架構的分層原理與各層的頻率選擇。
- 能設計姿態控制器並說明角速度內迴路的必要性。
- 能說明位置控制如何轉換為姿態指令。
- 能比較四旋翼與固定翼、直升機、VTOL 的特性與適用場合。
- 能說明無人機的狀態估測架構與各感測器的角色。
- 能分析無人機的安全機制與常見失效模式。

先備知識

第 13 章的動力學與慣性張量、第 16 ~ 19 章的控制理論（尤其是第 18 章的 LQR 與第 19 章的欠驅動概念）、第 24 章的狀態估測與互補濾波、第 8 章的姿態表示（四元數在此極為重要）、第 6.3 節的 IMU。

本章是全書整合度最高的章節之一。若對第 18、19、24 章有任何不確定，建議先回頭複習——本章會直接使用那些工具而不重新推導。

核心名詞中英對照

表 27-1 本章核心名詞

中文	英文	說明
四旋翼	Quadrotor / Quadcopter	以四個螺旋槳產生升力的飛行器
推力係數	Thrust Coefficient	推力與轉速平方的比例常數
扭矩係數	Drag/Torque Coefficient	反扭矩與轉速平方的比例常數
分配矩陣	Mixer / Allocation Matrix	由推力力矩指令求各馬達轉速的矩陣
正反槳	CW/CCW Propellers	順逆時針旋轉的螺旋槳配置
懸停	Hover	維持位置不變的飛行狀態
滾轉 / 俯仰 / 偏航	Roll / Pitch / Yaw	繞機體三軸的旋轉
欠驅動	Underactuated	控制輸入少於自由度
微分平坦	Differential Flatness	所有狀態與輸入可由少數輸出及其導數表示
串級控制	Cascade Control	多層巢狀迴路的控制架構
姿態控制	Attitude Control	控制機體朝向的內迴路
位置控制	Position Control	控制空間位置的外迴路
飛行控制器	Flight Controller, FC	執行飛行控制的嵌入式系統
地面效應	Ground Effect	接近地面時升力增加的現象
視覺慣性里程計	VIO	融合視覺與 IMU 的位姿估計
失效保護	Failsafe	異常時的自動保護行為
垂直起降	VTOL	能垂直起降的固定翼混合構型

27.1 飛行的原理與構型

27.1.1 四旋翼為何成為主流

旋翼飛行器有多種構型，而四旋翼在過去二十年成為絕對的主流。原因不在於它的飛行效率（實際上比直升機差），而在於**機構的極簡**。

傳統直升機需要**變距機構**（swashplate）——透過改變槳葉的攻角來控制升力與方向。這個機構包含數十個精密的連桿與軸承，製造與維護成本極高。

四旋翼則完全不需要：**所有的控制都靠改變四顆馬達的轉速**。沒有任何活動的機械連桿，只有四個固定的螺旋槳。這使它能夠被大量、便宜地製造——這正是消費級無人機爆發的關鍵。

◎ 為什麼是四個而不是三個或兩個

兩個螺旋槳不夠：無法同時控制推力、滾轉、俯仰與偏航這四個量。**三個也不行**：三個轉速只能控制三個量，且反扭矩無法平衡（機身會持續自轉）。**四個剛好**：四個轉速對應推力、滾轉、俯仰、偏航四個控制量，且能透過正反槳配置抵消反扭矩。**六個或八個**（六旋翼、八旋翼）則提供了**冗餘**——損失一顆馬達仍能飛行，這對載人或高價值任務極為重要。這是一個典型的「剛好夠用」與「冗餘備份」的工程取舍，與第 4 章討論自由度時的思維一致。

27.1.2 正反槳與反扭矩

四旋翼有一個容易被忽略但極為關鍵的設計：**相鄰的螺旋槳旋轉方向相反**（對角線上的兩個同向）。

原因來自牛頓第三定律。螺旋槳推動空氣向下時，空氣也對機身施加一個**反扭矩**（drag torque），使機身朝反方向旋轉。若四個螺旋槳都同向轉，反扭矩會累加，機身會持續自轉。

採用正反槳配置後，四個反扭矩兩兩抵消。而**偏航控制正是刻意打破這個平衡**——讓一對槳加速、另一對減速，總推力不變但反扭矩不再平衡，機身就會繞垂直軸旋轉。

$$\text{偏航力矩 } \tau_{\psi} \propto (\omega_1^2 + \omega_3^2) - (\omega_2^2 + \omega_4^2)$$

偏航由「兩組對角槳的轉速差」產生。這是四旋翼最弱的一個控制通道——因為反扭矩遠小於推力。

實務意義：偏航的控制權限（authority）遠低於滾轉與俯仰，因此無人機**轉向較慢**，且在強風中偏航容易被吹偏。這是四旋翼的固有限制。

27.1.3 各種飛行構型的比較

表 27-2 飛行機器人構型的比較

構型	續航	載重	機動性	定點懸停	機構複雜度	典型應用
四旋翼	短 (15~40 分)	小~中	極佳	可	極低	航拍、巡檢、教學
六 / 八旋翼	短	中~大	佳	可	低	農噴、專業航拍 (有冗餘)
傳統直升機	中	大	佳	可	極高	載人、大載重
固定翼	長 (數小時)	中	差	不可	低	測繪、長程巡邏
VTOL 混合翼	長	中	中	可	高	長程且需定點起降
撲翼 / 仿生	短	極小	特殊	部分可	高	研究、微型偵察

選型的核心取舍是「續航」與「懸停能力」。固定翼靠機翼產生升力，效率極高（升阻比可達 10 以上），但必須持續前進；旋翼機把全部能量用於對抗重力，效率低但能停在空中。VTOL 試圖兼得，代價是機構複雜與重量增加。

27.2 四旋翼的動力學

27.2.1 由轉速到推力與力矩

每個螺旋槳產生的推力與反扭矩，都近似正比於轉速的平方：

$$F_i = k_f \cdot \omega_i^2 \quad M_i = k_m \cdot \omega_i^2$$

k_f 為推力係數、 k_m 為扭矩係數。兩者的比值 k_m/k_f 是一個重要的機構參數，決定了偏航的控制權限。

四個螺旋槳的效果合成為四個控制量。設機臂長度為 L （機心到槳心），採用「十」字配置：

$$\begin{bmatrix} F \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} = \begin{bmatrix} k_f & k_f & k_f & k_f \\ 0 & -L \cdot k_f & 0 & L \cdot k_f \\ L \cdot k_f & 0 & -L \cdot k_f & 0 \\ k_m & -k_m & k_m & -k_m \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}$$

分配矩陣 (mixer)。由四個轉速平方，映射到總推力與三軸力矩。

控制器實際輸出的是 $(F, \tau_\phi, \tau_\theta, \tau_\psi)$ ，再透過這個矩陣的**反矩陣**求出各馬達應有的轉速——這一步稱為**控制分配** (control allocation)。

◎ 飽和與分配的優先順序

實務上有一個重要的細節：當控制器要求的推力與力矩超出馬達能力時，該犧牲哪一個？標準的答

案是**優先保證姿態力矩**。原因是：推力不足只會使無人機緩慢下降（還有時間補救），但姿態失控會立即翻覆墜毀。因此飛控的分配演算法會先滿足 τ_ϕ 、 τ_θ 、 τ_ψ ，再把剩餘的餘裕給總推力 F 。這是「先活著，再談去哪裡」的工程原則，與第 20 章「安全優先於精度」的思維一致。

27.2.2 簡化的動力學模型

四旋翼的完整動力學相當複雜，但有一個標準的簡化模型，在多數情況下已足夠準確。

平移動力學（在世界座標系中）：

$$m \cdot a = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + R \cdot \begin{bmatrix} 0 \\ 0 \\ F \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ m \cdot g \end{bmatrix}$$

推力永遠沿機體的 Z 軸，經旋轉矩陣 R 轉換到世界座標系，再減去重力。

展開後可以清楚看見欠驅動的本質：

$$\begin{aligned} m \cdot \ddot{x} &= F \cdot (\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \\ m \cdot \ddot{y} &= F \cdot (\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \\ m \cdot \ddot{z} &= F \cdot \cos \phi \cos \theta - m \cdot g \end{aligned}$$

水平方向的加速度，完全由姿態角決定。想往前飛，就必須俯仰（低頭）。

轉動動力學（在機體座標系中）——這正是第 14.1.1 節的尤拉方程：

$$I \cdot \dot{\omega} + \omega \times (I \cdot \omega) = \tau$$

I 為慣性張量、 ω 為機體角速度、 τ 為控制力矩。第二項是陀螺項，在高速旋轉時不可忽略。

27.2.3 懸停條件與線性化

懸停是最重要的平衡點。此時所有速度與角速度為零、姿態水平：

$$F_{\text{hover}} = m \cdot g \quad \phi = \theta = 0$$

懸停條件。總推力恰好等於重量。

在懸停點附近做線性化（小角度近似： $\sin \phi \approx \phi$ 、 $\cos \phi \approx 1$ ），得到極為簡潔的結果：

$$\ddot{x} \approx g \cdot \theta \quad \ddot{y} \approx -g \cdot \phi \quad \ddot{z} \approx (F - mg)/m$$

線性化的平移動力學。俯仰角 θ 直接對應 X 方向的加速度、滾轉角 ϕ 對應 Y 方向（負號）。

這三條式子是整個無人機控制的基礎。它們揭示了一個關鍵的結構：

位置控制器輸出「期望加速度」→ 除以 g → 得到「期望姿態角」

這就是外迴路與內迴路的介面。位置控制不直接控制馬達，而是產生姿態指令交給下層。

27.3 欠驅動與微分平坦性

27.3.1 欠驅動的精確意義

四旋翼有 6 個自由度 ($x, y, z, \varphi, \theta, \psi$) 但只有 4 個控制輸入 ($F, \tau_\varphi, \tau_\theta, \tau_\psi$)。這是典型的欠驅動系統。

但哪四個量可以獨立控制？答案是：位置 (x, y, z) 加上偏航角 ψ 。

為什麼是這四個？因為滾轉與俯仰已經被位置的需求佔用了——要往某個方向加速，就必須傾斜到對應的角度。 φ 與 θ 不是自由的變數，而是位置控制的副產品。

可獨立控制： (x, y, z, ψ)	被決定： (φ, θ)
--------------------------	--------------------------

四旋翼的控制結構。這也解釋了為何開頭故事中「往右平移但機身不傾斜」是做不到的。

27.3.2 微分平坦性：一個極為優美的性質

四旋翼有一個令人驚訝的數學性質，稱為微分平坦 (differential flatness)。它由 Mellinger 與 Kumar 於 2011 年提出，徹底改變了無人機的軌跡規劃。

◎ 微分平坦性

若一個系統的所有狀態與所有控制輸入，都能由少數幾個「平坦輸出」(flat outputs) 及其有限階導數代數地表示出來，則稱該系統是微分平坦的。對四旋翼而言，平坦輸出恰好就是 (x, y, z, ψ)——也就是那四個可獨立控制的量。

這個性質的實用價值極大。它意味著：

給定一條 $(x(t), y(t), z(t), \psi(t))$ 的軌跡 $\Rightarrow$ 所需的姿態角、角速度、推力、力矩，全部可以直接算出來
--

不需要任何積分或求解微分方程。

具體的推導鏈是：

1. 由位置軌跡的二階導數 (加速度)，可算出所需的推力向量 (大小與方向)。
2. 推力方向即機體 Z 軸方向；結合期望的偏航角 ψ ，即可完全確定姿態 (旋轉矩陣 R)。
3. 由位置的三階導數 (急跳度 jerk)，可算出角速度。
4. 由位置的四階導數 (snap)，可算出角加速度，進而由尤拉方程得到所需的力矩。

27.3.3 最小 snap 軌跡

上述推導揭示了一個重要的結論：力矩正比於位置的四階導數（snap）。

由於力矩由馬達轉速差產生，而轉速差的變化受限於馬達的響應速度，因此最小化 snap 就等於最小化對馬達的要求。這導出了無人機軌跡規劃的標準作法：

$$\min \int ||d^4r/dt^4||^2 dt \quad (\text{最小 snap 軌跡})$$

在通過指定航點的約束下，最小化 snap 的平方積分。解為分段的七次多項式。

這與第 15 章的軌跡生成一脈相承——那裡最小化的是急跳度（三階導數，因為機械手臂的力矩正比於加速度），這裡因為多了一層（推力方向由姿態決定，而姿態又需要力矩來改變），所以要到四階。

最小 snap 軌跡是目前競速無人機與專業航拍的標準方法。它產生的軌跡極為平滑，能讓無人機以極高的速度通過複雜的路徑而不失控。

27.4 串級控制架構

27.4.1 為何必須分層

開頭故事中叫獸提到的「分層」，是無人機控制的核心架構。它的理論依據是時間尺度分離（time-scale separation）。

比較兩個迴路的動態：

表 27-3 姿態與位置動態的時間尺度

面向	姿態迴路	位置迴路
驅動方式	力矩直接驅動角加速度	推力 → 傾斜 → 水平分力 → 加速度
等效慣量	小（機體轉動慣量）	大（整機質量）
積分層數	2（力矩 → 角速度 → 角度）	4（含姿態迴路在內）
時間常數	數十毫秒	數百毫秒~數秒
控制頻率	500~1000 Hz	20~100 Hz
感測器	IMU（陀螺儀為主）	GNSS、氣壓計、視覺、光流

時間常數相差十倍以上，這正是分層的依據。當外迴路（位置）發出一個姿態指令時，內迴路（姿態）已經快到可以視為「瞬間達成」——這使得外迴路的設計可以完全忽略內迴路的動態，

大幅簡化問題。

◎ 時間尺度分離的普遍性

這個「快的內迴路 + 慢的外迴路」架構，在本書中已經出現多次：第 5 章伺服驅動器的電流迴路（10 kHz）套在速度迴路（1 kHz）內；第 17 章的計算力矩控制把非線性抵消放在內層、線性 PD 放在外層；第 3 章的時間尺度金字塔更是把整個機器人系統依頻率分層。共同的原則是：讓每一層只處理自己時間尺度上的問題，並把下層視為理想的執行器。這是複雜系統設計最重要的技巧之一。

27.4.2 四層架構

一個完整的無人機控制系統包含四層，由外而內：

1. **軌跡規劃層（1~10 Hz）**：產生位置與偏航的參考軌跡（第 26 章的路徑規劃 + 27.3.3 節的最小 snap）。
2. **位置控制層（20~100 Hz）**：比較實際位置與參考軌跡，輸出期望加速度，再依 27.2.3 節的關係轉換為期望姿態角與期望總推力。
3. **姿態控制層（250~500 Hz）**：比較實際姿態與期望姿態，輸出期望角速度。
4. **角速度控制層（500~1000 Hz）**：比較實際角速度與期望角速度，輸出力矩指令，最後經分配矩陣轉為四個馬達的轉速。

注意姿態控制本身又分成兩層（姿態角與角速度）。這是因為角速度可以由陀螺儀直接量測（高精度、低延遲），而姿態角必須經過融合估計（第 24 章）才能得到，精度與延遲都較差。把最內層建立在最可靠的感測上，是這個設計的關鍵。

圖 27-1 無人機的串級控制架構

建議配圖：由左至右的方塊圖，四個巢狀的控制迴路。最外層「軌跡規劃」（10 Hz）輸出位置參考；「位置控制」（50 Hz）輸出期望加速度，經過一個標註「 $a \rightarrow$ 姿態角」的轉換方塊（含公式 $\theta_d = a_x/g$ 、 $\varphi_d = -a_y/g$ ）；「姿態控制」（500 Hz）輸出期望角速度；「角速度控制」（1000 Hz）輸出力矩；最後經「分配矩陣」轉為四個馬達轉速。各層的回授來源以不同顏色標註：角速度來自陀螺儀（紅）、姿態來自 IMU 融合（橙）、位置來自 GNSS/VIO（藍）。右下角標註各層的更新頻率與時間常數對照。

27.4.3 角速度內迴路的關鍵性

最內層的角速度控制（俗稱 rate loop）是整個系統中最關鍵的一環，理由有三：

理由一：它決定了系統的極限性能。所有上層的指令最終都必須透過它執行。若 rate loop 不夠快或不夠準，上層再好的規劃也無法實現。

理由二：它是唯一能直接對抗擾動的層級。陣風打到機身時，最先反應的就是角速度的變化——rate loop 能在數毫秒內偵測並修正，遠快於姿態或位置層。

理由三：它的調參最為敏感。競速無人機玩家花最多時間調的就是 rate loop 的 PID 參數。增益太低則反應遲鈍、太高則產生高頻振盪（並發出特有的嗡嗡聲，稱為 motor noise）。

實務上 rate loop 幾乎都採用 PID，並在微分項前加上低通濾波（因為陀螺儀的雜訊在微分後會被嚴重放大——這正是第 16.7 節提到的問題）。

27.5 姿態控制

27.5.1 為何不能用歐拉角

姿態控制的第一個關鍵決定是：**用什麼表示姿態**。第 8 章已經給出答案——**內部計算必須用四元數或旋轉矩陣，絕不能用歐拉角**。

對無人機而言，這個選擇尤其關鍵，因為：

- **萬向節鎖是真實的威脅：**競速無人機做出翻滾動作時，俯仰角會經過 $\pm 90^\circ$ ——歐拉角在此處會發散（第 8.3.3 節）。
- **大角度誤差的計算會出錯：**若期望姿態與實際姿態相差 170° ，用歐拉角相減可能得到錯誤的旋轉方向（繞遠路）。
- **內插不平滑：**軌跡追隨時需要在姿態之間內插，歐拉角的線性內插會產生扭曲的路徑。

因此標準作法是：**內部用四元數儲存與計算、只在人機介面上轉換為歐拉角顯示**。這正是第 8.5.4 節的結論在實務上的體現。

27.5.2 姿態誤差的計算

設當前姿態為四元數 q 、期望姿態為 q_d 。姿態誤差不是相減，而是**相對旋轉**：

$$q_e = q^* \otimes q_d$$

q^* 為 q 的共軛（即反向旋轉）。 q_e 代表「由當前姿態轉到期望姿態所需的旋轉」。

由 q_e 取出誤差向量（用於回授控制）：

$$e = 2 \cdot \text{sign}(q_e, w) \cdot [q_e, x, q_e, y, q_e, z]^T$$

取向量部乘以 2（因為四元數用半角，第 8.5.1 節）。**sign 項處理雙重覆蓋——確保永遠選擇較短的旋轉路徑。**

那個 sign 函數極為重要。若省略它，當誤差超過 180° 時，控制器會選擇繞遠路的方向——無人機會朝錯誤的方向翻轉。這是實務上一個經典的除錯陷阱。

27.5.3 串級的姿態控制器

姿態控制採用兩層串級（第 27.4.2 節）：

外層（姿態角，250~500 Hz）：通常只用比例控制（P），輸出期望角速度。

$$\omega_d = K_{p,att} \cdot e_{att}$$

姿態外層。不需要微分項，因為下一層本身就是速度控制。

內層（角速度，500~1000 Hz）：採用完整的 PID。

$$\tau = K_{p,rate} \cdot (\omega_d - \omega) + K_{i,rate} \cdot \int (\omega_d - \omega) dt + K_{d,rate} \cdot d(\omega_d - \omega)/dt$$

角速度內層。這是整個無人機最關鍵的控制器（第 27.4.3 節）。

三個項的作用在無人機上有具體的意義：

表 27-4 角速度 PID 各項在無人機上的作用

項	作用	調太低的症狀	調太高的症狀
P（比例）	主要的響應力道	反應遲鈍、追不上指令	高頻振盪、馬達嗡嗡聲
I（積分）	消除穩態誤差（如重心偏移、風）	持續偏航、無法保持角度	積分飽和、大動作後過衝
D（微分）	阻尼、抑制超越	超越後來回擺盪	放大陀螺儀雜訊、機身抖動

微分項的雜訊問題在無人機上特別嚴重，因為馬達與螺旋槳產生大量的機械振動，直接傳到陀螺儀上。因此實務上必定會在 D 項前加低通濾波器，且截止頻率的調整是飛控調參的重點之一。

27.5.4 為何加入前饋

純回授的姿態控制有一個固有的延遲——它必須等誤差出現才能反應。對於快速的軌跡追隨，這個延遲會造成明顯的落後。

因應之道與第 17 章相同：加入前饋。由 27.3.2 節的微分平坦性，期望的角速度與角加速度都可以由軌跡直接算出：

$$\tau = \tau_{ff} + \tau_{fb} \quad , \text{ 其中 } \tau_{ff} = I \cdot \dot{\omega}_d + \omega_d \times (I \cdot \omega_d)$$

前饋項來自尤拉方程 (第 14.1.1 節) , 直接算出「執行這個期望運動所需的力矩」。

這正是第 17 章「用模型處理已知的部分、用回授處理未知的部分」哲學的又一次體現。在競速無人機與高速航拍中，前饋能顯著改善軌跡追隨的精度。

27.6 位置控制

27.6.1 由位置誤差到姿態指令

位置控制的核心，是把「該往哪裡去」翻譯成「該傾斜多少」。其流程分三步。

步驟一：計算期望加速度。用 PID (或 LQR) 對位置誤差產生加速度指令：

$$a_d = K_p(r_d - r) + K_d(\dot{r}_d - \dot{r}) + K_i \int (r_d - r) dt + \ddot{r}_d$$

最後一項 $\ddot{r}_d$ 是軌跡的前饋加速度——這使控制器能提前反應而非事後追趕。

步驟二：計算所需的推力向量。加速度加上重力補償：

$$F_{vec} = m \cdot (a_d + g \cdot \hat{z})$$

所需的推力向量 (在世界座標系中) 。注意必須加上抵抗重力的分量。

步驟三：由推力向量反推姿態與推力大小。

$$F = \|F_{vec}\| \quad (\text{總推力大小})$$

$$\hat{z}_{body} = F_{vec} / \|F_{vec}\| \quad (\text{機體 } z \text{ 軸應指的方向})$$

這是四旋翼控制最核心的一步：推力永遠沿機體 z 軸，因此「推力向量的方向」就直接決定了「機體該朝哪個方向」。

有了 $\hat{z}_{body}$ 與期望的偏航角 ψ_d ，即可完全確定期望的姿態矩陣 R_d (第 8 章的方法：先由 ψ_d 定出參考的 X 方向，再用外積建構正交基底)。

27.6.2 小角度的簡化形式

在小角度 (一般飛行，傾角小於 20°) 的情況下，上述流程可以簡化為第 27.2.3 節的線性關係：

$$\theta_d = a_x / g \quad (\text{俯仰角，控制前後})$$

$$\phi_d = -a_y / g \quad (\text{滾轉角，控制左右})$$

$$F = m \cdot (g + a_z) \quad (\text{總推力，控制上下})$$

小角度近似。實作極為簡單，是入門級飛控的標準作法。

這個近似的誤差在傾角 20° 時約為 2%、 30° 時約為 5%。對一般飛行完全足夠，但在高機動飛行（傾角超過 45° ）時必須使用 27.6.1 節的完整向量形式。

27.6.3 高度控制的特殊性

高度（Z 軸）的控制與水平方向有兩個重要的差異：

差異一：它是直接驅動的。不需要透過姿態——增加總推力就能上升。因此高度控制的頻寬可以比水平方向高得多。

差異二：重力補償是必須的。懸停時推力必須恰好等於 mg （第 27.2.3 節）。因此控制律中必定包含一個常數的前饋項：

$$F = m \cdot g + m \cdot a_{z,desired} + \text{PID 修正}$$

重力前饋。若沒有這一項，控制器必須靠積分項慢慢累積出 mg 這麼大的推力——起飛時會嚴重下沉，這是第 17.5 節重力補償概念的直接應用。

電池電壓下降的影響：隨著電池放電，同樣的 PWM 指令產生的推力會下降。若不補償，無人機會在飛行後期逐漸下沉。專業飛控會量測電池電壓並做即時補償——這是一個典型的「已知擾動用前饋處理」的例子。

27.6.4 外部擾動

無人機面對三種主要的外部擾動：

- **風：**最主要的擾動。持續的風會被積分項補償（表現為機身持續傾斜對抗風力），但陣風則會造成瞬間的位置偏移。
- **地面效應：**接近地面（約一個槳徑的高度內）時，螺旋槳下洗氣流被地面反射，使升力增加 5~20%。這使無人機在降落時「浮起來」而難以觸地，也使起飛時的推力估計不準。
- **渦環狀態（Vortex Ring State）：**快速垂直下降時，螺旋槳會落入自己產生的下洗氣流中，導致升力急遽下降甚至失控。因應之道是降落時保持一定的水平速度，避免純垂直的快速下降。這是無人機失事的常見原因之一。

27.7 狀態估測

27.7.1 需要估測什麼

無人機的控制需要六個量：**位置 (x, y, z)**、**速度**、**姿態**、**角速度**。其中只有角速度可以直接量測（陀螺儀），其餘都必須估測。

這正是第 24 章的完整應用場合。各感測器的角色分工如下：

表 27-5 無人機的感測器配置

感測器	提供的資訊	更新率	特性	失效情境
陀螺儀	角速度	1~8 kHz	極準但會漂移	長時間積分
加速度計	重力方向（→姿態）	1~8 kHz	長期準、雜訊大	機動飛行時受干擾
磁力計	偏航角（絕對）	50~100 Hz	不漂移	受鐵件與馬達電流干擾
氣壓計	相對高度	20~50 Hz	解析度約 10 cm	受氣流與溫度影響
GNSS	絕對位置	1~10 Hz	公尺級精度	室內、樹下、峽谷失效
光流相機	水平速度	50~100 Hz	需要紋理與光線	白牆、暗處、高空
測距（雷射/超音波）	對地高度	20~50 Hz	精度高	有效距離短（數公尺）
視覺 VIO	位姿（六自由度）	20~50 Hz	精度高、不需 GNSS	快速運動、低紋理

27.7.2 姿態估測：互補濾波的經典應用

姿態估測是第 24.8 節互補濾波的最經典應用場景。三個感測器的互補關係極為漂亮：

- **陀螺儀**：積分角速度得到姿態。短期極準，但**會漂移**（每分鐘可達數度）。
- **加速度計**：靜止時量到的就是重力方向，可推算滾轉與俯仰。**不漂移**，但飛行中的加速度會污染讀值。
- **磁力計**：量測地磁方向，提供**偏航角**的絕對參考。加速度計無法提供偏航（因為繞垂直軸旋轉不改變重力方向）。

滾轉、俯仰：陀螺儀（高通）+ 加速度計（低通）

偏航：陀螺儀（高通）+ 磁力計（低通）

互補濾波的分工。這正是第 24.8.2 節「頻率響應互補」的具體實現。

磁力計的校正極為關鍵。機身上的鐵件、馬達的電流、甚至電池的位置都會產生磁場干擾。因此每台無人機在首次使用前都必須做**羅盤校正**——那個「拿著無人機轉三圈」的儀式，其實是在收集不同姿態下的磁場讀值，以估算並補償硬鐵與軟鐵的干擾。

27.7.3 位置估測的三種情境

位置估測的方案完全取決於環境：

情境一：戶外開闊（GNSS 可用）。標準方案是 GNSS + IMU 的 EKF 融合。GNSS 提供不漂移的絕對位置（1~10 Hz）、IMU 提供高頻的平滑預測（1 kHz）——這正是第 24.10.1 節的融合邏輯。典型精度可達 1~2 公尺（一般 GNSS）或數公分（RTK）。

情境二：室內或 GNSS 拒止。必須改用視覺方案：

- **光流 + 測距：**最輕量的方案。光流相機量測水平速度（透過影像的移動）、測距感測器量測高度。只能估測速度而非絕對位置，因此仍會緩慢漂移，但足以維持懸停穩定。這是消費級無人機室內定高懸停的標準配置。
- **VIO（視覺慣性里程計）：**完整的六自由度位姿估計（第 25.9.2 節）。精度高但計算量大，且仍會漂移（沒有迴路閉合的話）。
- **VSLAM：**加入迴路閉合，可消除長期漂移（第 25 章）。計算量最大。

情境三：外部定位系統。在實驗室或表演場地，可用動作捕捉系統（如 OptiTrack）或 UWB 提供公釐級的絕對位置。這使研究者能專注在控制演算法而不受估測誤差干擾——多數無人機特技表演的影片，都是在這種環境下拍攝的。

◎ 為何室內飛行比室外困難

直覺上室內沒有風、空間又小，應該比較容易。但實際上**室內飛行的技術門檻高得多**，關鍵就在於定位。室外有 GNSS 這個「免費的絕對參照」，而室內完全沒有——必須靠視覺自己建立參照，而視覺會受光線、紋理、運動模糊影響。這也解釋了一個常見的現象：同一台無人機在室外能穩穩懸停，進到室內卻會緩慢地飄移——因為它失去了絕對位置的參照，只能靠光流估測速度，而速度的積分必然漂移（第 23.8 節的宿命）。

27.8 安全與失效保護

27.8.1 為何安全設計特別關鍵

無人機的安全問題與地面機器人有本質的差異：它沒有「安全停止」這個選項。第 7.9 節的緊急停止在無人機上是無效的——切斷動力等於墜落。

因此無人機的安全設計必須建立在不同的原則上：不是「停止」，而是「降級」——在部分能力失效時，仍能維持最低限度的可控性，並安全著陸。

27.8.2 常見的失效模式

表 27-6 無人機的失效模式與因應

失效	症狀	後果	因應機制
電池電量不足	電壓下降、推力衰減	墜落	低電量自動返航、強制降落
GNSS 失鎖	位置估計跳動或消失	位置失控、飄移	切換至姿態模式、維持高度
磁力計干擾	偏航角錯誤	朝錯誤方向飛 (toilet bowl 現象)	偵測不一致、改用 GPS 航向
馬達或槳失效	推力不平衡	翻覆墜落	六旋翼可容錯；四旋翼可犧牲偏航保平衡
通訊中斷	失去遙控訊號	失控飛離	自動返航 (RTH)
IMU 故障	姿態估計錯誤	立即失控	多 IMU 冗餘與表決
渦環狀態	垂直下降時失去升力	急速墜落	降落時保持水平速度

27.8.3 分級降級的設計

專業飛控採用分級的飛行模式，能力由高到低依序降級：

1. **位置模式 (Position Hold)**：完整的位置控制，鬆手即懸停。需要 GNSS 或視覺定位。
2. **高度模式 (Altitude Hold)**：只維持高度，水平方向由操作員控制。GNSS 失效時降級到此。
3. **姿態模式 (Attitude / Stabilize)**：只穩定姿態，高度與位置全由操作員控制。氣壓計失效時降級到此。
4. **手動模式 (Acro / Rate)**：只控制角速度，連姿態都不穩定。這是競速飛行的模式，也是最後的保底——只要陀螺儀還在運作就能飛。

這個降級序列的設計邏輯是「依感測器的可靠度排序」：陀螺儀最可靠（純內部感測）、其次是氣壓計、再來是 GNSS 或視覺（依賴外部環境）。每失去一個感測來源，就降到對應的模式，而非直接失控。

這與第 7 章「失效導向安全」的原則一脈相承，但形式不同——地面機器人的安全狀態是「停止」，無人機的安全狀態是「維持最低限度的可控性」。

27.8.4 法規與責任

無人機是少數受到嚴格法規管制的機器人。台灣的《遙控無人機管理規則》（2020 年起施行）規定了：

- **註冊與檢驗**：250 公克以上須註冊，特定重量以上須檢驗。
- **操作證照**：超過 2 公斤或從事特定活動須考取操作證。
- **飛行限制**：最大高度 400 英尺（約 122 公尺）、不得超出目視範圍、禁飛區（機場、政府機關等）。
- **保險**：商業用途須投保責任險。

工程上的意義：這些法規直接影響設計。例如「不得超出目視範圍」使長程自主飛行在多數情況下不合法；「400 英尺高度限制」則限制了通訊與測繪的規劃。在開發任何無人機應用之前，必須先確認法規的可行性——這與第 35 章討論的機器人倫理與社會接受度密切相關。

27.9 自主飛行與應用

27.9.1 由遙控到自主的三個層次

無人機的自主程度可分為三個層次，對應第 1 章討論的自主性光譜：

層次一：穩定輔助。操作員下達速度或姿態指令，飛控負責穩定。這是消費級無人機的基本模式——人決定去哪裡，機器負責不摔。

層次二：航點自主。事先規劃一連串 GPS 航點，無人機自動飛完全程。這是測繪、農噴、巡檢的標準模式。路徑是人規劃的，執行是自主的。

層次三：任務自主。給定目標（如「檢查這棟建築的所有窗戶」），無人機自行規劃路徑、避障、並決定何時完成。這需要完整的感知—規劃—控制鏈，是目前的研究前沿。

27.9.2 典型應用與其技術需求

表 27-7 無人機應用與技術需求

應用	核心需求	關鍵技術	難點
航拍攝影	平穩、精確的軌跡	最小 snap 軌跡、雲台穩定	風中的穩定性
測繪	精確定位、規律覆蓋	RTK-GNSS、航線規劃	影像重疊率與後處理
農業噴灑	低空穩定、大載重	地形跟隨、流量控制	藥液晃動、下洗氣流
基礎設施巡檢	接近結構物、細節拍攝	視覺定位、避障	GNSS 遮蔽、金屬干擾磁力計
物流配送	長程、精確降落	VTOL、視覺標記降落	法規、天候、安全
搜救	室內外皆可、自主探索	SLAM、自主探索	無 GNSS、環境未知
競速	極限機動	高頻 rate loop、低延遲影傳	飛行員技術、機體強度

27.9.3 無人機叢集

多架無人機的協同 (swarm) 是近年的熱點，其技術挑戰有三：

挑戰一：相對定位。叢集中的每一架都必須知道其他架的位置。GNSS 的公尺級精度不足以避免碰撞，因此需要 RTK (公分級) 或機間的相對測距 (UWB)。

挑戰二：通訊與同步。數十架無人機的狀態需要即時共享，且動作必須同步。這是一個典型的分散式系統問題。

挑戰三：分散式決策。集中式控制 (一台電腦指揮全部) 簡單但脆弱 (單點失效)；分散式 (各自決策) 強健但難以保證整體行為。實務上多採混合式——集中規劃、分散執行。

表演型叢集 (如燈光秀) 與 **任務型叢集** (如搜救) 的差異很大：前者的軌跡完全預先規劃、環境已知、且可用外部定位；後者則需要即時的感知與決策。目前商業化成熟的主要是前者。

27.10 由飛行邁向行走

27.10.1 本章的成果

本章完成了第一個「全書技術綜合」的案例。你現在能夠：分析四旋翼的推力與力矩產生機制、推導其動力學並辨識欠驅動的本質、理解微分平坦性及其在軌跡規劃上的價值、設計串級的姿態與位置控制器、選擇合適的狀態估測方案、並設計分級降級的安全機制。

更重要的是，你看到了前面各章的理論如何在一個真實系統中**同時**發揮作用：

表 27-8 本章對前面各章的整合

技術	來源章節	在無人機上的角色
尤拉方程與慣性張量	第 13、14 章	轉動動力學、前饋力矩計算
軌跡生成與急跳度	第 15 章	最小 snap 軌跡（推廣到四階）
PID 與微分項濾波	第 16 章	角速度內迴路
前饋加回授	第 17 章	軌跡追隨的精度提升
LQR	第 18 章	姿態與位置控制器的最佳增益
欠驅動與穩定性	第 19 章	本質不穩定系統的分析
四元數與萬向節鎖	第 8 章	姿態表示與誤差計算
互補濾波與 EKF	第 24 章	姿態與位置的狀態估測
VIO 與 SLAM	第 25 章	GNSS 拒止環境下的定位
路徑規劃	第 26 章	航點規劃與避障

27.10.2 接下來：更困難的欠驅動

無人機雖然欠驅動且不穩定，但它有一個關鍵的簡化：**它的動態是連續的**。螺旋槳持續轉動、推力連續變化、沒有任何突變。

第 28 章的腿式機器人則面對一個更困難的問題：**接觸狀態不斷切換**。腳著地時，機器人是**一個受約束的多體系統**；腳離地時，它是一個自由飛行的物體。每一步都是一次**混合動態**（hybrid dynamics）的切換。

此外，腿式機器人還有三個無人機沒有的困難：

- **自由度極高**：四足機器人 12~18 個自由度、人形機器人可達 30 個以上，遠超四旋翼的 4 個輸入。
- **接觸力的不確定性**：地面的摩擦係數、軟硬程度都未知且會變化（第 14.7 節與第 21.2.3 節的困難再次出現）。
- **騰空階段完全無法控制質心**：一旦雙腳離地，質心的軌跡就由拋物線決定，只能靠落地點的選擇來修正——這是一種特殊的欠驅動。

但兩者也有深刻的共同點：**都是欠驅動、都本質不穩定、都必須持續控制、也都採用分層架構**。理解了無人機，第 28 章會容易許多。

換句話說：第 27 章讓機器人離開地面，第 28 章讓它用腿走回地面——而後者，出乎意料地更難。

完整計算例題

例題 27-1 四旋翼的推力與分配

題目：某四旋翼質量 $m = 1.2 \text{ kg}$ ，機臂長 $L = 0.25 \text{ m}$ ，推力係數 $k_f = 1.5 \times 10^{-5} \text{ N} \cdot \text{s}^2$ 、扭矩係數 $k_m = 2.4 \times 10^{-7} \text{ N} \cdot \text{m} \cdot \text{s}^2$ 。(a) 求懸停時所需的總推力與每顆馬達的轉速。(b) 若要產生 $0.15 \text{ N} \cdot \text{m}$ 的滾轉力矩，求四顆馬達的轉速調整量。(c) 求偏航與滾轉的控制權限比。(d) 若馬達最高轉速為 1200 rad/s ，求最大可用的推力與推重比。

解 (a)：懸停時總推力等於重量：

$$F_{\text{hover}} = m \cdot g = 1.2 \times 9.8 = 11.76 \text{ N}$$

總推力。

四顆馬達平均分擔，每顆 $F_1 = 11.76/4 = 2.94 \text{ N}$ 。由 $F = k_f \cdot \omega^2$ ：

$$\omega = \sqrt{(F/k_f)} = \sqrt{(2.94/1.5e-5)} = \sqrt{196000} = 442.7 \text{ rad/s}$$

每顆馬達的懸停轉速 (約 4228 RPM)。

解 (b)：採「十」字配置，滾轉力矩由左右兩顆馬達的推力差產生 (第 27.2.1 節的分配矩陣)：

$$\tau_{\phi} = L \cdot (F_4 - F_2) = 0.15 \text{ N} \cdot \text{m} \Rightarrow F_4 - F_2 = 0.15/0.25 = 0.6 \text{ N}$$

左右馬達的推力差。

為維持總推力不變，一顆增加 0.3 N 、另一顆減少 0.3 N ：

$$\begin{aligned} F_4 &= 2.94 + 0.3 = 3.24 \text{ N} \Rightarrow \omega_4 = \sqrt{(3.24/1.5e-5)} = 464.8 \text{ rad/s} \\ F_2 &= 2.94 - 0.3 = 2.64 \text{ N} \Rightarrow \omega_2 = \sqrt{(2.64/1.5e-5)} = 419.5 \text{ rad/s} \end{aligned}$$

調整後的轉速。相差約 45 rad/s (約 10%)。

解 (c)：比較兩種力矩在相同轉速差下的大小。

滾轉力矩：由推力差乘力臂產生： $\tau_{\phi} = L \cdot \Delta F = L \cdot k_f \cdot \Delta(\omega^2)$ 。

偏航力矩：由反扭矩差產生： $\tau_{\psi} = k_m \cdot \Delta(\omega^2)$ 。

$$\tau_{\varphi}/\tau_{\psi} = (L \cdot k_f)/k_m = (0.25 \times 1.5e-5)/2.4e-7 = 3.75e-6/2.4e-7 = 15.6$$

滾轉的控制權限是偏航的 15.6 倍。

工程意義：這解釋了第 27.1.2 節提到的現象——四旋翼的偏航反應明顯較慢、且在強風中容易被吹偏。若要改善，可以加長機臂（增大 L ，但會增加慣量）或選用扭矩係數較大的螺旋槳（但通常效率較差）。

解（d）：最大推力：

$$F_{\max} = 4 \times k_f \times \omega_{\max}^2 = 4 \times 1.5e-5 \times 1200^2 = 4 \times 1.5e-5 \times 1.44e6 = 86.4 \text{ N}$$

四顆馬達全速的總推力。

$$\text{推重比} = F_{\max}/(m \cdot g) = 86.4/11.76 = 7.35$$

推重比約 7.3。

判讀：推重比是無人機最重要的性能指標之一。

- 小於 1.5：無法起飛或極為勉強。
- 2 ~ 3：一般航拍機，穩定但機動性有限。
- 4 ~ 6：運動型，能做較大的機動動作。
- 7 以上：競速機，能做極限特技。

本例的 7.35 屬於競速等級。但要注意：高推重比意味著懸停時馬達只用了 13.6% 的推力，這使油門的解析度變差、懸停時較難精確控制——這是競速機不適合航拍的原因之一。

例題 27-2 串級控制的頻寬設計

題目：某四旋翼的轉動慣量 $I_{xx} = 0.012 \text{ kg} \cdot \text{m}^2$ 。（a）若要求姿態迴路的自然頻率為 15 rad/s、阻尼比 1.0，求 PD 增益。（b）依時間尺度分離原則，建議位置迴路的頻寬。（c）依第 19.9.3 節的準則，求各層所需的最低控制頻率。（d）討論若姿態頻寬只有 5 rad/s 會有什麼後果。

解（a）：姿態動態近似為 $I \cdot \ddot{\varphi} = \tau$ （忽略陀螺項）。這是一個雙積分器，套用第 16 章的公式：

$$\begin{aligned} K_p &= I \cdot \omega_n^2 = 0.012 \times 15^2 = 0.012 \times 225 = 2.70 \text{ N} \cdot \text{m/rad} \\ K_d &= I \cdot 2\zeta\omega_n = 0.012 \times 2 \times 1.0 \times 15 = 0.36 \text{ N} \cdot \text{m} \cdot \text{s/rad} \end{aligned}$$

姿態迴路的 PD 增益。

解 (b)：時間尺度分離要求外迴路比內迴路慢至少 3~5 倍 (實務上常取 5~10 倍)：

$$\omega_{n,pos} \leq 15/5 = 3 \text{ rad/s}$$

位置迴路的頻寬建議不超過 3 rad/s (約 0.48 Hz)。

這個數字看似很低，但符合實際——無人機的位置響應確實較慢，從發出「往前 1 公尺」的指令到真正到位，通常需要 1~2 秒。

解 (c)：依第 19.9.3 節，取樣頻率應為閉環頻寬的 20~30 倍：

$$\text{姿態迴路：} f \geq 20 \times 15/(2\pi) = 20 \times 2.39 = 47.8 \text{ Hz} \rightarrow \text{建議 } 250 \text{ Hz 以上}$$

$$\text{位置迴路：} f \geq 20 \times 3/(2\pi) = 20 \times 0.48 = 9.6 \text{ Hz} \rightarrow \text{建議 } 50 \text{ Hz}$$

各層的最低控制頻率。

注意：實務上姿態迴路遠高於理論最低值 (500~1000 Hz)，原因有二：一、角速度內迴路的頻寬更高 (可達 50 rad/s 以上)；二、需要足夠的取樣率才能有效濾除馬達振動的雜訊。

解 (d)：若姿態頻寬只有 5 rad/s：

後果一：位置迴路必須降到 1 rad/s 以下——無人機的反應會極為遲鈍，從指令到位需要 5 秒以上。

後果二：抗擾動能力大幅下降。陣風造成的姿態偏差需要更長的時間才能修正，期間位置已經偏離很遠。

後果三：無法執行快速軌跡。最小 snap 軌跡若要求快速的姿態變化，控制器根本跟不上。

改善方向：提高姿態增益 (但受限於雜訊與結構共振)、降低轉動慣量 (縮短機臂、減輕外圍重量)、或使用響應更快的馬達與電調。**降低慣量往往是最有效的**——這也是競速無人機把電池與電子件都集中在中心的原因。

例題 27-3 位置控制與姿態指令的轉換

題目：某四旋翼質量 1.5 kg，當前位置 (2.0, 1.0, 3.0) m、速度 (0.5, 0, 0) m/s。目標位置 (5.0, 3.0, 3.0) m、目標速度為零。位置控制增益 $K_p = 2.0$ 、 $K_d = 3.0$ 。(a) 計算期望加速度。(b) 用小角度近似計算期望的滾轉與俯仰角。(c) 計算所需的總推力。(d) 驗證小角度近似是否適用。

解 (a)：位置誤差與速度誤差：

$$\begin{aligned} \mathbf{e}_p &= (5.0-2.0, 3.0-1.0, 3.0-3.0) = (3.0, 2.0, 0) \text{ m} \\ \mathbf{e}_v &= (0-0.5, 0-0, 0-0) = (-0.5, 0, 0) \text{ m/s} \end{aligned}$$

誤差向量。

$$\begin{aligned} \mathbf{a}_d &= \mathbf{K}_p \cdot \mathbf{e}_p + \mathbf{K}_d \cdot \mathbf{e}_v = 2.0(3.0, 2.0, 0) + 3.0(-0.5, 0, 0) \\ &= (6.0, 4.0, 0) + (-1.5, 0, 0) = (4.5, 4.0, 0) \text{ m/s}^2 \end{aligned}$$

期望加速度。

注意：這個加速度相當大（水平合成約 6.0 m/s^2 ，約 $0.61g$ ）。實務上會加上飽和限制，例如把水平加速度限制在 $0.5g$ 以內——否則會要求過大的傾角。

解（b）：由第 27.6.2 節的小角度近似：

$$\begin{aligned} \theta_d &= a_x/g = 4.5/9.8 = 0.459 \text{ rad} = 26.3^\circ \quad (\text{俯仰, 前傾}) \\ \phi_d &= -a_y/g = -4.0/9.8 = -0.408 \text{ rad} = -23.4^\circ \quad (\text{滾轉}) \end{aligned}$$

期望姿態角。

解（c）：總推力（垂直方向要求加速度為零，即維持高度）：

$$F = m \cdot (g + a_z) = 1.5 \times (9.8 + 0) = 14.7 \text{ N}$$

小角度近似下的總推力。

但這個近似在此處已不夠準確。完整的計算應為：

$$\begin{aligned} \mathbf{F}_{\text{vec}} &= m \cdot (\mathbf{a}_d + g \cdot \hat{\mathbf{z}}) = 1.5 \times (4.5, 4.0, 9.8) = (6.75, 6.0, 14.7) \text{ N} \\ F &= \|\mathbf{F}_{\text{vec}}\| = \sqrt{(6.75)^2 + (6.0)^2 + (14.7)^2} = \sqrt{(45.6 + 36 + 216.1)} = \sqrt{297.7} = 17.25 \text{ N} \end{aligned}$$

完整向量形式的推力，比小角度近似大了 17%。

解（d）：小角度近似的誤差來源是 $\cos$ 項。當傾角為 26.3° 時：

$$\text{誤差} \approx 1/\cos(26.3^\circ) - 1 = 1/0.897 - 1 = 0.115 = 11.5\%$$

推力誤差達 11.5%。

結論：此情況下小角度近似不適用，必須使用 27.6.1 節的完整向量形式。

實務原則：

- 傾角 $< 15^\circ$ ：小角度近似的誤差 $< 3.5\%$ ，可以接受。
- 傾角 $15^\circ \sim 30^\circ$ ：誤差 $3.5\% \sim 15\%$ ，建議使用完整形式。
- 傾角 $> 30^\circ$ ：必須使用完整形式，否則高度會明顯下沉。

這也解釋了一個常見的現象：入門級無人機在做大角度轉彎時會掉高度——因為它的飛控使用了小角度近似而未補償推力的餘弦損失。

程式範例：四旋翼的動力學與串級控制模擬

以下 Python 程式實作四旋翼的動力學模型、分配矩陣、串級控制器，並模擬懸停與軌跡追隨。同時驗證本章三個例題的計算。

```
import numpy as np

# ===== 四旋翼參數與動力學 =====
class Quadrotor:
    def __init__(s, m=1.2, L=0.25, kf=1.5e-5, km=2.4e-7,
                 Ixx=0.012, Iyy=0.012, Izz=0.022, w_max=1200.0):
        s.m, s.L, s.kf, s.km, s.w_max = m, L, kf, km, w_max
        s.I = np.diag([Ixx, Iyy, Izz])
        s.g = 9.8
        # 分配矩陣 ( 27.2.1 節 · 「十」字配置 )
        s.M = np.array([
            [ kf,      kf,      kf,      kf ], # 總推力
            [ 0,      -L*kf,      0,      L*kf ], # 滾轉  $\tau_\phi$ 
            [ L*kf,      0,      -L*kf,      0 ], # 俯仰  $\tau_\theta$ 
            [ km,      -km,      km,      -km ]]) # 偏航  $\tau_\psi$ 
        s.M_inv = np.linalg.inv(s.M)

    def allocate(s, F, tx, ty, tz):
        """由推力矩指令求各馬達轉速 ( 含飽和處理 ) """
        w2 = s.M_inv @ np.array([F, tx, ty, tz])
        w2 = np.clip(w2, 0, s.w_max**2) # 不能為負、不能超速
        return np.sqrt(w2)

    def hover_speed(s):
        F1 = s.m*s.g/4
        return np.sqrt(F1/s.kf)

    def max_thrust(s):
        return 4*s.kf*s.w_max**2

    def thrust_to_weight(s):
        return s.max_thrust()/(s.m*s.g)

# ===== 例題 27-1 =====
print("=== 例題 27-1：推力與分配 ===")
q = Quadrotor()
print(f"(a) 懸停總推力 = {q.m*q.g:.2f} N")
print(f"    每顆馬達推力 = {q.m*q.g/4:.3f} N")
print(f"    懸停轉速 = {q.hover_speed():.1f} rad/s "
      f"({q.hover_speed()*60/(2*np.pi):.0f} RPM)")

w = q.allocate(q.m*q.g, 0.15, 0, 0)
print(f"\n(b) 產生  $\tau_\phi=0.15$  N·m 時的四顆轉速:")
```

```

for i, wi in enumerate(w, 1):
    F_i = q.kf*wi**2
    print(f"    馬達{i}: {wi:7.1f} rad/s (推力 {F_i:.3f} N)")
print(f"    轉速差 = {w.max()-w.min():.1f} rad/s")

ratio = (q.L*q.kf)/q.km
print(f"\n(c) 滾轉/偏航控制權限比 = (L·kf)/km = {ratio:.1f}")
print(f"    → 偏航是最弱的控制通道")
print(f"\n(d) 最大推力 = {q.max_thrust():.1f} N")
print(f"    推重比 = {q.thrust_to_weight():.2f} "
      f"(懸停時油門 {100/q.thrust_to_weight():.1f}%)")

# ===== 例題 27-2：串級頻寬設計 =====
print("\n=== 例題 27-2：串級控制的頻寬設計 ===")
Ixx, wn_att, zeta = 0.012, 15.0, 1.0
Kp_att = Ixx*wn_att**2
Kd_att = Ixx*2*zeta*wn_att
print(f"(a) 姿態 PD: Kp = {Kp_att:.3f} N·m/rad, "
      f"Kd = {Kd_att:.3f} N·m·s/rad")
print(f"(b) 位置頻寬建議 ≤ {wn_att/5:.1f} rad/s "
      f"({wn_att/5/(2*np.pi):.2f} Hz)")
print(f"(c) 最低控制頻率:")
for name, wn in (("姿態", wn_att), ("位置", wn_att/5)):
    f_min = 20*wn/(2*np.pi)
    print(f"    {name}迴路: ≥ {f_min:.1f} Hz")
print(f"    → 實務上姿態用 500~1000 Hz、位置用 50 Hz")

# ===== 例題 27-3：位置到姿態的轉換 =====
print("\n=== 例題 27-3：位置控制與姿態指令 ===")
m3 = 1.5
pos = np.array([2.0, 1.0, 3.0]); vel = np.array([0.5, 0.0, 0.0])
pos_d = np.array([5.0, 3.0, 3.0]); vel_d = np.zeros(3)
Kp, Kd = 2.0, 3.0

a_d = Kp*(pos_d - pos) + Kd*(vel_d - vel)
print(f"(a) 期望加速度 = ({a_d[0]:.2f}, {a_d[1]:.2f}, {a_d[2]:.2f}) m/s²")
print(f"    水平合成 = {np.hypot(a_d[0], a_d[1]):.2f} m/s² "
      f"({np.hypot(a_d[0], a_d[1])/9.8:.2f} g)")

# 小角度近似
th_d = a_d[0]/9.8; ph_d = -a_d[1]/9.8
F_small = m3*(9.8 + a_d[2])
print(f"\n(b) 小角度近似: θ_d = {np.rad2deg(th_d):.1f}°, "
      f"φ_d = {np.rad2deg(ph_d):.1f}°")
print(f"(c) 小角度推力 F = {F_small:.2f} N")

# 完整向量形式

```

```

F_vec = m3*(a_d + np.array([0, 0, 9.8]))
F_full = np.linalg.norm(F_vec)
tilt = np.arccos(F_vec[2]/F_full)
print(f"    完整形式 F = {F_full:.2f} N "
      f"(比近似大 {(F_full/F_small-1)*100:.1f}%)")
print(f"\n(d) 實際傾角 = {np.rad2deg(tilt):.1f}°")
print(f"    近似誤差 = {(1/np.cos(tilt)-1)*100:.1f}% → 應使用完整形式")

print("\n 傾角與近似誤差的關係:")
print(f"{'傾角':>8}{ '推力誤差':>12} 建議")
for deg in (5, 10, 15, 20, 30, 45):
    err = (1/np.cos(np.deg2rad(deg)) - 1)*100
    rec = "近似可用" if err < 3.5 else ("建議完整" if err < 15 else "必須完整")
    print(f"{deg:7d}°{err:11.1f}% {rec}")

# ===== 串級控制模擬 =====
print("\n=== 串級控制模擬：懸停點的階躍響應 ===")
class CascadeController:
    """27.4~27.6 節的四層串級控制"""
    def __init__(s, quad):
        s.q = quad
        s.Kp_pos, s.Kd_pos = 2.0, 3.0          # 位置層
        s.Kp_att = 12.0                       # 姿態層 (輸出角速度)
        s.Kp_rate = np.array([0.30, 0.30, 0.15]) # 角速度層
        s.Kd_rate = np.array([0.008, 0.008, 0.004])
        s.prev_e_rate = np.zeros(3)

    def update(s, pos, vel, rpy, omega, pos_d, yaw_d, dt):
        # ---- 位置層 (外迴路·慢) ----
        a_d = s.Kp_pos*(pos_d - pos) + s.Kd_pos*(-vel)
        a_d[:2] = np.clip(a_d[:2], -0.5*s.q.g, 0.5*s.q.g) # 限制傾角
        # ---- 轉換為姿態指令 (完整向量形式·27.6.1 節) ----
        F_vec = s.q.m*(a_d + np.array([0, 0, s.q.g]))
        F = np.linalg.norm(F_vec)
        zb = F_vec/F                                # 期望的機體 Z 軸
        # 由 zb 與 yaw_d 建構期望姿態 (簡化：轉為 roll/pitch)
        th_d = np.arctan2(zb[0]*np.cos(yaw_d) + zb[1]*np.sin(yaw_d), zb[2])
        ph_d = np.arcsin(np.clip(zb[0]*np.sin(yaw_d) - zb[1]*np.cos(yaw_d),
                                -1, 1))
        rpy_d = np.array([ph_d, th_d, yaw_d])
        # ---- 姿態層 (中迴路) ----
        e_att = rpy_d - rpy
        e_att[2] = np.arctan2(np.sin(e_att[2]), np.cos(e_att[2])) # 角度包裹
        omega_d = s.Kp_att*e_att
        # ---- 角速度層 (內迴路·快) ----
        e_rate = omega_d - omega
        d_e = (e_rate - s.prev_e_rate)/dt
        s.prev_e_rate = e_rate
        tau = s.Kp_rate*e_rate + s.Kd_rate*d_e
        return F, tau, rpy_d

```

```

def simulate(pos0, pos_d, T=6.0, dt=0.001):
    quad = Quadrotor()
    ctrl = CascadeController(quad)
    pos = np.array(pos0, float); vel = np.zeros(3)
    rpy = np.zeros(3); omega = np.zeros(3)
    log = []
    for k in range(int(T/dt)):
        F, tau, rpy_d = ctrl.update(pos, vel, rpy, omega,
                                    np.array(pos_d), 0.0, dt)

        # 分配並飽和 ( 27.2.1 節的知識框：優先保證力矩 )
        w = quad.allocate(F, tau[0], tau[1], tau[2])
        actual = quad.M @ (w**2)
        F_a, tau_a = actual[0], actual[1:]
        # 動力學積分
        ph, th, ps = rpy
        R_z = np.array([
            np.cos(ph)*np.sin(th)*np.cos(ps) + np.sin(ph)*np.sin(ps),
            np.cos(ph)*np.sin(th)*np.sin(ps) - np.sin(ph)*np.cos(ps),
            np.cos(ph)*np.cos(th)])
        acc = F_a*R_z/quad.m - np.array([0, 0, quad.g])
        vel = vel + acc*dt; pos = pos + vel*dt
        omega_dot = np.linalg.solve(
            quad.I, tau_a - np.cross(omega, quad.I @ omega))
        omega = omega + omega_dot*dt; rpy = rpy + omega*dt
        if k % int(0.5/dt) == 0:
            log.append((k*dt, pos.copy(), np.rad2deg(rpy).copy()))
    return log, pos

print(f"{'時間':>7}{ 'x':>8}{ 'y':>8}{ 'z':>8}"
      f"{'roll':>9}{ 'pitch':>9}")
log, final = simulate([0, 0, 2.0], [2.0, 1.0, 3.0])
for t, p, r in log[:9]:
    print(f"{t:6.1f}s{p[0]:8.3f}{p[1]:8.3f}{p[2]:8.3f}"
          f"{r[0]:9.2f}{r[1]:9.2f}")
err = np.linalg.norm(final - np.array([2.0, 1.0, 3.0]))
print(f"最終位置誤差 = {err*1000:.1f} mm")
print("> 注意 pitch 先增大 ( 前傾產生水平力 ) 再回到零 ( 到位後回正 ) ")

# ===== 馬達失效的影響 =====
print("\n=== 馬達推力衰減對懸停的影響 ===")
print(f"{'推力衰減':>10}{ '可用推重比':>14}{ '懸停油門':>12} 狀態")
for decay in (0.0, 0.2, 0.5, 0.7, 0.87):
    q2 = Quadrotor(kf=1.5e-5*(1-decay))
    twr = q2.thrust_to_weight()
    throttle = 100/twr if twr > 1 else float('inf')
    st = "正常" if twr > 2 else ("勉強" if twr > 1.2 else
                               "臨界" if twr > 1 else "***無法起飛**")
    tstr = f"{throttle:.0f}%" if twr > 1 else "-"
    print(f"{decay*100:9.0f}%{twr:14.2f}{tstr:>12} {st}")
print("> 電池電量下降會使 kf 等效降低，這是低電量必須返航的原因")

```

請試著修改：(一) 把位置控制的水平加速度限制由 0.5g 改為 0.2g，觀察響應變慢但傾角

變小。(二)把角速度層的 K_p 加倍，觀察是否出現振盪。(三)在動力學中加入一個持續的水平風力(如 0.5 N)，觀察無人機如何自動傾斜對抗，以及穩態位置誤差有多大。

實作：無人機的組裝、調校與飛行

本章實作以小型四旋翼為對象。**安全提醒：螺旋槳極為危險，測試時必須拆下螺旋槳或在防護網內進行；戶外飛行必須遵守法規並選擇空曠場地。**

- 參數量測：**量測機身質量、機臂長度。用第 13 章的方法估算轉動慣量(可用三線擺法，或由 CAD 估算)。
- 推力測試：**把單一馬達固定在電子秤上，記錄不同 PWM 指令下的推力與轉速。繪出「轉速 vs 推力」曲線，用二次擬合求出推力係數 k_f ，驗證是否符合 $F = k_f \cdot \omega^2$ 。
- 推重比計算：**由測得的最大推力與機身質量計算推重比，判斷屬於哪個等級(例題 27-1d)。
- 分配矩陣驗證：**在拆下螺旋槳的狀態下，用飛控的除錯介面觀察——當手動傾斜機身時，四顆馬達的輸出如何變化。驗證是否符合分配矩陣的預期。
- IMU 校正：**執行加速度計與磁力計的校正程序。**特別記錄磁力計校正前後的差異——**把手機的指南針 App 靠近馬達，觀察讀值如何被干擾(第 27.7.2 節)。
- 姿態估測驗證：**把無人機固定在雲台或轉盤上，緩慢旋轉到已知角度，比較飛控輸出的姿態與實際角度。測試互補濾波的參數 α 對響應速度與雜訊的影響。
- 角速度 PID 調校：**這是最關鍵的一步。從保守的增益開始，逐步提高 P 直到出現高頻振盪(會聽到馬達的嗡嗡聲)，再退回 70%。接著調 D 抑制超越、最後加入少量 I。**每次只改一個參數。**
- 姿態迴路測試：**在安全的防護下(或用測試架)，驗證姿態的階躍響應——手動傾斜後放開，觀察回正的速度與是否超越。
- 懸停測試：**在空曠場地進行首次懸停。觀察是否有持續的漂移(可能是重心偏移或 IMU 未校正)、以及油門懸停點是否符合推重比的預期。
- 位置控制測試：**切換到位置模式，測試定點懸停的精度(用地面標記量測漂移範圍)。比較有無 GNSS 時的差異。
- 失效保護測試：**在安全高度下，測試各項保護機制——關閉遙控器(應觸發返航)、模擬

低電量（應強制降落）。這一步極為重要，必須在實際任務前完成。

12. （進階）軌跡追隨：規劃一個簡單的方形或圓形航線，比較實際飛行軌跡與規劃軌跡的誤差。測試不同速度下的追隨精度。

表 27-9 無人機調校記錄表（前兩列為範例）

項目	設定值	觀察	問題	調整
Rate P (roll)	0.15	反應遲鈍、追不上	增益不足	提高至 0.25
Rate D (roll)	0.010	機身高頻抖動	放大陀螺雜訊	降至 0.006 並加濾波
（自行填寫其餘項目）				

國中小也看得懂：無人機往前飛要先低頭

你有沒有想過，四旋翼無人機只有**四個螺旋槳**，為什麼可以前後左右上下自由飛？

先想一個問題：如果四個螺旋槳**一起加速**，會發生什麼事？答案是**往上飛**。一起減速就往下降。這很簡單。

但如果要**往前飛**呢？這裡有一個很有趣的祕密：**無人機必須先低頭！**

你可以這樣想像：螺旋槳的推力永遠是「垂直於機身」往上推的。如果無人機是平的，推力就完全往上，它只會上下動。但如果它稍微**前傾**，推力就會分成兩份——一份往上（撐住重量）、一份往前（推它前進）。

所以無人機往前飛的時候，其實是**歪著身體**在飛的！下次看到無人機飛過去，仔細觀察它的姿勢，你會發現它真的是斜的。

那怎麼讓它低頭呢？很簡單：**讓後面兩個螺旋槳轉快一點、前面兩個慢一點**。後面被推得比較高，前面就低下去了。

最後告訴你一件很重要的事：無人機和地上跑的機器人有一個**天大的差別**——

地上的機器人如果程式當掉了，它就停在原地，沒事。但無人機如果程式當掉了，它會從天上掉下來！

所以無人機的電腦必須**一秒鐘計算一千次**，不停地問自己「我現在歪了嗎？要怎麼調整？」一刻都不能休息。這就是為什麼工程師說：無人機是「**必須一直控制才能活著**」的機器。

叫獸提醒與常見錯誤

- 誤區一：以為無人機能「平移而不傾斜」。四旋翼欠驅動，水平加速度必須由傾斜產生，位置與姿態不能獨立控制。
- 誤區二：混淆欠驅動與不穩定。差速車欠驅動但停下來仍安全；無人機欠驅動且本質不穩定，失控即墜落。
- 誤區三：內部用歐拉角表示姿態。大角度機動時會遇到萬向節鎖，必須改用四元數或旋轉矩陣。
- 誤區四：計算姿態誤差時忽略四元數的雙重覆蓋。缺少 sign 項會使無人機朝錯誤方向翻轉。
- 誤區五：不做時間尺度分離。姿態與位置迴路的頻寬應相差 3~5 倍以上，否則兩層會互相干擾而振盪。
- 誤區六：角速度迴路的微分項未加濾波。馬達振動會被微分放大，造成機身高頻抖動與馬達過熱。
- 誤區七：大傾角時仍用小角度近似。傾角超過 30° 時推力誤差達 15%，會造成明顯的高度下沉。
- 誤區八：高度控制缺少重力前饋。控制器必須靠積分項累積出 mg ，起飛時會嚴重下沉。
- 誤區九：忽略電池電壓下降的影響。同樣的 PWM 在低電量時推力較小，會使飛行後期逐漸下沉。
- 誤區十：磁力計未校正或安裝位置不當。靠近馬達或鐵件會使偏航角錯誤，導致「馬桶效應」（繞圈飛行）。
- 誤區十一：垂直快速下降。可能進入渦環狀態而急遽失去升力，降落時應保持水平速度。
- 誤區十二：把緊急停止設計為切斷動力。這在無人機上等於墜落；正確的作法是分級降級而非停止。

課堂討論

1. 27.4 節的時間尺度分離在本書中已多次出現（第 5 章的伺服迴路、第 17 章的計算力矩控制、第 3 章的時間金字塔）。請歸納這個設計原則的共同結構，並討論它在非機器人領域的對應。
2. 微分平坦性使「給定位置軌跡即可算出所有狀態與輸入」。請討論這個性質為何如此珍貴，

以及它對規劃與控制的分工有何影響。

3. 無人機的安全哲學是「降級」而非「停止」。請比較這與第 7 章「失效導向安全」的異同，並討論還有哪些系統適用「降級」的思維。
4. 例題 27-1 顯示偏航的控制權限只有滾轉的 $1/15.6$ 。請討論這個限制的三個實務後果，並提出至少兩種機構層面的改善方向。
5. 27.7.3 節指出「室內飛行比室外困難」，關鍵在於缺少 GNSS 這個絕對參照。請討論：這個「絕對參照」的概念在本書其他章節如何出現？（提示：第 23 章的里程計漂移、第 25 章的迴路閉合）

章末摘要

- 四旋翼因機構極簡（無變距機構、只靠四顆馬達轉速）而成為主流；六旋翼以上則提供馬達失效的冗餘。
- 正反槳配置使反扭矩兩兩抵消；偏航控制則刻意打破此平衡，因此是最弱的控制通道。
- 推力與反扭矩皆正比於轉速平方；分配矩陣把 $(F, \tau_\phi, \tau_\theta, \tau_\psi)$ 映射到四顆馬達轉速。
- 控制分配飽和時應優先保證姿態力矩——「先活著，再談去哪裡」。
- 推力永遠沿機體 Z 軸，因此水平加速度完全由姿態決定：這是欠驅動的本質。
- 懸停點線性化後： $\ddot{x} \approx g \cdot \theta$ 、 $\ddot{y} \approx -g \cdot \phi$ ，這正是外迴路與內迴路的介面。
- 四旋翼有 6 自由度但只有 4 個輸入；可獨立控制的是 (x, y, z, ψ) ，滾轉與俯仰被位置需求佔用。
- 四旋翼具微分平坦性，平坦輸出為 (x, y, z, ψ) ；所有狀態與輸入可由其導數代數求出。
- 力矩正比於位置的四階導數（snap），因此軌跡規劃採最小 snap，解為分段七次多項式。
- 串級控制的依據是時間尺度分離：姿態時間常數數十毫秒、位置數百毫秒以上，相差十倍以上。
- 四層架構：軌跡規劃（10 Hz）→ 位置控制（50 Hz）→ 姿態控制（500 Hz）→ 角速度控制（1000 Hz）。
- 角速度內迴路最為關鍵：決定極限性能、唯一能直接對抗擾動、且調參最敏感。
- 姿態必須用四元數表示；姿態誤差為相對旋轉 $q_e = q^* \otimes q_d$ ，並須以 sign 項處理雙重覆蓋。

- 位置控制的三步驟：算期望加速度 → 求推力向量 → 由向量方向反推姿態與推力大小。
- 小角度近似在傾角 15° 內誤差小於 3.5%，超過 30° 則必須使用完整向量形式。
- 姿態估測是互補濾波的經典應用：陀螺儀高通、加速度計與磁力計低通。
- 位置估測依環境分三種方案：GNSS+IMU（戶外）、光流或 VIO（室內）、外部定位（實驗室）。
- 無人機沒有「安全停止」選項，其安全哲學是分級降級：位置→高度→姿態→手動模式。
- 常見失效包含電量不足、GNSS 失鎖、磁力計干擾、通訊中斷與渦環狀態，各有對應的保護機制。

觀念題與計算題

觀念題

1. 說明四旋翼為何以四個螺旋槳為最少配置，以及六旋翼以上的優勢。
2. 解釋正反槳配置的原理，以及偏航力矩如何產生。
3. 寫出分配矩陣的形式，並說明控制分配飽和時的優先順序原則。
4. 由平移動力學說明四旋翼的欠驅動本質，並解釋為何「往右平移不能不傾斜」。
5. 寫出懸停點附近的線性化關係，並說明它如何構成串級控制的介面。
6. 說明四旋翼可獨立控制哪四個量，以及為何滾轉與俯仰不是自由的。
7. 定義微分平坦性，並說明四旋翼的平坦輸出為何。
8. 說明為何四旋翼的軌跡規劃採用最小 snap 而非最小 jerk。
9. 說明時間尺度分離的原理，以及它為何是串級控制的依據。
10. 列出無人機控制的四層架構及其頻率，並說明角速度內迴路為何最關鍵。
11. 說明為何姿態內部計算必須用四元數，並寫出姿態誤差的計算方式。
12. 說明位置控制轉換為姿態指令的三個步驟。
13. 說明小角度近似的適用範圍，以及超出範圍時的後果。
14. 說明姿態估測中三個感測器的互補關係，以及磁力計校正的必要性。
15. 比較無人機在戶外、室內與實驗室三種環境下的位置估測方案。
16. 說明為何無人機不能採用「切斷動力」的緊急停止，以及分級降級的設計邏輯。

計算題

1. 某四旋翼 $m = 0.9 \text{ kg}$ 、 $L = 0.20 \text{ m}$ 、 $k_f = 1.2\text{e-}5 \text{ N}\cdot\text{s}^2$ 、 $k_m = 1.8\text{e-}7 \text{ N}\cdot\text{m}\cdot\text{s}^2$ 、馬達最高轉速 1500 rad/s 。(a) 求懸停時每顆馬達的轉速。(b) 求最大推力與推重比。(c) 求滾轉與偏航的控制權限比。(d) 若要產生 $0.1 \text{ N}\cdot\text{m}$ 的俯仰力矩，求前後馬達的推力差。
2. 某四旋翼 $I_{xx} = 0.008 \text{ kg}\cdot\text{m}^2$ 。(a) 若要求姿態自然頻率 20 rad/s 、阻尼比 0.8 ，求 PD 增益。(b) 建議位置迴路的頻寬上限。(c) 求兩層所需的最低控制頻率。(d) 若把 I_{xx} 減半（縮短機臂），重新計算 (a) 並討論其影響。
3. 某四旋翼 $m = 1.8 \text{ kg}$ ，當前位置 $(0, 0, 5) \text{ m}$ 、速度 $(1.0, 0.5, 0) \text{ m/s}$ ，目標位置 $(4, 3, 6) \text{ m}$ 、目標速度為零。 $K_p = 1.5$ 、 $K_d = 2.5$ 。(a) 求期望加速度。(b) 求完整向量形式的推力大小。(c) 求實際傾角。(d) 判斷小角度近似的誤差，並說明是否適用。
4. 某四旋翼的推重比為 3.0 。(a) 求懸停時的油門百分比。(b) 若電池電量下降使推力衰減 25% ，求新的推重比與懸停油門。(c) 求推力衰減到多少百分比時無法起飛。(d) 若要保留「推重比不低於 1.5 」的安全餘裕，求可容許的最大推力衰減。

動手做與專題延伸

本章實作：四旋翼模擬器

建立一個完整的四旋翼模擬環境，這是測試控制演算法最安全也最有效率的方式。

- 階段一：實作四旋翼的動力學模型與分配矩陣，驗證懸停平衡與例題 27-1。
- 階段二：實作角速度與姿態的串級控制器，測試姿態的階躍響應。
- 階段三：實作位置控制與姿態指令的轉換（含完整向量形式），測試定點懸停。
- 階段四：實作最小 snap 軌跡生成，並驗證微分平坦性——由軌跡直接算出所需的姿態與推力。
- 階段五：加入感測器雜訊與互補濾波，模擬完整的估測—控制迴路。

專題延伸

- 時間尺度分離的驗證：系統性地改變姿態迴路與位置迴路的頻寬比（ $1:1$ 、 $2:1$ 、 $5:1$ 、 $10:1$ ），記錄系統的響應與穩定性。你會看到當比值太小時，兩層會互相干擾而產生振盪——這是理解串級控制原理最直接的實驗。繪出「頻寬比 vs 超越量」的曲線。
- 微分平坦性的視覺化：給定一條三維的最小 snap 軌跡，用平坦性關係直接計算出全程所需

的姿態角、角速度與推力。把姿態以小座標軸的形式沿軌跡繪出，你會看到無人機如何在轉彎時自動傾斜、在加速時前傾——這張圖能讓抽象的平坦性變得直觀。

- **進階：馬達失效的容錯控制。**模擬一顆馬達完全失效的情況，觀察四旋翼如何失控翻覆。接著實作一個容錯策略——**犧牲偏航控制**（讓機身持續自轉），只用剩餘三顆馬達維持推力與滾轉俯仰的平衡。驗證這個策略能否使無人機維持高度並安全降落。這個題目結合了控制分配、欠驅動分析與安全設計，是本章的綜合考驗。

參考文獻與延伸閱讀

- Mellinger, D. and Kumar, V., "Minimum Snap Trajectory Generation and Control for Quadrotors," ICRA 2011 (微分平坦性與最小 snap 軌跡的奠基論文)。
- Mahony, R., Kumar, V. and Corke, P., "Multirotor Aerial Vehicles: Modeling, Estimation, and Control of Quadrotor," IEEE Robotics & Automation Magazine, 2012 (四旋翼建模與控制的優秀綜述)。
- Lee, T., Leok, M. and McClamroch, N. H., "Geometric Tracking Control of a Quadrotor UAV on SE(3)," CDC 2010 (幾何控制方法，避免歐拉角的奇異性)。
- Beard, R. W. and McLain, T. W., Small Unmanned Aircraft: Theory and Practice, Princeton University Press (無人機的完整教材，含固定翼)。
- Quan, Q., Introduction to Multicopter Design and Control, Springer (多旋翼設計與控制的系統性教材)。
- Mahony, R., Hamel, T. and Pflimlin, J.-M., "Nonlinear Complementary Filters on the Special Orthogonal Group," IEEE Trans. Automatic Control, 2008 (姿態估測的互補濾波理論)。
- Faessler, M., Franchi, A. and Scaramuzza, D., "Differential Flatness of Quadrotor Dynamics Subject to Rotor Drag," RA-L, 2018 (考慮空氣阻力的平坦性推廣)。
- Loianno, G. et al., "Estimation, Control, and Planning for Aggressive Flight with a Small Quadrotor," RA-L, 2017 (高機動飛行的完整系統)。
- PX4 與 ArduPilot 開源飛控專案文件：兩套最主流的開源飛控，含完整的控制架構與調參指南。
- 交通部民用航空局《遙控無人機管理規則》與相關公告：台灣無人機的法規依據。

第 28 章

腿式機器人與運動控制

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為降維抽象的對照：上方是一台三十自由度的人形機器人，下方以粗箭頭指向一根簡單的倒立擺（質點、無質量桿、單一接觸點），標註「 $LIPM: 30+ DOF \rightarrow 3$ 個狀態量」。中央畫出關鍵幾何——質心高度 z 、質心水平位置 x 、 ZMP 位置，以及捕捉點 $x + \dot{x}/\omega$ 的位置（以紅色標出，並畫出一隻正要踩上去的腳）。右半為四足小跑步態的四格連續動作，每格下方以色塊時序圖標出四隻腳的支撐相（實心）與擺動相（空心），並以虛線標出當下的支撐多邊形。下緣文字：透過選擇落腳點，反覆消除運動的發散分量。

叫獸開講 「走路的本質，是一連串被打斷的跌倒。」

叫獸請全班站起來：「請你們往前走三步，然後告訴我——在走路的過程中，有沒有哪一個瞬間，你其實是正在跌倒的？」

學生們慢慢走著，小矧先開口：「腳抬起來的時候……身體是往前倒的，只是下一隻腳及時撐住了。」

「完全正確。」叫獸點頭，「人類走路的本質，是一連串被打斷的跌倒。你不是先站穩再移動，而是先讓自己失去平衡，再用下一步接住自己。」

他在白板上畫了兩個字：**靜態**、**動態**。「有一種走路方式叫靜態步行——重心永遠保持在支撐腳的正上方，任何一刻停下來都不會倒。這就是早期人形機器人那種『慢慢挪、慢慢挪』的走法。安全，但慢得可怕，而且能量效率極差。」

「人類不是這樣走的。我們用的是**動態步行**——大部分時間重心根本不在支撐面內，我們靠的是『下一步會接住我』這個預測。」

小翰舉手：「那不是很危險嗎？萬一沒接到？」

「就跌倒啊。」叫獸笑了，「這就是為什麼腿式機器人比輪型機器人難這麼多。輪子隨時可以停下來，腿不行——你一旦開始動態行走，就進入了一個必須持續正確預測的狀態。」

接著他問了一個更關鍵的問題：「而且腿有一個輪子沒有的麻煩。當腳踩在地上時，機器人是一個**被約束的系統**——腳不能穿透地面、不能滑動。但當腳抬起來時，那些約束**全部消失**了。每走一步，系統的方程式就要換一次。」

他在白板上寫下一個詞：**混合動態**。「第 27 章的無人機雖然難，但它的動態是**連續**的——螺旋槳一直在轉，方程式從頭到尾都一樣。腿式機器人不是，它每一步都在切換。這是完全不同層級的困難。」

最後他拿出一張紙，畫了一根倒立的桿子，底下是一個點。「但有一個好消息。**這麼複雜的系統，在規劃層面上可以簡化成一根倒立擺**——第 18 章那個東西。三十個自由度、每步切換的接觸狀態，最後歸結成一個三維的倒立擺模型。今天你們會看到，**簡化模型是如何讓不可能變成可能的**。」

本章為什麼重要

腿式機器人是機器人學中最困難、也最能展現技術整合度的領域。過去十年，波士頓動力的 Atlas 與 Spot、宇樹的四足機器人、以及各家人形機器人的快速進展，使它從實驗室走向了實用。

為什麼要用腿？ 輪子在平整地面上效率遠高於腿——這是無可爭議的。但地球上**只有約一半的陸地表面是輪型載具能到達的**。樓梯、碎石、瓦礫、沙地、森林、以及所有為人類設計的建築內部——這些環境對輪子是障礙，對腿卻是可以跨越的。

在技術上，腿式機器人是本書所有內容的**最終整合考驗**。它需要：第 13、14 章的動力學（自由度極高）、第 15 章的軌跡生成（步態規劃）、第 17~19 章的控制與穩定性（本質不穩定）、第 20 章的力控制（接觸力分配）、第 24 章的狀態估測（沒有絕對參照）、第 26 章的規劃（地形上的落腳點）、以及第 31 章的學習（近年最重要的突破）。

本章也將揭示一個貫穿全書的方法論頂點：**用簡化模型處理複雜系統**。一台三十自由度的人形機器人，在規劃層面可以簡化為一根三維倒立擺——這個抽象使原本不可解的問題變得可解。理解這個思維，比記住任何一條公式都重要。

學習目標

- 能說明腿式機器人相對於輪型的優勢與代價。
- 能說明混合動態的意義，以及接觸切換帶來的困難。
- 能區分靜態穩定與動態穩定，並說明各自的判準。
- 能定義支撐多邊形與零力矩點（ZMP），並說明其物理意義。
- 能說明線性倒立擺模型的推導與其在規劃上的價值。
- 能說明捕捉點的概念及其對落腳點選擇的指導。
- 能描述四足與雙足的常見步態並比較其特性。
- 能說明步態的相位、佔空比與其對穩定性的影響。
- 能說明腿式機器人控制的分層架構。
- 能說明全身控制的最佳化形式與接觸力分配。
- 能說明腿式機器人狀態估測的特殊困難與解法。

- 能說明強化學習在腿式運動上的突破與其與模型式方法的關係。

先備知識

第 13、14 章的動力學（尤其是自由度與慣性）、第 18 章的 LQR 與倒立擺、第 19 章的穩定性與欠驅動、第 20 章的力控制與接觸、第 24 章的狀態估測、第 27 章的欠驅動系統與分層控制。

本章是全書整合度最高的一章。強烈建議先確認第 18、19、20、27 章已充分理解——本章會直接引用那些結論而不重新推導。若時間有限，至少要掌握 28.4 節的線性倒立擺模型與 28.7 節的分層架構。

核心名詞中英對照

表 28-1 本章核心名詞

中文	英文	說明
混合動態	Hybrid Dynamics	連續動態與離散切換交替的系統
支撐多邊形	Support Polygon	所有觸地點所張的凸包
靜態穩定	Static Stability	重心投影落在支撐多邊形內
動態穩定	Dynamic Stability	含慣性效應下仍不傾倒
零力矩點	Zero Moment Point, ZMP	地面反作用力合力矩為零的點
壓力中心	Center of Pressure, CoP	地面壓力分布的等效作用點
線性倒立擺模型	Linear Inverted Pendulum Model, LIPM	把質心運動簡化為倒立擺的模型
捕捉點	Capture Point	踩下去就能停止的落腳位置
發散分量	Divergent Component of Motion, DCM	捕捉點的推廣形式
步態	Gait	各腿抬放的時序模式
支撐相	Stance Phase	腳著地承重的階段
擺動相	Swing Phase	腳離地前擺的階段
佔空比	Duty Factor	支撐相佔整個步態週期的比例
落腳點規劃	Footstep Planning	決定每一步該踩在哪裡
全身控制	Whole-Body Control, WBC	統一協調所有關節的控制架構
接觸力分配	Contact Force Distribution	決定各腳承受多少力
摩擦錐	Friction Cone	不打滑的接觸力方向範圍

中文	英文	說明
被動行走	Passive Dynamic Walking	不需驅動即可下坡行走的機構

28.1 為何要用腿

28.1.1 輪子的極限

在平整的地面上，輪子的效率遠高於腿。一台輪型機器人的**運輸成本**（cost of transport，單位距離每單位重量所需的能量）可以低到 0.1 以下，而人類行走約為 0.2、多數腿式機器人則在 0.5~2 之間。

但輪子有三個無法克服的限制：

- **必須連續的支撐面**：輪子無法跨越大於其半徑的縫隙或台階。一台輪徑 10 公分的機器人，連 5 公分的門檻都過不去。
- **無法選擇接觸點**：輪子走過的每一點都必須是可通行的。腿則可以**跳過**不可踩的區域——這在碎石、瓦礫、濕滑的地面上是決定性的差異。
- **對地面破壞**：履帶與輪子在轉向時會刮傷地面（第 23.6.1 節），這在草坪、農地與室內是問題。

◎ 為何自然界沒有輪子

生物演化出了飛行、游泳、爬行、行走，卻沒有任何大型生物使用輪子移動。原因有二：一、輪子需要連續旋轉的關節，而這在生物的血管與神經供應上幾乎不可能（必須有一個完全分離、自由旋轉的部件）；二、輪子在自然地形上根本不佔優勢。自然界的地表充滿不規則的起伏，而腿能夠選擇落腳點——這個能力在崎嶇地形上的價值，遠超過輪子在地上的效率優勢。這也解釋了為何在災難現場、森林、與為人類設計的建築中，腿式機器人有其不可取代性。

28.1.2 腿的代價

但腿的代價極為高昂，這解釋了為何工業界至今仍以輪型為主：

表 28-2 輪型與腿式機器人的比較

面向	輪型	腿式
平地效率	極高	低（3~10 倍耗能）
崎嶇地形	受限	佳

面向	輪型	腿式
跨越障礙	小於輪半徑	可達腿長的一半以上
自由度	2~4	12~30 以上
控制複雜度	低	極高
穩定性	天然穩定	本質不穩定
成本	低	高 (多倍)
可靠度	高	低 (故障點多)
典型應用	工廠、倉儲、平整環境	巡檢、災難、樓梯、戶外

28.1.3 被動行走：一個重要的啟發

1990 年 McGeer 展示了一個令人震驚的裝置：一組完全沒有馬達、沒有感測器、沒有控制器的機械腿，放在一個緩坡上輕輕一推，它就能穩定地走下去——步態自然、甚至比許多有動力的機器人更像人。

這稱為**被動動態行走** (passive dynamic walking)。它的能量完全來自重力位能，控制則完全由機構的動力學自然產生。

它的啟示極為深刻：行走的穩定性，有很大一部分可以由**機構本身的動力學**提供，而非全靠控制器。這與第 21 章「柔順性取代感測與計算」、第 20 章「一個倒角勝過複雜控制」的思維完全一致——在把問題丟給控制器之前，先問能不能用機構解決。

後續的研究由此發展出「**極簡驅動行走**」 (minimally actuated walking) ——只在必要處加入少量驅動，補充能量並提供轉向與適應能力，其餘交給被動動力學。這類機器人的能量效率能接近人類水準，遠優於全主動控制的機器人。

28.2 混合動態

28.2.1 每一步都在換方程式

如開頭故事所述，腿式機器人最根本的困難是**混合動態** (hybrid dynamics) ——系統的方程式會隨接觸狀態而改變。

以雙足行走為例，一個步態週期至少包含四種不同的動態模式：

表 28-3 雙足行走的動態模式

階段	接觸狀態	系統性質	約束數
雙腳支撐	兩腳皆著地	過約束（閉環）	多（12）
單腳支撐（左）	左腳著地	受約束	中（6）
單腳支撐（右）	右腳著地	受約束	中（6）
騰空（跑步時）	雙腳離地	自由飛行	0

每一次狀態切換，都伴隨兩件事：

一、約束方程改變。著地的腳提供了約束（不能穿透、不能滑動），這使系統的有效自由度減少。抬起時約束消失。

二、碰撞（impact）。腳落地的瞬間是一次碰撞——速度會不連續地改變。這在數學上是一個離散的狀態跳變：

$$q^+ = q^- \quad (\text{位置連續}) \quad q^+ = \Delta(q) \cdot q^- \quad (\text{速度跳變})$$

碰撞映射。 Δ 為碰撞矩陣，由動量守恆推導。部分動能會在碰撞中損失——這是腿式行走能量效率低的主因之一。

28.2.2 為何這使問題困難得多

混合動態帶來三個層次的困難：

困難一：無法用單一模型。第 13~19 章的所有工具都假設一個固定的動力學方程。混合系統則需要為每個模式各建一套，並處理切換。

困難二：切換時機也是決策變數。「什麼時候該抬腳」本身就是一個要規劃的量。這使問題成為混合整數最佳化——比連續最佳化困難得多。

困難三：穩定性理論不直接適用。第 19 章的 Lyapunov 方法假設連續動態。對混合系統，必須使用龐加萊映射（Poincaré map）——分析「每走一步後，狀態如何映射到下一步」，若這個映射有穩定的固定點，則週期步態是穩定的。

◎ 龐加萊映射：把「一步」當成一次迭代

分析週期運動有一個優雅的技巧：不去追蹤連續的軌跡，而是只看每一步結束時的狀態。例如在雙足行走中，每次左腳觸地的瞬間記錄一次狀態，得到一個離散序列 $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots$ 。若這個序列收斂到一個固定點，就代表機器人能穩定地重複同樣的步態。這把一個複雜的連續混合系統，化簡

為一個離散映射的穩定性問題——而後者可以用第 19.9 節的離散系統判準（特徵值落在單位圓內）分析。這是降維思維的又一個範例。

28.3 平衡的判準

28.3.1 支撐多邊形與靜態穩定

最基本的平衡概念是**支撐多邊形**（support polygon）——所有觸地點所張成的凸包。

靜態穩定的判準極為簡單：

重心（CoM）在地面的投影，落在支撐多邊形內部

靜態穩定判準。若成立，機器人在任何時刻停下來都不會倒。

這個判準的價值在於它的保守與可靠。早期的人形機器人（如 Honda P2、ASIMO 的早期版本）與多數四足機器人的慢速步態，都採用靜態穩定。

但它的代價也極為明顯：

- **速度受限**：重心必須始終保持在支撐面內，這使步態必須緩慢。
- **能量效率差**：為了維持重心位置，關節必須持續施力，無法利用擺動的自然動力學。
- **四足必須至少三腳著地**：因為兩點只能張出一條線段，無法形成有面積的多邊形。這使爬行步態（crawl gait）成為唯一選擇。

28.3.2 動態穩定與 ZMP

要走得快，就必須放棄靜態穩定。但這時需要一個新的判準——**零力矩點**（Zero Moment Point, ZMP）。

◎ ZMP 的定義

零力矩點是地面上的一個點，在該點處，地面反作用力所產生的**水平方向力矩為零**。等價地說，它是地面壓力分布的**等效作用點**（在腳未傾倒時，ZMP 與壓力中心 CoP 重合）。

ZMP 的判準是：

ZMP 落在支撐多邊形內部 ⇔ 腳不會繞邊緣翻轉

動態穩定判準。注意這裡是 ZMP 而非重心投影——兩者在有加速度時會分離。

為何 ZMP 比重心投影更適用？因為它把慣性效應納入了考量。ZMP 的位置可由下式計算

(簡化的線性倒立擺情形)：

$$\mathbf{x}_{zmp} = \mathbf{x}_{com} - (\mathbf{z}_{com}/g) \cdot \ddot{\mathbf{x}}_{com}$$

ZMP 與重心的關係。第二項是慣性的修正——重心加速度越大，ZMP 偏離重心投影越遠。

這條式子有極為重要的物理意義：**當重心向前加速時，ZMP 會往後移**。這正是人類前傾起步時的物理——重心已經超出腳掌前緣，但因為向前加速，ZMP 仍在腳掌內，所以不會摔倒。

28.3.3 ZMP 的限制

ZMP 判準雖然實用，但有三個重要的限制：

限制一：只適用於平坦地面且腳為面接觸。若在碎石上單點著地，或在斜坡上，ZMP 的定義需要推廣（有 FRI、CWC 等推廣形式）。

限制二：無法處理騰空階段。跑步或跳躍時雙腳離地，沒有支撐多邊形，ZMP 判準完全失效。

限制三：它是必要而非充分條件。ZMP 在支撐面內只保證「腳不會翻轉」，不保證「機器人不會跌倒」——例如上半身可能因為其他原因失去平衡。

因此現代的方法逐漸轉向 28.5 節的**捕捉點**與更一般的接觸扳手錐（CWC）判準。但 ZMP 因其直觀與計算簡單，至今仍是雙足行走規劃的主力工具。

28.4 線性倒立擺模型

28.4.1 把三十個自由度變成三個

這是本章最重要的觀念，也是腿式機器人得以實用的關鍵。

一台人形機器人有三十個以上的自由度，其完整動力學方程極為龐大。要在毫秒級的控制迴路中規劃步態，直接使用完整模型是不可能的。

線性倒立擺模型 (Linear Inverted Pendulum Model, LIPM) 提供了一個極簡但足夠好的抽象：

◎ LIPM 的三個假設

假設一：全身的质量集中在質心 (CoM) ——忽略各連桿的分布，把機器人視為一個點質量。假

設二：質心維持在固定高度 ($z_{com} = \text{常數}$) ——這使垂直方向的動態消失。**假設三：**不產生繞

質心的角動量——忽略手臂擺動等效應。在這三個假設下，機器人簡化為「一根長度可變、頂端有質點的倒立擺」。

在這些假設下，質心的水平動態變得極為簡潔：

$$\ddot{x} = (g/z_{com}) \cdot (x - x_{zmp})$$

LIPM 方程。這是一個線性方程！質心的加速度正比於「質心與 ZMP 的水平距離」。

定義一個關鍵的參數：

$$\omega = \sqrt{g/z_{com}}$$

自然頻率。它是 LIPM 的唯一參數，決定了系統的時間尺度。以 $z_{com} = 0.8\text{ m}$ 為例， $\omega = \sqrt{9.8/0.8} = 3.5\text{ rad/s}$ ，對應時間常數 0.29 秒。

28.4.2 解與其物理意義

LIPM 方程 $\ddot{x} = \omega^2(x - x_{zmp})$ 的解（設 x_{zmp} 為常數）包含兩個模態：

$$x(t) = A \cdot e^{\omega t} + B \cdot e^{-\omega t} + x_{zmp}$$

一個發散模態 ($e^{\omega t}$) 與一個收斂模態 ($e^{-\omega t}$)。

那個**發散**的模態正是問題的核心——它代表倒立擺會越倒越快。這是第 18.9 節倒立擺不穩定性的重現，只是現在有了明確的時間常數 $1/\omega$ 。

控制的本質，就是消除發散模態。而唯一的手段是**移動 ZMP**——也就是換腳、選擇下一個落腳點。這給出了腿式行走最核心的洞見：

行走 = 透過選擇落腳點，反覆消除質心運動的發散分量

這一句話概括了整個腿式運動控制的本質。

28.4.3 LIPM 為何如此有效

LIPM 的三個假設看似粗暴，但實務效果極佳，原因有三：

理由一：質心高度確實近似固定。人類行走時質心的垂直起伏只有約 5 公分（相對於 0.9 公尺的高度，變化不到 6%）。

理由二：質量確實集中。人形機器人的軀幹通常佔總質量的 50% 以上，且腿部的質量分布可以用等效方式併入。

理由三：規劃與控制的分工。LIPM 只用於**規劃層**（決定質心軌跡與落腳點），實際執行時

仍使用完整的動力學模型（28.7 節的全身控制）。簡化模型負責「該往哪走」，完整模型負責「怎麼走到」——這正是第 27 章分層思維的延續。

圖 28-1 線性倒立擺模型

建議配圖：左半為對照圖——上方是一台複雜的人形機器人（標註「30+ 自由度」），下方箭頭指向一根簡單的倒立擺（一個質點、一根無質量的桿、一個接觸點），標註「LIPM：3 個狀態量」。中央標出關鍵幾何：質心高度 z_{com} （虛線水平面表示「維持固定高度」）、質心水平位置 x 、ZMP 位置 x_{zmp} ，並以箭頭標示兩者的水平距離。右半為 LIPM 的相平面圖（橫軸 x 、縱軸 $\dot{x}$ ），畫出發散模態（沿 $\dot{x} = \omega x$ 這條線遠離）與收斂模態（沿 $\dot{x} = -\omega x$ 這條線趨近），並標出捕捉點的位置。下緣公式： $\ddot{x} = \omega^2(x - x_{zmp})$ ， $\omega = \sqrt{g/z}$ 。

28.5 捕捉點

28.5.1 踩在哪裡才能停下來

由 28.4.2 節的結論，控制的核心是消除發散模態。這引出一個極為實用的問題：如果我現在想停下來，該把腳踩在哪裡？

答案就是捕捉點（Capture Point，也稱瞬時捕捉點 ICP）：

$$x_{cp} = x + \dot{x}/\omega$$

捕捉點。它是「當前質心位置」加上「速度除以自然頻率」。

這個定義的意義極為直觀：若把 ZMP（也就是腳）放在捕捉點上，質心會恰好在無窮遠的時間後停在該點，不會超過也不會回頭。

驗證：把 $x_{zmp} = x_{cp}$ 代入 LIPM 方程並解之，可以證明發散模態的係數恰好為零——只剩下收斂模態。

◎ 捕捉點的直觀理解

想像你被人從背後推了一把，身體開始往前倒。你會下意識地跨出一步——而那一步該跨多遠？太近會撐不住繼續倒，太遠會被自己絆住而往後彈。捕捉點就是那個「剛剛好」的位置。而公式 $x + \dot{x}/\omega$ 說得很清楚：你倒得越快（ $\dot{x}$ 越大），就要跨得越遠。這正是人類的本能反應，而它有一個精確的數學形式。

28.5.2 捕捉點的三個用途

用途一：判斷是否還能站住。若捕捉點落在可達的落腳範圍內，機器人就能透過一步（或數步）恢復平衡；若落在範圍外，則必定會跌倒——此時應立即執行保護動作（第 28.9 節）。

用途二：決定落腳點。這是最直接的應用。要停下來就踩在捕捉點上；要繼續前進，就踩在

捕捉點稍前方（留下一些殘餘速度）；要加速，就踩在捕捉點後方。

踩在捕捉點上 → 停止

踩在捕捉點前方 → 減速

踩在捕捉點後方 → 加速

落腳點與速度控制的關係。這是腿式運動最核心的控制律之一。

用途三：量化穩定裕度。捕捉點到支撐多邊形邊界的距離，就是一個直觀的穩定裕度指標——距離越大，能承受的擾動越大。

28.5.3 N 步捕捉

上述討論是「一步捕捉」（1-step capturable）。更一般地，可以定義 **N 步捕捉區域**——需要 N 步才能停下來的初始狀態集合。

這個概念在實務上極為有用：一台機器人被推了一把之後，控制器可以判斷「我需要跨兩步才能穩住」，並據此規劃一連串的落腳點。波士頓動力 Atlas 那些被推後踉蹌數步再站穩的著名影片，其底層邏輯正是這個。

28.5.4 DCM：捕捉點的推廣

捕捉點的一個重要推廣稱為**發散運動分量**（Divergent Component of Motion, DCM）。它的定義形式相同，但被視為一個**獨立的狀態變數**來控制：

$$\xi = x + \dot{x}/\omega \quad (\text{DCM}) \quad \xi = \omega(\xi - x_{\text{zmp}})$$

DCM 的動態。它只有一個發散模態——這使控制問題大幅簡化。

這個轉換的巧妙之處在於：原本的二階系統（含一個發散、一個收斂模態）被**分解**為兩個一階系統——DCM（發散，需要控制）與質心（收斂，會自動跟隨）。

控制 DCM $\Rightarrow$ 質心自動穩定

只需處理發散的部分。這是一個典型的「分而治之」——把不穩定的部分隔離出來單獨處理。

DCM 是目前雙足行走規劃的主流方法，被廣泛用於各國的人形機器人研究（如 DLR、IHMC、以及多數 DARPA 機器人挑戰賽的隊伍）。

28.6 步態

28.6.1 步態的描述

步態 (gait) 是各條腿抬起與放下的時序模式。描述步態需要三個要素：

- **相位 (phase)**：各腿在週期中的相對時間偏移。例如四足的對角小跑，是「左前與右後」同相、「右前與左後」相差半個週期。
- **佔空比 (duty factor)**：支撐相佔整個週期的比例。佔空比大於 0.5 代表隨時至少有一半的腿著地 (較穩定但較慢)。
- **步長與步頻**：決定行進速度。速度 = 步長 × 步頻。

28.6.2 四足的常見步態

表 28-4 四足機器人的常見步態

步態	腿的時序	佔空比	同時著地	速度	穩定性
爬行 Crawl	一次抬一腳	0.75	3 腳	最慢	靜態穩定
小跑 Trot	對角兩腳同時	0.5	2 腳 (對角)	中	動態穩定
溜蹄 Pace	同側兩腳同時	0.5	2 腳 (同側)	中	動態 (側向不穩)
跳躍 Bound	前兩腳、後兩腳	0.4~0.5	2 腳 (前或後)	快	動態
飛奔 Gallop	四腳依序、含騰空	< 0.4	1~2 腳或騰空	最快	高度動態

爬行是唯一的靜態穩定步態——因為它保證隨時有三腳著地，能形成有面積的支撐多邊形。其餘步態都需要動態平衡。

小跑 (trot) 是實務上最常用的步態，原因有三：對角的兩腳形成的支撐線通過質心附近 (側向較平衡)、機構的對稱性使控制簡單、且速度與穩定性的取舍適中。多數四足機器人的預設步態都是小跑。

自然界的啟示：馬會依速度自動切換步態——慢走用 walk、中速用 trot、快速用 gallop。切換的時機由能量效率決定：每種步態在特定的速度範圍內最省能。這給了機器人設計一個重要的參考——不應只用一種步態，而應依速度切換。

28.6.3 雙足的步態

雙足的步態較單純，主要分兩類：

行走 (walking)：佔空比大於 0.5，因此有**雙腳支撐期** (兩腳同時著地)。這個重疊期提

供了穩定的過渡，也是能量交換的時機。

跑步 (running)：佔空比小於 0.5，因此有**騰空期**（雙腳皆離地）。此時機器人是一個自由飛行的物體——完全無法控制質心軌跡，只能為下一次落地做準備。

行走：無騰空期，有雙腳支撐期

跑步：有騰空期，無雙腳支撐期

兩者的定義差異。這也是競走比賽的規則依據——「必須隨時有一腳著地」。

騰空期是雙足跑步最困難的部分。在這段時間內：質心沿拋物線運動、無法施加任何外力、只能調整腿的姿態為落地做準備。而落地的瞬間又是一次碰撞，必須精確控制才不會失衡。

28.6.4 彈簧負載倒立擺 (SLIP)

對於跑步與跳躍，LIPM 不再適用（因為質心高度會大幅變化）。此時的標準簡化模型是 **SLIP** (Spring-Loaded Inverted Pendulum) ——一個質點加上一根**彈簧腿**。

SLIP 模型的價值在於它捕捉了跑步的本質：**能量在動能、位能與彈性位能之間循環**。著地時腿被壓縮（動能轉為彈性位能）、蹬離時彈回（彈性位能轉回動能）。

這也解釋了為何許多腿式機器人採用**串聯彈性驅動器**（SEA，第 5.9 節）——彈簧不只是為了安全與力控制，更能**儲存並回收能量**，大幅提升跑步的效率。這是「機構參與控制」的又一個範例（呼應 28.1.3 節的被動行走）。

28.7 分層控制架構

28.7.1 三層架構

腿式機器人的控制採用與第 27 章無人機類似、但更複雜的分層架構。典型分為三層：

表 28-5 腿式機器人的控制分層

層級	頻率	使用的模型	輸出	任務
步態與落腳點規劃	1~10 Hz	LIPM / SLIP (簡化)	落腳點序列、質心軌跡	決定往哪走、踩哪裡
質心軌跡最佳化 (MPC)	20~100 Hz	LIPM 或單剛體	質心軌跡、接觸力	在未來數步內最佳化
全身控制 (WBC)	200~1000 Hz	完整動力學	各關節力矩	把上層指令轉為關節力矩

這個分層的核心邏輯是「模型複雜度隨頻率遞增而遞減」——最上層用最簡化的模型看得最遠、最下層用最完整的模型算得最細。這正是第 27.4 節時間尺度分離思想的推廣。

28.7.2 中層：模型預測控制

中層的 MPC (第 17.8.4 節) 是近十年腿式機器人最重要的突破之一。它的作法是：

1. 以 LIPM 或單剛體模型，預測未來 0.5~1.5 秒 (涵蓋 2~4 步) 的運動。
2. 求解一個最佳化問題：在滿足接觸約束 (摩擦錐、力的上下限) 與 ZMP 約束的前提下，最小化「軌跡追隨誤差」與「接觸力大小」。
3. 只執行最佳解的第一步。
4. 下一個週期以新的狀態重新求解 (滾動時域)。

MPC 的關鍵優勢正是第 18.8.2 節提到的：它能明確處理約束。摩擦錐、力的極限、ZMP 必須在支撐面內——這些都是硬約束，用 LQR 或 PID 都無法直接表達，但 MPC 可以。

MIT Cheetah 3 是這個方法的代表性成果——它用凸最佳化的 MPC 實現了四足機器人的盲走上樓、後空翻與高速奔跑。

28.7.3 底層：全身控制

最底層的全身控制 (Whole-Body Control, WBC) 負責把上層的指令，轉換為每一個關節的力矩。它面對的問題是：

已知：期望的質心加速度、期望的腳部軌跡、接觸的狀態

求：n 個關節力矩 + 各接觸點的力

這是一個帶約束的最佳化問題。

典型的表述形式是一個二次規劃 (QP)：

$$\min \|\text{任務誤差}\|^2 + \|\text{力矩}\|^2$$

s.t. 動力學方程、接觸約束、摩擦錐、力矩極限

WBC 的 QP 表述。每個控制週期求解一次 (1 kHz 時只有 1 毫秒)。

它與第 12.8 節分層逆運動學、第 17.9 節動態一致的多任務控制是同一個譜系——都是在處理「多個任務、有限的自由度」的問題。差別在於腿式機器人多了接觸約束這個關鍵的複雜性。

28.7.4 接觸力分配與摩擦錐

WBC 中最特殊的部分是**接觸力分配**——決定每隻腳該承受多少力。這個問題有三個約束：

- **單向性**：腳只能**推**地面，不能拉。因此法向力必須為正 ($f_n \geq 0$) ——這與第 21.3.3 節抓取的「手指只能推」完全相同。
- **摩擦錐**：切向力不能超過 $\mu \cdot f_n$ ，否則會打滑（第 21.2.2 節）。
- **合力要求**：所有接觸力的合成，必須等於支撐機器人運動所需的力與力矩。

當有多隻腳著地時，這個問題是**冗餘的**（無限多組解），因此可以最佳化——例如讓各腳的受力盡量平均、或讓接觸力盡量遠離摩擦錐邊界（增加抗滑裕度）。

◎ 第 21 章的抓取與第 28 章的行走是同一個問題

這是一個值得停下來欣賞的觀察：**四足機器人站在地上，與機械手抓住一個物體，在數學上是同一個問題**。四隻腳就是四根手指、地面就是被「抓住」的物體、摩擦錐與力封閉的分析完全相同。差別只在於：抓取時是手指抓物體、行走時是「地面抓住機器人」（或者說，機器人用腳抓住地球）。因此第 21 章的力封閉、摩擦錐、正線性跨度等所有工具，都可以直接應用到腿式機器人的接觸力分析上。這種跨領域的結構相似性，正是理論的力量所在。

28.8 狀態估測的特殊困難

28.8.1 沒有絕對參照

腿式機器人的狀態估測，比輪型機器人困難得多。核心問題是：**它沒有可靠的里程計**。

輪型機器人可以數輪子的轉動（第 23.7 節），雖然會漂移但短期精準。腿式機器人呢？

- **腳會打滑**：碎石、濕滑地面上的滑動量無法預測。
- **腳會變形**：橡膠腳墊受力會壓縮數公釐。
- **接觸時機不確定**：什麼時候真正著地？觸地感測器有延遲與雜訊。
- **騰空時完全無資訊**：跑步時雙腳離地，此時只能靠 IMU 積分。

28.8.2 腿式里程計

儘管困難，仍可以建立一個**腿式里程計**（leg odometry）。其原理是：

若腳 A 著地且不滑動，則「腳 A 在世界座標系中的位置」保持不變

接觸點作為暫時的固定參照。由關節編碼器可算出「軀幹相對於腳」的位置（正向運動學），因而反推出軀幹

的移動。

這個方法的三個要點：

- **必須偵測接觸**：用力感測器、接觸開關、或由關節力矩推算（第 11.6.3 節）。誤判接觸是最主要的誤差來源。
- **必須偵測打滑**：若多隻腳同時著地，可以交叉比對——若各腳算出的軀幹速度不一致，代表有腳在滑。
- **必須融合 IMU**：腿式里程計的更新率受步頻限制（數 Hz），必須用 IMU 填補中間的高頻資訊。

28.8.3 標準的估測架構

現代腿式機器人的狀態估測，典型採用三層融合：

表 28-6 腿式機器人的狀態估測

來源	提供的資訊	更新率	特性
IMU	姿態、角速度、加速度	500~1000 Hz	高頻、會漂移
關節編碼器 + 接觸偵測	軀幹相對速度（腿式里程計）	步頻限制	會因打滑而錯誤
視覺 / 光達 SLAM	絕對位姿	10~30 Hz	不漂移但慢
外部定位（實驗室）	絕對位姿	100+ Hz	精確但僅限特定場地

融合的邏輯與第 24.10.1 節相同：IMU 提供高頻預測、腿式里程計提供速度修正、SLAM 提供不漂移的絕對參照。三者以 EKF 或因子圖融合。

一個實務上的關鍵細節：腿式機器人的高度估計特別困難。因為 IMU 的垂直加速度必須扣除重力，而任何姿態估計的微小誤差都會造成巨大的垂直積分誤差。因此高度通常主要依賴「腳的接觸點高度 + 腿的正向運動學」，而非 IMU 積分。

28.9 強化學習的突破

28.9.1 一個典範轉移

2019 年前後，腿式機器人領域發生了一次典範轉移。ETH Zurich 的 ANYmal 團隊展示了一件事：用強化學習訓練出的控制器，在崎嶇地形上的表現超越了數十年來精心設計的模式方法。

這個結果之所以震撼，是因為模型式方法（28.4 ~ 28.7 節）建立在紮實的物理與最佳化理論之上，而強化學習只是「在模擬中試錯數百萬次」。但後者在**真實崎嶇地形**上的強健性確實更高。

為什麼？三個原因：

- **模型式方法依賴簡化模型**。LIPM 假設質心高度固定、忽略角動量——這些假設在崎嶇地形上會失效。學習式方法則直接從完整的模擬動態中學習，不受簡化的限制。
- **接觸的不確定性難以建模**。地面的軟硬、摩擦係數、是否會滑動——這些在模型中只能用粗略的假設。學習式方法則透過**領域隨機化**（第 14.7.3 節）在訓練中經歷各種可能性，學到對這些變異不敏感的策略。
- **反應式行為難以設計**。踩到石頭滑了一下該怎麼辦？模型式方法需要明確地寫出每種情況的處置；學習則能自動發展出這類「本能反應」。

28.9.2 訓練的流程

典型的腿式運動強化學習流程為：

1. **在模擬中建立環境**：用物理引擎（第 14.8 節）模擬機器人與地形。地形通常是程序化生成的（隨機的台階、斜坡、碎石）。
2. **領域隨機化**：每次訓練隨機改變物理參數——質量（ $\pm 20\%$ ）、摩擦係數（0.3~1.2）、馬達力矩上限、感測延遲、地形起伏。**這是 Sim-to-Real 成功的關鍵**。
3. **設計獎勵函數**：鼓勵前進速度、懲罰能耗、懲罰關節極限、懲罰身體傾倒、鼓勵平順的動作。這是最需要工程直覺的部分。
4. **大規模平行訓練**：在 GPU 上同時模擬數千個環境（如 Isaac Gym），數小時內完成相當於數年的真實經驗。
5. **部署到實機**：訓練好的策略是一個神經網路，推論極快（微秒級），可直接在機器人上執行。

28.9.3 教師—學生架構

有一個極為巧妙的技巧值得特別介紹，稱為**教師—學生**（teacher-student）訓練：

◎ 特權資訊與知識蒸餾

問題：在模擬中，我們知道所有的「特權資訊」——精確的地形高度、摩擦係數、機器人的真實狀

態。但在真實機器人上，這些都無法量測。**解法**：先訓練一個「教師」策略，讓它使用所有特權資訊，學到最佳的行為；再訓練一個「學生」策略，它只能看到真實可得的感測資料（IMU、關節編碼器），但被要求**模仿教師的動作**。這樣學生就學會了「如何從有限的感測中推斷出隱藏的狀態」，並做出教師等級的決策。這個技巧使 ANYmal 能在完全看不見地形的情況下（純本體感覺）走過碎石與樓梯。

28.9.4 模型與學習的分工

經過數年的發展，目前的共識並非「學習取代模型」，而是**兩者各有所長**：

表 28-7 模型式與學習式方法的比較

面向	模型式 (MPC + WBC)	學習式 (RL)
可解釋性	高 (每一步都有物理意義)	低 (黑盒子)
保證	有 (約束明確滿足)	無 (只有統計上的表現)
崎嶇地形的強健性	中	高
調整新行為	需重新設計	需重新訓練 (數小時)
處理未建模效應	差	佳
計算量 (部署時)	中 (需即時最佳化)	低 (一次前向傳播)
開發成本	高 (需專家知識)	中 (需大量計算資源)

混合式架構因而成為趨勢：用學習處理「反應式的低層行為」（如踩空時的即時修正），用模型式方法處理「需要保證的高層規劃」（如避障、任務約束）。這與第 21.9.5 節的抓取、第 25.10.3 節的 SLAM 是同一種思維——**學習做直覺，模型做保證**。

28.10 跌倒與安全

28.10.1 跌倒是不可避免的

與無人機類似（第 27.8 節），腿式機器人也沒有「安全停止」這個選項——但情況又不同：無人機停止就墜落，腿式機器人停止則會傾倒。

因此設計哲學必須是：**承認跌倒會發生，並讓它盡可能無害**。三個層次的處理：

層次一：避免跌倒。持續監測捕捉點（第 28.5.2 節）——若捕捉點仍在可達範圍內，就執行恢復步伐；若已超出，進入層次二。

層次二：受控跌倒。既然無法避免，就**主動選擇怎麼倒**。具體策略包括：

- **選擇倒下的方向：**避開人員、避開陡坡或樓梯。
- **降低質心：**屈膝下蹲，減少落下的高度與衝擊能量。
- **用手臂緩衝：**人形機器人可用手臂先觸地分散衝擊（與人類的本能相同）。
- **保護關鍵部位：**轉動身體使頭部（含感測器）朝上，用背部或側面著地。

層次三：跌倒後的復原。多數現代腿式機器人具備**自主站起**的能力。這是一個獨立的控制問題——需要規劃一連串的姿態轉換，且過程中接觸狀態不斷改變。

28.10.2 與人共處的安全

腿式機器人（尤其是人形）在與人共處時有特殊的安全考量：

- **質量與速度：**一台 80 公斤的人形機器人以 1.5 m/s 移動時，動能達 90 焦耳——足以造成嚴重傷害。這遠超過第 20 章協作機器人的安全標準。
- **跌倒的方向不可控：**不像固定式手臂能限制工作範圍，腿式機器人可能往任何方向倒。
- **踩到人的風險：**腳的接觸壓力極高（小面積承受全身重量）。

因應之道包括：限制運行速度、設定人員接近時的減速區、加裝軟性外殼、以及最重要的一——在有人的環境中，優先採用保守的靜態穩定步態。

28.11 第四篇的總結

28.11.1 本章的成果

本章完成了全書技術的最高整合。你現在能夠：說明腿式機器人的優勢與代價、理解混合動態的困難、區分靜態與動態穩定並運用 ZMP 判準、推導並運用線性倒立擺模型、以捕捉點決定落腳點、比較各種步態的特性、理解三層控制架構與全身控制的最佳化形式、處理腿式狀態估測的特殊困難、並理解強化學習帶來的突破。

最重要的是那個貫穿全章的方法論：**用簡化模型處理複雜系統**。三十個自由度的人形機器人，在規劃層面簡化為一根三維倒立擺——這個抽象使原本不可解的問題變得可解。而底層仍用完整模型執行，兩者透過分層架構銜接。

28.11.2 第四篇的整合回顧

第 22 ~ 28 章共同構成了一個完整的自主移動機器人系統：

看見 (22) → 移動 (23) → 估測 (24) → 建圖定位 (25) → 規劃導航 (26) → 飛行
(27) → 行走 (28)

第四篇的邏輯鏈。

這七章與第三篇形成了鮮明的對比。**第三篇的世界是確定的**——給定關節角度，末端位置唯一確定；**第四篇的世界充滿不確定性**——每一個感測讀值都帶著雜訊、每一次辨識都可能出錯、每一步都可能打滑。

因應這個轉變，第四篇反覆出現三個原則：

- 量化不確定性而非忽略它 (第 24 章的協方差、第 28 章的捕捉點裕度) 。
- 依可信度融合而非二選一 (第 24 章的 Kalman 增益、第 28 章的多來源估測) 。
- 用簡化模型處理複雜系統 (第 26 章的位形空間、第 27 章的小角度近似、第 28 章的 LIPM) 。

28.11.3 接下來：從「會動」到「懂得做什麼」

經過二十八章，機器人已經具備了完整的**物理能力**：它能感知環境、能精確地運動、能與環境安全地互動、能在未知空間中導航、能飛行也能行走。

但它仍然缺少一件關鍵的東西——**理解與決策**。目前為止的所有能力，都建立在「人類明確地告訴它該做什麼」之上：軌跡是人規劃的、任務是人指定的、行為是人設計的。

第五篇「智慧機器人與具身人工智慧」(第 29 ~ 33 章)將處理這個轉變。我們會探討：機器人如何理解自然語言的指令、如何從示範中學習新技能、如何用強化學習自主發展行為、如何用大型模型進行任務層級的推理、以及「具身智慧」這個近年最重要的概念——**智慧不只存在於運算中，也存在於身體與環境的互動裡**。

屆時你會發現，前四篇建立的所有能力，都將成為第五篇的**執行基礎**。一個大型語言模型可以理解「幫我把桌上的杯子拿到廚房」，但要真正做到，需要第 22 章的視覺辨識、第 21 章的抓取、第 26 章的導航、以及第 17 章的控制——**智慧與身體，缺一不可**。

換句話說：前四篇讓機器人**做得到**，第五篇讓它**知道該做什麼**。

完整計算例題

例題 28-1 線性倒立擺與捕捉點

題目：某人形機器人質心高度 $z_{\text{com}} = 0.85 \text{ m}$ 。(a) 求 LIPM 的自然頻率 ω 與時間常數。(b) 若當前質心位置 $x = 0.05 \text{ m}$ (相對於支撐腳)、速度 $\dot{x} = 0.4 \text{ m/s}$ ，求捕捉點位置。(c) 若腳掌長 0.25 m (前緣在 $+0.15 \text{ m}$ 、後緣在 -0.10 m)，判斷是否能靠當前這隻腳穩住。(d) 若機器人的最大跨步距離為 0.6 m ，判斷能否一步捕捉。

解 (a)：由第 28.4.1 節：

$$\omega = \sqrt{g/z_{\text{com}}} = \sqrt{9.8/0.85} = \sqrt{11.53} = 3.396 \text{ rad/s}$$

自然頻率。

$$\text{時間常數 } \tau = 1/\omega = 1/3.396 = 0.294 \text{ 秒}$$

這是質心發散的特徵時間——約 0.3 秒內誤差會放大 e 倍 (2.72 倍)。這解釋了為何腿式機器人的反應必須極快。

解 (b)：由第 28.5.1 節的捕捉點公式：

$$x_{\text{cp}} = x + \dot{x}/\omega = 0.05 + 0.4/3.396 = 0.05 + 0.1178 = 0.168 \text{ m}$$

捕捉點在質心前方約 12 公分處。

解 (c)：腳掌的可用範圍是 -0.10 m 到 $+0.15 \text{ m}$ 。捕捉點在 0.168 m ：

$$0.168 \text{ m} > 0.15 \text{ m} \text{ (腳掌前緣)}$$

捕捉點超出腳掌範圍，無法靠當前這隻腳穩住。

物理意義：即使把 ZMP 移到腳尖 (最大的可能)，仍不足以消除發散——機器人必定會繼續往前倒。此時必須跨步。

這正是人類的經驗：當你被推得很急時，光靠腳踝的調整 (把重心壓向腳尖) 是不夠的，必須跨出一步。

解 (d)：需要跨到 0.168 m 處，而最大跨步為 0.6 m ：

$$0.168 \text{ m} < 0.6 \text{ m} \Rightarrow \text{可以一步捕捉 } \checkmark$$

只需跨出約 17 公分即可穩住。

進一步分析：若速度更快，例如 $\dot{x} = 2.5 \text{ m/s}$ ：

$$x_{cp} = 0.05 + 2.5/3.396 = 0.05 + 0.736 = 0.786 \text{ m} > 0.6 \text{ m}$$

超出單步能力，需要兩步以上才能停下（ N 步捕捉，第 28.5.3 節）。

工程意義：這個計算給出了一個明確的**速度上限**——超過某個速度後，機器人就無法在一步內停止。這是步態規劃時必須考慮的安全約束。

例題 28-2 ZMP 與動態穩定

題目：某雙足機器人質量 $m = 60 \text{ kg}$ 、質心高度 0.9 m 。腳掌支撐區域由 -0.08 m 到 $+0.14 \text{ m}$ （以踝關節為原點）。（a）若質心位置在 $+0.05 \text{ m}$ 且靜止，判斷是否靜態穩定。（b）若質心以 2.0 m/s^2 向前加速，求 ZMP 位置並判斷穩定性。（c）求不使 ZMP 超出前緣的最大向前加速度。（d）由（c）的結果，估算機器人能達到的最大水平推力。

解（a）：靜止時 ZMP 與質心投影重合。

$$x_{zmp} = x_{com} = 0.05 \text{ m}$$

在 $-0.08 \sim +0.14$ 的範圍內。

判定：靜態穩定 ✓。機器人可以無限期地保持這個姿勢。

解（b）：由第 28.3.2 節的公式：

$$x_{zmp} = x_{com} - (z_{com}/g) \cdot \ddot{x}_{com} = 0.05 - (0.9/9.8) \times 2.0 = 0.05 - 0.1837 = -0.134 \text{ m}$$

ZMP 位置。

判定：不穩定 ✗—— -0.134 m 超出了後緣 -0.08 m 。

物理意義：向前加速時 ZMP 往後移（注意公式中的負號）。若加速太猛，ZMP 會退出腳掌後緣，機器人會向後翻倒——這正是快速起步時「往後仰」的感覺。人類的因應之道是前傾身體（把 x_{com} 往前移），使 ZMP 回到腳掌內。

解（c）：要求 $x_{zmp} \geq -0.08 \text{ m}$ ：

$$0.05 - (0.9/9.8) \cdot \ddot{x} \geq -0.08 \Rightarrow 0.0918 \cdot \ddot{x} \leq 0.13 \Rightarrow \ddot{x} \leq 1.416 \text{ m/s}^2$$

最大向前加速度約 1.42 m/s^2 （約 $0.14g$ ）。

若把質心前移到 0.10 m (前傾) :

$$\ddot{x}_{\max} = (0.10 + 0.08)/0.0918 = 1.961 \text{ m/s}^2$$

加速能力提升了 38%。這定量地說明了為何起跑時要前傾。

解 (d) : 由牛頓第二定律 :

$$F_{\max} = m \cdot \ddot{x}_{\max} = 60 \times 1.416 = 85 \text{ N}$$

最大水平推力約 85 牛頓 (前傾後可達 118 N) 。

工程判讀：這個數字看似不大，但它揭示了一個重要的限制——**腿式機器人的水平出力受限於腳掌尺寸，而非馬達功率**。要提高推力，可以：加大腳掌 (但影響行走靈活度)、前傾身體 (本例的作法)、或**跨步支撐** (把後腳往後蹬，形成更大的支撐多邊形)——這正是人類推重物時的姿勢。

例題 28-3 步態參數與速度

題目：某四足機器人腿長 0.35 m，採用小跑 (trot) 步態。(a) 若步頻為 2.5 Hz、步長 0.25 m，求行進速度。(b) 求該步態的佔空比與單腳的支撐時間。(c) 估算此速度下的 Froude 數並判斷步態是否合理。(d) 若要提升到 2.0 m/s，討論可行的策略。

解 (a) : 行進速度為步長乘以步頻。注意小跑步態每個週期兩腳交替，故一個週期前進一個步長：

$$v = \text{步長} \times \text{步頻} = 0.25 \times 2.5 = 0.625 \text{ m/s}$$

行進速度約 0.63 m/s。

解 (b) : 小跑的佔空比為 0.5 (第 28.6.2 節) ——每隻腳有一半的時間著地。

$$\text{週期 } T = 1/2.5 = 0.4 \text{ 秒}$$

$$\text{單腳支撐時間} = 0.5 \times 0.4 = 0.2 \text{ 秒}$$

$$\text{單腳擺動時間} = 0.2 \text{ 秒}$$

時序分配。

工程意義：0.2 秒的擺動時間，決定了腿部馬達必須有多快——腿要在 0.2 秒內抬起、前擺 0.25 公尺、再放下。這是選擇馬達與減速比的直接依據 (第 5 章) 。

解 (c) : **Froude 數**是步態研究中的重要無量綱參數，定義為：

$$Fr = v^2/(g \cdot L)$$

L 為特徵長度（通常取腿長或髖關節高度）。它衡量「慣性效應相對於重力效應」的大小。

$$Fr = 0.625^2/(9.8 \times 0.35) = 0.3906/3.43 = 0.114$$

Froude 數約 0.11。

生物學的經驗值：

- $Fr < 0.5$ ：行走（walk）較有效率——包含四足的爬行與小跑。
- $Fr \approx 0.5$ ：步態轉換的臨界點——動物在此速度附近會由 walk 換成 trot 或 run。
- $Fr > 1.0$ ：必須使用含騰空期的步態（gallop、run）。

本例 $Fr = 0.11$ 遠低於 0.5，小跑步態完全合理（甚至可以說偏保守，還有加速的餘裕）。

解（d）：要達到 2.0 m/s：

$$Fr = 2.0^2/(9.8 \times 0.35) = 4.0/3.43 = 1.166 > 1.0$$

超過 1.0，小跑步態已不適用，必須改用含騰空期的步態（bound 或 gallop）。

三種可行的策略：

- 提高步頻： $v = 0.25 \times f = 2.0 \Rightarrow f = 8 \text{ Hz}$ 。但擺動時間縮短為 0.0625 秒——這對馬達的要求極高（腿要在 62 毫秒內完成前擺），且慣性力大幅增加。
- 加大步長： $v = L \times 2.5 = 2.0 \Rightarrow L = 0.8 \text{ m}$ 。但步長受限於腿長（0.35 m），最多約 0.5 m（腿張開的極限）——不足以達成目標。
- 改變步態：改用 bound 或 gallop，利用騰空期延長每一步的前進距離。這是唯一可行的方案，也是自然界動物的選擇。

結論：從 0.63 m/s 提升到 2.0 m/s，不是單純調參數就能達成的——必須改變步態的本質。這與汽車換檔的道理相同，也印證了第 28.6.2 節「應依速度切換步態」的建議。

程式範例：LIPM、捕捉點與步態模擬

以下 Python 程式實作線性倒立擺模型、捕捉點計算、ZMP 分析與步態時序，並驗證本章三個例題。

```
import numpy as np
```

```
# ===== 線性倒立擺模型 =====
class LIPM:
    """線性倒立擺 ( 28.4 節 ) """
    def __init__(s, z_com=0.85, g=9.8):
        s.z, s.g = z_com, g
        s.w = np.sqrt(g/z_com)          # 自然頻率  $\omega$ 

    def capture_point(s, x, xd):
        """捕捉點 ( 28.5.1 節 ) """
        return x + xd/s.w

    def dcm(s, x, xd):
        """DCM 與捕捉點同義 ( 28.5.4 節 ) """
        return x + xd/s.w

    def step(s, x, xd, x_zmp, dt):
        """LIPM 動態積分： $\ddot{x} = \omega^2(x - x_{zmp})$ """
        xdd = s.w**2 * (x - x_zmp)
        xd_new = xd + xdd*dt
        x_new = x + xd_new*dt
        return x_new, xd_new

    def zmp_from_com(s, x, xdd):
        """由質心加速度反推 ZMP ( 28.3.2 節 ) """
        return x - (s.z/s.g)*xdd

    def max_accel(s, x_com, zmp_limit):
        """在 ZMP 限制下的最大加速度"""
        return (x_com - zmp_limit)*s.g/s.z

# ===== 例題 28-1 =====
print("=== 例題 28-1：LIPM 與捕捉點 ===")
lip = LIPM(z_com=0.85)
print(f"(a)  $\omega = \sqrt{g/z} = \{{lip.w:.3f}\}$  rad/s")
print(f"    時間常數  $\tau = 1/\omega = \{{1/lip.w:.3f}\}$  秒")
print(f"    → 約 0.3 秒內誤差放大 e 倍，反應必須極快")

x, xd = 0.05, 0.4
cp = lip.capture_point(x, xd)
print(f"\n(b) x={x} m,  $\dot{x}=\{{xd}\}$  m/s → 捕捉點 = {cp:.4f} m")

foot_back, foot_front = -0.10, 0.15
ok_ankle = foot_back <= cp <= foot_front
print(f"\n(c) 腳掌範圍 [{foot_back}, {foot_front}] m")
print(f"    捕捉點 {cp:.3f} m {'在' if ok_ankle else '超出'}腳掌範圍"
      f" → {'可靠腳踝穩住' if ok_ankle else '**必須跨步**'}")

max_step = 0.6
```

```

print(f"\n(d) 最大跨步 {max_step} m:")
for v in (0.4, 1.0, 2.0, 2.5, 3.0):
    cp_v = lip.capture_point(0.05, v)
    n_step = "1 步" if cp_v <= max_step else "***需 2 步以上**"
    print(f"     $\dot{x}=\{v:.1f\}$  m/s  $\rightarrow$  捕捉點  $\{cp_v:.3f\}$  m {n_step}")

# ===== 例題 28-2 : ZMP 分析 =====
print("\n=== 例題 28-2 : ZMP 與動態穩定 ===")
lip2 = LIPM(z_com=0.9)
m_body = 60.0
back, front = -0.08, 0.14
x_com = 0.05

print(f"(a) 靜止時 ZMP = x_com = {x_com} m  $\rightarrow$  ")
    f"'穩定  $\checkmark$ ' if back <= x_com <= front else '不穩定 X'")

acc = 2.0
zmp = lip2.zmp_from_com(x_com, acc)
print(f"(b) 加速 {acc} m/s2  $\rightarrow$  ZMP = {zmp:.4f} m  $\rightarrow$  ")
    f"'穩定  $\checkmark$ ' if back <= zmp <= front else '***不穩定 X ( 向後翻倒 ) **'")

print(f"\n(c) 最大向前加速度 ( ZMP 不超出後緣 {back} m ) :")
for xc in (0.05, 0.08, 0.10, 0.12):
    a_max = lip2.max_accel(xc, back)
    F_max = m_body*a_max
    print(f"    質心在 {xc:.2f} m  $\rightarrow$  a_max = {a_max:.3f} m/s2"
          f"    ({a_max/9.8:.3f} g), 推力 = {F_max:.1f} N")
print("     $\rightarrow$  前傾 ( 質心前移 ) 能顯著提升加速能力")

# ===== ZMP 隨加速度的變化 =====
print("\n=== ZMP 隨加速度的變化 ( 質心固定在 0.05 m ) ===")
print(f"'加速度':>10}'{ZMP':>10}'{狀態':>12}")
for a in (-1.0, 0.0, 0.5, 1.0, 1.4, 2.0):
    z = lip2.zmp_from_com(0.05, a)
    st = "穩定" if back <= z <= front else ("超出後緣" if z < back
                                           else "超出前緣")
    print(f"{a:9.1f}2{z:10.4f}{st:>12}")
print("> 向前加速使 ZMP 後移 ; 減速 ( 負加速度 ) 則使 ZMP 前移")

# ===== 例題 28-3 : 步態參數 =====
print("\n=== 例題 28-3 : 步態參數與 Froude 數 ===")
leg_len = 0.35
def froude(v, L=leg_len, g=9.8):
    return v**2/(g*L)

stride, freq = 0.25, 2.5
v = stride*freq

```

```

print(f"(a) 速度 = {stride} × {freq} = {v:.3f} m/s")
T = 1/freq
print(f"(b) 週期 = {T:.3f} s, 佔空比 0.5")
print(f"    支撐時間 = {T*0.5:.3f} s, 擺動時間 = {T*0.5:.3f} s")
print(f"(c) Froude 數 = {froude(v):.4f}")

print(f"\n{'速度':>9}{ 'Froude':>10}{ '建議步態':>16}")
for vv in (0.3, 0.625, 1.0, 1.5, 2.0, 3.0):
    fr = froude(vv)
    if fr < 0.3:    gait = "爬行/慢走"
    elif fr < 0.5: gait = "小跑 trot"
    elif fr < 1.0: gait = "快跑 bound"
    else:          gait = "**飛奔 gallop**"
    print(f"{vv:8.2f}m/s{fr:10.3f}{gait:>16}")
print("→ Fr ≈ 0.5 是步態轉換點 · Fr > 1.0 必須含騰空期")

print(f"\n(d) 要達 2.0 m/s 的三種策略:")
print(f"    提高步頻: f = 2.0/{stride} = {2.0/stride:.1f} Hz "
      f"(擺動時間僅 {0.5/(2.0/stride)*1000:.0f} ms – 馬達要求極高)")
print(f"    加大步長: L = 2.0/{freq} = {2.0/freq:.2f} m "
      f"(超過腿長 {leg_len} m – **不可行**)")
print(f"    改變步態: Fr = {froude(2.0):.2f} > 1.0 – **唯一可行**")

# ===== 捕捉點控制的行走模擬 =====
print("\n=== 以捕捉點決定落腳點的行走模擬 ===")
def walk_sim(v_target=0.5, n_steps=8, T_step=0.4, z=0.85):
    """每步結束時，依捕捉點決定下一個落腳點 ( 28.5.2 節 ) """
    lp = LIPM(z_com=z)
    x, xd = 0.0, 0.0 # 相對於支撐腳
    x_foot = 0.0 # 支撐腳的絕對位置
    dt = 0.001
    print(f"{'步':>3}{ '落腳點':>10}{ '步末 x':>10}{ '步末 ẋ':>10}"
          f"{'捕捉點':>10}")
    for k in range(n_steps):
        # 依捕捉點加上目標速度的偏移，決定下一落腳點
        cp = lp.capture_point(x, xd)
        offset = v_target/lp.w * np.tanh(1.0) # 踩在捕捉點後方以維持速度
        x_next_rel = cp - offset
        # 在這一步內以當前腳為 ZMP 積分
        for _ in range(int(T_step/dt)):
            x, xd = lp.step(x, xd, 0.0, dt)
        cp_end = lp.capture_point(x, xd)
        print(f"{k:3d}{x_foot + x_next_rel:10.4f}{x:10.4f}"
              f"{xd:10.4f}{cp_end:10.4f}")
        # 換腳：座標原點移到新的支撐腳
        x_foot += x_next_rel
        x -= x_next_rel
    return x_foot

```



```

final = walk_sim()
print(f"共前進 {final:.3f} m")
print("> 每步的落腳點都由捕捉點決定，這是腿式行走的核心控制律")

# ===== 步態時序表 =====
print("\n=== 四足步態的相位時序 (0=著地起點, 相位 $\in$ [0,1)) ===")
gaits = {
    "爬行 Crawl": ([0.0, 0.5, 0.25, 0.75], 0.75),
    "小跑 Trot": ([0.0, 0.5, 0.5, 0.0], 0.5),
    "溜蹄 Pace": ([0.0, 0.5, 0.0, 0.5], 0.5),
    "跳躍 Bound": ([0.0, 0.0, 0.5, 0.5], 0.45),
}
print(f"{'步態':<14}{'LF':>7}{'RF':>7}{'LH':>7}{'RH':>7}"
      f"{'佔空比':>9}{'同時著地':>10}")
for name, (ph, duty) in gaits.items():
    # 計算相位 0 時有幾隻腳著地
    n_contact = sum(1 for p in ph if (0.0 - p) % 1.0 < duty)
    print(f"{'name':<14}{'ph[0]':7.2f}{'ph[1]':7.2f}{'ph[2]':7.2f}"
          f"{'ph[3]':7.2f}{'duty':9.2f}{'n_contact':9d} 腳")
print("> 只有爬行能保證 3 腳著地 (靜態穩定)")

```

請試著修改：(一) 把質心高度由 0.85 改為 1.2 公尺 (較高的機器人)，觀察 ω 變小、時間常數變長——這代表反應可以慢一些，但捕捉點會離得更遠。(二) 在 ZMP 分析中把腳掌長度加倍，觀察最大加速度提升多少。(三) 在行走模擬中改變 v_{target} ，觀察落腳點如何自動調整。

實作：腿式機器人的模擬與實驗

本章實作以模擬為主 (實體腿式機器人成本高且危險)。若有四足平台 (如市售的教育型四足機器人)，後段項目可實機驗證。

- LIPM 模擬**：實作線性倒立擺模型，驗證其發散特性——由一個小的初始偏移出發，觀察質心如何指數地遠離。量測實際的發散時間常數並與 $1/\omega$ 比較。
- 捕捉點驗證**：在 LIPM 模擬中，把 ZMP 設在捕捉點上，驗證質心確實會漸近地停止。再故意設在捕捉點前後各 5 公分處，觀察質心會往回彈或繼續前衝。
- ZMP 與腳掌限制**：實作 ZMP 計算，掃描不同的質心位置與加速度組合，繪出「可行區域」的圖。驗證例題 28-2 的結論。

4. **步態產生器**：實作四足的步態時序產生器，支援爬行、小跑、跳躍三種步態。以動畫顯示各腿的抬放時序與支撐多邊形的變化。
5. **靜態穩定判定**：對每個時刻計算支撐多邊形與重心投影，判定是否靜態穩定。驗證只有爬行步態能全程保持靜態穩定。
6. **簡化的行走控制**：整合 LIPM、捕捉點與步態產生器，實作一個能在平地上行走的簡化控制器。測試不同的目標速度。
7. **擾動恢復測試**：在行走過程中注入一個突然的速度擾動（模擬被推），觀察捕捉點如何改變、以及控制器如何調整落腳點恢復平衡。找出「能恢復」與「必定跌倒」的臨界擾動大小。
8. **（若有實機）步態參數調校**：在實體四足機器人上測試不同的步長、步頻與佔空比，記錄行進速度、穩定性與耗電量。繪出「速度 vs 能耗」曲線，找出最省能的參數組合。
9. **（若有實機）地形適應測試**：在平地、斜坡、階梯與碎石上分別測試，記錄各步態的成功率。觀察機器人在何種地形上開始失效。
10. **（進階）ROS 2 + Gazebo 模擬**：使用開源的四足機器人模型（如 Champ 或 Unitree 的開源專案），在 Gazebo 中建立完整的模擬環境，實作並測試自己的步態控制器。

表 28-8 腿式機器人實驗記錄表（前兩列為範例）

測試項目	設定	結果	觀察	改善方向
LIPM 發散驗證	$z=0.85, x_0=0.01$ m	0.29 s 後放大 2.7 倍	與 $1/\omega$ 相符	—
擾動恢復	小跑, 推力 20 N	一步恢復	捕捉點在跨步範圍內	測試更大擾動
（自行填寫其餘測試）				

國中小也看得懂：走路就是不斷地往前倒

先做一個實驗：站起來，然後往前走三步。走的時候仔細感覺一下——當你抬起一隻腳的那一瞬間，你的身體是不是正在往前倒？

是的！走路其實就是「不斷地往前倒，然後用下一隻腳接住自己」。你不是先站穩再移動，而是先讓自己失去平衡，再趕快跨一步接住。是不是很神奇？

再試一個：請朋友從背後輕輕推你一下。你會發現自己**自動跨出一步**——而且推得越用力，你跨得越遠。你的身體會自己算出「該跨多遠才接得住」。

科學家幫這個位置取了一個名字，叫做「**捕捉點**」。它有一個公式：**你倒得越快，捕捉點就離得越遠**。機器人也是用這個公式決定要把腳踩在哪裡的！

那為什麼機器人走路這麼難呢？有兩個原因：

第一，機器人要一直算。你走路不用想，但機器人每一秒要計算好幾百次「我現在倒到哪裡了？下一步該踩哪裡？」只要算錯一次就會摔倒。

第二，腳踩地和抬起來完全不一樣。腳踩在地上時，機器人是被地面「撐著」的；腳抬起來的瞬間，那個支撐就消失了。所以機器人每走一步，計算的方式就要換一次——這比一直用同一個方式難很多。

最後告訴你一件很酷的事：科學家發現，**一台有三十個關節的機器人，在規劃走路的時候可以簡化成「一根倒立的棍子」**！就像你只要知道自己往哪邊倒、倒得多快，就知道該跨多遠一樣。把複雜的東西變簡單，這就是科學家最厲害的本事。

叫獸提醒與常見錯誤

- 誤區一：以為腿式機器人只是「多幾個自由度的手臂」。混合動態（接觸切換）是本質上不同的困難，第 13~19 章的工具無法直接套用。
- 誤區二：用重心投影判斷動態穩定。有加速度時必須用 ZMP，兩者會分離（第 28.3.2 節）。
- 誤區三：以為 ZMP 在支撐面內就一定不會跌倒。它只保證腳不翻轉，是必要而非充分條件。
- 誤區四：在騰空期使用 ZMP 判準。此時沒有支撐多邊形，該判準完全失效。
- 誤區五：忽略 LIPM 的假設限制。質心高度大幅變化（跑步、跳躍）時應改用 SLIP 模型。
- 誤區六：把捕捉點當成「重心該去的地方」。它是**腳該踩的地方**——兩者意義完全不同。
- 誤區七：只用一種步態。應依速度切換（Froude 數約 0.5 與 1.0 是轉換點），這也是自然界動物的作法。
- 誤區八：忽略腳部打滑對狀態估測的破壞。腿式里程計的誤差主要來自誤判接觸與未偵測的

打滑。

- 誤區九：接觸力分配時忘記單向性與摩擦錐。腳只能推不能拉，且切向力受 $\mu \cdot f_n$ 限制——這與第 21 章的抓取完全相同。
- 誤區十：以為強化學習能完全取代模型式方法。學習擅長反應式的低層行為，模型式方法提供保證與可解釋性，兩者互補。
- 誤區十一：把「避免跌倒」當成唯一目標。跌倒不可避免，應設計受控跌倒與自主站起的能力。
- 誤區十二：低估腿式機器人與人共處的風險。其質量與速度遠超協作機器人的安全標準，且跌倒方向不可控。

課堂討論

1. 28.1.3 節的被動行走顯示「穩定性可以由機構本身提供」。請討論這與第 21 章的欠驅動手、第 20 章的 RCC 有何共通之處，並歸納出一個設計原則。
2. 28.7.4 節指出「四足站立與機械手抓取在數學上是同一個問題」。請進一步討論：第 21 章的哪些工具可以直接應用到腿式機器人？哪些需要修改？
3. 強化學習在崎嶇地形上超越了模型式方法，但缺乏保證。請討論：在什麼應用中這個取舍是可接受的？在什麼應用中不可接受？並設計一個混合式架構。
4. 例題 28-2 顯示腿式機器人的水平出力受限於腳掌尺寸而非馬達功率。請討論這對機構設計的三個啟示，並比較人類與四足動物的解法差異。
5. 本書反覆出現「用簡化模型處理複雜系統」的思維（第 26 章的位形空間、第 27 章的小角度近似、第 28 章的 LIPM）。請歸納這個方法論的共同結構，並討論它可能失效的情況。

章末摘要

- 腿式機器人的優勢在於能選擇落腳點、跨越障礙、適應崎嶇地形；代價是效率低、自由度高、控制複雜且本質不穩定。
- 被動動態行走證明穩定性可由機構本身提供，這與「用機構取代控制」的思維一脈相承。
- 混合動態是腿式機器人的根本困難：接觸狀態切換使方程式改變，且落地時有速度的離散跳變。

- 分析週期步態可用龐加萊映射——只看每步結束的狀態，把連續混合系統化為離散映射。
- 靜態穩定要求重心投影在支撐多邊形內，任何時刻停下都不會倒；代價是速度慢、效率差。
- ZMP 是地面反力合力矩為零的點；動態穩定要求 ZMP 在支撐多邊形內，這保證腳不會翻轉。
- ZMP 與重心的關係 $x_{zmp} = x_{com} - (z/g) \cdot \ddot{x}_{com}$ ：向前加速時 ZMP 後移。
- 線性倒立擺模型 (LIPM) 以三個假設把 30 自由度簡化為一個線性方程 $\ddot{x} = \omega^2(x - x_{zmp})$ ， $\omega = \sqrt{g/z}$ 。
- LIPM 的解含一個發散模態與一個收斂模態；控制的本質是透過選擇落腳點消除發散分量。
- 捕捉點 $x_{cp} = x + \dot{x}/\omega$ 是「踩下去就能停止」的位置；倒得越快就要跨得越遠。
- 捕捉點的三個用途：判斷能否站住、決定落腳點、量化穩定裕度；踩在其前方減速、後方加速。
- DCM 把二階系統分解為一個發散與一個收斂的一階系統，控制 DCM 即可使質心自動穩定。
- 四足步態中只有爬行是靜態穩定的；小跑因對稱性與速度平衡而最常用；速度提高時應切換步態。
- 行走與跑步的差別在於有無騰空期；跑步需改用含彈簧的 SLIP 模型，串聯彈性驅動器可回收能量。
- 控制採三層架構：步態規劃（簡化模型、低頻）、MPC（中等模型、中頻）、全身控制（完整模型、高頻）。
- 全身控制是一個二次規劃問題，需同時滿足動力學、接觸單向性、摩擦錐與力矩極限。
- 四足站立與多指抓取在數學上是同一個問題——摩擦錐與力封閉的分析完全相通。
- 腿式狀態估測缺乏可靠里程計；腿式里程計以「著地腳不動」為參照，需偵測接觸與打滑並融合 IMU。
- 強化學習在崎嶇地形上表現優於模型式方法，關鍵在領域隨機化與教師—學生訓練；兩者應互補而非取代。
- 跌倒不可避免，設計應包含受控跌倒（選方向、降質心、用手緩衝）與自主站起的能力。

觀念題與計算題

觀念題

1. 說明腿式機器人相對於輪型的三個優勢與三個代價。
2. 說明被動動態行走的原理，以及它對機器人設計的啟示。
3. 什麼是混合動態？它為腿式機器人帶來哪三個層次的困難？
4. 說明龐加萊映射如何用於分析週期步態的穩定性。
5. 區分靜態穩定與動態穩定，並說明各自的判準與代價。
6. 定義 ZMP，並說明它與重心投影在有加速度時為何會分離。
7. 寫出 ZMP 與質心加速度的關係，並解釋「向前加速時 ZMP 後移」的物理意義。
8. 列舉 ZMP 判準的三個限制。
9. 說明 LIPM 的三個假設，並解釋為何這些簡化在實務上仍然有效。
10. 由 LIPM 的解說明「控制的本質是消除發散模態」。
11. 定義捕捉點並說明其物理意義；列舉其三個用途。
12. 說明 DCM 的定義與其「分而治之」的巧妙之處。
13. 比較四足的五種步態，並說明為何只有爬行是靜態穩定的。
14. 說明行走與跑步的定義差異，以及為何跑步需改用 SLIP 模型。
15. 說明腿式機器人三層控制架構的分工與各層使用的模型。
16. 說明全身控制的最佳化形式，以及接觸力分配的三個約束。
17. 說明「四足站立與多指抓取是同一個問題」的意義。
18. 說明腿式里程計的原理與其三個誤差來源。
19. 說明強化學習在腿式運動上突破的三個原因，以及教師—學生架構的巧妙之處。

計算題

1. 某人形機器人質心高度 1.0 m。(a) 求 ω 與時間常數。(b) 若 $x = 0.03$ m、 $\dot{x} = 0.6$ m/s，求捕捉點。(c) 若腳掌範圍為 $-0.09 \sim +0.16$ m，判斷能否靠腳踝穩住。(d) 若最大跨步 0.7 m，求能一步捕捉的最大速度。
2. 某雙足機器人 $m = 75$ kg、質心高度 0.95 m、腳掌範圍 $-0.07 \sim +0.15$ m。(a) 質心在 $+0.06$ m 且以 1.5 m/s² 加速時，求 ZMP 並判斷穩定性。(b) 求在此質心位置下的最大向

前加速度。(c) 求對應的最大水平推力。(d) 若質心前移到 $+0.12\text{ m}$ ，重新計算 (b) (c) 並計算改善百分比。

3. 某四足機器人腿長 0.4 m ，採小跑步態、步長 0.3 m 、步頻 3.0 Hz 。(a) 求速度與 Froude 數。(b) 求週期、支撐時間與擺動時間。(c) 判斷此步態是否合理。(d) 求該機器人使用小跑步態的理論最高速度 (取 $Fr = 1.0$ 為上限)。
4. 某 LIPM 系統 $z_{\text{com}} = 0.8\text{ m}$ 。初始 $x = 0.02\text{ m}$ 、 $\dot{x} = 0$ 。(a) 求 ω 。(b) 若 ZMP 固定在 0，求 0.3 秒後的質心位置 (提示：用雙曲函數解)。(c) 計算此時的捕捉點。(d) 若要在 0.3 秒時停止，ZMP 應設在何處？

動手做與專題延伸

本章實作：腿式運動模擬器

建立一個腿式機器人的模擬與控制環境，這是本書技術的最終整合。

- 階段一：實作 LIPM 與捕捉點計算，驗證發散特性與捕捉點的物理意義。
- 階段二：實作 ZMP 計算與支撐多邊形判定，建立穩定性的視覺化工具。
- 階段三：實作四足步態產生器 (支援多種步態) 與相位时序的動畫顯示。
- 階段四：整合為簡化的行走控制器——以捕捉點決定落腳點、以 LIPM 規劃質心軌跡。
- 階段五：加入擾動並實作恢復策略，測試 N 步捕捉的能力。

專題延伸

- 捕捉點的視覺化動畫：模擬一個 LIPM 系統，即時繪製質心位置、速度與捕捉點的變化。加入互動功能——讓使用者用滑鼠「推」機器人一下，觀察捕捉點如何瞬間跳遠，以及控制器如何選擇落腳點恢復。這是理解本章核心概念最直觀的工具。
- 步態的能耗分析：對同一台四足機器人模型，實作三種步態 (爬行、小跑、跳躍)，在多個速度下模擬並估算能耗 (可用關節力矩乘角速度的積分近似)。繪出「速度 vs 能耗」的曲線族，找出各步態的最佳速度範圍與轉換點。與 28.6.2 節提到的生物學結論比較——你應該會看到與 Froude 數相符的轉換點。
- 進階：ZMP 可行區域的三維視覺化。對一台雙足機器人，以「質心位置」「質心加速度」「腳掌尺寸」為三個維度，掃描所有組合並判定 ZMP 是否在支撐面內。把可行區域繪成三維的體積圖。接著討論：這個圖如何指導機構設計 (腳掌該多大) 與控制設計 (加速度該如

何限制)？並延伸思考——為何人類的腳掌是這個比例？用你的圖來驗證演化的選擇是否接近最佳。

參考文獻與延伸閱讀

- Kajita, S. et al., Introduction to Humanoid Robotics, Springer (人形機器人的標準教材，LIPM 與 ZMP 的完整處理)。
- Wieber, P.-B., Tedrake, R. and Kuindersma, S., "Modeling and Control of Legged Robots," in Springer Handbook of Robotics, 2nd ed. (腿式機器人的權威綜述)。
- Vukobratović, M. and Borovac, B., "Zero-Moment Point — Thirty Five Years of Its Life," Int. J. Humanoid Robotics, 2004 (ZMP 概念的回顧，由其提出者撰寫)。
- Pratt, J. et al., "Capture Point: A Step toward Humanoid Push Recovery," Humanoids 2006 (捕捉點的原始論文)。
- Englsberger, J., Ott, C. and Albu-Schäffer, A., "Three-Dimensional Bipedal Walking Control Based on Divergent Component of Motion," IEEE Trans. Robotics, 2015 (DCM 方法)。
- McGeer, T., "Passive Dynamic Walking," IJRR, 1990 (被動動態行走的原始論文)。
- Raibert, M. H., Legged Robots That Balance, MIT Press, 1986 (腿式機器人的開創性著作，SLIP 模型與跳躍控制)。
- Di Carlo, J. et al., "Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control," IROS 2018 (四足 MPC 的代表性工作)。
- Hwangbo, J. et al., "Learning Agile and Dynamic Motor Skills for Legged Robots," Science Robotics, 2019 (強化學習在 ANYmal 上的突破)。
- Lee, J. et al., "Learning Quadrupedal Locomotion over Challenging Terrain," Science Robotics, 2020 (教師—學生架構與盲走崎嶇地形)。

[@prof](#)

機器人學導論

機構、感知、控制與具身人工智慧

第 五 篇

智慧機器人與具身人工智慧

第 29 章 ROS 2 與機器人軟體框架

第 30 章 模擬與 Sim-to-Real

第 31 章 機器學習與強化學習

第 32 章 大型模型與任務規劃

第 33 章 具身人工智慧

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

2026 年 9 月

第五篇 篇章導讀

前四篇讓機器人**做得到**，這一篇要讓它**知道該做什麼**。

經過二十八章，機器人已經具備了完整的物理能力：它能感知環境、精確地運動、與環境安全地互動、在未知空間中導航、能飛行也能行走。但這些能力全都建立在一個前提上——**人類明確地告訴它該做什麼**。軌跡是人規劃的、任務是人指定的、行為是人設計的。

第五篇的五章，就是要補上這最後一塊拼圖。

第 29 章 (ROS 2) 先解決一個看似平凡卻極為關鍵的問題：**軟體整合**。四組學生各自做出了不錯的成果，卻接不起來——因為語言不同、座標系定義不同、時間基準不同。ROS 2 提供的不是演算法，而是一套讓不同模組能夠對話的約定。它也是後續四章的共同基礎。

第 30 章 (模擬與 Sim-to-Real) 處理一個殘酷的現實：模擬中 98% 的成功率，到實機可能只剩兩成。但本章的答案出乎意料——**不是把模擬做得更真，而是刻意把模擬弄亂**。領域隨機化的思想是：與其縮小模擬與現實的距離，不如擴大模擬的涵蓋範圍，使現實成為其中一個樣本。

第 31 章 (機器學習與強化學習) 回答一個根本的問題：**當問題寫不出方程式時怎麼辦？** 逆運動學可以算到微米級，但摺一件 T 恤至今無人能及五歲小孩——因為衣服沒有方程式。這一章也回收了第 18 章最重要的伏筆：**強化學習與最佳控制共用同一條 Bellman 方程**，差別只在於模型是否已知。

第 32 章 (大型模型與任務規劃) 讓機器人第一次能「聽懂人話」。「幫我把桌上的杯子拿到廚房」這句話背後，藏著三十年來寫不完的常識——而大型語言模型從文字中學到了它們。但它也帶來新的危險：**它會非常有自信地說錯話**，而在機器人上，說錯就是一個真實的動作。

第 33 章 (具身人工智慧) 是全書理論部分的收束。它回到第 1 章的 Moravec 悖論，給出三層答案，並提出一個統一的觀點：**智慧不只在腦裡，也在身體裡，還在身體與環境的互動裡**。

這一篇要回答的六個問題

- 四組的成果都不錯，為什麼接不起來？
- 模擬裡 98% 的成功率，為什麼到實機只剩兩成？
- 算逆運動學能到微米級，為什麼摺不好一件 T 恤？

- 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？
- 為什麼電腦下棋能贏世界冠軍，卻摺不好一件衣服？
- 一隻蜜蜂的腦只有一百萬個神經元，為什麼能做到我們的機器人做不到的事？

三條貫穿全篇的線索

線索一：模型提供保證，學習提供泛化。 這是本篇最核心的分工原則。第 31 章明確指出：能寫出方程式的就用模型（更準、更可靠、不需資料），寫不出來的才用學習。第 32 章的架構更是三者並用——語言模型提供常識、符號規劃提供正確性、底層控制提供安全。**不是誰取代誰，而是各自處理擅長的部分。**

線索二：智慧可以是機率性的，但安全必須是確定性的。 學習與大型模型都沒有保證、都可能在分布外做出不可預測的行為。因應之道不是讓它們變得更可靠，而是**在其外加上一層模型式的、可驗證的安全機制**。一個力矩限制不該由神經網路判斷，而應該是一行明確的 if 敘述。

線索三：與其消除不確定性，不如設計對它不敏感的系統。 這條主線從第 20 章的柔順控制、第 21 章的軟性夾爪一路延伸到第 30 章的領域隨機化，並在第 33 章以「形態計算」得到統一的解釋——**當身體的結構與任務相匹配時，大量的計算可以由物理本身完成。**

讀完這一篇之後

你將能夠：以 ROS 2 建立完整的機器人軟體系統、設計模擬環境並跨越 Sim-to-Real 落差、運用監督式學習與強化學習訓練機器人技能、以大型模型實現語言驅動的任務規劃、並為系統設計三層安全防護。

更重要的是，你將獲得三種判斷力：**知道何時該用哪個工具**（模型或學習、精確或柔順、規劃或反應）、**知道每個方法的邊界**（在什麼假設下成立、在什麼情況下失效）、以及**知道問題的本質**——很多時候，正確的答案不是更好的演算法，而是重新定義問題。

第 33 章的最後留下了一句話：**現實會回應你，這是機器人學最好的老師。**而第六篇「系統整合、產業應用與專題」將帶你真正走進現實——把這三十三章的知識，變成一個能運作的系統。

本篇章節配置

章次	章名	核心內容	叫獸開講
第 29	ROS 2 與機器	節點與話題、服務與動作、參數與啟動檔、	「四組的成果都不錯，為什麼接不起

章次	章名	核心內容	叫獸開講
章	人軟體框架	TF2、URDF、QoS、工具生態	來？」——整合的困難平方成長
第 30 章	模擬與 Sim-to-Real	物理引擎、保真度、落差的五個來源、領域隨機化、系統辨識、平行模擬	「模擬 98%，實機只剩兩成？」——模擬不必更真，策略必須更穩
第 31 章	機器學習與強化學習	三種典範、模仿學習與 DAgger、MDP、Bellman 方程、獎勵設計、PPO	「算逆運動學到微米級，為什麼摺不好 T 恤？」——衣服沒有方程式
第 32 章	大型模型與任務規劃	符號接地、PDDL、語言模型作為規劃器、SayCan、VLM 與 VLA、三層防護	「幫我把杯子拿到廚房」——那些寫不完的常識
第 33 章	具身人工智慧	Moravec 悖論的三層答案、具身認知、形態計算、主動感知、倫理與社會	「蜜蜂只有一百萬個神經元卻能飛行」——智慧不只在腦裡

第五篇 智慧機器人與具身人工智慧

第 29 章

ROS 2 與機器人軟體框架

編著：李世淵（ 機器人叫獸 ）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩種架構的對照：上圖是「點對點」——八個模組以雜亂交錯的線兩兩相連，共二十八條線，以紅色標註「介面平方成長」；下圖是「發布訂閱」——同樣八個模組各自連到中央的話題匯流排，線條清爽，標註「模組完全解耦」。右半為一台服務機器人的節點圖：以圓圈代表節點（驅動、SLAM、導航、辨識、手臂規劃、任務管理），以箭頭與話題名稱（/scan、/map、/cmd_vel）連接，並在旁側標出三種通訊模式的圖示。下緣以一棵 TF 樹（map → odom → base_link → 各感測器）收尾。標題文字：讓不同的程式能夠對話。

叫獸開講 「四組的成果都不錯，為什麼接不起來？」

學期末的專題發表會上，四組學生輪流上台。第一組做了視覺辨識、第二組做了機械手臂控制、第三組做了移動底盤導航、第四組做了語音介面。每一組單獨看都做得不錯。

叫獸聽完後問了一個問題：「現在，請你們把四組的成果組合起來，做成一台『聽到指令就去拿東西』的服務機器人。需要多久？」

教室安靜了。小翰苦笑：「視覺組用 Python 3.10、手臂組用 C++、底盤組寫在 Arduino 上、語音組是 Node.js.....而且大家的座標系定義都不一樣。」

「所以答案是？」「.....可能要重寫一半。」

「這就是機器人軟體最大的問題。」叫獸在白板上寫下一個詞：**整合**。「每一個子系統都不難，但把它們接起來極其困難。而且這個困難會隨著模組數量平方成長——三個模組要接三條線，十個模組要接四十五條。」

他停頓一下：「更糟的是，這個工作沒有任何學術價值。沒有人會因為『我把視覺和手臂接起來了』而發表論文。但如果不做，什麼系統都做不出來。」

小矧舉手：「所以有現成的解法嗎？」

「有，而且它改變了整個領域。」叫獸寫下三個字母：**ROS**。「機器人作業系統。但它的名字有點誤導——它不是作業系統，它是一套讓不同的程式能夠互相溝通的約定。」

他在白板上畫了幾個圓圈，用線連起來：「它的核心思想極為簡單：把每個功能寫成一個獨立的程式（節點），讓它們透過標準化的訊息互相溝通。視覺節點發布『我看到一個杯子在這個位置』，手臂節點訂閱這個訊息並執行抓取。兩者不需要知道對方是誰、用什麼語言寫的、甚至不需要在同一台電腦上。」

「這個設計解決的不只是技術問題，」叫獸最後說，「它解決的是協作問題。全世界的實驗室可以各自開發一個節點，然後互相使用。這就是為什麼今天你可以下載一個現成的 SLAM 套件、接上你的光達，十分鐘就跑起來——而這件事在二十年前，需要一個博士生做兩年。」

本章為什麼重要

前面二十八章建立了機器人的理論與能力：運動學、動力學、控制、感知、定位、規劃。但

這些知識要變成一台能運作的機器人，還缺少一樣東西——**把它們組織起來的軟體架構**。

這個問題常被低估。在教科書中，各章的內容是獨立的；但在真實的機器人上，它們必須**同時運行、互相通訊、共享時間基準、並在某個環節失效時仍能安全處置**。一個完整的機器人系統可能包含數十個並行的程式，處理著不同頻率的資料流（第 3 章的時間尺度金字塔）。

ROS 2 是目前這個問題的標準答案。它提供了三樣東西：**一套通訊機制**（讓模組能互相傳訊息）、**一套工具生態**（視覺化、記錄、除錯、模擬）、以及**一個社群**（數千個現成的套件可以直接使用）。學會它，就等於獲得了整個機器人社群數十年的累積。

本章也是全書的一個轉折點。前四篇教的是「**機器人學的知識**」，本章開始則是「**如何把知識變成系統**」——而這正是第五篇「**智慧機器人**」的必要基礎：第 30 章的模擬、第 31 章的學習、第 32 章的大型模型，全部都建立在這套軟體架構之上。

學習目標

- 能說明機器人軟體整合的困難，以及框架所解決的問題。
- 能說明 ROS 的核心設計哲學與其三個貢獻。
- 能比較 ROS 1 與 ROS 2 的架構差異及其動機。
- 能說明節點、話題、訊息的概念並實作發布與訂閱。
- 能區分話題、服務與動作三種通訊模式並選擇合適者。
- 能說明參數與啟動檔的作用並撰寫基本的啟動檔。
- 能運用 TF2 管理座標變換並理解其時間查詢機制。
- 能撰寫 URDF 描述機器人並在 RViz 中視覺化。
- 能說明 QoS 設定的意義並為不同的資料流選擇適當設定。
- 能運用 rosbag、rqt 與 RViz 進行記錄與除錯。
- 能說明即時性的需求與 ROS 2 的因應機制。
- 能設計一個完整機器人系統的節點架構。

先備知識

第 3 章的系統架構與時間尺度、第 8 章的座標系與齊次變換（TF2 的基礎）、第 4 章的連桿

與關節（URDF 的基礎）、第 7 章的嵌入式與即時性概念。

程式方面需要 Python 或 C++ 的基本能力（本章以 Python 為主）、以及 Linux 命令列的基本操作。**不需要**事先接觸過 ROS。若對 Linux 生疏，可先閱讀附錄 C。

核心名詞中英對照

表 29-1 本章核心名詞

中文	英文	說明
機器人作業系統	Robot Operating System, ROS	機器人軟體的中介層框架
節點	Node	執行特定功能的獨立程式單元
話題	Topic	多對多的非同步訊息通道
發布者 / 訂閱者	Publisher / Subscriber	發送與接收話題訊息的角色
訊息	Message	標準化的資料結構定義
服務	Service	一對一的同步請求回應模式
動作	Action	長時間執行且可回報進度的任務模式
參數	Parameter	節點的可設定值
啟動檔	Launch File	一次啟動多個節點的設定腳本
座標變換樹	TF2	管理各座標系關係的系統
統一機器人描述格式	URDF	描述機器人結構的 XML 格式
服務品質	Quality of Service, QoS	通訊可靠度與時效性的設定
資料分發服務	DDS	ROS 2 底層的通訊中介軟體
工作空間	Workspace	存放與編譯套件的目錄結構
套件	Package	ROS 程式的組織單位
生命週期節點	Lifecycle Node	具有明確狀態轉換的受管節點
執行器	Executor	決定回呼如何被執行的排程器
回呼群組	Callback Group	控制回呼並行方式的機制

29.1 為何需要框架

29.1.1 整合的困難

如開頭故事所述，機器人軟體最大的困難不在單一功能，而在**整合**。這個困難有四個具體的

來源。

來源一：介面的組合爆炸。若有 n 個模組要互相溝通，最壞情況需要 $n(n-1)/2$ 條介面。三個模組要 3 條，十個模組要 45 條——而且每一條都要各自定義資料格式、錯誤處理與同步機制。

來源二：異質性。不同的模組可能用不同的語言（C++ 求效能、Python 求開發速度）、跑在不同的硬體（PC、嵌入式板、微控制器）、甚至在不同的機器上（機器人本體與遠端工作站）。

來源三：時間與並行。第 3 章的時間尺度金字塔說明了機器人系統中並存著多種頻率——控制迴路 1 kHz、感知 30 Hz、規劃 1 Hz。這些必須**並行**執行且不互相阻塞。

來源四：除錯的困難。當系統由數十個並行程式構成時，「哪裡出錯了」極難判斷。若沒有工具能觀察系統內部的資料流，除錯幾乎不可能。

29.1.2 框架提供了什麼

一個機器人軟體框架，本質上是提供**四樣東西**：

表 29-2 機器人框架的四項核心功能

功能	解決的問題	ROS 2 中的對應
通訊機制	模組間如何傳資料（來源一、二）	話題、服務、動作
標準介面	資料格式如何統一	訊息類型（msg）、TF2、URDF
執行管理	多個程式如何啟動與並行（來源三）	啟動檔、執行器、生命週期
工具生態	如何觀察與除錯（來源四）	rqt、RViz、rosbag、ros2 CLI

◎ 為何不自己寫一套就好

許多團隊在初期會覺得「ROS 太複雜，我自己寫一個簡單的訊息機制就好」。這在專案初期確實較快，但通常在三個月後就會後悔。原因是：**你會逐步重新發明 ROS 的每一個功能**——先是需要記錄資料（於是寫一個 log 系統）、再需要視覺化（於是寫一個 GUI）、接著需要處理座標轉換（於是寫一個 TF）、然後需要多機通訊……而每一項你自己寫的版本，都不如 ROS 成熟，且**無法使用社群的任何現成套件**。這個決策的關鍵不在於「ROS 好不好」，而在於**你是否要放棄整個生態系**。

29.1.3 ROS 不是作業系統

必須釐清一個常見的誤解：**ROS 不是作業系統**。它跑在 Linux（或其他 OS）之上，是一個中介層（middleware）。

它之所以叫「作業系統」，是因為它提供了類似 OS 的服務：行程管理（啟動與監控節點）、行程間通訊（話題與服務）、硬體抽象（標準化的感測器與致動器介面）、以及套件管理。但底層的排程、記憶體管理、驅動程式，仍然由真正的作業系統負責。

29.2 ROS 1 到 ROS 2

29.2.1 ROS 1 的成功與限制

ROS 1 於 2007 年由 Willow Garage 推出，徹底改變了機器人研究。它的成功來自兩點：**開源**（任何人都能使用與貢獻）與**模組化**（節點可以獨立開發與重用）。

但隨著應用由研究走向產品，ROS 1 的四個限制逐漸浮現：

- **單點失效**：ROS 1 需要一個中央的 `roscore`（Master）來協調節點的連線。若它掛掉，整個系統瓦解。
- **無即時性保證**：底層的通訊機制不支援即時排程，無法用於安全關鍵的控制迴路。
- **無安全機制**：訊息以明文傳輸，任何人接上網路都能讀取或注入指令。
- **多機器人支援薄弱**：跨網段、跨機器的通訊需要複雜的設定。

29.2.2 ROS 2 的三個關鍵改變

ROS 2 於 2017 年推出，針對上述問題做了根本性的重新設計。

改變一：底層改用 DDS。DDS（Data Distribution Service）是一套工業標準的通訊中介軟體，原本用於航空、國防與工業自動化。它提供了三樣 ROS 1 沒有的東西：

- **去中心化的探索**：節點透過廣播自動找到彼此，**不需要中央的 Master**。這消除了單點失效。
- **QoS 設定**：可以為每個通道指定可靠度、時效性與歷史深度（29.6 節）。
- **內建安全機制**：支援加密、認證與存取控制。

改變二：支援即時性。ROS 2 的設計避免了動態記憶體配置與不確定的鎖定，配合即時作業系統（如 PREEMPT_RT）可用於控制迴路。

改變三：生命週期管理。引入了「受管節點」（managed node）的概念——節點有明確的狀態（未設定、非活動、活動、完成），可以被外部協調地啟動、暫停與重設。這對產品級的

系統極為重要。

表 29-3 ROS 1 與 ROS 2 的比較

面向	ROS 1	ROS 2
中央協調	需要 roscore (單點失效)	去中心化 (DDS 探索)
通訊層	自訂的 TCPROS/UDPROS	DDS (工業標準)
即時性	不支援	可支援 (配合 RTOS)
安全性	無 (明文)	加密、認證、存取控制
QoS 設定	無	完整支援
多平台	主要 Linux	Linux、Windows、macOS、RTOS
生命週期	無	受管節點
Python 版本	Python 2 (舊版)	Python 3
現況	已停止支援 (2025 EOL)	現行標準

本書一律使用 ROS 2。ROS 1 的最後一個版本 Noetic 已於 2025 年停止支援，新的專案不應再使用。

29.3 核心概念：節點與話題

29.3.1 節點

節點 (node) 是 ROS 中最基本的執行單位——一個執行特定功能的獨立程式。典型的節點包括：

- **驅動節點**：讀取感測器或控制致動器 (如光達驅動、馬達控制器)。
- **處理節點**：對資料做運算 (如影像辨識、SLAM、路徑規劃)。
- **介面節點**：與人或外部系統互動 (如遙控器接收、網頁介面)。

設計節點的核心原則是「單一職責」——每個節點只做一件事。這使得節點容易測試、重用與替換。若一個節點同時做了影像辨識與路徑規劃，就應該考慮拆分。

29.3.2 話題：非同步的多對多通訊

話題 (topic) 是節點之間最主要的溝通方式。它的運作模式是**發布—訂閱** (publish-subscribe)：

發布者 → [話題名稱] → 訂閱者 (可以有多個)

發布者不知道有誰在訂閱、訂閱者也不知道是誰在發布。兩者完全解耦。

這個解耦是 ROS 最重要的設計。它帶來三個好處：

- **可替換性**：可以把真實的相機驅動換成模擬器，只要發布同樣的話題，下游節點完全不需修改——這是第 30 章 Sim-to-Real 的基礎。
- **可觀察性**：任何話題都可以被額外訂閱來監看或記錄，不影響原本的資料流。
- **可擴充性**：新增一個訂閱者不需要修改發布者。

29.3.3 訊息類型

話題上傳輸的資料有明確的**類型** (message type)，以 `.msg` 檔定義。ROS 2 提供了大量的標準訊息，這是「標準介面」的具體體現：

表 29-4 常用的標準訊息類型

訊息類型	內容	典型用途
<code>geometry_msgs/Twist</code>	線速度與角速度 (各三維)	移動機器人的速度指令
<code>sensor_msgs/LaserScan</code>	一圈的距離讀值陣列	二維光達
<code>sensor_msgs/Image</code>	影像資料	相機
<code>sensor_msgs/PointCloud2</code>	三維點雲	三維光達、深度相機
<code>sensor_msgs/Imu</code>	角速度、加速度、姿態	IMU
<code>sensor_msgs/JointState</code>	各關節的位置、速度、力矩	機械手臂狀態
<code>nav_msgs/Odometry</code>	位姿與速度 (含協方差)	里程計
<code>nav_msgs/OccupancyGrid</code>	佔據柵格地圖	SLAM 輸出 (第 25 章)
<code>geometry_msgs/PoseStamped</code>	帶時間戳的位姿	目標點、定位結果

標準訊息的價值在於**互通性**。因為所有的移動機器人都用 `Twist` 接收速度指令，所以任何導航套件都能驅動任何底盤——這正是社群能夠共用套件的關鍵。

29.3.4 最簡單的例子

以下是一個發布者與訂閱者的完整範例 (Python)：

```
# publisher_node.py — 發布速度指令
import rclpy
from rclpy.node import Node
```

```

from geometry_msgs.msg import Twist

class VelocityPublisher(Node):
    def __init__(self):
        super().__init__('velocity_publisher')    # 節點名稱
        # 建立發布者：話題名稱 'cmd_vel'、佇列深度 10
        self.pub = self.create_publisher(Twist, 'cmd_vel', 10)
        # 建立定時器：每 0.1 秒呼叫一次 (10 Hz)
        self.timer = self.create_timer(0.1, self.timer_callback)
        self.get_logger().info('速度發布節點已啟動')

    def timer_callback(self):
        msg = Twist()
        msg.linear.x = 0.2    # 前進 0.2 m/s
        msg.angular.z = 0.1   # 左轉 0.1 rad/s
        self.pub.publish(msg)

def main():
    rclpy.init()
    node = VelocityPublisher()
    rclpy.spin(node)        # 進入事件迴圈，等待定時器觸發
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

```

# subscriber_node.py — 訂閱並顯示速度指令
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class VelocitySubscriber(Node):
    def __init__(self):
        super().__init__('velocity_subscriber')
        # 建立訂閱者：收到訊息時呼叫 listener_callback
        self.sub = self.create_subscription(
            Twist, 'cmd_vel', self.listener_callback, 10)

    def listener_callback(self, msg):
        self.get_logger().info(
            f'收到指令: v={msg.linear.x:.2f} m/s, '
            f'w={msg.angular.z:.2f} rad/s')

def main():
    rclpy.init()
    node = VelocitySubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```



```
if __name__ == '__main__':
    main()
```

注意兩個節點之間沒有任何直接的引用——它們只透過話題名稱 `cmd\_vel` 與訊息類型 `Twist` 這個「約定」連接。這正是解耦的具體體現。

29.4 服務與動作

29.4.1 三種通訊模式的分工

話題適合**持續的資料流**（感測器讀值、狀態回報），但不適合所有情況。ROS 2 提供三種通訊模式：

表 29-5 三種通訊模式的比較

模式	特性	是否等待回應	適用情境	範例
話題 Topic	多對多、非同步、持續	否	資料流、狀態廣播	感測器讀值、速度指令
服務 Service	一對一、同步、單次	是（阻塞）	快速的請求回應	查詢參數、觸發校正
動作 Action	一對一、非同步、可回報進度	是（非阻塞）	長時間任務	導航到目標、執行軌跡

29.4.2 服務

服務（service）是「問一個問題，等一個答案」的模式。它由**請求**（request）與**回應**（response）兩部分構成。

典型的用途包括：查詢機器人的狀態、切換模式、觸發一次性的動作（如拍照、校正）。

服務的**關鍵限制是它會阻塞**——呼叫者必須等待回應才能繼續。因此**絕不應該用服務執行長時間的任務**。若一個服務要花五秒才回應，呼叫的節點就會卡住五秒，可能導致整個系統的反應遲鈍。

29.4.3 動作

動作（action）正是為了解決這個問題而設計的。它適用於「**需要一段時間、且過程中想知道進度、還可能需要中途取消**」的任務。

動作由三部分構成：

- **目標 (Goal)**：要執行什麼。例如「導航到 (5, 3)」。
- **回饋 (Feedback)**：執行過程中持續回報。例如「已走 60%、剩餘距離 2.1 公尺」。
- **結果 (Result)**：完成後的最終回報。例如「成功抵達」或「失敗：路徑被阻擋」。

此外，動作還支援**取消 (cancel)**——呼叫者可以在中途要求中止。這對機器人極為重要：規劃好的導航可能因為突發狀況需要放棄。

最典型的動作範例是 Nav2 的導航介面（第 26.8.2 節）。上層下達「去廚房」的目標，導航系統持續回報進度，若途中被人擋住太久則回報失敗，而上層也可以隨時取消。

◎ 選擇通訊模式的判準

問三個問題：一、需要回應嗎？不需要 → 話題。二、會很快回應嗎（毫秒級）？是 → 服務。

三、需要進度回報或可能取消嗎？是 → 動作。一個常見的錯誤是濫用服務——把「移動手臂到某個姿態」寫成服務，結果呼叫者被卡住數秒。這類任務應該用動作。反之，也有人把單純的狀態查詢寫成動作，徒增複雜度。**選對模式，程式碼會自然變簡單。**

29.5 參數與啟動檔

29.5.1 參數：把設定與程式碼分離

一個節點通常有許多可調的設定：控制增益、感測器的裝置路徑、更新頻率、機器人的尺寸。這些不應該寫死在程式碼裡——否則每改一個數值就要重新編譯。

ROS 2 的參數 (parameter) 機制把設定外部化。節點在啟動時宣告自己需要哪些參數，這些參數可以由啟動檔、YAML 檔或命令列指定，甚至可以在執行中動態修改。

```
# 節點中宣告與讀取參數
class ControllerNode(Node):
    def __init__(self):
        super().__init__('controller')
        # 宣告參數並設定預設值
        self.declare_parameter('kp', 2.0)
        self.declare_parameter('kd', 0.5)
        self.declare_parameter('max_speed', 1.0)
        self.declare_parameter('frame_id', 'base_link')

        # 讀取參數值
        self.kp = self.get_parameter('kp').value
        self.kd = self.get_parameter('kd').value
        self.get_logger().info(f'控制增益: Kp={self.kp}, Kd={self.kd}')
```

```
# 註冊回呼，支援執行中動態調參
self.add_on_set_parameters_callback(self.param_callback)

def param_callback(self, params):
    from rcl_interfaces.msg import SetParametersResult
    for p in params:
        if p.name == 'kp':
            self.kp = p.value
            self.get_logger().info(f'Kp 已更新為 {self.kp}')
    return SetParametersResult(successful=True)
```

動態調參的價值極大。第 16 章的 PID 調校、第 26 章的導航參數、第 27 章的飛控增益——這些都需要反覆試驗。若每次都要重新編譯與重啟，調參會極為緩慢；有了動態參數，可以在機器人運行中即時調整並觀察效果。

29.5.2 YAML 參數檔

參數通常以 YAML 檔管理，這使得不同的機器人或不同的場景可以有各自的設定檔：

```
# config/controller_params.yaml
controller:
  ros__parameters:
    kp: 3.5
    kd: 0.8
    max_speed: 0.6
    frame_id: "base_link"

# 不同場景可以有不同的檔案
# config/controller_params_indoor.yaml → max_speed: 0.4
# config/controller_params_outdoor.yaml → max_speed: 1.2
```

29.5.3 啟動檔：一次啟動整個系統

一個完整的機器人系統可能需要同時啟動十幾個節點。若要一個一個手動啟動，既繁瑣又容易出錯。**啟動檔** (launch file) 解決了這個問題。

ROS 2 的啟動檔以 Python 撰寫 (也支援 XML 與 YAML)，這使它能夠包含邏輯——例如依參數決定是否啟動模擬器、或依機器人型號載入不同的設定。

```
# launch/robot_bringup.launch.py
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.substitutions import LaunchConfiguration
import os
from ament_index_python.packages import get_package_share_directory
```

```
def generate_launch_description():
    pkg_dir = get_package_share_directory('my_robot')
    params = os.path.join(pkg_dir, 'config', 'controller_params.yaml')

    # 可由命令列覆寫的啟動參數
    use_sim = LaunchConfiguration('use_sim')

    return LaunchDescription([
        DeclareLaunchArgument('use_sim', default_value='false',
                               description='是否使用模擬器'),

        # 底盤驅動節點
        Node(package='my_robot', executable='base_driver',
             name='base_driver', parameters=[params],
             output='screen'),

        # 光達驅動節點
        Node(package='rplidar_ros', executable='rplidar_node',
             name='lidar', parameters=[{'serial_port': '/dev/ttyUSB0'}]),

        # 控制器節點 (重新命名話題)
        Node(package='my_robot', executable='controller',
             name='controller', parameters=[params],
             remappings=[('cmd_vel', 'cmd_vel_smoothed')]),

        # 機器人狀態發布 (讀取 URDF 並發布 TF)
        Node(package='robot_state_publisher',
             executable='robot_state_publisher',
             parameters=[{'robot_description': open(
                 os.path.join(pkg_dir, 'urdf', 'robot.urdf')).read()}]),
    ])

```

三個值得注意的機制：

- 參數檔的載入：`parameters=[params]` 使多個節點共用同一份設定。
- 話題重新映射 (remapping)：`remappings` 讓節點在不修改程式碼的情況下改變它發布或訂閱的話題名稱。這是重用第三方節點的關鍵技巧。
- 啟動參數：`LaunchConfiguration` 使同一個啟動檔能依情境調整 (如切換真實硬體與模擬器)。

29.6 TF2：座標變換的管理

29.6.1 為何需要專門的系統

第 8 章建立了座標變換的數學基礎。但在真實的機器人系統中，座標系的管理有三個超出數學的困難：

困難一：數量龐大。一台移動操作機器人可能有數十個座標系：世界、地圖、里程計、底盤、雷射、相機、手臂各連桿、夾爪、工具中心點.....

困難二：分散在不同節點。相機的外參由校正節點提供、手臂各連桿的變換由關節狀態計算、地圖到里程計的變換由定位節點提供。**沒有任何一個節點知道全部。**

困難三：時間問題。這是最關鍵也最容易被忽略的。相機在 t 時刻拍到一個物體，但這張影像的處理需要 50 毫秒。當結果出來時，機器人已經移動了——此時該用哪個時刻的座標變換？

TF2 正是為了解決這三個問題而生。它是一個分散式的座標變換系統，維護著一棵隨時間變化的變換樹。

29.6.2 變換樹的結構

TF2 把所有座標系組織成一棵樹 (tree)，這個結構有兩個嚴格的規則：

規則一：每個座標系只能有一個父座標系

規則二：不能有迴圈

樹的結構保證了任意兩個座標系之間有唯一的變換路徑。

一個典型的移動機器人 TF 樹：

```
map                ← 全域地圖座標系 ( 由 SLAM 或定位提供 )
├── odom            ← 里程計座標系 ( 連續但會漂移 )
│   ├── base_footprint ← 機器人在地面的投影
│   │   └── base_link   ← 機器人本體中心
│   │       ├── laser_link ← 光達
│   │       ├── camera_link ← 相機
│   │       ├── imu_link  ← IMU
│   │       └── arm_base_link ← 手臂基座
│   │           └── link1 → link2 → ... → gripper
```

◎ map → odom → base_link 的深意

這個三層結構是 ROS 導航的標準設計 (第 24.10.3 節)，其分工極為巧妙。**odom → base_link** 由里程計提供：**連續、高頻、但會漂移**。**map → odom** 由定位系統 (AMCL 或 SLAM) 提供：**低頻、會跳變、但不漂移**。兩者串接後得到的 **map → base_link** 就同時具備了連續性與不漂移。關

鍵在於：定位系統修正的不是機器人的位置，而是「地圖與里程計之間的偏差」。這使得下游的控制器可以放心使用 odom 座標系（保證連續，不會突然跳動），而不必擔心長期漂移。這是一個值得細細品味的架構設計。

29.6.3 時間查詢

TF2 最重要的功能是帶時間戳的查詢。它會儲存過去一段時間（預設 10 秒）的變換歷史，允許查詢「在某個特定時刻，A 相對於 B 的變換是什麼」。

```
import rclpy
from rclpy.node import Node
from tf2_ros import Buffer, TransformListener
from geometry_msgs.msg import PointStamped
import tf2_geometry_msgs # 註冊 PointStamped 的轉換支援

class TFExample(Node):
    def __init__(self):
        super().__init__('tf_example')
        self.buffer = Buffer()
        self.listener = TransformListener(self.buffer, self)

    def transform_point(self, point_in_camera):
        """把相機座標系中的一個點，轉換到 base_link 座標系"""
        try:
            # 關鍵：使用該點自身的時間戳查詢，而非「現在」
            result = self.buffer.transform(
                point_in_camera, 'base_link',
                timeout=rclpy.duration.Duration(seconds=0.1))
            return result
        except Exception as e:
            self.get_logger().warn(f'座標轉換失敗: {e}')
            return None

    def lookup_at_time(self, stamp):
        """查詢特定時刻的變換"""
        try:
            t = self.buffer.lookup_transform(
                'map', 'base_link', stamp,
                timeout=rclpy.duration.Duration(seconds=0.1))
            return t
        except Exception as e:
            self.get_logger().warn(f'查詢失敗: {e}')
            return None
```

注意 ``self.buffer.transform(point_in_camera, ...)`` 這一行——它使用了 ``point_in_camera`` 自身攜帶的時間戳。這意味著：即使處理花了 50 毫秒，TF2 也會回溯到影像拍攝的那一刻的座標變換來計算。

這正是第 24.9.3 節「時間戳不同步」問題的系統級解法。若沒有 TF2，每個節點都必須自己維護變換的歷史紀錄——這既繁瑣又容易出錯。

29.6.4 靜態與動態變換

TF2 區分兩種變換：

- **靜態變換 (Static)**：不隨時間改變的，如相機相對於底盤的安裝位置。只需發布一次，TF2 會永久記住。用 ``static_transform_publisher`` 或在 URDF 中定義。
- **動態變換 (Dynamic)**：隨時間改變的，如機器人在地圖中的位置、手臂各關節的角度。必須持續發布 (典型 10~100 Hz)。

一個常見的錯誤是把靜態變換當成動態的持續發布——這會浪費頻寬並增加系統負擔。相機的安裝位置一小時都不會變，卻每秒發布 50 次，是不必要的。

29.7 URDF 與機器人描述

29.7.1 用 XML 描述機器人

URDF (Unified Robot Description Format) 是描述機器人結構的標準格式。它以 XML 撰寫，直接對應第 4 章的連桿與關節概念。

一份 URDF 描述三類資訊：

- **運動學**：連桿之間的連接關係、關節類型與軸向、關節極限——這正是第 9 章 DH 參數的等價資訊。
- **動力學**：各連桿的質量、質心位置、慣性張量——第 13 章動力學計算所需的參數。
- **幾何**：視覺模型 (用於顯示) 與碰撞模型 (用於碰撞檢測，第 26 章)。

```
<?xml version="1.0"?>
<robot name="two_link_arm">

  <!-- 基座連桿 -->
  <link name="base_link">
    <visual>
      <geometry><cylinder radius="0.05" length="0.02"/></geometry>
      <material name="grey"><color rgba="0.5 0.5 0.5 1"/></material>
    </visual>
    <collision>
      <geometry><cylinder radius="0.05" length="0.02"/></geometry>
    </collision>
    <inertial>
      <mass value="1.0"/>
    </inertial>
  </link>
  <link name="link1">
    <parent link="base_link" joint="revolute" offset="0.05 0 0" />
    <axis joint="revolute" axis="x" />
    <limit joint="revolute" lower="0" upper="3.14" />
    <visual>
      <geometry><cylinder radius="0.05" length="0.02"/></geometry>
      <material name="red"><color rgba="1 0 0 1"/></material>
    </visual>
    <collision>
      <geometry><cylinder radius="0.05" length="0.02"/></geometry>
    </collision>
    <inertial>
      <mass value="1.0"/>
    </inertial>
  </link>
  <link name="link2">
    <parent link="link1" joint="revolute" offset="0.05 0 0" />
    <axis joint="revolute" axis="x" />
    <limit joint="revolute" lower="0" upper="3.14" />
    <visual>
      <geometry><cylinder radius="0.05" length="0.02"/></geometry>
      <material name="blue"><color rgba="0 0 1 1"/></material>
    </visual>
    <collision>
      <geometry><cylinder radius="0.05" length="0.02"/></geometry>
    </collision>
    <inertial>
      <mass value="1.0"/>
    </inertial>
  </link>
</robot>
```



```

    <inertia ixx="0.001" iyy="0.001" izz="0.002"
            ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>

<!-- 第一個關節：旋轉關節，繞 Z 軸 -->
<joint name="joint1" type="revolute">
  <parent link="base_link"/>
  <child link="link1"/>
  <origin xyz="0 0 0.01" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-3.14" upper="3.14" effort="10" velocity="2.0"/>
</joint>

<!-- 第一節連桿：長 0.3 m -->
<link name="link1">
  <visual>
    <origin xyz="0.15 0 0" rpy="0 1.5708 0"/>
    <geometry><cylinder radius="0.02" length="0.3"/></geometry>
    <material name="blue"><color rgba="0.2 0.4 0.8 1"/></material>
  </visual>
  <collision>
    <origin xyz="0.15 0 0" rpy="0 1.5708 0"/>
    <geometry><cylinder radius="0.025" length="0.3"/></geometry>
  </collision>
  <inertial>
    <origin xyz="0.15 0 0"/>
    <mass value="0.5"/>
    <inertia ixx="0.0001" iyy="0.0038" izz="0.0038"
            ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>

<!-- 第二個關節 -->
<joint name="joint2" type="revolute">
  <parent link="link1"/>
  <child link="link2"/>
  <origin xyz="0.3 0 0" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-2.5" upper="2.5" effort="5" velocity="2.0"/>
</joint>

<link name="link2">
  <visual>
    <origin xyz="0.1 0 0" rpy="0 1.5708 0"/>
    <geometry><cylinder radius="0.015" length="0.2"/></geometry>
    <material name="green"><color rgba="0.2 0.7 0.3 1"/></material>
  </visual>
  <inertial>
    <origin xyz="0.1 0 0"/>
    <mass value="0.3"/>
    <inertia ixx="0.00005" iyy="0.001" izz="0.001"
            ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>

</robot>

```

這份檔案正是第 4 章實作的延續。當時我們用 `check\_urdf` 驗證了語法，現在要讓它真正動起來。

29.7.2 由 URDF 到 TF

URDF 與 TF2 之間有一個關鍵的橋樑：**robot_state_publisher** 節點。它的運作方式是：

1. 讀取 URDF，建立連桿與關節的結構。
2. 訂閱 `/joint\_states` 話題（訊息類型 `sensor\_msgs/JointState`），取得各關節的當前角度。
3. 用正向運動學計算各連桿的位姿（第 9 章的內容！）。
4. 把結果發布到 TF 樹上。

這是一個極為優雅的設計：只要有任何節點發布關節角度，整個機器人的座標變換就自動建立。無論這些角度來自真實的編碼器、模擬器、或是規劃器的輸出——下游完全不需要知道。

這也再次體現了 29.3.2 節的解耦原則：**真實機器人與模擬器可以完全互換**，只要它們都發布 `/joint\_states`。

29.7.3 Xacro：讓 URDF 可維護

純 URDF 有一個嚴重的缺點：**大量重複**。一台六軸手臂的六個連桿，其 XML 結構幾乎相同，只有尺寸與慣量不同。若手動撰寫，一份 URDF 可能有上千行，且極難維護。

Xacro (XML Macro) 解決了這個問題。它為 URDF 加上了巨集、變數與數學運算：

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="arm">

  <!-- 定義常數 -->
  <xacro:property name="link_radius" value="0.02"/>
  <xacro:property name="pi" value="3.14159265"/>

  <!-- 定義可重用的巨集 -->
  <xacro:macro name="arm_link" params="name length mass color">
    <link name="${name}">
      <visual>
        <origin xyz="${length/2} 0 0" rpy="0 ${pi/2} 0"/>
        <geometry>
          <cylinder radius="${link_radius}" length="${length}"/>
        </geometry>
      </visual>
    </link>
  </xacro:macro>
</robot>
```

```

    <material name="${color}"/>
  </visual>
  <inertial>
    <origin xyz="${length/2} 0 0"/>
    <mass value="${mass}"/>
    <!-- 圓柱體的慣量公式 (第 13.1.3 節) -->
    <inertia ixx="${mass*link_radius*link_radius/2}"
            iyy="${mass*length*length/12}"
            izz="${mass*length*length/12}"
            ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>
</xacro:macro>

<!-- 使用巨集，一行產生一個連桿 -->
<xacro:arm_link name="link1" length="0.3" mass="0.5" color="blue"/>
<xacro:arm_link name="link2" length="0.2" mass="0.3" color="green"/>
<xacro:arm_link name="link3" length="0.15" mass="0.2" color="red"/>

</robot>

```

注意慣量的計算直接寫在 XML 中 (``${mass*length*length/12}``) ——這正是第 13.1.3 節的圓柱體慣量公式。改變長度時，慣量會自動更新，不會出現「改了尺寸卻忘了改慣量」這種常見的錯誤。

29.8 QoS：通訊品質設定

29.8.1 為何需要設定

ROS 1 的通訊只有一種模式：可靠傳輸 (類似 TCP) 。但機器人系統中不同的資料流有截然不同的需求：

- 影像串流：每秒 30 張，若掉了一張無所謂——寧可丟棄舊資料，也要拿到最新的。
- 地圖資料：只在更新時發布，但新加入的節點必須拿到——必須可靠，且要能取得最後一筆。
- 緊急停止指令：絕對不能遺失。
- 控制指令：高頻、且舊的指令毫無價值——寧可丟也不要延遲。

用同一種傳輸模式處理這些，必定有一方受害。QoS (Quality of Service) 讓每個話題可以各自設定。

29.8.2 主要的 QoS 選項

表 29-6 主要的 QoS 設定

設定	選項	意義	典型用途
可靠度 Reliability	RELIABLE	保證送達（會重傳）	指令、地圖、狀態
	BEST_EFFORT	盡力而為（不重傳）	感測器串流
持久性 Durability	VOLATILE	只送給當前的訂閱者	一般資料
	TRANSIENT_LOCAL	保留給後來的訂閱者	地圖、機器人描述
歷史 History	KEEP_LAST(n)	只保留最近 n 筆	多數情況
	KEEP_ALL	全部保留	不可遺失的資料
時效 Deadline	時間值	超過此間隔未收到即警告	監控感測器是否失效
生命期 Lifespan	時間值	超過此時間的訊息作廢	即時控制指令

29.8.3 實務上的設定

ROS 2 提供了預設的 QoS 組合 (profile)，涵蓋常見情境：

```

from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy
from rclpy.qos import qos_profile_sensor_data

# 情境一：感測器資料——用內建的 sensor_data profile
# ( BEST_EFFORT + KEEP_LAST(5) · 適合高頻串流 )
self.scan_sub = self.create_subscription(
    LaserScan, 'scan', self.scan_callback, qos_profile_sensor_data)

# 情境二：地圖——必須可靠，且晚加入的節點也要拿得到
map_qos = QoSProfile(
    depth=1,
    reliability=ReliabilityPolicy.RELIABLE,
    durability=DurabilityPolicy.TRANSIENT_LOCAL) # 關鍵設定
self.map_pub = self.create_publisher(OccupancyGrid, 'map', map_qos)

# 情境三：控制指令——可靠但只要最新的
cmd_qos = QoSProfile(
    depth=1,
    reliability=ReliabilityPolicy.RELIABLE)
self.cmd_pub = self.create_publisher(Twist, 'cmd_vel', cmd_qos)

```

◎ QoS 不相容：一個經典的除錯陷阱

ROS 2 有一個規則：**發布者與訂閱者的 QoS 必須相容**，否則完全不會連線——而且不會報錯，只是「什麼都收不到」。最常見的情況是：發布者用 BEST_EFFORT，訂閱者用 RELIABLE (預設值)。訂閱者要求「保證送達」，但發布者不提供這個保證，於是**兩者不建立連線**。當你發現「明

明有節點在發布，但我就是收不到」時，第一件事就是用 `ros2 topic info -v <話題名稱>` 檢查兩端的 QoS 設定。這是 ROS 2 初學者最常遇到的坑，也是從 ROS 1 轉換過來的人最容易忽略的差異。

29.9 工具生態

29.9.1 命令列工具

ROS 2 的命令列工具（`ros2` 指令）是日常開發與除錯的第一道防線。掌握這幾個指令，能解決大部分的問題：

表 29-7 常用的 `ros2` 命令列工具

指令	用途	典型使用時機
<code>ros2 node list</code>	列出所有執行中的節點	確認節點是否啟動
<code>ros2 topic list</code>	列出所有話題	確認話題名稱是否正確
<code>ros2 topic echo <topic></code>	即時顯示話題內容	檢查資料是否正確
<code>ros2 topic hz <topic></code>	量測發布頻率	檢查是否掉頻
<code>ros2 topic info -v <topic></code>	顯示發布訂閱者與 QoS	除錯 QoS 不相容
<code>ros2 param list / get / set</code>	查看與設定參數	調參
<code>ros2 service call <srv></code>	手動呼叫服務	測試服務
<code>ros2 run <pkg> <exec></code>	執行單一節點	快速測試
<code>ros2 launch <pkg> <file></code>	執行啟動檔	啟動完整系統
<code>ros2 doctor</code>	檢查環境設定	環境有問題時

三個最有價值的除錯指令：`ros2 topic hz` 能立刻看出是否掉頻（第 3 章的時間尺度問題）、`ros2 topic echo` 能確認資料內容、`ros2 topic info -v` 能診斷 QoS 不相容（29.8.3 節的陷阱）。

29.9.2 RViz：三維視覺化

RViz 是 ROS 最重要的視覺化工具。它能同時顯示：

- 機器人模型：由 URDF 與 TF 即時呈現機器人的姿態。
- 感測資料：光達點雲、相機影像、深度點雲。
- 地圖與規劃：佔據柵格、路徑、代價地圖。

- **座標系**：TF 樹中所有座標系的位置與方向。
- **自訂標記**：以 `visualization\_msgs/Marker` 繪製任意的箭頭、線段、文字——用於顯示演算法的內部狀態。

RViz 的核心價值在於「讓不可見的變可見」。第 24 章的協方差橢圓、第 25 章的粒子分布、第 26 章的規劃路徑——這些抽象的內部狀態，都可以用 Marker 畫出來。當演算法出錯時，看見它在想什麼，往往比讀程式碼更快找到問題。

◎ RViz 不是模擬器

這是初學者最常見的誤解。**RViz 只是「顯示」——它不模擬任何物理**。它畫出的機器人會動，是因為有節點在發布 TF 與關節狀態；若那些資料來自真實機器人，RViz 就是即時的監控畫面；若來自模擬器（第 30 章的 Gazebo），RViz 顯示的就是模擬結果。**RViz 是眼睛，Gazebo 是世界——兩者角色完全不同，卻常被混淆。**

29.9.3 rosbag：記錄與重放

rosbag 能把所有話題的資料錄下來，之後完整重放。這個功能對機器人開發的價值極為重大：

- **離線除錯**：實機測試時錄下所有資料，回到辦公室慢慢分析。不需要重複跑實驗。
- **演算法比較**：用同一份資料測試不同的演算法版本，確保比較是公平的。
- **回歸測試**：修改程式後，用歷史資料驗證行為沒有退化。
- **資料集建立**：第 31 章的機器學習需要大量資料，rosbag 是最自然的收集方式。

```
# 錄製所有話題
ros2 bag record -a -o my_experiment

# 只錄製特定話題 (節省空間)
ros2 bag record /scan /odom /tf /camera/image_raw -o experiment_01

# 查看錄製內容的資訊
ros2 bag info experiment_01

# 重放 (可調速度)
ros2 bag play experiment_01
ros2 bag play experiment_01 --rate 0.5    # 半速，便於觀察
ros2 bag play experiment_01 --loop        # 循環播放
```

一個實務上的重要習慣：每次實機測試都錄 bag。硬碟很便宜，但「回到現場重做一次實驗」極為昂貴——尤其是那些偶發的、難以重現的錯誤。

29.9.4 rqt：圖形化工具集

rqt 是一組圖形化的除錯工具，常用的包括：

- **rqt_graph**：顯示節點與話題的连接圖。這是理解一個陌生系統最快的方式——一眼看出誰在跟誰講話。
- **rqt_plot**：即時繪製數值話題的曲線。用於觀察控制器的響應（第 16 章的階躍響應就可以這樣看）。
- **rqt_reconfigure**：圖形化的動態調參介面。配合 29.5.1 節的參數回呼，可以拖動滑桿即時調整增益。
- **rqt_console**：集中顯示所有節點的 log 訊息，可依等級與節點過濾。

29.10 即時性與系統設計

29.10.1 ROS 2 能做即時控制嗎

這是一個需要仔細回答的問題。簡短的答案是：可以，但需要條件。

ROS 2 在設計上支援即時性，但要真正達成，需要三個層次的配合：

1. **作業系統層**：必須使用即時核心（如 Linux 的 PREEMPT_RT 修補）或即時作業系統。標準的 Linux 無法保證延遲上限。
2. **中介軟體層**：DDS 的實作必須支援即時（如 eProsima 的 Fast DDS 有即時模式）。
3. **應用層**：節點的程式碼必須避免動態記憶體配置、避免不確定的鎖定、並使用即時安全的 API。

實務上的常見架構是分層混合：

表 29-8 典型的即時性分層

層級	頻率	執行環境	為何如此
伺服控制	1~10 kHz	微控制器或即時核心	硬即時要求，不容許延遲
狀態估測	100~1000 Hz	即時核心或高優先權	軟即時，偶爾延遲可容忍
感知與規劃	1~30 Hz	一般的 ROS 2 節點	無即時要求

層級	頻率	執行環境	為何如此
任務決策	< 1 Hz	一般節點或雲端	完全無即時要求

這正是第 3 章時間尺度金字塔的軟體實現。最底層的硬即時通常交給專用的硬體（第 5 章的伺服驅動器、第 7 章的微控制器），ROS 2 則負責其上的所有層級。

29.10.2 執行器與回呼群組

一個 ROS 2 節點可能有多個回呼（訂閱、定時器、服務）。它們如何被排程？這由執行器（executor）決定。

預設的 `SingleThreadedExecutor` 是單執行緒的——所有回呼依序執行，一次只能跑一個。這帶來一個重要的後果：

◎ 一個常見的阻塞陷阱

假設一個節點有兩個回呼：一個是 10 Hz 的感測器訂閱（處理很快）、另一個是服務（需要執行 2 秒的規劃）。在單執行緒執行器下，當服務在執行的那 2 秒內，感測器回呼完全不會被呼叫——資料全部堆積或遺失。這是 ROS 2 初學者最常遇到的效能問題。解法有三：一、改用 `MultiThreadedExecutor` 並搭配回呼群組（callback group）；二、把耗時的工作改成動作（29.4.3 節）而非服務；三、把耗時的工作拆到另一個節點。

```
import rclpy
from rclpy.node import Node
from rclpy.executors import MultiThreadedExecutor
from rclpy.callback_groups import (MutuallyExclusiveCallbackGroup,
                                   ReentrantCallbackGroup)

class MixedNode(Node):
    def __init__(self):
        super().__init__('mixed_node')
        # 群組一：感測器回呼，彼此互斥但可與其他群組並行
        sensor_group = MutuallyExclusiveCallbackGroup()
        # 群組二：耗時的服務，可重入
        service_group = ReentrantCallbackGroup()

        self.sub = self.create_subscription(
            LaserScan, 'scan', self.scan_cb, 10,
            callback_group=sensor_group)
        self.srv = self.create_service(
            Trigger, 'plan', self.plan_cb,
            callback_group=service_group)

def main():
    rclpy.init()
```

```
node = MixedNode()
executor = MultiThreadedExecutor(num_threads=4) # 多執行緒
executor.add_node(node)
executor.spin()
```

29.10.3 系統設計的原則

整合前面各節，設計一個 ROS 2 系統時應遵循五個原則：

- **單一職責**：每個節點只做一件事。若一個節點超過五百行，通常該拆了。
- **依頻率分層**：把不同時間尺度的工作放在不同的節點，避免慢的拖累快的。
- **使用標準訊息**：能用標準訊息就不要自訂——這決定了你能否使用社群的套件。
- **參數外部化**：所有可能需要調整的數值都應該是參數，而非寫死的常數。
- **設計失效行為**：每個節點都應該考慮「若上游沒有資料怎麼辦」。第 26.8.4 節的復原行為正是這個原則的體現。

29.11 由框架邁向智慧

29.11.1 本章的成果

本章建立了機器人系統的軟體基礎。你現在能夠：說明機器人軟體整合的困難與框架的價值、理解 ROS 2 的節點與話題架構、選擇適當的通訊模式、運用參數與啟動檔管理系統、以 TF2 處理座標變換與時間查詢、撰寫 URDF 描述機器人、設定 QoS、運用工具生態除錯、並設計符合即時性需求的系統架構。

我們也回收了兩個伏筆：**第 2 章**提到的軟體環境建置、以及**第 4 章**那份 `two\_link.urdf`——現在它能在 RViz 中真正動起來了。

更重要的是，本章展示了一個貫穿全書的主題的**軟體版本：解耦**。第 17 章用模型與回授解耦已知與未知、第 27 章用時間尺度解耦快慢動態、而本章用話題解耦了模組之間的依賴。**好的架構，就是讓每一部分都能獨立地改變。**

29.11.2 接下來：讓機器人變聰明

有了軟體框架，第五篇的其餘四章將處理「智慧」的各個面向：

第 30 章 (模擬與 Sim-to-Real)：在真實機器人上測試既昂貴又危險，而模擬能提供無限

的試錯機會。但模擬與現實之間有落差（第 14.7.3 節已預告過）——這一章要處理如何縮小它。屆時 29.3.2 節「模擬器與真實硬體可以互換」的設計將發揮關鍵作用。

第 31 章（機器學習與強化學習）：第 21 章的抓取、第 28 章的運動控制都已提過學習方法的威力。這一章將系統性地建立其原理，並回收第 18 章 Bellman 方程的伏筆——強化學習與最佳控制共用同一套數學。

第 32 章（大型模型與任務規劃）：近年最重大的變化。大型語言模型使機器人能理解自然語言的指令、並自行分解任務。但它們也帶來新的問題：如何確保生成的計畫是**可執行且安全的**？

第 33 章（具身人工智慧）：整個第五篇的收束。它要探討一個根本的問題——**智慧是否需要身體**？而答案將把我們帶回第 1 章的 Moravec 悖論。

換句話說：第 29 章讓各個模組**能夠對話**，第 30~33 章則讓它們**開始思考**。

完整計算例題

例題 29-1 系統架構設計

題目：設計一台室內服務機器人的 ROS 2 節點架構。需求為：以光達 SLAM 建圖與定位、用 RGB-D 相機辨識物體、能自主導航到指定地點、有一支六軸手臂能抓取物體、並提供語音指令介面。（a）列出所需的節點與其職責。（b）標明各節點之間的話題連接。（c）指出應使用服務與動作的地方。（d）分析各節點的頻率需求並分層。

解（a）：依「單一職責」原則，規劃十個節點：

表 29-9 服務機器人的節點設計

節點	職責	訂閱	發布
lidar_driver	讀取光達硬體	—	/scan
camera_driver	讀取 RGB-D 相機	—	/image_raw, /depth, /points
base_driver	底盤馬達控制與里程計	/cmd_vel	/odom, /joint_states
arm_driver	手臂關節控制	/arm_cmd	/joint_states
slam	建圖與定位	/scan, /odom	/map, /tf(map→odom)
object_detector	物體辨識與位姿估	/image_raw, /points	/detected_objects

節點	職責	訂閱	發布
	計		
navigation	路徑規劃與追隨	/map, /goal_pose, /scan	/cmd_vel
arm_planner	手臂運動規劃	/detected_objects	/arm_cmd
speech_interface	語音辨識與合成	—	/user_command
task_manager	任務協調與決策	/user_command, /detected_objects	/goal_pose (透過動作)

解 (b) : 資料流可分為三條主鏈 :

感知鏈 : **lidar_driver** → **slam** → **/map** → **navigation**
 視覺鏈 : **camera_driver** → **object_detector** → **arm_planner**
 控制鏈 : **navigation** → **/cmd_vel** → **base_driver**

三條並行的資料流。task_manager 位於最上層協調三者。

解 (c) : 三種通訊模式的分配 :

- **話題** : 所有的感測資料流 (/scan 、 /image_raw 、 /odom) 、 狀態回報 (/joint_states) 、 以及高頻的控制指令 (/cmd_vel) 。
- **服務** : 一次性且快速的操作——如「儲存目前的地圖」、「重設里程計」、「切換手臂的控制模式」。
- **動作** : 兩個長時間任務——導航到目標點 (Nav2 的 NavigateToPose) 與手臂執行抓取。兩者都需要進度回報與取消能力。

特別注意 task_manager 與 navigation 之間必須用動作而非服務。導航可能需要數十秒，若用服務，task_manager 會被完全阻塞，無法回應使用者的中止指令。

解 (d) : 依第 3 章的時間尺度金字塔分層 :

表 29-10 節點的頻率分層

層級	節點	頻率	即時性需求
硬體控制	base_driver, arm_driver (內部迴路)	1 kHz	硬即時 (在微控制器上)
感測擷取	lidar_driver, camera_driver	10~40 Hz	軟即時
狀態估測	slam	10~20 Hz	軟即時
感知處理	object_detector	5~15 Hz	無

層級	節點	頻率	即時性需求
規劃控制	navigation (局部)	10~20 Hz	軟即時
規劃控制	navigation (全域) , arm_planner	0.5~2 Hz	無
決策	task_manager, speech_interface	< 1 Hz	無

設計判讀：只有最底層的伺服控制需要硬即時，因此應在專用硬體上執行（第 7 章）；其餘各層都能在標準的 ROS 2 環境中運行。這個分層也決定了 QoS 的選擇——感測資料用 BEST_EFFORT、地圖用 TRANSIENT_LOCAL、控制指令用 RELIABLE。

例題 29-2 TF 樹與時間查詢

題目：某移動機器人在 $t = 10.000$ 秒時，相機拍到一個物體，其在相機座標系中的位置為 $(0.5, 0.1, 0.8)$ m。影像處理花了 80 毫秒，於 $t = 10.080$ 秒完成。機器人以 0.5 m/s 前進、0.2 rad/s 左轉。（a）說明應查詢哪個時刻的 TF。（b）估算若誤用「現在」時刻會產生多大的位置誤差。（c）說明 TF2 如何解決這個問題。（d）若 TF 的緩衝時間設為 0.05 秒，會發生什麼事？

解（a）：應查詢 $t = 10.000$ 秒——也就是影像被拍攝的時刻，而非處理完成的時刻。

理由：那個位置 $(0.5, 0.1, 0.8)$ 是相機在 $t = 10.000$ 時「看到」的。若要把它轉換到世界座標系，必須使用相機在當時的位姿。

解（b）：80 毫秒內機器人的移動量：

$$\text{平移} = 0.5 \text{ m/s} \times 0.08 \text{ s} = 0.04 \text{ m} = 4 \text{ 公分}$$

$$\text{旋轉} = 0.2 \text{ rad/s} \times 0.08 \text{ s} = 0.016 \text{ rad} = 0.92^\circ$$

機器人在處理期間的移動。

旋轉造成的額外誤差（物體距離約 0.95 m）：

$$\text{旋轉誤差} \approx 0.95 \times 0.016 = 0.0152 \text{ m} = 1.52 \text{ 公分}$$

角度誤差在距離上的放大。

$$\text{總誤差} \approx \sqrt{4^2 + 1.52^2} \approx 4.3 \text{ 公分}$$

若誤用「現在」的 TF，物體位置會偏差超過 4 公分。

工程意義：對抓取任務而言，4 公分的誤差是災難性的——夾爪會完全抓空（第 21 章的容差通常只有數公釐到 1 公分）。這說明了時間戳處理絕非細節，而是**成敗關鍵**。

解 (c)：TF2 儲存變換的**歷史紀錄** (29.6.3 節)。當呼叫 `buffer.transform(point, target_frame)` 時，它會：

1. 讀取 `point` 訊息中攜帶的時間戳 ($t = 10.000$)。
2. 在緩衝區中找出該時刻前後的兩筆變換紀錄。
3. 在兩者之間做內插，得到 $t = 10.000$ 時的精確變換。
4. 用這個變換計算結果。

這個內插機制是 TF2 最有價值的功能之一。它讓每個節點都不必自己維護歷史紀錄，也不必擔心資料到達的順序。

解 (d)：若緩衝時間只有 0.05 秒 (50 毫秒)，而查詢的時刻已經是 80 毫秒之前——該筆紀錄已經被丟棄了。

結果是拋出 `ExtrapolationException` 或 `LookupException`，轉換失敗。程式必須妥善處理這個例外 (如放棄這次偵測結果並記錄警告)，否則節點可能崩潰。

實務建議：TF 緩衝時間應設為**最長處理延遲的三到五倍**。預設的 10 秒對多數應用綽綽有餘；但若記憶體吃緊 (如嵌入式平台)，可以縮短——但絕不應短於系統中最慢的處理鏈。

例題 29-3 QoS 設定與頻寬估算

題目：某機器人系統包含：640×480 的 RGB 影像 (30 Hz)、640×480 的深度圖 (30 Hz)、二維光達 (每圈 720 點，10 Hz)、以及 200 Hz 的關節狀態 (12 個關節)。(a) 估算各話題的頻寬需求。(b) 為各話題選擇適當的 QoS。(c) 若透過 100 Mbps 的網路傳輸，判斷是否可行。(d) 提出三個降低頻寬的策略。

解 (a)：逐一計算：

RGB 影像： $640 \times 480 \times 3 \text{ bytes} \times 30 \text{ Hz} = 27.6 \text{ MB/s} = 221 \text{ Mbps}$

未壓縮的影像頻寬極大。

深度圖： $640 \times 480 \times 2 \text{ bytes} \times 30 \text{ Hz} = 18.4 \text{ MB/s} = 147 \text{ Mbps}$

深度圖每點 16 位元。

光達： $720 \text{ 點} \times 4 \text{ bytes} \times 10 \text{ Hz} = 28.8 \text{ KB/s} = 0.23 \text{ Mbps}$

光達的頻寬極小。

關節狀態：12 關節 × 3 值 × 8 bytes × 200 Hz = 57.6 KB/s = 0.46 Mbps

同樣極小。

總計 ≈ 369 Mbps

主要由影像主導，佔 99.8%。

解 (b)：QoS 的選擇：

表 29-11 各話題的 QoS 建議

話題	可靠度	歷史深度	持久性	理由
/image_raw	BEST_EFFORT	KEEP_LAST(1)	VOLATILE	高頻串流，掉幀無妨，只要最新的
/depth	BEST_EFFORT	KEEP_LAST(1)	VOLATILE	同上
/scan	BEST_EFFORT	KEEP_LAST(5)	VOLATILE	感測資料，保留少量歷史
/joint_states	RELIABLE	KEEP_LAST(10)	VOLATILE	狀態資料，不應遺失
/map	RELIABLE	KEEP_LAST(1)	TRANSIENT_LOCAL	晚加入的節點也要取得
/cmd_vel	RELIABLE	KEEP_LAST(1)	VOLATILE	指令必須送達，但只要最新的

解 (c)：369 Mbps 遠超過 100 Mbps 的網路容量——不可行。實際上連 1 Gbps 網路也會相當吃緊（佔用 37%），且會與其他流量競爭。

解 (d)：三個降低頻寬的策略：

- **策略一：壓縮。**使用 `image\_transport` 的壓縮傳輸（JPEG 或 PNG）。JPEG 通常能達到 10~20 倍的壓縮率，把 221 Mbps 降到約 15 Mbps。這是最有效的一步。
- **策略二：就地處理。**把影像處理節點放在機器人本體上，只傳輸處理後的結果（如偵測到的物體位姿，僅數百 bytes）。這是 29.10.3 節「依頻率分層」的延伸——頻寬也應該分層。
- **策略三：降低解析度或頻率。**若辨識任務不需要 30 Hz，降到 10 Hz 就能省下三分之二。同樣地，320×240 的解析度對多數辨識任務已經足夠。

實務上三者並用：本體上做高頻的處理、只把低頻的結果傳出去、且必要的影像串流一律壓縮。這也解釋了為何機器人上通常會裝一台效能不錯的電腦（第 3 章的運算平台選擇）——把資料留在本地處理，遠比傳出去便宜。

程式範例：一個完整的 ROS 2 套件

以下是一個完整可運行的 ROS 2 套件，包含一個簡單的速度控制器節點——它訂閱目標位置、發布速度指令，並示範參數、TF 查詢與生命週期管理。

```
# =====
# 檔案結構
# my_robot/
#   ├── package.xml
#   ├── setup.py
#   └── my_robot/
#       ├── __init__.py
#       └── goal_controller.py
#   ├── launch/
#       └── controller.launch.py
#   └── config/
#       └── params.yaml
# =====

# ----- my_robot/goal_controller.py -----
import math
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy
from geometry_msgs.msg import Twist, PoseStamped
from nav_msgs.msg import Odometry
from tf2_ros import Buffer, TransformListener
from rcl_interfaces.msg import SetParametersResult

class GoalController(Node):
    """簡單的目標點控制器：以 Pure Pursuit 概念導向目標 (第 23.9.3 節)"""

    def __init__(self):
        super().__init__('goal_controller')

        # ----- 參數宣告 (29.5.1 節) -----
        self.declare_parameter('k_linear', 0.5)
        self.declare_parameter('k_angular', 1.5)
        self.declare_parameter('max_linear', 0.4)
        self.declare_parameter('max_angular', 1.0)
        self.declare_parameter('goal_tolerance', 0.10)
        self.declare_parameter('control_rate', 20.0)
        self._load_params()
        self.add_on_set_parameters_callback(self.on_param_change)

        # ----- TF (29.6 節) -----
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)

        # ----- 通訊 (29.3、29.8 節) -----
        cmd_qos = QoSProfile(depth=1,
```

```

        reliability=ReliabilityPolicy.RELIABLE)
self.cmd_pub = self.create_publisher(Twist, 'cmd_vel', cmd_qos)

self.goal_sub = self.create_subscription(
    PoseStamped, 'goal_pose', self.goal_callback, 10)
self.odom_sub = self.create_subscription(
    Odometry, 'odom', self.odom_callback, 10)

# ----- 狀態 -----
self.goal = None
self.pose = None

# ----- 控制迴路 -----
period = 1.0 / self.get_parameter('control_rate').value
self.timer = self.create_timer(period, self.control_loop)

self.get_logger().info(
    f'目標控制器已啟動 ( {1/period:.0f} Hz ) ')

# ===== 參數處理 =====
def _load_params(self):
    g = lambda n: self.get_parameter(n).value
    self.k_lin = g('k_linear')
    self.k_ang = g('k_angular')
    self.v_max = g('max_linear')
    self.w_max = g('max_angular')
    self.tol = g('goal_tolerance')

def on_param_change(self, params):
    """支援執行中動態調參"""
    for p in params:
        self.get_logger().info(f'參數 {p.name} → {p.value}')
    # 先讓框架寫入新值，再重新載入
    result = SetParametersResult(successful=True)
    self.create_timer(0.0, self._reload_once)
    return result

def _reload_once(self):
    self._load_params()
    self.destroy_timer(self.timer) # 避免重複觸發

# ===== 回呼 =====
def goal_callback(self, msg):
    self.goal = msg
    self.get_logger().info(
        f'收到新目標: ({msg.pose.position.x:.2f}, '
        f'{msg.pose.position.y:.2f}) in {msg.header.frame_id}')

def odom_callback(self, msg):
    self.pose = msg.pose.pose

```

```

# ===== 控制迴路 =====
def control_loop(self):
    if self.goal is None or self.pose is None:
        return

    # 把目標轉換到機器人座標系 ( 29.6.3 節：用目標自身的時間戳 )
    try:
        goal_in_base = self.tf_buffer.transform(
            self.goal, 'base_link',
            timeout=rclpy.duration.Duration(seconds=0.1))
    except Exception as e:
        self.get_logger().warn(f'TF 轉換失敗: {e}', throttle_duration_sec=2.0)
        return

    dx = goal_in_base.pose.position.x
    dy = goal_in_base.pose.position.y
    dist = math.hypot(dx, dy)

    cmd = Twist()
    if dist < self.tol:
        # 到達目標 · 停止
        self.cmd_pub.publish(cmd)      # 全零
        self.get_logger().info('已抵達目標', once=True)
        self.goal = None
        return

    # 簡化的 Pure Pursuit：朝目標方向轉、同時前進
    alpha = math.atan2(dy, dx)
    cmd.angular.z = max(-self.w_max,
                        min(self.w_max, self.k_ang * alpha))
    # 角度誤差大時減速 ( 避免走出大弧線 )
    speed_scale = max(0.0, math.cos(alpha))
    cmd.linear.x = max(0.0, min(self.v_max,
                                self.k_lin * dist * speed_scale))
    self.cmd_pub.publish(cmd)

def main(args=None):
    rclpy.init(args=args)
    node = GoalController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        # 安全停止：離開前送出零速度 ( 重要的失效處理！ )
        node.cmd_pub.publish(Twist())
        node.destroy_node()
        rclpy.shutdown()

```

```
if __name__ == '__main__':
    main()
```

```
# ----- config/params.yaml -----
goal_controller:
  ros__parameters:
    k_linear: 0.6
    k_angular: 1.8
    max_linear: 0.35
    max_angular: 1.2
    goal_tolerance: 0.08
    control_rate: 20.0

# ----- launch/controller.launch.py -----
from launch import LaunchDescription
from launch_ros.actions import Node
import os
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    params = os.path.join(
        get_package_share_directory('my_robot'), 'config', 'params.yaml')
    return LaunchDescription([
        Node(package='my_robot', executable='goal_controller',
            name='goal_controller', parameters=[params],
            output='screen'),
    ])

# ----- setup.py 的關鍵部分 -----
# entry_points={
#     'console_scripts': [
#         'goal_controller = my_robot.goal_controller:main',
#     ],
# },
# data_files=[
#     ('share/my_robot/launch', ['launch/controller.launch.py']),
#     ('share/my_robot/config', ['config/params.yaml']),
# ],
```

四個值得注意的設計：

- **參數全部外部化**：所有增益與限制都是參數，可用 `rqt_reconfigure` 即時調整（29.5.1 節）。
- **TF 使用目標自身的時間戳**：而非「現在」，這正是例題 29-2 的正確作法。
- **離開前送出零速度**：`finally` 區塊中的 `publish(Twist())` 確保程式中斷時機器人會停止——這是失效導向安全（第 7.9 節）的軟體實現。
- **警告訊息使用 throttle**：`throttle\_duration\_sec=2.0` 避免 TF 失敗時每秒印出二十筆相同的警告，淹沒 log。

請試著修改：（一）把控制律改為第 23.9.3 節完整的 Pure Pursuit（含前視距離）。（二）加入一個服務，讓外部能查詢「距離目標還有多遠」。（三）把導航改為動作介面，支援進度回報與取消。

實作：建立你的第一個 ROS 2 系統

本章實作將前面各節整合為一個可運行的系統。建議在 Ubuntu 22.04 或 24.04 上安裝 ROS 2（Humble 或 Jazzy），Windows 使用者可用 WSL2。

1. **環境建置**：安裝 ROS 2、設定環境變數、執行 `ros2 doctor` 確認無誤。這回收了第 2.13.4 節的伏筆。
2. **Hello ROS**：執行內建的 talker/listener 範例，用 `ros2 topic list` 與 `ros2 topic echo` 觀察話題。用 `rqt_graph` 看兩者的連接。
3. **建立套件**：用 `ros2 pkg create` 建立自己的套件，撰寫 29.3.4 節的發布者與訂閱者並執行。
4. **參數與啟動檔**：把節點的參數外部化，撰寫 YAML 與啟動檔，用一個指令啟動整個系統。
5. **動態調參**：加入參數回呼，用 `ros2 param set` 或 `rqt_reconfigure` 在執行中改變數值，觀察行為變化。
6. **URDF 與 RViz**：把第 4 章的 `two_link.urdf` 擴充（加入慣量與碰撞模型），用 `robot_state_publisher` 與 `joint_state_publisher_gui` 在 RViz 中拖動關節——這回收了第 4 章的伏筆。
7. **TF 實驗**：用 `ros2 run tf2_tools view_frames` 產生 TF 樹的圖形。加入一個靜態變換發布者（模擬相機的安裝位置），驗證變換是否正確。
8. **QoS 實驗**：故意讓發布者用 BEST_EFFORT、訂閱者用 RELIABLE，觀察「什麼都收不到」的現象。用 `ros2 topic info -v` 診斷，再修正。這個經驗極為寶貴。
9. **rosvbag 錄製與重放**：錄下一段資料，用 `ros2 bag info` 查看，再以半速重放並在 RViz 中觀察。
10. **阻塞實驗**：撰寫一個節點，同時有 10 Hz 的訂閱與一個耗時 2 秒的服務。用單執行緒執行器觀察訂閱被阻塞的現象，再改用多執行緒執行器與回呼群組解決。
11. **（進階）整合模擬器**：安裝 Gazebo（或 TurtleBot 模擬套件），讓你的控制器節點驅動

模擬中的機器人。驗證 29.3.2 節「模擬與真實可互換」的設計——同一份控制器程式碼不需修改。

表 29-12 ROS 2 實作記錄表 (前兩列為範例)

實作項目	遇到的問題	診斷方式	解法
發布訂閱	訂閱者收不到訊息	ros2 topic info -v	QoS 不相容，改為一致
TF 查詢	ExtrapolationException	檢查時間戳	改用訊息自身的時間戳
(自行填寫其餘項目)			

國中小也看得懂：園遊會的公布欄

想像你們班要辦一場園遊會，有四個小組：一組負責做食物、一組負責收錢、一組負責招呼客人、一組負責整理場地。

問題來了：這四組要怎麼溝通？

最笨的方法是「每一組都要認識其他三組」——做食物的要知道收錢的是誰、收錢的要知道整理場地的是誰……四組要建立六條聯絡管道。如果有十組呢？就要四十五條！光是記住誰要跟誰講話，就會累死。

聰明的方法是：設一個公布欄。做食物的把「食物好了！」貼在公布欄上，需要知道的人自己去看。收錢的把「今天賣了多少錢」貼上去，老師想看就去看。

這樣一來，每一組都不需要認識其他組，只要知道「該把消息貼在哪個公布欄」就好。而且新加入的小組也很簡單——直接看公布欄就行了。

機器人的軟體也是這樣運作的！這個「公布欄」在機器人的世界裡叫做「話題」(topic)。

舉個例子：機器人的相機看到一個杯子，它就把「我看到杯子在這個位置」貼到公布欄上。手臂的程式看到這個消息，就伸手去抓。相機完全不需要知道手臂是誰寫的、用什麼程式語言、甚至不需要在同一台電腦上。

這個方法有多厲害呢？因為全世界的科學家都用同一套公布欄的規則，所以你可以直接下載別人寫好的程式來用。想讓機器人畫地圖？下載一個現成的就好，十分鐘就跑起來了——而這件事在二十年前，一個研究生要花兩年才做得出來！

叫獸提醒與常見錯誤

- 誤區一：以為 ROS 是作業系統。它是跑在 Linux 之上的中介層，負責通訊、工具與標準介面。
- 誤區二：初期覺得 ROS 太複雜而自己寫一套。你會逐步重新發明它的每一個功能，且無法使用社群的任何套件。
- 誤區三：一個節點做太多事。應遵循單一職責原則，超過五百行通常就該拆分。
- 誤區四：用服務執行長時間任務。服務會阻塞呼叫者，長任務應改用動作（可回報進度與取消）。
- 誤區五：忽略 QoS 不相容。發布者與訂閱者的 QoS 不相容時「完全不會連線且不報錯」，應用 `ros2 topic info -v` 診斷。
- 誤區六：TF 查詢使用「現在」時刻。應使用資料自身的時間戳，否則處理延遲會造成顯著的位置誤差（例題 29-2）。
- 誤區七：把靜態變換當成動態的持續發布。相機的安裝位置不會變，不需要每秒發布五十次。
- 誤區八：參數寫死在程式碼中。所有可能調整的數值都應外部化，否則調參要重新編譯。
- 誤區九：在單執行緒執行器中執行耗時的回呼。這會阻塞其他所有回呼，應改用多執行緒與回呼群組。
- 誤區十：未壓縮就在網路上傳輸影像。單一路 $640 \times 480 @ 30\text{fps}$ 的未壓縮影像就需要 221 Mbps。
- 誤區十一：把 RViz 當成模擬器。RViz 只負責顯示，不模擬任何物理——世界是由 Gazebo 提供的。
- 誤區十二：程式結束時未送出停止指令。應在 `finally` 區塊中發布零速度，這是失效導向安全的軟體實現。

課堂討論

1. 開頭故事中四組學生的成果無法整合。請討論：除了通訊機制之外，還需要哪些「約定」才能讓不同團隊的成果互通？（提示：座標系、單位、時間基準、命名規範）
2. 話題的發布—訂閱模式使模組完全解耦。請討論這個設計的代價——在什麼情況下，這種

解耦反而造成困難？

3. $\text{map} \rightarrow \text{odom} \rightarrow \text{base_link}$ 的三層設計，把「連續但漂移」與「不漂移但跳變」分離。請討論這個思維在本書其他章節的對應（提示：第 24 章的互補濾波、第 17 章的前饋加回授）。
4. 例題 29-3 顯示影像佔了 99.8% 的頻寬。請討論「就地處理」這個策略對系統架構的影響，以及它與雲端運算的取捨。
5. ROS 2 用 DDS 取代了 ROS 1 的自訂通訊。請討論「採用工業標準」相對於「自行開發」的優缺點，並舉出本書中其他類似的取捨。

章末摘要

- 機器人軟體的核心困難是整合：介面組合爆炸、異質性、並行與時間、以及除錯困難。
- 框架提供四項功能：通訊機制、標準介面、執行管理與工具生態。
- ROS 不是作業系統，而是跑在 OS 之上的中介層。
- ROS 2 相對 ROS 1 的三個關鍵改變：改用 DDS（去中心化、QoS、安全）、支援即時性、生命週期管理。
- 節點是基本執行單位，應遵循單一職責原則。
- 話題採發布—訂閱模式，發布者與訂閱者完全解耦，這使模擬器與真實硬體可以互換。
- 標準訊息類型是社群能共用套件的關鍵——所有底盤都用 Twist、所有光達都用 LaserScan。
- 三種通訊模式：話題（非同步資料流）、服務（同步請求回應）、動作（長時間任務含進度與取消）。
- 選擇模式的三個問題：需要回應嗎、會很快回應嗎、需要進度或取消嗎。
- 參數使設定與程式碼分離，並支援執行中動態調整——這對調參極為重要。
- 啟動檔一次啟動多個節點，並支援話題重新映射與條件邏輯。
- TF2 管理座標變換樹，其最重要的功能是帶時間戳的查詢與內插。
- $\text{map} \rightarrow \text{odom} \rightarrow \text{base_link}$ 的三層設計，分離了「連續但漂移」與「不漂移但跳變」兩種特性。
- URDF 描述機器人的運動學、動力學與幾何；robot_state_publisher 用正向運動學把它轉為

TF。

- Xacro 為 URDF 加上巨集與運算，避免重複並使慣量能隨尺寸自動更新。
- QoS 讓每個話題各自設定可靠度、持久性與歷史深度；不相容時會完全不連線且不報錯。
- 工具生態是 ROS 的核心價值：ros2 CLI 診斷、RViz 視覺化、rosviz 記錄重放、rqt 圖形化除錯。
- RViz 只負責顯示不模擬物理；Gazebo 才是模擬器。
- 即時性需要作業系統、中介軟體與應用層三個層次的配合；實務上採分層混合架構。
- 單執行緒執行器中的耗時回呼會阻塞其他回呼，應改用多執行緒與回呼群組。

觀念題與計算題

觀念題

1. 說明機器人軟體整合困難的四個來源，並解釋為何介面數量會平方成長。
2. 列舉機器人框架提供的四項核心功能及其在 ROS 2 中的對應。
3. 為何說「ROS 不是作業系統」？它與真正的作業系統如何分工？
4. 列舉 ROS 1 的四個限制，以及 ROS 2 針對它們所做的三個關鍵改變。
5. 說明採用 DDS 帶來的三個好處。
6. 說明發布—訂閱模式的解耦特性，以及它帶來的三個好處。
7. 為何標準訊息類型對社群生態如此重要？
8. 比較話題、服務與動作三種通訊模式，並說明選擇的三個判準。
9. 為何不應該用服務執行長時間任務？應改用什麼？
10. 說明參數機制的價值，以及動態調參對哪些章節的工作特別有幫助。
11. 說明啟動檔的三個機制：參數載入、話題重新映射、啟動參數。
12. 說明 TF2 解決的三個困難，以及變換樹的兩個結構規則。
13. 解釋 map → odom → base_link 三層設計的分工與其巧妙之處。
14. 說明 TF2 的時間查詢與內插機制，以及它為何是時間戳問題的系統級解法。
15. 說明 URDF 描述的三類資訊，以及 robot_state_publisher 的運作方式。

16. 說明 Xacro 相對於純 URDF 的優勢，並舉例說明它如何避免錯誤。
17. 列舉四個主要的 QoS 設定選項，並各舉一個適用情境。
18. 說明 QoS 不相容的現象與診斷方式，以及為何它是常見的除錯陷阱。
19. 說明 RViz 與 Gazebo 的角色差異。
20. 說明 ROS 2 達成即時性所需的三個層次，以及實務上的分層混合架構。

計算題

1. 某系統有 8 個模組需要互相通訊。(a) 若採用點對點的直接介面，最壞情況需要幾條介面？(b) 若採用發布訂閱模式，每個模組只需知道話題名稱，共需幾個「約定」？(c) 若模組數增至 20，重新計算兩者。(d) 由此說明框架的價值。
2. 某機器人以 0.8 m/s 前進、0.3 rad/s 轉向。相機在 $t = 5.000$ s 拍到距離 1.2 m 的物體，處理耗時 120 ms。(a) 求處理期間的平移量與旋轉量。(b) 求旋轉造成的橫向誤差。(c) 求總位置誤差。(d) 判斷此誤差對抓取任務（容差 8 mm）是否可接受。
3. 某系統的話題包含：1280×720 RGB 影像（15 Hz）、三維點雲（每幀 30 萬點、每點 16 bytes、10 Hz）、IMU（100 Hz、9 個 float64）。(a) 分別計算三者的頻寬（Mbps）。(b) 求總頻寬。(c) 若使用 1 Gbps 網路，計算佔用率。(d) 若影像以 JPEG 壓縮 15 倍、點雲改為就地處理只傳結果，重新計算總頻寬。
4. 某節點有三個回呼：感測器訂閱（20 Hz、每次處理 5 ms）、控制定時器（50 Hz、每次 3 ms）、規劃服務（每分鐘一次、每次 800 ms）。(a) 計算前兩者的 CPU 佔用率。(b) 在單執行緒執行器下，規劃服務執行時會遺失多少次感測器回呼與控制回呼？(c) 說明這對系統的影響。(d) 提出兩種解法並比較。

動手做與專題延伸

本章實作：機器人軟體框架

把前面各章的演算法，整合到 ROS 2 的架構中。這是全書程式的整合起點。

- 階段一：建立工作空間與套件，實作基本的發布訂閱節點。
- 階段二：把第 23 章的里程計與第 23.9.3 節的 Pure Pursuit 包裝成 ROS 2 節點。
- 階段三：把第 24 章的 Kalman 濾波器包裝成節點，訂閱多個感測話題並發布融合後的位姿。

- **階段四**：撰寫完整的 URDF (含慣量與碰撞)，在 RViz 中視覺化並驗證 TF 樹。
- **階段五**：撰寫啟動檔，一個指令啟動整個系統；並用 rosbag 錄製資料供離線分析。

專題延伸

- **系統架構圖的繪製與分析**：為一台你設計的機器人 (可用例題 29-1 的服務機器人) 繪製完整的節點與話題連接圖。標註每個話題的訊息類型、頻率、QoS 設定與估計頻寬。接著用 rqt_graph 產生實際的圖，與你的設計圖比對——你會發現真實系統往往比設計時複雜得多。討論哪些連接是必要的、哪些可以簡化。
- **時間戳誤差的實驗驗證**：撰寫兩個版本的座標轉換節點——一個使用資料自身的時間戳、一個使用「現在」。讓機器人 (或模擬器) 以不同速度移動，並人為加入處理延遲 (如 sleep 100 ms)。量測兩者的位置誤差差異，繪出「速度 vs 誤差」曲線，驗證例題 29-2 的計算。這個實驗能讓「時間戳很重要」從一句話變成一個具體的數字。
- **進階：從零建立一個最小的通訊框架**。用 Python 的 socket 或 ZeroMQ，自行實作一個極簡的發布訂閱機制 (約兩百行)。接著逐步加入需求：多對多連線、訊息序列化、話題的自動探索、時間戳、以及跨機器通訊。**記錄每加入一項功能所需的程式碼量與遇到的問題**。這個練習的目的不是要取代 ROS，而是親身體會 ROS 為你解決了多少問題——完成後你會對 29.1.2 節的知識框有完全不同的體會。

參考文獻與延伸閱讀

- ROS 2 官方文件 (docs.ros.org)：最權威且持續更新的資料來源，含完整的教學與 API 參考。
- Macenski, S. et al., "Robot Operating System 2: Design, Architecture, and Uses In The Wild," Science Robotics, 2022 (ROS 2 的設計理念與實際應用回顧)。
- Quigley, M. et al., "ROS: An Open-Source Robot Operating System," ICRA Workshop on Open Source Software, 2009 (ROS 的原始論文)。
- Quigley, M., Gerkey, B. and Smart, W. D., Programming Robots with ROS, O'Reilly (ROS 程式設計的實務教材)。
- Newman, W. S., A Systematic Approach to Learning Robot Programming with ROS, CRC Press (系統性的 ROS 教材，含大量範例)。

- Foote, T., "tf: The Transform Library," TePRA 2013 (TF 系統的設計論文) 。
- OMG DDS 標準文件：ROS 2 底層通訊中介軟體的規格。
- Macenski, S. et al., "The Marathon 2: A Navigation System," IROS 2020 (Nav2 的設計，本章分層架構的實例) 。
- ROS 2 Design 網站 (design.ros2.org)：記錄了 ROS 2 各項設計決策的理由，對理解「為何這樣設計」極有幫助。
- Awesome ROS 2 ([GitHub](#))：社群整理的 ROS 2 資源清單，含套件、教學與工具。

第 30 章

模擬與 Sim-to-Real

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩格對照：上格是模擬畫面中機械手臂流暢抓起物體，標註「模擬成功率 98%」；下格是實驗室的真實照片——手臂抓空、物件倒了一桌，標註「實機成功率 20%」。右半為領域隨機化的核心示意：左側「傳統思路」是兩個相距的小圓點（SIM 與 REAL）以雙向箭頭標註「努力縮小落差」；右側「領域隨機化」是一個涵蓋範圍極大的橢圓（內部散布數百個代表不同參數組合的小點），REAL 落在其中，標註「真實世界只是其中一個樣本」。下緣文字：模擬不必更真，策略必須更穩。

叫獸開講 「模擬裡 98% 的成功率，為什麼到實機只剩兩成？」

小妤花了兩週，在模擬器裡訓練出一個抓取策略。成功率 98%——她把畫面投影出來，機械手臂流暢地抓起各種形狀的物體，連歪斜的圓柱都能穩穩夾住。

「很漂亮。」叫獸點頭，「現在搬到實機上跑。」

三十分鐘後，實驗室裡一片狼藉。手臂抓空了七次、把三個物件推倒、有兩次夾爪碰到桌面觸發了過載保護。實測成功率：約兩成。

「98% 變成 20%。」小妤盯著螢幕，「模擬器是不是寫錯了？」

「模擬器沒錯。」叫獸拿起一個剛才被推倒的塑膠瓶，「錯的是你以為模擬器該是什麼。」

他把瓶子放在桌上輕輕一推，瓶子滑了一小段停下來。「你的模擬裡，這個瓶子的摩擦係數是多少？」「0.6，我查表填的。」

「那實際上呢？」叫獸把瓶子倒過來，「這一面沾了點灰，摩擦係數可能只有 0.3。而且桌面有一層薄薄的油膜——你上禮拜擦桌子留下的。」

小翰插話：「所以只要把摩擦係數量準一點就好了？」

「那你要量幾個？」叫獸反問，「每一個物體、每一面、每一種濕度、每一次擦桌子之後。而且還有質量分布、表面彈性、夾爪墊的老化程度、馬達的溫度……你永遠追不完。」

他在白板上寫下一行字：模擬永遠不等於現實，這不是缺陷，而是定義。

「所以正確的問題不是『怎麼讓模擬更真』，而是『怎麼讓學到的東西不依賴那些不準的細節』。」叫獸停頓一下，「如果你的策略只在摩擦係數恰好是 0.6 時才有效，那它本來就是脆弱的——就算模擬器完美無缺，換一個沾了灰的瓶子它還是會失敗。」

最後他寫下今天的核心：「與其追求完美的模擬，不如刻意把模擬弄亂——每次訓練都隨機改變摩擦、質量、光線、延遲。逼你的策略學會在**所有可能的世界**裡都能運作。這樣它到了真實世界，只不過是又多了一個它沒見過、但也不特別的世界罷了。」

本章為什麼重要

模擬是現代機器人開發不可或缺的環節。它的價值來自三個現實的限制：真實機器人昂貴

(一台協作手臂數十萬元、一台四足機器人上百萬元)、**測試危險** (第 27 章的無人機會墜落、第 28 章的腿式機器人會摔壞)、**時間緩慢** (一次抓取嘗試需數秒，而第 31 章的強化學習需要數百萬次)。

模擬把這三個限制通通解除：可以同時跑一千個環境、可以讓機器人摔一萬次、可以用比實時快百倍的速度執行。**這正是近年強化學習能在機器人上取得突破的物質基礎** (第 28.9 節的 ANYmal、第 21.9.3 節的抓取資料集)。

但模擬有一個根本的限制——**它永遠不等於現實**。這個落差稱為 **Sim-to-Real Gap**，是本章的核心議題。第 14.7.3 節已經預告過接觸模擬的困難，第 21.2.3 節提過摩擦係數的不確定性，本章要系統性地處理：**落差從哪裡來、如何量化、以及如何跨越**。

本章的核心洞見與全書多處呼應：**不要追求消除不確定性，而要設計對不確定性不敏感的系統**。這與第 20 章的柔順控制、第 21 章的軟性夾爪、第 24 章的機率估測，是同一種工程哲學的不同體現。

學習目標

- 能說明模擬在機器人開發中的三個價值與其限制。
- 能描述物理引擎的四個核心模組及其計算瓶頸。
- 能區分不同層次的模擬保真度並依需求選擇。
- 能分析 Sim-to-Real Gap 的五個主要來源。
- 能說明領域隨機化的原理、實作與參數選擇。
- 能區分領域隨機化與領域自適應兩種策略。
- 能說明系統辨識如何縮小落差。
- 能比較常見模擬器的特性並為特定任務選擇。
- 能說明大規模平行模擬的架構與其對學習的意義。
- 能設計模擬到實機的漸進式驗證流程。
- 能說明數位孿生的概念及其與模擬的差異。
- 能評估一個模擬環境是否足以支持特定的開發目標。

先備知識

第 14 章的正向動力學與數值積分（模擬器的核心）、第 14.7 節的接觸模型、第 21.2 節的摩擦錐、第 29 章的 ROS 2 架構（模擬器與真實硬體的可互換性）、第 22 章的相機模型（視覺模擬）。

本章的數學不深，但**觀念的轉換很關鍵**——必須從「模擬要多真才夠」轉向「哪些不真無所謂」。建議讀完 30.4 節後停下來思考自己過去的專案，是否曾誤以為「模擬對了就等於做對了」。

核心名詞中英對照

表 30-1 本章核心名詞

中文	英文	說明
模擬器	Simulator	以計算重現物理與感測的軟體
物理引擎	Physics Engine	計算剛體動力學與接觸的核心模組
模擬到現實落差	Sim-to-Real Gap	模擬行為與真實行為的差異
保真度	Fidelity	模擬與現實相符的程度
領域隨機化	Domain Randomization	訓練時隨機改變模擬參數的策略
領域自適應	Domain Adaptation	使模型適應目標領域的技術
系統辨識	System Identification	由實測資料估計模型參數
即時因子	Real-Time Factor, RTF	模擬速度與真實時間的比值
渲染	Rendering	產生影像的圖形計算
光線追蹤	Ray Tracing	以模擬光線路徑產生逼真影像
碰撞偵測	Collision Detection	判斷物體是否接觸
約束求解器	Constraint Solver	計算接觸與關節約束力的模組
軟接觸	Soft Contact	允許微小穿透的接觸模型
數位孿生	Digital Twin	與實體同步的虛擬複本
硬體在環	Hardware-in-the-Loop, HIL	真實硬體接入模擬迴路
場景隨機化	Scene Randomization	隨機改變環境外觀與配置
合成資料	Synthetic Data	由模擬產生的訓練資料
漸進式驗證	Progressive Validation	由模擬逐步過渡到實機的流程

30.1 為何需要模擬

30.1.1 三個現實的限制

模擬的價值，來自真實機器人開發的三個限制。

限制一：成本。一台協作機械手臂數十萬元、一台四足機器人上百萬元、一台自駕測試車更是千萬等級。而研究通常需要**多台**——不同的構型、不同的配置、以及備品。模擬則可以無限複製。

限制二：風險。第 27 章的無人機失控會墜毀、第 28 章的腿式機器人跌倒會損壞關節、第 20 章的力控制參數錯誤會壓壞工件。而學習演算法的本質就是「試錯」——第 31 章的強化學習在初期幾乎全是失敗的動作。在真機上訓練，代價無法承受。

限制三：時間。這是最根本的限制。一次抓取嘗試約需 5 秒，而訓練一個抓取策略可能需要數十萬次嘗試：

100,000 次 × 5 秒 ≈ 139 小時 (不眠不休近 6 天)

而且還要人工重置場景。在真機上，這幾乎不可行。

模擬把這三個限制通通解除：

表 30-2 模擬解除的三個限制

限制	真實機器人	模擬
成本	數十萬~千萬元/台	近乎零 (只需算力)
風險	損壞、傷人	無 (重來即可)
速度	實時 (1×)	可達 100~1000×
平行度	1 台	可達數千個環境
重置	需人工	自動、瞬間
可觀測性	受感測器限制	全知 (所有狀態皆可讀)

最後一列「可觀測性」常被低估。在模擬中我們知道每一個物體的精確位置、每一個接觸點的力、每一個關節的真實速度——這些在真機上要嘛量不到、要嘛有雜訊。這使得**除錯與分析變得容易得多**，也使第 28.9.3 節的「教師—學生」訓練成為可能。

30.1.2 模擬的四種用途

模擬在機器人開發中扮演四種不同的角色，需求各不相同：

- **演算法開發與除錯**：驗證程式邏輯是否正確。**不需要高保真度**——只要能跑起來、能看見行為即可。
- **控制器驗證**：測試控制器在各種情況下的表現。**需要動力學準確**，但視覺可以很粗糙。
- **學習訓練**：產生大量的訓練資料。**需要速度與多樣性**，而非單一環境的精確度。
- **視覺感知訓練**：產生標註好的影像資料。**需要視覺逼真**，但物理可以簡化。

這四種用途對「保真度」的需求截然不同（30.3 節）。一個常見的錯誤是不分用途地追求「最真實的模擬」——這既浪費資源，也往往找錯了方向。

30.2 物理引擎的架構

30.2.1 四個核心模組

第 14.8.1 節已介紹過物理引擎的組成，此處深入其計算特性。一個模擬步驟包含四個階段：

表 30-3 物理引擎的四個模組

模組	任務	計算複雜度	通常的瓶頸
碰撞偵測	找出哪些物體接觸、接觸點在哪	$O(n \log n) \sim O(n^2)$	常是最大瓶頸
約束求解	計算接觸力與關節約束力	迭代，視接觸數而定	接觸多時很慢
正向動力學	由力求加速度（第 14.5 節）	$O(n) \sim O(n^3)$	自由度高時
數值積分	推進到下一個時刻（第 14.6 節）	$O(n)$	很少是瓶頸

碰撞偵測通常是最大的瓶頸，這與直覺不符——多數人以為動力學計算最耗時。原因是碰撞偵測需要處理大量的幾何配對，且複雜的網格模型（mesh）的檢測成本極高。

因應之道是使用簡化的碰撞幾何：用球、膠囊、盒子等基本形狀近似複雜的模型。這正是第 29.7.1 節 URDF 中「視覺模型」與「碰撞模型」分開的原因——視覺可以精細（只影響顯示），碰撞則應盡量簡單。

◎ 一個常見的效能陷阱

許多初學者直接把 CAD 匯出的高精度網格（數萬個三角面）當成碰撞模型。結果是模擬慢得無法使用——即時因子可能只有 0.05（模擬一秒要花二十秒）。正確的作法是為碰撞另外建立簡化模型：一根連桿用一個膠囊、一個底盤用一個盒子、一個夾爪用兩個小盒子。這通常能把速度提升一

到兩個數量級，而對物理行為的影響微乎其微。

30.2.2 時間步長的取捨

第 14.6.3 節討論過步長的選擇，在模擬器中這個取捨更為關鍵：

步長太大 → 穿透、彈跳、發散

步長太小 → 模擬太慢

而含接觸的模擬需要極小的步長 ($0.1 \sim 1 \text{ ms}$)，這是模擬速度的主要限制。

兩個常用的技巧：

- 子步進 (substepping)：物理以小步長計算 (如 1 ms)，但只在較大的間隔 (如 10 ms) 回報結果給控制器。這使物理準確而通訊負擔小。
- 分離的時間尺度：物理 1 kHz 、控制 100 Hz 、渲染 30 Hz 、感測器發布依各自頻率。這正是第 3 章時間尺度金字塔在模擬中的實現。

30.2.3 即時因子

即時因子 (Real-Time Factor, RTF) 是模擬速度的標準指標：

$\text{RTF} = \text{模擬時間} / \text{實際耗費的時間}$

$\text{RTF} = 1$ 代表與現實同速； $\text{RTF} = 100$ 代表快一百倍； $\text{RTF} = 0.1$ 代表慢十倍。

不同用途對 RTF 的需求不同：

表 30-4 各用途的 RTF 需求

用途	RTF 需求	理由
硬體在環 (HIL)	必須 = 1	真實硬體以真實時間運行
人在迴路測試 (遙控)	接近 1	操作員的反應需要真實時間感
控制器驗證	≥ 0.5 即可	可接受慢一些，但要能觀察
強化學習訓練	越高越好 ($100+$)	決定訓練所需的時間
視覺資料生成	不重要	離線批次產生即可

30.3 保真度的層次

30.3.1 「夠真」是相對的

「模擬要多真才夠？」這個問題沒有絕對的答案——它取決於你要用模擬回答什麼問題。

以一台移動機器人的開發為例，不同階段所需的保真度截然不同：

表 30-5 依開發階段的保真度需求

階段	要回答的問題	所需保真度	可以簡化的部分
架構設計	節點該怎麼切分？	極低（可用假資料）	幾乎全部
演算法邏輯	程式會不會當？	低（運動學即可）	動力學、接觸、視覺
導航測試	路徑規劃對不對？	中（需碰撞與里程計）	精確動力學、視覺細節
控制器調校	增益設得對嗎？	高（需準確動力學）	視覺、外觀
抓取學習	策略能否泛化？	中（但需多樣性）	單一環境的精確度
視覺辨識訓練	能否辨識物體？	視覺高、物理低	動力學可完全省略

最後兩列特別值得注意：抓取學習需要的不是「更真」，而是「更多樣」；視覺訓練需要的是渲染逼真，物理則幾乎可以不要。

30.3.2 三個保真度的維度

保真度不是一個單一的量，而是至少三個獨立的維度：

維度一：動力學保真度。質量、慣量、摩擦、關節特性是否準確。影響控制器的行為。

維度二：感測保真度。感測器的雜訊、延遲、失效模式是否模擬。**這一項最常被忽略**——多數模擬器預設輸出「完美的」感測資料，這與現實差距極大。

維度三：視覺保真度。光照、材質、陰影、反射是否逼真。只影響視覺相關的任務。

這三個維度可以獨立調整，而且應該依需求分別決定。追求全部都高，只會拖慢模擬速度卻沒有對應的收益。

◎ 最容易被忽略的保真度：感測器缺陷

多數 Sim-to-Real 失敗的案例，問題不在物理而在**感測**。模擬中的相機沒有動態模糊、沒有曝光變化、沒有鏡頭畸變；模擬中的光達沒有多重反射、不會被玻璃穿透、不會在陽光下失效；模擬中的編碼器沒有量化誤差、IMU 沒有零漂。一個在完美感測下訓練出的策略，遇到真實的雜訊就會崩潰。因此在模擬中「刻意加入缺陷」——雜訊、延遲、偶爾的資料遺失——往往比提升物理精度更有價值。這正是 30.5 節領域隨機化的一個重要面向。

30.4 Sim-to-Real Gap 的來源

30.4.1 五個主要來源

模擬與現實的落差，可以歸納為五個來源。理解它們的相對重要性，才能把資源投在對的地方。

來源一：接觸與摩擦（最嚴重）。這是第 14.7 節與第 21.2.3 節反覆提及的困難：

- 摩擦係數不確定且會變（灰塵、濕度、磨損），實測範圍可達數倍。
- 接觸模型本身是近似的——懲罰法允許穿透、約束法需要正則化。
- 真實的接觸涉及材料變形、微觀嵌合，這些幾乎無法模擬。

來源二：致動器動態。模擬中的馬達常被理想化為「指令什麼就輸出什麼」。真實的馬達有：延遲、力矩上限、摩擦、背隙、發熱降額、電池電壓下降（第 27.8.2 節）。

來源三：感測器缺陷（30.3.2 節的知識框）。雜訊、延遲、量化、偶發失效。

來源四：未建模的物理。空氣阻力、線材的拉扯力、結構的柔性、溫度影響、以及所有「你沒想到」的效應。

來源五：環境的差異。模擬中的世界是你建的，而真實世界永遠更複雜——地面不平、光線變化、有人走過、物體被移動。

表 30-6 Sim-to-Real Gap 的來源與嚴重程度

來源	嚴重程度	可否測量	因應策略
接觸與摩擦	最高	困難（且會變）	領域隨機化、柔順設計
致動器動態	高	可（實驗辨識）	系統辨識、加入延遲模型
感測器缺陷	高	可（統計量測）	在模擬中加入雜訊
未建模的物理	中	難	領域隨機化、保守設計
環境差異	中~高	部分可	場景隨機化、實地測試

30.4.2 哪些落差重要

一個關鍵的判斷是：不是所有落差都同等重要。判準是「這個落差會不會改變決策」。

舉例說明：

- 若模擬中的桌面顏色與實際不同——對力控制的抓取毫無影響，但對視覺辨識可能是致命

的。

- 若模擬中的馬達延遲少了 20 毫秒——對慢速的取放無影響，但對第 27 章的無人機姿態控制可能導致失穩。
- 若模擬中的摩擦係數高了一倍——對導航無影響，但對抓取是決定性的。

因此正確的作法是：先確認任務對哪些物理量敏感，再針對那些量下功夫。這需要做敏感度分析——系統性地改變各個參數，看性能如何變化（30.10 節的專題）。

30.4.3 一個常被誤解的觀念

最後要澄清一個核心的觀念，這正是開頭故事的重點：

◎ 模擬不必更真，策略必須更穩

許多人面對 Sim-to-Real 失敗時的第一反應是「把模擬做得更真」。這個方向有時有效，但往往是投資報酬率最低的路徑——因為真實世界的參數本身就在變動。若一個策略只在摩擦係數恰好為 0.6 時有效，那麼即使模擬完美無缺，它在真實世界遇到沾灰的物體時仍然會失敗。真正的問題不是模擬不夠真，而是策略太脆弱。因此更有效的方向是：讓策略在參數的整個合理範圍內都能運作。這就是下一節領域隨機化的核心思想，也是本書反覆出現的哲學——與其消除不確定性，不如設計對不確定性不敏感的系統（第 20 章的柔順控制、第 21 章的軟性夾爪、第 24 章的機率估測）。

30.5 領域隨機化

30.5.1 核心思想

領域隨機化（Domain Randomization）是目前跨越 Sim-to-Real Gap 最有效的方法。它的思想極為簡單卻極為有力：

◎ 領域隨機化的核心思想

不要試圖精確模擬真實世界，而是模擬「所有可能的世界」。每次訓練時隨機改變物理參數、外觀、感測雜訊。若一個策略能在這數百萬個各不相同的世界裡都成功，那麼真實世界對它而言，只不過是又一個它沒見過、但也不特別的世界。

這個轉換極為關鍵。傳統的思路是「縮小 sim 與 real 的距離」；領域隨機化則是「擴大 sim 的涵蓋範圍，使 real 落在其中」。

圖 30-1 領域隨機化的概念

建議配圖：兩格對照的示意圖。左格「傳統思路」——一個標示 *SIM* 的小圓點與一個標示 *REAL* 的小圓點相距一段距離，中間以雙向箭頭標註「努力縮小落差」，並註明「但 *REAL* 本身也在移動」（以虛線圓表示真實參數的變動範圍）。右格「領域隨機化」——一個涵蓋範圍極大的橢圓標示 *SIM*（內部散布著數百個小點，代表各種隨機參數組合），*REAL* 的圓點落在這個橢圓之內，標註「真實世界只是其中一個樣本」。下緣文字：不要縮小距離，而要擴大涵蓋。

30.5.2 該隨機化什麼

領域隨機化的對象可分為三類，各對應第 30.3.2 節的三個保真度維度：

表 30-7 領域隨機化的三類參數

類別	隨機化的對象	典型範圍	主要影響
動力學	質量、慣量、摩擦係數、關節阻尼、彈性	$\pm 20\sim 50\%$	控制與接觸行為
	致動器延遲、力矩上限、背隙	$\pm 10\sim 30\%$	控制的穩定性
感測	雜訊大小、延遲、量化、偶發失效	依實測分布	估測與感知的強健性
視覺	光照、材質、顏色、紋理、相機位姿	大範圍	視覺辨識的泛化
場景	物體位置、數量、種類、地形起伏	任務相關	策略的適應性

一個實務上的優先順序建議：先隨機化感測雜訊與致動器延遲（成本低、效果顯著）、再隨機化摩擦與質量（對接觸任務關鍵）、最後才是視覺（只在需要視覺的任務中重要）。

30.5.3 範圍該設多寬

這是領域隨機化最需要工程判斷的部分。範圍的設定有一個明確的取舍：

範圍太窄 → 涵蓋不到真實世界，策略仍會失敗

範圍太寬 → 任務變得過難，策略學不會或變得過度保守

這個取舍與第 24 章 *Q* 矩陣的設定、第 18 章 *LQR* 權重的選擇是同一類問題。

三個實用的原則：

- 以實測為中心：先量測真實系統的參數（30.6 節的系統辨識），再以該值為中心設定範圍。盲目地隨機化是無效的——若真實的摩擦是 0.4 而你隨機化 0.8~1.2，涵蓋不到就沒有意義。
- 寧寬勿窄：若不確定，寧可設寬一些。代價是訓練較慢、策略較保守，但至少能運作；設太窄則可能完全失敗。
- 漸進式擴大（Curriculum）：訓練初期用窄範圍（讓策略先學會基本行為），再逐步擴大。

這能顯著加速收斂——這與人類的學習方式一致：先在簡單環境練熟，再挑戰困難的。

30.5.4 實作範例

以下是一個典型的領域隨機化實作：

```
import numpy as np

class DomainRandomizer:
    """領域隨機化的參數產生器"""

    def __init__(self, nominal, curriculum_level=1.0):
        """
        nominal: 由系統辨識得到的標稱參數 (30.6 節)
        curriculum_level: 0.0~1.0，控制隨機化的強度
        """
        self.nom = nominal
        self.level = curriculum_level

    def _rand_scale(self, low, high):
        """依 curriculum 調整範圍: level 小時範圍窄"""
        mid = 1.0
        lo = mid + (low - mid) * self.level
        hi = mid + (high - mid) * self.level
        return np.random.uniform(lo, hi)

    def sample(self):
        """為一次訓練回合產生一組隨機參數"""
        p = {}

        # ---- 動力學參數 (以標稱值為中心) ----
        p['mass'] = self.nom['mass'] * self._rand_scale(0.7, 1.3)
        p['friction'] = self.nom['friction'] * self._rand_scale(0.5, 1.8)
        p['damping'] = self.nom['damping'] * self._rand_scale(0.6, 1.5)
        p['inertia_scale'] = self._rand_scale(0.8, 1.2)

        # ---- 致動器 (延遲與力矩上限) ----
        p['actuator_delay'] = np.random.uniform(0, 0.02 * self.level) # 0~20 ms
        p['torque_limit'] = self.nom['torque_limit'] * self._rand_scale(0.8, 1.1)
        p['motor_gain'] = self._rand_scale(0.9, 1.1)

        # ---- 感測雜訊 (依實測的分布) ----
        p['imu_noise_std'] = self.nom['imu_noise'] * self._rand_scale(0.5, 2.0)
        p['enc_quantize'] = self.nom['enc_res']
        p['sensor_delay'] = np.random.uniform(0, 0.01 * self.level)
        p['dropout_prob'] = np.random.uniform(0, 0.02 * self.level) # 資料遺失

        # ---- 外部擾動 ----
        p['ext_force'] = np.random.normal(0, 5.0 * self.level, size=3) # N
        p['ground_tilt'] = np.random.uniform(-0.05, 0.05) * self.level # rad
```

```

    return p

def set_level(self, level):
    """訓練過程中逐步提高難度 ( curriculum ) """
    self.level = np.clip(level, 0.0, 1.0)

# ----- 使用方式 -----
nominal_params = {
    'mass': 2.5, 'friction': 0.6, 'damping': 0.05,
    'torque_limit': 10.0, 'imu_noise': 0.01, 'enc_res': 0.001,
}
randomizer = DomainRandomizer(nominal_params, curriculum_level=0.2)

for episode in range(1_000_000):
    # 逐步提高隨機化強度 ( 前 20% 的訓練用於暖身 )
    randomizer.set_level(min(1.0, episode / 200_000))
    params = randomizer.sample()
    env.reset(params)          # 用這組參數重設模擬環境
    # ... 執行一回合的訓練 ...

```

三個值得注意的設計：

- **以標稱值為中心**：所有隨機化都是「標稱值 × 隨機倍率」，而非憑空取值。這確保涵蓋範圍是有意義的。
- **curriculum 逐步提高**：訓練初期 level 小、範圍窄；隨訓練進行逐步擴大。
- **包含感測缺陷與外部擾動**：不只是動力學參數，還有雜訊、延遲、資料遺失與外力——這些往往比物理參數更關鍵。

30.5.5 為何有效：一個直觀的解釋

領域隨機化為何能奏效？有兩個互補的解釋。

解釋一：把「未知參數」變成「觀測不到的干擾」。當策略無法依賴某個參數的特定值時，它被迫學會依即時的回授來調整——也就是學會了一個更接近「控制器」而非「開迴路播放」的行為。這與第 16 章「回授對抗未知干擾」的原理相通。

解釋二：隱含的系統辨識。若策略能觀察一段歷史（如過去 0.5 秒的狀態序列），它其實可以從行為中推斷出當前的參數——例如「我施加了這個力矩卻只動了這麼多，看來這次比較重」。這使策略具備了線上適應的能力。這正是第 28.9.3 節「教師—學生」架構要教會學生的東西。

30.6 系統辨識

30.6.1 與領域隨機化的互補

領域隨機化需要一個「標稱值」作為中心 (30.5.3 節) 。而取得這個值，就是**系統辨識** (system identification) 的工作。

兩者是**互補而非競爭**的：

系統辨識 → 找出參數的中心值
領域隨機化 → 在中心值周圍設定合理的範圍

先辨識、再隨機化，這是標準的流程。

若跳過辨識而盲目隨機化，可能整個範圍都涵蓋不到真實值 (30.5.3 節的原則一) ；若只辨識而不隨機化，則策略會過度依賴那個特定的值。

30.6.2 辨識什麼、怎麼辨識

第 13.10 節已介紹過動力學參數的辨識，此處整理模擬相關的重點：

表 30-8 常見的系統辨識項目

參數	辨識方法	難度
質量與質心	秤重、平衡點法	低
慣性張量	三線擺、CAD 估算	中
關節摩擦	等速運動下的力矩 (第 13.8.2 節)	中
致動器延遲	階躍指令與實際響應的時間差	低但常被忽略
力矩常數	電流與力矩的關係	低
接觸摩擦係數	斜面法 (第 21 章實作)	中 (且會變)
感測器雜訊	靜止時的統計量測	低，應必做
感測器延遲	與已知訊號的互相關	中

最容易被忽略但最有價值的三項：致動器延遲、感測器雜訊、感測器延遲。它們都很容易量測 (一個下午即可完成) ，但對 Sim-to-Real 的影響極大。

30.6.3 由軌跡資料反推參數

更系統性的作法是：讓真實機器人執行一段**激勵軌跡**，記錄輸入與輸出，再以最佳化反推參

數，使模擬的行為與實測吻合。

$$\min_{\theta} \sum_t ||x_{\text{real}}(t) - x_{\text{sim}}(t; \theta)||^2$$

參數辨識的最佳化形式。 θ 為待辨識的參數、 x 為狀態軌跡。

這與第 22.3 節的相機校正、第 25.6 節的圖最佳化是同一類問題——都是「調整參數使預測符合觀測」。

激勵軌跡的設計是關鍵（呼應第 19.10.2 節的持續激勵條件）：軌跡必須足夠「豐富」，才能把各個參數分離出來。若機器人只做單一方向的等速運動，就無法辨識出慣量（因為加速度為零）。

◎ 可微分模擬的新可能

第 14.8.3 節提過的**可微分模擬**，為系統辨識帶來了新工具。傳統的辨識需要反覆試錯或使用無梯度的最佳化；可微分模擬則能直接計算「模擬結果對參數的梯度」，使辨識能用梯度下降高效求解。這使得辨識**數十個參數**成為可能（傳統方法通常只能處理少數幾個）。但它的限制也如第 14.8.3 節所述——接觸的不連續性使梯度不可靠，因此目前主要用於接觸較少的場景。

30.7 常見模擬器的比較

30.7.1 主流選擇

第 14.8.2 節已比較過物理引擎，此處從**完整模擬器**的角度比較（模擬器 = 物理引擎 + 渲染 + 感測器模型 + 介面）：

表 30-9 常見機器人模擬器的比較

模擬器	物理引擎	渲染	ROS 2 整合	平行能力	主要優勢	典型用途
Gazebo	多種可選	中等	原生支援	低	ROS 生態最成熟	導航、SLAM、系統整合
Isaac Sim	PhysX (GPU)	極佳 (光追)	支援	極高 (GPU)	大規模平行、視覺逼真	強化學習、合成資料
MuJoCo	自有 (軟接觸)	簡單	需自行整合	高	接觸平滑可微、極快	控制研究、RL
Webots	ODE	良好	支援	低	易上手、教學友善	教學、競賽
PyBullet	Bullet	簡單	需自行	中	輕量、Python	快速原型、RL 入

模擬器	物理引擎	渲染	ROS 2 整合	平行能力	主要優勢	典型用途
			整合		友善	門
CoppeliaSim	多種可選	良好	支援	低	圖形化編輯、機構豐富	工業應用、教學

30.7.2 選擇的判準

選擇模擬器應依**主要用途**而非「哪個最好」：

- **做 ROS 系統整合與導航** → **Gazebo**。它與 ROS 2 的整合最深，感測器外掛最完整，且社群範例最多。
- **做大規模強化學習** → **Isaac Sim** 或 **MuJoCo**。前者能在單張 GPU 上跑數千個環境，後者速度極快且接觸模型平滑（有利於學習）。
- **做控制演算法研究** → **MuJoCo**。它的接觸模型可微且穩定，適合需要精確動力學的研究。
- **教學或快速原型** → **Webots** 或 **PyBullet**。前者圖形化易上手，後者用純 Python 幾行就能跑起來。
- **做視覺感知訓練** → **Isaac Sim**。它的光線追蹤渲染能產生接近照片級的合成影像。

一個常見的實務模式是「用兩個」：用 MuJoCo 或 Isaac 做大量的訓練（速度優先），再用 Gazebo 做系統整合驗證（生態優先）。因為第 29.3.2 節的解耦設計，控制器程式碼不需要修改。

30.7.3 模擬器與 ROS 2 的介接

模擬器與 ROS 2 的整合，本質上是讓模擬器**扮演硬體驅動節點的角色**——發布同樣的話題、訂閱同樣的指令：

```
真實硬體：驅動節點 → /scan, /odom, /joint_states ; ← /cmd_vel
模擬器：模擬節點 → /scan, /odom, /joint_states ; ← /cmd_vel
```

兩者對上層完全一樣。這正是第 29.3.2 節解耦設計的價值。

因此切換模擬與實機，理想上只需要改變啟動檔中的一行——這也是為何第 29.5.3 節的啟動檔要設計 `use\_sim` 這個參數。

一個重要的細節：**時間來源**。模擬中的時間可能比真實時間快或慢，因此所有節點必須使用**模擬時間**而非系統時鐘。ROS 2 透過 `/clock` 話題與 `use\_sim\_time` 參數處理這件事——忘記設

定它是初學者常見的錯誤，症狀是 TF 查詢一直失敗或行為異常。

30.8 大規模平行模擬

30.8.1 為何需要平行

第 30.1.1 節算過：十萬次抓取嘗試在真機上需要六天。即使模擬能快十倍，仍需十四小時。而第 31 章的強化學習往往需要數百萬次互動——這使平行化成為必要。

平行模擬的思想很直接：同時跑 N 個獨立的環境，每個環境各自進行。若 $N = 4096$ ，理論上可以把訓練時間縮短四千倍。

30.8.2 CPU 平行與 GPU 平行

平行化有兩種截然不同的作法：

CPU 平行：在多核心的 CPU 上開多個行程，每個跑一個模擬。這是傳統的作法。

- **優點**：實作簡單、可用任何模擬器、環境之間完全獨立。
- **缺點**：受限於 CPU 核心數（典型 8~64），且行程間的資料傳輸有開銷。

GPU 平行：把整個模擬移到 GPU 上，用向量化的方式同時計算數千個環境。這是近年的突破（如 Isaac Gym）。

- **優點**：可達數千甚至上萬個環境、且資料直接留在 GPU 上——訓練的神經網路也在 GPU 上，省去了資料搬移的開銷。
- **缺點**：需要 GPU 友善的物理實作（不是所有引擎都支援）、環境必須同構（相同的機器人與場景結構）、且除錯較困難。

◎ 為何 GPU 平行是一次典範轉移

在 Isaac Gym 之前，強化學習的瓶頸通常在**模擬**而非**訓練**——CPU 跑模擬跑不過 GPU 訓練網路。研究者必須用數十台電腦組成叢集才能收集足夠的資料。GPU 平行模擬改變了這件事：現在**單張 GPU 就能同時跑數千個環境**，且資料不需要在 CPU 與 GPU 之間搬移。這使得原本需要數天與一整個叢集的訓練，變成數小時與一台工作站。第 28.9 節腿式機器人的強化學習突破，很大程度上是這個技術突破的結果——不是演算法變聰明了，而是能試的次數多了千倍。

30.8.3 平行模擬的實作要點

三個實務上的關鍵：

要點一：環境的異質性。若所有環境都一樣，平行只是重複同樣的經驗，價值有限。**應搭配領域隨機化**——每個環境用不同的參數、不同的初始狀態、不同的場景配置。**平行 × 隨機化**，才能產生真正多樣的經驗。

要點二：非同步重置。各環境的回合長度不同（有的早早失敗、有的跑完全程）。若等所有環境都結束才一起重置，會浪費大量算力。正確的作法是**個別重置**——某個環境結束就立刻重來，不影響其他環境。

要點三：批次的資料流。所有環境的觀測應該以一個大批次（batch）送進神經網路，一次前向傳播處理全部——這正是 GPU 的強項。若逐一處理，GPU 的平行能力就浪費了。

```
# 平行環境的典型架構（概念性程式碼）
import numpy as np

class ParallelEnv:
    """N 個平行環境的向量化介面"""

    def __init__(self, num_envs=4096, randomizer=None):
        self.N = num_envs
        self.randomizer = randomizer
        # 所有環境的狀態存成一個大陣列（GPU 上則為 tensor）
        self.states = np.zeros((self.N, STATE_DIM))
        self.params = [None] * self.N
        self.step_count = np.zeros(self.N, dtype=int)

    def reset_all(self):
        for i in range(self.N):
            self._reset_one(i)
        return self.states.copy()

    def _reset_one(self, i):
        """個別重置：每個環境用不同的隨機參數（要點一）"""
        self.params[i] = self.randomizer.sample()
        self.states[i] = self._sample_initial_state()
        self.step_count[i] = 0

    def step(self, actions):
        """一次推進所有環境（批次處理，要點三）"""
        # 向量化的物理計算—GPU 上這是一次核心呼叫
        self.states = self._physics_step(self.states, actions, self.params)
        self.step_count += 1

        rewards = self._compute_rewards(self.states, actions)
        dones = self._check_termination(self.states, self.step_count)

        # 非同步重置：只重置已結束的環境（要點二）
```

```

    for i in np.where(dones)[0]:
        self._reset_one(i)

    return self.states.copy(), rewards, dones

# ----- 訓練迴圈 -----
env = ParallelEnv(num_envs=4096, randomizer=randomizer)
obs = env.reset_all()

for iteration in range(10000):
    # 一次前向傳播處理 4096 個環境的觀測
    actions = policy(obs) # shape: (4096, ACTION_DIM)
    obs, rewards, dones = env.step(actions)
    # ... 更新策略 ...

    # 每次迭代收集 4096 筆經驗——相當於 4096 台真實機器人
    if iteration % 100 == 0:
        print(f'迭代 {iteration}: 已收集 {(iteration+1)*4096:,} 筆經驗')

```

注意最後的計算：一萬次迭代 $\times$ 4096 個環境 = 4096 萬筆經驗。若在真機上收集（每筆 0.1 秒），需要 47 年。這就是平行模擬的價值。

30.9 漸進式驗證流程

30.9.1 不要一步跨到實機

開頭故事中「98% 變成 20%」的挫折，很大一部分來自**驗證流程的缺失**——直接從模擬跳到實機。正確的作法是**分階段漸進**，每一階段驗證不同的東西。

表 30-10 漸進式驗證的五個階段

階段	環境	驗證什麼	失敗時的診斷
一、單元測試	純程式（無物理）	邏輯正確性、邊界條件	程式錯誤
二、理想模擬	無雜訊、參數精確	演算法本身可行	演算法設計問題
三、隨機化模擬	領域隨機化開啟	對參數變異的強健性	策略過度依賴特定參數
四、硬體在環	真實控制器 + 模擬機器人	時序、通訊、延遲	軟體架構或即時性問題
五、實機受限測試	真機但低速、有防護	真實的物理與感測	未建模的效應
六、實機完整測試	真機、正常條件	整體性能	—

每一階段的價值在於「隔離變因」。若在階段三失敗，你知道是策略不夠強健（而非硬體問題）；若階段三通過但階段四失敗，問題就在通訊與時序。這使除錯有明確的方向。

◎ 硬體在環（HIL）：最被低估的一步

階段四的**硬體在環**常被跳過，但它能捕捉一類模擬永遠測不到的問題：**時序與通訊**。在這個階段，真實的控制器（如飛控板、工控機）連接到模擬的機器人——控制器不知道自己接的是假的。這能測出：控制迴路是否真的能在 1 ms 內完成、通訊是否會塞車、CPU 負載是否過高、以及第 29.10.2 節的執行器阻塞問題。這些在純軟體模擬中完全不會出現（因為模擬會等你算完），卻是實機失敗的常見原因。

30.9.2 量化落差

每一階段之間，應該**量化**性能的下降。這比「感覺變差了」有用得多：

$$\text{落差} = (\text{模擬性能} - \text{實機性能}) / \text{模擬性能}$$

以成功率為例：98% → 20% 的落差是 79.6%。

追蹤這個數字的變化，能判斷改善是否有效。例如：

- 加入領域隨機化後，落差由 80% 降到 30% → **策略的強健性改善了**。
- 做了系統辨識後，落差由 30% 降到 15% → **標稱參數更準了**。
- 加入感測雜訊模擬後，落差由 15% 降到 8% → **感知的強健性改善了**。

若某項改善沒有降低落差，就代表你找錯了方向——這是避免浪費時間的關鍵判準（呼應 30.4.2 節「哪些落差重要」）。

30.9.3 實機測試的安全原則

進入階段五時，必須有明確的安全措施（呼應第 7.9 節）：

- **降速執行**：以正常速度的 10~30% 開始，確認行為正常後再逐步提高。
- **限制工作空間**：用軟體邊界限制機器人的活動範圍。
- **人在迴路**：手持緊急停止，且操作者的注意力不可分散。
- **先用替代物**：抓取測試先用便宜的替代品（保麗龍塊而非玻璃杯）。
- **全程錄製**：用 rosbag 記錄所有資料（第 29.9.3 節）——失敗時才有資料可分析。

30.10 數位學生

30.10.1 與一般模擬的差異

數位學生 (Digital Twin) 近年成為工業界的熱門概念，但它常被誤用為「模擬」的同義詞。兩者有一個關鍵的差異：

模擬：一個獨立的虛擬環境，用於測試「如果.....會怎樣」

數位學生：與「特定的實體」持續同步的虛擬複本

關鍵在於「同步」。數位學生會持續接收實體的資料並更新自己。

這個差異帶來不同的用途：

表 30-11 模擬與數位學生的比較

面向	一般模擬	數位學生
對象	一類機器人 (通用模型)	特定的那一台
資料流	單向 (模擬 → 分析)	雙向持續同步
參數	標稱值或隨機	由實測持續更新
主要用途	開發、訓練、測試	監控、預測、最佳化
典型問題	「這個演算法可行嗎？」	「這台機器什麼時候該保養？」

30.10.2 三個實務應用

應用一：預測性維護。持續比對實體與學生的行為差異。若某個關節需要的力矩逐漸增加 (而學生模型顯示不該如此)，代表軸承或減速機可能在劣化——在故障發生前就發出警示。

應用二：離線編程與驗證。在學生上測試新的程式，確認無誤後才下載到實體。這對產線極為重要——停機測試的代價極高。

應用三：遠端監控與診斷。專家不需要到現場，透過學生就能看見機器的狀態並診斷問題。

30.10.3 建立數位學生的要件

一個有效的數位學生需要三樣東西：

- **準確的初始模型：**由 CAD 與系統辨識 (30.6 節) 建立。
- **持續的資料連結：**實體必須把感測資料傳給學生。這正是第 29 章 ROS 2 架構的用途之一——同樣的話題，一份給控制器、一份給學生。

- **參數的線上更新**：用實測資料持續修正模型（如第 24 章的估測技術）。

數位學生的價值與其同步的精度成正比。若模型與實體差距太大，比對就沒有意義——這使得 30.6 節的系統辨識成為數位學生的必要基礎。

30.11 由模擬邁向學習

30.11.1 本章的成果

本章建立了模擬的正確使用方式。你現在能夠：說明模擬的三個價值與四種用途、理解物理引擎的計算瓶頸與時間步長的取舍、依需求選擇適當的保真度、分析 Sim-to-Real Gap 的五個來源、實作領域隨機化與系統辨識、選擇適合的模擬器、運用大規模平行模擬、並設計漸進式的驗證流程。

最重要的是那個貫穿全章的觀念轉換：**模擬不必更真，策略必須更穩**。追求完美的模擬是一條報酬遞減的路；讓策略在**參數的整個合理範圍內**都能運作，才是根本的解法。

這個哲學與全書多處呼應——第 20 章用柔順取代精確、第 21 章用軟性夾爪取代精密定位、第 24 章用機率描述取代確定值。它們說的是同一件事：**與其消除不確定性，不如設計對它不敏感的系統**。

30.11.2 接下來：讓機器人自己學

本章建立的模擬環境，其最大的用途將在下一章展現。

第 31 章（機器學習與強化學習）將處理「機器人如何自己學會技能」。我們會看到：監督式學習如何用於感知（第 21.9 節的抓取姿勢偵測、第 22.8 節的物體辨識）、模仿學習如何從人類示範中學習、以及強化學習如何透過試錯自主發展行為。

而**強化學習正是本章模擬技術的最大受益者**——它需要數百萬次的試錯，這在真機上不可能，只有靠 30.8 節的大規模平行模擬才成為可行。第 28.9 節腿式機器人的突破、第 21.9.3 節的抓取資料集，都建立在這個基礎上。

屆時我們也會回收第 18 章的伏筆——**強化學習與最佳控制共用同一條 Bellman 方程**，差別只在於模型是否已知。

換句話說：第 30 章提供了**練習的場地**，第 31 章則教機器人**如何在其中學習**。

完整計算例題

例題 30-1 模擬速度與訓練時間

題目：某抓取任務每次嘗試需 4 秒。強化學習需要 200 萬次嘗試才能收斂。(a) 在真機上訓練需要多久？(b) 若模擬的 RTF 為 20，單環境訓練需要多久？(c) 若改用 GPU 平行，同時跑 2048 個環境且總 RTF 為 8000，需要多久？(d) 若加入領域隨機化使所需的嘗試次數增為 3 倍，重新計算 (c)。

解 (a)：真機訓練時間：

$$2,000,000 \times 4 \text{ 秒} = 8,000,000 \text{ 秒} = 2,222 \text{ 小時} = 92.6 \text{ 天}$$

而且這還沒算人工重置場景的時間。實務上不可行。

解 (b)：RTF = 20 代表模擬時間比真實時間快 20 倍：

$$8,000,000 / 20 = 400,000 \text{ 秒} = 111 \text{ 小時} = 4.6 \text{ 天}$$

仍然太慢——每次調整超參數就要等近五天。

解 (c)：總 RTF 為 8000 (2048 個環境，每個環境的等效 RTF 約 3.9)：

$$8,000,000 / 8,000 = 1,000 \text{ 秒} = 16.7 \text{ 分鐘} \checkmark$$

由 92.6 天縮短到 16.7 分鐘——這正是 GPU 平行模擬帶來的典範轉移 (30.8.2 節的知識框)。

解 (d)：領域隨機化使任務變難，需要更多次嘗試：

$$\begin{aligned} 2,000,000 \times 3 &= 6,000,000 \text{ 次} \\ 6,000,000 \times 4 / 8,000 &= 3,000 \text{ 秒} = 50 \text{ 分鐘} \checkmark \end{aligned}$$

仍在可接受範圍內。

關鍵的工程判讀：領域隨機化的代價是訓練時間增為 3 倍 (16.7 分鐘 → 50 分鐘)，但換來的是 Sim-to-Real 的成功。以本章開頭的故事為例，若不做隨機化，訓練再快也只有 20% 的實機成功率——快速地得到一個沒用的策略，並不是效率。

例題 30-2 落差的量化與歸因

題目：某團隊開發抓取策略，記錄了不同階段的成功率：理想模擬 96%、加入感測雜訊 91%、加入動力學隨機化 84%、實機測試 71%。(a) 計算各階段之間的性能下降。(b) 計算總落差。(c) 判斷主要的落差來源。(d) 若要進一步改善，建議下一步該做什麼。

解 (a)：逐階段的下降：

表 30-12 各階段的性能變化

階段	成功率	相對前一階段的下降	累積下降
理想模擬	96%	—	—
+ 感測雜訊	91%	5.2%	5.2%
+ 動力學隨機化	84%	7.7%	12.5%
實機測試	71%	15.5%	26.0%

解 (b)：總落差 (以理想模擬為基準)：

$$\text{落差} = (96 - 71)/96 = 26.0\%$$

相對於 30.4 節開頭故事的 79.6%，這已經是相當好的結果。

解 (c)：最大的單一下降出現在「隨機化模擬 → 實機」這一步 (15.5%)。這代表：

- 感測雜訊與動力學隨機化**確實有效**——它們各自吸收了 5.2% 與 7.7% 的落差 (若不做，這些下降會在實機階段一次爆發)。
- 但仍有 15.5% 的落差**未被模擬涵蓋**——存在某種隨機化沒有涵蓋到的因素。

解 (d)：建議的診斷步驟：

- 分析失敗案例：錄下實機失敗的 rosbag，逐一檢視。是抓空？滑落？還是碰撞？不同的失敗模式指向不同的原因。
- 擴大隨機化範圍：若失敗集中在特定的物體 (如較滑的)，可能是摩擦係數的範圍設得不夠寬。
- 檢查未隨機化的項目：是否有遺漏的因素？常見的是致動器延遲與視覺的光線變化。
- 做敏感度分析：在模擬中系統性地改變各參數，找出策略最敏感的那一個，優先擴大它的範圍。

一個實務判準：若擴大某個參數的隨機化範圍後，模擬成功率明顯下降但實機成功率上升，代表那個參數正是關鍵。反之若模擬與實機都沒變，代表方向錯了。

例題 30-3 保真度與計算成本的取舍

題目：某模擬環境的計算時間分配為：碰撞偵測 45 ms、約束求解 20 ms、動力學 8 ms、

渲染 35 ms、感測器模擬 12 ms (每個模擬步驟，步長 5 ms)。(a) 計算目前的 RTF。(b) 若任務不需要視覺，關閉渲染後的 RTF。(c) 若把碰撞網格由 5000 面簡化為 200 面 (碰撞時間降為 1/8)，重新計算。(d) 若目標是 $RTF \geq 50$ ，判斷是否達成並提出建議。

解 (a)：每步的總計算時間：

$$45 + 20 + 8 + 35 + 12 = 120 \text{ ms}$$

模擬一個 5 ms 的步長，需要 120 ms 的計算。

$$RTF = 5 \text{ ms} / 120 \text{ ms} = 0.0417$$

RTF 僅 0.042——模擬比現實慢 24 倍。這幾乎無法使用。

解 (b)：關閉渲染 (省下 35 ms)：

$$45 + 20 + 8 + 12 = 85 \text{ ms} \Rightarrow RTF = 5/85 = 0.0588$$

改善了 41%，但仍遠低於實用水準。

解 (c)：碰撞時間由 45 ms 降為 $45/8 = 5.6 \text{ ms}$ ：

$$5.6 + 20 + 8 + 12 = 45.6 \text{ ms} \Rightarrow RTF = 5/45.6 = 0.110$$

再改善 87%。碰撞簡化的效果比關閉渲染更顯著——這印證了 30.2.1 節「碰撞偵測通常是最大瓶頸」的說法。

解 (d)：目前 $RTF = 0.110$ ，距離目標 50 還差 455 倍。單靠上述優化不可能達成。三個建議：

- **平行化 (最有效)**：跑 512 個環境，總吞吐量即可達到 $0.110 \times 512 = 56.3$ —— 超過目標。這說明了為何 30.8 節的平行化如此重要：與其把單一環境優化到極限，不如同時跑很多個。
- **加大時間步長**：若接觸不多，步長可由 5 ms 加大到 10 ms，RTF 直接加倍。但需確認不會發生穿透 (第 14.6.3 節)。
- **簡化約束求解**：20 ms 是第二大項。減少迭代次數或降低接觸點數量可以改善，但會影響精度——這是保真度與速度的直接取捨。

工程結論：優化的優先順序應為「平行化 > 碰撞簡化 > 步長 > 關閉不需要的模組」。而最重要的判斷是：先確認任務真正需要什麼保真度 (30.3 節)，不需要的一律關閉。

程式範例：領域隨機化的效果驗證

以下程式以一個簡化的抓取任務為例，比較「無隨機化」與「有隨機化」訓練出的策略，在參數變動下的表現差異。

```
import numpy as np

# ===== 簡化的抓取模擬 =====
class SimpleGraspEnv:
    """
    極簡的抓取模型：夾爪以某個力夾住物體，
    若「摩擦力 × 夾持力」足以支撐重量則成功。
    這是第 21 章力封閉的一維簡化。
    """
    def __init__(s, mu=0.6, mass=0.5, sensor_bias=0.0,
                 actuator_gain=1.0, g=9.8):
        s.mu = mu                # 摩擦係數
        s.mass = mass            # 物體質量
        s.sensor_bias = sensor_bias # 感測器偏差 (估計的質量誤差)
        s.gain = actuator_gain    # 致動器增益誤差
        s.g = g

    def observe(s):
        """策略能看到的資訊 (帶偏差的質量估計)"""
        return s.mass * (1 + s.sensor_bias)

    def execute(s, commanded_force):
        """執行夾持，回傳是否成功"""
        actual_force = commanded_force * s.gain    # 致動器誤差
        friction_force = s.mu * actual_force        # 最大摩擦力
        weight = s.mass * s.g
        # 成功條件：摩擦力足以支撐重量，且不過度用力壓壞
        holds = friction_force >= weight
        crushes = actual_force > 50.0              # 超過 50 N 壓壞
        return holds and not crushes

# ===== 兩種策略 =====
class NaivePolicy:
    """在標稱參數下訓練——假設  $\mu$  恰好是 0.6"""
    def __init__(s, mu_assumed=0.6, safety=1.05):
        s.mu = mu_assumed
        s.safety = safety          # 只留 5% 餘裕

    def act(s, observed_mass):
        #  $F = mg / \mu$ ，只留很小的餘裕
        return observed_mass * 9.8 / s.mu * s.safety
```

```

class RobustPolicy:
    """在隨機化環境中訓練——學會保留足夠餘裕"""
    def __init__(s, mu_worst=0.35, safety=1.25):
        s.mu = mu_worst          # 假設最差的摩擦
        s.safety = safety         # 留 25% 餘裕

    def act(s, observed_mass):
        return observed_mass * 9.8 / s.mu * s.safety

# ===== 評估 =====
def evaluate(policy, n_trials=10000, randomize=True, seed=0):
    """在隨機的真實參數下測試策略"""
    rng = np.random.default_rng(seed)
    success = 0
    for _ in range(n_trials):
        if randomize:
            # 真實世界的參數分布
            env = SimpleGraspEnv(
                mu=rng.uniform(0.30, 0.90),          # 摩擦變動大
                mass=rng.uniform(0.3, 0.8),
                sensor_bias=rng.normal(0, 0.08),      # 8% 的估計誤差
                actuator_gain=rng.uniform(0.85, 1.10))
        else:
            env = SimpleGraspEnv()                   # 標稱參數
            if env.execute(policy.act(env.observe())):
                success += 1
    return success / n_trials

print("=== 兩種策略在不同環境下的成功率 ===")
naive, robust = NaivePolicy(), RobustPolicy()
print(f"{'策略':<20}{'理想環境':>12}{'隨機環境':>12}{'落差':>10}")
for name, pol in (("天真策略 (無隨機化)", naive),
                  ("強健策略 (有隨機化)", robust)):
    ideal = evaluate(pol, randomize=False)
    real = evaluate(pol, randomize=True)
    gap = (ideal - real)/ideal*100 if ideal > 0 else 0
    print(f"{name:<20}{ideal:11.1%}{real:11.1%}{gap:9.1f}%")
print("→ 天真策略在理想環境完美，但真實環境崩潰 (重現開頭故事)")

# ===== 逐項分析落差來源 =====
print("\n=== 落差的來源分析 (每次只隨機化一項) ===")
def eval_single(policy, vary, n=10000, seed=1):
    rng = np.random.default_rng(seed)
    kw = dict(mu=0.6, mass=0.5, sensor_bias=0.0, actuator_gain=1.0)
    ok = 0
    for _ in range(n):
        p = dict(kw)
        if vary == 'mu':      p['mu'] = rng.uniform(0.30, 0.90)
        if vary == 'mass':    p['mass'] = rng.uniform(0.3, 0.8)
        if vary == 'sensor':  p['sensor_bias'] = rng.normal(0, 0.08)
        if vary == 'actuator':p['actuator_gain'] = rng.uniform(0.85, 1.10)
        env = SimpleGraspEnv(*p)

```

```

        if env.execute(policy.act(env.observe())): ok += 1
    return ok/n

print(f"{'隨機化的項目':<16}{'天真策略':>12}{'強健策略':>12}")
for vary, label in (('none', '無 (理想)'), ('mu', '摩擦係數'),
                    ('mass', '質量'), ('sensor', '感測偏差'),
                    ('actuator', '致動器增益')):
    a = eval_single(naive, vary)
    b = eval_single(robust, vary)
    print(f"{label:<16}{a:11.1%}{b:11.1%}")
print("> 摩擦係數是最主要的落差來源 ( 呼應 30.4.1 節 ) ")

# ===== 隨機化範圍的取舍 ( 30.5.3 節 ) =====
print("\n=== 隨機化範圍對策略的影響 ===")
print(f"{'訓練時假設的最差  $\mu$ ':>20}{'餘裕':>8}"
      f"{'理想環境':>11}{'真實環境':>11}{'平均用力':>11}")
for mu_train, safety in ((0.60, 1.05), (0.50, 1.10),
                        (0.40, 1.20), (0.35, 1.25), (0.25, 1.40)):
    pol = RobustPolicy(mu_worst=mu_train, safety=safety)
    ideal = evaluate(pol, randomize=False)
    real = evaluate(pol, randomize=True)
    # 平均施力 ( 衡量是否過度保守 )
    avg_force = np.mean([pol.act(m) for m in np.linspace(0.3, 0.8, 100)])
    print(f"{mu_train:19.2f}{safety:8.2f}{ideal:10.1%}"
          f"{real:10.1%}{avg_force:10.1f} N")
print("> 假設越保守 · 真實成功率越高 · 但用力也越大 ( 可能壓壞物體 ) ")
print("> 這正是 30.5.3 節「範圍太窄涵蓋不到、太寬過度保守」的取舍")

# ===== 漸進式驗證的模擬 ( 30.9 節 ) =====
print("\n=== 漸進式驗證：逐階段加入不確定性 ===")
stages = [
    ("階段二：理想模擬", dict(mu=(0.6,0.6), sb=0.00, ag=(1.0,1.0))),
    ("階段三 a：+感測雜訊", dict(mu=(0.6,0.6), sb=0.08, ag=(1.0,1.0))),
    ("階段三 b：+致動器", dict(mu=(0.6,0.6), sb=0.08, ag=(0.85,1.10))),
    ("階段三 c：+摩擦變動", dict(mu=(0.3,0.9), sb=0.08, ag=(0.85,1.10))),
]
def eval_stage(policy, cfg, n=10000, seed=2):
    rng = np.random.default_rng(seed)
    ok = 0
    for _ in range(n):
        env = SimpleGraspEnv(
            mu=rng.uniform(*cfg['mu']),
            mass=rng.uniform(0.3, 0.8),
            sensor_bias=rng.normal(0, cfg['sb']),
            actuator_gain=rng.uniform(*cfg['ag']))
        if env.execute(policy.act(env.observe())): ok += 1
    return ok/n

print(f"{'階段':<20}{'天真策略':>12}{'強健策略':>12}{'下降':>10}")
prev_a = prev_b = None
for label, cfg in stages:

```

```

a, b = eval_stage(naive, cfg), eval_stage(robust, cfg)
drop = f"{{(prev_a-a)/prev_a*100:.1f}}%" if prev_a else "-"
print(f"{{label:<20}}{{a:11.1%}}{{b:11.1%}}{{drop:>10}}")
prev_a, prev_b = a, b
print("> 逐階段測試能明確指出「哪一項因素造成最大的下降」")
print("> 本例中摩擦變動是關鍵—應優先擴大其隨機化範圍")

```

請試著修改：（一）把 RobustPolicy 的 safety 提高到 1.6，觀察是否開始出現「壓壞物體」的失敗。（二）加入一個「自適應策略」——它能觀察前一次的結果並調整力道，比較其表現。（三）把真實環境的摩擦範圍改為 0.2~0.5（比訓練時假設的更差），觀察強健策略是否仍然有效，驗證 30.5.3 節「涵蓋不到就無效」的原則。

實作：建立模擬環境與驗證流程

本章實作以 Gazebo 或 PyBullet 為主。若已完成第 29 章的 ROS 2 環境，可直接延續。

- 環境建置**：安裝模擬器（建議 Gazebo 搭配 ROS 2，或 PyBullet 用於快速實驗）。載入一個現成的機器人模型並確認能運行。
- RTF 量測**：量測你的模擬環境的即時因子。用 `ros2 topic hz /clock` 或模擬器內建的統計功能。記錄下來作為後續優化的基準。
- 碰撞簡化實驗**：把機器人的碰撞模型由精細網格改為簡化的幾何（膠囊、盒子），重新量測 RTF。驗證 30.2.1 節與例題 30-3 的結論——你應該會看到顯著的提升。
- 時間步長實驗**：逐步加大物理步長（1、2、5、10、20 ms），記錄 RTF 與行為的變化。找出「開始出現穿透或不穩」的臨界步長。
- 感測器雜訊模擬**：為模擬的感測器加入雜訊（多數模擬器支援）。用實測的統計值（第 24 章實作量測過的）設定參數。
- 系統辨識**：對你的實體機器人（或一個已知的模型）做簡單的辨識——量測質量、致動器延遲、感測器雜訊。**特別是延遲**，用階躍指令與實際響應的時間差即可測得。
- 領域隨機化實作**：實作 30.5.4 節的隨機化器，在每次重設環境時套用。從窄範圍開始，逐步擴大。
- 敏感度分析**：系統性地改變單一參數（其餘固定），記錄性能變化。繪出「參數 vs 性能」曲線，找出**最敏感的參數**。這決定了隨機化的優先順序。
- 漸進式驗證**：依 30.9.1 節的五個階段逐步測試你的控制器或策略。在每個階段記錄性能，

繪出「階段 vs 性能」的下降曲線。

10. 硬體在環（若有硬體）：把真實的控制器（如 Arduino 或工控機）接到模擬器，驗證時序與通訊。觀察是否出現純軟體模擬中沒有的問題。

11. （進階）平行模擬：實作多環境的平行模擬（可用 Python 的 multiprocessing 或 PyBullet 的多實例）。量測總吞吐量並與單環境比較。

表 30-13 Sim-to-Real 驗證記錄表（前兩列為範例）

階段 / 設定	性能指標	相對下降	推測原因	改善措施
理想模擬	成功率 94%	—	—	—
+ 感測雜訊	成功率 88%	6.4%	位置估計誤差	加入雜訊訓練
（自行填寫其餘階段）				

國中小也看得懂：為什麼電動玩得好不代表真的會開車

你有沒有玩過賽車電動遊戲？在遊戲裡你可能是高手，每次都第一名。但如果真的讓你去開卡丁車，你一定會撞牆——為什麼？

因為**遊戲不是真的**。遊戲裡沒有風、輪胎不會磨損、地上不會有小石頭、你也感覺不到車子在震動。遊戲把很多東西「簡化」了。

機器人的工程師也用同樣的方法：他們先在電腦裡蓋一個「假的世界」，讓機器人在裡面練習。這樣機器人摔一萬次也不會壞、而且電腦可以跑得比真實世界快一百倍！

但問題來了——在假世界裡練得很好的機器人，到了真實世界常常就不行了。

有一個學姊就遇到這種事：她的機器手臂在電腦裡抓東西成功率 98%，但真的去抓，一百次只成功二十次！

為什麼呢？因為電腦裡的瓶子**永遠一樣滑**，但真實的瓶子可能沾了灰、桌子可能有一點油、夾子用久了會變鬆.....真實世界有太多小細節，永遠算不完。

那怎麼辦？聰明的科學家想出一個很妙的辦法：**故意把假世界弄亂！**

每一次練習，就隨機改變一次設定——這次瓶子很滑、下次很粗糙；這次很重、下次很輕；這次燈很亮、下次很暗。**逼機器人在幾百萬種不同的世界裡都要能成功。**

這樣一來，當它到了真實世界，就會想：「喔，這個世界我雖然沒見過，但也沒什麼特別的嘛！」然後就成功了。

這個道理其實對你也適用：如果你只在一種情況下練習，換個場地就不會了；但如果你在各種情況下都練過——大場地、小場地、有風、沒風——那你就真的學會了。

叫獸提醒與常見錯誤

- 誤區一：以為「模擬夠真就能上實機」。真實參數本身就在變動，過度依賴特定參數的策略必定脆弱。
- 誤區二：面對 Sim-to-Real 失敗時只想提升模擬保真度。應優先檢視策略的強健性——這是報酬率更高的方向。
- 誤區三：盲目地隨機化而未先做系統辨識。若隨機化的範圍涵蓋不到真實值，就完全沒有意義。
- 誤區四：隨機化範圍設得太寬。任務會變得過難而學不會，或使策略過度保守（例題的過度用力）。
- 誤區五：只隨機化動力學而忽略感測缺陷。多數 Sim-to-Real 失敗的原因在感測而非物理。
- 誤區六：用高精度網格作為碰撞模型。這可能使模擬慢一到兩個數量級，應改用簡化的幾何。
- 誤區七：不分用途地追求最高保真度。四種用途對保真度的需求截然不同，應依需求分別決定。
- 誤區八：忘記設定 `use_sim_time`。模擬時間與系統時鐘不同步，會導致 TF 查詢失敗與行為異常。
- 誤區九：跳過硬體在環測試。時序、通訊與 CPU 負載的問題在純軟體模擬中完全不會出現。
- 誤區十：直接從模擬跳到實機全速測試。應遵循漸進式驗證，並在實機階段降速、限制範圍、全程錄製。
- 誤區十一：平行模擬時所有環境用相同參數。這只是重複同樣的經驗，應搭配領域隨機化才有價值。
- 誤區十二：把數位孿生當成一般模擬的同義詞。前者的關鍵在於與特定實體的持續同步。

課堂討論

1. 30.4.3 節主張「模擬不必更真，策略必須更穩」。請討論這個原則的適用邊界——在什麼情況下，提升模擬保真度反而是正確的選擇？
2. 領域隨機化把「未知參數」變成「觀測不到的干擾」。請討論這與第 16 章「回授對抗未知干擾」、第 24 章「用機率描述不確定性」的異同。
3. 例題 30-1 顯示 GPU 平行模擬把訓練時間由 92.6 天縮短到 16.7 分鐘。請討論這個「量變引起質變」的現象——它如何改變了研究的方式？
4. 30.9.1 節的漸進式驗證強調「隔離變因」。請討論這個科學方法的原則，在本書其他章節的除錯情境中如何應用（提示：第 29 章的 QoS 診斷、第 24 章的創新序列檢驗）。
5. 本章的哲學「與其消除不確定性，不如設計對它不敏感的系統」在全書多次出現。請找出至少四處，並歸納它們的共同結構與各自的實現方式。

章末摘要

- 模擬的三個價值：解除成本、風險與時間的限制；另有「全知的可觀測性」這個常被低估的優勢。
- 模擬有四種用途——演算法除錯、控制器驗證、學習訓練、視覺訓練——各自對保真度的需求截然不同。
- 物理引擎的四個模組中，碰撞偵測通常是最大瓶頸；應使用簡化的碰撞幾何而非高精度網格。
- 即時因子（RTF）是模擬速度的指標；不同用途的需求由 HIL 的必須為 1 到 RL 的越高越好。
- 保真度有三個獨立的維度：動力學、感測、視覺；應依需求分別決定，不必全部追求最高。
- 最容易被忽略的保真度是感測器缺陷——多數 Sim-to-Real 失敗的原因在感測而非物理。
- Sim-to-Real Gap 的五個來源：接觸與摩擦（最嚴重）、致動器動態、感測器缺陷、未建模的物理、環境差異。
- 判斷落差是否重要的準則是「它會不會改變決策」；應先做敏感度分析再投入資源。
- 核心觀念：模擬不必更真，策略必須更穩——追求完美模擬是報酬遞減的路徑。
- 領域隨機化的思想是「不模擬真實世界，而模擬所有可能的世界」，使真實世界成為其中的

一個樣本。

- 隨機化的三類對象：動力學、感測、視覺與場景；優先順序建議為感測雜訊與延遲 > 摩擦與質量 > 視覺。
- 隨機化範圍的取舍：太窄涵蓋不到、太寬過度保守；應以系統辨識的結果為中心，並採漸進式擴大。
- 領域隨機化有效的兩個解釋：把未知參數變成干擾（迫使依賴回授）、以及隱含的線上系統辨識。
- 系統辨識與領域隨機化互補：先辨識找出中心值，再隨機化設定範圍。
- 最容易量測卻最常被忽略的三項辨識：致動器延遲、感測器雜訊、感測器延遲。
- 模擬器的選擇應依用途：ROS 整合用 Gazebo、大規模 RL 用 Isaac Sim 或 MuJoCo、教學用 Webots 或 PyBullet。
- GPU 平行模擬是一次典範轉移——單張 GPU 可跑數千環境且資料不需搬移，這是近年 RL 突破的物質基礎。
- 平行模擬應搭配領域隨機化與非同步重置，否則只是重複同樣的經驗。
- 漸進式驗證分六階段，每階段隔離不同的變因；硬體在環能捕捉純軟體模擬測不到的時序問題。
- 數位學生與一般模擬的關鍵差異在於「與特定實體的持續同步」，用途為監控、預測與最佳化。

觀念題與計算題

觀念題

1. 說明模擬解除的三個限制，並解釋「可觀測性」為何是被低估的優勢。
2. 列舉模擬的四種用途，並說明各自對保真度的需求為何不同。
3. 說明物理引擎的四個模組，並解釋為何碰撞偵測通常是最大瓶頸。
4. 定義即時因子，並比較五種用途對它的需求。
5. 說明保真度的三個維度，並解釋為何應分別決定。
6. 為何說「感測器缺陷是最容易被忽略的保真度」？請舉三個具體的例子。

7. 列舉 Sim-to-Real Gap 的五個來源，並依嚴重程度排序。
8. 說明判斷「哪些落差重要」的準則，並舉例說明。
9. 解釋「模擬不必更真，策略必須更穩」的意義，並說明其理由。
10. 說明領域隨機化的核心思想，以及它與傳統思路的根本差異。
11. 列舉領域隨機化的三類對象，並說明實務上的優先順序。
12. 說明隨機化範圍設定的取捨，以及三個實用的原則。
13. 說明領域隨機化為何有效的兩個解釋。
14. 說明系統辨識與領域隨機化的互補關係，以及為何順序不可顛倒。
15. 列舉三項最容易量測卻常被忽略的辨識項目。
16. 比較至少四種模擬器的特性，並說明選擇的判準。
17. 說明 GPU 平行模擬相對於 CPU 平行的優勢，以及它為何是一次典範轉移。
18. 說明平行模擬的三個實作要點。
19. 列舉漸進式驗證的六個階段，並說明硬體在環能捕捉什麼問題。
20. 區分數位學生與一般模擬，並列舉數位學生的三個應用。

計算題

1. 某任務每次嘗試需 6 秒，強化學習需要 150 萬次嘗試。(a) 真機訓練需多久？(b) $RTF = 15$ 的單環境模擬需多久？(c) 若用 1024 個環境平行且總 RTF 為 5000，需多久？(d) 若領域隨機化使所需次數增為 2.5 倍，重新計算 (c)。
2. 某團隊記錄的成功率為：理想模擬 92%、加入感測雜訊 87%、加入動力學隨機化 79%、實機 68%。(a) 計算各階段的下降百分比。(b) 計算總落差。(c) 指出最大的單一下降出現在何處。(d) 提出兩個改善的方向。
3. 某模擬的每步計算時間為：碰撞 60 ms、約束求解 25 ms、動力學 10 ms、渲染 40 ms、感測 15 ms，步長 4 ms。(a) 計算 RTF 。(b) 關閉渲染後的 RTF 。(c) 若碰撞簡化使其降為 1/10，重新計算。(d) 若要達到總 $RTF \geq 100$ ，計算所需的平行環境數。
4. 某抓取任務的物體質量在 0.2~1.0 kg 之間，摩擦係數在 0.35~0.85 之間。策略以「假設最差情況」的方式設計。(a) 若假設 $\mu = 0.85$ (最樂觀)，求質量 1.0 kg 時所需的夾持力，並判斷在 $\mu = 0.35$ 時是否會失敗。(b) 若假設 $\mu = 0.35$ (最保守)，求同樣情況下的夾持

力。(c) 計算兩者的比值。(d) 若夾持力超過 40 N 會壓壞物體，判斷保守策略是否可行。

動手做與專題延伸

本章實作：模擬與驗證環境

建立一套完整的模擬與驗證流程，這將是第 31 章學習訓練的基礎。

- **階段一：**建立基本的模擬環境，載入機器人模型並確認能運行；量測 RTF 作為基準。
- **階段二：**優化模擬速度——簡化碰撞幾何、調整時間步長、關閉不需要的模組。記錄每項優化的效果。
- **階段三：**實作領域隨機化器，支援動力學、感測與場景三類參數，並具備 curriculum 功能。
- **階段四：**實作漸進式驗證的自動化流程——一鍵跑完各階段並產生性能比較表。
- **階段五：**實作平行模擬，量測吞吐量並與單環境比較。

專題延伸

- **敏感度分析地圖：**對一個具體的任務（如抓取或導航），系統性地掃描各個物理參數（摩擦、質量、延遲、雜訊），記錄性能的變化。繪出「參數 vs 性能」的曲線族，並用斜率量化敏感度。這張圖能直接告訴你：該優先隨機化哪些參數、以及該優先辨識哪些參數。這是本章方法論最實用的產出。
- **隨機化範圍的最佳化：**以程式範例的抓取任務為基礎，系統性地改變隨機化的範圍寬度（從極窄到極寬），記錄「模擬成功率」與「真實環境成功率」兩條曲線。你會看到一個明確的取舍——模擬成功率隨範圍變寬而下降，但真實成功率先上升後下降（過寬時策略變得過度保守）。找出那個最佳點，並討論它與真實參數分布的關係。
- **進階：完整的 Sim-to-Real 專案。**選一個具體的任務（建議：用循跡車走一條有轉彎的路線，或用手臂抓取三種不同的物體）。（一）在模擬中開發並訓練；（二）做系統辨識並量化真實參數；（三）加入領域隨機化重新訓練；（四）依 30.9 節的六階段逐步驗證，每階段記錄性能；（五）繪出完整的落差曲線並歸因。這個專題會讓你親身體會本章的每一個概念——尤其是「哪些改善有效、哪些沒效」這個最寶貴的工程經驗。

參考文獻與延伸閱讀

- Tobin, J. et al., "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," IROS 2017 (領域隨機化的奠基論文) 。
- Peng, X. B. et al., "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization," ICRA 2018 (動力學隨機化的代表性工作) 。
- Makoviychuk, V. et al., "Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning," NeurIPS 2021 (GPU 平行模擬的突破) 。
- Todorov, E., Erez, T. and Tassa, Y., "MuJoCo: A Physics Engine for Model-Based Control," IROS 2012 (MuJoCo 的設計理念) 。
- Koenig, N. and Howard, A., "Design and Use Paradigms for Gazebo," IROS 2004 (Gazebo 的原始論文) 。
- Zhao, W., Queralta, J. P. and Westerlund, T., "Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: A Survey," SSCI 2020 (Sim-to-Real 的完整綜述) 。
- Muratore, F. et al., "Robot Learning From Randomized Simulations: A Review," Frontiers in Robotics and AI, 2022 (隨機化模擬的系統性回顧) 。
- Chebotar, Y. et al., "Closing the Sim-to-Real Loop: Adapting Simulation Randomization with Real World Experience," ICRA 2019 (用實機資料自動調整隨機化範圍) 。
- Kaspar, M. et al., "Sim2Real Transfer for Reinforcement Learning without Dynamics Randomization," IROS 2020 (以系統辨識為主的替代路線) 。
- Gazebo、Isaac Sim、MuJoCo、PyBullet 官方文件：各模擬器的使用與最佳實務。

第 31 章

機器學習與強化學習

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩個任務的對照：上格是六軸手臂精確地定位到一個標記點，旁註「逆運動學：兩頁公式，微米級精度」；下格是一件皺成一團的 T 恤與一雙手，旁註「摺衣服：至今無機器人能及五歲小孩」。中央以一條垂直的分界線區隔「模型」與「學習」兩條路徑，左側標註「寫得出方程式 → 建模、推導、保證」，右側標註「寫不出方程式 → 從經驗中找規律」。右半為 Bellman 方程的雙重身分：同一條方程式，左邊標「最佳控制：模型已知，用算的」、右邊標「強化學習：模型未知，用試的」。下緣文字：模型提供保證，學習提供泛化。

叫獸開講 「算逆運動學到微米級，為什麼摺不好一件 T 恤？」

叫獸在白板上寫下兩個任務，請全班判斷哪一個比較難。

任務一：計算一支六軸手臂的逆運動學，誤差不超過 0.01 公釐。

任務二：把一件皺成一團的 T 恤攤平摺好。

學生們幾乎都選了任務一——畢竟那要解三角函數、處理八組解、還要避開奇異點。

「錯了。」叫獸說，「任務一，第 10 章我們用兩頁的公式就解決了，而且**精度可以到微米**。任務二，目前全世界沒有任何一台機器人能做得像五歲小孩那樣好。」

教室安靜了。小好問：「為什麼？摺衣服看起來很簡單啊。」

「因為衣服沒有方程式。」叫獸拿起一件 T 恤揉成一團，「你能寫出它的動力學嗎？布料有無限多的自由度、會皺、會摩擦、會黏在一起。第 13 章的 $M(q)\ddot{q} + C\dot{q} + G = \tau$ 在這裡完全派不上用場。」

他把衣服攤在桌上：「前面三十章，我們的方法都是「先建模型，再解方程」。這套方法極為強大——但它有一個前提：問題必須寫得出方程式。」

小翰舉手：「那寫不出來的怎麼辦？」

「那就**不要寫**。」叫獸說，「換一條路——讓機器人自己從經驗中找出規律。這就是機器學習。」

他在白板上畫了一條分界線，左邊寫「模型」、右邊寫「學習」：「這兩條路不是互相取代，而是分工。左邊處理你**理解得夠清楚**的部分——運動學、動力學、控制律，這些有嚴格的保證。右邊處理你**寫不出來但有很多例子**的部分——什麼樣的地方適合抓、皺成什麼樣的布該怎麼拉。」

最後他補了一句，指向第 18 章的板書痕跡：「而且今天你會發現一件有趣的事。**強化學習**跟我們在第 18 章學的最佳控制，用的是同一條方程式——Bellman 方程。差別只在於：第 18 章我們知道世界的規則所以能用算的，強化學習則是不知道規則，只好用試的。」

本章為什麼重要

前面三十章建立的是一套**模型導向**的方法論：建立數學模型、推導控制律、證明穩定性。這

套方法在有明確物理規律的問題上極為成功——第 17 章的計算力矩控制能達到微米級精度、第 18 章的 LQR 有最佳性保證、第 19 章的 Lyapunov 方法能嚴格證明不會失控。

但這套方法有一個前提：**問題必須能被建模**。而真實世界中有大量的問題無法滿足這個前提：

- **難以建模的物體**：布料、繩索、液體、顆粒、生物組織。
- **難以建模的接觸**：第 14.7 節與第 21.2.3 節反覆提及的摩擦與接觸不確定性。
- **難以描述的判斷**：「這個物體看起來該從哪裡抓」、「這個地形該怎麼踩」。
- **組合爆炸的決策**：雜亂桌面上該先移開哪一個物體。

機器學習提供了另一條路：不建模，而是從資料中學習規律。近年機器人領域的重大突破——第 21.9 節的抓取姿勢偵測、第 28.9 節的四足運動控制、以及第 32 章的大型模型——全部來自這條路線。

但本章要傳達的核心訊息不是「學習取代模型」，而是**兩者的分工**。正如第 21.9.5 節的混合式抓取、第 25.10.3 節的 SLAM 前後端、第 28.9.4 節的模型與學習比較所示：**模型提供保證，學習提供泛化**。理解何時該用哪一個，是本章最重要的收穫。

學習目標

- 能說明模型導向與資料導向方法的適用邊界。
- 能區分監督式、非監督式與強化學習三種典範。
- 能說明監督式學習在機器人感知中的應用與資料需求。
- 能說明模仿學習的原理、優勢與其複合誤差問題。
- 能說明 DAgger 如何解決分布偏移。
- 能定義馬可夫決策過程的五個要素。
- 能說明強化學習與最佳控制的關係並指出關鍵差異。
- 能區分價值型與策略型方法及其適用場合。
- 能說明獎勵函數設計的困難與獎勵駭客現象。
- 能說明策略梯度與 PPO 的基本原理。
- 能說明從模擬到實機的學習流程與其成功要件。

- 能為特定的機器人問題選擇適當的學習方法。

先備知識

第 18 章的 Bellman 方程與價值函數（本章直接建立在其上，務必先複習）、第 30 章的模擬與領域隨機化、第 22.8 節的深度學習概念、第 21.9 節的學習式抓取、第 28.9 節的腿式運動學習。

數學方面需要：機率的基本概念、梯度的直覺。**不需要**深度學習的完整背景——本章聚焦於「機器人如何學習」，而非神經網路的細節。若需要補充，可先閱讀附錄 E。

核心名詞中英對照

表 31-1 本章核心名詞

中文	英文	說明
監督式學習	Supervised Learning	由帶標籤的資料學習輸入到輸出的映射
非監督式學習	Unsupervised Learning	由無標籤資料中發現結構
強化學習	Reinforcement Learning, RL	透過與環境互動並依獎勵學習
模仿學習	Imitation Learning	由專家示範中學習策略
行為複製	Behavior Cloning	以監督式學習直接模仿示範動作
分布偏移	Distribution Shift	執行時的狀態分布偏離訓練資料
複合誤差	Compounding Error	小誤差逐步累積導致大偏離
馬可夫決策過程	Markov Decision Process, MDP	強化學習的標準數學框架
策略	Policy	由狀態決定動作的規則
獎勵	Reward	衡量行為好壞的純量訊號
回報	Return	未來獎勵的折扣累加
折扣因子	Discount Factor, γ	衡量未來獎勵重要性的係數
價值函數	Value Function	由某狀態出發的期望回報
Q 函數	Q-function / Action-Value	在某狀態採取某動作的期望回報
探索與利用	Exploration vs Exploitation	嘗試新動作與採用已知好動作的取捨
策略梯度	Policy Gradient	直接對策略參數求梯度的方法
獎勵塑形	Reward Shaping	設計輔助獎勵以加速學習
獎勵駭客	Reward Hacking	找到獎勵的漏洞而非完成任務

中文	英文	說明
樣本效率	Sample Efficiency	達到目標性能所需的互動次數

31.1 模型與學習的分界

31.1.1 模型導向方法的成功與邊界

前面三十章的方法論可以概括為三步：**建模** → **推導** → **驗證**。它的成功建立在物理定律的普適性上——牛頓定律不會因為換一台機器人而改變。

這帶來三個無可取代的優勢：

- **保證**：第 19 章能嚴格證明「這個控制器不會失控」。學習方法給不出這種保證。
- **可解釋**：當出問題時，可以逐項檢查——是模型參數錯了？增益不對？還是感測失準？
- **資料效率**：不需要任何訓練資料。一台新機器人裝好、量好參數，控制器立刻能用。

但它的邊界也很明確：**問題必須寫得出方程式**。

表 31-2 模型導向方法的適用邊界

問題類型	能否建模	範例	本書章節
剛體運動學	完全可以	手臂的正逆運動學	第 9、10 章
剛體動力學	可以（有誤差）	力矩計算、軌跡規劃	第 13~15 章
線性控制	可以且有保證	PID、LQR	第 16、18 章
接觸與摩擦	部分可以	抓取、行走	第 20、21、28 章
變形物體	極困難	布料、繩索、食物	—
視覺理解	幾乎不可能	「這是什麼物體」	第 22 章
常識判斷	不可能	「杯子該正放」	第 32 章

31.1.2 學習方法的價值與代價

學習方法的思路完全相反：不問「規律是什麼」，只問「資料告訴我們什麼」。

價值：能處理無法建模的問題、能從經驗中改善、能泛化到新情況。

代價：需要大量資料、沒有保證、難以解釋、且在訓練分布之外的行為不可預測。

◎ 一個關鍵的判準

面對一個機器人問題，該用模型還是學習？一個實用的判準是問三個問題：一、你能寫出方程式嗎？能 → 用模型（它更準、更可靠、且不需要資料）。二、你有大量的資料或能低成本地產生嗎？有 → 學習可行（第 30 章的模擬正是為此）。三、失敗的代價有多大？極高（如載人系統）→ 至少關鍵部分必須用有保證的方法。多數實務系統的答案是「兩者都要」——用模型處理有物理規律的部分、用學習處理需要判斷的部分。

31.1.3 三種混合的模式

實務上模型與學習的結合有三種常見模式，本書前面各章都出現過：

表 31-3 模型與學習的三種混合模式

模式	作法	本書中的例子
學習做前端、模型做後端	學習負責辨識與判斷，模型負責計算與驗證	第 25.10.3 節的 SLAM、第 21.9.5 節的抓取
模型做骨架、學習做修正	模型提供標稱行為，學習補償未建模的部分	第 17 章的前饋加回授（概念相同）
模型產生資料、學習做泛化	用模型或模擬產生大量標註資料訓練網路	第 21.9.3 節的解析式標註

31.2 三種學習典範

31.2.1 由「資料長什麼樣」區分

機器學習傳統上分為三種典範，其差別在於訓練資料的形式：

表 31-4 三種學習典範的比較

典範	資料形式	學什麼	機器人上的典型應用
監督式學習	(輸入, 正確答案) 配對	輸入到輸出的映射	物體辨識、抓取點預測、深度估計
非監督式學習	只有輸入，無標籤	資料的內在結構	特徵學習、異常偵測、分群
強化學習	(狀態, 動作, 獎勵) 序列	能最大化長期回報的策略	運動控制、操作技能、決策

三者的關鍵差異在於「回饋的形式」：

- 監督式學習得到的是**正確答案**——「這張圖是杯子」。回饋最豐富、學習最快。
- 強化學習得到的是**評分**——「這次抓取得 1 分」。它不告訴你正確答案是什麼，只說結果好

不好。這使學習困難得多。

- 非監督式學習完全沒有回饋——只能從資料本身的結構中找規律。

31.2.2 機器人領域的特殊性

機器人上的機器學習，與影像辨識或自然語言處理有三個重要的差異：

差異一：資料昂貴。網路上有數十億張標註好的影像，但沒有任何現成的「機器人抓取經驗」資料集能與之相比。每一筆資料都需要真實的物理互動——這正是第 30 章模擬如此重要的原因。

差異二：動作有後果。影像辨識錯了只是一個錯誤的標籤；機器人動作錯了可能打翻杯子、撞壞設備、甚至傷人。這使「試錯」的代價極高。

差異三：序列決策。辨識是一次性的判斷；機器人的動作則會改變自己接下來看到的世界。這個「動作影響後續狀態」的特性，正是強化學習與監督式學習的根本差異。

◎ 為何機器人學習比影像辨識困難

2012 年之後，深度學習在影像辨識上取得了驚人的進展。許多人期待同樣的突破會迅速發生在機器人上——但實際上慢得多。三個原因：**一、資料量差了好幾個數量級**（ImageNet 有 1400 萬張標註影像，而最大的機器人資料集也只有數百萬筆互動，且多來自模擬）；**二、無法用網路上的資料**（機器人需要的是「動作與後果」的配對，而不是圖片與標籤）；**三、錯誤的代價不對稱**（辨識錯 1% 無妨，但機器人 1% 的機率打翻東西就無法部署）。理解這三點，就能理解為何機器人領域的學習方法必須格外重視樣本效率與安全性。

31.3 監督式學習在機器人上的應用

31.3.1 最成熟的應用：感知

監督式學習在機器人上最成功的應用是**感知**——因為感知本質上就是「由輸入預測輸出」的映射問題。

本書前面已經多次提及：

- **物體偵測與分類**（第 22.8 節）：輸入影像，輸出物體的類別與位置。
- **位姿估計**（第 22.8 節）：輸入影像，輸出物體的六自由度位姿。
- **抓取姿勢偵測**（第 21.9 節）：輸入深度圖，輸出可行的抓取位姿與成功率預測。
- **語意分割**（第 25.10.1 節）：輸入影像，輸出每個像素的類別——用於剔除動態物體。

- 深度估計（第 22.4 節）：由單張影像預測深度。

31.3.2 資料從哪裡來

監督式學習的瓶頸永遠是**標註資料**。機器人領域有四個來源：

表 31-5 機器人學習資料的四個來源

來源	成本	真實性	規模	典型用途
人工標註	高	高	小（千~萬）	驗證集、微調
模擬產生	極低	中（有落差）	極大（百萬+）	預訓練、大規模訓練
解析式標註	低	中	大	抓取品質評分（第 21.9.3 節）
自我監督	中	高	中	機器人自行嘗試並記錄結果

實務上的標準作法是混合：用模擬產生大量資料做預訓練（學會通用的特徵）、再用少量的真實資料微調（校正 Sim-to-Real 的落差）。這正是第 30 章模擬技術的直接應用。

31.3.3 一個常被忽略的陷阱

監督式學習有一個在機器人上特別危險的問題：它假設訓練資料與實際使用時的資料來自同一個分布。

但機器人的部署環境往往與訓練時不同——不同的光線、不同的物體、不同的背景。當這個假設被打破時，模型的預測**可能錯得很離譜，而且它自己不知道**（會給出很高的信心分數）。

因應之道有三：

- **訓練時涵蓋多樣性**：這正是第 30.5 節領域隨機化的視覺版本。
- **輸出不確定性**：讓模型除了預測之外，也回報自己的信心。低信心時交由其他機制處理（如改用保守的策略、或請求人類介入）。
- **執行時的驗證**：用幾何或物理的方法驗證學習的輸出——這正是第 21.9.5 節「學習產生候選、解析驗證品質」的混合式架構。

31.4 模仿學習

31.4.1 從示範中學習

模仿學習（Imitation Learning）的思路極為直觀：與其讓機器人自己摸索，不如讓人示範

幾次，機器人照著學。

這在機器人上特別有吸引力，因為：

- 不需要設計獎勵函數（31.7 節會說明這有多困難）。
- 樣本效率高：幾十次示範可能就夠，而強化學習需要數百萬次嘗試。
- 安全：不需要在真機上做危險的隨機探索。
- 直觀：非專家也能「教」機器人——這對第 20.5.3 節的手動導引示教是自然的延伸。

31.4.2 行為複製及其致命缺陷

最簡單的模仿學習稱為**行為複製**（Behavior Cloning）：把示範資料當成監督式學習的訓練集。

訓練資料： $\{(s_1, a_1), (s_2, a_2), \dots\}$

學習目標： $\pi(s) \approx a_{\text{expert}}(s)$

這就是一個標準的監督式學習問題。實作極為簡單。

但它有一個致命的缺陷，稱為**複合誤差**（compounding error）：

◎ 複合誤差：為何模仿學習會逐漸偏離

假設機器人學會了 99% 準確地模仿專家。第一步它有 1% 的誤差，來到一個專家從未到過的狀態。在這個新狀態下，它的預測更不準（因為訓練資料裡沒有），於是誤差變成 3%。下一步來到更陌生的狀態，誤差變成 10%.....**誤差會像滾雪球一樣累積**。這正是為何行為複製訓練出的自駕車，在直線上開得很好，卻會在第一個彎道逐漸偏離車道並最終衝出去——**因為專家從來沒有偏離過，所以資料中沒有「偏離後該怎麼修正」的範例**。

數學上可以證明：若每一步的誤差率為 ϵ ，在長度 T 的軌跡上，總誤差會以 $O(\epsilon T^2)$ 成長——與時間的平方成正比，而非線性。

31.4.3 DAgger：讓專家糾正錯誤

DAgger（Dataset Aggregation）是解決複合誤差的經典方法。它的核心洞見是：

問題的根源：訓練資料只涵蓋「專家走過的狀態」

解法：讓資料涵蓋「機器人自己會走到的狀態」

關鍵是改變資料的來源。

具體流程為：

1. 用現有的示範資料訓練一個初始策略。
2. 讓機器人自己執行（它會犯錯並偏離）。
3. 在它走過的每一個狀態上，詢問專家「你會怎麼做」。
4. 把這些新的（狀態，專家動作）加入資料集。
5. 重新訓練，回到步驟 2。

這個流程的精髓在於步驟三：專家不是重新示範一遍，而是在機器人犯錯後的狀態上給出正確答案。這使資料集逐漸涵蓋「偏離後如何修正」——正是行為複製所缺少的。

DAgger 的代價是需要專家持續參與（而非一次性的示範），這在實務上可能很昂貴。因此有多種變體嘗試降低專家的負擔——例如只在模型不確定時才詢問。

31.4.4 近年的發展

模仿學習近年有三個重要的進展，都與第 32 章的大型模型密切相關：

進展一：擴散策略（Diffusion Policy）。傳統的行為複製只能學到「平均的動作」——若專家有多種合理的做法（如繞過障礙可左可右），平均之後可能兩邊都不是。擴散模型能表示多峰的動作分布，解決了這個問題。

進展二：大規模的示範資料集。多個實驗室合作收集跨機器人、跨任務的示範資料（如 Open X-Embodiment），使「預訓練一個通用的操作模型」成為可能——這正是第 32 章的主題。

進展三：遙操作介面的改善。用 VR 手把或第 20.5.3 節的手動導引收集示範，大幅降低了資料收集的門檻。

◎ 模仿學習與強化學習的分工

兩者不是競爭關係，而是常常搭配使用。典型的流程是：先用模仿學習取得一個「還可以」的初始策略（避免強化學習初期完全隨機的危險探索），再用強化學習微調到最佳。這個組合兼顧了樣本效率與最終性能——模仿學習提供了好的起點，強化學習則能超越示範者的水準（因為它不受限於專家的表現）。第 28.9 節的腿式運動、以及 AlphaGo 的訓練流程，都採用了這個架構。

31.5 強化學習的框架

31.5.1 馬可夫決策過程

強化學習的標準數學框架是**馬可夫決策過程** (Markov Decision Process, MDP)，由五個要素構成：

表 31-6 MDP 的五個要素

要素	符號	意義	機器人上的對應
狀態空間	S	所有可能的狀態	關節角度、速度、感測讀值
動作空間	A	所有可能的動作	關節力矩、速度指令
轉移機率	$P(s' s,a)$	在 s 做 a 後到達 s' 的機率	機器人動力學 (第 13 章)
獎勵函數	$R(s,a)$	行為好壞的評分	由設計者定義 (31.7 節)
折扣因子	$\gamma \in [0,1)$	未來獎勵的重要性	典型 0.95~0.99

第三列值得特別注意：轉移機率 $P(s'|s,a)$ 就是**系統的動力學**。第 13、14 章我們花了整整兩章推導它；而在強化學習中，它被假設為**未知**——這正是兩者最根本的差異。

31.5.2 目標：最大化回報

強化學習的目標是找出一個**策略** $\pi(a|s)$ ，使**期望回報**最大：

$$G = r_0 + \gamma \cdot r_1 + \gamma^2 \cdot r_2 + \gamma^3 \cdot r_3 + \dots$$

回報 (return) 是未來所有獎勵的折扣累加。

折扣因子 γ 有兩個作用：**數學上**使無限長的序列收斂；**直覺上**代表「未來的獎勵不如現在的**值錢**」。

$\gamma = 0.99 \rightarrow$ 有效視野約 100 步

$\gamma = 0.95 \rightarrow$ 有效視野約 20 步

$\gamma = 0.9 \rightarrow$ 有效視野約 10 步

有效視野 $\approx 1/(1-\gamma)$ 。這是一個實用的判準—— γ 應依任務所需的規劃長度來選。

γ 的選擇有一個實務上的取舍：太小則策略「短視」（只顧眼前）、太大則學習困難（因為要考慮的未來太長，訊號被稀釋）。

31.5.3 價值函數與 Q 函數

與第 18.3.2 節完全相同，我們定義**價值函數**：

$$V^{\pi}(s) = E[G \mid s_0 = s, \text{依策略 } \pi \text{ 行動}]$$

由狀態 s 出發、依策略 π 行動，能得到的期望回報。

強化學習還常用另一個相關的函數——**Q 函數**（動作價值函數）：

$$Q^\pi(s,a) = E[G \mid s_0 = s, a_0 = a, \text{ 之後依 } \pi \text{ 行動 }]$$

在狀態 s 先做動作 a 、之後依 π 行動的期望回報。

Q 函數比 V 函數實用的地方在於：它直接告訴你該做哪個動作：

$$\text{最佳策略：} \pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$$

只要有了 Q^* ，最佳策略就是「選 Q 值最大的動作」——不需要知道環境的模型。這正是 Q 函數在無模型情境下的價值。

31.6 伏筆回收：強化學習與最佳控制

31.6.1 同一條方程式

現在可以回收第 18 章的伏筆。強化學習的核心方程式是：

$$V^*(s) = \max_a [R(s,a) + \gamma \cdot \sum_{s'} P(s'|s,a) \cdot V^*(s')]$$

Bellman 最佳方程（強化學習版本）。

而第 18.4.1 節的最佳控制版本是：

$$V_k(x) = \min_u [L(x,u) + V_{k+1}(f(x,u))]$$

Bellman 方程（最佳控制版本）。

兩者是同一條方程式。差別只在三個表面的地方：

表 31-7 最佳控制與強化學習的對照（完整版）

最佳控制（第 18 章）	強化學習（本章）	說明
狀態 x	狀態 s	相同
控制輸入 u	動作 a	相同
代價 L （最小化）	獎勵 r （最大化）	符號相反，本質相同
系統模型 $f(x,u)$	轉移機率 $P(s' s,a)$	確定性 vs 隨機
模型已知	模型未知	最關鍵的差異
價值函數 $V(x)$	價值函數 $V(s)$	相同概念
動態規劃（計算）	時序差分學習（取樣）	求解方式不同
有限或無限時域	折扣因子 γ	處理無限未來的方式

31.6.2 關鍵的差異：模型是否已知

表中的第五列是唯一本質的差異，而它導致了完全不同的求解方式：

最佳控制知道模型，因此可以**計算**：對每一個狀態，直接算出「做這個動作會到哪裡、值多少」。第 18.4.2 節的動態規劃就是這樣由後往前推的。

強化學習不知道模型，因此只能**取樣**：實際去做一個動作，看看發生了什麼、得到多少獎勵，再用這個經驗更新估計。

最佳控制： $V(s) \leftarrow \max_a [R + \gamma \sum P(s'|s,a) \cdot V(s')]$ （用模型算期望值）

強化學習： $V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$ （用一次經驗更新）

後者稱為**時序差分學習 (TD Learning)**。中括號中的量稱為**TD 誤差**——「實際看到的」與「原本預期的」之差。

◎ TD 誤差：一個優美的想法

時序差分學習的核心是一個極簡的想法：我原本以為這個狀態值 $V(s)$ ，但走了一步之後看到獎勵 r 且來到了值 $V(s')$ 的狀態，所以它其實應該值 $r + \gamma V(s')$ 。兩者的差 $\delta = r + \gamma V(s') - V(s)$ 就是我的估計錯了多少，據此修正即可。這個想法的美妙之處在於：不需要等到回合結束才知道結果，走一步就能學一點。這與人類的學習方式驚人地相似——你不需要下完整盤棋才知道剛才那步是好是壞，走完立刻就有感覺。神經科學研究甚至發現，大腦中多巴胺神經元的活動模式與 TD 誤差高度吻合。

31.6.3 這個關聯的三個啟示

理解兩者的同源，帶來三個實用的啟示：

啟示一：有模型就別用強化學習。若你能寫出動力學方程（如第 13 章的機械手臂），用第 18 章的 LQR 或 MPC 會快得多、準得多、且有保證。強化學習是「沒有模型時的備案」，不是更先進的替代品。

啟示二：可以用模型加速學習。若有一個近似的模型，可以用它產生虛擬經驗來訓練，減少真實互動的次數。這稱為**模型基礎的強化學習 (Model-based RL)**，樣本效率通常高出一到兩個數量級。

啟示三：第 18 章的直覺仍然適用。維數詛咒、價值函數的意義、bang-bang 特性——這些在強化學習中同樣存在。第 18.4.3 節提到的「用函數近似逼近價值函數」，正是深度強化學習

的核心作法。

31.7 獎勵函數的設計

31.7.1 最困難也最被低估的一環

強化學習的教科書往往聚焦於演算法，但實務上真正困難的是獎勵函數的設計。原因是：

演算法有標準答案（PPO、SAC 等都是現成的）

獎勵函數沒有一它必須為每個任務重新設計

而獎勵函數決定了機器人會學到什麼。

一個常見的經驗是：同一個演算法，換一個獎勵函數，結果可能天差地別。許多「演算法無效」的案例，實際上是獎勵設計的問題。

31.7.2 稀疏獎勵與稠密獎勵

獎勵函數有兩種基本形式，各有優劣：

表 31-8 稀疏獎勵與稠密獎勵的比較

面向	稀疏獎勵	稠密獎勵
定義	只在成功時給獎勵	每一步都給部分獎勵
範例	抓到物體 +1，否則 0	距離目標越近獎勵越高
設計難度	低（不易出錯）	高（容易誤導）
學習難度	極高（探索困難）	低（有明確方向）
風險	可能永遠學不到	可能學到錯的東西
適用	簡單任務、有好的探索機制	複雜任務、需要引導

稀疏獎勵的困難在於探索：若隨機動作幾乎不可能成功，機器人就永遠得不到任何正回饋，也就無從學起。這稱為探索問題。

稠密獎勵的困難在於設計：你必須把「什麼叫做進步」量化——而這往往比想像中困難。下一節的獎勵駭客正是這個困難的體現。

31.7.3 獎勵駭客

獎勵駭客（Reward Hacking）是強化學習中最惡名昭彰的問題：機器人找到了獲得高獎勵的方法，但那不是你想要的行為。

◎ 三個經典的獎勵駭客案例

案例一：抓取任務。獎勵設為「夾爪與物體的距離越近越好」。結果機器人學會了把夾爪貼在物體旁邊來回摩擦——距離永遠很近，獎勵拿滿，但從未抓起任何東西。**案例二：行走任務。**獎勵設為「質心的前進距離」。結果機器人學會了往前撲倒——一次撲倒能前進很多，摔倒後重置再撲一次。**案例三：清潔任務。**獎勵設為「偵測到的髒污減少」。結果機器人學會了把攝影機轉開不看髒的地方。這三個案例的共通點是：演算法完美地達成了你「寫下的」目標，而不是你「想要的」目標。

因應獎勵駭客有三個原則：

- **盡量使用稀疏或半稀疏的獎勵：**越接近「真正的目標」，被鑽漏洞的空間越小。
- **加入懲罰項：**對不希望的行為明確扣分（如「摔倒 -10」、「關節超限 -5」、「能量消耗」）。
- **觀察行為而非只看分數：**訓練時務必實際看機器人在做什麼。分數上升不代表學對了——這是最重要也最常被省略的一步。

31.7.4 實務上的獎勵設計

以第 28 章的四足行走為例，一個典型的獎勵函數包含多項：

```
def compute_reward(state, action, next_state):
    """四足機器人行走的典型獎勵函數"""
    r = 0.0

    # ---- 主要目標：追蹤指令速度 ----
    vel_error = np.linalg.norm(next_state.base_vel[:2] - cmd_vel[:2])
    r += 1.0 * np.exp(-vel_error**2 / 0.25)      # 越接近指令速度越高

    # ---- 姿態獎勵：保持身體水平與高度 ----
    r += 0.5 * np.exp(-abs(next_state.base_height - 0.35)**2 / 0.01)
    r -= 0.5 * abs(next_state.base_roll)        # 懲罰側傾
    r -= 0.5 * abs(next_state.base_pitch)

    # ---- 平滑性：懲罰劇烈的動作（呼應第 15 章的急跳度） ----
    r -= 0.01 * np.sum(action**2)               # 力矩大小
    r -= 0.05 * np.sum((action - prev_action)**2) # 動作變化率

    # ---- 能耗：懲罰無效的功 ----
    power = np.sum(np.abs(action * next_state.joint_vel))
    r -= 0.001 * power
```



```
# ---- 安全約束：明確懲罰不希望的狀態 ----
if next_state.base_height < 0.15: r -= 10.0 # 趴下
if abs(next_state.base_roll) > 1.0: r -= 10.0 # 翻覆
r -= 1.0 * np.sum(next_state.joint_limit_violation)

# ---- 步態品質：鼓勵抬腳（避免拖行） ----
r += 0.1 * np.sum(next_state.foot_clearance > 0.03)

return r
```

注意這個獎勵函數的結構：一個主要目標（速度追蹤）、數個輔助項（姿態、平滑、能耗）、以及明確的懲罰（翻覆、超限）。這是實務上的典型配置。

各項的權重需要大量的調整——這是強化學習中最耗時的工作之一，也是為何第 30 章的快速模擬如此重要（每次調整都要重新訓練）。

31.8 主要的強化學習方法

31.8.1 兩大 family

強化學習的演算法可分為兩大類，其差別在於學什麼：

價值型方法 (Value-based)：學習 Q 函數，動作由 $\arg\max$ 決定。

- 代表：Q-learning、DQN（深度 Q 網路）。
- 優點：樣本效率較高（可重複使用歷史經驗）、原理直觀。
- 限制：只適用於離散的動作空間——因為 $\arg\max$ 需要列舉所有動作。

策略型方法 (Policy-based)：直接學習策略 $\pi(a|s)$ ，不經過價值函數。

- 代表：REINFORCE、PPO、TRPO。
- 優點：能處理連續動作（機器人的力矩、速度都是連續的）、可表示隨機策略。
- 限制：樣本效率較低、訓練較不穩定。

機器人幾乎都用策略型或其變體，原因很簡單——關節力矩是連續的，無法列舉。

31.8.2 Actor-Critic：兩者的結合

Actor-Critic 架構結合了兩者的優點，是目前的主流：

Actor (演員) : 策略網路 $\pi(a|s)$, 負責決定動作

Critic (評論家) : 價值網路 $V(s)$ 或 $Q(s,a)$, 負責評估好壞

Actor 做決定、Critic 給回饋——這個分工使學習更穩定。

為何需要 Critic ? 純策略梯度需要等到回合結束才知道結果 (變異極大) ; 有了 Critic , 每一步都能得到「這個動作比預期好還是壞」的即時評估——這正是 31.6.2 節 TD 誤差的應用。

31.8.3 PPO : 機器人上最常用的演算法

PPO (Proximal Policy Optimization) 是目前機器人強化學習的事實標準。第 28.9 節的四足運動、多數的 Isaac Gym 範例, 用的都是它。

它解決的核心問題是: 策略更新的步伐該多大?

更新太小 → 學得太慢

更新太大 → 策略突然變差, 且可能再也回不來

後者是強化學習特有的危險——因為策略變差後, 收集到的資料也跟著變差, 形成惡性循環。

PPO 的解法極為務實: 限制每次更新的幅度, 不要讓新策略與舊策略差太多。

$$L = E[\min(\text{ratio} \cdot \hat{A}, \text{clip}(\text{ratio}, 1-\epsilon, 1+\epsilon) \cdot \hat{A})]$$

PPO 的裁剪目標函數。ratio 是新舊策略的機率比、 $\hat{A}$ 是優勢估計、 ϵ 典型取 0.2。clip 函數限制了更新的幅度。

PPO 之所以成為標準, 不是因為它最先進, 而是因為它「夠好且好調」——超參數不敏感、實作簡單、在各種任務上都能穩定運作。這是工程上極為可貴的特質 (呼應第 26 章 Pure Pursuit 「參數少」的價值) 。

表 31-9 常見強化學習演算法的比較

演算法	類型	動作空間	樣本效率	穩定性	機器人適用性
DQN	價值型	離散	中	中	低 (動作需離散化)
DDPG	Actor-Critic	連續	高	低 (易發散)	中
TD3	Actor-Critic	連續	高	中	中高
SAC	Actor-Critic	連續	高	高	高 (真機訓練常用)
PPO	策略型	連續	低	高	極高 (模擬訓練標準)

選擇的判準: 若在模擬中訓練 (樣本便宜), 用 PPO——穩定且好調。若必須在真機上訓練 (樣本昂貴), 用 SAC——樣本效率高出許多。

31.8.4 樣本效率的問題

強化學習最大的實務障礙是**樣本效率極低**。以 PPO 訓練一個四足行走策略為例，典型需要 1 億到 10 億步的互動。

$$10 \text{ 億步} \times 0.02 \text{ 秒/步} = 2000 \text{ 萬秒} \approx 231 \text{ 年 (真機)}$$

這就是為何強化學習幾乎必須在模擬中進行——第 30 章的內容不是選配，而是必要條件。

三個提升樣本效率的方向：

- **模型基礎的方法**：用學到的模型產生虛擬經驗（31.6.3 節的啟示二）。
- **離線策略方法 (Off-policy)**：如 SAC，能重複使用歷史經驗，效率高出 10~100 倍。
- **從示範出發**：先用模仿學習取得初始策略（31.4.4 節的知識框），大幅減少探索的需求。

31.9 完整的學習流程

31.9.1 七個步驟

整合前面各節，一個完整的機器人強化學習專案包含七個步驟：

1. **問題形式化**：定義狀態空間、動作空間、以及成功的判準。**這一步決定了後續一切——狀態若不含必要的資訊（如未觀測的接觸狀態），策略再好也學不會。**
2. **建立模擬環境（第 30 章）**：包含機器人模型、環境、感測器模擬。優化速度以支援大規模訓練。
3. **設計獎勵函數（31.7 節）**：主要目標加輔助項加懲罰。這一步通常需要多次迭代。
4. **設定領域隨機化（第 30.5 節）**：先做系統辨識取得標稱值，再設定範圍。
5. **訓練**：以 PPO 或 SAC 在平行環境中訓練。**務必同時觀察行為與分數。**
6. **在模擬中驗證**：測試泛化能力——用訓練時沒見過的地形、物體、擾動。
7. **部署到實機（第 30.9 節）**：漸進式驗證，由低速與受限環境開始。

其中**步驟一與步驟三最耗時間**，卻最常被低估。多數初學者急著跑演算法，結果在錯誤的問題形式化或獎勵設計上浪費數週。

31.9.2 觀測空間的設計

步驟一中最關鍵的決定是：**策略能看到什麼**。這比想像中重要得多。

以四足機器人為例，典型的觀測包含：

表 31-10 四足機器人的典型觀測空間

類別	內容	維度	說明
本體感覺	關節角度與速度	24	編碼器直接讀取，最可靠
	軀幹角速度與重力方向	6	由 IMU 取得 (第 24 章)
指令	期望的前進與轉向速度	3	由上層或遙控器給定
歷史	前幾步的動作與觀測	可變	使策略能推斷未觀測的狀態
外部感知	地形高度圖 (可選)	大	第 22 章的深度感知

第四列的「歷史」極為關鍵。單一時刻的觀測無法分辨「地面是硬的還是軟的」、「負載是輕還是重」——但一段歷史可以 (呼應第 30.5.5 節的解釋二)。這正是第 28.9.3 節「教師—學生」架構中，學生能從有限感測推斷隱藏狀態的原因。

31.9.3 除錯的方法

強化學習的除錯特別困難——它可能「沒有錯誤訊息但就是學不起來」。五個實用的診斷步驟：

- **先確認環境正確**：手動下達已知正確的動作，確認機器人的行為與獎勵符合預期。**環境有錯是最常見的問題。**
- **簡化任務**：先讓它學會最簡單的版本 (如「站著不動」而非「行走」)。若連這都學不會，問題在環境或獎勵，不在演算法。
- **檢查獎勵的量級**：各項的數值範圍應該相當。若某一項是 100 而另一項是 0.01，後者形同不存在。
- **觀察行為而非分數**：錄影看機器人在做什麼。獎勵駭客 (31.7.3 節) 只有這樣才發現得了。
- **檢查觀測的正規化**：神經網路對輸入的尺度敏感。關節角度 (± 3) 與速度 (± 20) 的量級差異應該正規化。

31.10 學習方法的限制與展望

31.10.1 四個根本的限制

儘管成果驚人，學習方法仍有四個根本的限制，理解它們才能正確地使用：

限制一：沒有保證。第 19 章能證明「這個控制器一定穩定」；學習方法只能說「在測試的一萬個案例中成功了 99%」。對安全關鍵的系統，這個差異是決定性的。

限制二：分布外行為不可預測。策略在訓練涵蓋的範圍內表現良好，但遇到完全沒見過的情況時，行為可能任意奇怪——而且它不會告訴你「我不確定」。

限制三：難以除錯與修改。若一個學好的策略有某個特定的缺陷（如「遇到玻璃就撞上去」），你無法像改程式一樣直接修正——只能重新設計獎勵並重新訓練，而這可能破壞其他已經學好的行為。

限制四：樣本效率。31.8.4 節算過，即使有模擬，訓練仍需數小時到數天。每一次獎勵函數的調整都要重來。

31.10.2 因應的架構

面對這些限制，實務上的因應是分層與混合：

表 31-11 學習與模型的分層架構

層級	方法	理由
安全監督層	模型式的規則檢查	必須有保證——如力矩極限、工作空間、緊急停止
高階決策	學習或大型模型	需要判斷與泛化（第 32 章）
中層技能	學習（RL 或模仿）	難以建模但有大量資料
低階控制	模型式（PID、WBC）	有明確物理規律且需要保證

最上層的安全監督層是關鍵：無論學習的策略輸出什麼，都必須經過模型式的檢查——超出力矩極限就限制、要撞到人就停止。這使系統即使在學習出錯時也不會造成災難。

這正是第 21.9.5 節「學習產生候選、解析驗證品質」的推廣——學習可以犯錯，但錯誤必須被攔截。

31.10.3 近年的方向

三個值得關注的發展：

方向一：大規模預訓練。與其為每個任務從頭訓練，不如先在大量的資料上預訓練一個通用的模型，再針對特定任務微調。這正是第 32 章的主題，也是目前最受矚目的方向。

方向二：安全強化學習。在學習過程中明確地納入安全約束——例如用第 19 章的 Lyapunov 函數或控制屏障函數 (CBF) 限制探索的範圍。這試圖為學習方法補上「保證」這一塊。

方向三：離線強化學習。從已收集的資料中學習，完全不需要新的互動。這對真機訓練極有價值——可以用歷史的操作紀錄訓練，而不需要冒險探索。

31.11 由學習邁向理解

31.11.1 本章的成果

本章建立了機器人學習的完整圖像。你現在能夠：判斷何時該用模型、何時該用學習、區分三種學習典範、理解模仿學習的複合誤差問題與 DAgger 的解法、掌握 MDP 框架與價值函數、設計獎勵函數並避免獎勵駭客、選擇適當的強化學習演算法、並設計完整的學習流程與安全架構。

我們也回收了第 18 章最重要的伏筆：**強化學習與最佳控制共用同一條 Bellman 方程**，差別只在於「模型是否已知」。這個認識帶來三個實用的啟示——有模型就別用強化學習、可以用模型加速學習、以及第 18 章的所有直覺仍然適用。

最重要的是那個貫穿全章的分工原則：**模型提供保證，學習提供泛化**。這不是誰取代誰，而是各自處理擅長的部分——與第 17 章「模型處理已知、回授處理未知」、第 25 章「學習做前端、幾何做後端」，是同一種工程智慧的不同體現。

31.11.2 接下來：從技能到任務

但本章的方法有一個共同的限制：**它們學的都是「技能」，而非「任務」**。

一個強化學習訓練出的策略，可能極為擅長「走路」或「抓取圓柱體」。但若你對它說「**幫我把桌上的杯子拿到廚房，順便看看冰箱裡還有沒有牛奶**」——它完全無法理解，因為：

- 它不懂**語言**。
- 它不會**分解任務**（要先導航、再辨識、再抓取、再導航、再放置）。
- 它沒有**常識**（杯子要正著拿、牛奶在冰箱裡、開冰箱要拉門把）。
- 它無法**處理未預期的情況**（若杯子是滿的怎麼辦？）。

第 32 章（大型模型與任務規劃）將處理這個層次的問題。近年大型語言模型與視覺語言模

型的進展，使機器人第一次具備了「理解指令並自行分解任務」的能力。我們會探討：如何把語言連接到動作、如何確保生成的計畫是可執行的、以及這些模型帶來的新問題（它們會**自信地說錯話**，而在機器人上這意味著危險的動作）。

第 33 章 (具身人工智慧) 則是整個第五篇的收束，將回到第 1 章的 Moravec 悖論——**為何對電腦而言下棋容易而摺衣服困難？智慧是否需要身體？**

換句話說：第 31 章讓機器人**學會做一件事**，第 32 章讓它**知道該做哪些事**。

完整計算例題

例題 31-1 折扣因子與有效視野

題目：某機器人的控制頻率為 50 Hz。(a) 若 $\gamma = 0.99$ ，求有效視野的步數與對應的時間。(b) 若任務需要規劃未來 5 秒的行為，求所需的 γ 。(c) 若 $\gamma = 0.9$ ，計算 100 步之後的獎勵權重。(d) 討論 γ 過大或過小的後果。

解 (a)：有效視野的常用估計為 $1/(1-\gamma)$ ：

$$\text{有效視野} = 1/(1 - 0.99) = 100 \text{ 步}$$

步數。

$$\text{時間} = 100 \text{ 步} / 50 \text{ Hz} = 2.0 \text{ 秒}$$

策略大約只「看到」未來 2 秒。

解 (b)：5 秒對應的步數為 $5 \times 50 = 250$ 步：

$$1/(1 - \gamma) = 250 \Rightarrow \gamma = 1 - 1/250 = 0.996$$

所需的折扣因子。

注意 γ 對視野的敏感度：由 0.99 提高到 0.996（只差 0.006），視野卻由 2 秒延長到 5 秒。**這是一個高度非線性的關係**——這也解釋了為何 γ 的微調常有出乎意料的效果。

解 (c)：第 100 步的獎勵權重為 γ^{100} ：

$$0.9^{100} = 2.66 \times 10^{-5}$$

權重只有 0.0027%——形同不存在。

這說明 $\gamma = 0.9$ 的策略**完全看不到 100 步之後的事**。若任務需要長期規劃（如「先繞路避開

障礙，雖然短期較慢但長期較快」)，這個設定必定失敗。

解 (d)：兩端的後果：

- γ 太小 (如 0.9)：策略「短視」，只顧眼前的獎勵。在需要犧牲短期利益的任務中會失敗 (如必須先後退才能繞過障礙)。
- γ 太大 (如 0.999)：有效視野達 1000 步，價值函數的估計變異極大、學習極為緩慢。且遠期的獎勵訊號被稀釋，難以分辨哪個動作真正有貢獻 (信用分配問題)。

實務建議：由任務所需的規劃長度反推 γ ，而非盲目取一個「常見值」。多數機器人任務的合理範圍是 0.95~0.995。

例題 31-2 複合誤差的估算

題目：某模仿學習訓練出的策略，單步的動作誤差率為 $\varepsilon = 0.02$ (2%)。(a) 依線性模型估計 100 步後的累積誤差。(b) 依複合誤差的 $O(\varepsilon T^2)$ 模型重新估計。(c) 比較兩者。(d) 若要把 100 步的失敗率控制在 10% 以下，求所需的單步誤差率。

解 (a)：線性模型假設誤差獨立累加：

$$\text{累積誤差} \approx \varepsilon \times T = 0.02 \times 100 = 2.0$$

但機率不能超過 1——這顯示線性模型在此已不適用。

解 (b)：複合誤差模型的失敗率上界為：

$$\text{失敗率} \approx \varepsilon \cdot T^2 = 0.02 \times 100^2 = 200$$

遠大於 1，代表幾乎必定失敗。

(嚴格地說，理論給出的是「期望的總代價上界為 $O(\varepsilon T^2)$ 」，實務上表現為「軌跡幾乎必定偏離」。))

解 (c)：兩者的比值為 $T = 100$ 倍。這正是複合誤差的可怕之處——它不是線性累加，而是與時間的平方成正比。

直觀的理解：每一次小偏離都會把機器人帶到「訓練資料中沒有的狀態」，而在那裡它的表現更差，於是偏離更多——這是一個正回饋。

解 (d)：要求 $\varepsilon \cdot T^2 \leq 0.1$ ，其中 $T = 100$ ：

$$\epsilon \leq 0.1/10000 = 1 \times 10^{-5}$$

單步誤差率必須低於 0.001%。

工程判讀：這個要求極為嚴苛，純粹的行為複製幾乎不可能達成。這正是 31.4.3 節 DAgger 存在的理由——它改變了資料的分布，使「偏離後如何修正」也在訓練資料中，從而把誤差成長由 $O(\epsilon T^2)$ 降為 $O(\epsilon T)$ 。

另一個實務上的因應是縮短 T ：把長任務分解為多個短片段，每段結束時重新定位或由更高層的規劃器介入。這正是第 32 章任務分解的價值之一。

例題 31-3 訓練資源的估算

題目：某四足行走任務，PPO 需要 5 億步的互動才能收斂。模擬環境的單環境速度為每秒 2000 步。(a) 單環境訓練需要多久？(b) 若用 4096 個平行環境 (GPU)，總吞吐量為每秒 300 萬步，需要多久？(c) 若獎勵函數需要調整 20 次，每次都要重新訓練，求總時間。(d) 討論這對開發流程的影響。

解 (a)：單環境：

$$5 \times 10^8 / 2000 = 250,000 \text{ 秒} = 69.4 \text{ 小時} \approx 2.9 \text{ 天}$$

近三天。

解 (b)：平行環境：

$$5 \times 10^8 / 3 \times 10^6 = 167 \text{ 秒} \approx 2.8 \text{ 分鐘}$$

不到三分鐘——這正是第 30.8.2 節 GPU 平行的威力。

解 (c)：二十次迭代：

$$\text{單環境：} 2.9 \text{ 天} \times 20 = 58 \text{ 天 (約兩個月)}$$

$$\text{平行環境：} 2.8 \text{ 分鐘} \times 20 = 56 \text{ 分鐘 (不到一小時)}$$

差距約 1500 倍。

解 (d)：這個差距根本地改變了開發流程：

- **單環境 (兩個月)：**每次調整獎勵都要等三天，一個學期只能試十幾次。研究者必須極為謹慎地設計每一次實驗，且無法探索大量的想法。
- **平行環境 (一小時)：**可以在一個下午試十次不同的獎勵設計。這使「快速迭代」成為可能

——可以先粗略地試，看到結果再修正。

這正是第 30.8.2 節「量變引起質變」的具體體現。它不只是「快一點」，而是使一種完全不同的工作方式成為可能——從「深思熟慮的少數實驗」轉變為「快速迭代的大量嘗試」。第 28.9 節腿式機器人的突破，很大程度上來自這個轉變。

程式範例：從 Bellman 方程到 Q-learning

以下程式用一個極簡的網格世界，展示「已知模型的動態規劃」與「未知模型的 Q-learning」如何求解同一個問題——具體呈現第 18 章與本章的關係。

```
import numpy as np

# ===== 環境：一個 4x4 的網格世界 =====
class GridWorld:
    """
    機器人在 4x4 網格中移動，目標是到達右下角。
    有一個「陷阱」格子會給予懲罰。
    這是 MDP 的最小範例 (31.5.1 節)。
    """
    def __init__(s, size=4):
        s.size = size
        s.goal = (size-1, size-1)
        s.trap = (1, 2)
        s.actions = [(0,1), (0,-1), (1,0), (-1,0)] # 右左下上
        s.n_states = size * size
        s.n_actions = 4

    def idx(s, pos): return pos[0]*s.size + pos[1]
    def pos(s, idx): return (idx // s.size, idx % s.size)

    def step(s, state_idx, action_idx):
        """執行動作，回傳 (下一狀態，獎勵，是否結束)"""
        r0, c0 = s.pos(state_idx)
        dr, dc = s.actions[action_idx]
        r, c = np.clip(r0+dr, 0, s.size-1), np.clip(c0+dc, 0, s.size-1)
        nxt = s.idx((r, c))
        if (r, c) == s.goal: return nxt, 10.0, True
        if (r, c) == s.trap: return nxt, -10.0, True
        return nxt, -0.1, False # 每步小懲罰 (鼓勵快速到達)

# ===== 方法一：動態規劃 (第 18 章，已知模型) =====
def value_iteration(env, gamma=0.95, tol=1e-6):
    """
    價值迭代：用 Bellman 方程直接「計算」(31.6.2 節)
    前提：知道 step() 函式，也就是知道模型
    """
    V = np.zeros(env.n_states)
```

```

    iters = 0
    while True:
        delta = 0.0
        for si in range(env.n_states):
            if env.pos(si) in (env.goal, env.trap): continue
            # Bellman 最佳方程 :  $V(s) = \max_a [ R + \gamma \cdot V(s') ]$ 
            q_values = []
            for ai in range(env.n_actions):
                nxt, r, done = env.step(si, ai)
                q_values.append(r + (0 if done else gamma*V[nxt]))
            v_new = max(q_values)
            delta = max(delta, abs(v_new - V[si]))
            V[si] = v_new
        iters += 1
        if delta < tol: break
    return V, iters

# ===== 方法二：Q-learning (本章·未知模型) =====
def q_learning(env, episodes=3000, alpha=0.1, gamma=0.95,
               eps_start=1.0, eps_end=0.05, seed=0):
    """
    Q-learning：只能「試」·不能「算」(31.6.2 節)
    它不使用 env 的內部結構·只透過 step() 取得經驗
    """
    rng = np.random.default_rng(seed)
    Q = np.zeros((env.n_states, env.n_actions))
    steps_used = 0
    for ep in range(episodes):
        #  $\epsilon$ -greedy 的探索率逐步下降 (探索與利用的取捨)
        eps = eps_end + (eps_start-eps_end)*np.exp(-3.0*ep/episodes)
        si = 0 # 由左上角出發
        for t in range(50):
            # 選動作：以機率 eps 隨機探索
            if rng.random() < eps:
                ai = rng.integers(env.n_actions)
            else:
                ai = int(np.argmax(Q[si]))
            nxt, r, done = env.step(si, ai)
            steps_used += 1
            # TD 更新 (31.6.2 節的核心)
            target = r + (0 if done else gamma*np.max(Q[nxt]))
            td_error = target - Q[si, ai]
            Q[si, ai] += alpha * td_error
            si = nxt
            if done: break
    return Q, steps_used

# ===== 比較 =====
env = GridWorld()
print("=== 方法一：動態規劃 (已知模型·第 18 章) ===")
V_dp, iters = value_iteration(env)
print(f"收斂所需迭代次數: {iters}")
print("價值函數 V(s):")
for r in range(env.size):
    print(" " + " ".join(f"{V_dp[env.idx((r,c))]:7.3f}" for c in range(env.size)))

```

```

print("\n=== 方法二：Q-learning ( 未知模型 · 本章 ) ===")
Q_rl, steps = q_learning(env)
V_rl = Q_rl.max(axis=1)
print(f"所需的環境互動步數：{steps:,}")
print("由 Q 導出的價值函數 max_a Q(s,a):")
for r in range(env.size):
    print(" " + " ".join(f"{V_rl[env.idx((r,c))]:7.3f}" for c in range(env.size)))

# 比較兩者的差異
mask = np.array([env.pos(i) not in (env.goal, env.trap)
                  for i in range(env.n_states)])
err = np.abs(V_dp[mask] - V_rl[mask]).max()
print(f"\n 兩者的最大差異：{err:.4f}")
print(f"動態規劃：{iters} 次迭代 ( 每次掃過所有狀態 · 需要模型 ) ")
print(f"Q-learning：{steps:,} 步互動 ( 不需模型 · 但要試很多次 ) ")
print("→ 同一條 Bellman 方程 · 兩種求解方式 ( 31.6 節 ) ")

# ===== 導出的策略 =====
print("\n=== 兩者導出的策略 ( →↔↑↓ · G=目標 X=陷阱 ) ===")
arrows = ['→', '↔', '↓', '↑']
def show_policy(Q_or_V, name, use_q=True):
    print(f"{name}:")
    for r in range(env.size):
        row = []
        for c in range(env.size):
            si = env.idx((r,c))
            if (r,c) == env.goal: row.append(' G'); continue
            if (r,c) == env.trap: row.append(' X'); continue
            if use_q:
                ai = int(np.argmax(Q_or_V[si]))
            else:
                qs = []
                for a in range(env.n_actions):
                    nxt, rr, dn = env.step(si, a)
                    qs.append(rr + (0 if dn else 0.95*Q_or_V[nxt]))
                ai = int(np.argmax(qs))
            row.append(' ' + arrows[ai])
        print(" " + " ".join(row))

show_policy(V_dp, "動態規劃的策略", use_q=False)
show_policy(Q_rl, "Q-learning 的策略", use_q=True)

# ===== 折扣因子的影響 ( 例題 31-1 ) =====
print("\n=== 例題 31-1：折扣因子與有效視野 ===")
print(f"{'γ':>8}{ '有效視野(步)':>14}{ '@50Hz(秒)':>12}{ '100 步後權重':>14}")
for g in (0.9, 0.95, 0.99, 0.996, 0.999):
    horizon = 1/(1-g)
    print(f"{g:8.3f}{horizon:13.0f}{horizon/50:11.2f}{g**100:14.2e}")
print("→ γ 由 0.99 到 0.996 只差 0.006 · 視野卻由 2 秒延長到 5 秒")

```

```

# ===== 探索率的影響 =====
print("\n=== 探索策略對學習的影響 ===")
print(f"{'探索設定':<22}{'最終誤差':>12}{'說明':>4}")
for label, e0, e1 in (("完全不探索 ( $\epsilon=0$ )", 0.0, 0.0),
                      ("少量探索 ( $\epsilon=0.1$ )", 0.1, 0.1),
                      ("遞減探索 ( $1 \rightarrow 0.05$ )", 1.0, 0.05),
                      ("持續高探索 ( $\epsilon=0.5$ )", 0.5, 0.5)):
    Qt, _ = q_learning(env, episodes=3000, eps_start=e0, eps_end=e1, seed=1)
    Vt = Qt.max(axis=1)
    e = np.abs(V_dp[mask] - Vt[mask]).max()
    print(f"{'label':<22}{'e':12.4f}")
print("> 完全不探索會困在局部；遞減探索（先探索後利用）效果最好")

# ===== 獎勵設計的影響（31.7 節）=====
print("\n=== 獎勵設計對行為的影響 ===")
class ModifiedGrid(GridWorld):
    """把每步的懲罰改掉·觀察行為變化"""
    def __init__(s, step_penalty=-0.1, size=4):
        super().__init__(size); s.pen = step_penalty
    def step(s, si, ai):
        r0, c0 = s.pos(si)
        dr, dc = s.actions[ai]
        r, c = np.clip(r0+dr,0,s.size-1), np.clip(c0+dc,0,s.size-1)
        nxt = s.idx((r,c))
        if (r,c) == s.goal: return nxt, 10.0, True
        if (r,c) == s.trap: return nxt, -10.0, True
        return nxt, s.pen, False

print(f"{'每步懲罰':>10}{'到達目標的步數':>16} 行為")
for pen in (0.0, -0.1, -1.0, -3.0):
    e2 = ModifiedGrid(step_penalty=pen)
    V2, _ = value_iteration(e2)
    # 依策略走一遍·計算步數
    si, n = 0, 0
    for _ in range(50):
        qs = []
        for a in range(e2.n_actions):
            nxt, rr, dn = e2.step(si, a)
            qs.append(rr + (0 if dn else 0.95*V2[nxt]))
        si, _, done = e2.step(si, int(np.argmax(qs)))
        n += 1
        if done: break
    behav = ("不急著到達" if pen == 0 else
            "正常" if pen >= -1.0 else "急於結束（可能冒險）")
    print(f"{'pen':10.1f}{'n':16d} {behav}")
print("> 獎勵的細節直接決定了行為——這正是 31.7 節的核心")

```

請試著修改：（一）把陷阱的懲罰由 -10 改為 -100 ，觀察策略是否變得更保守（繞更遠的路避開）。（二）把 Q-learning 的學習率 α 設為 0.9 ，觀察是否變得不穩定。（三）加入一個

「牆壁」使某些格子不可通行，觀察兩種方法是否都能找到繞路。

實作：訓練你的第一個機器人策略

本章實作以模擬環境為主。建議延續第 30 章建立的環境，或使用現成的 Gymnasium 與 Isaac Gym 範例。

1. **環境理解**：先用現成的環境（如 Gymnasium 的 Pendulum 或 CartPole）跑通流程。**手動下達動作**，觀察狀態與獎勵，確認自己理解了 MDP 的五個要素。
2. **動態規劃 vs Q-learning**：實作本章的程式範例，親眼看見兩者求解同一問題的差異。記錄各自所需的計算量。
3. **PPO 訓練**：用現成的實作（如 Stable-Baselines3）訓練一個簡單的任務。記錄學習曲線。
4. **折扣因子實驗**：改變 γ （0.9、0.95、0.99、0.999），觀察對學習速度與最終行為的影響。驗證例題 31-1 的分析。
5. **獎勵設計實驗**：對同一個任務設計三種不同的獎勵函數（稀疏、稠密、含懲罰項），比較學習速度與最終行為。**特別觀察是否出現獎勵駭客**。
6. **觀測空間實驗**：故意移除某些觀測（如速度資訊），觀察策略是否還能學會。接著加入歷史觀測，驗證 31.9.2 節「歷史使策略能推斷隱藏狀態」的說法。
7. **行為複製**：手動（或用一個已知的控制器）產生一批示範資料，用監督式學習訓練一個策略。**觀察複合誤差**——策略是否在執行一段時間後逐漸偏離。
8. **DAGger 實作**：對上一步的策略實作 DAGger——讓它自己跑、在它走過的狀態上用專家控制器標註、重新訓練。比較改善的幅度。
9. **領域隨機化**：把第 30.5 節的隨機化加入訓練，比較「有無隨機化」的策略在參數變動下的表現。
10. **（進階）四足或機械手臂**：若有 Isaac Gym 或類似環境，訓練一個四足行走或手臂到達的策略。這會讓你體會 31.8.4 節樣本效率的問題。
11. **（進階）安全層實作**：在學習的策略外加上一層模型式的安全檢查（力矩限制、工作空間限制），驗證即使策略輸出異常也不會造成危險。

表 31-12 強化學習實驗記錄表（前兩列為範例）

實驗項目	設定	結果	觀察	下一步
折扣因子	$\gamma = 0.9$	收斂快但行為短視	不會繞路避障	提高至 0.99
獎勵設計	只有距離獎勵	在目標旁徘徊	獎勵駭客	改用稀疏 + 懲罰
(自行填寫其餘實驗)				

國中小也看得懂：兩種學習方式

你是怎麼學會騎腳踏車的？

是先讀完一本《腳踏車力學原理》，把公式都背熟，然後一次就騎成功嗎？當然不是！你是一直摔、一直試——摔一次就知道「剛才那樣不行」，慢慢地身體就記住了怎麼保持平衡。

而且很有趣的是：你到現在都說不出來自己是怎麼做到的。如果有人問你「車子往左倒的時候，龍頭要轉幾度？」你根本答不出來——但你就是會騎。

機器人也有這兩種學習方式：

第一種：用算的。 像我們前面學的，先寫出公式，再算出答案。這個方法很準，而且保證不會出錯。但它有一個條件——你必須寫得出公式。

第二種：用試的。 讓機器人自己一直嘗試，做對了就給它「獎勵」（加分），做錯了就扣分。試了幾百萬次之後，它就會找出最好的方法了。

那什麼時候要用哪一種呢？很簡單：

能寫出公式的，就用算的——像是「手臂要轉幾度才能碰到那個點」，這個算得出來，而且算得超級準。

寫不出公式的，就用試的——像是「怎麼把一件皺成一團的衣服摺好」。衣服是軟的、會皺、會黏在一起，沒有人寫得出它的公式。這種就只能讓機器人自己去試。

最後告訴你一件很妙的事：**教機器人最難的部分，不是寫程式，而是「決定怎麼給分」。**

有一次，科學家想教機器人抓東西，就設定「夾子離物體越近，分數越高」。結果機器人學會了什麼？它把夾子貼在物體旁邊一直摩來摩去！分數拿滿了，但一次都沒抓起來過。

所以啊，你要很小心地想清楚「什麼才是真正想要的」——不然機器人會找到你想不到的

偷懶方法。這其實跟養小孩很像對不對？

叫獸提醒與常見錯誤

- 誤區一：以為學習方法比模型方法「更先進」。兩者是分工——有物理規律且需要保證的部分，模型永遠是更好的選擇。
- 誤區二：能建模卻用強化學習。若能寫出動力學，LQR 或 MPC 會更快、更準且有保證（31.6.3 節的啟示一）。
- 誤區三：用行為複製處理長時序任務。複合誤差以 $O(\epsilon T^2)$ 成長，必須用 DAgger 或縮短任務長度。
- 誤區四：折扣因子隨便取一個常見值。應由任務所需的規劃長度反推（有效視野 $\approx 1/(1-\gamma)$ ）。
- 誤區五：只看獎勵曲線而不看行為。獎勵上升不代表學對了——獎勵駭客只有實際觀察才發現得了。
- 誤區六：獎勵函數各項的量級不平衡。若一項是 100 而另一項是 0.01，後者形同不存在。
- 誤區七：把稠密獎勵設計得太具體。越接近「你想要的最終結果」越安全，越是中間步驟的引導越容易被鑽漏洞。
- 誤區八：在連續動作空間使用 DQN。價值型方法需要 $\arg\max$ ，只適用於離散動作。
- 誤區九：忽略觀測空間的設計。若狀態不含必要資訊（或缺少歷史），策略再好也學不會。
- 誤區十：在真機上做大規模的隨機探索。樣本效率使其不可行且危險——應在模擬中訓練（第 30 章）。
- 誤區十一：忘記正規化觀測。神經網路對輸入尺度敏感，量級差異大的觀測必須正規化。
- 誤區十二：把學習的策略直接部署而無安全層。必須有模型式的檢查攔截異常輸出（31.10.2 節）。

課堂討論

1. 31.6 節顯示強化學習與最佳控制共用 Bellman 方程，差別只在模型是否已知。請討論：這個認識如何改變你對「該用哪種方法」的判斷？
2. 獎勵駭客的三個案例中，機器人都「完美地達成了寫下的目標」。請討論這個現象對人工智

慧安全的啟示，並嘗試設計一個不容易被鑽漏洞的抓取獎勵函數。

3. 例題 31-3 顯示平行模擬使開發流程由「兩個月」變成「一小時」。請討論這種「量變引起質變」還出現在本書的哪些地方，以及它對研究方法的影響。
4. 31.10.1 節指出學習方法「沒有保證」且「分布外行為不可預測」。請討論：在什麼應用中這是可接受的？在什麼應用中絕對不行？並設計對應的安全架構。
5. 本章與第 21、25、28 章都得出「學習做判斷、模型做保證」的結論。請歸納這個分工原則的共同結構，並討論它是否有例外。

章末摘要

- 模型導向方法的優勢是保證、可解釋與資料效率；其邊界是「問題必須寫得出方程式」。
- 學習方法能處理無法建模的問題，代價是需要資料、沒有保證、且分布外行為不可預測。
- 模型與學習的三種混合模式：學習做前端、模型做骨架、模型產生資料。
- 三種學習典範的差別在於回饋的形式：正確答案（監督式）、評分（強化）、無回饋（非監督式）。
- 機器人學習的三個特殊性：資料昂貴、動作有後果、以及序列決策（動作改變後續狀態）。
- 監督式學習在機器人上最成功的應用是感知；其資料來源有人工標註、模擬、解析式標註與自我監督。
- 監督式學習的陷阱是分布偏移——部署環境與訓練不同時，模型可能自信地犯錯。
- 模仿學習從示範中學習，不需設計獎勵、樣本效率高、且安全。
- 行為複製的致命缺陷是複合誤差：誤差以 $O(\epsilon T^2)$ 成長，因為偏離後會進入訓練資料未涵蓋的狀態。
- DAgger 的核心是改變資料來源——在機器人自己走過的狀態上詢問專家，使資料涵蓋「如何修正」。
- MDP 的五個要素：狀態、動作、轉移機率、獎勵、折扣因子；其中轉移機率就是系統動力學。
- 有效視野 $\approx 1/(1-\gamma)$ ； γ 應由任務所需的規劃長度反推，而非取一個常見值。
- 強化學習與最佳控制共用同一條 Bellman 方程，唯一的本質差異是「模型是否已知」。
- 模型已知則可「計算」（動態規劃），未知則只能「取樣」（時序差分學習）。

- TD 誤差 $\delta = r + \gamma V(s') - V(s)$ 是「實際看到的」與「原本預期的」之差，走一步就能學一點。
- 獎勵函數的設計是實務上最困難的一環；稀疏獎勵探索困難、稠密獎勵容易被鑽漏洞。
- 獎勵駭客：機器人完美達成「寫下的」目標而非「想要的」目標；防範之道是使用接近真實目標的獎勵並實際觀察行為。
- 價值型方法只適用於離散動作；機器人的連續動作空間需要策略型或 Actor-Critic 方法。
- PPO 是模擬訓練的事實標準（穩定好調）；SAC 則因樣本效率高而適合真機訓練。
- 學習方法的四個限制：沒有保證、分布外不可預測、難以修改、樣本效率低。
- 因應之道是分層架構：低階用模型式控制、中階用學習、最上層必須有模型式的安全監督。

觀念題與計算題

觀念題

1. 說明模型導向方法的三個優勢與其適用邊界。
2. 列舉四類難以建模的機器人問題，並說明為何模型方法在此失效。
3. 比較三種學習典範的資料形式與回饋，並說明為何強化學習比監督式學習困難。
4. 說明機器人學習相對於影像辨識的三個特殊性。
5. 列舉監督式學習在機器人感知上的五個應用。
6. 說明機器人學習資料的四個來源及其優缺點。
7. 什麼是分布偏移？它在機器人上為何特別危險？有哪三種因應之道？
8. 說明模仿學習的四個優勢。
9. 解釋複合誤差的成因，並說明為何誤差以 $O(\epsilon T^2)$ 而非 $O(\epsilon T)$ 成長。
10. 說明 DAgger 的核心洞見與其四個步驟。
11. 列舉 MDP 的五個要素，並指出哪一個對應機器人的動力學。
12. 說明折扣因子的兩個作用，以及有效視野的估計方式。
13. 區分價值函數與 Q 函數，並說明後者為何在無模型情境下特別實用。
14. 說明強化學習與最佳控制的關係，並指出唯一的本質差異。
15. 解釋 TD 誤差的意義，並說明時序差分學習相對於動態規劃的差異。

16. 說明「有模型就別用強化學習」的理由，以及模型能如何加速學習。
17. 比較稀疏獎勵與稠密獎勵的優缺點。
18. 什麼是獎勵駭客？請說明三個案例的共通點與三個防範原則。
19. 說明價值型與策略型方法的差異，以及為何機器人幾乎都用後者。
20. 說明 PPO 解決的核心問題，以及它成為標準的原因。
21. 列舉學習方法的四個根本限制，並說明因應的分層架構。

計算題

1. 某機器人控制頻率 100 Hz。(a) 若 $\gamma = 0.995$ ，求有效視野的步數與時間。(b) 若任務需規劃未來 8 秒，求所需的 γ 。(c) 計算 $\gamma = 0.95$ 時第 200 步的獎勵權重。(d) 由此討論 γ 的選擇原則。
2. 某模仿學習策略的單步誤差率 $\epsilon = 0.005$ 。(a) 依 $O(\epsilon T^2)$ 模型估計 50 步與 200 步後的累積誤差。(b) 計算兩者的比值並說明。(c) 若要求 200 步的失敗率低於 5%，求所需的 ϵ 。(d) 討論達成此要求的兩種途徑。
3. 某任務用 PPO 需要 3 億步收斂。單環境速度為每秒 1500 步、平行環境 (2048 個) 總速度為每秒 200 萬步。(a) 分別計算兩者的訓練時間。(b) 若獎勵函數需調整 15 次，求總時間。(c) 計算兩者的比值。(d) 討論這對研究方式的影響。
4. 某網格世界的獎勵設定為：到達目標 +20、掉入陷阱 -20、每步 -0.5， $\gamma = 0.9$ 。(a) 若最短路徑為 6 步，計算沿最短路徑的回報。(b) 若繞路避開陷阱需 10 步，計算其回報。(c) 判斷策略會選擇哪一條。(d) 若把每步的懲罰改為 -2.0，重新判斷並討論。

動手做與專題延伸

本章實作：學習框架

建立一套機器人學習的實驗框架，這將是第 32 章的基礎。

- 階段一：實作 MDP 的基本框架與動態規劃，驗證第 18 章與本章的等價性。
- 階段二：實作 Q-learning 與 ϵ -greedy 探索，比較與動態規劃的差異。
- 階段三：接上現成的 PPO 實作 (Stable-Baselines3)，在標準環境上訓練。
- 階段四：實作行為複製與 DAgger，量化複合誤差與改善幅度。

- **階段五**：整合第 30 章的領域隨機化，訓練一個可轉移的策略。

專題延伸

- **Bellman 方程的兩種求解方式對照**：以一個中等規模的網格世界（如 10×10 含障礙），並排實作價值迭代與 Q-learning。繪出「所需計算量 vs 問題規模」的曲線，並記錄兩者收斂到相同解所需的資源。接著把環境改為「模型未知」（不允許動態規劃存取 step 函式），驗證只有 Q-learning 仍然可行。**這個專題能讓第 18 章與第 31 章的關係變得具體。**
- **獎勵駭客的實驗**：刻意設計三個「有漏洞」的獎勵函數（如只獎勵距離、只獎勵速度、只獎勵接觸次數），訓練並錄影記錄機器人學到的行為。你很可能會親眼看到 31.7.3 節描述的那些荒謬行為。接著逐步修正獎勵函數，記錄行為如何改變。**這個專題的教育價值極高——它會讓你永遠記得「要小心你許的願」。**
- **進階：模仿學習與強化學習的組合**。選一個中等難度的任務（如二維的到達或避障）。
 - （一）先用一個已知的控制器（如第 26 章的 DWA）產生示範資料；
 - （二）用行為複製訓練並量測複合誤差；
 - （三）用 DAgger 改善；
 - （四）以模仿學習的結果為初始策略，再用 PPO 微調；
 - （五）比較四種方式的最終性能與所需的樣本數。**你應該會看到組合方式兼具了樣本效率與最終性能**，驗證 31.4.4 節知識框的說法。

參考文獻與延伸閱讀

- Sutton, R. S. and Barto, A. G., Reinforcement Learning: An Introduction, 2nd ed., MIT Press（強化學習的權威教材，線上公開；第 3~6 章對應本章）。
- Levine, S., CS285 Deep Reinforcement Learning 課程講義（UC Berkeley，線上公開）（機器人強化學習的最佳課程資源之一）。
- Schulman, J. et al., "Proximal Policy Optimization Algorithms," arXiv 2017（PPO 的原始論文）。
- Haarnoja, T. et al., "Soft Actor-Critic: Off-Policy Maximum Entropy Deep RL," ICML 2018（SAC，真機訓練的主流選擇）。
- Ross, S., Gordon, G. and Bagnell, D., "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," AISTATS 2011（DAgger 與複合誤差的理論分析）。

- Chi, C. et al., "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," RSS 2023 (擴散策略・模仿學習的近年突破) 。
- Hwangbo, J. et al., "Learning Agile and Dynamic Motor Skills for Legged Robots," Science Robotics, 2019 (腿式機器人的強化學習突破・呼應第 28.9 節) 。
- Kober, J., Bagnell, J. A. and Peters, J., "Reinforcement Learning in Robotics: A Survey," IJRR, 2013 (機器人強化學習的完整綜述) 。
- Amodei, D. et al., "Concrete Problems in AI Safety," arXiv 2016 (獎勵駭客與 AI 安全的系統性討論) 。
- Stable-Baselines3 與 Gymnasium 官方文件：強化學習演算法與環境介面的標準實作 。

第 32 章

大型模型與任務規劃

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為一句話的拆解：中央是「幫我把桌上的杯子拿到廚房」這句話，以四條放射線連向四個問號框——「哪一個杯子？」（畫面中三個不同的杯子）、「廚房在哪？」（地圖上只有座標沒有標籤）、「要哪些步驟？」（六個空白的步驟框）、「什麼是常識？」（杯子要正著拿的圖示）。右半為 SayCan 的機制圖：上方一列是語言模型對五個技能的評分（長條圖）、中間一列是可供性的評分、下方是兩者相乘的結果，並以箭頭標出「Say 高但 Can 低」的技能如何被排除。下緣文字：語言模型知道世界通常是怎樣的，但不知道現在這裡是怎樣的。

叫獸開講 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？

實驗室裡那台服務機器人已經很能幹了。它會導航、會避障、會辨識物體、會抓取——第 22 到 31 章教的東西，它樣樣都會。

叫獸對著麥克風說：「幫我把桌上的杯子拿到廚房。」

機器人靜止不動。螢幕上跳出一行錯誤：`Unknown command`。

「問題在哪裡？」叫獸問。小翰說：「語音辨識沒開？」叫獸搖頭：「辨識沒問題，它聽到了每一個字。問題是它不知道這句話要做什麼。」

他在白板上把這句話拆開：「你想想，要完成這件事，它必須知道多少事情？

第一，什麼是『杯子』？它的視覺模型或許認得杯子，但如果桌上有三個杯子呢？我說的是哪一個？

第二，什麼是『廚房』？地圖上只有座標，沒有『廚房』這個標籤。

第三，要做哪些步驟？導航到桌邊、辨識杯子、規劃抓取、抓起來、導航到廚房、放下。這六步是我說的嗎？不是，是它要自己想出來的。

第四，什麼是常識？杯子要正著拿（不然裡面的東西會灑）、要放在平面上（不能放在地上）、經過門要先確認門是開的。」

小好說：「所以要把這些全部寫成規則？」

「以前確實是這樣做的。」叫獸點頭，「而且做了三十年。但你有沒有發現一個問題——你永遠寫不完。今天寫了『杯子』的規則，明天要處理『碗』；今天處理了『廚房』，明天要處理『陽台』。這就是第 1 章那個 Moravec 悖論的另一面：常識太多了，多到無法窮舉。」

他停頓一下：「但最近三年，事情有了根本的變化。大型語言模型讀了幾乎整個網際網路——它知道杯子要正著拿、知道牛奶在冰箱裡、知道『廚房』通常有水槽。那些我們寫不完的常識，它從文字中學到了。」

最後他補了一個關鍵的警告：「但這帶來一個新的、而且很危險的問題。語言模型會非常有自信地說錯話。在聊天時，說錯了頂多是笑話；在機器人上，說錯了就是一個真實的動作——可能打翻東西、可能撞到人。今天我們要學的，一半是怎麼用它，另一半是怎麼防它。」

本章為什麼重要

經過第 22 ~ 31 章，機器人已經具備了完整的**執行能力**：它能感知、移動、抓取、學習技能。但這些能力都停留在「**技能**」的層次——它會抓杯子，但不知道「什麼時候該抓杯子」。

任務層級的智慧是最後一塊拼圖。它要回答的問題是：

- **理解**：使用者說的話是什麼意思？「那個」指的是哪一個？
- **分解**：這個任務要拆成哪些步驟？順序如何？
- **常識**：哪些事情是「理所當然」但沒有明說的？
- **應變**：若某一步失敗了，該怎麼辦？

這些問題困擾了機器人學三十年。傳統的方法是**手工編寫規則與知識庫**，但如叫獸所說——**常識太多，寫不完**。

近三年，**大型語言模型 (LLM)** 與**視覺語言模型 (VLM)** 帶來了根本的改變。它們從海量的文字與影像中，隱含地學到了大量的常識與任務知識。這使機器人第一次能夠「理解」自然語言的指令，並自行分解任務。

但這也帶來了新的風險。**語言模型會產生看似合理但實際錯誤的內容 (幻覺)**——在聊天中這是笑話，在機器人上則是危險的動作。因此本章不只講「如何使用」，更要講**如何約束**：如何確保生成的計畫是**可執行、可驗證、且安全的**。

學習目標

- 能說明符號接地問題及其為何是機器人智慧的核心困難。
- 能說明傳統符號式任務規劃的原理與其三個限制。
- 能說明 PDDL 的基本結構並撰寫簡單的領域描述。
- 能說明大型語言模型的基本原理與其在機器人上的能力來源。
- 能區分語言模型作為規劃器的三種使用方式。
- 能說明程式碼生成作為機器人介面的優勢。
- 能說明可供性與價值函數如何約束語言模型的輸出。
- 能說明視覺語言模型如何連接感知與語言。

- 能說明視覺語言動作模型的架構與其意義。
- 能分析大型模型在機器人上的四類風險並設計防護。
- 能設計一個結合語言模型與傳統規劃的混合架構。
- 能評估任務規劃系統的正确性與安全性。

先備知識

第 26 章的路徑規劃（本章的任務規劃是其上一層）、第 31 章的學習方法與其限制、第 22 章的視覺感知、第 29 章的軟體架構、第 1 章的 Moravec 悖論與自主性光譜。

不需要深度學習或自然語言處理的專業背景。本章聚焦於「大型模型如何用於機器人」，而非模型本身的訓練細節。若對神經網路完全陌生，可先閱讀附錄 E。

核心名詞中英對照

表 32-1 本章核心名詞

中文	英文	說明
符號接地問題	Symbol Grounding Problem	如何把符號與真實世界連結
任務規劃	Task Planning	決定達成目標所需的動作序列
任務與運動規劃	Task and Motion Planning, TAMP	同時處理離散決策與連續運動
PDDL	Planning Domain Definition Language	描述規劃問題的標準語言
前提條件	Precondition	動作可執行所需滿足的條件
效果	Effect	動作執行後世界狀態的改變
大型語言模型	Large Language Model, LLM	以海量文字訓練的生成式模型
視覺語言模型	Vision-Language Model, VLM	同時處理影像與文字的模型
視覺語言動作模型	Vision-Language-Action, VLA	直接輸出機器人動作的多模態模型
提示工程	Prompt Engineering	設計輸入以引導模型輸出的技術
思維鏈	Chain-of-Thought	引導模型逐步推理的提示方式
可供性	Affordance	環境或物體所提供的行動可能性
幻覺	Hallucination	模型產生看似合理但錯誤的內容
接地	Grounding	把語言連結到真實的感知與動作
技能庫	Skill Library	機器人可執行的原子動作集合
閉環回饋	Closed-loop Feedback	以執行結果修正後續計畫

中文	英文	說明
語意地圖	Semantic Map	含物體類別與空間關係的地圖
零樣本	Zero-shot	未經該任務訓練即可執行

32.1 符號接地問題

32.1.1 三十年的老問題

機器人要理解「把杯子拿到廚房」，必須解決一個哲學層次的問題：「杯子」這兩個字，如何連結到眼前那個具體的物體？

這稱為**符號接地問題** (Symbol Grounding Problem)，由 Harnad 於 1990 年提出。它的核心是：

◎ 符號接地問題

符號本身沒有意義。「杯子」這兩個字，對電腦而言只是一串位元組。若我們用其他符號來定義它（「杯子是一種容器」），那「容器」又是什麼？這會陷入無窮的循環——**符號只能用符號解釋，永遠接觸不到真實世界**。要打破這個循環，符號必須**接地** (grounded) 到感知與行動：「杯子」意味著「看起來像這樣的東西」、以及「可以這樣抓起來的東西」。

這個問題對機器人特別關鍵，因為機器人**必須行動**。一個純語言的系統可以在符號之間打轉；但機器人必須把「杯子」變成一個具體的抓取姿態、把「廚房」變成一個具體的座標。

32.1.2 三個層次的接地

實務上，接地問題可以分解為三個層次：

表 32-2 符號接地的三個層次

層次	問題	所需技術	本書章節
物體接地	「杯子」對應畫面中的哪個區域？	物體偵測與分割	第 22.8 節
空間接地	「桌上」「廚房」對應什麼座標？	語意地圖、空間關係推理	第 25 章 + 本章
動作接地	「拿」對應什麼運動指令？	技能庫、抓取規劃	第 21、26 章

第三列最容易被忽略但最關鍵。「拿起杯子」這個符號，最終必須變成一連串的關節力矩——而中間的每一層轉換，都是前面三十一章的內容。

32.1.3 為何近年有了突破

符號接地問題困擾了三十年，為何近年突然有了進展？三個原因：

原因一：視覺模型能辨識開放詞彙。傳統的物體偵測只能辨識訓練過的固定類別（如 COCO 的 80 類）。近年的視覺語言模型（如 CLIP）能處理任意的文字描述——你可以問「畫面中哪個是藍色的馬克杯」，即使它從未見過這個特定的類別。這直接解決了物體接地。

原因二：語言模型內含大量常識。它從文字中學到了「杯子通常在桌上」、「廚房有水槽」、「牛奶要冷藏」。這些常識過去必須手工編寫，現在是免費附贈的。

原因三：語言成為通用的介面。過去每個模組都有自己的資料格式；現在語言可以作為模組之間共同語言——視覺模型輸出「桌上有一個紅色杯子」，語言模型讀懂它並輸出「移動到桌子、抓取紅色杯子」。

32.2 傳統的符號式任務規劃

32.2.1 STRIPS 與 PDDL

在大型模型出現之前，任務規劃有一套成熟的形式化方法。其核心是把世界描述為一組符號命題，把動作描述為前提條件與效果。

最經典的表示法是 STRIPS，其現代標準是 PDDL（Planning Domain Definition Language）。一個動作的定義包含三部分：

動作名稱（參數）

前提條件（Preconditions）：什麼情況下可以執行

效果（Effects）：執行後世界如何改變

這個結構極為簡潔，卻能表達複雜的任務。

```
;; PDDL 領域定義：一個簡化的取放任務
(define (domain robot-pickplace)
  (:requirements :strips :typing)

  (:types robot object location)

  (:predicates
    (at ?r - robot ?l - location)      ; 機器人在某位置
    (on ?o - object ?l - location)     ; 物體在某位置
    (holding ?r - robot ?o - object)  ; 機器人正握著物體
```

```

(hand-empty ?r - robot)          ; 手是空的
(reachable ?l - location))        ; 該位置可到達

;; 動作一：移動
(:action move
 :parameters (?r - robot ?from ?to - location)
 :precondition (and (at ?r ?from) (reachable ?to))
 :effect (and (at ?r ?to) (not (at ?r ?from))))

;; 動作二：拿起
(:action pick
 :parameters (?r - robot ?o - object ?l - location)
 :precondition (and (at ?r ?l) (on ?o ?l) (hand-empty ?r))
 :effect (and (holding ?r ?o)
              (not (on ?o ?l))
              (not (hand-empty ?r))))

;; 動作三：放下
(:action place
 :parameters (?r - robot ?o - object ?l - location)
 :precondition (and (at ?r ?l) (holding ?r ?o))
 :effect (and (on ?o ?l)
              (hand-empty ?r)
              (not (holding ?r ?o))))

;; 問題定義：把杯子從桌子拿到廚房
(define (problem cup-to-kitchen)
  (:domain robot-pickplace)
  (:objects
   r1 - robot
   cup - object
   table kitchen - location)
  (:init
   (at r1 table)
   (on cup table)
   (hand-empty r1)
   (reachable table)
   (reachable kitchen))
  (:goal (on cup kitchen)))

```

規劃器讀入這兩個檔案，會自動輸出動作序列：

pick(r1, cup, table) → move(r1, table, kitchen) → place(r1, cup, kitchen)

這正是叫獸開場時說的「六個步驟」中的核心三步——而它是自動推導出來的，不是人寫的。

32.2.2 規劃的搜尋

PDDL 規劃器的工作，本質上是在狀態空間中搜尋——與第 26 章的路徑規劃是同一類問題：

表 32-3 路徑規劃與任務規劃的對照

面向	路徑規劃 (第 26 章)	任務規劃 (本章)
狀態	機器人的位形 (x, y, θ)	世界的符號狀態 (哪些命題為真)
動作	連續的運動	離散的動作 (pick、place、move)
狀態空間	連續、低維	離散、極高維 (2^n 個命題組合)
演算法	A^* 、RRT	A^* 、啟發式搜尋 (如 FF、Fast Downward)
主要困難	高維連續空間	組合爆炸

兩者的演算法思想相通——都用啟發式搜尋。差別在於任務規劃的狀態空間是離散的，且會因為命題數量而組合爆炸 (n 個命題有 2^n 種狀態)。

32.2.3 傳統方法的三個限制

符號式規劃在理論上優雅、且有**正確性的保證** (規劃器找到的計畫必定滿足所有前提條件)。但它有三個致命的實務限制：

限制一：知識工程的負擔。每一個物體、每一個動作、每一個相關的性質，都必須手工編碼。一個真實的家庭環境有數百種物體與數千種可能的狀態——**這個工作永遠做不完**。

限制二：無法處理常識。「杯子要正著拿」、「熱的東西要小心」、「玻璃會碎」——這些常識若要編碼，數量是無窮的。這正是第 1 章 Moravec 悖論在任務層次的體現。

限制三：符號從哪來。PDDL 假設世界狀態已經是符號形式 (`(on cup table)`)。但機器人看到的是**像素**——由像素到符號的轉換，正是 32.1 節的接地問題，而傳統方法對此無解。

◎ 為何符號式規劃仍然重要

儘管有上述限制，符號式規劃**並未過時**——它在本章的架構中扮演關鍵角色。原因是它提供了大型模型所缺乏的東西：**正確性的保證**。一個 PDDL 規劃器產生的計畫，**保證每一步的前提條件都被滿足**；而語言模型產生的計畫可能包含「拿起一個不存在的杯子」這種錯誤。因此現代的架構常是**混合式**：用語言模型處理理解與常識 (它擅長的)，用符號式規劃驗證計畫的可行性 (它擅長的)。這與第 21.9.5 節的抓取、第 25.10.3 節的 SLAM、第 31.10.2 節的安全層，是同一種分工。

32.3 大型語言模型的能力來源

32.3.1 它到底在做什麼

大型語言模型的訓練目標極為簡單：**預測下一個詞**。給定「今天天氣很」，預測下一個詞可

能是「好」、「熱」、「糟」。

這個看似平凡的目標，在**極大規模**的訓練下產生了意想不到的能力。原因是：**要準確地預測下一個詞，模型必須理解語意、掌握常識、甚至進行推理。**

「杯子從桌上掉下來，結果它____」

要預測「破了」，模型必須知道：杯子易碎、地板是硬的、重力會使它加速

預測下一個詞的目標，隱含地要求了對世界的理解。

32.3.2 對機器人有價值的三種能力

大型語言模型有許多能力，但對機器人特別有價值的是三種：

能力一：常識知識。這是最直接的價值。模型知道：

- **物體的性質：**杯子可以裝液體、刀子很鋒利、玻璃會碎。
- **空間的常識：**廚房有水槽、書在書架上、鞋子在門口。
- **行為的規範：**熱的東西要小心、杯子要正著拿、開門要先確認沒人。

能力二：任務分解。給定一個高階目標，模型能拆解為步驟。這正是叫獸開場提到的「第三」——六個步驟不是人說的，是機器人要自己想出來的。

能力三：語意理解與指代消解。「把那個紅色的拿過來」、「放在旁邊」——這些模糊的指代，模型能依上下文推斷。

32.3.3 三個根本的限制

但語言模型也有三個對機器人特別危險的限制：

限制一：幻覺。模型會產生看似合理但完全錯誤的內容，且帶著高度的自信。它可能建議「打開櫃子取出杯子」——但那個櫃子根本不存在。

限制二：沒有物理直覺。模型從文字學習，而文字中很少描述物理細節。它知道「杯子會破」，但不知道「這個角度施力 5 牛頓會不會破」。第 13 到 21 章的所有物理知識，它幾乎都不具備。

限制三：不知道機器人能做什麼。模型可能規劃出「跳上桌子」或「用兩隻手同時操作」——而這台機器人可能根本沒有腿或只有一隻手臂。

◎ 一個關鍵的認識

語言模型知道「世界通常是怎樣的」，但不知道「現在這裡是怎樣的」，也不知道「這台機器人能做什麼」。這三者的落差，正是本章後續所有技術要處理的問題——32.5 節的可供性用來告訴它「現在能做什麼」、32.6 節的視覺語言模型用來告訴它「現在看到什麼」、而 32.9 節的安全層則負責攔截它的錯誤。把語言模型當成一個「見多識廣但沒到過現場的顧問」，是最貼切的理解。

32.4 語言模型作為任務規劃器

32.4.1 三種使用方式

把語言模型用於機器人任務規劃，有三種漸進的方式：

表 32-4 語言模型的三種使用方式

方式	輸出形式	優點	缺點
自然語言步驟	「先走到桌子，然後拿起杯子」	最直觀	難以自動執行（需再解析）
技能序列	goto(table); pick(cup); goto(kitchen)	可直接執行	受限於預定義的技能
程式碼生成	完整的 Python 程式（含迴圈與判斷）	最靈活、可表達複雜邏輯	更容易出錯、需嚴格驗證

32.4.2 方式一與二：從語言到技能

最務實的作法是**方式二**：定義一組**技能庫**（機器人能執行的原子動作），要求語言模型只用這些技能組合出計畫。

提示（Prompt）的典型結構

SYSTEM_PROMPT = """

你是一台服務機器人的任務規劃器。你只能使用以下技能：

```
goto(location)      - 移動到指定地點
pick(object)        - 拿起物體（手必須是空的）
place(location)     - 把手中的物體放下
open(container)     - 打開容器
close(container)    - 關閉容器
say(text)           - 說話
```

目前可見的物體：{visible_objects}

目前可到達的地點：{known_locations}

機器人狀態：{robot_state}

規則：

1. 只能使用上列技能，不可發明新技能。
2. 只能操作「目前可見的物體」清單中的物體。
3. `pick` 之前手必須是空的；`place` 之前手必須有東西。
4. 若指令無法達成，回覆 `CANNOT`：加上原因。

請輸出計畫，每行一個技能呼叫，不要有其他文字。

"""

USER = "幫我把桌上的杯子拿到廚房"

模型的輸出

```
PLAN = """
goto(table)
pick(cup)
goto(kitchen)
place(counter)
"""
```

這個提示設計包含四個關鍵元素：

- **技能清單**：明確限定模型能用什麼——防止它發明機器人做不到的動作（32.3.3 節的限制三）。
- **當前狀態**：告訴它現在看到什麼、能去哪裡——彌補「不知道現在這裡是怎樣」的落差。
- **規則**：明確的前提條件——這其實是把 PDDL 的知識用自然語言表達。
- **失敗的出口**：允許它說「做不到」——**這極為重要**，否則模型會硬生出一個錯誤的計畫。

32.4.3 方式三：程式碼作為介面

程式碼生成是近年最受矚目的方式（如 Google 的 Code as Policies）。它的核心洞見是：

◎ 為何程式碼是好的介面

程式語言恰好具備任務規劃所需要的一切結構：順序（一行接一行）、條件（if 判斷）、迴圈（重複處理多個物體）、函式（技能的呼叫與組合）、以及變數（記住中間結果）。而純粹的技能序列（方式二）只能表達「順序」——若任務是「把桌上**所有**的杯子都收起來」，技能序列必須事先知道有幾個杯子，程式碼卻可以用一個 for 迴圈處理。此外，語言模型在程式碼上的訓練資料極為豐富且結構嚴謹，因此生成程式碼的品質往往優於生成自然語言的計畫。

```

# 語言模型生成的程式碼 (Code as Policies 的風格)
# 指令：「把桌上所有的杯子都放到水槽裡，但如果杯子是滿的就先倒掉」

def execute_task():
    goto('table')
    cups = detect_objects('cup')          # 視覺模組回傳所有杯子

    if len(cups) == 0:
        say('桌上沒有看到杯子')
        return

    for cup in cups:
        pick(cup)
        # 用視覺語言模型判斷杯子是否是滿的
        if vlm_query(f'這個杯子裡有液體嗎?', get_image()):
            goto('sink')
            pour_out()                    # 倒掉
            goto('sink')
            place('sink')
            goto('table')                # 回來拿下一個

    say(f'已經收好 {len(cups)} 個杯子')

```

注意這段程式碼展現的能力：迴圈處理未知數量的物體、條件判斷、以及呼叫視覺語言模型做即時的判斷（`vlm\_query`）。這是純技能序列完全做不到的。

32.4.4 思維鏈：讓它先想再做

一個簡單卻極為有效的技巧是**思維鏈**（Chain-of-Thought）：要求模型在輸出計畫之前，先寫出推理過程。

直接輸出計畫 → 錯誤率較高

先推理再輸出計畫 → 錯誤率顯著降低

這與人類「先想清楚再動手」是同樣的道理。

```

# 加入思維鏈的提示
PROMPT_WITH_COT = """
在輸出計畫之前，請先分析：
1. 使用者的目標是什麼？
2. 目前的狀態如何？有哪些相關的物體？
3. 需要哪些步驟？每一步的前提條件是否滿足？
4. 有什麼可能出錯的地方？

分析完成後，再輸出計畫。

```

```

格式：
分析：<你的推理>
計畫：
<技能呼叫，每行一個>
"""

# 模型的輸出
"""
分析：
1. 目標是把杯子移到廚房。
2. 目前機器人在起始點，杯子在桌上，手是空的。
   可見物體：cup, book, plate。可到達：table, kitchen, door。
3. 步驟：先到桌邊（因為 pick 需要在物體所在位置）、
   拿起杯子（手是空的，前提滿足）、移動到廚房、放下。
4. 可能的問題：杯子若是滿的，移動時可能灑出。
   但指令未提及，先假設是空的。

計畫：
goto(table)
pick(cup)
goto(kitchen)
place(counter)
"""

```

思維鏈還有一個額外的好處：**可解釋性**。當計畫出錯時，可以從推理過程中看出它是在哪一步想錯了——這對除錯極為有價值，也部分彌補了 31.10.1 節提到的「學習方法難以除錯」的問題。

32.5 可供性：把語言接地到能力

32.5.1 問題：模型不知道自己能做什麼

32.3.3 節指出，語言模型的第三個限制是「不知道機器人能做什麼」。這在實務上會產生兩類錯誤：

- **能力錯誤**：規劃出機器人做不到的動作（如「爬上桌子」，但這是一台輪型機器人）。
- **情境錯誤**：規劃出當下不可行的動作（如「拿起杯子」，但杯子在關著的櫃子裡）。

解法的核心概念是「**可供性**」（affordance）——這個詞源自心理學家 Gibson，意指「環境提供給行動者的行動可能性」。

語言模型說：「這件事「應該」做」（依常識）

可供性說：「這件事「現在能不能」做」（依實況）

兩者結合 → 可執行的計畫

這是 SayCan 這類方法的核心思想。

32.5.2 SayCan：說的 × 能的

Google 的 SayCan 提出了一個極為優雅的框架。對每一個候選技能，計算兩個分數並相乘：

$$\text{分數(技能)} = P_{\text{LLM}}(\text{技能} \mid \text{指令}) \times P_{\text{affordance}}(\text{成功} \mid \text{當前狀態})$$

「Say」是語言模型的意見、「Can」是實際的可行性。

兩個分數的來源不同：

- $P_{\text{LLM}}(\text{Say})$ ：語言模型評估「這個技能對達成目標有多相關」。它有常識但不知道現場狀況。
- $P_{\text{affordance}}(\text{Can})$ ：由價值函數提供——這正是第 31 章強化學習訓練出的 Q 函數！它評估「在當前的視覺觀測下，這個技能有多大機率成功」。

這個設計的巧妙之處在於兩者的互補：

表 32-5 Say 與 Can 的互補

情境	Say (語言模型)	Can (可供性)	相乘後	結果
合理且可行	高	高	高	執行 ✓
合理但不可行	高	低	低	跳過 (如杯子在關著的櫃中)
可行但不相關	低	高	低	跳過 (如「打開電視」與任務無關)
既不合理也不可行	低	低	低	跳過

第二列是關鍵：語言模型說「拿杯子」很合理，但視覺告訴我們杯子在關著的櫃子裡，抓取的成功率極低——相乘後這個選項被排除，模型會轉而選擇「打開櫃子」。系統因此自動學會了正確的順序，而不需要有人寫下這條規則。

32.5.3 可供性從哪裡來

可供性分數有三種來源，各有優缺：

表 32-6 可供性的三種來源

來源	作法	優點	缺點
學習的價值函數	用第 31 章的 RL 訓練 Q 函數	準確、反映真實能力	需要大量訓練
幾何檢查	用第 26 章的規劃器檢查可達性	有保證、快速	只涵蓋幾何，不含成功率
視覺語言模型	問 VLM 「這個動作現在可行嗎」	零樣本、易實作	可能不準

實務上常三者並用：先用幾何檢查快速排除不可能的（如超出工作空間）、再用價值函數評估成功率、必要時用 VLM 補充判斷。

32.5.4 閉環：讓執行結果回饋到規劃

閉環的規劃有一個根本問題：**若某一步失敗了，後續的計畫全部失效**。例如規劃了「拿杯子 → 去廚房 → 放下」，但抓取失敗了——機器人仍會空手走到廚房並做出放下的動作。

閉環的架構在每一步執行後檢查結果，並據此更新計畫：

```
def execute_with_feedback(instruction, max_steps=20):
    """閉環的任務執行 ( Inner Monologue 的架構 ) """
    history = []

    for step in range(max_steps):
        # 1. 取得當前的觀測
        image = camera.capture()
        scene = vlm_describe(image)          # 視覺語言模型描述場景
        robot_state = get_robot_state()

        # 2. 由語言模型決定下一步 ( 考慮歷史與當前狀態 )
        prompt = build_prompt(instruction, scene, robot_state, history)
        candidates = llm_propose_skills(prompt)  # 產生候選技能

        # 3. 用可供性過濾 ( SayCan 的核心 )
        scored = []
        for skill in candidates:
            p_say = candidates[skill]          # LLM 的分數
            p_can = affordance_model(skill, image)  # 可供性分數
            scored.append((skill, p_say * p_can))
        best_skill, score = max(scored, key=lambda x: x[1])

        # 4. 安全檢查 ( 32.9 節 )
        if not safety_check(best_skill, robot_state):
            history.append(f'{best_skill}: 未通過安全檢查')
            continue
        if score < THRESHOLD:
```

```

say('我不確定該怎麼做，請給我更多指示')
return False

# 5. 執行並取得結果
success = execute_skill(best_skill)

# 6. 把結果加入歷史——這是「閉環」的關鍵
result = '成功' if success else '失敗'
history.append(f'{best_skill}: {result}')

if not success:
    # 讓語言模型看到失敗，它會自行調整策略
    # 例如：抓取失敗 → 它可能會提議「換一個角度再試」
    continue

# 7. 檢查是否完成
if llm_check_done(instruction, scene, history):
    return True

return False

```

這個架構稱為「內心獨白」(Inner Monologue) ——機器人在每一步之後「告訴自己」發生了什麼，並依此調整。它的價值在於：

- 能從失敗中恢復：抓取失敗時，模型看到「pick(cup): 失敗」，可能會提議換個角度或先移開遮擋物。
- 能處理環境變化：若有人把杯子移走了，下一步的場景描述會反映這件事。
- 能處理意外：發現杯子是滿的、發現門是鎖的——這些都能即時反應。

32.6 視覺語言模型

32.6.1 連接看見與說出

前面的架構有一個環節尚未說明：機器人如何把「看到的」變成「語言模型能讀的」？這就是視覺語言模型 (Vision-Language Model, VLM) 的工作。

VLM 能同時處理影像與文字，執行三類任務：

表 32-7 VLM 在機器人上的三類用途

用途	輸入	輸出	機器人上的應用
場景描述	影像	文字描述	告訴語言模型「現在看到什麼」
視覺問答	影像 + 問題	答案	「杯子是滿的嗎？」「門開著嗎？」
開放詞彙定位	影像 + 物體描述	位置（框或遮罩）	「藍色馬克杯」在畫面的哪裡

第三列直接解決了 32.1.2 節的物體接地問題。傳統的物體偵測只認得訓練過的固定類別；VLM 則能處理任意的文字描述——「左邊那個有把手的杯子」、「最大的那個盒子」。

32.6.2 開放詞彙的意義

這個能力的價值極大。比較兩種作法：

傳統：訓練一個 80 類的偵測器 → 只認得那 80 類
VLM：用文字查詢 → 可以認得任何能用語言描述的東西

這是質的差異，而非量的差異。

具體的影響：一台家用機器人不可能事先訓練所有可能遇到的物體（每個家庭的東西都不同）。有了開放詞彙的能力，使用者可以直接說「拿我的藍色水壺」——即使系統從未見過那個特定的水壺。

32.6.3 VLM 的限制

但 VLM 在機器人上有三個重要的限制：

限制一：空間關係的精度不足。VLM 能說「杯子在盤子旁邊」，但難以精確判斷「杯子在盤子右邊 5 公分處」。對抓取而言，這個精度遠遠不夠（第 21 章的容差只有數公釐）。因此實務上仍需傳統的幾何方法計算精確位姿。

限制二：對物理性質不敏感。它能認出「這是玻璃杯」，但無法判斷「這個杯子有多重」、「這個表面有多滑」。第 30 章的 Sim-to-Real 落差在此重現。

限制三：推論成本高。大型 VLM 的單次推論可能需要數百毫秒到數秒——這遠慢於控制迴路。因此它只能用於低頻的決策層（第 3 章的時間尺度金字塔的最上層），而非即時控制。

◎ VLM 與傳統視覺的分工

這再次體現了本書反覆出現的分工原則。**VLM 負責「這是什麼、該怎麼理解」**——開放詞彙的辨識、語意的判斷、常識的應用。**傳統視覺負責「它精確地在哪裡」**——第 22 章的相機模型、位姿

估計、深度計算。一個典型的流程是：VLM 回答「藍色杯子在畫面的這個區域」，再用傳統的方法（點雲配準或 PnP）算出它精確到公釐的六自由度位姿。**語意用學習、幾何用模型**——與第 25.10.3 節的 SLAM 前後端完全相同的思維。

32.7 視覺語言動作模型

32.7.1 更激進的一步

前面各節的架構是**模組化**的：VLM 看、LLM 想、技能庫做。近年出現了一個更激進的方向——把三者合而為一。

視覺語言動作模型（Vision-Language-Action, VLA）直接由「影像 + 語言指令」輸出「機器人動作」：

傳統模組化：影像 → VLM → 文字 → LLM → 技能 → 控制器 → 動作

VLA：（影像，指令）→ 單一模型 → 動作

中間的所有轉換都被吸收進一個模型中。

32.7.2 為何可能有效

這個作法初看違反直覺——它放棄了模組化的所有好處。但它有三個潛在的優勢：

優勢一：避免資訊損失。模組化的每一次轉換都會丟失資訊。「杯子在畫面右側」這句話，遠不如原始影像所含的資訊豐富。端到端的模型能保留全部。

優勢二：從網路知識中遷移。VLA 通常以預訓練的 VLM 為基礎（已經理解了視覺與語言），再用機器人資料微調。這使它能**把網路上學到的常識，直接帶到動作層**。例如它可能從未見過「拿起恐龍玩具」的機器人示範，但因為它認得恐龍玩具，就能泛化過去。

優勢三：不需要設計介面。模組化架構需要人為定義技能庫、狀態表示、以及模組間的協定。VLA 完全跳過這些設計。

32.7.3 RT-2 與其後續

Google 的 RT-2（Robotic Transformer 2）是這個方向的代表。它的核心技巧極為巧妙：

◎ 把動作當成文字

RT-2 的關鍵洞見是：**把機器人的動作編碼成「詞」**。例如把每個關節的位移離散化為 256 個區間，每個區間對應一個特殊的詞元（token）。這樣一來，「輸出動作」就變成了「輸出一串文

字」——而這正是語言模型最擅長的事。於是整個預訓練好的視覺語言模型，可以幾乎不做架構修改就用於機器人控制。這個技巧使得「網路上的知識」與「機器人的動作」能在同一個模型中共存——RT-2 展示了它能理解「把水果放到能裝水的容器裡」，即使訓練資料中沒有這個組合。

後續的發展包括 **Open X-Embodiment** (跨機構、跨機器人的大規模資料集) 與各種開源的 VLA 模型。這個方向目前發展極快。

32.7.4 目前的限制

但 VLA 距離實用仍有距離，四個主要的限制：

表 32-8 VLA 目前的四個限制

限制	說明	影響
推論速度	大型模型的單次推論需數十至數百毫秒	只能做低頻控制 (3~10 Hz)
資料需求	需要大量的真實機器人示範資料	收集成本極高
沒有保證	無法驗證輸出動作的安全性	必須有外部的安全層
精度不足	離散化的動作解析度有限	難以做精密操作 (第 20 章的裝配)

因應之道是分層：VLA 負責高層的、粗略的決策 (3~10 Hz)，底層仍用傳統的控制器 (1 kHz) 執行精確的運動。這正是第 27、28 章分層架構思想的延續。

32.8 任務與運動規劃的整合

32.8.1 符號與幾何的落差

32.2 節的符號式規劃有一個根本的問題：它假設每個符號動作都能執行。

但實際上不一定。考慮這個計畫：

pick(cup) → place(cup, shelf)

符號上完全合理——前提條件都滿足。

但實際執行時可能發現：手臂構不到那個位置 (超出工作空間，第 12 章)、路徑被擋住 (第 26 章的規劃無解)、或抓取姿態會使手臂與桌子碰撞。

符號層不知道這些幾何的細節，而幾何層不知道任務的目標——這個落差稱為 **TAMP 問題** (Task and Motion Planning)。

32.8.2 為何困難

TAMP 的困難在於兩層必須互相依賴：

要選符號動作 → 必須知道它幾何上可行

要檢查幾何可行 → 必須先選定符號動作

這是一個雞生蛋的循環——與第 25 章 SLAM 的結構相同。

更麻煩的是連續參數的選擇。「把杯子放到架子上」這個符號動作，實際上包含無限多種可能——放在架子的哪個位置？手臂用什麼姿態？而這個選擇會影響後續動作的可行性（若放得太靠內側，下一次可能拿不到）。

32.8.3 常見的解法

TAMP 有三類主要的解法：

表 32-9 TAMP 的三類解法

方法	作法	優點	缺點
先符號後幾何	先規劃符號序列，逐一檢查幾何可行性，失敗則回溯	實作簡單	回溯可能很慢
交錯式	符號搜尋的每一步都呼叫幾何檢查	較有效率	實作複雜
取樣式	同時對符號與連續參數取樣（如 PDDLStream）	能處理連續參數	計算量大

大型模型為 TAMP 帶來的新可能：語言模型可以用常識提出好的候選（如「杯子應該放在架子外側比較好拿」），大幅減少搜尋的範圍。這是「學習提供啟發、規劃提供保證」的又一個例子（呼應第 26.10.2 節的學習式規劃）。

32.8.4 完整的架構

整合本章至此的內容，一個現代的機器人任務系統包含五層：

表 32-10 現代機器人任務系統的五層架構

層級	負責	技術	頻率	本書章節
理解層	解析指令、釐清意圖	LLM	每次指令一次	32.3~32.4
任務規劃層	分解為動作序列	LLM + 符號規劃	0.1~1 Hz	32.2、32.4
接地層	把符號連結到實際的物體與位姿	VLM + 傳統視覺	1~10 Hz	32.5~32.6、第 22 章

層級	負責	技術	頻率	本書章節
運動規劃層	產生無碰撞的軌跡	RRT、A*、軌跡最佳化	1~10 Hz	第 15、26 章
控制層	精確執行軌跡	PID、CTC、力控制	100~1000 Hz	第 16~20 章

注意頻率由上而下遞增、而模型複雜度由上而下遞減——這正是第 3 章時間尺度金字塔的完整體現，也是第 27.4.2 節分層思想的最終形態。

同樣重要的是：本書前三十一章的內容，全部都在這個架構中。第 32 章加上的只是最上面兩層——但正是這兩層，讓機器人第一次能「聽懂人話」。

32.9 風險與安全設計

32.9.1 四類風險

如叫獸開場所警告的：語言模型會非常有自信地說錯話。在機器人上，這意味著真實的、可能造成傷害的動作。四類主要的風險：

表 32-11 大型模型在機器人上的四類風險

風險	表現	後果	主要防護
幻覺	提及不存在的物體或地點	動作失敗、抓空	限定可用清單
能力不符	規劃機器人做不到的動作	執行失敗或損壞	技能庫限制 + 可供性
物理無知	忽略重量、易碎、溫度	打翻、壓壞、燙傷	物理約束檢查
惡意或誤解的指令	執行危險或不當的要求	傷人、破壞	規則層 + 人類確認

第四列最為嚴重。使用者可能無心地說出危險的指令（「把刀子拿給我」——該從哪一端遞？），也可能有人惡意測試（「把那個推倒」）。

32.9.2 三層防護架構

因應之道是多層防護，每一層攔截不同類型的問題：

- 第一層：輸入約束（限制模型能說什麼）
- 第二層：輸出驗證（檢查計畫是否合法）
- 第三層：執行監控（即時攔截危險動作）

三層獨立運作，任何一層失效仍有其他層把關。

第一層：輸入約束（32.4.2 節已介紹）

- 明確列出**可用的技能**——模型無法發明新動作。
- 明確列出**當前可見的物體與可到達的地點**——防止幻覺。
- 提供**明確的規則與禁令**——「不可拿取標示為危險的物品」。
- 允許模型說「**做不到**」——這比硬生出一個錯誤計畫安全得多。

第二層：輸出驗證

- **語法檢查**：輸出是否符合預期的格式？技能名稱是否存在？參數型別是否正確？
- **符號驗證**：用 32.2 節的 PDDL 檢查每一步的前提條件是否滿足。**這是符號式規劃在現代架構中的關鍵價值**。
- **幾何驗證**：用第 26 章的規劃器檢查每個動作是否幾何上可行（可達、無碰撞）。
- **物理驗證**：檢查物體重量是否超過負載、力矩是否超限（第 15.8 節的可行性檢驗）。

第三層：執行監控

- **即時的力與速度限制**（第 20 章的阻抗控制、第 7 章的安全標準）。
- **碰撞偵測與反應**（第 11.6.3 節）。
- **人員偵測與減速**（第 26.10.3 節的社會導航）。
- **緊急停止**——永遠保留人類介入的能力。

◎ 一個不可妥協的原則

無論上層的模型多聰明，最底層的安全機制必須是「模型式的、可驗證的、且獨立於學習系統的」。一個力矩限制不應該由神經網路判斷，而應該是一行明確的 `if torque > MAX: clamp()`。理由很簡單：你無法證明神經網路在所有情況下都會做出安全的判斷，但你可以證明那一行 if 敘述必定生效。這呼應了第 7.9 節的失效導向安全、第 31.10.2 節的分層架構——智慧可以是機率性的，但安全必須是確定性的。

32.9.3 人在迴路

最後一道防線是**人類的參與**。三種常見的設計：

表 32-12 人在迴路的三種模式

模式	作法	適用情境
執行前確認	顯示計畫，等待使用者核可	高風險或不可逆的動作
主動詢問	模型不確定時主動提問	指令模糊（「那個」是哪個？）
隨時可中止	使用者能隨時喊停	所有情境（必備）

「主動詢問」特別值得強調。傳統的系統遇到模糊指令時，要嘛猜一個、要嘛報錯。而語言模型能自然地提問：「桌上有兩個杯子，一個藍色一個白色，您指的是哪一個？」

這個能力大幅提升了實用性——它把「失敗」轉化為「對話」，與第 1 章討論的自主性光譜一致：不是全自動或全手動，而是在需要時尋求協助。

32.10 實務上的系統設計

32.10.1 該用多大的模型

一個常見的實務問題是：該用什麼規模的模型、跑在哪裡？三種配置：

表 32-13 三種部署配置的比較

配置	延遲	成本	隱私	可靠度	適用
雲端大模型	0.5~3 秒	按用量計費	資料外流	受網路影響	高階推理、開發階段
本地中型模型	0.1~1 秒	一次性硬體	佳	穩定	多數實際部署
本地小模型	< 0.1 秒	低	佳	穩定	簡單的指令解析

實務上的常見架構是混合：用本地的小模型處理常見的指令（快速、離線可用），遇到複雜的情況才呼叫雲端的大模型。這與第 3 章的邊緣—雲端分工是同一個思維。

32.10.2 語意地圖

要回答「廚房在哪裡」，機器人需要一張語意地圖——在第 25 章的幾何地圖之上，加入物體與空間的語意標註。

建立語意地圖有三種方式：

- 人工標註：最準確但費工。適合固定的環境（如工廠、特定的家庭）。
- 自動建立：機器人探索時用 VLM 辨識物體並記錄位置。這是近年的主流——邊建圖邊標

註。

- **由使用者互動建立：**「這裡是廚房」——使用者直接告訴機器人。簡單且準確。

語意地圖是符號接地的空間版本（32.1.2 節的第二層）。有了它，「廚房」這個符號才能連結到一組具體的座標。

32.10.3 技能庫的設計

32.4.2 節提到要限定模型只能用預定義的技能。**技能庫的設計品質，直接決定系統的能力上限**。三個設計原則：

原則一：粒度要適中。太細（如「移動關節 1 到 30 度」）會使語言模型難以規劃；太粗（如「整理房間」）則失去了組合的彈性。**一個好的粒度是「人類會用一句話描述的動作」**——拿起、放下、打開、移動到。

原則二：介面要清晰。每個技能應明確定義：參數、前提條件、預期效果、以及失敗的判準。**這其實就是 32.2 節 PDDL 的結構**——只是現在同時給人、給規劃器、也給語言模型看。

原則三：要能回報結果。技能執行後必須明確回傳成功或失敗（以及失敗的原因），這是 32.5.4 節閉環架構的前提。

32.11 由任務邁向具身智慧

32.11.1 本章的成果

本章讓機器人第一次能「聽懂人話」。你現在能夠：說明符號接地問題與其三個層次、運用 PDDL 進行符號式任務規劃、理解大型語言模型的能力來源與三個限制、以三種方式運用語言模型做規劃、用可供性把語言接地到實際能力、運用視覺語言模型連接感知與語言、理解 VLA 的架構與限制、整合任務與運動規劃、並設計三層的安全防護。

最重要的是那個貫穿本章的分工：**語言模型提供常識與理解，符號規劃提供正確性的保證，而底層的控制提供安全**。這與第 31 章的「模型提供保證、學習提供泛化」、第 25 章的「學習做前端、幾何做後端」，是同一種工程智慧在不同層次的體現。

我們也看到了**全書技術的完整整合**——32.8.4 節的五層架構中，前三十一章的內容全部就位，而本章加上的是最上面兩層。

32.11.2 接下來：智慧需要身體嗎

但本章也留下了一個更深的問題。

大型語言模型知道「杯子要正著拿」、「玻璃會碎」——這些知識來自閱讀文字，而非親身經驗。它從未真正拿過一個杯子、從未感受過玻璃碎裂的觸感。

這樣的「知道」，與人類的「知道」是同一回事嗎？

一個五歲的孩子能摺衣服，不是因為他讀過《布料力學》，而是因為他用手摸過、拉過、失敗過無數次。他的知識存在於身體的經驗中，而非文字的描述裡。

第 33 章 (具身人工智慧) ——第五篇也是全書理論部分的收束——將處理這個根本的問題：

- 智慧是否必須有身體？這是「具身認知」(embodied cognition) 的核心主張。
- 為何 Moravec 悖論成立？第 1 章提出的問題，將在此得到完整的回答。
- 身體如何參與智慧？第 21 章的柔順夾爪、第 28 章的被動行走——這些「用機構取代計算」的例子，是否正是具身智慧的體現？
- 通往通用機器人的路徑為何？是更大的模型、更多的資料，還是更好的身體？

換句話說：第 32 章讓機器人知道該做什麼，第 33 章則要問——它真的「理解」自己在做什麼嗎？

完整計算例題

例題 32-1 符號規劃的狀態空間

題目：某環境有 5 個物體、4 個地點、1 台機器人。狀態由三類命題描述：``at(robot, loc)``、``on(obj, loc)``、``holding(obj)``。(a) 計算各類命題的數量。(b) 計算理論上的狀態空間大小。(c) 考慮物理約束後，計算實際可能的狀態數。(d) 討論這對搜尋的影響。

解 (a)：逐類計算：

`at(robot, loc) : 1 × 4 = 4 個命題`

`on(obj, loc) : 5 × 4 = 20 個命題`

`holding(obj) : 5 個命題`

總計：29 個命題

每個命題可為真或假。

解 (b)：理論上每個命題獨立取真假：

$$2^{29} = 536,870,912 \approx 5.4 \text{ 億種狀態}$$

這就是 32.2.2 節提到的組合爆炸。

解 (c)：但多數組合在物理上不可能。加入約束：

- 機器人只能在一個地點：4 種選擇（而非 $2^4 = 16$ ）。
- 每個物體要嘛在某個地點、要嘛被握著：每個物體有 5 種狀態（4 個地點 + 被握著）。
- 機器人最多握一個物體：這進一步限制了組合。

$$\text{先不考慮「最多握一個」：} 4 \times 5^5 = 4 \times 3125 = 12,500 \text{ 種}$$

已經由 5.4 億降到 1.25 萬。

再扣除「同時握兩個以上」的情況。設 k 個物體被握著，只有 $k \leq 1$ 合法：

$$k = 0 : 4 \times 4^5 = 4 \times 1024 = 4,096$$

$$k = 1 : 4 \times 5 \times 4^4 = 4 \times 5 \times 256 = 5,120$$

$$\text{合計：} 9,216 \text{ 種合法狀態}$$

由 5.4 億降到 9,216——減少了約 58,000 倍。

解 (d)：這個計算揭示了三件事：

- 約束極為有價值。加入物理約束後，搜尋空間縮小了近六萬倍。這正是 PDDL 中前提條件的作用——它不只保證正確性，還大幅加速搜尋。
- 但仍會爆炸。若物體增加到 20 個、地點增加到 10 個，合法狀態數變為約 10^{22} ——再多的約束也救不了。
- 這正是需要啟發式搜尋的原因。與第 26.3.3 節的 A^* 一樣，好的啟發函數能讓搜尋有方向感，避免盲目地展開整個空間。

例題 32-2 SayCan 的分數計算

題目：指令為「我把飲料打翻了，可以幫我嗎？」。語言模型與可供性模型對五個候選技能的評分如下表。(a) 計算各技能的綜合分數。(b) 判斷應執行哪一個。(c) 若機器人手上已經拿著海綿，可供性分數會如何改變？重新計算。(d) 討論兩個分數各自的作用。

表 32-14 例題 32-2 的評分（情境一：手是空的）

技能	P_LLM (Say)	P_affordance (Can)	綜合分數
find(sponge) 找海綿	0.75	0.90	(待計算)
pick(sponge) 拿海綿	0.80	0.15	(待計算)
go_to(table) 去桌邊	0.40	0.95	(待計算)
clean(spill) 擦拭	0.95	0.05	(待計算)
say('抱歉') 道歉	0.30	1.00	(待計算)

解 (a) : 綜合分數為兩者相乘 :

find(sponge) : $0.75 \times 0.90 = 0.675$ ← 最高
pick(sponge) : $0.80 \times 0.15 = 0.120$
go_to(table) : $0.40 \times 0.95 = 0.380$
clean(spill) : $0.95 \times 0.05 = 0.048$
say('抱歉') : $0.30 \times 1.00 = 0.300$

各技能的綜合分數。

解 (b) : 應執行 find(sponge) (分數 0.675) 。

這個結果值得細看。注意`clean(spill)`的語言模型分數最高 (0.95) ——它最直接地達成目標。但它的可供性極低 (0.05) , 因為機器人手上沒有海綿, 根本無法擦。相乘後被排除。

系統因此自動推導出了正確的順序: 要擦拭, 必須先有海綿; 要有海綿, 必須先找到它。這條「常識性的順序」沒有任何人寫下來——它是兩個分數相乘的自然結果。

解 (c) : 若機器人已經拿著海綿, 可供性大幅改變:

表 32-15 例題 32-2 的評分 (情境二: 手上有海綿)

技能	P_LLM	P_affordance	綜合分數	變化
find(sponge)	0.30	0.90	0.270	LLM 分數降低 (已經有了)
pick(sponge)	0.10	0.05	0.005	手已滿, 可供性極低
go_to(table)	0.70	0.95	0.665	上升 (該去現場了)
clean(spill)	0.95	0.30	0.285	可供性上升但仍需在現場
say('抱歉')	0.20	1.00	0.200	—

現在最高的是 go_to(table) (0.665) ——系統自然地進入了下一步。到了桌邊之後, `clean(spill)`的可供性會再上升到 0.9 以上, 成為最高分。

解 (d)：兩個分數的作用可以總結為：

Say (語言模型)：提供「目標導向」——什麼事情有助於達成任務

Can (可供性)：提供「現實約束」——什麼事情現在做得到

缺一不可。

只有 Say 會怎樣？機器人會直接嘗試 `clean(spill)` (分數最高) ——空手擦拭，失敗。

只有 Can 會怎樣？機器人會做 `say('抱歉')` (可供性 1.00，永遠可行) ——說了句抱歉就結束了，完全沒有解決問題。

這正是 32.5.2 節「說的 × 能的」的價值——一個提供方向、一個提供可行性，相乘才能得到既正確又可執行的行為。

例題 32-3 分層架構的時間預算

題目：某服務機器人採用 32.8.4 節的五層架構。各層的處理時間為：LLM 規劃 1200 ms、VLM 場景理解 350 ms、傳統視覺定位 40 ms、運動規劃 80 ms、控制迴路 1 ms。(a) 計算完成一個「拿起杯子」動作的總延遲。(b) 判斷各層應以什麼頻率運行。(c) 若使用者說話後要在 3 秒內開始動作，判斷是否可行。(d) 提出三個降低延遲的方法。

解 (a)：一次完整的流程需依序經過各層：

$$1200 \text{ (規劃)} + 350 \text{ (場景理解)} + 40 \text{ (定位)} + 80 \text{ (運動規劃)} = 1,670 \text{ ms}$$

約 1.7 秒才開始移動。控制迴路的 1 ms 不計入啟動延遲 (它是持續運行的)。

解 (b)：各層應依其時間常數決定頻率 (第 3 章的時間尺度金字塔)：

表 32-16 各層的合理運行頻率

層級	處理時間	合理頻率	理由
LLM 規劃	1200 ms	每次指令一次	只在需要重新規劃時執行
VLM 場景理解	350 ms	1~2 Hz	場景變化不快
傳統視覺定位	40 ms	10~25 Hz	抓取時需要較即時的位姿
運動規劃	80 ms	5~10 Hz	重新規劃以避開動態障礙
控制迴路	1 ms	1000 Hz	第 16~20 章的要求

注意頻率跨越了三個數量級 (1 次 vs 1000 Hz) ——這正是分層的必要性。若把 LLM 放進

控制迴路，系統完全無法運作。

解 (c)：總延遲 1.67 秒 < 3 秒。可行，且有 1.33 秒的餘裕。

但要注意：這是**理想情況**。實際上還要加上語音辨識（約 300~500 ms）、網路延遲（若用雲端 LLM，可能再加 200~1000 ms）。加總後可能接近或超過 3 秒。

解 (d)：三個降低延遲的方法：

- **平行化**：VLM 場景理解與 LLM 規劃可以同時進行——先用上一時刻的場景描述開始規劃，同時更新場景。這能省下 350 ms。
- **預先規劃與快取**：對常見的指令（「拿杯子」「開燈」），事先產生計畫並快取。使用者說出時直接取用，省下整整 1200 ms。
- **分層回應**：先執行不需要精確資訊的動作（如「往桌子的方向移動」），同時在背景完成規劃。這使機器人「看起來」立刻有反應——這對使用者體驗極為重要。

第三點特別值得強調。人類的互動有一個心理學上的門檻：**超過約 1 秒沒有任何反應，就會覺得「它是不是當機了」**。因此即使還在思考，也應該先有回應（如轉頭看向使用者、或說「好的，我看看」）——這與技術效能無關，卻直接決定了系統是否好用。

程式範例：SayCan 風格的任務規劃器

以下程式實作一個簡化但完整的任務規劃系統，包含：技能庫、可供性評估、SayCan 的分數結合、符號驗證、以及閉環執行。它**不需要真正的語言模型即可運行**——用一個規則式的模擬器代替，使核心邏輯清晰可見。

```
import numpy as np

# ===== 技能庫 (32.10.3 節) =====
class Skill:
    """每個技能明確定義前提條件與效果 (32.2.1 節的 PDDL 結構)"""
    def __init__(s, name, params, precondition, effect, desc):
        s.name, s.params = name, params
        s.precond = precondition    # 檢查函式：state -> bool
        s.effect = effect          # 轉移函式：state -> new_state
        s.desc = desc              # 給語言模型看的說明

    def __repr__(s):
        return f"{s.name}({', '.join(s.params)})"
```

```

class WorldState:
    """世界的符號狀態"""
    def __init__(s, robot_at='start', holding=None, objects=None,
                  containers=None):
        s.robot_at = robot_at
        s.holding = holding
        s.objects = objects or {}          # {物體: 位置}
        s.containers = containers or {}    # {容器: 是否打開}

    def copy(s):
        return WorldState(s.robot_at, s.holding,
                           dict(s.objects), dict(s.containers))

    def __repr__(s):
        return (f"機器人在 {s.robot_at}, 手持 {s.holding or '空'}, "
                f"物體 {s.objects}")

def build_skill_library():
    """定義機器人能做的所有動作"""
    return {
        'goto': Skill(
            'goto', ['location'],
            precondition=lambda st, loc: loc != st.robot_at,
            effect=lambda st, loc: _apply(st, robot_at=loc),
            desc='移動到指定地點'),
        'pick': Skill(
            'pick', ['object'],
            # 前提: 手是空的、物體在機器人所在位置、若在容器中則容器須打開
            precondition=lambda st, o: (st.holding is None
                                         and st.objects.get(o) == st.robot_at),
            effect=lambda st, o: _apply(st, holding=o, remove_obj=o),
            desc='拿起物體 (手必須是空的、且物體須在同一位置)'),
        'place': Skill(
            'place', ['location'],
            precondition=lambda st, loc: (st.holding is not None
                                           and st.robot_at == loc),
            effect=lambda st, loc: _apply(st, place_here=loc),
            desc='把手中的物體放在當前位置'),
        'open': Skill(
            'open', ['container'],
            precondition=lambda st, c: (c in st.containers
                                         and not st.containers[c]
                                         and st.holding is None),
            effect=lambda st, c: _apply(st, open_container=c),
            desc='打開容器 (手必須是空的)'),
    }

def _apply(st, robot_at=None, holding=None, remove_obj=None,
           place_here=None, open_container=None):
    n = st.copy()
    if robot_at is not None:      n.robot_at = robot_at
    if holding is not None:       n.holding = holding
    if remove_obj is not None:    n.objects.pop(remove_obj, None)

```

```

if place_here is not None:
    n.objects[n.holding] = place_here; n.holding = None
if open_container is not None: n.containers[open_container] = True
return n

# ===== 語言模型的模擬 ( Say ) =====
def llm_score(skill_call, instruction, state):
    """
    模擬語言模型對「這個技能對達成目標有多相關」的評分。
    真實系統中這裡會呼叫 LLM 並取得對數機率。
    """
    name, arg = skill_call
    scores = {}
    if '杯子' in instruction and '廚房' in instruction:
        # 目標：把杯子拿到廚房
        scores = {
            ('goto', 'table'): 0.7 if state.holding != 'cup' else 0.2,
            ('goto', 'kitchen'): 0.9 if state.holding == 'cup' else 0.3,
            ('pick', 'cup'): 0.95 if state.holding is None else 0.05,
            ('place', 'kitchen'): 0.9 if state.holding == 'cup' else 0.05,
            ('place', 'table'): 0.1,
            ('open', 'cabinet'): 0.4,
        }
    return scores.get((name, arg), 0.1)

# ===== 可供性模型 ( Can ) =====
def affordance_score(skill_call, skills, state):
    """
    評估「這個技能現在能不能成功」。
    真實系統中這來自第 31 章訓練的價值函數或幾何檢查。
    """
    name, arg = skill_call
    sk = skills[name]
    # 前提條件不滿足 → 可供性極低 ( 但不為零，保留探索空間 )
    if not sk.precond(state, arg):
        return 0.05
    # 前提滿足 → 依動作的難度給分
    base = {'goto': 0.95, 'pick': 0.75, 'place': 0.90, 'open': 0.70}
    return base.get(name, 0.5)

# ===== SayCan 規劃器 ( 32.5.2 節 ) =====
def saycan_step(instruction, state, skills, candidates, verbose=True):
    """一步的決策：Say x Can"""
    scored = []
    for call in candidates:
        p_say = llm_score(call, instruction, state)
        p_can = affordance_score(call, skills, state)
        scored.append((call, p_say, p_can, p_say * p_can))
    scored.sort(key=lambda x: -x[3])

    if verbose:
        print(f" 當前狀態：{state}")
        print(f"  {'技能':<22}{'Say':>7}{'Can':>7}{'綜合':>8}")

```

```

    for call, ps, pc, tot in scored[:5]:
        mark = ' ←選擇' if call == scored[0][0] else ''
        print(f" {str(call):<22}{ps:7.2f}{pc:7.2f}{tot:8.3f}{mark}")
    return scored[0]

# ===== 完整的閉環執行 ( 32.5.4 節 ) =====
def execute_task(instruction, state, skills, goal_check, max_steps=8):
    candidates = [('goto','table'), ('goto','kitchen'),
                  ('pick','cup'), ('place','kitchen'),
                  ('place','table'), ('open','cabinet')]
    history = []
    print(f"指令：「{instruction}」\n")

    for step in range(max_steps):
        print(f"--- 第 {step+1} 步 ---")
        call, p_say, p_can, total = saycan_step(
            instruction, state, skills, candidates)

        # 安全門檻 ( 32.9.2 節第二層：輸出驗證 )
        if total < 0.15:
            print(" 綜合分數過低 → 停止並請求協助")
            return False, history

        name, arg = call
        sk = skills[name]

        # 符號驗證：執行前再次檢查前提條件
        if not sk.precond(state, arg):
            print(f" X 前提條件不滿足 · 跳過")
            history.append((str(call), '前提不滿足'))
            continue

        # 執行
        state = sk.effect(state, arg)
        history.append((str(call), '成功'))
        print(f" ✓ 執行 {call}\n")

        if goal_check(state):
            print(f"=== 任務完成 ( 共 {step+1} 步 ) ===")
            return True, history
    return False, history

# ===== 執行 =====
skills = build_skill_library()
init = WorldState(robot_at='start', holding=None,
                  objects={'cup': 'table', 'book': 'table'},
                  containers={'cabinet': False})
goal = lambda st: st.objects.get('cup') == 'kitchen'

```

```

ok, hist = execute_task('幫我把桌上的杯子拿到廚房', init, skills, goal)

print("\n=== 執行歷史 ===")
for i, (c, r) in enumerate(hist, 1):
    print(f"  {i}. {c:<22} → {r}")

# ===== 例題 32-2 的驗證 =====
print("\n=== 例題 32-2 : Say x Can 的互補作用 ===")
cases = [
    ("find(sponge)", 0.75, 0.90), ("pick(sponge)", 0.80, 0.15),
    ("go_to(table)", 0.40, 0.95), ("clean(spill)", 0.95, 0.05),
    ("say('抱歉')", 0.30, 1.00),
]
print(f"'技能':<18}{ 'Say':>7}{ 'Can':>7}{ '綜合':>9}")
best = max(cases, key=lambda x: x[1]*x[2])
for n, ps, pc in cases:
    mark = ' ← 選擇' if (n, ps, pc) == best else ''
    print(f"{n:<18}{ps:7.2f}{pc:7.2f}{ps*pc:9.3f}{mark}")
print(f"\n 只看 Say → 會選 {max(cases, key=lambda x:x[1])[0]} (空手擦拭·失敗)")
print(f" 只看 Can → 會選 {max(cases, key=lambda x:x[2])[0]} (沒解決問題)")
print(f"Say x Can → 選 {best[0]} (正確：先找工具)")

# ===== 例題 32-1 : 狀態空間的計算 =====
print("\n=== 例題 32-1 : 狀態空間的組合爆炸 ===")
print(f"'物體數':>7}{ '地點數':>7}{ '命題數':>9}{ '理論狀態':>14}{ '合法狀態':>12}")
for n_obj, n_loc in ((5,4), (10,6), (20,10)):
    n_prop = n_loc + n_obj*n_loc + n_obj
    theoretical = 2**n_prop
    # 合法：機器人位置 × (無持物 or 持一物)
    legal = n_loc * (n_loc**n_obj + n_obj * n_loc**(n_obj-1))
    print(f"{n_obj:7d}{n_loc:7d}{n_prop:9d}{theoretical:14.2e}{legal:12.2e}")
print("→ 約束大幅縮減搜尋空間·但規模增大時仍會爆炸 (需啟發式搜尋)")

# ===== 例題 32-3 : 分層的時間預算 =====
print("\n=== 例題 32-3 : 分層架構的時間預算 ===")
layers = [("LLM 任務規劃", 1200, "每次指令"), ("VLM 場景理解", 350, "1~2 Hz"),
          ("傳統視覺定位", 40, "10~25 Hz"), ("運動規劃", 80, "5~10 Hz"),
          ("控制迴路", 1, "1000 Hz")]
print(f"'層級':<16}{ '耗時(ms)':>10}{ '頻率':>14}")
total = 0
for name, t, f in layers:
    print(f"{name:<16}{t:10d}{f:>14}")
    if name != "控制迴路": total += t
print(f"'啟動總延遲':<16}{total:10d}{':>14}")
print(f"\n 使用者要求 3 秒內開始動作 → ")
    f"'可行' if total < 3000 else '不可行' ( 餘裕 {3000-total} ms )"
print("優化：LLM 與 VLM 平行 → 省 350 ms ; 常見指令快取 → 省 1200 ms")

```

請試著修改：（一）把杯子的初始位置改到 `cabinet`（櫃子裡）並讓櫃子關著，觀察系統是否會自動先執行 `open(cabinet)`。（二）把 `pick` 的可供性基準分數降到 0.1（模擬抓取很困難），觀察決策如何改變。（三）加入一個「杯子是滿的」的狀態，並設計對應的技能與獎勵邏輯。

實作：建立語言驅動的機器人系統

本章實作可在模擬環境或實體機器人上進行。若使用雲端的語言模型 API，請注意費用與資料隱私。

- 技能庫設計**：為你的機器人定義 5~10 個原子技能。每個技能明確寫出：參數、前提條件、預期效果、失敗判準。**這是整個系統的基礎，務必仔細設計。**
- PDDL 規劃**：把技能庫轉寫為 PDDL 領域檔，用現成的規劃器（如 Fast Downward）求解幾個任務。**體會符號規劃的正確性保證。**
- 提示工程**：設計 32.4.2 節的提示模板，包含技能清單、當前狀態、規則與失敗出口。用語言模型測試十個不同的指令。
- 幻覺測試**：故意問一些做不到的事（「幫我泡咖啡」但沒有咖啡機技能），觀察模型是否會誠實回覆「做不到」，還是硬生出一個計畫。**記錄失敗率。**
- 思維鏈比較**：對同樣的十個指令，比較「直接輸出計畫」與「先推理再輸出」的正確率。驗證 32.4.4 節的說法。
- 輸出驗證**：實作 32.9.2 節的第二層——語法檢查與符號驗證。統計有多少比例的計畫被攔截，以及攔截的原因分類。
- 可供性整合**：實作簡化的可供性評估（可用幾何可達性檢查），並實作 SayCan 的分數相乘。比較「有無可供性」的成功率差異。
- 閉環執行**：實作 32.5.4 節的閉環架構。故意讓某一步失敗（如移走物體），觀察系統是否能重新規劃。
- VLM 整合**：接上一個視覺語言模型，讓它描述場景並回答問題（「桌上有幾個杯子？」）。測試開放詞彙的定位能力。
- 延遲量測**：量測各層的實際處理時間，繪出時間預算表（如例題 32-3）。找出瓶頸並嘗試

優化。

11. 安全測試：設計十個「危險或不當」的指令，測試三層防護是否能全部攔截。**這一步不可省略。**

12. (進階) 完整系統：整合語音輸入、VLM 感知、LLM 規劃、可供性過濾、與實際執行，完成一個能回應自然語言的服務機器人。

表 32-17 語言驅動系統的測試記錄表 (前兩列為範例)

測試指令	生成的計畫	是否可執行	攔截層	問題分析
「拿杯子到廚房」	goto/pick/goto/place	✓	—	正常
「幫我泡咖啡」	use(coffee_machine)	✗	第一層	幻覺：技能不存在
(自行填寫其餘測試)				

國中小也看得懂：那些「大家都知道」的事

假設你請一個從來沒來過你家的機器人：「幫我把桌上的杯子拿到廚房。」

這句話很簡單對不對？但你有沒有想過，要做到這件事，它得知道多少事情？

第一，什麼是「杯子」？如果桌上有三個杯子，是哪一個？

第二，「廚房」在哪裡？它的地圖上只有一堆座標，沒有寫「這裡是廚房」。

第三，要做哪些事？走到桌邊、找到杯子、拿起來、走到廚房、放下——這五個步驟你有說嗎？沒有！是它要自己想出來的。

第四，那些「大家都知道」的事。杯子要正著拿（不然裡面的水會灑出來）、要放在桌面上（不能丟在地上）、如果杯子是熱的要小心。

以前的科學家想把這些「大家都知道的事」一條一條寫進電腦裡。結果呢？永遠寫不完！今天寫了杯子，明天要寫碗；今天寫了廚房，明天要寫陽台.....

但最近幾年，事情有了很大的變化。有一種叫做「大型語言模型」的 AI，它讀過幾乎整個網路上的文字——所有的書、所有的網頁、所有的說明。所以它自然就知道：杯子要正著拿、牛奶放冰箱、廚房通常有水槽。

那些我們寫不完的常識，它從讀書中學會了！

不過呢，它有一個很麻煩的毛病：**它有時候會很有自信地說錯話**。

比如它可能說「打開櫃子拿出杯子」——但那個櫃子根本不存在，是它自己想像出來的！

如果只是聊天，說錯了大不了笑一笑。但機器人不一樣——**它說什麼就會真的去做**。所以工程師必須加上好幾層「檢查關卡」：

第一關：先告訴它「你只能用這幾個動作」，不准自己發明。

第二關：它想出計畫之後，先檢查「這樣做真的可行嗎？」

第三關：真的動起來的時候，如果快撞到東西就立刻停下來。

這就像騎腳踏車要戴安全帽一樣——不是因為你一定會摔倒，而是萬一摔倒的時候，要有東西保護你。

叫獸提醒與常見錯誤

- 誤區一：以為大型模型能取代所有傳統方法。它提供常識與理解，但沒有正確性保證，也不懂物理。
- 誤區二：不限制模型的輸出。必須明確提供技能清單與可見物體，否則會產生幻覺（不存在的物體或動作）。
- 誤區三：不給模型「說做不到」的出口。否則它會硬生出一個錯誤的計畫，而非誠實地回報無法達成。
- 誤區四：只用語言模型而不檢查可供性。它會規劃出「空手擦拭」這類合理但當下不可行的動作。
- 誤區五：開環執行而不檢查結果。某一步失敗後，後續的計畫全部失效，機器人會做出荒謬的動作。
- 誤區六：把 VLM 用於精確定位。它的空間關係精度不足以支持抓取，仍需傳統的幾何方法。
- 誤區七：把大型模型放進控制迴路。其推論延遲達數百毫秒，只能用於低頻的決策層。
- 誤區八：以為符號式規劃已經過時。它提供大型模型所缺乏的正確性保證，是驗證層的核心。

- 誤區九：技能庫的粒度設計不當。太細則模型難以規劃、太粗則失去組合的彈性。
- 誤區十：把安全機制交給神經網路判斷。安全必須是確定性、可驗證、且獨立於學習系統的。
- 誤區十一：忽略「回應延遲」的心理門檻。超過一秒沒有任何反應，使用者就會覺得系統當機了。
- 誤區十二：未測試惡意或誤解的指令。應主動設計危險指令測試三層防護是否有效。

課堂討論

1. 32.1 節的符號接地問題困擾了三十年。請討論：大型模型是「解決」了這個問題，還是只是「繞過」了它？兩者的差別重要嗎？
2. 例題 32-2 顯示 Say $\times$ Can 能自動推導出「先找工具再擦拭」的順序，而這條規則沒有人寫下來。請討論這種「湧現的常識」的可靠性——它在什麼情況下會失效？
3. 32.9.2 節主張「智慧可以是機率性的，但安全必須是確定性的」。請討論這個原則的邊界——有沒有情況是安全機制也必須靠學習？
4. VLA 模型把感知、理解與動作合而為一，放棄了模組化。請討論這個取捨，並比較它與本書其他章節的分層架構思想。
5. 32.11.2 節提問：語言模型「知道杯子要正著拿」，這與人類的「知道」是同一回事嗎？請在讀第 33 章之前，先寫下你的看法。

章末摘要

- 符號接地問題：符號只能用符號解釋會陷入循環，必須接地到感知與行動才有意義。
- 接地分三個層次：物體接地（哪個區域）、空間接地（什麼座標）、動作接地（什麼運動指令）。
- 近年突破的三個原因：開放詞彙的視覺模型、語言模型內含的常識、以及語言成為模組間的通用介面。
- PDDL 以「前提條件與效果」描述動作，規劃器能自動推導出動作序列並保證正確性。
- 符號式規劃的三個限制：知識工程負擔、無法處理常識、以及符號從何而來（接地問題）。
- 大型語言模型的訓練目標是預測下一個詞，但這隱含地要求了對世界的理解。

- 它對機器人有價值的三種能力：常識知識、任務分解、語意理解與指代消解。
- 它的三個限制：幻覺、沒有物理直覺、不知道機器人能做什麼。
- 把語言模型視為「見多識廣但沒到過現場的顧問」是最貼切的理解。
- 語言模型的三種使用方式：自然語言步驟、技能序列、程式碼生成；後者最靈活且品質最佳。
- 提示設計的四個要素：技能清單、當前狀態、明確規則、以及「做不到」的出口。
- 思維鏈（先推理再輸出）能顯著降低錯誤率，並提供可解釋性。
- SayCan 以「 $P_{LLM} \times P_{affordance}$ 」結合目標導向與現實約束，缺一則會失敗。
- 可供性有三種來源：學習的價值函數、幾何檢查、以及視覺語言模型的判斷。
- 閉環架構在每一步後把結果回饋給模型，使其能從失敗中恢復並應對環境變化。
- VLM 的開放詞彙能力直接解決了物體接地；但其空間精度不足，仍需傳統幾何方法。
- VLA 把動作編碼為詞元，使預訓練的視覺語言模型能直接輸出動作，但受限於速度、資料與精度。
- TAMP 的困難在於符號與幾何互相依賴；大型模型可提供好的候選以縮小搜尋範圍。
- 現代機器人任務系統分五層：理解、任務規劃、接地、運動規劃、控制；頻率由上而下遞增。
- 四類風險：幻覺、能力不符、物理無知、惡意指令；防護分三層：輸入約束、輸出驗證、執行監控。
- 不可妥協的原則：智慧可以是機率性的，但安全必須是確定性、可驗證、且獨立於學習系統的。

觀念題與計算題

觀念題

1. 說明符號接地問題的核心，以及為何純符號的定義會陷入循環。
2. 列舉符號接地的三個層次，並指出各自所需的技術。
3. 說明近年符號接地問題有所突破的三個原因。
4. 說明 PDDL 中動作定義的三個部分，並解釋規劃器如何運作。

5. 比較路徑規劃與任務規劃的異同。
6. 列舉符號式規劃的三個限制，並說明為何它仍然重要。
7. 說明大型語言模型的訓練目標，以及為何這隱含地要求了對世界的理解。
8. 列舉語言模型對機器人有價值的三種能力與三個限制。
9. 為何說語言模型是「見多識廣但沒到過現場的顧問」？
10. 比較語言模型的三種使用方式，並說明程式碼生成的四個優勢。
11. 說明提示設計的四個要素，並解釋「做不到的出口」為何重要。
12. 說明思維鏈的作用與其兩個好處。
13. 說明 SayCan 的核心公式，並解釋只有 Say 或只有 Can 會分別產生什麼問題。
14. 列舉可供性的三種來源及其優缺點。
15. 說明閉環架構的三個價值。
16. 說明 VLM 的三類用途，以及開放詞彙能力為何是「質的差異」。
17. 說明 VLM 的三個限制，以及它與傳統視覺的分工。
18. 說明 VLA 把動作編碼為詞元的巧妙之處，以及其四個限制。
19. 說明 TAMP 問題的困難，以及大型模型能如何協助。
20. 列出現代機器人任務系統的五層架構及各層的頻率。
21. 列舉四類風險與三層防護，並說明「安全必須是確定性的」這個原則。

計算題

1. 某環境有 8 個物體、6 個地點、1 台機器人。(a) 計算三類命題的數量與總數。(b) 計算理論上的狀態空間大小。(c) 加入「機器人只在一處」「每物體在一處或被握著」「最多握一個」的約束後，計算合法狀態數。(d) 計算縮減的倍數。
2. 某任務的候選技能與評分為：A (Say 0.9, Can 0.2)、B (Say 0.6, Can 0.85)、C (Say 0.4, Can 0.95)、D (Say 0.85, Can 0.7)。(a) 計算各技能的綜合分數。(b) 判斷應執行哪一個。(c) 若只看 Say 或只看 Can，分別會選哪一個？(d) 討論此例中三種判準的差異。
3. 某系統各層的處理時間為：語音辨識 400 ms、LLM 規劃 900 ms、VLM 理解 300 ms、視覺定位 50 ms、運動規劃 120 ms。(a) 計算串列執行的總延遲。(b) 若 LLM 與 VLM 可

平行，重新計算。(c) 若對常見指令做快取（省下 LLM 的時間），再計算。(d) 判斷各情況是否滿足「1 秒內有回應」的要求。

4. 某語言模型產生的計畫有 12% 的機率包含至少一個錯誤。系統的三層防護各能攔截：第一層 60%、第二層 80%（針對通過第一層者）、第三層 90%（針對通過前兩層者）。(a) 計算通過所有防護的錯誤比例。(b) 計算整體的攔截率。(c) 若要把漏網率降到 0.01% 以下，求第三層需要提升到多少。(d) 討論增加第四層防護的邊際效益。

動手做與專題延伸

本章實作：語言驅動的任務系統

建立一個完整的語言驅動機器人系統，這是全書技術的最高整合。

- **階段一：**設計技能庫並以 PDDL 形式化，用現成的規劃器驗證。
- **階段二：**實作提示模板與語言模型的介接，測試指令的解析。
- **階段三：**實作三層防護（輸入約束、輸出驗證、執行監控）。
- **階段四：**加入可供性評估並實作 SayCan 的分數結合。
- **階段五：**整合為閉環系統，並接上模擬器或實體機器人執行。

專題延伸

- **幻覺率的量化研究：**設計 50 個測試指令（其中 20 個是機器人能力範圍外的）。分別測試四種提示設計：（甲）無任何限制、（乙）只給技能清單、（丙）技能清單加當前狀態、（丁）再加上思維鏈與失敗出口。統計各自的幻覺率與正確率。繪出「提示設計 vs 錯誤率」的長條圖。這個實驗能量化證明提示工程的價值。
- **Say-Can 分數的視覺化：**對一個多步驟的任務，記錄每一步所有候選技能的 Say 分數、Can 分數與綜合分數。以熱力圖呈現整個決策過程——你會清楚看到「當條件改變時，分數如何重新分配」。特別觀察那些「Say 高但 Can 低」的技能如何被自動排除，以及在條件滿足後又如何浮現。
- **進階：混合式規劃器的實作與比較。**實作三種任務規劃方式：（甲）純 PDDL 符號規劃、（乙）純語言模型規劃、（丙）語言模型提出候選 + PDDL 驗證的混合式。在一組 30 個任務上測試（包含 10 個需要常識、10 個需要嚴格順序、10 個是不可能達成的）。比較三者的成功率、規劃時間、以及對「不可能任務」的處理方式。你應該會發現：純符號在需要常識

的任務上失敗、純語言在需要嚴格順序的任務上出錯、而混合式在兩者都表現良好——這將以實驗驗證本章的核心主張。

參考文獻與延伸閱讀

- Ahn, M. et al., "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances," CoRL 2022 (SayCan 的原始論文) 。
- Liang, J. et al., "Code as Policies: Language Model Programs for Embodied Control," ICRA 2023 (程式碼生成作為機器人介面) 。
- Huang, W. et al., "Inner Monologue: Embodied Reasoning through Planning with Language Models," CoRL 2022 (閉環的語言規劃架構) 。
- Brohan, A. et al., "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control," CoRL 2023 (VLA 的代表性工作) 。
- Open X-Embodiment Collaboration, "Open X-Embodiment: Robotic Learning Datasets and RT-X Models," ICRA 2024 (跨機器人的大規模資料集) 。
- Harnad, S., "The Symbol Grounding Problem," Physica D, 1990 (符號接地問題的原始論文) 。
- Ghallab, M., Nau, D. and Traverso, P., Automated Planning and Acting, Cambridge University Press (任務規劃的權威教材) 。
- Garrett, C. R. et al., "Integrated Task and Motion Planning," Annual Review of Control, Robotics, and Autonomous Systems, 2021 (TAMP 的完整綜述) 。
- Radford, A. et al., "Learning Transferable Visual Models From Natural Language Supervision," ICML 2021 (CLIP · 開放詞彙視覺的基礎) 。
- Firoozi, R. et al., "Foundation Models in Robotics: Applications, Challenges, and the Future," IJRR, 2024 (基礎模型在機器人上的綜述) 。

第 33 章

具身人工智慧

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為 Moravec 悖論的對照：上格是超級電腦棋盤上擊敗世界冠軍的畫面，旁註「抽象推理：演化 10 萬年」；下格是一雙孩童的手正在摺衣服，旁註「知覺行動：演化 5 億年」。中央為具身智慧的三角關係圖——「腦」「身體」「環境」三個節點以雙向箭頭相連，中央寫著「智慧」，並註明「純計算只複製了三分之一」。右半為形態計算的實例拼貼：被動行走的雙腿、欠驅動手指包絡物體、柔性指墊的變形、以及一個倒角引導插銷入孔，四者以共同的標籤串起——「身體免費地做了原本需要計算的事」。下緣文字：智慧不只在腦裡，也在身體裡，還在身體與環境的互動裡。

叫獸開講 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？

最後一堂課，叫獸沒有帶投影片。他在白板上寫下第 1 章開學時就寫過的那個問題：

為什麼電腦下棋能贏世界冠軍，卻摺不好一件衣服？

「三十二章之前，我問過你們這個問題。」他說，「現在你們學完了運動學、動力學、控制、感知、規劃、學習、大型模型——**誰能回答我？**」

小翰試著說：「因為摺衣服的物理很複雜，布料有無限多自由度，寫不出方程式。」

「這是對的，但只是表面。」叫獸搖頭，「我再問一個問題：**一隻蜜蜂的腦只有一百萬個神經元，比你手機的算力差了幾億倍。但它能在風中飛行、找到花、找到路回家。而我們用一整個資料中心訓練出來的模型，還是做不到。為什麼？**」

教室安靜了很久。小好慢慢地說：「因為.....蜜蜂有身體？」

「說下去。」

「牠的翅膀本身就會處理氣流，不需要用腦去算。牠的複眼結構就適合偵測運動，不需要跑演算法。」小好越說越快，「就像第 21 章那個軟手套——**變形這件事本身就在做計算。**」

叫獸點了點頭，在白板上寫下這一章的核心：

智慧不只在腦裡，也在身體裡，還在身體與環境的互動裡。

「這叫做**具身認知**。它主張：**你不能把智慧從身體裡切出來。一個沒有身體的智慧，不是「智慧少了身體」，而是根本不是同一種東西。**」

他走到窗邊：「第 32 章那個語言模型知道『杯子要正著拿』——因為它讀過。但一個兩歲的孩子知道這件事，是因為**他打翻過**。這兩種『知道』一樣嗎？」

「今天這堂課，我不會教你們新的公式。」叫獸最後說，「我要問的是三個問題：**Moravec 悖論的真正答案是什麼？機器人的智慧該往哪裡走？以及——當機器人真的能走進我們的生活，我們準備好了嗎？**」

本章為什麼重要

這是全書理論部分的最後一章。前面三十二章教的是**方法**——如何建模、如何控制、如何

感知、如何學習。本章要退後一步，問一個更根本的問題：**這些方法為什麼是這樣的？**

具身人工智慧 (Embodied AI) 提供了一個統一的觀點。它主張：智慧不是一個能獨立存在的抽象能力，而是**身體、大腦與環境三者互動的產物**。這個觀點能解釋許多本書中反覆出現卻未被明說的現象：

- 為何第 21 章的**軟性夾爪**能用簡單的機構取代複雜的感測與計算？
- 為何第 28 章的**被動行走**不需要任何馬達與控制器就能穩定？
- 為何第 20 章的**一個倒角**能把裝配的容差放寬 41 倍？
- 為何第 1 章的 **Moravec 悖論**成立——對電腦而言，推理容易而知覺行動困難？

這些不是巧合，而是同一個原理的不同體現：當身體的結構與任務相匹配時，大量的「計算」可以由物理本身完成。

本章也將處理三個更廣的議題：機器人的**社會影響**（就業、責任、隱私）、**倫理框架**、以及**通往通用機器人的路徑**。這些不是技術問題，卻決定了本書所有技術最終能否被社會接受。

學習目標

- 能完整解釋 Moravec 悖論的成因並提出三個層次的答案。
- 能說明具身認知的核心主張與其四個支柱。
- 能定義形態計算並辨識本書中的實例。
- 能說明感知—行動迴路的意義及其與被動感知的差異。
- 能說明主動感知與具身探索的價值。
- 能區分符號主義、連結主義與具身觀點三種典範。
- 能說明基礎模型對機器人的意義與其限制。
- 能分析通往通用機器人的三條路徑及其取捨。
- 能說明機器人對就業與社會的影響並提出平衡的觀點。
- 能運用倫理框架分析機器人的責任歸屬問題。
- 能說明機器人在安全、隱私與偏見上的風險。
- 能為自己的機器人專案定位其在技術與社會脈絡中的位置。

先備知識

本章是全書的綜合，建議在讀完前面三十二章後閱讀。特別重要的是：第 1 章的 Moravec 悖論與自主性光譜、第 21 章的柔順抓取、第 28 章的被動行走、第 31 章的學習限制、第 32 章的大型模型。

本章沒有數學推導，但需要的是回顧與連結的能力——把前面各章看似獨立的洞見，串成一個統一的圖像。建議閱讀時隨手翻回被引用的章節，這個「重新連結」的過程本身就是最重要的學習。

核心名詞中英對照

表 33-1 本章核心名詞

中文	英文	說明
具身人工智慧	Embodied AI	強調身體與環境互動的人工智慧觀點
具身認知	Embodied Cognition	認知依賴於身體結構與感知運動經驗
Moravec 悖論	Moravec's Paradox	推理容易而知覺行動困難的現象
形態計算	Morphological Computation	由身體結構本身完成的「計算」
感知—行動迴路	Perception-Action Loop	感知與行動互相構成的循環
主動感知	Active Perception	透過行動主動獲取資訊
可供性	Affordance	環境提供給特定身體的行動可能性
符號主義	Symbolic AI / GOFAI	以符號操作為核心的人工智慧典範
連結主義	Connectionism	以神經網路為核心的典範
基礎模型	Foundation Model	大規模預訓練的通用模型
通用機器人	General-Purpose Robot	能執行多樣任務的機器人
自動化紅皇后	Automation Paradox	自動化程度越高、人的角色越關鍵
責任歸屬	Accountability	系統造成損害時的責任認定
演算法偏見	Algorithmic Bias	系統對特定群體的系統性不利
價值對齊	Value Alignment	使系統的目標符合人類價值
恐怖谷	Uncanny Valley	過度擬人卻不完美所引發的不適感

33.1 Moravec 悖論的答案

33.1.1 問題的重述

第 1 章提出了這個現象：

◎ Moravec 悖論 (1988)

「要讓電腦在智力測驗或下棋上表現出成人的水準相對容易，但要讓它具備一歲小孩的知覺與行動能力，卻極為困難或根本不可能。」——Hans Moravec

三十多年後，這個觀察依然成立。大型語言模型能通過律師考試、能寫程式、能做數學證明；但沒有任何機器人能像五歲孩子那樣摺衣服、綁鞋帶、或在陌生的房間裡自在地移動。

33.1.2 第一層答案：演化的時間

Moravec 自己給出的解釋是演化的觀點：

知覺與運動能力：演化了約 5 億年

抽象推理能力：演化了不到 10 萬年

前者的優化時間是後者的五千倍。

這個解釋的洞見在於：我們覺得容易的事，往往是被優化得最徹底的事——徹底到我們完全意識不到它有多複雜。

一個具體的對照：你「毫不費力」地拿起一個杯子，這個過程動用了：視覺辨識、深度估計、抓取點選擇、軌跡規劃、力控制、滑動偵測與即時修正——本書第 21、22、26 章花了近百頁描述的內容，你的大腦在 0.3 秒內完成，且完全不需要意識參與。

反之，我們覺得困難的事（如心算三位數乘法），往往是演化上最新、最未經優化的。而這恰恰是電腦最擅長的。

33.1.3 第二層答案：知識的形式

但演化的解釋只說明了「為什麼難」，沒說明「難在哪裡」。第二層的答案是知識的形式不同：

表 33-2 兩種知識的對照

面向	抽象推理的知識	知覺行動的知識
形式	顯性、可陳述	隱性、難以言說
範例	「西洋棋的規則是……」	「腳踏車要這樣平衡」
能否寫下	可以（規則、公式）	極難（你說得出怎麼騎車嗎？）

面向	抽象推理的知識	知覺行動的知識
獲得方式	學習、閱讀	練習、身體經驗
表示	符號、邏輯	感知運動的模式
對電腦	容易編碼	難以編碼

這正是第 32.2.3 節「知識工程的負擔」的根源。符號式方法要求把知識寫成明確的規則——而知覺行動的知識本質上就寫不出來。這不是我們還沒找到方法，而是這類知識的形式就不是符號的。

這也解釋了為何第 31 章的學習方法在此有效：既然寫不出來，就不要寫——讓系統從經驗中直接學習那個「說不出來的模式」。

33.1.4 第三層答案：身體的參與

但還有更深的一層，而這正是本章的核心主張：

◎ 第三層答案

知覺與行動之所以困難，是因為它們不只發生在「腦」裡——它們發生在「腦、身體、環境」的互動中。我們試圖用純計算的方式重現它們，等於是只複製了三分之一的系統。一隻蜜蜂的腦只有一百萬個神經元（不到人腦的十萬分之一），卻能完成我們的機器人做不到的飛行任務——因為牠的翅膀、複眼、體重分布，本身就在「計算」。

這個觀點帶來一個關鍵的推論：

若身體與任務不匹配 → 大腦必須做更多計算來補償

若身體與任務匹配 → 大量的計算可以被「省掉」

這正是 33.3 節形態計算的核心，也是本書多處實例的統一解釋。

33.2 具身認知

33.2.1 核心主張

具身認知 (Embodied Cognition) 是認知科學中的一個立場，其核心主張是：

認知不是「腦中的計算」，而是「身體在環境中的活動」

這是對傳統認知科學的根本挑戰。

傳統的觀點把認知視為資訊處理：感官輸入 → 大腦計算 → 動作輸出。在這個觀點下，身體

只是「輸入裝置」與「輸出裝置」，本身不參與思考。

具身認知則主張：**身體的結構直接塑造了認知的形式**。同樣的「智慧」在不同的身體中，會是不同的東西。

33.2.2 四個支柱

具身認知的主張可以拆解為四個支柱，本書中都有對應：

表 33-3 具身認知的四個支柱

支柱	主張	本書中的體現
身體塑造認知	認知的形式取決於身體的結構	第 21 章：軟手的抓取策略與硬手完全不同
環境是資源	環境不只是問題，也是計算的一部分	第 28 章：被動行走利用重力與地形
行動構成感知	感知不是被動接收，而是主動探索	第 22 章：主動視覺；33.4 節
即時互動優於內部表示	與其建立完整的世界模型，不如快速反應	第 26 章：DWA 的局部反應優於全域重規劃

33.2.3 一個具體的對照

以「抓取一個未知的物體」為例，比較兩種取徑：

取徑一（傳統計算觀點）：

1. 用視覺重建物體的完整三維模型。
2. 估計其質量、質心、摩擦係數。
3. 計算力封閉（第 21.3 節）並最佳化抓取品質。
4. 規劃無碰撞的軌跡（第 26 章）。
5. 以精確的力控制執行（第 20 章）。

取徑二（具身觀點）：

1. 用一隻柔軟、欠驅動的手（第 21.8 節）。
2. 粗略地移動到物體附近。
3. 閉合手指——機構會自動貼合物體的形狀。
4. 若滑動，材料的變形會自動增加摩擦（第 21.7.1 節）。

取徑二用了幾乎為零的計算，成功率卻可能更高。這不是因為它「更聰明」，而是因為它把計算的負擔轉移到了身體上。

◎ 這不是說計算不重要

必須澄清一個常見的誤解。具身認知不是主張「不需要計算」，而是主張「計算、身體與環境應該協同設計」。第 20 章的力控制、第 26 章的規劃、第 31 章的學習——這些都極為重要。具身觀點所補充的是：在投入計算資源之前，先問「這個問題能不能用身體解決」。這正是本書從第 20 章開始反覆出現的那句話——「在把問題丟給控制器之前，先問能不能用機構把問題變簡單」。

33.3 形態計算

33.3.1 身體如何「計算」

形態計算 (Morphological Computation) 是具身認知在工程上最具體的體現。它的定義是：

◎ 形態計算

由身體的物理結構 (形態) 本身所完成的功能，若用控制器實現則需要感測與計算。換句話說：身體「免費地」做了原本需要計算的事。

判斷的標準是：若把這個結構換成剛性的、簡單的版本，是否需要加上感測器與控制演算法才能達成同樣的功能？若是，那麼原本的結構就在做形態計算。

33.3.2 本書中的實例

回顧全書，形態計算的實例俯拾皆是——只是當時沒有用這個名字：

表 33-4 本書中的形態計算實例

實例	章節	身體做了什麼	否則需要什麼
被動動態行走	28.1.3	機構的動力學自然產生穩定步態	完整的步態規劃與平衡控制
欠驅動手指	21.8.2	連桿自動分配力量、包絡物體	多個力感測器 + 各指獨立控制
柔性指墊	21.7.1	變形貼合、滑動時自動增加摩擦	滑動偵測 + 即時力控制
RCC 順應裝置	20.8.2	順應中心落在尖端，自動對位	高精度視覺 + 力控制迴路
孔口倒角	20.9.3	把精確對準轉為落入範圍	定位精度提升 41 倍
串聯彈性驅動器	5.9、28.6.4	儲存並回收能量、吸收衝擊	高頻力控制 + 更大的馬達
顆粒堵塞夾爪	21.8.3	同時實現柔軟適應與剛性固定	可變剛度的複雜機構與控制

實例	章節	身體做了什麼	否則需要什麼
麥克納姆輪	23.5	機構解除非完整約束	複雜的路徑規劃（第 26.5 節）

這張表是全書一條隱藏主線的顯影。從第 5 章到第 28 章，我們反覆遇到「用機構解決問題」的例子——現在它們有了一個統一的名字。

33.3.3 三個層次的形態計算

形態計算可以分為三個層次，複雜度遞增：

層次一：被動的結構特性。最單純的形式——形狀、剛度、阻尼的設計。例如倒角、柔性指墊。這一層幾乎是免費的（不增加成本與複雜度），卻常帶來巨大的效益。

層次二：機構的耦合。透過連桿、彈簧、筋腱把多個自由度耦合起來，使它們自動協調。例如欠驅動手指、被動行走的腿。這一層需要巧妙的設計，但一旦設計完成，運作極為可靠。

層次三：與環境的動態耦合。利用環境的物理特性（重力、氣流、地形）作為系統的一部分。例如被動行走利用重力位能、鳥類利用氣流滑翔。這一層最為精妙，但也最難設計與分析。

33.3.4 形態計算的限制

但形態計算不是萬靈丹，它有三個明確的限制：

- **不可重新編程。**一個為抓球設計的柔性手，無法透過「改參數」變成適合抓平板的手。機構一旦製造出來就固定了——而軟體可以更新。
- **難以分析與驗證。**柔性結構的行為難以用第 13 章的剛體動力學描述，也難以用第 19 章的方法證明穩定性。
- **設計困難。**要設計出「恰好能自動完成某功能」的機構，需要深厚的直覺與大量的試誤——這比寫一個控制器困難得多。

因此正確的態度是「協同設計」：用形態計算處理那些穩定不變、且需要快速反應的部分（如接觸的順應性）；用控制與學習處理那些需要彈性與適應的部分（如任務的切換）。

33.4 感知—行動迴路

33.4.1 感知不是被動的

傳統的觀點把感知視為單向的資訊流：世界 → 感測器 → 處理 → 理解。但具身觀點主張：感知本質上是一個迴路。

行動 → 改變感知 → 影響下一個行動 → 再改變感知 → ...

感知與行動互相構成，無法分割。

三個具體的體現：

- 你的眼睛不停地在動。人眼每秒有 3~4 次跳視 (saccade) ——你「看見」的完整畫面，其實是主動掃描後拼湊出來的。被動盯著看一個點，你反而什麼都看不清楚 (除了中央凹的一小塊)。
- 你摸東西時手會動。要判斷一個表面的粗糙度，你會來回摩擦而非靜止按壓。靜止的觸覺幾乎無法分辨材質。
- 你會走過去看。若視線被遮擋，你不會用演算法推測遮擋後面是什麼——你會移動身體換一個角度。

33.4.2 主動感知

主動感知 (Active Perception) 把這個洞見變成工程方法：與其被動地處理接收到的資料，不如主動採取行動以獲取更好的資料。

表 33-5 被動感知與主動感知的對照

問題	被動感知的作法	主動感知的作法	本書章節
物體被遮擋	用推論或先驗補全	移動到另一個視角	第 22 章
不知道物體多重	由外觀估計質量	輕輕提起試試	第 21 章
深度不確定	用單目深度估計	移動相機產生視差	第 22.4.2 節
定位不確定	增加濾波器的複雜度	主動走向地標	第 24、25 章
不知道抓不抓得住	計算力封閉品質	先夾一下看會不會滑	第 21 章

主動感知常常比被動的演算法改良更有效。以「判斷物體重量」為例：由影像估計質量的誤差可能達 50%，而輕輕提起試一下的誤差只有幾個百分點。

這也是第 24.1.3 節「綁架機器人問題」的最佳解——與其設計更複雜的濾波器，不如主動移動到一個特徵明顯的位置重新定位。

33.4.3 探索與具身經驗

感知—行動迴路還有一個更深的含義：**機器人可以透過行動來「學習世界是什麼」。**

一個嬰兒如何知道「球會滾、積木不會」？不是有人告訴他，而是他**推過、丟過、看過結果**。這個過程稱為**具身探索** (embodied exploration) 。

在機器人上，這對應到：

- **互動式感知**：推一下物體看它怎麼動，從而判斷它的質量、摩擦與是否被固定。這比純視覺可靠得多。
- **自我監督學習**：機器人自己嘗試並記錄結果 (第 31.3.2 節)，不需要人工標註。
- **好奇心驅動的探索**：主動尋找「預測不準」的情況——因為那裡才有新資訊。這是強化學習中處理稀疏獎勵的重要方法。

◎ 為何這對第 32 章的大型模型是個提醒

第 32 章的語言模型「知道」杯子會破——因為它讀過。但它**從未經歷過**。這個差異在本書的脈絡下有具體的後果：它**沒有物理直覺** (32.3.3 節的限制二)。它知道「玻璃會破」這個命題，卻不知道「這個角度施力 5 牛頓會不會破」——因為後者不是一個能從文字中學到的知識，而是**必須從身體經驗中獲得的**。這正是為何目前最有效的架構仍是混合式：語言模型提供從文字學到的常識，而具身的經驗 (無論是模擬或真機) 提供物理的直覺。兩者缺一不可。

33.5 三種典範的辯證

33.5.1 人工智慧的三條路線

要理解具身觀點的定位，必須把它放在人工智慧發展史的脈絡中。三種典範依序登場，而**每一種都是對前一種局限的回應**：

表 33-6 人工智慧的三種典範

典範	盛期	核心主張	代表成果	遇到的困境
符號主義	1960~1980s	智慧 = 符號的操作與推理	專家系統、下棋程式	符號接地問題、常識無法窮舉
連結主義	1990s~至今	智慧 = 從資料中學習的統計模式	深度學習、大型模型	沒有保證、需要海量資料、缺乏物理直覺
具身觀點	1990s 興起	智慧 = 身體與環境的互動	行為式機器人、軟體機器人	難以擴展到高階認知、設計困難

33.5.2 Brooks 的挑戰

具身觀點在機器人學中的關鍵人物是 **Rodney Brooks**。他於 1990 年發表了一篇著名的論文，標題極為挑釁：《大象不下棋》（Elephants Don't Play Chess）。

他的論點是：

◎ Brooks 的核心主張 (1986~1991)

「智慧不需要表示。」（Intelligence without representation）傳統的機器人架構是「感知 → 建立世界模型 → 規劃 → 行動」，Brooks 主張這個中間的「世界模型」是不必要的，甚至是有害的——因為世界本身就是最好的模型（The world is its own best model）。他提出的包容式架構（Subsumption Architecture）把行為分成多個並行的層次，每一層直接把感知連到行動，不經過中央的世界模型。他用這個架構做出的機器人（如 Genghis）能在真實環境中穩健地行走，而當時基於世界模型的機器人在實驗室外幾乎寸步難行。

這個主張在當時極具衝擊，因為它挑戰了整個符號主義的基礎。而它的實證支持也很有力——簡單的反應式機器人在真實世界中的表現，往往勝過複雜的規劃式機器人。

33.5.3 三者的辯證與整合

但三十年後，我們可以看得更清楚：三種典範各自處理不同層次的問題，而完整的系統需要三者。

本書的架構本身就是這個整合的體現（回顧第 32.8.4 節的五層）：

表 33-7 三種典範在本書架構中的位置

層級	主導的典範	本書章節	為何是這個典範
任務理解與規劃	符號 + 連結	第 32 章	需要常識（連結）與正確性保證（符號）
感知與辨識	連結主義	第 22、31 章	難以寫出規則，但有大量資料
運動規劃	符號 / 幾何	第 26 章	有明確的數學結構與保證
運動控制	符號 / 模型	第 16~20 章	物理定律明確、需要即時保證
接觸與互動	具身	第 20、21、28 章	用機構的柔順取代感測與計算
身體設計	具身	第 4、5、21 章	形態計算：讓身體分擔計算

最後兩列是本書一條隱藏的主線。從第 4 章討論機構、第 5 章的驅動器選型、到第 21 章的

軟性夾爪與第 28 章的被動行走——「身體的設計」始終與「演算法的設計」同等重要。

◎ 為何典範之爭是假問題

科技史上常見「A 取代 B」的敘事——符號主義失敗了、深度學習贏了、接下來輪到具身智慧。但這個敘事幾乎總是錯的。真實的情況是：每個典範都解決了前者無法處理的問題，同時暴露了自己的局限。符號主義給了我們形式化與保證（第 26、32 章的規劃器至今仍在用）；連結主義給了我們泛化與感知（第 22、31 章）；具身觀點給了我們對「身體參與智慧」的理解（第 20、21、28 章）。一個真實的機器人系統，三者缺一不可——而知道哪一層該用哪一個，正是本書最想傳達的工程判斷力。

33.6 基礎模型與機器人

33.6.1 「機器人的 GPT 時刻」會來嗎

自然語言處理領域在 2020 年前後經歷了一次典範轉移：與其為每個任務訓練一個模型，不如訓練一個通用的大模型再微調。這種模型稱為**基礎模型**（Foundation Model）。

機器人領域自然會問：同樣的事會發生在機器人上嗎？

支持的論點：

- 第 32.7 節的 VLA 已展現了跨任務的泛化——RT-2 能理解「把水果放到能裝水的容器裡」，即使訓練資料中沒有這個組合。
- Open X-Embodiment 等跨機構的資料集正在建立，規模持續擴大。
- 視覺與語言的基礎模型可以直接遷移——機器人不需要從零學會「什麼是杯子」。

懷疑的論點：

- 資料的規模差了幾個數量級。網路上有數兆個詞元，而機器人資料集只有數百萬筆軌跡——而且無法從網路上取得（第 31.2.2 節）。
- 具身的多樣性。每台機器人的形態、感測器、動力學都不同。語言的「詞」是通用的，但「動作」不是——一個為七軸手臂學到的動作，無法直接用在六軸手臂上。
- 物理的不可壓縮性。語言可以「說得含糊」，但動作必須精確到公釐與毫秒。錯誤的代價不對稱（第 31.2.2 節）。

33.6.2 一個折衷的判斷

目前較合理的判斷是：**基礎模型會顯著改變機器人的「上層」**，但「下層」仍需傳統方法。

上層（理解、規劃、常識）→ 基礎模型帶來根本的改變

下層（控制、接觸、安全）→ 物理定律與即時性的要求不變

這正是第 32.8.4 節五層架構的預測。

具體的理由：上層處理的是「語意與判斷」——這正是基礎模型的強項，且對延遲不敏感（可以想一秒）。下層處理的是「物理與時序」——需要 1 kHz 的確定性運算與可證明的穩定性，這是神經網路的弱項。

33.6.3 資料的瓶頸

若基礎模型的路線要成功，**資料是最關鍵的瓶頸**。目前有四條路：

表 33-8 機器人資料的四條路徑

路徑	作法	優勢	限制
模擬	大規模平行模擬（第 30 章）	量大、便宜、安全	Sim-to-Real 落差
遙操作示範	人類遙控機器人收集資料	真實、高品質	極為昂貴（每筆需人力）
人類影片	從網路影片學習人的動作	量極大	沒有動作標註、形態不同
機器人自主探索	機器人自己嘗試（第 33.4.3 節）	真實、可持續	慢、且需要安全機制

目前的共識是**四者並用**：用模擬取得規模、用遙操作取得品質、用影片取得多樣性、用自主探索持續改善。

33.7 通往通用機器人的三條路徑

33.7.1 三種賭注

「通用機器人」——能像人一樣執行多樣任務的機器人——是這個領域的長期目標。目前有三條主要的路徑，代表三種不同的賭注：

表 33-9 通往通用機器人的三條路徑

路徑	核心主張	代表方向	主要風險
規模路線	「資料與算力夠大就會湧現」	VLA、基礎模型、大規模預訓練	資料瓶頸、物理精度不足
結構路線	「架構與模型才是關鍵」	分層架構、模型式控制加學習	設計複雜、泛化受限

路徑	核心主張	代表方向	主要風險
身體路線	「更好的身體勝過更好的演算法」	軟體機器人、形態計算、仿生設計	難以擴展到高階認知

33.7.2 三條路徑的檢視

規模路線是目前最受矚目的。它的論據是自然語言處理的成功先例——當模型與資料規模跨越某個門檻，能力會「湧現」。

但機器人有一個語言沒有的限制：**物理不能靠規模繞過**。無論模型多大，若機器人的手指精度只有 5 公釐，就無法完成需要 1 公釐容差的裝配（第 20.9 節）。這是硬體的限制，不是資料的限制。

結構路線主張精心設計的分層架構——正是本書第 32.8.4 節所呈現的。它的優勢是**每一層都可驗證**，且能結合各典範的長處。缺點是**設計的負擔重**，且模組之間的介面需要人為定義。

身體路線則主張：與其讓演算法變聰明，不如讓身體變得更適合任務。第 33.3 節的所有實例都支持這個方向——**一隻好的手，能讓平庸的演算法表現優異**。

◎ 一個歷史的教訓

這三條路徑的爭論，在機器人學史上已經出現過多次，而每一次的答案都是「都需要」。1980 年代有人相信只要專家系統夠完整就能通用；1990 年代 Brooks 主張丟掉世界模型；2010 年代深度學習興起時，也有人預測「端到端會取代一切」。每一次，單一路線都在某個範圍內取得成功，然後撞上自己的邊界。本書的立場是：**三條路徑處理的是不同的瓶頸**——規模解決「常識與泛化」、結構解決「保證與整合」、身體解決「接觸與效率」。一個真正通用的機器人，很可能三者都需要。

33.7.3 還缺什麼

誠實地說，目前的機器人距離「通用」仍有明顯的距離。四個最關鍵的缺口：

- **靈巧操作**：第 21.10.2 節的手內操作至今仍極為笨拙。人類能在手中轉筆、能單手打開瓶蓋——這些看似簡單的動作，機器人做不到。
- **長時序任務**：目前的系統能執行數十秒的任務，但「整理一個房間」需要數十分鐘與數百個步驟，且中間會不斷遇到意外。
- **真正的物理直覺**：第 33.4.3 節的知識框已指出——知道「玻璃會破」與知道「這個力道會不

會破」是兩回事。

- **可靠度**：實驗室的 90% 成功率，在真實部署中是不可接受的。**要達到工業級的 99.9%，往往比從 0% 到 90% 更難。**

最後一項特別值得強調。第 30、31 章反覆提到的「沒有保證」問題，在此成為部署的最大障礙。一台每一百次會失敗一次的家用機器人，若失敗意味著打翻熱湯——**它就無法上市。**

33.8 機器人與社會

33.8.1 就業：一個需要細緻討論的議題

「機器人會不會搶走工作」是最常被問到的問題，也是最容易被簡化的問題。

歷史的證據是混合的。過去兩百年的自動化並未造成長期的大規模失業——但它確實造成了劇烈的結構性轉移，且轉移的成本由特定的群體承擔。

表 33-10 自動化對工作的三種影響

影響	說明	機器人領域的例子
取代 (Substitution)	機器完全接手該工作	產線的重複性焊接、搬運
互補 (Complementation)	機器承擔部分，人的生產力提高	外科手術機器人、協作機器人
創造 (Creation)	產生原本不存在的工作	機器人維護、系統整合、資料標註

關鍵的判斷不是「會不會取代」，而是「哪些任務會被取代」。近年的研究指出，自動化影響的是**任務**而非**職業**——多數職業由多項任務構成，其中部分可被自動化、部分不能。

對機器人領域而言，一個具體的觀察是：

最容易被自動化的：重複、可預測、在結構化環境中的體力工作

最難被自動化的：需要靈巧操作、應變、或人際互動的工作

這正是 Moravec 悖論的社會版本——那些「低技能」的工作（如清潔、照護）反而最難自動化，因為它們需要的正是知覺與行動的能力。

33.8.2 自動化的悖論

有一個常被忽略的現象，稱為**自動化悖論** (Automation Paradox)：

◎ 自動化悖論

系統的自動化程度越高，剩下那少數需要人介入的時刻就越關鍵、也越困難。因為：一、人的技能

會退化（長期不操作就生疏）；二、需要介入的必定是異常情況（正常的都被自動化了）；三、人失去了情境覺察（不知道系統剛才在做什麼）。航空業對此有深刻的教訓——多起事故的原因是自動駕駛在異常時交還控制權，而飛行員無法在數秒內重建情境並正確反應。

這對機器人的設計有直接的啟示：

- 不要設計「平常完全自動、出事才叫人」的系統——那個「叫人」的瞬間往往來不及。
- 應保持人的參與度（第 32.9.3 節的人在迴路）——即使只是定期的確認。
- 介面必須快速地傳達情境——不只是「出錯了」，而是「剛才發生了什麼、現在是什麼狀態」。

33.8.3 責任歸屬

當機器人造成損害時，誰該負責？這個問題在法律上仍未有定論，但可以辨識出幾個關鍵的角色：

表 33-11 責任歸屬的可能對象

對象	何時應負責	困難之處
製造商	設計缺陷、未盡安全義務	如何界定「合理的安全」
軟體開發者	演算法錯誤、未做適當測試	學習式系統的行為無法完全預測
部署者 / 雇主	使用不當、未做風險評估	是否具備判斷的專業
操作者	誤用、忽視警告	自動化悖論下的技能退化
機器人本身	（目前法律上不成立）	無法律人格、無資產

最困難的是第二列。第 31.10.1 節指出，學習式系統「在訓練分布之外的行為不可預測」——那麼開發者對「無法預測的行為」該負多少責任？

目前的實務傾向是：要求可追溯性與盡職義務。開發者不需要保證「永不出錯」，但必須能證明：做了合理的測試、有適當的安全機制、且事故發生後能追溯原因。這正是第 29.9.3 節「每次測試都錄 rosbag」的深層理由——它不只是除錯工具，也是責任的證據。

33.9 倫理與風險

33.9.1 四類倫理風險

除了就業與責任，機器人還帶來四類需要正視的風險：

表 33-12 機器人的四類倫理風險

風險	具體表現	本書中的相關章節
安全	碰撞傷害、失控、系統故障	第 7、20、27、28 章
隱私	持續的影像與位置紀錄	第 22、25 章
偏見	對特定群體的辨識或服務品質較差	第 22、31 章
自主與尊嚴	照護機器人取代人際接觸、決策權轉移	第 1 章的自主性光譜

33.9.2 隱私：一個被低估的問題

一台家用機器人為了導航與操作，必須**持續地掃描家中的環境**（第 25 章的 SLAM）。這意味著：

- 它建立了你家的**完整三維地圖**——包含每個房間的格局與家具擺設。
- 它的相機**看見了一切**——包含你在做什麼、家裡有誰、桌上放著什麼。
- 這些資料若上傳雲端（第 32.10.1 節的雲端模型），就**離開了你的控制**。

這比手機更為敏感，因為機器人會**主動移動**——它會走到你沒預期的地方，且是**持續而非間歇地**記錄。

工程上的因應（呼應第 32.10.1 節）：

- **優先在本地處理**：把影像處理留在機器人上，只上傳必要的抽象結果（如「已完成任務」而非原始影像）。
- **明確的指示燈**：讓使用者知道相機何時在運作。
- **資料最小化**：只保留任務所需的資料，用完即刪。
- **可控的禁區**：允許使用者指定「不進入、不記錄」的區域。

33.9.3 偏見：不只是資料問題

第 31 章的學習方法會**繼承訓練資料中的偏見**。在機器人上，這有具體的後果：

- **辨識的不平等**：若訓練資料中某些膚色的樣本較少，人員偵測的準確率會較低——而這直接關係到安全（第 20.10 節的碰撞避免）。
- **互動的不平等**：語音辨識對某些口音的準確率較低，使部分使用者難以操作。
- **設計的不平等**：機器人的高度、力道、介面若以特定的身材或能力為基準設計，會排除其他

使用者。

最後一項尤其值得注意——它不是「演算法偏見」而是**設計偏見**，且往往在需求階段就已注定。若測試者都是實驗室的年輕研究生，就不會發現長者難以使用的問題。

33.9.4 一個工程師的檢核清單

面對這些問題，一個實務的作法是在專案中定期檢視以下問題：

1. 失效時會怎樣？最壞的情況是什麼？有沒有機制阻止它？（第 7.9、31.10.2、32.9 節）
2. 誰可能受影響？不只是使用者，還有旁觀者、被服務者、以及被取代的工作者。
3. 測試涵蓋了誰？測試的樣本是否涵蓋了實際使用者的多樣性？
4. 資料去了哪裡？收集了什麼、儲存多久、誰能存取？
5. 人能介入嗎？使用者能否理解系統在做什麼、並在必要時停止它？
6. 出事後能追溯嗎？是否記錄了足夠的資料以釐清原因？

這六個問題不需要哲學訓練就能回答，但認真回答它們，能避免絕大多數的倫理問題。

33.10 全書的主線回顧

33.10.1 六條貫穿全書的線索

在結束之前，讓我們回顧貫穿三十三章的六條主線。這些線索比任何單一的公式都更值得記住：

線索一：模型的價值不在完美，而在減負。第 13 章的動力學模型有 10~20% 的誤差，第 17 章仍能靠它把回授的負擔降低五倍；第 30 章更指出「模擬不必更真，策略必須更穩」。不完美的模型仍然極有價值。

線索二：分層與時間尺度分離。第 3 章的時間尺度金字塔、第 17 章的內外迴路、第 27 章的四層飛控、第 28 章的三層腿式控制、第 32 章的五層任務系統——**同一個原則反覆出現**：讓每一層只處理自己時間尺度上的問題，並把下層視為理想的執行器。

線索三：與其消除不確定性，不如設計對它不敏感的系統。第 20 章的柔順控制、第 21 章的軟性夾爪、第 24 章的機率估測、第 30 章的領域隨機化——**這是本書最重要的工程哲學**。

線索四：機構與控制必須協同設計。第 20 章的倒角把容差放寬 41 倍、第 21 章的欠驅動手

取代了感測與計算、第 28 章的被動行走不需要控制器——在把問題丟給控制器之前，先問能不能用機構把問題變簡單。這正是本章 33.3 節形態計算的統一解釋。

線索五：模型提供保證，學習提供泛化。第 21 章的抓取、第 25 章的 SLAM、第 28 章的運動控制、第 31 章的學習架構、第 32 章的任務規劃——每一次的結論都是「兩者分工」而非「誰取代誰」。

線索六：理論的各部分互相支撐。第 13 章那個「看似無用」的斜對稱性，在第 19 章成為證明的樞紐；第 11 章的雅可比對偶，先後成為第 17 章操作空間控制、第 20 章力映射、第 21 章抓取矩陣的基礎；第 18 章的 Bellman 方程，在第 31 章以強化學習的形式重現。這不是巧合，而是理論的內在結構。

33.10.2 三十三章的地圖

最後，用一張表回顧全書的架構：

表 33-13 全書架構總覽

篇	章	核心問題	關鍵洞見
導讀篇	1~2	機器人是什麼？該怎麼學？	Moravec 悖論、自主性光譜
第一篇	3~7	機器人由什麼構成？	時間尺度金字塔、機構與驅動的取捨
第二篇	8~12	它能到哪裡？	座標變換、逆解的多重性、奇異點
第三篇	13~21	該用多大的力氣？	模型加回授、最佳性與穩定性、柔順接觸
第四篇	22~28	它在哪裡？該往哪走？	機率估測、SLAM 的聯合求解、分層導航
第五篇	29~33	它知道自己在做什麼嗎？	模擬與學習、大型模型、具身智慧

注意每一篇的核心問題如何層層遞進——由「是什麼」到「能到哪」、由「怎麼動」到「在哪裡」、最終到「知道自己在做什麼」。

33.11 結語：給讀者的話

33.11.1 你學到了什麼

三十三章之後，你已經具備了完整的機器人學基礎：能建立運動學與動力學模型、能設計並證明控制器、能處理感知的不確定性、能規劃路徑與任務、能運用學習方法、也能整合為一個完整的系統。

但更重要的可能不是這些知識本身，而是三種判斷力：

- 知道何時該用哪個工具——模型或學習、精確或柔順、規劃或反應。
- 知道每個方法的邊界——它在什麼假設下成立、在什麼情況下會失效。
- 知道問題的本質——很多時候，正確的答案不是更好的演算法，而是重新定義問題。

33.11.2 這個領域還很年輕

讀完一本教科書容易產生一個錯覺：這個領域已經成熟了。但機器人學遠非如此。

回顧本書，有太多地方寫著「目前仍是研究前沿」：手內操作、長時序任務、變形物體、可靠度、以及本章討論的通用機器人。這些不是留給後人的習題，而是留給你的。

一個提醒：本書中那些看起來最成熟的部分（如第 9 章的運動學），在四十年前也是前沿；而那些現在最熱門的部分（如第 32 章的大型模型），十年後可能已被更好的方法取代。唯一不變的是那六條主線——它們是三十年後仍然適用的思維方式。

33.11.3 最後一段

第 1 章開頭，我們問：為什麼電腦下棋能贏世界冠軍，卻摺不好一件衣服？

現在你知道答案了：因為摺衣服需要的不只是計算，還需要一雙能感受布料的手、一個能與環境互動的身體、以及無數次「試了才知道」的經驗。

而這正是機器人學迷人的地方——它是唯一一個必須同時面對數學、物理、與真實世界的學科。你可以在紙上證明一個控制器穩定，但機器人會用它的行為告訴你，你的模型漏掉了什麼。

這種「現實會回應你」的特質，是機器人學最好的老師。

願你在往後的路，保持那份「想看看它到底會怎麼動」的好奇心。

那才是機器人學真正的起點。

綜合思考題

本章沒有計算例題，取而代之的是四個綜合性的思考題。它們沒有標準答案，但認真思考的過程，會把前面三十三章的知識連結起來。

思考題 33-1 形態計算的辨識與設計

題目：以下三個情境，各自可以用「加強演算法」或「改良機構」來解決。請為每一個提出兩種方案並比較。

情境一：一台移動機器人在磁磚地面上打滑，里程計誤差極大（第 23.8 節）。

參考分析：

- **演算法路線：**加裝 IMU 並用第 24 章的融合估測、或用第 25 章的 SLAM 提供絕對參照。優點是不需改硬體；缺點是增加了計算與感測成本，且打滑時的里程計資料本身就是錯的（垃圾進、垃圾出）。
- **機構路線：**換用摩擦係數更高的輪胎、增加配重以提高正向力、或改用第 23.6 節的履帶。優點是從源頭減少打滑；缺點是增加重量與能耗。
- **判斷：**兩者應並用。機構改善減少了誤差的來源，融合估測處理剩餘的誤差。單靠演算法無法補償嚴重的打滑（第 23.8.2 節的隨機誤差無法校正）。

情境二：機械手臂在插銷入孔時，因定位誤差而卡住（第 20.9 節）。

情境三：四足機器人在碎石地面上容易失去平衡（第 28 章）。

請自行分析情境二與三，並回答：在什麼情況下，機構路線的投資報酬率會高於演算法路線？

提示：考慮改變的頻率（機構固定、演算法可更新）、以及第 33.3.4 節的三個限制。

思考題 33-2 Moravec 悖論的當代檢驗

題目：2020 年之後，大型語言模型在許多「智力測驗」上超越了人類平均水準。請分析：

1. 這是否強化了 Moravec 悖論？（提示：抽象推理又進步了，而知覺行動仍然困難）
2. 第 32.7 節的 VLA 模型能由影像直接輸出動作。這是否代表悖論正在被打破？
3. 若要設計一個實驗來檢驗 Moravec 悖論是否仍然成立，你會怎麼設計？需要控制哪些變因？

一個參考的思考方向：注意 VLA 目前的成功案例多集中在「桌面上的簡單取放」——這與五歲孩子能做的事（摺衣服、綁鞋帶、在陌生房間移動）仍有明顯差距。差距在哪裡？是資料量、是身體、還是別的？

思考題 33-3 為一個真實場景做完整的系統設計

題目：為「醫院中運送檢體與藥品的服務機器人」做完整的系統設計。請涵蓋：

1. **技術架構**：依第 32.8.4 節的五層列出各層的技術選擇與理由。
2. **硬體選型**：移動平台（第 23 章）、感測器（第 6、22 章）、是否需要手臂（第 4、5 章）。
3. **安全設計**：三層防護（第 32.9 節）、失效行為（第 7.9 節）、與人共處的考量（第 20.10 節）。
4. **倫理檢核**：用 33.9.4 節的六個問題檢視你的設計。
5. **社會影響**：這台機器人會取代什麼工作？會創造什麼工作？醫護人員會如何看待它？

這個题目的價值在於「沒有純技術的答案」。一個技術上完美但護理人員不願使用的系統，是失敗的設計。

思考題 33-4 你的立場是什麼

題目：回答 33.7 節的三條路徑之爭——你認為通往通用機器人的關鍵是什麼？

請以本書的具體內容支持你的論點，並誠實地說明你的立場可能錯在哪裡。

一個提示：最好的答案往往不是「選一個」，而是能清楚地說明「在什麼條件下，哪一條路徑會勝出」。例如：若資料收集的成本大幅下降，規模路線的勝算會提高；若材料科學有突破（如更好的人工肌肉），身體路線的價值會上升。

實作：回顧、整合與展望

本章的「實作」不是寫程式，而是**回顧與整合**。以下九項活動，建議在課程結束時進行。

1. **繪製你的知識地圖**：不看書，憑記憶畫出全書三十三章的架構圖，標出各章之間的連結。畫完後再與表 33-13 對照，看看哪些連結你記得、哪些遺漏了。遺漏的部分正是需要複習的。
2. **找出五個形態計算的實例**：除了表 33-4 列出的，在你的生活或專案中再找出五個「用結構取代計算」的例子。可以是機器人的、生物的、或日常用品的。
3. **重讀第 1 章**：讀完全書後再讀一次第 1 章。你會發現當時看不懂或覺得空泛的段落，現在都有了具體的意義。記下你的感受變化。
4. **分析一個真實的機器人**：選一台你感興趣的機器人（掃地機器人、協作手臂、四足機器人、或自駕車），試著推測它用了本書的哪些技術。特別注意它「沒有做」的事——那往往反映了工程上的取捨。

5. **設計一個形態計算的解法**：選一個你曾經用演算法解決的問題，試著設計一個「用機構解決」的替代方案。比較兩者的成本、可靠度與彈性。
6. **做一次倫理檢核**：用 33.9.4 節的六個問題檢視你做過的任何一個專案（不限機器人）。誠實地回答，並記下你發現的問題。
7. **寫下三個你想解決的問題**：讀完全書後，有哪三個機器人問題是你最想解決的？為每一個寫下：為何重要、目前的困難、以及你會從哪裡開始。
8. **與人討論**：找一位非工程背景的朋友或家人，用五分鐘解釋「為什麼機器人摺衣服很難」。若你能講清楚，代表你真的懂了。
9. **（給教師）課程回顧討論**：以本章的四個思考題為基礎，安排一次沒有標準答案的討論課。重點不是得出結論，而是讓學生練習「用學過的知識支持自己的判斷」。

國中小也看得懂：聰明不只在腦袋裡

在這本書的第一頁，我們問過一個問題：為什麼電腦可以打敗世界棋王，卻不會摺衣服？

現在，讓我們來回答它。

先做一個實驗：閉上眼睛，用手在口袋或書包裡摸索，找出你的鑰匙。

你做到了對不對？很輕鬆，幾秒鐘就找到了。

但你有沒有想過，你剛才做了多少事情？你要分辨鑰匙和硬幣的形狀差別、要感覺金屬和塑膠的不同、要在摸到的時候調整手指的力道、還要判斷「這個形狀是不是我要的」——而這一切你完全不用思考，身體自己就會了。

這就是答案的第一部分：我們覺得簡單的事，其實超級複雜——只是我們的身體太熟練了，熟練到我們根本感覺不到。

答案的第二部分更有趣：因為你有一雙手。

想想看，你的手指是軟的、會變形、指腹有紋路增加摩擦、手掌可以彎起來包住東西。這些「身體的特性」本身就在幫你做事——你不需要用腦袋計算「該用幾牛頓的力」，因為軟軟的手指自己就會調整。

科學家發現了一件很重要的事：一隻蜜蜂的腦只有一百萬個腦細胞，比你的手機笨好幾億倍。

但牠可以在風中飛、可以找到花、還可以飛回家。為什麼？

因為牠的翅膀本身就會處理氣流，不需要用腦去算。

所以這本書最後想告訴你的是：

聰明，不只在腦袋裡，也在身體裡，還在身體跟世界互動的過程裡。

這就是為什麼，如果你想做出厲害的機器人，不能只會寫程式——你還要懂機構、懂材料、懂它會怎麼碰到這個世界。

最後，送你一句話：

這個世界上還有好多好多事情，是機器人不會做的。摺衣服、綁鞋帶、在陌生的房間裡走來走去不撞到東西——這些五歲小孩都會的事，全世界最厲害的科學家還在努力。

也許，解決其中一個人的人，就是正在讀這本書的你。

叫獸提醒與常見錯誤

- 誤區一：以為 Moravec 悖論只是「還沒做到」。它反映的是知識形式的根本差異——知覺行動的知識本質上難以符號化。
- 誤區二：把具身認知理解為「不需要計算」。它主張的是「計算、身體與環境應協同設計」，而非否定計算。
- 誤區三：忽略形態計算的可能性。在投入演算法之前，應先問「這個問題能不能用機構解決」。
- 誤區四：以為形態計算萬能。它不可重新編程、難以驗證、且設計困難，只適合穩定不變的需求。
- 誤區五：把感知視為被動的。主動感知（移動視角、輕推試探）常比改良演算法更有效。
- 誤區六：相信「新典範取代舊典範」的敘事。符號、連結、具身三者各有其層次，完整系統缺一不可。
- 誤區七：以為基礎模型會解決一切。它會改變上層的理解與規劃，但下層的控制與安全仍需傳統方法。
- 誤區八：低估資料瓶頸。機器人資料無法從網路取得，且具身的多樣性使遷移困難。

- 誤區九：把可靠度視為次要問題。從 90% 提升到 99.9% 往往比從 0% 到 90% 更難，卻是部署的門檻。
- 誤區十：設計「平常全自動、出事才叫人」的系統。自動化悖論指出，那個交還控制權的瞬間往往來不及。
- 誤區十一：忽略隱私問題。機器人會主動移動且持續記錄，其敏感度高於手機。
- 誤區十二：以為倫理是「別人的事」。33.9.4 節的六個問題不需要哲學訓練，但認真回答能避免多數問題。

課堂討論

1. 33.1.4 節主張知覺行動的困難源自「只複製了三分之一的系統」。請討論：若要「複製另外三分之二」，工程上具體該做什麼？
2. 表 33-4 列出了本書中八個形態計算的實例。請討論它們的共同結構，並嘗試提出一個判準——什麼樣的問題適合用形態計算解決？
3. 33.5.3 節主張三種典範各有其層次。請討論：有沒有可能出現第四種典範？它會處理什麼目前無法處理的問題？
4. 33.8.2 節的自動化悖論指出「自動化程度越高，人的介入越關鍵也越困難」。請以本書中的具體系統（如第 27 章的無人機、第 32 章的語言驅動系統）為例，設計一個能緩解這個悖論的人機介面。
5. 33.11.3 節說「現實會回應你，這是機器人學最好的老師」。請分享一個你在實作中「模型與現實不符」的經驗，以及你從中學到了什麼。

章末摘要

- Moravec 悖論有三層答案：演化的時間（知覺行動優化了 5 億年）、知識的形式（隱性且難以符號化）、以及身體的參與。
- 第三層是最深的：知覺與行動發生在「腦、身體、環境」的互動中，純計算只複製了三分之一的系統。
- 具身認知的四個支柱：身體塑造認知、環境是資源、行動構成感知、即時互動優於內部表示。

- 具身認知不否定計算，而是主張計算、身體與環境應協同設計。
- 形態計算是指由身體結構本身完成的功能——若換成剛性簡單的版本則需要感測與計算。
- 本書中的形態計算實例包含：被動行走、欠驅動手指、柔性指墊、RCC、倒角、串聯彈性驅動器、顆粒堵塞夾爪。
- 形態計算分三個層次：被動結構特性、機構的耦合、與環境的動態耦合。
- 形態計算的三個限制：不可重新編程、難以驗證、設計困難；因此應與控制協同設計。
- 感知不是被動接收而是一個迴路——行動改變感知、感知影響行動。
- 主動感知（移動視角、輕推試探）常比改良被動演算法更有效。
- 人工智慧的三種典範：符號主義（形式化與保證）、連結主義（泛化與感知）、具身觀點（身體參與智慧）。
- 三種典範各處理不同層次，完整的機器人系統三者缺一不可。
- 基礎模型會顯著改變機器人的上層（理解與規劃），但下層（控制與安全）仍需傳統方法。
- 機器人資料的四條路徑：模擬、遙操作示範、人類影片、自主探索；目前需四者並用。
- 通往通用機器人的三條路徑：規模、結構、身體；三者處理不同的瓶頸，很可能都需要。
- 目前的四個關鍵缺口：靈巧操作、長時序任務、物理直覺、以及可靠度。
- 自動化影響的是任務而非職業；最難自動化的反而是需要靈巧與應變的工作（Moravec 悖論的社會版本）。
- 自動化悖論：自動化程度越高，剩下需要人介入的時刻越關鍵也越困難。
- 責任歸屬的實務傾向是要求可追溯性與盡職義務，而非保證永不出錯。
- 四類倫理風險：安全、隱私、偏見、自主與尊嚴；六個檢核問題能避免多數問題。
- 全書的六條主線：模型的價值在減負、分層與時間尺度分離、設計對不確定性不敏感的系統、機構與控制協同設計、模型提供保證而學習提供泛化、理論各部分互相支撐。

觀念題與綜合題

觀念題

1. 陳述 Moravec 悖論，並說明其三層答案。
2. 說明演化的時間差如何解釋「我們覺得容易的事對電腦卻困難」。

3. 比較抽象推理與知覺行動兩種知識的形式，並說明這對編碼方式的影響。
4. 說明具身認知的核心主張及其對傳統認知科學的挑戰。
5. 列舉具身認知的四個支柱，並各舉一個本書中的例子。
6. 定義形態計算，並說明判斷的標準。
7. 由表 33-4 中選三個實例，說明「身體做了什麼」與「否則需要什麼」。
8. 說明形態計算的三個層次與三個限制。
9. 解釋為何說「感知是一個迴路而非單向的資訊流」，並舉三個人類的例子。
10. 比較被動感知與主動感知，並說明主動感知為何常更有效。
11. 說明具身探索的意義，以及它對第 32 章大型模型的提醒。
12. 比較人工智慧的三種典範，並說明各自的核心主張與困境。
13. 說明 Brooks 的「智慧不需要表示」主張及其實證支持。
14. 為何說「典範之爭是假問題」？請以本書的架構說明三者的分工。
15. 說明基礎模型對機器人的可能影響，並列舉支持與懷疑的論點各三個。
16. 列舉機器人資料的四條路徑及其優缺點。
17. 比較通往通用機器人的三條路徑，並說明各自的主要風險。
18. 列舉目前機器人的四個關鍵缺口，並說明為何可靠度特別困難。
19. 說明自動化對工作的三種影響，以及「最難自動化的是什麼工作」。
20. 說明自動化悖論及其對機器人介面設計的三個啟示。
21. 列舉責任歸屬的可能對象，並說明學習式系統帶來的特殊困難。
22. 列舉四類倫理風險，並說明隱私問題為何在機器人上比手機更敏感。
23. 列出 33.9.4 節的六個檢核問題。
24. 陳述全書的六條主線，並各舉一個具體的章節例證。

綜合題

1. **跨章整合：**以第 21 章的軟性夾爪為例，說明它如何同時體現：形態計算（33.3 節）、對不確定性不敏感的設計（第 20、30 章）、以及機構與控制的協同（第 5 章）。並討論若改用剛性夾爪，需要額外增加哪些感測與計算。

2. **方法論的比較**：對「讓機器人學會開門」這個任務，分別設計三種方案：（甲）純模型式（第 20、26 章）、（乙）純學習式（第 31 章）、（丙）混合式。比較三者所需的開發成本、可靠度、與泛化能力，並說明你會選哪一個及理由。
3. **架構設計**：為一台「協助長者的居家機器人」設計完整的五層架構（第 32.8.4 節）。除技術選擇外，請特別說明：如何處理自動化悖論（33.8.2 節）、如何保護隱私（33.9.2 節）、以及如何避免設計偏見（33.9.3 節）。
4. **前瞻分析**：選擇本書提到的一個「目前仍是研究前沿」的問題（手內操作、長時序任務、變形物體操作、或可靠度），分析：（a）目前的主要障礙為何；（b）三條路徑（規模、結構、身體）分別會如何嘗試解決；（c）你認為哪一條最有希望，以及需要什麼條件才能成功。

動手做與專題延伸

本章實作：整合與展望

本章的實作是**回顧與連結**，詳見前面的「實作」一節。此處提供三個延伸的方向。

- **方向一：形態計算的實驗**。設計一組對照實驗，量化「機構改良」與「演算法改良」的效益。例如：同一個抓取任務，分別測試（甲）剛性夾爪加精密視覺、（乙）柔性夾爪加粗略視覺。比較成功率、成本與開發時間。
- **方向二：主動感知的驗證**。設計一個實驗比較被動與主動感知。例如：判斷未知物體的重量——（甲）純視覺估計、（乙）輕輕提起試探。量化兩者的誤差差異。
- **方向三：完整系統的整合**。把前面各章的模組整合為一個完整的系統，並用 33.9.4 節的六個問題做倫理檢核。**這是本書所有內容的最終整合。**

專題延伸

- **形態計算的量化研究**：選擇一個具體的任務（建議：抓取形狀多變的物體）。設計並製作三種夾爪：剛性平行夾爪、加了柔性指墊的夾爪、以及欠驅動的自適應夾爪（第 21.8.2 節）。對同一組物體（至少 15 種形狀）各測試 20 次，記錄成功率、所需的定位精度、以及所需的感測與計算資源。繪出「機構複雜度 vs 演算法需求」的關係圖。這個專題能把本章最核心的主張變成一組具體的數據。
- **Moravec 悖論的當代測試**：設計一組任務，涵蓋由「純抽象推理」到「純知覺行動」的光譜

(例如：解數學題 → 下棋 → 辨識物體 → 抓取物體 → 摺衣服)。分別評估：目前的 AI 系統能達到什麼水準、五歲孩子能達到什麼水準。**繪出兩條曲線並找出交叉點**。討論這個交叉點在過去十年如何移動、以及未來可能如何移動。

- **進階：一個完整的畢業專題**。以本書的全部內容為基礎，設計並實作一個完整的機器人系統。建議的規格：(一) 能接受自然語言指令 (第 32 章)；(二) 能在未知環境中導航 (第 25、26 章)；(三) 能抓取與操作物體 (第 20、21 章)；(四) 具備三層安全防護 (第 32.9 節)；(五) 通過 33.9.4 節的倫理檢核。**這個專題會用到本書幾乎每一章的內容——而完成它，就是你真正掌握了機器人學的證明。**

參考文獻與延伸閱讀

- Moravec, H., Mind Children: The Future of Robot and Human Intelligence, Harvard University Press, 1988 (Moravec 悖論的出處)。
- Brooks, R. A., "Intelligence Without Representation," Artificial Intelligence, 1991 (具身觀點在機器人學中的宣言)。
- Brooks, R. A., "Elephants Don't Play Chess," Robotics and Autonomous Systems, 1990 (包容式架構與對符號主義的批判)。
- Pfeifer, R. and Bongard, J., How the Body Shapes the Way We Think: A New View of Intelligence, MIT Press, 2006 (具身智慧的權威著作，形態計算的系統性論述)。
- Clark, A., Being There: Putting Brain, Body, and World Together Again, MIT Press, 1997 (具身認知的哲學基礎)。
- Gibson, J. J., The Ecological Approach to Visual Perception, 1979 (可供性概念的原始出處)。
- Bajcsy, R., Aloimonos, Y. and Tsotsos, J. K., "Revisiting Active Perception," Autonomous Robots, 2018 (主動感知的回顧與展望)。
- Firoozi, R. et al., "Foundation Models in Robotics: Applications, Challenges, and the Future," IJRR, 2024 (基礎模型在機器人上的綜述)。
- Autor, D. H., "Why Are There Still So Many Jobs? The History and Future of Workplace Automation," Journal of Economic Perspectives, 2015 (自動化對就業影響的經濟學分析)。

- IEEE Global Initiative on Ethics of Autonomous and Intelligent Systems, Ethically Aligned Design (自主系統倫理設計的國際指引) 。
- Lin, P., Abney, K. and Bekey, G. A. (Eds.), Robot Ethics: The Ethical and Social Implications of Robotics, MIT Press (機器人倫理的論文集) 。
- Bicchi, A. and Tonietti, G., "Fast and Soft Arm Tactics: Dealing with the Safety-Performance Tradeoff," IEEE Robotics & Automation Magazine, 2004 (安全與性能取舍的經典分析，呼應本章的形態計算) 。

台灣自編大學教科書

[@prof](#)

機器人學導論

機構、感知、控制與具身人工智慧

第 六 篇

系統整合、產業應用與專題

第 34 章 系統整合與產業應用

第 35 章 法規、商業化與社會接受度

第 36 章 總整專題方法論

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

2026 年 9 月

第六篇 篇章導讀

前五篇教你**機器人學是什麼**，這一篇要教你**如何用它做出真正能運作的東西**。

這一篇的性質與前面五篇截然不同。前三十三章的內容是「知識」——運動學、動力學、控制、感知、規劃、學習，每一項都有明確的理論與可驗證的答案。但這三章談的是「判斷」：什麼題目值得做、什麼需求是真的、什麼風險該先驗證、什麼時候該用機構取代演算法、以及——什麼時候該誠實地說「這件事我做不到」。

第 34 章（系統整合與產業應用）從一個殘酷的統計數字開始：機器人專案的失敗原因中，技術只佔一成。需求不明佔 35%、環境假設錯誤佔 25%。一個在實驗室成功率 95% 的系統，到工廠第一天就停了七次——而那七次的原因，沒有一個是機器人學的問題。這一章教的，是把技術變成一個能在產線上跑三年的系統所需要的方法論。

第 35 章（法規、商業化與社會接受度）處理另一道門檻：當機器人走出工廠、走進有人的地方。餐廳老闆的一個問題讓整個團隊愣住——「如果它燙傷客人，是誰要賠？」這個問題的答案不在程式裡，而在法律、保險與社會共識裡。這一章談認證、責任、隱私、商業模式，以及一個常被忽略的事實：使用者信任的不是最聰明的機器人，而是最可預測的機器人。

第 36 章（總整專題方法論）是全書的最後一章，也是所有內容的最終整合。它從兩組學生的對照開始：一組做了五種功能但每種都不能用，另一組只做一件事卻測了 1,260 次並分析了每一次失敗。這一章教的是如何選題、如何界定範圍、如何規劃、如何測試、如何從失敗中提取知識、以及如何讓別人理解你的價值。

這一篇要回答的六個問題

- 實驗室 95% 的系統，為什麼在工廠第一天就停了七次？
- 客戶說「我要一台機器手臂」，他真正需要的是什麼？
- 當誤差無法降低時，還有什麼辦法？
- 如果它燙傷客人，是誰要賠？
- 做出原型之後，距離「可以上市」還有多遠？
- 一個專題的價值，究竟從哪裡來？

三條貫穿全篇的線索

線索一：先讓環境變得可控，再讓演算法變得聰明。這是第 34 章最核心的原則，也是第 33 章形態計算在產線層級的延伸。一個定位治具能讓視覺定位完全省略；一個成本兩萬的倒角，勝過花兩百萬提升精度。在把問題丟給演算法之前，先問能不能把問題本身變簡單。

線索二：盡早失敗，而且要失敗得便宜。第 34 章的 POC、第 35 章的場域試驗、第 36 章的最小可行系統——三者是同一個思維：**與其把系統做完美再驗證，不如盡早用最粗糙的版本暴露最大的風險。**因為在第二階段發現不可行，損失是三週；拖到現場驗收才發現，損失是九個月與客戶的信任。

線索三：證明比嘗試更有價值。這是第 36 章的核心，也貫穿了整個第六篇。一個能明確說出「在什麼條件下、測了幾次、成功率多少、失敗的原因是什麼」的系統，遠勝於一個宣稱「功能很多」卻無法量化的系統。而一個被充分理解的失敗，比一個沒被理解的成功更有價值。

讀完這一篇之後

你將能夠：運用系統工程的方法把技術變成可運作的系統、以五問法釐清真實需求、用誤差預算與經濟評估做技術決策、規劃產品的認證與上市路徑、設計符合社會期待的人機互動、以及——**獨立完成一個有價值的專題並讓別人理解它。**

但更重要的是三種判斷力：知道何時該用哪個工具、知道每個方法的邊界、以及知道問題的本質。因為很多時候，正確的答案不是更好的演算法，而是重新定義問題。

第 36 章的最後留下了一句話，也是這本書的最後一句話：**它會動，而那是你讓它動的。現在，換你了。**

本篇章節配置

章次	章名	核心內容	叫獸開講
第 34 章	系統整合與產業應用	V 模型、需求釐清、介面與治具、誤差預算、OEE、投資評估、安全法規	「實驗室 95%，為什麼工廠第一天就停了七次？」——技術只佔三成
第 35 章	法規、商業化與社會接受度	認證體系、產品責任、個資與隱私、RaaS、TRL 與鴻溝、人機互動設計	「如果它燙傷客人，是誰要賠？」——技術可行不等於可以上市
第 36 章	總整專題方法論	選題、範圍界定、倒推規劃、最小可行系統、量化測試、失敗分析、發表	「做到一件事的 95%，勝過五件事的 60%」——證明比嘗試更有價值

第六篇 系統整合、產業應用與專題

第 34 章

系統整合與產業應用

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩格對照：上格是明亮整潔的實驗室，機械手臂穩定地抓取乾淨的樣品，旁註「實驗室成功率 95%」；下格是真實的工廠現場——黃色照明、輸送帶持續移動、零件沾著切削油，旁註「現場第一天停機七次」，並以五個小標籤標出失敗原因（輸送帶不停、燈色不同、班別光線差異、油污、震動）。右半為 V 模型的圖示：左臂由上而下標「使用者需求 → 系統規格 → 子系統規格 → 元件設計」，右臂由下而上標「元件測試 → 子系統測試 → 系統測試 → 現場驗收」，兩臂之間以虛線橫向連接對應的層級。下緣文字：技術只佔三成，另外七成是把問題想清楚。

叫獸開講 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？

一家做汽車零件的工廠找上叫獸，希望導入機器人取代人工的檢測與包裝。他們的需求聽起來很單純：「一台手臂，把零件從輸送帶拿起來，檢查有沒有瑕疵，好的放 A 箱、壞的放 B 箱。」

小翰很有信心：「這不就是第 21 章的抓取加第 22 章的視覺辨識嗎？我一個學期就能做出來。」

三個月後，小翰帶著做好的系統去現場測試。實驗室裡成功率 95% 的系統，在工廠裡第一天就停了七次。

叫獸帶著全班去看。「說說看，發生了什麼事。」

小翰翻著筆記：「第一，輸送帶不會停——我以為零件會靜止，結果它一直在動。第二，工廠的燈是黃光，我在實驗室用白光校正的相機，顏色全跑掉了。第三，早班和晚班的光線不一樣。第四，零件上有切削油，跟我測試用的乾淨樣品完全不同。第五……」

「停。」叫獸打斷他，「這五個問題，有哪一個是**機器人學**的問題？」

小翰愣住了。小妤說：「好像……都不是。都是環境的問題。」

「這就是今天的第一課。」叫獸說，「一個**機器人系統**的失敗，通常不是因為演算法不夠好，而是因為對環境的假設錯了。你們前面三十三章學的東西，在真實的工廠裡只佔工作量的三成。」

他在白板上寫了一個數字：「另外七成是什麼？釐清需求、界定範圍、設計治具、規劃動線、訂定驗收標準、教育操作人員、處理異常、以及回答『這台機器壞了誰來修』。」

小翰不服氣：「那前面學的不就沒用了？」

「恰恰相反。」叫獸搖頭，「正因為你懂那三成，你才知道哪些需求做得到、哪些做不到、以及該用什麼治具把不可能變成可能。第 20 章那個倒角記得嗎？一個成本幾乎為零的機構，把容差放寬了 41 倍。這種判斷，只有懂技術的人做得出來。」

最後他補了一句：「今天要教的，是如何把知識變成一個真的能在工廠裡跑三年的系統。這需要的不只是技術，還有一種思維方式——**系統工程**。」

本章為什麼重要

前面三十三章教的是**技術**。但技術本身不會自動變成一個能用的系統——這中間隔著一個常被低估的鴻溝，稱為**系統整合**。

這個鴻溝的具體表現是：

- **實驗室 95%，現場 60%**：環境的差異遠比想像中大（這正是第 30 章 Sim-to-Real 落差的產業版本）。
- **技術可行 ≠ 商業可行**：一個技術上完美但投資回收要八年的方案，不會有人買單。
- **做得出來 ≠ 用得下去**：若操作員不會用、維修人員不敢修、主管看不懂報表，系統就會被閒置。

本章要建立的是一套「把技術變成系統」的方法論。它涵蓋：如何釐清真正的需求（而非客戶說出口的需求）、如何界定範圍以避免失控、如何設計整合的架構、如何評估經濟效益、如何規劃安全與驗收、以及如何處理部署後的維運。

這些內容在多數機器人教科書中缺席，卻是**業界最看重的能力**。一個能寫出 SLAM 演算法的工程師很有價值；但一個能判斷「這個案子該不該接、該怎麼報價、風險在哪裡」的工程師，**價值更高且更稀缺**。

學習目標

- 能說明系統工程的 V 模型並運用於機器人專案。
- 能區分表面需求與真實需求，並運用五問法釐清。
- 能撰寫可驗證的需求規格並定義驗收標準。
- 能說明範圍潛變的成因與三種控制方法。
- 能設計機器人工作站的整合架構並辨識關鍵介面。
- 能說明治具與周邊設計如何降低技術難度。
- 能計算投資回收期並評估專案的經濟可行性。
- 能運用風險評估方法辨識並降低專案風險。
- 能說明機器人安全標準的體系與風險評估流程。
- 能規劃系統的驗收測試與交付流程。
- 能說明部署後的維運需求與常見的失效模式。

- 能為一個真實的產業需求提出完整的系統提案。

先備知識

本章需要的是**前面三十三章的整體理解**，而非特定的章節。特別會用到：第 7 章的安全標準、第 21 章的抓取、第 22 章的視覺、第 29 章的軟體架構、第 30 章的 Sim-to-Real 落差、第 33 章的倫理與社會考量。

本章沒有數學推導，但有大量的**工程判斷**。建議讀法：把每一節的內容對照自己做過的專題，思考「當時若知道這些，會怎麼做得不一樣」。

核心名詞中英對照

表 34-1 本章核心名詞

中文	英文	說明
系統整合	System Integration	把各子系統組合為可運作整體的工程
系統工程	Systems Engineering	從需求到交付的整體工程方法論
V 模型	V-Model	需求分解與驗證對應的開發流程
需求規格	Requirements Specification	明確定義系統應達成什麼
範圍潛變	Scope Creep	專案範圍不受控地擴大
工作單元	Work Cell	機器人與周邊構成的作業單位
治具	Fixture / Jig	固定或定位工件的裝置
節拍時間	Cycle Time	完成一個作業循環所需的時間
稼動率	Availability / Uptime	設備實際可運作時間的比例
投資回收期	Payback Period	累積效益抵過投資所需的時間
整體設備效率	OEE	稼動率×性能×良率的綜合指標
風險評估	Risk Assessment	辨識危害並評估其嚴重度與機率
安全等級	Performance Level, PL	安全功能可靠度的分級
驗收測試	Acceptance Test	確認系統符合規格的正式測試
現場驗收	Site Acceptance Test, SAT	在客戶現場進行的驗收
平均故障間隔	MTBF	兩次故障之間的平均時間
平均修復時間	MTTR	從故障到恢復的平均時間
預防保養	Preventive Maintenance	定期執行以避免故障的保養

34.1 系統工程的思維

34.1.1 為何需要方法論

一個機器人專案的失敗，很少是因為「某個演算法不夠好」。統計上，失敗的原因依序是：

表 34-2 機器人專案失敗的主要原因

原因	佔比 (概略)	說明
需求不明確或改變	~35%	客戶說不清楚要什麼，或做到一半改需求
對環境的假設錯誤	~25%	現場的光線、震動、髒污與實驗室不同
整合的介面問題	~20%	各子系統單獨都對，接起來就出錯
技術本身的困難	~10%	真正的演算法或硬體限制
人的因素	~10%	操作員不會用、維修人員不支持

注意「技術本身」只佔一成。這不是說技術不重要——而是說其他九成若沒處理好，技術再好也沒用。

系統工程 (Systems Engineering) 就是處理這九成的方法論。

34.1.2 V 模型

系統工程最經典的框架是 **V 模型**。它的形狀說明了一切：左邊是逐層分解、右邊是逐層驗證，且每一層的分解都對應一層的驗證。

左側 (分解)：使用者需求 → 系統規格 → 子系統規格 → 元件設計

右側 (驗證)：元件測試 → 子系統測試 → 系統測試 → 使用者驗收

每一層的規格，都必須在對應的層級被驗證。

表 34-3 V 模型在機器人專案中的對應

階段	左側：定義什麼	右側：驗證什麼	本書相關章節
使用者需求	「我要自動化檢測與包裝」	現場驗收：實際生產三天	34.2 節
系統規格	節拍 8 秒、良率 99.5%、稼動率 95%	系統測試：連續運行測試	34.2、34.9 節
子系統規格	視覺辨識率 99.8%、抓取成功率 99.9%	子系統測試：各自的指標測試	第 21、22 章
元件設計	相機型號、夾爪設計、控制器程式	元件測試：單元測試	第 5、6、29 章

V 模型最重要的洞見是「驗證的層級對應」。一個常見的錯誤是：只做元件測試（「我的視覺辨識率 99.8%」），卻沒做系統測試（「整條線的良率是多少」）。**兩者可能天差地別——因為誤差會在系統層級累積（34.5 節）。**

34.1.3 迭代與 V 模型的結合

V 模型看起來是瀑布式的（一路往下再一路往上），這在軟體界已被批評多年。但在機器人專案中，它仍然有價值，只是需要迭代地執行：

大 V：整體專案的一次完整流程（數月）

小 v：每個子系統的快速迭代（數天到數週）

用小 v 快速試錯，用大 V 確保整體不偏離。

具體的作法是：先用第 30 章的模擬快速跑幾輪小 v（驗證概念可行）、再進入實體的原型（驗證關鍵風險）、最後才是完整的大 V（正式開發與驗收）。這正是 34.4 節「先做概念驗證」的理由。

34.2 需求：最關鍵也最容易出錯的一步

34.2.1 客戶說的不是他要的

開頭故事中，客戶的需求是「把零件拿起來、檢查、分類」。這聽起來很明確——但它其實幾乎什麼都沒說。

沒說清楚的至少有十件事：

- 零件的**種類有幾種**？未來會不會增加？
- 輸送帶會不會停？**節拍時間**是多少？
- 「瑕疵」的定義是什麼？**多小的刮痕算瑕疵**？
- 誤判的代價：把好的判成壞的（浪費）vs 把壞的判成好的（客訴）——**哪個嚴重**？
- 要求的**良率與稼動率**是多少？
- 現場的**空間**有多大？電力、氣源、網路呢？
- **誰來操作**？他們的技術水準如何？
- **壞了誰來修**？有沒有備品？
- **預算與投資回收期**的要求？

- 未來的擴充：這條線會不會改產品？

這十個問題若沒問清楚，專案幾乎必定出問題。而客戶通常不會主動說——因為他們不知道這些會影響什麼。

34.2.2 五問法：挖到真實的需求

一個實用的技巧是反覆追問「為什麼」，直到找出真正的目的。

◎ 五問法的實例

客戶說：「我要一台機器手臂。」問一：為什麼需要手臂？——「因為要把零件搬到檢測站。」問二：為什麼要搬到檢測站？——「因為檢測設備在那裡。」問三：為什麼檢測設備不能移到產線上？——「呃……好像可以，只是從來沒想過。」問四：那真正的目的是什麼？——「減少人力、提高檢測的一致性。」問五：現在的人力成本與誤判率是多少？——「兩班共四人，誤判率約 3%。」結論：真正的需求是「降低檢測的人力與誤判率」，而不是「一台機器手臂」。也許把檢測設備移到線上、加裝一套視覺系統，成本只有手臂方案的三分之一。問對問題，往往比解對問題更有價值。

這個技巧的價值在於：它能找出「更便宜的解法」。而在產業中，便宜且夠用的方案，永遠勝過昂貴而完美的方案。

34.2.3 可驗證的需求規格

需求必須寫成可驗證的形式。比較兩種寫法：

表 34-4 需求的兩種寫法

模糊的寫法（不可驗證）	明確的寫法（可驗證）
「系統要快」	節拍時間 ≤ 8.0 秒/件（連續 100 件的平均）
「辨識要準」	瑕疵漏檢率 $\leq 0.1\%$ 、誤檢率 $\leq 2\%$ （以 500 件標準樣本測試）
「要穩定」	稼動率 $\geq 95\%$ （連續 30 天，扣除計畫性保養）
「要好操作」	操作員經 2 小時訓練後能獨立完成換線與異常排除
「要安全」	符合 ISO 10218 與 ISO/TS 15066，經風險評估達 PL d

右欄的每一條都能用一個明確的測試來判定通過與否。這不只是文書工作——它是驗收時避免爭議的唯一保障。

◎ 一個慘痛的教訓

一個真實的案例：某團隊交付了一套視覺檢測系統，合約寫「辨識準確率 99%」。驗收時客戶用自己現場的樣本測試，準確率只有 92%。爭議點在於：團隊用的是乾淨的標準樣本，客戶用的是沾了油、有輕微變形的實際件。合約沒有定義「用什麼樣本測試」。最終雙方各退一步，團隊免費追加了三個月的工作。教訓是：需求規格中必須明確定義「測試的條件與樣本」——這比數字本身更重要。

34.2.4 範圍潛變

範圍潛變 (Scope Creep) 是專案殺手：需求在過程中不斷增加，但時程與預算沒有跟著調整。

它的成因通常是善意的：客戶看到成果後想到更多可能、工程師想把事情做得更完美、業務為了成交而承諾額外的功能。

三種控制方法：

- **明確的範圍文件**：白紙黑字寫下「本專案包含什麼、不包含什麼」。後者尤其重要——明確寫出「不含：多品項切換、不含產線連動、不含遠端監控」。
- **變更管理流程**：任何新增的需求都要重新評估時程與費用，並經雙方書面同意。這不是刁難，而是保護雙方。
- **分階段交付**：把「一定要有」與「有更好」分開，先交付核心功能。這也降低了雙方的風險。

34.3 整合的架構

34.3.1 工作單元的組成

一個機器人工作站 (work cell) 遠不只是「一台機器人」。以開頭故事的檢測包裝站為例，完整的組成包含：

表 34-5 典型機器人工作單元的組成

類別	元件	常被低估之處
機器人本體	手臂、控制器、示教器	—
末端工具	夾爪、快換裝置、氣壓管路	線材與氣管的走線常造成干涉
感測	相機、光源、力覺、觸發訊號	光源比相機更關鍵 (34.3.3 節)
治具與周邊	定位治具、輸送帶、料倉、擋停器	這部分常佔一半以上的工時

類別	元件	常被低估之處
安全裝置	圍籬、光柵、安全門、急停	必須通過風險評估 (34.7 節)
控制與整合	PLC、工業電腦、網路、HMI	與既有產線的訊號整合最易出錯
公用設施	電力、氣源、網路、照明	現場常不足，需另行拉線

第四列與第六列是新手最容易低估的。在實務上，「機器人本體與程式」往往只佔專案工時的 30%，而治具設計與訊號整合各佔 25% 以上。

34.3.2 介面：最容易出問題的地方

34.1.1 節指出「整合的介面問題」佔失敗原因的 20%。原因是：各子系統單獨測試都正常，但接起來就出錯。

四類常見的介面問題：

- **時序 (Timing)**：訊號來得太早或太晚。例如：相機在零件還沒定位就拍照、或機器人在治具還沒夾緊就開始動作。**這是最常見的一類。**
- **資料格式**：座標系定義不同 (第 8 章)、單位不同 (公釐 vs 公尺)、旋轉表示不同 (歐拉角的順序，第 8.3 節)。
- **握手協定**：誰先動、誰等誰、失敗時如何回報。**若沒有明確定義，就會出現死鎖** (兩邊都在等對方)。
- **異常處理**：其中一方出錯時，另一方怎麼知道、怎麼反應。**這一項最常被完全忽略**——大家都只設計了「正常流程」。

◎ 介面規格書：一份被低估的文件

在多方合作的專案中，介面規格書 (Interface Control Document) 比任何技術文件都重要。它應該明確定義：訊號的名稱與意義、電氣規格 (電壓、極性)、時序圖 (誰在什麼時候發什麼)、資料格式與單位、握手的流程、以及**異常時的行為**。花兩天寫這份文件，能省下兩週的除錯——因為多數整合問題的根源是「雙方對介面的理解不同」，而這在文件中會立刻暴露。這也呼應了第 29 章 ROS 2 的核心價值：標準化的介面。

34.3.3 周邊設計：把難題變簡單

這是本章與前面各章最重要的連結。一個好的周邊設計，能把技術上極困難的問題變得平凡。

表 34-6 周邊設計降低技術難度的實例

困難的技術問題	周邊設計的解法	效果	本書呼應
零件位置不確定 → 需要高精度視覺	加一個定位治具	位置變成已知，視覺可省略	第 20.9.3 節的倒角
零件會滑動 → 需要力控制	加一個擋停器與夾具	位置固定，只需位置控制	第 20 章
光線變化 → 辨識不穩	加一個遮光罩與固定光源	環境變成可控的	第 22 章
零件形狀多變 → 抓取困難	加一個整列機構	姿態統一，抓取變簡單	第 21 章
節拍太緊 → 機器人太慢	改為雙工位輪流	有效節拍減半	—

這張表的每一列，都是「用機構解決演算法問題」的實例——正是第 33.3 節形態計算思維在產線層級的延伸。

一個實務的原則：

先問「能不能把環境變得可控」，再問「能不能用演算法處理不可控的環境」

前者通常便宜十倍、可靠十倍。

但也要注意其限制：治具是為特定工件設計的，換產品就要重做。因此在多品項、少量生產的場合，反而應該用彈性較高的視覺方案。這個取捨的判斷，正是系統整合工程師的核心價值。

34.4 專案的推進流程

34.4.1 七個階段

一個典型的機器人整合專案，包含七個階段：

表 34-7 機器人整合專案的七個階段

階段	主要工作	產出	典型佔比
一、需求訪談	釐清真實需求、現場勘查	需求規格書	5%
二、概念驗證 (POC)	驗證關鍵風險是否可行	可行性報告	10%
三、方案設計	架構、選型、佈局、報價	系統設計書、報價單	10%
四、詳細設計與製造	機構設計、治具製作、程式開發	圖面、程式、治具	35%
五、廠內組裝與測試	組裝、調試、內部測試 (FAT)	測試報告	20%

階段	主要工作	產出	典型佔比
六、現場安裝與驗收	運送、安裝、調機、驗收 (SAT)	驗收報告	15%
七、教育訓練與保固	操作訓練、文件交付、保固維護	手冊、訓練紀錄	5%

34.4.2 概念驗證：最划算的投資

第二階段的 POC (Proof of Concept) 是整個流程中投資報酬率最高的一步。它的目的不是做出完整的系統，而是驗證那個「若不成立則整個專案失敗」的關鍵假設。

如何找出關鍵假設？問這個問題：「這個專案最可能失敗的地方是什麼？」

- 若是「不確定視覺能不能辨識出那種瑕疵」→ POC 就做視覺測試，用現場的真實樣本。
- 若是「不確定節拍趕不趕得上」→ POC 就做一次動作的時間量測。
- 若是「不確定夾爪抓不抓得住」→ POC 就做抓取測試。

POC 的三個原則：

- 只驗證最關鍵的一兩件事，不要做成迷你版的完整系統。
- 必須用現場的真實條件——真實的樣本、真實的光線、真實的節拍。這正是開頭故事失敗的教訓。
- 時間要短（一至三週）。POC 拖太久，就失去了「快速止損」的意義。

◎ POC 的價值：早失敗比晚失敗好一百倍

假設一個專案在第二階段 (POC) 發現不可行，損失是十萬元與三週；若拖到第六階段 (現場驗收) 才發現，損失可能是三百萬元、九個月、以及客戶的信任。POC 的價值不在於「證明可行」，而在於「盡早發現不可行」。因此一個好的 POC 應該主動去測那些最可能失敗的情況，而非挑最容易的來展示。這與第 30.9 節的漸進式驗證是同一種思維——每一階段隔離不同的風險，讓失敗盡早且廉價地發生。

34.4.3 FAT 與 SAT

驗收分為兩個階段，這個區分極為重要：

FAT (Factory Acceptance Test，廠內驗收)：在整合商的工廠進行。優點是環境可控、問題容易修改。應盡可能在此階段解決所有問題。

SAT (Site Acceptance Test，現場驗收)：在客戶的實際產線進行。這裡會暴露所有「環

境假設」的問題——真實的光線、震動、髒污、以及與既有設備的互動。

經驗法則：能在 FAT 解決的問題，成本是 SAT 的十分之一

因此 FAT 應該盡可能模擬真實環境——用客戶提供的真實樣本、模擬現場的光線條件。

一個實務的建議：在 FAT 時請客戶到場參與。這有三個好處：他們能提早發現期待的落差、能參與問題的討論、且在 SAT 時不會有「這跟我想的不一樣」的爭議。

34.5 誤差預算與可靠度

34.5.1 誤差會累積

一個常見的錯誤是：各子系統單獨測試都達標，但整合後卻不達標。原因是誤差會在系統層級累積。

以「把零件插入孔中」為例，最終的位置誤差來自多個來源：

表 34-8 誤差來源的累積 (插件任務)

誤差來源	典型值	說明	本書章節
機器人重複精度	± 0.05 mm	規格書上的數字	第 9.9.2 節
機器人絕對精度	± 0.5 mm	與重複精度差十倍	第 9.9.2 節
視覺定位誤差	± 0.3 mm	含相機校正與辨識誤差	第 22.3 節
手眼校正誤差	± 0.2 mm	常被忽略	第 22.6 節
治具定位誤差	± 0.1 mm	治具本身的公差	34.3.3 節
工件本身公差	± 0.15 mm	來料的變異	—
夾持的偏移	± 0.2 mm	每次夾持的位置不完全相同	第 21 章
熱變形	± 0.1 mm	運轉數小時後的漂移	第 9 章

若把這些誤差直接相加 (最壞情況)：

$$\text{總誤差} = 0.5 + 0.3 + 0.2 + 0.1 + 0.15 + 0.2 + 0.1 = 1.55 \text{ mm}$$

遠超過多數插件任務所需的容差 (典型 $0.1 \sim 0.3$ mm)。

若假設誤差獨立且用均方根相加 (統計上較合理)：

$$\text{總誤差} = \sqrt{(0.5^2 + 0.3^2 + 0.2^2 + 0.1^2 + 0.15^2 + 0.2^2 + 0.1^2)} = 0.69 \text{ mm}$$

仍然過大。

這說明了一個重要的事實：即使每個環節都「很準」，整合後也可能完全不可行。

34.5.2 誤差預算

誤差預算 (Error Budget) 是系統設計的核心工具。它的作法是：

1. 由需求反推：任務的容差是多少？（如插件需要 $\pm 0.2\text{ mm}$ ）
2. 分配給各個環節：每個來源可以佔多少？
3. 檢查是否可行：若總和超過容差，就必須改變設計。
4. 改變設計：或提升某個環節的精度、或改變架構使某個誤差源消失。

第四步的「或」極為關鍵。以本例而言，有三種改善的方向：

表 34-9 三種改善誤差的策略

策略	作法	效果	成本
提升精度	換更精密的機器人、更好的相機	各項誤差減半	極高
消除誤差源	用治具取代視覺定位（34.3.3 節）	視覺 0.3 + 手眼 0.2 直接歸零	低
放寬容差	在孔口加倒角（第 20.9.3 節）	容差由 0.2 放寬到 1.0 mm	極低

第三列是最漂亮的解法——它不改善任何誤差，而是改變了問題本身。這正是第 20.9.3 節那個「一個倒角把容差放寬 41 倍」的實例，在系統層級的重現。

◎ 誤差預算的思維價值

做誤差預算最大的價值，不是算出那個數字，而是「知道該把資源投在哪裡」。若視覺誤差是 0.3 mm 而機器人絕對精度是 0.5 mm，那麼把相機從 500 萬畫素升級到 1200 萬畫素毫無意義——因為瓶頸不在那裡。這與第 30.4.2 節「哪些落差重要」、第 33 章「先做敏感度分析」是完全相同的思維：找出瓶頸，把資源投在那裡。

34.5.3 可靠度與稼動率

除了精度，可靠度同樣關鍵。三個核心指標：

$\text{MTBF (平均故障間隔)} = \text{總運轉時間} / \text{故障次數}$

$\text{MTTR (平均修復時間)} = \text{總修復時間} / \text{故障次數}$

$\text{稼動率} = \text{MTBF} / (\text{MTBF} + \text{MTTR})$

稼動率是客戶最在意的數字。

一個重要的觀察：提升稼動率有兩條路——增加 MTBF（更可靠）或降低 MTTR（更好修）。而後者往往便宜得多：

- 模組化設計：故障時整組更換，而非現場維修。MTTR 由數小時降到十分鐘。
- 良好的診斷：明確的錯誤訊息與指示燈，讓操作員能自行排除簡單的異常。
- 備品策略：關鍵易損件（夾爪指片、感測器、氣管）現場備有存貨。
- 遠端診斷：工程師能遠端查看日誌與狀態，不需到場。

這也是第 29.9.3 節「每次都錄 rosbag」的產業版本——沒有紀錄，遠端診斷就無從談起。

34.5.4 整體設備效率 (OEE)

製造業衡量設備表現的標準指標是 OEE：

$$\text{OEE} = \text{稼動率} \times \text{性能效率} \times \text{良率}$$

三者相乘——任何一項低落都會嚴重拉低整體。

這個相乘的結構有重要的意義。假設三項都是 90%（聽起來都不錯）：

$$\text{OEE} = 0.90 \times 0.90 \times 0.90 = 0.729 = 72.9\%$$

三個「不錯」相乘之後只有七成。

而製造業的標竿水準是 $\text{OEE} \geq 85\%$ ，這要求三項都在 95% 左右。這說明了為何工業級的要求如此嚴苛——也呼應了第 33.7.3 節「從 90% 到 99.9% 比從 0% 到 90% 更難」的觀察。

34.6 經濟評估

34.6.1 為何工程師必須懂

一個技術上完美的方案，若投資回收要八年，**不會有人買單**。因此經濟評估不是業務的事，而是工程師必須具備的判斷力——因為它會影響技術方案的選擇。

34.6.2 投資回收期

最常用也最直觀的指標是投資回收期 (Payback Period)：

$$\text{投資回收期} = \text{總投資額} / \text{年度效益}$$

製造業的常見要求是 2~3 年以內；若超過 5 年，多數企業不會投資。

總投資額包含的項目常被低估：

表 34-10 自動化專案的成本項目

項目	說明	常被低估的程度
機器人本體	手臂與控制器	低 (有明確報價)
周邊與治具	夾爪、治具、輸送帶、料倉	高 (常佔總額的 40%)
視覺與感測	相機、光源、鏡頭	中
安全設備	圍籬、光柵、安全繼電器	高 (法規要求，不可省)
系統整合工時	設計、程式、調試	極高 (常佔 30~50%)
安裝與試車	運送、安裝、現場調機	中
產線停機損失	安裝期間無法生產	極高 (常被完全忽略)
教育訓練	操作與維護人員的訓練	低

倒數第二列特別值得注意。若一條產線每天產值一百萬，安裝停機五天就是五百萬的損失——這可能比機器人本身還貴。因此實務上常採「分段導入」或「假日安裝」以降低這項成本。

34.6.3 年度效益的計算

效益不只是「省下的人力」，還包含四類：

- **人力節省**：最直接的一項。但要注意——通常不是「省下 N 個人」，而是「這 N 個人去做別的事」。若公司不裁員，效益要以「產能提升」而非「薪資節省」計算。
- **品質提升**：不良率降低所減少的報廢與客訴成本。這一項常被低估，但在高價值產品上可能是最大的效益。
- **產能提升**：機器人可以連續運轉 (不需休息、可上夜班)，使產能提升。
- **間接效益**：職業傷害減少、人力招募壓力降低、交期穩定度提升。這些難以量化但真實存在。

◎ 一個常見的誤區：只算人力

許多提案只計算「省下兩個人 × 年薪 60 萬 = 120 萬」。但這往往低估了真實的效益。以開頭故事的檢測站為例：人工檢測的誤判率 3%，若每件產品價值 500 元、月產 10 萬件，則誤判造成的損失為 $10 \text{ 萬} \times 3\% \times 500 = 150 \text{ 萬/月}$ 。若自動化能把誤判率降到 0.5%，每月省下 125 萬——遠超過人力節省的效益。因此提案時務必把品質效益納入計算，這往往是說服決策者的關鍵。

34.6.4 不只看回收期

投資回收期簡單直觀，但有三個限制：

- **忽略回收後的效益**：一台能用八年的設備，回收期兩年，但後面六年都是淨賺。
- **忽略資金的時間價值**：今天的一百萬比五年後的一百萬值錢。
- **忽略風險**：兩個回收期都是三年的方案，風險可能完全不同。

因此財務上還會用淨現值（NPV）與內部報酬率（IRR）。但對工程師而言，理解回收期已足以做技術判斷——關鍵是知道它會影響方案的選擇。

一個實用的判斷：若你的方案回收期是四年而客戶要求兩年，不要只想著降低報價——應該回頭檢視：能不能用更簡單的方案（34.2.2 節的五問法）？能不能把效益算得更完整（34.6.3 節）？能不能分階段導入以降低初期投資？

34.7 安全與法規

34.7.1 安全是法律義務

第 7 章介紹過機器人的安全概念，本節從**產業實務**的角度補充。最重要的認識是：

◎ 安全不是「做好一點」，而是法律義務

在多數國家，工業機器人的安全防護是強制性的法規要求，而非「可以斟酌的品質項目」。若因未達安全標準而發生職災，雇主與設備供應商都可能負法律責任（呼應第 33.8.3 節的責任歸屬）。因此安全的預算**不可刪減**——這是報價時必須向客戶明確說明的。

34.7.2 標準體系

機器人安全的標準是分層的，理解這個結構有助於查找正確的規範：

表 34-11 機器人安全標準的體系

層級	標準	涵蓋範圍
A 類（基礎）	ISO 12100	機械安全的通則、風險評估的方法
B 類（群組）	ISO 13849、IEC 62061	安全控制系統的設計與可靠度（PL/SIL）
	ISO 13855	安全距離的計算（光柵該裝多遠）
C 類（產品）	ISO 10218-1	機器人本體的安全要求
	ISO 10218-2	機器人系統與整合的安全要求

層級	標準	涵蓋範圍
	ISO/TS 15066	協作機器人的力與壓力限制 (第 20.10 節)

C 類標準優先於 A、B 類——若 ISO 10218 有明確規定，就依它；沒規定的部分才回到通則。

34.7.3 風險評估的流程

風險評估 (Risk Assessment) 是安全設計的核心程序，依 ISO 12100 分為五步：

1. 界定範圍：機器的使用限制、預期用途、以及可預見的誤用。
2. 辨識危害：列出所有可能的危害——夾擠、撞擊、切割、電擊、噪音、以及維修時的危害 (常被忽略)。
3. 風險估計：對每個危害評估「傷害的嚴重度 × 暴露頻率 × 迴避的可能性」。
4. 風險評價：判斷風險是否可接受。
5. 風險減低：若不可接受，依三階法採取措施 (見下)。

第五步的「三階法」有明確的優先順序，不可跳過：

第一階：本質安全設計 (消除危害本身)
第二階：工程防護 (圍籬、光柵、安全控制)
第三階：資訊警告 (標示、手冊、訓練)

必須依序考慮——不能因為「貼個警告標示比較便宜」就跳過前兩階。

第一階最常被忽略但最有效。例如：把夾點的間隙加大到手指伸不進去、把移動速度限制在安全範圍、或把危險的動作區域設計成人不會進入的位置。這正是第 7.9 節「本質安全」的體現，也與第 33.3 節的形態計算是同一種思維——用設計消除問題，而非用防護遮蔽問題。

34.7.4 協作機器人的特殊考量

協作機器人 (第 20.10 節) 常被誤解為「不需要安全防護」。這是危險的誤解。

正確的認識是：協作機器人允許在特定條件下與人共處，但仍必須經過風險評估。ISO/TS 15066 定義了四種協作模式：

表 34-12 ISO/TS 15066 的四種協作模式

模式	作法	適用
安全級監控停止	人進入時機器人停止	偶爾需要人介入的作業
手動導引	人握著機器人移動它	示教、輔助搬運
速度與間距監控	依人的距離動態調整速度	人機頻繁交錯的作業
功率與力限制	限制接觸力不致傷害	真正的並肩協作（第 20.5 節）

最後一種最常被稱為「協作」，但它的限制也最嚴格——必須經過實測驗證接觸力與壓力不超過標準值（依身體部位而異，如手部 140 N、臉部禁止接觸）。且一旦裝上尖銳的工具或抓取重物，原本的安全評估就失效了，必須重新評估。

34.8 產業應用的樣貌

34.8.1 四大應用領域

機器人的產業應用，可依「環境的結構化程度」與「任務的變異程度」分為四個象限：

表 34-13 機器人的四大應用領域

領域	環境	任務變異	代表應用	關鍵技術
傳統工業	高度結構化	低（大量重複）	焊接、噴塗、搬運、組裝	精度、速度、可靠度
彈性製造	結構化	高（多品項少量）	檢測、包裝、混線生產	視覺、快速換線
物流倉儲	半結構化	中（品項多但動作單純）	分揀、揀貨、搬運	抓取泛化、導航
服務與戶外	非結構化	高	清潔、巡檢、農業、醫療	感知、自主、安全

這四個象限的技術重點截然不同，也對應了本書不同章節的內容：

- 傳統工業依賴的是第 9~19 章的精確運動學與控制——環境已知，重點是快與準。
- 彈性製造加入了第 22 章的視覺——因為工件位置與種類會變。
- 物流倉儲需要第 21 章的泛化抓取與第 26 章的導航——品項無法窮舉。
- 服務與戶外則需要第 24~33 章的全部——不確定性、自主決策、與人共處。

34.8.2 技術成熟度的落差

一個誠實的觀察：四個領域的技術成熟度差距極大。

表 34-14 各領域的成熟度與挑戰

領域	成熟度	典型稼動率	主要挑戰
傳統工業	極成熟（數十年）	> 98%	已非技術問題，而是成本與彈性
彈性製造	成熟中	90~97%	換線時間、多品項的視覺穩定度
物流倉儲	快速發展	85~95%	抓取泛化（品項無法窮舉）
服務與戶外	early stage	70~90%	可靠度、成本、法規、社會接受度

最後一系列的「可靠度」是關鍵瓶頸。這正是第 33.7.3 節的觀察——實驗室的 90% 在真實部署中不可接受。一台每十次會失敗一次的服務機器人，會產生大量的人工介入需求，反而增加成本。

34.8.3 台灣的產業脈絡

對台灣的讀者而言，有幾個特殊的脈絡值得認識：

- **中小企業為主**：台灣製造業以中小企業居多，**多品項、少量、快速換線**是常態。這使得「大批量的專用自動化」不適用，反而需要**彈性製造**的方案。
- **缺工問題嚴重**：這使自動化的驅動力來自「**找不到人**」而非「省人力成本」——這改變了經濟評估的邏輯（34.6.3 節）。
- **產業聚落完整**：機構加工、電控、模具的供應鏈健全，**治具與周邊的製作成本相對低**。這使 34.3.3 節「用機構解決問題」的策略特別有利。
- **半導體與電子業的高精度需求**：這個領域對精度、潔淨度與稼動率的要求極高，是台灣機器人應用最成熟的場域。

一個實務的建議：台灣的機器人工程師若要創造價值，「**彈性製造**」與「**快速換線**」往往比「**追求極致精度**」更有市場——因為前者切中中小企業的痛點，而後者已有國際大廠的成熟方案。

34.9 部署與維運

34.9.1 交付不是結束

一個常見的錯誤觀念是：**驗收通過就等於專案完成**。實際上，系統要在產線上運作三到五年，維運的總成本往往超過建置成本。

交付時必須包含的四樣東西：

- **操作手冊**：不是技術文件，而是**操作員看得懂的圖文步驟**——如何開機、換線、排除常見異常。
- **維護手冊**：保養項目與週期、易損件清單與料號、故障排除的決策樹。
- **教育訓練**：且必須包含**實作**。只做簡報的訓練，一個月後就忘光了。
- **備品清單**：哪些零件應該現場備有存貨（34.5.3 節）。

34.9.2 常見的失效模式

實務上，機器人系統在部署後最常見的問題**不是程式錯誤**，而是以下六類：

表 34-15 部署後的常見失效模式

失效	成因	預防
夾爪指片磨損	長期夾持造成尺寸變化	定期更換（易損件）、監測夾持力
視覺辨識漂移	光源老化、鏡頭積灰、環境光變化	定期校正、光源亮度監測、遮光
治具鬆動或磨損	長期震動、碰撞	定期檢查與校正、加防鬆設計
氣壓不足	廠務氣源波動、管路洩漏	加裝壓力監測與警報
來料變異	供應商換料或製程變動	來料檢查、擴大系統的容忍範圍
人為變更	有人移動了相機或治具	明確的標示與固定、變更管理

第二列與第五列最值得警惕——它們是**緩慢漸進**的，不會立刻造成停機，而是使良率逐漸下降。等到有人注意到時，可能已經生產了大量的不良品。

因應之道是「**監測趨勢而非只看當下**」：記錄每天的辨識信心分數、夾持力、節拍時間，繪出**趨勢圖**。當某個指標開始漂移時就介入——這正是第 30.10 節數位學生的預測性維護思想。

34.9.3 持續改善

一個系統上線後，通常還有 10~30% 的改善空間。三個常見的著力點：

- **節拍優化**：分析動作的時間分布，找出瓶頸。常見的改善是**動作的重疊**（如相機拍照的同時機器人已開始移動）與**軌跡的平滑化**（第 15 章）。
- **參數微調**：依實際的資料調整視覺的門檻、力控制的增益、加減速的參數。
- **異常處理的完善**：上線後會遇到當初沒想到的異常。**每一次異常都應該被記錄並納入處理邏輯**。

◎ 為何「上線後才是開始」

一個系統的價值，不在於交付那一天有多完美，而在於它三年後還在穩定運轉。這需要三件事：一、可監測（能看見狀態的趨勢）；二、可維護（模組化、有備品、有文件）；三、可調整（參數外部化，第 29.5.1 節）。這三件事都必須在設計階段就考慮——若等到上線後才想加，成本會高出數倍。這也是為何本章要放在全書的最後：只有理解了完整的技術，才能在設計時做出這些前瞻的判斷。

34.10 一個完整的案例

34.10.1 情境

讓我們用開頭故事的檢測包裝站，走一遍完整的流程。

客戶的原始需求：「一台手臂，把零件從輸送帶拿起來，檢查有沒有瑕疵，好的放 A 箱、壞的放 B 箱。」

34.10.2 需求釐清（34.2 節）

經過訪談與五問法，釐清出的真實需求為：

表 34-16 釐清後的需求規格

項目	規格	驗證方式
零件種類	3 種，尺寸 30~80 mm，未來可能增至 5 種	—
節拍時間	≤ 6.0 秒/件	連續 200 件的平均
瑕疵定義	刮痕 > 0.5 mm、缺料、變形 > 1 mm	以標準樣本判定
漏檢率	$\leq 0.1\%$ （壞判成好，客訴風險）	500 件含 50 件已知不良的測試
誤檢率	$\leq 3\%$ （好判成壞，浪費）	同上
稼動率	$\geq 95\%$	連續 30 天運轉紀錄
投資回收期	≤ 2.5 年	財務試算
安全	符合 ISO 10218-2，達 PL d	風險評估報告

注意「漏檢率」與「誤檢率」的不對稱——前者要求嚴格十倍，因為客訴的代價遠高於浪費。這個判斷必須由客戶提供，工程師不能自行假設。

34.10.3 關鍵風險與 POC（34.4.2 節）

最可能失敗的地方是什麼？團隊判斷是「視覺能否在現場的光線與油污下，穩定辨識 0.5

mm 的刮痕」。

POC 的設計：

1. 向客戶索取 100 件真實樣本（含 20 件已知不良，且不清洗）。
2. 在客戶現場架設相機與光源測試（而非在實驗室）。
3. 分別在早班與晚班各測一次，觀察環境光的影響。
4. 為期兩週，產出可行性報告。

POC 的結果：在加裝遮光罩與同軸光源後，漏檢率 0.08%、誤檢率 2.1%——達標，可以進入正式開發。若沒做這一步而直接開發，很可能在 SAT 才發現不可行。

34.10.4 架構設計（34.3 節）

依 34.3.3 節「先讓環境可控」的原則，方案設計為：

表 34-17 系統架構與設計決策

問題	決策	理由
零件在輸送帶上移動	加擋停器 + 定位治具	位置變已知，視覺只需檢測不需定位
環境光變化	遮光罩 + 同軸光源	環境變可控（POC 已驗證）
3 種零件尺寸不同	可調式治具 + 自適應夾爪	換線只需調治具，不需換夾爪
節拍 6 秒偏緊	檢測與搬運平行	拍照的同時機器人移向下一件
未來可能增至 5 種	視覺模型可線上新增類別	預留擴充性（避免範圍潛變）
安全	圍籬 + 安全門開關 + 光柵	風險評估後達 PL d

注意前三列都是「用機構解決」——這正是 34.3.3 節與第 33.3 節形態計算的實踐。

34.10.5 誤差預算（34.5 節）

抓取的容差需求為 ± 1.5 mm（夾爪有 V 型槽導引）。誤差來源：

機器人絕對精度 ± 0.5 、治具定位 ± 0.3 、工件公差 ± 0.2 、夾持偏移 ± 0.2

$$\text{均方根總和} = \sqrt{(0.5^2 + 0.3^2 + 0.2^2 + 0.2^2)} = 0.66 \text{ mm}$$

$$0.66 < 1.5, \text{ 餘裕充足 } \checkmark$$

注意視覺的誤差沒有出現在這裡——因為治具已經固定了位置，視覺只負責「判斷好壞」而不負責「定位」。這正是 34.5.2 節「消除誤差源」的策略。

34.10.6 經濟評估 (34.6 節)

投資成本：

機器人與控制器 85 萬、視覺系統 45 萬、治具與周邊 60 萬
安全設備 25 萬、系統整合工時 95 萬、安裝試車 20 萬
總計 = 330 萬元

注意整合工時佔 29%、治具佔 18%——符合 34.6.2 節的觀察。

年度效益：

人力節省：2 人 × 2 班 × 60 萬 = 240 萬/年
品質效益：誤判率由 3% 降至 0.5%
月產 10 萬件 × 2.5% × 500 元 = 125 萬/月 = 1500 萬/年

品質效益遠大於人力效益 (34.6.3 節的知識框)。

保守起見，品質效益只計入四成 (因為部分不良品原本就會在下游被攔截)：

年度效益 = 240 + 1500 × 0.4 = 840 萬/年
投資回收期 = 330 / 840 = 0.39 年 ≈ 4.7 個月

遠優於 2.5 年的要求 ✓

這個案例說明了 34.6.3 節的重點：若只計算人力節省 (回收期 $330/240 = 1.4$ 年)，雖然也達標，但說服力遠不如把品質效益納入。

34.11 由整合邁向專題

34.11.1 本章的成果

本章建立了「把技術變成系統」的方法論。你現在能夠：運用 V 模型規劃專案、以五問法釐清真實需求、撰寫可驗證的規格、設計整合架構並辨識介面風險、用周邊設計降低技術難度、計算誤差預算與經濟效益、規劃安全與驗收、以及設計可維運的系統。

最重要的是三個判斷：

- 先問「能不能把環境變得可控」，再問「能不能用演算法處理不可控的環境」——前者通常便宜十倍、可靠十倍。
- 找出瓶頸，把資源投在那裡——誤差預算與敏感度分析的核心價值。
- 盡早失敗——POC 的價值不在證明可行，而在盡早發現不可行。

34.11.2 接下來

第 35 章將處理本章未涵蓋的一個面向：當機器人走出工廠、進入公共空間與家庭時，會遇到什麼技術之外的問題——法規、標準、認證、商業模式、以及社會的接受度。第 33 章已從哲學與倫理的角度談過，第 35 章則從**產業實務**的角度補完。

第 36 章則是全書的最終整合——**完整的專題方法論**。它會帶你從「選題」開始，經過規劃、開發、測試、到成果發表，並特別著重**失敗原因的分析與設計改良**。若說第 34 章教的是「產業專案怎麼做」，第 36 章教的就是「**你自己的專題怎麼做**」。

換句話說：第 34 章讓你知道業界如何把技術變成系統，第 36 章讓你親手走一遍。

完整計算例題

例題 34-1 誤差預算與方案選擇

題目：某插件任務要求容差 ± 0.25 mm。目前的誤差來源為：機器人絕對精度 ± 0.5 mm、視覺定位 ± 0.35 mm、手眼校正 ± 0.2 mm、工件公差 ± 0.15 mm、夾持偏移 ± 0.2 mm。(a) 計算最壞情況與均方根的總誤差。(b) 判斷是否可行。(c) 提出三種改善方案並比較。(d) 選擇最佳方案並計算改善後的誤差。

解 (a)：兩種計算方式：

$$\text{最壞情況 (直接相加)} = 0.5 + 0.35 + 0.2 + 0.15 + 0.2 = 1.40 \text{ mm}$$

保守的估計。

$$\begin{aligned} \text{均方根} &= \sqrt{(0.5^2 + 0.35^2 + 0.2^2 + 0.15^2 + 0.2^2)} \\ &= \sqrt{(0.25 + 0.1225 + 0.04 + 0.0225 + 0.04)} = \sqrt{0.475} = 0.689 \text{ mm} \end{aligned}$$

統計上較合理的估計。

解 (b)：即使用較樂觀的均方根計算，0.689 mm 仍是容差 0.25 mm 的 2.8 倍——完全不可行。

解 (c)：三種方案：

表 34-18 三種改善方案的比較

方案	作法	改善後誤差	成本	彈性
甲：提	換高精度機器人 (± 0.1)	$\sqrt{(0.1^2 + 0.15^2 + 0.2^2 + 0.15^2 + 0.2^2)} =$	極高	高

方案	作法	改善後誤差	成本	彈性
升精度	+ 更好的視覺 (± 0.15)	0.35 mm	(+200 萬)	
乙：消除誤差源	加定位治具，取消視覺定位	$\sqrt{(0.5^2+0.1^2+0.15^2+0.2^2)} = 0.58$ mm	低 (+15 萬)	低 (換件要改治具)
丙：放寬容差	在孔口加 1 mm 倒角 (第 20.9.3 節)	誤差不變，但容差變 1.25 mm	極低 (+2 萬)	高

分析：

- 方案甲改善到 0.35 mm，仍然超過 0.25 mm——花了兩百萬還是不行。
- 方案乙改善有限 (因為機器人的 0.5 mm 仍在)，也不行。
- 方案丙誤差不變，但容差由 0.25 放寬到 1.25 mm，而總誤差 $0.689 < 1.25$ ✓ 可行。

解 (d)：選擇方案丙 (加倒角)。改善後：

誤差 0.689 mm < 容差 1.25 mm，餘裕 45%

成本僅 2 萬元，遠低於方案甲的 200 萬。

核心的工程判讀：這個例題最重要的教訓是——當誤差無法降低時，就改變容差。方案甲與乙都在「努力降低誤差」，而方案丙改變了問題本身。這正是第 20.9.3 節、第 33.3 節形態計算的思維：在把問題丟給精度之前，先問能不能用機構讓問題消失。

但也要注意其前提：倒角必須是客戶的工件設計允許的。若工件已量產且不能修改，方案丙就不成立——此時可能要考慮在夾爪端加 V 型導引，達到同樣的效果。

例題 34-2 OEE 與稼動率的分析

題目：某產線的班別為 8 小時，其中計畫性停機 (保養、換線) 40 分鐘。實際運轉時因故障停機 25 分鐘。理論節拍 5.0 秒/件，實際共生產 4,800 件，其中不良 96 件。(a) 計算稼動率。(b) 計算性能效率。(c) 計算良率。(d) 計算 OEE 並判斷水準。(e) 若要提升 OEE 至 85%，分析應優先改善哪一項。

解 (a)：稼動率 = 實際運轉時間 / 計畫運轉時間：

計畫運轉時間 = 480 - 40 = 440 分鐘

$$\text{實際運轉時間} = 440 - 25 = 415 \text{ 分鐘}$$

$$\text{稼動率} = 415/440 = 94.3\%$$

故障停機造成 5.7% 的損失。

解 (b)：性能效率 = 理論所需時間 / 實際運轉時間：

$$\text{理論所需時間} = 4,800 \text{ 件} \times 5.0 \text{ 秒} = 24,000 \text{ 秒} = 400 \text{ 分鐘}$$

$$\text{性能效率} = 400/415 = 96.4\%$$

有 3.6% 的微停與速度損失 (如短暫的卡料、節拍略慢於理論值)。

解 (c)：良率 = 良品數 / 總生產數：

$$\text{良率} = (4,800 - 96)/4,800 = 4,704/4,800 = 98.0\%$$

不良率 2%。

解 (d)：

$$\text{OEE} = 0.943 \times 0.964 \times 0.980 = 0.891 = 89.1\%$$

超過 85% 的標竿水準 ✓

判讀：三項都在 94% 以上，相乘後 89.1%——這印證了 34.5.4 節的觀察：相乘的結構使得「每一項都要很高」。

解 (e)：目前已達 89.1%，但若要進一步提升，應分析各項的改善空間：

表 34-19 各項的改善潛力分析

項目	目前	損失	改善難度	優先度
稼動率	94.3%	5.7% (故障 25 分)	中 (需分析故障原因)	高
性能效率	96.4%	3.6% (微停與速度)	中 (需優化節拍)	中
良率	98.0%	2.0% (96 件不良)	高 (涉及製程)	低

建議優先改善稼動率，理由有三：

- 損失最大 (5.7%)。
- 故障停機通常有明確的原因，分析 25 分鐘的停機記錄，往往能找出一兩個反覆出現的問題 (如 34.9.2 節的夾爪磨損或氣壓不足)。
- 改善成本低：多數是保養與備品的問題，而非需要重新設計。

若能把故障停機由 25 分降到 10 分：

$$\text{新稼動率} = 430/440 = 97.7\%$$

$$\text{新 OEE} = 0.977 \times 0.964 \times 0.980 = 0.923 = 92.3\%$$

提升 3.2 個百分點。

例題 34-3 投資評估與方案決策

題目：某工廠考慮自動化一個包裝工位。現況為 3 班制、每班 2 人，人力成本每人每年 55 萬。三個方案的資料如下表。（a）計算各方案的投資回收期。（b）若要求回收期 ≤ 2.5 年，判斷哪些可行。（c）考慮五年的總效益，比較三者。（d）綜合判斷應選哪一個。

表 34-20 三個方案的比較

方案	投資額	取代人力	額外效益	預期壽命
甲：半自動	180 萬	3 人（每班 1 人）	無	8 年
乙：全自動（標準）	420 萬	6 人（全部）	品質效益 60 萬/年	8 年
丙：全自動（高規）	680 萬	6 人（全部）	品質效益 60 萬/年 + 產能提升 150 萬/年	10 年

解（a）：先計算各方案的年度效益：

$$\text{甲：} 3 \times 55 = 165 \text{ 萬/年}$$

$$\text{乙：} 6 \times 55 + 60 = 390 \text{ 萬/年}$$

$$\text{丙：} 6 \times 55 + 60 + 150 = 540 \text{ 萬/年}$$

年度效益。

$$\text{甲：} 180/165 = 1.09 \text{ 年}$$

$$\text{乙：} 420/390 = 1.08 \text{ 年}$$

$$\text{丙：} 680/540 = 1.26 \text{ 年}$$

投資回收期。

解（b）：三個方案的回收期都在 1.1~1.3 年，全部遠優於 2.5 年的要求 ✓

這是一個常見的情況——當人力成本高（本例 3 班 6 人，年成本 330 萬）時，多數自動化方案都能快速回收。此時回收期就不是決策的關鍵，必須看其他因素。

解 (c)：五年的累積淨效益 (效益 $\times$ 年數 - 投資)：

$$\text{甲：} 165 \times 5 - 180 = 825 - 180 = 645 \text{ 萬}$$

$$\text{乙：} 390 \times 5 - 420 = 1,950 - 420 = 1,530 \text{ 萬}$$

$$\text{丙：} 540 \times 5 - 680 = 2,700 - 680 = 2,020 \text{ 萬}$$

丙的累積效益最高。

解 (d)：綜合判斷應選方案丙，但需檢視三個前提：

- 「產能提升 150 萬」是否真的能實現？若市場需求不足，多出來的產能賣不掉，這 150 萬就不存在——此時丙的效益降為 390 萬/年，與乙相同，但投資多了 260 萬，反而較差。
- 6 個人去哪裡？若公司不裁員而是轉調，則「人力節省」應改以「產能提升」計算 (34.6.3 節)。
- 丙的技術風險是否較高？高規方案通常較複雜，稼動率可能較低。

這個例題的核心教訓：回收期只是門檻，不是決策依據。當多個方案都通過門檻時，應該比較累積效益、並檢視效益假設是否成立。而最後一點——質疑自己的假設——往往是工程師最欠缺卻最重要的能力。

實作：一個完整的系統提案

本章的實作是撰寫一份完整的系統提案。這是業界最常見的工作產出，也是把本章知識整合的最佳方式。

1. **選定題目**：選一個你熟悉的重複性工作作為自動化的對象。可以是校內的 (如實驗室的樣品分類)、家庭的 (如衣物分類)、或產業的 (若有實習經驗)。選一個你能實際觀察的。
2. **現場觀察**：實際去看那個工作至少三次，記錄：動作的步驟與時間、環境的條件 (光線、空間、干擾)、異常發生的頻率與處理方式。這一步不可省略——它是所有需求的來源。
3. **需求訪談**：訪談實際執行者與其主管。用 34.2.2 節的五問法，找出「真實的需求」而非「表面的要求」。記錄兩者的差異。
4. **撰寫需求規格**：依 34.2.3 節的原則，寫出至少 8 條可驗證的需求。每一條都要有明確的數值與測試方式。
5. **界定範圍**：明確寫出「本專案包含什麼、不包含什麼」 (34.2.4 節)。
6. **辨識關鍵風險**：這個專案最可能失敗的地方是什麼？設計一個 POC 來驗證它 (34.4.2

節)。

7. **架構設計**：繪製系統的組成圖 (34.3.1 節)，標出所有的元件與介面。特別標出你認為最容易出問題的介面。
8. **周邊設計**：用 34.3.3 節的思維，找出至少三個「可以用機構解決的技術問題」。
9. **誤差預算**：若涉及精度，做一次誤差預算 (34.5 節)，判斷是否可行。
10. **經濟評估**：計算投資額 (含 34.6.2 節的所有項目) 與年度效益 (含 34.6.3 節的四類)，求出投資回收期。
11. **安全評估**：依 34.7.3 節的流程做一次簡化的風險評估，列出至少五項危害與其防護措施。
12. **撰寫提案書**：整合以上內容為一份 10~15 頁的提案書，包含：現況分析、需求規格、方案架構、實施計畫、經濟效益、風險與對策。
13. **(進階) 向他人簡報**：找一位不熟悉技術的人 (如家人或其他系的同學) 聽你的提案。若他能理解你要解決什麼問題、以及為什麼值得投資，代表你的提案是成功的。

表 34-21 系統提案的自我檢核表

檢核項目	是否完成	備註
需求都是可驗證的 (有數值與測試方式)	☐	34.2.3 節
明確寫出了「不包含什麼」	☐	34.2.4 節
辨識出關鍵風險並設計了 POC	☐	34.4.2 節
至少找出三個「用機構解決」的機會	☐	34.3.3 節
投資額包含了整合工時與停機損失	☐	34.6.2 節
效益包含了品質與產能 (非只有人力)	☐	34.6.3 節
做了風險評估並列出防護措施	☐	34.7.3 節
考慮了維運 (備品、訓練、文件)	☐	34.9 節

國中小也看得懂：先想清楚要做什麼

假設你想做一台「自動整理書包」的機器人。你會怎麼開始？

大部分的人會直接想：「要用什麼手臂？要怎麼寫程式？」

但真正厲害的工程師會先問一堆問題：

「書包裡有幾種東西？」（課本、鉛筆盒、水壺.....如果有一百種，就很難做）

「要多快整理完？」（一分鐘還是十分鐘？差很多！）

「整理錯了會怎樣？」（把課本放錯格子沒關係，但把水壺放倒會濕掉全部）

「誰來用它？」（如果是小朋友自己用，操作就不能太複雜）

「壞掉了誰修？」（如果只有你會修，那你不在的時候怎麼辦？）

有沒有發現？這些問題都跟「怎麼寫程式」沒關係，但它們才是最重要的。

這一章要告訴你一個很重要的祕密：

在真實的世界裡做一台機器人，「技術」只佔三成，另外七成是「想清楚要做什麼」。

而且還有一個超級好用的技巧：與其把機器人做得更聰明，不如把環境變得更簡單。

舉個例子：如果書包裡的東西亂七八糟，機器人要很聰明才能認出來。但如果你在書包裡加幾個隔層，讓每樣東西都有固定的位置——**機器人就變得超簡單了！**

這就像考試一樣：與其把題目背得滾瓜爛熟，不如先看清楚題目在問什麼。**很多時候，把問題想清楚，答案就自己出現了。**

所以下次你想做什麼東西的時候，先別急著動手。**先問自己五個「為什麼」**，你會發現真正要解決的問題，常常跟你一開始想的不一樣。

叫獸提醒與常見錯誤

- 誤區一：以為專案的困難在技術。統計上技術只佔失敗原因的一成，需求不明與環境假設錯誤才是主因。
- 誤區二：接受客戶說出口的需求就開始做。應用五問法追問真實的目的——往往有更便宜的解法。
- 誤區三：需求寫成「要快」「要準」這類無法驗證的文字。必須有明確的數值、測試條件與樣本定義。
- 誤區四：沒有明確寫出「不包含什麼」。這是防止範圍潛變最有效的一句話。
- 誤區五：跳過 POC 直接開發。早失敗比晚失敗便宜一百倍，且 POC 必須用現場的真實條

件。

- 誤區六：只做元件測試而不做系統測試。誤差會在系統層級累積，各項達標不代表整體達標。
- 誤區七：先想演算法再想機構。應先問「能不能讓環境變得可控」——通常便宜十倍、可靠十倍。
- 誤區八：投資額只算機器人本體。整合工時常佔 30~50%、治具佔 40%、停機損失常被完全忽略。
- 誤區九：效益只算人力節省。品質提升的效益往往更大，且是說服決策者的關鍵。
- 誤區十：把安全預算當成可刪減的項目。它是法律義務，且風險評估必須依三階法的順序。
- 誤區十一：以為協作機器人不需要安全評估。它仍須經風險評估，且裝上工具後原評估即失效。
- 誤區十二：驗收通過就以為結束。維運成本常超過建置成本，可監測、可維護、可調整必須在設計階段就考慮。

課堂討論

1. 34.1.1 節指出「技術只佔失敗原因的一成」。請討論：這是否意味著技術能力不重要？一個技術強但不懂系統整合的工程師，與一個懂整合但技術普通的工程師，各自的價值在哪裡？
2. 例題 34-1 顯示「加一個倒角」勝過「花兩百萬提升精度」。請討論這個思維在本書其他章節的對應，並嘗試提出一個判準——什麼時候該改變問題、什麼時候該提升能力？
3. 34.6.3 節的知識框指出品質效益常大於人力效益。請討論：為何提案時人們仍傾向只算人力？這反映了什麼樣的認知偏誤？
4. 34.8.3 節提到台灣以中小企業為主，需要的是「彈性製造」而非「大批量專用自動化」。請討論這對機器人工程師的職涯選擇有什麼啟示？
5. 34.9.3 節說「上線後才是開始」。請討論：一個機器人系統要能穩定運轉三年，設計階段需要做哪些「當下看不出價值」的準備工作？

章末摘要

- 機器人專案的失敗原因中，技術只佔一成；需求不明（35%）與環境假設錯誤（25%）才是

主因。

- V 模型的核心是「每一層的分解都對應一層的驗證」；常見的錯誤是只做元件測試而不做系統測試。
- 客戶說的需求通常不是真實的需求；五問法能挖出真正的目的，並常找出更便宜的解法。
- 需求必須寫成可驗證的形式——有明確的數值、測試條件與樣本定義。
- 範圍潛變的三種控制方法：明確的範圍文件（含「不包含什麼」）、變更管理流程、分階段交付。
- 機器人工作單元中，本體與程式只佔約 30% 的工時；治具與訊號整合各佔 25% 以上。
- 介面問題有四類：時序、資料格式、握手協定、異常處理；其中異常處理最常被完全忽略。
- 核心原則：先問「能不能把環境變得可控」，再問「能不能用演算法處理不可控的環境」。
- POC 的價值不在證明可行，而在盡早發現不可行；必須用現場的真實條件測試。
- 能在 FAT 解決的問題，成本是 SAT 的十分之一；應請客戶參與 FAT。
- 誤差會在系統層級累積；各子系統達標不代表整體達標。
- 誤差預算的三種改善策略：提升精度（貴）、消除誤差源（中）、放寬容差（最便宜）。
- 稼動率的提升有兩條路——增加 MTBF 或降低 MTTR，後者往往便宜得多。
- $OEE = \text{稼動率} \times \text{性能效率} \times \text{良率}$ ；相乘的結構使「每一項都要很高」。
- 投資額常被低估的項目：整合工時、治具與周邊、以及產線停機損失。
- 效益不只是人力節省，還包含品質提升、產能提升與間接效益；品質效益常是最大的一項。
- 安全是法律義務而非品質項目；風險評估依三階段：本質安全 → 工程防護 → 資訊警告。
- 協作機器人仍須風險評估；裝上工具或抓取重物後，原有的安全評估即失效。
- 四大應用領域依環境結構化程度與任務變異度區分，技術重點與成熟度差距極大。
- 部署後最常見的失效不是程式錯誤，而是磨損、光源老化、來料變異等漸進性問題。
- 系統的價值不在交付當天多完美，而在三年後還在穩定運轉——需要可監測、可維護、可調整。

觀念題與計算題

觀念題

1. 列舉機器人專案失敗的五個主要原因及其佔比，並說明「技術只佔一成」的意義。
2. 說明 V 模型的結構與其核心洞見，並舉例說明「只做元件測試」的風險。
3. 說明五問法的作法，並解釋為何它常能找出更便宜的解法。
4. 比較「模糊的需求」與「可驗證的需求」，並各舉三個例子。
5. 說明需求規格中為何必須定義「測試的條件與樣本」。
6. 說明範圍潛變的成因與三種控制方法。
7. 列舉機器人工作單元的七類組成，並指出最常被低估的兩類。
8. 列舉介面問題的四類，並說明為何「異常處理」最常被忽略。
9. 說明「先讓環境可控」的原則，並舉三個用周邊設計降低技術難度的例子。
10. 說明治具方案的限制，以及在什麼情況下應改用視覺方案。
11. 說明 POC 的目的與三個原則，並解釋「早失敗比晚失敗好一百倍」。
12. 區分 FAT 與 SAT，並說明為何應請客戶參與 FAT。
13. 說明誤差為何會在系統層級累積，並比較最壞情況與均方根兩種計算方式。
14. 說明誤差預算的三種改善策略，並指出哪一種通常最划算。
15. 說明 MTBF、MTTR 與稼動率的關係，並解釋為何降低 MTTR 往往較便宜。
16. 說明 OEE 的三個組成，並解釋相乘結構的意義。
17. 列舉自動化投資成本中最常被低估的三個項目。
18. 列舉年度效益的四個類別，並說明為何品質效益常被忽略卻可能最大。
19. 說明機器人安全標準的三層體系，以及風險評估的五個步驟。
20. 說明風險減低的三階法及其優先順序，並解釋為何不可跳過第一階。
21. 說明協作機器人的四種模式，並解釋「協作不等於不需要安全評估」。
22. 比較四大應用領域的環境、任務變異、關鍵技術與成熟度。
23. 列舉部署後的六種常見失效模式，並說明哪兩種特別值得警惕。

計算題

1. 某任務容差 $\pm 0.3 \text{ mm}$ ，誤差來源為：機器人 ± 0.4 、視覺 ± 0.25 、手眼校正 ± 0.15 、工件公差 ± 0.2 、夾持 $\pm 0.15 \text{ mm}$ 。(a) 計算最壞情況與均方根總誤差。(b) 判斷是否可行。

(c) 若加裝定位治具可消除視覺與手眼誤差，重新計算。(d) 若改為在孔口加 0.8 mm 倒角，判斷是否可行並比較兩方案。

2. 某產線班別 8 小時，計畫性停機 45 分鐘，故障停機 30 分鐘。理論節拍 4.5 秒/件，實際生產 5,200 件，不良 130 件。(a) 計算稼動率。(b) 計算性能效率。(c) 計算良率。(d) 計算 OEE 並判斷是否達標竿水準。(e) 若把故障停機減半，計算新的 OEE。
3. 某自動化專案的成本為：機器人 120 萬、視覺 60 萬、治具周邊 80 萬、安全設備 30 萬、整合工時 130 萬、安裝試車 25 萬、安裝期停機損失 3 天（日產值 80 萬）。效益為：取代 4 人（每人年成本 58 萬）、不良率由 2.5% 降至 0.4%（月產 8 萬件、每件價值 300 元）。(a) 計算總投資額。(b) 計算年度效益（品質效益以五成計）。(c) 計算投資回收期。(d) 若只計人力節省，回收期為何？比較兩者並討論。
4. 某工位有三個自動化方案：甲投資 250 萬、年效益 200 萬、壽命 8 年；乙投資 480 萬、年效益 420 萬、壽命 8 年；丙投資 750 萬、年效益 600 萬、壽命 12 年。(a) 計算各方案的投資回收期。(b) 計算五年的累積淨效益。(c) 計算各自壽命期間的累積淨效益。(d) 綜合判斷應選哪一個，並列出你的判斷需要確認的三個前提。

動手做與專題延伸

本章實作：系統提案

本章的實作是撰寫一份完整的系統提案，詳見前面的「實作」一節。此處提供三個延伸的方向。

- **方向一：現場觀察與時間研究。**實際到一個有重複性工作的場所（工廠、餐廳廚房、圖書館），用碼表記錄每個動作的時間，繪出時間分布圖。**找出最耗時的三個動作——這就是自動化的優先目標。**
- **方向二：介面規格書的撰寫。**為你的專題中兩個子系統之間的介面，撰寫一份完整的介面規格書（34.3.2 節的知識框）。**特別要定義異常時的行為。**
- **方向三：安全風險評估。**對一個實際的機器人設備（可用實驗室的），依 34.7.3 節的五步驟做一次完整的風險評估，列出所有危害與防護措施。

專題延伸

- **需求釐清的實證研究：**找三位不同的人（如三位老師或三位同學），請他們各自描述「一台

自動整理實驗室的機器人應該做什麼」。記錄他們的原始描述，接著用五問法各訪談 20 分鐘，記錄挖掘出的真實需求。比較三份「表面需求」與「真實需求」的差異，並歸納出你發現的模式。這個練習能讓你親身體會 34.2 節的核心——**問題往往不是別人說的那個問題**。

- **誤差預算的實測驗證**：對實驗室的機器手臂做一次完整的誤差量測——重複精度（同一點來回 30 次）、絕對精度（用尺量測實際位置與指令的差）、以及溫度漂移（開機後每小時量一次，共 4 小時）。把實測值與規格書的數字比較，你很可能會發現絕對精度遠差於規格書上的重複精度（第 9.9.2 節）。接著用這些實測值做一次誤差預算，判斷你的專題是否可行。
- **進階：一個真實的產業案例研究**。聯繫一家有導入機器人的工廠（可透過學校的產學管道），實地訪談並記錄：他們當初為何導入、遇到什麼問題、實際的投資回收期與預期差多少、以及**如果重來一次會怎麼做**。最後一個問題往往能得到最有價值的答案。把訪談整理成一份案例報告，並用本章的框架（V 模型、誤差預算、OEE、投資評估）分析他們的決策。這個專題能讓你看見教科書與真實產業之間的落差，而那正是最寶貴的學習。

參考文獻與延伸閱讀

- INCOSE, Systems Engineering Handbook, 5th ed., Wiley（系統工程的權威指引，V 模型的標準定義）。
- Blanchard, B. S. and Fabrycky, W. J., Systems Engineering and Analysis, Pearson（系統工程的經典教材，含經濟分析）。
- ISO 12100:2010, Safety of Machinery — General Principles for Design（機械安全的基礎標準，風險評估的方法）。
- ISO 10218-1 & -2:2011, Robots and Robotic Devices — Safety Requirements for Industrial Robots（工業機器人與其系統整合的安全要求）。
- ISO/TS 15066:2016, Robots and Robotic Devices — Collaborative Robots（協作機器人的力與壓力限制）。
- ISO 13849-1:2015, Safety-Related Parts of Control Systems（安全控制系統的設計與 PL 等級）。
- Nakajima, S., Introduction to TPM: Total Productive Maintenance, Productivity Press（OEE 概念的出處）。
- Hitomi, K., Manufacturing Systems Engineering, Taylor & Francis（製造系統工程，含節

拍與產能分析) 。

- Groover, M. P., Automation, Production Systems, and Computer-Integrated Manufacturing, Pearson (自動化系統的完整教材，含經濟分析) 。
- 勞動部職業安全衛生署，〈機械設備器具安全資訊網〉與相關法規：台灣的機械安全法規與檢查基準。
- 經濟部產業發展署，〈智慧機械產業推動方案〉相關報告：台灣機器人與智慧製造的產業脈絡。
- International Federation of Robotics (IFR), World Robotics Report (年度出版)：全球機器人市場與應用趨勢的權威資料。

第 35 章

法規、商業化與社會接受度

編著：李世淵（ 機器人叫獸 ）

助教：李天宇、李宇晴

【章首情境圖建議】左半為餐廳現場的三格情境：第一格是送餐機器人與一位拄拐杖的長者在走道上正面相對、兩者都靜止，後方排了數人，旁註「技術上正確，結果是堵塞」；第二格是小孩伸手觸碰托盤、機器人急停、湯汁潑灑，旁註「急停不一定是最安全的反應」；第三格是餐廳老闆指著機器人提問，對話框寫著「如果它燙傷客人，是誰要賠？」。右半為上市路徑的階梯圖：由左下的「實驗室原型 (TRL 5) 」向右上經過場域試驗、設計定案、認證取得、小量試產，抵達「上市 (TRL 9) 」，並在「認證取得」處標註「3~9 個月、數十萬至數百萬元」。下緣文字：技術可行，不等於可以上市。

叫獸開講 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？

小矧的團隊做出了一台送餐機器人。技術上相當成功——SLAM 建圖穩定、避障靈敏、抓取平穩，在實驗室走廊上跑了三個月沒出過問題。他們決定去一家餐廳試營運。

第一天，機器人在走道上遇到一位拄拐杖的長者。它偵測到障礙、正確地停下、等待——然後就一直等下去。長者不知道該讓路，機器人不知道該繞行，後面排了五個人。

第二天，一個小孩把手伸向機器人的托盤。它偵測到有東西靠近，緊急停止——整盤湯灑了。

第三天，餐廳老闆問了一個問題，讓整個團隊愣住了：「如果它燙傷客人，是誰要賠？」

回到實驗室，叫獸把這三件事寫在白板上。「你們注意到了嗎？這三個問題，沒有一個是技術問題。」

小翰說：「第一個是.....不知道該怎麼跟人互動？」

「對。而且更深一層——它不知道社會的規則。人在走道上相遇時，會有一個微妙的協商：誰先動、誰讓路、要不要點頭示意。這套規則沒有寫在任何地方，但每個人都懂。」

「第二個呢？」小矧說：「它的反應是對的啊，偵測到人就停止。」

「技術上對，但結果是錯的。」叫獸說，「急停灑了湯，可能比慢慢閃開更危險。安全不是一個技術指標，而是一個情境判斷。」

「至於第三個問題.....」叫獸停頓了一下，「那才是最難的。因為它的答案不在你們的程式裡，而在法律、保險、與社會共識裡——而這三樣東西，目前都還沒準備好。」

他在白板上寫下這一章的主題：

技術可行 ≠ 可以上市。

「第 34 章教你們怎麼把技術變成一個能在工廠裡跑的系統。今天要教的是：當機器人走出工廠、走進有人的地方，還要跨過哪些門檻。」

本章為什麼重要

第 34 章的场景是工廠——一個高度可控的環境：進入者受過訓練、有圍籬隔離、有明確的

作業規範。但機器人的下一個戰場是**有人的空間**：餐廳、醫院、賣場、辦公室、以及家庭。

這個轉換帶來三類全新的挑戰，而它們都不是技術問題：

- **法規與認證**：工廠裡的機器人只需符合職安法規；但一台在公共空間運行的機器人，可能同時涉及電氣安全、電磁相容、無線電、個資保護、以及各地方政府的特別規定。**沒有這些文件，做得再好也不能賣。**
- **責任與保險**：工廠的操作員知道風險並受過訓練；但餐廳的客人沒有。一旦出事，責任歸屬（第 33.8.3 節）就成為真實的法律問題。
- **社會接受度**：一台技術完美但讓人不舒服的機器人，會被使用者拒絕——而這種拒絕往往是無聲的：他們不會抱怨，只會不用它。

本章的價值在於「**讓你知道有這些門檻存在**」。多數技術背景的團隊在做出原型後才發現，離「能賣」還有很長的路——而那條路的長度，往往超過開發本身。**提早知道，就能提早準備。**

學習目標

- 能說明工廠環境與公共空間對機器人要求的三類差異。
- 能說明產品標準與認證的體系及其在不同市場的差異。
- 能列舉一台服務機器人上市所需的主要認證項目。
- 能說明台灣、歐盟與美國在機器人法規上的主要差異。
- 能說明個資與隱私法規對機器人設計的具體要求。
- 能分析產品責任的歸屬並說明降低風險的三種手段。
- 能說明保險在機器人商業化中的角色。
- 能區分四種機器人的商業模式並比較其適用場合。
- 能說明技術成熟度與市場採用曲線的關係。
- 能分析社會接受度的四個構面並設計改善策略。
- 能說明人機互動設計的基本原則。
- 能為一個機器人產品規劃完整的上市路徑。

先備知識

第 34 章的系統整合與安全（本章是其在公共空間的延伸）、第 7 章的安全標準、第 33.8 ~ 33.9 節的社會影響與倫理、第 26.10.3 節的社會導航、第 20.10 節的協作安全。

本章沒有數學，但有大量的判斷與取捨。建議讀法：把每一節對照你熟悉的某個產品（如掃地機器人、送餐機器人），思考「它是怎麼處理這個問題的」。

核心名詞中英對照

表 35-1 本章核心名詞

中文	英文	說明
產品認證	Product Certification	由第三方確認符合標準的程序
符合性聲明	Declaration of Conformity	製造商自行聲明符合法規
CE 標誌	CE Marking	歐盟市場的強制性符合標誌
電磁相容	EMC	設備不干擾他人也不被干擾
低電壓指令	Low Voltage Directive	歐盟的電氣安全要求
型式認證	Type Approval	對產品型號的一次性認證
產品責任	Product Liability	產品缺陷造成損害的賠償責任
召回	Recall	產品有安全疑慮時的回收程序
個人資料保護	Personal Data Protection	對可識別個人資訊的法律保護
一般資料保護規則	GDPR	歐盟的個資保護法規
設計即隱私	Privacy by Design	在設計階段納入隱私保護
技術成熟度	Technology Readiness Level, TRL	技術由概念到量產的九級分類
市場採用曲線	Technology Adoption Curve	新技術被市場接受的階段模型
鴻溝	The Chasm	早期採用者與主流市場之間的斷層
機器人即服務	Robot-as-a-Service, RaaS	以訂閱制提供機器人服務
社會接受度	Social Acceptance	使用者與社會對技術的接納程度
恐怖谷	Uncanny Valley	過度擬人卻不完美引發的不適
社會可接受行為	Socially Acceptable Behavior	符合社會慣例的機器人行為

35.1 走出工廠

35.1.1 三類環境的對照

機器人的運行環境可分為三類，而**每跨越一類，難度就跳一個等級**：

表 35-2 三類運行環境的對照

面向	工廠（受控）	半公共（商業空間）	家庭與公共
周圍的人	受訓的員工	顧客、訪客	任何人（含幼童、長者）
環境的變異	低（固定佈局）	中（人流變動）	極高（無法預期）
隔離手段	圍籬、光柵	不可隔離	不可隔離
法規適用	職安法規	職安 + 公共安全	產品安全 + 消費者保護 + 個資
失敗的後果	停線、損壞	客訴、商譽	傷害、法律責任
維護支援	廠內有工程師	定期巡檢	使用者自行處理
可靠度要求	95~98%	98%+	99.9%+

最後一系列的差異特別值得注意。工廠裡稼動率 95% 是可接受的（每天停機一小時，有人會來修）；但一台家用機器人若每天故障一次，**使用者會直接把它收進櫃子裡**。

35.1.2 為何「不能隔離」改變了一切

工廠安全的核心手段是**隔離**（第 34.7.3 節的第二階工程防護）：用圍籬把人與機器分開。這個手段簡單、可靠、且成本低。

但在公共空間，**隔離是不可能的**——機器人存在的目的就是要在人的環境中活動。這使得安全必須改用另外兩種手段：

- **本質安全**（第一階）：限制速度、限制力量、圓角化外殼、降低質量。**這是最可靠的手段**（呼應第 20.10 節與第 33.3 節）。
- **主動偵測與反應**（第三階的延伸）：偵測人的存在並減速或停止。**但如開頭故事所示，這不見得總是正確的反應。**

◎ 急停不一定是最安全的反應

開頭故事的第二天，機器人偵測到小孩的手而緊急停止，結果湯灑了。**這揭示了一個在工廠中不存在的問題：在有人的環境中，「停止」未必是最安全的選項。**工廠的機器人停止就等於安全（因為人不該在那裡）；但服務機器人停在走道中央可能造成阻塞、急停可能潑灑液體、在樓梯口停止可能造成絆倒。**正確的設計是「情境化的安全反應」**——依當下的情況選擇減速、繞行、緩停或完全停止。這比單一的急停邏輯複雜得多，卻是進入公共空間的必要條件。

35.1.3 從「設備」到「產品」

還有一個法律上的關鍵轉變：工廠裡的機器人通常被視為**設備**（equipment），而消費或商用的機器人是**產品**（product）。這個區別有實質的後果：

表 35-3 設備與產品的法律差異

面向	設備（B2B 客製）	產品（量產銷售）
適用法規	職業安全衛生法規	商品檢驗法、消費者保護法
安全驗證	由整合商做風險評估	須通過型式認證
責任基礎	契約責任為主	產品責任（無過失責任）
使用者	受訓的操作員	一般消費者（不需受訓）
說明義務	操作手冊	警語、標示、使用說明
瑕疵時	維修或索賠	可能需召回

「無過失責任」是最關鍵的差異。在產品責任的架構下，即使製造商沒有過失，只要產品有缺陷造成損害，就要負賠償責任。這使得產品的安全設計必須遠比設備嚴謹。

35.2 標準與認證

35.2.1 標準與法規的區別

首先要釐清兩個常被混用的概念：

標準（Standard）：技術規範，通常是自願性的

法規（Regulation）：法律要求，強制性的

但兩者常有連結——法規常「引用」標準作為符合性的推定依據。

具體的運作方式是：法規說「產品必須安全」（原則），標準說「怎樣算安全」（具體要求）。若你的產品符合標準，通常就被推定符合法規。

35.2.2 一台服務機器人需要哪些認證

這是許多團隊在做出原型後才發現的清單。以一台在台灣販售的室內服務機器人為例：

表 35-4 服務機器人的主要認證項目

類別	項目	依據	說明
電氣安全	商品檢驗（BSMI）	商品檢驗法	強制性，未取得不得販售

類別	項目	依據	說明
電磁相容	EMC 測試	CNS 13438 等	確保不干擾他人設備
無線電	NCC 型式認證	電信管理法	含 WiFi、藍牙者皆須
電池	鋰電池安全	CNS 15364 等	電池是高風險項目
機械安全	風險評估報告	ISO 12100	第 34.7 節
服務機器人安全	ISO 13482	—	個人照護機器人的專用標準
個資	隱私政策與告知	個人資料保護法	35.4 節
(若含醫療宣稱)	醫材許可	醫療器材管理法	門檻極高，應避免醫療宣稱

最後一列是一個常見的陷阱。若行銷文案寫「可監測長者的健康狀況」，就可能被認定為醫療器材——而醫材的認證成本與時程是一般產品的十倍以上。因此許多產品刻意使用「生活紀錄」「活動提醒」等非醫療的用語。

35.2.3 ISO 13482：服務機器人的專用標準

值得特別介紹的是 ISO 13482——它是第一個專門針對個人照護機器人的國際安全標準（2014 年發布）。

它把服務機器人分為三類，各有不同的要求：

- 行動輔助機器人：協助行走、移位的設備。風險在於跌倒與夾擠。
- 個人載具機器人：載人移動的設備（如自動輪椅）。風險在於翻覆與碰撞。
- 人型服務機器人：在人的環境中執行任務。風險在於碰撞、夾擠與失控。

它與工業標準（ISO 10218）最大的差異是：工業標準假設「人不該在機器人附近」，因此重點是隔離；而 ISO 13482 假設「人必定在附近」，因此重點是限制傷害的程度——與第 20.10 節協作機器人的思維一致。

35.2.4 不同市場的差異

若要銷往國際，還必須面對各地的差異：

表 35-5 主要市場的認證體系

市場	體系	特點	難度
台灣	BSMI + NCC	商品檢驗 + 無線電型式認證	中

市場	體系	特點	難度
歐盟	CE 標誌	自我宣告 + 技術文件；機械指令、EMC 指令、RED	中高（文件要求嚴謹）
美國	FCC + UL/ETL	FCC 管無線電；UL 非強制但實務上必要	中高
日本	PSE + 技適	電氣用品安全法 + 無線電	高
中國	CCC + SRRC	強制性產品認證	高

一個實務的建議：若有國際市場的規劃，應在設計階段就把目標市場的要求納入。事後修改（如更換不合規的零件、重新佈線以通過 EMC）的成本，遠高於一開始就設計好。

◎ 認證的時間與成本，遠超過多數團隊的預期

一個常見的誤判是：「產品做好了，送測一下就好」。實務上，認證通常需要 3~9 個月與數十萬到數百萬元，且很少一次通過——EMC 不合格需要修改屏蔽或濾波、電氣安全不合格需要改變絕緣設計，每一次修改都要重新送測。因此認證應該在專案的時程與預算中被明確規劃，而非當成最後的一個小步驟。更好的作法是「預測試」：在開發中期就送去做一次非正式的 EMC 掃描，及早發現問題。

35.3 責任、保險與召回

35.3.1 產品責任的三種缺陷

法律上，產品缺陷分為三類，機器人三類都可能發生：

表 35-6 產品缺陷的三種類型

類型	定義	機器人的例子
設計缺陷	設計本身就有問題	夾點設計不當、速度上限過高
製造缺陷	設計正確但製造出錯	某批次的感測器焊接不良
警告缺陷	未充分告知風險與正確用法	未說明不可用於樓梯、未標示夾點

第三類最常被工程團隊忽略。一個技術上完美的產品，若因為說明書沒寫清楚而導致誤用受傷，製造商仍要負責。

35.3.2 降低責任風險的三種手段

手段一：設計上消除風險（最有效）。這是第 34.7.3 節三階段的第一階——能消除的危害就不要靠警告。例如：把夾點的間隙加大到手指伸不進去、限制最大速度、外殼圓角化。

手段二：完整的文件與可追溯性。第 33.8.3 節提過，實務的責任判斷傾向要求「可追溯性與盡職義務」。這意味著必須保存：

- **設計與測試紀錄：**證明做了合理的驗證。
- **風險評估報告：**證明辨識了危害並採取了措施。
- **運行日誌：**事故發生時能還原當時的狀況。這正是第 29.9.3 節「每次都錄 rosbag」的法律意義。
- **軟體版本紀錄：**能確認事故當時運行的是哪個版本。

手段三：明確的使用限制與警告。誠實地說明「這個產品不適合什麼情境」——如「不可用於有樓梯或落差的環境」「不可載運熱湯」「不建議在有幼童無人看管的空間使用」。

◎ 誠實的限制說明反而是優勢

許多團隊擔心「寫出限制會讓產品看起來很弱」。但實務經驗恰恰相反：**明確的限制說明能降低客戶的期待落差，減少客訴，也在法律上保護自己。**一台明確說「我只能在平坦地面運作」的機器人，客戶在鋪了地毯的地方使用出問題時，責任較輕；而一台宣稱「適用各種環境」的機器人，就必須為所有環境負責。這與第 34.2.4 節「明確寫出不包含什麼」是同一個道理——界定範圍不是示弱，而是專業。

35.3.3 保險的角色

保險是商業化中一個常被忽略但關鍵的環節。三種相關的保險：

- **產品責任險：**涵蓋產品缺陷造成第三人損害的賠償。多數 B2B 客戶會要求供應商投保——沒有它，可能連投標的資格都沒有。
- **公共意外責任險：**涵蓋在公共場所發生的意外。餐廳、賣場等場域通常要求。
- **營業中斷險：**若機器人故障造成客戶的營業損失。

一個實務的觀察：保險公司的核保會問很多問題——有沒有認證？有沒有風險評估？有沒有事故紀錄？**這些問題本身就是一份檢核表**，能幫助團隊發現自己的疏漏。而保費的高低，也反映了保險公司對這項技術風險的評估。

35.3.4 召回與軟體更新

機器人有一個傳統產品沒有的特性：**它的行為可以透過軟體更新改變**。這帶來機會也帶來風險。

機會：發現安全問題時，可以**遠端推送修正**，不需實體召回——成本可能相差百倍。

風險：

- **更新本身可能引入新問題。**一個推送到上千台機器人的錯誤更新，後果可能極為嚴重。
- **版本管理的複雜度：**現場可能有多個版本並存，事故時難以釐清。
- **法律上的模糊：**軟體更新是否算「修理」？更新後的產品責任如何認定？**這些在多數國家尚無明確的規範。**

因應之道：分批推送（先小範圍驗證）、保留回滾機制、完整記錄每台設備的版本歷程、以及更新前的完整回歸測試（第 29.9.3 節的 rosbag 在此再次派上用場）。

35.4 隱私與資料保護

35.4.1 機器人是一個移動的感測器

第 33.9.2 節已指出隱私問題的嚴重性，本節從**法規遵循**的角度補充。

關鍵的認識是：**機器人蒐集的多數資料，在法律上屬於個人資料：**

- **影像：**只要能辨識出個人（甚至只是背影加上時間地點），就是個資。
- **語音：**內容與聲紋皆是。
- **位置與軌跡：**能推知一個人的作息與行為模式。
- **環境地圖：**你家的格局本身就是敏感資訊。

35.4.2 個資法的核心要求

台灣的《個人資料保護法》與歐盟 GDPR 的核心要求相似，可歸納為五點：

表 35-7 個資保護的五項核心要求

原則	要求	機器人上的具體作法
告知	蒐集前須告知目的與範圍	首次使用時的說明、機身標示
目的限制	只能用於告知的目的	不可把導航用的影像拿去訓練模型 （除非另行同意）
最小化	只蒐集必要的資料	能在本地處理就不上傳、能用低解析度就不用高的
保存期限	目的達成後應刪除	設定自動刪除的週期
當事人權利	可查詢、更正、刪除	提供資料刪除的功能

第二列是最容易違反的一項。許多團隊會想「把使用者的資料拿來改善模型」——但若當初告知的目的只是「導航」，這就是違法的。正確的作法是明確告知並取得額外的同意。

35.4.3 設計即隱私

設計即隱私 (Privacy by Design) 是現代個資法規的核心原則：隱私保護必須在設計階段就納入，而非事後補救。

七個具體的設計原則：

- 本地優先：能在機器人上處理的就不要上傳（呼應第 32.10.1 節）。
- 去識別化：若必須上傳，先做模糊化或去除人臉。
- 明確的指示：以指示燈或聲音讓人知道相機何時運作。這也是社會接受度的關鍵（35.7 節）。
- 可控的禁區：讓使用者能指定「不進入、不記錄」的區域。
- 預設最保守：預設不蒐集、不上傳，由使用者主動開啟。
- 資料可攜與刪除：提供匯出與一鍵刪除的功能。
- 透明的政策：用一般人看得懂的語言說明蒐集了什麼、為什麼、存多久。

◎ 隱私設計不只是合規，更是市場的差異化

許多團隊把隱私當成「法務要求的麻煩」。但近年的市場趨勢顯示：隱私正在成為消費者的購買考量。一台明確標示「所有影像僅在本機處理、絕不上傳」的家用機器人，相對於一台把影像上傳雲端的競品，在行銷上是明確的優勢。而技術上，本地處理也帶來延遲低、不依賴網路的好處（第 32.10.1 節）。這是一個「合規、體驗、行銷」三者一致的少見案例——值得優先投入。

35.5 商業模式

35.5.1 四種模式

同樣的技術，用不同的商業模式銷售，會有截然不同的結果。機器人領域主要有四種模式：

表 35-8 四種機器人商業模式

模式	作法	優點	缺點	適用
設備銷售	一次買斷	現金流快、關係單純	客戶初期投資高、後續無收入	工業設備、已成熟的技術

模式	作法	優點	缺點	適用
系統整合	客製化專案 (第 34 章)	單價高、切中需求	難以規模化、依賴人力	產線自動化
機器人即服務 (RaaS)	訂閱制，按月/按件付費	降低客戶門檻、持續收入	初期資金壓力大、需長期維運	服務業、中小企業
平台與軟體	賣軟體或開發平台	邊際成本低、可規模化	需要生態系、競爭激烈	已有硬體生態的市場

35.5.2 RaaS 為何在服務機器人上特別重要

RaaS (Robot-as-a-Service) 是近年服務機器人的主流模式，原因有四：

- **降低採用門檻：**一台送餐機器人可能要價六十萬，對小餐廳是難以承受的投資；但每月一萬五的租金，與雇用一個工讀生相當，決策就容易得多。
- **風險轉移：**客戶不必承擔「買了不好用」的風險——不好用就不續約。這對尚未完全成熟的技術特別重要。
- **維運內含：**機器人需要定期保養與軟體更新 (第 34.9 節)。RaaS 把這些內含在服務中，客戶不需自行處理。
- **持續改善的誘因：**供應商的收入來自「客戶持續使用」，因此有強烈的動機去改善產品——這使雙方的利益一致。

但 RaaS 也有一個嚴峻的挑戰：現金流。設備的成本一次投入，收入卻分期回收：

設備成本 60 萬，月租 1.5 萬 → 需 40 個月才回本 (不計維運成本)

這對新創公司是巨大的資金壓力。

因應之道包括：與租賃公司合作 (由金融業承擔資產)、提高月租但降低合約期、或採「混合模式」 (部分買斷加部分服務費)。

35.5.3 定價的邏輯

一個常見的錯誤是用**成本加成法定價** (成本 $\times 1.5$ = 售價)。更好的方式是**價值定價**——依客戶得到的效益定價。

以送餐機器人為例：

客戶的效益 = 節省的人力 + 提升的翻桌率 + 減少的送錯率

若每月效益 4 萬，則月租 1.5 萬對客戶而言是明確划算的

定價的上限由客戶的效益決定，而非你的成本。

這也解釋了為何第 34.6.3 節的效益計算如此重要——你必須能清楚地向客戶說明「你會賺回多少」，才能支撐合理的價格。

◎ 一個定價的陷阱：與人力成本直接對標

服務機器人常被拿來與人力比較：「一台機器人月租 1.5 萬，比請一個人便宜」。但這個對標有風險——因為機器人做不到人能做的全部。一個服務生能送餐、能收桌、能回答問題、能處理客訴；而送餐機器人只能做第一項。若客戶期待「取代一個人」，必定失望。更好的定位是：「它承擔了最耗體力的來回搬運，讓你的服務生能專注在與客人的互動上」——這是「互補」而非「取代」（呼應第 33.8.1 節的三種影響）。這個定位不只更誠實，也更能被員工接受（他們不會覺得自己要被取代）。

35.6 技術成熟度與市場採用

35.6.1 技術成熟度等級 (TRL)

TRL (Technology Readiness Level) 是評估技術成熟度的標準框架，源自 NASA，現廣泛用於研發管理與補助審查：

表 35-9 技術成熟度等級 (TRL)

等級	階段	機器人領域的對應
TRL 1-3	基礎研究、概念驗證	論文、實驗室的原理驗證
TRL 4-5	實驗室與相關環境驗證	實驗室的完整原型 (多數學術研究止於此)
TRL 6-7	真實環境的原型驗證	現場試營運 (第 34 章的 POC 與 SAT)
TRL 8-9	完成認證、實際運作	通過認證、量產、商業運轉

一個關鍵的觀察：學術研究通常在 TRL 4-5 就發表論文並宣告成功；但商業化需要 TRL 8-9。而 TRL 5 到 8 的距離，往往比 1 到 5 更遠——這正是所謂的「死亡之谷」。

TRL 1→5：技術問題為主 (研究能力)

TRL 5→9：工程、認證、可靠度、成本 (完全不同的能力)

許多優秀的研究成果止步於此，不是因為技術不好，而是因為缺乏後半段所需的能力與資源。

35.6.2 市場採用曲線與鴻溝

即使技術成熟，市場的採用仍有其節奏。經典的**採用曲線**把使用者分為五類：

表 35-10 市場採用的五個階段

階段	佔比	特徵	對機器人的態度
創新者	2.5%	熱愛新技術本身	願意忍受不完美，主動嘗試
早期採用者	13.5%	尋求競爭優勢	願意冒險，重視效益
早期大眾	34%	務實，要看到成功案例	要求可靠、有支援、有參考客戶
晚期大眾	34%	保守，跟隨主流	等到成為標準配備才採用
落後者	16%	抗拒改變	被迫才採用

「鴻溝」(The Chasm) 出現在早期採用者與早期大眾之間——這是最多產品陣亡的地方。原因是兩者的需求截然不同：

早期採用者要的是：新的可能性 (願意忍受問題)

早期大眾要的是：可靠的解決方案 (不能有問題)

同一個產品往往無法同時滿足兩者。

跨越鴻溝的三個關鍵：

- **專注一個明確的利基**：與其做「通用服務機器人」，不如做「專門送餐的機器人」——把一件事做到極致可靠，建立可複製的成功案例。
- **建立完整的支援體系**：早期大眾要的不只是產品，還有安裝、訓練、維護、備品、以及「壞了有人來修」的保證 (第 34.9 節)。
- **可驗證的參考客戶**：「某某連鎖店用了兩年」比任何規格表都有說服力。

35.6.3 機器人領域的現況

用這兩個框架檢視機器人的各個領域 (呼應第 34.8.2 節)：

表 35-11 各領域的成熟度與市場位置

領域	TRL	市場階段	主要瓶頸
工業手臂	9	晚期大眾	已成熟，競爭在成本與彈性
掃地機器人	9	早期大眾 ~ 晚期大眾	已跨越鴻溝，滲透率持續上升
送餐/服務機器人	8	早期大眾 (正在跨越)	可靠度、與人互動、投資回收
倉儲揀貨	7-8	早期採用者 ~ 早期大眾	抓取的泛化 (第 21 章)

領域	TRL	市場階段	主要瓶頸
自駕車	7-8	早期採用者	長尾情境、法規、責任
人形機器人	4-6	創新者	可靠度、成本、實用性尚未證明
家用操作機器人	3-5	研究階段	靈巧操作 (第 33.7.3 節)

最後兩列值得特別注意。人形機器人近年獲得大量關注與投資，但其 TRL 仍在 4-6 之間——這意味著它距離「可靠地在真實環境中創造價值」仍有距離。這不是否定其潛力，而是提醒：關注度與成熟度是兩回事。

35.7 社會接受度

35.7.1 技術完美不等於被接受

開頭故事的第一天，機器人「正確地」停下來等待——技術上完全符合設計，但結果是走道被堵住、五個人在排隊、餐廳的服務體驗變差。

這揭示了一個關鍵的差異：

技術指標：偵測到障礙 → 停止 (正確)
社會期待：遇到人 → 禮讓、繞行、或示意 (自然)

兩者的落差，就是社會接受度的問題。

35.7.2 接受度的四個構面

使用者是否接受一台機器人，取決於四個構面：

表 35-12 社會接受度的四個構面

構面	使用者的問題	設計上的因應
有用性	「它真的幫到我了嗎？」	解決真實的痛點 (第 34.2 節的五問法)
易用性	「我會用嗎？會不會很麻煩？」	簡化操作、明確的回饋、容錯設計
信任	「它安全嗎？可預測嗎？」	行為的可預測性、明確的意圖表達
社會規範	「別人會怎麼看？我會不會很奇怪？」	符合場域的社會慣例、不引人側目

第三與第四構面最常被工程團隊忽略，卻往往是決定成敗的關鍵。

35.7.3 信任來自可預測性

一個反直覺的發現：使用者信任的不是「最聰明」的機器人，而是「最可預測」的機器人。

原因是：人需要能夠預期機器人接下來會做什麼，才能安心地與它共處。一台行為多變、難以預測的機器人（即使它每次的決策都更「最佳」），會讓人感到不安。

三個提升可預測性的設計：

- 行為的一致性：同樣的情境應該有同樣的反應。不要因為演算法的隨機性而讓行為變來變去。
- 意圖的表達：讓人知道它「打算做什麼」。這可以透過：轉向前先減速並轉頭（模仿人的行為）、用光條指示行進方向、用聲音提示「我要通過了」。
- 明確的狀態顯示：它現在是在工作、待命、還是故障？一個看不出狀態的機器人會讓人不知所措。

◎ 「腿部語言」：一個優雅的設計案例

有研究團隊在送貨機器人上做過一個實驗：加裝一個簡單的「眼睛」，在轉彎前先「看」向要去的方向。結果行人與機器人的碰撞事件顯著減少——不是因為機器人變聰明了，而是因為行人能預測它要往哪走，於是自然地讓開。這個設計的成本極低（兩個 LED 眼睛），效果卻超過複雜的路徑規劃改良。它體現了一個重要的原則：與其讓機器人更會閃避人，不如讓人更容易閃避機器人。這也是第 26.10.3 節社會導航的核心思想——溝通往往比計算更有效。

35.7.4 恐怖谷與外觀設計

恐怖谷（Uncanny Valley）是機器人外觀設計的重要現象：隨著擬人程度提高，人的好感度會先上升，但在「非常像人卻不完全像」的區間急遽下降，直到完全擬真才回升。

卡通化的外觀 → 好感度高

寫實但不完美的人臉 → 強烈的不適感 X

完全擬真 → 好感度回升（但技術上難以達成）

多數服務機器人刻意停留在第一區。

實務上的三個設計建議：

- 不要追求擬真。除非有極強的理由，否則卡通化或機械感的外觀更安全。
- 功能性的外觀勝過擬人。一台送餐機器人不需要臉——它需要的是穩固的托盤、清楚的方向指示、以及讓人一眼看出「它是機器」。

- **避免過度的擬人化承諾**。若外觀像人，使用者會期待它能像人一樣互動——而它做不到，於是失望。外觀應該誠實地反映能力。

最後一點是一個重要的設計倫理：外觀是一種承諾。一台看起來很聰明的機器人，若實際上很笨，使用者的失望會轉為排斥。

35.8 人機互動設計

35.8.1 從「操作」到「互動」

工業機器人的使用者是**受訓的操作員**，介面可以複雜（如示教器）。但服務機器人的使用者是**未受訓的一般人**——這使介面設計的邏輯完全改變。

表 35-13 兩種介面設計的對照

面向	工業（操作）	服務（互動）
使用者	受訓的技術人員	任何人
學習時間	數小時到數天	零（必須直覺）
介面複雜度	可以複雜	必須極簡
錯誤的容忍	使用者會小心	必須容錯
回饋方式	螢幕與指示燈	光、聲、動作、語音
設計重點	功能完整	可理解、可預測

35.8.2 多模態的回饋

由於使用者不會盯著螢幕，服務機器人必須用**多種管道**傳達資訊：

- **光**：狀態指示（待命/工作/故障）、方向指示（要往哪走）。光的優點是遠距可見且不吵。
- **聲**：提示音（「請讓一下」）、警示音。但要極為節制——一台一直發出聲音的機器人在餐廳裡會令人厭煩。
- **動作**：轉向前的減速、輕微的「點頭」示意。這是最自然的溝通方式（35.7.3 節的知識框）。
- **語音**：最直接但也最侵入。適合需要明確指示的場合，但在安靜的環境（如圖書館、醫院夜間）應避免。

一個實務原則：回饋的強度應與情境相稱。日常的狀態用光即可；需要人配合時用輕聲提示；

只有真正的危險才用警示音。

35.8.3 失敗的優雅處理

服務機器人一定會失敗——這是第 33.7.3 節「可靠度」問題的必然結果。因此如何優雅地失敗，比「不失敗」更務實。

四個原則：

- 及早求助勝過反覆嘗試。若卡住了，與其嘗試二十次，不如在第三次後就請求協助。反覆的失敗會讓旁觀者失去耐心。
- 明確地說明狀況。不要只顯示「錯誤 E-042」，而要說「我被擋住了，可以請你幫我移開嗎？」——一般人看得懂的语言。
- 保持安全的姿態。失敗時應該停在不擋路、不危險的位置——這需要在設計時就規劃「失敗時該退到哪裡」。
- 讓人容易介入。提供明確的手動模式、緊急停止、以及「推得動」的設計（斷電後可自由推動）。

◎ 一個容易被忽略的設計：讓它推得動

許多服務機器人在斷電或故障後完全推不動（因為減速機的自鎖，第 5.5 節）。這在實務上是嚴重的問題——一台故障的機器人卡在走道中央，沒有人能移動它，只能等工程師來。因應之道有三：使用可反向驅動的傳動（低減速比，第 5.5.2 節）、加裝手動釋放的離合器、或設計成可以整台抬起（控制總重量）。這是一個典型的「設計時想到就很簡單、事後才發現就很麻煩」的細節——也呼應了第 34.9 節「可維護性必須在設計階段考慮」。

35.8.4 與現場人員的關係

最後一個常被忽略的面向：機器人不只服務顧客，還要與現場的員工共處。

若員工不接受它，機器人就會被「架空」——他們會找理由不用它、或在它出問題時不願處理。三個實務的建議：

- 讓員工參與導入的過程。在規劃階段就聽他們的意見——他們最清楚現場的實際狀況（呼應第 34.2 節的需求訪談）。
- 明確定位為「輔助」而非「取代」（35.5.3 節的知識框）。強調它承擔的是最累的部分。
- 讓他們能掌控它。提供簡單的手動操作、暫停、與呼叫功能。能掌控的東西才不會被視為威

脅。

這也呼應第 33.8.2 節的自動化悖論：系統越自動化，人的角色越關鍵。而一個被員工排斥的系統，在需要人介入時就會失敗。

35.9 上市的完整路徑

35.9.1 六個階段

整合前面各節，一個機器人產品從原型到上市的完整路徑為：

表 35-14 產品上市的六個階段

階段	主要工作	常見的時程	關鍵風險
一、原型驗證	技術可行性 (TRL 5-6)	6~18 個月	技術做不出來
二、場域試驗	真實環境的試營運	3~9 個月	實際情境與預期不符
三、設計定案	為量產與認證而重新設計	3~6 個月	低估重新設計的工作量
四、認證取得	各項法規認證 (35.2 節)	3~9 個月	測試不過需修改再送
五、小量試產	驗證製造與品質	2~4 個月	良率、供應鏈
六、上市與維運	銷售、支援、持續改善	持續	維運能量不足

第三階段最常被低估。一個實驗室的原型與一個可量產的產品，差異極大：

- **零件的取得性**：實驗室可以用單價五千的實驗級零件，量產必須找到穩定供應且成本可行的替代品。
- **製造的可行性**：3D 列印的外殼在量產時要改為射出成型——而射出成型有拔模角、肉厚、分模線等一堆限制。
- **維修的設計**：實驗室的機器壞了就拆開修；產品必須考慮誰來修、怎麼拆、要不要防止使用者自行拆解。
- **成本的壓縮**：原型的物料成本可能是目標售價的三倍——需要大幅的重新設計。

35.9.2 場域試驗：最有價值的一步

第二階段的場域試驗，其價值等同於第 34.4.2 節的 POC——它會暴露所有你想不到的問題。

開頭故事的三天，正是場域試驗的縮影：

- **第一天的問題（不知如何與人協商）**：這在實驗室永遠測不出來，因為實驗室裡沒有拄拐杖的長者。
- **第二天的問題（急停灑湯）**：這需要真實的液體、真實的小孩、真實的意外。
- **第三天的問題（責任歸屬）**：這只有真正的營運者才會問。

場域試驗的三個原則：

- **盡早開始，即使產品不完美**。等到「做好了再去試」，通常已經投入太多而難以改變設計方向。
- **觀察勝過詢問**。使用者說「還不錯」但實際上很少用它——**行為比言語誠實**。應記錄實際的使用頻率與情境。
- **記錄所有的意外**。每一個「怎麼會這樣」都是一個設計的缺口。**這些累積起來，就是產品與競品的差距**。

35.9.3 一份上市檢核表

以下是一份實用的檢核表，涵蓋本章的所有面向：

表 35-15 機器人產品上市檢核表

類別	檢核項目	本章節次
安全	完成風險評估報告	第 34.7 節
	符合適用的安全標準（ISO 13482 等）	35.2.3
	失效時的行為安全且不阻塞	35.8.3
認證	電氣安全、EMC、無線電認證	35.2.2
	確認未涉及醫療或其他特許宣稱	35.2.2
	目標市場的在地認證需求已確認	35.2.4
責任	投保產品責任險	35.3.3
	使用限制與警告已明確標示	35.3.2
	運行日誌可追溯	35.3.2
隱私	隱私政策已撰寫並告知	35.4.2
	資料最小化與本地處理已實作	35.4.3
	提供資料刪除的功能	35.4.2
互動	非技術人員能零學習使用	35.8.1

類別	檢核項目	本章節次
	行為可預測、意圖可辨識	35.7.3
	斷電後可推動或有手動釋放	35.8.3
維運	備品清單與供應鏈已確認	第 34.9 節
	遠端診斷與更新機制已建立	35.3.4
	現場人員的訓練材料已備妥	35.8.4

35.10 台灣的機會與策略

35.10.1 三個結構性的優勢

第 34.8.3 節談過台灣的產業脈絡，本節從**機器人產品化**的角度補充三個優勢：

優勢一：完整的硬體供應鏈。從機構加工、電子組裝、馬達、到模具，台灣的供應鏈密度極高且反應快速。這使得「快速試錯」的成本遠低於其他地區——一個機構件的修改，在台灣可能三天完成，在其他地方可能三週。

優勢二：ICT 與機械的交會。機器人正好需要這兩者的結合，而台灣同時具備兩個產業的深厚基礎。這在全球是少見的組合。

優勢三：高齡化與缺工的內需市場。台灣的高齡化速度居世界前列，加上服務業的缺工——這創造了真實的需求，而非想像的需求。且台灣市場的規模適中，適合作為驗證的場域（不像大市場那樣競爭激烈，也不像小市場那樣沒有規模）。

35.10.2 三個結構性的挑戰

但也必須誠實面對三個挑戰：

- **內需市場規模有限：**單靠台灣市場難以支撐大規模的研發投入，必須及早規劃國際化——而這又回到 35.2.4 節的認證問題。
- **軟體與系統整合的人才相對稀缺：**台灣的強項在硬體製造，但機器人的價值越來越多在軟體與系統。
- **品牌與通路的經驗不足：**長期的代工模式使得「做出好產品」與「賣出好產品」之間有落差。

35.10.3 一個務實的策略建議

綜合以上，對台灣的機器人團隊，有三個務實的建議：

建議一：從利基市場切入，而非通用市場。與其做「通用服務機器人」與國際大廠競爭，不如專注一個明確的場域（如特定類型的餐廳、特定的檢測任務），**把它做到極致可靠**。這也符合 35.6.2 節「跨越鴻溝」的原則。

建議二：善用供應鏈優勢做快速迭代。台灣最大的優勢是「改得快」——**應該把這個優勢用在場域試驗的快速迭代上**（35.9.2 節），而非追求一次到位的完美設計。

建議三：把系統整合能力視為核心競爭力。第 34 章指出整合佔專案的 30~50%，而這正是難以被複製的能力——它需要對現場的深刻理解，而這是本地團隊的天然優勢。

35.11 由上市邁向專題

35.11.1 本章的成果

本章補完了「技術之外」的最後一塊。你現在能夠：說明公共空間與工廠環境的三類差異、列舉服務機器人所需的主要認證、分析產品責任的歸屬與降低手段、依個資法規設計隱私保護、比較四種商業模式並理解 RaaS 的邏輯、用 TRL 與採用曲線評估技術的位置、設計提升社會接受度的互動、以及規劃完整的上市路徑。

最重要的是三個認識：

- **技術可行 ≠ 可以上市。**認證、責任、保險、社會接受度——每一項都可能是致命的門檻，且都需要時間與成本。
- **可預測性勝過聰明。**使用者信任的不是最聰明的機器人，而是最可預測的機器人。
- **外觀是一種承諾。**它應該誠實地反映能力——過度的擬人化只會帶來失望與排斥。

35.11.2 接下來：把知識變成你自己的作品

第 36 章是全書的最後一章，也是所有內容的最終整合——**完整的專題方法論**。

它會帶你走過一個完整的專題：從**選題**（如何找到一個「值得做且做得完」的題目）、**規劃**（如何在有限的時間內完成）、**開發**（如何有效地推進與除錯）、**測試**（如何驗證你真的做到了）、到**發表**（如何讓別人理解你的價值）。

而它會特別著重一件事：失敗原因的分析與設計改良。因為在真實的專題中，失敗是常態而非例外——而能否從失敗中學到東西，決定了這個專題對你的價值。

換句話說：前三十五章教你**機器人學是什麼**，第 36 章教你**如何用它做出你自己的東西**。

完整案例分析

本章沒有計算例題，取而代之的是三個貼近實務的案例分析。

案例 35-1 送餐機器人的上市評估

情境：某團隊完成了送餐機器人的原型，在實驗室運行穩定。他們計畫六個月後上市，售價 55 萬元/台。請評估這個計畫的可行性。

分析一：時程的檢視 (35.9.1 節)

目前的狀態約在 TRL 5-6 (實驗室原型)。要達到可上市的 TRL 8-9，仍需經過：

場域試驗 3~9 個月 + 設計定案 3~6 個月 + 認證 3~9 個月 + 試產 2~4 個月 = 最快 11 個月，一般 18~28 個月

六個月的計畫明顯過於樂觀。

最可能的結果：趕在六個月上市，但跳過了場域試驗與完整認證——這會導致上市後大量的問題與潛在的法律風險。

分析二：認證的檢視 (35.2.2 節)

這台機器人含 WiFi、鋰電池、且在公共場所運行，因此至少需要：

- BSMI 商品檢驗 (電氣安全) ——強制，未取得不得販售。
- NCC 型式認證 (無線電) ——強制。
- 鋰電池安全測試——高風險項目，測試較嚴格。
- EMC 測試——馬達驅動器是常見的干擾源，首次測試不過的機率很高。

預估：認證費用 60~150 萬、時程 4~8 個月 (含一次修改重測)
--

這筆預算與時間必須明確納入計畫。

分析三：商業模式的檢視 (35.5 節)

售價 55 萬對小餐廳而言是很高的門檻。以一家月營收 200 萬的餐廳而言，55 萬是近一個月的營收——決策層級會拉高到老闆，決策周期會拉長。

建議改為 RaaS：

月租 1.8 萬（約等於一個工讀生的成本）
設備成本 30 萬 → 回本需 17 個月（不計維運）
三年合約的總收入 = 64.8 萬 > 買斷的 55 萬

對雙方都更有利：客戶門檻降低、供應商總收入更高且有持續關係。

但要注意現金流：若第一年要部署 50 台，需要 $30 \times 50 = 1,500$ 萬的資金投入，而第一年的收入只有約 1,080 萬。這個資金缺口必須事先規劃（35.5.2 節）。

綜合建議：

1. 把時程調整為 18 個月，並明確規劃各階段。
2. 立即開始場域試驗（不要等產品完美），與 2~3 家餐廳合作。
3. 同步啟動認證的預測試（尤其是 EMC），及早發現問題。
4. 改採 RaaS 模式，並規劃資金來源（自有資金、租賃公司合作、或投資）。
5. 投保產品責任險，並在合約中明確界定責任範圍。

案例 35-2 三天的問題該如何解決

情境：回到開頭故事的三個問題。請為每一個提出具體的設計解法。

問題一：與拄拐杖長者的僵局

問題的本質：機器人只有「停止」與「前進」兩種行為，缺乏「協商」的能力。

三層解法：

- 表達意圖（35.7.3 節）：加裝方向指示燈，讓長者知道機器人打算往哪走。這是成本最低也最有效的一步。
- 主動溝通：偵測到僵局超過三秒時，發出輕聲的「不好意思，我從您右邊過去」，同時慢速地往右移動——用行動配合語言。
- 繞行策略：若走道寬度足夠，主動貼邊繞行而非正面停止。這需要第 26 章的局部規劃器支援「側向繞行」而非只有「停止」。

問題二：急停造成潑灑

問題的本質：把「安全」等同於「停止」（35.1.2 節的知識框）。

三層解法：

- **機構層（最可靠）**：加裝托盤的防潑設計——凹槽、防滑墊、或加蓋。這是第 33.3 節形態計算的思維：與其讓控制更聰明，不如讓機構容錯。
- **控制層**：把「急停」改為「緩停」——依偵測到的距離決定減速率。只有在極近距離（如 20 公分內）才急停。
- **規劃層**：降低整體速度上限，並在人多的區域自動減速（第 26.7 節 DWA 的速度限制）。

注意三層解法的優先順序：機構層最可靠且不會失效，應優先採用。

問題三：燙傷的責任歸屬

這個問題沒有技術解，但有工程與商業上的因應（35.3 節）：

- **設計上降低風險**：限制可載運液體的溫度（如加裝溫度感測，超過 60°C 拒絕載運）、要求加蓋、限制加速度。
- **明確的使用限制**：在合約與說明書中寫明「不建議載運超過 60°C 的液體」——這是警告缺陷的防範（35.3.1 節）。
- **投保產品責任險**（35.3.3 節）。
- **合約中界定責任**：明確約定餐廳方的責任（如確保走道淨空、確保容器加蓋）與供應商的責任（設備的正常運作）。
- **運行日誌**：記錄事故當時的速度、感測資料、與控制指令——這是釐清責任的關鍵證據（35.3.2 節）。

一個誠實的結論：這個問題無法完全消除風險，只能降低機率並明確分擔。而這正是所有真實產品的處境——沒有零風險的產品，只有風險被合理管理的產品。

案例 35-3 隱私設計的取舍

情境：一台居家陪伴機器人需要辨識家中成員以提供個人化服務。團隊面臨三個設計選項，請分析其取舍。

表 35-16 三種隱私設計方案的比較

方案	作法	隱私	功能	成本
甲：雲端辨識	影像上傳雲端做人臉辨識	最差	最強（可用大模型）	低（不需高階硬體）
乙：本地辨識	在機器人上做人臉辨識	佳	中（受限於本地算力）	高（需 AI 加速硬體）
丙：非視覺辨識	用語音或穿戴裝置辨識	最佳（不拍攝）	弱（需主動配合）	低

分析：

- 方案甲功能最強但隱私最差。在家庭場景中，這可能是致命的——使用者對「家中的攝影機把影像傳到雲端」有強烈的疑慮（第 33.9.2 節）。
- 方案乙是多數產品的選擇。成本較高，但「所有影像僅在本機處理」是強而有力的行銷訴求（35.4.3 節的知識框）。
- 方案丙隱私最好但功能受限。適合作為輔助而非主要方案。

建議：以方案乙為主，方案丙為輔。具體的設計為：

1. 本地辨識：人臉辨識完全在機器人上執行，影像絕不上傳。
2. 只儲存特徵而非影像：辨識完成後即刪除原始影像，只保留無法還原的特徵向量。
3. 明確的指示：相機運作時亮起明確的指示燈。
4. 可控的禁區：讓使用者設定「不進入、不辨識」的房間（如臥室、浴室）。
5. 一鍵刪除：提供清除所有個人資料的功能。
6. 保留方案丙作為備援：若使用者選擇關閉視覺辨識，可改用語音辨識——把選擇權交給使用者。

核心的判斷：在家庭場景中，隱私不是一個可以用功能交換的項目。一個功能稍弱但令人安心的產品，勝過一個功能強大但讓人不敢放在客廳的產品。

實作：產品上市的規劃

本章的實作是為一個機器人產品（可用你的專題）規劃完整的上市路徑。

1. 定位分析：用 35.6 節的框架，評估你的產品目前在 TRL 的哪一級、目標客戶屬於採用曲線

的哪一群。**誠實地評估。**

2. **環境分析**：用表 35-2 判斷你的產品要在哪一類環境運行，並列出因此增加的要求。
3. **認證清查**：列出你的產品需要的所有認證（35.2.2 節）。上網查詢至少三項的實際費用與時程——你可能會被數字嚇到，而這正是重點。
4. **責任分析**：列出你的產品可能造成的三種傷害，並依 35.3.1 節分類（設計/製造/警告缺陷）。為每一項提出防範措施。
5. **隱私盤點**：列出你的產品會蒐集的所有資料，判斷哪些屬於個資，並依 35.4.3 節的七原則設計保護措施。
6. **商業模式設計**：比較買斷與 RaaS 兩種模式，計算各自的客戶成本與你的現金流。**特別要算出 RaaS 的資金缺口。**
7. **接受度評估**：用 35.7.2 節的四個構面，評估你的產品在每一項的表現。**找出最弱的一項並提出改善。**
8. **互動設計檢視**：檢查你的產品是否符合 35.8 節的原則——非技術人員能否零學習使用？行為是否可預測？失敗時是否優雅？斷電後推得動嗎？
9. **場域試驗規劃**：設計一個場域試驗的計畫——去哪裡、待多久、觀察什麼、如何記錄。**重點是「觀察勝過詢問」。**
10. **填寫上市檢核表**：用表 35-15 逐項檢視，標出「已完成」「進行中」「尚未開始」。
11. **撰寫上市計畫書**：整合以上為一份 8~12 頁的計畫書，包含：產品定位、目標市場、上市時程、認證計畫、商業模式、風險與對策。
12. **（進階）實地訪談**：找一位潛在的使用者（如餐廳老闆、長照機構主管），呈現你的計畫並訪談。**特別問他們「你最擔心什麼」——那往往是你沒想到的。**

國中小也看得懂：做得出來不等於賣得出去

假設你和同學發明了一台超厲害的機器人，可以幫忙送東西。你們很興奮，想把它拿去賣。

但等一下——做出來，跟可以拿去賣，是兩回事喔！

先想想這幾個問題：

問題一：它會不會電到人？ 你必須送去給專門的機構檢驗，證明它是安全的。這個檢驗要

花好幾個月、好幾十萬。

問題二：它的 WiFi 會不會干擾別人？也要送檢驗。

問題三：如果它撞到人，誰要賠錢？ 是做的人？賣的人？還是用的人？這要看合約怎麼寫、有沒有保險。

問題四：它拍到的照片存在哪裡？ 如果傳到網路上，那就是別人的隱私。法律規定你必須先告訴人家、而且用完要刪掉。

有沒有發現？**這些問題都跟「機器人做得好不好」沒關係，但每一個都可能讓你賣不出去。**

還有一個更有趣的問題：**就算全部都過關了，人家會喜歡它嗎？**

有一個真實的故事：一台送餐機器人在走道上遇到一位拿拐杖的老爺爺。機器人很「乖」地停下來等——結果**兩個都不動，後面排了一長串人。**

為什麼？因為人跟人相遇的時候，會有一種默契：誰先讓、誰先走、要不要點個頭。**這種默契沒有寫在任何地方，但每個人都懂——只有機器人不懂。**

後來工程師想到一個很妙的辦法：**在機器人上裝兩個像眼睛的燈，轉彎前先「看」向要去的方向。**結果撞到人的次數大幅減少了！

不是因為機器人變聰明了，而是因為**人可以猜到它要往哪走，就自己讓開了。**

這就是這一章最重要的一句話：與其讓機器人更會躲人，不如讓人更容易躲機器人。

所以啊，做機器人不只要懂技術，**還要懂人。**這可能是最難、但也最有趣的部分。

■ 叫獸提醒與常見錯誤

- 誤區一：以為做出原型就快上市了。TRL 5 到 8 的距離往往比 1 到 5 更遠，這正是「死亡之谷」。
- 誤區二：把認證當成最後的小步驟。它通常需要 3~9 個月與數十萬元，且很少一次通過，必須納入時程與預算。
- 誤區三：行銷文案使用醫療宣稱。這可能使產品被認定為醫療器材，認證成本暴增十倍以上。

- 誤區四：以為「技術上正確」就是安全。急停可能潑灑液體、停在走道可能造成阻塞——安全是情境判斷。
- 誤區五：忽略警告缺陷。技術完美的產品若說明不足導致誤用，製造商仍要負責。
- 誤區六：擔心寫出使用限制會讓產品看起來很弱。明確的限制能降低期待落差、減少客訴，並在法律上保護自己。
- 誤區七：把導航蒐集的影像拿去訓練模型。若當初告知的目的只是導航，這就違反了目的限制原則。
- 誤區八：用成本加成法定價。應以客戶的效益定價，而效益的計算必須完整（第 34.6.3 節）。
- 誤區九：把機器人定位為「取代一個人」。它做不到人的全部，應定位為承擔最耗體力的部分。
- 誤區十：追求最聰明而非最可預測。使用者信任的是行為一致、意圖明確的機器人。
- 誤區十一：追求擬真的外觀。恐怖谷效應使「像人卻不完美」引發強烈不適，且外觀是一種能力承諾。
- 誤區十二：忘記設計「斷電後推得動」。一台推不動的故障機器人會卡在走道上，只能等工程師來。

課堂討論

1. 開頭故事的三個問題中，第三個（責任歸屬）沒有技術解。請討論：工程師在面對「無法用技術解決的問題」時，應該扮演什麼角色？
2. 35.7.3 節的知識框顯示「兩個 LED 眼睛」勝過複雜的路徑規劃改良。請討論這個現象，並嘗試找出本書其他章節中類似的「低成本高效益」設計。
3. 35.5.3 節主張機器人應定位為「互補」而非「取代」。請討論這個定位的誠實性與商業性之間是否有衝突，以及如何平衡。
4. 35.4.3 節的知識框指出隱私是「合規、體驗、行銷三者一致」的少見案例。請討論還有哪些設計決策具備這種特性，以及為何它們如此稀少。
5. 35.6.3 節指出人形機器人的 TRL 僅在 4-6，卻獲得大量關注與投資。請討論「關注度」與「成熟度」脫節的原因與後果。

章末摘要

- 三類環境（工廠、半公共、家庭公共）的要求逐級提高，可靠度需求由 95% 提升到 99.9%。
- 公共空間不可用隔離作為安全手段，必須改用本質安全與情境化的反應。
- 急停不一定是安全的反應——在有人的環境中，停止可能造成阻塞或潑灑。
- 從「設備」變成「產品」帶來法律上的關鍵轉變：適用產品責任的無過失責任。
- 標準是技術規範（自願），法規是法律要求（強制），兩者常有引用關係。
- 服務機器人的主要認證：電氣安全、EMC、無線電、電池、機械安全、個資；應避免醫療宣稱。
- ISO 13482 是個人照護機器人的專用標準，其思維是「假設人必定在附近」。
- 認證通常需要 3~9 個月與數十萬到數百萬元，且很少一次通過；應做預測試。
- 產品缺陷分三類：設計、製造、警告；第三類最常被工程團隊忽略。
- 降低責任風險的三種手段：設計上消除、完整的可追溯文件、明確的使用限制。
- 誠實的限制說明能降低期待落差、減少客訴，並在法律上保護自己。
- 機器人蒐集的影像、語音、位置與環境地圖，多數在法律上屬於個人資料。
- 個資保護的五項核心：告知、目的限制、最小化、保存期限、當事人權利。
- 設計即隱私的七原則中，「本地優先」同時滿足合規、體驗與行銷三個目的。
- 四種商業模式中，RaaS 因降低採用門檻與利益一致而成為服務機器人的主流，但有現金流壓力。
- 定價應依客戶的效益而非成本加成；且應定位為互補而非取代。
- TRL 5 到 8 的距離往往比 1 到 5 更遠，這是「死亡之谷」的成因。
- 採用曲線的鴻溝出現在早期採用者與早期大眾之間；跨越需要專注利基、完整支援與參考客戶。
- 社會接受度的四個構面：有用性、易用性、信任、社會規範；後兩者最常被忽略。
- 使用者信任的不是最聰明的機器人，而是最可預測的機器人。
- 恐怖谷效應使擬真但不完美的外觀引發不適；且外觀是一種能力的承諾。

- 服務機器人必用多模態回饋（光、聲、動作、語音），且回饋強度應與情境相稱。
- 失敗是常態，優雅的失敗處理包含：及早求助、明確說明、安全姿態、讓人容易介入。
- 機器人不只服務顧客，還要與現場員工共處；被員工排斥的系統會被架空。

觀念題與分析題

觀念題

1. 比較工廠、半公共與家庭公共三類環境的七個面向差異。
2. 說明為何「不能隔離」根本地改變了公共空間機器人的安全設計。
3. 解釋為何「急停不一定是安全的反應」，並舉三個例子。
4. 比較「設備」與「產品」的六個法律差異，並說明無過失責任的意義。
5. 區分標準與法規，並說明兩者的連結方式。
6. 列舉一台台灣販售的服務機器人所需的主要認證項目。
7. 說明 ISO 13482 與 ISO 10218 在思維上的根本差異。
8. 說明認證的典型時程與成本，以及「預測試」的價值。
9. 列舉產品缺陷的三種類型，並說明為何警告缺陷最常被忽略。
10. 說明降低責任風險的三種手段，並解釋運行日誌的法律意義。
11. 說明為何「誠實的限制說明」反而是優勢。
12. 列舉三種與機器人相關的保險，並說明保險核保的附帶價值。
13. 說明軟體更新帶來的機會與三個風險。
14. 列舉機器人蒐集的四類個人資料，並說明環境地圖為何敏感。
15. 說明個資保護的五項核心要求，並指出最容易違反的一項。
16. 列舉設計即隱私的七個原則，並說明「本地優先」為何是三贏。
17. 比較四種商業模式，並說明 RaaS 在服務機器人上的四個理由。
18. 說明 RaaS 的現金流挑戰與三種因應方式。
19. 說明價值定價的邏輯，以及「與人力成本對標」的陷阱。
20. 說明 TRL 的分級，以及「死亡之谷」的成因。

21. 說明採用曲線的五個階段，以及鴻溝出現的原因與跨越方法。
22. 列舉社會接受度的四個構面，並說明哪兩個最常被忽略。
23. 解釋「使用者信任的是可預測而非聰明」，並列舉三個提升可預測性的設計。
24. 說明恐怖谷效應與三個外觀設計建議，並解釋「外觀是一種承諾」。
25. 列舉優雅失敗處理的四個原則，並說明「讓它推得動」為何重要。

分析題

1. **上市評估**：為你的專題（或一個你熟悉的機器人產品）做完整的上市評估。包含：TRL 定位、所需認證清單與預估時程費用、三種可能的產品缺陷與防範、以及建議的商業模式。**特別要誠實地評估「距離可上市還有多遠」。**
2. **安全反應的設計**：針對「服務機器人在走道上遇到人」這個情境，設計一套完整的反應邏輯。要求：（a）至少三個層級的反應（依距離或風險）；（b）每個層級都要說明「為何這個反應在此情境下是最適當的」；（c）考慮至少三種特殊情況（長者、幼童、載運液體時）。
3. **隱私設計的取舍**：為一台居家機器人設計隱私保護方案。列出它會蒐集的所有資料、判斷哪些是個資、並為每一項設計保護措施。接著分析：若要提供「跌倒偵測」的功能（需要持續的影像分析），該如何在功能與隱私之間取舍？
4. **社會接受度的改善**：選一台你觀察過的服務機器人（餐廳、賣場、機場皆可），用 35.7.2 節的四個構面評估它，找出最弱的一項。**接著提出三個具體的改善設計**，並說明每一個的成本與預期效果。

動手做與專題延伸

本章實作：上市規劃

本章的實作是為一個機器人產品規劃完整的上市路徑，詳見前面的「實作」一節。此處提供三個延伸方向。

- **方向一：認證的實地調查**。實際致電或查詢一家檢測機構（如台灣電子檢驗中心、SGS），詢問一台含 WiFi 的服務機器人所需的認證項目、費用與時程。**把真實的數字記錄下來——這會比任何教科書的描述更有說服力。**
- **方向二：使用者觀察**。到一個有服務機器人的場所（餐廳、賣場），觀察至少一小時。記

錄：人們如何與它互動、發生了幾次尷尬或衝突、店員如何處理異常。用 35.7.2 節的四個構面評分。

- **方向三：隱私政策的撰寫。**為你的專題撰寫一份使用者看得懂的隱私政策（不超過一頁）。用一般人的語言說明：蒐集什麼、為什麼、存多久、如何刪除。

專題延伸

- **服務機器人的接受度研究：**設計一份問卷或訪談提綱，調查一般人對服務機器人的態度。**關鍵是要區分「他們說的」與「他們做的」**——除了問卷，也要實地觀察他們的實際行為。你很可能會發現落差（如問卷說「很願意使用」但實際上會繞開它）。分析這個落差的成因，並提出設計上的改善。
- **意圖表達的實驗：**實作 35.7.3 節知識框中的「機器人眼睛」概念。在你的移動機器人上加裝方向指示（LED 或簡單的螢幕），設計一個實驗：讓機器人在走道上與行人相遇，分別在「有指示」與「無指示」兩種條件下各測 30 次，記錄：僵局發生的次數、行人讓路的比例、平均通過時間。**這個實驗成本極低但結果往往很驚人。**
- **進階：完整的產品化規劃。**選一個你的專題，做一份完整的產品化規劃書（20~30 頁），涵蓋本章與第 34 章的所有面向：需求與定位、技術架構、認證計畫（含實際查詢的費用與時程）、責任與保險、隱私設計、商業模式與財務試算、上市時程、風險與對策。**接著找一位業界人士（可透過學校的產學管道）審閱並給予意見。**這份文件不只是作業——它可能是你創業或求職時最有力的作品集。

參考文獻與延伸閱讀

- ISO 13482:2014, Robots and Robotic Devices — Safety Requirements for Personal Care Robots（個人照護機器人的安全標準）。
- ISO 12100:2010, Safety of Machinery — General Principles for Design（風險評估的基礎標準）。
- 經濟部標準檢驗局（BSMI）商品檢驗相關規定與國家標準（CNS）：台灣的商品檢驗要求。
- 國家通訊傳播委員會（NCC），電信管制射頻器材型式認證相關規定：無線電設備的認證要求。
- 《個人資料保護法》及其施行細則（中華民國）：個資保護的法源。

- European Commission, Regulation (EU) 2023/1230 on Machinery (歐盟機械法規，2027 年起取代機械指令) 。
- Moore, G. A., Crossing the Chasm, HarperBusiness (市場採用曲線與鴻溝理論的經典著作) 。
- Rogers, E. M., Diffusion of Innovations, 5th ed., Free Press (創新擴散理論的原始著作) 。
- Mori, M., "The Uncanny Valley," IEEE Robotics & Automation Magazine, 2012 (英譯版；恐怖谷概念的原始論文) 。
- Bartneck, C. et al., Human-Robot Interaction: An Introduction, Cambridge University Press (人機互動的完整教材) 。
- Kanda, T. and Ishiguro, H., Human-Robot Interaction in Social Robotics, CRC Press (社交機器人的互動設計) 。
- International Federation of Robotics (IFR), World Robotics: Service Robots (年度出版)：服務機器人的市場資料與趨勢。

第 36 章

總整專題方法論

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩組專題的對照：上方是一台掛滿功能標籤（導航、辨識、抓取、語音、陪聊）的機器人，每個標籤旁都標著低落的數字（60%、40%、55%.....），旁註「五件事的 60%」；下方是一台只做一件事的小車正精準對接充電座，旁附一張完整的測試表格（五種地板 × 三種光線 × 六個角度，成功率 98.7%），旁註「一件事的 95%」。右半為專題的推進路徑圖：由左至右五個階段（選題界定、規劃、最小可行系統、測試量化、失敗分析與改良），其中「最小可行系統」以一條貫穿所有層級的垂直細線標示（對比另一條逐層堆疊的水平線並打叉），並在「失敗分析」處畫出五個為什麼的追問鏈。下緣文字：專題的價值，不在於你嘗試了多少，而在於你證明了什麼。

叫獸開講 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？

專題發表會的前一週，兩組學生來找叫獸。

第一組做了一台「全自動智慧居家服務機器人」。他們的規劃是：能導航、能辨識物體、能抓取、能語音對話、還能陪聊天。做了八個月，現在的狀況是——導航會撞牆、辨識只認得三種物體、手臂抓不穩、語音常誤判。「**每一項都做了，但每一項都不能用。**」

第二組只做了一件事：**一台會自己找到充電座並精準對接的小車**。功能簡單到有人笑他們「這也算專題？」。但他們做了完整的測試——在五種地板材質、三種光線條件、六個不同的起始角度下，各測 50 次，成功率 98.7%，並記錄了失敗的 13 次分別是什麼原因。

叫獸看完後說：「**第二組的分數會比較高。**」

第一組不服氣：「可是我們做的功能多那麼多！」

「我問你們一個問題。」叫獸說，「**你們的機器人，能不能穩定地做完一件事？**」

沉默。

「這就是差別。」叫獸在白板上寫下：

做到一件事的 95%，勝過做到五件事的 60%。

「而且原因不只是分數。」他繼續說，「**第二組學到的東西比你們多**。他們知道為什麼在磁磚上會失敗、知道光線角度如何影響對接、知道 13 次失敗中有 9 次是同一個原因。**這些是真正的知識。**」

「而第一組呢——你們花了八個月，最後只知道『每一項都很難』。**這不是知識，這只是感想。**」

小翰低聲說：「那我們現在該怎麼辦？」

「還有一週。」叫獸拿起筆，「**選一個功能，把它做到能穩定運作，並且量化它**。剩下的功能就誠實地說『這是未來的工作』——**這比假裝都做到了誠實得多，也專業得多。**」

最後他補了一句，那句話成為這一章的核心：

「一個專題的價值，不在於你嘗試了多少，而在於你證明了什麼。」

本章為什麼重要

這是全書的最後一章。前面三十五章教的是**機器人學是什麼**，本章要教的是——**如何用它做出你自己的東西**。

專題 (capstone project) 是多數學生第一次獨立完成一個完整的工程作品。它與課堂作業有本質的不同：

- **沒有標準答案**：課本的習題有解答，專題沒有。你必須自己判斷什麼是對的。
- **資源有限**：時間、經費、人力都受限。**必須做取捨**——而取捨的能力，正是工程師與學生的差別。
- **會失敗**：而且失敗是常態。**如何從失敗中提取知識**，決定了這個專題對你的價值。
- **需要溝通**：做出來只是一半，讓別人理解你做了什麼、為什麼有價值，是另一半。

本章的目標不是教你「**怎麼做一個好專題**」，而是教你「**怎麼判斷什麼是好專題**」——因為前者依題目而異，後者卻是通用的。

而本章也是全書所有內容的收束：第 34 章的系統工程、第 35 章的產品化思維、第 30 章的漸進式驗證、第 33 章的倫理檢核——它們都將在你的專題中被實際使用。

學習目標

- 能說明專題的價值來源並區分「嘗試」與「證明」。
- 能運用三個判準評估一個題目是否適合。
- 能界定專題的範圍並辨識過度擴張的徵兆。
- 能撰寫可驗證的專題目標與成功判準。
- 能運用倒推法規劃時程並辨識關鍵路徑。
- 能設計最小可行系統並規劃迭代的順序。
- 能建立有效的實驗記錄與版本管理習慣。
- 能設計量化的測試方案並統計地呈現結果。
- 能進行系統性的失敗原因分析。
- 能依失敗分析提出並驗證設計改良。

- 能撰寫結構清晰的專題報告與有效的簡報。
- 能誠實地評估專題的貢獻與限制。

先備知識

本章需要的是全書的整體理解。特別會用到：第 34 章的系統工程與需求管理、第 30 章的漸進式驗證、第 29 章的軟體工程實務、第 35 章的社會與倫理考量、以及第 2 章的學習方法。

建議的讀法：若你正要開始專題，現在就讀第 36.2~36.5 節（選題與規劃）；若你已在進行中，讀 36.6~36.8 節（開發與測試）；若接近尾聲，讀 36.9~36.11 節（分析與發表）。這一章是可以「按需查閱」的。

核心名詞中英對照

表 36-1 本章核心名詞

中文	英文	說明
總整專題	Capstone Project	整合所學的綜合性專題
範圍界定	Scoping	明確定義做什麼與不做什麼
最小可行系統	Minimum Viable System	能端到端運作的最簡版本
垂直切片	Vertical Slice	貫穿所有層級的最小功能
關鍵路徑	Critical Path	決定專案總時程的工作序列
緩衝時間	Buffer / Slack	為不確定性預留的時間
里程碑	Milestone	可驗證的階段性成果
消融實驗	Ablation Study	移除某元件以驗證其貢獻
基準線	Baseline	用於比較的參考方法
對照組	Control Group	實驗中未施加處理的比較組
可重現性	Reproducibility	他人能重複得到相同結果
失效模式分析	Failure Mode Analysis	系統性地分析失敗的原因
根本原因	Root Cause	問題的最終來源而非表象
五個為什麼	Five Whys	反覆追問以找出根本原因
設計改良	Design Iteration	依分析結果修改設計
技術債	Technical Debt	為求快速而累積的品質欠缺
同儕評閱	Peer Review	由同領域者審查工作

中文	英文	說明
電梯簡報	Elevator Pitch	極短時間內說明核心價值

36.1 專題的價值從哪裡來

36.1.1 嘗試與證明的差別

開頭故事的核心，是一個常被誤解的問題：**專題的價值是什麼？**

多數學生以為是「**做了多少功能**」。但實際上，評審（以及未來的雇主）看的是：

你證明了什麼？
你怎麼知道你證明了它？
別人能相信你的結論嗎？

這三個問題，構成了專題價值的全部。

第一組的問題不在於「功能太多」，而在於「什麼都沒證明」。他們無法回答「你的導航有多準？」「辨識率是多少？」「在什麼條件下會失敗？」——因為他們從未量化過。

第二組雖然只做一件事，卻能明確地說：「在五種材質、三種光線、六個角度下各測 50 次，成功率 98.7%，失敗的 13 次中有 9 次源於同一個原因。」——這是一個可以被檢驗、被引用、被延續的結論。

36.1.2 專題的四種價值

一個專題可以在四個面向創造價值，而多數專題只需要在其中一項做得好：

表 36-2 專題的四種價值來源

價值	問題	評判標準	例子
新穎性	「這件事以前沒人做過」	文獻中確實沒有	新的演算法、新的機構
性能	「我做得比現有的好」	與基準線的量化比較	成功率提升 15%
整合	「我把這些技術串起來了」	系統確實能端到端運作	完整的服務機器人原型
理解	「我搞清楚了為什麼」	分析的深度與可信度	失敗原因的系統性分析

大學部的專題，最容易創造價值的是第三與第四項——整合與理解。而第四項「理解」尤其被低估：一個誠實地分析「為什麼我的方法在這種情況下會失敗」的專題，其價值往往高於一

個宣稱「全部成功」卻沒有細節的專題。

◎ 為何「失敗的分析」是有價值的貢獻

在學術與工程領域，一個被充分理解的失敗，比一個沒被理解的成功更有價值。原因是：成功可能來自運氣或未被察覺的特殊條件（第 30 章的 Sim-to-Real 落差就是最好的例子）；而一個被分析清楚的失敗，告訴了後人「這條路的邊界在哪裡」。本書多處引用的重要論文中，有不少正是「指出某方法的失效條件」的工作。因此若你的專題結果不如預期，不要隱藏它——要把它變成你的主要貢獻。這需要誠實與勇氣，但那正是專業的表現。

36.1.3 三個常見的誤區

誤區一：功能越多越好（開頭故事的第一組）。實際上，**深度勝過廣度**——做到一件事的 95%，勝過五件事的 60%。

誤區二：用最新的技術就是好。使用大型模型不會自動使專題變好；若一個簡單的方法就能解決，用複雜的方法反而顯示判斷力不足（呼應第 31.1.2 節的判準）。

誤區三：等做完了再測試。這是最致命的一項——若最後一週才發現不行，就沒有補救的機會了（36.5 節會詳述）。

36.2 選題

36.2.1 三個判準

一個好的專題題目，必須同時滿足三個條件。**缺一項就會出問題**：

表 36-3 選題的三個判準

判準	問題	不滿足的後果
有價值	做出來之後，誰會在意？	做完了但「所以呢？」
做得完	在你的時間與資源內能完成嗎？	最常見的失敗原因
你能學到東西	這個題目能讓你成長嗎？	做完了但沒有收穫

第二項是最常被低估的。學生往往高估自己在單位時間內能完成的量——這是所謂的「**規劃謬誤**」（planning fallacy），是一個普遍的認知偏誤。

一個實用的校準方法：估算你認為需要的時間，然後乘以 2.5。這個係數聽起來誇張，但符合多數專案的實際經驗。

36.2.2 題目的三種來源

來源一：真實的痛點（最推薦）。觀察你周遭的重複性工作、不便之處、或危險的作業。

- **優點：**需求真實、容易說明價值、且**你能實際驗證**。
- **如何找：**用第 34.2.2 節的五問法訪談實際的使用者。**去現場觀察至少三次**。

來源二：延續既有的工作。接續學長姊的專題、實驗室的研究方向、或開源專案。

- **優點：**有基礎可以站，不必從零開始；且有人能指導。
- **風險：**可能繼承了難以理解的程式碼與未記錄的坑。**接手前務必先確認能不能跑起來**。

來源三：複現與延伸。先複現一篇論文或一個開源專案，再做一個小的改進。

- **優點：**目標明確、有基準線可比較（這對第 36.2 節的「性能」價值極為重要）。
- **注意：**複現本身就有價值——許多論文的結果難以複現，若你能誠實地報告「我複現的結果與論文有差距，原因是……」，這就是貢獻。

36.2.3 題目的健康檢查

在確定題目前，用以下六個問題自我檢查：

1. **能不能用一句話說清楚？**若需要三分鐘才能解釋，題目可能太模糊或太大。
2. **成功長什麼樣子？**若說不出「做到什麼程度算成功」，就無法規劃也無法驗收。
3. **最難的部分是什麼？**若答不出來，代表你還沒真正理解這個題目。
4. **如果那個最難的部分做不出來，還剩下什麼？**——這是最重要的一個問題（36.3.3 節）。
5. **現有的方法為什麼不夠？**若現成的工具就能解決，那你的貢獻是什麼？
6. **你能取得所需的資源嗎？**硬體、資料、場地、以及能指導你的人。

◎ 一個危險的題目類型：「等硬體到就開始」

有一類題目特別危險：**整個專題都建立在某個尚未到手的硬體上**。「等那台機器手臂到貨我就開始寫程式」「等感測器買回來就能測試」——**這在實務上幾乎必定延誤**。採購可能延遲、規格可能不符、到貨後可能發現不能用。**因應之道有三：**一、**先用模擬開發**（第 30 章的價值在此體現）；二、**先用手邊的替代品**（如用手機相機代替工業相機做初步驗證）；三、**把硬體相依的部分隔離**，使其他部分能並行推進。**最重要的原則是：不要讓任何單一的外部依賴，成為整個專題的必要條件。**

36.3 範圍界定

36.3.1 範圍是專題成敗的關鍵

第 34.2.4 節提過「範圍潛變」是專案殺手。在學生專題中，這個問題**更加嚴重**——因為沒有客戶會限制你，只有你自己會不斷加東西。

過度擴張的四個徵兆：

- 「順便加一個……」：每次想到新功能就加進來。
- 「這個做完再說」：核心功能還沒完成就開始做次要功能。
- 「應該不難」：對未做過的事情估計過於樂觀。
- 無法說出「不做什麼」：若你列不出明確的排除清單，代表範圍還沒界定。

36.3.2 寫下「不做什麼」

這是界定範圍最有效的一招（呼應第 34.2.4 節）。一份好的範圍文件應該有兩欄：

表 36-4 範圍界定的範例（自動對接充電的小車）

本專題包含（✓）	本專題不包含（X）
在已知地圖中導航到充電站附近	未知環境的自主探索
用視覺標記做精準對接（誤差 < 5 mm）	無標記的自然特徵對接
在平坦室內地面運作	斜坡、門檻、戶外
電量低於 20% 時自動返回	電池管理與充電曲線最佳化
對接失敗時重試三次並回報	多台機器人的充電排程
量化測試五種地板、三種光線	溫度、濕度等環境變因

右欄不是「做不到」，而是「這次不做」。明確地寫出來有三個好處：

- 保護自己：避免被要求「順便做一下」。
- 聚焦資源：把有限的時間投在核心功能上。
- 誠實地呈現：報告中寫「本研究不涵蓋 X」，比被人問到才說「呃我們沒做」專業得多。

36.3.3 最重要的問題：如果最難的部分失敗了呢

這是 36.2.3 節第四個問題的展開，也是專題規劃中最關鍵的一步。

每個專題都有一個「若失敗則全盤皆輸」的環節。你必須提前識別它，並準備退路：

表 36-5 關鍵風險與退路的規劃範例

關鍵風險	若失敗	退路 (Plan B)
視覺對接的精度不足	無法自動充電	改用機械導引槽 (第 33.3 節的形態計算)
深度學習模型訓練不起來	辨識失效	改用傳統的特徵比對 (第 22.5 節)
實體機器人採購延遲	無法測試	先在模擬中完成 (第 30 章)
SLAM 在該場地漂移嚴重	導航失準	改用視覺標記定位 (第 22.4.4 節)

注意每一個退路都指向本書中「更簡單但更可靠」的方法。這不是妥協，而是工程判斷——一個能運作的簡單方案，勝過一個做不出來的先進方案。

◎ 退路不是失敗，而是專業

許多學生認為「準備 Plan B 就是對自己沒信心」。恰恰相反——沒有 Plan B 才是不專業。工程的本質就是在不確定性中推進，而識別風險並準備因應，正是工程師的核心能力 (第 34.4.2 節的 POC 也是同樣的思維)。更重要的是：有了退路，你才敢大膽地嘗試困難的方法——因為你知道即使失敗也不會全盤皆輸。這反而使你更可能成功。

36.4 規劃

36.4.1 倒推法

規劃時程最有效的方法是由截止日往回推，而非由今天往前排。

具體的步驟：

1. 寫下最終的截止日 (發表日)。
2. 往前推兩週：這是「必須完全停止開發」的日期——剩下的時間用於整理數據、撰寫報告、製作簡報、排練。這兩週不可壓縮。
3. 再往前推兩週：這是「必須完成所有測試」的日期。若測試發現問題，這兩週是修補的緩衝。
4. 剩下的時間才是開發。
5. 在開發時間中扣除 30% 作為緩衝 (意外、生病、其他課業、硬體延遲)。

一個常見的震撼：以 16 週的專題為例：

總時程 16 週

$$\begin{aligned}
 & - \text{報告與簡報 } 2 \text{ 週} - \text{測試與修補 } 2 \text{ 週} = 12 \text{ 週} \\
 & \times (1 - 30\% \text{ 緩衝}) = \text{實際可用的開發時間僅 } 8.4 \text{ 週}
 \end{aligned}$$

這通常只有學生原本預期的一半。

這個計算的價值在於：它會逼你重新界定範圍（36.3 節）。與其在最後兩週手忙腳亂，不如一開始就把範圍縮到 8.4 週能完成的程度。

36.4.2 里程碑的設計

好的里程碑必須是**可驗證的**——能明確判斷「達成了」或「還沒」。

表 36-6 里程碑的兩種寫法

模糊的（不可驗證）	明確的（可驗證）
「完成視覺模組」	能在測試影像上輸出標記的位姿，誤差 < 10 mm
「機器人會動」	能接收速度指令並在地面移動，誤差 < 5%
「整合完成」	能端到端執行一次完整流程（即使成功率低）
「開始測試」	完成 50 次測試並記錄成功率

第三列特別重要——「端到端執行一次」應該是一個很早的里程碑（36.5 節）。

36.4.3 辨識關鍵路徑

關鍵路徑是決定總時程的那條工作序列——它上面的任何延誤都會直接延誤整個專題。

辨識的方法：畫出各項工作的相依關係，找出最長的那條鏈。

例：硬體採購（3 週）→ 組裝（1 週）→ 校正（1 週）→ 整合測試（2 週）= 7 週
而軟體開發可以並行（5 週）

關鍵路徑是硬體那一條——因此應優先啟動採購，而非先寫程式。

三個實務建議：

- 最長前置期的項目最先啟動：硬體採購、需要申請的場地、需要協調的受試者。
- 關鍵路徑上的工作要留最多緩衝。
- 盡可能把工作並行化：例如用模擬先開發軟體，不等硬體到貨（36.2.3 節的知識框）。

36.4.4 團隊的分工

多數專題是團隊進行的，而分工的方式會深刻影響成敗。

兩種常見的分工模式：

表 36-7 兩種分工模式的比較

模式	作法	優點	風險
依模組分工	A 做視覺、B 做控制、C 做機構	各自專精、效率高	整合時才發現接不起來（第 34.3.2 節）
依垂直切片分工	每人負責一個完整的功能流程	整合風險低、每人都懂全貌	需要更多的溝通與共同學習

實務建議是「混合」：初期用垂直切片讓每個人都跑通一次端到端的流程（建立共同的理解），之後再依模組分工深化。

三個必要的團隊機制：

- 共用的版本管理（Git）：這不是選配。沒有版本管理的團隊專題，必定會發生「檔案覆蓋」與「不知道誰改了什麼」的災難。
- 定期的同步：每週一次固定的會議，各自報告進度、卡關、與下週計畫。卡關要早說——這是團隊最大的價值。
- 明確的介面約定：即使依模組分工，也要先約定好模組之間的介面（第 34.3.2 節的介面規格書）。

36.5 最小可行系統

36.5.1 先讓它跑起來，再讓它跑得好

這是本章最重要的一個原則，也是最常被違反的一個。

◎ 垂直切片優於水平堆疊

錯誤的作法（水平）：先把視覺模組做到完美、再把控制模組做到完美、最後整合。結果是：直到最後一個月才第一次整合，然後發現接不起來。正確的作法（垂直）：先做一個粗糙但能端到端運作的版本——視覺只認得一個固定的標記、控制只會直線移動、抓取只會固定姿態——但它能從頭走到尾。之後再逐項深化。這個「粗糙的完整版本」稱為最小可行系統（Minimum Viable System），它應該在專題的前三分之一就完成。

為何垂直切片如此重要？三個理由：

- 及早暴露整合問題（第 34.3.2 節）：時序、座標系、資料格式的問題會立刻浮現，而非在最後一週。
- 建立可運作的基準：之後每一次改進都能與它比較，知道自己是否真的變好了。
- 降低心理壓力：有一個「至少能動」的版本在手，你才敢大膽地嘗試困難的改進。

36.5.2 如何設計最小可行系統

以「自動對接充電」的專題為例：

表 36-8 最小可行系統與完整系統的對照

環節	最小可行版本	完整版本
定位	手動放在起點附近	自主導航到充電站附近
偵測	只認一個大尺寸的固定標記	多尺度、抗遮蔽的標記偵測
對位	開迴路移動固定距離	視覺伺服的閉環對位
判定	人工判斷是否對接成功	電流偵測自動判定
異常	失敗就停止	重試策略與回報

這個最小版本可能一週就能完成——而它一旦運作，你就知道整個流程是通的，接下來只需逐項替換為完整版本。

深化的順序原則：先做風險最高的（36.3.3 節的關鍵風險），而非最容易的。若視覺對位是關鍵風險，就先深化它。

36.5.3 用模擬加速迭代

第 30 章的模擬在專題中有三個具體的價值：

- 不等硬體：軟體開發可以在硬體到貨前開始（36.2.3 節的知識框）。
- 快速試錯：一個參數的效果，在模擬中十分鐘就能測完，在實機上可能要半天。
- 安全地測極端情況：故意讓機器人撞牆、故意讓感測器失效——這些在實機上代價太高。

但也要注意第 30 章的警告：模擬中成功不代表實機成功。因此最小可行系統應該盡早在實機上跑通一次，即使很粗糙。

36.6 開發的實務

36.6.1 版本管理：不是選配

任何專題都應該使用 **Git**（或其他版本管理）。三個理由：

- 能回到任何一個版本：當你改壞了，能立刻退回昨天能跑的版本。這救過無數個專題。
- 能記錄「為什麼改」：good commit message 是最好的實驗紀錄。
- 團隊協作的唯一方式：沒有 Git 的團隊專題，必定會發生檔案覆蓋的災難。

三個實用的習慣：

- 每次「能跑」就 **commit**，並在訊息中寫清楚改了什麼、為什麼。
- 用 **tag** 標記重要的版本：`v1-milestone-first-e2e`（第一次端到端成功）——之後要做比較或回退時極為方便。
- 參數與程式碼分離（第 29.5.1 節）：把可調的參數放在設定檔中，這樣調參時不會弄髒程式的歷史。

36.6.2 實驗紀錄

這是專題中最容易被忽略、但價值最高的習慣。

為什麼需要：三個月後你會完全忘記「當時為什麼把那個參數設成 0.3」。而寫報告時，你會需要那些數據。

一份好的實驗紀錄應該包含：

```
## 2026-03-15 視覺對位測試 #7

### 目的
測試不同光照下的標記偵測穩定度

### 設定
- 程式版本：commit a3f9c21
- 參數：threshold=0.65, min_marker_size=40px
- 環境：實驗室，日光燈全開 / 半開 / 關閉（僅窗光）
- 標記尺寸：10 cm x 10 cm

### 結果
| 光照條件 | 測試次數 | 偵測成功 | 平均誤差 |
```


-----	-----	-----	-----
全開	30	30	3.2 mm
半開	30	28	4.1 mm
僅窗光	30	19	8.7 mm

觀察

- 光線不足時偵測率明顯下降 (63%)
- 失敗的 11 次中，有 8 次是標記邊緣模糊導致角點偵測失敗
- 誤差在低光下也變大，推測與角點精度有關

下一步

- 試著把 threshold 降到 0.5，看能否改善低光的偵測率
- 考慮加裝補光 (但要注意反光，見 #5 的教訓)

注意這份紀錄的四個要素：目的、設定 (含程式版本)、量化的結果、以及觀察與下一步。

最後一項使紀錄不只是數據，而是思考的軌跡。

◎ 實驗紀錄的複利效應

寫實驗紀錄看起來是額外的工作，實際上它會省下更多時間。三個具體的回報：一、避免重複的錯誤——「這個我上次試過了，不行，原因是……」；二、寫報告時直接可用——你不需要回想，數據和分析都在那裡；三、討論時的依據——當指導老師問「你怎麼知道？」，你能立刻拿出證據。一個實務的建議：每次做完實驗立刻寫，不要拖到明天。因為細節會在幾小時內就模糊掉。

36.6.3 除錯的方法

除錯佔了開發時間的大半。四個有效的原則：

原則一：二分法。當一個長流程出錯時，先確認**中間點**是對是錯，就能把問題範圍減半。反覆數次即可定位。

原則二：先驗證假設，而非猜測。與其猜「應該是視覺的問題」，不如印出視覺的輸出來看。第 29.9.2 節的 RViz 與第 29.9.1 節的 `ros2 topic echo` 正是為此。

原則三：一次只改一件事。同時改三個地方然後「好像好了」，你不會知道是哪一個起了作用——而下次遇到類似問題時也無從累積經驗。

原則四：能重現才能修。若一個問題偶爾才發生，第一件事是找出讓它穩定重現的條件。無法重現的問題無法確認是否修好。

36.6.4 技術債的管理

技術債是指為了求快而暫時採用的「不好但能動」的作法。在專題中，適度地欠債是合理的——但必須知道自己欠了什麼。

實用的作法：在程式中用統一的標記記下：

```
# TODO: 這裡的座標轉換寫死了，應該改用 TF ( 第 29.6 節 )
#       目前能動是因為相機位置固定。若相機會動就會壞。

# HACK: 用 sleep(0.5) 等待視覺結果，應該改用回呼
#       這會使節拍變慢，但先求能動

# FIXME: 這個閾值 0.65 是試出來的，不知道為什麼
#       在低光下會失效 ( 見實驗紀錄 #7 )
```

三種標記的區別：`TODO` 是「該做但還沒做」、`HACK` 是「能動但不對」、`FIXME` 是「已知有問題」。在報告的「限制」一節中，這些標記就是現成的素材。

36.7 測試與量化

36.7.1 從「能動」到「有多好」

開頭故事的核心差異就在這裡。「我的機器人能自動對接」是一個沒有資訊的陳述——它可能是十次成功一次，也可能是百次成功九十九次。

量化的三個要素：

在什麼條件下 (測試的情境)
測了幾次 (樣本數)
結果如何 (成功率、誤差、時間)

缺任何一項，結論都無法被檢驗。

36.7.2 測試方案的設計

一份好的測試方案應該涵蓋四類情境：

表 36-9 測試情境的四個類別

類別	目的	例子 (自動對接)
標準情境	驗證基本功能	正常光線、平坦地面、正對充電站

類別	目的	例子 (自動對接)
變異情境	測試強健性	不同光線、不同地板、不同起始角度
邊界情境	找出能力的極限	最遠可以從多遠開始對接？最大偏角？
異常情境	驗證失效的行為	標記被部分遮蔽、途中有障礙、電量極低

第三類「邊界情境」最能展現專業度。一個能明確說出「本系統在起始偏角 $\pm 30^\circ$ 內成功率 $> 95\%$ ，超過 40° 後急遽下降」的專題，遠勝於只說「我們的系統很穩定」。

36.7.3 樣本數要多少

這是學生最常問的問題。一個實用的判準：

要區分 90% 與 95% 的成功率，每組至少需 30~50 次
要宣稱「 $> 99\%$ 」，至少需要 300 次以上

因為 30 次全成功，只能說明成功率大約在 88% 以上 (95% 信心)。

一個常見的錯誤：測了 10 次全成功就宣稱「成功率 100%」。統計上，10 次全成功只能支持「成功率大於 74%」的結論 (95% 信心區間的下界)。

實務建議：每個測試情境至少 30 次；若要做情境之間的比較 (如「改良前 vs 改良後」)，每組至少 50 次。

36.7.4 基準線與消融實驗

基準線 (baseline) 是用來比較的參考。沒有基準線的性能數字沒有意義——「成功率 92%」是好是壞？要看比什麼。

三種常見的基準線：

- 現有的方法：論文中的方法、開源的實作、或商用產品。
- 簡單的方法：一個「笨」但能動的作法。若你的複雜方法只比簡單方法好一點點，這本身就是重要的發現。
- 你自己的前一版：改良前後的比較。

消融實驗 (ablation study) 則是驗證「每個元件是否真的有貢獻」的方法：逐一移除某個元件，看性能下降多少。

表 36-10 消融實驗的範例

配置	成功率	平均誤差	結論
完整系統	98.7%	3.2 mm	—
移除視覺伺服（改開迴路）	62.3%	18.4 mm	視覺伺服貢獻極大
移除補光	84.0%	4.1 mm	補光在低光時重要
移除重試機制	94.1%	3.2 mm	貢獻有限（僅救回 4.6%）
移除濾波	97.9%	3.5 mm	幾乎沒有貢獻

最後一列的發現極有價值——它顯示「濾波」這個元件幾乎沒有作用。誠實地報告這件事，比假裝每個元件都很重要更專業（而且可以在未來版本中移除它以簡化系統）。

36.7.5 結果的呈現

三個原則：

- 用表格呈現數字，用圖呈現趨勢。若要比較數個離散的配置，用表格；若要顯示某個參數的影響，用曲線圖。
- 標示變異性：不要只報平均值，要報標準差或範圍。「成功率 92%」與「92% ± 3%」是不同的資訊量。
- 誠實地呈現失敗：失敗的案例應該被分析而非隱藏（36.8 節）。

36.8 失敗原因的分析

36.8.1 這是專題最有價值的部分

如 36.1.2 節的知識框所述，一個被充分理解的失敗，比一個沒被理解的成功更有價值。而失敗分析正是把「失敗」轉化為「知識」的程序。

多數學生的處理方式是：「我們的系統成功率 85%，失敗的原因是環境干擾。」——這幾乎沒有資訊。「環境干擾」是什麼？哪一種干擾？佔多少比例？

專業的處理方式是：對每一次失敗逐一記錄與分類。

36.8.2 失敗的分類

以開頭故事第二組的 13 次失敗為例：

表 36-11 失敗案例的分類與分析

失敗模式	次數	佔比	初步原因	根本原因
標記偵測失敗	9	69%	光線不足	曝光時間固定，未依環境調整
對位誤差過大	2	15%	地板打滑	輪速回授失準（第 23.8 節）
通訊逾時	1	8%	WiFi 不穩	未設重連機制
未知	1	8%	無法重現	—

這張表的價值在於三點：

- 它指出了主要矛盾：69% 的失敗來自同一個原因——因此改善的資源應該集中在那裡（呼應第 34.5.2 節「找出瓶頸」）。
- 它區分了初步原因與根本原因：「光線不足」是現象，「曝光固定」才是可以修改的原因。
- 它誠實地保留了「未知」：不強行歸類無法解釋的案例。

36.8.3 五個為什麼

要從現象追到根本原因，最實用的方法是**五個為什麼**（Five Whys）——與第 34.2.2 節的五問法同源，但用於故障分析。

◎ 五個為什麼的實例

現象：對接失敗。**為什麼？**——因為機器人停在偏離充電座 2 公分的地方。**為什麼？**——因為視覺算出的位置有誤差。**為什麼？**——因為標記的角點偵測不準。**為什麼？**——因為影像模糊。**為什麼？**——因為曝光時間固定為 20 ms，在低光環境下不足，導致影像雜訊高且對比不足。**根本原因：**曝光參數未依環境自適應。——注意這個結論是可以被修改的，而最初的「對接失敗」不是。這就是根本原因分析的價值：把「不知道怎麼辦」轉化為「知道要改哪裡」。

36.8.4 由分析到改良

找出根本原因後，改良的方案應該**針對根本原因**，並且**再次量化驗證**。

完整的改良循環有**四步**：

1. **提出改良方案：**針對根本原因。以上例而言：改用自動曝光、或加裝補光。
2. **預測效果：**在改之前先預測——「若曝光是主因，改善後低光下的偵測率應由 63% 提升到 90% 以上」。這一步極為重要（見下方知識框）。
3. **實作並重測：**用相同的測試條件重測，才能比較。

4. 比對預測與實際：若符合預測，代表你的因果分析正確；若不符合，代表根本原因找錯了一—回到第一步。

◎ 為何要「先預測再驗證」

這一步是區分「工程」與「試誤」的關鍵。若你改了一個東西然後變好了，這不代表你理解了原因——可能是巧合，也可能是別的因素。但若你先預測「改了之後應該會從 63% 提升到 90%」，然後實際測出 91%，這就強烈地支持了你的因果理解。反之，若你預測 90% 卻只得到 68%，你就知道「還有別的原因沒找到」——這比盲目地繼續試更有價值。這個習慣會讓你的專題報告從「我們試了很多方法」升級為「我們理解了這個系統」。

36.8.5 改良的成果呈現

改良的成果應該用改良前後的對照呈現：

表 36-12 改良前後的對照 (範例)

測試條件	改良前	改良後	提升
全開日光燈	100% (30/30)	100% (30/30)	—
半開	93.3% (28/30)	100% (30/30)	+6.7%
僅窗光 (低光)	63.3% (19/30)	93.3% (28/30)	+30.0%
整體	85.6%	97.8%	+12.2%

注意這張表的說服力來源：它顯示改良只在預期的情境 (低光) 產生顯著效果——這正好驗證了「根本原因是曝光」的分析。若改良在所有情境都提升了，反而值得懷疑是否有其他因素。

36.9 報告的撰寫

36.9.1 報告的結構

一份技術報告的標準結構，其實反映了一個論證的邏輯：

表 36-13 報告的結構與其論證功能

章節	回答的問題	常見的錯誤
摘要	整篇的濃縮 (問題、方法、結果、結論)	寫成「背景介紹」而非結果的摘要
緒論	為什麼這個問題重要？	只講背景不講動機
文獻回顧	別人做到哪裡？還缺什麼？	只是列舉而未指出「缺口」
方法	你做了什麼？別人能複現嗎？	細節不足、參數未交代

章節	回答的問題	常見的錯誤
實驗與結果	你怎麼知道它有效？	只有「能動」的展示而無量化
討論	這些結果代表什麼？限制是什麼？	沒有討論限制
結論	所以呢？下一步是什麼？	只是重複摘要

「實驗與結果」是報告的核心，也是最常被寫得最薄弱的一節。多數學生的報告在「方法」寫了十頁，「結果」只有兩頁——這個比例應該倒過來。

36.9.2 摘要的寫法

摘要是被讀最多次的部分（許多人只讀摘要）。它應該包含四個要素，各一到兩句：

【問題】室內移動機器人的自動充電對接，在低光環境下的可靠度不足。

【方法】本研究提出一套結合視覺標記偵測與自適應曝光的對接系統，並以視覺伺服進行閉環對位。

【結果】在五種地板材質、三種光照條件下各測試 50 次，整體成功率由基準版本的 85.6% 提升至 97.8%；低光環境的改善尤其顯著（63.3% → 93.3%）。失敗案例的分析顯示，剩餘的失敗中有六成源於地板打滑造成的里程計誤差。

【結論】自適應曝光是低光對接的關鍵；未來的改善應聚焦於里程計的強健性而非視覺模組。

注意這份摘要的特徵：有具體的數字、有與基準的比較、有失敗的分析、且結論指出了下一步的方向。這比「本研究成功實現了自動對接功能」有價值一百倍。

36.9.3 誠實地寫「限制」

「限制」一節是專業度的試金石。許多學生擔心寫出限制會扣分——恰恰相反。

理由有三：

- 評審看得出來。你的系統一定有限制，若你不寫，評審會在提問時指出——而那時你就顯得沒有自知之明。
- 主動寫出來反而顯示理解深度。能清楚說出「本系統在何種條件下會失效」，證明你真正理解了它。

- 這是後續研究的價值（呼應第 35.3.2 節「誠實的限制說明反而是優勢」）。

一份好的限制說明的例子：

研究限制

1. 環境限制：本系統僅在平坦室內地面驗證。斜坡、門檻與地毯未經測試，預期在地毯上因里程計誤差增大而性能下降。
2. 標記依賴：本系統依賴事先貼設的視覺標記（第 22.4.4 節）。無標記的自然特徵對接不在本研究範圍內。
3. 樣本數：每個測試情境 50 次，足以區分 90% 與 98% 的成功率，但不足以精確估計 99% 以上的可靠度。
4. 單一機器人：所有測試在同一台機器人上進行，未驗證跨機台的一致性。不同的相機與輪徑可能需要重新校正。
5. 未量化的因素：溫度、濕度、電池電量對性能的影響未納入測試。

注意每一條都具體且可檢驗，而非空泛的「本研究仍有改進空間」。

36.9.4 可重現性

若別人無法複現你的結果，你的結論就無法被檢驗。三個具體的作法：

- 完整交代參數：所有的閾值、增益、尺寸都應該列出（可放附錄）。「經過調整後效果良好」不是可重現的敘述。
- 交代環境：硬體型號、軟體版本、測試場地的條件。
- 公開程式與資料（若可能）：附上 Git 儲存庫的連結。這是現代技術報告的標準作法。

36.10 發表與簡報

36.10.1 簡報不是報告的縮短版

這是最常見的錯誤：把報告的每一節各做一張投影片。

兩者的目的不同：

報告：讓人「完整地理解」（可以慢慢讀、可以回頭）

簡報：讓人「快速地相信」（只有一次機會、無法回頭）

簡報必須有取捨。

36.10.2 簡報的結構

一份 12 分鐘的專題簡報，建議的結構是：

表 36-14 簡報的建議結構（12 分鐘）

段落	時間	內容	關鍵
問題	1.5 分	為什麼這件事值得做	用具體的情境而非抽象的敘述
目標	1 分	你要做到什麼（可驗證的）	明確的數字
方法	3 分	怎麼做的（只講關鍵的設計決策）	不要講每一個細節
展示	2 分	影片：系統實際運作	這是最有說服力的一段
結果	3 分	量化的數據與比較	這是最重要的一段
限制與未來	1 分	誠實的自我評估	顯示專業度
結論	0.5 分	一句話的核心貢獻	讓人記住

注意「結果」佔了最多時間——這正是 36.1.1 節「你證明了什麼」的體現。

36.10.3 影片的重要性

一段 60~90 秒的展示影片，勝過十張投影片。三個製作的建議：

- 一鏡到底，不要剪接。剪接會讓人懷疑「是不是剪掉了失敗的部分」。若一定要剪，明確標示。
- 顯示時間或計數。畫面角落顯示「第 37 次測試」或計時器，增加可信度。
- 也放一段失敗的。這聽起來反直覺，但展示失敗並解釋原因，會大幅提升可信度——它顯示你真的做了大量測試，而非只錄了成功的那一次。

36.10.4 回答問題

問答往往比簡報本身更能顯示你的理解深度。四個原則：

- 先確認問題：若不確定，複述一次「您是問.....對嗎？」。答錯問題比答不出來更糟。
- 誠實地說「不知道」：「這一點我們沒有測試，但我推測.....因為.....」——這比硬掰專業得多。

- 把質疑當成禮物：評審指出的弱點，往往正是你該寫在「未來工作」的內容。
- 準備常見的問題：「為什麼不用 X 方法？」「若換成 Y 環境會怎樣？」「成本多少？」——這三個幾乎必問。

◎ 電梯簡報：三十秒說清楚

練習用三十秒說明你的專題——這個練習的價值極高。若三十秒說不清楚，通常代表你自己還沒想清楚核心是什麼。一個好的三十秒版本的結構是：「很多 [某類使用者] 面臨 [某個問題]，現有的方法 [某個不足]。我們做了 [你的方法]，在 [測試條件] 下達到 [量化結果]，比 [基準] 提升了 [多少]。」——這五句話涵蓋了問題、缺口、方法、驗證與貢獻。這個版本在求職面試、實驗室介紹、與任何非正式的場合都用得上。

36.11 結語：從讀者到創造者

36.11.1 全書的終點

三十六章之後，這本書的內容結束了。

回顧這趟旅程：第一篇我們認識了機器人的身體；第二篇學會描述它能到哪裡；第三篇讓它動得精確而穩定；第四篇讓它理解自己身在何處；第五篇讓它知道該做什麼；而第六篇，我們把這一切變成能在真實世界運作的東西。

而第 33 章留下的那句話，此刻有了新的意義：

現實會回應你，這是機器人學最好的老師。

而本章教的，正是如何聽懂那個回應——透過量化、透過失敗分析、透過誠實的紀錄。

36.11.2 六條主線的最後一次回顧

第 33.10 節整理過全書的六條主線。在你的專題中，它們會全部出現：

表 36-15 六條主線在專題中的體現

主線	在專題中的意義
模型的價值在減負	不完美的模型仍能大幅簡化問題——不要因為模型不準就放棄它
分層與時間尺度分離	把快的與慢的分開處理，是系統能運作的關鍵
設計對不確定性不敏感的系統	與其追求精確，不如追求強健——這往往是專題成敗的分界
機構與控制協同設計	先問能不能用機構解決（36.3.3 節的退路多半屬於此類）
模型提供保證，學習提供泛化	選對工具，而非選最新的工具

主線	在專題中的意義
理論各部分互相支撐	你會發現當初覺得無用的章節，在專題中突然派上用場

36.11.3 最後的話

這本書從第 1 章的一個問題開始：為什麼電腦下棋能贏世界冠軍，卻摺不好一件衣服？

第 33 章給了答案：因為智慧不只在腦裡，也在身體裡，還在身體與環境的互動裡。

但還有一個問題，這本書無法替你回答：

你想做出什麼？

這個問題的答案，只能由你自己給。

三十六章的內容，說到底只是**工具**。運動學、動力學、控制、感知、規劃、學習——它們的價值，取決於你用它們做了什麼。

而做出一個東西的過程，必定包含大量的失敗、挫折、與「為什麼又不行」的深夜。**這是這門學科的代價，也是它的魅力**——因為當它終於動起來的那一刻，那個東西是**真實存在於世界上的**，不是紙上的公式，不是螢幕上的數字。

它會動。而那是你讓它動的。

願你在往後的路上，做出一些讓自己驚訝的東西。

現在，換你了。

完整案例：一個專題的全程

以下用一個虛構但貼近真實的案例，串起本章的所有內容。

案例背景

題目：實驗室移動機器人的自動充電對接系統

時程：16 週 **團隊：**3 人 **資源：**一台差速輪機器人、一支 USB 相機、實驗室場地

第 1~2 週：選題與範圍界定

選題的判斷（36.2.1 節）：

- **有價值 ✓**：實驗室的機器人現在需要人工插電，常因忘記充電而無法使用。
- **做得完 ✓**：範圍明確、硬體現成、且有明確的成功判準。
- **能學到 ✓**：涵蓋視覺、控制、系統整合。

範圍界定 (36.3.2 節)：如表 36-4 所示，明確寫出「不含未知環境探索、不含無標記對接、不含斜坡」。

關鍵風險的識別 (36.3.3 節)：團隊判斷是「視覺對位的精度能否達到 5 mm」，並準備了退路——若不行，改用機械導引槽（漏斗狀的開口引導）。

第 3~4 週：規劃與最小可行系統

倒推法的時程 (36.4.1 節)：

$$16 \text{ 週} - \text{報告簡報} 2 \text{ 週} - \text{測試修補} 2 \text{ 週} = 12 \text{ 週}$$

$$12 \times 0.7 \text{ (緩衝)} = 8.4 \text{ 週的實際開發時間}$$

團隊據此把範圍再縮減——原本想做的「多台排程」被移到「不包含」欄。

最小可行系統 (36.5.2 節)：第 4 週結束時，完成了一個「手動放在起點、只認一個大標記、開迴路移動、人工判定」的版本。雖然粗糙，但端到端跑通了——這使團隊立刻發現了兩個沒預料到的問題：相機的視野比預期窄、以及標記在近距離時會超出畫面。

第 5~10 週：逐項深化

深化的順序 (36.5.2 節)：先做風險最高的視覺對位。

開發實務 (36.6 節)：

- 全程使用 Git，每次能跑就 commit，重要版本打 tag。
- **每次實驗都寫紀錄** (如 36.6.2 節的範例)——這在第 12 週寫報告時省下了大量時間。
- 遇到「偶爾對不準」的問題時，**先找出穩定重現的條件** (36.6.3 節原則四)，發現是在特定角度才發生。

第 11~12 週：測試與量化

測試方案 (36.7.2 節)：四類情境

表 36-16 案例的測試方案

情境類別	具體條件	次數
標準	日光燈全開、磁磚地、正對	50
變異	五種地板 × 三種光線	$50 \times 15 = 750$ (抽樣至 300)
邊界	起始偏角 $10^\circ \sim 50^\circ$ (每 10° 一組)	$5 \times 30 = 150$
異常	標記部分遮蔽、途中有障礙	60

結果：整體成功率 85.6%；邊界測試發現偏角超過 40° 後成功率由 92% 驟降至 47%——這是一個明確且有價值的發現。

第 13 週：失敗分析與改良

失敗分類 (36.8.2 節)：如表 36-11，發現 69% 的失敗源於低光下的標記偵測。

五個為什麼 (36.8.3 節)：追到根本原因是「曝光時間固定」。

改良與驗證 (36.8.4 節)：

1. 方案：改用自動曝光。
2. 預測：低光下的偵測率應由 63% 提升到 90% 以上；其他光照條件應無顯著變化。
3. 實測：低光 93.3%、其他條件不變。
4. 比對：符合預測 ✓——這強烈支持了因果分析的正確性。

整體成功率由 85.6% 提升至 97.8% (表 36-12)。

第 14~16 週：報告與發表

報告的重點分配 (36.9.1 節)：結果與討論佔了報告的一半篇幅，方法只佔四分之一。

誠實的限制 (36.9.3 節)：明確寫出五項限制，包含「偏角超過 40° 性能急遽下降」——這個「缺點」反而成為簡報時最受好評的部分，因為它顯示團隊真正理解了系統的邊界。

簡報 (36.10 節)：12 分鐘中，結果佔 3 分鐘、展示影片 2 分鐘 (含一段失敗的案例並解釋原因)。

評審的回饋：「你們是唯一一組能明確說出自己系統在什麼條件下會失效的。」

◎ 這個案例的核心教訓

回顧全程，這個專題成功的關鍵不在於技術有多先進——視覺標記偵測是第 22 章的標準技術、視

覺伺服是第 22.7 節的基本方法。它成功是因為五件事：一、範圍界定得夠小（8.4 週能完成）；二、第 4 週就跑通了端到端（及早暴露問題）；三、全程有實驗紀錄（寫報告時省下大量時間）；四、做了充分的量化測試（1,260 次）；五、失敗分析做到了根本原因並驗證了預測。這五件事，任何一個題目都能做到——而它們正是本章的全部內容。

實作：啟動你的專題

本章的實作就是你自己的專題。以下是一份可以直接使用的行動清單。

階段一：選題與界定（第 1~2 週）

1. 列出五個候選題目，用 36.2.1 節的三個判準各自評分（1~5 分）。
2. 對最高分的題目做健康檢查（36.2.3 節的六個問題）。特別是第四個問題——若最難的部分失敗了，還剩下什麼？
3. 寫下範圍文件：兩欄式，明確列出「包含」與「不包含」各至少五項（36.3.2 節）。
4. 識別關鍵風險並準備退路（36.3.3 節），至少列出三個風險與其 Plan B。
5. 寫下可驗證的成功判準：不是「做出一台機器人」，而是「在 X 條件下達到 Y 的成功率」。

階段二：規劃（第 2~3 週）

1. 用倒推法排出時程（36.4.1 節）。算出你實際可用的開發週數——這個數字通常會讓你回頭修改範圍。
2. 設定 4~6 個可驗證的里程碑（36.4.2 節），每個都要能明確判定達成與否。
3. 畫出工作的相依關係圖，標出關鍵路徑（36.4.3 節）。最長前置期的項目今天就啟動。
4. （團隊）約定分工模式與介面規格（36.4.4 節），並建立 Git 儲存庫與每週的同步時間。

階段三：開發（第 3~11 週）

1. 設計最小可行系統（36.5.2 節）：列出每個環節的「最簡版本」。
2. 在前三分之一的時間內完成端到端的跑通——即使很粗糙。
3. 建立實驗紀錄的習慣（36.6.2 節）：每次實驗立刻寫，包含目的、設定、結果、觀察。
4. 每次能跑就 commit，重要版本打 tag。
5. 依風險高低決定深化的順序——先做最可能失敗的。

階段四：測試與分析（第 11~13 週）

1. 設計涵蓋四類情境的測試方案（36.7.2 節）。
2. 每個情境至少測 30 次；若要比較改良前後，每組至少 50 次。
3. 建立基準線：至少要有「簡單方法」或「改良前」作為比較。
4. 逐一記錄每次失敗的現象，不要只記「失敗」。
5. 做失敗分類（36.8.2 節），找出佔比最高的失敗模式。
6. 用五個為什麼追到根本原因（36.8.3 節）。
7. 提出改良、預測效果、實作、比對（36.8.4 節的四步循環）。先預測再驗證。

階段五：發表（第 14~16 週）

1. 先寫摘要（36.9.2 節的四要素）——這會逼你想清楚核心貢獻。
2. 結果與討論應佔報告的一半篇幅。
3. 誠實地寫出至少五項限制（36.9.3 節），每一項都要具體。
4. 附上完整的參數與環境資訊（36.9.4 節的可重現性）。
5. 製作 60~90 秒的展示影片，含一段失敗的案例。
6. 練習三十秒的電梯簡報（36.10.4 節的知識框）。
7. 預想三個必問的問題並準備答案。
8. （最後）用第 33.9.4 節的六個倫理問題檢視你的專題。

國中小也看得懂：不要貪心，選一件事做到最好

假設你要參加科展，有兩個題目可以選：

題目一：「超級無敵萬能機器人」——會走路、會說話、會抓東西、會掃地、會陪你玩。

題目二：「會自己找到插座充電的小車」——只做這一件事。

你會選哪一個？

大部分的人會選第一個，因為聽起來厲害多了。但評審通常會給第二個比較高的分數。為什麼？

因為第一個題目做出來，很可能**每一樣都會，但每一樣都不好用**——走路會撞牆、說話常聽錯、抓東西會掉。

而第二個題目，你可以**把它做到非常好**。而且你可以說出：「我測試了 300 次，成功了 294 次。失敗的 6 次裡，有 4 次是因為地板太滑。」

這就是差別：第一組只能說「我試了很多東西」，第二組可以說「**我證明了一件事**」。

那怎麼做到呢？這一章教了三個祕訣：

祕訣一：先做一個「很爛但完整」的版本。不要先把每個零件做到完美再組起來——**要先讓它從頭到尾動一次**，就算很醜也沒關係。因為這樣你才知道哪裡會出問題。

祕訣二：每次做實驗都要寫下來。寫什麼？做了什麼、結果如何、你發現了什麼。因為三個月後你一定會忘記，而且寫報告的時候會用到。

祕訣三：失敗的時候，要問「為什麼」五次。

比如：對不準 → 為什麼？因為位置算錯 → 為什麼？因為照片模糊 → 為什麼？因為太暗 → 為什麼？因為相機的設定沒有跟著光線調整。

你看，一直問下去，最後就找到「真正可以修的地方」了！

最後告訴你這一章最重要的一句話：

一個專題的價值，不在於你嘗試了多少，而在於你證明了什麼。

所以下次做東西的時候，不要貪心。**選一件事，把它做到最好，然後測試它、記錄它、理解它**。這樣做出來的東西，才真的是你的。

叫獸提醒與常見錯誤

- 誤區一：以為功能越多分數越高。做到一件事的 95%，勝過做到五件事的 60%。
- 誤區二：用最新的技術就以為比較好。若簡單的方法就能解決，用複雜的方法反而顯示判斷力不足。
- 誤區三：先把每個模組做到完美再整合。應先做垂直切片，在前三分之一就跑通端到端。
- 誤區四：低估所需的時間。應把估算乘以 2.5，並用倒推法算出實際可用的開發週數。

- 誤區五：整個專題建立在尚未到手的硬體上。應先用模擬或替代品開發，並隔離硬體相依的部分。
- 誤區六：說不出「不做什麼」。無法列出排除清單，代表範圍還沒界定。
- 誤區七：沒有準備 Plan B。識別關鍵風險並準備退路是專業，不是沒信心。
- 誤區八：不寫實驗紀錄。三個月後你會忘記所有細節，而寫報告時會需要那些數據。
- 誤區九：測了十次全成功就宣稱 100%。統計上這只能支持「大於 74%」的結論。
- 誤區十：沒有基準線就報告性能數字。「成功率 92%」是好是壞，取決於比什麼。
- 誤區十一：把失敗歸因為「環境干擾」就結束。應逐一分類並用五個為什麼追到根本原因。
- 誤區十二：報告中不寫限制。評審看得出來，而主動寫出反而顯示你真正理解了系統。

課堂討論

1. 開頭故事中，叫獸說第二組「學到的東西比較多」。請討論：為何深度勝過廣度？這個原則在什麼情況下不成立？
2. 36.8.4 節主張「先預測再驗證」。請討論這個習慣為何能區分「工程」與「試誤」，並舉出本書中運用了同樣思維的其他例子。
3. 36.9.3 節主張誠實地寫出限制反而是優勢。請討論這與第 35.3.2 節「誠實的限制說明」的異同，並思考為何人們仍傾向隱藏缺點。
4. 36.5.1 節的垂直切片原則，與第 30.9 節的漸進式驗證、第 34.4.2 節的 POC 有共同的結構。請歸納這個結構，並說明它為何有效。
5. 回顧全書，哪一章的內容讓你最意外？哪一個觀念改變了你原本的想法？請具體說明。

章末摘要

- 專題的價值不在於嘗試了多少，而在於證明了什麼——關鍵是「你怎麼知道」與「別人能否相信」。
- 專題的四種價值：新穎性、性能、整合、理解；大學部專題最容易在後兩者創造價值。
- 一個被充分理解的失敗，比一個沒被理解的成功更有價值。
- 選題的三個判準：有價值、做得完、能學到東西；第二項最常被低估。
- 估算時間時應乘以 2.5，這符合「規劃謬誤」的普遍現象。

- 題目的三種來源：真實的痛點、延續既有工作、複現與延伸；複現本身就有價值。
- 選題健康檢查最重要的一問：若最難的部分失敗了，還剩下什麼？
- 不要讓任何單一的外部依賴（如未到貨的硬體）成為整個專題的必要條件。
- 界定範圍最有效的一招是寫下「不做什麼」，這能保護自己、聚焦資源、並誠實呈現。
- 每個專題都有「若失敗則全盤皆輸」的環節，必須提前識別並準備退路。
- 用倒推法規劃：扣除報告、測試與 30% 緩衝後，實際的開發時間常只有預期的一半。
- 里程碑必須可驗證；「端到端跑通一次」應該是很早的里程碑。
- 垂直切片優於水平堆疊：先做粗糙但完整的最小可行系統，在前三分之一完成。
- 版本管理不是選配；實驗紀錄有複利效應，應在實驗後立刻寫。
- 除錯的四原則：二分法、驗證假設而非猜測、一次只改一件事、能重現才能修。
- 量化的三要素：在什麼條件下、測了幾次、結果如何；缺一項結論就無法檢驗。
- 測試應涵蓋標準、變異、邊界、異常四類情境；邊界情境最能展現專業度。
- 每個情境至少 30 次；10 次全成功只能支持「大於 74%」的結論。
- 沒有基準線的性能數字沒有意義；消融實驗能驗證每個元件是否真有貢獻。
- 失敗分析應逐一分類並用五個為什麼追到根本原因——把「不知道怎麼辦」轉為「知道要改哪裡」。
- 改良的四步循環：提出方案、預測效果、實作重測、比對預測——先預測再驗證是工程與試誤的分界。
- 報告中「結果與討論」應佔一半篇幅；誠實地寫出限制反而顯示專業度。
- 簡報不是報告的縮短版；結果應佔最多時間，並用影片展示（含一段失敗的案例）。
- 練習三十秒的電梯簡報：問題、缺口、方法、驗證、貢獻五句話。

觀念題與實踐題

觀念題

1. 說明專題價值的三個核心問題，並解釋「嘗試」與「證明」的差別。
2. 列舉專題的四種價值來源，並說明大學部專題最容易在哪兩項創造價值。
3. 解釋為何「一個被充分理解的失敗，比一個沒被理解的成功更有價值」。

4. 列舉選題的三個判準，並說明「規劃謬誤」與 2.5 倍法則。
5. 比較題目的三種來源及其優缺點，並說明「複現本身就有價值」的理由。
6. 列舉選題健康檢查的六個問題，並說明哪一個最重要。
7. 說明「等硬體到就開始」為何是危險的題目類型，以及三種因應方式。
8. 說明「寫下不做什麼」的三個好處。
9. 解釋為何準備 Plan B 是專業而非沒信心。
10. 說明倒推法的五個步驟，並計算 16 週專題的實際開發時間。
11. 比較里程碑的兩種寫法，並說明「端到端跑通」為何應是很早的里程碑。
12. 說明關鍵路徑的意義與辨識方法，以及三個實務建議。
13. 比較兩種團隊分工模式，並說明推薦的混合作法。
14. 說明垂直切片優於水平堆疊的三個理由。
15. 說明最小可行系統的設計原則與深化的順序。
16. 列舉版本管理的三個理由與三個實用習慣。
17. 說明實驗紀錄應包含的四個要素及其複利效應。
18. 列舉除錯的四個原則，並說明「能重現才能修」的意義。
19. 說明技術債的三種標記及其區別。
20. 說明量化的三個要素，以及為何缺一項結論就無法檢驗。
21. 列舉測試情境的四個類別，並說明哪一類最能展現專業度。
22. 說明樣本數的判準，並解釋「10 次全成功」在統計上的意義。
23. 說明基準線的三種來源，以及消融實驗的目的。
24. 說明失敗分析的作法，以及五個為什麼如何把現象轉為可修改的原因。
25. 說明改良的四步循環，並解釋「先預測再驗證」為何是工程與試誤的分界。
26. 列舉報告的七個章節及其論證功能，並指出最常被寫得薄弱的一節。
27. 說明摘要的四個要素，並解釋為何應包含失敗的分析。
28. 說明誠實地寫出限制的三個理由。
29. 比較報告與簡報的目的差異，並說明簡報中哪一段應佔最多時間。

30. 說明展示影片的三個製作建議，以及為何應包含失敗的案例。

實踐題

- 選題評估：**列出三個你有興趣的專題題目，用 36.2.1 節的三個判準各自評分並說明理由。
接著對得分最高的做完整的健康檢查（36.2.3 節的六個問題），特別詳細回答第四個問題。
- 範圍與風險：**為你選定的題目撰寫完整的範圍文件（兩欄式，各至少五項），並識別三個關鍵風險與其退路（36.3.3 節）。每個退路都要說明它會損失什麼、保留什麼。
- 時程規劃：**用倒推法為你的專題排出時程，計算實際可用的開發週數。設定 5 個可驗證的里程碑，並畫出工作的相依關係圖標出關鍵路徑。若算出的開發時間不足，說明你會如何調整範圍。
- 測試方案設計：**為你的專題設計完整的測試方案，涵蓋四類情境（36.7.2 節）。列出每個情境的具體條件、測試次數、以及要記錄的指標。並說明你的基準線是什麼。
- 失敗分析的演練：**回想一個你過去做過的專題或作業中失敗的經驗。用五個為什麼追到根本原因，並提出一個針對根本原因的改良方案。接著預測：若當時做了這個改良，結果會如何改變？

動手做與專題延伸

本章實作：你的專題

本章的實作就是完成你自己的專題，詳見前面「實作」一節的五階段行動清單。

專題延伸

- 專題後的復盤：**專題結束後（無論成敗），花兩小時做一次完整的復盤。回答五個問題：
（一）哪些時間被浪費了？（二）哪個決定是最關鍵的？（三）如果重來一次，最先改變什麼？（四）我學到最重要的一件事是什麼？（五）這個專題的哪一部分值得延續？把答案寫下來——這份文件在你下一個專題、以及求職面試時都會極有價值。
- 把專題變成作品集：**整理成一份對外的版本，包含：一段 90 秒的影片、一頁的圖文說明（含量化的結果）、以及 GitHub 的程式碼與說明文件。求職時，一個能實際展示的作品，勝過十行履歷上的描述。
- 進階：延續與深化。**若你的專題有值得延續的部分，考慮：（一）投稿學生競賽或研討會——這會逼你把工作整理到可被審查的水準；（二）開源它——讓後續的人能站在你的基礎

上；（三）寫成技術部落格——把你踩過的坑寫下來，這對後人的價值可能超過專題本身。本書多次強調「一個被理解的失敗是有價值的貢獻」——你的踩坑紀錄，正是這樣的貢獻。

參考文獻與延伸閱讀

- Booth, W. C., Colomb, G. G. and Williams, J. M., The Craft of Research, University of Chicago Press (研究方法與論證結構的經典教材) 。
- Zobel, J., Writing for Computer Science, 3rd ed., Springer (技術寫作的實用指南，含實驗設計與結果呈現) 。
- Brooks, F. P., The Mythical Man-Month, Addison-Wesley (軟體專案管理的經典，含規劃謬誤的深刻討論) 。
- Ries, E., The Lean Startup, Crown Business (最小可行產品與快速迭代的思想來源) 。
- Reason, J., Human Error, Cambridge University Press (失效分析與根本原因的理論基礎) 。
- Ohno, T., Toyota Production System, Productivity Press (五個為什麼的原始出處) 。
- Kahneman, D., Thinking, Fast and Slow, Farrar, Straus and Giroux (規劃謬誤等認知偏誤的系統性討論) 。
- Chacon, S. and Straub, B., Pro Git (線上免費)：版本管理的完整教材。
- Wilson, G. et al., "Good Enough Practices in Scientific Computing," PLOS Computational Biology, 2017 (研究專案的實務建議，含資料與程式管理) 。
- Duke, A., Thinking in Bets, Portfolio (在不確定性下決策，含「先預測再驗證」的思維) 。
- Reynolds, G., Presentation Zen, New Riders (簡報設計的實用指南) 。
- 本書第 2 章的學習方法、第 30 章的漸進式驗證、第 34 章的系統工程：本章的許多原則在前面各章已有對應，建議一併回顧。